CADERNO DE ENGENHARIA

System Design

Staff+ & Principal

Pensamento, mecanismos e decisões para sistemas distribuídos e cenários bancários.

171 capítulos / 5.130 perguntas respondidas

1.710 trade-offs explicados / 342 alternativas

Edição de estudo - 4 de novembro de 2026

Material independente. As arquiteturas propostas são exercícios de engenharia, não documentação interna do Nubank nem garantia de aprovação.

PDF integral com índice navegável e referências técnicas.

Como estudar sem decorar um desenho

Comece por M001. Em cada problema, negocie resultado, invariantes e restrições antes de desenhar. Faça uma proposta simples, percorra uma operação de ponta a ponta e então introduza falhas, concorrência e limites de escala.

Cada capítulo contém definição, uso e limites; uma fala de abertura; tecnologias e fluxo principal; passos de construção; 30 perguntas com respostas; 10 trade-offs; duas alternativas; e fontes técnicas. As perguntas formam um repertório de treino, não uma previsão do roteiro da banca.

T001-T120 cobrem fundamentos e mecanismos. C001-C050 desenvolvem os cinquenta cenários fornecidos. M001 transforma esse repertório num método de conversa. Use os links entre capítulos para investigar uma decisão sem perder o problema original.

Nas respostas, “eu faria” significa uma escolha proposta sob as premissas do exercício. Confirme volumes, contratos, versões e exigências do ambiente real. Exemplos numéricos são hipóteses, a menos que uma fonte seja explicitamente indicada.

Uma fonte comprova o mecanismo descrito no seu escopo; ela não comprova automaticamente toda a arquitetura proposta nem que uma empresa a utiliza. URLs preservam o acesso ao texto original e podem mudar depois desta edição.

Para treinar: leia a pergunta, responda em voz alta sem olhar, exponha um custo da escolha e diga que evidência faria você mudá-la. Depois compare com a resposta. Explique o mecanismo antes de citar o produto.

A edição em áudio segue a mesma ordem. Diagramas são descritos, tabelas viram explicações e URLs ficam para consulta neste PDF. A narração usa voz sintética neural em português brasileiro.

Um roteiro de 60 minutos

0-8: problema e invariantes. 8-18: contratos e modelo de dados. 18-32: fluxo principal e responsabilidades. 32-48: concorrência, falhas, recuperação e gargalos. 48-60: alternativas, operação, evolução e síntese. Ajuste o tempo à condução do entrevistador.

Correções que fazem diferença

Senioridade não é uma lista de tecnologias

SQL, locks, batch e monólitos podem ser escolhas excelentes. Staff+ exige justificar limites, custo, segurança, evolução e colaboração.

Timeout não é fracasso comprovado

Uma operação externa pode ter ocorrido. Preserve identidade, estado ambíguo e reconciliação antes de produzir outro efeito.

Exactly-once exige uma fronteira

Broker, banco e fornecedor têm garantias distintas. Deduplicação, transações e identidade de negócio precisam se conectar.

Dinheiro exige uma autoridade definida

Cache, projeção e replicação assíncrona não garantem saldo correto por si. Explicite quem autoriza e o que acontece numa partição.

Multi-região não obriga multiwriter

Cells, afinidade e múltiplas regiões podem coexistir com um único escritor por chave. Failover requer controle de propriedade e dados recuperáveis.

Degradação é política, não improviso

Stand-in e falhar aberto só cabem onde risco e regras permitem. Uma tela parcial e uma autorização financeira têm consequências diferentes.

Imutabilidade não elimina privacidade

Retenção, acesso, backups, chaves e obrigações de exclusão precisam de desenho próprio. Crypto-shredding não resolve sozinho toda obrigação.

Observabilidade não substitui correção

Trace é amostrado e pode se perder. Estado financeiro, auditoria e obrigações pendentes precisam sobreviver independentemente do APM.

Índice navegável

M001 - Mindset Staff+: raciocínio, risco, falhas e influência 11

T001 - Requisitos funcionais e escopo 18

T002 - SLI, SLO, SLA e error budget 25

T003 - Estimativas de capacidade e crescimento 32

T004 - DDD, domínio, agregados e invariantes 39

T005 - Monólito modular e microsserviços 46

T006 - Fronteiras, ownership e limites transacionais 53

T007 - Arquitetura hexagonal e ports and adapters 60

T008 - Contratos de API: REST, gRPC e GraphQL 67

T009 - Comunicação síncrona, assíncrona e fan-out 74

T010 - Decisões arquiteturais orientadas a evidências 81

T011 - CAP e PACELC 88

T012 - Modelos de consistência 95

T013 - ACID, MVCC e isolamento 102

T014 - Consenso, Raft, Paxos e eleição de líder 109

T015 - Quóruns e suas garantias 116

T016 - Replicação, lag e promoção 123

T017 - Sharding e rebalanceamento 130

T018 - Relógios distribuídos e causalidade 137

T019 - Concorrência, locks, CAS e deadlocks 144

T020 - Múltiplas regiões e split-brain 151

T021 - SQL versus NoSQL e modelagem 158

T022 - PostgreSQL: consultas, planos e paginação 165

T023 - Índices: B-tree, hash, GIN e BRIN 172

T024 - WAL, fsync, LSM e compaction 179

T025 - Connection pooling 186

T026 - DynamoDB 193

T027 - Redis: estruturas, Cluster e persistência 200

T028 - Datomic, Datalog e Clojure 207

T029 - MongoDB e Cassandra 214

T030 - Elasticsearch, OpenSearch e busca 221

T031 - Cache-aside, read-through, write-through e write-behind 228

T032 - Invalidação de cache, TTL e versões 235

T033 - Eviction e admissão em cache 242

T034 - Cache stampede, single-flight e hot keys 249

T035 - Cache penetration e Bloom filters 256

T036 - Cache distribuído e particionamento 263

T037 - CQRS e materialized views 270

T038 - Event-driven architecture e contratos 277

T039 - Event sourcing, snapshots e replay 284

T040 - CDC e Debezium 291

T041 - Kafka: partições, grupos e offsets 298

T042 - Filas e pub/sub: SQS, SNS, RabbitMQ e Service Bus 304

T043 - Ordenação e sequência por agregado 311

T044 - At-most-once, at-least-once e exactly-once 318

T045 - Backpressure e admission control 325

T046 - DLQ, quarentena e redrive 332

T047 - Sagas e compensações 339

T048 - Workflows duráveis: Temporal e Step Functions 346

T049 - Batch, arquivos, manifestos e checkpoints 353

T050 - Cron, calendários, cutoff e quotas 360

T051 - DNS, TLS, mTLS e descoberta 367

T052 - TCP, UDP, HTTP/2 e HTTP/3 374

T053 - Gateway, BFF e WAF 381

T054 - Load balancing e sticky sessions 388

T055 - Traffic shaping e rate limiting 395

T056 - CDN, ETag e cache HTTP 402

T057 - WebSocket, SSE e long polling 409

T058 - Object storage, multipart e URLs assinadas 416

T059 - VPC, subnets, NAT e egress 423

T060 - APIs externas, webhooks e SFTP 430

T061 - Idempotência 437

T062 - Transactional Outbox 444

T063 - Transactional Inbox 451

T064 - Timeouts, retries, backoff, jitter e UNKNOWN 458

T065 - Leases e fencing tokens 465

T066 - Circuit breaker, bulkhead e degradação 472

T067 - Backups, PITR, RPO, RTO e disaster recovery 479

T068 - Observabilidade: métricas, logs e traces 486

T069 - Incidentes, runbooks e post-mortems 493

T070 - Canary, blue-green e rollback 500

T071 - Autenticação: OAuth, OIDC e sessões 507

T072 - Autorização, RBAC, ABAC e multi-tenancy 514

T073 - Criptografia, KMS, HSM e segredos 521

T074 - Threat modeling e OWASP API Security 528

T075 - Governança, retenção, exclusão e auditoria 535

T076 - Testes de contrato, propriedades, carga e chaos 542

T077 - Migrações, expand-contract e backfills 549

T078 - Docker, ECS e Kubernetes 556

T079 - Serverless: Lambda e Azure Functions 563

T080 - IaC, Terraform, OpenTofu e GitOps 570

T081 - Runtime, threads, event loop e GC 577

T082 - Hardware, CPU, NUMA, SSD e page cache 583

T083 - Algoritmos e estruturas de dados 589

T084 - Escalabilidade e autoscaling 596

T085 - Little’s Law, tail latency e profiling 602

T086 - OLTP e OLAP 608

T087 - Stream processing, Flink e watermarks 615

T088 - ETL, ELT, data lake e lineage 621

T089 - IA, RAG, embeddings e avaliações 627

T090 - UX, leituras desatualizadas e estados pendentes 634

T091 - Ledger, partidas dobradas e imutabilidade 641

T092 - Saldo disponível, reservas e limites 648

T093 - Precisão monetária, arredondamento e câmbio 654

T094 - Pagamentos, Pix e cartões 661

T095 - Chargeback: ida, volta e resultados parciais 668

T096 - Reconciliação operacional e financeira 675

T097 - Antifraude e revisão humana 682

T098 - Gestão de casos e maker-checker 689

T099 - Atendimento omnichannel e capacidade 696

T100 - Shuffle, BFF e composição de interfaces 703

T101 - Notificações: push, e-mail e SMS 710

T102 - Geração e entrega de documentos 717

T103 - Feeds e timelines 724

T104 - Geolocalização e índices espaciais 731

T105 - UUID, Snowflake e encurtador de URLs 738

T106 - Reservas, estoque e concorrência 745

T107 - Execução distribuída de jobs 752

T108 - Metering, billing e quotas 759

T109 - A/B tests e feature flags 766

T110 - Recomendação, ranking e personalização 773

T111 - RFCs e ADRs 780

T112 - Build versus buy 787

T113 - FinOps, TCO e custo por operação 794

T114 - Ownership, times e carga cognitiva 801

T115 - Plataformas e golden paths 808

T116 - Strangler, dívida técnica e evolução 815

T117 - Gestão de riscos e dependências 822

T118 - Migrações organizacionais e adoção 829

T119 - Condução da entrevista de 60 minutos 836

T120 - Validação por invariantes e contraexemplos 843

C001 - Desenhe o sistema de envio e recebimento de Pix 850

C002 - Como você arquitetaria o autorizador de cartão de crédito? 857

C003 - Desenhe o Ledger (Livro-Razão) financeiro do banco 864

C004 - Arquitetura para cálculo de rendimento diário de 90 milhões de contas 871

C005 - Sistema de detecção de fraude transacional em tempo real 878

C006 - Como lidar com picos de tráfego extremos (ex: quinto dia útil ou Black Friday)? 885

C007 - Desenhe a infraestrutura de Open Finance (iniciação de pagamentos) 892

C008 - Migração de um banco de dados relacional grande para particionamento (Sharding) 899

C009 - Sistema centralizado de Notificações (Push, Email, SMS) 906

C010 - Onboarding de usuários e KYC (Know Your Customer) 913

C011 - Arquitetura Multi-Região / Active-Active 920

C012 - Gerenciamento de chaves e segurança PCI-DSS para PIN do cartão 927

C013 - Agregação de saldo (Conta, Cartão, Cripto, Investimentos) na tela inicial 934

C014 - Prevenção de duplicidade em compras com cartão físico intermitente 941

C015 - Distribuição e consumo massivo de webhooks para parceiros B2B 948

C016 - Tolerância a falhas na emissão de cartões físicos 955

C017 - Serviço de Chat de Atendimento ao Cliente 962

C018 - Processamento de fechamento da fatura do cartão 969

C019 - Rastreabilidade distribuída em centenas de microsserviços 976

C020 - Gerenciamento de campanhas de Cashback/Rewards 983

C021 - Criação de um ambiente de homologação fidedigno (Sandbox) 990

C022 - Mudança de esquema de banco de dados sem indisponibilidade 997

C023 - Prevenção contra ataques de Credential Stuffing/Brute Force no Login 1004

C024 - Sistema de simulação de empréstimo (Loan Engine) 1011

C025 - Lidando com versionamento de APIs REST consumidas pelo App Mobile 1018

C026 - Ingestão massiva de logs de aplicativos móveis 1025

C027 - Sincronização offline-first do aplicativo para consultas sem internet 1032

C028 - Desenho de um sistema de "Distributed Lock" (Bloqueio Distribuído) 1039

C029 - Otimização de custos de infraestrutura no pico da escalabilidade 1046

C030 - Criação de um Internal Developer Platform (IDP) 1053

C031 - Arquitetura Analítica: Data Mesh vs Data Lakehouse 1060

C032 - Direito ao Esquecimento (LGPD/GDPR) em sistemas imutáveis 1067

C033 - Engenharia do Caos e Testes em Produção 1074

C034 - Motor de busca para Histórico de Transações de milhões de clientes 1081

C035 - Service Mesh e Comunicação Inter-serviços (L7) 1088

C036 - Integração com APIs legadas externas (ex: SPB, Câmaras de Compensação) 1095

C037 - GraphQL Federation para dezenas de Frontends/Mobile Apps 1102

C038 - Controle de Acesso Baseado em Regras e Relacionamentos (Global IAM/RBAC) 1109

C039 - Gerenciamento e Rotação de Segredos 1116

C040 - Consistência e Idempotência em Sagas Distribuídas 1123

C041 - MLOps e Model Serving em Alta Concorrência (Scoring de Crédito) 1130

C042 - Agendamento Distribuído de Jobs (Distributed Cron) 1137

C043 - Defesa L7 na Borda (Edge Computing e WAF Avançado) 1144

C044 - Roteamento Inteligente de Pagamentos P2P e B2B 1151

C045 - Cálculo de Saldo em Tempo Real com Extrema Concorrência 1158

C046 - Motor de Reconciliação Contábil Massiva (Banco x BACEN) 1165

C047 - Integração e Pooling de Hardware Security Modules (HSMs) 1172

C048 - Controle de Feature Toggles em Dispositivos Móveis 1179

C049 - Evolução de Contratos de Eventos e Schema Registry 1186

C050 - Câmbio e Pagamentos Cross-Border (Operações de FX) 1193

M001 / N. Mindset e condução da entrevista

Mindset Staff+: raciocínio, risco, falhas e influência

Mindset Staff+ é transformar ambiguidade em uma decisão técnica justificável, entregável e operável. Inclui produto, correção, falhas, custo, pessoas e evolução. Pensar em incidentes é uma parte desse raciocínio, não sua definição inteira.

Quando usar. Use como roteiro mental em qualquer cenário: descubra o resultado desejado, proteja invariantes, proponha uma solução simples e teste seus limites.

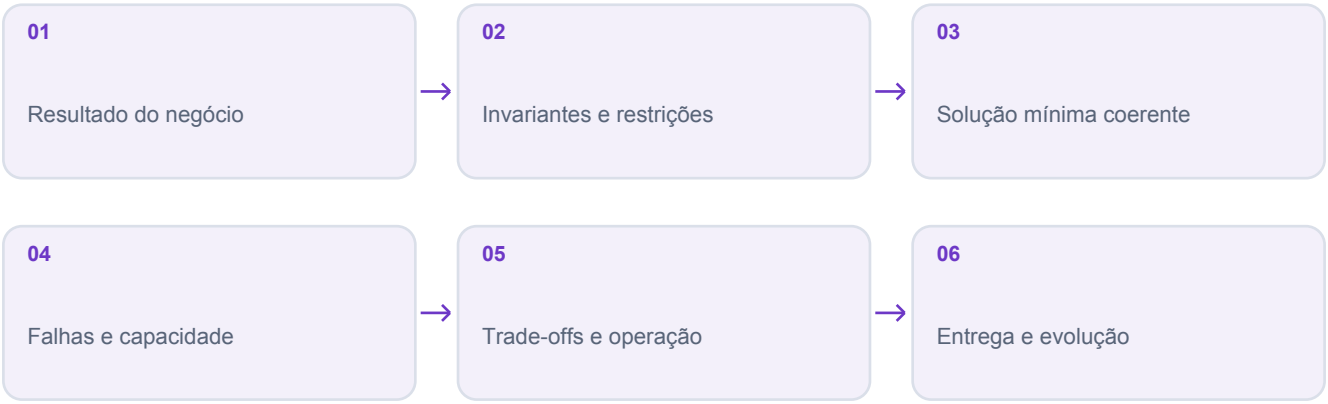
Quando evitar ou limitar. Evite começar por um catálogo de tecnologias, dramatizar toda seta, inventar riscos aceitos pelo negócio ou tratar papéis e níveis como uma escada universal.

Uma abertura possível

Qual resultado precisa acontecer, o que não pode dar errado e qual é a solução mais simples que entrega isso? Depois eu provo como ela se comporta quando cresce, falha e precisa mudar.

Peças e tecnologias. Invariantes · SLO e orçamento de risco · ADR · Capacity planning · Blast radius · Runbook e ownership

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Antes do desenho: diga o objetivo em uma frase e confirme duas ou três restrições que mudariam a solução.

2. Primeiro desenho: escolha a autoridade dos dados, o limite da transação e o fluxo principal com poucas peças.

3. Em cada fronteira relevante: pergunte por prazo, identidade, efeito, resultado desconhecido, capacidade e recuperação.

4. Ao comparar opções: explicite benefício, custo, risco residual, responsável pela política e evidência que mudaria sua decisão.

5. Ao fechar: resuma o que foi escolhido, como será operado e entregue, o limite atual e o próximo experimento necessário.

Cuidado técnico. O texto enviado é ponto de partida de estudo, não evidência da rubrica interna do Nubank.

Cuidado técnico. Não há regra universal de aprovar em 200 ms, escolher AP, active-active ou reconciliar no data lake.

Cuidado técnico. Sênior, Tech Lead, Staff e Principal não são definidos por presença ou ausência de um pattern.

30 perguntas para sustentar o desenho

01. Qual frase devo manter na cabeça antes de responder?

Eu quero entregar o resultado do cliente sem violar as invariantes do negócio. Vou escolher uma solução simples, explicar suas fronteiras e demonstrar como ela cresce, falha e se recupera. Essa frase orienta prioridades sem me prender a ferramentas.

Termo para lembrar: Enquadramento

02. Staff é o engenheiro que pensa em tudo que pode dar errado?

Pensar em falhas é necessário, mas não suficiente. Também preciso entender produto, reduzir escopo, escolher o que entregar primeiro e tornar decisões compreensíveis. Uma análise infinita de riscos pode impedir uma solução útil e não demonstra bom julgamento.

Termo para lembrar: Escopo

03. Qual pergunta faço ao receber um cenário novo?

Pergunto qual jornada é essencial e qual consequência seria inaceitável. Num chargeback, perder uma solicitação e duplicar crédito são riscos diferentes. Isso revela onde preciso de durabilidade, identidade e confirmação antes de escolher banco ou fila.

Termo para lembrar: Primeira pergunta

04. Como diferencio requisito de invariante?

Requisito descreve comportamento ou qualidade desejada; invariante é uma condição que deve permanecer verdadeira durante as transições relevantes. Eu escrevo, por exemplo, uma intenção financeira não gerar dois créditos, e verifico cada caminho contra ela.

Termo para lembrar: Invariante

05. Como evito parecer simples demais para Staff?

Apresento a solução mínima que satisfaz as restrições e mostro por que ela é suficiente. Depois identifico seus limites mensuráveis. Adicionar Kafka, Kubernetes ou múltiplas regiões sem necessidade aumenta obrigações operacionais, não comprova senioridade.

Termo para lembrar: Solução mínima

06. Que pergunta faço diante de uma dependência externa?

Se ela ficar lenta ou indisponível, quais jornadas e recursos serão afetados? Limito espera, concorrência e acoplamento conforme essa resposta. Não adiciono apenas um timeout: explico o que acontece com o trabalho já aceito e com o usuário.

Termo para lembrar: Blast radius

07. O que devo enxergar em cada seta do diagrama?

Um contrato entre participantes: identidade, autorização, prazo, semântica de entrega, efeito e confirmação. Nas setas críticas, descrevo também duplicatas e resultado desconhecido. Nem toda chamada precisa da mesma infraestrutura ou do mesmo nível de garantia.

Termo para lembrar: Fronteira

08. Como respondo quando o entrevistador corta uma seta?

Primeiro identifico em que instante a falha aconteceu. Antes do envio, depois do commit e depois do efeito remoto exigem respostas diferentes. Eu preservo a identidade e explico qual evidência permite retentar ou concluir.

Termo para lembrar: Timeout

09. Posso escolher aceitar inconsistência porque parece mais barato?

Eu posso apresentar opções e quantificar consequências, mas não inventar a tolerância financeira ou regulatória. Algumas invariantes não são negociáveis. Para risco permitido, proponho limites, responsável, monitoramento e caminho de reconciliação antes de aceitar a exposição.

Termo para lembrar: Negócio

10. Staff sempre prefere manter a compra aprovada?

Não. Uma recusa controlada pode ser a decisão correta quando faltam controles essenciais. Disponibilidade tem significado por jornada e contrato. Aprovar indiscriminadamente durante falhas pode produzir perda e violar obrigações que nenhuma reconciliação posterior desfaz.

Termo para lembrar: Disponibilidade

11. Timeout em fraude ou prevenção à lavagem autoriza bypass?

Não assumo essa equivalência. São controles com finalidades e obrigações diferentes. Defino o tratamento aprovado para cada operação e sinal, incluindo revisão ou recusa quando necessário. Circuit breaker protege recursos; não concede permissão de negócio.

Termo para lembrar: Antifraude e AML

12. Devo dizer que um banco escolhe AP para impressionar?

Não. Eu identifico qual dado e qual operação precisam de qual consistência, especialmente sob partição. Uma arquitetura bancária contém garantias diferentes. CAP não determina sozinho stand-in, reconciliação nem o risco que a empresa deve aceitar.

Termo para lembrar: CAP

13. Posso dizer que depois reconciliamos no data lake?

Eu preciso indicar autoridade, identidades, prazo, divergências e ações autorizadas. Um data lake pode apoiar análise, mas não é automaticamente o mecanismo operacional de correção. Reconciliação detecta e resolve evidências; não apaga qualquer erro financeiro sem custo.

Termo para lembrar: Reconciliação

14. Como apresento trade-off sem dizer só depende?

Eu digo de que depende e escolho sob hipóteses explícitas. Por exemplo: com este volume e prazo, usaria polling por simplicidade; mudaria para CDC se carga e latência medidas justificarem o custo operacional adicional.

Termo para lembrar: Custo

15. O entrevistador muda um requisito. Devo defender minha escolha original?

Reavalio a premissa e mostro qual parte da decisão mudou. Isso demonstra que a arquitetura deriva dos requisitos. Preservo componentes ainda válidos e explico custo de migração; não preciso proteger uma escolha por orgulho.

Termo para lembrar: Evidência

16. Qual profundidade matemática é esperada no raciocínio?

Faço contas que influenciem decisões: taxa de pico, concorrência, bytes, amplificação e tempo de drenagem. Declaro hipóteses e margens. Não preciso produzir precisão falsa, mas devo perceber quando um gargalo invalida o desenho.

Termo para lembrar: Capacidade

17. Como considero a experiência do cliente?

Descrevo o estado observável: pendente, indisponível, desatualizado ou concluído. Não retorno zero para saldo desconhecido nem sucesso antes da confirmação necessária. A interface faz parte da correção porque orienta a próxima ação do usuário.

Termo para lembrar: Falha parcial

18. Quando é correto não usar um pattern famoso?

Quando o problema que ele resolve não existe ou já está coberto por uma garantia mais simples. Explico a janela de falha e a fronteira transacional. Conhecer um padrão inclui reconhecer seu custo e quando dispensá-lo.

Termo para lembrar: Simplicidade

19. Como o aspecto organizacional aparece sem desviar da entrevista?

Relaciono cada autoridade e operação a um responsável, contrato e capacidade de suporte. Se proponho serviço compartilhado, explico quem evolui e atende incidentes. Não transformo o mock em organograma, mas evito componentes sem dono.

Termo para lembrar: Ownership

20. Toda solução deve virar plataforma para cinquenta squads?

Não. Primeiro verifico repetição real do problema e custo da adoção. Uma biblioteca, padrão documentado ou serviço gerenciado pode bastar. Plataforma exige usuários, manutenção, compatibilidade e governança; generalizar cedo também cria atrito.

Termo para lembrar: Paved road

21. Lead é necessariamente um nível entre Senior e Staff?

Não existe essa relação universal. Tech Lead pode ser uma responsabilidade exercida em diferentes níveis. Eu comparo escopo, autonomia e impacto demonstrados, sem alegar conhecer a rubrica interna do Nubank a partir de títulos.

Termo para lembrar: Tech Lead

22. Sênior só pensa no caminho feliz e em autoscaling?

Não. Engenheiros seniores competentes também tratam falhas, segurança e trade-offs. A distinção útil está no alcance e na ambiguidade enfrentada, não numa caricatura em que apenas Staff conhece resiliência.

Termo para lembrar: Sênior

23. O que acrescenta alcance Principal ao raciocínio?

Eu conecto a decisão a investimentos, dependências e evolução de vários domínios, mantendo profundidade suficiente para testar as garantias. Influência não significa centralizar tudo; significa ajudar a organização a tomar decisões coerentes e sustentáveis.

Termo para lembrar: Principal

24. Como soar humano sem perder profundidade?

Começo pelo efeito concreto: se a resposta sumir, não quero cobrar duas vezes. Depois nomeio o mecanismo e sua fronteira. Uso termos técnicos para explicar decisões, não como lista que o entrevistador precise interpretar sozinho.

Termo para lembrar: Comunicação

25. Como discordo de uma sugestão do entrevistador?

Reconheço o benefício e exponho a consequência observável. Eu diria: essa opção melhora latência, mas duas regiões poderiam reservar o mesmo limite; podemos aceitar essa exposição ou precisamos de uma autoridade comum? Mantenho a conversa colaborativa.

Termo para lembrar: Desacordo

26. E se eu não souber uma tecnologia citada?

Explico a garantia de que preciso e pergunto pelo contrato relevante. Não invento propriedades. Posso comparar mecanismos conhecidos e indicar o que verificaria na documentação, continuando o desenho em torno das invariantes.

Termo para lembrar: Incerteza

27. O que diferencia dizer resiliente de demonstrar resiliência?

Eu narro uma falha concreta, quem detecta, o estado preservado, quem assume e como sabemos que recuperou. Incluo limites de capacidade e efeito no cliente. Uma caixa de DLQ sem responsável e redrive não fecha essa história.

Termo para lembrar: Operação

28. Como evita um projeto perfeito impossível de entregar?

Defino uma primeira versão que atende o requisito essencial e preserva espaço de evolução. Separo decisões reversíveis das caras de mudar. Para cada etapa, indico validação, risco residual e condição de avanço.

Termo para lembrar: Entrega incremental

29. Como distribuo meu raciocínio durante o mock?

Uso o início para escopo e invariantes, depois modelo e desenho principal, aprofundando os riscos mais importantes antes de fechar. Ajusto ao diálogo. Não gasto quarenta minutos listando requisitos e deixo recuperação apenas como promessa final.

Termo para lembrar: Sessenta minutos

30. Como sei se minha resposta está madura?

Consigo explicar por que cada peça existe, qual garantia termina nela, o que acontece após falha e como operar e mudar a solução. Se um termo não ajuda a responder isso, preciso aprofundá-lo ou retirá-lo do desenho.

Termo para lembrar: Autoavaliação

10 trade-offs: ganho, custo e decisão

01. Profundidade versus cobertura

Ganho. Aprofundar revela domínio das garantias

Custo e motivo. Pode deixar jornadas essenciais sem desenho

Como decidir. Escolher dois ou três riscos decisivos e manter visão do todo

02. Simplicidade versus flexibilidade

Ganho. Menos peças facilitam entrega e operação

Custo e motivo. Pode exigir evolução quando requisitos mudarem

Como decidir. Preservar contratos e identificar gatilhos mensuráveis de mudança

03. Disponibilidade versus invariantes

Ganho. Manter serviço reduz fricção

Custo e motivo. Algumas aprovações sob falha assumem risco inaceitável

Como decidir. Decidir por operação e política, nunca por slogan AP

04. Automação versus intervenção

Ganho. Automação reduz tempo e toil

Custo e motivo. Decisões ambíguas podem produzir efeito incorreto

Como decidir. Automatizar o que tem evidência suficiente e escalar exceções

05. Padronização versus autonomia

Ganho. Padrões reduzem erros repetidos

Custo e motivo. Podem impor custos a domínios diferentes

Como decidir. Padronizar garantias e permitir exceções justificadas

06. Generalização versus problema local

Ganho. Reuso ajuda vários times

Custo e motivo. Plataforma prematura aumenta adoção e manutenção

Como decidir. Validar demanda e custo antes de criar produto interno

07. Rigor versus velocidade

Ganho. Validação evita decisões caras erradas

Custo e motivo. Análise ilimitada adia valor

Como decidir. Distinguir decisões reversíveis e usar experimentos pequenos

08. Detalhe técnico versus clareza

Ganho. Mecanismo concreto sustenta argumento

Custo e motivo. Jargão excessivo esconde a decisão

Como decidir. Explicar efeito, mecanismo e limite nessa ordem

09. Custo de infraestrutura versus risco

Ganho. Margem reduz impacto de picos e falhas

Custo e motivo. Recursos ociosos e complexidade custam

Como decidir. Comparar custo total com consequência e compromisso do produto

10. Defender decisão versus adaptar

Ganho. Consistência transmite critério

Custo e motivo. Teimosia ignora premissas novas

Como decidir. Mudar quando evidência muda e explicitar o motivo

Duas alternativas ao caminho proposto

1. Roteiro por invariantes

Quando faz sentido. Problema novo com risco de correção elevado

Como implementar. Objetivo, invariantes, autoridade, transação, falhas, recuperação e evolução

Limites. Pode ficar abstrato se não virar um fluxo concreto com dados e números

2. Roteiro por jornada e incidentes

Quando faz sentido. Sistema existente ou integração com parceiros

Como implementar. Narrar jornada normal, cortar fronteiras críticas e provar contenção e retomada

Limites. Não deve ignorar requisitos, simplicidade, custo e responsabilidades fora do incidente

Referências e alcance da evidência

1. Google SRE: Embracing Risk

Relaciona confiabilidade, risco e objetivos; não define níveis do Nubank.

https://sre.google/sre-book/embracing-risk/

2. AWS: bulkhead e células

Isolamento de falhas e limites de dependências compartilhadas.

https://docs.aws.amazon.com/wellarchitected/latest/reliability-pillar/rel_fault_isolation_use_bulkhead.html

3. Azure: ADR

Contexto, alternativas e consequências de decisões.

https://learn.microsoft.com/en-us/azure/well-architected/architect-role/architecture-decision-record

4. Google SRE: Simplicity

Redução de complexidade e equilíbrio entre estabilidade e evolução.

https://sre.google/sre-book/simplicity/

Continue em: T001 · T002 · T010 · T066 · T111 · T114 · T115 · T119 · T120

T001 / A. Enquadramento e arquitetura

Requisitos funcionais e escopo

Requisitos funcionais descrevem comportamentos observáveis; escopo define quais atores, fluxos e exceções serão projetados e quais ficarão como integrações ou evolução.

Quando usar. Antes de escolher armazenamento, serviços ou escala; especialmente quando o enunciado mistura aceitação, processamento e conclusão financeira.

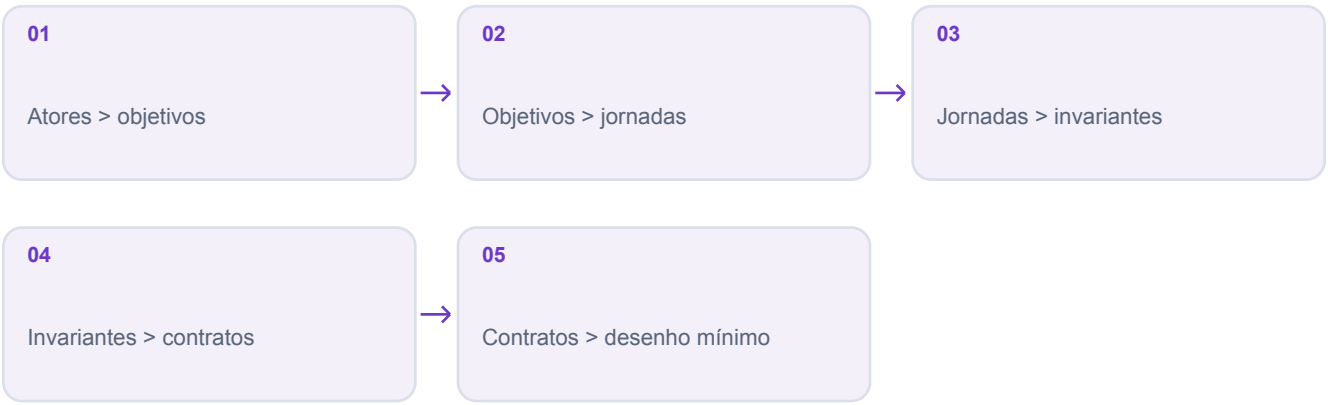
Quando evitar ou limitar. Evitar transformar descoberta em interrogatório interminável: confirmar hipóteses essenciais, registrar pendências e desenhar uma primeira solução revisável.

Uma abertura possível

Vou confirmar quem usa, qual resultado espera e o que significa sucesso. Depois priorizo um fluxo completo, incluindo falhas e retorno, para que cada peça tenha uma razão.

Peças e tecnologias. EventStorming · C4 · OpenAPI · máquina de estados · story mapping

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Identificar usuário, operador e sistemas externos.

2. Definir entrada, saída e significado de conclusão.

3. Priorizar fluxo essencial e exceções de alto risco.

4. Registrar hipóteses de volume, prazo e segurança.

5. Validar o escopo e marcar evoluções fora do MVP.

30 perguntas para sustentar o desenho

01. Como começaria um enunciado vago?

Eu identificaria o usuário e o resultado desejado antes de nomear serviços. Pediria um exemplo concreto, confirmaria a entrada e definiria como reconhecer sucesso, falha e processamento pendente.

Termo para lembrar: Problem framing | Base técnica: referência 1.

02. O que distingue requisito funcional de qualidade?

Transferir entre contas é comportamento funcional; concluir dentro de um prazo com determinada disponibilidade é qualidade. Eu escreveria ambos separadamente, porque um sistema pode executar a função e ainda frustrar o usuário.

Termo para lembrar: Requisito funcional | Base técnica: referência 1.

03. Como define o MVP sem perder segurança?

Eu preservaria um caminho completo com autorização, persistência e confirmação. Adiaria canais e relatórios adicionais, mas não retiraria invariantes financeiras para reduzir escopo; simplificaria o produto em vez da correção.

Termo para lembrar: Walking skeleton | Base técnica: referência 1.

04. Como trata uma exigência de mostrar tudo?

Eu perguntaria quais informações sustentam a primeira decisão do usuário. Carregaria esse contexto inicialmente e detalhes sob demanda, reduzindo fan-out, exposição de dados e dependência de serviços opcionais.

Termo para lembrar: Progressive disclosure | Base técnica: referência 1.

05. O que significa operação concluída?

Eu distinguiria recebimento durável, validação, execução e confirmação externa. A resposta HTTP e a interface informariam a etapa real; um registro criado não provaria que o dinheiro chegou ao destino.

Termo para lembrar: Acceptance versus completion | Base técnica: referência 1.

06. Como identifica os atores?

Eu incluiria cliente, atendente, supervisor, operação e integrações. Para cada ator, registraria ações permitidas e informação necessária; isso evita desenhar apenas o aplicativo e esquecer investigação e recuperação administrativa.

Termo para lembrar: Actor mapping | Base técnica: referência 1.

07. Como registra hipóteses quando faltam dados?

Eu declararia valores provisórios e sua influência na decisão. Se assumir cem operações por segundo, anotaria que sharding depende de medição posterior, permitindo corrigir a premissa sem refazer todo o raciocínio.

Termo para lembrar: Assumption log | Base técnica: referência 1.

08. O que faz com requisitos contraditórios?

Eu mostraria a incompatibilidade com um cenário observável, como resposta imediata dependente de parceiro indisponível. Negociaria confirmação pendente ou disponibilidade menor, explicitando qual experiência o negócio está disposto a aceitar.

Termo para lembrar: Constraint negotiation | Base técnica: referência 1.

09. Como traduz fala de negócio em modelo?

Eu pediria exemplos de estados e transições reais, mantendo os termos reconhecidos pela operação. Diferenciaria aprovado, enviado e liquidado, porque palavras parecidas podem esconder obrigações e consequências financeiras muito diferentes.

Termo para lembrar: Ubiquitous language | Base técnica: referência 1.

10. Como define o que fica fora?

Eu nomearia a responsabilidade excluída e o contrato esperado dela. Se o ledger é externo, especificaria comando, idempotência e consulta de resultado; fora do escopo não significa ignorar a integração.

Termo para lembrar: Scope boundary | Base técnica: referência 1.

11. Como evita desenhar para qualquer possibilidade?

Eu escolheria o fluxo mais frequente e duas falhas de maior impacto. Exploraria variações que mudam garantias, como timeout após efeito remoto, deixando pequenas diferenças de apresentação para uma etapa posterior.

Termo para lembrar: Risk prioritization | Base técnica: referência 1.

12. Como considera fluxos excepcionais?

Eu perguntaria sobre cancelamento, duplicata, falta de documento e rejeição externa. Ligaria cada exceção a um estado e responsável, evitando um campo genérico de erro que impede recuperação automática.

Termo para lembrar: Exception path | Base técnica: referência 1.

13. Como trata requisitos regulatórios?

Eu registraria retenção, localização e trilhas exigidas como restrições a confirmar com especialistas responsáveis. Não inventaria prazos legais; desenharia políticas configuráveis e evidência de execução para adaptar a implementação validada.

Termo para lembrar: Constraint discovery | Base técnica: referência 1.

14. Como pergunta sobre volume sem gastar dez minutos?

Eu confirmaria usuários ativos, operações por dia e fator de pico, usando contas aproximadas. Só aprofundaria bytes, regiões ou conexões quando esses números realmente alterassem uma escolha importante do desenho.

Termo para lembrar: Order of magnitude | Base técnica: referência 1.

15. Como lida com mudança de requisito no meio?

Eu identificaria qual hipótese deixou de valer e mostraria a parte afetada. Se aparecer necessidade de aprovação humana, acrescentaria estado e prazo duráveis, preservando os contratos que continuam corretos.

Termo para lembrar: Adaptive design | Base técnica: referência 1.

16. Como saber se um requisito é testável?

Eu o reescreveria como condição inicial, ação e resultado observável. Em vez de sistema rápido, definiria uma janela de latência e população de requisições, permitindo medir cumprimento sem interpretação subjetiva.

Termo para lembrar: Acceptance criteria | Base técnica: referência 1.

17. Como inclui experiência de falha?

Eu descreveria o que o usuário vê quando uma dependência falha. Um saldo indisponível não vira zero; uma tentativa desconhecida permanece em verificação, com histórico e canal de acompanhamento claros.

Termo para lembrar: Degraded experience | Base técnica: referência 1.

18. Como separa solução de requisito?

Eu trataria Kafka como hipótese de implementação, não como necessidade do usuário. Primeiro estabeleceria retenção, desacoplamento ou replay; depois compararia tecnologias capazes de fornecer as garantias requeridas no contexto.

Termo para lembrar: Solution bias | Base técnica: referência 1.

19. Como prioriza custo e latência?

Eu escolheria uma jornada e quantificaria o ganho esperado. Se reduzir cem milissegundos exige duplicar infraestrutura, perguntaria pelo impacto mensurável, evitando otimizar a infraestrutura sem benefício proporcional para o produto.

Termo para lembrar: Quality attribute scenario | Base técnica: referência 1.

20. Como trata o fluxo de atendimento?

Eu incluiria pesquisa por identificador, linha do tempo e ações autorizadas. A operação precisa investigar sem editar tabelas manualmente; por isso ferramentas administrativas entram no desenho conforme risco e frequência.

Termo para lembrar: Operational journey | Base técnica: referência 1.

21. Como modela anexos no escopo?

Eu confirmaria formatos, tamanho, origem e retenção. Separaria recebimento de liberação para leitura, incluindo quarentena e autorização; tratar arquivo como simples coluna escondia um ciclo de segurança e armazenamento próprio.

Termo para lembrar: Content lifecycle | Base técnica: referência 1.

22. Como evita esquecer o retorno de uma integração?

Eu desenharia ida, confirmação e reconciliação no mesmo quadro. Para cada envio perguntaria como descobrimos o resultado, inclusive parcial, impedindo considerar o projeto pronto apenas porque geramos um arquivo.

Termo para lembrar: End-to-end flow | Base técnica: referência 1.

23. Como define consistência com o negócio?

Eu perguntaria quais estados jamais podem ocorrer, como dois créditos pela mesma decisão. A partir disso escolheria transação, unicidade ou coordenação; não começaria com uma preferência abstrata por consistência forte.

Termo para lembrar: Business invariant | Base técnica: referência 1.

24. Como trata canais diferentes?

Eu distinguiria identificação do canal e autenticação bancária. WhatsApp pode iniciar atendimento, mas operações sensíveis exigem verificação apropriada; o contrato funcional precisa reconhecer essa diferença antes da escolha do gateway.

Termo para lembrar: Channel boundary | Base técnica: referência 1.

25. Como registra dependências de outra equipe?

Eu identificaria owner, capacidade, latência, quotas e mecanismo de suporte da dependência. Documentaria comportamento durante indisponibilidade, porque uma seta no desenho não informa se conseguimos concluir a jornada dentro do prazo.

Termo para lembrar: Dependency contract | Base técnica: referência 1.

26. Como escolhe perguntas iniciais em sessenta minutos?

Eu faria poucas perguntas que alteram arquitetura: resultado, escala, consistência, prazo e integrações. Explicitando hipóteses restantes, avançaria para desenho e usaria os aprofundamentos para validar decisões junto ao entrevistador.

Termo para lembrar: Timeboxing | Base técnica: referência 1.

27. Como evita responsabilidade financeira no lugar errado?

Eu nomearia a autoridade do saldo e da decisão em cada etapa. Uma tela ou projeção pode apresentar informação, mas o serviço responsável revalida a operação antes de alterar dinheiro.

Termo para lembrar: System of record | Base técnica: referência 1.

28. Como distingue cancelamento de compensação?

Eu perguntaria se a ação já produzira efeito. Antes disso, cancelar pode impedir execução; depois, é necessário outro movimento autorizado e auditável, com identidade própria e ligação ao movimento original.

Termo para lembrar: Business semantics | Base técnica: referência 1.

29. Como valida que entendeu o problema?

Eu devolveria um resumo curto da jornada, garantias e exclusões, pedindo correções relevantes. Esse alinhamento fornece um contrato de conversa e evita descobrir no fim que resolvi um problema diferente.

Termo para lembrar: Playback | Base técnica: referência 1.

30. Como fecha a descoberta e começa o desenho?

Eu escolheria um fluxo essencial e anunciaria suas premissas. Desenharia atores, autoridade de dados e integração primeiro, acrescentando componentes quando uma necessidade demonstrada justificar sua responsabilidade, custo e comportamento operacional.

Termo para lembrar: Decision checkpoint | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Escopo estreito

Ganho. Permite aprofundar garantias

Custo e motivo. Adia funcionalidades

Como decidir. Quando há pouco tempo ou alta incerteza

02. Descoberta extensa

Ganho. Reduz premissas incorretas

Custo e motivo. Atrasa construção

Como decidir. Quando uma resposta pode mudar a arquitetura

03. MVP com fluxo completo

Ganho. Valida resultado real

Custo e motivo. Entrega menos canais

Como decidir. Quando integração é o maior risco

04. Requisitos mensuráveis

Ganho. Permitem teste e acompanhamento

Custo e motivo. Exigem definição de população e janela

Como decidir. Quando qualidade afeta aceitação

05. Contexto mínimo inicial

Ganho. Reduz carga e exposição

Custo e motivo. Exige carregamento progressivo

Como decidir. Quando informações são numerosas

06. Hipóteses explícitas

Ganho. Aceleram avanço

Custo e motivo. Podem exigir revisão

Como decidir. Quando dados exatos não estão disponíveis

07. Políticas configuráveis

Ganho. Adaptam regras voláteis

Custo e motivo. Aumentam governança e testes

Como decidir. Quando regras mudam com frequência

08. Ferramenta operacional

Ganho. Reduz intervenção no banco

Custo e motivo. Aumenta escopo inicial

Como decidir. Quando falhas exigem investigação recorrente

09. Exclusão explícita

Ganho. Protege tempo e ownership

Custo e motivo. Deixa dependências externas

Como decidir. Quando responsabilidade é de outro domínio

10. Detalhe de exceções

Ganho. Reduz falhas sem dono

Custo e motivo. Consome tempo de descoberta

Como decidir. Quando a consequência é financeira ou irreversível

Duas alternativas ao caminho proposto

1. EventStorming

Quando faz sentido. Quando estados e responsabilidades estão confusos

Como implementar. Mapear eventos, comandos e políticas com especialistas

Limites. Precisa de tradução posterior para contratos e implementação

2. Story mapping

Quando faz sentido. Quando o desafio principal é priorização de jornadas

Como implementar. Organizar passos do usuário e cortes de entrega

Limites. Não substitui invariantes nem análise de falhas distribuídas

Referências e alcance da evidência

1. Microsoft: Domain analysis

Análise de domínio, bounded contexts e fronteiras; casos bancários são propostas de treino.

https://learn.microsoft.com/en-us/azure/architecture/microservices/model/domain-analysis

2. Microsoft: Architecture styles

Características e critérios de estilos arquiteturais; decisões particulares são análises autorais.

https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/

3. Google SRE: Implementing SLOs

Definição de indicadores, objetivos e políticas de orçamento de erro; os exemplos de entrevista são autorais.

https://sre.google/workbook/implementing-slos/

Continue em: T002 · T004 · T119

T002 / A. Enquadramento e arquitetura

SLI, SLO, SLA e error budget

SLI mede experiência observável, SLO define um objetivo nessa medida e SLA formaliza compromissos externos. Error budget é a margem de eventos ruins permitida pelo objetivo.

Quando usar. Ao transformar expectativas de disponibilidade, latência, frescor e conclusão em critérios operacionais verificáveis.

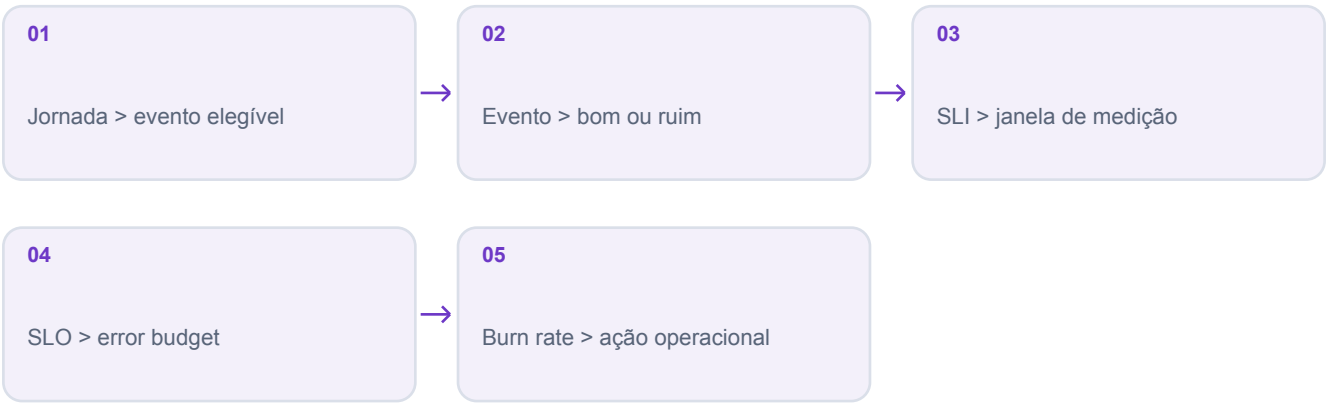
Quando evitar ou limitar. Evitar metas decorativas de 100% ou indicadores de infraestrutura que não representam a jornada do cliente.

Uma abertura possível

Vou medir sucesso do ponto de vista do cliente, separar aceitação de conclusão e negociar um objetivo cujo custo e orçamento de erro sejam explícitos.

Peças e tecnologias. Prometheus · OpenTelemetry · Grafana · CloudWatch · Azure Monitor

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Escolher jornadas e população elegível.

2. Definir sucesso, limiar e tratamento de exclusões.

3. Estabelecer janela e objetivo negociado.

4. Instrumentar borda e resultado de negócio.

5. Criar política de alertas, mudanças e recuperação.

30 perguntas para sustentar o desenho

01. Qual diferença entre SLI e SLO?

Eu definiria o SLI como proporção medida de experiências boas e o SLO como objetivo desejado. Por exemplo, contar consultas elegíveis abaixo de dois segundos e estabelecer a porcentagem exigida.

Termo para lembrar: Service level indicator | Base técnica: referência 1.

02. SLA e SLO precisam ser iguais?

Eu manteria o objetivo operacional coerente com o compromisso externo, normalmente com margem para reagir. O SLA inclui condições contratuais; não presumiria penalidades nem exclusões sem verificar o acordo aplicável.

Termo para lembrar: Service level agreement | Base técnica: referência 1.

03. Como calcula error budget?

Eu multiplicaria a fração permitida de eventos ruins pelo total elegível da janela. Em um objetivo de 99,9%, um milhão de operações permite mil ruins; esse número depende do denominador.

Termo para lembrar: Error budget | Base técnica: referência 1.

04. Por que não pedir 100%?

Eu mostraria o custo e as falhas inevitáveis de dependências e clientes. Um objetivo absoluto impede negociar risco; prefiro um patamar fundamentado e invariantes específicas que continuem obrigatórias para cada operação.

Termo para lembrar: Reliability target | Base técnica: referência 1.

05. Disponibilidade por tempo é suficiente?

Eu usaria tempo quando representar a experiência, mas operações por evento frequentemente capturam melhor o impacto. Dez minutos no pico podem prejudicar mais clientes que o mesmo intervalo durante uma madrugada.

Termo para lembrar: Request-based SLI | Base técnica: referência 1.

06. Como mede uma operação assíncrona?

Eu mediria tanto aceitação durável quanto conclusão dentro do prazo acordado. HTTP 202 não conta como sucesso financeiro final; a correlação pelo identificador permite acompanhar o resultado através das etapas.

Termo para lembrar: End-to-end SLI | Base técnica: referência 1.

07. Qual população entra no denominador?

Eu definiria requisições válidas e autorizadas, incluindo falhas causadas pelo serviço. Exclusões precisam ser explícitas e auditáveis; retirar automaticamente todo erro 4xx poderia esconder rate limiting indevido ou contrato quebrado.

Termo para lembrar: Eligible events | Base técnica: referência 1.

08. Como mede latência?

Eu escolheria limites associados à jornada e contaria eventos abaixo deles. Histogramas permitem percentis, mas não faria média de percentis entre instâncias; agregaria as observações ou buckets corretamente antes do cálculo.

Termo para lembrar: Latency threshold | Base técnica: referência 1.

09. Por que p99 em vez de média?

Eu examinaria a cauda porque poucos atendimentos muito lentos podem bloquear trabalho e gerar retries. A média esconde essa distribuição, embora o percentil também exija volume suficiente e segmentação útil.

Termo para lembrar: Tail latency | Base técnica: referência 1.

10. O que é burn rate?

Eu compararia a velocidade atual de erros à velocidade sustentável pelo orçamento. Um consumo muitas vezes superior ao previsto exige resposta rápida; a interpretação depende da janela e não apenas de uma porcentagem instantânea.

Termo para lembrar: Burn rate | Base técnica: referência 1.

11. Como evita alertas ruidosos?

Eu combinaria janelas curta e longa para confirmar consumo relevante e persistente do orçamento. Isso reduz alarmes por pequenas oscilações, sem esperar a janela mensal terminar para perceber degradações graves.

Termo para lembrar: Multiwindow alerting | Base técnica: referência 1.

12. Quando interrompe deploys?

Eu definiria previamente quem pode bloquear mudanças e quais exceções são permitidas. Orçamento esgotado direciona esforço à confiabilidade; correções urgentes de segurança ou incidentes continuam possíveis com risco explicitamente controlado.

Termo para lembrar: Error budget policy | Base técnica: referência 1.

13. Como calcula minutos de indisponibilidade?

Eu aplicaria a fração de erro à duração da janela: 99,9% em trinta dias corresponde a 43,2 minutos. Só usaria essa equivalência quando o indicador escolhido realmente for baseado em tempo.

Termo para lembrar: Time-based budget | Base técnica: referência 1.

14. RTO de uma hora combina com 99,99%?

Eu compararia o desastre previsto ao orçamento disponível e deixaria a incompatibilidade explícita. Um RTO longo pode consumir vários orçamentos; precisamos melhorar recuperação, rever a meta ou aceitar formalmente esse risco.

Termo para lembrar: Recovery alignment | Base técnica: referência 1.

15. Como trata manutenção planejada?

Eu seguiria o contrato definido, mas mediria também a experiência completa sem esconder interrupções. Excluir manutenção do SLA não remove seu impacto no cliente nem dispensa procurar formas mais seguras de mudança.

Termo para lembrar: Maintenance accounting | Base técnica: referência 1.

16. Como lida com baixo tráfego?

Eu evitaria conclusões fortes a partir de poucos eventos. Combinaria sondas sintéticas e janelas adequadas, mantendo distinção entre tráfego real e teste; um único erro pode distorcer bastante a porcentagem.

Termo para lembrar: Statistical confidence | Base técnica: referência 1.

17. Como mede frescor de dados?

Eu registraria o instante ou versão da atualização autoritativa e a posição aplicada na projeção. Definiria quanto atraso é aceitável para aquela tela; disponibilidade de uma leitura antiga não garante informação útil.

Termo para lembrar: Freshness SLI | Base técnica: referência 1.

18. Como mede correção financeira?

Eu acompanharia divergências de reconciliação e violações de invariantes separadamente da disponibilidade. Não trataria mil créditos duplicados como um orçamento aceitável; alguns eventos exigem bloqueio imediato e investigação mesmo sendo raros.

Termo para lembrar: Correctness SLI | Base técnica: referência 1.

19. SLO por serviço resolve a jornada?

Eu mapearia as dependências realmente necessárias e mediria a jornada inteira. Multiplicar disponibilidades pode ilustrar o efeito de cadeias, mas falhas correlacionadas e caminhos alternativos invalidam uma conta simplista de independência.

Termo para lembrar: Composite availability | Base técnica: referência 1.

20. Onde instrumenta o indicador?

Eu mediria próximo da experiência que prometo, complementando servidor com cliente quando necessário. Uma API pode responder rápido enquanto a tela trava; cada ponto de observação precisa declarar o que consegue enxergar.

Termo para lembrar: Measurement boundary | Base técnica: referência 1.

21. Como trata telemetria ausente?

Eu não classificaria ausência de métricas como sucesso. Criaria alerta de instrumentação e cruzaria fontes independentes; a indisponibilidade do coletor precisa ser distinguida da indisponibilidade do serviço observado durante o incidente.

Termo para lembrar: No-data condition | Base técnica: referência 1.

22. Pode usar sampling para SLO?

Eu avaliaria erro amostral e viés antes de usar amostras como denominador. Sampling de traces pode favorecer falhas ou excluir fluxos; contadores de métricas dedicados costumam ser mais adequados para contabilização.

Termo para lembrar: Sampling bias | Base técnica: referência 1.

23. Como segmenta os indicadores?

Eu separaria dimensões relevantes, como país, canal e tipo de operação, sem criar cardinalidade por cliente. Um agregado saudável pode esconder uma região quebrada; segmentação deve preservar precisão e viabilidade operacional.

Termo para lembrar: SLO segmentation | Base técnica: referência 1.

24. Como negocia confiabilidade e custo?

Eu estimaria impacto do erro e custo incremental de reduzir sua frequência. Disponibilidade adicional exige redundância e operação; a decisão precisa considerar perdas, experiência e criticidade, não apenas perseguir mais noves.

Termo para lembrar: Reliability economics | Base técnica: referência 1.

25. Como uma DLQ afeta o SLO?

Eu contaria operações na quarentena como não concluídas enquanto seu prazo corre. Mover uma mensagem para outra fila não resolve a jornada; ela precisa de responsável, diagnóstico e política de recuperação.

Termo para lembrar: Completion backlog | Base técnica: referência 1.

26. Como define alerta de fila?

Eu priorizaria idade e prazo restante das pendências, além do tamanho. Mil tarefas rápidas podem ser normais; uma única solicitação crítica esquecida há horas pode representar violação iminente e exigir intervenção.

Termo para lembrar: Age of oldest | Base técnica: referência 1.

27. Como revisa objetivos?

Eu compararia necessidade do usuário, histórico de incidentes e capacidade técnica em revisões periódicas. Mudanças são documentadas e negociadas; não relaxaria retroativamente a meta apenas para apagar uma violação já ocorrida.

Termo para lembrar: SLO governance | Base técnica: referência 1.

28. Como inclui parceiros externos?

Eu especificaria o que controlamos e a experiência oferecida durante indisponibilidade externa. Dependência fora da empresa continua afetando o usuário; distinguimos causas para ação, sem chamar pendência de conclusão bem-sucedida.

Termo para lembrar: Dependency SLO | Base técnica: referência 1.

29. Que teste valida a instrumentação?

Eu injetaria erros, lentidão e perda de coleta em ambiente controlado, verificando numerador, denominador e alertas. Compararia eventos conhecidos com métricas para detectar exclusões acidentais antes de confiar na política operacional.

Termo para lembrar: Telemetry verification | Base técnica: referência 1.

30. Como resumiria isso na entrevista?

Eu escolheria poucas jornadas críticas, daria limites mensuráveis e explicaria a reação ao consumo de orçamento. Mostraria também invariantes financeiras e recuperação, deixando claro que confiabilidade envolve resultado correto além de uptime.

Termo para lembrar: User-centric reliability | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Meta mais alta

Ganho. Menos falhas toleradas

Custo e motivo. Redundância e operação mais caras

Como decidir. Quando impacto da indisponibilidade justifica

02. SLI por requisição

Ganho. Pondera o tráfego afetado

Custo e motivo. Depende de denominador confiável

Como decidir. Quando operações representam experiência

03. SLI por tempo

Ganho. Simplifica monitoramento contínuo

Custo e motivo. Ignora variação de carga

Como decidir. Quando disponibilidade contínua é o contrato

04. Janela curta

Ganho. Detecta mudanças cedo

Custo e motivo. Maior variância

Como decidir. Quando precisamos agir rápido

05. Janela longa

Ganho. Estabiliza avaliação

Custo e motivo. Pode esconder incidente recente

Como decidir. Quando combinada com alerta de curto prazo

06. Segmentação fina

Ganho. Revela grupos prejudicados

Custo e motivo. Aumenta cardinalidade e ruído

Como decidir. Quando há jornadas distintas

07. Sondas sintéticas

Ganho. Cobrem períodos ociosos

Custo e motivo. Não reproduzem todo tráfego

Como decidir. Quando volume real é baixo

08. Métricas no cliente

Ganho. Medem experiência final

Custo e motivo. Telemetria pode chegar atrasada

Como decidir. Quando renderização e rede importam

09. Bloqueio de mudanças

Ganho. Protege orçamento

Custo e motivo. Atrasa entrega

Como decidir. Quando falhas recorrentes exigem estabilização

10. Objetivo de frescor

Ganho. Torna atraso explícito

Custo e motivo. Exige correlação de versões

Como decidir. Quando projeções assíncronas alimentam decisões

Duas alternativas ao caminho proposto

1. Objetivos por jornada

Quando faz sentido. Quando métricas isoladas de serviço escondem falhas compostas

Como implementar. Correlacionar aceitação, processamento e conclusão

Limites. Exige identificação persistente e política para resultados desconhecidos

2. Guardrails por invariante

Quando faz sentido. Quando perdas raras são inaceitáveis

Como implementar. Bloquear ou escalar diante de violação financeira ou de autorização

Limites. Não substituem metas de latência e disponibilidade

Referências e alcance da evidência

1. Google SRE: Implementing SLOs

Definição de indicadores, objetivos e políticas de orçamento de erro; os exemplos de entrevista são autorais.

https://sre.google/workbook/implementing-slos/

2. Google SRE: Handling Overload

Sobrecarga, filas e rejeição de trabalho; números e dimensionamentos são hipóteses didáticas.

https://sre.google/sre-book/handling-overload/

Continue em: T001 · T068 · T085

T003 / A. Enquadramento e arquitetura

Estimativas de capacidade e crescimento

Dimensionamento estima taxas, concorrência, bytes e crescimento a partir de jornadas e hipóteses explícitas; serve para identificar limites e planejar medições, não para prometer capacidade por aritmética.

Quando usar. Quando escolhas de banco, filas, rede, retenção e capacidade humana dependem do volume esperado.

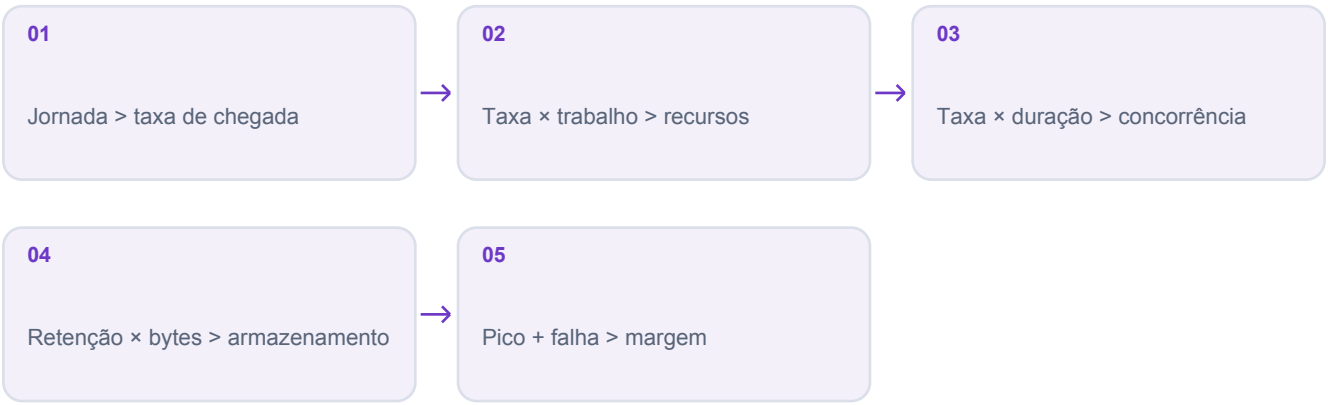
Quando evitar ou limitar. Evitar precisão artificial, usar média como pico ou confundir usuários cadastrados com operações simultâneas.

Uma abertura possível

Vou transformar a jornada em operações por segundo, bytes e trabalho acumulado. Usarei ordens de grandeza e depois validarei o gargalo com carga representativa.

Peças e tecnologias. Prometheus · k6 · Locust · fio · pgbench

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Estimar usuários ativos e frequência por jornada.

2. Converter volume diário em média e explicitar pico.

3. Calcular fan-out, payload, retenção e concorrência.

4. Considerar capacidade perdida e drenagem de backlog.

5. Testar hipóteses e revisar limites com telemetria.

30 perguntas para sustentar o desenho

01. Como transforma volume diário em RPS?

Eu dividiria operações diárias por 86.400 segundos para obter a média. Depois aplicaria um perfil de pico justificado; média diária não dimensiona um fluxo concentrado em poucas horas.

Termo para lembrar: Arrival rate | Base técnica: referência 1.

02. Como estima usuários simultâneos?

Eu separaria conexões abertas de usuários executando ações. Para trabalho em andamento, usaria chegada multiplicada pela duração média sob estabilidade, verificando filas e comportamento real antes de extrapolar.

Termo para lembrar: Concurrency | Base técnica: referência 1.

03. Como calcula carga de fan-out?

Eu multiplicaria a taxa de jornadas pelas chamadas necessárias por jornada, incluindo cache misses e retries. Uma API com cem RPS pode gerar milhares de consultas nas dependências.

Termo para lembrar: Request amplification | Base técnica: referência 1.

04. Como estima armazenamento mensal?

Eu multiplicaria eventos diários, bytes médios e dias de retenção, acrescentando índices, réplicas, WAL e backups separadamente. Payload lógico não equivale ao espaço físico total necessário em produção.

Termo para lembrar: Retention sizing | Base técnica: referência 1.

05. Como dimensiona largura de banda?

Eu calcularia bytes por operação vezes operações por segundo, distinguindo entrada, saída e replicação. Conversões entre bits e bytes e tráfego entre regiões podem mudar bastante custo e capacidade.

Termo para lembrar: Network throughput | Base técnica: referência 1.

06. O que faz quando só conhece usuários cadastrados?

Eu perguntaria a fração ativa e a frequência de ações por usuário, declarando hipóteses provisórias. Cadastros indicam potencial de armazenamento, mas não determinam diretamente concorrência nem taxa instantânea de acesso.

Termo para lembrar: Activity ratio | Base técnica: referência 1.

07. Como escolhe fator de pico?

Eu usaria histórico de sazonalidade, campanhas e horários de processamento. Sem histórico, apresentaria cenários conservadores distintos; não escolheria dez vezes a média como verdade universal sem relação com a jornada.

Termo para lembrar: Peak factor | Base técnica: referência 1.

08. Quanto tempo leva para drenar uma fila?

Eu dividiria o backlog pela capacidade de processamento menos a entrada corrente. Se saída não superar entrada, não há drenagem; aumentar consumidores só ajuda quando as dependências suportam a vazão adicional.

Termo para lembrar: Backlog drain | Base técnica: referência 1.

09. Como usa Little's Law?

Eu aplicaria L igual a lambda vezes W para médias em um sistema estável e fronteiras consistentes. Não substituiria duração média por p99 nem usaria a equação para provar baixa espera.

Termo para lembrar: Little’s Law | Base técnica: referência 1.

10. Como considera falha de uma zona?

Eu verificaria se a capacidade remanescente atende a carga essencial após perder a zona. Operar todos os nós perto do limite normal pode deixar nenhuma margem justamente durante o failover.

Termo para lembrar: N-minus-one capacity | Base técnica: referência 1.

11. Como estima CPU?

Eu mediria tempo efetivo de CPU por operação e multiplicaria pela taxa. Espera de rede não consome CPU da mesma forma; perfis de serialização, criptografia e coleta de lixo entram separadamente.

Termo para lembrar: CPU service demand | Base técnica: referência 1.

12. Como estima memória por conexão?

Eu mediria buffers, estado de sessão e overhead do runtime por conexão, considerando mensagens pendentes. Multiplicaria pelo pico e limitaria buffers; conexão lenta não pode gerar memória ilimitada no gateway.

Termo para lembrar: Connection footprint | Base técnica: referência 1.

13. Como identifica chave quente?

Eu observaria distribuição de acesso por chave e percentis de carga por partição. Uniformidade global não protege uma conta ou tenant popular; o maior consumidor pode determinar o limite real do shard.

Termo para lembrar: Skew | Base técnica: referência 1.

14. Como considera cache no cálculo?

Eu calcularia carga de origem a partir dos misses, incluindo aquecimento e indisponibilidade do cache. Planejaria proteção para perda do hit rate; assumir noventa e nove por cento continuamente é arriscado.

Termo para lembrar: Hit ratio | Base técnica: referência 1.

15. Como calcula capacidade de atendimento humano?

Eu estimaria conversas simultâneas, duração, paralelismo real por agente e ocupação desejada. Pausas e habilidades reduzem disponibilidade; escalar infraestrutura não reduz espera quando faltam pessoas aptas na fila correta.

Termo para lembrar: Human capacity | Base técnica: referência 1.

16. Como estima limite do banco?

Eu combinaria testes representativos e métricas de CPU, I/O, locks e latência. Contar consultas por segundo isoladamente engana, porque uma busca indexada e uma agregação extensa exigem trabalhos muito diferentes.

Termo para lembrar: Saturation point | Base técnica: referência 1.

17. Como trata crescimento anual?

Eu apresentaria cenários de crescimento e o recurso que esgota primeiro, como armazenamento ou throughput de escrita. A decisão incluiria tempo necessário para expansão, evitando reagir somente quando o limite chegar.

Termo para lembrar: Growth model | Base técnica: referência 1.

18. Quando escala verticalmente?

Eu compararia capacidade disponível, custo e tempo operacional. Uma máquina maior pode resolver o horizonte necessário com menos complexidade; avaliaria limites físicos e disponibilidade antes de introduzir particionamento distribuído prematuro.

Termo para lembrar: Scale up | Base técnica: referência 1.

19. Como mede recuperação de cold start?

Eu testaria instâncias novas com cache frio, compilação e pools ainda não estabelecidos. A capacidade inicial pode ser menor que a estável; por isso autoscaling precisa antecipação e limites de admissão.

Termo para lembrar: Warm-up capacity | Base técnica: referência 1.

20. Como escolhe o tamanho do lote?

Eu equilibraria amortização de overhead com memória e latência de espera. Lotes grandes aumentam impacto de retry e recuperação; escolheria limites por bytes e quantidade, não apenas número de registros.

Termo para lembrar: Batch sizing | Base técnica: referência 1.

21. Como dimensiona conexões de banco?

Eu distribuiria o orçamento de conexões entre réplicas da aplicação e reservaria capacidade administrativa. Autoscaling multiplica pools; muitas conexões ociosas ou concorrentes podem piorar o banco em vez de aumentar throughput.

Termo para lembrar: Pool budget | Base técnica: referência 1.

22. Como considera retries na capacidade?

Eu estimaria a carga adicional durante falhas e imporia orçamento de repetição. Se cada camada tenta três vezes, a amplificação pode crescer drasticamente; definiria uma camada responsável e propagaria deadlines.

Termo para lembrar: Retry amplification | Base técnica: referência 1.

23. Como considera tarefas de manutenção?

Eu reservaria recursos para vacuum, backups, compaction e reindexação. Medir capacidade apenas com tráfego online produz uma estimativa otimista que falha quando a manutenção obrigatória concorre com solicitações reais.

Termo para lembrar: Background load | Base técnica: referência 1.

24. Como valida um benchmark?

Eu reproduziria distribuição de dados, consultas, chaves quentes e concorrência, usando gerador independente da latência do servidor. Compararia throughput sustentável e cauda, não apenas um pico curto antes da saturação.

Termo para lembrar: Representative workload | Base técnica: referência 1.

25. Como trata estimativas incertas?

Eu variaria as hipóteses com maior efeito, como payload e fator de pico, mostrando quando a decisão muda. Essa análise informa quais dados precisamos medir primeiro e evita precisão sem evidência.

Termo para lembrar: Sensitivity analysis | Base técnica: referência 1.

26. Como calcula custo por operação?

Eu dividiria custo relevante pelo volume útil concluído e separaria custos fixos e variáveis. Retries e processamento descartado aumentam custo sem criar valor; regiões e retenção podem dominar a conta.

Termo para lembrar: Unit economics | Base técnica: referência 1.

27. Qual relação entre utilização e espera?

Eu esperaria crescimento acentuado de fila próximo da saturação, dependendo da variabilidade. Não planejaria operar continuamente a cem por cento; preservaria margem para picos, falhas e distribuição desigual de trabalho.

Termo para lembrar: Queueing delay | Base técnica: referência 1.

28. Como distingue limite de quota de capacidade física?

Eu verificaria limites configurados do serviço e capacidade efetivamente disponível. Aumentar uma quota não cria automaticamente recursos no parceiro; o plano precisa considerar negociação, lead time e teste de carga autorizado.

Termo para lembrar: Quota versus capacity | Base técnica: referência 1.

29. Como dimensiona replay?

Eu calcularia eventos históricos, taxa de reconstrução e tráfego concorrente, isolando consumidores e recursos. Replay pode exceder muito a carga normal; precisa de controle para não atrasar operações novas nem sobrecarregar dependências.

Termo para lembrar: Recovery workload | Base técnica: referência 1.

30. Como apresenta as contas na entrevista?

Eu mostraria poucas equações com unidades e hipóteses claras, conectando cada resultado a uma decisão. Encerraria indicando medições necessárias; números aproximados orientam o desenho, mas não substituem validação de capacidade real.

Termo para lembrar: Back-of-envelope | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Mais margem

Ganho. Tolera picos e falhas

Custo e motivo. Custa ociosidade

Como decidir. Quando recuperação exige capacidade imediata

02. Escala vertical

Ganho. Simplifica operação

Custo e motivo. Possui teto físico

Como decidir. Quando resolve o horizonte previsto

03. Escala horizontal

Ganho. Distribui trabalho

Custo e motivo. Exige coordenação

Como decidir. Quando tarefa é particionável

04. Lotes maiores

Ganho. Amortizam overhead

Custo e motivo. Aumentam espera

Como decidir. Quando prazo permite agrupamento

05. Retenção longa

Ganho. Permite histórico

Custo e motivo. Amplia armazenamento

Como decidir. Quando existe uso comprovado

06. Compressão

Ganho. Reduz bytes

Custo e motivo. Consome CPU

Como decidir. Quando rede ou disco predominam

07. Cache maior

Ganho. Reduz leitura de origem

Custo e motivo. Consome memória

Como decidir. Quando há reutilização

08. Benchmark realista

Ganho. Melhora estimativa

Custo e motivo. Custa preparação

Como decidir. Quando decisão é difícil de reverter

09. Reservar capacidade

Ganho. Reduz risco de quota

Custo e motivo. Aumenta custo fixo

Como decidir. Quando pico é previsível

10. Admissão limitada

Ganho. Protege estabilidade

Custo e motivo. Recusa ou adia trabalho

Como decidir. Quando demanda supera recursos

Duas alternativas ao caminho proposto

1. Teste de carga incremental

Quando faz sentido. Quando já existe implementação

Como implementar. Aumentar demanda até identificar limite sustentável

Limites. Resultado depende de representatividade e ambiente

2. Modelo de cenários

Quando faz sentido. Quando produto ainda não existe

Como implementar. Comparar hipóteses de pico, bytes e crescimento

Limites. Produz intervalo de decisões, não prova de throughput

Referências e alcance da evidência

1. Google SRE: Handling Overload

Sobrecarga, filas e rejeição de trabalho; números e dimensionamentos são hipóteses didáticas.

https://sre.google/sre-book/handling-overload/

2. Google SRE: Implementing SLOs

Definição de indicadores, objetivos e políticas de orçamento de erro; os exemplos de entrevista são autorais.

https://sre.google/workbook/implementing-slos/

Continue em: T025 · T084 · T085

T004 / A. Enquadramento e arquitetura

DDD, domínio, agregados e invariantes

DDD organiza software segundo conceitos do negócio. Bounded contexts delimitam significados; agregados delimitam consistência e invariantes que precisam ser preservadas conjuntamente.

Quando usar. Quando regras, linguagem e responsabilidades mudam entre produtos e equipes, especialmente em processos financeiros com decisões e estados complexos.

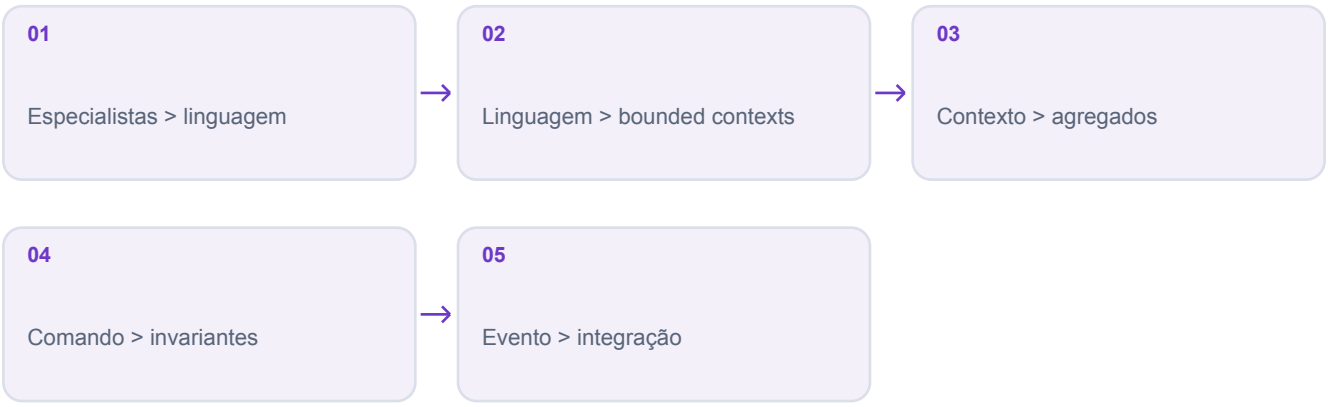
Quando evitar ou limitar. Evitar inventar agregados enormes ou transformar cada substantivo em microsserviço; DDD não exige distribuição física.

Uma abertura possível

Vou descobrir quais regras precisam ser verdadeiras após cada comando e quem tem autoridade sobre elas. Só depois defino agregado, contrato e eventual separação de serviços.

Peças e tecnologias. EventStorming · Clojure · Kotlin · PostgreSQL · OpenAPI

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Mapear termos e significados com especialistas.

2. Identificar invariantes e autoridades de dados.

3. Definir agregados pequenos com fronteiras transacionais.

4. Modelar comandos, eventos e estados inválidos.

5. Integrar contextos por contratos e revisar fronteiras.

30 perguntas para sustentar o desenho

01. O que é bounded context?

Eu o trataria como limite dentro do qual um modelo e seus termos têm significado consistente. Cliente no atendimento pode diferir de cliente no crédito, exigindo tradução explícita entre os contextos.

Termo para lembrar: Bounded context | Base técnica: referência 1.

02. Agregado é uma tabela?

Eu definiria agregado pela unidade de regras e consistência, não pela estrutura física. Ele pode ocupar várias tabelas, desde que sua alteração preserve invariantes dentro da fronteira transacional escolhida para aquele comando.

Termo para lembrar: Aggregate | Base técnica: referência 1.

03. Como encontra uma invariante?

Eu perguntaria que estado seria inadmissível mesmo sob concorrência. Uma reserva não pode ultrapassar limite disponível; essa regra orienta atualização atômica e autoridade, em vez de depender apenas de validação na interface.

Termo para lembrar: Invariant | Base técnica: referência 1.

04. Como escolhe a raiz do agregado?

Eu escolheria a entidade que controla alterações nas partes relacionadas e protege suas regras. Consumidores enviariam comandos pela raiz; acesso direto às estruturas internas não pode contornar validações e comprometer consistência.

Termo para lembrar: Aggregate root | Base técnica: referência 1.

05. Por que evitar agregado gigante?

Eu mostraria como reunir todo cliente, cartões e contas numa transação cria contenção e acoplamento. Separaria regras independentes, mantendo juntas apenas as alterações que realmente precisam de atomicidade naquele caso.

Termo para lembrar: Contention boundary | Base técnica: referência 1.

06. Quando duas entidades pertencem ao mesmo agregado?

Eu verificaria se uma regra exige atualizar ambas atomicamente e com frequência. Relacionamento no modelo não basta; quando consistência eventual é aceitável, referências entre agregados podem preservar autonomia e reduzir bloqueios.

Termo para lembrar: Transactional invariant | Base técnica: referência 1.

07. Como diferencia entidade de value object?

Eu daria identidade própria a objetos acompanhados ao longo do tempo, como conta. Para valor monetário, moeda e quantia definem igualdade; modelaria validações e arredondamento sem inventar um identificador desnecessário.

Termo para lembrar: Value object | Base técnica: referência 1.

08. Onde fica lógica de domínio?

Eu colocaria regras e transições perto do modelo que as possui, preservando testes independentes de HTTP. Persistência e integração ficam em componentes próprios, evitando regras duplicadas em controladores, jobs e interfaces.

Termo para lembrar: Domain behavior | Base técnica: referência 1.

09. O que é serviço de domínio?

Eu usaria quando uma regra relevante não pertence naturalmente a uma entidade ou valor. Ele expressa o negócio, enquanto o serviço de aplicação coordena transação e dependências necessárias para executar o caso.

Termo para lembrar: Domain service | Base técnica: referência 1.

10. Como evita modelo anêmico?

Eu faria o modelo expor operações que preservam invariantes, como aprovar uma decisão válida, em vez de setters irrestritos. Para CRUD simples, porém, não criaria camadas elaboradas sem regra suficiente para justificar.

Termo para lembrar: Rich domain model | Base técnica: referência 1.

11. Como representa evento de domínio?

Eu registraria um fato já ocorrido, com identidade, contexto e versão, como DisputaAprovada. Um comando solicita uma ação e pode ser rejeitado; misturar essas semânticas confunde quem deve validar e responder.

Termo para lembrar: Domain event | Base técnica: referência 1.

12. Como integra dois contextos?

Eu definiria o contrato e o modelo traduzido na fronteira, indicando upstream, downstream e responsabilidade. Não compartilharia classes de domínio indiscriminadamente, pois isso propaga mudanças internas e impede evolução independente.

Termo para lembrar: Context mapping | Base técnica: referência 1.

13. Quando usa anti-corruption layer?

Eu a usaria para traduzir estados, códigos e semântica de um legado ou parceiro. Essa camada protege nosso modelo, sem esconder falhas ou inventar garantias que a integração externa não oferece.

Termo para lembrar: Anti-corruption layer | Base técnica: referência 1.

14. DDD obriga microsserviços?

Eu começaria com módulos e interfaces quando a escala e a organização permitirem. Contextos podem compartilhar processo; separação física só vale quando autonomia, isolamento ou escala compensam o custo distribuído adicional.

Termo para lembrar: Modular domain | Base técnica: referência 1.

15. Como protege invariante sob duas requisições?

Eu combinaria a regra com um mecanismo no armazenamento, como versão ou lock. Validar saldo em memória e depois salvar não protege concorrência; a checagem precisa participar da decisão atômica.

Termo para lembrar: Concurrency control | Base técnica: referência 1.

16. Como lida com regra entre agregados?

Eu avaliaria se a fronteira foi mal escolhida ou se uma reserva resolve a necessidade. Quando há processo distribuído, modelaria estados intermediários e compensações, sem chamar consistência eventual de atomicidade global.

Termo para lembrar: Cross-aggregate rule | Base técnica: referência 1.

17. Como trata dinheiro no modelo?

Eu representaria quantia e moeda explicitamente, usando precisão adequada e regra de arredondamento definida. Conversões exigem cotação e validade; somar valores de moedas distintas sem contexto seria um estado inválido.

Termo para lembrar: Money value object | Base técnica: referência 1.

18. Como modela aprovação dupla?

Eu ligaria aprovação à revisão e ao conteúdo da decisão, verificando que aprovador e autor são distintos quando exigido. Se o conteúdo mudar, a aprovação antiga não valida automaticamente a nova proposta.

Termo para lembrar: Maker-checker | Base técnica: referência 1.

19. Como representa cancelamento?

Eu verificaria a fase do processo antes de aceitar o comando. Cancelar trabalho ainda não executado difere de compensar um efeito concluído; o modelo deve distinguir essas consequências e seus responsáveis.

Termo para lembrar: State transition | Base técnica: referência 1.

20. Como define identidade de negócio?

Eu escolheria uma chave que represente a intenção correta, considerando operações parciais e repetidas legitimamente. Um ID de compra sozinho pode bloquear duas disputas válidas; unicidade precisa refletir a regra real.

Termo para lembrar: Business identity | Base técnica: referência 1.

21. Como mantém histórico sem event sourcing?

Eu persistiria estado atual e transições relevantes na mesma transação, com ator e motivo. Isso fornece rastreabilidade sem exigir reconstrução total por eventos, desde que retenção e integridade da auditoria sejam definidas.

Termo para lembrar: Audit history | Base técnica: referência 1.

22. Como versiona regras?

Eu registraria a política aplicada à decisão para explicar resultados históricos. Reprocessar com regra nova pode produzir outra conclusão; o modelo precisa distinguir correção operacional de uma nova decisão de negócio.

Termo para lembrar: Policy version | Base técnica: referência 1.

23. Como trata tempo no domínio?

Eu passaria o instante e calendário necessários explicitamente, tornando vencimentos testáveis. Usar relógio global escondido dificulta reproduzir casos; dias úteis, timezone e cutoff pertencem à política do processo.

Termo para lembrar: Domain clock | Base técnica: referência 1.

24. Como organiza ownership?

Eu atribuiria a uma equipe responsabilidade por contrato, invariantes, operação e evolução do contexto. Dividir apenas por camada técnica multiplica coordenação para uma mudança simples e deixa regras sem dono claro.

Termo para lembrar: Domain ownership | Base técnica: referência 1.

25. Como testa agregados?

Eu testaria comandos válidos, transições proibidas e sequências aleatórias que tentam violar invariantes. Complementaria com concorrência no armazenamento, porque testes unitários do modelo não demonstram isolamento da implementação persistente.

Termo para lembrar: Property-based testing | Base técnica: referência 1.

26. Como identifica contexto errado?

Eu observaria mudanças que sempre exigem coordenação entre as mesmas equipes e transações distribuídas frequentes. Isso pode indicar fronteira ruim, embora dependência ocasional não justifique fundir modelos e responsabilidades distintos.

Termo para lembrar: Boundary feedback | Base técnica: referência 1.

27. Como usa linguagem ubíqua sem exagerar?

Eu manteria nomes reconhecidos pelos especialistas em contratos e código, esclarecendo ambiguidades por contexto. Não transformaria workshops em objetivo final; a linguagem precisa melhorar decisões, testes e entendimento do comportamento operacional.

Termo para lembrar: Ubiquitous language | Base técnica: referência 1.

28. Como migra um domínio legado?

Eu introduziria uma fronteira de tradução, moveria casos de uso progressivamente e definiria uma autoridade de escrita por etapa. Comparações em sombra ajudam, mas escrita dupla sem coordenação pode divergir silenciosamente.

Termo para lembrar: Strangler migration | Base técnica: referência 1.

29. Como trata um modelo de leitura?

Eu projetaria consultas conforme a tarefa do usuário, sem exigir que toda tela atravesse agregados completos. A projeção pode juntar contextos, mas não passa a ter autoridade para aprovar alterações financeiras.

Termo para lembrar: Read model | Base técnica: referência 1.

30. Como explicaria DDD em um minuto?

Eu diria que o modelo deve expressar regras, linguagem e responsabilidades reais. Usaria um exemplo de invariante e sua fronteira transacional, mostrando por que o desenho reduz mudanças acopladas e estados inválidos.

Termo para lembrar: Domain-driven design | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Agregado maior

Ganho. Protege mais regras atomicamente

Custo e motivo. Aumenta contenção

Como decidir. Quando invariantes exigem alteração conjunta

02. Agregado menor

Ganho. Permite paralelismo

Custo e motivo. Exige coordenação externa

Como decidir. Quando regras são independentes

03. Modelo rico

Ganho. Centraliza regras

Custo e motivo. Aumenta investimento inicial

Como decidir. Quando domínio tem comportamento complexo

04. CRUD simples

Ganho. Entrega rápido

Custo e motivo. Pode dispersar regras futuras

Como decidir. Quando comportamento é limitado

05. Eventos de domínio

Ganho. Expõem fatos relevantes

Custo e motivo. Exigem evolução de contrato

Como decidir. Quando outros contextos reagem

06. Contextos separados

Ganho. Permitem autonomia

Custo e motivo. Exigem tradução

Como decidir. Quando significados diferem

07. Biblioteca compartilhada

Ganho. Reduz duplicação

Custo e motivo. Acopla deploys e modelos

Como decidir. Quando semântica é estável e comum

08. Versão otimista

Ganho. Evita lock prolongado

Custo e motivo. Exige resolver conflitos

Como decidir. Quando colisões são raras

09. Políticas versionadas

Ganho. Explicam decisões antigas

Custo e motivo. Aumentam armazenamento e testes

Como decidir. Quando regras mudam

10. Projeção especializada

Ganho. Melhora consultas

Custo e motivo. Pode atrasar

Como decidir. Quando UX aceita frescor limitado

Duas alternativas ao caminho proposto

1. Transaction script

Quando faz sentido. Quando regras são simples e procedimentos claros

Como implementar. Organizar cada caso de uso com transação e validações explícitas

Limites. Pode duplicar regras conforme o domínio cresce

2. Modelo funcional

Quando faz sentido. Quando transformações e imutabilidade ajudam o raciocínio

Como implementar. Representar estado e comandos como funções puras com efeitos externos separados

Limites. Ainda exige autoridade e concorrência na persistência

Referências e alcance da evidência

1. Microsoft: Domain analysis

Análise de domínio, bounded contexts e fronteiras; casos bancários são propostas de treino.

https://learn.microsoft.com/en-us/azure/architecture/microservices/model/domain-analysis

2. Microsoft: Architecture styles

Características e critérios de estilos arquiteturais; decisões particulares são análises autorais.

https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/

3. PostgreSQL: Transaction isolation

Isolamento, snapshots e necessidade de repetição de transações sob conflitos.

https://www.postgresql.org/docs/current/transaction-iso.html

Continue em: T006 · T007 · T091

T005 / A. Enquadramento e arquitetura

Monólito modular e microsserviços

Monólito modular separa responsabilidades dentro de um deploy; microsserviços distribuem componentes com contratos e operação independentes. A escolha depende de autonomia, isolamento e custos de coordenação.

Quando usar. Ao decidir unidades de desenvolvimento, implantação, escala e falha.

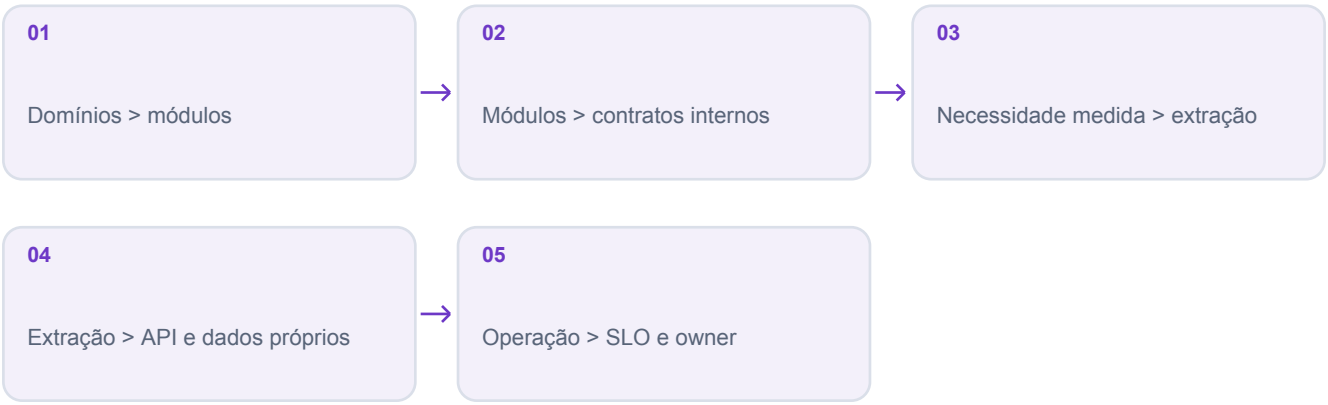
Quando evitar ou limitar. Evitar distribuição por moda ou monólito sem limites internos; ambos podem produzir forte acoplamento.

Uma abertura possível

Vou começar pela fronteira de domínio e pelas necessidades de escala. Separarei processos quando o benefício de autonomia e isolamento superar rede, consistência e operação adicionais.

Peças e tecnologias. Spring Modulith · Kotlin · Go · Docker · ECS · Kubernetes

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Identificar contextos e dependências.

2. Definir interfaces e acesso aos dados por módulo.

3. Medir gargalos e necessidades de implantação.

4. Extrair uma fronteira de forma incremental.

5. Comparar custo operacional e impacto nas equipes.

30 perguntas para sustentar o desenho

01. Quando começaria com monólito modular?

Eu escolheria um deploy com módulos bem delimitados quando equipe, carga e ciclo de mudança permitirem. Preservaria interfaces e ownership dos dados para evitar que simplicidade operacional vire acoplamento interno.

Termo para lembrar: Modular monolith | Base técnica: referência 1.

02. Quando extrairia um microsserviço?

Eu procuraria necessidade comprovada de escala, isolamento ou implantação independente. Extrair uma função estável reduz coordenação, mas adiciona rede e operação; o benefício precisa superar esses custos no contexto da equipe.

Termo para lembrar: Independent deployment | Base técnica: referência 1.

03. Um monólito não escala?

Eu lembraria que um monólito stateless pode ter várias instâncias e banco escalado separadamente. O limite aparece quando partes exigem recursos muito diferentes ou contenção interna impede aproveitar a capacidade disponível.

Termo para lembrar: Vertical and horizontal scaling | Base técnica: referência 1.

04. Como impede acesso entre tabelas dos módulos?

Eu definiria repositórios privados, interfaces públicas e verificações arquiteturais no build. Quando viável, schemas e permissões reforçam o limite; uma convenção ignorada por consultas diretas não garante modularidade real.

Termo para lembrar: Data ownership | Base técnica: referência 1.

05. Como identifica monólito distribuído?

Eu observaria deploys coordenados, chamadas encadeadas e mudanças que atravessam vários serviços. Ter processos separados sem autonomia conserva acoplamento e acrescenta falhas de rede, exigindo rever contratos e fronteiras.

Termo para lembrar: Distributed monolith | Base técnica: referência 1.

06. Banco compartilhado é sempre proibido?

Eu distinguiria infraestrutura compartilhada de propriedade compartilhada. Serviços podem usar o mesmo cluster com limites claros, mas escrever nas tabelas alheias acopla schema e regras, dificultando evolução e responsabilidade operacional.

Termo para lembrar: Shared database | Base técnica: referência 1.

07. Como trata transação antes da extração?

Eu identificaria quais alterações eram atômicas no processo original. Após separação, precisaremos manter a fronteira ou modelar estados intermediários; trocar chamada local por HTTP sem revisar garantias cria inconsistências.

Termo para lembrar: Transactional boundary | Base técnica: referência 1.

08. Como evita chamadas excessivamente pequenas?

Eu criaria operações alinhadas a tarefas de negócio, reduzindo requisições por entidade. Contratos muito finos multiplicam latência e falhas; agrupamento precisa preservar autorização e evitar APIs genéricas que vazam armazenamento.

Termo para lembrar: Chatty communication | Base técnica: referência 1.

09. Como organiza bibliotecas compartilhadas?

Eu limitaria compartilhamento a semântica estável e infraestrutura reutilizável. Uma biblioteca com todos os modelos de negócio obriga atualizações coordenadas; contratos versionados devem permitir consumidores evoluírem com independência real.

Termo para lembrar: Shared kernel | Base técnica: referência 1.

10. Como faz testes em microsserviços?

Eu combinaria testes locais, contratos entre consumidores e provedores e poucos fluxos integrados críticos. Um ambiente completo para todo teste é caro e frágil; mocks isolados também não comprovam compatibilidade real.

Termo para lembrar: Contract testing | Base técnica: referência 1.

11. Como monitora uma chamada distribuída?

Eu propagaria contexto de trace e identidade da operação, correlacionando latência por dependência. Traces complementam métricas de negócio; amostragem e dados sensíveis exigem cuidado para não produzir observabilidade cara ou incompleta.

Termo para lembrar: Distributed tracing | Base técnica: referência 1.

12. Como controla retries entre serviços?

Eu definiria qual camada repete e propagaria deadline total. Cada serviço repetir independentemente multiplica carga durante incidente; só repetiria operações seguras, com backoff e orçamento compatível com a jornada.

Termo para lembrar: Retry budget | Base técnica: referência 1.

13. Como compara custos?

Eu incluiria infraestrutura, pipelines, observabilidade, plantão e coordenação, não apenas CPU. Serviços pequenos podem custar mais em operação; a escolha deve considerar capacidade da equipe para manter contratos e incidentes.

Termo para lembrar: Total cost of ownership | Base técnica: referência 1.

14. Qual tamanho ideal de serviço?

Eu não usaria linhas de código como regra. Escolheria uma responsabilidade com invariantes e owner claros, ajustando conforme mudanças e carga; serviços minúsculos podem criar dependências maiores que seus benefícios.

Termo para lembrar: Service granularity | Base técnica: referência 1.

15. Como implanta sem quebrar consumidores?

Eu faria mudanças aditivas, manteria contratos anteriores durante migração e observaria uso antes de remover campos. Deploy independente exige compatibilidade; versionar a URL sozinho não elimina a necessidade de transição segura.

Termo para lembrar: Backward compatibility | Base técnica: referência 1.

16. Como decide entre sync e eventos?

Eu usaria síncrono quando o resultado imediato é necessário e a dependência cabe no prazo. Eventos servem para reações desacopladas, mas exigem atraso aceitável, identidade, recuperação e contratos duráveis de mensagem.

Termo para lembrar: Temporal coupling | Base técnica: referência 1.

17. Como faz migração incremental?

Eu rotearia um caso de uso para a implementação nova, com autoridade de escrita definida. Compararia resultados e ampliaria gradualmente; escrita dupla sem reconciliação apenas troca uma migração controlada por divergência silenciosa.

Termo para lembrar: Strangler pattern | Base técnica: referência 1.

18. Como evita big bang de dados?

Eu adicionaria estruturas compatíveis, faria backfill limitado e verificaria equivalência antes do corte. Depois removeria o caminho antigo quando não houvesse consumidores; rollback precisa considerar dados produzidos pela versão nova.

Termo para lembrar: Expand-contract | Base técnica: referência 1.

19. Como distribui equipes?

Eu alinharia uma equipe capaz de desenvolver e operar o serviço à responsabilidade de negócio. Separar times por camada pode exigir múltiplas filas organizacionais para cada entrega, anulando a autonomia técnica pretendida.

Termo para lembrar: Team ownership | Base técnica: referência 1.

20. Como isola uma falha?

Eu limitaria concorrência, pools e recursos por dependência ou classe de trabalho. Processos separados ajudam, mas um banco ou gateway compartilhado saturado continua podendo derrubar todos os serviços da jornada.

Termo para lembrar: Bulkhead | Base técnica: referência 1.

21. Quando não separaria um módulo?

Eu manteria junto quando as regras mudam em conjunto e não existe benefício de escala ou operação. A extração precoce congela fronteiras imaturas e transforma refatorações simples em migrações de contratos.

Termo para lembrar: Premature distribution | Base técnica: referência 1.

22. Como um monólito pode ser resiliente?

Eu usaria réplicas, health checks, isolamento de trabalho pesado e limites de recursos. Mesmo num processo único, filas e pools separados reduzem contágio; disponibilidade depende do desenho inteiro, não do rótulo arquitetural.

Termo para lembrar: Process resilience | Base técnica: referência 1.

23. Como trata consultas entre domínios?

Eu compararia composição por API com uma projeção específica. Consultar dezenas de serviços por tela eleva fan-out; uma visão materializada melhora leitura, mas introduz atraso e precisa respeitar autorização e ownership.

Termo para lembrar: Read composition | Base técnica: referência 1.

24. Como usa eventos dentro do monólito?

Eu distinguiria callbacks em memória de obrigações duráveis. Uma reação crítica precisa participar de transação ou persistir trabalho; publicar um evento local não garante execução depois de uma queda do processo.

Termo para lembrar: In-process event | Base técnica: referência 1.

25. Como sabe que a modularização funcionou?

Eu mediria mudanças que atravessam módulos, tempo de entrega e incidentes causados por dependências. Diagramas bonitos não bastam; a fronteira precisa reduzir coordenação sem esconder duplicação de regras ou dados.

Termo para lembrar: Change coupling | Base técnica: referência 1.

26. Como trata autorização nos serviços?

Eu autenticaria na entrada e faria autorização no serviço dono do recurso. Um gateway não conhece necessariamente todas as regras; confiar apenas na rede interna deixa ações sensíveis expostas a chamadas indevidas.

Termo para lembrar: Defense in depth | Base técnica: referência 1.

27. Como mantém ambiente de desenvolvimento simples?

Eu permitiria executar o serviço com dependências mínimas e contratos simulados, oferecendo integração real para testes necessários. Exigir dezenas de containers para editar uma regra é sinal de acoplamento ou plataforma insuficiente.

Termo para lembrar: Developer experience | Base técnica: referência 1.

28. Como trata versionamento do banco?

Eu coordenaria mudanças de schema com versões coexistentes da aplicação. Remover coluna antes de encerrar instâncias antigas quebra rollouts; compatibilidade durante deploy é necessária mesmo quando existe apenas um monólito.

Termo para lembrar: Schema compatibility | Base técnica: referência 1.

29. Qual é o custo de voltar atrás?

Eu avaliaria dependências externas, dados separados e mudanças de equipe antes de extrair. Código pode ser reunido, mas contratos públicos e históricos distribuídos tornam a reversão mais cara do que parece inicialmente.

Termo para lembrar: Reversibility | Base técnica: referência 1.

30. Como defenderia a escolha final?

Eu mostraria a necessidade atendida, as garantias preservadas e os custos assumidos. Definiria sinais para revisão, como contenção ou deploys bloqueados, evitando apresentar monólito ou microsserviços como solução universal.

Termo para lembrar: Architecture fitness | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Deploy único

Ganho. Simplifica release

Custo e motivo. Amplia raio de impacto

Como decidir. Quando módulos mudam juntos

02. Deploy independente

Ganho. Acelera autonomia

Custo e motivo. Exige contratos compatíveis

Como decidir. Quando times têm ritmos distintos

03. Dados locais

Ganho. Facilitam transações

Custo e motivo. Limitam autonomia física

Como decidir. Quando invariantes são conjuntas

04. Dados separados

Ganho. Isolam ownership

Custo e motivo. Criam coordenação distribuída

Como decidir. Quando contextos são independentes

05. Biblioteca comum

Ganho. Reutiliza código

Custo e motivo. Acopla versões

Como decidir. Quando semântica é estável

06. API entre módulos

Ganho. Define fronteira

Custo e motivo. Adiciona tradução

Como decidir. Quando extração futura é provável

07. Eventos

Ganho. Desacoplam temporalmente

Custo e motivo. Criam lag e replay

Como decidir. Quando reação pode esperar

08. Chamadas síncronas

Ganho. Simplificam resultado imediato

Custo e motivo. Acoplam disponibilidade

Como decidir. Quando prazo comporta dependências

09. Serviços pequenos

Ganho. Escalam seletivamente

Custo e motivo. Multiplicam operação

Como decidir. Quando responsabilidade é coesa

10. Plataforma padronizada

Ganho. Reduz custo por serviço

Custo e motivo. Exige investimento central

Como decidir. Quando há muitos serviços semelhantes

Duas alternativas ao caminho proposto

1. Monólito modular

Quando faz sentido. Quando autonomia física ainda não é necessária

Como implementar. Separar interfaces, dados e testes no mesmo deploy

Limites. Escala e release continuam compartilhados

2. Arquitetura por células

Quando faz sentido. Quando isolamento de clientes e falhas domina

Como implementar. Replicar um conjunto de serviços por grupo de tenants

Limites. Exige roteamento, capacidade e migração entre células

Referências e alcance da evidência

1. Microsoft: Architecture styles

Características e critérios de estilos arquiteturais; decisões particulares são análises autorais.

https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/

2. Microsoft: Domain analysis

Análise de domínio, bounded contexts e fronteiras; casos bancários são propostas de treino.

https://learn.microsoft.com/en-us/azure/architecture/microservices/model/domain-analysis

Continue em: T004 · T006 · T114

T006 / A. Enquadramento e arquitetura

Fronteiras, ownership e limites transacionais

Fronteiras definem autoridade sobre dados e regras; ownership inclui operação e evolução. Limites transacionais indicam quais mudanças podem ser confirmadas ou revertidas conjuntamente.

Quando usar. Ao dividir serviços, equipes, bancos e responsabilidades de integração.

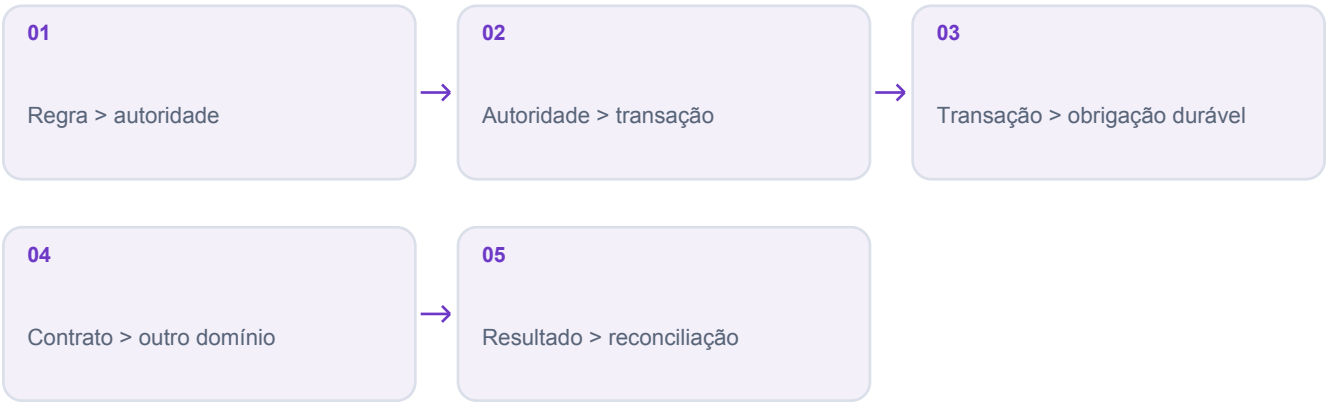
Quando evitar ou limitar. Evitar inferir atomicidade de um desenho com setas ou deixar várias equipes alterarem a mesma regra sem autoridade definida.

Uma abertura possível

Vou nomear quem decide cada mudança e onde ela é atômica. Entre fronteiras, definirei identidade, estado intermediário e recuperação, incluindo quem atende o incidente.

Peças e tecnologias. PostgreSQL · OpenAPI · AsyncAPI · ADRs · Outbox

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Listar invariantes e donos.

2. Desenhar fronteiras de dados e transações.

3. Especificar contratos e resultados desconhecidos.

4. Definir operação, suporte e evolução.

5. Testar falhas nas fronteiras e revisar acoplamento.

30 perguntas para sustentar o desenho

01. Quem deve ser dono do saldo?

Eu escolheria uma autoridade financeira explícita para validar movimentos e produzir saldo. Telas e caches podem apresentar projeções, mas não autorizam débitos com base em informação potencialmente atrasada de outra fronteira.

Termo para lembrar: System of record | Base técnica: referência 1.

02. Ownership significa manter um repositório?

Eu incluiria contrato, qualidade, plantão, capacidade e evolução, além do código. Um serviço sem responsável pelo comportamento em produção transfere falhas para equipes consumidoras e dificulta decisões durante incidentes.

Termo para lembrar: Service ownership | Base técnica: referência 1.

03. Como desenha a fronteira transacional?

Eu marcaria as alterações que compartilham commit e rollback no armazenamento escolhido. Uma função chamando dois bancos não cria uma transação única; a garantia precisa existir no mecanismo e ser demonstrável.

Termo para lembrar: Atomic boundary | Base técnica: referência 1.

04. Como trata regra que cruza dois serviços?

Eu verificaria se a separação faz sentido ou se uma reserva reduz a necessidade de atomicidade. Se for realmente distribuída, explicaria estados intermediários e compensação, sem prometer rollback instantâneo entre sistemas.

Termo para lembrar: Distributed invariant | Base técnica: referência 1.

05. Quem publica o fato de negócio?

Eu faria o domínio que confirma a decisão publicar seu evento, com obrigação persistida junto da mudança. Um consumidor não deveria inventar conclusão financeira apenas porque observou uma tentativa ou um status intermediário.

Termo para lembrar: Event authority | Base técnica: referência 1.

06. Duas equipes podem escrever na mesma tabela?

Eu evitaria escritas independentes porque ninguém controla integralmente invariantes e migrações. Se houver transição temporária, definiria protocolo, owner final e prazo de remoção, com auditoria das alterações e compatibilidade explícita.

Termo para lembrar: Shared ownership | Base técnica: referência 1.

07. Como trata dados duplicados?

Eu classificaria cópias como projeções e registraria origem, versão e frescor. Duplicação para leitura pode ser útil; o problema é permitir decisões conflitantes quando cópias passam a agir como autoridades equivalentes.

Termo para lembrar: Data sovereignty | Base técnica: referência 1.

08. Como define contrato de uma ação?

Eu especificaria autorização, precondições, identidade da intenção e estados possíveis da resposta. Também incluiria consulta posterior, porque perder a confirmação não significa que a ação externa deixou de acontecer.

Termo para lembrar: Command contract | Base técnica: referência 1.

09. Quem resolve uma falha entre domínios?

Eu atribuiria responsabilidade pela investigação e escalonamento, com correlação compartilhada. O emissor acompanha a obrigação, o destino informa seu resultado; a interface precisa mostrar pendência sem obrigar o cliente a coordenar equipes.

Termo para lembrar: Operational ownership | Base técnica: referência 1.

10. Como lida com dependência sem SLO?

Eu mediria comportamento atual e negociaria limites e suporte antes de prometer uma jornada mais forte. Sem contrato suficiente, acrescentaria degradação ou reserva de capacidade, deixando o risco residual explícito.

Termo para lembrar: Dependency agreement | Base técnica: referência 1.

11. Como evita bypass por ferramenta administrativa?

Eu faria a ferramenta chamar os mesmos comandos autorizados do domínio, registrando motivo e ator. Editar tabelas diretamente pode violar regras que a API protege e tornar a auditoria incapaz de explicar resultados.

Termo para lembrar: Invariant enforcement | Base técnica: referência 1.

12. Como define fronteira de segurança?

Eu identificaria onde identidade, dados ou privilégios mudam de confiança. A validação acompanha a entrada da fronteira; estar dentro da rede corporativa não prova autorização para alterar um recurso financeiro específico.

Termo para lembrar: Trust boundary | Base técnica: referência 1.

13. Como evita ciclo entre serviços?

Eu investigaria se o ciclo representa responsabilidades mal separadas ou coordenação legítima. Poderia mover a orquestração para um processo explícito, mantendo contratos unidirecionais e evitando chamadas recursivas que amplificam falhas.

Termo para lembrar: Dependency cycle | Base técnica: referência 1.

14. Como decide quem guarda o histórico?

Eu preservaria a trilha da decisão no domínio responsável e correlacionaria eventos entre sistemas. Uma plataforma central pode indexar cópias, mas precisa distinguir ausência de coleta de ausência de uma operação real.

Termo para lembrar: Audit authority | Base técnica: referência 1.

15. Como migra a autoridade de escrita?

Eu estabeleceria uma etapa de corte com writer único, versão de roteamento e verificação de consistência. Copiar dados não transfere autoridade automaticamente; clientes antigos e jobs precisam respeitar a nova fronteira.

Termo para lembrar: Ownership transfer | Base técnica: referência 1.

16. Como trata contratos internos informais?

Eu identificaria comportamentos realmente usados, como campos opcionais e ordenação, antes de alterar. Consumidores dependem do observado, mesmo sem documentação; testes e métricas ajudam transformar essas expectativas em contrato explícito.

Termo para lembrar: Implicit contract | Base técnica: referência 1.

17. Pode compartilhar a infraestrutura do banco?

Eu separaria propriedade lógica de isolamento físico. Schemas e permissões podem permitir economia no mesmo cluster, mas contenção e falhas continuam compartilhadas; avaliaria esse risco antes de prometer isolamento completo.

Termo para lembrar: Logical isolation | Base técnica: referência 1.

18. Como define transação longa?

Eu observaria locks, conexões e versões retidas, não apenas segundos absolutos. Chamadas de rede e aprovação humana ficam fora da transação local; persistiria estados para retomar o processo sem manter recursos bloqueados.

Termo para lembrar: Transaction duration | Base técnica: referência 1.

19. Como protege uma decisão concorrente?

Eu exigiria versão esperada ou condição atômica no comando do domínio. O consumidor pode ter lido uma cópia antiga; a autoridade revalida e devolve conflito em vez de sobrescrever silenciosamente outra decisão.

Termo para lembrar: Compare-and-swap | Base técnica: referência 1.

20. Como trata alteração de política?

Eu nomearia quem aprova e publica versões da regra, incluindo impacto em decisões em andamento. Configuração também altera comportamento; precisa de revisão, rollout e possibilidade de explicar qual versão foi aplicada.

Termo para lembrar: Policy ownership | Base técnica: referência 1.

21. Como distingue evento e solicitação entre equipes?

Eu usaria comando para pedir algo que pode ser rejeitado e evento para relatar fato confirmado. O nome e contrato deixam claro quem valida, quem responde e qual obrigação nasce da mensagem.

Termo para lembrar: Command versus event | Base técnica: referência 1.

22. Como organiza reconciliação?

Eu compararia identidades e estados relevantes nas duas autoridades, separando atraso de divergência definitiva. Definiria janela, responsável e correção permitida; um job genérico não pode criar ajustes financeiros sem política validada.

Termo para lembrar: Reconciliation boundary | Base técnica: referência 1.

23. Como evita duplicar regra no BFF?

Eu permitiria validação de experiência no BFF, mas manteria a regra autoritativa no domínio. Caso contrário, jobs e outros canais podem aplicar critérios diferentes ou contornar uma restrição financeira importante.

Termo para lembrar: Business rule placement | Base técnica: referência 1.

24. Como lida com dependência descontinuada?

Eu criaria inventário de consumidores, prazo de migração e indicadores de uso antes do desligamento. Ausência de chamadas recentes não garante inexistência de tarefas mensais ou processos de recuperação ainda dependentes.

Termo para lembrar: Contract lifecycle | Base técnica: referência 1.

25. Como define responsabilidade por dados pessoais?

Eu registraria finalidades, cópias, acessos e políticas aplicáveis por domínio. Exclusão e retenção precisam atravessar projeções e backups conforme regra validada; um delete numa tabela não resolve sozinho todo o ciclo.

Termo para lembrar: Data stewardship | Base técnica: referência 1.

26. Como decide por uma API compartilhada?

Eu ofereceria capacidades estáveis com ownership e suporte claros, evitando uma API genérica que dá acesso irrestrito às tabelas. A plataforma reduz repetição, mas não deve absorver arbitrariamente todas as regras dos produtos.

Termo para lembrar: Platform contract | Base técnica: referência 1.

27. Como avalia custo organizacional da fronteira?

Eu mediria quantas equipes precisam participar de uma mudança comum e quanto tempo ela espera por decisões. Uma separação tecnicamente elegante pode ser ruim se transforma cada entrega em coordenação contínua.

Termo para lembrar: Coordination cost | Base técnica: referência 1.

28. Como valida contratos de falha?

Eu testaria timeout, resposta duplicada, indisponibilidade e recuperação com o provedor. Documentaria o que permanece desconhecido; códigos HTTP não substituem uma definição compartilhada de confirmação durável e resultado de negócio.

Termo para lembrar: Failure contract | Base técnica: referência 1.

29. Como preserva autonomia sem duplicar tudo?

Eu padronizaria infraestrutura, segurança e observabilidade, deixando decisões de domínio com seus owners. Compartilhar ferramentas reduz carga cognitiva; centralizar toda decisão funcional cria gargalo e enfraquece responsabilidade local.

Termo para lembrar: Governed autonomy | Base técnica: referência 1.

30. Como apresenta isso no desenho?

Eu colocaria caixas de domínio, bancos autoritativos e transações locais, anotando contratos nas setas. Para cada travessia, explicaria prazo, repetição e owner; isso torna os limites verificáveis durante perguntas de falha.

Termo para lembrar: Responsibility mapping | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Fronteira ampla

Ganho. Simplifica atomicidade

Custo e motivo. Aumenta coordenação

Como decidir. Quando regras mudam juntas

02. Fronteira estreita

Ganho. Amplia autonomia

Custo e motivo. Cria integrações

Como decidir. Quando invariantes são locais

03. Infraestrutura compartilhada

Ganho. Reduz custo

Custo e motivo. Compartilha falhas

Como decidir. Quando isolamento físico não é exigido

04. Dados exclusivos

Ganho. Clarifica autoridade

Custo e motivo. Duplica projeções

Como decidir. Quando evolução independente importa

05. Orquestração central

Ganho. Explicita processo

Custo e motivo. Concentra responsabilidade

Como decidir. Quando fluxo cruza vários domínios

06. Coreografia

Ganho. Reduz coordenação central

Custo e motivo. Dificulta visão global

Como decidir. Quando reações são independentes

07. Contrato rígido

Ganho. Protege consumidores

Custo e motivo. Retarda mudança

Como decidir. Quando compatibilidade é crítica

08. Política configurável

Ganho. Acelera adaptação

Custo e motivo. Exige governança

Como decidir. Quando regra muda frequentemente

09. Owner único

Ganho. Facilita decisão

Custo e motivo. Pode virar gargalo

Como decidir. Quando responsabilidade precisa ser inequívoca

10. Projeção local

Ganho. Reduz dependência de leitura

Custo e motivo. Introduz atraso

Como decidir. Quando frescor tolera eventualidade

Duas alternativas ao caminho proposto

1. Agregado compartilhado local

Quando faz sentido. Quando uma invariante exige atomicidade frequente

Como implementar. Manter a regra e os dados no mesmo limite transacional

Limites. Reduz autonomia física dos participantes

2. Process manager

Quando faz sentido. Quando há coordenação longa entre domínios

Como implementar. Persistir etapas, resultados e compensações sob um owner

Limites. Não cria isolamento nem rollback global automático

Referências e alcance da evidência

1. Microsoft: Domain analysis

Análise de domínio, bounded contexts e fronteiras; casos bancários são propostas de treino.

https://learn.microsoft.com/en-us/azure/architecture/microservices/model/domain-analysis

2. PostgreSQL: Transaction isolation

Isolamento, snapshots e necessidade de repetição de transações sob conflitos.

https://www.postgresql.org/docs/current/transaction-iso.html

3. Microsoft: Architecture styles

Características e critérios de estilos arquiteturais; decisões particulares são análises autorais.

https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/

Continue em: T004 · T047 · T114

T007 / A. Enquadramento e arquitetura

Arquitetura hexagonal e ports and adapters

Arquitetura hexagonal separa regras da aplicação dos mecanismos externos. Portas expressam capacidades necessárias; adaptadores conectam HTTP, banco, broker e testes sem inverter a dependência do domínio.

Quando usar. Quando regras precisam sobreviver a mudanças de transporte, persistência ou fornecedores e quando testes isolados ajudam o desenvolvimento.

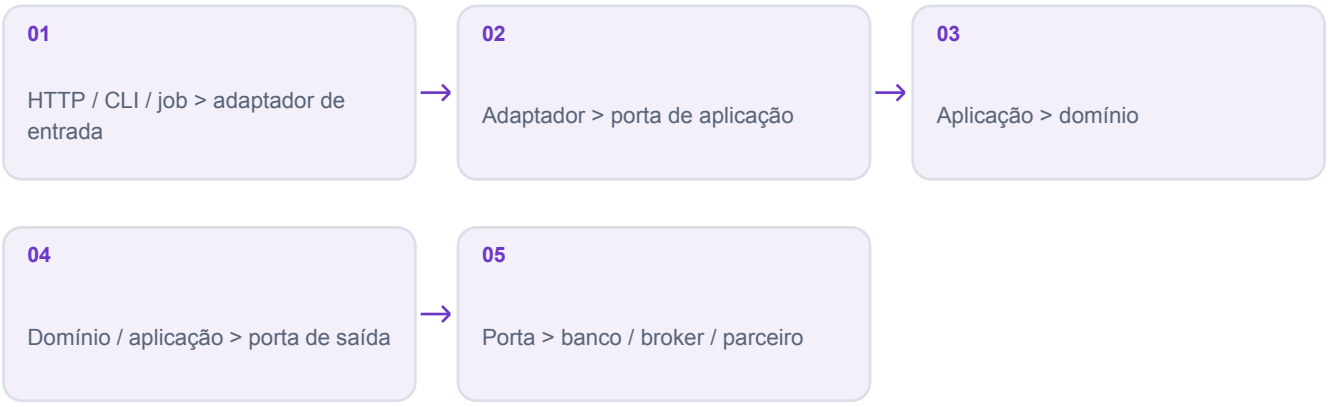
Quando evitar ou limitar. Evitar interfaces artificiais para cada classe ou abstrações que escondem garantias relevantes de transação e consistência.

Uma abertura possível

Vou manter o caso de uso independente do protocolo, mas deixar explícitas as garantias exigidas da persistência. Trocar um adaptador não significa que qualquer banco oferece o mesmo comportamento.

Peças e tecnologias. Kotlin · Go interfaces · Clojure protocols · JUnit · Testcontainers

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Definir casos de uso e invariantes.

2. Criar portas com semântica de negócio.

3. Implementar adaptadores de entrada e saída.

4. Testar domínio isolado e contratos com infraestrutura real.

5. Verificar dependências e custos de abstração.

30 perguntas para sustentar o desenho

01. O que é uma porta?

Eu a definiria como contrato de uma capacidade usada ou oferecida pela aplicação, como reservar limite. Não precisa corresponder a uma classe; seu significado deve expressar comportamento e garantias esperadas.

Termo para lembrar: Port | Base técnica: referência 1.

02. O que é um adaptador?

Eu chamaria de adaptador a implementação que traduz um mecanismo externo para uma porta, como HTTP ou PostgreSQL. Ele cuida de formatos e erros técnicos sem assumir indevidamente regras do domínio.

Termo para lembrar: Adapter | Base técnica: referência 1.

03. O hexágono exige seis componentes?

Eu explicaria que a forma representa separação entre interior e exterior, não quantidade de lados funcionais. O número de portas depende das interações necessárias, evitando criar camadas apenas para reproduzir um desenho.

Termo para lembrar: Architectural metaphor | Base técnica: referência 1.

04. Onde fica a validação?

Eu separaria formato de entrada de regra de negócio. O adaptador rejeita JSON inválido; o domínio impede transição proibida, porque jobs, testes e outros canais também precisam respeitar a mesma invariante.

Termo para lembrar: Validation boundary | Base técnica: referência 1.

05. Onde começa a transação?

Eu coordenaria a transação no caso de uso ou mecanismo equivalente, cobrindo as mudanças que precisam ser atômicas. A porta deve deixar essa garantia explícita; vários repositórios não podem confirmar independentemente por acidente.

Termo para lembrar: Unit of work | Base técnica: referência 1.

06. Como modela um repositório?

Eu ofereceria operações alinhadas ao agregado e suas condições, como salvar com versão esperada. Um CRUD universal tende a esconder concorrência e permitir estados inválidos; o contrato precisa expressar sucesso e conflito.

Termo para lembrar: Repository contract | Base técnica: referência 1.

07. Pode trocar SQL por NoSQL sem mudança?

Eu só afirmaria isso se ambos cumprirem o contrato de atomicidade, consultas e consistência. A interface pode permanecer igual enquanto a garantia desaparece; testes e documentação precisam impedir substituições semanticamente incompatíveis.

Termo para lembrar: Semantic substitutability | Base técnica: referência 1.

08. Como testa o domínio?

Eu exercitaria regras com valores e estados explícitos, sem rede nem banco. Depois complementaria com testes de integração, porque um modelo correto em memória não comprova índices únicos, isolamento ou persistência durável.

Termo para lembrar: Pure core | Base técnica: referência 1.

09. Mock é suficiente para testar adaptador?

Eu usaria mocks para explorar respostas do caso de uso, mas testaria o adaptador contra a tecnologia real. Um fake permissivo pode esconder diferenças de null, ordenação, concorrência e falhas do banco.

Termo para lembrar: Contract testing | Base técnica: referência 1.

10. Como evita interface para tudo?

Eu criaria portas em fronteiras relevantes de mudança ou efeito externo. Abstrair funções triviais aumenta navegação e manutenção sem benefício; a decisão deve reduzir acoplamento real ou tornar uma regra importante testável.

Termo para lembrar: Abstraction cost | Base técnica: referência 1.

11. Como trata erros do fornecedor?

Eu traduziria erros técnicos para categorias úteis, preservando diagnóstico e ambiguidade. Um timeout pode significar resultado desconhecido; não o converteria automaticamente em rejeição de negócio só para simplificar a interface.

Termo para lembrar: Error translation | Base técnica: referência 1.

12. Como representa tempo?

Eu injetaria o relógio ou passaria o instante ao caso de uso, permitindo testes de vencimento determinísticos. O contrato diferencia duração monotônica de horário civil quando expiração e calendário exigem comportamentos distintos.

Termo para lembrar: Clock port | Base técnica: referência 1.

13. Como representa geração de IDs?

Eu geraria identidades na fronteira apropriada e preservaria a intenção em retries. Testes podem controlar o gerador, mas uma nova chamada não deve criar outro identificador quando estamos repetindo a mesma operação.

Termo para lembrar: Identity port | Base técnica: referência 1.

14. Onde fica autorização?

Eu validaria identidade no adaptador e permissão sobre a ação no caso de uso ou política de domínio. Assim, uma entrada alternativa não consegue ignorar regras porque deixou de passar pelo controlador HTTP.

Termo para lembrar: Authorization policy | Base técnica: referência 1.

15. Como expõe várias interfaces?

Eu conectaria HTTP, CLI e consumidor de fila ao mesmo caso de uso, com tradução de contexto. Cada canal mantém autenticação e contrato próprios, mas compartilha as regras que precisam permanecer consistentes.

Termo para lembrar: Driving adapters | Base técnica: referência 1.

16. Onde fica retry?

Eu colocaria repetição na camada que entende se a operação é segura e qual prazo resta. Adaptadores podem repetir transporte quando permitido, mas retries ocultos podem duplicar efeitos ou ultrapassar o deadline global.

Termo para lembrar: Retry placement | Base técnica: referência 1.

17. Como preserva desempenho?

Eu mediria serialização, alocações e chamadas reais antes de culpar interfaces. Ainda evitaria mapeamentos repetidos de objetos grandes e contratos excessivamente finos que multiplicam consultas ao armazenamento ou chamadas de rede.

Termo para lembrar: Abstraction overhead | Base técnica: referência 1.

18. Como trata consultas complexas?

Eu criaria portas de leitura específicas para a necessidade, sem forçar toda consulta a reidratar agregados. Isso permite otimizar SQL mantendo autorização e sem conceder à projeção autoridade para alterar o domínio.

Termo para lembrar: Query port | Base técnica: referência 1.

19. Onde coloca outbox?

Eu exigiria que persistir a mudança e a obrigação de publicação compartilhassem uma transação. O adaptador implementa o mecanismo, mas o caso de uso precisa conhecer a garantia e os estados resultantes.

Termo para lembrar: Transactional messaging | Base técnica: referência 1.

20. Como organiza dependências no código?

Eu faria o núcleo depender de contratos próprios, com composição dos adaptadores na borda. Bibliotecas de banco e HTTP não deveriam dominar tipos do domínio, dificultando testes e mudanças de infraestrutura.

Termo para lembrar: Dependency inversion | Base técnica: referência 1.

21. Arquitetura hexagonal equivale a microsserviços?

Eu explicaria que são dimensões diferentes: um processo pode ter arquitetura hexagonal e vários serviços podem não tê-la. A separação interna ajuda manutenção, mas não determina granularidade, rede ou estratégia de implantação.

Termo para lembrar: Deployment independence | Base técnica: referência 1.

22. Como migra código legado?

Eu encontraria um caso de uso, criaria uma interface de fronteira e moveria regras gradualmente. Testes de caracterização protegem comportamento existente; não reescreveria tudo antes de entregar o primeiro benefício verificável.

Termo para lembrar: Seam extraction | Base técnica: referência 1.

23. Como trata eventos de entrada?

Eu validaria schema e contexto, aplicaria identidade de consumo e chamaria o caso de uso. Confirmação da mensagem deve ocorrer após persistência relevante; o adaptador não pode ocultar uma janela de perda.

Termo para lembrar: Message adapter | Base técnica: referência 1.

24. Como faz observabilidade sem poluir domínio?

Eu instrumentaria adaptadores e casos de uso com correlação, reservando eventos de negócio para significados reais. Logs técnicos não precisam invadir cada entidade, mas o fluxo deve manter rastreabilidade de decisões importantes.

Termo para lembrar: Instrumentation boundary | Base técnica: referência 1.

25. Como evita vazamento de detalhes do banco?

Eu não retornaria objetos de sessão ou queries arbitrárias para o domínio. Porém, não esconderia capacidades essenciais: versão esperada, atomicidade e paginação precisam aparecer quando afetam o comportamento prometido pela porta.

Termo para lembrar: Leaky abstraction | Base técnica: referência 1.

26. Como testa falha após commit?

Eu simularia perda de resposta depois da persistência e verificaria recuperação pela identidade da operação. Esse teste atravessa camadas, mostrando se o contrato distingue rejeição, confirmação e resultado desconhecido corretamente.

Termo para lembrar: Fault injection | Base técnica: referência 1.

27. Como controla cancelamento?

Eu propagaria contexto e deadline aos adaptadores, sem assumir que cancelar uma espera desfaz um efeito remoto. O caso de uso registra a situação e permite consultar resultado quando a operação pode ter concluído.

Termo para lembrar: Cancellation propagation | Base técnica: referência 1.

28. Quando não adotaria a estrutura completa?

Eu simplificaria scripts pequenos e CRUD temporário com poucas regras, mantendo organização suficiente. Criar dezenas de portas para um protótipo descartável pode custar mais que o risco de acoplamento que pretende reduzir.

Termo para lembrar: Pragmatic architecture | Base técnica: referência 1.

29. Como avalia qualidade da arquitetura?

Eu observaria se uma alteração de fornecedor ou regra fica localizada e se os testes detectam quebra de contrato. Contar interfaces não mede qualidade; dependências claras e garantias preservadas são evidências melhores.

Termo para lembrar: Change resilience | Base técnica: referência 1.

30. Como explicaria no desenho?

Eu desenharia o caso de uso no centro, entradas de um lado e efeitos do outro. Em cada porta anotaria a garantia exigida, evitando que a simplicidade visual esconda transações, deadlines ou autorização.

Termo para lembrar: Ports and adapters | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Portas específicas

Ganho. Expressam garantias

Custo e motivo. Criam mais contratos

Como decidir. Quando domínio tem regras fortes

02. Repository genérico

Ganho. Reduz código inicial

Custo e motivo. Esconde semântica

Como decidir. Quando CRUD é simples

03. Núcleo puro

Ganho. Facilita testes

Custo e motivo. Exige separação de efeitos

Como decidir. Quando regras são complexas

04. Mappers explícitos

Ganho. Isolam formatos

Custo e motivo. Aumentam manutenção

Como decidir. Quando contratos externos mudam

05. Fakes

Ganho. Aceleram testes

Custo e motivo. Podem divergir da realidade

Como decidir. Quando complementados por integração

06. Teste real de adapter

Ganho. Valida semântica

Custo e motivo. Custa infraestrutura

Como decidir. Quando persistência afeta correção

07. Query ports

Ganho. Permitem otimização

Custo e motivo. Duplicam modelos

Como decidir. Quando consultas diferem de escrita

08. Clock injetado

Ganho. Torna tempo testável

Custo e motivo. Adiciona parâmetro

Como decidir. Quando vencimento importa

09. Interface estável

Ganho. Reduz impacto de troca

Custo e motivo. Pode limitar capacidades

Como decidir. Quando fornecedores oferecem garantias equivalentes

10. Estrutura mínima

Ganho. Acelera protótipo

Custo e motivo. Aumenta refatoração futura

Como decidir. Quando risco e duração são baixos

Duas alternativas ao caminho proposto

1. Arquitetura em camadas

Quando faz sentido. Quando responsabilidades são simples e bem conhecidas

Como implementar. Separar apresentação, aplicação e dados com dependências controladas

Limites. Pode permitir domínio dependente de infraestrutura se mal aplicada

2. Functional core, imperative shell

Quando faz sentido. Quando transformação pura concentra regras

Como implementar. Manter cálculos puros e executar efeitos numa casca explícita

Limites. Não elimina necessidade de contratos e transações nos efeitos

Referências e alcance da evidência

1. Alistair Cockburn: Hexagonal Architecture

Artigo original sobre isolamento da aplicação por portas e adaptadores; exemplos e decisões são autorais.

https://alistair.cockburn.us/hexagonal-architecture

2. Microsoft: Architecture styles

Características e critérios de estilos arquiteturais; decisões particulares são análises autorais.

https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/

Continue em: T004 · T006 · T076

T008 / A. Enquadramento e arquitetura

Contratos de API: REST, gRPC e GraphQL

Contratos de API definem operações, tipos, erros, compatibilidade e garantias. REST usa semântica HTTP, gRPC define RPCs tipadas e GraphQL oferece consultas sobre um schema.

Quando usar. Ao expor capacidades a aplicativos, equipes ou parceiros e definir como clientes evoluem independentemente.

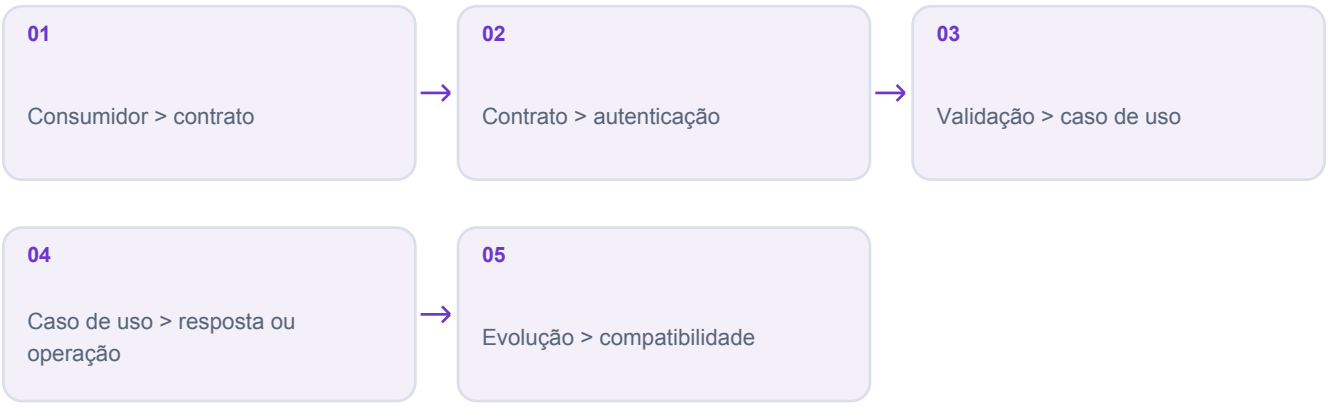
Quando evitar ou limitar. Evitar escolher protocolo como substituto de autorização, idempotência, controle de custo ou política de compatibilidade.

Uma abertura possível

Vou escolher o protocolo pelo consumidor e pelo padrão de interação, documentando erros e evolução. O contrato inclui o que acontece quando a resposta se perde, não apenas campos felizes.

Peças e tecnologias. OpenAPI · HTTP · gRPC · Protocol Buffers · GraphQL · Schema Registry

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Definir recursos, comandos e consumidores.

2. Escolher protocolo e semântica de erros.

3. Modelar identidade, paginação e concorrência.

4. Especificar compatibilidade e limites de custo.

5. Validar contratos e observar versões em produção.

30 perguntas para sustentar o desenho

01. Quando escolhe REST?

Eu escolheria HTTP com recursos e contratos claros para clientes variados e integração ampla. Usaria métodos e status conforme semântica, sem assumir que um verbo torna a implementação automaticamente segura ou idempotente.

Termo para lembrar: Resource-oriented API | Base técnica: referência 1.

02. Quando escolhe gRPC?

Eu usaria contratos tipados e geração de clientes quando comunicação interna e streaming justificarem. Consideraria suporte de proxies e clientes; desempenho depende de payload e implementação, não apenas do nome do protocolo.

Termo para lembrar: Typed RPC | Base técnica: referência 1.

03. Quando escolhe GraphQL?

Eu avaliaria quando clientes precisam compor dados variados com seleção de campos. Controlaria custo, profundidade e autorização por recurso; permitir consulta flexível não autoriza trabalho ilimitado nem acesso irrestrito ao grafo.

Termo para lembrar: Schema-based query | Base técnica: referência 1.

04. Como modela criação assíncrona?

Eu retornaria aceitação com um identificador consultável e estados de processamento. O cliente acompanha até conclusão; não usaria resposta de sucesso inicial para afirmar efeito financeiro que depende de trabalho posterior.

Termo para lembrar: Operation resource | Base técnica: referência 1.

05. PUT e POST são intercambiáveis?

Eu escolheria PUT para substituição no recurso conhecido conforme contrato e POST para processamento ou criação adequada. A semântica orienta clientes, mas a persistência ainda precisa garantir concorrência e comportamento de repetição.

Termo para lembrar: HTTP semantics | Base técnica: referência 1.

06. Como trata retries de POST?

Eu aceitaria uma chave estável por intenção, verificando payload e autorização. A constraint evita duas criações concorrentes; se o resultado já existe, devolvo a operação anterior em vez de executar novamente.

Termo para lembrar: Idempotency key | Base técnica: referência 1.

07. Como representa conflito concorrente?

Eu aceitaria versão esperada ou ETag com condição apropriada, rejeitando atualização baseada em estado antigo. O cliente pode reler e decidir; sobrescrever silenciosamente perde decisões feitas por outros participantes.

Termo para lembrar: Optimistic concurrency | Base técnica: referência 1.

08. Como define erros úteis?

Eu separaria código estável de máquina, mensagem humana e correlação, evitando expor detalhes sensíveis. O contrato indica retry ou ação necessária; todo erro genérico obriga consumidores a adivinhar o comportamento correto.

Termo para lembrar: Problem details | Base técnica: referência 1.

09. Como pagina listas grandes?

Eu usaria cursor opaco com ordenação determinística e desempate, limitando quantidade e bytes. O contrato explica mudanças durante navegação; um cursor não cria snapshot consistente automaticamente quando novos registros são inseridos.

Termo para lembrar: Cursor pagination | Base técnica: referência 1.

10. Como evolui sem quebrar mobile antigo?

Eu adicionaria campos opcionais e preservaria significados existentes, observando versões ativas antes de remover suporte. Um usuário pode não atualizar o aplicativo; deploy do servidor não controla o ciclo de todos os clientes.

Termo para lembrar: Backward compatibility | Base técnica: referência 1.

11. Campo novo sempre é compatível?

Eu verificaria clientes estritos, enums e validações, além do formato. Adicionar um estado desconhecido pode quebrar lógica mesmo com JSON válido; compatibilidade precisa considerar comportamento e testes de consumidores representativos.

Termo para lembrar: Compatibility semantics | Base técnica: referência 1.

12. Como altera um campo obrigatório?

Eu introduziria o novo formato, manteria leitura compatível e migraria consumidores antes de remover o antigo. Se significado mudar, preferiria contrato explícito novo em vez de reutilizar silenciosamente o mesmo nome.

Termo para lembrar: Expand-contract | Base técnica: referência 1.

13. Como evita N+1 no GraphQL?

Eu agruparia e reutilizaria leituras dentro do escopo seguro da requisição. O cache precisa respeitar autorização; batching reduz consultas, mas não elimina consultas caras ou fan-out excessivo provocado pelo formato solicitado.

Termo para lembrar: DataLoader | Base técnica: referência 1.

14. Como limita custo de GraphQL?

Eu definiria limites de profundidade, cardinalidade e custo estimado, com paginação obrigatória em listas. Para clientes controlados, operações persistidas ajudam; o servidor ainda precisa medir custo real e proteger dependências.

Termo para lembrar: Query complexity | Base técnica: referência 1.

15. Onde faz autorização?

Eu verificaria acesso ao recurso no backend responsável, não apenas ao endpoint. Um usuário autenticado pode trocar identificadores; a mesma regra deve valer em REST, resolvers GraphQL e handlers gRPC.

Termo para lembrar: Object-level authorization | Base técnica: referência 1.

16. Como usa deadlines em gRPC?

Eu definiria prazo total e propagaria o tempo restante às dependências. Cancelamento reduz trabalho inútil, mas não desfaz efeito já confirmado; o contrato permite consultar o resultado quando a resposta é perdida.

Termo para lembrar: Deadline propagation | Base técnica: referência 1.

17. Como trata streaming?

Eu limitaria buffers, respeitaria controle de fluxo e definiria retomada quando a conexão cair. Streaming transporta dados continuamente, mas não fornece sozinho persistência, replay ou confirmação de aplicação de cada mensagem.

Termo para lembrar: Flow control | Base técnica: referência 1.

18. Como evita logs com dados pessoais?

Eu registraria metadados operacionais e correlação, mascarando campos sensíveis. Debug não justifica armazenar documentos ou credenciais indiscriminadamente; contratos e ferramentas precisam classificar dados antes de observá-los em produção.

Termo para lembrar: Data minimization | Base técnica: referência 1.

19. Como testa compatibilidade?

Eu executaria exemplos representativos de consumidores contra mudanças do provedor, além de validação estrutural. Testes não substituem inventário de versões, mas detectam alterações de campos, erros e comportamento antes do rollout.

Termo para lembrar: Consumer-driven contract | Base técnica: referência 1.

20. API deve espelhar tabelas?

Eu exporia capacidades e recursos úteis ao consumidor, mantendo detalhes físicos privados. Renomear coluna não deveria quebrar clientes; um contrato ligado diretamente ao schema torna evolução e autorização mais difíceis.

Termo para lembrar: Domain-oriented contract | Base técnica: referência 1.

21. Como trata payload grande?

Eu imporia limites de bytes, campos e anexos, oferecendo upload separado quando necessário. Compressão ajuda rede, mas pode consumir CPU; o servidor deve recusar trabalho excessivo antes de alocar memória sem controle.

Termo para lembrar: Payload budget | Base técnica: referência 1.

22. Como representa dinheiro na API?

Eu enviaria moeda e quantia com representação exata, como unidades mínimas ou decimal textual com escala definida. Ponto flutuante binário pode introduzir arredondamento inesperado; o contrato determina validação e conversão permitida.

Termo para lembrar: Money encoding | Base técnica: referência 1.

23. Como representa horários?

Eu usaria instante com timezone inequívoco para eventos e campos separados para regras locais quando necessário. Um timestamp não representa sozinho calendário comercial, cutoff ou duração monotônica; essas semânticas precisam ser documentadas.

Termo para lembrar: Temporal contract | Base técnica: referência 1.

24. Quando versiona a URL?

Eu usaria versão explícita para quebra relevante quando evolução compatível não bastar. Manter versões tem custo operacional; estabeleceria política de suporte, métricas de adoção e migração antes de anunciar descontinuação.

Termo para lembrar: API versioning | Base técnica: referência 1.

25. Como trata rate limiting?

Eu documentaria escopo da quota e sinais de espera, distinguindo limite por cliente de indisponibilidade geral. O cliente pode reduzir ritmo com jitter; retries imediatos aumentam sobrecarga e pioram a experiência coletiva.

Termo para lembrar: Rate limit contract | Base técnica: referência 1.

26. Como evita cache indevido?

Eu definiria diretivas conforme sensibilidade e variação da resposta, incluindo autenticação e tenant. Cache compartilhado de dados personalizados exige configuração cuidadosa; usar GET não torna a informação automaticamente pública ou reutilizável.

Termo para lembrar: Cache control | Base técnica: referência 1.

27. Como trata webhooks como contrato?

Eu definiria assinatura, identidade de evento, repetição e prazo de confirmação. O receptor persiste antes de responder; eventos podem chegar duplicados ou fora de ordem, exigindo política de aplicação compatível com o domínio.

Termo para lembrar: Webhook delivery | Base técnica: referência 1.

28. Como decide entre endpoint agregado e granular?

Eu agruparia dados necessários à tarefa quando isso reduz latência sem ocultar falhas parciais. Endpoints genéricos demais transferem composição ao cliente; agregações enormes aumentam custo e acoplamento entre produtos.

Termo para lembrar: API composition | Base técnica: referência 1.

29. OpenAPI garante implementação correta?

Eu usaria a especificação para validação, documentação e geração de clientes, mas testaria semântica real. Ela não comprova autorização, idempotência ou durabilidade; essas garantias exigem código, armazenamento e testes de falha.

Termo para lembrar: Executable contract | Base técnica: referência 1.

30. Como defenderia a escolha na entrevista?

Eu compararia perfil do consumidor, necessidade de streaming, flexibilidade e operação. Escolheria um caminho inicial e declararia limites; protocolos diferentes podem coexistir quando há fronteiras claras e benefício concreto para cada público.

Termo para lembrar: Protocol trade-off | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. REST

Ganho. Ampla interoperabilidade

Custo e motivo. Pode exigir mais chamadas

Como decidir. Quando clientes são variados

02. gRPC

Ganho. Tipagem e streaming

Custo e motivo. Exige suporte de infraestrutura

Como decidir. Quando RPC interno predomina

03. GraphQL

Ganho. Consulta flexível

Custo e motivo. Exige controle de custo

Como decidir. Quando telas variam muito

04. Versão nova

Ganho. Isola quebra

Custo e motivo. Multiplica manutenção

Como decidir. Quando semântica muda

05. Evolução aditiva

Ganho. Reduz migração

Custo e motivo. Acumula campos legados

Como decidir. Quando compatibilidade é viável

06. Cursor

Ganho. Evita offset profundo

Custo e motivo. Dificulta saltos arbitrários

Como decidir. Quando listas são grandes

07. Agregação

Ganho. Reduz round trips

Custo e motivo. Amplia dependências

Como decidir. Quando jornada usa vários recursos

08. Contrato estrito

Ganho. Detecta erro cedo

Custo e motivo. Pode limitar evolução

Como decidir. Quando consumidores são coordenados

09. Streaming

Ganho. Reduz overhead contínuo

Custo e motivo. Complica retomada

Como decidir. Quando há fluxo prolongado

10. Operações persistidas

Ganho. Controlam consultas

Custo e motivo. Reduzem flexibilidade

Como decidir. Quando clientes são conhecidos

Duas alternativas ao caminho proposto

1. REST com BFF

Quando faz sentido. Quando vários frontends precisam de respostas distintas

Como implementar. Compor endpoints adequados a cada jornada

Limites. Adiciona camada e precisa preservar autoridade dos domínios

2. Eventos assíncronos

Quando faz sentido. Quando consumidor não exige resultado imediato

Como implementar. Publicar fatos ou aceitar comandos duráveis com acompanhamento

Limites. Exige idempotência, lag explícito e contratos de evolução

Referências e alcance da evidência

1. IETF: HTTP Semantics, RFC 9110

Semântica HTTP, métodos, respostas e condicionais, não garantias financeiras fim a fim.

https://www.rfc-editor.org/rfc/rfc9110

2. gRPC: Core concepts

Serviços, mensagens, tipos de RPC e ciclo de chamadas.

https://grpc.io/docs/what-is-grpc/core-concepts/

3. GraphQL Specification

Schema, tipos, execução, queries e mutations; recomendações operacionais são análise autoral.

https://spec.graphql.org/October2021/

Continue em: T053 · T061 · T077

T009 / A. Enquadramento e arquitetura

Comunicação síncrona, assíncrona e fan-out

Comunicação síncrona aguarda resposta no fluxo corrente; assíncrona registra trabalho para execução desacoplada. Fan-out envia trabalho a múltiplos destinos e exige controle da amplificação e das falhas parciais.

Quando usar. Ao combinar serviços, compor telas, disparar processamento e definir como o usuário acompanha resultados.

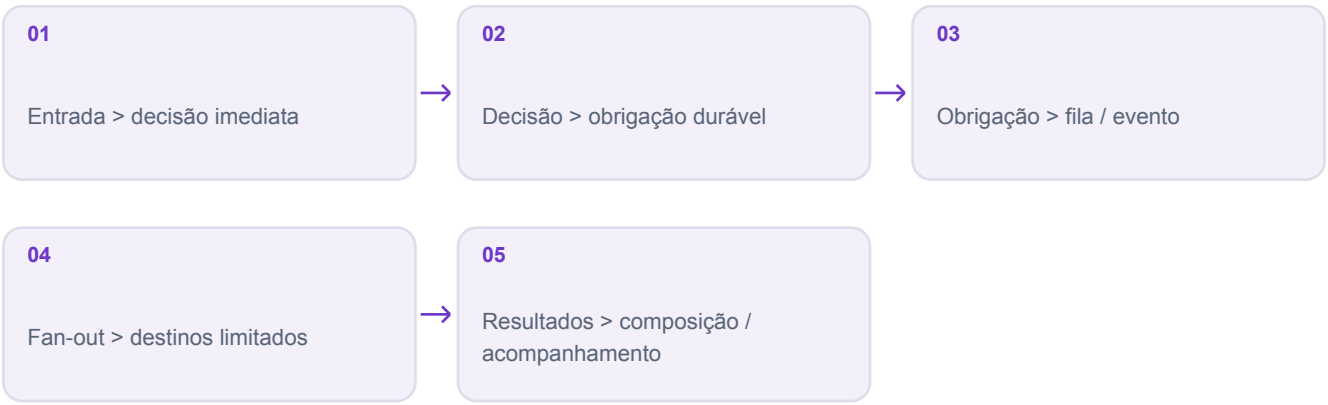
Quando evitar ou limitar. Evitar filas quando a decisão precisa ser imediata ou cadeias síncronas longas quando atraso e indisponibilidade se propagam.

Uma abertura possível

Vou escolher onde realmente preciso de resposta imediata. Nas demais etapas, persistirei a obrigação e mostrarei o progresso, limitando paralelismo e trabalho gerado por cada solicitação.

Peças e tecnologias. HTTP · gRPC · Kafka · SQS · RabbitMQ · SSE

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Separar resposta necessária de reação posterior.

2. Definir prazo total e falhas parciais.

3. Limitar fan-out e concorrência por dependência.

4. Persistir trabalho assíncrono e identidade.

5. Medir conclusão, backlog e recuperação.

30 perguntas para sustentar o desenho

01. Quando faria chamada síncrona?

Eu aguardaria quando a decisão depende imediatamente do resultado e a dependência cabe no prazo. Limitaria timeout e concorrência, explicando o que o usuário recebe se o serviço remoto não responder.

Termo para lembrar: Synchronous interaction | Base técnica: referência 1.

02. Quando escolheria assíncrono?

Eu usaria quando o trabalho pode continuar depois da aceitação e precisa sobreviver a falhas. Persistiria a obrigação, retornaria identificador e definiria prazo de conclusão; assíncrono não significa ausência de compromisso.

Termo para lembrar: Asynchronous processing | Base técnica: referência 1.

03. Assíncrono sempre é mais rápido?

Eu distinguiria resposta inicial de conclusão. Uma fila pode liberar a API rapidamente e melhorar throughput, mas acrescenta espera e etapas; a jornada só fica melhor se o usuário tolerar esse atraso.

Termo para lembrar: Latency versus throughput | Base técnica: referência 1.

04. O que é fan-out?

Eu definiria como multiplicação de uma solicitação em chamadas ou tarefas para vários destinos. Mediria amplificação por jornada e limitaria concorrência, porque uma API aparentemente leve pode sobrecarregar muitas dependências simultaneamente.

Termo para lembrar: Fan-out | Base técnica: referência 1.

05. Como calcula prazo de chamadas paralelas?

Eu observaria a chamada necessária mais lenta, acrescida da composição e rede. Paralelismo evita somar todos os tempos, mas amplia exposição à cauda e não remove limites globais de capacidade.

Termo para lembrar: Critical path | Base técnica: referência 1.

06. Como evita cascata de timeouts?

Eu propagaria o prazo restante e reservaria tempo para resposta e limpeza. Dependências não recebem cada uma o timeout inteiro da jornada; isso mantém o trabalho dentro do orçamento acordado com o cliente.

Termo para lembrar: Deadline budget | Base técnica: referência 1.

07. Como trata resultado parcial?

Eu classificaria partes essenciais e opcionais, devolvendo status e frescor por seção. Não substituiria falha por valores inventados; o consumidor precisa distinguir informação ausente, desatualizada e legitimamente vazia durante a degradação.

Termo para lembrar: Partial response | Base técnica: referência 1.

08. Como limita concorrência?

Eu colocaria limites por destino e classe de trabalho, além do limite por requisição. Quatro chamadas por tela ainda podem gerar milhares globalmente; protegeria a capacidade real da dependência compartilhada.

Termo para lembrar: Concurrency limiter | Base técnica: referência 1.

09. Como evita N+1?

Eu reuniria identificadores e buscaria dados em lote quando a autorização permitir. Isso reduz viagens de rede, mas imponho tamanho máximo e trato resultados individuais, evitando transformar o lote numa operação ilimitada.

Termo para lembrar: Batching | Base técnica: referência 1.

10. Como decide entre evento e comando?

Eu usaria comando para solicitar responsabilidade específica e evento para divulgar fato confirmado. O contrato deve indicar rejeição e resposta esperada; chamar toda mensagem de evento esconde quem deve agir e garantir conclusão.

Termo para lembrar: Command versus event | Base técnica: referência 1.

11. Fila em memória é suficiente?

Eu só aceitaria perda quando o produto permitir, como sinal de digitação. Para uma solicitação bancária confirmada, persistiria trabalho antes do ACK; memória do processo desaparece durante reinício ou falha da máquina.

Termo para lembrar: Durable acceptance | Base técnica: referência 1.

12. Como oferece consulta de progresso?

Eu persistiria estados e referências relevantes, oferecendo um recurso de operação consultável. Atualização por SSE melhora experiência, mas reconexão consulta a autoridade; a conexão aberta não é o registro durável do andamento.

Termo para lembrar: Operation status | Base técnica: referência 1.

13. Como trata cancelamento assíncrono?

Eu verificaria se o trabalho ainda pode ser impedido ou se já produziu efeito. Cancelar a espera do cliente não cancela execução remota; persistiria intenção de cancelamento e resultado real separadamente.

Termo para lembrar: Cancellation semantics | Base técnica: referência 1.

14. Como recupera worker morto?

Eu faria claim com tentativa e prazo, permitindo retomada após expiração. Um worker antigo pode retornar; gravações condicionadas à tentativa e idempotência no efeito externo protegem contra processamento concorrente indevido.

Termo para lembrar: Lease recovery | Base técnica: referência 1.

15. Como evita tempestade de retries?

Eu definiria uma camada responsável, backoff com jitter e orçamento de repetição. Se cada chamada do fan-out repetir autonomamente, um pequeno incidente pode multiplicar carga e impedir a recuperação da dependência.

Termo para lembrar: Retry amplification | Base técnica: referência 1.

16. Como lida com fila cheia?

Eu estabeleceria capacidade de armazenamento e idade máxima aceitável, reduzindo admissão antes de esgotar recursos. Responder sucesso indefinidamente enquanto o prazo se torna impossível apenas transforma indisponibilidade visível em dívida operacional.

Termo para lembrar: Admission control | Base técnica: referência 1.

17. Como organiza prioridade?

Eu combinaria urgência, cotas e envelhecimento para evitar starvation. Prioridade estrita pode abandonar trabalho comum; a política deve proteger solicitações críticas sem tornar as demais permanentemente incapazes de avançar.

Termo para lembrar: Fair scheduling | Base técnica: referência 1.

18. Como trata um destino lento no fan-out?

Eu isolaria seu pool e limite de chamadas, interrompendo espera conforme deadline. Outros destinos continuam quando a jornada permite; compartilhar toda capacidade com o serviço lento espalharia sua falha pelo sistema inteiro.

Termo para lembrar: Bulkhead | Base técnica: referência 1.

19. Quando faria hedged request?

Eu consideraria duplicar uma leitura lenta após um limiar, com orçamento restrito e cancelamento do perdedor. Só usaria em operações seguras; duplicar escrita ou sobrecarregar réplicas pode piorar justamente a cauda.

Termo para lembrar: Hedging | Base técnica: referência 1.

20. Como controla ordenação assíncrona?

Eu definiria a ordem necessária por agregado e manteria sequência explícita. Particionar pela chave ajuda, mas produtores concorrentes e retries ainda exigem cuidado; não prometeria ordem global sem requisito que justifique seu custo.

Termo para lembrar: Per-key ordering | Base técnica: referência 1.

21. Como aplica backpressure?

Eu reduziria produção ou admissão conforme consumidores atingem limites, propagando sinais úteis às camadas anteriores. Uma fila absorve variação temporária, mas não resolve diferença permanente entre taxa de chegada e capacidade disponível.

Termo para lembrar: Backpressure | Base técnica: referência 1.

22. Como identifica conclusão distribuída?

Eu modelaria quais resultados são obrigatórios, registrando cada etapa e sua identidade. Timeout de uma etapa não vira falha definitiva automaticamente; o processo pode precisar de consulta ou reconciliação antes de decidir o desfecho.

Termo para lembrar: Completion protocol | Base técnica: referência 1.

23. Como evita evento perdido após salvar?

Eu gravaria a mudança e a obrigação de publicar na mesma transação local. O relay pode repetir, então consumidores precisam reconhecer identidade; isso elimina a lacuna local sem criar atomicidade global entre serviços.

Termo para lembrar: Transactional outbox | Base técnica: referência 1.

24. Como trata múltiplos consumidores?

Eu daria progresso e política de erro independentes a consumidores com responsabilidades distintas. Um relatório lento não deve impedir crédito financeiro; separação também permite replay e escalabilidade sem alterar a conclusão de outras tarefas.

Termo para lembrar: Consumer isolation | Base técnica: referência 1.

25. Como observa uma jornada assíncrona?

Eu propagaria identidade de operação e contexto, relacionando aceite, publicação e processamento. Mediria idade e conclusão além da latência HTTP, porque uma API saudável pode esconder milhares de solicitações nunca finalizadas.

Termo para lembrar: Trace correlation | Base técnica: referência 1.

26. Como usa polling no cliente?

Eu limitaria frequência, aplicaria backoff e pararia em estados terminais ou timeout de acompanhamento. Polling é simples e recuperável, mas milhares de clientes sincronizados podem criar carga; jitter e cache apropriado ajudam.

Termo para lembrar: Polling strategy | Base técnica: referência 1.

27. Como trata tarefas expiradas?

Eu verificaria se executar depois do prazo ainda tem valor ou risco. Algumas tarefas podem ser descartadas com registro; obrigações financeiras não desaparecem por expiração técnica e exigem decisão de negócio ou reconciliação.

Termo para lembrar: Deadline-aware execution | Base técnica: referência 1.

28. Como evita resposta fantasma de tela anterior?

Eu associaria a resposta à seleção e versão atual da interface, descartando resultados antigos. Cancelar a chamada reduz trabalho, mas a checagem de identidade continua necessária quando a resposta já estava em trânsito.

Termo para lembrar: Request cancellation | Base técnica: referência 1.

29. Como dimensiona paralelismo dos workers?

Eu mediria tempo de serviço, limites do destino e variabilidade, aumentando consumidores até o gargalo real. Mais threads não superam quota externa e podem aumentar conexões, contenção e retries sem melhorar conclusão.

Termo para lembrar: Effective throughput | Base técnica: referência 1.

30. Como defenderia um fluxo híbrido?

Eu faria validação e aceitação durável no caminho síncrono, deixando processamento longo em workers. Explicaria estados, prazo e recuperação; a combinação responde rápido sem fingir que o resultado de negócio já aconteceu.

Termo para lembrar: Hybrid interaction | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Síncrono

Ganho. Resultado imediato

Custo e motivo. Acopla disponibilidade

Como decidir. Quando decisão depende da resposta

02. Assíncrono

Ganho. Isola tempo de execução

Custo e motivo. Adiciona estados e lag

Como decidir. Quando conclusão pode esperar

03. Paralelismo alto

Ganho. Reduz caminho feliz

Custo e motivo. Aumenta pressão

Como decidir. Quando destinos têm folga medida

04. Resposta parcial

Ganho. Preserva utilidade

Custo e motivo. Complica contrato

Como decidir. Quando partes são opcionais

05. Batch

Ganho. Amortiza chamadas

Custo e motivo. Aumenta espera

Como decidir. Quando itens podem agrupar

06. Fila grande

Ganho. Absorve pico

Custo e motivo. Acumula atraso

Como decidir. Quando pico é transitório

07. Hedging

Ganho. Reduz cauda de leitura

Custo e motivo. Duplica trabalho

Como decidir. Quando há orçamento e segurança

08. Polling

Ganho. Simplifica cliente

Custo e motivo. Gera consultas periódicas

Como decidir. Quando atualizações são pouco frequentes

09. Push

Ganho. Reduz consultas

Custo e motivo. Exige conexões e retomada

Como decidir. Quando eventos são frequentes

10. Prioridade

Ganho. Protege urgências

Custo e motivo. Pode causar starvation

Como decidir. Quando combinada com aging

Duas alternativas ao caminho proposto

1. Workflow durável

Quando faz sentido. Quando coordenação tem etapas e esperas longas

Como implementar. Persistir histórico de execução e retomada

Limites. Não elimina idempotência dos efeitos externos

2. Processamento local síncrono

Quando faz sentido. Quando operação é curta e atômica

Como implementar. Executar dentro de uma fronteira controlada

Limites. Não serve para dependências lentas ou tarefas humanas

Referências e alcance da evidência

1. Google SRE: Handling Overload

Sobrecarga, filas e rejeição de trabalho; números e dimensionamentos são hipóteses didáticas.

https://sre.google/sre-book/handling-overload/

2. gRPC: Core concepts

Serviços, mensagens, tipos de RPC e ciclo de chamadas.

https://grpc.io/docs/what-is-grpc/core-concepts/

3. Microsoft: Architecture styles

Características e critérios de estilos arquiteturais; decisões particulares são análises autorais.

https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/

Continue em: T045 · T048 · T062

T010 / A. Enquadramento e arquitetura

Decisões arquiteturais orientadas a evidências

Decisões orientadas a evidências relacionam hipóteses, opções, riscos e métricas, usando experimentos proporcionais à incerteza e registrando condições que justificariam revisão.

Quando usar. Quando existem alternativas plausíveis, restrições conflitantes ou decisões caras de reverter.

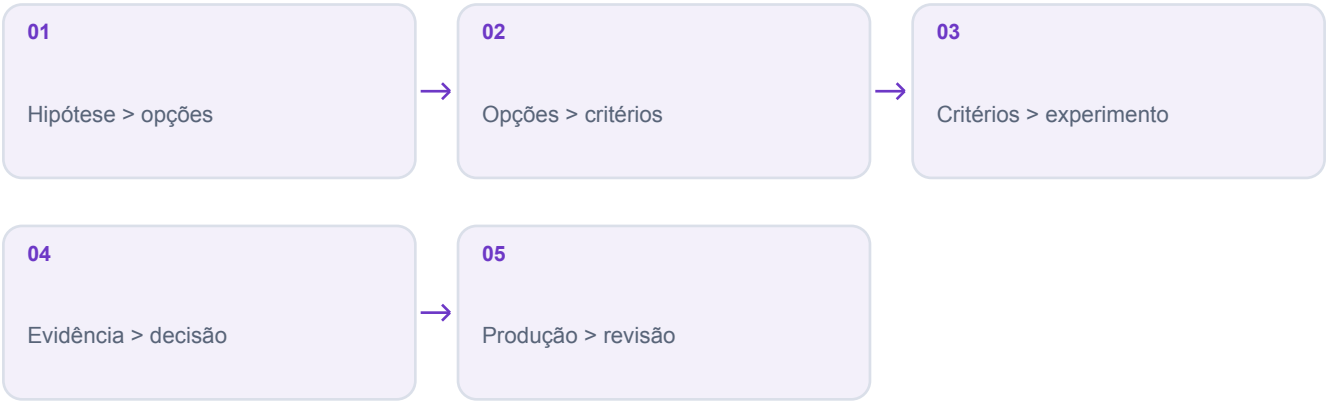
Quando evitar ou limitar. Evitar benchmarks irrelevantes, preferência pessoal apresentada como fato e provas de conceito sem critérios de sucesso.

Uma abertura possível

Vou separar o que sabemos do que assumimos. Compararei poucas opções pelos requisitos críticos e medirei primeiro a incerteza que pode mudar a decisão.

Peças e tecnologias. ADR · RFC · k6 · OpenTelemetry · feature flags · shadow traffic

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Registrar problema e restrições.

2. Comparar duas ou três alternativas reais.

3. Definir evidência mínima e critérios de parada.

4. Executar experimento representativo e registrar limites.

5. Decidir, acompanhar e revisar por gatilhos claros.

30 perguntas para sustentar o desenho

01. Como evita escolher pela tecnologia favorita?

Eu começaria por requisitos e restrições mensuráveis, comparando opções nos mesmos critérios. Familiaridade conta como custo operacional, mas não comprova adequação; explicaria quais evidências sustentam a escolha e quais hipóteses permanecem.

Termo para lembrar: Decision criteria | Base técnica: referência 1.

02. O que registra numa ADR?

Eu documentaria contexto, alternativas, decisão e consequências, incluindo riscos aceitos. Uma ADR precisa explicar por que a escolha fazia sentido naquele momento, permitindo revisão sem depender da memória de quem participou.

Termo para lembrar: Architecture decision record | Base técnica: referência 1.

03. Como distingue fato de hipótese?

Eu marcaria medições e garantias documentadas separadamente de estimativas e preferências. Essa distinção orienta o próximo experimento: validar uma hipótese crítica pode valer mais que detalhar uma parte já conhecida do sistema.

Termo para lembrar: Evidence classification | Base técnica: referência 1.

04. Quando faz prova de conceito?

Eu faria quando uma incerteza pode invalidar a arquitetura, como throughput ou compatibilidade. Definiria critérios antes de implementar; um protótipo que apenas funciona no caminho feliz não comprova viabilidade operacional.

Termo para lembrar: Proof of concept | Base técnica: referência 1.

05. Como escolhe benchmark representativo?

Eu reproduziria tamanho de dados, distribuição de chaves, concorrência e falhas relevantes. Comparar bancos com consultas distintas ou cache quente só em uma opção produz números atraentes, mas não evidência útil.

Termo para lembrar: Workload fidelity | Base técnica: referência 1.

06. Como trata resultados contraditórios?

Eu examinaria metodologia, variância, configuração e população antes de concluir. Repetiria o teste relevante com controle das diferenças, evitando escolher apenas a medição que confirma a preferência inicial da equipe.

Termo para lembrar: Experimental validity | Base técnica: referência 1.

07. Como avalia uma decisão reversível?

Eu estimaria custo de migração de dados, contratos e pessoas, não apenas de trocar código. Quanto mais difícil reverter, maior a evidência necessária; decisões locais baratas podem avançar com experimentação menor.

Termo para lembrar: Reversibility | Base técnica: referência 1.

08. Como compara custo operacional?

Eu incluiria plantão, atualização, backup, observabilidade e conhecimento disponível. Uma tecnologia com melhor benchmark pode exigir especialização indisponível; esse custo afeta confiabilidade e tempo de entrega tanto quanto recursos computacionais.

Termo para lembrar: Operational complexity | Base técnica: referência 1.

09. Como evita análise paralisante?

Eu definiria prazo de investigação e a menor evidência capaz de distinguir as opções. Se ambas atendem ao horizonte, escolheria a mais simples com gatilhos de revisão, em vez de buscar certeza impossível.

Termo para lembrar: Timeboxed investigation | Base técnica: referência 1.

10. Como usa dados de produção sem risco?

Eu copiaria tráfego ou consultas para um caminho sem efeitos, protegendo dados e capacidade. Compararia resultados e latência, mas impediria publicações ou pagamentos reais; sombra precisa de isolamento, não apenas outro endpoint.

Termo para lembrar: Shadow evaluation | Base técnica: referência 1.

11. Como testa escala futura?

Eu identificaria quais componentes crescem linearmente e quais mudam de comportamento sob contenção. Extrapolar dez vezes um teste pequeno ignora cache, índices e particionamento; validaria os pontos de inflexão mais relevantes.

Termo para lembrar: Extrapolation limits | Base técnica: referência 1.

12. Como lida com ausência de métricas?

Eu criaria uma linha de base mínima antes de otimizar, medindo a jornada e o recurso suspeito. Sem referência, fica difícil saber se a mudança resolveu o problema ou apenas deslocou seu custo.

Termo para lembrar: Baseline measurement | Base técnica: referência 1.

13. Como considera segurança na comparação?

Eu avaliaria dados expostos, fronteiras de confiança e mecanismos de recuperação por opção. Segurança não é uma nota genérica; identificaria ameaças concretas e o custo de mitigá-las na arquitetura proposta.

Termo para lembrar: Risk-based design | Base técnica: referência 1.

14. Como considera equipe pequena?

Eu reduziria componentes operacionais e favoreceria tecnologias sustentáveis pela equipe disponível. Capacidade humana é restrição arquitetural; um desenho sofisticado sem ownership e plantão adequado pode ser menos confiável que uma solução simples.

Termo para lembrar: Team capability | Base técnica: referência 1.

15. Como define sucesso da mudança?

Eu estabeleceria melhoria esperada e limites que não podem piorar, como latência, custo e taxa de erro. Sem guardrails, reduzir tempo médio pode esconder mais falhas financeiras ou degradação em clientes específicos.

Termo para lembrar: Success criteria | Base técnica: referência 1.

16. Como compara build e buy?

Eu compararia diferenciação de negócio, prazo, integração, custo total e saída do fornecedor. Construir oferece controle, mas cria obrigação operacional; comprar reduz implementação inicial sem eliminar dependências, contratos e riscos de migração.

Termo para lembrar: Build versus buy | Base técnica: referência 1.

17. Como usa um paper de outra empresa?

Eu extrairia o princípio e verificaria diferenças de escala, equipe e requisito. Um caso público inspira hipóteses, não prova que a mesma tecnologia é adequada nem revela toda a arquitetura interna daquela organização.

Termo para lembrar: External validity | Base técnica: referência 1.

18. Como registra risco aceito?

Eu nomearia cenário, impacto, probabilidade aproximada, owner e ação de contingência. Aceitar risco precisa ser uma decisão explícita e revisável, não um comentário vago que desaparece depois da reunião técnica.

Termo para lembrar: Risk acceptance | Base técnica: referência 1.

19. Quando muda uma decisão antiga?

Eu definiria gatilhos como crescimento, incidentes ou mudança regulatória, comparando custos de manter e migrar. Uma decisão pode ter sido correta e deixar de ser; revisão não exige tratar o passado como erro.

Termo para lembrar: Review trigger | Base técnica: referência 1.

20. Como evita solução para um pico imaginário?

Eu pediria histórico ou cenário de crescimento plausível e avaliaria o custo de adiar capacidade. Preparar caminho de evolução pode ser melhor que pagar agora por uma complexidade que talvez nunca seja necessária.

Termo para lembrar: Demand evidence | Base técnica: referência 1.

21. Como avalia lock-in?

Eu examinaria formatos de dados, APIs proprietárias, operação e tempo de migração. Lock-in não é automaticamente proibido; aceitaria quando benefícios compensarem e houver estratégia realista para mudanças de preço ou disponibilidade.

Termo para lembrar: Exit cost | Base técnica: referência 1.

22. Como usa feature flags?

Eu liberaria a decisão gradualmente para observar impacto, com critério de interrupção e fallback seguro. Flag não desfaz efeitos persistidos; mudanças de schema e dinheiro exigem compatibilidade além de desligar um botão.

Termo para lembrar: Controlled rollout | Base técnica: referência 1.

23. Como compara consistência entre opções?

Eu descreveria a anomalia que cada opção permite e o efeito na jornada. Rótulos como forte ou eventual são insuficientes sem saber escopo, leitura, concorrência e comportamento durante falhas relevantes.

Termo para lembrar: Semantic comparison | Base técnica: referência 1.

24. Como trata opinião de especialista?

Eu usaria experiência como evidência contextual, pedindo exemplos e condições de validade. Uma recomendação pode orientar o teste, mas não substitui requisito nem deve impedir revisão quando medições locais mostram outro comportamento.

Termo para lembrar: Expert judgment | Base técnica: referência 1.

25. Como apresenta tradeoffs sem indecisão?

Eu escolheria uma opção, explicaria o benefício principal e o custo aceito, mencionando quando mudaria. Enumerar possibilidades sem concluir não demonstra arquitetura; a decisão precisa ser assumida dentro das hipóteses declaradas.

Termo para lembrar: Decision rationale | Base técnica: referência 1.

26. Como evita otimização local?

Eu mediria o resultado da jornada antes e depois, não apenas o componente alterado. Acelerar publicação pode saturar consumidores e piorar conclusão; critérios precisam abranger o fluxo que entrega valor ao usuário.

Termo para lembrar: End-to-end objective | Base técnica: referência 1.

27. Como mede custo de migração?

Eu incluiria convivência de versões, backfill, reconciliação, treinamento e janela de risco. Comparar apenas custo mensal final ignora o investimento e os incidentes potenciais para alcançar a arquitetura pretendida.

Termo para lembrar: Migration economics | Base técnica: referência 1.

28. Como valida recuperação?

Eu ensaiaria restauração e failover com dados representativos, medindo perda e tempo real. Ter backup configurado não comprova recuperação; credenciais, quotas, dependências e efeitos externos precisam entrar no exercício.

Termo para lembrar: Recovery experiment | Base técnica: referência 1.

29. Como compartilha a decisão?

Eu escreveria para quem não participou da conversa, começando pelo problema e comportamento resultante. Explicaria evidência, limites e consequências, evitando um catálogo de tecnologias que não mostra por que escolhemos cada peça.

Termo para lembrar: Technical communication | Base técnica: referência 1.

30. Como fecha uma discussão sem certeza total?

Eu explicitaria a incerteza restante, sua consequência e o mecanismo de detecção. Escolheria a opção mais adequada ao contexto com revisão planejada, porque engenharia exige decisões responsáveis mesmo quando os dados são incompletos.

Termo para lembrar: Bounded uncertainty | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Experimento extenso

Ganho. Reduz incerteza

Custo e motivo. Atrasa entrega

Como decidir. Quando decisão é irreversível

02. Prova mínima

Ganho. Acelera aprendizado

Custo e motivo. Cobre menos falhas

Como decidir. Quando hipótese é localizada

03. Tecnologia familiar

Ganho. Reduz carga operacional

Custo e motivo. Pode limitar capacidades

Como decidir. Quando atende requisitos

04. Tecnologia nova

Ganho. Oferece benefício específico

Custo e motivo. Exige aprendizado

Como decidir. Quando ganho é demonstrado

05. Sombra

Ganho. Compara sem efeito

Custo e motivo. Consome recursos

Como decidir. Quando tráfego real importa

06. Canary

Ganho. Valida produção

Custo e motivo. Expõe parte dos usuários

Como decidir. Quando rollback é seguro

07. Padrão corporativo

Ganho. Reduz variedade

Custo e motivo. Pode ser inadequado localmente

Como decidir. Quando requisitos são comuns

08. Exceção arquitetural

Ganho. Atende necessidade especial

Custo e motivo. Aumenta manutenção

Como decidir. Quando evidência justifica

09. Compra

Ganho. Reduz construção inicial

Custo e motivo. Cria dependência

Como decidir. Quando produto não diferencia negócio

10. Construção

Ganho. Amplia controle

Custo e motivo. Exige operação contínua

Como decidir. Quando capacidade é estratégica

Duas alternativas ao caminho proposto

1. Arquitetura evolutiva

Quando faz sentido. Quando incerteza é alta e decisões são reversíveis

Como implementar. Entregar fatias e medir fitness functions

Limites. Exige disciplina para não acumular improvisações permanentes

2. Avaliação formal de riscos

Quando faz sentido. Quando consequências e dependências são grandes

Como implementar. Comparar cenários, mitigação e aprovação dos responsáveis

Limites. Pode ser lenta e não substitui testes técnicos

Referências e alcance da evidência

1. Microsoft: Architecture styles

Características e critérios de estilos arquiteturais; decisões particulares são análises autorais.

https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/

2. Google SRE: Implementing SLOs

Definição de indicadores, objetivos e políticas de orçamento de erro; os exemplos de entrevista são autorais.

https://sre.google/workbook/implementing-slos/

3. Google SRE: Handling Overload

Sobrecarga, filas e rejeição de trabalho; números e dimensionamentos são hipóteses didáticas.

https://sre.google/sre-book/handling-overload/

Continue em: T111 · T112 · T117

T011 / B. Sistemas distribuídos

CAP e PACELC

CAP estabelece uma impossibilidade sob partições no modelo considerado: garantir simultaneamente consistência linearizável e disponibilidade total. PACELC também destaca a escolha entre latência e consistência quando não há partição.

Quando usar. Para explicar decisões de leitura e escrita durante perda de comunicação, especialmente entre regiões.

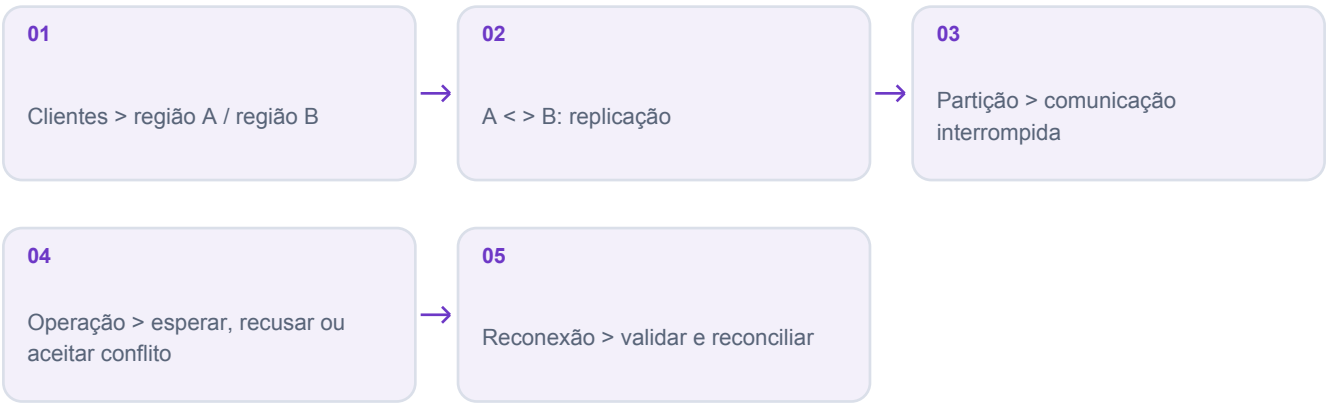
Quando evitar ou limitar. Evitar o slogan escolha dois, tratar partição como opção ou confundir disponibilidade formal de CAP com um SLO comercial.

Uma abertura possível

Vou descrever uma partição concreta e a operação afetada. Se não posso confirmar a autoridade, prefiro estado pendente ou recusa quando aceitar poderia violar a invariante financeira.

Peças e tecnologias. etcd · PostgreSQL · DynamoDB · Cassandra · Raft

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Definir operação e escopo de consistência.

2. Construir cenário de partição e autoridade.

3. Escolher comportamento conforme invariante.

4. Definir experiência e reconciliação.

5. Medir custo de latência também sem falhas.

30 perguntas para sustentar o desenho

01. O que CAP realmente afirma?

Eu diria que, sob partição no modelo distribuído considerado, não podemos garantir ao mesmo tempo leituras linearizáveis e resposta válida a toda requisição em nó não falho. A decisão deve ser descrita por operação.

Termo para lembrar: CAP theorem | Base técnica: referência 1.

02. Partição é uma escolha de arquitetura?

Eu trataria falhas de comunicação como possibilidade do ambiente, não um recurso desligável. Posso reduzir sua probabilidade com infraestrutura, mas ainda preciso definir comportamento quando réplicas deixam de trocar mensagens confiavelmente.

Termo para lembrar: Partition tolerance | Base técnica: referência 1.

03. Disponibilidade CAP equivale a 99,99%?

Eu separaria a propriedade formal de responder às requisições da disponibilidade estatística medida por um SLO. Uma arquitetura que interrompe escritas sob partição pode ter excelente disponibilidade prática quando esse evento é raro.

Termo para lembrar: Formal availability | Base técnica: referência 1.

04. Consistência CAP significa regras de negócio corretas?

Eu explicaria que o C se refere a uma visão compatível com ordem real das operações, não a qualquer regra contábil. Ainda preciso implementar autorização, partidas dobradas e validações mesmo num armazenamento linearizável.

Termo para lembrar: Linearizability | Base técnica: referência 1.

05. Por que escolha dois é enganoso?

Eu mostraria que a tensão central aparece quando a comunicação falha. Fora desse cenário, existem outros custos e escolhas; classificar um produto inteiro por três letras esconde operações e configurações diferentes.

Termo para lembrar: Failure model | Base técnica: referência 1.

06. O que PACELC acrescenta?

Eu diria que mesmo sem partição há custo para coordenar réplicas, especialmente distantes. A escolha entre menor latência e garantia de consistência ajuda discutir o caminho normal, que CAP sozinho não detalha.

Termo para lembrar: PACELC | Base técnica: referência 1.

07. Como aplica isso ao débito de saldo?

Eu impediria autorização concorrente sem coordenação quando isso puder exceder saldo. Poderia oferecer consulta degradada ou solicitação pendente, mas aceitar débito em ambos os lados exige outra garantia de negócio explicitamente desenhada.

Termo para lembrar: Financial invariant | Base técnica: referência 1.

08. Pode manter leitura disponível durante partição?

Eu avaliaria se aquela leitura tolera dado antigo e indicaria versão ou frescor. Mostrar histórico pode continuar útil; usar o mesmo dado para autorizar uma movimentação é outra operação com risco diferente.

Termo para lembrar: Stale read | Base técnica: referência 1.

09. O que acontece num cluster de três nós dividido em dois e um?

Eu esperaria que o lado com maioria pudesse avançar se o protocolo e a eleição permitirem. O nó isolado não deve confirmar escritas conflitantes; clientes ligados a ele precisam redirecionamento ou resposta indisponível.

Termo para lembrar: Majority partition | Base técnica: referência 1.

10. Ter cinco nós resolve CAP?

Eu aumentaria tolerância a certas falhas e possibilidades de maioria, mas não eliminaria a impossibilidade. Uma partição ainda pode separar nós de forma que algumas requisições não consigam obter coordenação necessária.

Termo para lembrar: Redundancy limits | Base técnica: referência 1.

11. Uma fila resolve o conflito?

Eu poderia persistir uma intenção para execução posterior, distinguindo aceite de conclusão. Isso mantém alguma utilidade ao usuário, mas não prova que a operação financeira foi autorizada ou executada durante a partição.

Termo para lembrar: Deferred acceptance | Base técnica: referência 1.

12. E se usar cache local?

Eu permitiria dados cacheados apenas onde o contrato tolera atraso, com limites e sinalização. Cache aumenta disponibilidade de leitura, mas não cria conhecimento sobre escritas que ocorreram no outro lado da partição.

Termo para lembrar: Degraded read path | Base técnica: referência 1.

13. Quórum resolve tudo?

Eu verificaria interseção, versão, protocolo de escrita e tratamento de concorrência. Uma fórmula de maioria não basta para garantir linearizabilidade; sloppy quorum, relógios e leituras incompletas podem mudar completamente a propriedade oferecida.

Termo para lembrar: Quorum assumptions | Base técnica: referência 1.

14. Como distingue nó lento de nó morto?

Eu não consigo saber perfeitamente apenas por timeout numa rede assíncrona. Uso suspeita operacional com protocolos seguros; reduzir timeout acelera reação, mas aumenta falsas suspeitas e pode provocar eleições ou retries desnecessários.

Termo para lembrar: Failure detector | Base técnica: referência 1.

15. Como trata active-active?

Eu perguntaria quais dados aceitam escritas simultâneas e como conflitos são prevenidos ou resolvidos. Active-active de entrada pode coexistir com autoridade única por conta; o nome não garante escrita global segura.

Termo para lembrar: Multiwriter architecture | Base técnica: referência 1.

16. CRDT torna qualquer operação AP?

Eu usaria estruturas convergentes para operações com semântica adequada, como certos contadores ou conjuntos. Limite de crédito global exige restrições adicionais; convergir depois não apaga uma autorização inválida feita durante a partição.

Termo para lembrar: Invariant compatibility | Base técnica: referência 1.

17. Pode dividir limite previamente?

Eu poderia reservar cotas independentes por região, permitindo operações locais dentro dessas parcelas. Isso preserva um limite global sob condições específicas, mas reduz flexibilidade e exige coordenação para redistribuir capacidade entre regiões.

Termo para lembrar: Escrow allocation | Base técnica: referência 1.

18. Como apresenta o comportamento ao cliente?

Eu distinguiria consulta desatualizada, solicitação recebida e operação confirmada. Durante perda de coordenação, estados pendentes ou indisponibilidade limitada podem ser mais corretos que mostrar sucesso e depois desfazer uma promessa financeira.

Termo para lembrar: Honest status | Base técnica: referência 1.

19. Como recupera após a rede voltar?

Eu reconciliaria posições, versões e resultados desconhecidos antes de liberar caminhos conflitantes. Reabrir tráfego não prova convergência; dependendo da política, preciso reparar réplicas e verificar efeitos externos executados durante a interrupção.

Termo para lembrar: Partition recovery | Base técnica: referência 1.

20. DNS elimina split-brain?

Eu usaria DNS para direcionar clientes, mas ele não impede um writer antigo de continuar operando. Fencing e autoridade precisam existir no protocolo ou destino, porque caches e conexões antigas podem permanecer ativos.

Termo para lembrar: Routing versus fencing | Base técnica: referência 1.

21. Qual relação com RPO?

Eu separaria visibilidade consistente de durabilidade diante de perda regional. Uma escrita pode ser confirmada localmente e ainda não existir na réplica assíncrona; promoção exige avaliar qual conjunto de dados pode ter sido perdido.

Termo para lembrar: Durability scope | Base técnica: referência 1.

22. Qual relação com latência geográfica?

Eu consideraria viagens de rede necessárias para confirmar entre regiões. A distância impõe um piso físico; replicação síncrona pode melhorar garantias, mas aumenta latência e sensibilidade à região mais lenta do quórum.

Termo para lembrar: Coordination latency | Base técnica: referência 1.

23. É aceitável recusar escrita?

Eu recusaria quando não consigo preservar uma invariante crítica, oferecendo informação e retomada seguras. A recusa tem custo de experiência, mas pode ser a escolha correta diante do risco de débito ou crédito indevido.

Termo para lembrar: Fail closed | Base técnica: referência 1.

24. Como trata operação não financeira?

Eu poderia aceitar preferências de interface com resolução posterior enquanto bloqueio movimentação financeira. O sistema não precisa de uma política uniforme; classifico operações pela consequência do conflito e pelas garantias exigidas.

Termo para lembrar: Per-operation policy | Base técnica: referência 1.

25. Pode escolher CP para todo banco?

Eu consultaria a garantia de cada operação, índice e configuração, incluindo failover. Um rótulo de produto não substitui contrato técnico; leituras de réplica e transações podem ter propriedades distintas no mesmo serviço.

Termo para lembrar: Configuration-specific guarantee | Base técnica: referência 1.

26. Como testa essa decisão?

Eu interromperia comunicação entre grupos de nós, executando operações concorrentes e verificando históricos e invariantes. Também testaria reconexão; sobreviver ao corte sem crash não demonstra que os resultados permaneceram corretos.

Termo para lembrar: Partition testing | Base técnica: referência 1.

27. CAP explica perda de mensagem?

Eu usaria CAP para a escolha de consistência e disponibilidade sob partição, não como explicação universal. Perder evento após commit pode ser um problema de escrita dupla que outbox resolve dentro de outra fronteira.

Termo para lembrar: Scope of theorem | Base técnica: referência 1.

28. Como inclui operação manual?

Eu permitiria intervenção com evidências, autorização e trilha, sem conceder liberdade para inventar resultado. Operação manual não supera impossibilidade informacional; precisa consultar autoridades e registrar decisões quando a ambiguidade finalmente for resolvida.

Termo para lembrar: Controlled recovery | Base técnica: referência 1.

29. Como evita discurso acadêmico demais?

Eu desenharia duas regiões sem comunicação e duas tentativas de gastar o mesmo saldo. Perguntaria qual resposta é permitida, conectando o teorema a uma consequência real em vez de apenas recitar letras.

Termo para lembrar: Concrete counterexample | Base técnica: referência 1.

30. Qual decisão faria para um portal bancário?

Eu manteria contexto não crítico disponível com frescor explícito e enviaria comandos financeiros à autoridade apropriada. Se ela não puder confirmar, apresentaria pendência ou recusa, preservando as garantias sem paralisar funcionalidades independentes.

Termo para lembrar: Mixed consistency policy | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Coordenação forte

Ganho. Evita certas anomalias

Custo e motivo. Pode bloquear na partição

Como decidir. Quando invariante exige autoridade comum

02. Leitura local

Ganho. Reduz latência

Custo e motivo. Pode estar atrasada

Como decidir. Quando atraso é aceitável

03. Escrita local concorrente

Ganho. Mantém acesso regional

Custo e motivo. Exige conflitos explícitos

Como decidir. Quando operação é conciliável

04. Writer único

Ganho. Simplifica ordem

Custo e motivo. Cria dependência de região

Como decidir. Quando domínio exige serialização

05. Quórum distante

Ganho. Amplia durabilidade

Custo e motivo. Aumenta RTT

Como decidir. Quando perda regional é crítica

06. Intenção pendente

Ganho. Preserva entrada

Custo e motivo. Adia resultado

Como decidir. Quando execução pode esperar

07. Cotas regionais

Ganho. Permitem autonomia limitada

Custo e motivo. Fragmentam capacidade

Como decidir. Quando limite é divisível

08. Fail closed

Ganho. Protege invariante

Custo e motivo. Recusa operações legítimas

Como decidir. Quando risco de conflito é alto

09. Stale reads

Ganho. Mantêm contexto

Custo e motivo. Exigem sinalização

Como decidir. Quando não autorizam dinheiro

10. Mais réplicas

Ganho. Toleram mais falhas

Custo e motivo. Custam coordenação

Como decidir. Quando o modelo de falha justifica

Duas alternativas ao caminho proposto

1. Autoridade por entidade

Quando faz sentido. Quando contas podem ter região proprietária

Como implementar. Roteamento global com writer único por chave e transferência controlada

Limites. Clientes podem pagar latência e ficar pendentes na perda da autoridade

2. Escrow de capacidade

Quando faz sentido. Quando recurso pode ser reservado antecipadamente

Como implementar. Distribuir direitos de consumo limitados e reconciliar depois

Limites. Não resolve invariantes arbitrárias nem redistribuição sem comunicação

Referências e alcance da evidência

1. Gilbert e Lynch: prova de CAP

Modelo formal de consistência atômica, disponibilidade e partições; exemplos bancários são análises autorais.

https://www.comp.nus.edu.sg/~gilbert/pubs/BrewersConjecture-SigAct.pdf

2. Daniel Abadi: PACELC

Trade-off entre disponibilidade e consistência sob partição e entre latência e consistência na operação normal.

https://www.cs.umd.edu/~abadi/papers/abadi-pacelc.pdf

3. Jepsen: Consistency Models

Definições e relações entre modelos de consistência, com atenção ao escopo das garantias.

https://jepsen.io/consistency/models

Continue em: T012 · T015 · T020

T012 / B. Sistemas distribuídos

Modelos de consistência

Modelos de consistência descrevem histórias observáveis permitidas. Linearizabilidade, serializabilidade, causalidade e garantias de sessão respondem a perguntas diferentes sobre ordem, concorrência e visibilidade.

Quando usar. Ao escolher leituras, transações e projeções e explicar quais anomalias o usuário pode observar.

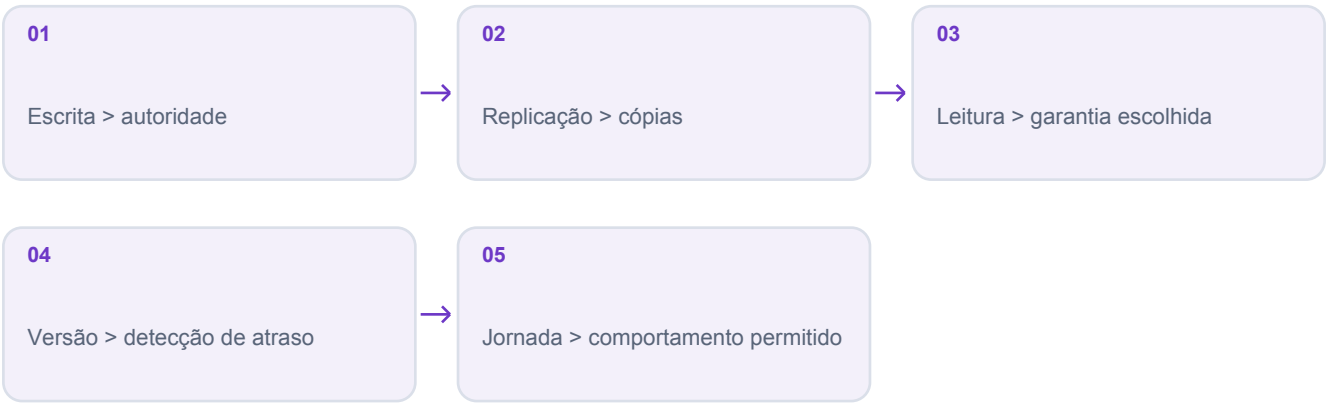
Quando evitar ou limitar. Evitar usar forte e eventual sem escopo, prazo ou operação; consistência de uma chave não implica transação entre várias.

Uma abertura possível

Vou escolher a garantia a partir da anomalia que precisamos impedir. Uma tela pode tolerar atraso, enquanto uma autorização financeira exige uma decisão coordenada sobre o estado relevante.

Peças e tecnologias. PostgreSQL · DynamoDB · etcd · Cassandra · session tokens

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Nomear anomalias e invariantes.

2. Definir escopo por chave, transação e sessão.

3. Escolher caminhos de leitura e escrita.

4. Propagar versões ou tokens necessários.

5. Testar históricos concorrentes e experiência degradada.

30 perguntas para sustentar o desenho

01. O que significa linearizabilidade?

Eu diria que cada operação parece ocorrer num ponto entre sua chamada e resposta, respeitando operações já concluídas. Verificaria o escopo: uma chave linearizável não torna automaticamente uma transferência entre duas contas atômica.

Termo para lembrar: Linearizability | Base técnica: referência 1.

02. O que é serializabilidade?

Eu exigiria que o efeito de transações concorrentes equivalesse a alguma execução serial. Isso protege contra anomalias transacionais, mas não implica sozinho a ordem de tempo real que uma garantia estritamente serializável acrescenta.

Termo para lembrar: Serializability | Base técnica: referência 1.

03. O que é strict serializability?

Eu combinaria serializabilidade com respeito à ordem real entre transações não sobrepostas. É útil quando precisamos raciocinar sobre múltiplos objetos e observações externas, assumindo o escopo e as condições documentadas pelo armazenamento.

Termo para lembrar: Strict serializability | Base técnica: referência 1.

04. Consistência sequencial é igual a linearizável?

Eu explicaria que uma ordem global pode respeitar a ordem de cada processo sem respeitar toda precedência de tempo real entre processos. A diferença importa quando um cliente espera observar uma escrita já confirmada externamente.

Termo para lembrar: Sequential consistency | Base técnica: referência 1.

05. O que significa eventual?

Eu diria que réplicas tendem a convergir quando atualizações cessam e a propagação funciona, conforme o sistema. Isso não define sozinho prazo máximo, causalidade ou resolução de conflitos; preciso contratar essas propriedades separadamente.

Termo para lembrar: Eventual consistency | Base técnica: referência 1.

06. Como garante ler a própria escrita?

Eu propagaria uma versão ou posição mínima e direcionaria a leitura a uma réplica suficientemente atualizada. Alternativamente, leria na autoridade após escrever; apenas manter sessão no mesmo frontend não garante frescor do banco.

Termo para lembrar: Read-your-writes | Base técnica: referência 1.

07. Como evita leitura voltar no tempo?

Eu guardaria a maior versão observada na sessão e não aceitaria uma réplica abaixo dela. Isso protege continuidade da experiência, mas pode exigir espera ou fallback quando a réplica local ainda está atrasada.

Termo para lembrar: Monotonic reads | Base técnica: referência 1.

08. O que é consistência causal?

Eu preservaria a ordem de operações dependentes, como criar uma conta antes de mostrar seu crédito associado. Operações independentes podem aparecer em ordens diferentes; causalidade não impõe uma sequência global de tudo.

Termo para lembrar: Causal consistency | Base técnica: referência 1.

09. Como representa dependências causais?

Eu propagaria tokens, versões ou vetores adequados ao sistema, evitando confiar apenas em horário de parede. O custo cresce com participantes e precisão necessária; escolheria metadados compatíveis com o domínio e o armazenamento.

Termo para lembrar: Causal metadata | Base técnica: referência 1.

10. Como trata saldo logo após Pix?

Eu retornaria a confirmação autoritativa com versão e faria a tela aguardar uma projeção ao menos nessa posição ou consultar o caminho forte. Não apresentaria saldo antigo sem explicar atualização pendente.

Termo para lembrar: Session freshness | Base técnica: referência 1.

11. Pode usar cache para autorização?

Eu só usaria dado cacheado se o protocolo preservar a regra necessária, como capacidade previamente reservada. Um saldo eventualmente consistente não é suficiente para autorizar débitos concorrentes sem outro mecanismo coordenado de limite.

Termo para lembrar: Stale authorization | Base técnica: referência 1.

12. Como distingue snapshot de dado atual?

Eu explicaria que um snapshot oferece visão coerente de um ponto lógico, mas pode não conter a última escrita. Coerência interna e frescor são dimensões diferentes; a consulta precisa saber qual delas exige.

Termo para lembrar: Snapshot semantics | Base técnica: referência 1.

13. Leitura forte elimina corrida posterior?

Eu lembraria que o estado pode mudar logo depois da leitura. Para agir, incorporo a condição na escrita ou transação; ler corretamente e atualizar depois sem controle ainda permite decisão concorrente inválida.

Termo para lembrar: Time-of-check/time-of-use | Base técnica: referência 1.

14. Como faz bounded staleness?

Eu definiria limite por tempo ou versões e verificaria a posição da réplica antes de responder. Se não conseguir cumprir, aguardo, redireciono ou falho; declarar um limite sem mecanismo não o torna garantido.

Termo para lembrar: Bounded staleness | Base técnica: referência 1.

15. Como compara consistência em índices secundários?

Eu verificaria a garantia específica do índice, que pode diferir da tabela principal. Uma busca pode omitir registro recém-criado mesmo quando leitura direta por chave está atualizada; o fluxo precisa prever essa diferença.

Termo para lembrar: Index consistency | Base técnica: referência 1.

16. Como resolve escritas concorrentes?

Eu escolheria política por significado: rejeitar conflito, mesclar campos compatíveis ou coordenar previamente. Last-write-wins pode descartar uma decisão válida; relógio mais recente não comprova que aquela alteração deveria prevalecer no negócio.

Termo para lembrar: Conflict resolution | Base técnica: referência 1.

17. Quando usaria CRDT?

Eu usaria operações com propriedades de convergência adequadas, como conjuntos específicos ou contadores. Confirmaria compatibilidade com invariantes; convergência matemática não torna seguro aceitar dois gastos sobre o mesmo saldo disponível.

Termo para lembrar: Convergent data type | Base técnica: referência 1.

18. Como trata leituras multichave?

Eu verificaria se valores precisam vir do mesmo snapshot. Ler débito e crédito em instantes diferentes pode mostrar uma transferência incompleta; uma transação de leitura ou projeção versionada pode preservar a visão necessária.

Termo para lembrar: Read skew | Base técnica: referência 1.

19. O que é write skew?

Eu mostraria duas transações que leem a mesma condição e alteram linhas diferentes, violando uma regra conjunta. Snapshot isolation pode permitir isso; serialização, lock apropriado ou redesign da invariante resolve conforme o caso.

Termo para lembrar: Write skew | Base técnica: referência 1.

20. Como explica consistência forte ao produto?

Eu usaria um exemplo: depois de confirmar bloqueio, consultas pelo caminho definido não devem mostrar o cartão como utilizável. Especificaria exceções de caches e sistemas externos, porque a garantia não atravessa fronteiras automaticamente.

Termo para lembrar: Observable guarantee | Base técnica: referência 1.

21. Como testa linearizabilidade?

Eu registraria invocações, respostas e concorrência, verificando se existe uma ordem válida compatível com o modelo. Testes de carga sem histórico podem mostrar rapidez, mas não detectar observações impossíveis sob a garantia prometida.

Termo para lembrar: History checking | Base técnica: referência 1.

22. Como usa versão em eventos?

Eu incluiria sequência por agregado e aplicaria eventos conforme sua semântica. Snapshots podem substituir versões antigas; deltas exigem detectar lacunas e recuperar, pois ignorar uma operação anterior pode perder efeito legítimo.

Termo para lembrar: Versioned projection | Base técnica: referência 1.

23. Consistência é escolha por banco ou por operação?

Eu configuraria conforme a necessidade e as capacidades documentadas: histórico pode usar réplica, enquanto decisão usa autoridade. O mesmo produto oferece caminhos diferentes; por isso classificações globais escondem limites importantes para a jornada.

Termo para lembrar: Per-operation consistency | Base técnica: referência 1.

24. Como define ordenação de mensagens?

Eu escolheria escopo por conversa, conta ou caso, evitando ordem global desnecessária. A ordem do transporte não equivale à ordem de commit se produtores publicam fora de sequência; origem e consumo também participam.

Termo para lembrar: Ordering scope | Base técnica: referência 1.

25. Como trata reconexão em outra região?

Eu carregaria o token de progresso e verificaria se a região consegue satisfazê-lo. Caso contrário, explicaria espera ou redirecionamento; trocar de região sem preservar sessão pode fazer o usuário observar estados anteriores.

Termo para lembrar: Session continuity | Base técnica: referência 1.

26. Como impede caches de regredir?

Eu associaria valor a versão e condicionaria atualização para não sobrescrever estado novo com resposta antiga. TTL limita duração, mas sozinho não impede uma corrida de preenchimento depois de uma invalidação correta.

Termo para lembrar: Version comparison | Base técnica: referência 1.

27. Como escolhe entre espera e dado antigo?

Eu classificaria a tarefa: contexto informativo pode tolerar atraso sinalizado; uma decisão sensível exige frescor ou recusa. O contrato deve permitir ao cliente distinguir os caminhos, em vez de degradar silenciosamente.

Termo para lembrar: Read availability policy | Base técnica: referência 1.

28. Como relaciona isolamento e replicação?

Eu avaliaria ambos separadamente. Uma transação serializável local pode ser lida por réplica atrasada; replicação síncrona não corrige automaticamente uma regra implementada com isolamento inadequado entre transações concorrentes no primário.

Termo para lembrar: Consistency dimensions | Base técnica: referência 1.

29. Como evita garantias exageradas?

Eu declararia objeto, região, operação e configuração aos quais a garantia se aplica. Também explicaria comportamento sob falha; palavras como consistente ou exatamente uma vez sem essas fronteiras impedem avaliação técnica séria.

Termo para lembrar: Scope declaration | Base técnica: referência 1.

30. Como resume a escolha para um ledger?

Eu manteria a autorização e os lançamentos sob uma fronteira que preserve as regras financeiras, usando projeções para consultas tolerantes a atraso. A experiência recebe versão e estado, sem transferir autoridade financeira para o cache.

Termo para lembrar: Invariant-driven consistency | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Linearizabilidade

Ganho. Respeita ordem real

Custo e motivo. Exige coordenação

Como decidir. Quando leitura guia decisão crítica

02. Serializabilidade

Ganho. Protege transações

Custo e motivo. Pode abortar conflitos

Como decidir. Quando invariantes cruzam linhas

03. Eventualidade

Ganho. Reduz latência local

Custo e motivo. Permite atraso

Como decidir. Quando convergência basta

04. Read-your-writes

Ganho. Melhora continuidade

Custo e motivo. Exige token ou roteamento

Como decidir. Quando usuário acabou de alterar

05. Snapshot

Ganho. Oferece visão coerente

Custo e motivo. Pode ser antigo

Como decidir. Quando consulta cruza objetos

06. Causalidade

Ganho. Preserva dependências

Custo e motivo. Requer metadados

Como decidir. Quando ordem global é desnecessária

07. Bounded staleness

Ganho. Limita atraso

Custo e motivo. Pode bloquear leitura

Como decidir. Quando frescor possui limite

08. Last-write-wins

Ganho. Simplifica resolução

Custo e motivo. Pode perder intenção

Como decidir. Quando sobrescrita é aceitável

09. Versão por agregado

Ganho. Detecta regressão

Custo e motivo. Exige gestão de lacunas

Como decidir. Quando eventos são assíncronos

10. Leitura no primário

Ganho. Simplifica frescor

Custo e motivo. Concentra carga

Como decidir. Quando volume cabe na autoridade

Duas alternativas ao caminho proposto

1. Garantias de sessão

Quando faz sentido. Quando principal problema é continuidade da UX

Como implementar. Propagar versão mínima e manter leituras monotônicas

Limites. Não oferece atomicidade global entre clientes

2. Transação serializável

Quando faz sentido. Quando regra depende de múltiplos registros

Como implementar. Executar leitura e decisão sob isolamento adequado com retries

Limites. Escopo e latência dependem do banco e da topologia

Referências e alcance da evidência

1. Jepsen: Consistency Models

Definições e relações entre modelos de consistência, com atenção ao escopo das garantias.

https://jepsen.io/consistency/models

2. PostgreSQL: Transaction isolation

Isolamento, snapshots e necessidade de repetição de transações sob conflitos.

https://www.postgresql.org/docs/current/transaction-iso.html

3. Daniel Abadi: PACELC

Trade-off entre disponibilidade e consistência sob partição e entre latência e consistência na operação normal.

https://www.cs.umd.edu/~abadi/papers/abadi-pacelc.pdf

Continue em: T011 · T013 · T032

T013 / B. Sistemas distribuídos

ACID, MVCC e isolamento

ACID descreve atomicidade, preservação de regras, isolamento e durabilidade. MVCC mantém versões para leituras concorrentes; o nível de isolamento determina quais histórias concorrentes são permitidas.

Quando usar. Ao implementar saldo, reservas, decisões, idempotência e qualquer atualização concorrente de dados relacionados.

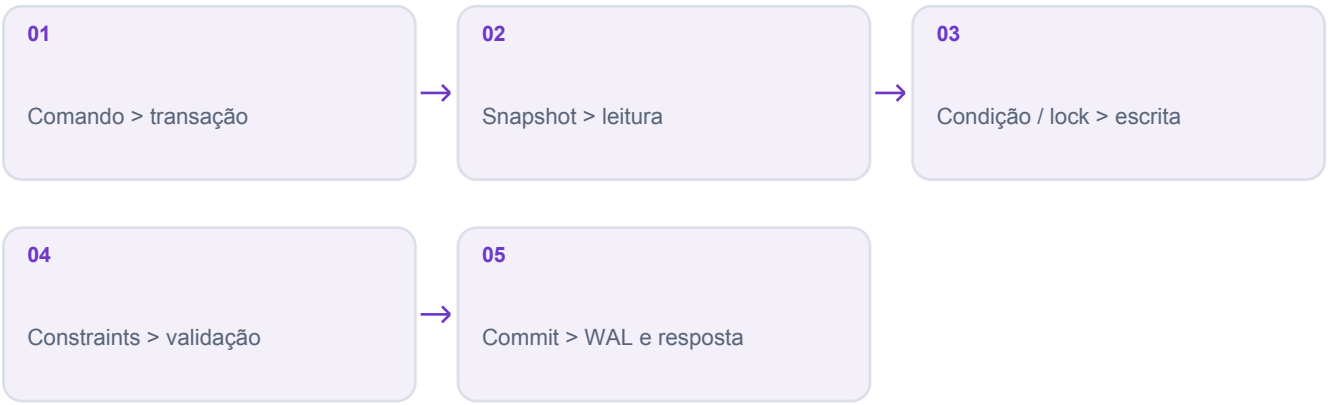
Quando evitar ou limitar. Evitar presumir que BEGIN e COMMIT eliminam todas as corridas ou manter transações abertas durante rede e trabalho humano.

Uma abertura possível

Vou identificar a invariante e escolher isolamento, constraints e ordem de locks suficientes. Depois explico os aborts e como repetir sem duplicar efeitos externos.

Peças e tecnologias. PostgreSQL · MVCC · SELECT FOR UPDATE · SERIALIZABLE · PgBouncer

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Escrever a invariante e duas operações concorrentes.

2. Escolher isolamento e constraints.

3. Manter transação curta e excluir rede.

4. Tratar conflitos e commit desconhecido.

5. Testar anomalias e observar locks e versões.

30 perguntas para sustentar o desenho

01. O que atomicidade garante?

Eu esperaria que as alterações dentro da transação confirmassem juntas ou fossem revertidas. A garantia não inclui automaticamente chamadas HTTP feitas no meio; efeitos externos precisam de outro protocolo de coordenação.

Termo para lembrar: Atomicity | Base técnica: referência 1.

02. O C de ACID é o C de CAP?

Eu separaria preservação de invariantes da transação e modelo de visibilidade distribuída. O banco ajuda com constraints, mas regras corretas ainda dependem da aplicação; ACID não significa que qualquer lógica financeira fica correta.

Termo para lembrar: Consistency distinction | Base técnica: referência 1.

03. O que MVCC resolve?

Eu usaria versões para permitir leituras consistentes sem bloquear toda escrita concorrente. Isso reduz certos conflitos, mas mantém custos de versões antigas, vacuum e controle de escrita; não elimina necessidade de isolamento adequado.

Termo para lembrar: Multiversion concurrency control | Base técnica: referência 1.

04. Read Committed mantém snapshot durante toda transação?

No PostgreSQL, eu consideraria um snapshot por comando nesse nível. Duas consultas na mesma transação podem observar commits diferentes; se a decisão exige visão estável ou proteção adicional, escolheria outro mecanismo.

Termo para lembrar: Statement snapshot | Base técnica: referência 1.

05. Repeatable Read impede todo erro concorrente?

Eu não assumiria isso. No PostgreSQL, a visão permanece estável, mas regras envolvendo alterações em linhas diferentes ainda podem sofrer write skew; preciso analisar a invariante ou usar serialização apropriada.

Termo para lembrar: Snapshot isolation | Base técnica: referência 1.

06. O que Serializable acrescenta?

Eu exigiria resultado equivalente a alguma execução serial, aceitando que transações possam abortar por conflito. A aplicação precisa repetir a unidade inteira corretamente; esse nível não transforma chamadas externas em parte da transação.

Termo para lembrar: Serializable isolation | Base técnica: referência 1.

07. Como faz retry de serialization failure?

Eu repetiria a transação inteira com novas leituras e limite de tentativas, usando backoff quando adequado. Valores calculados no snapshot abortado não podem ser reaproveitados cegamente; efeitos externos ficam fora desse bloco.

Termo para lembrar: Transaction retry | Base técnica: referência 1.

08. Como evita lost update?

Eu usaria atualização condicional ou operação atômica no banco, como decrementar sob condição de saldo suficiente. Ler, calcular e salvar sem versão permite sobrescrever concorrência, mesmo quando cada comando individual funciona corretamente.

Termo para lembrar: Atomic update | Base técnica: referência 1.

09. Quando usa SELECT FOR UPDATE?

Eu travaria as linhas que representam a regra quando a concorrência justificar. Manteria ordem consistente e transação curta; bloquear uma linha inexistente não protege automaticamente uma condição sobre registros que ainda podem aparecer.

Termo para lembrar: Pessimistic locking | Base técnica: referência 1.

10. Como lida com phantom?

Eu analisaria consultas por conjunto, como ausência de reserva existente. Constraint única, isolamento ou lock apropriado protege a regra; travar apenas linhas retornadas pode não impedir inserções que mudam o resultado depois.

Termo para lembrar: Predicate conflict | Base técnica: referência 1.

11. Como evita deadlock?

Eu padronizaria ordem de aquisição, como IDs de contas crescentes, e reduziria duração dos locks. Mesmo assim trataria aborts, porque dependências adicionais podem formar ciclos que o banco detecta e interrompe.

Termo para lembrar: Lock ordering | Base técnica: referência 1.

12. Constraint substitui validação da aplicação?

Eu usaria ambas com papéis distintos: aplicação explica o erro cedo; constraint protege a regra sob concorrência. Uma checagem prévia de unicidade não basta, pois outra transação pode inserir antes do nosso commit.

Termo para lembrar: Database constraint | Base técnica: referência 1.

13. Como implementa transferência?

Eu manteria débito, crédito e identidade da operação na mesma fronteira transacional quando possível. Validaria moeda e saldo, ordenaria locks e registraria lançamentos; notificação posterior sairia por obrigação durável separada.

Termo para lembrar: Atomic financial update | Base técnica: referência 1.

14. Como trata timeout no COMMIT?

Eu não concluiria que houve rollback. Consultaria a identidade estável da operação na autoridade correta; repetir com novo identificador pode executar novamente uma transação que já confirmou antes de perder a resposta.

Termo para lembrar: Unknown commit outcome | Base técnica: referência 1.

15. Durabilidade significa sobreviver a qualquer desastre?

Eu verificaria configuração de flush, replicação e falhas cobertas. Commit local durável não implica sobrevivência à perda de toda região; o contrato precisa definir armazenamento, réplicas e RPO considerado para aquela confirmação.

Termo para lembrar: Durability scope | Base técnica: referência 1.

16. Desligar fsync é otimização aceitável?

Eu não faria isso para dados bancários cuja confirmação exige persistência após falha. Reduzir sincronização muda a garantia; qualquer escolha desse tipo precisa de requisito que tolere perda e recuperação explicitamente validada.

Termo para lembrar: Write durability | Base técnica: referência 1.

17. Por que transação longa prejudica MVCC?

Eu observaria versões antigas que não podem ser removidas enquanto snapshots ainda precisam delas. Isso aumenta armazenamento e trabalho de manutenção; consultas e sessões esquecidas podem prejudicar o banco mesmo sem consumir CPU intensamente.

Termo para lembrar: Version retention | Base técnica: referência 1.

18. O que é idle in transaction?

Eu trataria sessão com transação aberta e sem trabalho como risco de locks e retenção de versões. Configuraria limites e encerramento seguro, corrigindo o código que espera rede ou usuário sem finalizar a transação.

Termo para lembrar: Transaction hygiene | Base técnica: referência 1.

19. Como usa savepoint?

Eu usaria para recuperar uma parte do trabalho dentro de uma transação quando a semântica permitir. Ele não confirma dados separadamente nem protege efeitos externos; também não elimina o custo de manter a transação aberta.

Termo para lembrar: Savepoint | Base técnica: referência 1.

20. Como escolhe entre lock e versão?

Eu preferiria versão quando colisões são raras e repetição é barata. Sob contenção alta, locks podem evitar trabalho desperdiçado; em ambos os casos preciso definir comportamento ao usuário e preservar a invariante.

Termo para lembrar: Optimistic versus pessimistic | Base técnica: referência 1.

21. Como trata unicidade com NULL?

Eu verificaria a semântica exata do banco e da constraint, porque valores nulos podem não ser considerados iguais. A chave de negócio precisa representar ausência corretamente para não permitir duplicatas inesperadas sob concorrência.

Termo para lembrar: Constraint semantics | Base técnica: referência 1.

22. Como lida com hot row?

Eu mediria tempo esperando lock e frequência por chave. Poderia particionar direitos ou serializar trabalho por entidade, mas não removeria coordenação necessária; otimizar uma linha quente exige preservar a regra que ela protege.

Termo para lembrar: Contention hotspot | Base técnica: referência 1.

23. Como testa write skew?

Eu executaria duas transações coordenadas para ler a mesma condição antes de alterar linhas distintas. Verificaria a regra final, mostrando se o isolamento escolhido impede a anomalia ou apenas produz resultados individualmente válidos.

Termo para lembrar: Concurrent counterexample | Base técnica: referência 1.

24. Uma transação só de leitura precisa isolamento?

Eu avaliaria se a consulta deve apresentar números coerentes do mesmo instante lógico. Relatórios que combinam saldos e movimentos podem exigir snapshot; leitura sem escrita ainda pode produzir uma visão enganosa do negócio.

Termo para lembrar: Read consistency | Base técnica: referência 1.

25. Como aplica outbox na transação?

Eu gravaria alteração e evento na mesma conexão e commit. O relay só observa registros confirmados; publicar antes do commit ou em outra transação reintroduz a possibilidade de evento sem fato ou fato sem evento.

Termo para lembrar: Atomic obligation | Base técnica: referência 1.

26. Como controla tempo de lock?

Eu definiria um limite coerente com o deadline da jornada, distinguindo espera por lock de tempo total da consulta. Timeout exige rollback e tratamento apropriado; simplesmente repetir imediatamente pode aumentar a contenção existente.

Termo para lembrar: Lock timeout | Base técnica: referência 1.

27. Como interage com pool de conexões?

Eu garantiria que a conexão permaneça vinculada durante a transação e seja devolvida limpa. Estado de sessão e pooling por transação exigem cuidado; uma configuração incompatível pode alterar comportamento sem mudança visível no código.

Termo para lembrar: Transaction scope | Base técnica: referência 1.

28. Como avalia custo do Serializable?

Eu mediria aborts, latência e throughput com carga representativa, em vez de assumir inviabilidade. O custo depende de padrões de acesso; mudanças no modelo ou índices podem reduzir conflitos sem enfraquecer garantias necessárias.

Termo para lembrar: Conflict rate | Base técnica: referência 1.

29. Como migra schema com tráfego?

Eu verificaria locks exigidos pela alteração e coexistência de versões da aplicação. Faria expansão compatível e backfill limitado; uma DDL curta no código pode bloquear uma tabela grande por tempo operacionalmente relevante.

Termo para lembrar: Online migration | Base técnica: referência 1.

30. Como resume a escolha de isolamento?

Eu apresentaria uma corrida concreta, o mecanismo que a impede e o tratamento de aborts. O objetivo não é escolher sempre o nível mais forte, mas demonstrar que nenhuma execução permitida viola a regra.

Termo para lembrar: Invariant-first isolation | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Read Committed

Ganho. Reduz coordenação

Custo e motivo. Permite leituras mudarem

Como decidir. Quando regra usa escrita condicional

02. Repeatable Read

Ganho. Mantém snapshot

Custo e motivo. Pode permitir write skew

Como decidir. Quando visão estável basta

03. Serializable

Ganho. Protege composição transacional

Custo e motivo. Pode abortar trabalho

Como decidir. Quando invariantes exigem

04. Lock pessimista

Ganho. Evita conflito tardio

Custo e motivo. Cria espera

Como decidir. Quando colisões são frequentes

05. Versão otimista

Ganho. Reduz bloqueio

Custo e motivo. Exige retry

Como decidir. Quando colisões são raras

06. Transação curta

Ganho. Libera recursos

Custo e motivo. Exige estados intermediários

Como decidir. Quando há trabalho externo

07. Constraint única

Ganho. Protege concorrência

Custo e motivo. Pode rejeitar no commit

Como decidir. Quando identidade é inequívoca

08. Snapshot longo

Ganho. Facilita relatório coerente

Custo e motivo. Retém versões

Como decidir. Quando duração é controlada

09. Flush síncrono

Ganho. Preserva durabilidade

Custo e motivo. Aumenta latência

Como decidir. Quando confirmação não pode perder

10. Particionamento de regra

Ganho. Reduz hot row

Custo e motivo. Complica modelo

Como decidir. Quando direitos podem ser divididos

Duas alternativas ao caminho proposto

1. CAS por versão

Quando faz sentido. Quando uma entidade concentra a regra

Como implementar. Atualizar somente se versão esperada ainda vale

Limites. Não protege automaticamente predicados entre entidades

2. Serialização por agregado

Quando faz sentido. Quando contenção é alta numa chave

Como implementar. Ordenar comandos por entidade e persistir atomicamente

Limites. Falha do executor ainda exige recuperação e identidade durável

Referências e alcance da evidência

1. PostgreSQL: Transaction isolation

Isolamento, snapshots e necessidade de repetição de transações sob conflitos.

https://www.postgresql.org/docs/current/transaction-iso.html

2. Jepsen: Consistency Models

Definições e relações entre modelos de consistência, com atenção ao escopo das garantias.

https://jepsen.io/consistency/models

Continue em: T019 · T024 · T091

T014 / B. Sistemas distribuídos

Consenso, Raft, Paxos e eleição de líder

Consenso permite a participantes acordar valores ou uma sequência apesar de certas falhas. Raft e Paxos organizam esse acordo; eleição de líder é parte do problema, não substituto de replicação segura.

Quando usar. Para metadados críticos, coordenação, locks com garantias adequadas e replicação de estados que exigem uma história consistente.

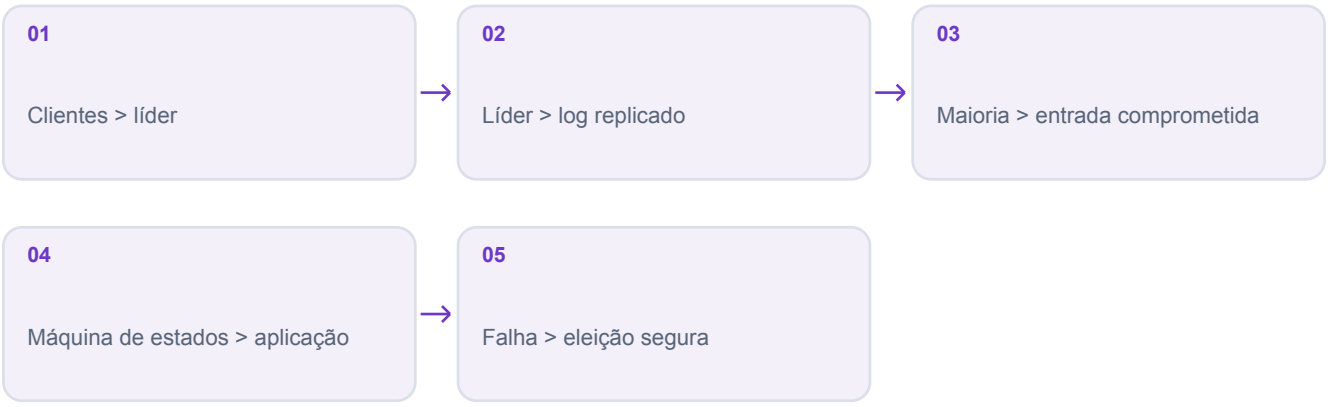
Quando evitar ou limitar. Evitar implementar consenso próprio sem necessidade, usar eleição isolada como prova de segurança ou confundir falha bizantina com modelo de crash.

Uma abertura possível

Vou usar uma implementação madura e explicar quórum, persistência e comportamento sem maioria. Ter um líder facilita coordenação, mas preciso impedir que líderes antigos confirmem decisões conflitantes.

Peças e tecnologias. Raft · Paxos · etcd · ZooKeeper · Consul

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Definir falhas toleradas e invariante.

2. Escolher implementação e número de votantes.

3. Explicar persistência, quórum e commit.

4. Definir leitura, eleição e fencing.

5. Testar partição, recuperação e reconfiguração.

30 perguntas para sustentar o desenho

01. O que consenso resolve?

Eu o usaria para acordar decisões entre réplicas sob um modelo de falhas definido. Ele não executa automaticamente regras de negócio; a máquina de estados ainda precisa validar comandos e aplicar resultados determinísticos.

Termo para lembrar: Consensus | Base técnica: referência 1.

02. Eleição de líder basta?

Eu não confundiria escolher um coordenador com manter uma história segura. Precisamos replicar decisões, preservar votos e impedir confirmações conflitantes; um líder antigo pode continuar vivo depois que outro foi eleito.

Termo para lembrar: Leader election | Base técnica: referência 1.

03. Por que usar três votantes?

Eu escolheria três para tolerar a indisponibilidade de um mantendo maioria, sob as hipóteses do protocolo. Dois votantes exigem ambos para maioria, oferecendo pouca disponibilidade quando um deles falha ou fica isolado.

Termo para lembrar: Majority quorum | Base técnica: referência 1.

04. Cinco votantes são sempre melhores?

Eu ganharia tolerância a duas perdas, mas aumentaria custo de replicação e operação. Distribuição de falhas e latência importam; mais nós não eliminam falta de maioria nem garantem desempenho melhor automaticamente.

Termo para lembrar: Quorum sizing | Base técnica: referência 1.

05. O que é um termo no Raft?

Eu trataria o termo como época lógica de liderança, aumentando durante eleições. Mensagens antigas podem ser identificadas; o armazenamento persistente dos metadados relevantes é necessário para manter segurança depois de reiniciar um participante.

Termo para lembrar: Raft term | Base técnica: referência 1.

06. Por que persistir o voto?

Eu explicaria que um nó não pode esquecer que já votou num termo e votar novamente após reinício. A persistência impede combinações de votos que poderiam permitir líderes concorrentes incompatíveis com a segurança do protocolo.

Termo para lembrar: Persistent vote | Base técnica: referência 1.

07. Como candidatos disputam eleição?

Eu usaria timeouts aleatorizados da implementação para reduzir eleições simultâneas repetidas. O valor precisa considerar rede e pausas; timeout agressivo pode gerar churn de liderança mesmo quando os servidores ainda funcionam.

Termo para lembrar: Randomized election timeout | Base técnica: referência 1.

08. O líder confirma ao gravar localmente?

Eu exigiria a condição de commit do protocolo, não apenas append local. No Raft, replicação em maioria e regras de termo determinam compromisso; dados locais não confirmados podem ser substituídos durante recuperação.

Termo para lembrar: Commit rule | Base técnica: referência 1.

09. O que acontece ao líder isolado?

Eu esperaria que ele deixasse de confirmar novas decisões sem quórum. Pode ainda receber clientes, mas não tem autoridade para prometer progresso seguro; roteamento e timeouts precisam tratar essa situação sem inventar sucesso.

Termo para lembrar: Minority leader | Base técnica: referência 1.

10. Como o novo líder sabe o histórico correto?

Eu confiaria nas regras de eleição e comparação de logs da implementação, que evitam eleger candidato sem entradas comprometidas necessárias. Copiar simplesmente o maior timestamp não oferece a mesma garantia de preservação da história.

Termo para lembrar: Election restriction | Base técnica: referência 1.

11. Entradas não comprometidas são definitivas?

Eu não as trataria como resultado confirmado ao cliente. Elas podem ser substituídas para alinhar o log ao líder legítimo; a aplicação precisa distinguir persistência local, compromisso e execução visível da máquina de estados.

Termo para lembrar: Uncommitted log | Base técnica: referência 1.

12. Commit significa já aplicado?

Eu separaria entrada comprometida de sua aplicação na máquina de estados. A API deve definir quando responde e como leituras aguardam a posição necessária; atraso de aplicação pode existir mesmo com replicação saudável.

Termo para lembrar: Commit versus apply | Base técnica: referência 1.

13. Como faz leitura linearizável?

Eu usaria o mecanismo oferecido pela implementação para confirmar liderança e posição aplicada, como ReadIndex. Ler memória de qualquer antigo líder não basta, porque ele pode estar isolado e apresentar estado desatualizado.

Termo para lembrar: ReadIndex | Base técnica: referência 1.

14. Lease de leitura exige o quê?

Eu verificaria hipóteses de tempo e implementação antes de usar leases para evitar comunicação por leitura. Relógios e pausas importam; uma otimização de latência não pode ampliar silenciosamente a garantia prometida pelo protocolo.

Termo para lembrar: Leader lease | Base técnica: referência 1.

15. Como reage a follower lento?

Eu permitiria progresso enquanto o quórum necessário permanece saudável, monitorando atraso e capacidade de recuperação. Um follower muito atrasado pode precisar de snapshot; promovê-lo sem as regras corretas arrisca perder história confirmada.

Termo para lembrar: Replication lag | Base técnica: referência 1.

16. Como funciona snapshot?

Eu compactaria histórico já representado por um estado consistente e sua posição no log. Snapshot reduz replay, mas precisa preservar metadados necessários; apagar log sem uma base recuperável compromete a retomada do participante.

Termo para lembrar: State snapshot | Base técnica: referência 1.

17. Como adiciona ou remove votantes?

Eu seguiria o protocolo de reconfiguração, como maiorias sobrepostas, evitando trocar conjuntos independentemente. Duas configurações sem interseção adequada podem confirmar decisões incompatíveis, especialmente durante falha ou implantação parcial da mudança.

Termo para lembrar: Membership change | Base técnica: referência 1.

18. Posso distribuir dois nós numa região e um noutra?

Eu avaliaria que perder a região com dois votantes elimina maioria, enquanto perder a outra não. A topologia determina disponibilidade por cenário; contar réplicas sem considerar domínios de falha produz falsa confiança.

Termo para lembrar: Failure-domain placement | Base técnica: referência 1.

19. Consenso impede requisição duplicada?

Eu manteria identidade de comando e resultado quando a aplicação exige idempotência. O log pode ordenar duas entradas iguais como comandos distintos; consenso não sabe sozinho que uma segunda tentativa representa a mesma intenção.

Termo para lembrar: Client deduplication | Base técnica: referência 1.

20. Como trata timeout do cliente?

Eu permitiria consulta ou repetição com identidade estável, porque a decisão pode ter sido comprometida antes da resposta se perder. Cancelar a chamada não remove uma entrada já confirmada no log replicado.

Termo para lembrar: Unknown outcome | Base técnica: referência 1.

21. Raft tolera nós maliciosos?

Eu não prometeria tolerância bizantina para um protocolo desenhado para falhas de crash e comunicação. Se participantes podem mentir arbitrariamente, precisamos de outro modelo e protocolo, com custos e premissas diferentes.

Termo para lembrar: Crash fault model | Base técnica: referência 1.

22. Qual diferença prática entre Paxos e Raft?

Eu explicaria que ambos fundamentam acordo seguro sob hipóteses apropriadas, mas organizam o raciocínio e implementação de maneiras diferentes. Escolheria pela implementação madura e requisitos, não por reimplementar um paper durante o projeto.

Termo para lembrar: Consensus formulation | Base técnica: referência 1.

23. Onde consenso fica caro?

Eu consideraria RTT do quórum, persistência e serialização da liderança. Operações globais frequentes podem limitar throughput; particionar a autoridade em grupos independentes ajuda quando as invariantes não exigem acordo entre todos.

Termo para lembrar: Coordination cost | Base técnica: referência 1.

24. Como particiona grupos de consenso?

Eu atribuiria entidades a grupos com logs e líderes próprios, distribuindo carga. Isso aumenta paralelismo, mas transações entre grupos exigem coordenação adicional; não basta dividir o cluster e manter a mesma promessa global.

Termo para lembrar: Consensus group | Base técnica: referência 1.

25. Como protege efeitos externos?

Eu faria o destino reconhecer uma época ou versão quando escritores antigos representam risco. O consenso elege corretamente, mas um processo pausado pode executar chamada externa atrasada; segurança precisa atravessar essa fronteira explicitamente.

Termo para lembrar: Fencing | Base técnica: referência 1.

26. Como testa o protocolo sem implementá-lo?

Eu validaria a configuração e integração com partições, reinícios e perda de respostas, verificando históricos e recuperação. Testar apenas disponibilidade de endpoints não demonstra que a aplicação usou corretamente as garantias do armazenamento.

Termo para lembrar: Failure testing | Base técnica: referência 1.

27. Como dimensiona timeouts de eleição?

Eu observaria latência entre votantes, pausas de runtime e fsync, mantendo margem adequada. Timeouts longos atrasam failover; curtos demais provocam eleições frequentes e reduzem a disponibilidade mesmo sem falha real.

Termo para lembrar: Election stability | Base técnica: referência 1.

28. O que monitora no cluster?

Eu acompanharia mudanças de líder, latência de commit, quorum disponível, atraso de aplicação e tamanho do log. Essas métricas ajudam distinguir rede, disco e configuração, evitando tratar toda lentidão como falta de CPU.

Termo para lembrar: Consensus observability | Base técnica: referência 1.

29. Como recupera cluster sem maioria?

Eu seguiria procedimento documentado, preservando a história mais confiável e impedindo retorno de membros antigos conflitantes. Forçar recuperação pode alterar garantias e perder dados; o runbook precisa explicitar RPO e validações.

Termo para lembrar: Disaster recovery | Base técnica: referência 1.

30. Como apresentaria consenso numa entrevista?

Eu explicaria que segurança impede decisões incompatíveis e progresso depende de condições como maioria comunicante. Mostraria uma partição concreta e o comportamento dos clientes, conectando o protocolo à invariante do sistema proposto.

Termo para lembrar: Safety and liveness | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Três votantes

Ganho. Menor coordenação

Custo e motivo. Tolera uma perda

Como decidir. Quando risco cabe nesse modelo

02. Cinco votantes

Ganho. Tolera duas perdas

Custo e motivo. Mais operação e tráfego

Como decidir. Quando falhas adicionais importam

03. Quórum regional

Ganho. Reduz RTT

Custo e motivo. Concentra risco geográfico

Como decidir. Quando desastre regional tem outro plano

04. Quórum entre regiões

Ganho. Preserva história distribuída

Custo e motivo. Aumenta latência

Como decidir. Quando RPO exige

05. Leitura linearizável

Ganho. Evita estado antigo

Custo e motivo. Exige confirmação adequada

Como decidir. Quando decisão é crítica

06. Leitura local

Ganho. Reduz custo

Custo e motivo. Pode atrasar

Como decidir. Quando contrato permite

07. Snapshot frequente

Ganho. Reduz recuperação

Custo e motivo. Consome I/O

Como decidir. Quando log cresce rápido

08. Timeout curto

Ganho. Acelera detecção

Custo e motivo. Aumenta falsas eleições

Como decidir. Quando rede é estável

09. Grupos particionados

Ganho. Aumentam paralelismo

Custo e motivo. Complicam transações cruzadas

Como decidir. Quando regras são locais

10. Implementação madura

Ganho. Reduz risco de protocolo

Custo e motivo. Limita customização

Como decidir. Quando consenso não é diferencial

Duas alternativas ao caminho proposto

1. Coordenação por banco transacional

Quando faz sentido. Quando a autoridade já está num banco confiável

Como implementar. Usar constraints, versões e transações sem serviço próprio de consenso

Limites. Depende das garantias e failover do banco

2. Coordenação limitada por direitos

Quando faz sentido. Quando recurso pode ser particionado previamente

Como implementar. Distribuir cotas que cada participante consome localmente

Limites. Não resolve acordo arbitrário nem redistribuição durante partição

Referências e alcance da evidência

1. Ongaro e Ousterhout: Raft

Eleição, replicação de log, safety e mudanças de configuração no protocolo Raft.

https://raft.github.io/raft.pdf

2. Gilbert e Lynch: prova de CAP

Modelo formal de consistência atômica, disponibilidade e partições; exemplos bancários são análises autorais.

https://www.comp.nus.edu.sg/~gilbert/pubs/BrewersConjecture-SigAct.pdf

3. Jepsen: Consistency Models

Definições e relações entre modelos de consistência, com atenção ao escopo das garantias.

https://jepsen.io/consistency/models

Continue em: T015 · T018 · T065

T015 / B. Sistemas distribuídos

Quóruns e suas garantias

Quórum define quantas réplicas devem participar de uma operação. Interseção é importante, mas garantias dependem também de protocolo, versão, configuração, concorrência e tratamento de falhas.

Quando usar. Ao discutir leitura e escrita replicadas, consenso, promoção e disponibilidade durante perda de nós.

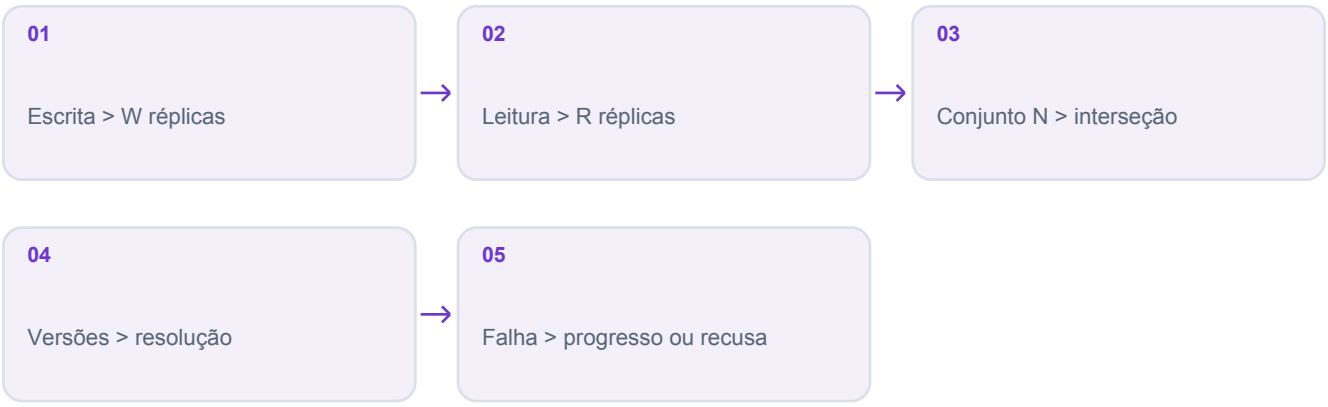
Quando evitar ou limitar. Evitar deduzir linearizabilidade apenas de R + W > N ou tratar confirmação em memória como durabilidade em disco.

Uma abertura possível

Vou contar réplicas, mas também explicar quais confirmam, o que confirmam e como comparamos versões. A matemática de interseção só vale sob as hipóteses corretas.

Peças e tecnologias. Cassandra · Raft · etcd · Dynamo-style replication · PostgreSQL synchronous replication

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Definir conjunto de réplicas e falhas.

2. Explicar W, R e persistência do ACK.

3. Verificar interseção e protocolo de versões.

4. Avaliar latência, hotspots e regiões.

5. Testar concorrência, timeout e reparo.

30 perguntas para sustentar o desenho

01. O que significa R + W > N?

Eu usaria a desigualdade para mostrar interseção entre conjuntos de leitura e escrita num conjunto fixo de réplicas. Ainda preciso de versões e protocolo correto; interseção isolada não prova todas as garantias de consistência.

Termo para lembrar: Quorum intersection | Base técnica: referência 1.

02. Por que maioria é comum?

Eu destacaria que duas maiorias do mesmo conjunto se intersectam, permitindo protocolos protegerem decisões. O nó comum precisa respeitar regras persistentes; contar votos sem lógica de épocas ou versão não impede conflitos.

Termo para lembrar: Majority quorum | Base técnica: referência 1.

03. N três, W dois e R dois significa o quê?

Eu aguardaria duas confirmações de escrita e consultaria duas réplicas para leitura, sob o protocolo escolhido. Isso pode tolerar uma indisponibilidade, mas latência, concorrência e resolução de versões continuam relevantes para a garantia.

Termo para lembrar: Read/write quorum | Base técnica: referência 1.

04. W um pode ser seguro?

Eu verificaria qual risco é aceitável e o que a única confirmação representa. Uma réplica pode falhar antes de propagar; para dados financeiros, não confundiria aceitação local com durabilidade suficiente para todo cenário.

Termo para lembrar: Write acknowledgement | Base técnica: referência 1.

05. R um sempre retorna dado velho?

Eu diria que pode estar atualizado, mas a garantia depende de qual réplica é escolhida e de sua posição. Latência baixa não comprova frescor; o contrato deve permitir ou impedir a leitura atrasada explicitamente.

Termo para lembrar: Single-replica read | Base técnica: referência 1.

06. W igual N é a melhor escolha?

Eu ganharia confirmação de todas as réplicas, mas qualquer uma indisponível pode bloquear escrita. Escolheria essa exigência apenas quando a durabilidade e topologia justificarem seu custo de latência e disponibilidade.

Termo para lembrar: All-replica write | Base técnica: referência 1.

07. Quórum equivale a consenso?

Eu explicaria que quórum é uma condição de participação usada por protocolos distintos. Consenso acrescenta regras para acordar decisões sob concorrência e falhas; um banco com leituras e escritas por quórum não é automaticamente linearizável.

Termo para lembrar: Quorum versus consensus | Base técnica: referência 1.

08. O que quebra a fórmula simples?

Eu verificaria mudanças no conjunto de réplicas, sloppy quorum, escritas concorrentes e versões ambíguas. A conta pressupõe conjuntos e regras compatíveis; réplicas substitutas fora do conjunto podem retirar a interseção esperada.

Termo para lembrar: Quorum assumptions | Base técnica: referência 1.

09. O que é sloppy quorum?

Eu o trataria como aceitar participantes temporários diferentes dos responsáveis originais para melhorar disponibilidade. Isso muda a interseção relevante; hinted handoff posterior ajuda propagação, mas não transforma automaticamente a operação numa garantia forte.

Termo para lembrar: Sloppy quorum | Base técnica: referência 1.

10. Hinted handoff basta para reparar tudo?

Eu usaria hints como mecanismo de entrega posterior, considerando retenção e falhas adicionais. Não dependeria deles como substituto de anti-entropy; interrupções longas e perda de hints exigem outros caminhos de reparo.

Termo para lembrar: Hinted handoff | Base técnica: referência 1.

11. Como resolve versões diferentes na leitura?

Eu seguiria a semântica do sistema, distinguindo relógio, vetor ou versão lógica. Escolher simplesmente o maior timestamp pode perder intenção concorrente; a política de resolução precisa ser adequada ao significado dos dados.

Termo para lembrar: Version reconciliation | Base técnica: referência 1.

12. O que é read repair?

Eu descreveria a correção de réplicas divergentes identificadas durante leitura, conforme implementação. Ela melhora convergência, mas não garante que todas as chaves nunca consultadas serão reparadas; manutenção independente continua necessária em muitos sistemas.

Termo para lembrar: Read repair | Base técnica: referência 1.

13. Como funciona anti-entropy?

Eu compararia conteúdo ou resumos entre réplicas para encontrar diferenças e transferir dados faltantes. O processo consome rede e disco; precisa ser agendado e monitorado para não competir descontroladamente com o tráfego online.

Termo para lembrar: Anti-entropy repair | Base técnica: referência 1.

14. Como trata timeout de escrita por quórum?

Eu não concluiria que nenhuma réplica aplicou a alteração. Algumas podem tê-la persistido; a aplicação consulta ou repete com identidade apropriada, evitando transformar uma confirmação incompleta em nova intenção de negócio.

Termo para lembrar: Ambiguous write | Base técnica: referência 1.

15. ACK significa fsync?

Eu leria o contrato da implementação e configuração para saber se a confirmação cobre memória, log ou disco remoto. O número de ACKs só tem significado de durabilidade quando sabemos exatamente o que cada confirmação garante.

Termo para lembrar: Durable acknowledgement | Base técnica: referência 1.

16. Como coloca réplicas entre zonas?

Eu distribuiria votantes para que a perda prevista preserve quórum, verificando falhas correlacionadas. Três réplicas no mesmo domínio físico não oferecem a tolerância que o número sugere durante uma falha compartilhada.

Termo para lembrar: Failure-domain quorum | Base técnica: referência 1.

17. Como coloca réplicas entre regiões?

Eu avaliaria quais regiões precisam responder para cada operação e o RTT resultante. Quórum local pode reduzir latência, mas não equivale a uma decisão global coordenada quando outra região também aceita escritas.

Termo para lembrar: Geographic quorum | Base técnica: referência 1.

18. LOCAL_QUORUM e QUORUM são iguais?

Eu verificaria o escopo exato definido pelo Cassandra para cada nível. Maioria local e maioria do conjunto total têm implicações diferentes em múltiplos datacenters; o nome da opção não substitui análise da topologia.

Termo para lembrar: Consistency level scope | Base técnica: referência 1.

19. Como trata mudança de N?

Eu coordenaria reconfiguração para preservar as propriedades necessárias durante a transição. Calcular quórum antigo em alguns nós e novo em outros sem protocolo pode permitir conjuntos disjuntos e decisões incompatíveis.

Termo para lembrar: Membership transition | Base técnica: referência 1.

20. Um nó lento conta como falho?

Eu posso continuar quando recebo as confirmações suficientes dos demais, mas monitoro o atrasado. Se ele for necessário ao quórum, determina latência; timeouts devem refletir o prazo da operação e o modelo de recuperação.

Termo para lembrar: Straggler tolerance | Base técnica: referência 1.

21. Como dimensiona reparo após outage?

Eu estimaria bytes divergentes e vazão disponível, reservando capacidade sem saturar produção. Um cluster pode voltar a responder antes de recuperar redundância; essa fase merece alerta porque tolera menos falhas adicionais.

Termo para lembrar: Repair capacity | Base técnica: referência 1.

22. Pode usar níveis diferentes por operação?

Eu escolheria nível conforme risco e garantia, documentando combinações válidas de leitura e escrita. Uma leitura fraca de dado escrito fortemente continua podendo estar atrasada; os dois lados do contrato precisam ser analisados juntos.

Termo para lembrar: Tunable consistency | Base técnica: referência 1.

23. Como trata exclusões?

Eu preservaria informação de remoção pelo tempo necessário para réplicas atrasadas não ressuscitarem dados. Política de tombstones, reparo e indisponibilidade máxima precisa ser coerente; apagar fisicamente cedo demais pode reintroduzir registros antigos.

Termo para lembrar: Tombstone retention | Base técnica: referência 1.

24. Como protege CAS distribuído?

Eu usaria o mecanismo específico do banco para condições concorrentes, verificando seu protocolo e escopo. Uma escrita normal por quórum não equivale automaticamente a compare-and-swap nem protege leitura seguida de atualização arbitrária.

Termo para lembrar: Conditional write protocol | Base técnica: referência 1.

25. Como mede a latência do quórum?

Eu observaria a resposta necessária para completar o conjunto, não a média de todos os nós. A distribuição e correlação de lentidão importam; um quórum geográfico pode ser limitado pelo caminho inter-regional.

Termo para lembrar: Order statistic latency | Base técnica: referência 1.

26. Quórum protege contra corrupção silenciosa?

Eu verificaria checksums, verificação e modelo de falha. Várias réplicas podem repetir dado corrompido ou bug lógico; tolerância a crash não significa tolerância automática a respostas arbitrariamente incorretas dos participantes.

Termo para lembrar: Failure model boundary | Base técnica: referência 1.

27. Como valida as garantias na entrevista?

Eu construiria escritas simultâneas, timeout parcial e leitura numa combinação diferente de réplicas. Se o protocolo não explica qual resultado é permitido, a fórmula de quórum ainda não sustenta a garantia prometida.

Termo para lembrar: Counterexample testing | Base técnica: referência 1.

28. Como escolhe N sob custo?

Eu equilibraria tolerância a falhas, latência, armazenamento e tráfego de reparo. Mais cópias custam operação e não substituem backups; a decisão depende dos cenários de perda e do prazo para restaurar redundância.

Termo para lembrar: Replication economics | Base técnica: referência 1.

29. Como explica indisponibilidade sem maioria?

Eu diria que o sistema preserva a propriedade escolhida recusando ou aguardando quando não reúne participantes suficientes. A interface mostra pendência ou indisponibilidade; inventar confirmação para manter um indicador verde violaria o contrato.

Termo para lembrar: Safety over progress | Base técnica: referência 1.

30. Como usa quórum num sistema bancário?

Eu escolheria o mecanismo que protege a autoridade financeira e deixaria leituras informativas usar caminhos menos fortes quando permitido. A configuração seria validada por histórico de falhas e documentação, sem considerar quórum um selo universal.

Termo para lembrar: Invariant-driven quorum | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. R maior

Ganho. Aumenta chance de observar versão

Custo e motivo. Eleva custo de leitura

Como decidir. Quando protocolo e frescor exigem

02. W maior

Ganho. Amplia confirmação replicada

Custo e motivo. Eleva custo de escrita

Como decidir. Quando durabilidade justifica

03. N maior

Ganho. Amplia tolerância potencial

Custo e motivo. Aumenta armazenamento e reparo

Como decidir. Quando domínios de falha são independentes

04. Quórum local

Ganho. Reduz RTT

Custo e motivo. Limita coordenação global

Como decidir. Quando escopo é regional

05. Quórum global

Ganho. Coordena regiões

Custo e motivo. Aumenta latência

Como decidir. Quando regra atravessa regiões

06. Sloppy quorum

Ganho. Mantém mais progresso

Custo e motivo. Enfraquece interseção simples

Como decidir. Quando conflito é aceitável

07. Read repair

Ganho. Corrige dados consultados

Custo e motivo. Adiciona trabalho à leitura

Como decidir. Quando implementação permite

08. Reparo periódico

Ganho. Cobre dados frios

Custo e motivo. Consome I/O

Como decidir. Quando réplicas podem divergir

09. Tombstone longa

Ganho. Evita ressurreição

Custo e motivo. Retém custo de leitura

Como decidir. Quando outage pode ser longo

10. Condição linearizável

Ganho. Protege CAS

Custo e motivo. Custa coordenação extra

Como decidir. Quando concorrência altera a regra

Duas alternativas ao caminho proposto

1. Leader-based replication

Quando faz sentido. Quando uma autoridade por chave é aceitável

Como implementar. Ordenar escritas num líder e replicar segundo protocolo seguro

Limites. Leitura local de follower ainda pode atrasar

2. Consistência eventual com reconciliação

Quando faz sentido. Quando conflitos são toleráveis por negócio

Como implementar. Aceitar operações e resolver versões com política explícita

Limites. Não serve para invariantes que não admitem violação temporária

Referências e alcance da evidência

1. Apache Cassandra: Dynamo architecture

Replicação, níveis de consistência e reparo no modelo Cassandra; não constitui promessa de linearizabilidade genérica.

https://cassandra.apache.org/doc/latest/cassandra/architecture/dynamo.html

2. Ongaro e Ousterhout: Raft

Eleição, replicação de log, safety e mudanças de configuração no protocolo Raft.

https://raft.github.io/raft.pdf

3. Jepsen: Consistency Models

Definições e relações entre modelos de consistência, com atenção ao escopo das garantias.

https://jepsen.io/consistency/models

Continue em: T014 · T016 · T026

T016 / B. Sistemas distribuídos

Replicação, lag e promoção

Replicação copia dados ou logs para outras instâncias. Lag mede atraso em receber ou aplicar mudanças; promoção transforma uma réplica em autoridade e exige prevenção de writers concorrentes e avaliação de perda.

Quando usar. Para disponibilidade, capacidade de leitura e recuperação, sempre separando esses objetivos de backup.

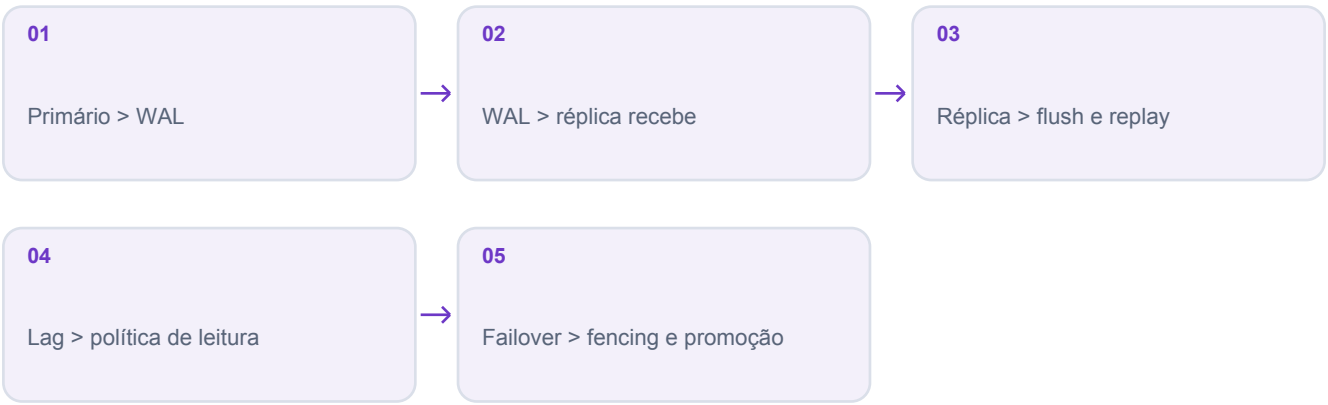
Quando evitar ou limitar. Evitar promover qualquer réplica automaticamente, confundir receive com replay ou tratar réplica como cópia histórica protegida de erros lógicos.

Uma abertura possível

Vou separar confirmação durável, aplicação da mudança e capacidade de servir leitura. Na promoção verificarei posição, impedirei o writer antigo e tratarei operações cujo resultado ficou desconhecido.

Peças e tecnologias. PostgreSQL streaming replication · RDS Multi-AZ · Patroni · WAL · replication slots

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Definir RPO, RTO e leitura permitida.

2. Escolher replicação síncrona ou assíncrona.

3. Monitorar posições e capacidade das réplicas.

4. Planejar fencing, promoção e roteamento.

5. Ensaiar retorno do antigo primário e reconciliação.

30 perguntas para sustentar o desenho

01. Replicação substitui backup?

Eu manteria ambos, porque exclusão indevida e corrupção lógica podem se propagar às réplicas. Backup com retenção permite voltar a um ponto anterior; réplica prioriza continuidade e não preserva automaticamente versões históricas recuperáveis.

Termo para lembrar: Replication versus backup | Base técnica: referência 1.

02. Qual diferença entre replicação física e lógica?

Eu explicaria que física reproduz mudanças de armazenamento do banco, enquanto lógica transmite alterações em estruturas selecionadas conforme mecanismo. Compatibilidade, DDL e conflitos diferem; escolheria pela necessidade de failover ou integração de dados.

Termo para lembrar: Replication granularity | Base técnica: referência 1.

03. Replicação assíncrona pode perder commit confirmado?

Eu reconheceria que a autoridade pode confirmar antes de outra região receber o log. Se ela for perdida, a réplica promovida pode não conter essas operações; o RPO precisa refletir essa janela real.

Termo para lembrar: Asynchronous RPO | Base técnica: referência 1.

04. Síncrona elimina qualquer perda?

Eu verificaria quais réplicas confirmam e se o ACK cobre flush ou aplicação. A garantia vale para os cenários cobertos; perder todas as cópias confirmadas ou restaurar backup antigo ainda exige recuperação e reconciliação.

Termo para lembrar: Synchronous guarantee | Base técnica: referência 1.

05. Receive lag e replay lag são iguais?

Eu separaria log recebido, persistido e aplicado. Uma réplica pode ter os bytes e ainda responder dados antigos; escolher leitura por atraso exige observar a posição relevante para a experiência prometida.

Termo para lembrar: Replication positions | Base técnica: referência 1.

06. Como faz read-your-writes com réplica?

Eu propagaria a posição mínima da escrita e aguardaria a réplica aplicá-la, ou leria no primário. Um tempo fixo de espera é apenas heurística; durante incidente, o lag pode ultrapassar essa suposição.

Termo para lembrar: Replay barrier | Base técnica: referência 1.

07. Como mede lag quando não há escritas?

Eu analisaria posições e atividade, não apenas um timestamp antigo. Uma réplica ociosa pode estar atualizada; métricas precisam distinguir ausência de novidades de incapacidade de receber ou aplicar mudanças recentes.

Termo para lembrar: Lag interpretation | Base técnica: referência 1.

08. Como escolhe candidato à promoção?

Eu verificaria integridade, posição e capacidade, seguindo o protocolo de failover. A réplica mais acessível não é necessariamente a mais atual; a decisão precisa preservar a história confirmada conforme a garantia contratada.

Termo para lembrar: Promotion eligibility | Base técnica: referência 1.

09. Como evita dois primários?

Eu impediria o antigo writer de aceitar ou produzir efeitos antes de liberar o novo, usando mecanismos adequados de infraestrutura ou protocolo. Alterar DNS não encerra conexões antigas nem impede um processo isolado de continuar.

Termo para lembrar: Fencing | Base técnica: referência 1.

10. O cliente precisa reconectar?

Eu esperaria conexões quebradas ou ligadas ao endpoint antigo, renovando o pool com backoff. Transações em andamento podem ter resultado desconhecido; a aplicação consulta identidades em vez de repetir tudo com novas operações.

Termo para lembrar: Connection recovery | Base técnica: referência 1.

11. Como trata commit durante failover?

Eu distinguiria rejeição confirmada de perda de resposta. Após estabilizar a autoridade, consultaria a operação por chave durável; reexecutar cegamente pode duplicar trabalho que já chegou ao novo primário ou ao parceiro externo.

Termo para lembrar: Ambiguous transaction | Base técnica: referência 1.

12. Como traz o antigo primário de volta?

Eu impediria sua escrita independente e o realinharia à história válida, usando ferramenta suportada ou reconstrução. Não juntaria logs divergentes como se fossem simples atraso; timelines diferentes exigem procedimento explícito de recuperação.

Termo para lembrar: Rejoin procedure | Base técnica: referência 1.

13. Réplica de leitura melhora escrita?

Eu reduziria carga de consultas no primário quando elas toleram atraso, liberando recursos indiretamente. Ela não aumenta automaticamente capacidade de atualização; replicação também consome rede, disco e trabalho de aplicação.

Termo para lembrar: Read offloading | Base técnica: referência 1.

14. Como consultas longas afetam standby?

Eu verificaria conflitos entre leitura e aplicação do log, conforme configuração. Preservar consulta pode atrasar replay ou reter recursos no primário; cancelar leitura ou aceitar lag é uma escolha operacional com impacto no usuário.

Termo para lembrar: Recovery conflict | Base técnica: referência 1.

15. O que replication slot protege?

Eu o usaria para preservar log necessário ao consumidor, monitorando atraso e armazenamento. Se o consumidor parar, a retenção pode crescer muito; limites e plano de reconstrução precisam existir antes de esgotar disco.

Termo para lembrar: Replication slot | Base técnica: referência 1.

16. Como evita réplica subdimensionada?

Eu dimensionaria também para replay e eventual promoção, não apenas consultas atuais. Um standby pequeno pode acumular lag e falhar justamente quando precisa assumir toda a carga durante o incidente do primário.

Termo para lembrar: Standby capacity | Base técnica: referência 1.

17. Como usa cascaded replication?

Eu consideraria replicar através de intermediários para reduzir carga direta no primário. Isso acrescenta dependência e possíveis atrasos; o desenho de failover precisa saber quais caminhos permanecem disponíveis quando um intermediário cai.

Termo para lembrar: Cascading replication | Base técnica: referência 1.

18. Como distribui réplicas em zonas?

Eu usaria domínios independentes para reduzir falhas correlacionadas e avaliaria o caminho de confirmação. Réplicas na mesma máquina ou zona não protegem contra todos os eventos que o nome alta disponibilidade sugere.

Termo para lembrar: Availability zone placement | Base técnica: referência 1.

19. Como distribui em regiões?

Eu compararia latência, RPO e capacidade de operação regional. Replicação distante aumenta custo e tempo de propagação; a aplicação precisa de roteamento, credenciais e serviços dependentes disponíveis na região de recuperação.

Termo para lembrar: Cross-region replication | Base técnica: referência 1.

20. Como trata indexação na réplica lógica?

Eu definiria quais estruturas e migrações são responsabilidade do destino, porque nem tudo é copiado automaticamente. Índices específicos ajudam leitura, mas exigem compatibilidade de schema e acompanhamento das mudanças no publicador.

Termo para lembrar: Logical subscriber schema | Base técnica: referência 1.

21. Como decide ler no primário?

Eu reservaria esse caminho para consultas que precisam de frescor ou após escrita sensível, dentro da capacidade. Redirecionar todas as leituras durante lag pode derrubar a autoridade; degradação precisa de limites explícitos.

Termo para lembrar: Consistency routing | Base técnica: referência 1.

22. Como faz teste de failover?

Eu interromperia o primário de forma controlada, medindo detecção, fencing, promoção e recuperação dos clientes. Validaria dados e operações externas, porque endpoint saudável não comprova que todas as confirmações foram preservadas.

Termo para lembrar: Failover drill | Base técnica: referência 1.

23. Como diferencia RTO de tempo de eleição?

Eu mediria até a jornada voltar corretamente, incluindo DNS, pools, caches, filas e aquecimento. Eleger um líder em segundos não implica que o sistema inteiro voltou a cumprir seu objetivo nesse mesmo tempo.

Termo para lembrar: End-to-end recovery | Base técnica: referência 1.

24. Como monitora recuperação após promoção?

Eu verificaria carga, erros, lag das novas réplicas e resultados desconhecidos, além de invariantes. O incidente não termina apenas porque há writer; a redundância precisa ser reconstruída e pendências reconciliadas com segurança.

Termo para lembrar: Post-failover validation | Base técnica: referência 1.

25. Replicação síncrona pode bloquear o primário?

Eu explicaria que esperar réplicas necessárias pode aumentar latência ou impedir commits quando elas ficam indisponíveis. A política de degradar para assíncrono, se existir, muda a garantia e precisa de decisão explícita.

Termo para lembrar: Commit dependency | Base técnica: referência 1.

26. Como faz manutenção de réplica?

Eu verificaria redundância e quórum antes de retirar participantes, atualizando um conjunto seguro por vez. Durante a manutenção, a tolerância a falhas diminui; capacidade e critérios de abortar a mudança precisam estar definidos.

Termo para lembrar: Rolling maintenance | Base técnica: referência 1.

27. Como trata lag de horas?

Eu estimaria se aplicar o backlog é mais viável que reconstruir a réplica, considerando log disponível e recursos. Recuperação deve ser limitada para não saturar o primário nem atrasar trabalho essencial do sistema.

Termo para lembrar: Catch-up versus rebuild | Base técnica: referência 1.

28. Como evita failover por falso positivo?

Eu combinaria sinais e protocolo seguro, ajustando timeouts ao ambiente. Uma rede lenta pode parecer falha; promover sem fencing aumenta risco, enquanto esperar demais aumenta indisponibilidade, exigindo um equilíbrio explicitamente testado.

Termo para lembrar: Failure detection | Base técnica: referência 1.

29. Como recupera dados ausentes após desastre?

Eu compararia a autoridade restaurada com fontes duráveis e efeitos externos, preservando IDs originais. Restaurar o banco não desfaz pagamentos já enviados; a retomada deve reconhecer resultados anteriores antes de executar pendências novamente.

Termo para lembrar: Reconciliation | Base técnica: referência 1.

30. Como defenderia sua política?

Eu explicaria quais falhas não perdem dados, quais podem perder dentro do RPO e quanto demora recuperar a jornada. Indicaria procedimento, owner e evidência de teste, evitando prometer failover transparente em qualquer circunstância.

Termo para lembrar: Recovery contract | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Replicação síncrona

Ganho. Reduz janela de perda

Custo e motivo. Aumenta latência

Como decidir. Quando RPO exige

02. Replicação assíncrona

Ganho. Reduz acoplamento

Custo e motivo. Pode perder commits no desastre

Como decidir. Quando perda limitada é aceita

03. Leitura em réplica

Ganho. Alivia primário

Custo e motivo. Pode estar atrasada

Como decidir. Quando frescor permite

04. Leitura no primário

Ganho. Simplifica atualidade

Custo e motivo. Concentra carga

Como decidir. Quando decisão é sensível

05. Réplica maior

Ganho. Facilita promoção

Custo e motivo. Custa ociosidade

Como decidir. Quando recuperação rápida importa

06. Slot de replicação

Ganho. Preserva log necessário

Custo e motivo. Pode encher disco

Como decidir. Quando monitorado e limitado

07. Consulta longa

Ganho. Entrega relatório coerente

Custo e motivo. Pode atrasar replay

Como decidir. Quando carga é isolada

08. Failover rápido

Ganho. Reduz indisponibilidade

Custo e motivo. Aumenta risco de falso positivo

Como decidir. Quando fencing é confiável

09. Região distante

Ganho. Protege desastre regional

Custo e motivo. Aumenta RTT e egress

Como decidir. Quando risco geográfico justifica

10. Rebuild

Ganho. Simplifica realinhamento

Custo e motivo. Consome rede e tempo

Como decidir. Quando backlog é inviável

Duas alternativas ao caminho proposto

1. Banco distribuído com consenso

Quando faz sentido. Quando autoridade replicada é requisito central

Como implementar. Usar protocolo integrado de replicação e eleição

Limites. Custo geográfico e limites transacionais continuam relevantes

2. Backup e restauração

Quando faz sentido. Quando interrupção maior é aceitável

Como implementar. Manter cópias verificadas e recuperar sob runbook

Limites. RTO é maior e efeitos externos exigem reconciliação

Referências e alcance da evidência

1. PostgreSQL: Log-Shipping Standby Servers

Streaming replication, standby e replicação síncrona; cenários de failover precisam de configuração específica.

https://www.postgresql.org/docs/current/warm-standby.html

2. PostgreSQL: Replication settings

Configuração de replicação, retenção e participantes síncronos.

https://www.postgresql.org/docs/current/runtime-config-replication.html

3. Ongaro e Ousterhout: Raft

Eleição, replicação de log, safety e mudanças de configuração no protocolo Raft.

https://raft.github.io/raft.pdf

Continue em: T014 · T020 · T067

T017 / B. Sistemas distribuídos

Sharding e rebalanceamento

Sharding distribui subconjuntos de dados entre unidades independentes. A chave determina localidade, distribuição e custo das operações; rebalanceamento move propriedade e dados sem perder ou duplicar decisões.

Quando usar. Quando limites medidos de armazenamento, throughput ou isolamento não são resolvidos por índices, escala vertical e réplicas.

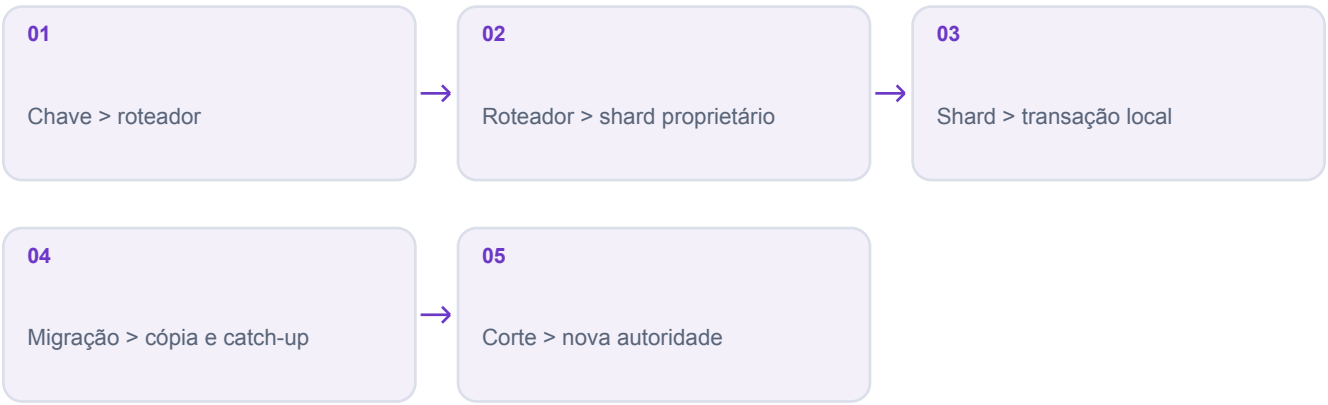
Quando evitar ou limitar. Evitar sharding precoce, chave com concentração previsível ou migração com dois writers sem protocolo.

Uma abertura possível

Vou escolher a chave pelas consultas e invariantes, não apenas pela uniformidade. Depois desenho como uma entidade muda de shard mantendo autoridade e operações em andamento.

Peças e tecnologias. PostgreSQL · Citus · DynamoDB · Cassandra · consistent hashing · shard directory

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Medir limite que exige particionamento.

2. Comparar hash, faixa e diretório.

3. Definir chave, transações e consultas cruzadas.

4. Planejar migração com versão e fencing.

5. Testar skew, falha e restauração por shard.

30 perguntas para sustentar o desenho

01. Quando faria sharding?

Eu exigiria evidência de limite de capacidade ou isolamento que opções mais simples não atendem. Sharding distribui carga, mas adiciona roteamento, migração e coordenação; uma tabela grande sozinha não prova necessidade imediata.

Termo para lembrar: Horizontal partitioning | Base técnica: referência 1.

02. Qual diferença para particionar tabela?

Eu distinguiria partições administradas dentro do mesmo banco de shards com recursos e autoridades independentes. Particionar ajuda poda e manutenção, mas pode manter o mesmo limite de CPU, disco e writer central.

Termo para lembrar: Logical versus physical partitioning | Base técnica: referência 1.

03. Como escolhe a shard key?

Eu combinaria distribuição de carga, consultas frequentes e invariantes que precisam ser locais. Uma chave uniforme pode ser ruim se toda operação exige consultar muitos shards; localidade e skew precisam ser avaliados juntos.

Termo para lembrar: Shard key | Base técnica: referência 1.

04. Hash ou faixa?

Eu usaria hash para espalhar chaves e faixa para consultas contíguas e movimentação dirigida. Hash dificulta intervalos globais; faixa pode concentrar escritas recentes, exigindo divisões e acompanhamento de hotspots ao longo do tempo.

Termo para lembrar: Hash versus range | Base técnica: referência 1.

05. Quando usa diretório de shards?

Eu escolheria um mapa explícito quando preciso mover tenants ou aplicar regras de localização. Isso oferece flexibilidade, mas o diretório vira dependência de disponibilidade, cache e versionamento; entradas antigas precisam ser tratadas com segurança.

Termo para lembrar: Directory routing | Base técnica: referência 1.

06. Consistent hashing resolve toda redistribuição?

Eu o usaria para reduzir a fração de chaves remapeadas quando participantes mudam. Ainda preciso mover dados, tratar operações em trânsito e proteger autoridade; a função de hash não executa uma migração segura sozinha.

Termo para lembrar: Consistent hashing | Base técnica: referência 1.

07. O que são virtual nodes?

Eu dividiria o espaço em mais unidades lógicas que servidores, facilitando distribuição e movimentação. Isso pode suavizar desequilíbrio, mas aumenta metadados e não resolve uma única chave cujo tráfego supera a capacidade local.

Termo para lembrar: Virtual nodes | Base técnica: referência 1.

08. Como detecta shard quente?

Eu compararia CPU, I/O, filas e latência por shard e chave, evitando olhar só média do cluster. Um tenant popular pode saturar uma unidade enquanto outras ficam ociosas, limitando a experiência global.

Termo para lembrar: Hot shard | Base técnica: referência 1.

09. Como resolve uma hot key?

Eu avaliaria cache, agregação ou divisão de direitos quando a semântica permitir. Espalhar escritas arbitrariamente pode quebrar ordenação; uma entidade realmente sequencial tem limite que não desaparece ao aumentar o número de shards.

Termo para lembrar: Hot key mitigation | Base técnica: referência 1.

10. Como consulta todos os shards?

Eu limitaria fan-out e prazo, agregando resultados com tratamento de falhas parciais. Consultas globais frequentes podem justificar projeção analítica; varrer todos os shards em cada tela compromete latência e disponibilidade.

Termo para lembrar: Scatter-gather | Base técnica: referência 1.

11. Como pagina resultado global?

Eu definiria ordenação total e cursors que representem progresso necessário nos shards. Concorrência de inserções complica estabilidade; para exportações coerentes, consideraria snapshot ou materialização em vez de prometer navegação perfeita sem custo.

Termo para lembrar: Distributed pagination | Base técnica: referência 1.

12. Como faz transferência entre shards?

Eu escolheria transação distribuída suportada ou processo com reservas e estados explícitos, conforme a invariante. Duas atualizações locais independentes não constituem transferência atômica; falhas intermediárias exigem protocolo e recuperação definidos.

Termo para lembrar: Cross-shard transaction | Base técnica: referência 1.

13. Como mantém unicidade global?

Eu escolheria identificadores com propriedades adequadas ou uma autoridade de reserva quando a unicidade de negócio exigir. Constraints locais só protegem seu shard; email ou chave financeira global precisa de outra estratégia consistente.

Termo para lembrar: Global uniqueness | Base técnica: referência 1.

14. Como começa uma migração?

Eu registraria origem, destino e época da migração, mantendo um writer autoritativo. Copiaria snapshot e acompanharia mudanças posteriores; o plano deve preservar operações novas enquanto os dados históricos são transferidos.

Termo para lembrar: Shard migration | Base técnica: referência 1.

15. Como corta o tráfego para o destino?

Eu verificaria catch-up, impediria novas escritas na origem conforme protocolo e atualizaria a versão do roteamento. Clientes com mapa antigo recebem redirecionamento ou rejeição segura, evitando dois writers confirmando estados incompatíveis.

Termo para lembrar: Ownership cutover | Base técnica: referência 1.

16. Pode escrever nos dois durante migração?

Eu só aceitaria com protocolo que defina autoridade, deduplicação e recuperação de falhas parciais. Duas chamadas independentes divergem; preferiria captura de mudanças ou log durável para sincronizar o destino antes do corte.

Termo para lembrar: Dual-write risk | Base técnica: referência 1.

17. Como desfaz uma migração?

Eu distinguiria voltar o roteamento de recuperar dados já escritos no destino. Após o corte, a origem pode estar atrasada; rollback exige sincronização inversa ou nova migração, não apenas restaurar uma configuração antiga.

Termo para lembrar: Migration rollback | Base técnica: referência 1.

18. Como faz backup por shard?

Eu definiria se basta recuperar cada shard isoladamente ou se preciso de ponto consistente global. Backups feitos em horários diferentes podem representar processos cruzados incompletos; reconciliação e metadados de posição entram no plano.

Termo para lembrar: Shard backup | Base técnica: referência 1.

19. Como trata falha de um shard?

Eu isolaria tenants afetados e manteria os demais atendendo, desde que dependências globais não propaguem a falha. O contrato de operações cruzadas deve explicar quando um shard indisponível impede a conclusão inteira.

Termo para lembrar: Blast radius | Base técnica: referência 1.

20. Como escolhe número inicial de shards?

Eu consideraria capacidade, crescimento e custo operacional, usando unidades lógicas quando possível. Shards demais criam overhead e manutenção; poucos podem exigir migração cedo, por isso planejo expansão antes de atingir o limite.

Termo para lembrar: Shard count | Base técnica: referência 1.

21. Como lida com tenant enorme?

Eu poderia dedicar recursos ou subdividir seu domínio com uma chave adicional compatível com as consultas. Separar por tenant simplifica isolamento, mas um único cliente pode ultrapassar o maior shard disponível.

Termo para lembrar: Tenant isolation | Base técnica: referência 1.

22. Como trata dados de referência?

Eu replicaria dados pequenos e estáveis quando reduzir joins remotos, com versão e atualização controlada. Se a referência governa decisão financeira, preciso garantir a política aplicada; cópia local não deve alterar regras silenciosamente.

Termo para lembrar: Reference data | Base técnica: referência 1.

23. Como faz joins?

Eu colocaria dados frequentemente relacionados no mesmo shard quando a invariante permitir. Para cruzamentos, compararia composição e projeções; joins distribuídos podem transferir muitos bytes e tornar a latência dependente dos participantes mais lentos.

Termo para lembrar: Join locality | Base técnica: referência 1.

24. Como mede custo de rebalanceamento?

Eu estimaria bytes, índices, tráfego de mudanças e impacto de I/O, limitando a cópia. Migração deve caber junto do tráfego online; mover dados sem controle pode provocar incidente no cluster que pretendia melhorar.

Termo para lembrar: Rebalance budget | Base técnica: referência 1.

25. Como trata schema diferente entre shards?

Eu usaria migrações compatíveis e acompanhamento por unidade, permitindo coexistência durante rollout. O roteador e a aplicação precisam funcionar com versões intermediárias; atualizar todos simultaneamente amplia risco e dificulta recuperação.

Termo para lembrar: Schema rollout | Base técnica: referência 1.

26. Como evita roteamento obsoleto?

Eu associaria a propriedade a uma versão e faria o shard validar solicitações incompatíveis. Cache do diretório melhora latência, mas só é seguro se a autoridade consegue rejeitar comandos enviados à localização antiga.

Termo para lembrar: Routing epoch | Base técnica: referência 1.

27. Como audita localização dos dados?

Eu registraria política de país, tenant e classe de dado no mecanismo de alocação. Rebalanceamento automático precisa respeitar essas restrições; distribuição uniforme não pode mover informação para uma região proibida pelo requisito validado.

Termo para lembrar: Placement governance | Base técnica: referência 1.

28. Como testa sharding?

Eu incluiria skew, mudança de mapa, falha parcial e operações cruzadas, verificando invariantes e completude. Um benchmark uniforme no caminho feliz não revela hot keys nem a perda de dados durante migração.

Termo para lembrar: Partitioned workload testing | Base técnica: referência 1.

29. Como sabe que sharding piorou o sistema?

Eu observaria mais fan-out, transações distribuídas e incidentes de roteamento que ganho de capacidade. Isso pode indicar chave inadequada ou adoção precoce; a solução deve ser revista, não defendida apenas pelo investimento realizado.

Termo para lembrar: Architectural feedback | Base técnica: referência 1.

30. Como resumiria a escolha?

Eu mostraria a chave, a consulta mais comum e a invariante que permanece local. Depois explicaria uma migração e uma falha, demonstrando que a arquitetura escala sem esconder a mudança de autoridade dos dados.

Termo para lembrar: Locality-first partitioning | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Hash

Ganho. Distribui chaves

Custo e motivo. Prejudica consultas por faixa

Como decidir. Quando acesso é por chave

02. Faixa

Ganho. Facilita intervalos

Custo e motivo. Pode concentrar escrita

Como decidir. Quando ordenação é essencial

03. Diretório

Ganho. Permite alocação flexível

Custo e motivo. Cria dependência de metadados

Como decidir. Quando tenants migram seletivamente

04. Mais shards

Ganho. Amplia paralelismo

Custo e motivo. Aumenta operação

Como decidir. Quando recursos locais saturam

05. Tenant dedicado

Ganho. Isola carga

Custo e motivo. Reduz eficiência

Como decidir. Quando cliente é dominante

06. Dados colocados juntos

Ganho. Simplificam transação

Custo e motivo. Limitam distribuição

Como decidir. Quando invariante é conjunta

07. Projeção global

Ganho. Facilita busca

Custo e motivo. Introduz atraso

Como decidir. Quando consulta cruza shards

08. Migração gradual

Ganho. Reduz impacto

Custo e motivo. Prolonga coexistência

Como decidir. Quando tráfego não pode parar

09. Pausa de escrita

Ganho. Simplifica corte

Custo e motivo. Causa indisponibilidade

Como decidir. Quando janela é aceitável

10. Virtual nodes

Ganho. Facilitam rebalanceamento

Custo e motivo. Aumentam metadados

Como decidir. Quando cluster muda frequentemente

Duas alternativas ao caminho proposto

1. Escala vertical e índices

Quando faz sentido. Quando um nó ainda atende o horizonte

Como implementar. Otimizar consultas e aumentar recursos antes de distribuir

Limites. Possui limite físico e pode custar caro

2. Células por grupos de clientes

Quando faz sentido. Quando isolamento operacional domina

Como implementar. Replicar stack e atribuir tenants a células

Limites. Exige roteamento e operação de múltiplas unidades

Referências e alcance da evidência

1. Microsoft: Sharding pattern

Estratégias de particionamento, distribuição e dificuldades de consultas e transações entre shards.

https://learn.microsoft.com/en-us/azure/architecture/patterns/sharding

2. Apache Cassandra: Dynamo architecture

Replicação, níveis de consistência e reparo no modelo Cassandra; não constitui promessa de linearizabilidade genérica.

https://cassandra.apache.org/doc/latest/cassandra/architecture/dynamo.html

3. Microsoft: Architecture styles

Características e critérios de estilos arquiteturais; decisões particulares são análises autorais.

https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/

Continue em: T003 · T020 · T036

T018 / B. Sistemas distribuídos

Relógios distribuídos e causalidade

Relógios físicos aproximam tempo civil; relógios monotônicos medem duração local; relógios lógicos representam ordem e causalidade. Nenhum timestamp comum prova sozinho a ordem real de eventos distribuídos.

Quando usar. Ao definir expiração, leases, ordenação, replay, calendário e resolução de conflitos entre máquinas.

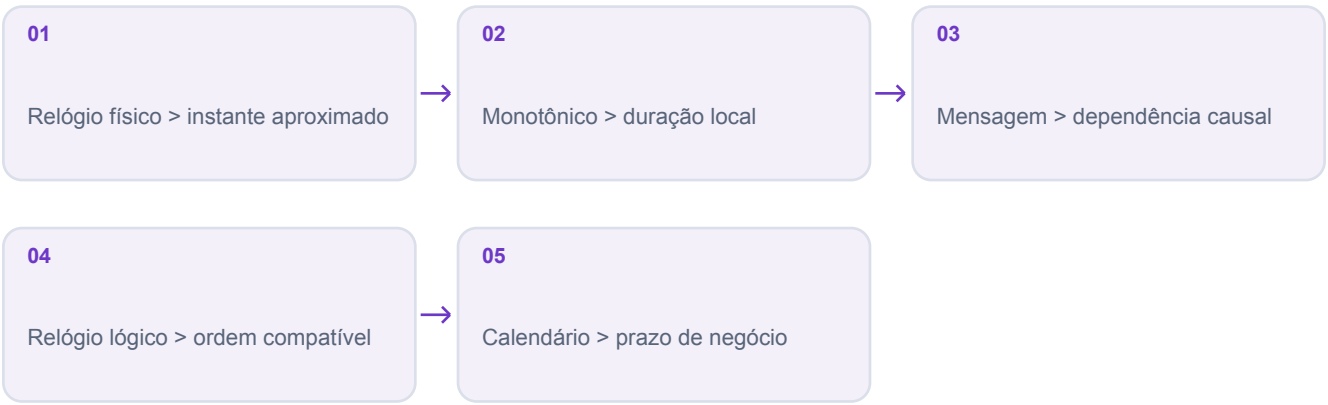
Quando evitar ou limitar. Evitar usar horário de parede para medir duração ou supor que timestamps únicos implicam causalidade ou ordem de commit.

Uma abertura possível

Vou separar horário de negócio, duração e ordem causal. Cada um exige um mecanismo diferente, especialmente quando relógios mudam, processos pausam ou mensagens chegam atrasadas.

Peças e tecnologias. NTP · monotonic clock · Lamport clocks · vector clocks · HLC · IANA TZDB

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Classificar instante, duração e causalidade.

2. Escolher relógio adequado para cada uso.

3. Propagar versões ou dependências entre processos.

4. Definir tolerância a skew e calendário.

5. Testar saltos, pausas e mensagens atrasadas.

30 perguntas para sustentar o desenho

01. Timestamp prova qual evento aconteceu primeiro?

Eu não assumiria isso entre máquinas, porque relógios podem divergir e ajustar. Para decisões concorrentes, usaria versões ou protocolo; o horário continua útil para observabilidade, desde que sua precisão e limites sejam conhecidos.

Termo para lembrar: Clock skew | Base técnica: referência 1.

02. Como mede timeout?

Eu usaria relógio monotônico para duração local, evitando saltos do horário civil. O valor não é necessariamente comparável entre máquinas ou reinícios; persistir prazo de negócio exige outra representação e política de retomada.

Termo para lembrar: Monotonic clock | Base técnica: referência 1.

03. O que é happened-before?

Eu definiria dependência pela ordem local e pelo envio e recebimento de mensagens, incluindo transitividade. Eventos sem relação conhecida podem ser concorrentes; impor uma ordem numérica não prova que um causou o outro.

Termo para lembrar: Happened-before | Base técnica: referência 1.

04. Como funciona relógio de Lamport?

Eu incrementaria o contador local e, ao receber mensagem, avançaria além do maior valor observado. Isso preserva precedência causal conhecida, mas dois valores ordenados não demonstram causalidade; ainda podem representar eventos concorrentes.

Termo para lembrar: Lamport clock | Base técnica: referência 1.

05. Como desempata timestamps lógicos iguais?

Eu poderia combinar contador com identificador do processo para obter uma ordem determinística. Esse desempate é uma convenção de processamento, não descoberta da ordem física real nem garantia de correção financeira automática.

Termo para lembrar: Total-order tie-break | Base técnica: referência 1.

06. O que relógio vetorial acrescenta?

Eu manteria progresso por participante para identificar certas relações causais e concorrência. O custo de metadados cresce com participantes; preciso definir compactação e identidade dos nós, especialmente quando clientes entram e saem frequentemente.

Termo para lembrar: Vector clock | Base técnica: referência 1.

07. Quando usaria HLC?

Eu consideraria combinar proximidade do tempo físico com ordenação lógica quando o sistema suporta esse modelo. Ainda verificaria hipóteses de sincronização e concorrência; HLC não torna qualquer escrita distribuída automaticamente linearizável.

Termo para lembrar: Hybrid logical clock | Base técnica: referência 1.

08. NTP elimina o problema?

Eu usaria sincronização para reduzir erro, mas manteria tolerância a desvios, falhas e ajustes. Aplicações não devem depender de relógios perfeitamente iguais; monitorar offset e políticas de correção faz parte da operação.

Termo para lembrar: Clock synchronization | Base técnica: referência 1.

09. Como define vencimento de uma reserva?

Eu escolheria uma autoridade para avaliar o prazo e registraria o instante conforme a política. Relógio do cliente serve à apresentação, não à decisão financeira; skew não pode permitir consumo depois do limite autorizado.

Termo para lembrar: Expiry authority | Base técnica: referência 1.

10. Como modela próximo dia útil?

Eu usaria calendário versionado, timezone e cutoff explícitos, convertendo o vencimento para um instante. Somar vinte e quatro horas não representa feriados nem horário de verão; a regra precisa ser reproduzível historicamente.

Termo para lembrar: Business calendar | Base técnica: referência 1.

11. UTC resolve tudo?

Eu armazenaria instantes em UTC, mas preservaria timezone e regra local quando o negócio depende deles. UTC não explica sozinho qual calendário determinou um prazo nem como tratar uma hora local ambígua.

Termo para lembrar: Temporal semantics | Base técnica: referência 1.

12. Como trata horário de verão?

Eu usaria biblioteca com base IANA e política para horários inexistentes ou repetidos. Não fixaria deslocamento manualmente; atualizações de regras podem mudar agendamentos futuros e precisam de revisão conforme o contrato do produto.

Termo para lembrar: Timezone rules | Base técnica: referência 1.

13. Como ordena transações?

Eu preferiria sequência atribuída pela autoridade do agregado ou posição do log, conforme a garantia necessária. IDs e timestamps criados antes do commit podem aparecer em ordem diferente da confirmação efetiva das transações.

Termo para lembrar: Logical sequence | Base técnica: referência 1.

14. Como usa timestamp em logs?

Eu registraria tempo, identidade e correlação, mas interpretaria diferenças pequenas com cautela. Traces e dependências ajudam reconstruir fluxo; ordenar linhas de máquinas diferentes apenas pelo horário pode produzir uma narrativa falsa do incidente.

Termo para lembrar: Observability ordering | Base técnica: referência 1.

15. Como timeout afeta uma lease?

Eu definiria quem mede validade e como renovação e expiração são verificadas. Pausa do processo pode ultrapassar a lease; por isso o recurso protegido precisa rejeitar writers antigos quando o risco exige fencing.

Termo para lembrar: Lease assumptions | Base técnica: referência 1.

16. O processo voltou depois de GC longo. E agora?

Eu não confiaria que ainda possui liderança ou lease. Revalidaria autoridade e usaria versão nas gravações; uma pausa local não impede outro participante de assumir o trabalho enquanto este processo estava congelado.

Termo para lembrar: Process pause | Base técnica: referência 1.

17. Como resolve last-write-wins?

Eu explicaria que escolher maior timestamp é uma política de conflito, não uma verdade de negócio. Relógio adiantado pode dominar atualizações legítimas; para dinheiro ou aprovação, preferiria coordenação ou rejeição de conflito.

Termo para lembrar: Timestamp conflict policy | Base técnica: referência 1.

18. Como trata mensagens fora de ordem?

Eu incluiria sequência ou dependência necessária e detectaria lacunas. Quando crédito depende de abertura de conta, o consumidor aguarda ou recupera contexto; ordenar por horário recebido não restaura automaticamente a causalidade original.

Termo para lembrar: Causal delivery | Base técnica: referência 1.

19. Como diferencia event time e processing time?

Eu separaria quando o fato ocorreu de quando o sistema o processou, usando ambos para diagnóstico. Processamento atrasado pode pertencer a uma janela passada; a política precisa dizer como corrigir resultados já publicados.

Termo para lembrar: Event time | Base técnica: referência 1.

20. Watermark significa que nada atrasado chegará?

Eu a trataria como estimativa ou limite operacional conforme a fonte, não certeza universal. Eventos posteriores ao watermark exigem política de atraso, atualização ou quarentena, principalmente quando relatórios financeiros precisam de reconciliação.

Termo para lembrar: Watermark assumption | Base técnica: referência 1.

21. Como agenda tarefas após reinício?

Eu persistiria prazo e identidade do job, recalculando elegibilidade na retomada. Um timer em memória desaparece; o scheduler precisa detectar vencidos e evitar que duas instâncias executem o mesmo efeito sem proteção.

Termo para lembrar: Durable scheduling | Base técnica: referência 1.

22. Como evita calendário duplicar execução?

Eu identificaria a ocorrência por regra, timezone e período de negócio, não só pelo instante de disparo. Isso ajuda tratar horário repetido e retries; a mesma ocorrência mantém identidade mesmo que seja reexecutada depois.

Termo para lembrar: Schedule identity | Base técnica: referência 1.

23. Como mede latência entre serviços?

Eu usaria durações locais e tracing com limites de sincronização, evitando subtrair timestamps arbitrários de hosts diferentes como medida perfeita. Para SLO, escolheria uma fronteira de medição que minimize esse erro operacional.

Termo para lembrar: Distributed timing | Base técnica: referência 1.

24. Como testa código dependente de tempo?

Eu injetaria relógio e calendário nos testes, cobrindo expiração, salto de horário e mudanças de regra. Também faria testes de integração com atraso real, porque controlar tempo lógico não reproduz todas as pausas do runtime.

Termo para lembrar: Clock injection | Base técnica: referência 1.

25. Como compara versões de dois objetos?

Eu verificaria se pertencem ao mesmo contador ou domínio de ordem. Versão dez de uma conta não é necessariamente posterior à versão nove de outra; comparar números sem escopo cria inferências causais inválidas.

Termo para lembrar: Causal scope | Base técnica: referência 1.

26. Pode usar UUID ordenável como relógio?

Eu o usaria para identidade e alguma localidade quando suas propriedades ajudarem, mas não como prova de commit ou causalidade. Geração concorrente e skew continuam possíveis; o contrato do algoritmo deve ser respeitado.

Termo para lembrar: Identifier semantics | Base técnica: referência 1.

27. Como trata certificados e tokens com skew?

Eu configuraria tolerância limitada conforme o protocolo e monitoraria relógios, sem aceitar validade indefinidamente. A margem melhora interoperabilidade, mas amplia janela de aceitação; operações sensíveis ainda precisam de autorização e revogação apropriadas.

Termo para lembrar: Clock tolerance | Base técnica: referência 1.

28. Como reconcilia dados de parceiros com horário diferente?

Eu preservaria o valor original, timezone declarado e instante normalizado, validando inconsistências. Não corrigiria silenciosamente registros ambíguos; identificadores e período de liquidação podem ser mais confiáveis que comparar timestamps aproximados.

Termo para lembrar: Temporal normalization | Base técnica: referência 1.

29. Como explica causalidade ao entrevistador?

Eu mostraria uma mensagem de crédito que depende da conta já criada. A regra exige observar a abertura primeiro; relógios físicos parecidos não garantem isso, enquanto uma dependência ou sequência explícita permite verificar a condição.

Termo para lembrar: Causal counterexample | Base técnica: referência 1.

30. Como fecha o desenho temporal?

Eu anotaria qual relógio mede duração, quem decide vencimento e qual versão ordena alterações. Também definiria comportamento sob atraso e skew; essa separação evita que um único timestamp assuma garantias incompatíveis no sistema.

Termo para lembrar: Time contract | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Relógio físico

Ganho. Permite horário humano

Custo e motivo. Tem skew

Como decidir. Quando representa instante aproximado

02. Monotônico

Ganho. Mede duração estável

Custo e motivo. É local

Como decidir. Quando controla timeout

03. Lamport

Ganho. Metadado pequeno

Custo e motivo. Não detecta toda concorrência

Como decidir. Quando ordem causal parcial basta

04. Vetor

Ganho. Identifica concorrência

Custo e motivo. Cresce com participantes

Como decidir. Quando merge depende disso

05. HLC

Ganho. Combina tempo e lógica

Custo e motivo. Exige hipóteses claras

Como decidir. Quando sistema oferece suporte

06. Timezone preservado

Ganho. Reproduz regra local

Custo e motivo. Complica modelo

Como decidir. Quando prazo depende de calendário

07. Tolerância de skew

Ganho. Evita rejeições indevidas

Custo e motivo. Amplia janela aceita

Como decidir. Quando protocolo permite

08. Sequência central

Ganho. Ordena agregado

Custo e motivo. Cria serialização

Como decidir. Quando ordem é obrigatória

09. Watermark conservador

Ganho. Reduz correções tardias

Custo e motivo. Aumenta espera

Como decidir. Quando atraso é frequente

10. Timer persistido

Ganho. Sobrevive a reinício

Custo e motivo. Exige scheduler

Como decidir. Quando execução tem obrigação durável

Duas alternativas ao caminho proposto

1. Versão por agregado

Quando faz sentido. Quando o problema é concorrência de uma entidade

Como implementar. Atribuir sequência transacional na autoridade

Limites. Não representa tempo físico nem ordem global

2. Coordenação por consenso

Quando faz sentido. Quando participantes precisam acordar ordem

Como implementar. Usar log replicado com protocolo maduro

Limites. Aumenta latência e pode bloquear sem maioria

Referências e alcance da evidência

1. Leslie Lamport: Time, Clocks and the Ordering of Events

Relação happened-before e ordenação lógica; não fornece por si só sincronização física perfeita.

https://www.microsoft.com/en-us/research/publication/time-clocks-ordering-events-distributed-system/

2. Ongaro e Ousterhout: Raft

Eleição, replicação de log, safety e mudanças de configuração no protocolo Raft.

https://raft.github.io/raft.pdf

3. Jepsen: Consistency Models

Definições e relações entre modelos de consistência, com atenção ao escopo das garantias.

https://jepsen.io/consistency/models

Continue em: T014 · T050 · T065

T019 / B. Sistemas distribuídos

Concorrência, locks, CAS e deadlocks

Controle de concorrência protege regras quando operações se sobrepõem. Locks coordenam acesso; CAS condiciona atualização a uma versão; deadlock é um ciclo de espera que impede progresso sem intervenção.

Quando usar. Em reservas, saldos, atribuição de casos, contadores e mudanças concorrentes de estado.

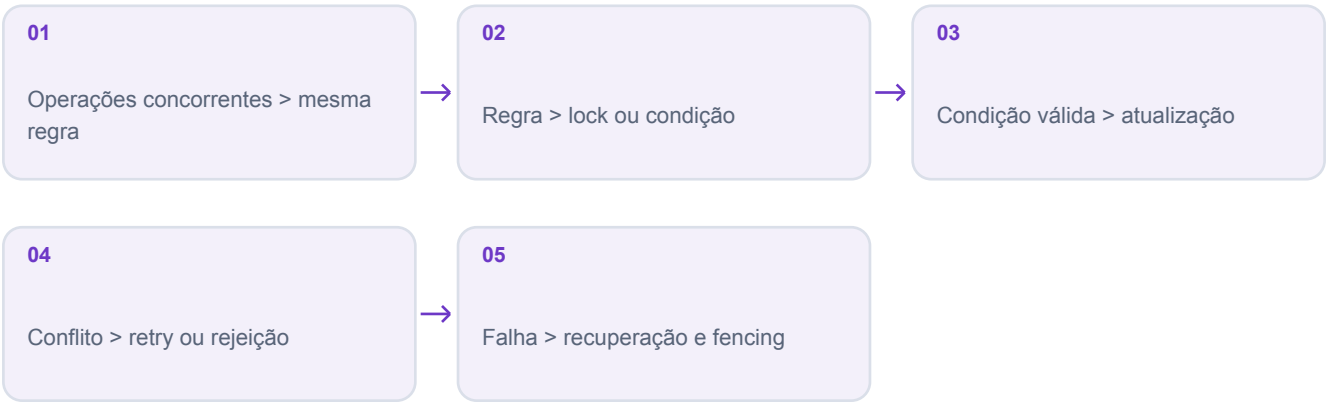
Quando evitar ou limitar. Evitar usar mutex local para vários processos, segurar lock durante rede ou supor que expiração impede o antigo proprietário de agir.

Uma abertura possível

Vou mostrar duas operações concorrentes e escolher o menor mecanismo que preserve a regra. Depois explico conflito, espera, retry e o retorno de um processo que ficou pausado.

Peças e tecnologias. PostgreSQL row locks · CAS · Redis SET NX · etcd leases · optimistic version

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Construir corrida que viola a regra.

2. Definir escopo do recurso protegido.

3. Escolher lock, CAS ou serialização.

4. Limitar espera e tratar aborts.

5. Testar pausas, deadlocks e efeitos externos.

30 perguntas para sustentar o desenho

01. Quando usa lock pessimista?

Eu usaria quando colisões são frequentes e refazer trabalho custa caro, mantendo a seção crítica curta. O lock deve proteger a regra correta; bloquear uma linha irrelevante apenas adiciona espera sem impedir a corrida.

Termo para lembrar: Pessimistic concurrency | Base técnica: referência 1.

02. Quando prefere CAS?

Eu atualizaria somente se a versão ainda fosse a esperada, retornando conflito caso contrário. Isso evita locks prolongados quando colisões são raras, mas a aplicação precisa reler e decidir se faz sentido repetir.

Termo para lembrar: Compare-and-swap | Base técnica: referência 1.

03. Mutex local protege várias instâncias?

Eu explicaria que ele coordena apenas participantes dentro daquele processo ou memória compartilhada. Com múltiplas instâncias, a autoridade precisa estar no armazenamento ou protocolo distribuído, caso contrário cada processo entra sozinho na mesma regra.

Termo para lembrar: Process-local lock | Base técnica: referência 1.

04. Como evita lost update?

Eu incorporaria a condição ou incremento na atualização atômica, ou verificaria versão. Ler saldo e depois salvar o valor calculado permite sobrescrever uma alteração concorrente, mesmo quando cada requisição parece correta isoladamente.

Termo para lembrar: Lost update | Base técnica: referência 1.

05. Como implementa reserva de estoque?

Eu executaria decremento condicionado a disponibilidade suficiente e verificaria quantas linhas foram alteradas. A condição pertence à mesma operação atômica; conferir estoque antes e atualizar sem condição deixa duas compras consumirem a última unidade.

Termo para lembrar: Conditional decrement | Base técnica: referência 1.

06. Como ordena locks de duas contas?

Eu definiria uma ordem estável, como identificadores crescentes, antes de adquirir ambos. Isso reduz ciclos de espera, mas ainda trataria aborts porque índices, triggers e outras rotinas podem introduzir dependências adicionais.

Termo para lembrar: Global lock order | Base técnica: referência 1.

07. O que acontece num deadlock?

Eu esperaria que o banco detectasse o ciclo e abortasse uma transação, conforme implementação. A aplicação repete a unidade inteira quando seguro, registrando frequência; retries não substituem corrigir uma ordem de locks sistematicamente ruim.

Termo para lembrar: Deadlock detection | Base técnica: referência 1.

08. Como limita espera?

Eu definiria timeout menor que o orçamento restante da jornada, distinguindo lock de execução. Ao expirar, faço rollback e retorno estado apropriado; repetir imediatamente sem limite pode intensificar a contenção que causou o problema.

Termo para lembrar: Lock timeout | Base técnica: referência 1.

09. SKIP LOCKED garante justiça?

Eu o usaria para workers evitarem linhas já ocupadas, aceitando uma visão parcial. Não garante FIFO nem ausência de starvation; monitoraria idade das tarefas e regras de prioridade para itens continuamente ultrapassados.

Termo para lembrar: Queue claiming | Base técnica: referência 1.

10. O que é ABA em CAS?

Eu explicaria que um valor pode mudar de A para B e voltar a A, escondendo alterações intermediárias. Uma versão monotônica ou identidade de geração ajuda detectar a diferença quando comparar apenas o valor é insuficiente.

Termo para lembrar: ABA problem | Base técnica: referência 1.

11. Como protege uma linha inexistente?

Eu usaria constraint única ou isolamento apropriado para a condição de ausência. SELECT FOR UPDATE sem resultado não trava magicamente futuras inserções; duas transações podem concluir que a mesma chave ainda não existe.

Termo para lembrar: Predicate protection | Base técnica: referência 1.

12. Lock distribuído com TTL basta?

Eu não trataria TTL como prova de exclusão após pausa. O proprietário antigo pode voltar depois de outro assumir; o recurso protegido deve reconhecer uma versão ou token que rejeite operações obsoletas quando necessário.

Termo para lembrar: Lease limitation | Base técnica: referência 1.

13. Como funciona fencing token?

Eu associaria cada nova propriedade a um token crescente e faria o destino rejeitar tokens menores que o já aceito. Sem verificação no recurso final, o token é apenas metadado e não impede escrita atrasada.

Termo para lembrar: Fencing token | Base técnica: referência 1.

14. Como libera lock com segurança?

Eu verificaria que ainda sou o proprietário antes de apagar ou liberar. Uma operação atômica compara o token; remover cegamente uma chave pode liberar o lock de outra tentativa que assumiu após expiração.

Termo para lembrar: Ownership verification | Base técnica: referência 1.

15. Advisory lock protege qualquer escrita?

Eu lembraria que ele depende de participantes seguirem o protocolo. Uma rotina que ignora o advisory lock pode alterar os mesmos dados; constraints e permissões continuam importantes para garantir regras sob caminhos alternativos.

Termo para lembrar: Advisory locking | Base técnica: referência 1.

16. Como trata starvation?

Eu mediria tempo de espera por tarefa e chave, usando aging ou filas adequadas quando a política exigir. Throughput alto não comprova justiça; um conjunto pequeno de operações pode ficar indefinidamente sem acesso ao recurso.

Termo para lembrar: Fairness | Base técnica: referência 1.

17. Como trata livelock?

Eu identificaria participantes que repetem e cedem continuamente sem concluir trabalho. Backoff com jitter, limite de tentativas e coordenação apropriada reduzem sincronização; também revisaria a estratégia para evitar conflito recorrente por desenho.

Termo para lembrar: Livelock | Base técnica: referência 1.

18. Como decide granularidade do lock?

Eu bloquearia a menor unidade que preserve a invariante. Locks amplos simplificam raciocínio, mas reduzem paralelismo; granularidade excessiva aumenta quantidade, complexidade e risco de não cobrir uma regra que cruza objetos.

Termo para lembrar: Lock granularity | Base técnica: referência 1.

19. Como evita lock durante API externa?

Eu persistiria intenção e estado antes da chamada, liberando a transação. A resposta é aplicada com condição de versão; manter lock enquanto espera rede prolonga contenção e não torna a operação remota atomicamente reversível.

Termo para lembrar: Critical section duration | Base técnica: referência 1.

20. Como trata conflito de edição humana?

Eu enviaria a versão lida junto da alteração, apresentando conflito quando outra pessoa mudou o caso. Reaplicar automaticamente pode apagar intenção; a interface oferece comparação ou atualização, preservando o histórico e a autorização atual.

Termo para lembrar: Optimistic editing | Base técnica: referência 1.

21. Como protege contador global?

Eu avaliaria atualização atômica local, particionamento ou direitos previamente distribuídos conforme a regra. Somar contadores eventualmente consistentes não protege um limite rígido instantâneo; a técnica precisa manter a garantia que o número representa.

Termo para lembrar: Hot counter | Base técnica: referência 1.

22. Como mede contenção?

Eu acompanharia tempo de espera, filas, transações bloqueadoras e chaves frequentes, relacionando a latência da jornada. CPU baixa com espera alta indica gargalo de coordenação; adicionar máquinas pode não resolver uma entidade serializada.

Termo para lembrar: Lock observability | Base técnica: referência 1.

23. Como usa retry com CAS?

Eu releria o estado e recalcularia a decisão, com limite e backoff. Repetir o mesmo UPDATE com versão antiga não avança; alguns conflitos exigem resposta ao usuário em vez de tentativa automática ilimitada.

Termo para lembrar: Optimistic retry | Base técnica: referência 1.

24. Como serializa por chave?

Eu encaminharia comandos da entidade para um executor ordenado, persistindo estado e progresso. Isso simplifica concorrência local, mas falha e reatribuição do executor ainda exigem identidade, recuperação e proteção contra proprietário antigo.

Termo para lembrar: Actor model | Base técnica: referência 1.

25. Lock de leitura sempre melhora?

Eu avaliaria proporção e duração de leituras, pois gerenciamento e preferência podem gerar starvation de escritores. MVCC pode ser alternativa; a escolha depende do runtime e do padrão de acesso, não de uma regra universal.

Termo para lembrar: Read-write lock | Base técnica: referência 1.

26. Como testa corridas?

Eu coordenaria threads ou processos com barreiras para forçar interleavings perigosos, verificando a invariante final. Testes repetidos aleatórios ajudam, mas um contraexemplo construído revela melhor se o mecanismo realmente cobre a condição crítica.

Termo para lembrar: Concurrency testing | Base técnica: referência 1.

27. Como garante visibilidade em memória?

Eu usaria primitivas corretas do runtime para sincronizar acesso, evitando variáveis compartilhadas sem garantia. Atomicidade de uma escrita isolada não implica ordenação ou visibilidade de todo o estado relacionado entre threads.

Termo para lembrar: Memory ordering | Base técnica: referência 1.

28. Como trata reentrância?

Eu verificaria se o mesmo fluxo pode readquirir o recurso e se isso é desejado. Reentrância pode evitar autobloqueio, mas também esconder dependências de chamada; prefiro seções críticas e ownership claramente definidos.

Termo para lembrar: Reentrant lock | Base técnica: referência 1.

29. Como recupera operação parcialmente executada?

Eu distinguiria estado local de efeito externo, usando identidade estável e consulta de resultado. O lock não prova que nada aconteceu antes da queda; recuperação precisa preservar intenção e evitar repetir uma consequência já confirmada.

Termo para lembrar: Idempotent recovery | Base técnica: referência 1.

30. Como escolhe mecanismo na entrevista?

Eu mostraria a execução que causa erro e como lock, condição ou serialização a impede. Depois explicaria o custo de espera e a recuperação, evitando chamar qualquer operação atômica de solução completa para concorrência.

Termo para lembrar: Invariant-driven concurrency | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Lock pessimista

Ganho. Evita trabalho conflitante

Custo e motivo. Cria espera

Como decidir. Quando conflito é frequente

02. CAS

Ganho. Mantém seção curta

Custo e motivo. Exige retry

Como decidir. Quando conflito é raro

03. Lock amplo

Ganho. Simplifica regra

Custo e motivo. Reduz paralelismo

Como decidir. Quando invariante cruza objetos

04. Lock fino

Ganho. Aumenta concorrência

Custo e motivo. Complica coordenação

Como decidir. Quando regras são locais

05. Lease curta

Ganho. Recupera rápido

Custo e motivo. Aumenta sobreposição

Como decidir. Quando fencing existe

06. Lease longa

Ganho. Reduz expiração falsa

Custo e motivo. Atrasa recuperação

Como decidir. Quando tarefas são longas

07. SKIP LOCKED

Ganho. Distribui jobs

Custo e motivo. Pode gerar starvation

Como decidir. Quando ordem estrita não é necessária

08. Actor por chave

Ganho. Simplifica sequência

Custo e motivo. Cria hot actor

Como decidir. Quando entidade é serial

09. Backoff

Ganho. Reduz colisões

Custo e motivo. Aumenta latência

Como decidir. Quando retries competem

10. Fencing

Ganho. Rejeita escritor antigo

Custo e motivo. Exige suporte do destino

Como decidir. Quando pausa pode causar efeito inválido

Duas alternativas ao caminho proposto

1. Constraint e atualização condicional

Quando faz sentido. Quando regra cabe numa operação do banco

Como implementar. Expressar condição e alteração atomicamente

Limites. Não cobre efeitos remotos nem invariantes fora do escopo

2. Fila por agregado

Quando faz sentido. Quando ordenação explícita simplifica alta contenção

Como implementar. Processar comandos sequencialmente com progresso durável

Limites. Hot keys e failover continuam exigindo desenho

Referências e alcance da evidência

1. PostgreSQL: Explicit Locking

Locks, conflitos, deadlocks e advisory locks no PostgreSQL; lock distribuído exige garantias adicionais.

https://www.postgresql.org/docs/current/explicit-locking.html

2. PostgreSQL: Transaction isolation

Isolamento, snapshots e necessidade de repetição de transações sob conflitos.

https://www.postgresql.org/docs/current/transaction-iso.html

3. Ongaro e Ousterhout: Raft

Eleição, replicação de log, safety e mudanças de configuração no protocolo Raft.

https://raft.github.io/raft.pdf

Continue em: T013 · T065 · T106

T020 / B. Sistemas distribuídos

Múltiplas regiões e split-brain

Arquiteturas multi-região distribuem entrada, dados e execução entre localidades. Split-brain ocorre quando autoridades incompatíveis avançam simultaneamente; evitar isso requer protocolo, não apenas roteamento.

Quando usar. Quando latência geográfica, continuidade ou requisitos de localização justificam custo e complexidade adicionais.

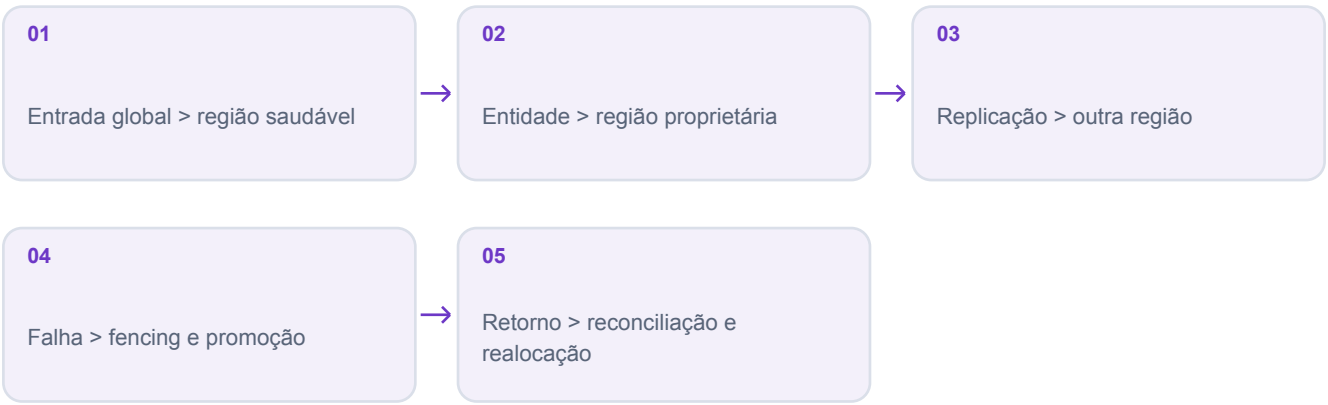
Quando evitar ou limitar. Evitar active-active como objetivo abstrato, prometer RPO zero com replicação assíncrona ou usar DNS como fencing.

Uma abertura possível

Vou definir o que continua funcionando quando uma região some e quem pode escrever em cada conta. Separarei redundância de entrada, autoridade financeira e recuperação de dependências.

Peças e tecnologias. Route 53 · Azure Front Door · PostgreSQL replication · DynamoDB global tables · cell architecture

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Definir falha regional e objetivos de recuperação.

2. Mapear autoridades por entidade e dependências.

3. Escolher replicação e consistência por operação.

4. Planejar roteamento, fencing e capacidade de reserva.

5. Ensaiar desastre, retorno e reconciliação.

30 perguntas para sustentar o desenho

01. Por que usar mais de uma região?

Eu buscaria reduzir impacto de desastre regional ou aproximar clientes, quantificando o benefício. A segunda região também precisa de dados, segredos, quotas e operação; duplicar servidores sem dependências não entrega continuidade real.

Termo para lembrar: Regional resilience | Base técnica: referência 1.

02. Active-active significa dois writers por conta?

Eu distinguiria entrada ativa em várias regiões de escrita simultânea no mesmo dado. Posso servir clientes globalmente e manter autoridade única por conta, preservando regras sem prometer resolução automática de conflitos financeiros.

Termo para lembrar: Active-active scope | Base técnica: referência 1.

03. Como escolhe região proprietária?

Eu atribuiria entidade a uma autoridade com roteamento versionado, considerando cliente, localização e capacidade. Transferir propriedade exige sincronização e fencing; mudar um campo de configuração não basta para impedir escritor antigo.

Termo para lembrar: Home region | Base técnica: referência 1.

04. Qual diferença de active-passive?

Eu manteria uma região principal e outra preparada para assumir, com grau de aquecimento definido. Isso simplifica autoridade, mas recuperação depende de lag, capacidade disponível e testes das dependências no destino.

Termo para lembrar: Active-passive | Base técnica: referência 1.

05. Como define RPO zero?

Eu especificaria quais falhas estão cobertas e quais confirmações remotas são exigidas antes do ACK. Replicação assíncrona não sustenta essa promessa para perda da origem; custo de latência e disponibilidade precisa ser aceito.

Termo para lembrar: Zero-data-loss scope | Base técnica: referência 1.

06. Como calcula RTO real?

Eu incluiria detecção, decisão, fencing, promoção, roteamento, reconexão e aquecimento, até a jornada cumprir o objetivo. Recuperar apenas o banco não encerra o desastre se filas, credenciais ou parceiros continuam indisponíveis.

Termo para lembrar: End-to-end RTO | Base técnica: referência 1.

07. DNS impede escrita na região antiga?

Eu usaria DNS para descoberta, mas assumiria caches e conexões persistentes. A região antiga precisa perder autoridade por mecanismo verificável; caso contrário pode continuar produzindo efeitos mesmo depois do tráfego novo mudar de destino.

Termo para lembrar: Routing limitation | Base técnica: referência 1.

08. Como faz fencing regional?

Eu escolheria mecanismo no armazenamento ou recurso final que invalide a época antiga, complementado por isolamento de infraestrutura quando necessário. O importante é provar que o writer anterior não confirma novas decisões conflitantes.

Termo para lembrar: Writer fencing | Base técnica: referência 1.

09. Como detecta desastre sem falso positivo?

Eu combinaria sinais independentes e um processo seguro de decisão. Uma partição pode parecer perda total; failover precipitado sem fencing cria risco maior que esperar alguns segundos para confirmar o cenário operacional.

Termo para lembrar: Failure detection policy | Base técnica: referência 1.

10. Como dimensiona capacidade secundária?

Eu garantiria recursos e quotas suficientes para a carga essencial após o corte. Autoscaling não cria capacidade instantânea ilimitada; instâncias, pools e caches frios podem exigir reserva ou redução explícita de funcionalidades.

Termo para lembrar: Failover capacity | Base técnica: referência 1.

11. Como trata cache após failover?

Eu invalidaria ou versionaria entradas cuja origem e atualidade não posso garantir, limitando reconstrução para proteger o banco. Um cache quente com dados antigos pode parecer saudável e induzir decisões incorretas na região promovida.

Termo para lembrar: Cache freshness | Base técnica: referência 1.

12. Como preserva sessões?

Eu manteria identidade e tokens necessários em mecanismos recuperáveis, revalidando autorização no destino. Sessão presa em memória regional pode exigir novo login; o contrato de experiência deve declarar o comportamento durante recuperação.

Termo para lembrar: Session continuity | Base técnica: referência 1.

13. Como trata idempotência entre regiões?

Eu preservaria o identificador da intenção e garantiria que a autoridade promovida conheça operações confirmadas conforme o RPO. Uma tabela de dedupe atrasada pode permitir repetição; efeitos externos precisam reconhecer a mesma identidade.

Termo para lembrar: Global operation identity | Base técnica: referência 1.

14. Como recupera mensagens?

Eu definiria replicação, retenção e posição de consumo, distinguindo envio ao broker e efeito no consumidor. Reprocessar após desastre pode repetir eventos; offsets e deduplicação precisam ser coerentes com o estado restaurado.

Termo para lembrar: Cross-region messaging | Base técnica: referência 1.

15. Como lida com parceiro que aceita só uma origem?

Eu prepararia certificados, allowlists, rotas e contratos de conexão para a região alternativa. Banco e aplicação podem estar prontos, mas uma restrição externa de rede ainda impediria concluir a jornada financeira.

Termo para lembrar: External dependency readiness | Base técnica: referência 1.

16. Como evita split-brain de scheduler?

Eu separaria ocorrência de job e tentativa de execução, com autoridade ou claim verificável entre regiões. Dois schedulers podem disparar a mesma ocorrência; o efeito precisa de identidade estável e proteção adicional contra duplicata.

Termo para lembrar: Scheduler ownership | Base técnica: referência 1.

17. Pode usar last-write-wins para saldo?

Eu não escolheria sobrescrita por timestamp para reconciliar movimentos financeiros concorrentes. Ela pode descartar uma intenção legítima; preferiria autoridade, coordenação ou direitos previamente reservados, mantendo lançamentos e compensações auditáveis conforme o domínio.

Termo para lembrar: Conflict policy | Base técnica: referência 1.

18. Quando usa multiwriter?

Eu consideraria dados com merge definido e invariantes compatíveis, como algumas preferências ou registros independentes. Precisaria explicar conflito e convergência; multiwriter não é uma promoção de senioridade nem solução universal para disponibilidade bancária.

Termo para lembrar: Conflict-compatible writes | Base técnica: referência 1.

19. Como funciona escrow regional?

Eu distribuiria direitos de consumo limitados para permitir decisões locais dentro de parcelas seguras. A técnica depende de recurso divisível e regras corretas; capacidade pode ficar presa numa região enquanto outra esgota sua parcela.

Termo para lembrar: Rights allocation | Base técnica: referência 1.

20. Como trata residência de dados?

Eu configuraria alocação, replicação e backups conforme restrições validadas para cada classe de dado. Roteamento global e observabilidade também podem mover informação; o plano precisa cobrir todo o ciclo, não apenas o banco principal.

Termo para lembrar: Data residency | Base técnica: referência 1.

21. Como faz retorno à região original?

Eu trataria como migração controlada, sincronizando estado e estabelecendo um novo corte de autoridade. Voltar automaticamente ao primeiro sinal de saúde pode causar oscilação, perda de progresso ou duas regiões atuando como writer.

Termo para lembrar: Failback | Base técnica: referência 1.

22. Como testa disaster recovery?

Eu executaria exercícios graduais com objetivos e abort criteria, medindo dados, jornada e operação. Um teste de desligar instâncias não cobre perda de controle regional, serviços gerenciados, segredos e limites de fornecedores externos.

Termo para lembrar: Regional drill | Base técnica: referência 1.

23. Como considera latência inter-regional?

Eu calcularia quantas viagens entram no caminho crítico e mediria cauda, não só média. Distância e variabilidade tornam coordenação global cara; a escolha deve preservar a invariante sem adicionar dependências desnecessárias a toda consulta.

Termo para lembrar: Geographic coordination | Base técnica: referência 1.

24. Como usa células?

Eu distribuiria grupos de clientes por stacks isoladas, limitando raio de falha e capacidade por unidade. Células ainda precisam de roteamento e migração; uma dependência global obrigatória pode reintroduzir o ponto único que tentamos remover.

Termo para lembrar: Cell-based architecture | Base técnica: referência 1.

25. Como trata configuração global?

Eu manteria caminho de dados capaz de operar por um período com configuração válida em cache, quando seguro. Se toda requisição depende do control plane global, sua falha pode derrubar todas as regiões simultaneamente.

Termo para lembrar: Control-plane dependency | Base técnica: referência 1.

26. Como mede custo total?

Eu incluiria egress, replicação, recursos reservados, duplicação de serviços e exercícios operacionais. O custo não é apenas dobrar compute; atrasos, coordenação de deploy e manutenção de competências também entram na decisão.

Termo para lembrar: Multi-region TCO | Base técnica: referência 1.

27. Como faz deploy entre regiões?

Eu atualizaria gradualmente, observando métricas e mantendo compatibilidade de schema e eventos. Implantar a mesma falha em todas as regiões remove independência operacional; rollout precisa preservar uma opção saudável de recuperação.

Termo para lembrar: Progressive regional rollout | Base técnica: referência 1.

28. Como reconcilia perda dentro do RPO?

Eu compararia operações com fontes e parceiros duráveis, preservando identidades originais. Se um pagamento externo aconteceu e o registro local foi perdido, não posso simplesmente repetir; preciso reconstruir resultado e auditoria antes de avançar.

Termo para lembrar: Post-disaster reconciliation | Base técnica: referência 1.

29. Como comunica degradação?

Eu mostraria quais funções estão disponíveis, pendentes ou temporariamente bloqueadas, sem declarar sucesso financeiro incerto. A experiência pode manter consulta e atendimento enquanto protege comandos cuja autoridade ainda está sendo restabelecida.

Termo para lembrar: Regional user experience | Base técnica: referência 1.

30. Como defenderia o desenho?

Eu percorreria perda de uma região e partição entre duas regiões, mostrando autoridade, dados e recuperação. Escolheria active-passive, células ou multiwriter pelo requisito concreto, nunca como indicador automático de sofisticação ou nível profissional.

Termo para lembrar: Failure-scenario rationale | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Active-passive

Ganho. Simplifica autoridade

Custo e motivo. Exige recuperação

Como decidir. Quando latência local global não é necessária

02. Entrada active-active

Ganho. Aproxima clientes

Custo e motivo. Complica roteamento

Como decidir. Quando serviços stateless permitem

03. Multiwriter

Ganho. Mantém escrita local

Custo e motivo. Exige conflitos seguros

Como decidir. Quando invariantes permitem

04. Home region

Ganho. Clarifica autoridade

Custo e motivo. Pode aumentar RTT

Como decidir. Quando entidade precisa de ordem

05. Replicação síncrona

Ganho. Reduz perda

Custo e motivo. Acopla regiões

Como decidir. Quando RPO exige

06. Replicação assíncrona

Ganho. Reduz latência

Custo e motivo. Aceita janela de perda

Como decidir. Quando reconciliação é viável

07. Standby quente

Ganho. Reduz RTO

Custo e motivo. Custa capacidade ociosa

Como decidir. Quando interrupção deve ser curta

08. Células

Ganho. Limitam blast radius

Custo e motivo. Duplicam operação

Como decidir. Quando escala organizacional justifica

09. Failover automático

Ganho. Reage rapidamente

Custo e motivo. Exige fencing rigoroso

Como decidir. Quando critérios são confiáveis

10. Failback manual controlado

Ganho. Evita oscilação

Custo e motivo. Aumenta trabalho

Como decidir. Quando retorno envolve estado complexo

Duas alternativas ao caminho proposto

1. Multi-AZ regional

Quando faz sentido. Quando falha de zona é o cenário prioritário

Como implementar. Distribuir réplicas e aplicação entre zonas com backup externo

Limites. Não cobre toda perda regional com baixa interrupção

2. Recuperação por restauração

Quando faz sentido. Quando RTO maior é aceitável

Como implementar. Restaurar infraestrutura e dados em região alternativa

Limites. Precisa de ensaio e reconciliação de efeitos externos

Referências e alcance da evidência

1. PostgreSQL: Log-Shipping Standby Servers

Streaming replication, standby e replicação síncrona; cenários de failover precisam de configuração específica.

https://www.postgresql.org/docs/current/warm-standby.html

2. AWS: DynamoDB global tables

Replicação multi-região e modos de consistência documentados; capacidades dependem da configuração escolhida.

https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/GlobalTables.html

3. Gilbert e Lynch: prova de CAP

Modelo formal de consistência atômica, disponibilidade e partições; exemplos bancários são análises autorais.

https://www.comp.nus.edu.sg/~gilbert/pubs/BrewersConjecture-SigAct.pdf

Continue em: T016 · T017 · T067

T021 / C. Bancos e armazenamento

SQL versus NoSQL e modelagem

SQL e NoSQL abrangem modelos e implementações variados. A decisão considera consultas, invariantes, escala, consistência e operação; formato de dados ou marketing não substituem análise do workload.

Quando usar. Ao escolher autoridade de dados, projeções e armazenamento especializado para cada jornada.

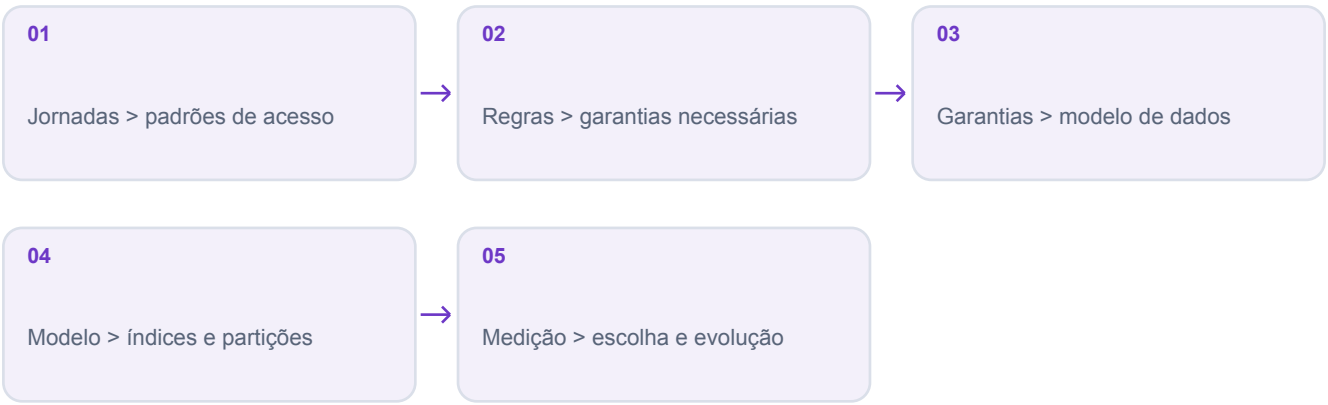
Quando evitar ou limitar. Evitar SQL não escala, NoSQL não tem transação ou banco sem schema; verificar capacidades concretas da implementação.

Uma abertura possível

Vou listar consultas e regras antes de escolher o banco. Usarei relacional quando transações e flexibilidade ajudarem, e modelos especializados quando o acesso justificar a complexidade adicional.

Peças e tecnologias. PostgreSQL · DynamoDB · MongoDB · Cassandra · OpenSearch · S3

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Listar leituras, escritas e cardinalidades.

2. Definir invariantes e escopo transacional.

3. Modelar chaves e índices por opção.

4. Comparar operação, custo e evolução.

5. Testar workload e caminho de migração.

30 perguntas para sustentar o desenho

01. Como escolhe entre SQL e NoSQL?

Eu compararia consultas, transações, distribuição e operação exigidas, não apenas formato JSON. Escolheria a implementação que atende às garantias com menor custo total, registrando quais padrões de acesso ficam caros ou difíceis.

Termo para lembrar: Workload-driven storage | Base técnica: referência 1.

02. SQL não escala horizontalmente?

Eu lembraria que existem réplicas, particionamento e bancos relacionais distribuídos, cada qual com custos. A pergunta é qual limite temos e quais consultas precisam preservar atomicidade; o modelo relacional não determina sozinho a topologia.

Termo para lembrar: Relational scalability | Base técnica: referência 1.

03. NoSQL significa ausência de transações?

Eu verificaria o produto e seu escopo, pois alguns oferecem transações e condições atômicas. Limites de itens, regiões e custos importam; a categoria NoSQL não descreve uma garantia única de concorrência ou durabilidade.

Termo para lembrar: Transaction scope | Base técnica: referência 1.

04. Como começa o modelo relacional?

Eu identificaria entidades, relações, chaves e constraints, evitando duplicação sem necessidade. Também consideraria consultas e índices desde o início; normalização ajuda integridade, mas não dispensa análise de desempenho do workload real.

Termo para lembrar: Normalization | Base técnica: referência 1.

05. Quando desnormaliza?

Eu duplicaria dados quando a leitura frequente e mensurada justificar, definindo autoridade e atualização. O ganho de consulta vem com custo de sincronização e espaço; sem owner, as cópias tendem a divergir silenciosamente.

Termo para lembrar: Denormalization | Base técnica: referência 1.

06. Como modela key-value?

Eu escolheria chaves que resolvem leituras previsíveis diretamente, incluindo tenant e identidade quando necessário. Consultas fora desse acesso exigem índices ou scans; a simplicidade da leitura depende de conhecer previamente as perguntas importantes.

Termo para lembrar: Key-value access | Base técnica: referência 1.

07. Quando usa documentos?

Eu agruparia dados consumidos e alterados juntos, com tamanho e crescimento controlados. Embutir tudo num documento cria contenção e limites; referências fazem sentido quando partes têm ciclo de vida ou cardinalidade independente.

Termo para lembrar: Document aggregate | Base técnica: referência 1.

08. Quando usa wide-column?

Eu modelaria partição e ordenação interna para consultas previsíveis e grande volume, evitando partições ilimitadas. Flexibilidade analítica pode ser menor; novas perguntas frequentemente exigem outra tabela ou pipeline de projeção especializado.

Termo para lembrar: Query-first modeling | Base técnica: referência 1.

09. Schema flexível elimina migração?

Eu explicaria que versões coexistentes de documentos ainda precisam ser compreendidas pela aplicação. Validação, backfill e compatibilidade continuam existindo; apenas mover a responsabilidade do banco para o código não elimina o problema.

Termo para lembrar: Schema evolution | Base técnica: referência 1.

10. Como representa relações muitos-para-muitos?

Eu compararia tabela associativa com duplicação orientada a consultas, considerando atualização e cardinalidade. Relações que mudam frequentemente podem tornar múltiplas cópias caras; a escolha depende de quem consulta e qual consistência exige.

Termo para lembrar: Relationship modeling | Base técnica: referência 1.

11. Como faz unicidade em NoSQL?

Eu usaria chave, condição ou transação suportada para o escopo necessário. Uma consulta antes de inserir não protege concorrência; unicidade global entre partições pode exigir uma autoridade adicional ou outro modelo de chave.

Termo para lembrar: Uniqueness guarantee | Base técnica: referência 1.

12. Como decide a fonte da verdade?

Eu definiria qual armazenamento confirma decisões e quais são projeções reconstruíveis. Usar bancos especializados é útil, mas não quero duas autoridades independentes discordando sobre saldo ou status financeiro sem protocolo explícito de resolução.

Termo para lembrar: Authoritative store | Base técnica: referência 1.

13. Como compara custo?

Eu incluiria operações, índices, replicas, armazenamento, manutenção e engenharia. Um serviço gerenciado reduz trabalho operacional, mas scans e acessos mal modelados podem custar muito; preço por unidade precisa ser aplicado ao workload real.

Termo para lembrar: Storage unit economics | Base técnica: referência 1.

14. Como evita polyglot persistence exagerada?

Eu acrescentaria outro banco apenas quando um requisito relevante não for bem atendido pelo existente. Cada tecnologia cria backup, segurança, plantão e migração; variedade por preferência pessoal aumenta risco sem benefício proporcional.

Termo para lembrar: Operational burden | Base técnica: referência 1.

15. Como escolhe para ledger?

Eu priorizaria atomicidade dos lançamentos, unicidade de intenção, auditoria e recuperação. O nome do banco é secundário à implementação; um relacional bem modelado pode atender, assim como outra tecnologia com garantias comprovadas.

Termo para lembrar: Financial storage | Base técnica: referência 1.

16. Como escolhe para busca textual?

Eu usaria índice de busca quando análise linguística e ranking justificarem, alimentado por fonte autoritativa. Ele pode atrasar e precisa de controle de acesso; não o tornaria autoridade financeira apenas porque responde consultas rapidamente.

Termo para lembrar: Specialized index | Base técnica: referência 1.

17. Como escolhe para arquivos?

Eu armazenaria objetos grandes em serviço apropriado, mantendo metadados e autorização no domínio. Colocar anexos no banco transacional aumenta backup e I/O; separação exige lidar com upload incompleto, órfãos e retenção.

Termo para lembrar: Object storage | Base técnica: referência 1.

18. Como trata consultas ad hoc?

Eu avaliaria a frequência e o custo de perguntas novas. Modelo relacional ou analítico pode oferecer flexibilidade; um desenho key-value extremamente otimizado exige novas projeções quando o produto muda seus padrões de acesso.

Termo para lembrar: Query flexibility | Base técnica: referência 1.

19. Como compara consistência?

Eu verificaria garantia por operação, índice e região, incluindo falhas e concorrência. Não classificaria todo SQL como forte nem todo NoSQL como eventual; configurações e caminhos de leitura podem alterar a experiência observada.

Termo para lembrar: Consistency contract | Base técnica: referência 1.

20. Como migra sem escrita dupla insegura?

Eu manteria writer autoritativo, copiaria histórico e acompanharia mudanças por mecanismo durável. Após comparar e alcançar a posição necessária, faria corte controlado; duas escritas independentes podem divergir quando uma falha.

Termo para lembrar: Migration authority | Base técnica: referência 1.

21. Como avalia índices secundários?

Eu incluiria custo de escrita, espaço e frescor, além do benefício da consulta. Índices podem ter consistência diferente da tabela; uma operação recém-criada talvez ainda não apareça na busca que depende deles.

Termo para lembrar: Index maintenance | Base técnica: referência 1.

22. Como trata cardinalidade crescente?

Eu evitaria arrays ou partições que crescem sem limite, usando páginas, buckets ou entidades separadas. A modelagem deve considerar o cliente mais antigo e ativo, não apenas o documento pequeno do protótipo.

Termo para lembrar: Bounded collection | Base técnica: referência 1.

23. Como protege integridade referencial?

Eu usaria constraints quando disponíveis e apropriadas, ou protocolos explícitos quando os dados estão distribuídos. A aplicação precisa tratar órfãos e exclusões concorrentes; ausência de foreign key não elimina a regra de negócio.

Termo para lembrar: Referential integrity | Base técnica: referência 1.

24. Como escolhe para histórico temporal?

Eu modelaria identidade, instante e ordenação para consultas por entidade e período, considerando retenção. Histórico append-only facilita auditoria, mas ainda exige correção de eventos, índices e política para dados pessoais e snapshots.

Termo para lembrar: Temporal access | Base técnica: referência 1.

25. Como considera backup na escolha?

Eu exigiria restauração testada, consistência e suporte ao horizonte de retenção. Um banco com excelente throughput, mas recuperação inadequada, não atende o requisito; também avaliaria como reconstruir índices e projeções derivados.

Termo para lembrar: Recoverability | Base técnica: referência 1.

26. Como evita scans acidentais?

Eu revisaria APIs e planos de consulta, limitando operações que não possuem índice ou chave adequada. Em bancos cobrados por leitura, um filtro posterior ao scan pode continuar consumindo grande capacidade sem entregar muitos resultados.

Termo para lembrar: Access path control | Base técnica: referência 1.

27. Como trata transações entre bancos?

Eu evitaria presumir commit conjunto, usando outbox, estados e reconciliação quando cruzar autoridades. Uma biblioteca que chama ambos não torna a atualização atômica; a arquitetura precisa declarar a janela e sua recuperação.

Termo para lembrar: Polyglot consistency | Base técnica: referência 1.

28. Como avalia lock-in do modelo?

Eu analisaria chaves, consultas e funcionalidades proprietárias, além do formato exportado. Copiar JSON é simples comparado a reproduzir transações, índices e desempenho; a saída do fornecedor precisa de plano compatível com a semântica usada.

Termo para lembrar: Data portability | Base técnica: referência 1.

29. Como testa uma escolha de banco?

Eu executaria consultas e escritas reais com distribuição, concorrência e tamanho futuros plausíveis, incluindo manutenção e falhas. Testar apenas inserts sequenciais em memória não sustenta uma decisão sobre um sistema bancário completo.

Termo para lembrar: Representative benchmark | Base técnica: referência 1.

30. Como resume a decisão?

Eu mostraria duas consultas, uma invariante e uma falha relevante, explicando como o banco as atende. Também nomearia o custo aceito e o sinal que justificaria adicionar outra tecnologia ou revisar o modelo.

Termo para lembrar: Fit-for-purpose persistence | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Normalização

Ganho. Reduz inconsistência

Custo e motivo. Pode exigir joins

Como decidir. Quando relações mudam frequentemente

02. Desnormalização

Ganho. Acelera leitura

Custo e motivo. Duplica atualização

Como decidir. Quando consultas são previsíveis

03. Relacional

Ganho. Oferece flexibilidade

Custo e motivo. Exige índices e capacidade

Como decidir. Quando transações e relações dominam

04. Key-value

Ganho. Acesso direto eficiente

Custo e motivo. Limita consultas livres

Como decidir. Quando chaves são conhecidas

05. Documento embutido

Ganho. Agrupa leitura

Custo e motivo. Pode crescer demais

Como decidir. Quando ciclo de vida é conjunto

06. Wide-column

Ganho. Escala acessos previstos

Custo e motivo. Exige modelagem por consulta

Como decidir. Quando volume é elevado

07. Índice especializado

Ganho. Melhora busca

Custo e motivo. Cria lag e operação

Como decidir. Quando ranking é requisito

08. Banco gerenciado

Ganho. Reduz manutenção

Custo e motivo. Aumenta dependência

Como decidir. Quando equipe quer foco no domínio

09. Múltiplos bancos

Ganho. Atendem workloads distintos

Custo e motivo. Complicam consistência

Como decidir. Quando benefício é medido

10. Banco único

Ganho. Simplifica operação

Custo e motivo. Pode limitar especialização

Como decidir. Quando requisitos cabem nele

Duas alternativas ao caminho proposto

1. Relacional com JSON

Quando faz sentido. Quando parte do modelo varia e transações importam

Como implementar. Combinar colunas e documentos com índices apropriados

Limites. Não oferece automaticamente todas as vantagens de bancos especializados

2. Autoridade mais projeções

Quando faz sentido. Quando escrita e leitura têm necessidades distintas

Como implementar. Manter decisões num store e alimentar modelos reconstruíveis

Limites. Exige pipeline, frescor e reconciliação

Referências e alcance da evidência

1. Microsoft: Understand data models

Modelos de armazenamento e critérios por padrão de acesso; propostas bancárias são análises autorais.

https://learn.microsoft.com/en-us/azure/architecture/data-guide/technology-choices/understand-data-store-models

2. AWS: NoSQL design for DynamoDB

Modelagem orientada a padrões de acesso no DynamoDB, sem generalizar limitações a todo banco relacional.

https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/bp-general-nosql-design.html

3. PostgreSQL: Transaction isolation

Isolamento, snapshots e necessidade de repetição de transações sob conflitos.

https://www.postgresql.org/docs/current/transaction-iso.html

Continue em: T022 · T026 · T029

T022 / C. Bancos e armazenamento

PostgreSQL: consultas, planos e paginação

Otimização PostgreSQL relaciona consulta, estatísticas, plano, índices e recursos. Paginação precisa de ordem determinística e custo controlado; velocidade deve ser avaliada com dados e concorrência representativos.

Quando usar. Ao diagnosticar lentidão, dimensionar OLTP e expor históricos ou filas grandes.

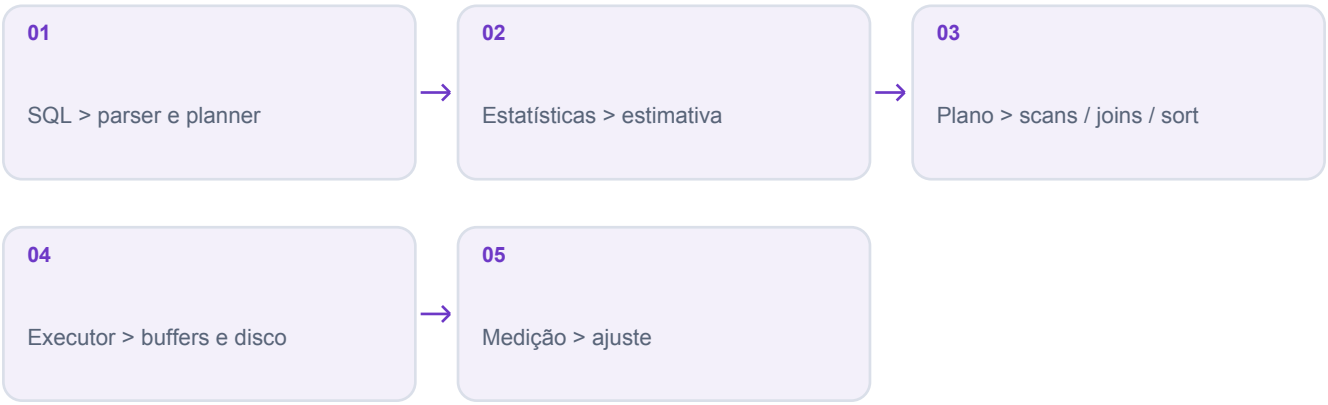
Quando evitar ou limitar. Evitar criar índice para toda coluna, usar EXPLAIN ANALYZE destrutivo sem proteção ou confiar em benchmark com tabela pequena.

Uma abertura possível

Vou olhar plano estimado e realizado, identificar cardinalidade incorreta e trabalho dominante. Depois ajusto consulta ou índice e testo sob concorrência, preservando semântica e custo de escrita.

Peças e tecnologias. PostgreSQL · EXPLAIN ANALYZE · pg_stat_statements · B-tree · keyset pagination

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Capturar consulta e parâmetros relevantes.

2. Comparar estimativas e linhas reais.

3. Identificar scan, join, sort ou lock dominante.

4. Ajustar índice, SQL ou estatísticas.

5. Validar latência, escrita e paginação sob carga.

30 perguntas para sustentar o desenho

01. Como começa a investigar consulta lenta?

Eu capturaria SQL, parâmetros e frequência, verificando se a espera é execução ou lock. Depois analisaria plano e métricas; otimizar uma consulta rara pode valer menos que reduzir uma pequena ineficiência repetida milhões de vezes.

Termo para lembrar: Query diagnosis | Base técnica: referência 1.

02. EXPLAIN e EXPLAIN ANALYZE diferem como?

Eu usaria EXPLAIN para estimativas e ANALYZE para executar e medir comportamento real. Para comandos que alteram dados, isso produz efeitos; preciso de ambiente seguro ou estratégia controlada antes de executar o diagnóstico.

Termo para lembrar: Execution plan | Base técnica: referência 1.

03. Cost é tempo em milissegundos?

Eu não interpretaria custo estimado como duração direta. Ele compara alternativas conforme parâmetros do planner; mediria tempo real e buffers, considerando que hardware, cache e concorrência alteram o desempenho observado.

Termo para lembrar: Planner cost | Base técnica: referência 1.

04. Por que estimativa de linhas importa?

Eu verificaria divergência entre linhas previstas e reais porque ela influencia joins e scans escolhidos. Estatísticas desatualizadas ou colunas correlacionadas podem produzir plano ruim, mesmo quando existe um índice aparentemente adequado à consulta.

Termo para lembrar: Cardinality estimation | Base técnica: referência 1.

05. Sequential scan é sempre ruim?

Eu aceitaria varredura quando a consulta precisa de grande parte da tabela ou ela é pequena. Forçar índice pode gerar muitos acessos aleatórios; a escolha correta depende de seletividade, cache e custo real.

Termo para lembrar: Sequential scan | Base técnica: referência 1.

06. Como funciona index scan?

Eu localizaria candidatos pelo índice e buscaria dados necessários na tabela, conforme o plano. Se muitos registros passam pelo filtro, essas visitas podem custar mais que uma leitura sequencial bem aproveitada pelo armazenamento.

Termo para lembrar: Index scan | Base técnica: referência 1.

07. Quando index-only ajuda?

Eu usaria um índice que contém os campos necessários e verificaria visibilidade das páginas. No PostgreSQL, ainda pode haver consultas ao heap para checar versões; incluir colunas não garante eliminar todo acesso à tabela.

Termo para lembrar: Index-only scan | Base técnica: referência 3.

08. O que é bitmap scan?

Eu o interpretaria como estratégia para reunir localizações antes de visitar páginas, podendo combinar índices. Ajuda em certas seletividades intermediárias, mas memória e quantidade de páginas influenciam rechecks e custo final da execução.

Termo para lembrar: Bitmap heap scan | Base técnica: referência 1.

09. Como escolhe ordem de índice composto?

Eu alinharia colunas a filtros de igualdade, faixa e ordenação frequentes, verificando o plano real. Não aplicaria uma regra cega de maior seletividade primeiro; a combinação de acesso e prefixos define o benefício.

Termo para lembrar: Composite index | Base técnica: referência 1.

10. Por que OFFSET profundo piora?

Eu explicaria que o banco pode precisar percorrer e descartar muitas linhas anteriores antes de devolver a página. Para navegação sequencial, cursor por chave ordenada evita trabalho proporcional à profundidade e melhora estabilidade operacional.

Termo para lembrar: Offset pagination | Base técnica: referência 1.

11. Como implementa keyset pagination?

Eu usaria ordenação estável com desempate, como data e ID, consultando valores posteriores ao último cursor. O índice acompanha essa ordem; o contrato explica inserções concorrentes e não promete saltar diretamente para qualquer página.

Termo para lembrar: Keyset pagination | Base técnica: referência 1.

12. Como evita registros repetidos entre páginas?

Eu definiria snapshot ou limites de navegação quando consistência entre páginas for necessária. Keyset reduz problemas de deslocamento, mas alterações nas colunas de ordenação ainda podem mover registros; a semântica precisa ser explícita.

Termo para lembrar: Pagination consistency | Base técnica: referência 1.

13. Como trata COUNT total?

Eu avaliaria se a interface realmente precisa do total exato em toda consulta. Contagem pode varrer muitos registros; aproximação, cache ou cálculo assíncrono reduzem custo quando o produto aceita diferença temporária.

Termo para lembrar: Exact count cost | Base técnica: referência 1.

14. Como evita SELECT estrela?

Eu retornaria apenas campos usados, reduzindo bytes, materialização e acoplamento de schema. Isso também ajuda índices de cobertura; para dados grandes, trazer colunas inúteis pode dominar a latência mesmo com filtro eficiente.

Termo para lembrar: Projection | Base técnica: referência 1.

15. Como resolve N+1 em SQL?

Eu agruparia consultas ou usaria join adequado, validando cardinalidade e tamanho do resultado. Um join que multiplica linhas excessivamente também pode ser ruim; o objetivo é reduzir viagens sem criar explosão combinatória.

Termo para lembrar: Query batching | Base técnica: referência 1.

16. Nested loop é sempre ruim?

Eu o consideraria eficiente quando a entrada externa é pequena e a busca interna usa índice. Para muitos registros, hash ou merge join podem ser melhores; estatísticas e distribuição determinam a escolha adequada do planner.

Termo para lembrar: Join strategy | Base técnica: referência 1.

17. Como diagnostica sort em disco?

Eu observaria método e espaço usado, investigando tamanho do conjunto e memória disponível. Ajustaria consulta ou índice antes de aumentar memória indiscriminadamente, porque work_mem pode ser consumido por várias operações e sessões simultâneas.

Termo para lembrar: Sort spill | Base técnica: referência 1.

18. Por que função na coluna pode atrapalhar?

Eu verificaria se a expressão impede usar o índice existente, como transformar cada valor antes da comparação. Poderia reescrever o predicado ou criar índice de expressão, mantendo semântica e custo de manutenção claros.

Termo para lembrar: Sargability | Base técnica: referência 1.

19. Como usa índice parcial?

Eu indexaria um subconjunto frequente, como jobs pendentes, quando o predicado da consulta corresponde à condição. Isso reduz tamanho e escrita, mas consultas fora do subconjunto precisam de outro caminho e validação do planner.

Termo para lembrar: Partial index | Base técnica: referência 1.

20. Como trata parâmetros com distribuição desigual?

Eu compararia planos para valores comuns e raros, porque um plano genérico pode ser inadequado para um tenant enorme. Testaria prepared statements e estatísticas, evitando concluir desempenho a partir de um único parâmetro conveniente.

Termo para lembrar: Parameter sensitivity | Base técnica: referência 1.

21. Como atualiza estatísticas?

Eu verificaria autovacuum e analyze, executando manutenção apropriada após grandes mudanças de distribuição. Estatísticas ajudam o planner, mas não corrigem uma consulta sem índice adequado ou uma invariante mal modelada na aplicação.

Termo para lembrar: ANALYZE | Base técnica: referência 1.

22. Como trabalha com tabelas particionadas?

Eu alinharia chave de partição ao filtro e confirmaria quais partições são acessadas no plano. Particionar por data não acelera automaticamente consulta apenas por cliente; índices e poda precisam corresponder às perguntas reais.

Termo para lembrar: Partition pruning | Base técnica: referência 1.

23. Como cria índice em produção?

Eu avaliaria construção concorrente quando suportada, espaço, duração e falhas possíveis. Ela reduz bloqueio de escrita em comparação com o caminho simples, mas tem custos e restrições; monitoraria o resultado antes de confiar no índice.

Termo para lembrar: Concurrent index build | Base técnica: referência 4.

24. Como diagnostica lock em vez de query?

Eu consultaria sessões bloqueadoras, eventos de espera e duração das transações. Uma consulta com plano excelente pode ficar parada aguardando outra; adicionar índice não resolve necessariamente uma transação esquecida segurando o recurso.

Termo para lembrar: Wait events | Base técnica: referência 1.

25. Como controla consultas de relatório?

Eu isolaria recursos ou usaria réplica e pipeline analítico quando consultas longas prejudicam OLTP. A cópia pode atrasar; relatórios financeiros precisam de snapshot ou posição coerente, não apenas de um servidor diferente.

Termo para lembrar: Workload isolation | Base técnica: referência 1.

26. Como evita exportação explodir memória?

Eu leria páginas ou cursor controlado e escreveria saída incrementalmente, com limites de tempo e bytes. Transações muito longas retêm versões; o desenho da exportação precisa equilibrar consistência, duração e impacto no banco.

Termo para lembrar: Streaming results | Base técnica: referência 1.

27. Como define statement timeout?

Eu escolheria limites por classe de operação, menores que o prazo total da jornada quando apropriado. Cancelar protege recursos, mas a aplicação precisa tratar erro e transação abortada, evitando devolver conexão suja ao pool.

Termo para lembrar: Query deadline | Base técnica: referência 1.

28. Como testa melhoria de índice?

Eu compararia planos e latência sob carga representativa, medindo também inserts, updates e espaço. Um índice pode acelerar uma consulta e piorar toda escrita; a decisão considera o benefício agregado do workload.

Termo para lembrar: Before-and-after measurement | Base técnica: referência 1.

29. Como detecta regressão após deploy?

Eu acompanharia fingerprints, tempo total, cauda e linhas processadas por chamada, correlacionando com release e crescimento. Mudança de parâmetros ou estatísticas pode alterar plano sem mudança de SQL; alertas precisam observar comportamento real.

Termo para lembrar: Query observability | Base técnica: referência 1.

30. Como explica otimização em Staff?

Eu identificaria trabalho desnecessário e sua causa, escolheria uma mudança pequena e mensurável e demonstraria limites. Não começaria trocando o banco; primeiro mostraria se o gargalo é consulta, contenção, I/O ou capacidade da aplicação.

Termo para lembrar: Evidence-based tuning | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Índice adicional

Ganho. Acelera acesso

Custo e motivo. Aumenta escrita e espaço

Como decidir. Quando consulta frequente justifica

02. Keyset

Ganho. Evita descarte profundo

Custo e motivo. Limita salto de páginas

Como decidir. Quando navegação é sequencial

03. OFFSET

Ganho. Simplifica API

Custo e motivo. Cresce com profundidade

Como decidir. Quando conjunto é pequeno

04. COUNT exato

Ganho. Entrega precisão

Custo e motivo. Pode custar varredura

Como decidir. Quando decisão exige total

05. Índice de cobertura

Ganho. Reduz heap

Custo e motivo. Aumenta tamanho

Como decidir. Quando colunas são estáveis

06. Mais work_mem

Ganho. Reduz spills

Custo e motivo. Amplia risco agregado

Como decidir. Quando orçamento global permite

07. Réplica analítica

Ganho. Isola consultas

Custo e motivo. Introduz lag

Como decidir. Quando frescor tolera

08. Particionamento

Ganho. Facilita poda e retenção

Custo e motivo. Complica manutenção

Como decidir. Quando filtros usam a chave

09. Plano genérico

Ganho. Reduz planejamento

Custo e motivo. Pode ignorar skew

Como decidir. Quando parâmetros são semelhantes

10. Snapshot longo

Ganho. Preserva exportação coerente

Custo e motivo. Retém versões

Como decidir. Quando consistência histórica é necessária

Duas alternativas ao caminho proposto

1. Modelo de leitura especializado

Quando faz sentido. Quando SQL online faz agregações repetidas caras

Como implementar. Pré-calcular projeção por evento ou job

Limites. Introduz frescor limitado e reconstrução

2. Armazenamento analítico

Quando faz sentido. Quando scans e agregações dominam

Como implementar. Enviar dados para engine colunar com isolamento

Limites. Não substitui autoridade transacional e exige pipeline

Referências e alcance da evidência

1. PostgreSQL: Using EXPLAIN

Leitura de planos, estimativas e execução com EXPLAIN ANALYZE; análise depende de estatísticas e workload.

https://www.postgresql.org/docs/current/using-explain.html

2. PostgreSQL: Index Types

Tipos de índices e operadores suportados no PostgreSQL.

https://www.postgresql.org/docs/current/indexes-types.html

3. PostgreSQL: Index-only scans

Requisitos de cobertura e verificação de visibilidade no heap.

https://www.postgresql.org/docs/current/indexes-index-only-scans.html

4. PostgreSQL: CREATE INDEX

Construção de índices, modo CONCURRENTLY, bloqueios, restrições e tratamento de falhas da construção.

https://www.postgresql.org/docs/current/sql-createindex.html

Continue em: T023 · T025 · T086

T023 / C. Bancos e armazenamento

Índices: B-tree, hash, GIN e BRIN

Índices mantêm estruturas auxiliares para localizar registros. B-tree atende igualdade e faixa; hash favorece igualdade; GIN indexa componentes; BRIN resume blocos e depende de correlação física.

Quando usar. Ao reduzir trabalho de leitura e implementar certas constraints, avaliando custo de escrita, memória e manutenção.

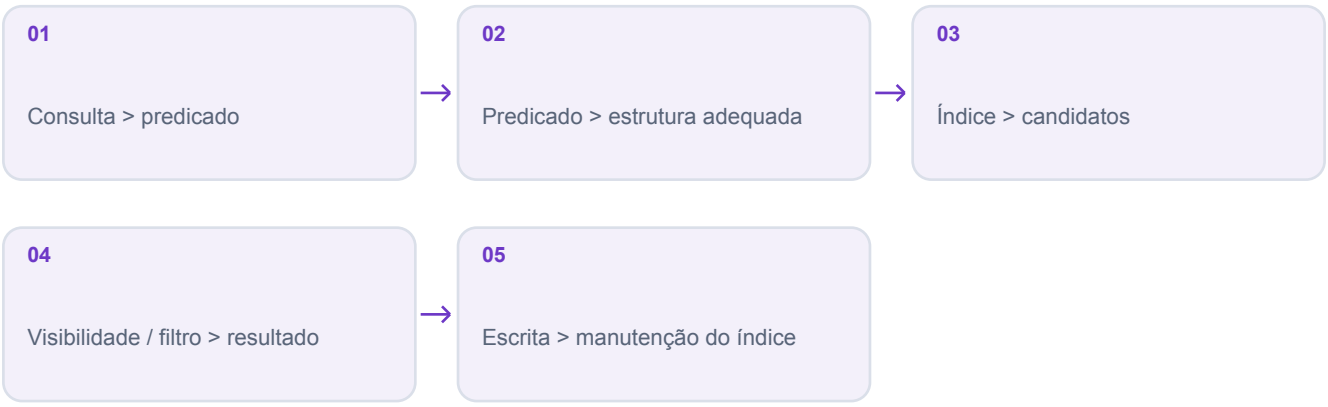
Quando evitar ou limitar. Evitar índice em toda coluna ou escolher estrutura apenas por Big O sem considerar seletividade, I/O e operadores suportados.

Uma abertura possível

Vou escolher o índice pela consulta e distribuição dos dados. Depois verifico o plano e o custo de manutenção, porque cada leitura acelerada pode acrescentar trabalho a toda escrita.

Peças e tecnologias. PostgreSQL B-tree · Hash index · GIN · BRIN · EXPLAIN · INCLUDE

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Mapear filtros, ordenação e cardinalidade.

2. Escolher estrutura e colunas.

3. Avaliar seletividade e correlação física.

4. Construir e medir plano e custo de escrita.

5. Monitorar uso, bloat e evolução do workload.

30 perguntas para sustentar o desenho

01. Quando escolhe B-tree?

Eu usaria para igualdade, intervalos e ordenação compatíveis com seus operadores. A estrutura mantém chaves ordenadas, mas o benefício depende de seletividade; retornar grande parte da tabela pode favorecer varredura sequencial.

Termo para lembrar: B-tree | Base técnica: referência 1.

02. Qual complexidade de busca em B-tree?

Eu explicaria altura logarítmica e páginas com muitos filhos, reduzindo acessos ao armazenamento. O custo real inclui cache, heap e quantidade de resultados; Big O sozinho não prevê latência de uma consulta grande.

Termo para lembrar: Tree height | Base técnica: referência 1.

03. Quando escolhe hash index?

Eu avaliaria igualdade quando o tipo e implementação suportam, comparando tamanho e desempenho com B-tree. Hash não atende faixa ou ordenação; colisões e manutenção continuam existindo, então não presumiria superioridade automática.

Termo para lembrar: Hash index | Base técnica: referência 1.

04. Quando usa GIN?

Eu o escolheria para valores com múltiplos componentes, como arrays, texto ou certos acessos JSON. Ele mapeia componentes a registros; leitura pode melhorar muito, mas alterações geram mais trabalho e exigem manutenção cuidadosa.

Termo para lembrar: Inverted index | Base técnica: referência 1.

05. Quando usa BRIN?

Eu o consideraria para tabelas grandes com correlação entre valor e posição física, como tempo crescente. Seus resumos são pequenos, mas retornam blocos candidatos que precisam ser rechecados; dados espalhados reduzem muito a eficiência.

Termo para lembrar: Block range index | Base técnica: referência 1.

06. Como escolhe índice composto?

Eu combinaria filtros de igualdade, faixa e ordenação usados juntos, verificando o plano. Colunas iniciais influenciam o trecho percorrido; regras simplistas de seletividade não substituem análise das consultas reais e da versão do banco.

Termo para lembrar: Composite key | Base técnica: referência 1.

07. Índice em boolean é inútil?

Eu analisaria distribuição e predicado. Se poucos registros estão pendentes, um índice parcial pode ser excelente; um boolean equilibrado isolado pode selecionar metade da tabela e oferecer pouco benefício à leitura.

Termo para lembrar: Selectivity | Base técnica: referência 1.

08. Como funciona índice parcial?

Eu indexaria somente registros que atendem a uma condição, reduzindo espaço e manutenção. O planner precisa reconhecer que a consulta implica esse predicado; parâmetros e formulações diferentes podem impedir o uso esperado.

Termo para lembrar: Partial index | Base técnica: referência 1.

09. Quando usa índice de expressão?

Eu criaria quando consultas usam uma transformação estável, como normalização específica de texto. A expressão e operadores precisam corresponder ao filtro; também avalio custo adicional de computá-la nas alterações da tabela.

Termo para lembrar: Expression index | Base técnica: referência 1.

10. Como trata collation?

Eu verificaria regras de comparação, acentos e case exigidas pelo negócio, porque elas afetam igualdade e ordenação. Mudanças de collation podem exigir manutenção dos índices; semântica textual não é apenas detalhe de apresentação.

Termo para lembrar: Collation semantics | Base técnica: referência 1.

11. INCLUDE torna coluna parte da chave?

Eu usaria INCLUDE para carregar dados necessários à consulta sem torná-los parte da ordenação da chave. Isso pode favorecer index-only scan, mas aumenta tamanho e custo de atualização, especialmente para colunas largas ou voláteis.

Termo para lembrar: Covering index | Base técnica: referência 1.

12. Index-only nunca acessa heap?

Eu verificaria a visibilidade das páginas no PostgreSQL. Mesmo com todos os campos no índice, pode ser necessário consultar o heap para garantir visibilidade MVCC; manutenção e frequência de updates influenciam esse benefício.

Termo para lembrar: Visibility map | Base técnica: referência 1.

13. Por que índice pode piorar insert?

Eu contabilizaria atualização de cada estrutura, páginas, WAL e possíveis divisões. Muitos índices transformam uma escrita simples em vários trabalhos; remover índices pouco úteis pode aumentar throughput e reduzir custo sem alterar a aplicação.

Termo para lembrar: Write amplification | Base técnica: referência 1.

14. Como índices ajudam unicidade?

Eu usaria mecanismo único do banco para arbitrar inserções concorrentes, escolhendo colunas e semântica de null corretamente. Verificação prévia na aplicação não protege a corrida; a constraint deve representar a identidade de negócio.

Termo para lembrar: Unique constraint | Base técnica: referência 1.

15. Como escolhe índice para histórico?

Eu começaria por entidade, instante e desempate, conforme filtro e paginação. Isso permite localizar um intervalo e continuar por cursor; um índice apenas em data pode varrer muitos clientes antes de encontrar os registros desejados.

Termo para lembrar: Ordered range access | Base técnica: referência 1.

16. Como escolhe para fila de jobs?

Eu indexaria elegibilidade e prazo num subconjunto de tarefas pendentes, limitando lote. Mudanças frequentes de estado também mantêm o índice; avalio contenção e vacuum para que a própria fila não sobrecarregue o banco.

Termo para lembrar: Pending-work index | Base técnica: referência 1.

17. Pode combinar vários índices simples?

Eu verificaria se o planner combina índices de forma eficiente para a consulta, comparando com um composto. Combinação pode ajudar filtros variados, mas não necessariamente preserva ordenação ou evita todo trabalho no heap.

Termo para lembrar: Bitmap combination | Base técnica: referência 1.

18. Como identifica índice redundante?

Eu examinaria consultas, prefixos, constraints e estatísticas de uso ao longo de período representativo. Pouco uso recente não basta para remover um índice necessário a rotinas mensais ou recuperação; validaria os consumidores relevantes.

Termo para lembrar: Index usage analysis | Base técnica: referência 1.

19. Como cria índice sem parar escrita?

Eu consideraria criação concorrente suportada pelo PostgreSQL, avaliando duração, espaço e possíveis resíduos inválidos após falha. Reduzir bloqueio não elimina consumo de recursos; programaria acompanhamento e critérios para interromper com segurança.

Termo para lembrar: Concurrent build | Base técnica: referência 1.

20. Como trata bloat de índice?

Eu mediria espaço e eficiência antes de reconstruir, investigando padrão de updates e manutenção. Reindexação pode recuperar recursos, mas também consome I/O e espaço temporário; corrigir a causa evita repetir uma operação cara.

Termo para lembrar: Index bloat | Base técnica: referência 1.

21. Como índices interagem com cache?

Eu compararia tamanho dos índices com memória disponível e frequência de acesso. Um índice amplo pode expulsar páginas úteis; reduzir colunas e dados indexados melhora localidade, mas precisa preservar os acessos mais importantes.

Termo para lembrar: Working set | Base técnica: referência 1.

22. Como trata JSON no PostgreSQL?

Eu escolheria operadores e índice conforme os caminhos realmente consultados, comparando GIN genérico e expressão específica. Indexar todo documento oferece flexibilidade, mas pode custar mais que uma chave bem definida para uma consulta frequente.

Termo para lembrar: JSON indexing | Base técnica: referência 1.

23. Como usa índice para busca textual?

Eu criaria representação adequada à busca e índice compatível, considerando idioma e normalização. Índice de igualdade sobre texto não oferece stemming ou ranking; a semântica procurada determina a estrutura necessária para a jornada.

Termo para lembrar: Text search index | Base técnica: referência 1.

24. BRIN substitui B-tree?

Eu o trataria como alternativa para certos acessos em dados correlacionados, não substituto geral. BRIN troca precisão por tamanho; consultas muito seletivas ou dados desordenados podem exigir B-tree ou outra estratégia de acesso.

Termo para lembrar: Lossy summarization | Base técnica: referência 1.

25. Como trata ordenação descendente?

Eu verificaria se varredura inversa atende ao ORDER BY e como múltiplas colunas combinam direções e nulls. Criar outro índice pode ser desnecessário; o plano real mostra se ainda existe sort caro.

Termo para lembrar: Index order | Base técnica: referência 1.

26. Como avalia índice com LIMIT?

Eu procuraria um caminho que encontre os primeiros resultados já na ordem exigida, evitando ordenar tudo. Porém, filtros pouco correlacionados podem obrigar percorrer muitas entradas antes de alcançar o limite solicitado pelo cliente.

Termo para lembrar: Top-N query | Base técnica: referência 1.

27. Como testa índice sob concorrência?

Eu executaria leituras e escritas juntas, medindo picos, WAL e latência de commit. Um índice excelente em teste somente leitura pode prejudicar o workload total quando transações concorrentes atualizam suas colunas frequentemente.

Termo para lembrar: Mixed workload benchmark | Base técnica: referência 1.

28. Como faz manutenção em partições?

Eu planejaria índices e construção por partição, verificando coerência do conjunto e limites da versão usada. Remover partições facilita retenção, mas consultas sem poda ainda podem tocar muitos índices e aumentar planejamento.

Termo para lembrar: Partitioned index | Base técnica: referência 1.

29. Como evita abuso de dicas de plano?

Eu investigaria estatísticas, SQL e configuração antes de forçar escolhas. Uma dica pode resolver um caso e piorar outro parâmetro; mudanças devem ter evidência e monitoramento para detectar quando a distribuição muda.

Termo para lembrar: Planner validation | Base técnica: referência 1.

30. Como defenderia o índice escolhido?

Eu mostraria o filtro, a ordem e a quantidade esperada de candidatos, comparando plano antes e depois. Também explicaria escrita, espaço e manutenção, deixando claro por que o ganho compensa o custo naquele workload.

Termo para lembrar: Access-path rationale | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. B-tree

Ganho. Suporta faixa e ordem

Custo e motivo. Mantém páginas e splits

Como decidir. Quando consultas são variadas

02. Hash

Ganho. Foca igualdade

Custo e motivo. Não suporta faixa

Como decidir. Quando igualdade domina e teste justifica

03. GIN

Ganho. Busca componentes

Custo e motivo. Amplifica escrita

Como decidir. Quando documentos e arrays são consultados

04. BRIN

Ganho. Ocupa pouco espaço

Custo e motivo. Exige rechecks

Como decidir. Quando dados são fisicamente correlacionados

05. Índice composto

Ganho. Atende combinação

Custo e motivo. Menos flexível a outras consultas

Como decidir. Quando filtros são recorrentes

06. Índice parcial

Ganho. Reduz tamanho

Custo e motivo. Só atende subconjunto

Como decidir. Quando poucos estados interessam

07. INCLUDE

Ganho. Pode evitar heap

Custo e motivo. Aumenta índice

Como decidir. Quando colunas são pequenas e estáveis

08. Mais índices

Ganho. Aceleram consultas diversas

Custo e motivo. Pioram escrita

Como decidir. Quando benefício agregado compensa

09. Construção concorrente

Ganho. Reduz bloqueio

Custo e motivo. Custa mais trabalho

Como decidir. Quando escrita deve continuar

10. Reindexação

Ganho. Recupera eficiência

Custo e motivo. Consome espaço e I/O

Como decidir. Quando bloat é comprovado

Duas alternativas ao caminho proposto

1. Materialized view

Quando faz sentido. Quando agregação repetida domina

Como implementar. Pré-calcular resultado e indexar a visão

Limites. Exige atualização e política de frescor

2. Índice de busca externo

Quando faz sentido. Quando linguagem e ranking excedem necessidade relacional

Como implementar. Projetar dados para engine especializada

Limites. Adiciona lag, segurança e operação

Referências e alcance da evidência

1. PostgreSQL: Index Types

Tipos de índices e operadores suportados no PostgreSQL.

https://www.postgresql.org/docs/current/indexes-types.html

2. PostgreSQL: Multicolumn indexes

Uso de colunas em índices compostos e características de acesso.

https://www.postgresql.org/docs/current/indexes-multicolumn.html

3. PostgreSQL: Using EXPLAIN

Leitura de planos, estimativas e execução com EXPLAIN ANALYZE; análise depende de estatísticas e workload.

https://www.postgresql.org/docs/current/using-explain.html

Continue em: T022 · T024 · T030

T024 / C. Bancos e armazenamento

WAL, fsync, LSM e compaction

WAL registra mudanças antes das páginas para permitir recuperação. Fsync coordena persistência com armazenamento. LSM acumula escritas e produz arquivos ordenados; compaction reorganiza versões, trocando amplificação de escrita, leitura e espaço.

Quando usar. Ao explicar durabilidade, custo físico de escrita, recuperação, checkpoints e comportamento de bancos sob ingestão intensa.

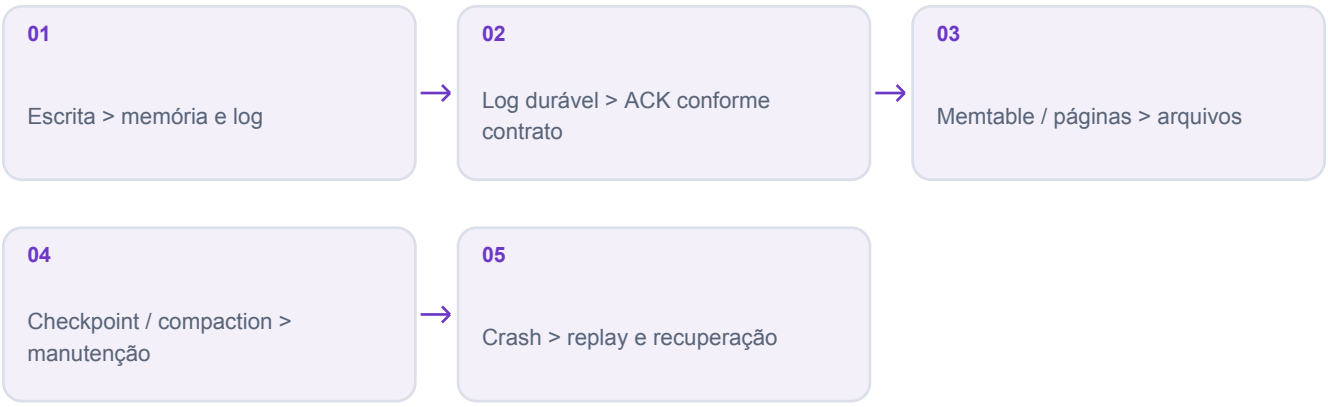
Quando evitar ou limitar. Evitar confundir ACK em memória com persistência ou comparar B-tree e LSM apenas pelo caminho rápido de ingestão sem manutenção.

Uma abertura possível

Vou seguir o dado do buffer ao armazenamento durável, indicando quando respondo ao cliente. Também reservarei capacidade para checkpoint e compaction, que fazem parte do custo real da escrita.

Peças e tecnologias. PostgreSQL WAL · fsync · RocksDB · LSM tree · SSTable · Bloom filter

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Definir o que a confirmação precisa sobreviver.

2. Explicar buffers, log e persistência.

3. Calcular amplificações e manutenção.

4. Planejar recuperação e retenção de log.

5. Testar falha e desempenho em regime estável.

30 perguntas para sustentar o desenho

01. Por que escrever no WAL antes dos dados?

Eu registraria informação suficiente de recuperação antes que páginas alteradas possam chegar ao armazenamento. Após queda, o sistema reconstrói mudanças necessárias; a ordem entre log e dados é parte da garantia, não detalhe opcional.

Termo para lembrar: Write-ahead logging | Base técnica: referência 1.

02. Fsync significa gravar qualquer dado do processo?

Eu verificaria quais arquivos e garantias do sistema operacional e dispositivo estão envolvidos. Fsync solicita persistência conforme contrato da plataforma; cache de hardware e configuração incorreta podem comprometer a hipótese de durabilidade assumida.

Termo para lembrar: Durability boundary | Base técnica: referência 1.

03. Pode responder antes de flush?

Eu só faria isso quando o contrato aceita perder operações recentes após falha. Resposta antecipada reduz latência, mas muda a promessa ao usuário; não apresentaria esse ganho como otimização gratuita de um ledger durável.

Termo para lembrar: Acknowledgement policy | Base técnica: referência 1.

04. O que é group commit?

Eu agruparia confirmações de várias transações num trabalho de sincronização compartilhado, amortizando custo físico. Isso pode melhorar throughput sem remover durabilidade, mas introduz espera de agrupamento e depende do comportamento da implementação.

Termo para lembrar: Group commit | Base técnica: referência 1.

05. Checkpoint substitui WAL?

Eu usaria checkpoint para estabelecer uma base de recuperação e limitar trabalho de replay. O log continua necessário para mudanças posteriores; frequência afeta I/O e tempo de recuperação, exigindo equilíbrio com a carga online.

Termo para lembrar: Checkpoint | Base técnica: referência 1.

06. O que acontece após crash?

Eu partiria do estado consistente e reaplicaria registros necessários conforme o engine, descartando transações não confirmadas quando apropriado. A aplicação ainda precisa tratar resposta perdida: recuperação do banco não informa automaticamente ao cliente seu resultado.

Termo para lembrar: Crash recovery | Base técnica: referência 1.

07. Por que páginas sujas não são gravadas imediatamente?

Eu manteria alterações em memória para agrupar trabalho e aproveitar localidade, respeitando a ordem de durabilidade do log. Escrever cada página a cada mudança aumentaria I/O; gerenciamento de buffers controla quando descarregar com segurança.

Termo para lembrar: Buffer pool | Base técnica: referência 1.

08. WAL cresce indefinidamente?

Eu definiria condições de reciclagem ligadas a checkpoints, backups e consumidores de replicação. Um slot atrasado pode reter log; monitoramento de espaço e estratégia de recuperação são necessários para evitar indisponibilidade por disco cheio.

Termo para lembrar: Log retention | Base técnica: referência 1.

09. Como funciona uma LSM?

Eu acumularia alterações em estrutura de memória e depois produziria arquivos ordenados imutáveis, conforme o engine. Leituras podem consultar várias camadas; compaction reduz esse custo e elimina versões quando for seguro fazê-lo.

Termo para lembrar: Log-structured merge tree | Base técnica: referência 1.

10. Memtable e WAL têm o mesmo papel?

Eu usaria memtable para acesso eficiente ao estado recente e WAL para recuperação durável, conforme configuração. Se o processo cair, a estrutura em memória desaparece; o log permite reconstruí-la quando a escrita foi confirmada adequadamente.

Termo para lembrar: Memtable versus WAL | Base técnica: referência 1.

11. O que é SSTable?

Eu explicaria como arquivo ordenado usado por engines LSM para armazenar pares e metadados de busca. Imutabilidade simplifica leitura e criação, mas atualizações geram novas versões que precisam ser reconciliadas e compactadas posteriormente.

Termo para lembrar: Sorted string table | Base técnica: referência 1.

12. Por que compaction é necessária?

Eu reorganizaria arquivos e versões para limitar busca e recuperar espaço, respeitando snapshots e exclusões. Sem manutenção suficiente, a ingestão inicial parece rápida, mas leituras, armazenamento e pausas de escrita podem piorar progressivamente.

Termo para lembrar: Compaction | Base técnica: referência 1.

13. O que é write amplification?

Eu compararia bytes físicos escritos ao volume lógico recebido, incluindo log e reorganizações. Uma atualização pode ser reescrita várias vezes; esse fator influencia SSD, custo e capacidade, mesmo quando o payload da aplicação é pequeno.

Termo para lembrar: Write amplification | Base técnica: referência 1.

14. O que é read amplification?

Eu contaria trabalho adicional para encontrar um valor entre índices, arquivos e versões. Filtros e cache ajudam reduzir acessos, mas a estratégia de compaction e distribuição dos dados determinam o custo sustentado das consultas.

Termo para lembrar: Read amplification | Base técnica: referência 1.

15. O que é space amplification?

Eu consideraria espaço ocupado por versões, arquivos sobrepostos e trabalho temporário comparado aos dados vivos. Compactar pode exigir folga para novos arquivos; operar quase sem espaço pode impedir a manutenção necessária para liberar armazenamento.

Termo para lembrar: Space amplification | Base técnica: referência 1.

16. Leveled e tiered compaction diferem como?

Eu compararia a preferência por menor sobreposição de leitura versus menor reescrita, conforme implementação. A escolha afeta amplificação e espaço; mediria o mix real de inserts, updates e scans antes de decidir.

Termo para lembrar: Compaction strategy | Base técnica: referência 1.

17. Como tombstones funcionam?

Eu registraria exclusão como informação que precisa sobreviver até ser seguro remover versões antigas. Eliminar cedo demais pode ressuscitar dados em réplicas ou arquivos atrasados; retenção e reparo precisam ser coerentes com essa regra.

Termo para lembrar: Tombstone | Base técnica: referência 1.

18. Como Bloom filter ajuda LSM?

Eu usaria o filtro para evitar ler arquivos que certamente não contêm a chave, aceitando falsos positivos. Ele reduz I/O de buscas negativas, mas não substitui a leitura autoritativa nem resolve atualização e compaction sozinho.

Termo para lembrar: Bloom filter | Base técnica: referência 1.

19. Como snapshots afetam limpeza?

Eu preservaria versões necessárias a leitores ou pontos de recuperação ativos. Um snapshot longo pode impedir remoção e aumentar espaço; a política de duração precisa considerar impacto no engine, mesmo quando a consulta parece inofensiva.

Termo para lembrar: Version retention | Base técnica: referência 1.

20. O que causa write stall?

Eu investigaria manutenção que não acompanha ingestão, como excesso de arquivos ou buffers aguardando flush. A limitação protege o engine; aumentar produtores sem ampliar capacidade de manutenção pode piorar latência e prolongar a pressão.

Termo para lembrar: Write stall | Base técnica: referência 1.

21. Como dimensiona SSD?

Eu mediria IOPS, largura de banda, latência de sincronização e amplificação sustentada, não apenas capacidade nominal. Mistura de leitura, compaction e checkpoint muda o perfil; desempenho de pico do fornecedor não garante o workload inteiro.

Termo para lembrar: Storage throughput | Base técnica: referência 1.

22. Como cache do sistema operacional interfere?

Eu consideraria dados e índices já residentes, evitando comparar engines com aquecimento desigual. Cache acelera leitura, mas confirmação de durabilidade exige mecanismos apropriados; estar na memória do sistema não equivale a sobreviver à energia perdida.

Termo para lembrar: Page cache | Base técnica: referência 1.

23. B-tree sempre faz escrita aleatória?

Eu evitaria simplificação absoluta: buffers, WAL e padrões de chave alteram o comportamento físico. B-tree e LSM possuem vantagens por workload; compararia regime estável, consultas e manutenção, não apenas slogans sobre escrita sequencial.

Termo para lembrar: Storage layout nuance | Base técnica: referência 1.

24. Como chaves sequenciais afetam B-tree?

Eu esperaria maior localidade em certas inserções, mas também possível contenção em páginas finais. Chaves aleatórias espalham trabalho e podem ampliar cache misses; a escolha de identidade precisa considerar semântica e desempenho conjuntamente.

Termo para lembrar: Page locality | Base técnica: referência 1.

25. Como testa durabilidade?

Eu usaria testes controlados de crash e recuperação, verificando quais confirmações sobrevivem conforme o contrato. Reiniciar apenas a aplicação não cobre falha do sistema operacional ou armazenamento; os cenários precisam corresponder à promessa feita.

Termo para lembrar: Power-loss testing | Base técnica: referência 1.

26. Como testa throughput de LSM?

Eu executaria por tempo suficiente para incluir flush e compaction, com tamanho de dados representativo. Um teste curto pode medir só buffers vazios; capacidade sustentável inclui manutenção, crescimento e consultas concorrentes do workload real.

Termo para lembrar: Steady-state benchmark | Base técnica: referência 1.

27. Como backup interage com log?

Eu manteria base consistente e sequência de logs necessária ao ponto desejado, testando restauração. Ter arquivos soltos não garante continuidade; lacunas, retenção ou incompatibilidade podem impedir recuperar exatamente a janela planejada.

Termo para lembrar: Point-in-time recovery | Base técnica: referência 1.

28. Como separa I/O de manutenção?

Eu limitaria trabalho de compaction, checkpoint e backup conforme capacidade, preservando o tráfego essencial. Limitar demais acumula dívida; o objetivo é manutenção sustentável, não apenas reduzir sua CPU durante alguns minutos.

Termo para lembrar: Background resource budget | Base técnica: referência 1.

29. Como explica latência de commit?

Eu decomporia espera por locks, WAL, flush e réplicas exigidas, medindo cada etapa. Adicionar CPU não resolve uma espera dominante de armazenamento ou rede; a otimização precisa atingir o recurso que realmente limita a confirmação.

Termo para lembrar: Durable commit latency | Base técnica: referência 1.

30. Como relaciona isso ao usuário?

Eu explicaria que o momento do ACK define o que prometemos preservar. O desenho precisa manter essa promessa durante crash e recuperação, enquanto a manutenção garante que a latência continue aceitável depois de dias de operação.

Termo para lembrar: Durability contract | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Flush por commit

Ganho. Confirma durabilidade

Custo e motivo. Custa latência

Como decidir. Quando perda recente é inaceitável

02. Group commit

Ganho. Amortiza sincronização

Custo e motivo. Adiciona espera

Como decidir. Quando há concorrência suficiente

03. Checkpoint frequente

Ganho. Reduz replay

Custo e motivo. Aumenta I/O

Como decidir. Quando RTO exige

04. Checkpoint espaçado

Ganho. Reduz manutenção imediata

Custo e motivo. Amplia recuperação

Como decidir. Quando log e RTO permitem

05. LSM

Ganho. Favorece ingestão

Custo e motivo. Exige compaction

Como decidir. Quando workload combina com o engine

06. B-tree

Ganho. Favorece certos acessos ordenados

Custo e motivo. Mantém páginas e splits

Como decidir. Quando consultas e updates justificam

07. Compaction agressiva

Ganho. Reduz leitura e espaço

Custo e motivo. Amplifica escrita

Como decidir. Quando SSD e orçamento permitem

08. Bloom maior

Ganho. Evita mais I/O

Custo e motivo. Consome memória

Como decidir. Quando buscas negativas dominam

09. Snapshot longo

Ganho. Preserva visão histórica

Custo e motivo. Retém versões

Como decidir. Quando relatório exige coerência

10. Mais memória

Ganho. Reduz acesso físico

Custo e motivo. Custa recursos e recuperação

Como decidir. Quando working set é reutilizado

Duas alternativas ao caminho proposto

1. B-tree com WAL

Quando faz sentido. Quando consultas e transações relacionais dominam

Como implementar. Usar engine maduro com índices ordenados e recuperação

Limites. Ainda exige vacuum, checkpoint e dimensionamento

2. Log append-only mais projeções

Quando faz sentido. Quando histórico é autoridade

Como implementar. Persistir eventos e reconstruir modelos de leitura

Limites. Exige controle de replay, versões e efeitos externos

Referências e alcance da evidência

1. PostgreSQL: Write-Ahead Logging

Persistência do log antes das páginas de dados e uso para recuperação.

https://www.postgresql.org/docs/current/wal-intro.html

2. RocksDB: Compaction

Estratégias de compaction e compromissos de amplificação de leitura, escrita e espaço.

https://github.com/facebook/rocksdb/wiki/Compaction

3. PostgreSQL: Log-Shipping Standby Servers

Streaming replication, standby e replicação síncrona; cenários de failover precisam de configuração específica.

https://www.postgresql.org/docs/current/warm-standby.html

Continue em: T023 · T028 · T067

T025 / C. Bancos e armazenamento

Connection pooling

Connection pooling reutiliza conexões e limita concorrência ao banco. Seu tamanho é um orçamento de trabalho simultâneo, não uma tentativa de abrir uma conexão para cada usuário.

Quando usar. Quando abertura de conexão é custosa, aplicações têm muitas instâncias ou precisamos proteger o banco contra excesso de concorrência.

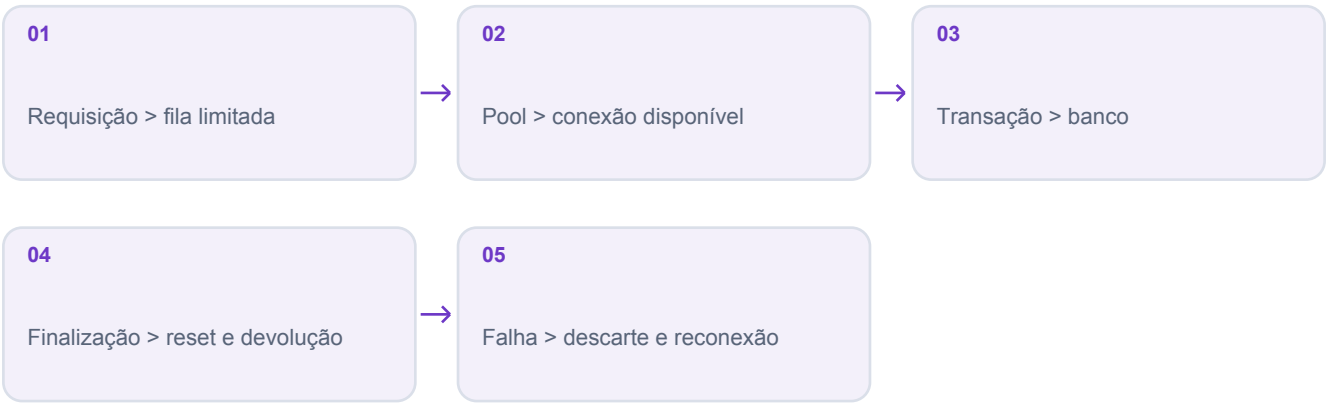
Quando evitar ou limitar. Evitar pools ilimitados, multiplicar tamanho sem considerar autoscaling ou usar pooling transacional com dependências incompatíveis de sessão.

Uma abertura possível

Vou limitar conexões conforme a capacidade do banco e medir espera pelo pool. A fila é finita e tem deadline, porque aumentar conexões além da capacidade pode reduzir throughput.

Peças e tecnologias. PgBouncer · HikariCP · RDS Proxy · JDBC · PostgreSQL

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Medir tempo de uso e capacidade do banco.

2. Dividir orçamento entre instâncias e classes.

3. Definir limites de fila, aquisição e execução.

4. Escolher modo compatível com sessão.

5. Testar leak, failover e escala da aplicação.

30 perguntas para sustentar o desenho

01. Por que usar pool?

Eu reutilizaria conexões para amortizar handshake e autenticação, limitando trabalho simultâneo no banco. O benefício não é atender infinitamente; um pool bem dimensionado protege recursos e torna a espera explícita antes da saturação.

Termo para lembrar: Connection reuse | Base técnica: referência 1.

02. Como escolhe tamanho?

Eu mediria capacidade sustentável e duração das transações, considerando todas as instâncias. Não usaria número de usuários como tamanho; cada conexão ativa consome recursos e pode aumentar contenção quando o banco já está saturado.

Termo para lembrar: Concurrency budget | Base técnica: referência 1.

03. Mais conexões aumentam throughput?

Eu testaria a curva real, porque concorrência excessiva pode aumentar locks, memória e troca de contexto. Depois do ponto ótimo, abrir mais conexões piora latência e até reduz vazão, sem criar capacidade física adicional.

Termo para lembrar: Saturation | Base técnica: referência 1.

04. Como calcula efeito do autoscaling?

Eu multiplicaria limite por instância pelo máximo de réplicas, incluindo jobs e ferramentas. Um autoscaling saudável de aplicação pode esgotar o banco; reservaria orçamento global e limites para expansão simultânea do sistema.

Termo para lembrar: Pool multiplication | Base técnica: referência 1.

05. Qual diferença entre session e transaction pooling?

Eu verificaria se a conexão física fica associada durante toda sessão ou somente durante uma transação. O modo transacional multiplexa melhor, mas funcionalidades que dependem de estado persistente de sessão exigem avaliação cuidadosa.

Termo para lembrar: Pooling mode | Base técnica: referência 1.

06. Statement pooling serve para qualquer aplicação?

Eu só consideraria quando cada comando pode funcionar isoladamente e as restrições são aceitas. Aplicações que precisam de transações com vários comandos não podem perder afinidade entre eles sem comprometer o comportamento esperado.

Termo para lembrar: Statement pooling | Base técnica: referência 1.

07. Prepared statements funcionam sempre?

Eu verificaria biblioteca, protocolo, versão e configuração do pooler, porque suporte varia. Não generalizaria que sempre quebram ou sempre funcionam; testaria a forma realmente usada pela aplicação antes de mudar o modo.

Termo para lembrar: Prepared statement compatibility | Base técnica: referência 1.

08. Como trata tabelas temporárias?

Eu identificaria dependência da mesma conexão ao longo da sessão. Num pool transacional, o próximo comando pode usar outra conexão; redesenharia o fluxo ou escolheria um modo que preserve a afinidade necessária.

Termo para lembrar: Session state | Base técnica: referência 1.

09. Como evita connection leak?

Eu garantiria fechamento em todos os caminhos com constructs apropriados e métricas de empréstimo longo. Timeouts ajudam detectar, mas não substituem corrigir código; conexões esquecidas reduzem capacidade até a fila inteira travar.

Termo para lembrar: Resource lifecycle | Base técnica: referência 1.

10. Como limpa conexão antes de devolver?

Eu finalizaria transação e removeria estado incompatível, como configurações e contexto de autorização. Reutilizar conexão suja pode vazar comportamento entre requisições; o pool e a aplicação precisam concordar sobre responsabilidade de limpeza.

Termo para lembrar: Session reset | Base técnica: referência 1.

11. Como trata tenant em sessão?

Eu evitaria confiar em estado residual de conexão, definindo contexto de forma transacional e verificável. Uma conexão reutilizada por outro cliente não pode herdar permissões; isolamento deve ser testado com alternância entre tenants.

Termo para lembrar: Tenant isolation | Base técnica: referência 1.

12. Qual timeout de aquisição?

Eu o definiria dentro do deadline da jornada, reservando tempo para a consulta. Esperar indefinidamente no pool mascara sobrecarga; quando não houver capacidade, retorno controlado evita acumular requisições e memória sem limite.

Termo para lembrar: Pool acquisition timeout | Base técnica: referência 1.

13. Timeout do pool é igual ao da query?

Eu separaria espera por conexão, execução e prazo total. Cada um informa um gargalo diferente; uma query rápida pode falhar por fila no pool, enquanto aquisição imediata não impede consulta presa em lock.

Termo para lembrar: Timeout layering | Base técnica: referência 1.

14. Como observa o pool?

Eu mediria conexões ativas, ociosas, espera, timeout e duração de uso, correlacionando com banco. Um pool cheio pode indicar transação longa, leak ou carga legítima; aumentar tamanho sem diagnóstico pode piorar a causa.

Termo para lembrar: Pool metrics | Base técnica: referência 1.

15. Como trata failover do banco?

Eu descartaria conexões quebradas e renovaria gradualmente com backoff, verificando a nova autoridade. Transações em andamento podem ter resultado desconhecido; o pool recupera transporte, mas a aplicação precisa resolver a intenção de negócio.

Termo para lembrar: Connection invalidation | Base técnica: referência 1.

16. Como evita reconnection storm?

Eu espalharia reconexões com jitter e limites, evitando todas as instâncias autenticarem simultaneamente após incidente. O banco precisa de espaço para recuperar; criar centenas de conexões ao mesmo tempo pode prolongar a indisponibilidade.

Termo para lembrar: Reconnect jitter | Base técnica: referência 1.

17. Como escolhe max lifetime?

Eu alinharia vida útil a timeouts de infraestrutura e rotação de credenciais, adicionando dispersão quando suportado. Conexões não devem morrer todas juntas; rotação precisa preservar transações em andamento e capacidade mínima disponível.

Termo para lembrar: Connection rotation | Base técnica: referência 1.

18. Health check de conexão basta?

Eu usaria validação apropriada, mas não consideraria um ping prova de que a próxima transação vai funcionar. A rede pode falhar imediatamente depois; cada operação continua precisando de tratamento de erro e resultado desconhecido.

Termo para lembrar: Liveness validation | Base técnica: referência 1.

19. Como usa proxy gerenciado?

Eu avaliaria multiplexação, autenticação e integração de failover, verificando limitações de pinning e sessão. Proxy reduz parte da operação, mas não remove orçamento de conexões nem torna consultas lentas eficientes automaticamente.

Termo para lembrar: Connection proxy | Base técnica: referência 1.

20. Como controla serverless?

Eu limitaria concorrência das funções e usaria mecanismos adequados de pooling ou proxy. Mil invocações simultâneas podem abrir conexões demais; elasticidade do compute precisa ser compatível com a capacidade finita do banco.

Termo para lembrar: Burst connection load | Base técnica: referência 1.

21. Como separa jobs de APIs?

Eu reservaria pools ou cotas por classe para tarefas em lote não consumirem todas as conexões interativas. A soma respeita o orçamento global; separar nomes de pools sem limites efetivos não oferece isolamento real.

Termo para lembrar: Workload pools | Base técnica: referência 1.

22. Como trata idle in transaction?

Eu identificaria sessões que mantêm transação aberta enquanto não executam, pois retêm locks e versões. Configuraria limites e corrigiria o fluxo; devolver conexão ao pool sem rollback pode propagar o problema.

Termo para lembrar: Transaction leak | Base técnica: referência 1.

23. Como dimensiona memória do banco?

Eu consideraria estruturas por conexão e operações que alocam memória por sort ou hash. O risco depende de consultas simultâneas; configurar memória alta multiplicada por muitas conexões pode esgotar o servidor rapidamente.

Termo para lembrar: Per-connection cost | Base técnica: referência 1.

24. Pode compartilhar uma conexão entre threads?

Eu seguiria o contrato do driver e evitaria uso concorrente indevido da mesma sessão. Uma conexão representa estado transacional; intercalar comandos de solicitações diferentes pode misturar operações mesmo quando o pool em si é thread-safe.

Termo para lembrar: Connection ownership | Base técnica: referência 1.

25. Como trata transação longa?

Eu removeria espera de rede e trabalho humano do período em que a conexão está emprestada. Quanto maior o tempo de posse, mais conexões precisamos para a mesma chegada; reduzir duração costuma ser melhor que ampliar o pool.

Termo para lembrar: Connection hold time | Base técnica: referência 1.

26. Como usa pooling em leitura e escrita?

Eu separaria destinos e garantias, direcionando leituras que toleram lag às réplicas. Após escrita sensível, o caminho pode exigir primário; o roteamento não deve sacrificar consistência silenciosamente para aliviar um pool cheio.

Termo para lembrar: Read/write routing | Base técnica: referência 1.

27. Como testa tamanho do pool?

Eu variaria concorrência sob workload representativo, observando throughput, cauda, espera e recursos do banco. Escolheria a faixa estável com margem, não o maior número que aceita conexões num teste sem carga real.

Termo para lembrar: Load sweep | Base técnica: referência 1.

28. Como usa Little's Law no pool?

Eu estimaria conexões ocupadas pela taxa de transações vezes tempo médio de posse, em condição estável. A conta orienta, mas variância, picos e locks exigem margem e medição; não substitui o teste de saturação.

Termo para lembrar: Queueing estimate | Base técnica: referência 1.

29. Como faz rotação de senha?

Eu prepararia período de coexistência ou mecanismo suportado, renovando conexões gradualmente e validando autenticação. Alterar o segredo sem renovar pools pode deixar parte da frota funcionando até falhar de maneira desigual depois.

Termo para lembrar: Credential rotation | Base técnica: referência 1.

30. Como explica pooling em Staff?

Eu apresentaria o pool como proteção de capacidade e ciclo de recursos, incluindo falhas e sessão. O desenho mostra onde a espera acontece, quando recusamos e como retomamos sem confundir reconexão com repetição segura da transação.

Termo para lembrar: Admission boundary | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Pool maior

Ganho. Reduz espera inicial

Custo e motivo. Aumenta contenção

Como decidir. Quando banco tem capacidade

02. Pool menor

Ganho. Protege banco

Custo e motivo. Pode limitar throughput

Como decidir. Quando recurso é escasso

03. Session pooling

Ganho. Preserva funcionalidades

Custo e motivo. Multiplexa menos

Como decidir. Quando sessão mantém estado

04. Transaction pooling

Ganho. Reutiliza melhor

Custo e motivo. Restringe afinidade

Como decidir. Quando aplicação é compatível

05. Proxy

Ganho. Centraliza conexões

Custo e motivo. Adiciona dependência

Como decidir. Quando muitas instâncias acessam banco

06. Fila longa

Ganho. Absorve pico

Custo e motivo. Aumenta latência e memória

Como decidir. Quando prazo permite

07. Timeout curto

Ganho. Falha rapidamente

Custo e motivo. Pode recusar picos breves

Como decidir. Quando UX exige resposta rápida

08. Pools separados

Ganho. Isolam classes

Custo e motivo. Fragmentam orçamento

Como decidir. Quando batch ameaça APIs

09. Rotação frequente

Ganho. Renova credenciais

Custo e motivo. Custa handshakes

Como decidir. Quando política de segurança exige

10. Validação ativa

Ganho. Detecta conexão morta

Custo e motivo. Adiciona chamadas

Como decidir. Quando ambiente mantém conexões ociosas longas

Duas alternativas ao caminho proposto

1. Proxy gerenciado

Quando faz sentido. Quando há grande número de clientes efêmeros

Como implementar. Multiplexar conexões com integração de autenticação e failover

Limites. Verificar pinning, transações e limites do serviço

2. Fila de comandos

Quando faz sentido. Quando excesso de escrita pode esperar

Como implementar. Persistir solicitações e limitar workers que usam banco

Limites. Muda contrato para conclusão assíncrona e exige acompanhamento

Referências e alcance da evidência

1. PgBouncer: Features

Modos session, transaction e statement pooling e restrições de funcionalidades de sessão.

https://www.pgbouncer.org/features.html

2. PostgreSQL: Transaction isolation

Isolamento, snapshots e necessidade de repetição de transações sob conflitos.

https://www.postgresql.org/docs/current/transaction-iso.html

3. Google SRE: Handling Overload

Sobrecarga, filas e rejeição de trabalho; números e dimensionamentos são hipóteses didáticas.

https://sre.google/sre-book/handling-overload/

Continue em: T003 · T022 · T079

T026 / C. Bancos e armazenamento

DynamoDB

DynamoDB é um banco gerenciado de chave-valor e documentos cuja modelagem depende dos padrões de acesso. Chaves, índices, condições e capacidade determinam desempenho e garantias de cada operação.

Quando usar. Para acessos previsíveis por chave, escala gerenciada e operações condicionais ou transacionais dentro do escopo suportado.

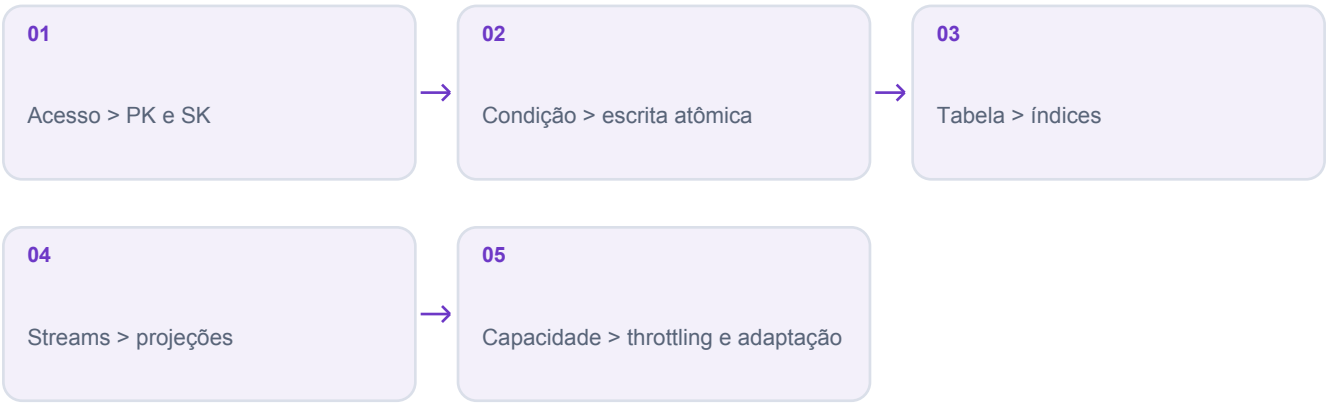
Quando evitar ou limitar. Evitar scans como estratégia principal, partições ilimitadas por tenant e suposição de que índices e regiões compartilham toda garantia da tabela base.

Uma abertura possível

Vou desenhar primeiro as consultas e chaves. Depois protejo invariantes com condições ou transações, verificando limites e custo, sem depender de consistência eventual para autorizar uma regra que exige coordenação.

Peças e tecnologias. DynamoDB · Partition key · Sort key · GSI · TransactWriteItems · DynamoDB Streams

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Listar padrões de acesso e cardinalidade.

2. Definir PK, SK e distribuição.

3. Escolher índices e consistência.

4. Implementar condições, transações e retries.

5. Testar hot keys, custo e recuperação.

30 perguntas para sustentar o desenho

01. Como escolhe partition key?

Eu buscaria distribuição de atividade e acesso direto, considerando também os índices. Uma chave de baixa cardinalidade, como status, pode concentrar tráfego; clientes muito grandes exigem análise além da média de uso.

Termo para lembrar: Partition key | Base técnica: referência 1.

02. Para que serve sort key?

Eu a usaria para organizar itens da mesma partição lógica e permitir consultas por intervalo ou prefixo conforme o modelo. Isso ajuda históricos, mas a coleção não pode crescer sem considerar tamanho, acesso e retenção.

Termo para lembrar: Sort key | Base técnica: referência 1.

03. Quando usa single-table design?

Eu agruparia entidades quando os acessos relacionados e a operação da equipe justificarem. Isso pode reduzir viagens, mas aumenta disciplina de chaves e evolução; não é requisito universal para usar DynamoDB corretamente.

Termo para lembrar: Single-table design | Base técnica: referência 1.

04. Query e Scan diferem como?

Eu usaria Query com chave e condições suportadas para um conjunto conhecido. Scan percorre dados amplamente e pode consumir muita capacidade; filtrar depois não transforma varredura numa busca eficiente por índice inexistente.

Termo para lembrar: Access pattern | Base técnica: referência 1.

05. FilterExpression reduz capacidade lida?

Eu lembraria que o filtro é aplicado após leitura dos candidatos conforme a operação. Uma resposta pequena pode ter consumido bastante trabalho; modelaria chave e índice para reduzir o conjunto lido antes da filtragem.

Termo para lembrar: Post-read filtering | Base técnica: referência 1.

06. Como protege saldo ou reserva?

Eu colocaria a condição necessária na atualização atômica, verificando falha de condição. Ler e depois escrever separadamente permite corrida; para regras entre itens, avaliaria transação suportada e seus limites concretos.

Termo para lembrar: Conditional write | Base técnica: referência 1.

07. Como implementa idempotência?

Eu usaria identidade estável e condição de ausência, ligando o registro ao efeito local por transação quando necessário. A retenção precisa cobrir o horizonte de retry; expirar cedo pode permitir executar novamente a mesma intenção.

Termo para lembrar: Idempotency record | Base técnica: referência 1.

08. TransactWriteItems resolve qualquer operação distribuída?

Eu verificaria itens, região, conta e limites documentados da API. A atomicidade não inclui Kafka ou parceiro externo; para publicação e efeitos, persistiria obrigação e manteria recuperação idempotente fora dessa fronteira local.

Termo para lembrar: Transaction scope | Base técnica: referência 1.

09. Como trata conflito transacional?

Eu distinguiria condição de negócio rejeitada de conflito transitório, repetindo somente quando apropriado. O retry usa novas leituras e prazo limitado; repetir uma condição impossível apenas desperdiça capacidade e piora latência da aplicação.

Termo para lembrar: Transaction conflict | Base técnica: referência 1.

10. Leitura forte funciona em GSI?

Eu verificaria o caminho: GSI oferece leitura eventual conforme a documentação, enquanto tabela e LSI têm opções diferentes. Se a jornada exige ver uma escrita imediatamente, consultaria a chave autoritativa adequada.

Termo para lembrar: Secondary-index consistency | Base técnica: referência 1.

11. Como usa GSI?

Eu modelaria outra chave para um padrão de acesso importante, considerando projeção e custo de manutenção. O índice tem seu próprio risco de concentração; distribuir bem a tabela não garante que o GSI esteja equilibrado.

Termo para lembrar: Global secondary index | Base técnica: referência 1.

12. Como usa índice esparso?

Eu incluiria no índice apenas itens que possuem os atributos necessários, como tarefas ainda pendentes. Isso reduz dados consultados, mas a atualização do estado e remoção dos atributos precisa manter o modelo correto.

Termo para lembrar: Sparse index | Base técnica: referência 1.

13. Como evita partição quente?

Eu mediria atividade por chave, usando buckets ou sharding de escrita quando consultas permitirem. Adaptive capacity ajuda em condições suportadas, mas não torna uma chave ilimitada; divisão também acrescenta custo de leitura e agregação.

Termo para lembrar: Hot partition | Base técnica: referência 1.

14. Como decide número de buckets?

Eu estimaria pico por entidade e capacidade alvo, mantendo estratégia evolutiva. Mais buckets distribuem escrita, mas consultas precisam reuni-los; escolheria o menor número que resolve a concentração com margem operacional adequada.

Termo para lembrar: Write sharding | Base técnica: referência 1.

15. On-demand elimina planejamento?

Eu usaria elasticidade gerenciada, mas ainda avaliaria quotas, picos súbitos, chaves quentes e custo. O modo de cobrança não elimina limites de recurso nem torna um scan grande aceitável para uma jornada interativa.

Termo para lembrar: Capacity mode | Base técnica: referência 1.

16. Como compara provisioned e on-demand?

Eu compararia previsibilidade de tráfego, utilização e esforço de gestão, usando preços vigentes no cálculo real. Capacidade provisionada pode ser eficiente sob carga estável; variabilidade alta pode favorecer outra escolha sem mudar o modelo.

Termo para lembrar: Capacity economics | Base técnica: referência 1.

17. Como trata throttling?

Eu reduziria ritmo e repetiria operações transitórias com jitter e limite, respeitando o prazo. Também investigaria chave quente e configuração; retry não corrige indefinidamente um padrão de acesso que excede a capacidade disponível.

Termo para lembrar: Backoff and jitter | Base técnica: referência 1.

18. BatchWrite é transacional?

Eu não presumiria atomicidade do lote. Trataria itens não processados conforme a resposta e preservaria identidade; quando a regra exige tudo ou nada, usaria a API transacional adequada em vez de confundir batching com transação.

Termo para lembrar: Batch semantics | Base técnica: referência 1.

19. Como pagina resultados?

Eu preservaria a chave de continuação retornada, mantendo parâmetros e escopo de autorização. Uma página pode conter poucos resultados após filtro e ainda ter continuação; não concluiria fim apenas pela quantidade de itens devolvida.

Termo para lembrar: Continuation key | Base técnica: referência 1.

20. TTL remove exatamente no vencimento?

Eu trataria TTL como limpeza assíncrona, não como relógio preciso de autorização. A aplicação verifica o instante de expiração antes de usar o item; registro expirado pode continuar presente por algum tempo.

Termo para lembrar: TTL semantics | Base técnica: referência 1.

21. Como usa Streams?

Eu consumiria mudanças com processamento idempotente e monitoramento de atraso, respeitando retenção e ordenação documentadas. Stream não substitui histórico permanente; se o consumidor ficar parado além da janela, preciso de recuperação planejada.

Termo para lembrar: Change stream | Base técnica: referência 1.

22. Streams dão ordem global?

Eu verificaria a ordem oferecida por item e shard, sem prometer sequência global entre entidades independentes. Se o domínio exige causalidade cruzada, incluo versão e dependências ou coordenação apropriada no modelo de aplicação.

Termo para lembrar: Ordering scope | Base técnica: referência 1.

23. Como usa global tables?

Eu escolheria explicitamente o modo de consistência e regiões suportadas, verificando o contrato vigente. Não assumiria que toda tabela global usa a mesma resolução; invariantes e transações precisam ser avaliadas para a configuração concreta.

Termo para lembrar: Global consistency mode | Base técnica: referência 1.

24. Como trata atributos grandes?

Eu manteria itens dentro dos limites e armazenaria objetos grandes fora, com referência e checksum. Tamanho influencia capacidade e custo; atualização de um documento muito amplo pode ser cara mesmo quando apenas um campo muda.

Termo para lembrar: Item sizing | Base técnica: referência 1.

25. Como modela histórico temporal?

Eu usaria entidade e buckets de período quando o volume exigir, ordenando por instante e desempate. Consultas entre períodos precisam de composição; retenção e limites do maior cliente determinam a granularidade adequada.

Termo para lembrar: Time buckets | Base técnica: referência 1.

26. Como controla acesso por tenant?

Eu incluiria tenant no modelo e aplicaria autorização no backend, verificando chaves e índices. Prefixo de chave ajuda organização, mas não substitui política; endpoints não podem permitir que o cliente consulte outro tenant arbitrariamente.

Termo para lembrar: Tenant keying | Base técnica: referência 1.

27. Como migra chave primária?

Eu planejaria novo modelo e cópia com captura de mudanças, mantendo uma autoridade de escrita até o corte. Alterar chave é uma migração de dados e consumidores; não trataria como simples renomeação de coluna.

Termo para lembrar: Key migration | Base técnica: referência 1.

28. Como testa custo real?

Eu mediria capacidade consumida por acesso com tamanhos e índices representativos, incluindo retries e transações. Um exemplo com item minúsculo pode esconder custo dominante; avaliaria também armazenamento, streams e tráfego regional conforme uso.

Termo para lembrar: Consumed capacity | Base técnica: referência 1.

29. Como faz backup e restauração?

Eu verificaria recursos de recuperação e ensaiaria restauração, incluindo reconstrução de consumidores e índices. Restaurar dados não desfaz efeitos externos; operações pendentes precisam de identidade e reconciliação antes de serem executadas novamente.

Termo para lembrar: Recoverability | Base técnica: referência 1.

30. Como defenderia DynamoDB contra PostgreSQL?

Eu mostraria acessos previsíveis e escala que se beneficiam do serviço gerenciado, reconhecendo consultas menos flexíveis e custos de modelagem. Se regras e consultas mudam frequentemente, um relacional pode ser uma escolha inicial mais simples.

Termo para lembrar: Workload comparison | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. On-demand

Ganho. Simplifica capacidade

Custo e motivo. Pode custar mais

Como decidir. Quando carga é variável

02. Provisioned

Ganho. Permite planejamento econômico

Custo e motivo. Exige ajuste

Como decidir. Quando tráfego é previsível

03. GSI

Ganho. Abre outro acesso

Custo e motivo. Amplia escrita e lag

Como decidir. Quando consulta é recorrente

04. Single-table

Ganho. Agrupa acessos

Custo e motivo. Complica evolução

Como decidir. Quando padrões são estáveis

05. Buckets

Ganho. Distribuem carga

Custo e motivo. Exigem fan-out

Como decidir. Quando uma chave é quente

06. Leitura forte

Ganho. Reduz atraso observável

Custo e motivo. Custa mais capacidade

Como decidir. Quando caminho suporta e exige

07. Leitura eventual

Ganho. Reduz custo

Custo e motivo. Pode atrasar

Como decidir. Quando UX tolera

08. Transação

Ganho. Protege vários itens

Custo e motivo. Aumenta coordenação

Como decidir. Quando invariante cruza itens

09. TTL

Ganho. Simplifica limpeza

Custo e motivo. Não vence instantaneamente

Como decidir. Quando aplicação verifica validade

10. Objeto externo

Ganho. Reduz item

Custo e motivo. Cria duas etapas

Como decidir. Quando conteúdo é grande

Duas alternativas ao caminho proposto

1. PostgreSQL

Quando faz sentido. Quando relações e consultas flexíveis dominam

Como implementar. Usar transações, constraints e índices conforme workload

Limites. Exige dimensionamento e operação de escala

2. Cassandra

Quando faz sentido. Quando distribuição e acesso previsível em larga escala justificam

Como implementar. Modelar tabelas por consulta e operar replicação adequada

Limites. Aumenta responsabilidade operacional e exige cuidado com consistência

Referências e alcance da evidência

1. AWS: DynamoDB partition keys

Distribuição de chaves e capacidade por partição; limites devem ser revistos na documentação vigente.

https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/bp-partition-key-design.html

2. AWS: DynamoDB transactions

APIs transacionais, condições, conflitos e limites de escopo.

https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/transaction-apis.html

3. AWS: DynamoDB read consistency

Consistência de tabela, LSI e GSI e diferenças por caminho de leitura.

https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.ReadConsistency.html

4. AWS: DynamoDB global tables

Replicação multi-região e modos de consistência documentados; capacidades dependem da configuração escolhida.

https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/GlobalTables.html

5. AWS: DynamoDB TTL

Expiração e remoção assíncrona de itens.

https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/TTL.html

Continue em: T015 · T017 · T021

T027 / C. Bancos e armazenamento

Redis: estruturas, Cluster e persistência

Redis oferece estruturas de dados e execução de comandos de baixa latência. Cluster distribui slots; replicação e persistência têm garantias específicas que precisam ser separadas da atomicidade de comandos.

Quando usar. Para cache, sessões, contadores, rate limiting e coordenação compatível com suas garantias.

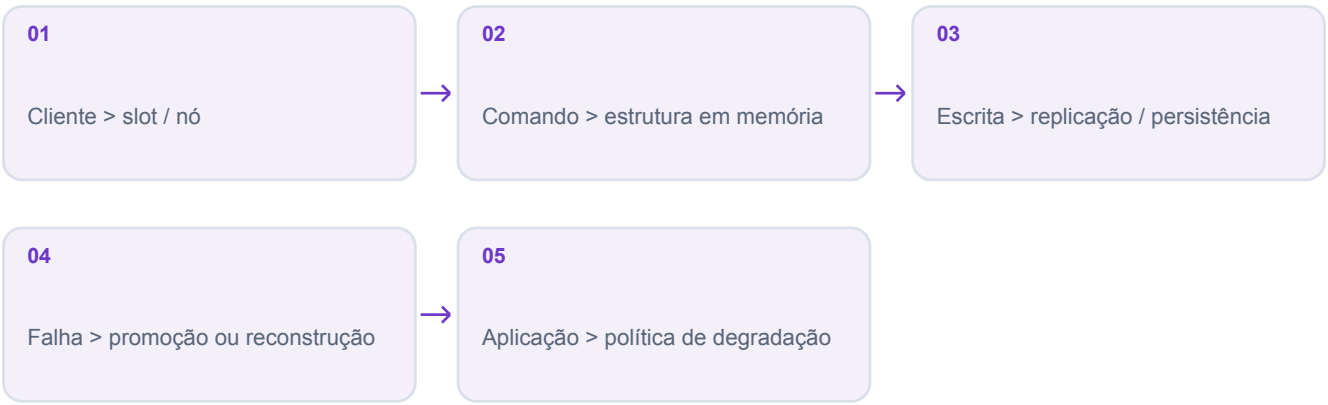
Quando evitar ou limitar. Evitar tratá-lo como ledger por ser rápido, depender de Pub/Sub para trabalho durável ou executar comandos ilimitados em chaves grandes.

Uma abertura possível

Vou escolher a estrutura pelo acesso e limitar tamanho e custo de comandos. Se Redis falhar, preciso saber se reconstruo dados ou se a aplicação assumiu uma durabilidade que a configuração não oferece.

Peças e tecnologias. Redis strings · Hashes · Sorted sets · Streams · Redis Cluster · AOF · RDB

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Classificar dado como autoridade ou acelerador.

2. Escolher estrutura e complexidade dos comandos.

3. Definir slots, tamanho e memória.

4. Configurar persistência e recuperação.

5. Testar hot key, eviction e falha do cluster.

30 perguntas para sustentar o desenho

01. Quando usa string?

Eu usaria para valor simples serializado ou contador compatível com comandos atômicos. Limitaria tamanho e TTL conforme o uso; guardar um documento enorme pode aumentar latência de rede e bloquear processamento por tempo relevante.

Termo para lembrar: Redis string | Base técnica: referência 1.

02. Quando usa hash?

Eu agruparia campos de uma entidade quando atualização parcial e acesso localizado ajudarem. Isso reduz algumas serializações, mas preciso considerar tamanho e expiração disponível na versão usada; estrutura compacta não significa crescimento ilimitado seguro.

Termo para lembrar: Redis hash | Base técnica: referência 1.

03. Quando usa sorted set?

Eu o escolheria para ranking, prazos ou prioridade com score, definindo desempate e limites. Operações sobre intervalos grandes ainda custam trabalho e bytes; a estrutura ordenada não autoriza retornar milhões de elementos numa chamada.

Termo para lembrar: Sorted set | Base técnica: referência 1.

04. Quando usa set?

Eu usaria para associação única, interseção ou presença quando o volume cabe em memória. Compararia com Bloom filter se falsos positivos forem aceitáveis; conjuntos exatos oferecem precisão, mas armazenam membros e consomem mais espaço.

Termo para lembrar: Set membership | Base técnica: referência 1.

05. Redis é sempre O(1)?

Eu verificaria a complexidade de cada comando e tamanho das estruturas. Algumas operações percorrem coleções ou ordenam resultados; latência baixa depende de comandos limitados, memória suficiente e ausência de trabalho que monopoliza o nó.

Termo para lembrar: Command complexity | Base técnica: referência 1.

06. Por que evitar KEYS em produção?

Eu evitaria varredura ampla que pode atrasar outros comandos, preferindo SCAN controlado quando apropriado. Mesmo SCAN exige limites e semântica de iteração; não o trataria como snapshot consistente durante alterações concorrentes.

Termo para lembrar: Blocking scan | Base técnica: referência 1.

07. Pipeline é transação?

Eu usaria pipeline para reduzir viagens de rede, não para garantir atomicidade conjunta. Comandos podem ter resultados individuais e falhas; quando a regra exige decisão indivisível, preciso de mecanismo adequado como script ou transação compatível.

Termo para lembrar: Pipelining | Base técnica: referência 1.

08. MULTI/EXEC equivale a SQL?

Eu verificaria semântica de execução e erros, sem esperar rollback relacional de todo comando que falha durante execução. A aplicação deve validar entradas e entender o comportamento real; atomicidade de execução não é uma transação SQL genérica.

Termo para lembrar: Redis transaction | Base técnica: referência 1.

09. Quando usa WATCH?

Eu observaria chaves para detectar mudanças antes do EXEC, repetindo a decisão quando necessário. Isso ajuda concorrência otimista, mas pode abortar muito sob contenção; o retry deve reler estado e respeitar o prazo da operação.

Termo para lembrar: Optimistic transaction | Base técnica: referência 1.

10. Quando usa Lua?

Eu combinaria operações curtas numa execução atômica no escopo suportado, evitando viagens extras e corridas locais. Scripts longos bloqueiam trabalho do nó; limito complexidade e não faço deles um motor de processamento pesado.

Termo para lembrar: Atomic script | Base técnica: referência 1.

11. Como Cluster distribui dados?

Eu explicaria que chaves são mapeadas para slots distribuídos entre nós, e clientes precisam entender redirecionamentos. Distribuição de slots não garante carga uniforme, porque algumas chaves podem concentrar quase todo o tráfego.

Termo para lembrar: Hash slots | Base técnica: referência 1.

12. Para que servem hash tags?

Eu colocaria chaves relacionadas no mesmo slot quando uma operação multichave exigir essa localidade. Isso facilita atomicidade suportada, mas pode criar hotspot; não agruparia todos os clientes no mesmo hash tag por conveniência.

Termo para lembrar: Key co-location | Base técnica: referência 1.

13. Pode transacionar entre slots?

Eu verificaria os limites do Redis Cluster e redesenharia a regra quando chaves não podem ser coordenadas numa operação suportada. Espalhar dados para escala e exigir atomicidade global simultaneamente requer outro mecanismo de consistência.

Termo para lembrar: Cross-slot constraint | Base técnica: referência 1.

14. Como trata MOVED e ASK?

Eu usaria cliente compatível com atualização do mapa e redirecionamento durante migração, com limites de retry. Ignorar esses sinais provoca erros ou loops; mudanças de slots fazem parte da operação normal do cluster.

Termo para lembrar: Cluster redirection | Base técnica: referência 1.

15. RDB e AOF diferem como?

Eu compararia snapshot periódico com registro de operações, considerando janela de perda e tempo de recuperação. Configuração e fsync mudam a garantia; escolher persistência exige conectar custo de I/O ao que a aplicação promete preservar.

Termo para lembrar: Persistence mode | Base técnica: referência 1.

16. Replicação elimina perda de escrita?

Eu verificaria confirmação e atraso antes de prometer sobrevivência a falhas. Replicação assíncrona pode perder alterações recentes na promoção; presença de réplica não torna todo ACK equivalente a um commit distribuído durável.

Termo para lembrar: Replication durability | Base técnica: referência 1.

17. Redis pode ser fonte da verdade?

Eu só o consideraria após validar durabilidade, recuperação e invariantes para o caso concreto. Para cache, perda é reconstruível; para dinheiro, não basta velocidade ou atomicidade de incremento sem um contrato financeiro completo.

Termo para lembrar: Authoritative use | Base técnica: referência 1.

18. Como trata hot key?

Eu mediria tráfego por chave e consideraria cache local, réplica de leitura ou divisão semântica quando possível. Adicionar shards não divide automaticamente uma única chave; a regra de atualização pode continuar serializada num nó.

Termo para lembrar: Hot key | Base técnica: referência 1.

19. Como trata big key?

Eu limitaria cardinalidade e bytes, dividindo estruturas por período ou página quando o acesso permitir. Operações e exclusão de valores gigantes podem gerar pausas; planejo limpeza gradual e monitoro tamanho antes do incidente.

Termo para lembrar: Big key | Base técnica: referência 1.

20. O que faz quando memória acaba?

Eu configuraria limite e política de eviction conforme o papel do dado, reservando overhead. Expulsar chaves pode ser aceitável para cache e perigoso para locks ou dedupe; misturar responsabilidades no mesmo cluster exige cuidado.

Termo para lembrar: Memory policy | Base técnica: referência 1.

21. Pub/Sub é fila durável?

Eu o usaria para sinais efêmeros quando perda é aceitável, não para obrigação financeira. Consumidor desconectado pode perder mensagens; histórico e processamento recuperável exigem armazenamento ou mecanismo de stream com semântica apropriada.

Termo para lembrar: Ephemeral messaging | Base técnica: referência 1.

22. Como usa Streams?

Eu definiria grupo, confirmação, pendências e recuperação de consumidores, limitando retenção conscientemente. Stream fornece recursos diferentes de Pub/Sub, mas a aplicação ainda precisa de idempotência, monitoramento e política de durabilidade do Redis usado.

Termo para lembrar: Redis Streams | Base técnica: referência 1.

23. Como faz rate limiter?

Eu escolheria algoritmo e executaria atualização de contador e expiração atomicamente no escopo correto. A política durante falha é explícita; fail-open pode ser aceitável para proteção leve e inadequado para limite financeiro rígido.

Termo para lembrar: Atomic rate limiting | Base técnica: referência 1.

24. Como usa Redis para lock?

Eu usaria token de propriedade, expiração e liberação condicional, reconhecendo limites de falha e pausa. Para efeitos críticos, preciso de fencing ou autoridade transacional; uma chave com TTL não impede o antigo proprietário de agir.

Termo para lembrar: Lease lock | Base técnica: referência 1.

25. Como protege acesso?

Eu restringiria rede, autenticação e permissões por capacidade, usando criptografia conforme o ambiente. Redis não deve ficar exposto sem controle; cache também pode conter informação sensível e precisa de política de retenção e observabilidade.

Termo para lembrar: Redis security | Base técnica: referência 1.

26. Como mede latência real?

Eu separaria tempo de rede, fila, execução e persistência, observando comandos lentos e tamanho de respostas. CPU média baixa não descarta pausas por comando caro, fork, memória ou contenção no host.

Termo para lembrar: Latency diagnosis | Base técnica: referência 1.

27. Como aquece cache após perda?

Eu reconstruiria progressivamente com limite de origem, priorizando chaves úteis. Liberar todas as requisições para o banco durante cache frio pode causar stampede; single-flight e admissão protegem a recuperação do sistema.

Termo para lembrar: Controlled warm-up | Base técnica: referência 1.

28. Como evita misturar sessão e cache descartável?

Eu separaria políticas e possivelmente clusters quando eviction, persistência e criticidade diferem. A mesma configuração que otimiza cache pode expulsar sessão ou dedupe; compartilhar infraestrutura exige análise de consequência e isolamento de capacidade.

Termo para lembrar: Workload separation | Base técnica: referência 1.

29. Como faz upgrade ou reshard?

Eu usaria procedimento suportado e cliente compatível, medindo erros e latência durante movimentação. Operações em trânsito podem precisar de retry seguro; manutenção precisa preservar identidade e não assumir que o cluster fica estático.

Termo para lembrar: Operational migration | Base técnica: referência 1.

30. Como defenderia Redis no desenho?

Eu nomearia o papel exato, a estrutura, o limite de tamanho e o comportamento na perda. Também explicaria por que a fonte autoritativa continua correta, evitando usar baixa latência como substituto de durabilidade ou consistência financeira.

Termo para lembrar: Role-specific guarantee | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Strings

Ganho. Simplificam payload

Custo e motivo. Reescrevem valor inteiro

Como decidir. Quando objeto é pequeno

02. Hashes

Ganho. Permitem campos separados

Custo e motivo. Complicam lifecycle

Como decidir. Quando atualizações são parciais

03. Sorted sets

Ganho. Ordenam eficientemente

Custo e motivo. Consomem mais memória

Como decidir. Quando ranking ou prazo importa

04. Pipeline

Ganho. Reduz RTT

Custo e motivo. Não cria atomicidade

Como decidir. Quando comandos são independentes

05. Lua

Ganho. Agrupa decisão atômica

Custo e motivo. Pode bloquear nó

Como decidir. Quando script é curto

06. RDB

Ganho. Simplifica snapshots

Custo e motivo. Aceita janela entre cópias

Como decidir. Quando perda recente é tolerável

07. AOF

Ganho. Reduz janela conforme política

Custo e motivo. Aumenta I/O

Como decidir. Quando recuperação exige mais detalhes

08. Cluster

Ganho. Distribui capacidade

Custo e motivo. Limita multichave

Como decidir. Quando carga é particionável

09. Cache local

Ganho. Alivia hot key

Custo e motivo. Duplica estado

Como decidir. Quando atraso é aceitável

10. Separar clusters

Ganho. Isola políticas

Custo e motivo. Aumenta custo

Como decidir. Quando criticidades diferem

Duas alternativas ao caminho proposto

1. Memcached

Quando faz sentido. Quando apenas cache simples é necessário

Como implementar. Usar valores descartáveis com origem protegida

Limites. Não oferece o mesmo conjunto de estruturas e persistência

2. Banco transacional

Quando faz sentido. Quando o dado precisa de autoridade durável

Como implementar. Usar constraints, versões e transações com índice adequado

Limites. Pode ter latência maior e exige dimensionamento

Referências e alcance da evidência

1. Redis: Cluster scaling

Particionamento por slots, redirecionamento e limites de operações entre chaves no Redis Cluster.

https://redis.io/docs/latest/operate/oss_and_stack/management/scaling/

2. Redis: Persistence

RDB, AOF, recuperação e compromissos de durabilidade e desempenho.

https://redis.io/docs/latest/operate/oss_and_stack/management/persistence/

Continue em: T031 · T033 · T036

T028 / C. Bancos e armazenamento

Datomic, Datalog e Clojure

Datomic representa fatos e valores imutáveis do banco, com transações e consultas temporais. Datalog expressa relações; Clojure favorece dados imutáveis e funções, mas regras financeiras e efeitos externos continuam explícitos.

Quando usar. Quando histórico, consultas sobre fatos e composição de transações combinam com requisitos e experiência da equipe.

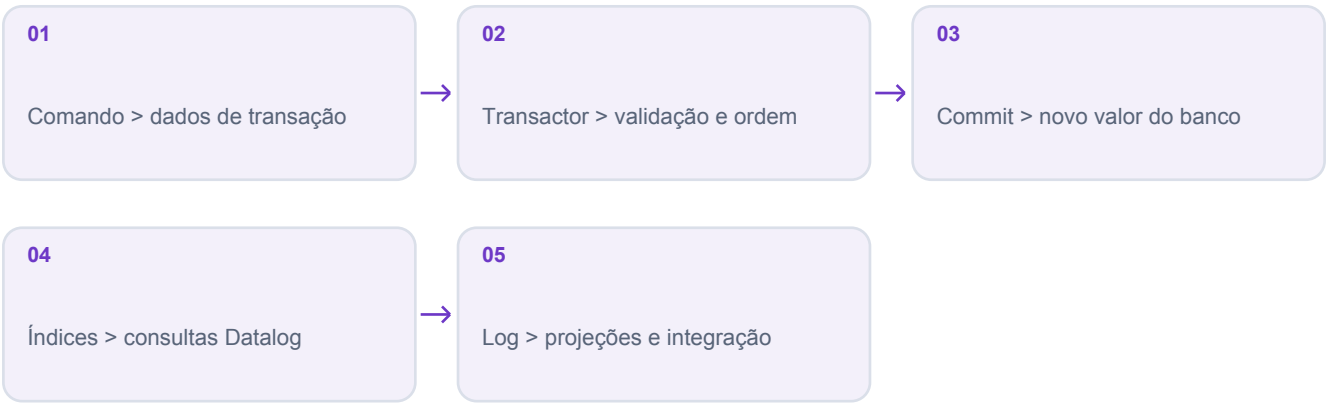
Quando evitar ou limitar. Evitar afirmar que toda arquitetura Nubank é conhecida, que transactor elimina toda coordenação ou que imutabilidade dispensa controle de autorização e concorrência externa.

Uma abertura possível

Vou separar o modelo de fatos da regra financeira. Datomic oferece uma fronteira transacional útil, mas preciso modelar partidas dobradas, identidade e recuperação, verificando a edição e a infraestrutura realmente escolhidas.

Peças e tecnologias. Datomic Pro · Datomic Cloud · Datalog · Clojure · datoms · transaction functions

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Escolher edição e confirmar capacidades.

2. Modelar entidades, atributos e identidades.

3. Expressar invariantes na transação correta.

4. Projetar consultas temporais e índices.

5. Tratar timeout, publicação e recuperação externa.

30 perguntas para sustentar o desenho

01. O que é um datom?

Eu o descreveria como entidade, atributo, valor, transação e indicação de adição ou retração. Esse fato permite reconstruir visões do banco; ele não equivale automaticamente a um lançamento contábil completo do domínio.

Termo para lembrar: EAVTOp | Base técnica: referência 1.

02. Retração apaga o passado?

Eu distinguiria retirar um fato da visão atual de eliminar seu registro histórico. A política de histórico e recursos específicos precisam ser considerados; para estorno financeiro, criaria movimentos compensatórios em vez de esconder o lançamento original.

Termo para lembrar: Retraction | Base técnica: referência 1.

03. O banco é imutável se posso transacionar?

Eu explicaria que um valor do banco representa um estado estável, enquanto novas transações produzem outros valores. Referências antigas continuam úteis para consultas coerentes; imutabilidade do valor não significa que o sistema nunca evolui.

Termo para lembrar: Immutable database value | Base técnica: referência 1.

04. Como funciona o transactor?

Eu diria que ele ordena e processa transações serialmente conforme o modelo documentado. Isso simplifica a fronteira local, mas não prova que toda atividade física é uma única thread nem resolve efeitos em outros sistemas.

Termo para lembrar: Serialized transactions | Base técnica: referência 1.

05. Isso elimina locks distribuídos?

Eu evitaria locks externos desnecessários para regras que cabem na transação Datomic, mas não afirmaria eliminação universal. Operações em brokers, parceiros e múltiplas autoridades ainda exigem identidade, coordenação ou reconciliação conforme o caso.

Termo para lembrar: Coordination scope | Base técnica: referência 1.

06. Como representa transferência?

Eu criaria operação e entradas equilibradas por moeda na mesma transação, ligadas a contas e identidade. O schema deve garantir significado dos sinais e tipos; guardar fatos imutáveis sem a regra de equilíbrio não cria um ledger correto.

Termo para lembrar: Double-entry model | Base técnica: referência 1.

07. Como calcula saldo histórico?

Eu consultaria entradas válidas até a posição temporal escolhida, considerando moeda e semântica de reversões. Para alta frequência, manteria acumulados verificáveis; somar todo o histórico a cada autorização pode ser caro mesmo com dados imutáveis.

Termo para lembrar: As-of balance | Base técnica: referência 1.

08. Como protege saldo suficiente?

Eu avaliaria a condição dentro da fronteira transacional, usando mecanismos adequados como função transacional ou condição suportada. Ler um valor antigo e montar débito fora dessa validação pode permitir decisões concorrentes inválidas.

Termo para lembrar: Transactional validation | Base técnica: referência 1.

09. Como faz idempotência?

Eu modelaria identidade única da operação e verificaria que o payload corresponde à intenção original. Unicidade ajuda arbitrar concorrência, mas replay precisa devolver resultado coerente; uma nova chave por tentativa destrói a proteção pretendida.

Termo para lembrar: Unique operation identity | Base técnica: referência 1.

10. Timeout significa transação falhou?

Eu seguiria a documentação: após timeout, o cliente pode não saber se a transação ocorreu. Consultaria um valor suficientemente recente pela identidade da operação, evitando repetir com uma nova intenção sem verificar o resultado anterior.

Termo para lembrar: Unknown transaction result | Base técnica: referência 1.

11. A lista de tx-data executa passo a passo?

Eu não interpretaria sua ordem textual como comandos SQL sequenciais. Os dados descrevem fatos aplicados atomicamente; funções transacionais validam e expandem a proposta conforme o modelo, exigindo compreender db-before e a semântica suportada.

Termo para lembrar: Transaction data semantics | Base técnica: referência 1.

12. O que é Datalog?

Eu usaria padrões e variáveis para expressar relações entre fatos, deixando o engine avaliar a consulta. Ainda preciso controlar seletividade e tamanho de resultados; linguagem declarativa não torna qualquer consulta barata automaticamente.

Termo para lembrar: Relational query | Base técnica: referência 1.

13. Como otimiza uma consulta?

Eu começaria por restrições seletivas e padrões compatíveis com índices, observando estatísticas oferecidas pelo produto. Evitaria trazer conjuntos enormes para filtrar na aplicação; o desenho precisa considerar cardinalidade e reutilização dos dados consultados.

Termo para lembrar: Query selectivity | Base técnica: referência 1.

14. Datomic é apenas event sourcing?

Eu distinguiria histórico de fatos e transações de uma arquitetura cujo domínio é reconstruído por eventos de negócio. Posso modelar eventos, mas escolher Datomic não define automaticamente contratos, snapshots e processos de replay da aplicação.

Termo para lembrar: Model distinction | Base técnica: referência 1.

15. Como integra com Kafka?

Eu persistiria a obrigação de publicar ou consumiria um log com posição recuperável, conforme a arquitetura suportada. Notificação em memória não basta; queda entre commit e envio exige retomada e consumidores capazes de reconhecer repetições.

Termo para lembrar: Durable publication | Base técnica: referência 1.

16. Tx-report-queue é histórico durável completo?

Eu a trataria conforme seu escopo de notificações enquanto o peer está conectado, cuidando também do consumo de memória. Para recuperação após desconexão, escolheria um mecanismo com posição e cobertura adequadas, não apenas uma fila local.

Termo para lembrar: Notification boundary | Base técnica: referência 1.

17. Como usa time travel para auditoria?

Eu consultaria o estado e fatos da posição relevante, ligando transações a ator, motivo e operação. História técnica ajuda, mas auditoria exige contexto de negócio e controle de acesso; timestamp sozinho não explica uma decisão.

Termo para lembrar: Temporal query | Base técnica: referência 1.

18. Como trata informação corrigida?

Eu registraria correção e vínculo com o fato ou decisão anterior, preservando o significado histórico necessário. Alterar uma referência atual pode ser legítimo, mas um estorno financeiro requer efeito compensatório explícito e autorizado.

Termo para lembrar: Correction semantics | Base técnica: referência 1.

19. Como escolhe infraestrutura de storage?

Eu verificaria documentação da edição e implantação reais, separando Datomic Pro e Cloud. Não desenharia DynamoDB e EBS como alternativas intercambiáveis sem confirmar suporte, papel e configuração; abstração lógica não elimina diferenças operacionais.

Termo para lembrar: Edition-specific architecture | Base técnica: referência 1.

20. Como escala leitura?

Eu exploraria valores imutáveis e componentes de consulta da edição escolhida, medindo cache e acesso ao storage. Escalar leitura não aumenta automaticamente a capacidade de transações; cada caminho tem recursos e limites próprios.

Termo para lembrar: Read scalability | Base técnica: referência 1.

21. Como escala escrita?

Eu mediria tamanho, custo de validação e fila de transações, mantendo funções curtas. Se necessário, dividiria autoridades de domínio com cuidado; separar bancos reduz escopo atômico e pode criar coordenação distribuída onde antes não existia.

Termo para lembrar: Transaction throughput | Base técnica: referência 1.

22. Como trata função transacional pesada?

Eu evitaria cálculos longos e chamadas externas no caminho que ordena decisões. Pré-computaria o que for seguro e revalidaria condições essenciais; trabalho caro nesse ponto aumenta fila e latência de operações independentes.

Termo para lembrar: Transaction critical path | Base técnica: referência 1.

23. Clojure imutável resolve concorrência sozinho?

Eu aproveitaria valores persistentes para reduzir compartilhamento mutável, mas ainda coordenaria identidades e efeitos. Uma coleção imutável não impede duas requisições de enviar o mesmo pagamento; a garantia precisa existir na fronteira de execução.

Termo para lembrar: Immutable values | Base técnica: referência 1.

24. Por que usar funções puras?

Eu isolaria cálculos de regras, tornando testes e replay mais previsíveis. Entradas como tempo e política seriam explícitas; efeitos de banco e rede ficam separados, mas precisam de tratamento próprio de falha e autorização.

Termo para lembrar: Functional core | Base técnica: referência 1.

25. Como representa dinheiro em Clojure?

Eu escolheria inteiro de unidades mínimas ou decimal com escala e arredondamento definidos, sempre acompanhado da moeda. Tipos numéricos disponíveis não definem regra financeira; evitaria conversão acidental para ponto flutuante no fluxo.

Termo para lembrar: Exact numeric representation | Base técnica: referência 1.

26. Como testa invariantes do ledger?

Eu geraria sequências de créditos, débitos e reversões, verificando equilíbrio e identidade, além de cenários concorrentes. Testes do modelo puro são úteis, mas complementaria com integração real para confirmar comportamento transacional e timeout.

Termo para lembrar: Property-based validation | Base técnica: referência 1.

27. Como evolui atributos?

Eu trataria mudanças de tipo e significado com cuidado, preferindo novos atributos quando necessário e migração explícita. Clientes históricos podem depender da semântica antiga; flexibilidade de schema não autoriza reinterpretar silenciosamente fatos passados.

Termo para lembrar: Schema evolution | Base técnica: referência 1.

28. Como trata dados pessoais no histórico?

Eu definiria minimização, referências e políticas de retenção validadas, considerando capacidades específicas de remoção e seus limites. Imutabilidade cria decisões importantes de governança; não prometeria conformidade automática por criptografar ou esconder um campo.

Termo para lembrar: Historical data governance | Base técnica: referência 1.

29. Como compara com PostgreSQL?

Eu compararia histórico e modelo de consulta com familiaridade, ecossistema e custo operacional. PostgreSQL com lançamentos append-only pode atender o ledger; Datomic pode simplificar outras dimensões, mas exige domínio de suas garantias e operação.

Termo para lembrar: Storage trade-off | Base técnica: referência 1.

30. Como explica sem atribuir arquitetura interna?

Eu diria que esta é uma proposta baseada nas capacidades documentadas, não um retrato completo do Nubank. Defenderia modelo, invariantes e falhas por mérito técnico, sem usar o nome da empresa como prova da escolha.

Termo para lembrar: Evidence boundary | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Histórico integrado

Ganho. Facilita consultas temporais

Custo e motivo. Amplia retenção

Como decidir. Quando auditoria usa estados passados

02. Valores imutáveis

Ganho. Simplificam leitura coerente

Custo e motivo. Exigem compreender versões

Como decidir. Quando consultas compartilham contexto

03. Transação serializada

Ganho. Clarifica ordem

Custo e motivo. Cria limite no caminho de escrita

Como decidir. Quando regras cabem nessa autoridade

04. Função transacional

Ganho. Valida no ponto correto

Custo e motivo. Pode aumentar fila

Como decidir. Quando condição depende do estado atual

05. Datalog

Ganho. Expressa relações

Custo e motivo. Exige aprendizado e tuning

Como decidir. Quando modelo de fatos ajuda

06. Acumulado de saldo

Ganho. Acelera leitura

Custo e motivo. Exige verificação

Como decidir. Quando histórico é grande

07. Consulta histórica completa

Ganho. Evita projeção própria

Custo e motivo. Pode custar mais

Como decidir. Quando acesso é eventual

08. Clojure

Ganho. Favorece dados e funções

Custo e motivo. Exige competência da equipe

Como decidir. Quando benefício é sustentado

09. Múltiplos bancos

Ganho. Distribuem escrita

Custo e motivo. Reduzem atomicidade conjunta

Como decidir. Quando domínios são separáveis

10. Outbox explícita

Ganho. Preserva publicação

Custo e motivo. Adiciona relay

Como decidir. Quando integração externa é obrigatória

Duas alternativas ao caminho proposto

1. PostgreSQL append-only

Quando faz sentido. Quando equipe domina relacional e invariantes cabem em transação

Como implementar. Modelar operações e lançamentos com constraints e histórico

Limites. Time travel e projeções exigem desenho próprio

2. Event store especializado

Quando faz sentido. Quando eventos de domínio são a autoridade

Como implementar. Persistir streams com concorrência esperada e construir projeções

Limites. Exige evolução de eventos, replay e governança de histórico

Referências e alcance da evidência

1. Datomic: Processing Transactions

Processamento serial de transações, resultados e ambiguidade de timeout.

https://docs.datomic.com/transactions/transaction-processing.html

2. Datomic: Transaction Data

Representação de transações como dados, assertions, retractions e funções transacionais.

https://docs.datomic.com/transactions/transaction-data-reference.html

3. Clojure: Data Structures

Coleções persistentes imutáveis e tipos de valores; não implica segurança automática dos efeitos externos.

https://clojure.org/reference/data_structures

Continue em: T024 · T039 · T091

T029 / C. Bancos e armazenamento

MongoDB e Cassandra

MongoDB organiza documentos e oferece replicação, índices e transações conforme escopo. Cassandra organiza dados por partição e consultas previstas, com replicação e consistência configuráveis; não são alternativas equivalentes por serem NoSQL.

Quando usar. Ao comparar modelo documental flexível com acesso distribuído orientado a partições e grande ingestão previsível.

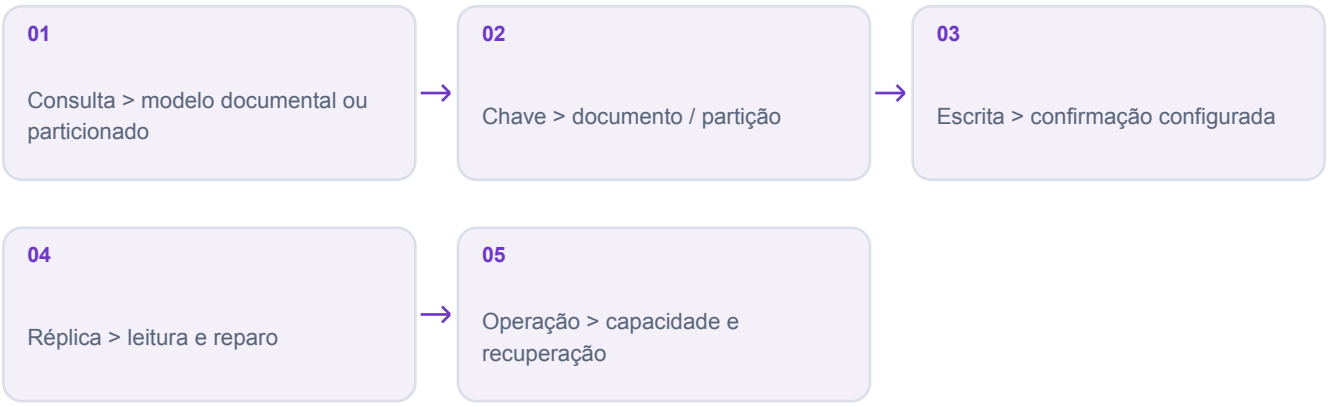
Quando evitar ou limitar. Evitar escolher pela categoria NoSQL, criar arrays ou partições ilimitadas e ignorar custos de repair, índices e transações.

Uma abertura possível

Vou separar as duas opções: documentos acessados juntos favorecem um modelo; consultas previsíveis por partição favorecem outro. Compararei as garantias concretas antes de colocar qualquer um como autoridade financeira.

Peças e tecnologias. MongoDB · Replica set · MongoDB transactions · Cassandra · CQL · LWT · repair

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Descrever consultas e tamanhos máximos.

2. Comparar documento e partição.

3. Definir índices, consistência e transações.

4. Planejar manutenção e falhas.

5. Testar distribuição, concorrência e crescimento.

30 perguntas para sustentar o desenho

01. MongoDB e Cassandra resolvem o mesmo problema?

Eu compararia documentos e consultas flexíveis com modelagem orientada a partições e acessos previstos. Ambos são NoSQL, mas suas garantias e operação diferem; a escolha depende da jornada e da equipe, não da categoria.

Termo para lembrar: Storage model | Base técnica: referência 1.

02. Quando embute dados no MongoDB?

Eu colocaria juntos dados lidos e alterados como unidade, com crescimento controlado. Isso reduz consultas, mas arrays sem limite e documentos muito disputados podem criar problemas; ciclo de vida e cardinalidade orientam a decisão.

Termo para lembrar: Embedding | Base técnica: referência 1.

03. Quando usa referências?

Eu separaria entidades com tamanho, atualização ou compartilhamento independentes, mantendo referências explícitas. Isso reduz duplicação e crescimento do documento, mas aumenta consultas ou agregação; a aplicação precisa preservar regras que atravessam essas fronteiras.

Termo para lembrar: Document references | Base técnica: referência 1.

04. MongoDB não tem schema?

Eu explicaria que flexibilidade de formato não elimina schema lógico. Aplicação e validação podem exigir campos e tipos; versões antigas continuam existindo, então mudanças precisam de compatibilidade, migração e tratamento de documentos incompletos.

Termo para lembrar: Schema validation | Base técnica: referência 1.

05. Como protege atualização de um documento?

Eu usaria operações condicionais no escopo suportado, verificando versão ou estado esperado. Ler e salvar todo o documento pode sobrescrever concorrência; a condição deve acompanhar a atualização que efetivamente confirma a decisão.

Termo para lembrar: Atomic document update | Base técnica: referência 1.

06. Quando usa transação MongoDB?

Eu consideraria quando a invariante realmente cruza documentos e cabe nas capacidades e limites da implantação. Não substitui modelagem; transações distribuídas têm custos e não incluem chamadas externas só porque o código as executa dentro do bloco.

Termo para lembrar: Multidocument transaction | Base técnica: referência 1.

07. O que é write concern?

Eu definiria quais confirmações uma escrita precisa receber, considerando replicação e durabilidade. O contrato depende da configuração; responder ao cliente após confirmação fraca pode permitir perda ou rollback em cenários que a aplicação não esperava.

Termo para lembrar: Write concern | Base técnica: referência 1.

08. O que é read concern?

Eu escolheria a garantia de dados visíveis conforme a consulta e a topologia, distinguindo snapshot e confirmação por maioria. Read preference escolhe onde ler; não substitui automaticamente a semântica de consistência exigida pela jornada.

Termo para lembrar: Read concern | Base técnica: referência 1.

09. Ler de secondary é sempre atual?

Eu assumiria possível atraso quando a replicação é assíncrona, usando sessão e garantias apropriadas quando necessário. Distribuir leituras melhora capacidade, mas não deve fazer o usuário observar regressão depois de uma atualização importante.

Termo para lembrar: Secondary lag | Base técnica: referência 1.

10. Como usa sessão causal no MongoDB?

Eu preservaria a sessão e as opções necessárias para manter relações causais documentadas. Isso ajuda continuidade entre operações, mas não transforma qualquer consulta arbitrária ou integração externa numa transação global com ordem total.

Termo para lembrar: Causal session | Base técnica: referência 1.

11. Como escolhe shard key no MongoDB?

Eu avaliaria distribuição, cardinalidade, consultas e crescimento, evitando chave que concentra todo tráfego recente. A escolha afeta roteamento e operações entre shards; uma mudança posterior pode exigir migração cuidadosa e capacidade adicional.

Termo para lembrar: MongoDB shard key | Base técnica: referência 1.

12. Como evita arrays ilimitados?

Eu separaria histórico em documentos ou buckets com limites claros, mantendo apenas resumo ou itens recentes quando útil. O modelo precisa suportar o cliente mais antigo e ativo, não apenas o exemplo pequeno da demonstração.

Termo para lembrar: Bounded document | Base técnica: referência 1.

13. Como usa índices no MongoDB?

Eu indexaria filtros e ordenações importantes, medindo custo de escrita e tamanho. Campos flexíveis podem multiplicar possibilidades; criar índice para tudo aumenta manutenção e memória sem garantir que as consultas reais melhorem.

Termo para lembrar: Document indexing | Base técnica: referência 1.

14. Como trata change streams?

Eu persistiria posição de retomada e processaria alterações de forma idempotente, considerando retenção e invalidações. Stream não é histórico infinito; se a janela necessária desaparecer, preciso de resnapshot ou reconstrução controlada do destino.

Termo para lembrar: Change stream resume | Base técnica: referência 1.

15. Como modela Cassandra?

Eu partiria de consultas, definindo partition key e clustering columns para acessar o conjunto necessário. Duplicação entre tabelas pode ser intencional, mas precisa de atualização e reconciliação; joins flexíveis não são a proposta central desse modelo.

Termo para lembrar: Query-first schema | Base técnica: referência 1.

16. O que define uma partição Cassandra?

Eu escolheria chave que agrupa o necessário sem crescer ou concentrar tráfego demais. Partição enorme afeta leitura, reparo e manutenção; buckets temporais podem ajudar, acrescentando composição para consultas que atravessam vários períodos.

Termo para lembrar: Partition key | Base técnica: referência 1.

17. Para que servem clustering columns?

Eu as usaria para ordenar e localizar linhas dentro da partição conforme os acessos previstos. A ordem influencia quais consultas são eficientes; não modelaria dezenas de filtros opcionais esperando que o banco resolva todos igualmente bem.

Termo para lembrar: Clustering order | Base técnica: referência 1.

18. Como escolhe consistency level?

Eu definiria quantidade e escopo das confirmações de leitura e escrita, considerando datacenters e anomalias aceitáveis. O nível faz parte do contrato da operação; escolher por latência apenas pode violar a regra que o dado representa.

Termo para lembrar: Tunable consistency | Base técnica: referência 1.

19. Quando usa LWT?

Eu consideraria condições concorrentes que exigem o mecanismo específico suportado pelo Cassandra, reconhecendo custo de coordenação. LWT não é substituto gratuito de toda transação relacional; escopo e comportamento precisam ser confirmados para a versão usada.

Termo para lembrar: Lightweight transaction | Base técnica: referência 1.

20. Por que repair é importante?

Eu planejaria reparo para convergência de réplicas e tratamento de dados que não aparecem em leituras frequentes. Hints não cobrem todos os cenários; atrasar manutenção pode aumentar divergência e comprometer exclusões sob certas condições.

Termo para lembrar: Anti-entropy | Base técnica: referência 1.

21. Como TTL afeta Cassandra?

Eu consideraria tombstones e o padrão de expiração no desenho de partições e compaction. Expirar muitos registros não significa custo zero; leituras podem atravessar marcas de exclusão até que seja seguro removê-las.

Termo para lembrar: Tombstone accumulation | Base técnica: referência 1.

22. Como lida com timestamp no Cassandra?

Eu verificaria a política de resolução e origem dos timestamps, evitando relógios incorretos dominarem atualizações. Last-write-wins não preserva automaticamente intenção financeira; regras críticas podem exigir condições ou outra autoridade de decisão.

Termo para lembrar: Conflict timestamp | Base técnica: referência 1.

23. ALLOW FILTERING resolve consulta ruim?

Eu não o usaria como atalho padrão para qualquer pergunta. Ele pode exigir trabalho imprevisível; preferiria tabela ou índice adequado ao acesso, medindo limites e evitando uma consulta interativa que varre dados excessivos.

Termo para lembrar: Query filtering | Base técnica: referência 1.

24. Como compara manutenção dos dois?

Eu incluiria compactação, repair, backups, índices, eleição e upgrades conforme o produto. Serviço gerenciado reduz tarefas, mas não elimina modelagem nem observabilidade; a equipe precisa entender os mecanismos que afetam latência e recuperação.

Termo para lembrar: Operational workload | Base técnica: referência 1.

25. Qual escolheria para cadastro variável?

Eu avaliaria MongoDB quando documentos relacionados e evolução de campos dominam, mantendo validação e índices. PostgreSQL com JSON também pode atender; flexibilidade de payload sozinha não justifica acrescentar outra tecnologia à operação.

Termo para lembrar: Document fit | Base técnica: referência 1.

26. Qual escolheria para telemetria por dispositivo?

Eu consideraria Cassandra com partições temporais e consultas previsíveis, avaliando ingestão, retenção e repair. Não afirmaria que sempre vence; o benchmark precisa incluir consultas, expiração e manutenção no volume sustentado esperado.

Termo para lembrar: Partitioned time series | Base técnica: referência 1.

27. Como trata autoridade financeira?

Eu exigiria identidade, condições, atomicidade e recuperação comprovadas no escopo necessário, independente do banco. Se o modelo torna invariantes difíceis, escolheria outra autoridade e usaria esses stores como projeções ou dados auxiliares.

Termo para lembrar: Financial correctness | Base técnica: referência 1.

28. Como migra entre modelos?

Eu traduziria consultas e regras, não apenas formatos, mantendo writer definido e sincronização durável durante a transição. Documento e partição não são equivalentes; índices, limites e garantias precisam ser reconstruídos e testados no destino.

Termo para lembrar: Model migration | Base técnica: referência 1.

29. Como testa falha e consistência?

Eu executaria concorrência, perda de nó, lag e retomada, verificando resultados permitidos pela configuração. Testar apenas disponibilidade de uma consulta simples não demonstra que transações ou condições mantêm a regra de negócio sob falha.

Termo para lembrar: Failure workload | Base técnica: referência 1.

30. Como defenderia a escolha final?

Eu mostraria acesso dominante, limite de crescimento e invariante, comparando custo operacional e evolução. Não venderia MongoDB como banco sem schema nem Cassandra como armazenamento ilimitado; cada benefício possui condições que precisam permanecer verdadeiras.

Termo para lembrar: Explicit capability comparison | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Embedding

Ganho. Reduz consultas

Custo e motivo. Pode crescer e disputar

Como decidir. Quando dados têm ciclo conjunto

02. Referências

Ganho. Separam crescimento

Custo e motivo. Exigem composição

Como decidir. Quando entidades são independentes

03. Transação multidocumento

Ganho. Protege regra

Custo e motivo. Aumenta coordenação

Como decidir. Quando embedding não atende

04. Secondary reads

Ganho. Distribuem carga

Custo e motivo. Podem atrasar

Como decidir. Quando frescor permite

05. Modelo por consulta

Ganho. Oferece acesso previsível

Custo e motivo. Duplica dados

Como decidir. Quando perguntas são estáveis

06. Partição temporal

Ganho. Limita tamanho

Custo e motivo. Exige fan-out por período

Como decidir. Quando histórico cresce

07. LWT

Ganho. Protege condição

Custo e motivo. Custa coordenação

Como decidir. Quando concorrência exige

08. TTL

Ganho. Automatiza expiração

Custo e motivo. Gera manutenção

Como decidir. Quando retenção é definida

09. Mais índices

Ganho. Atendem filtros

Custo e motivo. Aumentam escrita

Como decidir. Quando consultas justificam

10. Serviço gerenciado

Ganho. Reduz tarefas operacionais

Custo e motivo. Mantém dependência

Como decidir. Quando equipe aceita fornecedor

Duas alternativas ao caminho proposto

1. PostgreSQL com JSON

Quando faz sentido. Quando flexibilidade convive com relações e transações

Como implementar. Combinar colunas, constraints e documentos indexados

Limites. Requer tuning e não elimina limites de capacidade

2. DynamoDB

Quando faz sentido. Quando acesso por chave e gestão simplificada dominam

Como implementar. Modelar PK, SK, índices e condições

Limites. Consultas novas e hot keys exigem cuidado

Referências e alcance da evidência

1. MongoDB: Data Modeling

Modelagem documental, embedding, referências e validação de schema.

https://www.mongodb.com/docs/manual/data-modeling/

2. MongoDB: Read Isolation, Consistency and Recency

Read concern, write concern, sessões e leitura de secundários.

https://www.mongodb.com/docs/manual/core/read-isolation-consistency-recency/

3. Apache Cassandra: Dynamo architecture

Replicação, níveis de consistência e reparo no modelo Cassandra; não constitui promessa de linearizabilidade genérica.

https://cassandra.apache.org/doc/latest/cassandra/architecture/dynamo.html

Continue em: T021 · T026 · T024

T030 / C. Bancos e armazenamento

Elasticsearch, OpenSearch e busca

Elasticsearch e OpenSearch oferecem indexação invertida, análise textual e busca distribuída. São frequentemente projeções de dados autoritativos, com atraso de atualização, custo de shards e necessidade de governança de acesso.

Quando usar. Quando busca textual, filtros, facetas e ranking excedem o acesso simples do armazenamento principal.

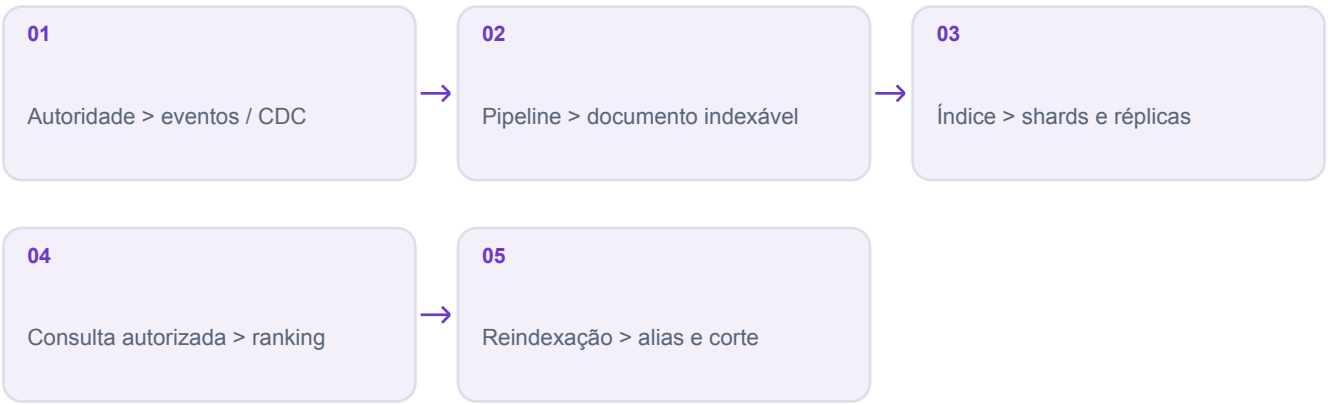
Quando evitar ou limitar. Evitar usar índice como autoridade financeira, mapping dinâmico ilimitado ou consultas profundas sem limites de custo.

Uma abertura possível

Vou manter o banco de negócio como autoridade e projetar um índice orientado à busca. O contrato informa frescor, autorização e recuperação, incluindo como reindexar sem perder alterações novas.

Peças e tecnologias. OpenSearch · Elasticsearch · Lucene · inverted index · index alias · CDC

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Definir perguntas, filtros e ranking.

2. Criar mapping e análise textual explícitos.

3. Alimentar índice com identidade e versão.

4. Controlar consultas, shards e acesso.

5. Planejar reindexação, retenção e reconstrução.

30 perguntas para sustentar o desenho

01. Quando precisa de engine de busca?

Eu escolheria quando tokenização, ranking e filtros complexos justificarem, comparando com busca do banco existente. Acrescentar engine cria pipeline e lag; uma consulta simples por ID ou igualdade não exige necessariamente outra infraestrutura.

Termo para lembrar: Search workload | Base técnica: referência 1.

02. O que é índice invertido?

Eu explicaria que ele relaciona termos aos documentos que os contêm, permitindo localizar candidatos sem ler todo conteúdo. Ranking e filtros completam a consulta; o custo depende da frequência dos termos e do conjunto avaliado.

Termo para lembrar: Inverted index | Base técnica: referência 1.

03. Text e keyword diferem como?

Eu usaria text para análise linguística e keyword para valor exato, agregação ou filtro conforme suporte. A escolha muda o que é indexado; um identificador financeiro não deve ser tokenizado como uma frase comum.

Termo para lembrar: Field mapping | Base técnica: referência 1.

04. Como escolhe analyzer?

Eu definiria tokenização, normalização e regras de idioma conforme o conteúdo, testando consultas reais. Aplicar stemming inadequado pode misturar significados; indexação e busca precisam de compatibilidade para produzir resultados úteis e previsíveis.

Termo para lembrar: Text analysis | Base técnica: referência 1.

05. O que significa near real-time?

Eu explicaria que escrita e visibilidade na busca são etapas distintas, ligadas ao refresh e configuração. Não prometeria leitura imediata por pesquisa após indexar; a jornada pode consultar a autoridade ou aguardar a condição apropriada.

Termo para lembrar: Refresh visibility | Base técnica: referência 1.

06. Refresh é fsync?

Eu separaria tornar segmentos pesquisáveis de confirmar persistência durável. Confundir as duas etapas produz garantias erradas; o contrato de escrita, replicação e recuperação precisa ser verificado na implementação e configuração realmente escolhidas.

Termo para lembrar: Refresh versus durability | Base técnica: referência 1.

07. Como alimenta o índice?

Eu usaria eventos duráveis ou CDC com posição de progresso, transformando registros em documentos de busca. Escrita direta no banco e no índice sem coordenação pode divergir; o pipeline precisa de retry e reconstrução.

Termo para lembrar: Projection pipeline | Base técnica: referência 1.

08. Como trata duplicatas de eventos?

Eu usaria ID estável de documento e versão de origem para impedir regressão por mensagens antigas. Um upsert simples pode sobrescrever dado novo com evento atrasado; a condição de versão preserva a ordem sem depender da entrega perfeita.

Termo para lembrar: Idempotent indexing | Base técnica: referência 1.

09. Como trata exclusão?

Eu propagaria remoção ou tombstone com versão, incluindo réplicas e índices derivados. Falha no pipeline pode deixar dados visíveis; monitoraria atraso e faria reconciliação, respeitando política de retenção e autorização durante a janela.

Termo para lembrar: Deletion propagation | Base técnica: referência 1.

10. Como evita mapping explosion?

Eu limitaria campos e evitaria transformar chaves arbitrárias de usuário em nomes de propriedades indexadas. Mapping explícito e representação adequada reduzem crescimento; validar payload na entrada evita que uma fonte descontrolada comprometa o cluster.

Termo para lembrar: Mapping governance | Base técnica: referência 1.

11. Como escolhe shards?

Eu consideraria volume, throughput e paralelismo, evitando muitos shards pequenos ou poucos enormes. Cada shard possui overhead e custo de recuperação; o número deve ser testado com crescimento e consultas reais do índice.

Termo para lembrar: Shard sizing | Base técnica: referência 1.

12. Réplicas aumentam throughput de escrita?

Eu esperaria benefício principalmente em disponibilidade e distribuição de leitura, enquanto escrita precisa ser replicada. Mais réplicas aumentam armazenamento e trabalho; não trataria adicionar cópias como expansão gratuita de ingestão.

Termo para lembrar: Replica trade-off | Base técnica: referência 1.

13. Como trata consulta em muitos shards?

Eu reduziria o conjunto por routing e filtros temporais quando seguros, limitando concorrência e prazo. Cada consulta global amplifica trabalho; a cauda pode depender do shard mais lento e de resultados parciais.

Termo para lembrar: Search fan-out | Base técnica: referência 1.

14. Como faz multi-tenancy segura?

Eu aplicaria autorização no backend e mecanismos apropriados do engine, incluindo tenant no documento e consulta. Um filtro de UI ou alias isolado não deve ser tratado como barreira universal de segurança contra acesso direto.

Termo para lembrar: Search authorization | Base técnica: referência 1.

15. Como indexa histórico bancário?

Eu guardaria campos mínimos necessários a busca, com referência à operação autoritativa. Resultado pode estar atrasado; ações financeiras revalidam no domínio, evitando que um documento encontrado se torne permissão para alterar dinheiro.

Termo para lembrar: Search projection | Base técnica: referência 1.

16. Como faz paginação profunda?

Eu consideraria search_after com ordenação estável e ponto no tempo quando necessário. Offset profundo exige guardar ou percorrer muitos candidatos; cursores precisam preservar contexto e expirar conforme limites de recursos da consulta.

Termo para lembrar: Search after | Base técnica: referência 1.

17. Como trata ranking?

Eu definiria exemplos de consultas e resultados esperados, avaliando relevância antes de ajustar pesos. Métrica técnica de latência não diz se o atendente encontra a transação correta; qualidade da busca precisa de avaliação própria.

Termo para lembrar: Relevance tuning | Base técnica: referência 1.

18. Como evita wildcard caro?

Eu limitaria padrões e campos pesquisáveis, oferecendo consultas estruturadas quando possível. Certas buscas expandem muitos termos e consomem recursos; limites de tempo e taxa protegem o cluster contra abuso e uso acidentalmente caro.

Termo para lembrar: Query cost control | Base técnica: referência 1.

19. Como usa bulk indexing?

Eu agruparia documentos por quantidade e bytes, tratando falhas por item e repetindo somente o necessário. Um lote pode ter sucesso parcial; reenviar tudo sem identidade e versão pode desperdiçar capacidade ou regredir documentos.

Termo para lembrar: Bulk ingestion | Base técnica: referência 1.

20. Como reindexa sem downtime?

Eu criaria índice novo com mapping desejado, copiaria dados e aplicaria mudanças ocorridas durante a cópia. Após validar posição e qualidade, trocaria alias; a troca atômica não substitui alcançar todos os dados novos.

Termo para lembrar: Versioned index | Base técnica: referência 1.

21. Como faz rollback da reindexação?

Eu manteria o índice anterior pelo período planejado e verificaria se ele continua recebendo mudanças necessárias. Voltar alias para uma cópia parada pode restaurar schema antigo com dados atrasados; o plano precisa preservar equivalência relevante.

Termo para lembrar: Index rollback | Base técnica: referência 1.

22. Como valida completude do índice?

Eu compararia contagens, versões e amostras ou hashes por faixa, considerando atraso esperado. Número igual de documentos não prova conteúdo correto; divergências precisam de reparo direcionado e capacidade de reconstruir sem afetar produção.

Termo para lembrar: Projection reconciliation | Base técnica: referência 1.

23. Como trata ingestão mais rápida que indexação?

Eu monitoraria backlog e tempo de frescor, limitando produtores ou ampliando capacidade onde existe gargalo. Uma fila absorve pico temporário; atraso contínuo exige ação e comunicação, não apenas mais retenção de mensagens.

Termo para lembrar: Backpressure | Base técnica: referência 1.

24. Como escolhe hot, warm e cold?

Eu distribuiria dados conforme frequência de consulta, prazo e custo, preservando requisitos de recuperação. Histórico antigo pode aceitar maior latência; mover dados sem considerar o atendimento pode tornar investigações legítimas impraticáveis.

Termo para lembrar: Data tiering | Base técnica: referência 1.

25. Como monitora heap e GC?

Eu observaria memória, pausas, circuit breakers e consultas caras, além de CPU. Agregações de alta cardinalidade podem pressionar heap; aumentar memória sem limitar workload pode apenas adiar a mesma falha em escala maior.

Termo para lembrar: Search memory pressure | Base técnica: referência 1.

26. Como trata falha de um shard?

Eu definiria se resposta parcial é aceitável e a sinalizaria claramente. Uma lista incompleta não pode parecer busca exaustiva para uma decisão crítica; a aplicação pode pedir retry ou consultar autoridade por identificador.

Termo para lembrar: Partial search failure | Base técnica: referência 1.

27. Como faz backup do índice?

Eu compararia snapshots com reconstrução pela fonte, estimando tempo e retenção dos eventos necessários. Projeção reconstruível ainda precisa de plano; se a origem foi apagada ou o replay expirou, reconstrução pode ser impossível.

Termo para lembrar: Search recoverability | Base técnica: referência 1.

28. Como protege dados sensíveis?

Eu indexaria apenas o necessário, evitando documentos e campos pessoais sem finalidade de busca. Logs de consultas também podem conter dados; retenção, acesso e mascaramento precisam cobrir pipeline, engine e ferramentas de operação.

Termo para lembrar: Index minimization | Base técnica: referência 1.

29. Como compara Elasticsearch e OpenSearch?

Eu avaliaria versões, plugins, licenciamento, APIs e operação atuais, sem assumir equivalência total. A origem comum não garante compatibilidade futura; a decisão precisa de teste com funcionalidades realmente usadas e plano de atualização.

Termo para lembrar: Product compatibility | Base técnica: referência 1.

30. Como defenderia o desenho?

Eu explicaria que busca otimiza descoberta, enquanto a autoridade confirma decisões. Mostraria atualização, frescor, autorização e reindexação, incluindo uma falha parcial; esse caminho é mais importante que apenas desenhar uma caixa de Elasticsearch.

Termo para lembrar: Search as projection | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Mapping explícito

Ganho. Controla schema

Custo e motivo. Exige evolução planejada

Como decidir. Quando dados são sensíveis ou variados

02. Mapping dinâmico

Ganho. Acelera ingestão

Custo e motivo. Pode explodir campos

Como decidir. Quando origem é controlada

03. Refresh frequente

Ganho. Reduz atraso

Custo e motivo. Aumenta trabalho

Como decidir. Quando frescor é crítico

04. Bulk maior

Ganho. Melhora ingestão

Custo e motivo. Aumenta memória e retry

Como decidir. Quando prazo permite

05. Mais shards

Ganho. Amplia paralelismo

Custo e motivo. Aumenta overhead

Como decidir. Quando volume justifica

06. Mais réplicas

Ganho. Melhora leitura e tolerância

Custo e motivo. Custa escrita e espaço

Como decidir. Quando disponibilidade exige

07. Routing por tenant

Ganho. Reduz fan-out

Custo e motivo. Pode criar hotspot

Como decidir. Quando distribuição é equilibrada

08. Índice por período

Ganho. Facilita retenção

Custo e motivo. Complica buscas amplas

Como decidir. Quando tempo é filtro comum

09. Ponto no tempo

Ganho. Estabiliza navegação

Custo e motivo. Retém recursos

Como decidir. Quando paginação precisa coerência

10. Reconstrução pela origem

Ganho. Reduz dependência de backup

Custo e motivo. Pode demorar

Como decidir. Quando fonte completa é preservada

Duas alternativas ao caminho proposto

1. PostgreSQL full-text

Quando faz sentido. Quando busca e volume cabem no banco existente

Como implementar. Usar representação textual e índices apropriados

Limites. Pode disputar recursos com OLTP e ter menos funcionalidades especializadas

2. Busca por atributos estruturados

Quando faz sentido. Quando usuários conhecem identificadores e filtros

Como implementar. Modelar índices relacionais ou key-value para consultas previsíveis

Limites. Não entrega relevância linguística ou pesquisa livre equivalente

Referências e alcance da evidência

1. Elastic: Near real-time search

Refresh torna documentos pesquisáveis; visibilidade em busca não é idêntica a commit durável.

https://www.elastic.co/docs/manage-data/data-store/near-real-time-search

2. OpenSearch: Index aliases

Aliases, troca de índices e escolha de destino de escrita.

https://docs.opensearch.org/latest/im-plugin/index-alias/

3. Microsoft: Understand data models

Modelos de armazenamento e critérios por padrão de acesso; propostas bancárias são análises autorais.

https://learn.microsoft.com/en-us/azure/architecture/data-guide/technology-choices/understand-data-store-models

Continue em: T023 · T037 · T040

T031 / D. Cache e modelos de dados

Cache-aside, read-through, write-through e write-behind

Padrões de cache definem quem carrega e grava dados: cache-aside deixa a aplicação coordenar; read-through encapsula carregamento; write-through confirma origem conforme contrato; write-behind posterga persistência e assume risco próprio.

Quando usar. Para reduzir latência e carga de leituras repetidas, com política explícita de frescor e falha.

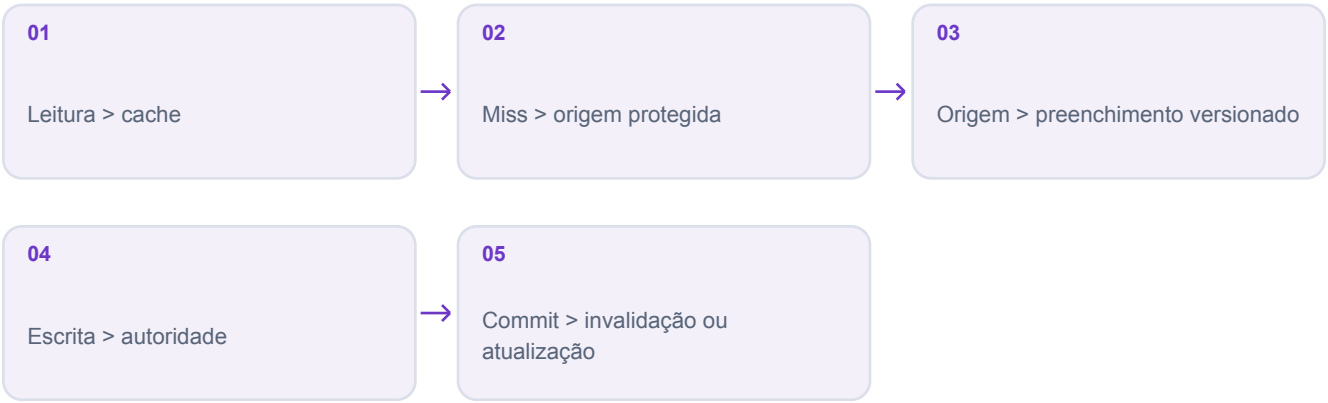
Quando evitar ou limitar. Evitar usar write-behind descartável como autoridade financeira ou presumir atomicidade entre cache e banco por causa do nome do padrão.

Uma abertura possível

Vou manter clara a fonte da verdade e o que acontece se cada etapa falhar. Cache é útil quando há reutilização e quando sabemos como reconstruir sem derrubar a origem.

Peças e tecnologias. Redis · Caffeine · Memcached · cache-aside · read-through · write-through · write-behind

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Medir reutilização e custo da origem.

2. Escolher padrão de leitura e escrita.

3. Definir chave, TTL e versão.

4. Proteger miss e indisponibilidade.

5. Testar corrida de preenchimento e recuperação.

30 perguntas para sustentar o desenho

01. Como funciona cache-aside?

Eu consultaria o cache e, no miss, buscaria a origem antes de preencher. A aplicação controla o fluxo e precisa limitar recomputação; o padrão melhora leitura, mas não garante consistência automática entre as duas cópias.

Termo para lembrar: Cache-aside | Base técnica: referência 1.

02. O que muda no read-through?

Eu encapsularia a busca na origem dentro de um loader ou camada de cache, reduzindo repetição no chamador. Ainda definiria timeout, autorização e concorrência; esconder o miss não elimina seu custo nem sua falha.

Termo para lembrar: Read-through | Base técnica: referência 1.

03. Como define write-through?

Eu exigiria que o caminho de escrita confirme a persistência autoritativa conforme o contrato antes de declarar sucesso. O componente pode atualizar cache também, mas duas tecnologias não se tornam atomicamente coordenadas apenas pelo nome do padrão.

Termo para lembrar: Write-through | Base técnica: referência 1.

04. O que é write-behind?

Eu aceitaria escrita em uma camada e persistiria na origem depois, reconhecendo a janela e o mecanismo durável necessário. Cache descartável não basta para confirmar dinheiro; perda antes da drenagem precisa ser tolerada ou impedida pelo desenho.

Termo para lembrar: Write-behind | Base técnica: referência 1.

05. Como escolhe o padrão?

Eu compararia frequência de leitura, escrita, custo e frescor, além da consequência de perder estado. A mesma aplicação pode usar cache-aside para perfil e nenhum cache na autorização de saldo, porque as garantias diferem.

Termo para lembrar: Cache policy | Base técnica: referência 1.

06. Quando cache não ajuda?

Eu evitaria quando dados quase nunca se repetem, são baratos de buscar ou exigem frescor incompatível. O cache adiciona rede, memória e invalidação; medir hit ratio e latência total evita otimização que apenas aumenta complexidade.

Termo para lembrar: Cache utility | Base técnica: referência 1.

07. Como calcula benefício esperado?

Eu ponderaria custo do hit e do miss pela distribuição real, incluindo serialização e rede. Também mediria cauda e carga na origem; um hit ratio alto pode esconder poucos misses extremamente caros ou concentrados.

Termo para lembrar: Miss cost | Base técnica: referência 1.

08. Como escolhe a chave?

Eu incluiria identidade, tenant, versão do formato e parâmetros que alteram a resposta. Omissões podem devolver dado de outro contexto; chave correta faz parte da segurança e da semântica, não apenas da organização do Redis.

Termo para lembrar: Cache key design | Base técnica: referência 1.

09. Pode cachear resposta autorizada?

Eu separaria dados canônicos de filtragem ou incluiria o escopo de permissão necessário na chave. Compartilhar resposta já mascarada entre usuários pode vazar dados ou esconder informação; autorização atual continua sendo verificada no backend.

Termo para lembrar: Authorization-aware cache | Base técnica: referência 1.

10. Como trata escrita no cache-aside?

Eu confirmaria a origem e então invalidaria ou atualizaria conforme protocolo, tratando falha dessa etapa. Preciso de TTL ou evento recuperável para limitar divergência; uma chamada de delete perdida não pode deixar dado antigo indefinidamente.

Termo para lembrar: Invalidation after commit | Base técnica: referência 1.

11. Atualizar cache antes do banco é seguro?

Eu não exporia estado ainda não confirmado pela autoridade. Se o banco rejeitar, o cache teria publicado um resultado falso; a ordem e a recuperação precisam preservar o significado de sucesso e evitar leitura de intenção como fato.

Termo para lembrar: Write ordering | Base técnica: referência 1.

12. Invalidar depois do commit elimina toda corrida?

Eu consideraria uma leitura antiga iniciada antes da escrita que preenche o cache depois da invalidação. Versões e publicação condicional ajudam impedir regressão; apenas escolher ordem de duas chamadas não resolve todas as interleavings possíveis.

Termo para lembrar: Cache fill race | Base técnica: referência 1.

13. Como trata cache indisponível?

Eu permitiria bypass dentro da capacidade protegida da origem, reduzindo funcionalidades ou admissão quando necessário. Mandar todo tráfego diretamente ao banco pode transformar uma falha opcional num incidente maior de toda a jornada.

Termo para lembrar: Cache bypass | Base técnica: referência 1.

14. Como trata origem indisponível?

Eu serviria valor antigo somente quando a política da tarefa permitir, com limite e sinalização. Para uma decisão financeira, dado velho pode ser inadequado; a alternativa correta pode ser pendência ou recusa temporária.

Termo para lembrar: Stale fallback | Base técnica: referência 1.

15. Como evita stampede no miss?

Eu reuniria solicitações concorrentes pela mesma chave em uma recomputação e limitaria loaders globalmente. Essa proteção precisa de prazo e recuperação; um loader travado não deve bloquear indefinidamente todos os consumidores daquela informação.

Termo para lembrar: Request coalescing | Base técnica: referência 1.

16. Como aquece cache?

Eu priorizaria chaves populares e capacidade da origem, evitando carregar todo universo sem evidência de reutilização. Aquecimento compete com tráfego real; limites e progresso gradual protegem o banco durante deploy ou recuperação do cluster.

Termo para lembrar: Warm-up | Base técnica: referência 1.

17. Como trata valor inexistente?

Eu cachearia ausência por prazo curto quando isso reduz consultas repetidas, distinguindo erro de não encontrado. Se um registro for criado, invalidaria a ausência; resposta negativa antiga não pode bloquear acesso legítimo indefinidamente.

Termo para lembrar: Negative caching | Base técnica: referência 1.

18. Pode cachear erro temporário?

Eu faria isso apenas com política explícita e prazo pequeno, evitando transformar timeout em inexistência permanente. Separaria indisponível de não encontrado; o cliente precisa saber se deve tentar depois ou corrigir sua requisição.

Termo para lembrar: Error caching | Base técnica: referência 1.

19. Como controla tamanho de entrada?

Eu imporia limite de bytes e avaliaria armazenar resumo ou referência. Uma entrada gigante consome memória e rede e pode expulsar muitas úteis; hit ratio por quantidade não revela esse custo desproporcional.

Termo para lembrar: Entry size limit | Base técnica: referência 1.

20. Como escolhe TTL?

Eu partiria do atraso aceitável e da frequência de mudança, não de um número universal. TTL curto aumenta misses; longo aumenta risco de informação antiga. Invalidação por evento e versão podem complementar, mas também falham.

Termo para lembrar: Freshness policy | Base técnica: referência 1.

21. Como usa jitter?

Eu espalharia expirações dentro de uma faixa permitida para evitar recomputação simultânea. O limite máximo de frescor deve continuar respeitado; adicionar jitter sem considerar a política pode prolongar dados além do que o produto aceita.

Termo para lembrar: Expiration jitter | Base técnica: referência 1.

22. Como trata L1 e L2?

Eu usaria cache local para acesso muito frequente e compartilhado para reutilização entre instâncias, com versão e invalidação coerentes. Cada nível acrescenta uma cópia; a estratégia precisa definir o que acontece quando a conexão de invalidação cai.

Termo para lembrar: Multilevel caching | Base técnica: referência 1.

23. Como observa cache?

Eu mediria hits, misses, bytes, latência, evictions e carga evitada na origem, segmentando dados críticos. Uma taxa de hit boa não comprova correção; comparações amostradas e frescor ajudam detectar valores antigos ou chaves mal definidas.

Termo para lembrar: Cache observability | Base técnica: referência 1.

24. Como faz rollout de formato?

Eu colocaria versão no namespace ou payload e manteria leitores compatíveis durante a transição. Reutilizar a mesma chave com formato incompatível pode quebrar instâncias antigas; expiração e limpeza completam a migração planejada.

Termo para lembrar: Cache schema version | Base técnica: referência 1.

25. Como protege write-behind contra perda?

Eu persistiria a obrigação antes de confirmar, com identidade, replay e ordenação apropriados. Nesse ponto não é apenas cache descartável; o log durável passa a fazer parte da autoridade e exige recuperação e monitoramento próprios.

Termo para lembrar: Durable write queue | Base técnica: referência 1.

26. Como trata write-behind fora de ordem?

Eu preservaria versão por entidade e impediria gravação antiga sobrescrever estado novo. Se forem deltas, detectaria lacunas; uma fila com retries independentes pode inverter operações, portanto a semântica do dado define o protocolo necessário.

Termo para lembrar: Ordered persistence | Base técnica: referência 1.

27. Como testa o padrão?

Eu simularia queda de cache, origem lenta, invalidação perdida e preenchimento concorrente após escrita. Verificaria correção e capacidade de recuperação; benchmark de hits em cache quente não cobre os riscos mais importantes do desenho.

Termo para lembrar: Cache fault testing | Base técnica: referência 1.

28. Como decide cache versus índice?

Eu primeiro verificaria se consulta e índice adequados resolvem o custo com menos complexidade. Cache ajuda reutilização, mas pode esconder consulta ruim; quando misses acontecem, a origem continua precisando atender de forma previsível.

Termo para lembrar: Optimization hierarchy | Base técnica: referência 1.

29. Como trata dados sensíveis?

Eu minimizaria conteúdo, controlaria acesso e definira retenção também nas cópias e snapshots. Ser cache não remove responsabilidade por dados; logs, dump e ferramenta de inspeção podem expor o mesmo conteúdo da origem.

Termo para lembrar: Cache governance | Base técnica: referência 1.

30. Como defenderia sua escolha?

Eu diria qual dado cacheio, por quanto tempo, como invalido e como opero sem o cache. Também explicaria por que a jornada tolera atraso, demonstrando que o ganho de latência não enfraquece uma invariante financeira.

Termo para lembrar: Cache contract | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Cache-aside

Ganho. Controle explícito

Custo e motivo. Repete lógica no cliente

Como decidir. Quando aplicação coordena acesso

02. Read-through

Ganho. Centraliza loading

Custo e motivo. Esconde custo do miss

Como decidir. Quando loader tem contrato claro

03. Write-through

Ganho. Mantém caminho de persistência explícito

Custo e motivo. Aumenta latência de escrita

Como decidir. Quando confirmação exige origem

04. Write-behind

Ganho. Agrupa persistência

Custo e motivo. Cria janela e replay

Como decidir. Quando há durabilidade e semântica adequadas

05. TTL curto

Ganho. Limita atraso

Custo e motivo. Aumenta carga

Como decidir. Quando frescor é importante

06. TTL longo

Ganho. Melhora reutilização

Custo e motivo. Amplia staleness

Como decidir. Quando dado muda pouco

07. Cache local

Ganho. Reduz RTT

Custo e motivo. Duplica estado

Como decidir. Quando chave é muito quente

08. Cache compartilhado

Ganho. Reutiliza entre instâncias

Custo e motivo. Adiciona rede

Como decidir. Quando working set é comum

09. Negative cache

Ganho. Reduz misses inúteis

Custo e motivo. Pode esconder criação recente

Como decidir. Quando ausência se repete

10. Bypass

Ganho. Mantém serviço parcial

Custo e motivo. Pode saturar origem

Como decidir. Quando limitado por capacidade

Duas alternativas ao caminho proposto

1. Índices e query tuning

Quando faz sentido. Quando origem é lenta por acesso inadequado

Como implementar. Reduzir trabalho no banco sem cópia adicional

Limites. Não elimina custo de consultas repetidas complexas

2. Materialized view

Quando faz sentido. Quando resultado é derivado e compartilhado

Como implementar. Manter projeção atualizada por eventos ou jobs

Limites. Exige frescor, reconstrução e validação de autoridade

Referências e alcance da evidência

1. Microsoft: Cache-Aside

Leitura sob demanda, invalidação e limites de consistência entre cache e origem.

https://learn.microsoft.com/en-us/azure/architecture/patterns/cache-aside

2. Redis: Client-side caching

Tracking e invalidação para caches locais, com tratamento de conexão e atualização.

https://redis.io/docs/latest/develop/reference/client-side-caching/

3. Redis: Cluster scaling

Particionamento por slots, redirecionamento e limites de operações entre chaves no Redis Cluster.

https://redis.io/docs/latest/operate/oss_and_stack/management/scaling/

Continue em: T032 · T034 · T037

T032 / D. Cache e modelos de dados

Invalidação de cache, TTL e versões

Invalidação remove ou substitui dados potencialmente antigos; TTL limita reutilização temporal; versões permitem reconhecer regressões. A estratégia precisa lidar com falhas, eventos atrasados e corridas de preenchimento.

Quando usar. Quando origem muda enquanto leituras usam cópias locais, compartilhadas ou HTTP.

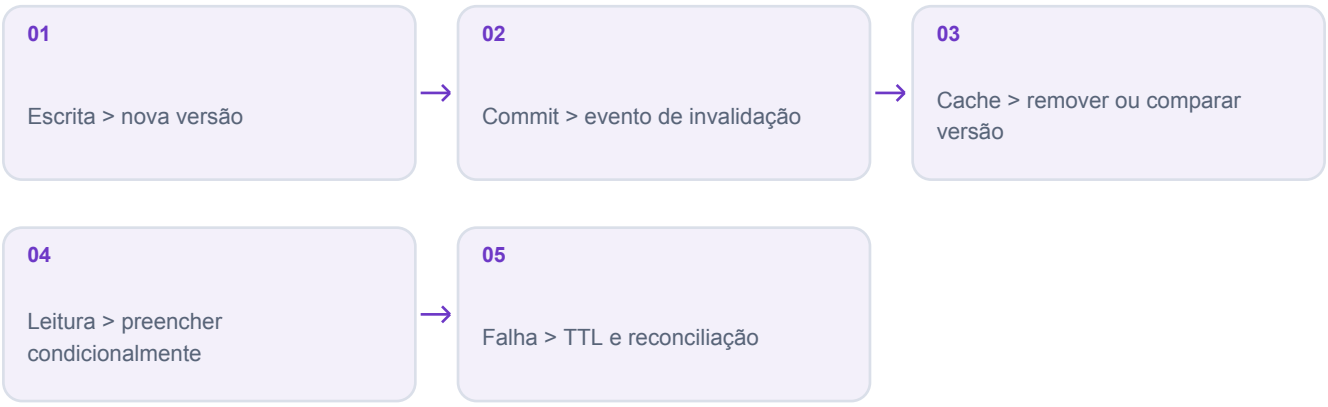
Quando evitar ou limitar. Evitar supor que TTL garante atualização instantânea, que delete perdido é inofensivo ou que evento sem versão pode sobrescrever qualquer valor.

Uma abertura possível

Vou desenhar uma escrita concorrente com um miss para mostrar a corrida. Usarei versão e limites de frescor, deixando explícito o que o usuário vê quando a invalidação atrasa.

Peças e tecnologias. Redis · ETag · Cache-Control · CDC · versioned keys · client tracking

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Definir atraso permitido por dado.

2. Associar versão à autoridade.

3. Escolher invalidação, atualização ou namespace.

4. Tratar perda e desordem de eventos.

5. Testar corrida, desconexão e criação após ausência.

30 perguntas para sustentar o desenho

01. TTL garante que o dado está atual?

Eu diria que TTL limita quanto tempo uma entrada é reutilizada segundo a política, não prova que a origem não mudou. Mesmo recém-carregado, o valor pode envelhecer imediatamente; decisões críticas precisam de outra garantia.

Termo para lembrar: TTL limitation | Base técnica: referência 1.

02. Como acontece preenchimento antigo após invalidação?

Eu mostraria uma leitura iniciada antes do commit que termina depois do delete e grava valor antigo. A ordem das chamadas individuais parece correta, mas a interleaving reintroduz staleness; versão ajuda reconhecer a regressão.

Termo para lembrar: Stale fill race | Base técnica: referência 1.

03. Como usa versão no cache?

Eu armazenaria valor e versão da origem, aplicando atualização somente quando não regredir. A comparação precisa ter escopo definido; versões de entidades diferentes não são intercambiáveis e um timestamp arbitrário pode não representar ordem confiável.

Termo para lembrar: Versioned value | Base técnica: referência 1.

04. Versão basta se a chave foi removida?

Eu avaliaria preservar um marcador mínimo ou usar namespace versionado, porque remover também a versão pode permitir preenchimento antigo. A solução depende de coordenação entre leitura e escrita; guardar somente o payload não resolve essa corrida.

Termo para lembrar: Version watermark | Base técnica: referência 1.

05. Como funciona chave versionada?

Eu incluiria versão imutável na chave e manteria referência ao valor atual quando necessário. Isso evita sobrescrita entre versões, mas a referência também precisa de consistência; entradas antigas exigem expiração ou coleta planejada.

Termo para lembrar: Versioned key | Base técnica: referência 1.

06. Como invalida por evento?

Eu publicaria a mudança por caminho durável e faria consumidores idempotentes, monitorando atraso. Eventos podem repetir ou chegar tarde; a invalidação deve tolerar isso e possuir TTL ou mecanismo de recuperação quando a conexão falha.

Termo para lembrar: Event-driven invalidation | Base técnica: referência 1.

07. Pode usar Pub/Sub para invalidação?

Eu usaria quando perda é coberta por expiração ou reconstrução segura, reconhecendo que consumidores desconectados podem perder sinais. Para garantias mais fortes, precisaria de progresso recuperável ou invalidar o cache local ao reconectar.

Termo para lembrar: Best-effort invalidation | Base técnica: referência 1.

08. Como usa client tracking do Redis?

Eu seguiria o protocolo de tracking e trataria perda de conexão como possível perda de invalidações. O cliente precisa limpar ou revalidar estado conforme a implementação; manter cache local indefinidamente após desconectar viola a suposição de frescor.

Termo para lembrar: Server-assisted caching | Base técnica: referência 1.

09. Como trata invalidar antes do commit?

Eu evitaria permitir que outro leitor recarregue o estado antigo enquanto a escrita ainda não confirmou. A invalidação precisa ser coordenada com a autoridade; caso contrário, o cache pode ficar desatualizado imediatamente após a transação.

Termo para lembrar: Commit ordering | Base técnica: referência 1.

10. Atualizar em vez de apagar é melhor?

Eu compararia custo de obter o valor correto, ordem e frequência de leitura. Atualização pode evitar miss, mas exige versão para impedir regressão; invalidação é simples, porém transfere recomputação ao próximo leitor.

Termo para lembrar: Update versus invalidate | Base técnica: referência 1.

11. Como escolhe TTL por entidade?

Eu usaria frequência de mudança, custo de miss e atraso permitido, com limites claros. Dados estáveis podem permanecer mais; informação usada para decisão sensível exige política mais restritiva e possivelmente consulta autoritativa.

Termo para lembrar: Adaptive expiry | Base técnica: referência 1.

12. Como usa soft e hard TTL?

Eu permitiria servir valor antigo durante uma janela curta enquanto um único loader atualiza, encerrando uso no limite rígido. Essa técnica depende de tolerância do produto; não aplicaria silenciosamente a dados que autorizam dinheiro.

Termo para lembrar: Stale-while-revalidate | Base técnica: referência 1.

13. Como evita expiração simultânea?

Eu distribuiria prazos numa faixa que preserve o limite máximo de frescor, reduzindo pico de recomputação. Jitter resolve sincronização entre chaves, mas uma chave muito popular ainda precisa de single-flight e proteção da origem.

Termo para lembrar: TTL jitter | Base técnica: referência 1.

14. Como invalida resultados de lista?

Eu identificaria quais consultas derivadas dependem da mudança, usando tags, versões de coleção ou TTL apropriado. Invalidar só o item não atualiza uma lista que deveria incluí-lo; o custo cresce com a quantidade de dependências.

Termo para lembrar: Dependency invalidation | Base técnica: referência 1.

15. Como trata paginação cacheada?

Eu incluiria filtros, ordenação, cursor e escopo de autorização na chave, definindo estabilidade entre páginas. Uma alteração pode invalidar apenas parte da navegação; não prometeria snapshot coerente sem mecanismo que preserve essa visão.

Termo para lembrar: Query cache key | Base técnica: referência 1.

16. Como cacheia ausência?

Eu usaria TTL curto e invalidaria quando a entidade fosse criada, distinguindo não encontrado de erro. Uma ausência antiga pode esconder cadastro recém-criado; a política precisa cobrir essa transição, não apenas updates de registros existentes.

Termo para lembrar: Negative-entry invalidation | Base técnica: referência 1.

17. Como trata revogação de permissão?

Eu avaliaria prazo máximo de revogação e revalidaria ações sensíveis na autoridade. Cache de permissão exige estratégia própria; manter dados visíveis por TTL longo pode ser incompatível com a promessa de bloqueio imediato ao usuário.

Termo para lembrar: Authorization freshness | Base técnica: referência 1.

18. ETag resolve concorrência e cache?

Eu o usaria conforme semântica HTTP para validação ou precondição, distinguindo esses usos. O servidor gera e verifica o identificador corretamente; um ETag não torna atomicamente consistente um conjunto de serviços independentes.

Termo para lembrar: Entity tag | Base técnica: referência 1.

19. Como diferencia no-cache e no-store?

Eu seguiria a semântica do protocolo: revalidação e proibição de armazenamento são objetivos distintos. Escolher a diretiva errada pode permitir cópia de informação sensível ou reduzir desempenho sem oferecer a proteção realmente necessária.

Termo para lembrar: HTTP cache directives | Base técnica: referência 1.

20. Como usa cache privado e compartilhado?

Eu classificaria se a resposta pode ser reutilizada entre usuários e quais campos variam. Autorização e Vary precisam ser considerados; cache compartilhado mal configurado pode devolver dados pessoais mesmo quando o endpoint exige autenticação.

Termo para lembrar: Cache scope | Base técnica: referência 1.

21. Como invalida várias regiões?

Eu propagaria versão e eventos com acompanhamento por região, permitindo limite de atraso ou recusa conforme a tarefa. Comunicação inter-regional pode falhar; o contrato não pode presumir invalidação simultânea universal sem coordenação correspondente.

Termo para lembrar: Regional invalidation | Base técnica: referência 1.

22. Como trata evento antigo depois do novo?

Eu compararia versões e ignoraria atualização regressiva quando o evento contém snapshot completo. Para deltas, não posso simplesmente pular lacunas; a semântica exige reconstrução ou aplicação na sequência correta antes de servir resultado.

Termo para lembrar: Monotonic cache update | Base técnica: referência 1.

23. Como mede staleness?

Eu registraria versão e instante autoritativo, comparando amostras ou progresso do pipeline. Hit ratio não mede atualidade; preciso detectar quanto tempo a cópia fica atrás e se esse atraso viola a jornada prometida.

Termo para lembrar: Freshness measurement | Base técnica: referência 1.

24. Como lida com cache sem origem disponível?

Eu definiria quais dados podem ser usados temporariamente e por quanto tempo, sinalizando a condição. Essa decisão depende do risco; aprovação financeira não pode reutilizar informação velha só para manter uma tela aparentemente funcional.

Termo para lembrar: Stale-if-error policy | Base técnica: referência 1.

25. Como faz invalidação em massa?

Eu poderia trocar uma versão de namespace para tornar entradas antigas inacessíveis, limpando depois. Isso é rápido no controle, mas provoca cache frio; limitaria recomputação para não deslocar o incidente inteiro para a origem.

Termo para lembrar: Namespace rotation | Base técnica: referência 1.

26. Como trata deploy com formato novo?

Eu usaria versão de formato e leitores compatíveis durante coexistência, sem confundir versão de schema com versão de dado. Rollback precisa entender entradas existentes ou mudar namespace; desserialização não deve depender da ordem do rollout.

Termo para lembrar: Schema-aware cache | Base técnica: referência 1.

27. Como protege contra poisoning?

Eu validaria entradas e construíria chaves apenas com parâmetros semanticamente relevantes e autorizados. Conteúdo controlado por atacante não pode poluir respostas compartilhadas; origem e headers precisam de normalização e políticas coerentes de cache.

Termo para lembrar: Cache poisoning defense | Base técnica: referência 1.

28. Como testa invalidação?

Eu forçaria leitura lenta antes de uma escrita, perderia o evento e reconectaria consumidores, verificando limite de atraso. Esses testes revelam corridas que não aparecem quando executamos salvar e ler sequencialmente numa única thread.

Termo para lembrar: Concurrency fault test | Base técnica: referência 1.

29. Como decide entre invalidação precisa e TTL?

Eu usaria evento e versão quando atraso precisa ser menor e dependências são conhecidas; TTL pode bastar para dados menos críticos. Precisão adicional custa metadados e operação, então a decisão acompanha a consequência do dado antigo.

Termo para lembrar: Coherence trade-off | Base técnica: referência 1.

30. Como explica a garantia final?

Eu declararia o atraso permitido, como detecto versão antiga e o que faço quando não consigo cumprir. A resposta pode aguardar, consultar origem ou falhar; o importante é não degradar a garantia silenciosamente durante incidentes.

Termo para lembrar: Bounded freshness contract | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. TTL curto

Ganho. Limita reutilização

Custo e motivo. Aumenta misses

Como decidir. Quando dado muda rápido

02. TTL longo

Ganho. Reduz origem

Custo e motivo. Amplia atraso

Como decidir. Quando dado é estável

03. Invalidação por evento

Ganho. Reage cedo

Custo e motivo. Pode atrasar ou perder

Como decidir. Quando pipeline é monitorado

04. Atualização por versão

Ganho. Evita regressão

Custo e motivo. Exige metadados

Como decidir. Quando eventos repetem

05. Namespace novo

Ganho. Invalida em massa

Custo e motivo. Provoca cache frio

Como decidir. Quando formato muda

06. Soft TTL

Ganho. Mantém resposta rápida

Custo e motivo. Serve dado antigo

Como decidir. Quando produto permite

07. Hard TTL

Ganho. Impõe limite

Custo e motivo. Pode indisponibilizar

Como decidir. Quando frescor é obrigatório

08. Tags de dependência

Ganho. Invalidam listas

Custo e motivo. Aumentam complexidade

Como decidir. Quando consultas derivadas são importantes

09. Revalidação HTTP

Ganho. Economiza payload

Custo e motivo. Mantém chamada

Como decidir. Quando cliente suporta validadores

10. Cache privado

Ganho. Protege contexto

Custo e motivo. Reduz compartilhamento

Como decidir. Quando resposta é personalizada

Duas alternativas ao caminho proposto

1. Leitura autoritativa

Quando faz sentido. Quando atraso é inaceitável e volume cabe

Como implementar. Consultar fonte com garantia adequada

Limites. Aumenta latência e carga da autoridade

2. Projeção versionada

Quando faz sentido. Quando várias consultas precisam de frescor rastreável

Como implementar. Manter read model com posição de origem e barreiras de leitura

Limites. Exige pipeline e recuperação, não apenas TTL

Referências e alcance da evidência

1. Microsoft: Cache-Aside

Leitura sob demanda, invalidação e limites de consistência entre cache e origem.

https://learn.microsoft.com/en-us/azure/architecture/patterns/cache-aside

2. Redis: Client-side caching

Tracking e invalidação para caches locais, com tratamento de conexão e atualização.

https://redis.io/docs/latest/develop/reference/client-side-caching/

3. IETF: HTTP Caching, RFC 9111

Diretivas de cache, validação e escopo de reutilização HTTP.

https://www.rfc-editor.org/rfc/rfc9111

Continue em: T012 · T031 · T034

T033 / D. Cache e modelos de dados

Eviction e admissão em cache

Eviction escolhe o que remover quando recursos faltam; admissão decide o que merece entrar. A política busca valor por memória e custo de recomputação, não apenas maximizar quantidade de hits.

Quando usar. Quando o working set excede memória ou acessos pouco reutilizados expulsam dados úteis.

Quando evitar ou limitar. Evitar misturar cache descartável com dedupe ou locks sujeitos à mesma eviction, e ignorar tamanho dos valores no dimensionamento.

Uma abertura possível

Vou medir quais entradas evitam trabalho relevante e reservar memória para overhead. A política deve reagir ao padrão real sem expulsar estados cuja perda quebra uma garantia de negócio.

Peças e tecnologias. Redis LRU · Redis LFU · Caffeine TinyLFU · maxmemory · weighted cache

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Medir working set e distribuição.

2. Separar dados descartáveis e críticos.

3. Escolher admissão e eviction.

4. Definir limite com overhead e bytes.

5. Testar scans, mudanças de popularidade e recuperação.

30 perguntas para sustentar o desenho

01. Qual diferença entre expiração e eviction?

Eu usaria expiração para limitar validade temporal e eviction para recuperar recursos sob pressão. Uma entrada válida pode ser expulsa antes do TTL; a aplicação deve tratar miss corretamente sem assumir permanência garantida.

Termo para lembrar: Expiry versus eviction | Base técnica: referência 1.

02. Qual diferença entre admissão e eviction?

Eu decidiria primeiro se um novo item merece espaço e depois qual vítima remover quando necessário. Admitir toda leitura pode poluir o cache com acessos únicos, expulsando dados que seriam reutilizados muitas vezes.

Termo para lembrar: Admission policy | Base técnica: referência 1.

03. Como funciona LRU?

Eu favoreceria itens acessados recentemente, esperando localidade temporal. A política pode funcionar bem, mas um scan grande de uso único pode expulsar o working set; implementação exata ou aproximada também possui custo operacional.

Termo para lembrar: Least recently used | Base técnica: referência 1.

04. Como funciona LFU?

Eu priorizaria frequência de acesso, usando envelhecimento para não preservar popularidade antiga indefinidamente. Sem adaptação, um item famoso no passado pode ocupar espaço depois que a carga mudou; a política precisa acompanhar o workload.

Termo para lembrar: Least frequently used | Base técnica: referência 1.

05. Quando FIFO pode bastar?

Eu aceitaria simplicidade quando padrão e custo justificarem, reconhecendo que ordem de entrada ignora reutilização. Mediria resultado; uma política sofisticada não vale seu overhead se o conjunto é pequeno ou quase todo acesso é novo.

Termo para lembrar: FIFO eviction | Base técnica: referência 1.

06. Por que TinyLFU considera admissão?

Eu compararia a frequência estimada do candidato com a vítima, evitando que um acesso isolado expulse um item útil. A aproximação economiza memória, mas exige configuração e avaliação sob mudanças de padrão e volume.

Termo para lembrar: Frequency-based admission | Base técnica: referência 1.

07. Hit ratio é a única métrica?

Eu mediria bytes, latência evitada e custo da origem, além da proporção de hits. Um hit barato e um hit que evita consulta pesada têm valores diferentes; otimizar só quantidade pode piorar o resultado econômico.

Termo para lembrar: Cache value | Base técnica: referência 1.

08. Como usa limite por peso?

Eu atribuiria peso aproximado pelo tamanho ou custo relevante, impedindo objetos grandes de dominar memória. O peso precisa ser atualizado quando o valor muda; medir apenas número de entradas esconde diferenças enormes de consumo.

Termo para lembrar: Weighted capacity | Base técnica: referência 1.

09. Como calcula overhead?

Eu incluiria chaves, metadados, estruturas, fragmentação, buffers e replicação, não apenas bytes do payload. O limite configurado não representa toda memória do processo; deixaria margem para manutenção e picos de alocação.

Termo para lembrar: Memory accounting | Base técnica: referência 1.

10. Noeviction evita falha?

Eu explicaria que preservar chaves pode fazer novas escritas falharem quando o limite chega. Isso pode ser adequado para certas responsabilidades, mas exige capacidade e tratamento de erro; não transforma memória finita em armazenamento ilimitado.

Termo para lembrar: Noeviction policy | Base técnica: referência 1.

11. Eviction pode remover idempotency key?

Eu não colocaria a garantia principal de dedupe num cache descartável sem outra autoridade. Se a chave some, o retry pode executar novamente; estado crítico precisa de retenção e durabilidade compatíveis com o horizonte de repetição.

Termo para lembrar: Correctness state | Base técnica: referência 1.

12. Como separa caches de criticidades diferentes?

Eu usaria limites ou instâncias separadas quando eviction e persistência divergem. Uma explosão de catálogo não deve expulsar sessões ou obrigações; compartilhar infraestrutura só é aceitável se o impacto de cada política estiver controlado.

Termo para lembrar: Policy isolation | Base técnica: referência 1.

13. O que é cache pollution?

Eu identificaria acessos de baixa reutilização que consomem espaço e expulsam itens valiosos. Poderia usar admissão por frequência, bypass de scans ou quotas por classe, medindo se o working set útil volta a permanecer.

Termo para lembrar: Cache pollution | Base técnica: referência 1.

14. Como trata um scan analítico?

Eu evitaria preencher o cache operacional com cada item de uma varredura única, usando bypass ou política resistente. Esse fluxo pode ter grande volume sem valor de reutilização, prejudicando consultas interativas durante e depois da execução.

Termo para lembrar: Scan resistance | Base técnica: referência 1.

15. Como reage a mudança de popularidade?

Eu escolheria mecanismos de envelhecimento e mediria tempo de adaptação, porque tráfego muda por campanhas e horários. Preservar itens antigos demais reduz hit ratio; reagir rápido demais pode perder estabilidade diante de ruído passageiro.

Termo para lembrar: Workload adaptation | Base técnica: referência 1.

16. Como dimensiona TTL junto com eviction?

Eu separaria validade de prioridade de memória, garantindo que nenhuma política sirva valor vencido. TTL curto pode reduzir o conjunto, mas também causar misses sincronizados; eviction não substitui a definição de frescor da informação.

Termo para lembrar: Policy interaction | Base técnica: referência 1.

17. Como trata item muito grande?

Eu poderia recusar admissão, armazenar resumo ou separar conteúdo, conforme a tarefa. Um único objeto pode expulsar centenas de itens úteis; a política deve considerar benefício e custo, não aceitar qualquer tamanho automaticamente.

Termo para lembrar: Oversized entry | Base técnica: referência 1.

18. Como trata tenant dominante?

Eu consideraria quotas ou partições lógicas de capacidade para evitar que um cliente expulse o working set dos demais. Isso melhora previsibilidade, mas pode deixar espaço subutilizado; ajustaria conforme isolamento e eficiência desejados.

Termo para lembrar: Cache fairness | Base técnica: referência 1.

19. Como avalia cache local na JVM?

Eu incluiria retenção de objetos e impacto de GC, usando limites e medição de alocação. Cache em processo reduz rede, mas ocupa o mesmo heap da aplicação; hit ratio alto não compensa pausas que prejudicam toda a jornada.

Termo para lembrar: Heap pressure | Base técnica: referência 1.

20. Como mede churn?

Eu acompanharia taxa de inserção e remoção, hits por idade e carga na origem. Entradas entrando e saindo rapidamente podem indicar cache pequeno ou admissão ruim; simplesmente aumentar TTL não resolve pressão de memória.

Termo para lembrar: Eviction churn | Base técnica: referência 1.

21. Como testa uma política?

Eu usaria sequência representativa de acessos com tamanhos e custos, comparando políticas antes de produção. Depois validaria overhead real; simulação de hits não mede serialização, concorrência e pausas do runtime que também afetam desempenho.

Termo para lembrar: Trace-driven simulation | Base técnica: referência 1.

22. Como faz rollout de nova política?

Eu aplicaria gradualmente e observaria origem, latência e memória, com critério de reversão. A mudança pode esfriar o cache ou alterar vítimas; comparar apenas uso de RAM não demonstra melhora para os usuários.

Termo para lembrar: Policy rollout | Base técnica: referência 1.

23. Como trata eviction em cluster?

Eu observaria memória e carga por nó, porque limite global aparente pode esconder um shard saturado. Distribuição de chaves e tamanho desigual causam expulsões localizadas mesmo quando outros nós ainda possuem bastante espaço livre.

Termo para lembrar: Per-node pressure | Base técnica: referência 1.

24. Pode usar eviction para cleanup de negócio?

Eu manteria expiração funcional e retenção em mecanismos explícitos, não na pressão de memória. Eviction é imprevisível para a aplicação; não serve como prova de que uma obrigação venceu ou um documento foi removido conforme política.

Termo para lembrar: Lifecycle separation | Base técnica: referência 1.

25. Como reage ao cache inteiro perdido?

Eu protegeria origem com admissão de requisições, single-flight e aquecimento prioritário. A política de eviction só governa memória ocupada; depois da perda, o principal risco é recomputação simultânea exceder a capacidade do banco.

Termo para lembrar: Repopulation strategy | Base técnica: referência 1.

26. Como compara LFU e LRU?

Eu observaria se frequência estável ou recência melhor prediz reutilização. LRU adapta rápido e pode sofrer com scans; LFU preserva populares, mas precisa envelhecer. Escolheria com tráfego representativo, não por preferência abstrata.

Termo para lembrar: Locality profile | Base técnica: referência 1.

27. Quando política aleatória é aceitável?

Eu consideraria overhead baixo e carga pouco previsível, comparando empiricamente. Aleatoriedade pode ser suficiente em alguns cenários, mas não protege itens caros ou críticos; essas classes exigem outra política ou isolamento explícito.

Termo para lembrar: Random eviction | Base técnica: referência 1.

28. Como evita metric cardinality explosion?

Eu agregaria por classe, tenant controlado ou faixa de tamanho, sem label para cada chave. Métricas detalhadas demais podem custar mais que o cache; amostragem e ferramentas específicas ajudam investigar hotspots sem explodir observabilidade.

Termo para lembrar: Cache metrics design | Base técnica: referência 1.

29. Como estima custo de um miss?

Eu mediria latência, CPU e I/O da origem por classe, incluindo trabalho concorrente gerado. Um miss em chave barata pode ser tolerável; itens que disparam fan-out caro merecem prioridade maior quando a política permite.

Termo para lembrar: Miss penalty | Base técnica: referência 1.

30. Como defenderia a configuração?

Eu mostraria o working set, limite de memória e benefício por classe, explicando quais dados podem desaparecer. A configuração é correta quando melhora a jornada e protege a origem sem transformar eviction em violação de regra financeira.

Termo para lembrar: Policy rationale | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. LRU

Ganho. Favorece recência

Custo e motivo. Sofre com scans

Como decidir. Quando localidade temporal domina

02. LFU

Ganho. Preserva populares

Custo e motivo. Precisa envelhecer

Como decidir. Quando frequência é estável

03. FIFO

Ganho. Reduz overhead

Custo e motivo. Ignora reutilização

Como decidir. Quando simplicidade atende

04. TinyLFU

Ganho. Rejeita candidatos fracos

Custo e motivo. Mantém estimativas

Como decidir. Quando scans poluem cache

05. Peso por bytes

Ganho. Controla memória

Custo e motivo. Pode rejeitar item valioso

Como decidir. Quando tamanhos variam

06. Quota por tenant

Ganho. Melhora isolamento

Custo e motivo. Pode desperdiçar espaço

Como decidir. Quando fairness importa

07. Noeviction

Ganho. Preserva existentes

Custo e motivo. Falha novas escritas

Como decidir. Quando expulsão é inaceitável

08. Cache maior

Ganho. Aumenta working set

Custo e motivo. Custa memória

Como decidir. Quando reutilização justifica

09. Bypass de scan

Ganho. Protege populares

Custo e motivo. Perde possível reutilização

Como decidir. Quando acesso é único

10. Envelhecimento rápido

Ganho. Adapta a mudanças

Custo e motivo. Perde estabilidade

Como decidir. Quando popularidade muda depressa

Duas alternativas ao caminho proposto

1. Sem cache para classe barata

Quando faz sentido. Quando miss custa pouco e reutilização é baixa

Como implementar. Consultar origem com índice e limites

Limites. A origem precisa suportar a carga

2. Projeção materializada

Quando faz sentido. Quando resultado caro deve permanecer disponível

Como implementar. Persistir modelo de leitura reconstruível

Limites. Exige pipeline e armazenamento, não apenas memória

Referências e alcance da evidência

1. Redis: Key eviction

Políticas de eviction, limites de memória e estatísticas associadas.

https://redis.io/docs/latest/develop/reference/eviction/

2. Caffeine: Efficiency

Políticas de admissão e eviction orientadas a frequência e comparação de eficiência.

https://github.com/ben-manes/caffeine/wiki/Efficiency

3. Redis: Persistence

RDB, AOF, recuperação e compromissos de durabilidade e desempenho.

https://redis.io/docs/latest/operate/oss_and_stack/management/persistence/

Continue em: T027 · T031 · T034

T034 / D. Cache e modelos de dados

Cache stampede, single-flight e hot keys

Stampede ocorre quando muitos misses disparam recomputação simultânea. Single-flight reúne chamadas pela mesma chave; hot keys concentram tráfego mesmo com hit e exigem limites, distribuição ou cache local.

Quando usar. Quando expiração, cache frio ou uma entidade popular ameaça a origem e a cauda de latência.

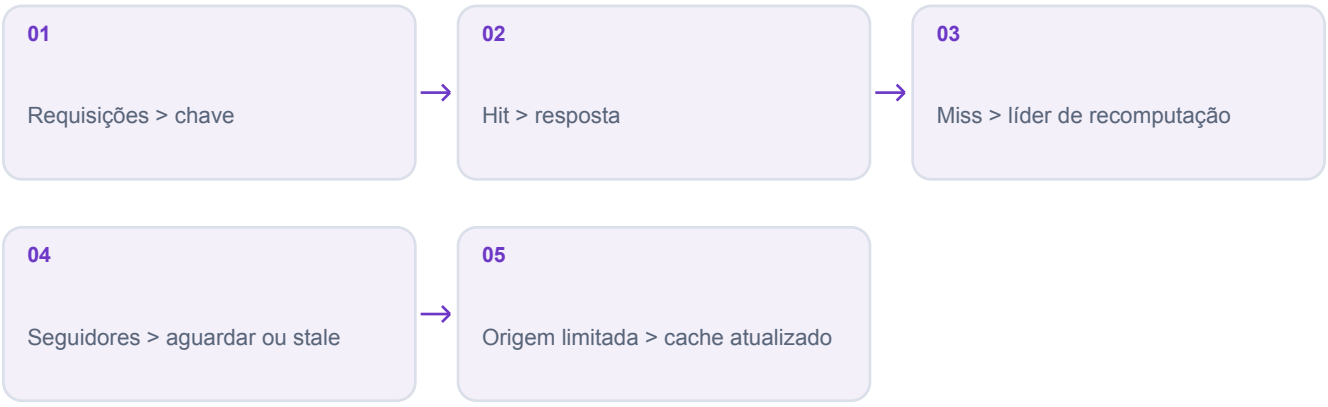
Quando evitar ou limitar. Evitar lock sem timeout, confundir single-flight local com coordenação global ou espalhar writes de uma chave sem preservar sua semântica.

Uma abertura possível

Vou impedir que uma chave expirada gere milhares de consultas iguais. Também separarei proteção por chave de limite global, porque muitas chaves diferentes podem derrubar a origem do mesmo jeito.

Peças e tecnologias. singleflight · Caffeine async loading · Redis · TTL jitter · stale-while-revalidate · semaphore

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Identificar expiração correlacionada e chaves quentes.

2. Coalescer recomputações por chave.

3. Limitar loaders globalmente e por destino.

4. Definir stale, timeout e recuperação.

5. Testar cache frio e falha do loader.

30 perguntas para sustentar o desenho

01. Como nasce um stampede?

Eu mostraria uma chave popular expirando enquanto muitas requisições chegam. Todas consultam a origem ao mesmo tempo, repetindo o mesmo trabalho; a latência cresce e retries podem amplificar ainda mais a carga gerada.

Termo para lembrar: Cache stampede | Base técnica: referência 1.

02. Como single-flight resolve?

Eu permitiria uma recomputação em andamento por chave e faria seguidores compartilhar seu resultado. Isso reduz duplicação local, mas preciso de limites de espera, memória e tratamento de erro quando o loader falha ou trava.

Termo para lembrar: Request coalescing | Base técnica: referência 1.

03. Single-flight de uma instância protege o cluster?

Eu explicaria que cada processo pode ainda executar um loader para a mesma chave. Dependendo da frota, isso já reduz bastante carga; coordenação distribuída só se justifica se o volume residual continuar excessivo.

Termo para lembrar: Coordination scope | Base técnica: referência 1.

04. Como faz coordenação distribuída?

Eu poderia usar uma lease curta por chave e liberação condicionada ao token, mantendo timeout e fallback. Não trataria o mecanismo como garantia financeira; sua função é reduzir trabalho, aceitando duplicação limitada quando a coordenação falha.

Termo para lembrar: Distributed recomputation lease | Base técnica: referência 1.

05. O que acontece se o loader morrer?

Eu encerraria espera por deadline e permitiria nova tentativa após a lease apropriada, preservando limites. Seguidores não podem ficar bloqueados para sempre; a política pode servir stale ou retornar indisponibilidade conforme a criticidade do dado.

Termo para lembrar: Loader recovery | Base técnica: referência 1.

06. Como lida com erro compartilhado?

Eu devolveria o erro aos seguidores e controlaria a próxima tentativa com backoff, evitando nova onda imediata. Diferenciaria indisponibilidade de ausência; cachear um timeout como não encontrado pode transformar falha temporária em informação falsa.

Termo para lembrar: Failure coalescing | Base técnica: referência 1.

07. TTL jitter resolve uma hot key?

Eu usaria jitter para espalhar expirações entre várias chaves, mas ele não impede milhares de acessos à mesma chave quando ela vence. Para isso preciso de coalescing, atualização antecipada ou política de stale adequada.

Termo para lembrar: Jitter scope | Base técnica: referência 1.

08. O que é stale-while-revalidate?

Eu serviria uma cópia ainda tolerável enquanto um loader atualiza em segundo plano, com limite rígido de idade. A técnica melhora cauda, mas depende do produto aceitar atraso; não a aplicaria automaticamente a autorização financeira.

Termo para lembrar: Stale-while-revalidate | Base técnica: referência 1.

09. Como define soft TTL?

Eu marcaria quando vale iniciar atualização sem impedir imediatamente o uso do valor existente. O hard TTL encerra reutilização; separar ambos ajuda estabilidade, mas exige comunicação clara da garantia de frescor oferecida ao consumidor.

Termo para lembrar: Soft expiry | Base técnica: referência 1.

10. Quando faz refresh antecipado?

Eu atualizaria itens populares antes do vencimento quando o benefício justificar chamadas extras. Não renovaria todo cache indiscriminadamente; isso pode gastar origem com dados nunca mais usados e manter carga alta durante períodos ociosos.

Termo para lembrar: Refresh ahead | Base técnica: referência 1.

11. Como limita recomputações distintas?

Eu usaria semáforo ou controle de concorrência por destino, porque single-flight só junta a mesma chave. Mil misses em chaves diferentes continuam gerando mil consultas; o limite agregado protege a capacidade real da origem.

Termo para lembrar: Global loader limit | Base técnica: referência 1.

12. Como trata cache totalmente frio?

Eu priorizaria tráfego essencial, aqueceria chaves populares e limitaria bypass, mantendo a origem dentro de capacidade. Liberar tudo após restart pode criar um pico maior que a carga normal e impedir o próprio aquecimento.

Termo para lembrar: Cold-cache recovery | Base técnica: referência 1.

13. Hot key sempre causa misses?

Eu separaria chave popular expirada de chave popular que acerta sempre, mas satura um nó ou rede. No segundo caso, cache local e replicação de leitura podem ajudar; coalescing de miss não resolve tráfego de hits.

Termo para lembrar: Hot-hit bottleneck | Base técnica: referência 1.

14. Como usa L1 para hot key?

Eu manteria cópia curta ou versionada nas instâncias, reduzindo chamadas ao cache compartilhado. Isso duplica estado e exige invalidação; para informação estável e muito acessada, o ganho pode compensar a complexidade adicional.

Termo para lembrar: Local cache | Base técnica: referência 1.

15. Pode dividir uma chave em várias?

Eu poderia replicar valor de leitura ou dividir agregação quando a semântica permitir. Escritas sobre um contador rígido não podem ser espalhadas arbitrariamente; a estratégia precisa preservar ordem, limite e atualização coerente das cópias.

Termo para lembrar: Key replication | Base técnica: referência 1.

16. Como evita fila de seguidores enorme?

Eu limitaria número e duração dos esperadores, oferecendo stale ou resposta controlada quando exceder. Coalescing reduz origem, mas uma lista ilimitada de clientes esperando o mesmo loader ainda pode esgotar memória da aplicação.

Termo para lembrar: Waiter bound | Base técnica: referência 1.

17. Como trata cancelamento de um seguidor?

Eu removeria a espera daquele cliente sem necessariamente cancelar o loader compartilhado usado por outros. O contexto da recomputação precisa de prazo próprio; vincular tudo ao primeiro cliente pode derrubar o trabalho quando ele desconecta.

Termo para lembrar: Cancellation ownership | Base técnica: referência 1.

18. Como evita loader antigo sobrescrever novo?

Eu associaria resultado à versão autoritativa e condicionaria publicação para não regredir. Uma tentativa lenta pode terminar depois de outra; a redução de duplicatas não substitui controle de frescor e ordenação do preenchimento.

Termo para lembrar: Versioned publication | Base técnica: referência 1.

19. Como trata valor inexistente popular?

Eu cachearia ausência por prazo curto e com invalidação na criação, evitando consultas repetidas inúteis. Também validaria entrada; um ataque com chaves aleatórias distintas pode escapar do single-flight e exigir outra proteção de admissão.

Termo para lembrar: Negative caching | Base técnica: referência 1.

20. Como detecta stampede?

Eu observaria aumento simultâneo de misses, loaders e latência da origem, correlacionado com expiração ou deploy. Hit ratio agregado pode parecer bom; métricas por classe e amostragem de chaves ajudam identificar concentração temporal.

Termo para lembrar: Correlated miss burst | Base técnica: referência 1.

21. Como mede eficiência do coalescing?

Eu compararia solicitações que compartilham resultado com execuções efetivas do loader, junto de espera e erros. Reduzir queries é útil, mas se todos aguardam um loader lento, a experiência pode continuar ruim.

Termo para lembrar: Shared-load ratio | Base técnica: referência 1.

22. Como escolhe timeout do loader?

Eu o alinharia ao orçamento de atualização e à opção de stale, não apenas ao timeout de cada cliente. Recomputação longa pode continuar útil, mas precisa de limite e proteção de origem para não ocupar recursos indefinidamente.

Termo para lembrar: Loader deadline | Base técnica: referência 1.

23. Como trata circuit breaker na origem?

Eu interromperia novas recomputações quando o destino está falhando, preservando dados toleráveis e limitando sondas de recuperação. Ao fechar o circuito, drenaria gradualmente; liberar todos os misses simultaneamente pode derrubar o serviço novamente.

Termo para lembrar: Origin protection | Base técnica: referência 1.

24. Como usa atualização probabilística?

Eu poderia antecipar recomputação com probabilidade relacionada à proximidade da expiração e ao custo, reduzindo sincronização. Ainda manteria coalescing e limites; aleatoriedade distribui trabalho, mas não oferece sozinha garantia de carga máxima.

Termo para lembrar: Probabilistic early refresh | Base técnica: referência 1.

25. Como protege múltiplos tenants?

Eu reservaria cotas ou fairness para loaders, evitando que um cliente com muitas chaves frias consuma toda capacidade. Limite global protege a origem, mas sem distribuição justa pode tornar os demais usuários indisponíveis.

Termo para lembrar: Tenant isolation | Base técnica: referência 1.

26. Como faz deploy sem esfriar tudo?

Eu substituiria instâncias gradualmente, preservando capacidade e aquecendo o necessário antes de ampliar tráfego. Reiniciar toda a frota de caches locais ao mesmo tempo cria carga sincronizada mesmo quando o cache compartilhado permanece saudável.

Termo para lembrar: Rolling warm-up | Base técnica: referência 1.

27. Como testa esse cenário?

Eu prepararia uma chave popular, expiraria sob concorrência e derrubaria o loader, medindo consultas e espera. Também testaria milhares de chaves diferentes; isso valida proteção por chave e agregada, que resolvem riscos distintos.

Termo para lembrar: Burst expiry test | Base técnica: referência 1.

28. Quando não usa stale?

Eu recusaria uso antigo quando pode autorizar uma ação inválida ou violar política de revogação. A disponibilidade aparente não compensa erro de negócio; retornaria pendência, consulta forte ou falha controlada conforme a jornada.

Termo para lembrar: Freshness boundary | Base técnica: referência 1.

29. Como evita lock virar dependência crítica?

Eu definiria fallback limitado quando coordenação de cache falhar, sem comprometer a autoridade. O objetivo é amortecer carga; se exigir disponibilidade perfeita do lock para toda leitura, posso adicionar mais um ponto de indisponibilidade desnecessário.

Termo para lembrar: Best-effort optimization | Base técnica: referência 1.

30. Como defenderia a solução?

Eu combinaria versão, coalescing por chave, limite global e experiência de stale quando permitida. Explicaria cache frio e loader morto, mostrando que a origem permanece protegida mesmo quando a taxa de hit cai abruptamente.

Termo para lembrar: Layered cache protection | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Single-flight local

Ganho. Reduz duplicação simples

Custo e motivo. Não coordena instâncias

Como decidir. Quando carga residual cabe

02. Lease distribuída

Ganho. Reduz duplicação global

Custo e motivo. Adiciona dependência

Como decidir. Quando origem é muito cara

03. Stale-while-revalidate

Ganho. Mantém latência

Custo e motivo. Serve atraso

Como decidir. Quando produto tolera

04. Refresh ahead

Ganho. Evita miss crítico

Custo e motivo. Pode desperdiçar trabalho

Como decidir. Quando chave é popular

05. TTL jitter

Ganho. Espalha expiração

Custo e motivo. Varia frescor

Como decidir. Quando muitas chaves vencem juntas

06. L1 local

Ganho. Alivia hot hits

Custo e motivo. Duplica estado

Como decidir. Quando valor é reutilizado intensamente

07. Limite global

Ganho. Protege origem

Custo e motivo. Pode rejeitar loaders

Como decidir. Quando capacidade é finita

08. Fila de seguidores

Ganho. Compartilha resultado

Custo e motivo. Consome memória

Como decidir. Quando espera é limitada

09. Negative cache

Ganho. Evita ausência repetida

Custo e motivo. Pode esconder criação

Como decidir. Quando prazo é curto

10. Warm-up gradual

Ganho. Suaviza recuperação

Custo e motivo. Demora preencher

Como decidir. Quando origem não suporta pico frio

Duas alternativas ao caminho proposto

1. Projeção pré-computada

Quando faz sentido. Quando resultado caro precisa de disponibilidade previsível

Como implementar. Atualizar por evento em vez de calcular no miss

Limites. Exige pipeline e frescor monitorado

2. Origem otimizada e admissão

Quando faz sentido. Quando reutilização é baixa

Como implementar. Melhorar índices e limitar concorrência sem cache

Limites. Pode manter latência maior, mas elimina coerência de cópias

Referências e alcance da evidência

1. Go: singleflight package

Supressão de chamadas duplicadas em andamento por chave dentro do escopo do Group.

https://pkg.go.dev/golang.org/x/sync/singleflight

2. Microsoft: Cache-Aside

Leitura sob demanda, invalidação e limites de consistência entre cache e origem.

https://learn.microsoft.com/en-us/azure/architecture/patterns/cache-aside

3. Google SRE: Handling Overload

Sobrecarga, filas e rejeição de trabalho; números e dimensionamentos são hipóteses didáticas.

https://sre.google/sre-book/handling-overload/

Continue em: T031 · T032 · T033

T035 / D. Cache e modelos de dados

Cache penetration e Bloom filters

Cache penetration ocorre quando acessos a dados ausentes atravessam continuamente o cache. Bloom filters respondem certamente ausente ou possivelmente presente para o conjunto representado, economizando espaço com falsos positivos.

Quando usar. Quando consultas negativas repetidas ou aleatórias pressionam uma origem cara e o conjunto de existentes pode ser mantido corretamente.

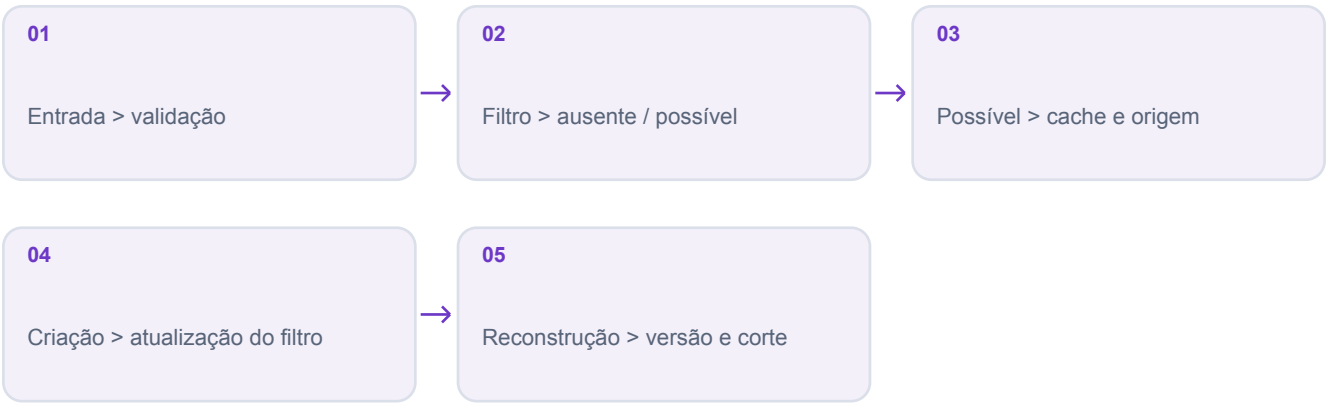
Quando evitar ou limitar. Evitar usar Bloom como prova de existência, autorização ou dedupe financeiro; filtros desatualizados podem rejeitar dados novos legitimamente existentes.

Uma abertura possível

Vou reduzir consultas inúteis sem transformar uma aproximação em autoridade. Um positivo continua consultando a origem; um negativo só pode bloquear quando sabemos que o filtro cobre a versão necessária do conjunto.

Peças e tecnologias. Bloom filter · RedisBloom · negative cache · Cuckoo filter · rate limiter

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Medir consultas negativas e sua origem.

2. Validar formato e autorização antes da busca.

3. Escolher negative cache ou filtro probabilístico.

4. Definir atualização, versão e política de falha.

5. Testar falsos positivos e criação concorrente.

30 perguntas para sustentar o desenho

01. O que é cache penetration?

Eu identificaria consultas a chaves ausentes que chegam repetidamente à origem porque não há valor para cachear. Pode ser uso legítimo ou abuso; a solução combina validação, cache negativo e limites, conforme o padrão.

Termo para lembrar: Cache penetration | Base técnica: referência 1.

02. Como negative caching ajuda?

Eu armazenaria ausência por um prazo curto, evitando repetir a mesma consulta vazia. Preciso distinguir não encontrado de erro e invalidar quando o dado nasce; caso contrário, escondo uma criação legítima do usuário.

Termo para lembrar: Negative caching | Base técnica: referência 1.

03. Por que não cachear toda ausência para sempre?

Eu limitaria retenção e admissão porque chaves inexistentes podem ser infinitas e posteriormente existir. Um ataque com identificadores aleatórios enche memória; a política deve reduzir trabalho sem criar outro recurso ilimitado para o atacante.

Termo para lembrar: Negative-entry lifecycle | Base técnica: referência 1.

04. Como funciona Bloom filter?

Eu representaria membros por bits selecionados por funções de hash. Se algum bit necessário está ausente, o item não pertence ao conjunto representado; todos presentes significam apenas possibilidade, exigindo confirmação na fonte autoritativa.

Termo para lembrar: Probabilistic membership | Base técnica: referência 1.

05. Bloom pode ter falso positivo?

Eu explicaria que colisões podem fazer uma chave ausente parecer presente, causando uma consulta extra. Isso afeta eficiência, não deve conceder acesso; qualquer decisão de existência ou autorização continua sendo validada pelo sistema responsável.

Termo para lembrar: False positive | Base técnica: referência 1.

06. Bloom pode ter falso negativo?

Eu distinguiria a estrutura corretamente mantida do sistema completo. Para membros inseridos, o filtro padrão não deve negar presença; porém atualização perdida, reconstrução incompleta ou dado recém-criado ainda não inserido pode gerar rejeição operacional indevida.

Termo para lembrar: Coverage assumption | Base técnica: referência 1.

07. Como dimensiona o filtro?

Eu estimaria quantidade de membros e taxa de falso positivo desejada, calculando bits e hashes adequados. Mais precisão consome memória e CPU; depois validaria o comportamento com cardinalidade e distribuição reais, incluindo crescimento futuro do conjunto.

Termo para lembrar: Bloom sizing | Base técnica: referência 1.

08. Qual fórmula orienta memória?

Eu usaria m aproximadamente igual a menos n vezes ln de p dividido por ln de dois ao quadrado. A fórmula orienta dimensão idealizada; implementação, escalabilidade e overhead precisam ser incluídos no orçamento operacional.

Termo para lembrar: Bits per element | Base técnica: referência 1.

09. Como escolhe número de hashes?

Eu usaria aproximadamente k igual a m dividido por n vezes ln de dois como referência, arredondando conforme a implementação. Hashes demais aumentam CPU e podem piorar ocupação; medir custo importa junto da taxa de erro.

Termo para lembrar: Hash count | Base técnica: referência 1.

10. O que acontece ao exceder capacidade prevista?

Eu esperaria aumento de falsos positivos, reduzindo a economia de consultas. Precisaria reconstruir ou usar estrutura escalável suportada; o filtro pode continuar respondendo, mas perder quase todo o valor como proteção da origem.

Termo para lembrar: Filter saturation | Base técnica: referência 1.

11. Como atualiza na criação?

Eu coordenaria a publicação do novo membro com a visibilidade do dado, definindo fallback para janelas de atraso. Se um negativo do filtro bloqueia consulta, a atualização precisa ser confiável; caso contrário devo revalidar caminhos críticos.

Termo para lembrar: Membership publication | Base técnica: referência 1.

12. Pode apagar um membro do Bloom padrão?

Eu não limparia bits diretamente, pois outros membros podem compartilhá-los e virar falsos negativos. Para exclusão, consideraria reconstrução, filtro com contadores ou outra estrutura, entendendo custos e garantias específicas da implementação escolhida.

Termo para lembrar: Deletion limitation | Base técnica: referência 1.

13. Como funciona counting Bloom?

Eu substituiria bits por contadores para permitir remoções sob condições adequadas, aumentando memória e complexidade. Preciso evitar decrementos indevidos e overflow; remover um item nunca inserido pode comprometer a cobertura do conjunto representado.

Termo para lembrar: Counting Bloom filter | Base técnica: referência 1.

14. Quando usa Cuckoo filter?

Eu avaliaria quando suporte a exclusão e características de espaço forem importantes, verificando limites de ocupação e inserção. Não o trataria como substituto universal; comportamento da implementação e workload determinam se o benefício compensa.

Termo para lembrar: Cuckoo filter | Base técnica: referência 1.

15. Bloom serve para dedupe de pagamento?

Eu não descartaria um pagamento apenas por positivo probabilístico, porque falso positivo poderia perder uma operação legítima. Posso usá-lo como aceleração antes de consultar um registro exato, mas a decisão final exige autoridade precisa.

Termo para lembrar: Financial deduplication | Base técnica: referência 1.

16. Bloom serve para autorização?

Eu nunca concederia acesso por possível presença e evitaria negar acesso crítico por filtro desatualizado. Permissões precisam de política autoritativa; probabilismo pode otimizar consultas, mas não substitui identidade, revogação e avaliação do recurso.

Termo para lembrar: Authorization boundary | Base técnica: referência 1.

17. Como trata filtro indisponível?

Eu escolheria fallback limitado à origem ou degradação conforme capacidade, sem responder inexistente só porque o filtro caiu. A camada é uma otimização; sua falha não pode inventar fatos nem disparar carga ilimitada no banco.

Termo para lembrar: Filter fallback | Base técnica: referência 1.

18. Como trata filtro vazio após restart?

Eu marcaria estado de prontidão e não interpretaria negativos como definitivos até carregar cobertura suficiente. Reconstrução precisa de versão e acompanhamento de mudanças; liberar um filtro vazio bloquearia praticamente todos os registros existentes.

Termo para lembrar: Warm-up safety | Base técnica: referência 1.

19. Como reconstrói sem perder criações novas?

Eu criaria uma base consistente e aplicaria alterações posteriores até uma posição conhecida antes do corte. Uma varredura isolada pode perder itens criados durante o processo; a cobertura precisa incluir essa janela explicitamente.

Termo para lembrar: Snapshot plus delta | Base técnica: referência 1.

20. Como faz troca de versão?

Eu manteria gerações identificadas e mudaria a referência quando a nova estiver validada, preservando a antiga conforme necessidade. Consumidores precisam saber qual conjunto e posição representam; misturar gerações pode produzir decisões inconsistentes entre instâncias.

Termo para lembrar: Filter generation | Base técnica: referência 1.

21. Como particiona filtros?

Eu dividiria por tenant, prefixo ou período quando isso ajuda tamanho e reconstrução, garantindo roteamento correto. Consultar o filtro errado gera negativo indevido; a chave de particionamento deve permanecer alinhada ao armazenamento autoritativo.

Termo para lembrar: Partitioned membership | Base técnica: referência 1.

22. Como combina Bloom com cache?

Eu validaria requisição, consultaria a cobertura e depois o cache ou origem quando necessário, medindo benefício. A ordem depende de custo e confiança; um cache positivo atual não deve ser contradito cegamente por filtro atrasado.

Termo para lembrar: Layered lookup | Base técnica: referência 1.

23. Como detecta ataque com IDs aleatórios?

Eu observaria proporção de ausências, diversidade de chaves e comportamento por origem ou identidade, aplicando limites apropriados. Bloom reduz custo por consulta, mas não substitui proteção contra volume de rede e CPU na entrada.

Termo para lembrar: Abuse pattern | Base técnica: referência 1.

24. Como valida formato de ID?

Eu rejeitaria identificadores impossíveis antes de tocar cache ou banco, usando limites de tamanho e sintaxe. Isso reduz trabalho barato, mas IDs bem formados ainda podem ser inexistentes; validação não substitui controle de taxa nem autorização.

Termo para lembrar: Input validation | Base técnica: referência 1.

25. Como mede eficiência do filtro?

Eu compararia consultas evitadas, falsos positivos estimados e custo de CPU e memória. Um filtro que quase sempre responde possível pode não valer a complexidade; medir somente tamanho compacto não demonstra benefício à jornada.

Termo para lembrar: Avoided-read ratio | Base técnica: referência 1.

26. Como estima falsos positivos em produção?

Eu amostraria respostas possíveis e verificaria a existência real, distinguindo atraso de replicação e exclusão. Medir corretamente exige uma fonte confiável; caso contrário posso atribuir ao filtro um erro que veio da consulta usada para validar.

Termo para lembrar: Sampling validation | Base técnica: referência 1.

27. Como trata exclusão de dados?

Eu aceitaria que um Bloom padrão mantenha positivo até reconstrução, pois isso apenas gera consulta extra. A origem confirma ausência; apagar bits sem protocolo seria pior, porque poderia bloquear outros membros legítimos do conjunto.

Termo para lembrar: Membership after deletion | Base técnica: referência 1.

28. Como testa ausência e criação concorrente?

Eu forçaria consulta negativa enquanto um registro é criado e durante troca de geração, verificando o contrato de visibilidade. Esse teste revela se a aplicação transformou uma otimização eventual numa barreira indevida para dados recém-confirmados.

Termo para lembrar: Race testing | Base técnica: referência 1.

29. Quando prefere conjunto exato?

Eu usaria set ou índice autoritativo quando o volume cabe e a decisão exige precisão, comparando custo de memória. Bloom é útil pela economia, mas sua aproximação exige uma segunda etapa para confirmar positivos de forma correta.

Termo para lembrar: Exact membership | Base técnica: referência 1.

30. Como resumiria a solução?

Eu diria que o filtro economiza consultas negativas sobre um conjunto coberto e versionado, sem decidir dinheiro ou permissão. Complementaria com negative cache, limites e fallback, demonstrando como evitamos tanto sobrecarga quanto rejeição de registros legítimos.

Termo para lembrar: Safe probabilistic acceleration | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Falso positivo menor

Ganho. Evita mais consultas

Custo e motivo. Consome memória

Como decidir. Quando origem é cara

02. Mais hashes

Ganho. Pode melhorar precisão ideal

Custo e motivo. Consome CPU

Como decidir. Quando cálculo de dimensionamento justifica

03. Negative cache

Ganho. Resolve ausências repetidas

Custo e motivo. Pode esconder criação

Como decidir. Quando chaves se repetem

04. Bloom

Ganho. Representa conjunto compacto

Custo e motivo. Não confirma presença

Como decidir. Quando falsos positivos são toleráveis

05. Counting Bloom

Ganho. Permite remoção controlada

Custo e motivo. Aumenta espaço

Como decidir. Quando exclusão frequente importa

06. Filtro particionado

Ganho. Facilita reconstrução

Custo e motivo. Exige roteamento correto

Como decidir. Quando conjunto é grande

07. Gerações paralelas

Ganho. Permitem corte seguro

Custo e motivo. Duplicam memória temporária

Como decidir. Quando atualização não pode parar

08. Fallback à origem

Ganho. Preserva correção

Custo e motivo. Pode aumentar carga

Como decidir. Quando limitado por admissão

09. Conjunto exato

Ganho. Oferece precisão

Custo e motivo. Consome mais espaço

Como decidir. Quando memória permite

10. Filtro local

Ganho. Reduz rede

Custo e motivo. Pode atrasar

Como decidir. Quando atualização é bem controlada

Duas alternativas ao caminho proposto

1. Negative cache com TTL

Quando faz sentido. Quando mesmas ausências se repetem

Como implementar. Guardar resultado não encontrado e invalidar na criação

Limites. Não cobre bem chaves aleatórias infinitas

2. Índice exato de existência

Quando faz sentido. Quando a decisão precisa ser precisa

Como implementar. Consultar set, tabela ou índice com garantia adequada

Limites. Custa mais memória ou I/O que filtro probabilístico

Referências e alcance da evidência

1. Redis: Bloom filter

Associação probabilística, falsos positivos, dimensionamento e operações oferecidas pelo produto.

https://redis.io/docs/latest/develop/data-types/probabilistic/bloom-filter/

2. Microsoft: Cache-Aside

Leitura sob demanda, invalidação e limites de consistência entre cache e origem.

https://learn.microsoft.com/en-us/azure/architecture/patterns/cache-aside

3. Caffeine: Efficiency

Políticas de admissão e eviction orientadas a frequência e comparação de eficiência.

https://github.com/ben-manes/caffeine/wiki/Efficiency

Continue em: T031 · T034 · T055

T036 / D. Cache e modelos de dados

Cache distribuído e particionamento

Cache distribuído reparte memória e tráfego entre nós, frequentemente com replicação e camadas locais. A topologia precisa preservar acesso, limitar remapeamento e tratar perda de nós sem sobrecarregar a origem.

Quando usar. Quando um cache único não atende capacidade, disponibilidade ou distribuição geográfica exigida.

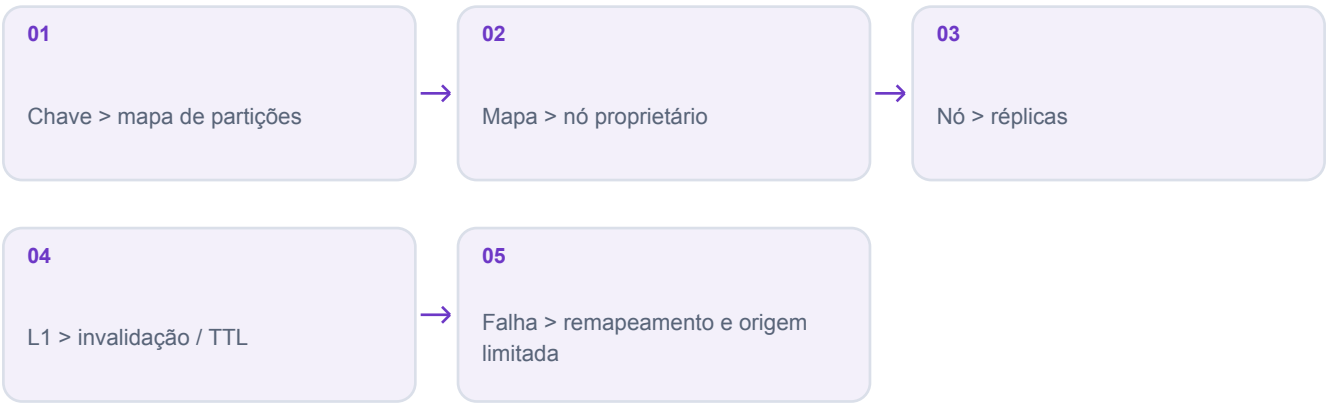
Quando evitar ou limitar. Evitar assumir que cluster resolve hot key ou que replicação transforma cache descartável em autoridade financeira.

Uma abertura possível

Vou separar distribuição de capacidade, redundância e coerência. Mostrarei como o cliente encontra a chave e como recuperamos um nó perdido mantendo a origem dentro de limites seguros.

Peças e tecnologias. Redis Cluster · Memcached · consistent hashing · virtual nodes · L1 cache · replica routing

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Medir tamanho e tráfego por chave.

2. Escolher particionamento e cliente compatível.

3. Definir replicação e consistência.

4. Planejar remapeamento, reshard e aquecimento.

5. Testar perda de nó, hot key e degradação.

30 perguntas para sustentar o desenho

01. Por que distribuir cache?

Eu buscaria mais memória, throughput ou isolamento do que um nó oferece, considerando custo de rede. Distribuição acrescenta roteamento e falhas parciais; se o working set cabe e a carga é baixa, simplicidade pode ser melhor.

Termo para lembrar: Distributed capacity | Base técnica: referência 1.

02. Como o cliente encontra a chave?

Eu usaria mapa de slots, hash consistente ou proxy conforme a tecnologia, com atualização durante mudanças. O roteamento precisa ter comportamento definido para mapa antigo; um cliente incompatível pode transformar rebalanceamento normal em indisponibilidade.

Termo para lembrar: Partition routing | Base técnica: referência 1.

03. Hash simples módulo N é suficiente?

Eu observaria que mudar N pode remapear grande parte das chaves, causando cache frio amplo. Hash consistente ou unidades lógicas reduzem esse impacto, mas ainda exigem capacidade para recomputar ou mover os dados afetados.

Termo para lembrar: Remapping cost | Base técnica: referência 1.

04. Como consistent hashing ajuda?

Eu distribuiria chaves num espaço estável para limitar movimentação ao adicionar ou remover nós. Isso não garante balanceamento perfeito nem resolve uma chave muito popular; o algoritmo cuida do mapeamento, não da semântica de atualização.

Termo para lembrar: Consistent hashing | Base técnica: referência 1.

05. Para que virtual nodes?

Eu atribuiria várias faixas lógicas por servidor para suavizar distribuição e facilitar movimento. Mais unidades aumentam flexibilidade e metadados; ainda preciso considerar tamanhos de valores e tráfego, que podem ser muito desiguais entre chaves.

Termo para lembrar: Virtual nodes | Base técnica: referência 1.

06. Replicação e sharding são iguais?

Eu usaria sharding para dividir conjuntos e replicação para copiar cada conjunto, com objetivos diferentes. Réplicas consomem memória adicional; um cluster pode ter muitos shards e continuar vulnerável se cada um possuir apenas uma cópia.

Termo para lembrar: Replication versus sharding | Base técnica: referência 1.

07. Como trata perda de um nó?

Eu redirecionaria conforme o protocolo e a existência de réplica, controlando misses e reconexões. Se dados forem perdidos, reconstrução é limitada; não permitiria que a ausência de um shard derrubasse a origem por carga súbita.

Termo para lembrar: Node failure | Base técnica: referência 1.

08. Como trata hot key?

Eu avaliaria L1, cópias de leitura ou divisão semântica, porque uma chave continua concentrada mesmo com muitos shards. Se a atualização exige sequência única, existe limite real; não prometeria escalabilidade infinita apenas com mais nós.

Termo para lembrar: Single-key bottleneck | Base técnica: referência 1.

09. Como trata big key?

Eu limitaria tamanho e dividiria por página ou período quando adequado. Uma chave enorme pode desequilibrar memória e operações, além de complicar migração; contagem uniforme de chaves não equivale a distribuição uniforme de bytes.

Termo para lembrar: Uneven memory distribution | Base técnica: referência 1.

10. Como usa cache L1?

Eu guardaria itens populares localmente para reduzir RTT e carga no cluster, com TTL e versão. Cada instância passa a ter cópia própria; invalidação e reconexão precisam impedir que valores antigos permaneçam além do contrato.

Termo para lembrar: Near cache | Base técnica: referência 1.

11. Como invalida entre instâncias?

Eu usaria eventos ou tracking com política para perda da conexão, complementando por expiração. Mensagens de invalidação podem falhar; uma instância que ficou isolada deve revalidar ou limpar estado antes de confiar nas cópias locais.

Termo para lembrar: Coherence channel | Base técnica: referência 1.

12. Como escolhe replicação de leitura?

Eu direcionaria leituras tolerantes a atraso para cópias quando suportado, medindo ganho e staleness. Réplica não deve autorizar regras críticas com estado antigo; roteamento precisa conhecer a garantia exigida por cada operação.

Termo para lembrar: Read replica | Base técnica: referência 1.

13. Como trata operações multichave?

Eu agruparia chaves no mesmo domínio de atomicidade quando a tecnologia exigir, sem criar um hotspot desnecessário. Se precisam estar em shards diferentes, a operação não mantém automaticamente a mesma garantia de um comando local.

Termo para lembrar: Co-location constraint | Base técnica: referência 1.

14. Como faz reshard sem parar?

Eu seguiria o protocolo da tecnologia para mover propriedade e dados, usando clientes que entendem redirecionamento. Limitaria taxa de migração e verificaria erro; o custo de rede e CPU compete com as requisições normais.

Termo para lembrar: Online resharding | Base técnica: referência 1.

15. Como evita cache frio durante expansão?

Eu copiaria ou aqueceria dados relevantes antes de ampliar tráfego quando possível, mantendo capacidade da origem protegida. Expandir cache pode inicialmente aumentar misses; o plano precisa considerar essa fase, não apenas o estado final.

Termo para lembrar: Migration warm-up | Base técnica: referência 1.

16. Proxy ou cliente inteligente?

Eu compararia simplicidade do cliente com latência e dependência adicional do proxy. Cliente inteligente evita um salto, mas precisa atualizar mapas corretamente; proxy centraliza lógica, porém exige escala e disponibilidade compatíveis com todo o tráfego.

Termo para lembrar: Routing architecture | Base técnica: referência 1.

17. Como isola tenants?

Eu combinaria chaves com escopo correto, quotas e eventualmente recursos dedicados para clientes dominantes. Particionar apenas por tenant simplifica isolamento, mas um tenant gigante pode exceder um nó; crescimento precisa ser previsto no modelo.

Termo para lembrar: Tenant-aware partitioning | Base técnica: referência 1.

18. Como decide TTL em várias regiões?

Eu definiria atraso aceitável e versão por dado, reconhecendo que invalidação inter-regional pode atrasar. Cache próximo melhora latência, mas não cria conhecimento imediato da outra região; decisões sensíveis usam autoridade ou barreira apropriada.

Termo para lembrar: Regional freshness | Base técnica: referência 1.

19. Pode compartilhar cache entre regiões?

Eu avaliaria RTT, egress e dependência geográfica antes de centralizar. Um cache remoto pode ser mais lento que acesso local otimizado; proximidade, replicação e coerência devem ser escolhidas pelo padrão real da jornada.

Termo para lembrar: Remote cache latency | Base técnica: referência 1.

20. Como controla memória por shard?

Eu monitoraria bytes, fragmentação e eviction por nó, reservando margem para replicação e migração. Um painel agregado pode esconder um shard saturado; limites e alertas precisam acompanhar a distribuição real do working set.

Termo para lembrar: Per-shard budget | Base técnica: referência 1.

21. Como trata mudanças de formato?

Eu versionaria namespace ou payload, permitindo coexistência entre clientes antigos e novos. Em cluster, diferentes instâncias podem preencher formatos simultaneamente; sem contrato, o mesmo valor pode quebrar consumidores dependendo de quem o escreveu.

Termo para lembrar: Distributed schema rollout | Base técnica: referência 1.

22. Como evita timeout multiplicado por retries?

Eu limitaria tentativas e redirecionamentos dentro de um deadline total, usando jitter após falhas. Um cliente que repete em todos os nós pode amplificar o incidente; a estratégia deve respeitar o estado do cluster e da origem.

Termo para lembrar: Client retry budget | Base técnica: referência 1.

23. Como usa health checks?

Eu verificaria disponibilidade e capacidade relevante, evitando retirar nó apenas por breve lentidão sem considerar remapeamento. Health check influencia carga; expulsar participantes demais pode concentrar tráfego e transformar uma degradação em colapso.

Termo para lembrar: Health semantics | Base técnica: referência 1.

24. Como trata perda do cluster inteiro?

Eu manteria caminho essencial com admissão controlada, aquecendo gradualmente e desabilitando consultas opcionais caras. A aplicação deve saber operar sem cache dentro de limites; reconstrução irrestrita pode derrubar o banco autoritativo.

Termo para lembrar: Cache disaster | Base técnica: referência 1.

25. Como protege dados sensíveis?

Eu restringiria rede, credenciais e escopo de acesso, minimizando payload e protegendo dumps. Compartilhar cache entre serviços amplia superfície de leitura; chaves com tenant não são uma barreira de autorização por si mesmas.

Termo para lembrar: Cache access control | Base técnica: referência 1.

26. Como mede eficiência distribuída?

Eu observaria latência por nó, hit ratio por classe, bytes, redirecionamentos e carga evitada. A média pode esconder hotspot ou cliente com mapa obsoleto; métricas devem permitir localizar falha sem cardinalidade ilimitada.

Termo para lembrar: Distributed cache metrics | Base técnica: referência 1.

27. Como testa particionamento?

Eu geraria distribuição desigual, valores grandes e mudança de nós, verificando latência e origem. Benchmark uniforme favorece artificialmente o cluster; o caso de uma chave dominante costuma revelar limites que o total de memória esconde.

Termo para lembrar: Skewed-load test | Base técnica: referência 1.

28. Como escolhe quantidade de réplicas?

Eu relacionaria falhas toleradas, capacidade de leitura e custo de memória, verificando a semântica de promoção. Mais cópias ajudam disponibilidade, mas também consomem rede e não eliminam perda recente quando confirmação é assíncrona.

Termo para lembrar: Replication budget | Base técnica: referência 1.

29. Como diferencia cache de banco distribuído?

Eu definiria se o dado pode ser reconstruído e qual promessa de durabilidade existe. Cache distribuído pode oferecer recursos avançados, mas o papel na arquitetura é decidido pela aplicação e pelas garantias, não apenas pelo produto usado.

Termo para lembrar: Role boundary | Base técnica: referência 1.

30. Como defenderia a topologia?

Eu mostraria chave, mapa, cópia e caminho de falha, incluindo uma expansão. A escolha precisa explicar por que distribuição melhora a jornada e como mantemos correção e capacidade quando o cluster perde memória ou conectividade.

Termo para lembrar: Topology rationale | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Mais shards

Ganho. Distribuem capacidade

Custo e motivo. Aumentam coordenação

Como decidir. Quando working set ou tráfego exige

02. Mais réplicas

Ganho. Melhoram tolerância

Custo e motivo. Duplicam memória

Como decidir. Quando perda de nó é prevista

03. Cliente inteligente

Ganho. Evita proxy

Custo e motivo. Complica SDK

Como decidir. Quando clientes suportam protocolo

04. Proxy

Ganho. Centraliza roteamento

Custo e motivo. Adiciona salto e dependência

Como decidir. Quando clientes são variados

05. L1

Ganho. Reduz hot hits

Custo e motivo. Complica coerência

Como decidir. Quando popularidade é alta

06. Co-location

Ganho. Permite multichave local

Custo e motivo. Pode concentrar carga

Como decidir. Quando atomicidade exige

07. Hash consistente

Ganho. Reduz remapeamento

Custo e motivo. Não resolve skew sozinho

Como decidir. Quando nós mudam

08. Tenant dedicado

Ganho. Isola ruído

Custo e motivo. Reduz eficiência

Como decidir. Quando carga é dominante

09. Cache regional

Ganho. Reduz RTT

Custo e motivo. Pode atrasar globalmente

Como decidir. Quando leitura tolera

10. Warm-up limitado

Ganho. Protege origem

Custo e motivo. Adia hit ratio alto

Como decidir. Quando recuperação é sensível

Duas alternativas ao caminho proposto

1. Cache local por instância

Quando faz sentido. Quando conjunto é pequeno e tolera cópias

Como implementar. Usar biblioteca com limites e TTL

Limites. Memória se duplica e invalidação exige cuidado

2. Origem com read replicas

Quando faz sentido. Quando cache traz pouco ganho

Como implementar. Distribuir leituras conforme consistência e índices

Limites. Custa mais I/O e não resolve consultas mal modeladas

Referências e alcance da evidência

1. Redis: Cluster scaling

Particionamento por slots, redirecionamento e limites de operações entre chaves no Redis Cluster.

https://redis.io/docs/latest/operate/oss_and_stack/management/scaling/

2. Redis: Client-side caching

Tracking e invalidação para caches locais, com tratamento de conexão e atualização.

https://redis.io/docs/latest/develop/reference/client-side-caching/

3. Microsoft: Sharding pattern

Estratégias de particionamento, distribuição e dificuldades de consultas e transações entre shards.

https://learn.microsoft.com/en-us/azure/architecture/patterns/sharding

Continue em: T017 · T027 · T034

T037 / D. Cache e modelos de dados

CQRS e materialized views

CQRS separa modelos de comando e consulta. Materialized views mantêm resultados derivados para leitura; podem compartilhar banco ou usar armazenamento distinto, com atualização síncrona ou assíncrona conforme o desenho.

Quando usar. Quando consultas e invariantes de escrita exigem modelos ou escalas diferentes, como ledger e extrato, casos e busca operacional.

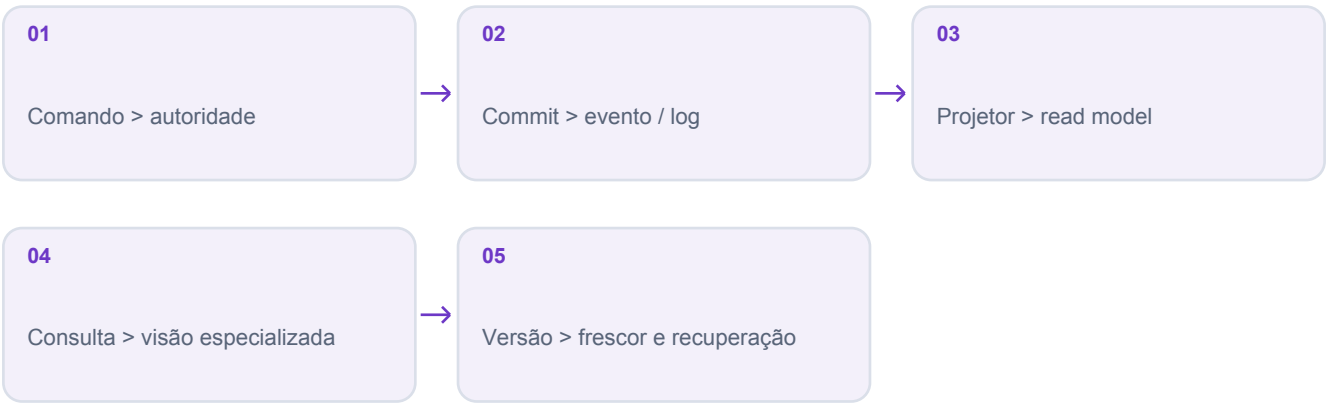
Quando evitar ou limitar. Evitar CQRS completo para CRUD simples ou usar projeção atrasada como autoridade de decisão financeira.

Uma abertura possível

Vou manter a escrita protegendo invariantes e desenhar a leitura para a tarefa do usuário. Explicarei o atraso, como ler após escrever e como reconstruir a projeção sem repetir efeitos externos.

Peças e tecnologias. PostgreSQL · materialized views · Kafka · Redis · OpenSearch · CDC

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Identificar divergência entre modelo de escrita e leitura.

2. Definir autoridade e contrato de comando.

3. Projetar consultas e atualização.

4. Implementar identidade, sequência e checkpoints.

5. Testar atraso, reconstrução e autorização.

30 perguntas para sustentar o desenho

01. CQRS exige dois bancos?

Eu começaria pela separação de modelos e responsabilidades, que pode existir no mesmo banco ou processo. Bancos distintos são uma opção quando escala ou acesso justificam, acrescentando pipeline e consistência eventual quando a atualização é assíncrona.

Termo para lembrar: Logical separation | Base técnica: referência 1.

02. CQRS exige event sourcing?

Eu explicaria que posso manter estado atual transacional e emitir eventos para projeções, sem reconstruir a autoridade por histórico. Event sourcing e CQRS combinam em alguns casos, mas cada um resolve um problema diferente.

Termo para lembrar: Pattern independence | Base técnica: referência 1.

03. Quando adotaria CQRS?

Eu procuraria consultas muito diferentes das regras de escrita, como telas agregadas ou busca textual. Se CRUD simples atende, separar tudo pode aumentar operação sem ganho; a decisão precisa de benefício mensurável para a jornada.

Termo para lembrar: Read/write asymmetry | Base técnica: referência 1.

04. Qual modelo é autoridade?

Eu manteria o serviço de comando responsável por validar invariantes e confirmar mudanças. A projeção otimiza apresentação e pode atrasar; qualquer ação sensível precisa revalidar no domínio, mesmo quando a tela mostra um estado aparentemente atual.

Termo para lembrar: Write authority | Base técnica: referência 1.

05. Como atualiza a projeção?

Eu escolheria evento durável, CDC ou atualização transacional conforme fronteira e requisitos. Escrita dupla independente entre banco e índice pode divergir; o mecanismo precisa permitir retomar do ponto correto depois de uma queda.

Termo para lembrar: Projection update | Base técnica: referência 1.

06. Como trata evento duplicado?

Eu combinaria aplicação e registro de progresso numa fronteira atômica quando possível, usando identidade ou versão. Incrementar um acumulado duas vezes muda o resultado; um simples ACK antes de persistir pode perder a atualização.

Termo para lembrar: Idempotent projector | Base técnica: referência 1.

07. Como trata evento fora de ordem?

Eu distinguiria snapshot completo de delta. Para snapshot, versão maior pode substituir; para delta, detecto lacuna e recupero a sequência, porque descartar o evento anterior pode perder um movimento legítimo necessário ao total.

Termo para lembrar: Projection sequence | Base técnica: referência 1.

08. Como implementa read-your-writes?

Eu retornaria posição da escrita e faria a consulta exigir uma projeção ao menos nessa versão, ou usaria autoridade temporariamente. Um delay fixo é frágil; sob falha, o lag pode ultrapassar qualquer espera escolhida sem verificação.

Termo para lembrar: Projection barrier | Base técnica: referência 1.

09. Como mostra atraso ao usuário?

Eu exibiria estado pendente e referência de atualização quando relevante, evitando simular conclusão com dado velho. A interface pode combinar confirmação do comando com projeção ainda atrasada, desde que não apresente duas verdades financeiras conflitantes.

Termo para lembrar: Freshness UX | Base técnica: referência 1.

10. Como reconstrói uma view?

Eu criaria uma geração nova a partir de fonte completa, aplicando mudanças até posição conhecida antes do corte. Não apagaria a view ativa para começar; reconstrução precisa coexistir com consultas e manter capacidade do sistema.

Termo para lembrar: Projection rebuild | Base técnica: referência 1.

11. Como valida equivalência?

Eu compararia contagens, versões e invariantes por faixa, com amostras ou hashes apropriados. Quantidade igual de linhas não prova valores iguais; diferenças devem ser investigadas antes de trocar a referência para a nova geração.

Termo para lembrar: Projection reconciliation | Base técnica: referência 1.

12. Como evita replay executar efeitos?

Eu separaria projetores de leitura de consumidores que enviam notificações ou dinheiro. Reconstruir uma visão não pode reenviar todos os pagamentos históricos; modo e destino do replay precisam ser controlados e testados explicitamente.

Termo para lembrar: Replay isolation | Base técnica: referência 1.

13. Como define checkpoint?

Eu persistiria posição de entrada coerente com o estado aplicado, evitando marcar progresso antes da alteração. Se destino e checkpoint não compartilham transação, uso protocolo idempotente que permita repetir sem perder ou duplicar o efeito da projeção.

Termo para lembrar: Projection checkpoint | Base técnica: referência 1.

14. Como faz várias views independentes?

Eu daria progresso e recursos próprios a cada projeção, permitindo busca atrasar sem bloquear saldo. Compartilhar todo processamento reduz operação inicial, mas pode criar acoplamento de disponibilidade e dificultar reconstruções seletivas depois.

Termo para lembrar: Consumer isolation | Base técnica: referência 1.

15. Como trata exclusão e dados pessoais?

Eu rastrearia quais views contêm dados derivados e propagaria mudanças conforme política validada. Reconstrução não pode ressuscitar informação removida indevidamente; retenção da fonte e regras de replay precisam permanecer compatíveis com a governança.

Termo para lembrar: Projection governance | Base técnica: referência 1.

16. Como faz autorização na view?

Eu incluiria atributos necessários e aplicaria política atual no backend, considerando atraso de permissões. Uma projeção com ACL antiga pode expor dados; ações e leituras sensíveis podem exigir verificação autoritativa adicional ou limite de frescor.

Termo para lembrar: Read-model authorization | Base técnica: referência 1.

17. Materialized view do banco é igual a consumidor Kafka?

Eu distinguiria atualização e consistência oferecidas pelo mecanismo local de uma projeção externa assíncrona. O nome da estrutura não informa frequência nem atomicidade; precisamos escolher como refresh, concorrência e recuperação funcionam na implementação real.

Termo para lembrar: Refresh mechanism | Base técnica: referência 1.

18. Como evita modelo de leitura gigante?

Eu projetaria por jornada e acesso, incluindo apenas dados necessários e origem clara. Uma view que replica todo o banco perde propósito, amplia exposição e cria dependência de muitos domínios para responder uma tarefa simples.

Termo para lembrar: Task-oriented view | Base técnica: referência 1.

19. Como trata mudança de schema?

Eu criaria nova versão do projetor e destino quando incompatível, com backfill e corte controlado. Consumidores antigos podem continuar durante migração; alterar a estrutura ativa abruptamente pode quebrar telas e tarefas em andamento.

Termo para lembrar: Projection versioning | Base técnica: referência 1.

20. Como calcula capacidade do projetor?

Eu compararia taxa de eventos e custo de atualização por view, incluindo picos e replay. A saída precisa superar entrada para drenar backlog; aumentar consumidores só ajuda quando particionamento e armazenamento permitem paralelismo real.

Termo para lembrar: Projection throughput | Base técnica: referência 1.

21. Como mede lag?

Eu observaria idade e posição do último evento aplicado, diferenciando ausência de novidades de atraso. Métricas de fila precisam estar ligadas à jornada; uma projeção importante atrasada pode violar o SLO mesmo com API de leitura rápida.

Termo para lembrar: Projection lag | Base técnica: referência 1.

22. Como trata erro de um evento?

Eu registraria diagnóstico e preservaria o evento, definindo se pode avançar conforme a semântica. Pular um delta financeiro corrompe o acumulado; quarentena precisa de owner, prazo e estratégia para recompor a visão corretamente.

Termo para lembrar: Poison event policy | Base técnica: referência 1.

23. Como usa agregados incrementais?

Eu aplicaria mudanças ao resultado em vez de recalcular tudo, protegendo identidade e sequência. Isso reduz custo por consulta, mas erros acumulam; checkpoints verificáveis e reconciliação contra a fonte ajudam detectar e reparar divergências.

Termo para lembrar: Incremental view maintenance | Base técnica: referência 1.

24. Como evita transação distribuída na leitura?

Eu materializaria os dados necessários a uma tela, aceitando frescor definido e preservando origem. Isso reduz fan-out, mas não torna a visão uma fotografia global perfeita de todos os domínios sem mecanismo adicional de coordenação.

Termo para lembrar: Denormalized query model | Base técnica: referência 1.

25. Como responde se a projeção caiu?

Eu ofereceria consulta limitada à autoridade quando houver capacidade e semântica compatíveis, ou indisponibilidade explícita. Bypass ilimitado pode derrubar escrita; a degradação deve preservar funções essenciais e evitar consultas caras improvisadas durante o incidente.

Termo para lembrar: Read-path degradation | Base técnica: referência 1.

26. Como escolhe armazenamento da view?

Eu escolheria pelo acesso: Redis para lookup descartável, relacional para filtros estruturados e busca para texto. Cada destino tem garantias diferentes; a view precisa de recuperação e política de frescor adequada, independente da tecnologia.

Termo para lembrar: Read-store selection | Base técnica: referência 1.

27. Como usa CQRS no Shuffle?

Eu agregaria contexto útil ao atendimento numa visão com versões e falhas parciais, mantendo comandos nos serviços donos. A bancada ganha velocidade, mas não passa a decidir bloqueios ou reembolsos com base apenas em dados projetados.

Termo para lembrar: Workspace projection | Base técnica: referência 1.

28. Como usa CQRS no ledger?

Eu manteria lançamentos e autorização na fronteira financeira, projetando extrato e saldo consultável conforme necessidade. A leitura atrasada pode ser informada ao cliente; autorizar débito pelo cache exigiria outra garantia explícita de saldo disponível.

Termo para lembrar: Ledger read model | Base técnica: referência 1.

29. Como testa projetor?

Eu aplicaria sequência conhecida com duplicatas, lacunas e versões antigas, comparando com cálculo de referência. Também testaria crash entre persistência e checkpoint; testes unitários felizes não revelam perdas decorrentes do protocolo de confirmação.

Termo para lembrar: Deterministic projection testing | Base técnica: referência 1.

30. Como defenderia o custo adicional?

Eu mostraria consulta ou escala que ficou significativamente melhor e explicaria a operação adicional aceita. Se não houver benefício concreto, manteria modelo único; CQRS é uma ferramenta para assimetria real, não um requisito de senioridade.

Termo para lembrar: CQRS justification | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Modelos separados

Ganho. Otimizam responsabilidades

Custo e motivo. Aumentam código

Como decidir. Quando leitura e escrita divergem

02. Mesmo banco

Ganho. Simplifica transação

Custo e motivo. Compartilha recursos

Como decidir. Quando escala cabe

03. Bancos distintos

Ganho. Isolam workloads

Custo e motivo. Criam lag e pipeline

Como decidir. Quando acessos justificam

04. View incremental

Ganho. Reduz recomputação

Custo e motivo. Acumula risco de divergência

Como decidir. Quando eventos são confiáveis e verificáveis

05. Rebuild completo

Ganho. Simplifica correção

Custo e motivo. Consome recursos

Como decidir. Quando fonte completa existe

06. Barreira de versão

Ganho. Preserva sessão

Custo e motivo. Pode esperar

Como decidir. Quando usuário acabou de escrever

07. Fallback autoritativo

Ganho. Mantém função essencial

Custo e motivo. Pressiona origem

Como decidir. Quando limitado

08. Views por jornada

Ganho. Reduzem fan-out

Custo e motivo. Duplicam dados

Como decidir. Quando tarefas são distintas

09. Checkpoint transacional

Ganho. Evita perda local

Custo e motivo. Acopla progresso ao destino

Como decidir. Quando armazenamento suporta

10. Geração paralela

Ganho. Permite migração segura

Custo e motivo. Duplica capacidade temporária

Como decidir. Quando leitura não pode parar

Duas alternativas ao caminho proposto

1. Modelo único com índices

Quando faz sentido. Quando CRUD e consultas cabem no relacional

Como implementar. Usar transações e tuning sem pipeline adicional

Limites. Pode limitar consultas muito especializadas

2. Cache de consultas

Quando faz sentido. Quando o problema é repetição de leituras

Como implementar. Guardar resultados com TTL e invalidação

Limites. Não oferece todas as capacidades de uma projeção durável

Referências e alcance da evidência

1. Microsoft: CQRS pattern

Separação de modelos de leitura e escrita e seus custos de consistência e complexidade.

https://learn.microsoft.com/en-us/azure/architecture/patterns/cqrs

2. Microsoft: Event-driven architecture

Produtores, consumidores e canais, com desafios de coordenação e observabilidade.

https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/event-driven

3. PostgreSQL: Transaction isolation

Isolamento, snapshots e necessidade de repetição de transações sob conflitos.

https://www.postgresql.org/docs/current/transaction-iso.html

Continue em: T028 · T039 · T040

T038 / D. Cache e modelos de dados

Event-driven architecture e contratos

Arquitetura orientada a eventos conecta produtores e consumidores por fatos publicados. Desacopla tempo e participantes, mas exige contratos, identidade, ordenação, observabilidade e responsabilidade pela conclusão dos processos.

Quando usar. Quando várias capacidades reagem a mudanças ou trabalho precisa ser desacoplado temporalmente.

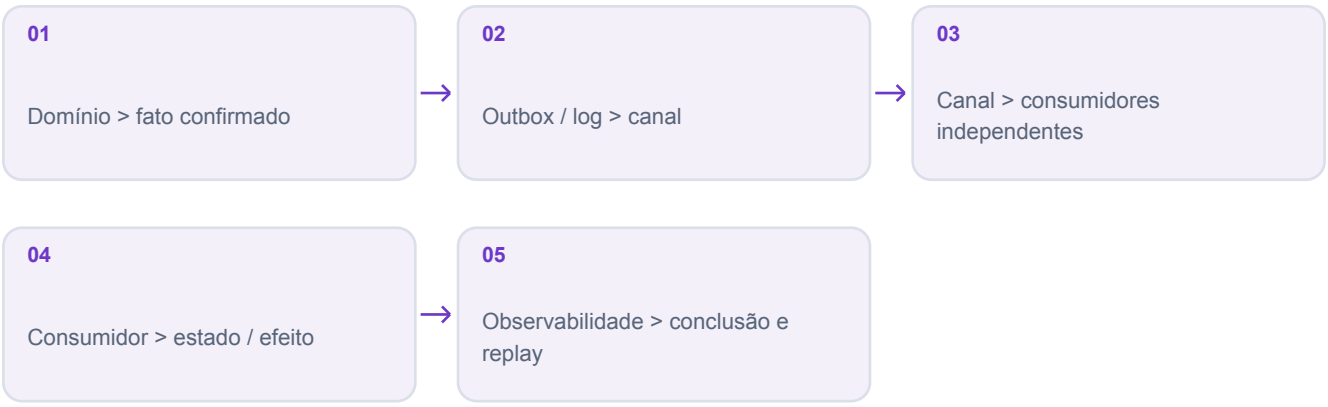
Quando evitar ou limitar. Evitar transformar todo comando em evento, ciclos invisíveis e broker usado como justificativa para não definir ownership.

Uma abertura possível

Vou publicar fatos do domínio com contratos estáveis e consumidores responsáveis. O broker transporta; a aplicação continua definindo o que precisa acontecer, como detectar atraso e como corrigir falhas.

Peças e tecnologias. Kafka · SNS · SQS · RabbitMQ · AsyncAPI · CloudEvents · Schema Registry

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Identificar fatos e responsáveis.

2. Definir envelope, schema e semântica.

3. Escolher canal e garantias necessárias.

4. Implementar publicação e consumo recuperáveis.

5. Medir jornada e governar evolução.

30 perguntas para sustentar o desenho

01. O que caracteriza um evento?

Eu o nomearia como fato já confirmado, com identidade e contexto, como ContaAberta. Comando solicita algo que pode ser recusado; essa diferença define quem valida e se esperamos resposta ou apenas reações independentes.

Termo para lembrar: Domain event | Base técnica: referência 1.

02. Quando eventos são melhores que chamada direta?

Eu usaria quando produtores não precisam aguardar todas as reações e consumidores têm ciclos independentes. O ganho vem com atraso e operação; se uma decisão imediata depende do resultado, chamada direta pode ser mais clara.

Termo para lembrar: Temporal decoupling | Base técnica: referência 1.

03. Evento notifica ou carrega estado?

Eu compararia payload mínimo com dados suficientes para o consumidor agir sem consultar origem. Referência reduz exposição, mas cria dependência temporal; estado carregado facilita autonomia e exige governança e evolução de schema mais cuidadosas.

Termo para lembrar: Event-carried state | Base técnica: referência 1.

04. Como define envelope?

Eu incluiria event_id, tipo, versão, agregado, correlação e informações temporais necessárias, evitando dados sensíveis inúteis. Campos têm semântica documentada; timestamp não substitui sequência quando consumidores precisam detectar ordem e lacunas.

Termo para lembrar: Event envelope | Base técnica: referência 1.

05. Como garante publicação após commit?

Eu persistiria fato e obrigação de publicar na mesma fronteira, usando outbox ou mecanismo equivalente. Chamar o broker depois do commit sem registro durável deixa uma janela em que a mudança existe, mas nenhum consumidor saberá.

Termo para lembrar: Transactional publication | Base técnica: referência 1.

06. Como trata duplicatas?

Eu preservaria event_id e faria o consumidor aplicar mudança com proteção contra repetição. O broker pode reenviar e o produtor pode republicar; a regra precisa sobreviver às duas origens de duplicata, não apenas ao retry de rede.

Termo para lembrar: At-least-once handling | Base técnica: referência 1.

07. Como trata ordem?

Eu declararia se a ordem importa por conta, conversa ou operação, incluindo sequência e particionamento. Ordem global custa coordenação; mesma partição não corrige publicação invertida por produtores concorrentes sem protocolo adicional na origem.

Termo para lembrar: Ordering contract | Base técnica: referência 1.

08. Como evita consumidor antigo quebrar?

Eu faria mudanças compatíveis e testaria versões coexistentes, usando registry quando útil. Compatibilidade estrutural não basta se significado muda; consumidores precisam entender novos estados ou ter comportamento explícito para valores desconhecidos do contrato.

Termo para lembrar: Schema compatibility | Base técnica: referência 1.

09. Como remove campo?

Eu observaria consumidores e versões, introduziria substituto e manteria transição antes de remover. Eventos retidos continuam sendo lidos em replay; desligar o produtor antigo não elimina a necessidade de interpretar mensagens históricas.

Termo para lembrar: Deprecation lifecycle | Base técnica: referência 1.

10. Como escolhe tópico?

Eu organizaria por domínio e características operacionais, considerando retenção, autorização e carga. Um tópico por entidade pode explodir metadados; um tópico para tudo mistura políticas e torna consumidores dependentes de tráfego irrelevante.

Termo para lembrar: Topic design | Base técnica: referência 1.

11. Como diferencia fila e log?

Eu compararia distribuição de trabalho e retenção com progresso independente. Um log facilita replay e múltiplos consumidores; uma fila pode ser mais simples para tarefas. A escolha depende das garantias reais do produto, não apenas do nome.

Termo para lembrar: Queue versus log | Base técnica: referência 1.

12. Como observa processo coreografado?

Eu correlacionaria eventos e estados numa visão operacional com prazo e owner, evitando depender de leitura manual de logs. Desacoplar serviços não pode deixar uma solicitação sem responsável por identificar que a jornada nunca terminou.

Termo para lembrar: Process visibility | Base técnica: referência 1.

13. Quando usa orquestrador?

Eu escolheria quando sequência, prazo e compensação precisam de coordenação explícita. O orquestrador registra progresso e chama participantes, mas não possui suas regras internas; centralizar processo não significa centralizar todo o domínio de negócio.

Termo para lembrar: Orchestration | Base técnica: referência 1.

14. Como evita ciclos de eventos?

Eu definiria causalidade e condições de reação, distinguindo evento de origem e atualização derivada. Um consumidor que publica a mesma mudança de volta pode criar loop; identidade e limites de origem ajudam detectar o problema.

Termo para lembrar: Feedback loop | Base técnica: referência 1.

15. Como trata consumidor lento?

Eu isolaria seu progresso e recursos, medindo idade e capacidade de drenagem. Um relatório lento não deve bloquear função financeira, mas retenção precisa cobrir seu atraso; simplesmente acumular dados não garante recuperação indefinida.

Termo para lembrar: Consumer lag | Base técnica: referência 1.

16. Como faz backpressure?

Eu limitaria admissão e concorrência conforme capacidade dos destinos, usando backlog como sinal. Brokers absorvem variação temporária; uma taxa de entrada permanentemente maior exige mudança de capacidade ou política, não apenas mais filas.

Termo para lembrar: Flow control | Base técnica: referência 1.

17. Como usa DLQ?

Eu preservaria evento, erro e contexto para diagnóstico, definindo responsável e redrive. DLQ não significa processo resolvido; a operação pode continuar fora do prazo, e reenvio precisa respeitar identidade, ordenação e correções necessárias.

Termo para lembrar: Quarantine | Base técnica: referência 1.

18. Como trata mensagem venenosa?

Eu classificaria erro de schema, dado inválido e falha transitória, evitando retry infinito no mesmo item. Se pular quebra uma sequência financeira, paro o agregado ou recupero contexto; disponibilidade do consumidor não justifica perder um delta.

Termo para lembrar: Poison message | Base técnica: referência 1.

19. Como faz replay seguro?

Eu definiria consumidores e efeitos permitidos, usando ambiente ou modo isolado para reconstrução. Reexecutar histórico de projeção não pode reenviar notificações ou pagamentos; identidade e autorização de replay precisam estar presentes no desenho operacional.

Termo para lembrar: Replay boundaries | Base técnica: referência 1.

20. Como trata retenção?

Eu alinharia janela do broker com recuperação, novos consumidores e obrigações de histórico, incluindo bytes e custo. Se replay necessário excede retenção, preciso de arquivo ou snapshot; não prometo reconstrução a partir de dados já removidos.

Termo para lembrar: Retention contract | Base técnica: referência 1.

21. Como evita dados pessoais demais?

Eu publicaria apenas o necessário e controlaria quem pode consumir, considerando que eventos se espalham por vários destinos. Dados imutáveis e retidos dificultam exclusão; referências e políticas precisam ser escolhidas antes de distribuir informação sensível.

Termo para lembrar: Event minimization | Base técnica: referência 1.

22. Como autoriza produtor e consumidor?

Eu restringiria permissões por tópico ou fila e validaria identidade de quem publica, sem confiar apenas em conectividade. Consumidor também autoriza efeitos; receber uma mensagem no canal não prova que qualquer ação financeira nela descrita é legítima.

Termo para lembrar: Messaging authorization | Base técnica: referência 1.

23. Como trata evento de parceiro externo?

Eu validaria autenticidade, formato e identidade, traduzindo para o modelo interno com rastreabilidade. Não deixaria códigos externos governarem diretamente estados financeiros; a integração precisa de regras para duplicatas, atraso e resultados parciais.

Termo para lembrar: Integration boundary | Base técnica: referência 1.

24. Como escolhe granularidade do evento?

Eu publicaria fatos úteis e estáveis, evitando um evento para cada alteração técnica irrelevante ou payload gigantesco com tudo. O contrato deve servir consumidores sem expor detalhes físicos que mudam a cada refatoração do banco.

Termo para lembrar: Event granularity | Base técnica: referência 1.

25. Como coordena várias mudanças do mesmo commit?

Eu avaliaria se consumidores precisam enxergar um conjunto atomicamente, incluindo metadados ou um evento composto quando adequado. Mensagens separadas podem ser observadas em momentos diferentes; commit local não se transfere automaticamente para a visão remota.

Termo para lembrar: Transaction grouping | Base técnica: referência 1.

26. Como testa contratos?

Eu validaria schemas, exemplos históricos e comportamento diante de campos novos, duplicatas e ausência de dependências. Testar somente serialização atual não cobre replay nem semântica; consumidores precisam demonstrar como reagem a evolução e falhas reais.

Termo para lembrar: Event contract testing | Base técnica: referência 1.

27. Como evita acoplamento por eventos?

Eu manteria contratos de domínio estáveis e evitaria consumidores dependerem de detalhes internos ou sequência acidental. Comunicação assíncrona ainda cria dependências; a diferença é temporal, não ausência completa de coordenação entre equipes.

Termo para lembrar: Semantic coupling | Base técnica: referência 1.

28. Como mede conclusão de negócio?

Eu acompanharia identidade da solicitação através dos eventos até resultado final, com prazo e estados desconhecidos. Mensagens publicadas por segundo são métricas técnicas; não provam que o cliente recebeu o serviço que solicitou.

Termo para lembrar: End-to-end completion | Base técnica: referência 1.

29. Quando não usaria EDA?

Eu evitaria para fluxo pequeno que precisa de resposta imediata e não possui consumidores independentes. Introduzir broker, schemas e replay sem necessidade aumenta operação; um módulo ou chamada direta bem controlada pode ser mais adequado.

Termo para lembrar: Architectural simplicity | Base técnica: referência 1.

30. Como defenderia a arquitetura?

Eu mostraria um fato confirmado, duas reações independentes e a recuperação de uma delas após falha. Também explicaria quem responde pela jornada, porque desacoplamento técnico só é útil quando responsabilidades e garantias permanecem compreensíveis.

Termo para lembrar: Event responsibility | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Eventos ricos

Ganho. Reduzem chamadas à origem

Custo e motivo. Duplicam dados

Como decidir. Quando autonomia de leitura importa

02. Eventos mínimos

Ganho. Reduzem payload

Custo e motivo. Criam dependência de consulta

Como decidir. Quando dados mudam e acesso é controlado

03. Coreografia

Ganho. Desacopla reações

Custo e motivo. Dificulta visão do processo

Como decidir. Quando reações são independentes

04. Orquestração

Ganho. Explicita sequência

Custo e motivo. Concentra coordenação

Como decidir. Quando prazo e compensação importam

05. Log retido

Ganho. Permite replay

Custo e motivo. Custa armazenamento

Como decidir. Quando consumidores precisam reconstruir

06. Fila simples

Ganho. Facilita tarefas

Custo e motivo. Pode limitar replay

Como decidir. Quando distribuição de trabalho basta

07. Schema registry

Ganho. Detecta incompatibilidade

Custo e motivo. Exige governança

Como decidir. Quando há muitos consumidores

08. Tópico granular

Ganho. Isola políticas

Custo e motivo. Aumenta metadados

Como decidir. Quando domínios divergem

09. Payload composto

Ganho. Preserva contexto conjunto

Custo e motivo. Amplia tamanho

Como decidir. Quando observação parcial é problemática

10. Consumer isolado

Ganho. Limita contágio

Custo e motivo. Duplica recursos

Como decidir. Quando criticidades diferem

Duas alternativas ao caminho proposto

1. API síncrona

Quando faz sentido. Quando chamador precisa da decisão imediata

Como implementar. Usar contrato, deadline e falha explícita

Limites. Acopla disponibilidade e latência

2. Workflow durável

Quando faz sentido. Quando fluxo precisa de coordenação central

Como implementar. Persistir etapas e resultados com retries e sinais

Limites. Exige engine e idempotência dos efeitos

Referências e alcance da evidência

1. Microsoft: Event-driven architecture

Produtores, consumidores e canais, com desafios de coordenação e observabilidade.

https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/event-driven

2. Microsoft: CQRS pattern

Separação de modelos de leitura e escrita e seus custos de consistência e complexidade.

https://learn.microsoft.com/en-us/azure/architecture/patterns/cqrs

3. Microsoft: Architecture styles

Características e critérios de estilos arquiteturais; decisões particulares são análises autorais.

https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/

Continue em: T041 · T047 · T062

T039 / D. Cache e modelos de dados

Event sourcing, snapshots e replay

Event sourcing mantém uma sequência de eventos como autoridade do estado. Projeções e snapshots aceleram acesso; replay reconstrói estado segundo regras versionadas e precisa ser separado de efeitos externos.

Quando usar. Quando histórico de decisões e reconstrução são capacidades centrais que justificam complexidade de eventos e evolução.

Quando evitar ou limitar. Evitar adotar apenas para auditoria simples ou assumir que replay pode chamar serviços externos sem duplicar consequências.

Uma abertura possível

Vou tratar eventos como fatos duráveis do domínio e usar versões esperadas para concorrência. Snapshots aceleram reconstrução, mas o histórico e sua interpretação continuam exigindo governança.

Peças e tecnologias. EventStoreDB · Kafka com modelo apropriado · PostgreSQL append-only · snapshots · CQRS · upcasting

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Definir eventos de negócio e invariantes.

2. Escolher append e concorrência por stream.

3. Implementar projeções puras e efeitos separados.

4. Versionar eventos e snapshots.

5. Testar replay, migração e recuperação.

30 perguntas para sustentar o desenho

01. O que vira fonte da verdade?

Eu manteria eventos confirmados como registro autoritativo das mudanças, derivando o estado atual. Uma tabela de auditoria anexada ao CRUD não é automaticamente event sourcing; a diferença está em como reconstruímos e validamos o estado.

Termo para lembrar: Event log authority | Base técnica: referência 1.

02. Como processa um comando?

Eu carregaria estado consistente do stream, validaria a intenção e produziria eventos, gravando com versão esperada. Se houver conflito, releio e decido novamente; não reaplico cegamente eventos calculados sobre um estado que já mudou.

Termo para lembrar: Command decision | Base técnica: referência 1.

03. Como garante concorrência por entidade?

Eu faria append condicional à versão lida, usando o mecanismo do event store. Dois comandos concorrentes não podem confirmar a mesma posição esperada; a aplicação trata conflito e mantém identidade para reconhecer retries da mesma intenção.

Termo para lembrar: Expected version | Base técnica: referência 1.

04. Evento é comando serializado?

Eu registraria o fato que ocorreu, não apenas o pedido recebido, porque pedidos podem ser rejeitados. Um evento precisa carregar informação suficiente para interpretação histórica sem depender de executar novamente decisões externas voláteis.

Termo para lembrar: Fact versus intent | Base técnica: referência 1.

05. Como escolhe stream?

Eu alinharia stream ao agregado e sua invariante, permitindo ordem local e paralelismo entre entidades. Um stream global simplifica sequência, mas pode limitar escala; muitos streams pequenos complicam regras que exigem atomicidade entre eles.

Termo para lembrar: Aggregate stream | Base técnica: referência 1.

06. Como reconstrói estado?

Eu aplicaria eventos em ordem a uma função de transição determinística, partindo de estado inicial ou snapshot válido. Essa função não consulta relógio ou parceiro para redefinir fatos; o replay deve reproduzir a interpretação documentada.

Termo para lembrar: Replay fold | Base técnica: referência 1.

07. O que é snapshot?

Eu armazenaria estado derivado e posição do stream para reduzir eventos reaplicados. Ele é otimização, não desculpa para perder histórico necessário; formato e versão precisam permitir reconstrução quando o snapshot fica incompatível ou corrompido.

Termo para lembrar: Snapshot | Base técnica: referência 1.

08. Quando cria snapshot?

Eu escolheria por custo de reconstrução e frequência de leitura, não a cada evento sem medição. Snapshots frequentes aumentam escrita e espaço; raros alongam startup e comandos em entidades com histórico muito extenso.

Termo para lembrar: Snapshot cadence | Base técnica: referência 1.

09. Como valida snapshot?

Eu verificaria identidade do stream, posição, versão do modelo e integridade, comparando amostras com replay completo. Um snapshot errado pode contaminar todo resultado futuro; a aplicação deve conseguir descartá-lo e reconstruir de fonte confiável.

Termo para lembrar: Snapshot integrity | Base técnica: referência 1.

10. Como muda schema de evento?

Eu favoreceria alterações compatíveis e novos tipos ou versões para mudanças de significado. Eventos antigos continuam existindo; o leitor precisa interpretá-los corretamente sem exigir que todo histórico seja reescrito a cada release da aplicação.

Termo para lembrar: Event evolution | Base técnica: referência 1.

11. O que é upcaster?

Eu usaria transformação controlada de eventos antigos para um formato compreendido pelo leitor atual, preservando significado. Upcaster não pode inventar fatos inexistentes; valores ausentes exigem política explícita e testes com exemplos históricos reais.

Termo para lembrar: Upcasting | Base técnica: referência 1.

12. Pode corrigir evento antigo?

Eu normalmente acrescentaria evento corretivo com referência ao original, mantendo trilha de decisão. Reescrever história exige governança específica e reconstrução de consumidores; não trataria alteração de log autoritativo como update comum sem consequências.

Termo para lembrar: Historical correction | Base técnica: referência 1.

13. Como faz estorno?

Eu registraria nova operação que reverte o efeito permitido, ligada à original e com identidade própria. O evento original continua verdadeiro historicamente; a compensação pode falhar ou exigir aprovação, portanto precisa de estado e recuperação próprios.

Termo para lembrar: Compensating event | Base técnica: referência 1.

14. Como evita replay reenviar pagamentos?

Eu manteria reconstrução de estado separada de executores de efeitos, usando identidades e modo explícito de replay. Eventos históricos podem alimentar uma view nova sem acionar novamente banco parceiro, notificações ou ações já concluídas.

Termo para lembrar: Effect separation | Base técnica: referência 1.

15. Como trata timeout no append?

Eu consultaria o stream pela identidade da operação antes de produzir outra intenção. A escrita pode ter confirmado sem resposta; versão esperada e dedupe ajudam, mas o cliente precisa entender o resultado desconhecido e sua recuperação.

Termo para lembrar: Unknown append | Base técnica: referência 1.

16. Kafka é automaticamente event store?

Eu verificaria retenção, unicidade, concorrência esperada, consultas por entidade e recuperação exigidas. Kafka fornece log distribuído, mas o modelo de autoridade precisa ser construído; colocar eventos num tópico não resolve todas as operações de um event store.

Termo para lembrar: Log suitability | Base técnica: referência 1.

17. Como combina com CQRS?

Eu criaria views para consultas, atualizadas a partir dos eventos com checkpoints e idempotência. O comando continua validando na autoridade; uma projeção atrasada pode servir contexto, mas não decide uma invariante financeira sem revalidação apropriada.

Termo para lembrar: Projection model | Base técnica: referência 1.

18. Como mede lag das views?

Eu compararia posição produzida e aplicada, além da idade do evento pendente mais antigo. Ausência de novos eventos não é atraso; a métrica deve refletir se a consulta atende o frescor prometido ao usuário.

Termo para lembrar: Projection freshness | Base técnica: referência 1.

19. Como trata exclusão de dados pessoais?

Eu minimizaria conteúdo sensível e definiria referências, retenção e mecanismos aprovados para remoção. Criptografia e destruição de chaves possuem condições e limites; não prometeria conformidade automática sem analisar cópias, metadados e requisitos legais aplicáveis.

Termo para lembrar: Historical privacy | Base técnica: referência 1.

20. Como modela tempo de negócio?

Eu registraria instante e política relevantes ao fato, distinguindo ocorrência de processamento. Replay não deve recalcular cotação ou prazo usando o relógio atual; a decisão histórica precisa ser explicável a partir das informações preservadas.

Termo para lembrar: Temporal facts | Base técnica: referência 1.

21. Como trata evento externo tardio?

Eu aplicaria uma nova decisão conforme o estado atual e a política do domínio, preservando referência temporal original. Inserir retroativamente no meio do stream pode invalidar consumidores; correção precisa de semântica explícita e auditável.

Termo para lembrar: Late event | Base técnica: referência 1.

22. Como calcula saldo rapidamente?

Eu manteria acumulado ou snapshot derivado com posição conhecida, aplicando somente a cauda necessária. Também verificaria contra lançamentos; performance não autoriza abandonar a capacidade de reconstruir e detectar divergência financeira no histórico.

Termo para lembrar: Verified accumulation | Base técnica: referência 1.

23. Como faz transação entre streams?

Eu verificaria suporte atômico do store ou redesenharia com processo e reservas. Dois appends independentes não formam transferência atômica; o protocolo deve proteger estados intermediários e resultados desconhecidos sem esconder a limitação do armazenamento.

Termo para lembrar: Cross-stream invariant | Base técnica: referência 1.

24. Como testa domínio event-sourced?

Eu usaria sequências de eventos e comandos para verificar decisões e invariantes, incluindo duplicatas e conflitos. Testes por propriedade ajudam encontrar combinações; integração valida append, snapshots e recuperação que funções puras não cobrem sozinhas.

Termo para lembrar: Sequence testing | Base técnica: referência 1.

25. Como faz migração de sistema CRUD?

Eu definiria um estado inicial ou eventos de abertura documentados, sem fingir reconstruir fatos históricos que não existem. A partir do corte, estabeleço autoridade e contratos; a migração precisa preservar operações em andamento e auditoria disponível.

Termo para lembrar: Event sourcing migration | Base técnica: referência 1.

26. Como evita evento técnico demais?

Eu registraria mudanças de negócio estáveis, não cada setter ou coluna alterada. Eventos excessivamente físicos acoplam consumidores à implementação e dificultam evolução; granularidade deve permitir explicar a decisão e reconstruir o comportamento necessário.

Termo para lembrar: Domain event design | Base técnica: referência 1.

27. Como faz retenção do log?

Eu separaria histórico necessário à autoridade de cópias de transporte e projeções, definindo política e capacidade. Apagar eventos sem base recuperável pode destruir reconstrução; snapshots isolados talvez não atendam auditoria ou novas consultas históricas futuras.

Termo para lembrar: History retention | Base técnica: referência 1.

28. Como recupera evento inválido no passado?

Eu identificaria o problema, definiria correção versionada e testaria a reconstrução antes de publicar. Pular silenciosamente um evento pode alterar saldo; a política precisa preservar explicação e permitir comparar o estado anterior com o corrigido.

Termo para lembrar: Replay compatibility | Base técnica: referência 1.

29. Quando não adotaria event sourcing?

Eu manteria estado atual com histórico quando isso atende auditoria e evolução, evitando custo de versões, replay e operação. Event sourcing é útil quando suas capacidades são centrais, não como requisito automático para escrever sistemas financeiros.

Termo para lembrar: Complexity threshold | Base técnica: referência 1.

30. Como defenderia a escolha?

Eu mostraria uma necessidade concreta de reconstrução ou explicação que o modelo atende melhor, junto de concorrência e recuperação. Também assumiria os custos de schema histórico e governança, demonstrando que o log não é apenas um slogan de imutabilidade.

Termo para lembrar: Historical authority rationale | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Eventos autoritativos

Ganho. Preservam história

Custo e motivo. Complicam evolução

Como decidir. Quando reconstrução é central

02. Snapshot frequente

Ganho. Acelera reidratação

Custo e motivo. Aumenta escrita

Como decidir. Quando streams são longos

03. Snapshot raro

Ganho. Reduz manutenção

Custo e motivo. Aumenta replay

Como decidir. Quando comandos são pouco frequentes

04. Upcasting

Ganho. Mantém leitores atuais

Custo e motivo. Acumula lógica histórica

Como decidir. Quando formatos antigos persistem

05. Evento corretivo

Ganho. Preserva trilha

Custo e motivo. Complica consultas

Como decidir. Quando passado deve ser explicado

06. Stream por agregado

Ganho. Permite paralelismo

Custo e motivo. Complica regras cruzadas

Como decidir. Quando invariantes são locais

07. Log global

Ganho. Simplifica sequência

Custo e motivo. Pode limitar escala

Como decidir. Quando ordem global é necessária

08. Views especializadas

Ganho. Aceleram consultas

Custo e motivo. Introduzem lag

Como decidir. Quando leitura difere da escrita

09. Payload rico

Ganho. Preserva contexto

Custo e motivo. Amplia dados sensíveis

Como decidir. Quando replay precisa dos fatos

10. Retenção longa

Ganho. Permite novos replays

Custo e motivo. Custa armazenamento e governança

Como decidir. Quando benefício é comprovado

Duas alternativas ao caminho proposto

1. Estado atual com auditoria

Quando faz sentido. Quando histórico serve investigação e não autoridade

Como implementar. Persistir transição e estado atomicamente

Limites. Não reconstrói qualquer versão sem modelagem adicional

2. Datomic

Quando faz sentido. Quando fatos e consultas temporais são adequados

Como implementar. Usar transações e valores imutáveis do banco

Limites. Não substitui automaticamente eventos de domínio e coordenação externa

Referências e alcance da evidência

1. Microsoft: Event Sourcing

Eventos como autoridade, projeções, replay e custos de evolução do modelo.

https://learn.microsoft.com/en-us/azure/architecture/patterns/event-sourcing

2. Microsoft: CQRS pattern

Separação de modelos de leitura e escrita e seus custos de consistência e complexidade.

https://learn.microsoft.com/en-us/azure/architecture/patterns/cqrs

3. Datomic: Processing Transactions

Processamento serial de transações, resultados e ambiguidade de timeout.

https://docs.datomic.com/transactions/transaction-processing.html

Continue em: T028 · T037 · T091

T040 / D. Cache e modelos de dados

CDC e Debezium

CDC captura mudanças confirmadas da fonte, frequentemente pelo log, para alimentar projeções ou integrações. Debezium fornece conectores, snapshots e posições; consumidores continuam responsáveis por semântica, idempotência e evolução.

Quando usar. Para busca, analytics, migrações e publicação de outbox sem polling intensivo.

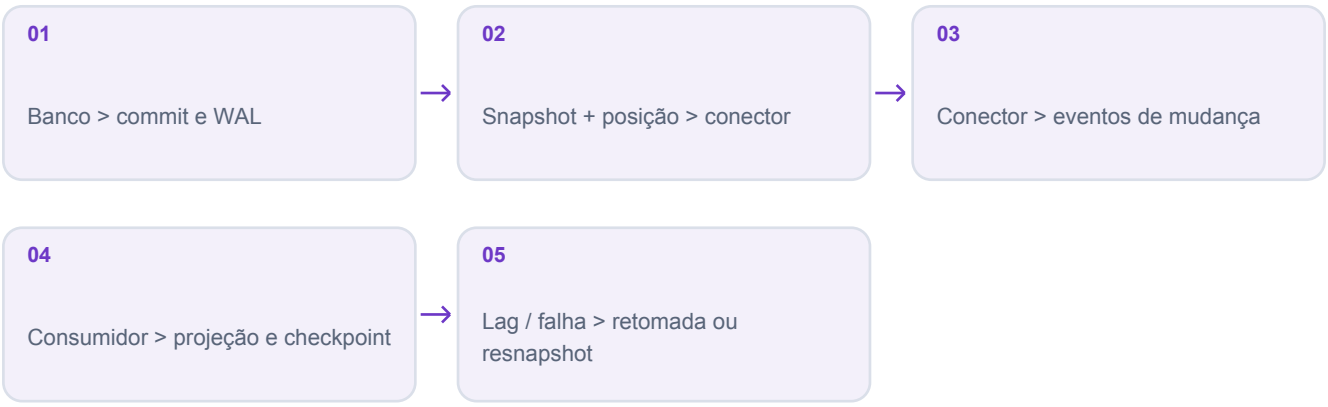
Quando evitar ou limitar. Evitar interpretar toda alteração de coluna como evento de negócio ou ignorar retenção do log, permissões e recuperação após perder a posição.

Uma abertura possível

Vou explicar a passagem do snapshot inicial para o fluxo contínuo, como persistimos progresso e como reconstruímos se a retenção acabar. Capturar mudanças não elimina o contrato do consumidor.

Peças e tecnologias. Debezium · Kafka Connect · PostgreSQL logical decoding · replication slots · LSN · Outbox SMT

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Definir tabelas e semântica de consumo.

2. Configurar permissões, log e retenção.

3. Executar snapshot com transição consistente.

4. Persistir progresso e aplicar idempotentemente.

5. Monitorar lag, schema e recuperação de failover.

30 perguntas para sustentar o desenho

01. O que CDC captura?

Eu capturaria alterações persistidas na fonte conforme o conector, incluindo inserções, atualizações e exclusões. Isso representa mudança de dados, não necessariamente intenção de negócio; consumidores precisam interpretar o contrato e a estrutura exposta pelo mecanismo.

Termo para lembrar: Change data capture | Base técnica: referência 1.

02. Por que usar log em vez de polling?

Eu reduziria consultas repetidas à tabela e aproveitaria a sequência de mudanças registrada pelo banco. Em troca, preciso operar permissões, posição e retenção do log; essa infraestrutura vira parte do caminho de atualização das projeções.

Termo para lembrar: Log-based capture | Base técnica: referência 1.

03. CDC substitui outbox?

Eu distinguiria capturar tabelas de domínio de capturar uma outbox explícita. A segunda preserva contratos de negócio definidos pela aplicação; CDC é o transporte do fato local, não uma garantia automática de que qualquer alteração expressa o evento correto.

Termo para lembrar: Intent versus storage change | Base técnica: referência 1.

04. Como começa com dados existentes?

Eu tiraria uma base consistente e registraria posição que permita continuar pelas mudanças subsequentes, usando o protocolo do conector. Uma exportação independente sem coordenação pode perder ou duplicar alterações ocorridas durante a leitura inicial.

Termo para lembrar: Initial snapshot | Base técnica: referência 1.

05. Como evita lacuna entre snapshot e streaming?

Eu seguiria o mecanismo documentado que relaciona snapshot e posição do log, mantendo retenção suficiente até a captura avançar. O consumidor tolera repetições; o requisito essencial é não deixar uma janela sem cobertura das mudanças confirmadas.

Termo para lembrar: Snapshot handoff | Base técnica: referência 1.

06. O que é LSN?

Eu o usaria como posição no log PostgreSQL dentro do contexto apropriado, não como horário universal ou versão de negócio. Ele ajuda retomada e diagnóstico, mas failover e timeline exigem tratamento conforme a implantação e o conector.

Termo para lembrar: Log sequence number | Base técnica: referência 1.

07. O que é replication slot?

Eu o trataria como estado que preserva recursos necessários à continuidade do consumidor, mesmo desconectado. Isso ajuda retomada, mas pode reter WAL e pressionar disco; monitoramento e limites são obrigatórios para proteger a fonte.

Termo para lembrar: Replication slot | Base técnica: referência 1.

08. Conector parado pode derrubar o banco?

Eu monitoraria bytes retidos, espaço livre e tempo de atraso, porque a retenção pode crescer até comprometer armazenamento. O plano define alerta, capacidade e consequência de abandonar a posição, incluindo possível resnapshot controlado.

Termo para lembrar: Retained WAL pressure | Base técnica: referência 1.

09. Como trata duplicatas após reinício?

Eu faria o destino aplicar por identidade ou versão, porque recuperação pode reenviar mudanças já vistas. A posição do conector não torna qualquer efeito externo único; consumidor precisa proteger sua própria fronteira de persistência e consequência.

Termo para lembrar: At-least-once capture | Base técnica: referência 1.

10. Como persiste progresso no destino?

Eu ligaria checkpoint à alteração da projeção numa transação quando possível. Se não há atomicidade comum, uso aplicação idempotente e protocolo de retomada; confirmar progresso antes da escrita cria perda silenciosa de mudança.

Termo para lembrar: Atomic checkpoint | Base técnica: referência 1.

11. CDC preserva ordem global?

Eu verificaria a ordem oferecida pela fonte, tópicos, partições e consumidores. Separar tabelas em canais diferentes pode alterar observação conjunta; se uma regra depende de sequência entre entidades, preciso de metadados e coordenação apropriados.

Termo para lembrar: Ordering scope | Base técnica: referência 1.

12. Como trata transação com várias tabelas?

Eu avaliaria se o destino precisa aplicar o conjunto junto e usaria metadados suportados ou contrato composto. Mudanças confirmadas atomicamente na origem podem ser observadas separadamente no consumidor; CDC não transfere automaticamente a atomicidade local.

Termo para lembrar: Transaction metadata | Base técnica: referência 1.

13. Como funciona outbox EventRouter?

Eu modelaria tabela conforme o contrato da transformação, com identidade de evento e agregado, usando inserts como publicação. Não misturaria ingenuamente updates de status do polling; a configuração e os filtros precisam respeitar os tipos de registros esperados.

Termo para lembrar: Outbox routing | Base técnica: referência 1.

14. Por que usar aggregate_id como chave?

Eu buscaria manter eventos relacionados no mesmo domínio de ordenação do broker, conforme o particionamento escolhido. Isso ajuda consumidores, mas ainda preciso preservar ordem na origem e considerar mudanças de partições, retries e operações concorrentes.

Termo para lembrar: Partition key | Base técnica: referência 1.

15. Como trata exclusão?

Eu interpretaria o evento de remoção e possíveis tombstones conforme o conector e o broker, preservando versão e identidade. Ignorar essa diferença pode deixar documentos antigos no índice ou apagar incorretamente informação durante replay.

Termo para lembrar: Delete event | Base técnica: referência 1.

16. Como obtém valores anteriores?

Eu verificaria configuração e disponibilidade dos campos antigos necessários, porque nem todo UPDATE carrega um before completo por padrão. Exigir mais informação pode aumentar volume e custo; o consumidor deve depender apenas do contrato configurado.

Termo para lembrar: Replica identity | Base técnica: referência 1.

17. Como trata mudança de schema?

Eu coordenaria DDL, conector e consumidores, testando adição, remoção e mudança de tipo. Capturar automaticamente não significa interpretar automaticamente; uma alteração física pode quebrar projeções que dependiam de formato ou significado antigo.

Termo para lembrar: Schema evolution | Base técnica: referência 1.

18. Como evita vazamento de colunas sensíveis?

Eu limitaria tabelas e colunas capturadas, validando também metadados e registros anteriores. O pipeline cria novas cópias de dados; permissões, retenção e destinos precisam ser governados antes de habilitar captura ampla da base.

Termo para lembrar: Capture minimization | Base técnica: referência 1.

19. Como lida com tabela sem chave?

Eu definiria uma identidade estável ou mudaria o modelo antes de confiar em upserts e deletes. Sem chave adequada, aplicar mudanças no destino pode ser ambíguo; o conector não inventa automaticamente uma identidade de negócio correta.

Termo para lembrar: Record identity | Base técnica: referência 1.

20. Como dimensiona o conector?

Eu consideraria volume de WAL, tamanho de eventos, serialização e capacidade do broker, incluindo bursts e snapshots. A taxa de transações sozinha não basta; uma transação grande pode gerar muito mais trabalho que milhares de pequenas.

Termo para lembrar: Capture throughput | Base técnica: referência 1.

21. Como controla snapshot pesado?

Eu usaria mecanismos suportados de divisão e limites, monitorando I/O e locks na fonte. A cópia inicial não deve derrubar o sistema que pretende integrar; capacidade de retenção precisa cobrir o tempo adicional de uma execução controlada.

Termo para lembrar: Snapshot throttling | Base técnica: referência 1.

22. Como trata backpressure do broker?

Eu acompanharia a propagação do atraso até a retenção na fonte, definindo limites e ações. Broker indisponível não é um problema isolado; pode pressionar disco do banco e exigir reduzir carga ou recuperar rapidamente o pipeline.

Termo para lembrar: Pipeline backpressure | Base técnica: referência 1.

23. Como monitora frescor?

Eu mediria posição capturada, publicada e aplicada no destino, distinguindo cada etapa. Conector saudável pode coexistir com consumidor parado; a jornada depende do último estágio, não apenas de o processo de captura estar em execução.

Termo para lembrar: End-to-end CDC lag | Base técnica: referência 1.

24. Como faz failover PostgreSQL com CDC?

Eu verificaria suporte da versão e configuração para continuidade de slots e posições no novo primário. Não assumiria retomada automática universal; o runbook precisa evitar lacunas e duplicatas perigosas, incluindo possível reconstrução quando necessário.

Termo para lembrar: Slot continuity | Base técnica: referência 1.

25. O que faz se o WAL necessário sumiu?

Eu reconheceria que não posso continuar de uma posição sem dados disponíveis e acionaria reconstrução controlada. O destino precisa de estratégia para comparar gerações e aplicar mudanças novas; reiniciar indefinidamente o conector não recupera log apagado.

Termo para lembrar: Resnapshot recovery | Base técnica: referência 1.

26. Como reprocessa sem afetar produção?

Eu usaria destino ou geração nova e capacidade limitada, mantendo consumidores de efeitos externos separados. Reindexar histórico não deve reenviar pagamentos ou notificações; identidade e autorização de replay fazem parte da operação segura.

Termo para lembrar: Replay isolation | Base técnica: referência 1.

27. Como usa CDC numa migração?

Eu copiaria a base e acompanharia mudanças até o corte, preservando uma autoridade de escrita. Antes de mudar tráfego, validaria equivalência e posição; escrita dupla sem protocolo não é substituto para sincronização confiável.

Termo para lembrar: Migration catch-up | Base técnica: referência 1.

28. Como compara CDC e eventos de domínio?

Eu usaria eventos de domínio para contratos de negócio estáveis e CDC para mudança física ou transporte de outbox. Consumir tabelas diretamente acopla consumidores ao schema; isso pode ser aceitável em analytics, mas exige governança explícita.

Termo para lembrar: Semantic coupling | Base técnica: referência 1.

29. Como testa recuperação do pipeline?

Eu interromperia conector e broker, alteraria schema e executaria failover controlado, verificando completude e duplicatas no destino. Contar mensagens no caminho feliz não prova que o fluxo cobre a janela entre snapshot, commit e retomada.

Termo para lembrar: CDC fault testing | Base técnica: referência 1.

30. Como defenderia Debezium no desenho?

Eu explicaria fonte, posição, retenção e aplicação idempotente, além do responsável por incidentes. A escolha reduz código de captura, mas mantém obrigações de operação; o benefício precisa superar complexidade de log, schema e recuperação da integração.

Termo para lembrar: Recoverable capture | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. CDC pelo log

Ganho. Reduz polling

Custo e motivo. Exige slots e retenção

Como decidir. Quando volume e frescor justificam

02. Polling

Ganho. Simplifica infraestrutura

Custo e motivo. Pressiona tabelas

Como decidir. Quando volume é pequeno

03. Snapshot amplo

Ganho. Copia base rapidamente

Custo e motivo. Compete por I/O

Como decidir. Quando fonte tem capacidade

04. Snapshot limitado

Ganho. Protege origem

Custo e motivo. Prolonga retenção

Como decidir. Quando tráfego online é prioritário

05. Evento de domínio

Ganho. Preserva semântica

Custo e motivo. Exige código de publicação

Como decidir. Quando contrato é externo ao schema

06. CDC de tabela

Ganho. Captura mudanças sem app

Custo e motivo. Acopla formato físico

Como decidir. Quando consumidores aceitam

07. Before completo

Ganho. Facilita interpretação

Custo e motivo. Aumenta bytes

Como decidir. Quando regras precisam do valor antigo

08. Retenção longa

Ganho. Permite retomada

Custo e motivo. Consome disco

Como decidir. Quando outages podem durar

09. Destino idempotente

Ganho. Tolera replay

Custo e motivo. Exige identidade e versão

Como decidir. Quando entrega pode repetir

10. Geração nova

Ganho. Permite reconstrução segura

Custo e motivo. Duplica armazenamento

Como decidir. Quando leitura deve continuar

Duas alternativas ao caminho proposto

1. Outbox com polling

Quando faz sentido. Quando simplicidade operacional é prioridade

Como implementar. Persistir evento junto da mudança e publicar em lotes

Limites. Exige claims, cleanup e controle de ordem

2. Eventos de aplicação

Quando faz sentido. Quando intenção de negócio deve ser o contrato

Como implementar. Gerar fatos explícitos com publicação durável

Limites. Não captura alterações feitas fora do caminho governado

Referências e alcance da evidência

1. Debezium: PostgreSQL connector

Snapshots, captura de mudanças, posições, slots e aspectos operacionais do conector PostgreSQL.

https://debezium.io/documentation/reference/stable/connectors/postgresql.html

2. PostgreSQL: Logical Decoding Concepts

Decodificação do WAL, slots, reenvio após falha e retenção de recursos.

https://www.postgresql.org/docs/current/logicaldecoding-explanation.html

3. Debezium: Outbox Event Router

Transformação de eventos de outbox e contrato de tabela orientada a inserts.

https://debezium.io/documentation/reference/stable/transformations/outbox-event-router.html

Continue em: T030 · T037 · T062

T041 / E. Mensageria e processamento

Kafka: partições, grupos e offsets

Kafka mantém um log particionado com leitura independente por grupos e posição por offset.

Quando usar. Replay, múltiplos consumidores e processamento de eventos com ordem por chave.

Quando evitar ou limitar. Uma tarefa simples sem necessidade de retenção ou replay pode usar uma fila gerenciada.

Uma abertura possível

Primeiro escolho a chave e o contrato de processamento; depois dimensiono partições, retenção e consumidores.

Peças e tecnologias. Kafka · MSK · Kafka Streams · Schema Registry

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Escolher agregado e chave estável

2. Medir bytes por segundo e capacidade por partição

3. Configurar replicação e acks conforme durabilidade

4. Implementar efeito idempotente antes do commit de progresso

5. Testar rebalanceamento, replay e perda de líder

30 perguntas para sustentar o desenho

01. Qual é a unidade de ordem?

Eu trato a partição como unidade de ordem. Eventos em partições diferentes não possuem ordem global; uso sequência de agregado quando essa relação importa para o processamento financeiro.

Termo para lembrar: Partição | Base técnica: referência 1.

02. Como escolhe a chave de pagamento?

Eu particiono pelo agregado cuja sequência preciso preservar, como payment_id. Não uso uma constante, pois concentraria tudo numa partição e limitaria o paralelismo mesmo com vários brokers.

Termo para lembrar: Partition key | Base técnica: referência 1.

03. Uma conta muito ativa vira gargalo?

Sim, uma hot partition limita aquele fluxo. Avalio dividir responsabilidades ou subagregados, mas somente quando a regra permitir; espalhar eventos de saldo arbitrariamente comprometeria a ordem necessária.

Termo para lembrar: Hot partition | Base técnica: referência 1.

04. Adicionar consumidores sempre acelera?

Não. Num grupo tradicional, cada partição tem um consumidor atribuído por vez. Consumidores além das partições ficam sem trabalho; também verifico banco, CPU e dependências antes de escalar.

Termo para lembrar: Consumer group | Base técnica: referência 1.

05. Para que serve outro grupo?

Outro consumer group mantém progresso independente e recebe sua própria visão do log. Uso isso para extrato e antifraude sem obrigá-los a compartilhar um offset de conclusão.

Termo para lembrar: Consumer group | Base técnica: referência 1.

06. Offset é identificador de negócio?

Não. Offset identifica posição dentro de uma partição, não a intenção financeira. Mantenho event_id e operation_id no envelope para deduplicação, auditoria e republicação em outro tópico.

Termo para lembrar: Offset | Base técnica: referência 1.

07. Quando confirma o offset?

Confirmo depois de tornar o processamento recuperável. Se o efeito ocorre no PostgreSQL, uso idempotência local; cair antes do commit do offset poderá repetir a entrega sem repetir o efeito.

Termo para lembrar: Commit de offset | Base técnica: referência 1.

08. O que causa perda com auto-commit?

Se o offset avançar antes de o efeito ficar durável, uma queda pode pular trabalho. Eu controlo a relação entre progresso confirmado e conclusão real, incluindo processamento paralelo.

Termo para lembrar: Auto-commit | Base técnica: referência 1.

09. Como trata registros paralelos na mesma partição?

Mantenho um watermark contíguo de conclusão. Se terminar o offset 12 antes do 11, não avanço além da lacuna; caso contrário uma queda poderia perder o registro incompleto.

Termo para lembrar: Watermark contíguo | Base técnica: referência 1.

10. O que ocorre num rebalanceamento?

Partições mudam de consumidor. Paro trabalho revogado, persisto progresso seguro e tolero tarefas antigas ainda em execução; idempotência e versões protegem efeitos que ultrapassam a posse anterior.

Termo para lembrar: Rebalanceamento | Base técnica: referência 1.

11. Como um consumidor lento afeta o grupo?

Se ultrapassar limites de polling ou processamento, pode perder atribuições. Reduzo lotes, separo trabalho pesado e ajusto configurações com métricas; aumentar timeouts indiscriminadamente piora detecção de falhas.

Termo para lembrar: Poll interval | Base técnica: referência 1.

12. acks=all significa persistência em todos os brokers?

Não. A confirmação depende das réplicas relevantes e da configuração de ISR. Examino min.insync.replicas e fator de replicação; acks sozinho não descreve toda a política de durabilidade.

Termo para lembrar: ISR | Base técnica: referência 1.

13. Qual custo de exigir mais réplicas?

Eu ganho resistência a perdas, mas posso recusar escrita quando réplicas suficientes não estiverem disponíveis. Escolho esse comportamento conforme o risco de perder eventos financeiros aceitos pelo sistema.

Termo para lembrar: min.insync.replicas | Base técnica: referência 1.

14. O produtor idempotente protege o ledger?

Não. Ele reduz duplicatas da produção dentro de seu escopo. Uma chamada ao ledger permanece externa; precisa de operation_id estável e contrato próprio de repetição ou reconciliação.

Termo para lembrar: Idempotent producer | Base técnica: referência 1.

15. Transação Kafka cobre PostgreSQL?

Não automaticamente. Transações Kafka podem coordenar registros e offsets no Kafka, mas não incluem uma transação arbitrária do PostgreSQL. Uso outbox ou proteção idempotente na fronteira externa.

Termo para lembrar: Transação Kafka | Base técnica: referência 1.

16. Por que read_committed?

Uso read_committed quando o consumidor precisa evitar registros de transações Kafka abortadas. Isso não transforma chamadas HTTP subsequentes em operações únicas nem substitui validações do domínio consumidor.

Termo para lembrar: read_committed | Base técnica: referência 1.

17. Como decide retenção?

Dimensiono retenção por janela de recuperação e replay, multiplicando vazão em bytes pelo período e replicação. Incluo margem para consumidores atrasados e custo de armazenar dados sensíveis.

Termo para lembrar: Retention | Base técnica: referência 1.

18. Lag em mensagens é suficiente?

Não. Um milhão de eventos pequenos difere de arquivos grandes ou tarefas lentas. Meço lag temporal, bytes, idade do evento mais antigo e prazo de negócio afetado.

Termo para lembrar: Consumer lag | Base técnica: referência 1.

19. Como recupera consumidor além da retenção?

Se os eventos expiraram, não prometo replay inexistente. Reconstruo de snapshot ou fonte autoritativa, estabeleço um ponto de corte consistente e volto ao consumo incremental depois da validação.

Termo para lembrar: Bootstrap | Base técnica: referência 1.

20. Compaction preserva cada mudança histórica?

Não. Log compaction preserva valores relevantes por chave, não necessariamente todas as versões. Para auditoria completa, mantenho histórico apropriado e não confundo tópico compactado com livro financeiro.

Termo para lembrar: Log compaction | Base técnica: referência 1.

21. Para que servem tombstones?

Um tombstone representa remoção lógica de uma chave em tópico compactado. Verifico retenção e consumidores atrasados, pois reconstrução incorreta pode ressuscitar registros se exclusões necessárias já desapareceram.

Termo para lembrar: Tombstone | Base técnica: referência 1.

22. Aumentar partições mantém o roteamento anterior?

Não necessariamente. Uma função de hash pode mover a mesma chave. Planejo transição e ordem durante a mudança; não suponho que aumentar partições seja semanticamente transparente.

Termo para lembrar: Repartitioning | Base técnica: referência 1.

23. Como evita payload gigante?

Envio metadados e referência imutável para object storage quando o corpo é grande. Incluo checksum e autorização de leitura; o consumidor precisa distinguir arquivo ausente de mensagem inválida.

Termo para lembrar: Claim check | Base técnica: referência 1.

24. Como publica evolução de schema?

Uso contratos versionados e testes de compatibilidade entre produtores e consumidores. Campo novo opcional é diferente de mudar o significado de um valor existente; compatibilidade binária não garante semântica.

Termo para lembrar: Schema evolution | Base técnica: referência 1.

25. O que faz com evento inválido?

Isolo com motivo e identidade numa quarentena, preservando dependências de ordem. Se eventos seguintes exigem o inválido, bloqueio aquele agregado e recupero a lacuna em vez de pular silenciosamente.

Termo para lembrar: Poison message | Base técnica: referência 2.

26. Kafka substitui controle de carga?

Não. A retenção absorve atraso por um período, mas não cria capacidade infinita. Limito consumo e produção, e comparo chegada com vazão sustentável para calcular tempo de drenagem.

Termo para lembrar: Backpressure | Base técnica: referência 1.

27. Como protege dados no tópico?

Uso autenticação, ACL por tópico e grupo, criptografia e payload mínimo. Restrinjo replay porque ele também concede acesso histórico; remover acesso da API não remove automaticamente acesso ao broker.

Termo para lembrar: ACL | Base técnica: referência 1.

28. Como migra consumidores sem duplicar notificações?

Uso identidade lógica estável e proteção de efeitos. Reconstruções usam destinos isolados e modo sem side effects; não renomeio o grupo e presumo que replay não terá consequências.

Termo para lembrar: Replay seguro | Base técnica: referência 1.

29. Como testa falha de broker?

Injeto perda de líder e indisponibilidade de réplicas dentro de limites controlados. Verifico confirmação de produção, retomada de consumidores e invariantes financeiros, não apenas a saúde do cluster.

Termo para lembrar: Failover | Base técnica: referência 1.

30. Quando rejeitaria Kafka?

Se o requisito for uma fila pequena de jobs sem replay independente, prefiro solução mais simples. Justifico Kafka por retenção, fan-out e ecossistema, considerando operação, custo e competência do time.

Termo para lembrar: Fit arquitetural | Base técnica: referência 2.

10 trade-offs: ganho, custo e decisão

01. Mais partições

Ganho. Mais paralelismo

Custo e motivo. Mais metadados e rebalanceamentos

Como decidir. Quando a vazão medida exceder capacidade atual

02. acks e ISR rigorosos

Ganho. Maior durabilidade

Custo e motivo. Escritas podem parar

Como decidir. Quando perder evento aceito é inaceitável

03. Chave por conta

Ganho. Ordem local clara

Custo e motivo. Conta quente limita throughput

Como decidir. Quando operações dependem do estado da conta

04. Chave por operação

Ganho. Distribuição melhor

Custo e motivo. Não ordena operações da conta

Como decidir. Quando o domínio resolve concorrência separadamente

05. Retenção longa

Ganho. Replay e auditoria operacional

Custo e motivo. Disco e exposição

Como decidir. Quando janela de recuperação exigir

06. Compaction

Ganho. Reconstrução do último estado

Custo e motivo. Histórico intermediário desaparece

Como decidir. Para projeções por chave

07. Lotes maiores

Ganho. Amortiza rede e disco

Custo e motivo. Aumenta latência e memória

Como decidir. Quando throughput é prioritário

08. Transações Kafka

Ganho. Atomicidade dentro do Kafka

Custo e motivo. Coordenação e latência

Como decidir. Quando offsets e saídas precisam avançar juntos

09. Múltiplos grupos

Ganho. Consumidores independentes

Custo e motivo. Mais leitura e operação

Como decidir. Para funções com SLAs diferentes

10. Referência de arquivo

Ganho. Mensagens pequenas

Custo e motivo. Dependência adicional

Como decidir. Para payload grande com identidade imutável

Duas alternativas ao caminho proposto

1. SQS ou Service Bus

Quando faz sentido. Jobs sem replay de log

Como implementar. Fila com ACK e consumidores idempotentes

Limites. Sem equivalência automática com log particionado

2. PostgreSQL com jobs

Quando faz sentido. Volume moderado e domínio relacional

Como implementar. Claim atômico e tarefas persistidas

Limites. Polling e crescimento pressionam o banco

Referências e alcance da evidência

1. Apache Kafka - Design

Log particionado, offsets, replicação e fronteiras de garantias transacionais.

https://kafka.apache.org/41/design/design/

2. AWS - SQS overview

Filas gerenciadas, modalidades e desacoplamento; efeitos externos exigem garantias próprias.

https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/welcome.html

Continue em: T043 · T044 · T045 · T062

T042 / E. Mensageria e processamento

Filas e pub/sub: SQS, SNS, RabbitMQ e Service Bus

Filas distribuem trabalho; pub/sub distribui cópias a consumidores independentes.

Quando usar. Desacoplar processamento, absorver picos e separar responsabilidades.

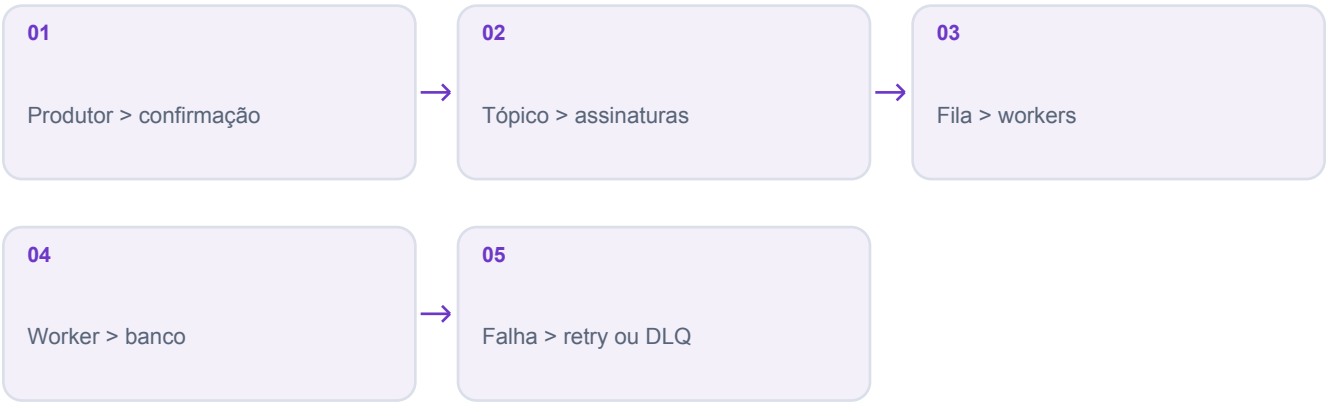
Quando evitar ou limitar. Não substitui uma transação nem torna efeitos externos únicos.

Uma abertura possível

Vou distinguir confirmação de publicação, entrega e conclusão do negócio antes de escolher o broker.

Peças e tecnologias. SQS · SNS · RabbitMQ · Azure Service Bus

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Definir work queue ou fan-out

2. Escolher política de ACK e retenção

3. Configurar limites de concorrência e prefetch

4. Aplicar idempotência ao efeito

5. Testar reentrega, indisponibilidade e redrive

30 perguntas para sustentar o desenho

01. Fila e tópico resolvem o mesmo problema?

Não. Uma work queue distribui cada trabalho entre consumidores concorrentes; pub/sub cria entregas para assinaturas independentes. Escolho conforme responsabilidade: vários workers iguais ou várias funções que precisam observar o evento.

Termo para lembrar: Work queue | Base técnica: referência 1.

02. SNS sozinho executa meu job?

SNS distribui notificações; não representa a conclusão do job. Para processamento amortecido por consumidores, posso ligar SNS a filas SQS, cada uma com sua retenção e política operacional.

Termo para lembrar: Fan-out | Base técnica: referência 1.

03. Dois workers na mesma fila recebem tudo?

Eles competem pelo trabalho, não formam duas cópias independentes do histórico. Se antifraude e extrato precisam de todos os eventos, crio assinaturas separadas com filas próprias e métricas distintas.

Termo para lembrar: Competing consumers | Base técnica: referência 3.

04. O que publisher confirm confirma?

Confirma a interação do produtor com o broker segundo a configuração. Não prova que o consumidor atualizou seu banco ou que o cliente recebeu uma notificação; acompanho cada fronteira separadamente.

Termo para lembrar: Publisher confirm | Base técnica: referência 2.

05. Quando o consumidor deve enviar ACK?

Depois de persistir o resultado ou uma obrigação durável de continuar. Confirmar antes permite perder trabalho numa queda; confirmar depois pode gerar duplicata, então implemento proteção idempotente no destino local.

Termo para lembrar: Acknowledgement | Base técnica: referência 2.

06. Fila durável basta para não perder mensagens?

Não isoladamente. Verifico persistência da mensagem, confirmação do produtor, replicação e política de recuperação. O significado de durável depende dessas configurações; também considero perda regional e retenção do produto.

Termo para lembrar: Durabilidade | Base técnica: referência 2.

07. Como escolhe SQS ou RabbitMQ?

SQS reduz administração de infraestrutura. RabbitMQ oferece roteamento e controle próprios, com operação de cluster. Escolho por protocolos, topologia e competência operacional; não apenas pela familiaridade com uma biblioteca.

Termo para lembrar: Broker gerenciado | Base técnica: referência 1.

08. Quando Service Bus é interessante?

Quando preciso integrar no ecossistema Azure com filas, tópicos e recursos como sessões. Verifico os contratos de lock e transações; recursos do broker não abrangem automaticamente meu banco externo.

Termo para lembrar: Service Bus | Base técnica: referência 3.

09. Prefetch alto sempre melhora throughput?

Prefetch alto pode amortizar round trips, mas concentra trabalho em um consumidor e aumenta memória. Ajusto conforme duração dos jobs e capacidade local, observando distribuição e tempo até confirmação.

Termo para lembrar: Prefetch | Base técnica: referência 2.

10. Como evita perder mensagens em desligamento?

Paro novas entregas, termino ou interrompo tarefas com segurança e confirmo somente efeitos duráveis. O broker pode reentregar o restante; o shutdown precisa respeitar orçamento de tempo e idempotência.

Termo para lembrar: Graceful shutdown | Base técnica: referência 2.

11. Mensagem invisível está processada?

Não. Visibilidade indica posse temporária de entrega, não conclusão. O efeito só está confirmado após a transação pertinente; timeout ou perda do worker pode tornar o mesmo trabalho novamente elegível.

Termo para lembrar: Visibility timeout | Base técnica: referência 1.

12. FIFO garante crédito único?

Não. Ordem e deduplicação do broker têm escopo próprio. Um worker pode executar crédito e cair antes da confirmação; o ledger precisa reconhecer a identidade da operação financeira independentemente do transporte.

Termo para lembrar: FIFO | Base técnica: referência 1.

13. Como identifica uma duplicata?

Uso event_id estável dentro do escopo do consumidor e, quando necessário, operation_id de negócio. Um delivery tag ou receipt handle identifica a tentativa de entrega e não substitui essa identidade.

Termo para lembrar: Identidade de evento | Base técnica: referência 2.

14. O que fazer quando ACK falha?

Presumo que o broker pode repetir a entrega. Consulto o estado persistido e reconheço a duplicata; não tento desfazer o negócio somente porque a confirmação de transporte não chegou.

Termo para lembrar: ACK ambíguo | Base técnica: referência 2.

15. Como lida com consumidor lento?

Limito prefetch, concorrência e duração de tarefas. Meço idade do backlog e separo classes de trabalho quando necessário. Aumentar workers sem capacidade no banco só muda o lugar da fila.

Termo para lembrar: Backpressure | Base técnica: referência 2.

16. Como calcula capacidade para drenar?

Comparo taxa de chegada com taxa sustentável de conclusão. Se chegam 100 jobs/s e processo 120, dreno 20/s; 72 mil pendências exigem aproximadamente uma hora sem novo agravamento.

Termo para lembrar: Drain rate | Base técnica: referência 1.

17. Pode usar uma fila para clientes de tamanhos diferentes?

Pode, mas controlo justiça. Um tenant ruidoso pode dominar o backlog; uso quotas, filas por classe ou escalonamento ponderado, evitando criar uma fila operacionalmente cara para cada usuário.

Termo para lembrar: Fair queuing | Base técnica: referência 3.

18. Como trata falhas permanentes?

Validação impossível não merece retry infinito. Encaminho a quarentena com motivo e rastreabilidade, preservando o conteúdo necessário para corrigir. Defino alerta e redrive; DLQ abandonada é trabalho perdido operacionalmente.

Termo para lembrar: DLQ | Base técnica: referência 1.

19. Quando usar atraso de entrega?

Uso atraso para retries e tarefas futuras dentro dos limites do produto. Para calendários complexos ou esperas longas, prefiro scheduler durável; uma fila de transporte não é calendário de negócio.

Termo para lembrar: Delayed delivery | Base técnica: referência 3.

20. Como suporta mensagem grande?

Guardo o corpo em object storage e envio referência imutável com checksum. Retenção do objeto precisa cobrir retries e replay; controle de acesso do broker não autoriza automaticamente o download.

Termo para lembrar: Claim check | Base técnica: referência 1.

21. O que muda com compressão?

Compressão reduz bytes trafegados, mas consome CPU e pode ocultar payload descompactado excessivo. Defino limites antes e depois da expansão para evitar que uma mensagem pequena derrube o consumidor.

Termo para lembrar: Compressão | Base técnica: referência 1.

22. Como trata reordenação?

Incluo agregado e sequência quando o processamento depende da ordem. Uma fila sem garantia necessária pode entregar eventos fora de sequência; adio lacunas e evito marcar conclusão prematuramente.

Termo para lembrar: Sequência | Base técnica: referência 3.

23. Qual risco de retry imediato por nack?

Uma mensagem defeituosa pode circular sem progresso, consumindo CPU e rede. Uso atraso, limites e classificação de erro, em vez de criar um ciclo apertado de requeue.

Termo para lembrar: Requeue loop | Base técnica: referência 2.

24. Como monitora um tópico com várias assinaturas?

Acompanho cada assinatura separadamente. Publicação saudável não significa entrega saudável em todas elas; verifico backlog, idade, erros e DLQ por consumidor para localizar qual função está atrasada.

Termo para lembrar: Subscription lag | Base técnica: referência 3.

25. Como publica e grava banco atomicamente?

Uso outbox no mesmo banco da alteração. O relay publica com identidade estável e admite repetição. Não faço commit no banco e depois uma publicação sem obrigação persistida de recuperar.

Termo para lembrar: Transactional outbox | Base técnica: referência 1.

26. Como migra de broker?

Faço transição com identidades preservadas, observação de duplicatas e pontos de corte. Dual publish sem controle pode mudar ordem e repetir efeitos; preparo consumidores para coexistência e reconciliação.

Termo para lembrar: Migração de transporte | Base técnica: referência 2.

27. Criptografar fila resolve autorização?

Não. Criptografia protege acesso ao conteúdo sob certas condições; permissões definem quem pode publicar, ler e administrar. Restrinjo também redrive e inspeção de mensagens porque podem expor dados bancários.

Termo para lembrar: Least privilege | Base técnica: referência 3.

28. Quando mensagem expira?

Expiração segue a retenção configurada do serviço. Alerto antes de o backlog alcançar esse limite e considero DLQ separadamente; um sistema disponível pode perder trabalho simplesmente por retenção insuficiente.

Termo para lembrar: Retention | Base técnica: referência 1.

29. Como testa entrega pelo menos uma vez?

Provoco queda após efeito local e antes do ACK, restauro o consumidor e observo reentrega. A prova é preservar a invariante de negócio, não apenas contar mensagens ou logs.

Termo para lembrar: Fault injection | Base técnica: referência 2.

30. Quando evitar um broker?

Se banco e worker atendem volume e recuperação, não adiciono broker automaticamente. A fila externa traz independência e buffering, mas também contratos, custo, segurança e operação para justificar.

Termo para lembrar: Simplicidade operacional | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Fila gerenciada

Ganho. Menos administração

Custo e motivo. Menor controle interno

Como decidir. Quando requisitos cabem no serviço

02. RabbitMQ próprio

Ganho. Protocolos e roteamento flexíveis

Custo e motivo. Operação de cluster

Como decidir. Quando topologia justifica

03. Fila por consumidor

Ganho. Isola atraso

Custo e motivo. Mais recursos e métricas

Como decidir. Para responsabilidades independentes

04. Fila compartilhada

Ganho. Simplicidade

Custo e motivo. Noisy neighbor

Como decidir. Quando trabalhos são homogêneos

05. Prefetch alto

Ganho. Menos round trips

Custo e motivo. Memória e desequilíbrio

Como decidir. Para tarefas pequenas medidas

06. ACK manual

Ganho. Controle de durabilidade

Custo e motivo. Código adicional

Como decidir. Quando efeito precisa de confirmação explícita

07. FIFO por grupo

Ganho. Ordem local

Custo e motivo. Grupo quente serializa

Como decidir. Quando agregado exige sequência

08. Payload externo

Ganho. Menor mensagem

Custo e motivo. Mais uma dependência

Como decidir. Quando tamanho cresce

09. Retry atrasado

Ganho. Reduz tempestade

Custo e motivo. Aumenta tempo de recuperação

Como decidir. Para falhas transitórias

10. Retenção maior

Ganho. Mais tempo para recuperar

Custo e motivo. Custo e exposição

Como decidir. Quando backlog pode durar dias

Duas alternativas ao caminho proposto

1. Kafka

Quando faz sentido. Replay e consumidores independentes

Como implementar. Log particionado com offsets por grupo

Limites. Mais operação e política de retenção

2. Jobs no PostgreSQL

Quando faz sentido. Volume moderado

Como implementar. Claim atômico e workers com leases

Limites. Contenção e manutenção do histórico

Referências e alcance da evidência

1. AWS - SQS overview

Filas gerenciadas, modalidades e desacoplamento; efeitos externos exigem garantias próprias.

https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/welcome.html

2. RabbitMQ - confirms e acknowledgements

Confirmação ao produtor, ACK do consumidor, prefetch e reentrega.

https://www.rabbitmq.com/docs/confirms

3. Azure - Service Bus

Filas, tópicos, assinaturas e coordenação de mensagens.

https://learn.microsoft.com/en-us/azure/service-bus-messaging/service-bus-messaging-overview

Continue em: T041 · T043 · T046 · T063

T043 / E. Mensageria e processamento

Ordenação e sequência por agregado

Sequência por agregado torna explícita a relação entre alterações que dependem umas das outras.

Quando usar. Abertura de conta antes de lançamentos, transições de caso e projeções incrementais.

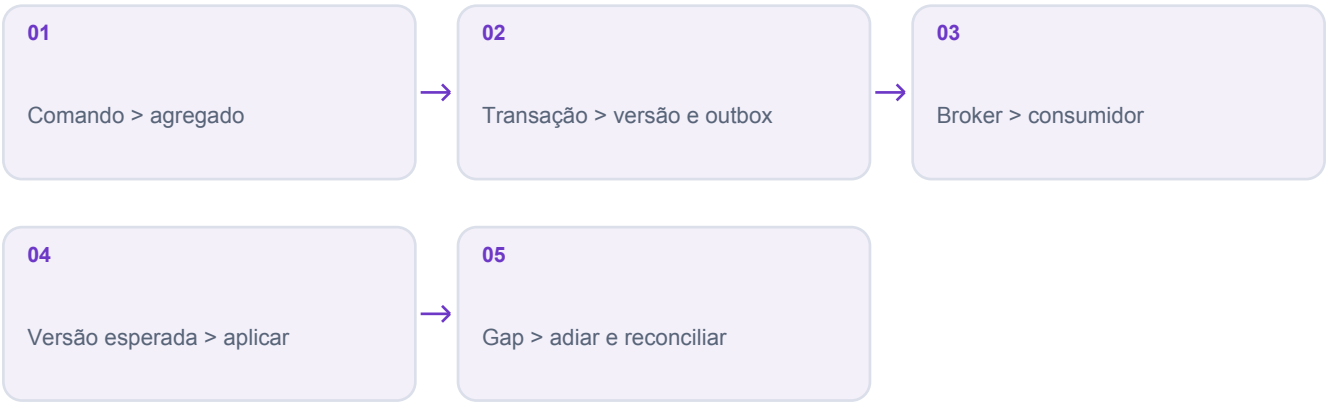
Quando evitar ou limitar. Não impor ordem global quando somente entidades individuais possuem dependências.

Uma abertura possível

Vou identificar qual operação depende de qual anterior; essa relação define a unidade de sequência.

Peças e tecnologias. Kafka · SQS FIFO · PostgreSQL · Service Bus Sessions

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Definir agregado e invariante

2. Gerar versão na mesma transação do estado

3. Publicar com identidade e sequência

4. Persistir gaps sem marcar efeito concluído

5. Recuperar lacunas e testar concorrência

30 perguntas para sustentar o desenho

01. Qual ordem precisa ser preservada?

Eu preservo a ordem exigida pela invariante, normalmente por agregado. Não imponho ordem global para contas independentes, pois isso serializaria trabalho sem melhorar a correção do domínio.

Termo para lembrar: Ordem por agregado | Base técnica: referência 1.

02. Timestamp define a sequência correta?

Não necessariamente. Relógios podem divergir e eventos simultâneos empatar. Uso uma versão monotônica gerada na fronteira autoritativa; timestamp continua útil para auditoria, mas não decide sozinho precedência financeira.

Termo para lembrar: Versão monotônica | Base técnica: referência 1.

03. Como gera a versão sem corrida?

Incremento a versão com atualização atômica ou lock na transação do agregado. O evento recebe essa versão na mesma gravação, evitando publicar uma posição que não corresponde ao estado confirmado.

Termo para lembrar: Controle otimista | Base técnica: referência 2.

04. E se dois produtores alteram a mesma conta?

Eles precisam compartilhar uma autoridade de concorrência ou protocolo que preserve a invariante. Apenas escolher a mesma chave Kafka não decide qual alteração concorrente de negócio é válida.

Termo para lembrar: Single writer lógico | Base técnica: referência 1.

05. Uma partição garante ordem entre produtores?

Ela registra uma ordem de chegada no log, mas essa ordem pode divergir da sequência de commits no banco. Preservo versão e controlo publicação ou corrijo lacunas no consumidor.

Termo para lembrar: Ordem de publicação | Base técnica: referência 1.

06. Como dois relays podem inverter eventos?

Um relay pode atrasar E11 enquanto outro publica E12. Evito claim indiscriminado dentro do mesmo agregado ou faço consumidor reconhecer versões. A outbox sozinha não garante ordem entre publishers concorrentes.

Termo para lembrar: Relay concorrente | Base técnica: referência 3.

07. O que faz ao receber versão 12 esperando 11?

Persisto o evento como adiado e procuro a lacuna. Não atualizo a versão para 12 se o efeito depender de 11; isso mascararia perda de atualização necessária.

Termo para lembrar: Gap detection | Base técnica: referência 1.

08. Pode confirmar evento adiado ao broker?

Sim, depois de persistir payload e obrigação durável de retomada. O ACK significa responsabilidade transferida para meu armazenamento, não efeito concluído; mantenho scheduler, prazo e recuperação dessas pendências.

Termo para lembrar: Deferred processing | Base técnica: referência 3.

09. Como evita adiar para sempre?

Registro idade da lacuna, tentativas e prazo de reconciliação. Busco na origem ou solicito snapshot consistente quando permitido; se faltar evento indispensável, escalo o agregado afetado sem inventar continuidade.

Termo para lembrar: Reconciliação de gap | Base técnica: referência 1.

10. Qual evento antigo pode ser descartado?

Um snapshot completo pode ser substituído por versão mais recente se não houver efeitos adicionais. Um delta não pode ser descartado por idade apenas; preciso comprovar que já foi aplicado.

Termo para lembrar: Snapshot versus delta | Base técnica: referência 1.

11. E se o evento antigo gera notificação?

Mesmo que o estado projetado já esteja atualizado, a obrigação de notificar pode ser distinta. Separo materialização de side effects e defino política explícita para notificações atrasadas ou obsoletas.

Termo para lembrar: Side effect | Base técnica: referência 3.

12. Como detecta duplicatas na sequência?

Uso identidade do evento e versão do agregado. A mesma versão com conteúdo diferente indica conflito, não repetição válida; registro divergência e evito escolher arbitrariamente qual payload representa a verdade.

Termo para lembrar: Conflito de versão | Base técnica: referência 2.

13. Por que usar ETag em atualização?

ETag pode carregar a versão observada e impedir lost update por precondição. O servidor valida a versão autoritativa antes de alterar; o navegador não decide sozinho se a transição é válida.

Termo para lembrar: Precondição | Base técnica: referência 2.

14. Sequência da conta impede transferência entre contas?

Ela organiza cada lado, mas não resolve atomicidade entre agregados. A transferência exige fronteira transacional adequada ou protocolo explícito de reserva e compensação; duas filas não criam uma transação financeira.

Termo para lembrar: Atomicidade entre agregados | Base técnica: referência 2.

15. Como escalar uma conta quente?

Primeiro meço contenção e separo leitura de escrita. Dividir o agregado só é válido se as invariantes puderem ser particionadas; sharding artificial de saldo pode permitir gasto concorrente indevido.

Termo para lembrar: Hot aggregate | Base técnica: referência 1.

16. Como o consumidor aplica evento atomicamente?

Na mesma transação verifico versão esperada, altero a projeção e avanço o checkpoint. Se houver obrigação externa, registro outbox; não avanço progresso antes de tornar todo trabalho recuperável.

Termo para lembrar: Checkpoint atômico | Base técnica: referência 2.

17. Pode processar E12 e E13 em paralelo?

Somente se seus efeitos forem independentes ou comutativos sob o contrato. Para transições dependentes, serializo aplicação por agregado, ainda permitindo paralelismo entre agregados diferentes e trabalho preparatório sem efeito.

Termo para lembrar: Comutatividade | Base técnica: referência 1.

18. Como trata exclusão seguida de recriação?

Uso identidade ou epoch que distinga a nova encarnação. Reutilizar a mesma chave com contador reiniciado pode fazer evento antigo parecer atual e ressuscitar estado de uma entidade anterior.

Termo para lembrar: Epoch | Base técnica: referência 2.

19. Como faz replay sem conflito com o vivo?

Reconstruo uma projeção separada até um ponto conhecido, alcanço o fluxo atual e troco leitura após validação. Evito dois escritores independentes aplicando versões sobre o mesmo destino sem coordenação.

Termo para lembrar: Rebuild | Base técnica: referência 1.

20. Qual risco de aumentar partições?

O roteamento por hash pode mover chaves e permitir coexistência de eventos em partições diferentes. Planejo corte ou versão de roteamento; sequência do domínio permanece necessária durante a migração.

Termo para lembrar: Repartitioning | Base técnica: referência 1.

21. FIFO elimina lacunas do domínio?

Não. FIFO ordena entregas dentro do escopo do broker, mas não cria eventos que o produtor deixou de publicar. Continuo verificando sequência e obrigação durável na origem.

Termo para lembrar: FIFO | Base técnica: referência 1.

22. O que muda ao mandar um evento para DLQ?

Se os seguintes dependem dele, removê-lo não libera semanticamente o agregado. Preservo bloqueio ou estado de investigação, enquanto entidades independentes continuam progredindo sem esperar aquele erro específico.

Termo para lembrar: Poison event | Base técnica: referência 1.

23. Como escolhe tamanho do buffer de gaps?

Limito por agregado, bytes e tempo. Buffer ilimitado permite exaustão por falha ou abuso; ao atingir limite, reduzo consumo daquela classe e inicio recuperação controlada da sequência.

Termo para lembrar: Bounded buffer | Base técnica: referência 1.

24. Versão pode ser global no banco?

Pode, mas um contador global pode gerar contenção e não representa necessariamente dependência entre entidades. Prefiro sequência local quando o contrato só exige ordem por agregado.

Termo para lembrar: Sequência local | Base técnica: referência 2.

25. Como audita mudança de ordem?

Registro event_id, aggregate_id, versão, instante de origem e de aplicação. Esses campos mostram atraso e inversões sem depender de relógios perfeitamente sincronizados para reconstruir a sequência lógica.

Termo para lembrar: Auditoria causal | Base técnica: referência 1.

26. Como trata rollback de uma versão?

Uma transação abortada não publica alteração confirmada. Se uso sequências que deixam buracos por design, não interpreto cada número ausente como evento perdido; o contrato deve distinguir contador técnico de sequência contígua.

Termo para lembrar: Sequência contígua | Base técnica: referência 2.

27. Como reconcilia snapshot com deltas pendentes?

O snapshot precisa declarar versão de corte. Descarto apenas deltas comprovadamente cobertos e aplico os posteriores na ordem exigida; sem esse corte, posso somar alterações duas vezes.

Termo para lembrar: Snapshot versionado | Base técnica: referência 1.

28. O cliente precisa enxergar toda sequência?

Não. Pode receber o estado mais recente e um marcador de versão, desde que estados intermediários não sejam necessários ao contrato. Para auditoria e execução, preservo histórico adequado separadamente.

Termo para lembrar: Coalescing | Base técnica: referência 1.

29. Que teste demonstra a correção?

Embaralho, duplico e atraso eventos, incluindo exclusão e recriação. Verifico a invariante e a convergência esperada; testar somente a ordem feliz do broker deixa justamente as lacunas mais perigosas.

Termo para lembrar: Property testing | Base técnica: referência 2.

30. Como explicaria a escolha ao entrevistador?

Eu diria que ordem é requisito do domínio, não atributo mágico da fila. Defino unidade, autoridade de versão e recuperação de gaps antes de selecionar Kafka, FIFO ou outra implementação.

Termo para lembrar: Contrato de ordem | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Ordem global

Ganho. Raciocínio simples

Custo e motivo. Serialização ampla

Como decidir. Somente se a invariante realmente exigir

02. Ordem por agregado

Ganho. Paralelismo entre entidades

Custo e motivo. Agregado quente

Como decidir. Para estados independentes

03. Sequência contígua

Ganho. Detecta lacunas

Custo e motivo. Exige geração coordenada

Como decidir. Para deltas dependentes

04. Timestamp

Ganho. Útil para auditoria

Custo e motivo. Não prova precedência

Como decidir. Quando horário é informativo

05. Snapshot completo

Ganho. Pode coalescer versões

Custo e motivo. Payload maior

Como decidir. Para projeções de estado

06. Delta

Ganho. Mensagem pequena

Custo e motivo. Depende de histórico

Como decidir. Quando mudanças são incrementais

07. Buffer durável

Ganho. Libera transporte

Custo e motivo. Scheduler adicional

Como decidir. Para gaps recuperáveis

08. Bloquear agregado

Ganho. Preserva dependência

Custo e motivo. Atrasa usuário afetado

Como decidir. Quando pular evento corrompe estado

09. Rebuild separado

Ganho. Isola replay

Custo e motivo. Armazenamento extra

Como decidir. Para mudar projeções

10. Single writer lógico

Ganho. Simplifica concorrência

Custo e motivo. Limite por chave

Como decidir. Para invariantes seriadas

Duas alternativas ao caminho proposto

1. Operações comutativas

Quando faz sentido. Domínio permite fusão independente

Como implementar. Modelar CRDT ou conjunto de fatos com identidade

Limites. Não serve automaticamente para saldo disponível

2. Consulta autoritativa

Quando faz sentido. Eventos apenas invalidam leitura

Como implementar. Ao receber aviso buscar snapshot versionado

Limites. Mais carga e perda de detalhes intermediários

Referências e alcance da evidência

1. Apache Kafka - Design

Log particionado, offsets, replicação e fronteiras de garantias transacionais.

https://kafka.apache.org/41/design/design/

2. PostgreSQL - Transaction isolation

Snapshots, concorrência, isolamento e falhas de serialização.

https://www.postgresql.org/docs/current/transaction-iso.html

3. AWS - Transactional outbox

Gravação atômica de estado e obrigação de publicar; duplicatas ainda possíveis.

https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/transactional-outbox.html

Continue em: T018 · T019 · T041 · T063

T044 / E. Mensageria e processamento

At-most-once, at-least-once e exactly-once

Semânticas de entrega descrevem fronteiras; efeito único exige proteção onde o efeito acontece.

Quando usar. Escolher confirmação, replay e deduplicação para operações importantes.

Quando evitar ou limitar. Não prometer exactly-once global a partir de uma configuração isolada do broker.

Uma abertura possível

Vou desenhar onde o efeito fica durável e onde o progresso é confirmado; a falha entre esses pontos define repetição ou perda.

Peças e tecnologias. Kafka · SQS · RabbitMQ · PostgreSQL

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Listar fronteiras de durabilidade

2. Definir identidade de intenção e evento

3. Escolher ordem efeito e confirmação

4. Adicionar atomicidade ou deduplicação

5. Injetar falhas em cada intervalo

30 perguntas para sustentar o desenho

01. O que significa at-most-once?

Significa que uma entrega pode não ocorrer e não será repetida naquele escopo. Se confirmo antes de processar, uma queda pode perder trabalho; uso apenas quando essa perda é aceitável.

Termo para lembrar: At-most-once | Base técnica: referência 1.

02. O que significa at-least-once?

Admito repetição para recuperar falhas de entrega ou confirmação. O consumidor precisa reconhecer efeitos já aplicados; a semântica do transporte não garante que o resultado financeiro esteja correto automaticamente.

Termo para lembrar: At-least-once | Base técnica: referência 2.

03. Exactly-once é impossível em qualquer sistema?

Não. Existem garantias delimitadas, como transações sobre registros e offsets dentro do Kafka. O erro é estender essa garantia a bancos ou APIs externas que não participam do mesmo mecanismo.

Termo para lembrar: Exactly-once scope | Base técnica: referência 1.

04. Onde nasce a duplicata mais comum?

O consumidor executa o efeito e cai antes do ACK. O broker reentrega porque não sabe da conclusão; uma identidade estável permite distinguir retentativa de nova intenção legítima.

Termo para lembrar: ACK gap | Base técnica: referência 3.

05. Como evitar perda no caminho inverso?

Não avanço o progresso antes de persistir o trabalho. Se preciso confirmar cedo, gravo primeiro uma obrigação durável e recuperável; uma variável em memória não substitui essa responsabilidade.

Termo para lembrar: Durable handoff | Base técnica: referência 3.

06. Uma transação de banco resolve tudo?

Resolve alterações dentro de sua fronteira. Não inclui automaticamente e-mail, SFTP ou ledger remoto. Eu registro obrigações locais e uso protocolos de idempotência ou reconciliação nas integrações externas.

Termo para lembrar: Fronteira transacional | Base técnica: referência 4.

07. Qual garantia oferece o produtor idempotente?

Ele protege determinadas repetições de publicação no Kafka. Não interpreta duas solicitações de crédito como a mesma intenção de negócio nem impede o consumidor de repetir uma chamada HTTP externa.

Termo para lembrar: Producer idempotence | Base técnica: referência 1.

08. SQS FIFO dispensa consumidor idempotente?

Não para efeitos externos. Mesmo com recursos de deduplicação do broker, uma execução pode cair após produzir efeito e antes de confirmar. Preservo identidade no destino responsável pela alteração.

Termo para lembrar: FIFO boundary | Base técnica: referência 2.

09. Mensagem única equivale a efeito único?

Não. Um único evento pode ser processado várias vezes por falhas; vários eventos podem representar a mesma intenção. Modelo event_id e operation_id conforme as duas fronteiras diferentes.

Termo para lembrar: Identidade semântica | Base técnica: referência 4.

10. Como tratar duas chaves diferentes para o mesmo crédito?

A deduplicação técnica não percebe essa igualdade. A regra de negócio precisa de uma chave ou constraint correspondente ao crédito autorizado, sem bloquear operações parciais ou correções legítimas.

Termo para lembrar: Business uniqueness | Base técnica: referência 4.

11. O que é effectively-once?

É uma descrição prática de efeitos observáveis protegidos contra repetição, normalmente sobre entrega at-least-once. Eu explico exatamente quais efeitos, identidades e janelas estão cobertos, evitando usar o termo como promessa universal.

Termo para lembrar: Effectively-once | Base técnica: referência 4.

12. Redis SETNX basta para deduplicar débito?

Não se o débito está em outro banco. Há falhas entre marcar Redis e debitar, além de expiração. Preciso ligar a proteção ao efeito autoritativo por transação ou contrato equivalente.

Termo para lembrar: Atomicidade | Base técnica: referência 4.

13. Como confirmar offsets e saídas Kafka juntos?

Uso transação Kafka quando o processamento permanece nesse ecossistema e configuro consumo compatível. Se também altero PostgreSQL, preciso projetar essa fronteira externa; a transação Kafka não a incorpora.

Termo para lembrar: Kafka transaction | Base técnica: referência 1.

14. Por que read_committed importa?

Ele evita expor ao consumidor registros de transações Kafka abortadas. A configuração protege aquela leitura; não certifica que etapas posteriores ocorrerão uma única vez ou serão revertidas conjuntamente.

Termo para lembrar: Isolation level | Base técnica: referência 1.

15. Como e-mail pode ficar duplicado?

O provedor pode aceitar o envio e a resposta se perder. Sem idempotência ou consulta inequívoca no provedor, não consigo garantir ausência absoluta de duplicatas; defino política explícita para essa incerteza.

Termo para lembrar: External side effect | Base técnica: referência 4.

16. Como SFTP limita a garantia?

Uma conexão pode cair depois de parte ou todo o arquivo chegar. Preservo artefato e identidade, verifico recibos previstos e não interpreto ausência de resposta como prova de não recebimento.

Termo para lembrar: UNKNOWN | Base técnica: referência 4.

17. O que deve acontecer após timeout de commit?

O commit pode ter ocorrido mesmo sem resposta ao cliente. Consulto pela identidade da operação antes de decidir nova execução; não converto falha de comunicação em evidência de rollback.

Termo para lembrar: Commit ambíguo | Base técnica: referência 4.

18. Retenção muda a semântica?

Sim. Se o registro de deduplicação expirar e a mensagem retornar, ela pode parecer nova. Alinho janela de replay e retenção ou mantenho unicidade duradoura no próprio efeito financeiro.

Termo para lembrar: Deduplication window | Base técnica: referência 4.

19. Como restauração de backup reabre duplicatas?

O backup pode apagar a memória local de operações já executadas externamente. Reconcilio antes do replay e preservo IDs no destino; recuperar dados antigos não desfaz efeitos do mundo externo.

Termo para lembrar: Disaster recovery | Base técnica: referência 4.

20. Como testa at-most-once?

Provoco falha após avanço do progresso e antes do efeito para observar a perda esperada. A decisão só é aceitável se o domínio tolerar essa perda e houver métrica correspondente.

Termo para lembrar: Fault injection | Base técnica: referência 3.

21. Como testa at-least-once?

Provoco queda depois do efeito e antes da confirmação. Espero reentrega e verifico que a proteção impede efeito duplicado; logs de tentativa devem mostrar repetição mesmo quando o resultado permanece único.

Termo para lembrar: Redelivery test | Base técnica: referência 3.

22. Como prova exatamente o efeito protegido?

Defino uma invariante verificável, como um lançamento por operation_id, e confronto todas as tentativas com o estado autoritativo. Contar mensagens no broker não demonstra unicidade financeira no destino.

Termo para lembrar: Invariante | Base técnica: referência 4.

23. Qual risco de compensar automaticamente uma incerteza?

Se a operação original ainda pode concluir, a compensação pode correr contra ela. Primeiro estabeleço resultado ou protocolo de cancelamento; compensar não transforma uma resposta desconhecida em rejeição comprovada.

Termo para lembrar: Compensação | Base técnica: referência 4.

24. O cliente pode repetir com novo ID após timeout?

Pode criar uma nova intenção sem querer. O cliente deve preservar a chave original e consultar status; o servidor também protege unicidade de negócio quando o risco justificar.

Termo para lembrar: Idempotency key | Base técnica: referência 4.

25. Como trata payload diferente com mesma chave?

Recuso como conflito de intenção e registro evidência. Reutilizar a resposta anterior silenciosamente pode esconder um valor diferente; sobrescrever o registro destruiria justamente a proteção contra execução ambígua.

Termo para lembrar: Payload hash | Base técnica: referência 4.

26. Todo evento exige deduplicação permanente?

Não. Telemetria aproximada pode aceitar contagens duplicadas; crédito financeiro costuma exigir proteção mais forte. Classifico efeitos e retenção pelo impacto, sem impor custo máximo a todos os fluxos.

Termo para lembrar: Risk-based guarantee | Base técnica: referência 2.

27. Como respostas idempotentes lidam com autorização revogada?

Verifico autorização atual antes de revelar o resultado armazenado. A chave identifica intenção, não concede acesso; replay não deve virar um caminho para consultar dados depois da revogação.

Termo para lembrar: Authorization on replay | Base técnica: referência 4.

28. Uma fila regional garante durabilidade global?

Não automaticamente. Analiso a estratégia de recuperação e o RPO regional. Se a confirmação ao cliente exige sobreviver à perda regional sem dados, preciso de durabilidade apropriada antes do ACK.

Termo para lembrar: RPO | Base técnica: referência 2.

29. Como evita conflito entre retries de várias camadas?

Escolho uma camada responsável e orçamento total de tentativas. Preservo identidade em todos os saltos; retries multiplicados aumentam carga e quantidade de janelas ambíguas sem fortalecer a garantia.

Termo para lembrar: Retry amplification | Base técnica: referência 4.

30. Qual conclusão você apresentaria?

Eu especificaria entrega, efeito local e efeito remoto separadamente. Para cada um, mostraria identidade, ponto de commit, confirmação e recuperação; assim a garantia pode ser testada em vez de presumida.

Termo para lembrar: End-to-end semantics | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. At-most-once

Ganho. Menos repetição

Custo e motivo. Pode perder trabalho

Como decidir. Para telemetria tolerante

02. At-least-once

Ganho. Recupera confirmação perdida

Custo e motivo. Exige idempotência

Como decidir. Para jobs importantes

03. Transação Kafka

Ganho. Coordena log e offsets

Custo e motivo. Escopo limitado

Como decidir. Para processamento dentro do Kafka

04. Constraint de negócio

Ganho. Proteção duradoura

Custo e motivo. Modelagem específica

Como decidir. Para efeitos financeiros

05. Deduplicação temporária

Ganho. Menos armazenamento

Custo e motivo. Replay tardio reabre risco

Como decidir. Quando janela é contratual

06. Resultado armazenado

Ganho. Retry previsível

Custo e motivo. Retenção e privacidade

Como decidir. Para APIs de comando

07. Reconciliação

Ganho. Resolve desconhecidos

Custo e motivo. Latência e operação

Como decidir. Quando destino pode ter executado

08. 2PC compatível

Ganho. Atomicidade entre participantes

Custo e motivo. Acoplamento e bloqueio

Como decidir. Quando participantes e requisitos justificam

09. Compensação

Ganho. Recuperação de negócio

Custo e motivo. Não apaga histórico

Como decidir. Para etapas reversíveis semanticamente

10. Identidade por intenção

Ganho. Distingue retries

Custo e motivo. Cliente precisa preservar chave

Como decidir. Quando há efeitos externos

Duas alternativas ao caminho proposto

1. Operação naturalmente idempotente

Quando faz sentido. Estado absoluto sem efeitos adicionais

Como implementar. Aplicar versão e precondições no destino

Limites. Atualizar saldo absoluto não serve para qualquer lançamento

2. Reconciliação orientada a estado

Quando faz sentido. Destino não oferece deduplicação

Como implementar. Consultar resultado e comparar evidências antes de agir

Limites. Pode exigir intervenção e não garante entrega única

Referências e alcance da evidência

1. Apache Kafka - Design

Log particionado, offsets, replicação e fronteiras de garantias transacionais.

https://kafka.apache.org/41/design/design/

2. AWS - SQS overview

Filas gerenciadas, modalidades e desacoplamento; efeitos externos exigem garantias próprias.

https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/welcome.html

3. RabbitMQ - confirms e acknowledgements

Confirmação ao produtor, ACK do consumidor, prefetch e reentrega.

https://www.rabbitmq.com/docs/confirms

4. AWS Builders Library - Idempotent APIs

Identidades estáveis, intenção, retries e requisições atrasadas.

https://aws.amazon.com/builders-library/making-retries-safe-with-idempotent-APIs/

Continue em: T061 · T062 · T063 · T064

T045 / E. Mensageria e processamento

Backpressure e admission control

Backpressure reduz produção ou consumo quando o próximo estágio não acompanha; admission control limita trabalho admitido.

Quando usar. Picos, dependências lentas, consumidores atrasados e limites externos.

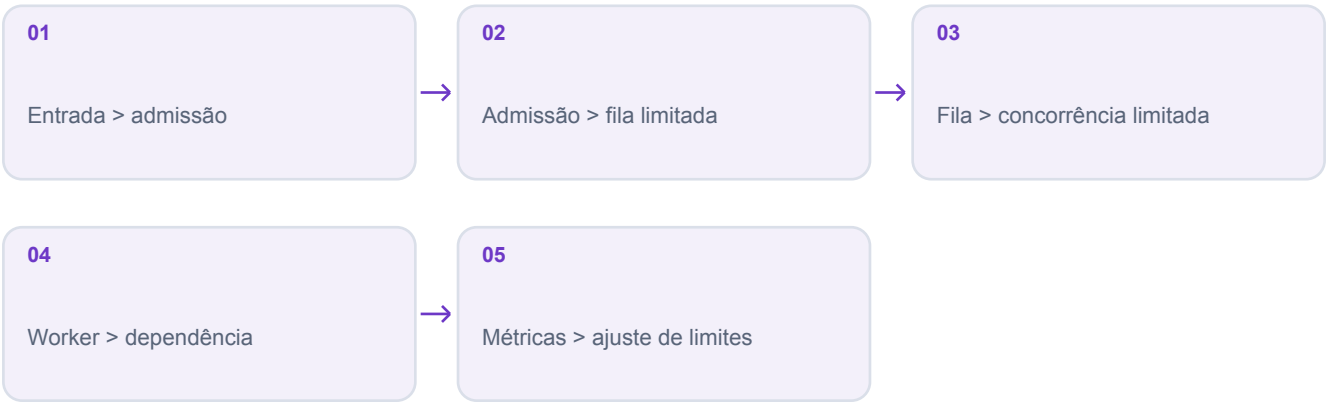
Quando evitar ou limitar. Não usar filas ilimitadas como substituto de capacidade ou prometer conclusão além do prazo possível.

Uma abertura possível

Vou dimensionar a taxa sustentável e recusar ou adiar trabalho antes de esgotar memória, conexões e prazos.

Peças e tecnologias. Envoy · SQS · Kafka · Resilience4j · Token bucket

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Medir vazão sustentável e custo por classe

2. Limitar filas e concorrência por recurso

3. Definir resposta de recusa ou aceitação durável

4. Propagar deadlines e orçamento de retries

5. Testar carga além da capacidade e recuperação

30 perguntas para sustentar o desenho

01. Qual diferença entre rate limit e backpressure?

Rate limiting controla uma taxa definida; backpressure responde à capacidade do caminho. Posso respeitar requisições por segundo e ainda saturar conexões se cada chamada ficar muito mais lenta.

Termo para lembrar: Backpressure | Base técnica: referência 1.

02. Como calcula trabalho simultâneo?

Uso a relação entre taxa e tempo no sistema como estimativa. Se entram 200 operações/s e cada uma ocupa recurso por dois segundos, espero aproximadamente 400 operações concorrentes em regime estável.

Termo para lembrar: Little's Law | Base técnica: referência 1.

03. Uma fila resolve sobrecarga sustentada?

Não. Se a chegada excede a conclusão continuamente, o backlog cresce até violar prazo ou retenção. A fila absorve variação temporária; preciso reduzir entrada, aumentar capacidade real ou mudar custo.

Termo para lembrar: Queue growth | Base técnica: referência 1.

04. Como dimensiona limite da fila?

Escolho tamanho considerando tempo máximo de espera, memória e vazão de drenagem. Um limite por quantidade pode ser insuficiente para payloads variados; também controlo bytes e idade do trabalho.

Termo para lembrar: Bounded queue | Base técnica: referência 1.

05. Quando devolve 429 ou 503?

Uso 429 para limite associado ao cliente conforme contrato e 503 para indisponibilidade temporária do serviço. Explico política de retry e evito orientar clientes a repetir imediatamente em sincronia.

Termo para lembrar: Admission control | Base técnica: referência 1.

06. O que acontece com uma operação já aceita?

Se confirmei aceitação durável, preservo a obrigação e informo pendência. Não descarto silenciosamente o job para aliviar carga; redução de trabalho precisa respeitar o contrato estabelecido com o cliente.

Termo para lembrar: Durable acceptance | Base técnica: referência 2.

07. Por que limitar concorrência além da taxa?

Uma dependência lenta aumenta operações simultâneas mesmo com taxa constante. O semáforo de concorrência protege sockets, threads e pool de banco; a taxa sozinha não acompanha essa mudança de duração.

Termo para lembrar: Concurrency limit | Base técnica: referência 1.

08. Qual risco de fila dentro do pool de conexões?

Ela consome deadline antes de executar a consulta e pode concentrar requisições vencidas. Limito espera pelo pool e cancelo trabalho sem utilidade, em vez de deixar filas implícitas sem observação.

Termo para lembrar: Connection pool | Base técnica: referência 1.

09. Como evita que retries derrubem o parceiro?

Defino orçamento de tentativas e uso backoff com jitter. Retries competem com novas operações por capacidade; reservo limite e não permito que cada camada multiplique tentativas independentemente.

Termo para lembrar: Retry budget | Base técnica: referência 3.

10. Autoscaling elimina admission control?

Não. Escalar demora e pode saturar dependências compartilhadas. Mantenho limites durante o crescimento e verifico se banco, quotas e parceiros suportam a nova concorrência antes de aumentar workers.

Termo para lembrar: Autoscaling lag | Base técnica: referência 1.

11. Como limita um consumidor Kafka?

Reduzo trabalho em voo e pauso partições quando necessário, preservando polling e atribuição conforme cliente. Controlo progresso confirmado; pausar ingestão não autoriza avançar offsets de tarefas incompletas.

Termo para lembrar: Consumer pause | Base técnica: referência 2.

12. Como limita consumidores SQS?

Controlo workers, prefetch lógico e chamadas simultâneas ao destino. Visibilidade acompanha duração real; receber milhares de mensagens para deixá-las paradas aumenta risco de expiração e reentrega sem benefício.

Termo para lembrar: In-flight limit | Base técnica: referência 2.

13. Qual política atende clientes pequenos e grandes?

Uso quotas e escalonamento ponderado por classe ou tenant. Um cliente ruidoso não deve consumir toda capacidade; a política precisa considerar custo por operação, não apenas número de requisições.

Termo para lembrar: Fairness | Base técnica: referência 1.

14. Prioridade absoluta é perigosa?

Sim, pode causar starvation. Combino urgência com aging e reserva mínima para classes comuns. Meço idade da solicitação mais antiga para detectar abandono que médias de latência esconderiam.

Termo para lembrar: Aging | Base técnica: referência 1.

15. Como degrada uma tela de atendimento?

Preservo contexto essencial e removo enriquecimentos caros. Cada widget informa indisponibilidade ou dado antigo; não substituo saldo desconhecido por zero nem habilito ação financeira baseada em fallback inseguro.

Termo para lembrar: Graceful degradation | Base técnica: referência 1.

16. Como funciona um token bucket?

Acumulo tokens até uma capacidade máxima e consumo conforme custo da chamada. Isso permite rajadas limitadas; calibro reposição e tamanho do balde conforme capacidade sustentável e contrato do parceiro.

Termo para lembrar: Token bucket | Base técnica: referência 1.

17. Leaky bucket serve para quê?

Ele suaviza a saída aproximando uma vazão constante. Pode aumentar espera e exige buffer limitado; uso quando o parceiro tolera pouca rajada, preservando deadlines e prioridade do trabalho pendente.

Termo para lembrar: Traffic shaping | Base técnica: referência 1.

18. Como dividir quota global entre instâncias?

Posso centralizar concessão ou distribuir cotas conservadoras com realocação. Contadores locais simples multiplicam o limite pelo número de réplicas; preciso escolher precisão, disponibilidade e custo de coordenação.

Termo para lembrar: Distributed quota | Base técnica: referência 1.

19. E se o serviço de limite cair?

Defino fail-open ou fail-closed por operação. Para proteger quota financeira rígida, falhar aberto pode ser inadmissível; para leitura barata, um limite local conservador pode preservar disponibilidade com risco controlado.

Termo para lembrar: Fail mode | Base técnica: referência 1.

20. Deadline vencido ainda deve executar?

Depende do contrato. Leitura cancelada pode ser abandonada; comando já aceito duravelmente continua como operação assíncrona rastreável. Não confundo o prazo da conexão HTTP com cancelamento do efeito financeiro.

Termo para lembrar: Deadline propagation | Base técnica: referência 3.

21. Como calcula tempo de drenagem?

Divido backlog pela capacidade líquida positiva: conclusão menos chegada. Se chegam 80/s, concluo 100/s e existem 72 mil tarefas, preciso de aproximadamente uma hora para drenar nesse regime.

Termo para lembrar: Drain time | Base técnica: referência 1.

22. Por que CPU baixa não prova folga?

O serviço pode estar bloqueado em rede, disco, locks ou pool de conexões. Observo recursos ocupados e latência por estágio; apenas CPU pode incentivar aumentar carga num sistema já saturado.

Termo para lembrar: Saturation | Base técnica: referência 1.

23. Como evita trabalho caro antes da admissão?

Faço validações baratas e identificação de classe cedo, antes de desserialização pesada ou fan-out. Ainda verifico autenticação e autorização adequadas; um atacante não deve obter processamento ilimitado por dados falsos.

Termo para lembrar: Early rejection | Base técnica: referência 1.

24. Como lida com payload de custo variável?

Uso pesos estimados por tamanho, quantidade de itens ou operação solicitada. Reavalio estimativas com métricas; limitar apenas requisições deixa um único batch enorme consumir a mesma cota de uma leitura simples.

Termo para lembrar: Weighted admission | Base técnica: referência 1.

25. Qual relação com circuit breaker?

O breaker interrompe chamadas a uma dependência falhando; admission control limita trabalho mesmo quando ela está saudável porém perto da capacidade. Uso ambos com responsabilidades e métricas diferentes.

Termo para lembrar: Circuit breaker | Base técnica: referência 3.

26. Como evita oscilação de limites adaptativos?

Uso janelas, suavização e margens de segurança, testando estabilidade sob carga variável. Ajustar agressivamente por uma amostra pode alternar entre sobrecarga e ociosidade, piorando a experiência do usuário.

Termo para lembrar: Feedback control | Base técnica: referência 1.

27. Como testa recuperação após pico?

Aumento carga além da capacidade, observo recusa previsível e retiro o excesso. Verifico se backlog drena, memória cai e p99 recupera sem tempestade de retries ou reinícios contínuos.

Termo para lembrar: Load test | Base técnica: referência 1.

28. Qual risco de aceitar para processar depois?

A resposta rápida pode esconder uma fila impossível de cumprir. Publico prazo e status, monitoro idade e limito aceitação pela capacidade prevista; 202 não deve maquiar indisponibilidade de negócio.

Termo para lembrar: Async admission | Base técnica: referência 2.

29. Pode descartar eventos de telemetria?

Se o contrato permitir, amostro ou descarto classes menos importantes para preservar operações essenciais. Registro perdas e mantenho auditoria obrigatória separada; não aplico essa política a eventos financeiros aceitos.

Termo para lembrar: Load shedding | Base técnica: referência 1.

30. Como defenderia a solução?

Eu mostraria capacidade por estágio, limite de trabalho em voo e comportamento ao excedê-lo. A resiliência vem de proteger recursos e compromissos, não de empilhar filas sem prazo ou orçamento.

Termo para lembrar: Capacity envelope | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Fila maior

Ganho. Absorve rajadas

Custo e motivo. Espera e memória

Como decidir. Para picos curtos conhecidos

02. Fila pequena

Ganho. Latência previsível

Custo e motivo. Mais rejeições

Como decidir. Quando prazo é estrito

03. Limite global

Ganho. Quota precisa

Custo e motivo. Coordenação remota

Como decidir. Para contratos rígidos

04. Limite local

Ganho. Baixa latência

Custo e motivo. Pode exceder soma global

Como decidir. Para proteção aproximada

05. Prioridade

Ganho. Protege urgentes

Custo e motivo. Pode causar starvation

Como decidir. Com aging e cotas

06. Degradação

Ganho. Preserva função essencial

Custo e motivo. Menos informação

Como decidir. Quando fallback é semanticamente seguro

07. Autoscaling

Ganho. Aumenta capacidade

Custo e motivo. Atraso e custo

Como decidir. Quando gargalo escala horizontalmente

08. Retry budget

Ganho. Controla amplificação

Custo e motivo. Algumas falhas não repetem logo

Como decidir. Durante incidentes

09. Admissão ponderada

Ganho. Reflete custo real

Custo e motivo. Estimativa complexa

Como decidir. Para operações heterogêneas

10. Aceitação assíncrona

Ganho. Libera conexão

Custo e motivo. Exige prazo e acompanhamento

Como decidir. Quando trabalho é durável e adiável

Duas alternativas ao caminho proposto

1. Processamento síncrono com recusa

Quando faz sentido. Sem necessidade de buffering

Como implementar. Concorrência limitada e resposta de indisponibilidade

Limites. Cliente precisa lidar com retry seguro

2. Provisionamento para pico conhecido

Quando faz sentido. Carga previsível e orçamento disponível

Como implementar. Reservar capacidade e testar margem

Limites. Picos inesperados ainda exigem limites

Referências e alcance da evidência

1. Google SRE - Handling Overload

Admissão de carga, degradação e prevenção de sobrecarga em cascata.

https://sre.google/sre-book/handling-overload/

2. AWS - SQS overview

Filas gerenciadas, modalidades e desacoplamento; efeitos externos exigem garantias próprias.

https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/welcome.html

3. AWS - Timeouts retries and jitter

Timeouts, backoff, jitter e risco de repetir efeitos após respostas perdidas.

https://aws.amazon.com/builders-library/timeouts-retries-and-backoff-with-jitter/

Continue em: T003 · T055 · T064 · T066

T046 / E. Mensageria e processamento

DLQ, quarentena e redrive

DLQ e quarentena isolam trabalho que não progride; redrive devolve trabalho após correção controlada.

Quando usar. Mensagens inválidas, falhas persistentes e recuperação rastreável.

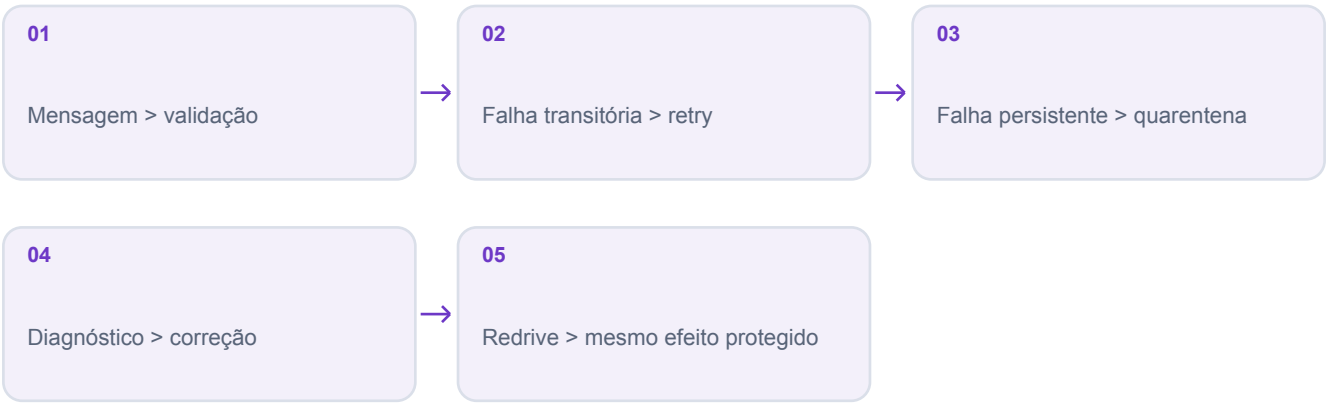
Quando evitar ou limitar. Não usar DLQ como arquivo morto sem responsável, prazo e política de retenção.

Uma abertura possível

Vou separar falha transitória, mensagem inválida e rejeição de negócio antes de definir retries e quarentena.

Peças e tecnologias. SQS DLQ · RabbitMQ DLX · Kafka quarantine topic · Service Bus DLQ

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Classificar erros e definir limite de tentativas

2. Persistir contexto seguro e identidade

3. Alertar por idade e impacto no negócio

4. Corrigir causa antes de redrive limitado

5. Validar ordem, deduplicação e retenção

30 perguntas para sustentar o desenho

01. Qual problema a DLQ resolve?

Ela retira mensagens que não progridem do caminho principal e preserva evidência para recuperação. Não corrige o defeito nem conclui o negócio; exige responsável, diagnóstico e procedimento de redrive.

Termo para lembrar: Dead-letter queue | Base técnica: referência 1.

02. Quais erros não merecem retry imediato?

Schema inválido, versão incompatível ou requisito impossível não melhoram com repetição instantânea. Distingo esses erros de timeout transitório e evito consumir capacidade indefinidamente com uma mensagem determinadamente defeituosa.

Termo para lembrar: Poison message | Base técnica: referência 1.

03. Rejeição de negócio deve ir para DLQ?

Normalmente não. Uma contestação inelegível pode ser resultado válido e terminal, persistido com motivo. A DLQ trata processamento que falhou; misturar os dois inflaria alertas e confundiria atendimento.

Termo para lembrar: Business rejection | Base técnica: referência 1.

04. Como define número máximo de tentativas?

Considero distribuição de falhas, prazo e custo de cada tentativa. Um limite muito baixo isola falhas recuperáveis; um limite enorme prolonga o bloqueio e sobrecarrega parceiros sem melhorar correção.

Termo para lembrar: Retry policy | Base técnica: referência 1.

05. Que dados guarda na quarentena?

Guardo event_id, origem, versão, erro, tentativas e referência ao payload necessário. Reduzo dados sensíveis e restrinjo leitura; a quarentena frequentemente recebe conteúdo bruto que não deveria aparecer em logs comuns.

Termo para lembrar: Diagnostic envelope | Base técnica: referência 1.

06. Como monitora a DLQ?

Alerto por entrada, idade e impacto no prazo de negócio. Uma fila com poucas mensagens pode conter pagamentos críticos; quantidade isolada não expressa severidade nem tempo restante para recuperação.

Termo para lembrar: DLQ age | Base técnica: referência 1.

07. Redrive é só reenviar tudo?

Não. Primeiro corrijo a causa e seleciono um lote controlado. Preservo identidades para deduplicação, limito vazão e acompanho efeitos; um redrive massivo pode recriar o incidente original.

Termo para lembrar: Controlled redrive | Base técnica: referência 1.

08. O ID do evento muda no redrive?

Preservo a identidade de negócio e de evento quando repito o mesmo conteúdo. Identificadores de transporte podem mudar; consumidores não devem depender exclusivamente deles para proteger o efeito final.

Termo para lembrar: Stable identity | Base técnica: referência 2.

09. E se precisar corrigir o payload?

Registro um evento corretivo com vínculo ao original e semântica explícita. Não reutilizo silenciosamente a mesma identidade com conteúdo diferente, porque isso quebra comparação de hash e trilha de auditoria.

Termo para lembrar: Corrective event | Base técnica: referência 2.

10. Retirar evento de FIFO mantém a ordem?

Não necessariamente. Se os seguintes dependem dele, a remoção abre uma lacuna semântica. Bloqueio ou adio o agregado afetado até resolver; filas ordenadas não tornam etapas ausentes dispensáveis.

Termo para lembrar: Ordering gap | Base técnica: referência 1.

11. Como evita bloquear outras contas?

Isolo por agregado ou grupo de mensagens quando o domínio permitir. Uma conta defeituosa não precisa interromper contas independentes; preservo apenas as dependências reais em vez de bloquear a fila inteira.

Termo para lembrar: Failure isolation | Base técnica: referência 1.

12. Quem deve operar o redrive?

Uma identidade autorizada com escopo e auditoria. Registrar operador, motivo, seleção e resultado permite investigar recuperações; acesso de leitura à DLQ não deve implicar permissão para reexecutar efeitos financeiros.

Termo para lembrar: Redrive authorization | Base técnica: referência 1.

13. Como trata mensagem cujo efeito já aconteceu?

O redrive pode encontrar processamento concluído. O consumidor reconhece a identidade e não repete o efeito; envio à DLQ não demonstra que a operação anterior falhou antes de produzir consequências.

Termo para lembrar: Idempotent consumer | Base técnica: referência 2.

14. Como lidar com timeout desconhecido?

Não classifico automaticamente como mensagem inválida. Persisto estado UNKNOWN e reconcilio com o destino; retentar pela DLQ sem resolver a incerteza pode duplicar uma operação externa já aceita.

Termo para lembrar: UNKNOWN | Base técnica: referência 3.

15. Como decide retenção da quarentena?

A retenção precisa cobrir investigação e recuperação, considerando regras do broker e prazo do negócio. Expirar a mensagem não resolve a obrigação; registro a pendência em armazenamento apropriado quando necessário.

Termo para lembrar: Retention | Base técnica: referência 1.

16. O que é uma parking lot queue?

É uma área de estacionamento para mensagens que exigem tratamento posterior específico. Defino estado e responsabilidade; mudar o nome da fila não elimina requisitos de retenção, segurança e redrive.

Termo para lembrar: Parking lot | Base técnica: referência 1.

17. Como distingue defeito de código e dado ruim?

Correlaciono versão do consumidor, tipo de erro e exemplos minimizados. Se todos os eventos novos falham após deploy, priorizo rollback; se apenas um contrato falha, isolo aquela classe com evidência.

Termo para lembrar: Error classification | Base técnica: referência 1.

18. Pode fazer retry infinito para garantir entrega?

Não é garantia útil. Pode ultrapassar retenção, prazo e capacidade, além de esconder erro permanente. Uso tentativas limitadas e escalonamento operacional, preservando a obrigação até existir resolução explícita.

Termo para lembrar: Bounded retry | Base técnica: referência 3.

19. Como evita loops entre fila principal e DLQ?

Registro histórico de recuperação e controlo políticas de redrive. Uma mensagem que retorna repetidamente sem mudança deve ser bloqueada para investigação; não automatizo reenvio periódico cego apenas por existir backlog.

Termo para lembrar: Redrive loop | Base técnica: referência 1.

20. Como trata evento de versão futura?

Posso adiar para consumidor compatível ou atualizar o parser, sem marcar o negócio concluído. Mantenho versão e payload original para recuperação; interpretar campos desconhecidos por suposição pode alterar significado financeiro.

Termo para lembrar: Schema compatibility | Base técnica: referência 1.

21. Qual papel de contratos no CI?

Testes de contrato detectam incompatibilidade antes da produção. Ainda mantenho quarentena porque dados e versões reais podem escapar; prevenção reduz volume de incidentes, mas não substitui recuperação operacional.

Termo para lembrar: Contract testing | Base técnica: referência 1.

22. Como valida antes do redrive?

Executo amostra em ambiente seguro ou modo sem efeitos, verifico parser e precondições atuais. Não assumo que a regra de negócio de ontem continua válida para todos os eventos atrasados.

Termo para lembrar: Replay validation | Base técnica: referência 2.

23. Como trata anexos referenciados que expiraram?

A mensagem pode sobreviver ao objeto e ficar irrecuperável. Alinho retenções e verifico existência e checksum; se o arquivo necessário sumiu, abro recuperação da evidência em vez de repetir downloads inúteis.

Termo para lembrar: Claim-check retention | Base técnica: referência 1.

24. Pode usar logs como única quarentena?

Logs normalmente não oferecem estado, seleção segura e controle de reexecução. Posso usá-los para diagnóstico, mas mantenho registro estruturado de pendência e payload recuperável conforme a necessidade do domínio.

Termo para lembrar: Durable quarantine | Base técnica: referência 1.

25. Como mede sucesso da recuperação?

Meço resolução da obrigação, não somente saída da DLQ. Confiro estado final, efeitos financeiros e mensagens que retornaram à quarentena; drenar a fila sem concluir o negócio é sucesso aparente.

Termo para lembrar: Business completion | Base técnica: referência 1.

26. O que faz se a DLQ estiver perto de expirar?

Prioritizo preservação autorizada das evidências e obrigações, avalio impacto e escalo responsáveis. Não espero a expiração silenciosa; o runbook precisa estabelecer como recuperar sem perder identidade ou confidencialidade.

Termo para lembrar: Retention deadline | Base técnica: referência 1.

27. Como controla custo de redrive?

Limito taxa por dependência e separo orçamento de recuperação do tráfego novo. Monitoro saturação e pauso se a recuperação degradar operações correntes além do limite aceito pelo negócio.

Termo para lembrar: Recovery budget | Base técnica: referência 3.

28. Como recupera uma ordem antiga cancelada?

Revalido estado e versão antes de executar. Redrive não autoriza ressuscitar uma intenção cancelada; registro resultado terminal apropriado e preservo evidência de que a mensagem atrasada foi tratada.

Termo para lembrar: State precondition | Base técnica: referência 2.

29. Quais testes são indispensáveis?

Injeto evento inválido, queda após efeito, redrive duplicado, expiração e lacuna de sequência. Verifico proteção do domínio e alarmes; apenas observar a mensagem chegando à DLQ testa pouco da recuperação.

Termo para lembrar: Failure tests | Base técnica: referência 1.

30. Como explica DLQ numa entrevista?

Eu a apresento como parte de um processo de recuperação com dono, prazo, segurança e invariantes. A caixa no desenho ajuda a isolar erro, mas a solução termina quando a obrigação é resolvida.

Termo para lembrar: Operational ownership | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Poucas tentativas

Ganho. Isola rápido

Custo e motivo. Mais falsos permanentes

Como decidir. Quando erro é determinístico

02. Muitas tentativas

Ganho. Tolera instabilidade

Custo e motivo. Carga e atraso

Como decidir. Para falhas transitórias medidas

03. Quarentena por tipo

Ganho. Diagnóstico preciso

Custo e motivo. Mais filas

Como decidir. Quando contratos diferem

04. Quarentena única

Ganho. Operação simples

Custo e motivo. Mistura criticidade

Como decidir. Para volume pequeno

05. Redrive automático

Ganho. Recuperação rápida

Custo e motivo. Pode repetir incidente

Como decidir. Após causa transitória comprovada

06. Redrive aprovado

Ganho. Controle de risco

Custo e motivo. Maior tempo operacional

Como decidir. Para efeitos financeiros ambíguos

07. Payload completo

Ganho. Reprodução fácil

Custo e motivo. Exposição de dados

Como decidir. Com acesso e retenção mínimos

08. Referência de payload

Ganho. Mensagem pequena

Custo e motivo. Objeto pode expirar

Como decidir. Com ciclo de vida coordenado

09. Bloquear agregado

Ganho. Preserva sequência

Custo e motivo. Atrasa aquele usuário

Como decidir. Para eventos dependentes

10. Rate limit de recuperação

Ganho. Protege tráfego vivo

Custo e motivo. Drenagem mais lenta

Como decidir. Durante backlog elevado

Duas alternativas ao caminho proposto

1. Tabela de pendências no banco

Quando faz sentido. Necessidade de consulta e workflow

Como implementar. Persistir estado, causa, tentativas e next_attempt_at

Limites. Exige scheduler e limpeza próprios

2. Rejeição terminal explícita

Quando faz sentido. Entrada é inválida por regra de negócio

Como implementar. Registrar resultado e notificar originador

Limites. Não resolve falhas técnicas desconhecidas

Referências e alcance da evidência

1. AWS - SQS dead-letter queues

Redrive, limites de recebimento, retenção e impacto na ordem.

https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-dead-letter-queues.html

2. AWS Builders Library - Idempotent APIs

Identidades estáveis, intenção, retries e requisições atrasadas.

https://aws.amazon.com/builders-library/making-retries-safe-with-idempotent-APIs/

3. AWS - Timeouts retries and jitter

Timeouts, backoff, jitter e risco de repetir efeitos após respostas perdidas.

https://aws.amazon.com/builders-library/timeouts-retries-and-backoff-with-jitter/

Continue em: T043 · T045 · T063 · T064

T047 / E. Mensageria e processamento

Sagas e compensações

Saga coordena etapas locais e ações compensatórias específicas do negócio sem uma transação global única.

Quando usar. Processos atravessam serviços e admitem estados intermediários controlados.

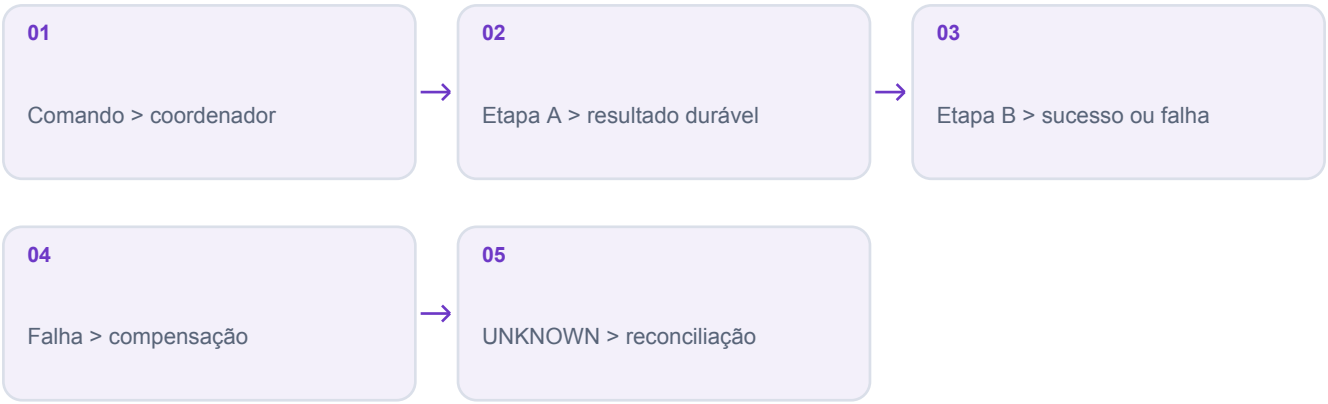
Quando evitar ou limitar. Não usar para mascarar uma invariante que exige atomicidade imediata e não tolera compensação.

Uma abertura possível

Vou listar etapas, efeitos irreversíveis e compensações; uma compensação é nova operação de negócio, não rollback mágico.

Peças e tecnologias. Temporal · Step Functions · Kafka · PostgreSQL

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Definir estados e invariantes intermediárias

2. Persistir intenção e resultado de cada etapa

3. Dar identidade própria a ação e compensação

4. Tratar timeout como potencial UNKNOWN

5. Testar falhas, concorrência e recuperação parcial

30 perguntas para sustentar o desenho

01. O que distingue saga de transação ACID?

Cada etapa confirma localmente e pode ficar visível antes das demais. Se algo falha, executo compensações de negócio; não prometo isolamento global nem retorno exato ao estado anterior como num rollback local.

Termo para lembrar: Saga | Base técnica: referência 1.

02. Quando você rejeitaria uma saga?

Se a invariante não admite exposição intermediária ou não existe compensação aceitável, reconsidero a fronteira do agregado. Distribuir serviços não justifica permitir dinheiro inconsistente; posso manter a operação crítica numa transação única.

Termo para lembrar: Fronteira de domínio | Base técnica: referência 1.

03. Compensar é desfazer bytes no banco?

Não. Outros processos podem ter alterado o estado após a etapa original. A compensação considera o contexto atual e cria uma ação rastreável, como lançamento inverso, sem apagar histórico legítimo.

Termo para lembrar: Compensating transaction | Base técnica: referência 1.

04. Como nomeia ação e compensação?

Uso identidades distintas e relacionadas: credit_operation_id e reversal_operation_id com referência ao original. Repetir a compensação conserva sua identidade; reutilizar a chave do crédito confundiria duas intenções diferentes no destino.

Termo para lembrar: Operation identity | Base técnica: referência 2.

05. Como evita compensação duplicada?

O destino impõe unicidade da intenção compensatória e valida o efeito original. A saga também persiste progresso, mas proteção somente no coordenador não cobre queda após execução remota e antes da resposta.

Termo para lembrar: Idempotent compensation | Base técnica: referência 2.

06. Timeout significa que devo compensar?

Não necessariamente. A etapa pode ter concluído. Primeiro classifico resultado desconhecido e reconcilio ou uso protocolo de cancelamento seguro; compensar uma operação ainda em voo pode criar corrida e estado incorreto.

Termo para lembrar: UNKNOWN | Base técnica: referência 3.

07. O que acontece se a compensação falhar?

Ela vira trabalho durável com retry seguro, limites e escalonamento. A saga permanece em recuperação, não concluída; métricas precisam mostrar obrigação pendente e prazo de negócio, mesmo quando não há erro HTTP.

Termo para lembrar: Recovery state | Base técnica: referência 1.

08. Orquestração ou coreografia?

Orquestração centraliza o fluxo e facilita acompanhar processos longos; coreografia reduz coordenador explícito, mas pode esconder dependências. Escolho conforme complexidade e ownership, evitando transformar eventos em um fluxo impossível de auditar.

Termo para lembrar: Orchestration | Base técnica: referência 1.

09. O coordenador é ponto único de falha?

Uma implementação ingênua pode ser. Persisto estado e permito retomada por outra instância com controle de concorrência; memória de processo não deve ser a única evidência de etapas já executadas.

Termo para lembrar: Durable coordinator | Base técnica: referência 1.

10. Como publica comandos após mudar estado?

Gravo estado da saga e outbox na mesma transação. O relay pode repetir, então cada comando preserva identidade. Não deixo uma etapa marcada como iniciada sem obrigação recuperável de enviá-la.

Termo para lembrar: Transactional outbox | Base técnica: referência 4.

11. Como recebe resposta duplicada?

Uso inbox ou transição condicional por versão e identidade da etapa. Uma resposta repetida não avança a saga duas vezes nem gera uma segunda compensação; respostas incompatíveis ficam para reconciliação.

Termo para lembrar: Idempotent transition | Base técnica: referência 4.

12. Uma resposta atrasada pode reabrir saga encerrada?

Não sem política explícita. Valido etapa, tentativa lógica e estado atual. Uma confirmação tardia de efeito antes desconhecido pode exigir correção, mas não sobrescreve simplesmente um estado terminal incompatível.

Termo para lembrar: Late response | Base técnica: referência 2.

13. Qual papel de reservas?

Reservas podem proteger capacidade ou saldo antes do efeito definitivo. Têm identidade, expiração e liberação controlada; expirar reserva enquanto confirmação externa ainda é possível exige protocolo que evite gastar o mesmo recurso.

Termo para lembrar: Reservation | Base técnica: referência 1.

14. Pode usar saldo em cache para aprovar etapa?

Não como autoridade se a invariante exige saldo disponível correto. A etapa responsável deve validar e reservar atomicamente no domínio financeiro; a saga coordena, mas não substitui controle de concorrência local.

Termo para lembrar: Authoritative balance | Base técnica: referência 1.

15. Como trata uma etapa irreversível?

Identifico o ponto de não retorno e tento executar validações antes dele. Depois, a recuperação pode exigir completar para frente ou intervenção; não invento uma compensação equivalente quando o efeito é irreversível.

Termo para lembrar: Pivot transaction | Base técnica: referência 1.

16. As compensações precisam seguir ordem inversa?

Nem sempre. Dependências e regras de negócio definem a ordem; algumas podem ocorrer em paralelo, outras precisam esperar. Documento essas relações em vez de aplicar uma pilha de rollback mecanicamente.

Termo para lembrar: Compensation ordering | Base técnica: referência 1.

17. Como lida com duas sagas sobre a mesma entidade?

Cada serviço protege suas invariantes com versões, reservas ou locks locais. A saga não oferece isolamento global; identifico conflitos e defino quais estados intermediários outras operações podem observar e modificar.

Termo para lembrar: Semantic locking | Base técnica: referência 1.

18. Como calcula o prazo total?

Incluo execução, espera, retries e recuperação, com deadlines por etapa. Um timeout técnico curto não encerra automaticamente o processo; o prazo de negócio determina escalonamento e experiência apresentada ao cliente.

Termo para lembrar: Business deadline | Base técnica: referência 3.

19. É possível cancelar a saga?

Cancelamento é uma intenção persistida, não garantia instantânea. Paro etapas ainda não iniciadas e trato as em andamento conforme contrato; comunico cancelamento pendente enquanto efeitos externos não forem confirmados.

Termo para lembrar: Cancellation protocol | Base técnica: referência 1.

20. Como faz evolução do workflow?

Versiono definições e mantenho interpretação para instâncias antigas. Não mudo a ordem das etapas de processos em andamento sem migração explícita e testes de recuperação; o histórico precisa continuar compreensível.

Termo para lembrar: Workflow versioning | Base técnica: referência 1.

21. Que informação guarda por etapa?

Guardo identidade, payload relevante ou referência imutável, status, resultado externo, timestamps e motivo de falha. Diferencio tentativa física de intenção lógica para repetir comunicação sem criar uma operação nova.

Termo para lembrar: Step journal | Base técnica: referência 2.

22. Como testa uma saga?

Injeto queda antes e depois de cada efeito, duplico respostas e atraso compensações. Verifico invariantes intermediárias e estado terminal; testar somente sucesso não exercita a característica principal desse padrão.

Termo para lembrar: Failure matrix | Base técnica: referência 1.

23. Compensação pode devolver valor diferente?

Pode, se a regra contratual exigir tarifas ou câmbio distintos, mas isso precisa ser explícito. Não assumo que compensar significa subtrair o mesmo número sem considerar moeda, precisão e efeitos já liquidados.

Termo para lembrar: Business semantics | Base técnica: referência 1.

24. Como evita transação de banco durante rede?

Persisto intenção em transação curta, executo a chamada fora dela e persisto resultado em outra etapa protegida. Manter lock durante rede aumenta contenção e não torna o destino participante da transação.

Termo para lembrar: Short transaction | Base técnica: referência 4.

25. Como opera uma saga parada?

Disponibilizo linha do tempo, etapa bloqueada, resultado conhecido e ações autorizadas. A intervenção usa os mesmos comandos idempotentes e deixa auditoria; não exige editar campos diretamente em produção.

Termo para lembrar: Operational recovery | Base técnica: referência 1.

26. Como trata publicação confirmada sem resposta de negócio?

Confirmação do broker prova apenas aquela entrega ao transporte. A etapa continua aguardando resultado de domínio; uso correlação e deadline, sem interpretar publisher confirm como crédito ou reembolso executado.

Termo para lembrar: Transport versus outcome | Base técnica: referência 4.

27. Que SLA mede?

Meço conclusão dentro do prazo, tempo em UNKNOWN e idade de compensações pendentes. Disponibilidade da API inicial não revela se clientes ficaram presos em processos incompletos por horas ou dias.

Termo para lembrar: End-to-end SLI | Base técnica: referência 1.

28. Pode reconstruir estado só pelos eventos?

Pode com contrato e retenção adequados, mas precisa controlar versões e efeitos de replay. Reconstruir o estado do coordenador não autoriza repetir comandos externos já executados e não revertidos.

Termo para lembrar: Replay safety | Base técnica: referência 4.

29. Como restauração regional afeta a saga?

Ela pode restaurar um progresso antigo enquanto parceiros mantêm efeitos recentes. Reconcilio etapas antes de retomar e preservo identidades no destino; um backup não constitui rollback distribuído do mundo externo.

Termo para lembrar: Disaster reconciliation | Base técnica: referência 2.

30. Como fecharia a defesa da solução?

Eu mostraria estados intermediários permitidos, identidades, ponto irreversível e recuperação de cada falha. A saga é adequada quando essas compensações são aceitáveis e operáveis, não apenas porque o sistema tem microsserviços.

Termo para lembrar: Saga suitability | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Orquestração

Ganho. Fluxo observável

Custo e motivo. Coordenador adicional

Como decidir. Para processos longos e auditáveis

02. Coreografia

Ganho. Autonomia local

Custo e motivo. Dependências implícitas

Como decidir. Para poucas reações independentes

03. Reserva antecipada

Ganho. Protege recurso

Custo e motivo. Bloqueia capacidade

Como decidir. Quando concorrência ameaça invariante

04. Compensação tardia

Ganho. Maior disponibilidade

Custo e motivo. Estado intermediário visível

Como decidir. Quando negócio tolera

05. Completar para frente

Ganho. Evita desfazer irreversível

Custo e motivo. Pode aumentar prazo

Como decidir. Após ponto de não retorno

06. Estado persistido

Ganho. Retomada confiável

Custo e motivo. Mais armazenamento

Como decidir. Para processos além de uma chamada

07. Paralelizar etapas

Ganho. Menor latência

Custo e motivo. Recuperação complexa

Como decidir. Quando etapas são independentes

08. Serializar etapas

Ganho. Raciocínio simples

Custo e motivo. Mais tempo total

Como decidir. Quando dependências são fortes

09. Automação de compensação

Ganho. Menos toil

Custo e motivo. Exige contratos seguros

Como decidir. Para efeitos reversíveis conhecidos

10. Intervenção humana

Ganho. Resolve exceções

Custo e motivo. Custo e atraso

Como decidir. Quando evidência externa é ambígua

Duas alternativas ao caminho proposto

1. Transação local

Quando faz sentido. Invariante cabe num domínio

Como implementar. Co-localizar estado e aplicar ACID

Limites. Pode limitar autonomia de serviços

2. Workflow sem compensação automática

Quando faz sentido. Processo exige decisão humana

Como implementar. Persistir etapas e encaminhar exceções

Limites. Maior prazo e carga operacional

Referências e alcance da evidência

1. Microsoft - Compensating transaction

Compensação específica do negócio e consideração de mudanças concorrentes.

https://learn.microsoft.com/en-us/azure/architecture/patterns/compensating-transaction

2. AWS Builders Library - Idempotent APIs

Identidades estáveis, intenção, retries e requisições atrasadas.

https://aws.amazon.com/builders-library/making-retries-safe-with-idempotent-APIs/

3. AWS - Timeouts retries and jitter

Timeouts, backoff, jitter e risco de repetir efeitos após respostas perdidas.

https://aws.amazon.com/builders-library/timeouts-retries-and-backoff-with-jitter/

4. AWS - Transactional outbox

Gravação atômica de estado e obrigação de publicar; duplicatas ainda possíveis.

https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/transactional-outbox.html

Continue em: T048 · T061 · T064 · T091

T048 / E. Mensageria e processamento

Workflows duráveis: Temporal e Step Functions

Workflows duráveis persistem progresso e retomam coordenação após falhas; Activities externas continuam exigindo garantias próprias.

Quando usar. Esperas longas, sinais humanos, retries e processos com várias etapas.

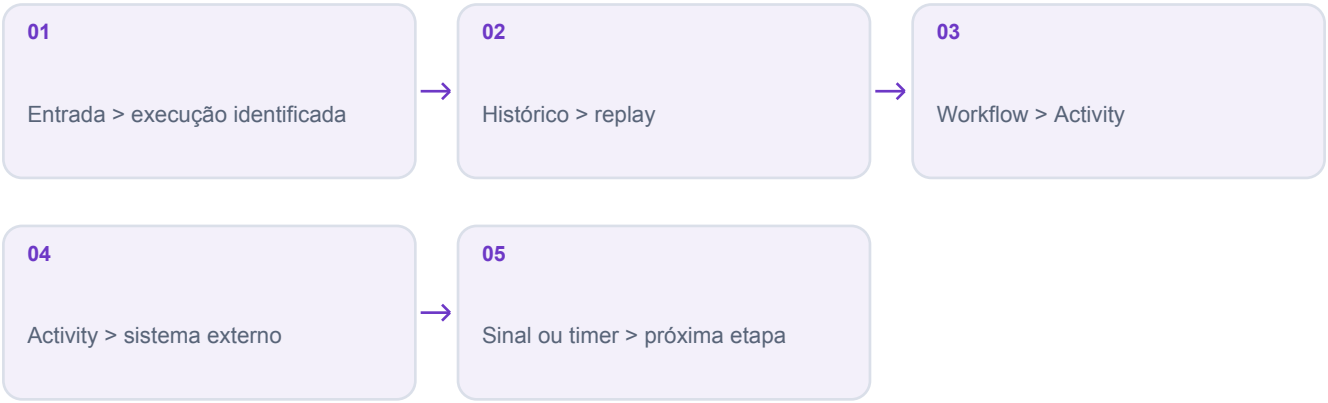
Quando evitar ou limitar. Não adicionar motor de workflow quando uma transação e um job simples resolvem o problema.

Uma abertura possível

Vou separar código determinístico de coordenação e efeitos externos, definindo identidade, timeouts e versão para processos em andamento.

Peças e tecnologias. Temporal · Step Functions · PostgreSQL · Workers

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Modelar estados, sinais e deadlines

2. Definir ID estável da execução

3. Isolar efeitos em Activities idempotentes

4. Versionar código e testar replay

5. Monitorar tarefas pendentes e recuperação

30 perguntas para sustentar o desenho

01. O que torna um workflow durável?

Seu progresso é persistido para que outra execução retome a coordenação após falhas. Isso evita depender de uma thread viva durante dias; a garantia concreta depende do motor e da configuração escolhidos.

Termo para lembrar: Durable execution | Base técnica: referência 1.

02. Replay executa novamente todas as chamadas HTTP?

Não deveria no código de workflow determinístico. Efeitos ficam em Activities, cujos resultados entram no histórico; retries de Activities ainda podem repetir chamadas e precisam de idempotência no destino.

Termo para lembrar: Workflow replay | Base técnica: referência 1.

03. Por que determinismo é necessário?

O motor compara decisões com o histórico ao reconstruir execução. Ler relógio ou aleatoriedade diretamente pode mudar decisões; uso APIs próprias do workflow e mantenho efeitos externos fora da coordenação determinística.

Termo para lembrar: Deterministic workflow | Base técnica: referência 1.

04. Activity é exactly-once?

Não assumo isso. Uma Activity pode produzir efeito e perder a resposta antes de registrar conclusão. Ela precisa de identidade estável ou reconciliação; durabilidade da coordenação não elimina ambiguidades de rede.

Termo para lembrar: Activity retry | Base técnica: referência 1.

05. Como escolhe Workflow ID?

Uso identidade do processo de negócio, com política explícita de unicidade e reutilização. Gerar um UUID novo em cada retry do cliente pode iniciar processos duplicados para a mesma intenção.

Termo para lembrar: Workflow identity | Base técnica: referência 1.

06. Como diferencia retry de novo processo?

Preservo ID e payload esperado na repetição. Uma nova revisão ou intenção ganha identidade própria e relação com a anterior; reutilização sem comparar conteúdo pode executar parâmetros errados silenciosamente.

Termo para lembrar: Intent identity | Base técnica: referência 3.

07. Como representa espera de três dias?

Uso timer durável do motor, não sleep numa thread ocupando recurso. A retomada revalida estado e prazo; esperar três dias corridos pode diferir de três dias úteis segundo o contrato.

Termo para lembrar: Durable timer | Base técnica: referência 1.

08. Como recebe aprovação humana?

Uso sinal ou mecanismo equivalente ligado à execução e revisão aprovada. Autorizo o remetente fora ou na fronteira apropriada e rejeito aprovação de conteúdo que mudou após ser apresentado.

Termo para lembrar: Workflow signal | Base técnica: referência 1.

09. Como lida com sinal duplicado?

A aprovação possui identidade e versão da decisão. O workflow reconhece repetição e não avança duas vezes; confiar apenas no canal de entrega seria insuficiente diante de retry ou recuperação.

Termo para lembrar: Signal idempotency | Base técnica: referência 1.

10. Como trata aprovação atrasada?

Verifico estado, revisão e deadline atuais. Se o processo já expirou ou foi substituído, registro o sinal como tardio sem executar o efeito antigo; não reabro automaticamente uma decisão financeira.

Termo para lembrar: State precondition | Base técnica: referência 1.

11. Como define timeout de Activity?

Separo espera na fila, duração de execução e prazo total de tentativas conforme o motor. Um único timeout genérico dificulta diagnosticar worker indisponível versus parceiro lento e pode gerar retries inadequados.

Termo para lembrar: Activity timeout | Base técnica: referência 1.

12. Heartbeat impede execução duplicada?

Não é prova absoluta de exclusividade. Heartbeats ajudam detectar progresso e retomar trabalho, mas um worker antigo pode voltar; efeitos externos exigem idempotência e, quando aplicável, fencing no recurso.

Termo para lembrar: Heartbeat | Base técnica: referência 1.

13. O que acontece quando worker cai?

O motor mantém o histórico e redistribui trabalho conforme suas políticas. A Activity interrompida pode ser repetida; desenho checkpoints e efeitos de forma que retomar não gere uma segunda operação financeira.

Termo para lembrar: Worker recovery | Base técnica: referência 1.

14. O motor pode substituir meu ledger?

Não. Ele coordena etapas, enquanto o ledger mantém invariantes financeiras. O workflow registra intenção e acompanha resultado, mas não transforma estado de coordenação em autoridade contábil automaticamente.

Termo para lembrar: Separation of concerns | Base técnica: referência 1.

15. Standard e Express são intercambiáveis?

Não. Step Functions possui modalidades com características de duração, histórico e execução diferentes. Escolho após verificar o contrato necessário; nomes de produto não substituem análise de efeito externo e idempotência.

Termo para lembrar: Workflow type | Base técnica: referência 2.

16. Standard dispensa idempotência nas tarefas?

Não para qualquer integração. Retries configurados e respostas ambíguas do destino precisam de análise. A garantia de execução do workflow não torna uma API externa participante atômica do histórico.

Termo para lembrar: External boundary | Base técnica: referência 2.

17. Como mantém histórico sob controle?

Uso os mecanismos do motor para encerrar ou continuar execuções com estado resumido, quando apropriado. Não deixo um processo infinito acumular eventos sem considerar limites, custo e tempo de replay.

Termo para lembrar: History growth | Base técnica: referência 1.

18. Como atualiza código com processos antigos?

Emprego versionamento e estratégias compatíveis com replay. Testo históricos reais minimizados antes de liberar; mudar uma decisão anterior sem preservar compatibilidade pode impedir retomada de processos ainda ativos.

Termo para lembrar: Workflow versioning | Base técnica: referência 1.

19. Rollback de deploy restaura processo financeiro?

Não. Ele troca código, mas efeitos já executados continuam existentes. Verifico compatibilidade do histórico e trato recuperação de negócio separadamente; rollback técnico não equivale a compensação financeira.

Termo para lembrar: Rollback boundary | Base técnica: referência 1.

20. Como usa compensação no workflow?

Registro etapas concluídas e compensações específicas, com identidade própria. Se a compensação falha, ela permanece pendente e observável; não marco a saga encerrada só porque o caminho principal terminou.

Termo para lembrar: Compensation | Base técnica: referência 3.

21. Pode consultar diretamente banco no workflow?

Em modelos de replay determinístico, acesso externo deve ocorrer por mecanismo apropriado, normalmente Activity. Ler banco diretamente durante replay pode mudar decisões históricas e comprometer a reconstituição da execução.

Termo para lembrar: Deterministic boundary | Base técnica: referência 1.

22. Como protege segredos no histórico?

Evito inserir credenciais e dados desnecessários em parâmetros persistidos. Uso referências e mecanismos de proteção apropriados; histórico durável aumenta a janela de exposição se conteúdo sensível for gravado indiscriminadamente.

Termo para lembrar: Payload protection | Base técnica: referência 1.

23. Como limita carga no parceiro?

Defino concorrência e rate limit nos workers ou fila de atividades, com orçamento de retry. Milhares de timers vencendo juntos não devem gerar uma rajada que derrube a integração externa.

Termo para lembrar: Worker throttling | Base técnica: referência 1.

24. Como cancela um processo?

Cancelamento é sinal persistido e protocolo de recuperação. Activities em andamento podem já ter produzido efeito; acompanho confirmação, compensação ou reconciliação antes de anunciar que tudo foi cancelado ao cliente.

Termo para lembrar: Cancellation | Base técnica: referência 1.

25. Quando usar child workflow?

Quando uma parte possui ciclo de vida, ownership ou volume que merece isolamento. Não decomponho cada função em child workflow; isso aumenta histórico e coordenação sem necessariamente melhorar entendimento ou recuperação.

Termo para lembrar: Workflow decomposition | Base técnica: referência 1.

26. Como testa sem esperar dias?

Uso ferramentas de teste do motor e avanço controlado de tempo quando disponível. Exercito sinais, retries e cancelamento, além de testar Activities contra contratos reais; simulação não comprova integração externa.

Termo para lembrar: Time-skipping tests | Base técnica: referência 1.

27. Quais métricas importam?

Idade de tarefas pendentes, tempo por etapa, retries, workflows bloqueados e conclusão dentro do prazo. Disponibilidade do cluster do motor não demonstra que os processos de negócio estão efetivamente progredindo.

Termo para lembrar: Workflow observability | Base técnica: referência 1.

28. Como recupera após desastre regional?

Verifico replicação e recuperação do histórico, além de efeitos nos parceiros. Execuções restauradas podem desconhecer respostas recentes; mantenho identidade no destino e reconcilio antes de repetir operações financeiras ambíguas.

Termo para lembrar: Disaster recovery | Base técnica: referência 3.

29. Qual custo de adotar um motor?

Ganho timers, histórico e recuperação padronizados, mas adiciono plataforma, modelos de programação e operação. Avalio quantidade e complexidade dos fluxos; um scheduler com tabela pode atender um processo muito simples.

Termo para lembrar: Platform trade-off | Base técnica: referência 1.

30. Como defenderia sua escolha?

Eu demonstraria uma falha de worker durante espera e outra após efeito externo. O motor resolve a retomada da coordenação; explico separadamente como a Activity e o destino evitam efeito duplicado.

Termo para lembrar: Guarantee boundary | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Motor durável

Ganho. Retomada padronizada

Custo e motivo. Plataforma adicional

Como decidir. Para fluxos longos

02. Código determinístico

Ganho. Replay confiável

Custo e motivo. Restrições de programação

Como decidir. Quando usa histórico de execução

03. Activities pequenas

Ganho. Retry granular

Custo e motivo. Mais coordenação

Como decidir. Para efeitos independentes

04. Activities grandes

Ganho. Menos etapas

Custo e motivo. Recuperação ampla

Como decidir. Quando há atomicidade interna

05. Workflow longo

Ganho. Contexto contínuo

Custo e motivo. Histórico cresce

Como decidir. Com estratégia de continuação

06. Execuções segmentadas

Ganho. Histórico limitado

Custo e motivo. Passagem de contexto

Como decidir. Para processos recorrentes

07. Worker dedicado

Ganho. Isola carga

Custo e motivo. Capacidade extra

Como decidir. Para parceiro crítico

08. Worker compartilhado

Ganho. Melhor utilização

Custo e motivo. Noisy neighbor

Como decidir. Para atividades semelhantes

09. Standard

Ganho. Histórico e processos apropriados

Custo e motivo. Custo por transição

Como decidir. Quando contrato exigir essas características

10. Express

Ganho. Modelo adequado a alto volume curto

Custo e motivo. Garantias e limites distintos

Como decidir. Após validar semântica da modalidade

Duas alternativas ao caminho proposto

1. PostgreSQL com jobs e timers

Quando faz sentido. Poucas etapas simples

Como implementar. Persistir estado, due_at, lease e versão

Limites. Equipe implementa retries e recuperação

2. Máquina de estados orientada a eventos

Quando faz sentido. Domínios reagem de modo independente

Como implementar. Outbox, inbox e contratos explícitos

Limites. Fluxo completo pode ficar difícil de acompanhar

Referências e alcance da evidência

1. Temporal - Workflows

Execução durável, replay e definição de workflows.

https://docs.temporal.io/workflows

2. AWS - Standard e Express Workflows

Diferenças de execução, histórico e características entre modalidades.

https://docs.aws.amazon.com/step-functions/latest/dg/choosing-workflow-type.html

3. AWS Builders Library - Idempotent APIs

Identidades estáveis, intenção, retries e requisições atrasadas.

https://aws.amazon.com/builders-library/making-retries-safe-with-idempotent-APIs/

Continue em: T047 · T050 · T061 · T065

T049 / E. Mensageria e processamento

Batch, arquivos, manifestos e checkpoints

Processamento em lote exige identidade estável, conjunto de entrada definido, artefato verificável e retomada sem repetir efeitos.

Quando usar. Integrações por arquivo, reconciliação e processamento de grandes conjuntos.

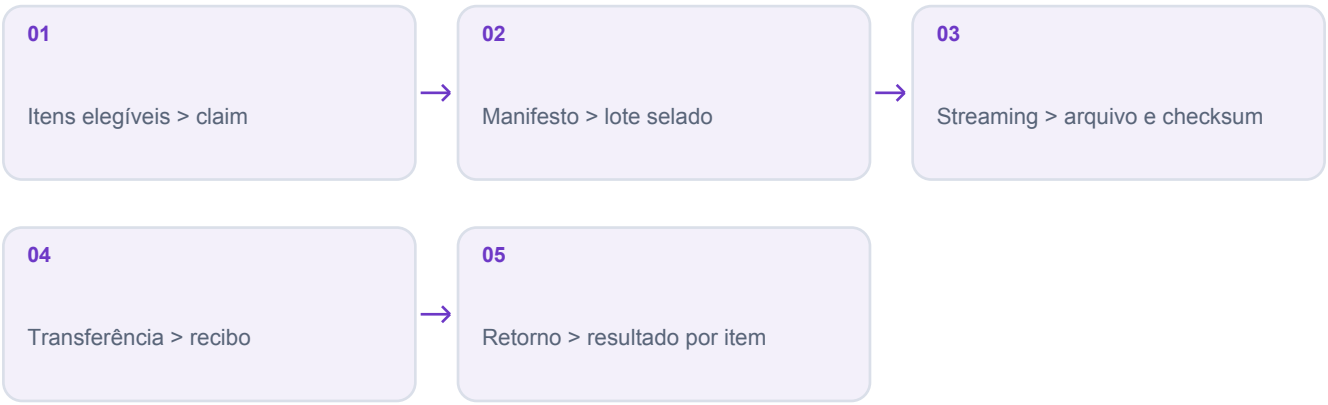
Quando evitar ou limitar. Não montar um novo lote com itens de resultado desconhecido sem resolver identidade e aceitação externa.

Uma abertura possível

Vou congelar quais itens pertencem ao lote e distinguir gerar arquivo, transferir, receber e processar.

Peças e tecnologias. S3 · Azure Blob · ECS · AWS Batch · PostgreSQL · SFTP

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Selecionar itens com cutoff e claim atômico

2. Persistir manifesto imutável e versão de layout

3. Gerar arquivo por streaming e checksum

4. Transferir com identidade estável e estado UNKNOWN

5. Aplicar retorno idempotente e reconciliar totais

30 perguntas para sustentar o desenho

01. Como define os itens de um lote?

Persisto um manifesto com identidades e versões dos itens selecionados. Depois de selado, o conjunto não muda silenciosamente; novos itens entram em outro lote, permitindo reproduzir exatamente o que foi enviado.

Termo para lembrar: Immutable manifest | Base técnica: referência 1.

02. Selecionar todos os pending é suficiente?

Não. Dois workers podem selecionar os mesmos registros. Faço claim atômico com batch_id e versão ou locks curtos, e só depois gero arquivo fora da transação para evitar contenção prolongada.

Termo para lembrar: Atomic claim | Base técnica: referência 2.

03. Como escolhe o cutoff?

Defino o instante e a regra de elegibilidade, incluindo timezone e dados atrasados. O cutoff é persistido no lote; não dependo de um filtro que muda enquanto o worker percorre páginas.

Termo para lembrar: Cutoff | Base técnica: referência 2.

04. Paginação por offset é segura durante montagem?

Pode pular ou repetir itens se o conjunto mudar. Prefiro seleção congelada e keyset pagination sobre identidade estável, com checkpoint; o manifesto determina quais registros pertencem à execução.

Termo para lembrar: Keyset pagination | Base técnica: referência 2.

05. Como evita carregar arquivo inteiro em memória?

Leio itens paginados e escrevo por streaming, mantendo buffers limitados. Calculo totais e checksum incrementalmente; tamanho do lote não deve multiplicar diretamente a memória necessária do processo.

Termo para lembrar: Streaming | Base técnica: referência 1.

06. Checksum comprova aprovação pela bandeira?

Não. Ele verifica integridade dos bytes segundo o algoritmo, não aceitação de negócio. Preciso de recibo e resultado do parceiro; arquivo íntegro pode conter registros rejeitados por regras externas.

Termo para lembrar: Checksum | Base técnica: referência 1.

07. ETag é sempre MD5 do arquivo?

Não assumo isso, especialmente em multipart e configurações diferentes. Uso checksum explicitamente suportado e registrado no manifesto; o contrato de integridade precisa dizer qual algoritmo e quais bytes foram calculados.

Termo para lembrar: Object checksum | Base técnica: referência 1.

08. O nome do arquivo pode ser a chave de deduplicação?

Pode participar do contrato, mas só se o parceiro reconhecer essa identidade e preservar semântica. Incluo batch_id estável e checksum; nome aleatório em cada retry pode transformar repetição em novo envio.

Termo para lembrar: Batch identity | Base técnica: referência 3.

09. Como retoma geração interrompida?

Uso checkpoint em limite reproduzível e objeto temporário ou partes verificadas. Antes de publicar como pronto, valido quantidade, checksum e manifesto; um arquivo parcial não deve parecer artefato final ao consumidor.

Termo para lembrar: Checkpoint | Base técnica: referência 1.

10. Por que separar objeto temporário e pronto?

Isso impede que um consumidor leia um artefato incompleto como lote válido. A publicação da referência final ocorre após validação; o protocolo de prontidão deve ser explícito, não inferido pelo prefixo de data.

Termo para lembrar: Commit marker | Base técnica: referência 1.

11. Pode alterar o CSV depois de uma tentativa?

Não sob a mesma identidade sem contrato. Preservo bytes e checksum do lote tentado; correção gera revisão ou novo artefato rastreável, porque o destino pode já ter recebido a versão anterior.

Termo para lembrar: Artifact immutability | Base técnica: referência 3.

12. O que significa transferência concluída?

Significa o estágio que o protocolo confirmou, não necessariamente processamento de todos os registros. Modelo separado upload concluído, arquivo aceito e resultado de negócio, evitando anunciar sucesso financeiro prematuramente.

Termo para lembrar: Transfer versus processing | Base técnica: referência 3.

13. Conexão SFTP caiu: monto outro lote?

Não automaticamente. O arquivo pode ter chegado parcialmente ou por completo. Marco UNKNOWN quando necessário, preservo identidade e busco evidência contratual antes de incluir novamente os mesmos efeitos num lote diferente.

Termo para lembrar: UNKNOWN | Base técnica: referência 3.

14. Arquivo ausente no diretório prova não recebimento?

Não. O parceiro pode mover ou consumir arquivos após recebê-los. A ausência só tem significado definido pelo contrato; uso recibos, consulta ou reconciliação, sem inferir rejeição pelo diretório vazio.

Termo para lembrar: External evidence | Base técnica: referência 3.

15. Como trata rejeição parcial?

Registro resultado por item e motivo, além de totais do lote. Itens aprovados não voltam à fila apenas porque outros falharam; cada nova tentativa respeita elegibilidade e identidade da operação correspondente.

Termo para lembrar: Partial result | Base técnica: referência 2.

16. Como deduplica arquivo de retorno?

Uso identidade do retorno, checksum e proteção por item na mesma transação da atualização. Uma segunda entrega não repete crédito ou transição; conflito de conteúdo com a mesma identidade exige investigação.

Termo para lembrar: Inbound deduplication | Base técnica: referência 2.

17. Como verifica que o retorno está completo?

Comparo itens esperados, quantidades, valores por moeda e estados finais. Uma linha ausente não equivale a rejeição; mantenho pendência e prazo de reconciliação para registros sem resultado confirmado.

Termo para lembrar: Completeness reconciliation | Base técnica: referência 2.

18. Como lida com layout novo?

Versiono parser e gerador e testo exemplos do parceiro. Campos monetários e encoding precisam de contrato explícito; aceitar uma coluna deslocada silenciosamente pode produzir erro financeiro mesmo com checksum correto.

Termo para lembrar: Layout versioning | Base técnica: referência 1.

19. Como representa valores monetários no arquivo?

Uso formato decimal ou inteiro em unidade definida, com moeda e regras de arredondamento explícitas. Não serializo float binário sem controle; valido totais antes do envio e após o retorno.

Termo para lembrar: Money encoding | Base técnica: referência 2.

20. Como impede dois workers enviando o lote?

Uso claim com lease e attempt_id, mas reconheço que um worker antigo pode voltar. A proteção final depende de fencing aceito pelo destino ou deduplicação externa pela identidade do lote.

Termo para lembrar: Lease boundary | Base técnica: referência 3.

21. Uma nova tentativa recebe novo batch_id?

Não quando representa o mesmo artefato e intenção. Distingo attempt_id físico de batch_id lógico; mudar o lote exige justificar semântica e garantir que itens anteriores não foram aceitos externamente.

Termo para lembrar: Attempt versus intent | Base técnica: referência 3.

22. Como organiza S3 por data?

Uso prefixos com formato ordenável e identificadores, como ano/mês/dia/batch_id. O prefixo facilita operação, mas não é a autoridade de estado; manifesto e banco registram elegibilidade, versão e processamento.

Termo para lembrar: Object key | Base técnica: referência 1.

23. O evento de S3 pode disparar processamento duplicado?

Projeto o consumidor para tolerar notificações repetidas e eventos atrasados. Valido objeto e manifesto antes de agir; um aviso de objeto criado não substitui idempotência nem confirmação de prontidão do negócio.

Termo para lembrar: Event idempotency | Base técnica: referência 3.

24. Como dimensiona batch e paralelismo?

Considero limite do parceiro, tamanho de arquivo, tempo de geração e prazo. Lotes maiores amortizam overhead, mas aumentam tempo de recuperação e impacto de rejeição; particiono por contrato e unidade independente.

Termo para lembrar: Batch sizing | Base técnica: referência 1.

25. Como testa arquivo sem enviar dinheiro?

Uso fixtures validadas, ambiente de homologação e modo de geração sem efeitos. Comparo bytes, layout e totais; teste sintático local não prova que a integração real aceitará todas as regras de negócio.

Termo para lembrar: Contract test | Base técnica: referência 1.

26. Como restaura lote depois de desastre?

Recupero manifesto, artefato e estado de tentativas de forma coerente. Antes de reenviar, reconcilio o que o parceiro já recebeu; restaurar banco antigo não apaga arquivos ou efeitos externos recentes.

Termo para lembrar: Disaster recovery | Base técnica: referência 3.

27. Como audita intervenção manual?

Operador usa comando autorizado com motivo, lote e itens selecionados. Registro revisão e resultado; editar CSV ou status diretamente sem rastreabilidade destrói a possibilidade de explicar uma duplicidade posterior.

Termo para lembrar: Controlled redrive | Base técnica: referência 2.

28. Quais métricas prioriza?

Idade do item elegível mais antigo, lotes UNKNOWN, prazo até cutoff, divergência de totais e taxa de rejeição. Sucesso do upload não basta para medir conclusão do processo bancário.

Termo para lembrar: Batch SLI | Base técnica: referência 1.

29. Como trata chegada tardia após fechar lote?

Ela entra em nova janela conforme regra e prazo, com referência ao período de negócio correto. Não reabro silenciosamente um lote selado; atrasos precisam aparecer na reconciliação e nos indicadores.

Termo para lembrar: Late arrival | Base técnica: referência 2.

30. Como defenderia sua arquitetura?

Eu mostraria o manifesto imutável, a separação de tentativa e intenção e o retorno por item. A recuperação repete comunicação quando seguro, preservando evidência do que foi efetivamente enviado e aceito.

Termo para lembrar: End-to-end batch | Base técnica: referência 3.

10 trade-offs: ganho, custo e decisão

01. Lote grande

Ganho. Amortiza overhead

Custo e motivo. Recuperação e atraso maiores

Como decidir. Quando prazo e parceiro permitem

02. Lote pequeno

Ganho. Falha afeta menos itens

Custo e motivo. Mais transferências

Como decidir. Para resultados rápidos

03. Manifesto persistido

Ganho. Reprodução exata

Custo e motivo. Escrita adicional

Como decidir. Para integração auditável

04. Streaming

Ganho. Memória limitada

Custo e motivo. Checkpoint mais complexo

Como decidir. Para arquivos grandes

05. Arquivo imutável

Ganho. Retry rastreável

Custo e motivo. Correções geram revisão

Como decidir. Após primeira tentativa externa

06. Prefixo temporal

Ganho. Organização operacional

Custo e motivo. Não garante unicidade

Como decidir. Com batch_id independente

07. Paralelismo

Ganho. Maior vazão

Custo e motivo. Quota e ordem

Como decidir. Para lotes independentes

08. Retorno por item

Ganho. Recuperação precisa

Custo e motivo. Modelo mais detalhado

Como decidir. Quando há rejeição parcial

09. Checksum forte

Ganho. Verifica bytes

Custo e motivo. Custo de cálculo

Como decidir. Para transferência e auditoria

10. Quarentena de UNKNOWN

Ganho. Evita duplicação cega

Custo e motivo. Mais latência

Como decidir. Quando aceitação externa é incerta

Duas alternativas ao caminho proposto

1. API por operação

Quando faz sentido. Parceiro oferece endpoint apropriado

Como implementar. Enviar intenções idempotentes e consultar status

Limites. Mais chamadas e limites de taxa

2. Streaming de eventos

Quando faz sentido. Integração suporta eventos contínuos

Como implementar. Publicar com sequência e confirmação por efeito

Limites. Exige contrato e operação diferentes de arquivos

Referências e alcance da evidência

1. AWS - S3 object integrity

Verificação de integridade por checksums em objetos.

https://docs.aws.amazon.com/AmazonS3/latest/userguide/checking-object-integrity.html

2. PostgreSQL - Transaction isolation

Snapshots, concorrência, isolamento e falhas de serialização.

https://www.postgresql.org/docs/current/transaction-iso.html

3. AWS Builders Library - Idempotent APIs

Identidades estáveis, intenção, retries e requisições atrasadas.

https://aws.amazon.com/builders-library/making-retries-safe-with-idempotent-APIs/

Continue em: T050 · T058 · T060 · T095

T050 / E. Mensageria e processamento

Cron, calendários, cutoff e quotas

Agendamento dispara tentativas; calendário e quota definem quando trabalho é elegível e quanto pode executar.

Quando usar. Rotinas bancárias, cutoff, processamento periódico e limites de parceiros.

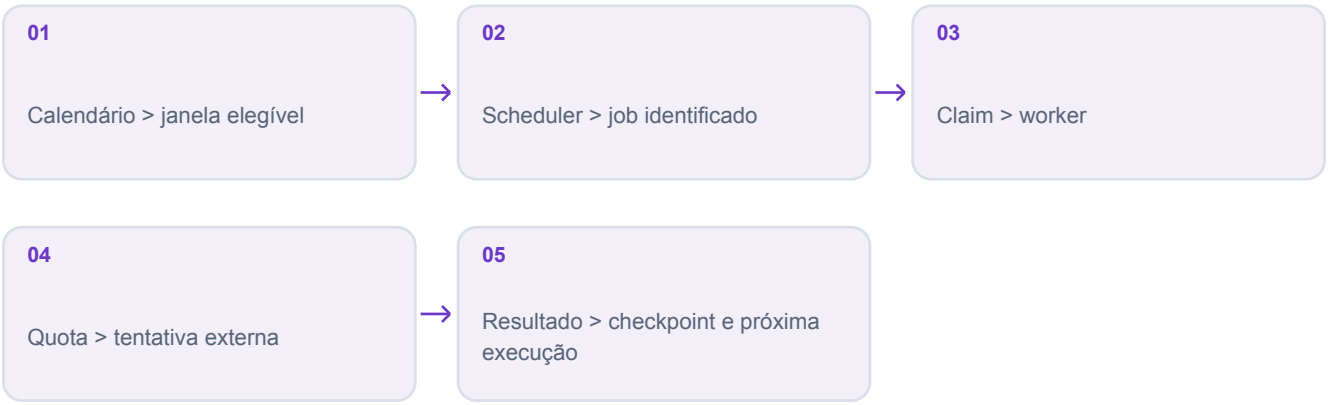
Quando evitar ou limitar. Não interpretar uma execução cron como garantia de conclusão única ou quatro horários como quatro transmissões físicas.

Uma abertura possível

Vou persistir a janela de negócio e a identidade do job; cron apenas acorda o processamento, que precisa recuperar atrasos e duplicatas.

Peças e tecnologias. EventBridge Scheduler · Kubernetes CronJob · Temporal · PostgreSQL

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Definir timezone, calendário e cutoff

2. Persistir chave única por janela e escopo

3. Implementar claim e recuperação de jobs

4. Aplicar quota a tentativas efetivas

5. Monitorar atrasos, duplicatas e janelas perdidas

30 perguntas para sustentar o desenho

01. Cron garante uma execução por horário?

Não assumo execução única. Agendadores podem atrasar, perder ou repetir disparos conforme condições; a identidade do job e a persistência do progresso precisam tornar a execução recuperável e idempotente.

Termo para lembrar: Scheduled job identity | Base técnica: referência 2.

02. Qual chave usa para um job diário?

Uso tipo de job, escopo e janela de negócio, como país e data operacional. O identificador não depende do instante em que o worker iniciou, pois atrasos não devem criar nova intenção.

Termo para lembrar: Business window | Base técnica: referência 2.

03. UTC resolve todo problema de calendário?

UTC representa instantes, mas regras como dia útil às 17h dependem de timezone e calendário. Persisto a regra e sua versão, além do instante calculado para aquela execução específica.

Termo para lembrar: Business calendar | Base técnica: referência 1.

04. Por que usar timezone IANA?

Ela representa regras históricas e mudanças de horário local melhor que um offset fixo. Ainda testo a política do scheduler para horários inexistentes ou repetidos e documento o comportamento esperado.

Termo para lembrar: IANA timezone | Base técnica: referência 1.

05. O que acontece no horário de verão?

Depende do agendador e da expressão. Confirmo como ele trata hora ausente ou repetida, e protejo a janela por unicidade; não deduzo a política apenas da sintaxe cron utilizada.

Termo para lembrar: DST policy | Base técnica: referência 1.

06. Rate de 24 horas equivale a todo dia às 4h?

Não necessariamente. Intervalo fixo e horário de calendário possuem semânticas diferentes, especialmente com mudanças de timezone. Escolho conforme requisito de negócio e preservo a janela calculada no registro do job.

Termo para lembrar: Rate versus cron | Base técnica: referência 1.

07. Como trata feriados bancários?

Uso calendário de negócio versionado e verifico país, produto e parceiro. Feriado não é uma propriedade universal do scheduler; atualizações de calendário precisam reavaliar execuções futuras de forma auditável.

Termo para lembrar: Calendar versioning | Base técnica: referência 1.

08. O que significa cutoff?

É a fronteira de elegibilidade ou processamento definida pelo negócio. Persisto o instante e a regra aplicados; um worker atrasado não deve reinterpretar silenciosamente quais itens pertenciam à janela original.

Termo para lembrar: Cutoff | Base técnica: referência 1.

09. Como recupera um disparo perdido?

Um reconciliador compara janelas esperadas com jobs persistidos e cria as faltantes de maneira idempotente. O cron principal não é a única fonte de recuperação; evito depender de memória do agendador.

Termo para lembrar: Catch-up | Base técnica: referência 2.

10. Catch-up deve executar todas as janelas imediatamente?

Não. Avalio prazo, supersessão e capacidade. Algumas janelas podem ser agregadas, outras exigem execução separada; aplico limites para que recuperação de atraso não derrube parceiros ou tráfego atual.

Termo para lembrar: Catch-up policy | Base técnica: referência 2.

11. O que faz quando execução anterior ainda roda?

Escolho concorrência permitida pelo domínio. Posso bloquear nova execução, enfileirar ou particionar trabalho, mas protejo efeitos com identidade; política do scheduler sozinha não impede um worker antigo de continuar.

Termo para lembrar: Concurrency policy | Base técnica: referência 2.

12. Como escolhe lease do job?

A lease precisa de renovação e limite operacional apropriados. Sua expiração permite recuperação, mas não mata o processo antigo; uso attempt_id e fencing onde suportado para impedir gravações obsoletas.

Termo para lembrar: Lease | Base técnica: referência 2.

13. Quatro horários significam quatro envios?

Não. Retries podem produzir várias transmissões dentro de cada horário. Se o contrato limita tentativas físicas, contabilizo cada tentativa real; diferencio slot lógico de processamento e quota de envio externo.

Termo para lembrar: Quota accounting | Base técnica: referência 1.

14. Como reserva uma unidade de quota?

Faço concessão atômica no escopo e janela corretos antes da tentativa. Defino quando a unidade é consumida ou devolvida segundo contrato; timeout externo não prova que a tentativa deixou de ocorrer.

Termo para lembrar: Atomic quota | Base técnica: referência 3.

15. Quota por instância funciona globalmente?

Não sem coordenação ou divisão conservadora. Dez instâncias com limite quatro podem realizar quarenta tentativas; distribuo orçamento explícito e controlo realocação, evitando soma acima do contrato global do parceiro.

Termo para lembrar: Distributed quota | Base técnica: referência 1.

16. Como separa pendência de resultado desconhecido?

PENDING indica trabalho elegível ainda não concluído conforme o modelo; UNKNOWN indica que o parceiro pode ter recebido. Não junto os dois cegamente num lote novo só porque chegou o próximo horário.

Termo para lembrar: UNKNOWN | Base técnica: referência 3.

17. Uma falha de conexão sempre permite retry?

Não. Dependendo do ponto da falha, bytes e efeito podem ter chegado. Registro evidência e uso contrato de idempotência ou consulta; o relógio do scheduler não resolve essa ambiguidade.

Termo para lembrar: Retry safety | Base técnica: referência 3.

18. Como representa next_attempt_at?

Persisto o próximo instante elegível com causa e tentativa lógica. O scheduler busca trabalhos devidos por índice e claim limitado; reiniciar o processo não deve zerar backoff nem esquecer a pendência.

Termo para lembrar: Durable retry | Base técnica: referência 3.

19. Como evita tempestade às 4h?

Uso espalhamento dentro da janela permitida, limites de concorrência e quotas. Se o cutoff exige um horário estrito, dimensiono capacidade correspondente; jitter não pode violar arbitrariamente o contrato de negócio.

Termo para lembrar: Scheduled jitter | Base técnica: referência 1.

20. O que faz com relógio local errado?

Não confio apenas no relógio de cada worker para decisões críticas. Uso fonte de tempo operacional controlada e validações na autoridade de estado, monitorando skew e mantendo identidade da janela independente do processo.

Termo para lembrar: Clock skew | Base técnica: referência 2.

21. Como mede atraso do scheduler?

Comparo instante planejado, disparo, início e conclusão. Essas quatro medidas distinguem atraso de agendamento, fila e execução; sucesso do disparo não demonstra que o lote terminou antes do cutoff.

Termo para lembrar: Schedule lag | Base técnica: referência 1.

22. Como testa calendários complexos?

Crio casos de feriado, fim de mês, ano bissexto, DST e mudança de regra. Verifico elegibilidade e identidade da janela, usando relógio controlável para reproduzir limites sem esperar dias reais.

Termo para lembrar: Calendar testing | Base técnica: referência 1.

23. Como muda o horário sem duplicar janela?

Versiono a regra e defino ponto de corte para aplicação. Reconcilio jobs já criados e preservo a identidade de negócio; alterar expressão cron sem migração pode executar a mesma janela duas vezes.

Termo para lembrar: Schedule migration | Base técnica: referência 2.

24. Pode apagar job concluído imediatamente?

Somente se outra proteção preservar a identidade durante a janela de repetição. Remover histórico cedo pode fazer catch-up recriar trabalho; retenção considera auditoria, recuperação e risco de efeitos duplicados.

Termo para lembrar: Job retention | Base técnica: referência 3.

25. Como trata desligamento durante o job?

Paro novos claims, persisto checkpoints e termino trabalho dentro do orçamento. Se não concluir, a lease permite retomada; o novo worker conserva a identidade e verifica efeitos externos já tentados.

Termo para lembrar: Graceful shutdown | Base técnica: referência 2.

26. Como divide um job muito grande?

Crio shards com limites estáveis e checkpoints independentes, ligados ao job pai. O pai só conclui após verificar todos os shards e totais; quantidade de workers não deve mudar silenciosamente a seleção.

Termo para lembrar: Partitioned batch | Base técnica: referência 2.

27. O scheduler deve executar regra financeira?

Prefiro que ele apenas dispare uma intenção identificada. A regra fica no domínio ou worker versionado, com autorização e auditoria; misturar lógica de negócio na expressão de agenda dificulta teste e evolução.

Termo para lembrar: Separation of concerns | Base técnica: referência 1.

28. Como recupera após perda regional?

Reconstruo janelas e progresso a partir de estado durável e reconcilio tentativas externas antes do catch-up. Um scheduler secundário ativo sem fencing pode duplicar execução durante partição regional.

Termo para lembrar: Disaster recovery | Base técnica: referência 3.

29. Qual alerta é mais importante?

Depende do impacto, mas priorizo trabalho elegível prestes a perder prazo e janelas não concluídas. Uma métrica de cron executado pode estar verde enquanto todo processamento permanece bloqueado no parceiro.

Termo para lembrar: Business deadline | Base técnica: referência 1.

30. Como defenderia o desenho?

Eu separaria calendário, identidade do job, claim, quota e resultado externo. Cada camada tem falhas diferentes; essa separação permite recuperar execução atrasada sem transformar a próxima janela em duplicação financeira.

Termo para lembrar: Scheduling contract | Base técnica: referência 2.

10 trade-offs: ganho, custo e decisão

01. Cron simples

Ganho. Baixa complexidade

Custo e motivo. Pouca semântica de negócio

Como decidir. Para tarefas sem calendário complexo

02. Scheduler durável

Ganho. Recupera progresso

Custo e motivo. Plataforma adicional

Como decidir. Para prazos e esperas longas

03. Hora local

Ganho. Alinha operação humana

Custo e motivo. DST e calendário

Como decidir. Quando contrato usa horário local

04. UTC fixo

Ganho. Instante inequívoco

Custo e motivo. Pode mudar horário local

Como decidir. Para intervalos técnicos

05. Catch-up completo

Ganho. Preserva janelas

Custo e motivo. Rajada de recuperação

Como decidir. Quando todas são necessárias

06. Coalescer janelas

Ganho. Menos trabalho

Custo e motivo. Pode perder granularidade

Como decidir. Para snapshots substituíveis

07. Lease curta

Ganho. Recupera rápido

Custo e motivo. Mais renovação e sobreposição

Como decidir. Com fencing e trabalho curto

08. Quota central

Ganho. Precisão global

Custo e motivo. Coordenação e disponibilidade

Como decidir. Para limite externo rígido

09. Jitter

Ganho. Suaviza pico

Custo e motivo. Varia instante

Como decidir. Dentro da janela permitida

10. Shards estáveis

Ganho. Paralelismo recuperável

Custo e motivo. Coordenação de conclusão

Como decidir. Para lotes grandes

Duas alternativas ao caminho proposto

1. Eventos de negócio

Quando faz sentido. Não há necessidade de horário fixo

Como implementar. Disparar ao atingir condição persistida

Limites. Ainda exige reconciliador de eventos perdidos

2. Polling de jobs devidos

Quando faz sentido. Agenda moderada e banco disponível

Como implementar. Tabela com due_at, claim e checkpoints

Limites. Carga de polling e manutenção próprias

Referências e alcance da evidência

1. AWS - Schedule types

Cron, rate, execução única, timezone e horário de verão.

https://docs.aws.amazon.com/scheduler/latest/UserGuide/schedule-types.html

2. Kubernetes - CronJob

Agendamento aproximado, concorrência e necessidade de jobs idempotentes.

https://kubernetes.io/docs/concepts/workloads/controllers/cron-jobs/

3. AWS Builders Library - Idempotent APIs

Identidades estáveis, intenção, retries e requisições atrasadas.

https://aws.amazon.com/builders-library/making-retries-safe-with-idempotent-APIs/

Continue em: T049 · T060 · T064 · T065

T051 / F. Rede e entrada

DNS, TLS, mTLS e descoberta

DNS encontra destinos; TLS protege e autentica conexões; mTLS também autentica o cliente por certificado.

Quando usar. Entrada HTTPS, comunicação entre serviços e descoberta de endpoints.

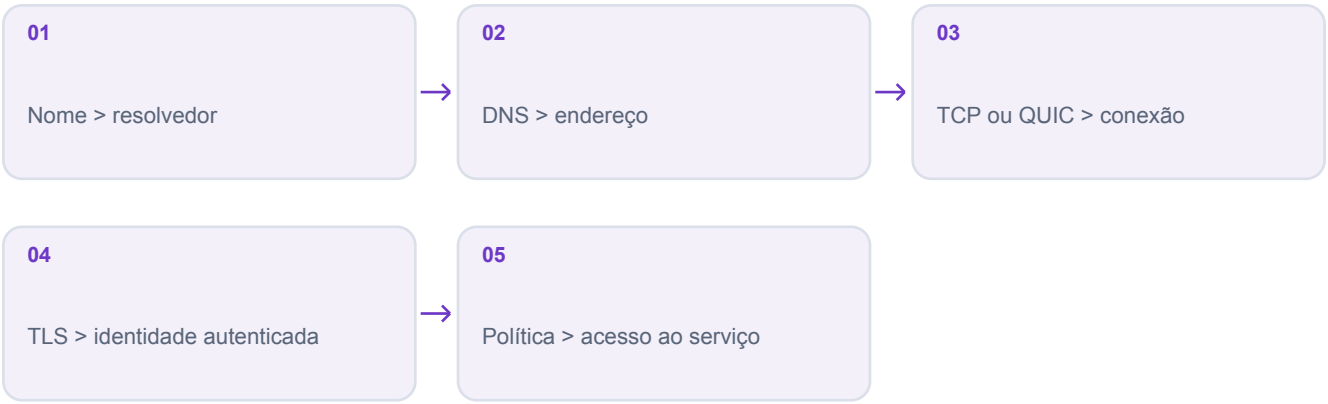
Quando evitar ou limitar. Não confundir IP privado, certificado válido ou nome resolvido com autorização de negócio.

Uma abertura possível

Vou separar resolução, conexão segura e política de acesso, pois cada etapa possui cache, timeout e recuperação próprios.

Peças e tecnologias. DNS · Route 53 · CoreDNS · TLS 1.3 · Envoy · mTLS

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Definir nomes e autoridades de descoberta

2. Configurar validação de cadeia e hostname

3. Automatizar emissão e rotação de certificados

4. Limitar cache e timeouts de resolução

5. Testar expiração, falha de DNS e revogação

30 perguntas para sustentar o desenho

01. DNS autentica o serviço?

Não. DNS resolve um nome para informações de destino; a conexão precisa validar identidade pelo mecanismo apropriado, como TLS. Um endereço resolvido não autoriza a aplicação nem comprova permissão para acessar dados.

Termo para lembrar: Name resolution | Base técnica: referência 1.

02. Qual diferença entre resolver e servidor autoritativo?

O resolver consulta e pode cachear respostas; o autoritativo responde pela zona. Diagnostico qual camada falhou antes de alterar registros, porque uma resposta antiga pode permanecer em caches intermediários.

Termo para lembrar: DNS hierarchy | Base técnica: referência 1.

03. TTL garante troca imediata após expirar?

Ele orienta a validade do cache, mas a recuperação também depende de resolvers, clientes e conexões existentes. Planejo failover considerando esses comportamentos, sem prometer migração instantânea apenas alterando um registro.

Termo para lembrar: DNS TTL | Base técnica: referência 1.

04. Por que TTL muito baixo custa?

Aumenta consultas e dependência do caminho de resolução, além de poder reduzir eficiência do cache. Uso quando preciso de atualização frequente, medindo latência e disponibilidade do resolvedor em vez de reduzir indiscriminadamente.

Termo para lembrar: TTL trade-off | Base técnica: referência 1.

05. O que é cache negativo?

Também posso cachear respostas de ausência conforme regras DNS. Se criar um nome após uma consulta malsucedida, clientes podem continuar vendo ausência por algum tempo; considero isso em bootstrap e migrações.

Termo para lembrar: Negative caching | Base técnica: referência 1.

06. Como separa falha DNS de serviço fora?

Registro tempo e resultado de resolução, conexão e handshake. Se DNS falha, aumentar timeout da aplicação não corrige a causa; métricas por fase evitam culpar o backend por falha anterior.

Termo para lembrar: Connection phases | Base técnica: referência 1.

07. TLS protege quais propriedades?

Protege confidencialidade e integridade do canal e autenticação conforme configuração. Não impede que um endpoint autorizado use dados indevidamente; controles de aplicação e autorização permanecem necessários além da criptografia do transporte.

Termo para lembrar: TLS | Base técnica: referência 2.

08. Certificado válido basta sem hostname?

Não. Valido cadeia, período e identidade esperada do destino. Aceitar qualquer certificado confiável sem conferir nome permite conectar ao serviço errado; desabilitar validação para resolver incidente cria uma vulnerabilidade persistente.

Termo para lembrar: Hostname verification | Base técnica: referência 2.

09. O que muda com mTLS?

Além de autenticar o servidor, o handshake autentica o cliente por certificado quando configurado. Ainda mapeio essa identidade para políticas; possuir certificado não significa poder executar qualquer operação bancária.

Termo para lembrar: Mutual TLS | Base técnica: referência 2.

10. Como distribui certificados entre serviços?

Uso emissão automatizada e identidade de workload com escopo limitado. Evito copiar a mesma chave privada para todos os serviços, pois um comprometimento ampliaria a capacidade de se passar por identidades distintas.

Termo para lembrar: Workload identity | Base técnica: referência 2.

11. Como rotaciona sem interromper tráfego?

Introduzo certificados e confiança novos com sobreposição controlada, atualizo workloads e retiro material antigo depois de validar. A aplicação precisa recarregar credenciais ou renovar conexões sem depender de intervenção manual em massa.

Termo para lembrar: Certificate rotation | Base técnica: referência 2.

12. O que acontece se a CA ficar indisponível?

Conexões e certificados válidos podem continuar conforme configuração, mas emissão e renovação ficam ameaçadas. Monitoro prazo de validade e preparo recuperação antes de expiração coletiva, evitando transformar falha temporária em outage programado.

Termo para lembrar: Certificate lifetime | Base técnica: referência 2.

13. Como trata expiração simultânea?

Espalho renovação e automatizo alertas antecipados. Testo falha do emissor e recarga do cliente; renovar todo o parque no mesmo instante pode criar dependência concentrada e reconexão massiva.

Termo para lembrar: Renewal jitter | Base técnica: referência 2.

14. Pode usar 0-RTT para um débito?

Não sem análise de replay no protocolo e aplicação. Early data pode ser repetível em condições específicas; comandos financeiros exigem proteção de intenção e política que não trate menor latência como autorização para risco.

Termo para lembrar: TLS 0-RTT | Base técnica: referência 2.

15. TLS encerrado no load balancer protege o resto?

Somente o trecho até aquele terminador. Defino proteção também entre proxy e backend conforme ameaças; o desenho precisa mostrar onde dados ficam em claro e quem pode acessar cada segmento.

Termo para lembrar: TLS termination | Base técnica: referência 2.

16. Recriptografar ao backend custa o quê?

Adiciona handshakes, CPU e operação de certificados, embora pooling reduza parte do custo. Escolho por necessidade de proteger rede interna e autenticar workloads, medindo impacto em latência e capacidade.

Termo para lembrar: Re-encryption | Base técnica: referência 2.

17. Como evita spoofing de headers de identidade?

O proxy remove headers não confiáveis e injeta identidade após autenticação, com canal protegido até o backend. O serviço não aceita o mesmo header vindo diretamente de clientes externos.

Termo para lembrar: Trusted proxy boundary | Base técnica: referência 2.

18. DNS service discovery substitui health check?

Não. Resolver endereço não prova prontidão nem capacidade do destino. Combino descoberta com checks e política de roteamento, evitando encaminhar para processo vivo que não consegue atender a função essencial.

Termo para lembrar: Service discovery | Base técnica: referência 1.

19. Como lida com conexões antigas após mudança DNS?

Conexões persistentes podem continuar no endereço anterior. Uso drenagem e política de renovação apropriadas; derrubar todas simultaneamente pode gerar pico de handshake e falhas evitáveis durante a migração.

Termo para lembrar: Connection draining | Base técnica: referência 1.

20. IP fixo simplifica descoberta?

Pode simplificar um contrato específico, mas dificulta mudanças e recuperação se incorporado ao código. Prefiro nome estável ou descoberta controlada quando possível, preservando requisitos de allowlist por mecanismos apropriados.

Termo para lembrar: Endpoint indirection | Base técnica: referência 1.

21. mTLS resolve autorização por cliente bancário?

Não. A identidade do serviço chamador é diferente da identidade do usuário e do recurso. O domínio ainda verifica tenant, ownership e ação permitida; transporte autenticado não elimina autorização fina.

Termo para lembrar: Authorization boundary | Base técnica: referência 2.

22. Como um service mesh ajuda?

Pode automatizar certificados e políticas de transporte entre workloads, mas adiciona plano de controle e proxies. Avalio complexidade operacional; não transfiro regras financeiras de autorização para uma política genérica de rede.

Termo para lembrar: Service mesh | Base técnica: referência 2.

23. DNSSEC e TLS são substitutos?

Não. Autenticidade de dados DNS e segurança do canal têm objetivos distintos. Mesmo com resolução protegida, valido o endpoint TLS e mantenho autorização; escolher uma camada não torna as outras dispensáveis.

Termo para lembrar: Defense in depth | Base técnica: referência 1.

24. Como reduz custo de handshake?

Reutilizo conexões e avalio resumption conforme o protocolo e risco. Pools precisam de limites e renovação; otimizar conexão não pode remover verificação de certificado ou permitir sessões sobreviverem indefinidamente a mudanças de política.

Termo para lembrar: Session resumption | Base técnica: referência 2.

25. Como protege chave privada?

Restrinjo acesso, uso armazenamento e emissão apropriados e evito logs ou imagens contendo chave. Se comprometida, a recuperação envolve revogação ou expiração controlada e substituição, não somente reiniciar o serviço.

Termo para lembrar: Private key custody | Base técnica: referência 2.

26. Que timeout inclui DNS e TLS?

Verifico a semântica da biblioteca e estabeleço orçamento total que cubra fases necessárias. Alguns limites abrangem apenas parte da conexão; testes com resolução lenta e handshake bloqueado revelam essas lacunas.

Termo para lembrar: End-to-end deadline | Base técnica: referência 2.

27. Como testa failover de resolução?

Altero destino em ambiente controlado e observo caches, reconexões e tráfego residual. Meço tempo real de convergência e validação de certificado; não considero apenas a atualização bem-sucedida do registro autoritativo.

Termo para lembrar: DNS failover test | Base técnica: referência 1.

28. Qual risco de confiar só em rede privada?

Um serviço comprometido ou configuração errada pode acessar a rede. Uso identidade e autorização além de segmentação; privado descreve conectividade, não legitimidade de toda chamada que chega ao destino.

Termo para lembrar: Zero trust principle | Base técnica: referência 2.

29. Quais métricas são úteis?

Tempo de resolução, erros por código, duração de handshake, falhas de validação e dias até expiração. Correlaciono com indisponibilidade percebida, sem registrar chaves, tokens ou conteúdo sensível para diagnosticar conectividade.

Termo para lembrar: TLS observability | Base técnica: referência 2.

30. Como defenderia essa camada?

Eu explicaria quem resolve o nome, quem autentica cada lado e onde a política decide acesso. Depois demonstraria rotação e falha de resolução, preservando segurança sem depender de desativar validações.

Termo para lembrar: Trust boundary | Base técnica: referência 2.

10 trade-offs: ganho, custo e decisão

01. TTL curto

Ganho. Mudança mais rápida

Custo e motivo. Mais consultas

Como decidir. Para endpoints dinâmicos

02. TTL longo

Ganho. Cache eficiente

Custo e motivo. Convergência lenta

Como decidir. Para destinos estáveis

03. mTLS

Ganho. Identidade bilateral

Custo e motivo. Operação de certificados

Como decidir. Entre workloads críticos

04. TLS no edge

Ganho. Menos custo interno

Custo e motivo. Trecho interno separado

Como decidir. Conforme modelo de ameaça

05. TLS até backend

Ganho. Protege mais segmentos

Custo e motivo. CPU e gestão

Como decidir. Para rede interna não confiável

06. Conexão persistente

Ganho. Menos handshake

Custo e motivo. Destino antigo permanece

Como decidir. Com drenagem planejada

07. Certificado curto

Ganho. Reduz janela de exposição

Custo e motivo. Renovação crítica

Como decidir. Com automação confiável

08. CA central

Ganho. Governança uniforme

Custo e motivo. Dependência compartilhada

Como decidir. Com alta disponibilidade e isolamento

09. Service mesh

Ganho. Automação uniforme

Custo e motivo. Complexidade e proxies

Como decidir. Quando escala organizacional justifica

10. 0-RTT

Ganho. Menor latência inicial

Custo e motivo. Risco de replay

Como decidir. Apenas para semântica compatível

Duas alternativas ao caminho proposto

1. Gateway autenticado

Quando faz sentido. Poucos serviços internos

Como implementar. Centralizar terminação e propagar identidade protegida

Limites. Backend ainda exige autorização e fronteira confiável

2. PKI gerenciada sem mesh

Quando faz sentido. Precisa de mTLS com menos plataforma

Como implementar. SDKs e agentes renovam certificados diretamente

Limites. Cada runtime precisa integrar recarga e validação

Referências e alcance da evidência

1. IETF - RFC 1034 DNS

Resolução hierárquica, cache e conceitos de DNS.

https://www.rfc-editor.org/rfc/rfc1034

2. IETF - RFC 8446 TLS 1.3

Handshake, autenticação, confidencialidade e limites de replay de early data.

https://www.rfc-editor.org/rfc/rfc8446

Continue em: T052 · T053 · T059 · T071

T052 / F. Rede e entrada

TCP, UDP, HTTP/2 e HTTP/3

Protocolos de transporte e HTTP afetam latência, multiplexação, perdas e controle de fluxo, mas não confirmam efeito de negócio.

Quando usar. APIs, comunicação entre serviços, clientes móveis e conexões persistentes.

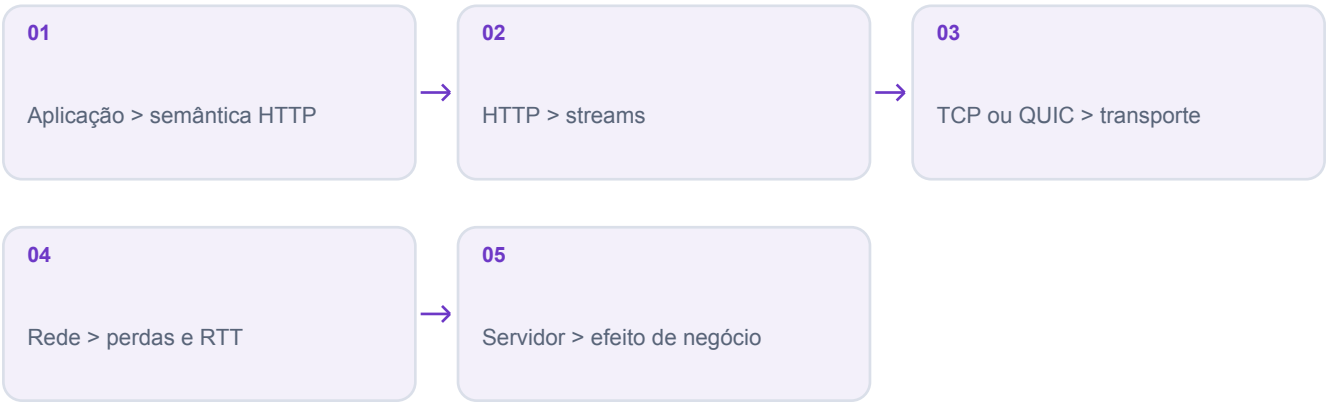
Quando evitar ou limitar. Não atribuir exactly-once financeiro ao TCP nem assumir que HTTP/3 resolve toda latência.

Uma abertura possível

Vou separar confiabilidade de bytes, semântica HTTP e confirmação do domínio antes de otimizar conexões.

Peças e tecnologias. TCP · UDP · HTTP/1.1 · HTTP/2 · HTTP/3 · QUIC · gRPC

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Medir RTT, perda e tamanho de payload

2. Escolher protocolo compatível com clientes

3. Configurar pools, streams e deadlines

4. Implementar idempotência no domínio

5. Testar perda, reconexão e caminhos de fallback

30 perguntas para sustentar o desenho

01. TCP garante pagamento único?

Não. TCP oferece transporte confiável de bytes numa conexão, não confirmação transacional do pagamento. Se a resposta se perder após commit, a aplicação ainda precisa reconhecer a intenção e consultar o resultado.

Termo para lembrar: Transport boundary | Base técnica: referência 1.

02. UDP sempre é mais rápido?

Não em qualquer carga. UDP oferece datagramas sem as mesmas garantias do TCP; a aplicação ou protocolo superior pode implementar confiabilidade e controle. Comparo requisitos e medições, não apenas overhead do cabeçalho.

Termo para lembrar: UDP semantics | Base técnica: referência 3.

03. HTTP/2 elimina toda espera entre requisições?

Ele multiplexa streams na conexão, mas perdas no TCP podem afetar o progresso de vários streams. A melhoria sobre HTTP/1.1 não elimina dependências de transporte, servidor ou limites de concorrência.

Termo para lembrar: Head-of-line blocking | Base técnica: referência 2.

04. Como HTTP/3 muda esse problema?

HTTP/3 usa QUIC e separa entrega confiável por streams, reduzindo bloqueio entre streams causado por perdas. Ainda existem congestionamento, dependências da aplicação e limites de compressão; não promete latência constante.

Termo para lembrar: QUIC streams | Base técnica: referência 3.

05. O que é RTT e por que importa?

É o tempo aproximado de ida e volta. Handshakes e chamadas sequenciais multiplicam seu impacto; reduzo round trips desnecessários e reutilizo conexões antes de tentar resolver tudo aumentando CPU.

Termo para lembrar: Round-trip time | Base técnica: referência 1.

06. Como pooling de conexão ajuda?

Reutiliza conexão estabelecida e reduz custo de handshake, mas precisa de limites, timeouts e drenagem. Um pool ilimitado pode esgotar sockets, portas e recursos do servidor mesmo com requisições pequenas.

Termo para lembrar: Connection pooling | Base técnica: referência 1.

07. Por que muitas conexões podem piorar?

Elas consomem memória, descritores, estado de rede e capacidade de handshake. Mais paralelismo não significa mais throughput se o destino estiver saturado; dimensiono conexões pelo trabalho sustentável e protocolo utilizado.

Termo para lembrar: Connection pressure | Base técnica: referência 1.

08. O que controla flow control?

Ele limita dados em voo conforme capacidade de recepção no escopo do protocolo. Não substitui admission control da aplicação, pois o servidor ainda pode aceitar trabalho semanticamente caro ou persistir backlog excessivo.

Termo para lembrar: Flow control | Base técnica: referência 2.

09. Congestion control e flow control diferem?

Sim. Um reage à capacidade e sinais do caminho de rede; outro evita sobrecarregar o receptor no escopo do protocolo. Nenhum conhece sozinho a quota financeira ou o custo de uma consulta.

Termo para lembrar: Congestion control | Base técnica: referência 1.

10. HTTP/2 muda os métodos REST?

Não. Métodos e status mantêm semântica HTTP; a representação em frames e streams muda o transporte. Uma operação POST continua precisando de contrato idempotente se o cliente puder repeti-la após falha.

Termo para lembrar: HTTP semantics | Base técnica: referência 4.

11. gRPC precisa de cuidado com deadlines?

Sim. Propago orçamento e cancelo trabalho apropriado quando o prazo acaba. Uma chamada RPC conveniente continua sendo distribuída; cancelamento local não prova que o servidor deixou de produzir um efeito financeiro.

Termo para lembrar: RPC deadline | Base técnica: referência 2.

12. Como lida com stream reset?

Classifico se houve possível processamento antes do reset e aplico política de retry segura. Não assumo que reset significa operação não executada; preservo identidade e consulto resultado quando houver efeito relevante.

Termo para lembrar: Stream reset | Base técnica: referência 2.

13. Como desliga servidor HTTP/2?

Dreno conexões e uso os mecanismos do protocolo para indicar encerramento conforme implementação. Novas chamadas migram, enquanto trabalho aceito termina ou é recuperado; fechamento abrupto aumenta respostas ambíguas.

Termo para lembrar: Graceful drain | Base técnica: referência 2.

14. HTTP/3 funciona onde UDP é bloqueado?

Pode exigir fallback para outro protocolo suportado. Testo redes corporativas e móveis reais; disponibilidade não pode depender de todos os caminhos permitirem QUIC sem uma estratégia de compatibilidade.

Termo para lembrar: Protocol fallback | Base técnica: referência 3.

15. Qual risco de 0-RTT em comando financeiro?

Dados antecipados podem ter risco de replay conforme o protocolo. Só habilito onde a semântica e a proteção de intenção forem adequadas; menor tempo de handshake não compensa duplicidade de operação.

Termo para lembrar: Early data replay | Base técnica: referência 3.

16. Conexão viva comprova aplicação saudável?

Não. O socket pode existir enquanto a aplicação está travada ou incapaz de acessar dependências. Uso health checks e deadlines de aplicação, distinguindo conectividade, prontidão e conclusão do negócio.

Termo para lembrar: Application liveness | Base técnica: referência 1.

17. Como controla payload enorme?

Limito tamanho, duração e taxa de leitura, além de usar streaming quando apropriado. Multiplexação não impede que um cliente consuma memória ou CPU desproporcional com conteúdo grande ou descompactação excessiva.

Termo para lembrar: Payload limits | Base técnica: referência 2.

18. Compressão sempre reduz latência?

Ela reduz bytes, mas consome CPU e pode aumentar atraso para conteúdo pequeno ou já comprimido. Meço tamanho e custo; também limito expansão para não aceitar bombas de descompressão.

Termo para lembrar: Compression trade-off | Base técnica: referência 2.

19. HTTP/2 melhora qualquer backend?

Não. Se o gargalo é consulta lenta ou lock, mudar protocolo pouco ajuda. Avalio métricas por estágio e número de round trips, evitando atribuir todo p99 à camada de transporte.

Termo para lembrar: Bottleneck analysis | Base técnica: referência 2.

20. Como trata redes móveis instáveis?

Projeto reconexão, retries com identidade e retomada de leitura. Protocolos podem ajudar a mobilidade, mas a aplicação precisa preservar mensagens pendentes e distinguir resposta perdida de nova intenção.

Termo para lembrar: Connection migration | Base técnica: referência 3.

21. O que significa TCP half-open?

Os lados podem divergir sobre o estado da conexão após falhas de rede ou processo. Timeouts e probes ajudam detectar, mas o estado de negócio precisa ser consultável independentemente daquele socket antigo.

Termo para lembrar: Half-open connection | Base técnica: referência 1.

22. Por que retransmissão aumenta p99?

Perdas exigem recuperação e podem atrasar dados necessários, afetando a cauda mesmo com média boa. Observo perda, RTT e retransmissões junto de traces; apenas aumentar threads não corrige esse caminho.

Termo para lembrar: Tail latency | Base técnica: referência 1.

23. Como HTTP/1.1 difere no uso de conexões?

Ele não oferece a mesma multiplexação independente de streams do HTTP/2. Clientes frequentemente usam múltiplas conexões para concorrência; comparo comportamento real do cliente e proxy antes de dimensionar pools.

Termo para lembrar: Multiplexing | Base técnica: referência 2.

24. Um proxy pode mudar o protocolo no meio?

Sim. O cliente pode usar HTTP/2 até o edge e o backend receber HTTP/1.1. Observo cada trecho, pois limites de conexão e concorrência podem mudar na tradução e criar gargalos inesperados.

Termo para lembrar: Protocol termination | Base técnica: referência 2.

25. Como escolhe limites de streams?

Considero memória por stream, capacidade de processamento e limites do destino. Muitos streams numa conexão ainda representam trabalho concorrente; limito admissão e não confundo menos sockets com capacidade infinita.

Termo para lembrar: Concurrent streams | Base técnica: referência 2.

26. Qual relação entre timeout e retransmissão?

O timeout da aplicação pode vencer enquanto o transporte ainda tenta recuperar bytes. Defino prazo pelo valor do trabalho e política de retry, lembrando que encerrar espera não desfaz execução no servidor.

Termo para lembrar: Timeout boundary | Base técnica: referência 1.

27. Como garante ordem de mensagens da aplicação?

O transporte ordena no seu escopo, mas reconexões e múltiplos canais podem reordenar intenções. Uso sequência de domínio quando necessária; uma conexão TCP anterior não define ordem global após retomada.

Termo para lembrar: Application sequence | Base técnica: referência 1.

28. Como testa protocolo com evidência?

Introduzo latência, perda, reconexão e limites de banda em ambiente controlado. Comparo p95, p99, CPU e conclusão correta, incluindo fallback; benchmark numa rede perfeita pode esconder o comportamento importante.

Termo para lembrar: Network fault test | Base técnica: referência 3.

29. Qual otimização tentaria primeiro?

Mediria chamadas sequenciais, tamanho de payload e reutilização de conexão. Eliminar um round trip desnecessário pode valer mais que trocar protocolo; escolho a alteração pelo gargalo observado e compatibilidade exigida.

Termo para lembrar: Latency budget | Base técnica: referência 4.

30. Como fecharia a análise?

Eu explicaria qual trecho usa cada protocolo, quais falhas ele recupera e o que permanece responsabilidade da aplicação. A escolha melhora transporte, enquanto idempotência, autorização e confirmação financeira continuam explícitas.

Termo para lembrar: Layered guarantees | Base técnica: referência 4.

10 trade-offs: ganho, custo e decisão

01. HTTP/2

Ganho. Multiplexação eficiente

Custo e motivo. Perda TCP afeta streams

Como decidir. Para clientes e proxies compatíveis

02. HTTP/3

Ganho. Melhora isolamento de streams

Custo e motivo. UDP e operação diferentes

Como decidir. Para caminhos que se beneficiem

03. Conexões persistentes

Ganho. Menos handshake

Custo e motivo. Drenagem e estado

Como decidir. Para tráfego recorrente

04. Muitos sockets

Ganho. Paralelismo

Custo e motivo. Portas e memória

Como decidir. Somente com capacidade medida

05. Streaming

Ganho. Memória limitada

Custo e motivo. Tratamento parcial

Como decidir. Para payload grande

06. Compressão

Ganho. Menos bytes

Custo e motivo. CPU e expansão

Como decidir. Para conteúdo compressível

07. Timeout curto

Ganho. Libera recurso

Custo e motivo. Respostas ambíguas adicionais

Como decidir. Com retry seguro

08. 0-RTT

Ganho. Menor handshake

Custo e motivo. Replay possível

Como decidir. Para operações compatíveis

09. Fallback de protocolo

Ganho. Compatibilidade

Custo e motivo. Mais caminhos de teste

Como decidir. Para redes variadas

10. Limite de streams

Ganho. Protege servidor

Custo e motivo. Pode enfileirar

Como decidir. Conforme capacidade real

Duas alternativas ao caminho proposto

1. HTTP/1.1 com pooling

Quando faz sentido. Compatibilidade é prioridade

Como implementar. Reutilizar conexões e controlar concorrência

Limites. Menos multiplexação e mais sockets

2. Mensageria assíncrona

Quando faz sentido. Resultado não precisa ser imediato

Como implementar. Persistir intenção e consumir por broker

Limites. Outro contrato de latência e acompanhamento

Referências e alcance da evidência

1. IETF - RFC 9293 TCP

Fluxo confiável de bytes, conexão e controle de transporte.

https://www.rfc-editor.org/rfc/rfc9293

2. IETF - RFC 9113 HTTP/2

Multiplexação por streams e controle HTTP/2.

https://www.rfc-editor.org/rfc/rfc9113.html

3. IETF - RFC 9114 HTTP/3

HTTP sobre QUIC, streams e limites de transporte.

https://www.rfc-editor.org/rfc/rfc9114

4. IETF - RFC 9110 HTTP Semantics

Métodos, precondições, representações e semântica HTTP.

https://www.rfc-editor.org/rfc/rfc9110.html

Continue em: T051 · T054 · T057 · T064

T053 / F. Rede e entrada

Gateway, BFF e WAF

Gateway controla entrada, BFF adapta composição à interface e WAF filtra padrões de tráfego suspeito.

Quando usar. Portais bancários, aplicativos e múltiplas interfaces com necessidades diferentes.

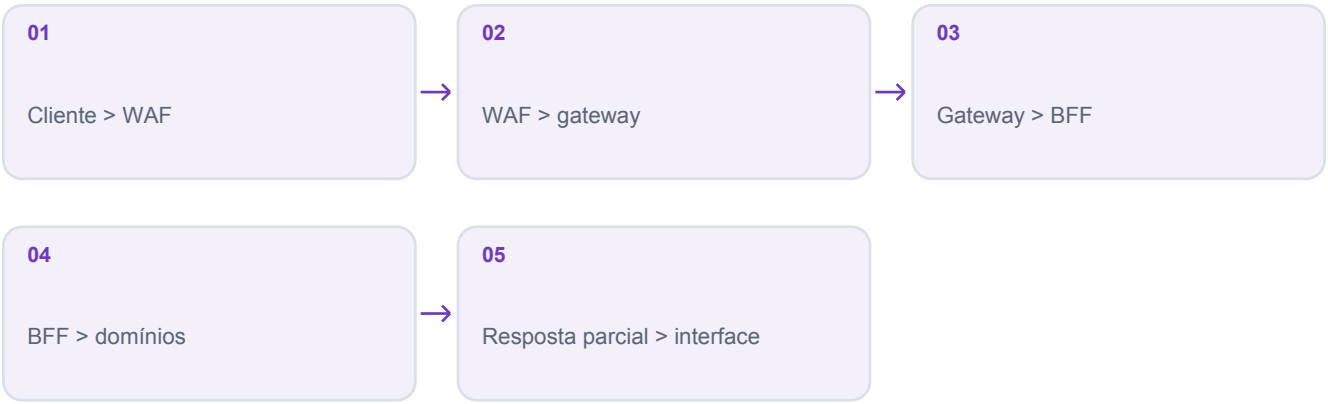
Quando evitar ou limitar. Não colocar regras financeiras no gateway nem tratar WAF como autorização de objetos.

Uma abertura possível

Vou separar políticas de entrada, composição da experiência e regras dos domínios para evitar uma camada central que decide tudo.

Peças e tecnologias. Envoy · Kong · API Gateway · AWS WAF · React · Go

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Definir contratos por interface

2. Aplicar autenticação e limites na entrada

3. Propagar identidade confiável e revalidar autorização

4. Limitar fan-out e tratar falhas parciais

5. Medir experiência e custo por composição

30 perguntas para sustentar o desenho

01. Gateway e BFF são a mesma coisa?

Não. Gateway aplica políticas de entrada e roteamento; BFF adapta dados e operações à interface. Podem coexistir, mas separar responsabilidades evita transformar regras financeiras e apresentação num único componente central.

Termo para lembrar: BFF | Base técnica: referência 1.

02. Quando criaria mais de um BFF?

Quando interfaces têm necessidades e evolução significativamente diferentes, como mobile e console operacional. Não crio um por tela; duplicação de composição só se justifica pelo ganho de autonomia e contrato adequado.

Termo para lembrar: Backend per frontend | Base técnica: referência 1.

03. O BFF pode calcular saldo oficial?

Não deveria assumir a autoridade do domínio financeiro. Ele apresenta dados e sinaliza atualização, enquanto o serviço responsável valida e calcula o saldo; duplicar essa regra cria divergência entre canais.

Termo para lembrar: Domain ownership | Base técnica: referência 1.

04. Como evita fan-out excessivo?

Defino contexto mínimo, limite de chamadas concorrentes e orçamento total. Carrego detalhes por demanda e evito consultar todos os produtos ao abrir uma tela; a composição deve respeitar capacidade das dependências.

Termo para lembrar: Bounded fan-out | Base técnica: referência 1.

05. O que retorna quando um widget falha?

Retorno estado explícito de indisponibilidade ou dado antigo com marcador temporal, preservando widgets independentes. Não converto ausência em valor zero nem deixo uma consulta opcional bloquear toda a bancada de atendimento.

Termo para lembrar: Partial response | Base técnica: referência 1.

06. WAF impede acesso a conta de outro cliente?

Não por si só. Ele pode filtrar padrões e abuso, mas autorização por recurso pertence à aplicação. Cada leitura ou comando verifica identidade, tenant e permissão sobre aquele objeto específico.

Termo para lembrar: Object authorization | Base técnica: referência 2.

07. Como trata headers de identidade?

Removo valores não confiáveis na entrada e propago identidade autenticada por canal protegido. O backend não deve aceitar um header arbitrário enviado pelo cliente como prova de usuário ou papel autorizado.

Termo para lembrar: Trusted headers | Base técnica: referência 1.

08. Autenticar no gateway dispensa domínio?

Não. Gateway pode validar credencial, mas o domínio precisa autorizar a ação no recurso e considerar estado atual. Uma sessão válida não demonstra que aquele usuário pode aprovar um reembolso específico.

Termo para lembrar: Authentication versus authorization | Base técnica: referência 1.

09. Como controla consultas GraphQL caras?

Limito profundidade, custo estimado, tamanho e concorrência conforme o contrato. Autenticação não impede consulta legitimamente formada de consumir recursos excessivos; resolvers também precisam evitar N+1 e aplicar autorização.

Termo para lembrar: Query cost | Base técnica: referência 1.

10. Cache no BFF pode vazar dados?

Pode, se a chave ou composição ignorar escopo de autorização. Cacheio fatos canônicos e autorizo depois, ou incluo contexto correto na chave; jamais compartilho uma resposta personalizada indiscriminadamente entre atendentes.

Termo para lembrar: Cache isolation | Base técnica: referência 1.

11. Como evita N+1?

Agrupo consultas compatíveis e reutilizo dados dentro da requisição, respeitando permissões. Batch reduz round trips, mas não deve transformar uma busca pequena em consulta ilimitada ou incluir recursos que o usuário não pode acessar.

Termo para lembrar: Batching | Base técnica: referência 1.

12. O gateway deve repetir POST automaticamente?

Somente com contrato de repetição seguro e identidade preservada. Um timeout pode ocorrer após efeito remoto; retries invisíveis no proxy podem duplicar comandos ou multiplicar tentativas já feitas pelo cliente.

Termo para lembrar: Proxy retry | Base técnica: referência 3.

13. Como define timeout do BFF?

Estabeleço deadline total e limites por dependência, incluindo tempo de fila. Uma chamada opcional recebe orçamento menor; não espero o timeout máximo de todos os serviços sequencialmente para montar a resposta.

Termo para lembrar: Deadline composition | Base técnica: referência 3.

14. Qual risco de regras WAF agressivas?

Falsos positivos podem bloquear clientes legítimos e fluxos bancários importantes. Uso observação inicial, rollout gradual e métricas por rota, com revisão de exceções; não desativo toda proteção para corrigir um caso.

Termo para lembrar: WAF tuning | Base técnica: referência 2.

15. WAF resolve DDoS volumétrico sozinho?

Não assumo equivalência entre filtragem de aplicação e proteção de rede. Combino camadas e capacidade apropriadas, entendendo escopo do serviço contratado; um ataque pode saturar antes de chegar à regra de aplicação.

Termo para lembrar: Defense layers | Base técnica: referência 2.

16. Como protege origem contra bypass do WAF?

Restrinjo acesso direto ao backend e aceito tráfego apenas pelos caminhos autorizados conforme infraestrutura. Caso contrário, um atacante pode contornar a filtragem acessando a origem mesmo que o domínio público esteja protegido.

Termo para lembrar: Origin protection | Base técnica: referência 2.

17. Como versiona contrato de mobile?

Prefiro evolução compatível e observo versões ainda ativas. Não exijo atualização imediata de todos os dispositivos; o BFF pode adaptar contratos, mas mantém regras e autorização consistentes nos domínios responsáveis.

Termo para lembrar: API evolution | Base técnica: referência 1.

18. Como evita acoplamento do BFF ao banco dos produtos?

Chamo contratos dos domínios em vez de consultar suas tabelas diretamente. A leitura direta pode parecer rápida, mas quebra ownership, autorização e evolução de schema, além de espalhar conhecimento interno pela interface.

Termo para lembrar: Service contract | Base técnica: referência 1.

19. Pode executar transferência entre serviços no BFF?

Ele pode iniciar uma intenção, mas a coordenação durável deve ter responsabilidade explícita. Não deixo uma sequência financeira depender da vida da requisição da tela; persisto operação e acompanho seu resultado.

Termo para lembrar: Durable command | Base técnica: referência 1.

20. Como trata mudança de cliente na interface?

Cancelo leituras desnecessárias e identifico a seleção à qual a resposta pertence. Uma resposta atrasada do cliente anterior não pode substituir dados na tela atual nem habilitar ações sobre a pessoa errada.

Termo para lembrar: Request correlation | Base técnica: referência 1.

21. O BFF precisa ser stateless?

Preferencialmente mantenho estado durável fora da instância para escalar e recuperar. Caches locais podem existir, mas a sessão ou operação não deve desaparecer porque o load balancer escolheu outra réplica.

Termo para lembrar: Stateless compute | Base técnica: referência 1.

22. Como suporta observabilidade ponta a ponta?

Propago contexto de trace e correlaciono a interação com chamadas de domínio, sem registrar dados sensíveis desnecessários. Meço tempo útil da tela e falhas parciais, não apenas latência média do gateway.

Termo para lembrar: Distributed tracing | Base técnica: referência 1.

23. Como decide fallback?

Aceito fallback somente quando o significado permanece correto. Histórico pode mostrar dado antigo identificado; autorização ou saldo disponível para débito não deve usar informação desatualizada apenas para manter uma resposta verde.

Termo para lembrar: Semantic fallback | Base técnica: referência 1.

24. Qual limite de payload na entrada?

Defino por rota e operação, incluindo tamanho descompactado e número de itens. O gateway rejeita cedo quando possível, mas o backend também valida porque clientes internos e rotas alternativas podem existir.

Termo para lembrar: Payload admission | Base técnica: referência 2.

25. Como faz deploy de regra de roteamento?

Versiono configuração, valido sintaxe e contratos e libero gradualmente com métricas de erro e latência. Um gateway central pode afetar muitos serviços; trato mudanças de configuração com rigor semelhante ao código.

Termo para lembrar: Configuration rollout | Base técnica: referência 1.

26. Como evita ponto único de falha?

Replico a camada e isolo configurações ou rotas quando necessário, mas verifico dependências compartilhadas como identidade e limites. Muitas réplicas com a mesma configuração defeituosa ainda podem falhar simultaneamente.

Termo para lembrar: Failure domain | Base técnica: referência 1.

27. Como protege ações administrativas?

Exijo autorização específica, motivo quando necessário e auditoria no domínio. Esconder botão no frontend não é controle suficiente; o BFF não deve aceitar papel ou identidade declarados livremente no corpo da requisição.

Termo para lembrar: Server-side authorization | Base técnica: referência 1.

28. Quando não usar BFF?

Se uma única interface usa contratos simples e estáveis, outra camada pode acrescentar latência e manutenção sem benefício. Começo com API adequada e introduzo BFF quando composição e autonomia justificarem o custo.

Termo para lembrar: Architectural fit | Base técnica: referência 1.

29. Como testa a composição?

Simulo serviço lento, resposta parcial, permissão revogada e payload grande. Verifico que a interface permanece coerente e que ações sensíveis continuam bloqueadas corretamente; sucesso de cada API isolada não comprova a composição.

Termo para lembrar: Failure composition test | Base técnica: referência 1.

30. Como defenderia o desenho?

Eu mostraria qual política pertence ao gateway, qual transformação ao BFF e qual regra ao domínio. Depois explicaria falha parcial e autorização por recurso, evitando depender de uma única camada para toda segurança.

Termo para lembrar: Responsibility boundary | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. BFF por interface

Ganho. Contrato adequado

Custo e motivo. Duplicação de composição

Como decidir. Quando interfaces divergem

02. API única

Ganho. Menos componentes

Custo e motivo. Contrato genérico pesado

Como decidir. Para necessidades semelhantes

03. Agregação no servidor

Ganho. Menos round trips do cliente

Custo e motivo. Fan-out central

Como decidir. Com orçamento de dependências

04. Carregamento por demanda

Ganho. Menor carga inicial

Custo e motivo. Mais estados de UI

Como decidir. Para contexto opcional

05. WAF rigoroso

Ganho. Bloqueia abuso

Custo e motivo. Falsos positivos

Como decidir. Com rollout e métricas

06. Cache de resposta

Ganho. Latência menor

Custo e motivo. Risco de escopo incorreto

Como decidir. Com chave e autorização corretas

07. Fallback antigo

Ganho. Preserva contexto

Custo e motivo. Dado desatualizado

Como decidir. Somente para leitura tolerante

08. Gateway central

Ganho. Políticas uniformes

Custo e motivo. Blast radius de configuração

Como decidir. Com isolamento e canary

09. Validação no domínio

Ganho. Protege todos os caminhos

Custo e motivo. Alguma redundância

Como decidir. Para regras críticas

10. Comando assíncrono

Ganho. Sobrevive à conexão

Custo e motivo. UX de pendência

Como decidir. Para operação longa

Duas alternativas ao caminho proposto

1. API de domínio direta

Quando faz sentido. Interface simples

Como implementar. Cliente chama poucos contratos estáveis

Limites. Maior acoplamento de frontend aos produtos

2. GraphQL com composição governada

Quando faz sentido. Múltiplas fontes e consultas flexíveis

Como implementar. Resolvers autorizados com limites de custo

Limites. Complexidade de execução e N+1 precisam de controle

Referências e alcance da evidência

1. Microsoft - Backends for Frontends

Composição de backend adaptada a necessidades de interfaces diferentes.

https://learn.microsoft.com/en-us/azure/architecture/patterns/backends-for-frontends

2. AWS - AWS WAF

Filtragem de requisições web e escopo de proteção por regras.

https://docs.aws.amazon.com/waf/latest/developerguide/what-is-aws-waf.html

3. AWS - Timeouts retries and jitter

Timeouts, backoff, jitter e risco de repetir efeitos após respostas perdidas.

https://aws.amazon.com/builders-library/timeouts-retries-and-backoff-with-jitter/

Continue em: T055 · T056 · T071 · T100

T054 / F. Rede e entrada

Load balancing e sticky sessions

Load balancing distribui tráfego; afinidade mantém preferência por destino sem garantir durabilidade de sessão.

Quando usar. Serviços replicados, pools heterogêneos e conexões longas.

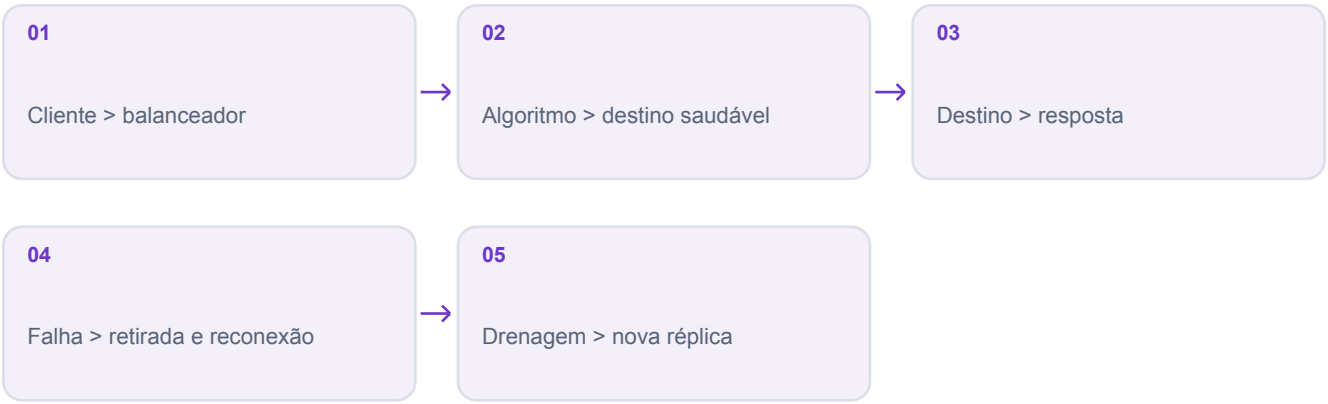
Quando evitar ou limitar. Não depender de sticky sessions para recuperar estado crítico nem confundir health check com sucesso do negócio.

Uma abertura possível

Vou escolher o algoritmo pelo custo das requisições e mostrar o que acontece quando o destino escolhido desaparece.

Peças e tecnologias. ALB · NLB · Envoy · HAProxy · Nginx

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Medir distribuição e custo de requisições

2. Escolher L4 ou L7 e algoritmo

3. Definir checks e drenagem

4. Externalizar estado necessário à retomada

5. Testar perda de instância e redistribuição

30 perguntas para sustentar o desenho

01. Round-robin é suficiente para qualquer carga?

Não. Ele distribui requisições de forma simples, mas ignora duração e custo diferentes. Se algumas chamadas ocupam recursos por muito tempo, comparo algoritmos sensíveis a carga e limites locais de concorrência.

Termo para lembrar: Round robin | Base técnica: referência 1.

02. Quando least outstanding requests ajuda?

Quando requisições têm duração variável e o número em andamento aproxima carga. Ainda verifico custo real: uma consulta pesada pode consumir muito mais recursos que várias leituras pequenas simultâneas.

Termo para lembrar: Least outstanding | Base técnica: referência 1.

03. Least connections equivale a menos trabalho?

Não necessariamente. HTTP/2 pode multiplexar muitos streams numa conexão, enquanto outra está ociosa. Escolho métrica adequada ao protocolo e observo saturação do destino, não apenas contagem bruta de sockets.

Termo para lembrar: Connection versus request | Base técnica: referência 1.

04. Como balanceia WebSocket?

A escolha inicial determina a conexão persistente; mensagens seguintes usam aquele canal. Para redistribuir, preciso reconexão ou migração explícita, e o estado recuperável não pode depender apenas da memória daquele gateway.

Termo para lembrar: Long-lived connection | Base técnica: referência 1.

05. Sticky session garante disponibilidade?

Não. Ela mantém preferência enquanto o destino existe, mas uma queda pode remover a sessão local. Persisto estado necessário fora da instância ou implemento retomada; afinidade é otimização, não plano de recuperação.

Termo para lembrar: Session affinity | Base técnica: referência 1.

06. Por que afinidade pode desequilibrar carga?

Clientes de custo desigual ficam presos a destinos diferentes. Alguns servidores podem saturar enquanto outros ficam ociosos; monitoro distribuição e uso afinidade somente quando o benefício supera essa perda de flexibilidade.

Termo para lembrar: Hot target | Base técnica: referência 1.

07. Como escolhe L4 ou L7?

L4 distribui por conexão e tem menor conhecimento da aplicação; L7 pode rotear por host, caminho e políticas HTTP. Escolho conforme protocolo, necessidade de inspeção e custo, mantendo segurança em cada trecho.

Termo para lembrar: L4 versus L7 | Base técnica: referência 1.

08. Health check deve consultar todas as dependências?

Não indiscriminadamente. Se uma dependência compartilhada falhar, retirar todas as réplicas pode piorar a situação. Defino prontidão mínima e degradação por função, distinguindo processo vivo e capacidade útil de atendimento.

Termo para lembrar: Readiness | Base técnica: referência 1.

09. Qual diferença entre liveness e readiness?

Liveness indica se reiniciar pode ajudar; readiness decide se deve receber tráfego. Misturar os dois pode causar reinícios em cascata durante falha externa, quando o processo está saudável mas uma dependência não.

Termo para lembrar: Probe semantics | Base técnica: referência 1.

10. Como evita oscilação de health check?

Uso thresholds, intervalos e critérios que tolerem variação pequena sem esconder falha real. Observo tempo de retirada e reinserção; checks agressivos demais podem alternar destinos e causar instabilidade no tráfego.

Termo para lembrar: Health hysteresis | Base técnica: referência 1.

11. O que é slow start?

É introduzir capacidade nova gradualmente para permitir aquecimento de cache e runtime. Enviar carga total imediatamente pode derrubar uma réplica recém-iniciada; verifico suporte do algoritmo e comportamento do produto.

Termo para lembrar: Slow start | Base técnica: referência 1.

12. Como faz connection draining?

Paro novas atribuições e permito concluir trabalho existente dentro de um prazo. Conexões longas exigem estratégia de reconexão; desligar processo imediatamente após removê-lo do pool ainda pode interromper chamadas em andamento.

Termo para lembrar: Deregistration delay | Base técnica: referência 1.

13. O balanceador pode repetir requisição perdida?

Pode conforme configuração, mas comandos exigem contrato seguro. Se o backend executou antes de perder resposta, retry em outro destino pode repetir efeito; preservo identidade e controlo tentativas entre proxy e aplicação.

Termo para lembrar: Proxy retry | Base técnica: referência 2.

14. Como trata cookie de afinidade?

Defino escopo, duração e proteção compatíveis com a aplicação. O cookie direciona tráfego, mas não deve ser usado como autenticação; roubar ou editar afinidade não pode conceder acesso aos dados da sessão.

Termo para lembrar: Affinity cookie | Base técnica: referência 1.

15. Hash consistente resolve sessão durável?

Não. Ele reduz remapeamento quando destinos mudam, mas não replica o estado nem recupera um servidor perdido. Uso para localidade ou cache, mantendo uma autoridade durável quando o estado importa.

Termo para lembrar: Consistent hashing | Base técnica: referência 1.

16. Como pondera máquinas de capacidades diferentes?

Uso pesos baseados em capacidade medida e observo saturação. Pesos estáticos podem envelhecer com mudança de carga; mantenho limites locais e ajusto gradualmente para evitar transferir sobrecarga entre destinos.

Termo para lembrar: Weighted balancing | Base técnica: referência 1.

17. O que é power of two choices?

Escolho dois destinos candidatos e comparo uma métrica de carga, reduzindo custo de coordenação com boa distribuição em muitos cenários. Ainda preciso definir métrica correta e comportamento para destinos indisponíveis.

Termo para lembrar: P2C | Base técnica: referência 3.

18. Balanceamento entre zonas sempre melhora?

Pode aumentar disponibilidade e uso de capacidade, mas adiciona tráfego entre zonas e dependências. Dimensiono cada zona para o cenário de falha escolhido e avalio custos e limites do serviço utilizado.

Termo para lembrar: Cross-zone balancing | Base técnica: referência 1.

19. DNS é balanceador por requisição?

Não. Resolução e cache influenciam escolha de endereço, mas conexões podem durar e clientes reutilizam respostas. Para controle fino por requisição, uso uma camada apropriada e considero convergência real do DNS.

Termo para lembrar: DNS balancing | Base técnica: referência 1.

20. Como evita que uma zona falha sature as outras?

Reservo margem e aplico admission control durante redistribuição. Remover destinos sem capacidade sobrevivente suficiente desloca a falha; alta disponibilidade exige dimensionar o cenário degradado, não só o normal.

Termo para lembrar: Failover capacity | Base técnica: referência 1.

21. Por que sessão externa melhora operação?

Outra réplica pode recuperar contexto após mudança de destino. O armazenamento vira dependência própria e precisa de disponibilidade, expiração e autorização; externalizar estado não remove custos, mas torna a retomada explícita.

Termo para lembrar: External session store | Base técnica: referência 1.

22. Como lida com IP de origem atrás de proxies?

Confio somente em headers de encaminhamento inseridos por proxies autorizados e removo valores externos falsos. Caso contrário rate limiting, auditoria e políticas por IP podem ser manipulados pelo cliente.

Termo para lembrar: Trusted forwarding | Base técnica: referência 1.

23. Um destino saudável pode estar sobrecarregado?

Sim. Um check simples pode responder enquanto filas internas crescem. Uso métricas de saturação e limites de admissão, além de saúde binária, para evitar continuar encaminhando trabalho que não terminará no prazo.

Termo para lembrar: Load-aware routing | Base técnica: referência 1.

24. Como evita retry storm após failover?

Uso orçamento de retry, jitter e retomada gradual. Milhares de clientes reconectando simultaneamente podem saturar o destino sobrevivente; o balanceador e os clientes precisam compartilhar uma política coerente de recuperação.

Termo para lembrar: Reconnect storm | Base técnica: referência 2.

25. Como separa tráfego administrativo e público?

Uso rotas, pools ou limites independentes conforme risco e capacidade. Isso evita que pico público elimine ferramentas de recuperação, mas mantém autenticação e autorização fortes também no caminho administrativo.

Termo para lembrar: Bulkhead routing | Base técnica: referência 1.

26. Como decide timeout de conexão ociosa?

Alinho cliente, proxy e servidor ao padrão de tráfego, incluindo WebSocket. Diferenças podem gerar resets inesperados; heartbeats devem ser baratos e não esconder conexões sem utilidade indefinidamente.

Termo para lembrar: Idle timeout | Base técnica: referência 1.

27. Qual teste demonstra distribuição correta?

Misturo requisições rápidas e lentas, máquinas diferentes e quedas. Meço carga, latência e erros por destino; apenas contar requisições iguais pode esconder desequilíbrio quando os custos são heterogêneos.

Termo para lembrar: Load distribution test | Base técnica: referência 1.

28. Como migra de algoritmo?

Faço rollout limitado, comparo p99, saturação e distribuição e mantenho reversão. Trocar algoritmo pode alterar localidade de cache e afinidade; considero esse efeito antes de atribuir regressão ao backend.

Termo para lembrar: Algorithm rollout | Base técnica: referência 1.

29. Quando manteria round-robin?

Quando destinos e requisições são relativamente homogêneos e as métricas mostram boa distribuição. Simplicidade é valiosa; um algoritmo mais complexo só se justifica se resolver um desequilíbrio observado ou requisito claro.

Termo para lembrar: Algorithm fit | Base técnica: referência 1.

30. Como defenderia sua escolha?

Eu relacionaria algoritmo ao custo das chamadas, explicaria saúde e drenagem e demonstraria retomada sem depender de afinidade. O objetivo é distribuir trabalho útil mantendo capacidade durante a perda de destinos.

Termo para lembrar: Balancing contract | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Round-robin

Ganho. Simplicidade

Custo e motivo. Ignora custo variável

Como decidir. Para carga homogênea

02. Least outstanding

Ganho. Considera trabalho ativo

Custo e motivo. Métrica imperfeita

Como decidir. Para duração variável

03. Afinidade

Ganho. Localidade de cache

Custo e motivo. Desequilíbrio e dependência

Como decidir. Quando benefício é medido

04. Sem afinidade

Ganho. Redistribuição flexível

Custo e motivo. Estado externo necessário

Como decidir. Para serviços recuperáveis

05. L4

Ganho. Menos inspeção

Custo e motivo. Pouca semântica HTTP

Como decidir. Para protocolos gerais

06. L7

Ganho. Roteamento e políticas

Custo e motivo. Mais processamento

Como decidir. Para APIs HTTP

07. Slow start

Ganho. Aquecimento seguro

Custo e motivo. Capacidade entra devagar

Como decidir. Para runtime e cache frios

08. Drenagem longa

Ganho. Preserva conexões

Custo e motivo. Deploy mais lento

Como decidir. Para requisições longas

09. Cross-zone

Ganho. Mais flexibilidade

Custo e motivo. Custo e tráfego

Como decidir. Conforme modelo de falha

10. Checks profundos

Ganho. Detecta dependência

Custo e motivo. Pode retirar todos

Como decidir. Somente com semântica de prontidão clara

Duas alternativas ao caminho proposto

1. Balanceamento no cliente

Quando faz sentido. Clientes controlados e descoberta madura

Como implementar. Selecionar destinos e aplicar limites na biblioteca

Limites. Distribui complexidade e versões pelo parque

2. Fila de trabalho

Quando faz sentido. Processamento não precisa de resposta imediata

Como implementar. Consumidores competem por jobs duráveis

Limites. Muda contrato de latência e acompanhamento

Referências e alcance da evidência

1. AWS - ALB target group attributes

Algoritmos, afinidade, drenagem e atributos de target groups.

https://docs.aws.amazon.com/elasticloadbalancing/latest/application/edit-target-group-attributes.html

2. AWS - Timeouts retries and jitter

Timeouts, backoff, jitter e risco de repetir efeitos após respostas perdidas.

https://aws.amazon.com/builders-library/timeouts-retries-and-backoff-with-jitter/

3. Envoy - Supported load balancers

Algoritmos de balanceamento, least request, P2C e hashing.

https://www.envoyproxy.io/docs/envoy/latest/intro/arch_overview/upstream/load_balancing/load_balancers

Continue em: T051 · T052 · T055 · T057

T055 / F. Rede e entrada

Traffic shaping e rate limiting

Rate limiting controla admissões por taxa ou custo; traffic shaping suaviza o ritmo de saída.

Quando usar. Proteção de APIs, fairness entre tenants e quotas rígidas de parceiros.

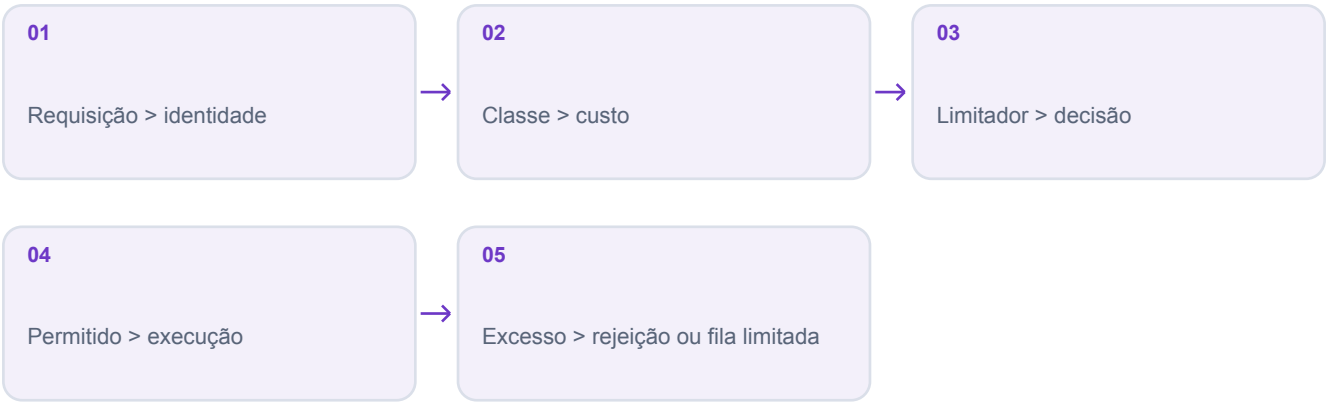
Quando evitar ou limitar. Não tratar limites por instância como quota global nem substituir controle de concorrência por RPS.

Uma abertura possível

Vou definir quem consome a quota, qual janela vale e se exceder significa recusar, adiar ou degradar.

Peças e tecnologias. Envoy · Redis · Token bucket · Leaky bucket · API Gateway

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Definir escopo, janela e custo

2. Escolher algoritmo de contagem

3. Coordenar quota global ou dividir orçamento

4. Definir falha do limitador e resposta

5. Testar rajadas, concorrência e recuperação

30 perguntas para sustentar o desenho

01. Qual problema resolve token bucket?

Ele permite rajadas limitadas sem perder controle da taxa média de reposição. Defino capacidade do balde e custo por operação; um balde grande demais pode ultrapassar a capacidade instantânea do destino.

Termo para lembrar: Token bucket | Base técnica: referência 1.

02. Como leaky bucket difere?

Ele suaviza saída para uma vazão controlada, normalmente com fila limitada. Isso aumenta espera; uso quando o parceiro tolera pouca rajada e rejeito ou adio explicitamente trabalho que não cabe no prazo.

Termo para lembrar: Leaky bucket | Base técnica: referência 2.

03. Qual falha da janela fixa?

Requisições no fim de uma janela e começo da seguinte podem produzir rajada maior no intervalo curto. Se isso violar capacidade, uso janela deslizante ou token bucket com burst explicitamente limitado.

Termo para lembrar: Fixed window | Base técnica: referência 1.

04. Janela deslizante exata custa o quê?

Guardar cada ocorrência permite precisão, mas aumenta memória e operação por chave. Aproximações por buckets reduzem custo com erro controlado; escolho conforme severidade da quota, não por preferência abstrata de algoritmo.

Termo para lembrar: Sliding window | Base técnica: referência 1.

05. Quem deve ser a chave do limite?

Depende do objetivo: usuário, tenant, credencial, IP ou parceiro. IP sozinho pode agrupar clientes legítimos atrás de NAT; também combino limites para impedir que múltiplas identidades contornem proteção global.

Termo para lembrar: Rate-limit key | Base técnica: referência 1.

06. Como limita operações de custos diferentes?

Atribuo pesos por tipo, quantidade de itens ou tamanho estimado. Uma consulta simples e um relatório massivo não devem necessariamente consumir um token igual; valido estimativa com métricas de custo real.

Termo para lembrar: Weighted tokens | Base técnica: referência 2.

07. Por que RPS não protege contra serviço lento?

Com latência maior, a mesma taxa acumula mais chamadas simultâneas. Uso limite de concorrência junto da taxa para proteger conexões e memória; um contador por segundo não captura ocupação prolongada.

Termo para lembrar: Concurrency control | Base técnica: referência 2.

08. Como impõe quota global com dez réplicas?

Uso autoridade compartilhada ou distribuição conservadora de orçamento. Limite igual em cada réplica multiplica a quota; a escolha precisa explicitar precisão, disponibilidade e comportamento quando a coordenação fica indisponível.

Termo para lembrar: Global quota | Base técnica: referência 1.

09. Redis INCR sozinho é suficiente?

Preciso tratar atualização, janela e expiração atomicamente conforme algoritmo. Separar incremento e configuração de TTL pode deixar chave sem expiração em uma falha; também avalio durabilidade exigida pela quota financeira.

Termo para lembrar: Atomic counter | Base técnica: referência 1.

10. Pode perder quota numa promoção de réplica?

Pode conforme persistência e replicação do armazenamento. Se o limite é contratualmente rígido, avalio essa perda de estado e uso autoridade apropriada; um cache com failover não garante contabilidade perfeita automaticamente.

Termo para lembrar: Quota durability | Base técnica: referência 1.

11. Quando falhar aberto?

Para certas leituras de baixo risco, posso usar limite local conservador quando a autoridade falha. Para controle financeiro ou proteção rígida do parceiro, falhar aberto pode ser inaceitável; defino a política por operação.

Termo para lembrar: Fail-open policy | Base técnica: referência 1.

12. Quando falhar fechado?

Quando exceder a quota ameaça dinheiro, segurança ou contrato essencial. Aceito indisponibilidade temporária e apresento estado correto; ainda preparo redundância e recuperação para que o limitador não vire dependência frágil desnecessária.

Termo para lembrar: Fail-closed policy | Base técnica: referência 1.

13. O que responde ao cliente limitado?

Uso status e contrato adequados, com orientação de espera quando possível. O cliente aplica jitter e preserva identidade de comandos; repetir imediatamente com nova chave pode agravar carga e duplicar intenções.

Termo para lembrar: 429 response | Base técnica: referência 2.

14. Rate limit é igual a quota mensal?

Não. Rate limit protege ritmo; quota mensal contabiliza consumo acumulado. Posso precisar dos dois, com períodos, reservas e reconciliação diferentes; um burst permitido não autoriza ultrapassar o orçamento total contratado.

Termo para lembrar: Rate versus quota | Base técnica: referência 1.

15. Como contabiliza tentativa externa que deu timeout?

Defino consumo conforme contrato do parceiro. Se a tentativa pode ter chegado, não devolvo quota automaticamente por falta de resposta; isso permitiria exceder transmissões enquanto o destino realmente as recebeu.

Termo para lembrar: Attempt accounting | Base técnica: referência 3.

16. O que significa reservar quota?

Separo concessão de orçamento e execução, com identidade e política de expiração. Reservas abandonadas exigem recuperação; devolver uma reserva enquanto a chamada ainda pode ocorrer cria risco de ultrapassar o limite.

Termo para lembrar: Quota reservation | Base técnica: referência 3.

17. Como evita um tenant monopolizar capacidade?

Distribuo orçamento mínimo ou pesos entre classes e monitoro uso. Um limite global protege o serviço, mas pode ser consumido inteiro por um único tenant; fairness requer política adicional.

Termo para lembrar: Tenant fairness | Base técnica: referência 2.

18. Prioridade deve ignorar o limite?

Não automaticamente. Reservo capacidade para operações críticas, mas continuo protegendo o destino. Permitir que toda requisição se declare prioritária anula a política; a classificação precisa vir de contexto confiável.

Termo para lembrar: Priority admission | Base técnica: referência 2.

19. Como impede explosão de chaves?

Limito cardinalidade, retenção e escopos aceitos. Um atacante pode gerar identificadores aleatórios para consumir memória do limitador; identidade autenticada e políticas de criação evitam que o mecanismo de proteção vire alvo.

Termo para lembrar: Cardinality attack | Base técnica: referência 1.

20. Qual papel do relógio no algoritmo?

Janelas e reposição precisam de tempo consistente no escopo da autoridade. Evito relógios arbitrários de clientes e considero skew; mudanças de tempo não podem gerar tokens extras ou prender quota indefinidamente.

Termo para lembrar: Clock discipline | Base técnica: referência 1.

21. Como usa limite local e global juntos?

O local corta rajadas antes de chamar a autoridade global; o global coordena orçamento entre instâncias. Calibro ambos para evitar rejeições desnecessárias e documento qual garante o contrato rígido.

Termo para lembrar: Hierarchical limiting | Base técnica: referência 1.

22. Como limita retries separadamente?

Reservo um orçamento de recuperação que compartilha capacidade com tráfego novo. Sem isso, retries podem consumir toda a quota e impedir operações saudáveis; observo taxa de sucesso recuperado por tentativa adicional.

Termo para lembrar: Retry budget | Base técnica: referência 3.

23. Fila é melhor que rejeição?

Depende do prazo e contrato. Enfileirar suaviza carga, mas cria espera e obrigação durável; rejeitar cedo permite ao cliente decidir. Não aceito uma fila que matematicamente não conseguirá cumprir o prazo.

Termo para lembrar: Shaping versus rejection | Base técnica: referência 2.

24. Como trata mudança de plano do cliente?

Versiono a política e aplico transição coerente ao consumo já registrado. Uma redução de limite pode exigir bloquear novas operações sem apagar histórico; uma atualização não deve reiniciar quota inadvertidamente.

Termo para lembrar: Policy versioning | Base técnica: referência 1.

25. Como testa a precisão do algoritmo?

Disparo chamadas concorrentes em limites de janela e simulo falha do armazenamento. Confiro concessões totais, não somente respostas de uma instância; testes precisam incluir clock skew, expiração e múltiplas réplicas.

Termo para lembrar: Boundary testing | Base técnica: referência 1.

26. Como evita overshoot ao dividir orçamento?

Distribuo cotas cuja soma respeita o total e controlo realocação com versões. Aceito capacidade ociosa como custo possível de precisão; concessões independentes sem contabilização podem somar acima da quota.

Termo para lembrar: Budget allocation | Base técnica: referência 1.

27. Pode usar rate limit para autenticação?

Ele ajuda a reduzir abuso, mas não substitui validação de credenciais ou autorização. Também considero distribuição de ataques e usuários atrás de NAT, evitando bloquear toda uma rede legítima por uma tentativa maliciosa.

Termo para lembrar: Abuse control | Base técnica: referência 1.

28. Quais métricas acompanho?

Concessões, recusas, consumo por classe, latência do limitador, falhas e saturação do destino. Rejeição baixa não prova segurança se a quota estiver sendo ultrapassada durante failover ou uso de múltiplas instâncias.

Termo para lembrar: Limiter telemetry | Base técnica: referência 1.

29. Como atualiza limites sem instabilidade?

Uso configuração versionada e rollout gradual, observando resposta do sistema. Reduzir abruptamente pode criar tempestade de retries; comunico contrato e mantenho comportamento previsível para clientes e operadores.

Termo para lembrar: Policy rollout | Base técnica: referência 1.

30. Como defenderia o desenho?

Eu apontaria a autoridade da quota, o algoritmo, a unidade contada e o comportamento sob falha. Depois mostraria rajada e timeout externo, porque esses casos revelam se o limite realmente protege o contrato.

Termo para lembrar: Quota proof | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Janela fixa

Ganho. Baixo custo

Custo e motivo. Rajada na fronteira

Como decidir. Para limites aproximados

02. Janela deslizante exata

Ganho. Precisão temporal

Custo e motivo. Memória por ocorrência

Como decidir. Para volume compatível

03. Buckets aproximados

Ganho. Menor armazenamento

Custo e motivo. Erro de aproximação

Como decidir. Quando contrato tolera

04. Token bucket

Ganho. Permite burst limitado

Custo e motivo. Rajada exige capacidade

Como decidir. Para APIs interativas

05. Leaky bucket

Ganho. Saída uniforme

Custo e motivo. Espera adicional

Como decidir. Para parceiro sensível

06. Autoridade global

Ganho. Quota coordenada

Custo e motivo. Latência e dependência

Como decidir. Para contrato rígido

07. Cotas locais

Ganho. Baixa latência

Custo e motivo. Ociosidade ou overshoot

Como decidir. Com distribuição controlada

08. Falhar aberto

Ganho. Preserva disponibilidade

Custo e motivo. Pode exceder proteção

Como decidir. Para baixo risco

09. Falhar fechado

Ganho. Preserva limite

Custo e motivo. Recusa legítimos

Como decidir. Para risco financeiro

10. Pesos por custo

Ganho. Admissão mais justa

Custo e motivo. Estimativa e configuração

Como decidir. Para carga heterogênea

Duas alternativas ao caminho proposto

1. Semáforo de concorrência

Quando faz sentido. Risco principal é trabalho em voo

Como implementar. Limitar chamadas simultâneas por dependência

Limites. Não garante quota por período

2. Fila com despacho controlado

Quando faz sentido. Trabalho pode aguardar

Como implementar. Persistir intenção e liberar na taxa permitida

Limites. Exige prazo, retenção e limite de backlog

Referências e alcance da evidência

1. Envoy - Global rate limiting

Rate limiting global e cooperação com políticas locais.

https://www.envoyproxy.io/docs/envoy/latest/intro/arch_overview/other_features/global_rate_limiting

2. Google SRE - Handling Overload

Admissão de carga, degradação e prevenção de sobrecarga em cascata.

https://sre.google/sre-book/handling-overload/

3. AWS - Timeouts retries and jitter

Timeouts, backoff, jitter e risco de repetir efeitos após respostas perdidas.

https://aws.amazon.com/builders-library/timeouts-retries-and-backoff-with-jitter/

Continue em: T045 · T050 · T054 · T064

T056 / F. Rede e entrada

CDN, ETag e cache HTTP

Cache HTTP usa diretivas e validadores para reutilizar representações; CDN aproxima conteúdo e reduz carga da origem.

Quando usar. Assets, documentos públicos e leituras que admitem política explícita de cache.

Quando evitar ou limitar. Não cachear dados bancários personalizados em chave compartilhada nem confundir no-cache com no-store.

Uma abertura possível

Vou definir quem pode armazenar a resposta, por quanto tempo e qual chave identifica a representação autorizada.

Peças e tecnologias. CloudFront · Azure Front Door · CDN · ETag · Cache-Control · Vary

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Classificar conteúdo público e privado

2. Definir chave de cache e diretivas

3. Versionar assets e emitir validadores

4. Planejar invalidação e fallback permitido

5. Testar vazamento entre usuários e carga de misses

30 perguntas para sustentar o desenho

01. Qual diferença entre no-cache e no-store?

No-cache permite armazenamento, mas exige revalidação conforme contrato antes de reutilizar. No-store instrui a não armazenar. Para dados sensíveis, escolho diretiva correta e não confio nela como única medida de segurança.

Termo para lembrar: Cache-Control | Base técnica: referência 1.

02. O que ETag identifica?

Ele é um validador da representação conforme a origem. Uso em requisições condicionais para evitar retransmitir conteúdo igual; sua geração e força precisam corresponder ao significado de equivalência exigido pelo recurso.

Termo para lembrar: ETag | Base técnica: referência 2.

03. Como funciona resposta 304?

O cliente envia validador e a origem confirma que pode reutilizar a representação armazenada, sem enviar o corpo completo. Isso economiza bytes, mas ainda envolve requisição e validação, não um hit puramente local.

Termo para lembrar: Conditional GET | Base técnica: referência 2.

04. ETag pode proteger atualização concorrente?

Pode participar de precondição como If-Match. O servidor valida contra a versão autoritativa antes de alterar; cache ou navegador não decide sozinho se o objeto continua na versão observada pelo usuário.

Termo para lembrar: If-Match | Base técnica: referência 2.

05. Quando usar ETag fraco?

Quando equivalência semântica de representações é suficiente para a finalidade permitida. Não uso validador fraco onde comparação forte é necessária; separo cache de leitura e precondições de escrita conforme semântica HTTP.

Termo para lembrar: Weak validator | Base técnica: referência 2.

06. O que entra na cache key?

Incluo dimensões que mudam a representação, como caminho, query relevante e headers autorizados. Ignorar idioma ou escopo pode servir conteúdo errado; incluir valores desnecessários destrói hit rate e aumenta cardinalidade.

Termo para lembrar: Cache key | Base técnica: referência 1.

07. Para que serve Vary?

Ele indica quais campos da requisição influenciam a representação. Configuração incorreta pode misturar respostas ou criar muitas variantes; testo a CDN e a origem juntas em vez de assumir comportamento idêntico.

Termo para lembrar: Vary | Base técnica: referência 1.

08. Pode cachear resposta autenticada numa CDN?

Somente com política explícita e isolamento adequado. Para dados bancários personalizados, prefiro evitar cache compartilhado sem necessidade clara; token válido não torna uma resposta segura para qualquer usuário que compartilhe a URL.

Termo para lembrar: Shared cache isolation | Base técnica: referência 1.

09. private significa criptografado?

Não. É uma diretiva de armazenamento para caches, não criptografia. TLS, autorização e proteção do dispositivo continuam necessários; palavras semelhantes no protocolo não devem ser confundidas com garantias de confidencialidade completas.

Termo para lembrar: Private cache | Base técnica: referência 1.

10. Como versiona arquivos estáticos?

Uso nomes com hash do conteúdo e política longa para assets imutáveis. O HTML que referencia esses arquivos tem atualização controlada; isso reduz necessidade de invalidar o mesmo caminho a cada deploy.

Termo para lembrar: Content addressing | Base técnica: referência 1.

11. Por que não sobrescrever app.js com TTL enorme?

Clientes e CDNs podem continuar usando a versão antiga, criando incompatibilidade com backend ou outros assets. Prefiro URL versionada e compatibilidade de rollout, sem depender de purge instantâneo em todos os caches.

Termo para lembrar: Immutable asset | Base técnica: referência 1.

12. Purge garante atualização imediata global?

Não assumo instantaneidade em todos os clientes e camadas. Planejo propagação e versões compatíveis; invalidação da CDN não apaga automaticamente cache do navegador nem arquivos já carregados em sessões longas.

Termo para lembrar: Invalidation scope | Base técnica: referência 1.

13. Como trata cache stampede na origem?

Uso coalescing, jitter de expiração e limites de concorrência quando disponíveis. Muitos misses simultâneos podem saturar a origem; a CDN reduz carga normal, mas não dispensa proteção para cold start.

Termo para lembrar: Request coalescing | Base técnica: referência 1.

14. O que acontece quando a CDN cai?

Defino comportamento e capacidade da origem para o cenário escolhido. Bypass irrestrito pode derrubar o backend; redundância ou degradação precisam ser testadas, considerando que o tráfego sem cache é muito maior.

Termo para lembrar: Origin capacity | Base técnica: referência 1.

15. Pode servir stale durante falha?

Quando o contrato de leitura permitir e a resposta indicar contexto adequado. Não uso informação antiga para autorizar débito ou acesso revogado; fallback de conteúdo precisa preservar significado e segurança do domínio.

Termo para lembrar: Stale response | Base técnica: referência 1.

16. TTL do saldo pode ser longo?

Depende da experiência e uso. Exibir saldo antigo identificado pode ser aceitável em consulta específica, mas a decisão de gastar requer validação autoritativa; cache de leitura não define disponibilidade financeira real.

Termo para lembrar: Read versus command | Base técnica: referência 1.

17. Como evita vazamento entre tenants?

Testo cache key, cookies, headers e autorização com usuários distintos. Uma configuração que remove token da chave sem separar conteúdo pode servir dados de outro tenant; isolamento precisa ser comprovado no caminho completo.

Termo para lembrar: Tenant isolation | Base técnica: referência 1.

18. Como protege origem de acesso direto?

Restrinjo origem ao caminho autorizado e mantenho autenticação apropriada. Caso contrário, políticas da CDN ou WAF podem ser contornadas; cache na borda não substitui controle de entrada do serviço originador.

Termo para lembrar: Origin access | Base técnica: referência 1.

19. Qual efeito de query strings irrelevantes?

Podem fragmentar o cache e reduzir hit rate, ou ser usadas para forçar misses. Normalizo somente parâmetros sem significado para a representação; remover parâmetros relevantes pode servir conteúdo incorreto ou inseguro.

Termo para lembrar: Cache fragmentation | Base técnica: referência 1.

20. O que é cache poisoning?

É influenciar uma resposta armazenada para que outros recebam conteúdo indevido. Valido entradas e alinho a chave com geração da resposta; headers não considerados na chave não devem alterar conteúdo compartilhado de forma perigosa.

Termo para lembrar: Cache poisoning | Base técnica: referência 1.

21. Respostas de erro devem ser cacheadas?

Somente com política deliberada. Um erro transitório cacheado por muito tempo prolonga indisponibilidade após recuperação; negative caching pode proteger origem, mas exige TTL e escopo adequados ao tipo de erro.

Termo para lembrar: Negative caching | Base técnica: referência 1.

22. Como usa max-age e s-maxage?

Distingo validade para caches privados e compartilhados conforme diretivas. Isso permite políticas diferentes para navegador e CDN; verifico precedência e configuração real do intermediário, evitando confiar apenas no header enviado pela aplicação.

Termo para lembrar: Freshness lifetime | Base técnica: referência 1.

23. Age mede o quê?

Ajuda indicar tempo da resposta no sistema de cache conforme o protocolo, mas não substitui a versão de negócio do dado. Um objeto recém-cacheado pode ter sido produzido a partir de informação antiga.

Termo para lembrar: Age header | Base técnica: referência 1.

24. Compressão muda a representação?

Pode criar variantes que exigem chave e validadores coerentes. Coordeno origem e CDN para evitar dupla compressão ou ETag incompatível; também avalio CPU, bytes e segurança do conteúdo servido.

Termo para lembrar: Representation variant | Base técnica: referência 2.

25. Como funciona cache para download privado?

Autorizo antes de emitir acesso e defino política adequada para URL ou cookie assinado. O link pode funcionar como credencial temporária; logs, compartilhamento e duração precisam ser tratados como parte da segurança.

Termo para lembrar: Signed access | Base técnica: referência 3.

26. Service worker muda a análise?

Sim. Ele adiciona lógica de cache no cliente que pode sobreviver a navegações e deploys. Versiono caches e estratégia de atualização; invalidar a CDN não garante que o aplicativo deixará de usar conteúdo local.

Termo para lembrar: Client cache layer | Base técnica: referência 1.

27. Como mede benefício da CDN?

Acompanho hit rate por classe, bytes evitados, latência do usuário e carga da origem. Um hit rate global alto pode esconder misses caros numa rota crítica; segmentação ajuda decidir otimização real.

Termo para lembrar: Cache effectiveness | Base técnica: referência 1.

28. Como testa deploy com clientes antigos?

Mantenho sessão carregada, atualizo assets e backend e verifico compatibilidade. Usuários não recarregam sincronizadamente; versões de frontend e API coexistem, então cache e rollout precisam ser planejados conjuntamente.

Termo para lembrar: Version coexistence | Base técnica: referência 2.

29. Quando evitar cache HTTP?

Quando reutilização não oferece benefício ou o risco de resposta indevida supera a economia, especialmente em comandos. Posso manter assets cacheados e dados sensíveis sem armazenamento, escolhendo por recurso e contrato.

Termo para lembrar: Cache suitability | Base técnica: referência 1.

30. Como defenderia sua política?

Eu mostraria quais recursos são públicos, privados ou não armazenáveis, a cache key e os validadores. Depois testaria troca de usuário e origem indisponível para comprovar correção além da latência média.

Termo para lembrar: Cache contract | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. TTL longo

Ganho. Mais hits

Custo e motivo. Atualização lenta

Como decidir. Para conteúdo imutável

02. TTL curto

Ganho. Mais frescor

Custo e motivo. Mais carga

Como decidir. Para leitura mutável

03. Revalidação

Ganho. Economiza corpo

Custo e motivo. Ainda usa round trip

Como decidir. Quando validador é barato

04. URL com hash

Ganho. Invalidação simples

Custo e motivo. Mais versões armazenadas

Como decidir. Para assets

05. Purge

Ganho. Atualiza mesmo caminho

Custo e motivo. Propagação operacional

Como decidir. Quando URL não pode mudar

06. Stale em falha

Ganho. Preserva leitura

Custo e motivo. Dado antigo

Como decidir. Somente se semântica permitir

07. Cache privado

Ganho. Isola usuário

Custo e motivo. Menor compartilhamento

Como decidir. Para conteúdo personalizado

08. Cache compartilhado

Ganho. Grande economia

Custo e motivo. Risco de chave incorreta

Como decidir. Para conteúdo seguro para grupo

09. Vary amplo

Ganho. Representação correta

Custo e motivo. Cardinalidade elevada

Como decidir. Somente para dimensões necessárias

10. Negative caching

Ganho. Protege origem

Custo e motivo. Pode prolongar erro

Como decidir. Com TTL específico e curto

Duas alternativas ao caminho proposto

1. Sem cache de dados

Quando faz sentido. Frescor e risco dominam

Como implementar. Consultar origem autorizada com limites

Limites. Maior latência e custo

2. Cache de aplicação versionado

Quando faz sentido. Precisa de controle semântico

Como implementar. Guardar fatos por versão e autorizar ao compor

Limites. Invalidação e capacidade ficam sob responsabilidade da equipe

Referências e alcance da evidência

1. IETF - RFC 9111 HTTP Caching

Freshness, revalidação e diretivas de cache HTTP.

https://www.rfc-editor.org/rfc/rfc9111

2. IETF - RFC 9110 HTTP Semantics

Métodos, precondições, representações e semântica HTTP.

https://www.rfc-editor.org/rfc/rfc9110.html

3. AWS - S3 presigned URLs

Acesso temporário baseado nas permissões do emissor e possibilidade de reutilização.

https://docs.aws.amazon.com/AmazonS3/latest/userguide/using-presigned-url.html

Continue em: T031 · T032 · T053 · T058

T057 / F. Rede e entrada

WebSocket, SSE e long polling

Conexões em tempo real transportam atualizações; histórico durável e sequência permitem retomar após desconexão.

Quando usar. Chat, acompanhamento de operações e painéis atualizados continuamente.

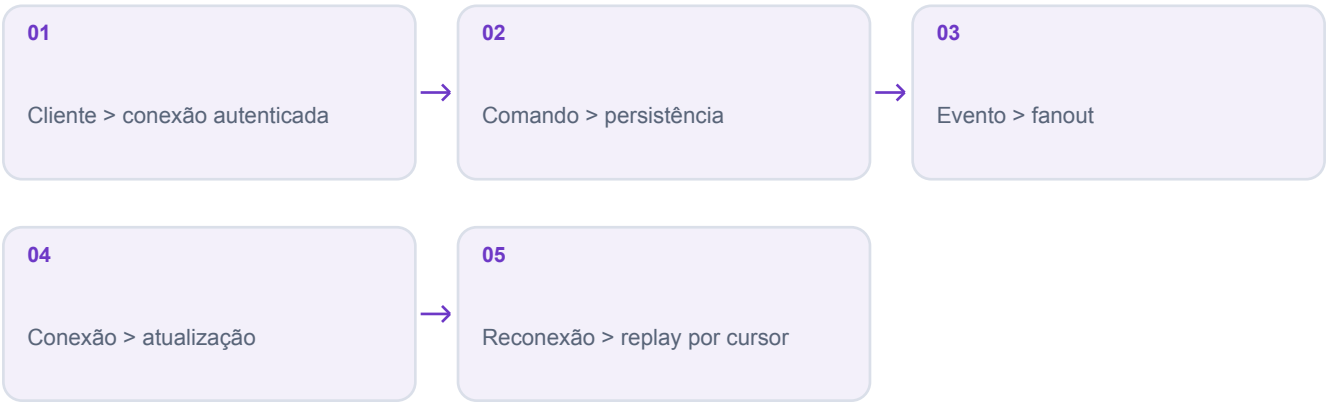
Quando evitar ou limitar. Não usar conexão viva como fonte da verdade nem acumular buffers ilimitados para clientes lentos.

Uma abertura possível

Vou separar transporte de durabilidade: a mensagem é confirmada depois de persistida, e a reconexão usa cursor para recuperar lacunas.

Peças e tecnologias. WebSocket · SSE · HTTP long polling · Redis · PostgreSQL

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Escolher direção e frequência do fluxo

2. Persistir mensagens e sequência

3. Autenticar conexão e autorizar assinaturas

4. Limitar buffers e implementar replay

5. Testar queda, cliente lento e revogação

30 perguntas para sustentar o desenho

01. Quando escolhe WebSocket?

Quando há comunicação bidirecional frequente, como chat interativo. Ainda envio identidade por mensagem e persisto efeitos importantes; a conexão reduz overhead de transporte, mas não oferece por si só durabilidade ou deduplicação.

Termo para lembrar: WebSocket | Base técnica: referência 1.

02. Quando SSE é suficiente?

Quando o servidor envia atualizações e o cliente pode mandar comandos por HTTP separado. Isso simplifica o fluxo unidirecional, mas mantenho autorização, reconexão e recuperação de eventos perdidos com contrato explícito.

Termo para lembrar: Server-sent events | Base técnica: referência 2.

03. Long polling é uma opção inferior sempre?

Não. Pode atender volume e frequência moderados com infraestrutura HTTP simples. Avalio overhead de requisições, latência e conexões pendentes; a escolha depende do uso, não de parecer tecnologicamente mais avançada.

Termo para lembrar: Long polling | Base técnica: referência 2.

04. Uma mensagem enviada no socket está entregue?

Não necessariamente ao usuário. Diferencio enviada, persistida, recebida pelo cliente e lida. TCP ou WebSocket não prova renderização nem leitura humana; cada confirmação precisa de significado e identidade definidos.

Termo para lembrar: Delivery states | Base técnica: referência 1.

05. Quando confirma mensagem do cliente?

Depois de persistir mensagem e obrigação de distribuição no armazenamento autoritativo. Se a resposta se perder, o cliente repete com o mesmo client_message_id, permitindo recuperar resultado sem criar outra mensagem.

Termo para lembrar: Durable ACK | Base técnica: referência 3.

06. Como ordena mensagens de uma conversa?

Atribuo sequência no servidor dentro da transação apropriada. A ordem representa aceitação pelo sistema, não o instante absoluto em que pessoas digitaram em dispositivos distintos com relógios diferentes.

Termo para lembrar: Conversation sequence | Base técnica: referência 3.

07. Como o cliente reconecta?

Ele autentica e envia o último cursor aplicado. O servidor oferece replay ou snapshot versionado; o cliente deduplica mensagens e detecta gaps, porque uma conexão nova não contém automaticamente o histórico perdido.

Termo para lembrar: Resume cursor | Base técnica: referência 2.

08. Como evita lacuna entre replay e assinatura viva?

Estabeleço um ponto de corte e buffer limitado durante a transição. Aplico histórico até esse ponto e depois eventos posteriores, deduplicando; assinar somente após terminar a leitura pode perder eventos intermediários.

Termo para lembrar: Replay handoff | Base técnica: referência 2.

09. O que faz com cliente muito lento?

Limito bytes e mensagens em buffer por conexão. Descarto sinais efêmeros primeiro e, se necessário, desconecto com orientação de ressincronização; não deixo um dispositivo ocupar memória ilimitada no gateway.

Termo para lembrar: Bounded buffer | Base técnica: referência 1.

10. WebSocket tem backpressure automático no navegador?

A API tradicional não oferece controle automático suficiente para esse problema. A aplicação precisa monitorar buffers e ritmo, ou escolher tecnologia apropriada; receber mais rápido do que processa pode esgotar recursos.

Termo para lembrar: Backpressure | Base técnica: referência 1.

11. Presença precisa ser durável?

Normalmente presença é efêmera e pode usar TTL. Não a confundo com estado de mensagem; perder presença pode mostrar usuário offline temporariamente, mas não deve apagar conversa nem liberar ação financeira indevida.

Termo para lembrar: Ephemeral presence | Base técnica: referência 1.

12. Redis pode guardar todo histórico?

Pode tecnicamente em certos desenhos, mas eu justificaria durabilidade e recuperação. Para histórico bancário, separo sessões efêmeras da fonte autoritativa; perder cache não deve apagar mensagens confirmadas ao cliente.

Termo para lembrar: Durability boundary | Base técnica: referência 3.

13. Sticky sessions são obrigatórias?

Não para correção se histórico e sessão recuperável estão externos. Afinidade pode ajudar localidade, mas uma queda exige reconexão em outra instância; depender dela para encontrar mensagens torna o sistema frágil.

Termo para lembrar: Session affinity | Base técnica: referência 1.

14. Como autentica uma conexão longa?

Valido credencial inicial e defino renovação, expiração e revogação durante a sessão. Uma conexão aberta há horas não deve conservar acesso indefinidamente após mudança de permissão ou desligamento de atendente.

Termo para lembrar: Session lifecycle | Base técnica: referência 1.

15. Como autoriza assinatura de conversa?

Verifico acesso ao recurso no servidor, incluindo tenant e participação atual. O cliente não pode assinar qualquer conversation_id apenas por conhecer o identificador; autorização ocorre também em mensagens e ações relevantes.

Termo para lembrar: Subscription authorization | Base técnica: referência 1.

16. Como impede envio por atendente removido?

Valido responsável e versão da atribuição ao persistir a mensagem. Um socket antigo pode continuar conectado após reatribuição; transporte ativo não representa permissão vigente para agir naquela conversa.

Termo para lembrar: Assignment fencing | Base técnica: referência 3.

17. Heartbeat prova que usuário está lendo?

Não. Ele ajuda detectar conectividade ou manter caminho ativo, mas não comprova atenção humana. Read receipt precisa de evento próprio e semântica limitada; ainda pode significar apenas que a interface marcou leitura.

Termo para lembrar: Heartbeat | Base técnica: referência 1.

18. Qual risco de heartbeat frequente?

Consome rede, bateria e processamento em grande número de conexões. Ajusto intervalo aos timeouts do caminho e necessidade de detecção, evitando usar uma cadência agressiva sem benefício mensurado.

Termo para lembrar: Heartbeat interval | Base técnica: referência 1.

19. Como trata proxies que encerram conexão ociosa?

Alinho timeouts e, quando apropriado, envio sinais leves de manutenção. O cliente tolera reconexão com backoff e replay; não dependo de uma conexão permanecer viva para sempre através de intermediários.

Termo para lembrar: Idle timeout | Base técnica: referência 1.

20. Como escala fanout?

Localizo sessões por gateway e distribuo eventos apenas aos destinos interessados, com limites. Para grupos grandes, controlo amplificação e evito enviar payload pesado repetidamente; fanout também precisa de observabilidade e backpressure.

Termo para lembrar: Fanout routing | Base técnica: referência 1.

21. Pub/sub Redis garante recuperação de mensagem perdida?

Não como substituto de histórico durável. Uso notificações como caminho rápido e o banco ou log como fonte de replay; se o assinante cair, ele recupera pelo cursor autoritativo.

Termo para lembrar: Fast path versus replay | Base técnica: referência 3.

22. Como evita tempestade de reconexão?

Aplico backoff com jitter, admissão e autenticação dimensionada. Um deploy ou perda de zona pode desconectar milhares de clientes; reconectar todos imediatamente pode derrubar tanto gateways quanto serviço de identidade.

Termo para lembrar: Reconnect storm | Base técnica: referência 1.

23. SSE exige uma conexão por widget?

Não necessariamente. Posso multiplexar eventos lógicos numa conexão autorizada e encaminhar por tipo ou recurso. Isso reduz overhead, mas exige contratos e limites para que um fluxo ruidoso não bloqueie os demais.

Termo para lembrar: Logical multiplexing | Base técnica: referência 2.

24. Como usa Last-Event-ID?

Ele pode transportar o ponto de retomada no SSE, mas o servidor precisa manter ou reconstruir eventos correspondentes. O header não cria histórico sozinho nem resolve cursor expirado sem uma política de ressincronização.

Termo para lembrar: Last-Event-ID | Base técnica: referência 2.

25. O cursor pode expirar?

Sim, se o histórico de replay tiver retenção limitada. Retorno uma condição clara e ofereço snapshot ou nova sincronização; não finjo continuidade quando eventos intermediários necessários já não estão disponíveis.

Termo para lembrar: Cursor retention | Base técnica: referência 2.

26. Como trata upload em chat?

Uso fluxo de arquivo separado, com autorização, limites e quarentena, enviando referência na mensagem. Não transformo o socket em canal ilimitado de bytes que compete com atualizações pequenas e importantes.

Termo para lembrar: Attachment offload | Base técnica: referência 1.

27. Como protege contra mensagens enormes?

Valido tamanho, schema, quantidade de assinaturas e taxa por conexão. Também limito conteúdo descompactado; conexão autenticada continua podendo causar sobrecarga acidental ou abusiva se não houver controles específicos.

Termo para lembrar: Connection limits | Base técnica: referência 1.

28. Quais métricas importam?

Conexões ativas, reconexões, buffer por conexão, tempo entre persistência e renderização e gaps recuperados. Uma conexão aberta não indica experiência saudável; meço entrega útil e atraso do histórico no cliente.

Termo para lembrar: Realtime SLI | Base técnica: referência 1.

29. Como testa sem depender do caminho feliz?

Desconecto após persistência, atraso eventos, duplico mensagens e simulo cliente lento e permissão revogada. Verifico sequência e retomada; um teste que apenas troca mensagens numa conexão estável deixa os riscos principais descobertos.

Termo para lembrar: Realtime fault test | Base técnica: referência 3.

30. Como defenderia sua escolha?

Eu justificaria WebSocket, SSE ou polling pela direção e frequência, depois mostraria persistência, cursor e limites. A parte importante é conseguir recuperar a conversa e respeitar acesso quando a conexão inevitavelmente falha.

Termo para lembrar: Transport independence | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. WebSocket

Ganho. Bidirecionalidade eficiente

Custo e motivo. Estado de conexão

Como decidir. Para interação frequente

02. SSE

Ganho. Fluxo servidor-cliente simples

Custo e motivo. Comandos por outro canal

Como decidir. Para atualizações unidirecionais

03. Long polling

Ganho. Compatibilidade HTTP

Custo e motivo. Mais requisições

Como decidir. Para frequência moderada

04. Heartbeat curto

Ganho. Detecta rápido

Custo e motivo. Bateria e tráfego

Como decidir. Quando reação precisa ser rápida

05. Buffer maior

Ganho. Tolera atraso breve

Custo e motivo. Memória por cliente

Como decidir. Com limite explícito

06. Desconectar lento

Ganho. Protege servidor

Custo e motivo. Exige replay

Como decidir. Quando cliente não acompanha

07. Histórico durável

Ganho. Retomada confiável

Custo e motivo. Persistência e retenção

Como decidir. Para mensagens confirmadas

08. Presença efêmera

Ganho. Baixo custo

Custo e motivo. Pode ficar imprecisa

Como decidir. Para indicador não financeiro

09. Uma conexão multiplexada

Ganho. Menos overhead

Custo e motivo. Fluxos competem

Como decidir. Com limites por assinatura

10. Gateway stateless lógico

Ganho. Failover simples

Custo e motivo. Store e roteamento externos

Como decidir. Para escala horizontal

Duas alternativas ao caminho proposto

1. Polling periódico

Quando faz sentido. Atualização tolera atraso

Como implementar. GET com cursor ou ETag em intervalos controlados

Limites. Mais carga e latência entre ciclos

2. Push notification com consulta

Quando faz sentido. Aplicativo pode ficar offline

Como implementar. Notificar mudança e buscar estado autorizado

Limites. Push não garante entrega nem contém histórico completo

Referências e alcance da evidência

1. MDN - WebSocket API

Comunicação bidirecional e ausência de backpressure automático na API tradicional.

https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API

2. MDN - Server-sent events

Eventos servidor-cliente, reconexão e formato do fluxo.

https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events/Using_server-sent_events

3. AWS Builders Library - Idempotent APIs

Identidades estáveis, intenção, retries e requisições atrasadas.

https://aws.amazon.com/builders-library/making-retries-safe-with-idempotent-APIs/

Continue em: T043 · T052 · T054 · T099

T058 / F. Rede e entrada

Object storage, multipart e URLs assinadas

Object storage guarda objetos identificados por chave; multipart e URLs assinadas controlam transferência, não validam conteúdo de negócio.

Quando usar. Documentos, anexos, arquivos de lote e histórico volumoso.

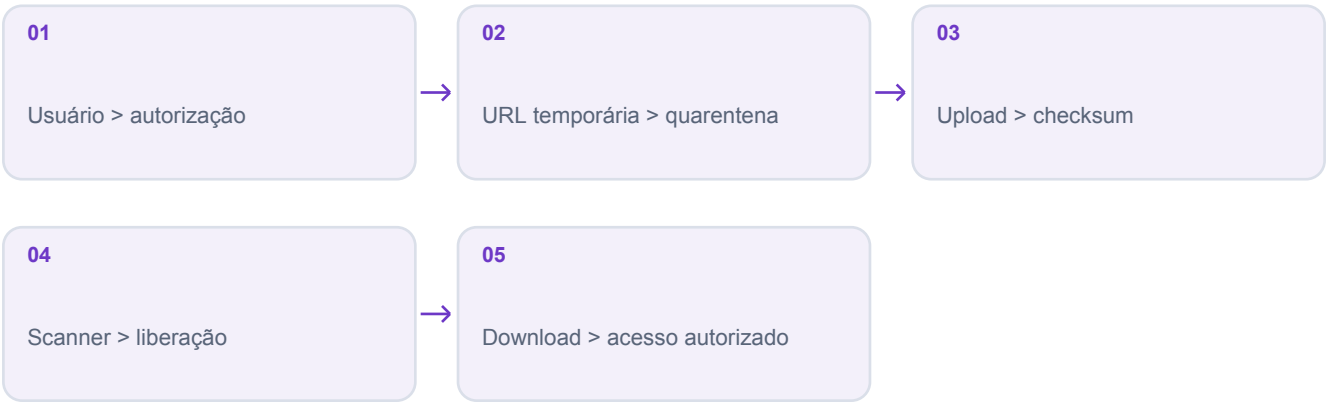
Quando evitar ou limitar. Não tratar URL assinada como uso único nem liberar upload diretamente para consumo sem validação.

Uma abertura possível

Vou separar autorização, transferência, integridade e liberação do arquivo; objeto enviado não significa documento seguro ou aprovado.

Peças e tecnologias. S3 · Azure Blob · Ceph · Multipart upload · Presigned URL

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Gerar chave e permissões no servidor

2. Restringir tamanho, tipo e validade

3. Enviar partes com checkpoint e checksum

4. Validar conteúdo antes de liberar

5. Definir retenção, limpeza e auditoria

30 perguntas para sustentar o desenho

01. Object storage é um filesystem remoto?

Não possui necessariamente a mesma semântica de diretórios, append ou rename de um filesystem. Modelo objetos e chaves conforme o serviço; adaptar uma biblioteca de arquivos sem verificar operações pode produzir comportamento incorreto.

Termo para lembrar: Object semantics | Base técnica: referência 1.

02. Como escolhe a chave do objeto?

Gero chave no servidor com escopo e identidade imutável, evitando confiar no nome enviado pelo usuário. O nome original pode virar metadado validado; a chave não deve permitir sobrescrever documento de outro cliente.

Termo para lembrar: Object key | Base técnica: referência 1.

03. URL assinada equivale a login?

Ela concede acesso temporário à operação e objeto definidos, conforme permissões do emissor. Quem obtém o link pode usá-lo nas condições permitidas; trato-o como credencial e não o exponho em logs públicos.

Termo para lembrar: Presigned URL | Base técnica: referência 1.

04. URL assinada pode ser usada uma única vez?

Não assumo isso. No S3 ela pode ser reutilizada enquanto válida. Se preciso de uso único, adiciono controle no fluxo e considero a corrida entre tentativas; assinatura sozinha não oferece essa propriedade.

Termo para lembrar: Bearer capability | Base técnica: referência 1.

05. Como impede sobrescrita indesejada?

Uso chaves únicas, políticas e precondições adequadas ao serviço, além de versionamento quando necessário. Emitir PUT assinado para uma chave existente pode permitir substituição; o backend precisa controlar essa decisão.

Termo para lembrar: Overwrite protection | Base técnica: referência 1.

06. Como limita validade do link?

Escolho prazo mínimo compatível com a transferência e considero expiração das credenciais emissoras. Também preparo renovação autorizada; um download longo ou cliente lento não deve exigir exposição desnecessariamente prolongada.

Termo para lembrar: URL expiration | Base técnica: referência 1.

07. Revogar acesso na aplicação invalida link existente?

Não necessariamente de forma imediata. O link pode continuar válido conforme credenciais e políticas. Para recursos sensíveis, uso validade curta ou caminho de autorização mais controlado, explicando essa janela de revogação.

Termo para lembrar: Revocation window | Base técnica: referência 1.

08. Quando usa multipart?

Para objetos grandes ou redes onde retomar partes reduz custo de falha. Dimensiono partes e paralelismo conforme limites e memória; multipart melhora transferência, mas exige acompanhar upload_id e concluir ou abortar corretamente.

Termo para lembrar: Multipart upload | Base técnica: referência 2.

09. Como retoma multipart interrompido?

Persisto upload_id, partes e checksums necessários para continuar conforme o serviço. Reenvio apenas partes ausentes ou inválidas e verifico conclusão; iniciar outro upload sem limpar o anterior pode acumular custo.

Termo para lembrar: Upload checkpoint | Base técnica: referência 2.

10. Partes enviadas já formam objeto final?

Não até a conclusão apropriada do multipart. O fluxo precisa diferenciar partes armazenadas de objeto pronto, e o consumidor só recebe referência liberada após validação de integridade e estado.

Termo para lembrar: Multipart completion | Base técnica: referência 2.

11. Como limpa uploads abandonados?

Configuro política de abortar multipart incompleto e acompanho pendências. Partes podem continuar armazenadas e cobradas até limpeza; a aplicação também deve encerrar tentativas canceladas sem depender apenas de intervenção manual.

Termo para lembrar: Multipart lifecycle | Base técnica: referência 2.

12. ETag é checksum universal?

Não. Seu significado varia com forma de upload e configuração. Uso checksum explícito suportado e guardo algoritmo e valor; comparar ETag como MD5 em todos os casos pode dar conclusão errada sobre integridade.

Termo para lembrar: Checksum contract | Base técnica: referência 3.

13. Checksum garante arquivo seguro?

Não. Ele detecta alteração de bytes conforme algoritmo, mas malware pode ter checksum perfeitamente válido. Scanner, validação de formato e políticas de acesso são etapas separadas da integridade de transporte.

Termo para lembrar: Integrity versus safety | Base técnica: referência 3.

14. Como valida tipo do arquivo?

Não confio somente em extensão ou Content-Type do cliente. Examino conteúdo com limites e ferramentas adequadas, mantenho quarentena e libero apenas após resultado; parsers também precisam proteção contra arquivos malformados.

Termo para lembrar: Content validation | Base técnica: referência 1.

15. Por que usar quarentena?

Ela impede que arquivo recém-enviado seja consumido antes de validação. O estado de liberação é autoritativo; mover ou renomear objeto sem atualizar corretamente esse estado pode criar acesso prematuro ou órfãos.

Termo para lembrar: Quarantine state | Base técnica: referência 1.

16. Como relaciona objeto e banco atomicamente?

Normalmente não existe transação única entre ambos. Modelo estados de upload, confirmação e limpeza de órfãos; uma referência só fica utilizável após verificar objeto, enquanto reconciliadores tratam gravações incompletas.

Termo para lembrar: Cross-store consistency | Base técnica: referência 1.

17. Evento ObjectCreated basta para aprovar documento?

Não. Ele sinaliza criação no storage, não validade do conteúdo ou conclusão de negócio. O consumidor valida identidade e estado, tolera repetição e registra resultado antes de tornar o documento disponível.

Termo para lembrar: Storage event | Base técnica: referência 1.

18. Como evita duplicar scanner?

Uso identidade do objeto e versão ou checksum para deduplicar processamento. Repetir a notificação não deve gerar efeitos múltiplos; se bytes mudaram, a nova versão precisa passar por validação própria.

Termo para lembrar: Versioned processing | Base técnica: referência 3.

19. Como protege download privado?

Autorizo usuário e recurso antes de emitir URL ou servir conteúdo. A chave não é segredo suficiente; enumeração ou conhecimento de object_id não pode permitir baixar documento bancário de outro cliente.

Termo para lembrar: Object authorization | Base técnica: referência 1.

20. Como trata cache de documento privado?

Defino diretivas e política compatíveis com confidencialidade e revogação. Um link assinado pode terminar em logs ou caches; não suponho que autenticação inicial controle todas as cópias posteriores da resposta.

Termo para lembrar: Private delivery | Base técnica: referência 1.

21. Qual vantagem de versionamento?

Ajuda recuperar sobrescritas e identificar bytes históricos, mas aumenta armazenamento e complexidade de exclusão. Defino qual versão é autorizada e válida; ter versões antigas não significa que todos possam acessá-las.

Termo para lembrar: Object versioning | Base técnica: referência 1.

22. Replicação substitui backup?

Não necessariamente. Ela pode propagar exclusões ou corrupção conforme configuração. Analiso retenção e recuperação independentes, e testo restauração de versões e metadados; cópia adicional sem procedimento não comprova recuperabilidade.

Termo para lembrar: Recovery copy | Base técnica: referência 1.

23. Como define retenção e exclusão?

Inventario objetos, versões, índices, caches e backups derivados. A política precisa considerar obrigação de preservar e de excluir, com responsáveis; apagar apenas a referência do banco pode deixar o arquivo acessível.

Termo para lembrar: Data lifecycle | Base técnica: referência 1.

24. Pode usar storage frio para qualquer documento?

Somente se o tempo e custo de recuperação atenderem ao contrato. Histórico raro pode tolerar espera; um anexo necessário durante atendimento imediato não deve ir para uma classe que inviabilize essa experiência.

Termo para lembrar: Storage tiering | Base técnica: referência 1.

25. Como controla custo de muitas partes pequenas?

Dimensiono tamanho e paralelismo, considerando operações por parte e overhead. Otimizar somente throughput pode aumentar requisições e custo; arquivos pequenos normalmente não precisam da mesma estratégia multipart de objetos enormes.

Termo para lembrar: Multipart economics | Base técnica: referência 2.

26. Como evita SSRF ao buscar arquivo externo?

Valido destino e fluxo de rede, restrinjo protocolos e endereços permitidos e considero redirecionamentos. Uma URL fornecida pelo usuário não deve permitir ao backend consultar metadados internos ou serviços privados.

Termo para lembrar: SSRF boundary | Base técnica: referência 1.

27. Como lida com objeto ausente após referência criada?

Retorno estado de indisponibilidade e inicio reconciliação, sem fingir documento vazio. Verifico se upload não concluiu, houve exclusão ou permissão mudou; cada causa tem recuperação e impacto operacional distintos.

Termo para lembrar: Orphan reconciliation | Base técnica: referência 1.

28. Quais métricas acompanha?

Falhas de upload, multipart abandonado, tempo de scanner, objetos em quarentena e referências quebradas. Também meço custo por classe e tempo até disponibilidade; bytes armazenados sozinhos não mostram saúde do fluxo.

Termo para lembrar: Object pipeline SLI | Base técnica: referência 2.

29. Como testa o caminho completo?

Interrompo upload, repito conclusão, envio arquivo inválido e revogo acesso antes do download. Verifico que objeto parcial não fica público e que limpeza não remove documentos válidos referenciados pelo domínio.

Termo para lembrar: Upload fault test | Base técnica: referência 2.

30. Como defenderia a arquitetura?

Eu mostraria quem autoriza, como o objeto é identificado, quando fica pronto e como órfãos são reconciliados. Multipart e assinatura resolvem transferência; segurança e conclusão do processo permanecem explícitas no backend.

Termo para lembrar: Storage boundary | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Upload direto

Ganho. Poupa banda do backend

Custo e motivo. Controle distribuído

Como decidir. Com autorização e quarentena

02. Proxy de upload

Ganho. Controle central

Custo e motivo. Custo e gargalo

Como decidir. Para requisitos específicos

03. URL curta

Ganho. Menor exposição

Custo e motivo. Mais renovações

Como decidir. Para documento sensível

04. URL longa

Ganho. Cliente lento tolerado

Custo e motivo. Janela maior

Como decidir. Quando risco é aceitável

05. Multipart paralelo

Ganho. Maior throughput

Custo e motivo. Memória e requisições

Como decidir. Para objeto grande

06. Partes pequenas

Ganho. Retry barato

Custo e motivo. Mais operações

Como decidir. Para rede instável

07. Versionamento

Ganho. Recupera sobrescrita

Custo e motivo. Armazenamento extra

Como decidir. Para artefatos auditáveis

08. Storage frio

Ganho. Menor custo

Custo e motivo. Recuperação lenta

Como decidir. Para histórico raro

09. Checksum explícito

Ganho. Integridade verificável

Custo e motivo. Cálculo e metadados

Como decidir. Para transferências críticas

10. Quarentena

Ganho. Impede consumo inseguro

Custo e motivo. Atraso no acesso

Como decidir. Para anexos externos

Duas alternativas ao caminho proposto

1. Filesystem compartilhado

Quando faz sentido. Aplicação exige semântica POSIX

Como implementar. Usar armazenamento de arquivos apropriado e HA

Limites. Operação e escala diferem de object storage

2. Banco para blobs pequenos

Quando faz sentido. Atomicidade com metadados domina

Como implementar. Guardar conteúdo limitado na mesma transação

Limites. Aumenta banco, backup e custo de I/O

Referências e alcance da evidência

1. AWS - S3 presigned URLs

Acesso temporário baseado nas permissões do emissor e possibilidade de reutilização.

https://docs.aws.amazon.com/AmazonS3/latest/userguide/using-presigned-url.html

2. AWS - S3 multipart upload

Upload em partes, conclusão, retomada e limpeza de uploads incompletos.

https://docs.aws.amazon.com/AmazonS3/latest/userguide/mpuoverview.html

3. AWS - S3 object integrity

Verificação de integridade por checksums em objetos.

https://docs.aws.amazon.com/AmazonS3/latest/userguide/checking-object-integrity.html

Continue em: T049 · T056 · T060 · T075

T059 / F. Rede e entrada

VPC, subnets, NAT e egress

Rede virtual define conectividade, rotas e fronteiras; segurança também exige identidade e autorização.

Quando usar. Serviços privados, bancos, parceiros externos e integração híbrida.

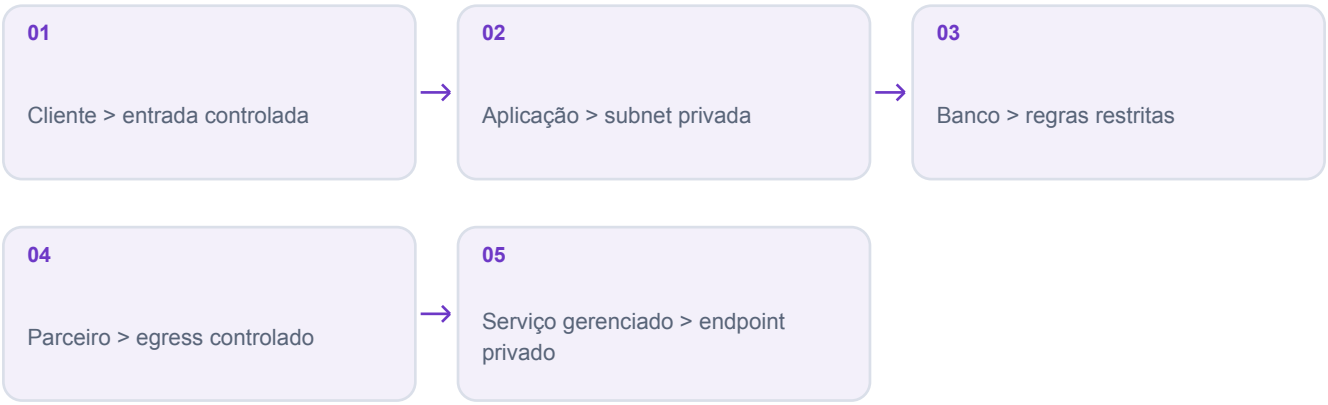
Quando evitar ou limitar. Não chamar uma subnet de privada como prova de segurança nem usar NAT como firewall de aplicação.

Uma abertura possível

Vou desenhar cada caminho de entrada e saída, incluindo DNS, rotas, identidade e falha de zona.

Peças e tecnologias. VPC · Subnets · Security Groups · NAT Gateway · PrivateLink · VPN · Envoy

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Separar subnets por função e zona

2. Definir rotas e controles mínimos

3. Controlar egress e identidades

4. Dimensionar NAT e caminhos de falha

5. Testar conectividade, isolamento e custos

30 perguntas para sustentar o desenho

01. O que torna uma subnet pública?

Sua conectividade e tabela de rotas, não o nome escolhido. Analiso caminho para internet gateway e endereçamento dos recursos; uma etiqueta pública ou privada não comprova quais conexões são possíveis.

Termo para lembrar: Routing table | Base técnica: referência 1.

02. Subnet privada significa sem internet?

Não necessariamente. Recursos podem iniciar saída por NAT ou outros caminhos configurados. Desenho egress explicitamente, porque atualizações, APIs externas e exfiltração potencial dependem dessas rotas e permissões.

Termo para lembrar: Private subnet egress | Base técnica: referência 1.

03. NAT gateway é um firewall completo?

Não. Ele traduz endereços e participa da conectividade de saída, mas não substitui políticas de identidade ou inspeção de aplicação. Defino controles adicionais conforme destinos e riscos do tráfego.

Termo para lembrar: NAT boundary | Base técnica: referência 1.

04. Como serviço privado recebe requisição pública?

Uso entrada controlada, como load balancer ou gateway, com regras que permitam somente o caminho necessário até a aplicação. O backend não precisa ficar diretamente acessível da internet para atender o cliente.

Termo para lembrar: Ingress path | Base técnica: referência 2.

05. Security group e autorização são equivalentes?

Não. Security group controla tráfego em determinado escopo de rede; autorização decide qual identidade pode agir sobre qual recurso. Uma conexão permitida ao banco ou API não legitima toda operação solicitada.

Termo para lembrar: Network versus application | Base técnica: referência 2.

06. Por que limitar egress?

Reduz caminhos de exfiltração e acesso indevido, além de tornar integrações conhecidas. Uma aplicação comprometida não deveria alcançar qualquer destino; mantenho exceções e dependências necessárias de maneira rastreável.

Termo para lembrar: Egress policy | Base técnica: referência 2.

07. Como permite parceiro com IP fixo de origem?

Uso caminho de saída com endereços conhecidos conforme o contrato e configuração do provedor. Ainda autentico a integração; allowlist de IP ajuda filtrar rede, mas não prova identidade de negócio sozinha.

Termo para lembrar: Static egress | Base técnica: referência 1.

08. Um NAT para todas as zonas é sempre adequado?

Pode simplificar custo inicial, mas concentra dependência e pode gerar tráfego entre zonas. Avalio recuperação e capacidade do cenário de falha; economia normal não deve esconder indisponibilidade compartilhada.

Termo para lembrar: Zonal failure domain | Base técnica: referência 1.

09. O que acontece se acabar porta de saída?

Novas conexões podem falhar apesar de CPU baixa. Monitoro conexões e padrões de destino, reutilizo pools e dimensiono egress; criar mais workers pode aumentar pressão em vez de resolver o gargalo.

Termo para lembrar: Port exhaustion | Base técnica: referência 1.

10. Endpoint privado elimina autenticação?

Não. Ele muda o caminho de rede, mas serviços continuam exigindo políticas e credenciais adequadas. Limito acesso ao endpoint e ao recurso; privado descreve conectividade, não autorização irrestrita.

Termo para lembrar: Private endpoint | Base técnica: referência 3.

11. Quando usar endpoint em vez de NAT?

Quando o serviço oferece conectividade privada compatível e isso atende segurança, custo e operação. Comparo tráfego, disponibilidade e configuração de DNS; não assumo que todos os endpoints possuem o mesmo tipo ou cobrança.

Termo para lembrar: PrivateLink | Base técnica: referência 3.

12. Como DNS afeta endpoint privado?

A resolução pode direcionar nomes conhecidos para endereços privados conforme configuração. Testo de dentro e fora da rede, porque split DNS incorreto pode mandar tráfego ao caminho público ou tornar serviço inacessível.

Termo para lembrar: Private DNS | Base técnica: referência 3.

13. VPC peering permite trânsito automático?

Não assumo transitividade. Analiso rotas e capacidades específicas da conexão; topologias com várias redes podem precisar de hub ou serviço apropriado, evitando depender de um caminho que o produto não oferece.

Termo para lembrar: Routing topology | Base técnica: referência 4.

14. Como conecta ambiente on-premises?

Avalio VPN ou conexão dedicada conforme latência, capacidade e disponibilidade, com rotas e redundância. O desenho inclui falha do link e autenticação dos serviços, não apenas uma linha entre dois datacenters.

Termo para lembrar: Hybrid connectivity | Base técnica: referência 3.

15. Conexão dedicada dispensa criptografia?

Não automaticamente. Decido pelo modelo de ameaça e contrato; conectividade privada e transporte criptografado oferecem propriedades diferentes. Dados bancários podem exigir proteção mesmo quando o caminho não atravessa internet pública.

Termo para lembrar: Transport protection | Base técnica: referência 3.

16. Como evita CIDRs sobrepostos?

Planejo endereçamento e inventário antes de conectar redes. Sobreposição dificulta roteamento e pode exigir tradução ou migração; não trato isso como detalhe tardio quando há múltiplas contas, regiões e aquisições.

Termo para lembrar: IP address planning | Base técnica: referência 3.

17. Como separa banco de aplicação?

Coloco banco em caminho restrito e permito apenas identidades e origens necessárias, com autenticação própria. A separação reduz exposição, mas backups, administração e replicação também precisam de rotas e controles explícitos.

Termo para lembrar: Network segmentation | Base técnica: referência 2.

18. Como administra sem abrir SSH ao mundo?

Uso acesso controlado e auditado por mecanismos adequados, com identidades temporárias e escopo. Acesso emergencial deve existir, mas não justificar uma porta permanente ampla que contorne a arquitetura de segurança.

Termo para lembrar: Administrative access | Base técnica: referência 2.

19. Como investiga timeout de rede?

Verifico resolução, rota, regras, estabelecimento de conexão e destino, correlacionando logs de fluxo e aplicação. Mudar todas as regras para permitir tudo pode mascarar a causa e ampliar exposição sem necessidade.

Termo para lembrar: Network troubleshooting | Base técnica: referência 2.

20. Log de fluxo mostra conteúdo da requisição?

Não necessariamente; ele descreve metadados de tráfego no escopo do serviço. Uso traces e logs de aplicação para semântica, com proteção de dados; não espero diagnosticar toda falha financeira apenas por bytes e portas.

Termo para lembrar: Flow logs | Base técnica: referência 2.

21. Como controla tráfego entre serviços?

Uso regras mínimas por função e identidade quando disponível, além de autenticação de aplicação. Um serviço comprometido não deve ter acesso lateral amplo só porque compartilha a mesma VPC com outros domínios.

Termo para lembrar: Lateral movement | Base técnica: referência 2.

22. O que muda com IPv6?

Reavalio endereçamento, rotas, políticas e mecanismos de saída apropriados. Não transporto automaticamente suposições de NAT IPv4; a aplicação e controles precisam tratar ambos os protocolos sem criar caminho esquecido.

Termo para lembrar: Dual stack | Base técnica: referência 1.

23. Como limita download de dependências em runtime?

Prefiro artefatos preparados e repositórios controlados quando possível, reduzindo dependência de internet durante inicialização. Se houver acesso externo, aplico egress e verificação de integridade; deploy não deve executar conteúdo arbitrário.

Termo para lembrar: Supply-chain egress | Base técnica: referência 2.

24. Como calcula custo de rede?

Considero bytes por caminho, NAT, transferência entre zonas e regiões e endpoints. Uma arquitetura com muitas chamadas pode pagar mais por tráfego que por compute; meço custo por operação relevante.

Termo para lembrar: Network economics | Base técnica: referência 1.

25. Como prepara failover regional?

Defino rotas, resolução, identidade e capacidade na região alternativa, além de dados e fencing. Ter subnets prontas não resolve consistência de escrita nem garante que parceiros aceitarão novos endereços de origem.

Termo para lembrar: Regional connectivity | Base técnica: referência 3.

26. Como evita bypass de controles de saída?

Restrinjo rotas e permissões de alterar rede, além da configuração dos workloads. Um proxy egress só funciona se caminhos alternativos não estiverem liberados; governança de infraestrutura faz parte da garantia.

Termo para lembrar: Egress enforcement | Base técnica: referência 2.

27. Que teste comprova isolamento?

Executo tentativas autorizadas e proibidas entre funções, incluindo acesso direto à origem e destinos externos. Verifico políticas reais após deploy, porque um diagrama correto não garante que as regras aplicadas correspondam a ele.

Termo para lembrar: Segmentation test | Base técnica: referência 2.

28. Como trata mudança de IP do parceiro?

Uso contrato de descoberta ou atualização controlada de allowlists e monitoro conectividade. Não relaxo para qualquer destino sem análise; mudanças precisam preservar autenticação e reduzir janela de interrupção da integração.

Termo para lembrar: Partner endpoint change | Base técnica: referência 1.

29. Quais métricas são essenciais?

Falhas de conexão, uso de egress, tráfego entre zonas, erros de resolução e saturação de portas. Correlaciono com latência e prazo do negócio, evitando tratar toda falha de rede como problema do aplicativo.

Termo para lembrar: Network SLI | Base técnica: referência 1.

30. Como defenderia a topologia?

Eu seguiria uma chamada real de entrada até banco e parceiro, mostrando rota, controle e identidade em cada trecho. Depois removeria uma zona para verificar que segurança e capacidade continuam coerentes.

Termo para lembrar: Path analysis | Base técnica: referência 3.

10 trade-offs: ganho, custo e decisão

01. NAT central

Ganho. Menos recursos

Custo e motivo. Dependência e tráfego cruzado

Como decidir. Para risco e volume compatíveis

02. Egress por zona

Ganho. Isola falha

Custo e motivo. Mais custo

Como decidir. Para disponibilidade zonal

03. Endpoint privado

Ganho. Caminho controlado

Custo e motivo. Configuração e cobrança

Como decidir. Para serviços compatíveis

04. Saída aberta

Ganho. Facilita integração

Custo e motivo. Exfiltração e abuso

Como decidir. Evitar em domínios críticos

05. Allowlist estrita

Ganho. Reduz superfície

Custo e motivo. Manutenção de destinos

Como decidir. Com inventário confiável

06. VPN

Ganho. Implantação acessível

Custo e motivo. Variação de caminho

Como decidir. Para conectividade híbrida adequada

07. Link dedicado

Ganho. Capacidade previsível

Custo e motivo. Custo e prazo

Como decidir. Para demanda sustentada

08. Segmentação fina

Ganho. Limita movimento lateral

Custo e motivo. Mais políticas

Como decidir. Para fronteiras de risco

09. Dual stack

Ganho. Compatibilidade futura

Custo e motivo. Mais caminhos de teste

Como decidir. Quando clientes e rede exigem

10. Logs de fluxo

Ganho. Diagnóstico de rede

Custo e motivo. Volume e retenção

Como decidir. Com perguntas operacionais claras

Duas alternativas ao caminho proposto

1. Plataforma gerenciada com rede abstrata

Quando faz sentido. Equipe quer menos operação

Como implementar. Usar controles nativos de entrada e saída do serviço

Limites. Ainda exige conhecer limites e dependências

2. Datacenter on-premises segmentado

Quando faz sentido. Infraestrutura própria necessária

Como implementar. VLANs ou redes definidas por software e firewalls

Limites. Equipe assume disponibilidade e capacidade física

Referências e alcance da evidência

1. AWS - VPC NAT gateways

Saída de sub-redes privadas e conectividade NAT pública ou privada.

https://docs.aws.amazon.com/vpc/latest/userguide/vpc-nat-gateway.html

2. AWS - VPC security groups

Regras de tráfego e controle de acesso de rede stateful.

https://docs.aws.amazon.com/vpc/latest/userguide/VPC_SecurityGroups.html

3. AWS - VPC endpoints

Conectividade privada a serviços e opções de endpoints.

https://docs.aws.amazon.com/vpc/latest/userguide/vpc-endpoints.html

4. AWS - VPC peering basics

Rotas, limitações e ausência de transitividade de VPC peering.

https://docs.aws.amazon.com/vpc/latest/peering/vpc-peering-basics.html

Continue em: T051 · T053 · T073 · T078

T060 / F. Rede e entrada

APIs externas, webhooks e SFTP

Integrações externas exigem contratos de identidade, confirmação, repetição e reconciliação além do protocolo de transporte.

Quando usar. APIs de parceiros, webhooks, SFTP e retorno de arquivos bancários.

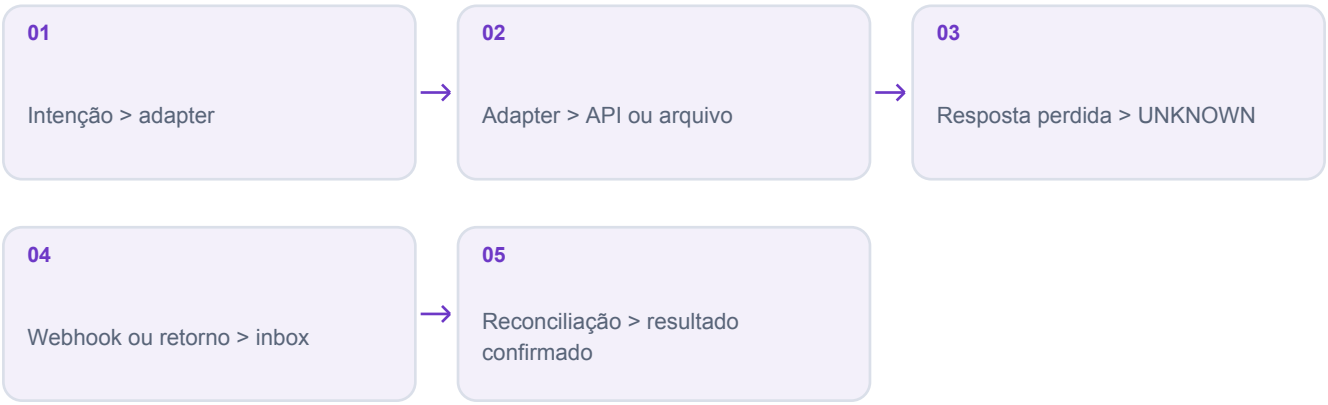
Quando evitar ou limitar. Não interpretar timeout como rejeição nem confirmação de transferência como resultado financeiro.

Uma abertura possível

Vou modelar envio e retorno separadamente e preservar a intenção quando a resposta se perde; a integração precisa explicar o que sabemos e o que permanece desconhecido.

Peças e tecnologias. HTTP · SFTP · AWS Transfer Family · IBM SFG · SQS · PostgreSQL

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Ler contrato de IDs, quotas e resultados

2. Persistir intenção e artefato imutável

3. Executar fora da transação com identidade

4. Validar e deduplicar retornos

5. Reconciliar pendências e resultados parciais

30 perguntas para sustentar o desenho

01. Qual é a primeira pergunta sobre o parceiro?

Pergunto como ele identifica uma intenção e como consulto resultado após perder a resposta. Sem esse contrato, retries podem duplicar efeitos; protocolo HTTP ou SFTP sozinho não define a garantia financeira.

Termo para lembrar: Partner contract | Base técnica: referência 3.

02. Por que usar um adapter?

Ele traduz contrato externo para o domínio e concentra autenticação, layout e classificação de erros. Evito espalhar códigos e peculiaridades do parceiro por todo sistema, preservando uma fronteira clara de evolução.

Termo para lembrar: Anti-corruption layer | Base técnica: referência 1.

03. O adapter pode esconder toda falha?

Não deve apagar informação importante. Traduzo erros para categorias úteis, mantendo evidência e referência externa; timeout ambíguo não pode virar rejeição definitiva apenas para simplificar o contrato interno.

Termo para lembrar: Error semantics | Base técnica: referência 3.

04. O que representa uma resposta HTTP 202?

Aceitação segundo o contrato, geralmente sem conclusão imediata. Persisto referência e acompanho resultado; não apresento pagamento ou chargeback concluído antes da confirmação de negócio correspondente, mesmo que a chamada inicial tenha sucesso.

Termo para lembrar: Async acceptance | Base técnica: referência 3.

05. Como autentica webhook?

Valido assinatura conforme o provedor, usando bytes e parâmetros esperados e política temporal adequada. Origem autenticada não dispensa validar tenant, objeto e transição; o evento ainda pode ser duplicado ou atrasado.

Termo para lembrar: Webhook signature | Base técnica: referência 1.

06. Por que preservar corpo bruto para assinatura?

A assinatura pode depender exatamente dos bytes recebidos. Desserializar e serializar novamente pode alterar representação; valido conforme SDK ou especificação antes de transformar o conteúdo para processamento interno.

Termo para lembrar: Raw payload verification | Base técnica: referência 1.

07. Quando responde sucesso ao webhook?

Depois de persistir recebimento durável ou concluir processamento local atômico, conforme desenho. Trabalho pesado pode seguir assíncrono; confirmar antes de qualquer durabilidade pode perder evento quando o processo cai.

Termo para lembrar: Durable webhook intake | Base técnica: referência 1.

08. Webhook pode chegar mais de uma vez?

Sim. Uso identidade do evento e consumidor, junto da alteração local. Também considero eventos diferentes sobre o mesmo objeto e tipo; deduplicação técnica não substitui regras de negócio sobre o efeito pretendido.

Termo para lembrar: Webhook deduplication | Base técnica: referência 1.

09. Webhook pode chegar fora de ordem?

Sim conforme provedor. Uso versão, estado e consulta autoritativa quando necessário; não deixo um evento antigo regredir uma operação concluída nem descarto um delta importante apenas por chegar depois.

Termo para lembrar: Event ordering | Base técnica: referência 1.

10. Como trata callback sem solicitação conhecida?

Persisto para investigação ou correlação posterior conforme contrato, sem executar efeito não autorizado. Pode haver atraso de replicação, erro de ambiente ou evento legítimo anterior; classifico antes de rejeitar ou aplicar cegamente.

Termo para lembrar: Orphan callback | Base técnica: referência 1.

11. Por que retornar rápido ao webhook?

Evita timeout e reentrega desnecessária, desde que já exista responsabilidade durável. A resposta rápida não significa processar tudo em memória; separo confirmação de recebimento e conclusão do negócio.

Termo para lembrar: Webhook acknowledgement | Base técnica: referência 1.

12. Como testa assinatura e replay?

Uso eventos válidos, payload alterado, segredo incorreto e evento repetido ou antigo conforme a janela do provedor. Verifico rejeição e deduplicação sem expor segredos nem aceitar bypass especial em produção.

Termo para lembrar: Signature testing | Base técnica: referência 1.

13. SFTP confirma o processamento da bandeira?

Não. O status de transferência informa um estágio do envio de arquivo. Preciso de recibo e retorno de negócio para saber quais itens foram aceitos, rejeitados ou continuam sem resultado.

Termo para lembrar: Transfer status | Base técnica: referência 2.

14. O que fazer quando envio SFTP perde conexão?

Preservo arquivo, checksum e batch_id. Dependendo do ponto da falha, o parceiro pode ter recebido; registro UNKNOWN e consulto evidência prevista antes de criar outro lote com os mesmos efeitos.

Termo para lembrar: SFTP UNKNOWN | Base técnica: referência 3.

15. Arquivo não está no diretório remoto: posso reenviar?

Não só por isso. O parceiro pode mover arquivos após consumi-los. A ausência precisa ter semântica contratual; uso recibos, consulta ou reconciliação para determinar se repetir comunicação é seguro.

Termo para lembrar: Remote absence | Base técnica: referência 3.

16. Novo attempt_id deve mudar nome do arquivo?

Não necessariamente. A tentativa física e a identidade lógica são diferentes. Se o nome integra deduplicação do parceiro, mudá-lo em cada retry pode transformar a mesma intenção em novo lote aceito.

Termo para lembrar: Attempt identity | Base técnica: referência 3.

17. Como garante integridade do arquivo?

Calculo checksum sobre bytes finais e registro layout e manifesto. Integridade não comprova validade de negócio; o retorno pode rejeitar registros mesmo quando a transferência e o checksum estão corretos.

Termo para lembrar: Artifact checksum | Base técnica: referência 2.

18. Como evita leitura de arquivo parcial?

Uso protocolo de publicação combinado, como nome temporário e sinal de prontidão quando suportado. Não assumo atomicidade de rename ou processamento do parceiro sem verificar seu contrato e implementação.

Termo para lembrar: Ready marker | Base técnica: referência 2.

19. Como trata retorno parcial?

Atualizo resultado por item com identidade e versão, mantendo pendência dos ausentes. Um lote pode ter aceitos e rejeitados; repetir todos porque o status agregado falhou pode duplicar operações já concluídas.

Termo para lembrar: Per-item outcome | Base técnica: referência 1.

20. Quotas contam chamadas ou operações?

Depende do contrato. Registro a unidade exata, incluindo tentativas físicas quando exigido. Timeout pode consumir quota; devolver orçamento automaticamente porque não recebi resposta pode ultrapassar o limite real do parceiro.

Termo para lembrar: External quota | Base técnica: referência 3.

21. Como lida com parceiro indisponível por horas?

Persisto trabalho, aplico backoff e circuito e monitoro prazo e backlog. Recuperação drena com limite; não despejo toda a fila assim que o parceiro responde, evitando causar nova indisponibilidade.

Termo para lembrar: Store-and-forward | Base técnica: referência 3.

22. Como troca de provedor durante uma falha?

Só encaminho a mesma intenção para outro quando houver segurança contra efeito duplo. Uma tentativa anterior UNKNOWN precisa de resolução ou protocolo de cancelamento; disponibilidade aparente não justifica pagamento duplicado.

Termo para lembrar: Provider failover | Base técnica: referência 3.

23. Como protege credenciais SFTP?

Uso gerenciamento de segredos, rotação e validação da identidade do servidor. Desativar host-key verification para conectar rapidamente permite destino falso; a operação precisa ter procedimento seguro para troca legítima de chave.

Termo para lembrar: Host key verification | Base técnica: referência 2.

24. Como evolve layout de arquivo?

Versiono parser e gerador e testo exemplos e totais com o parceiro. Encoding, separadores, casas decimais e campos obrigatórios fazem parte do contrato; compatibilidade não é apenas conseguir abrir o arquivo.

Termo para lembrar: File contract | Base técnica: referência 2.

25. Como trata resposta do parceiro contraditória?

Preservo ambas as evidências e aplico regra de precedência contratual ou reconciliação. Não sobrescrevo um estado financeiro confirmado por qualquer callback posterior sem verificar identidade, versão e significado da mudança.

Termo para lembrar: Conflicting evidence | Base técnica: referência 1.

26. Como evita timeout dentro de transação de banco?

Persisto intenção em transação curta, chamo parceiro fora dela e salvo resultado depois com versão. Segurar lock durante rede aumenta contenção e ainda não torna o parceiro participante atômico do banco.

Termo para lembrar: Short transaction | Base técnica: referência 3.

27. Como recupera após restore local?

Reconcilio IDs e resultados com o parceiro antes de reenviar pendências restauradas. O sistema externo pode manter operações posteriores ao backup; usar a mesma identidade ajuda, mas preciso conhecer sua retenção de deduplicação.

Termo para lembrar: Restore reconciliation | Base técnica: referência 3.

28. Que painel operacional oferece?

Mostro intenção, tentativas, artefato, evidências recebidas e próximo passo permitido. Ações de recuperação são autorizadas e auditadas; não obrigo operador a editar status no banco para tentar entender resultado desconhecido.

Termo para lembrar: Operational console | Base técnica: referência 2.

29. Quais testes de contrato são essenciais?

Simulo duplicata, atraso, assinatura inválida, rejeição parcial, quota e queda após aceitação. Também valido ambiente e credenciais; homologação só do caminho feliz deixa sem resposta os incidentes mais prováveis.

Termo para lembrar: Integration fault matrix | Base técnica: referência 1.

30. Como defenderia a integração?

Eu separaria intenção, transferência e resultado, mostrando IDs e evidências em cada etapa. A recuperação automática existe onde o contrato permite; onde falta certeza, preservo UNKNOWN e reconciliação em vez de inventar sucesso.

Termo para lembrar: External guarantee boundary | Base técnica: referência 3.

10 trade-offs: ganho, custo e decisão

01. API síncrona

Ganho. Resposta rápida

Custo e motivo. Acoplamento à disponibilidade

Como decidir. Quando parceiro oferece resultado imediato

02. Arquivo batch

Ganho. Amortiza operações

Custo e motivo. Atraso e retorno parcial

Como decidir. Quando contrato exige lote

03. Webhook

Ganho. Atualização assíncrona

Custo e motivo. Duplicata e reordenação

Como decidir. Com inbox e validação

04. Polling de status

Ganho. Controle de consulta

Custo e motivo. Carga e atraso

Como decidir. Quando callback não basta

05. Adapter central

Ganho. Contrato isolado

Custo e motivo. Componente adicional

Como decidir. Para parceiro com peculiaridades

06. Retry automático

Ganho. Recupera transitório

Custo e motivo. Risco de duplicar efeito

Como decidir. Com identidade reconhecida

07. UNKNOWN explícito

Ganho. Preserva verdade

Custo e motivo. Exige reconciliação

Como decidir. Após resposta ambígua

08. Artefato imutável

Ganho. Auditoria e repetição segura

Custo e motivo. Correção gera revisão

Como decidir. Após primeira tentativa

09. Circuito por parceiro

Ganho. Isola indisponibilidade

Custo e motivo. Adia trabalho

Como decidir. Com fila e prazo definidos

10. Intervenção controlada

Ganho. Resolve exceção

Custo e motivo. Tempo e custo

Como decidir. Quando evidência externa é insuficiente

Duas alternativas ao caminho proposto

1. API assíncrona com consulta

Quando faz sentido. Parceiro oferece operação identificada

Como implementar. Persistir operation_id e acompanhar status

Limites. Depende da retenção e semântica do destino

2. Mensageria contratual direta

Quando faz sentido. Ambos suportam broker ou protocolo acordado

Como implementar. Eventos com identidade, confirmação e reconciliação

Limites. Não elimina fronteira de efeito externo

Referências e alcance da evidência

1. Stripe - Webhooks

Validação, entregas repetidas, ordem e processamento assíncrono de eventos.

https://docs.stripe.com/webhooks

2. AWS - Monitoring SFTP connectors

Status de transferência, eventos e logs; processamento de negócio exige confirmação própria.

https://docs.aws.amazon.com/transfer/latest/userguide/track-connector-progress.html

3. AWS Builders Library - Idempotent APIs

Identidades estáveis, intenção, retries e requisições atrasadas.

https://aws.amazon.com/builders-library/making-retries-safe-with-idempotent-APIs/

Continue em: T049 · T050 · T061 · T064

T061 / G. Correção e resiliência

Idempotência

Idempotência reconhece a repetição da mesma intenção e preserva o efeito observável sob um contrato explícito.

Quando usar. Comandos com efeitos, retries, webhooks, jobs e integrações financeiras.

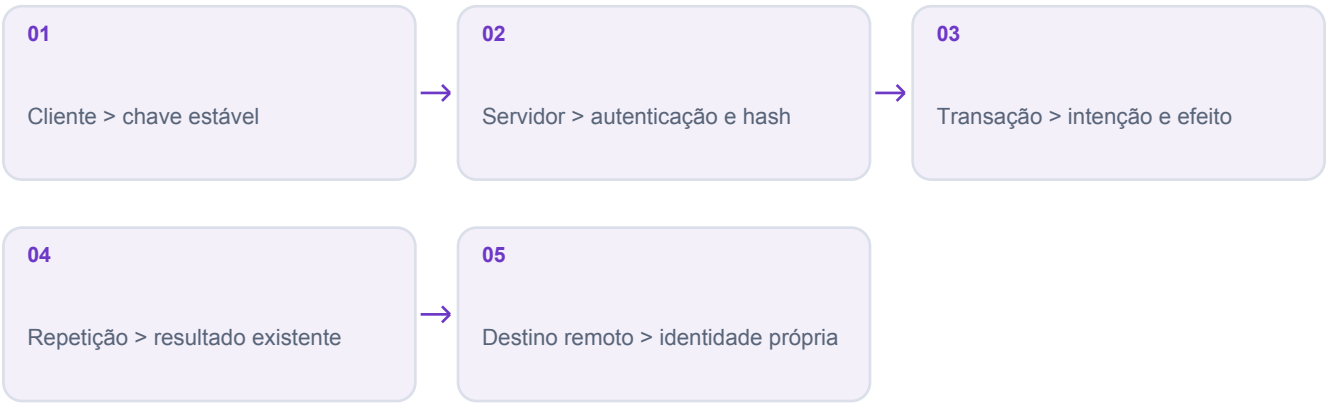
Quando evitar ou limitar. Não confundir resposta repetível com autorização, controle de concorrência ou unicidade de negócio.

Uma abertura possível

Vou escolher a identidade da intenção, guardar seu resultado com o efeito local e explicar a proteção no destino remoto.

Peças e tecnologias. PostgreSQL UNIQUE · DynamoDB conditional writes · Redis como aceleração · API Idempotency-Key

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Definir escopo e duração da chave

2. Normalizar payload e detectar conflitos

3. Gravar proteção e efeito local atomicamente

4. Preservar identidade em retries externos

5. Testar concorrência, timeout e replay tardio

30 perguntas para sustentar o desenho

01. O que exatamente fica idempotente?

Defino o efeito observável protegido, como criar uma operação por intenção. A resposta pode variar conforme progresso, mas repetir a mesma intenção não deve executar novamente o efeito coberto pelo contrato.

Termo para lembrar: Idempotency contract | Base técnica: referência 1.

02. Quem gera a chave?

Normalmente o originador cria a chave antes da primeira tentativa e a preserva durante retries. Se o servidor gera outra a cada recebimento, não consegue reconhecer a intenção repetida após timeout.

Termo para lembrar: Idempotency key | Base técnica: referência 1.

03. Qual é o escopo da chave?

Uso identidade do cliente ou tenant e tipo de operação, conforme contrato. Uma chave isolada pode colidir entre usuários; um escopo amplo demais pode impedir duas operações legítimas independentes.

Termo para lembrar: Key scope | Base técnica: referência 1.

04. O payload participa da comparação?

Sim. Guardo hash de uma representação estável dos campos relevantes. Se a mesma chave chegar com outro valor ou destinatário, retorno conflito; não reutilizo resposta nem substituo intenção silenciosamente.

Termo para lembrar: Payload hash | Base técnica: referência 1.

05. Hash do payload pode substituir a chave?

Nem sempre. Duas transferências legítimas podem ter conteúdo igual. O hash verifica equivalência da intenção identificada; ele não demonstra sozinho que duas requisições representam a mesma decisão do usuário.

Termo para lembrar: Semantic intent | Base técnica: referência 1.

06. Como decide concorrência entre retries?

Faço inserção protegida por constraint única na transação autoritativa. Requisições concorrentes disputam a mesma identidade; apenas a vencedora inicia o efeito, enquanto as demais recuperam estado ou aguardam resultado.

Termo para lembrar: Unique constraint | Base técnica: referência 2.

07. SELECT antes de INSERT é suficiente?

Não. Duas transações podem observar ausência simultaneamente. A unicidade precisa ser garantida na escrita; a consulta é útil para recuperar resposta, mas não substitui a constraint contra corrida.

Termo para lembrar: Check-then-act race | Base técnica: referência 2.

08. Como acopla registro e efeito local?

Gravo identidade, alteração de negócio e obrigação de saída na mesma transação. Se algo falha, tudo sofre rollback; registrar a chave em outro sistema cria uma nova janela de inconsistência.

Termo para lembrar: Atomic transaction | Base técnica: referência 3.

09. Posso usar SETNX e depois debitar PostgreSQL?

Isso não oferece atomicidade entre os dois sistemas. Se marcar Redis e cair antes do débito, posso perder trabalho; se debitar e perder a marca, posso repetir. Prefiro proteção junto do efeito.

Termo para lembrar: Dual write | Base técnica: referência 3.

10. O que retorna enquanto está em processamento?

Retorno um identificador consultável e estado explícito, como ACCEPTED ou PROCESSING. Não apresento sucesso financeiro antes da confirmação; o cliente pode repetir a mesma chave ou consultar a operação existente.

Termo para lembrar: Async operation | Base técnica: referência 1.

11. Devo guardar a resposta inteira?

Depende de estabilidade e privacidade. Posso guardar referência e resultado necessário para reconstruir resposta autorizada; dados sensíveis e conteúdo mutável exigem retenção e semântica definidas para retries futuros.

Termo para lembrar: Response replay | Base técnica: referência 1.

12. Como trata autorização revogada no replay?

Verifico identidade e autorização atuais antes de revelar resultado persistido. A chave reconhece intenção, não concede permissão; um usuário que perdeu acesso não deve ler dados apenas porque conhece uma chave antiga.

Termo para lembrar: Authorization on replay | Base técnica: referência 1.

13. Idempotência impede dois débitos com chaves diferentes?

Não. Isso exige regra de negócio adicional, como uma operação por autorização de pagamento. Separo deduplicação técnica de invariantes financeiras, preservando a possibilidade de operações parciais ou repetidas legitimamente.

Termo para lembrar: Business uniqueness | Base técnica: referência 3.

14. Como diferencia idempotência de controle otimista?

Idempotência reconhece repetição da mesma intenção. Controle otimista verifica se a decisão foi baseada numa versão ainda válida. Posso precisar dos dois para impedir retry duplicado e alteração sobre estado desatualizado.

Termo para lembrar: Optimistic concurrency | Base técnica: referência 3.

15. O que fazer após timeout externo?

Preservo a identidade e considero que o destino pode ter executado. Consulto resultado ou repito conforme contrato idempotente; sem esse suporte, mantenho UNKNOWN e reconcilio em vez de gerar nova operação.

Termo para lembrar: UNKNOWN | Base técnica: referência 1.

16. O ledger deve receber a mesma chave da API?

Pode receber um identificador derivado estável, com escopo próprio. O importante é mapear a intenção autorizada ao efeito financeiro sem criar novo ID a cada tentativa do worker ou troca de transporte.

Termo para lembrar: Operation mapping | Base técnica: referência 1.

17. Estorno usa a chave do crédito?

Não. O estorno é uma intenção distinta, ligada ao lançamento original. Ele possui identidade própria e proteção contra repetição; reutilizar a chave do crédito confundiria dois efeitos de negócio diferentes.

Termo para lembrar: Compensation identity | Base técnica: referência 1.

18. Qual retenção da chave é suficiente?

Considero retries, reentrega, DLQ, replay e recuperação de desastre. Se apagar cedo, a intenção antiga pode parecer nova; efeitos financeiros podem exigir unicidade duradoura no registro do próprio efeito.

Termo para lembrar: Deduplication retention | Base técnica: referência 1.

19. O que acontece depois do TTL?

A proteção temporária pode desaparecer. Defino o contrato para retries tardios e mantenho invariantes de negócio adicionais quando necessário; não prometo idempotência eterna usando apenas uma chave com expiração curta.

Termo para lembrar: TTL boundary | Base técnica: referência 1.

20. Como reutilizar uma chave antiga com segurança?

Prefiro não reutilizar dentro do escopo protegido. Se o contrato permitir reutilização, preciso distinguir gerações e deixar a janela explícita; caso contrário uma mensagem atrasada pode colidir com intenção nova.

Termo para lembrar: Key generation | Base técnica: referência 1.

21. Como migrar o formato de chave?

Mantenho mapeamento e compatibilidade durante a transição. Não recalculo IDs históricos de modo incompatível; produtores antigos, redrive e backups precisam continuar reconhecendo a mesma intenção financeira já processada.

Termo para lembrar: Identity migration | Base técnica: referência 1.

22. E se restaurar um backup antigo?

O estado restaurado pode esquecer intenções executadas externamente. Antes do retry, reconcilio com destinos e preservo IDs reconhecidos por eles; backup local não desfaz efeitos que ocorreram depois daquele ponto.

Termo para lembrar: Restore reconciliation | Base técnica: referência 1.

23. O produtor Kafka idempotente resolve minha API?

Não. Sua garantia cobre publicação sob condições do protocolo, não a intenção do usuário nem qualquer chamada externa. Mantenho idempotência na API e no domínio responsável pelo efeito final.

Termo para lembrar: Guarantee scope | Base técnica: referência 1.

24. PUT é sempre seguro financeiramente?

A semântica HTTP ajuda a definir repetição, mas a implementação pode incluir efeitos não idempotentes. Um PUT que dispara novo crédito a cada chamada viola o contrato; verifico o comportamento real.

Termo para lembrar: Semantic idempotency | Base técnica: referência 1.

25. Como cancela uma intenção em processamento?

Registro cancelamento como comando sujeito a estado e versão. O efeito pode estar em voo; acompanho resultado e eventual compensação antes de afirmar cancelamento concluído. Repetir cancelamento também precisa ser seguro.

Termo para lembrar: Cancellation state | Base técnica: referência 3.

26. Como tratar mesma chave com schema novo?

Comparo significado conforme versão de normalização ou contrato original. Mudança de serialização não deve transformar retry legítimo em conflito, nem permitir alteração semântica escondida sob hash calculado de forma diferente.

Termo para lembrar: Canonicalization version | Base técnica: referência 1.

27. Quais métricas indicam problema?

Acompanho conflitos de payload, retries por intenção, operações presas e divergências com o destino. Duplicatas tratadas são esperadas em algum nível; aumento súbito pode revelar timeout inadequado ou falha de publicação.

Termo para lembrar: Idempotency telemetry | Base técnica: referência 1.

28. Qual teste mais importante?

Executo chamadas concorrentes com a mesma chave e interrompo o processo após o efeito antes da resposta. Verifico um único efeito protegido e recuperação do resultado, incluindo payload divergente e autorização revogada.

Termo para lembrar: Concurrency test | Base técnica: referência 2.

29. Idempotência substitui reconciliação?

Não. Ela reduz risco de duplicação, mas não garante que todo trabalho terminou nem resolve dados divergentes entre sistemas. Reconciliação verifica completude e resultado, especialmente após falhas e restaurações.

Termo para lembrar: Reconciliation | Base técnica: referência 1.

30. Como defenderia o desenho em entrevista?

Eu apontaria a chave, seu escopo, onde a unicidade é imposta e qual transação protege o efeito. Depois mostraria a falha após commit e explicaria como o retry recupera o resultado.

Termo para lembrar: Proof by failure | Base técnica: referência 2.

10 trade-offs: ganho, custo e decisão

01. Chave do cliente

Ganho. Preserva intenção antes do envio

Custo e motivo. Cliente precisa persistir

Como decidir. Para comandos sujeitos a timeout

02. Chave de negócio

Ganho. Proteção independente do transporte

Custo e motivo. Modelagem complexa

Como decidir. Quando várias requisições representam um efeito

03. Retenção longa

Ganho. Protege replay tardio

Custo e motivo. Custo e privacidade

Como decidir. Para risco financeiro duradouro

04. Retenção curta

Ganho. Menos armazenamento

Custo e motivo. Retry antigo pode repetir

Como decidir. Somente com contrato e proteção adicional

05. Resposta persistida

Ganho. Replay previsível

Custo e motivo. Pode conter dados sensíveis

Como decidir. Quando resultado precisa ser estável

06. Referência ao resultado

Ganho. Menos duplicação

Custo e motivo. Reconstrução exige consulta

Como decidir. Quando estado permanece acessível

07. Constraint local

Ganho. Atomicidade com efeito

Custo e motivo. Contenção na chave

Como decidir. Para dados no mesmo banco

08. Cache auxiliar

Ganho. Reduz leitura repetida

Custo e motivo. Pode ficar desatualizado

Como decidir. Sem remover proteção autoritativa

09. Conflito de hash

Ganho. Detecta intenção diferente

Custo e motivo. Normalização precisa evoluir

Como decidir. Quando chave é reutilizável por cliente

10. Reconciliação remota

Ganho. Resolve ambiguidade

Custo e motivo. Aumenta prazo

Como decidir. Quando resposta externa se perde

Duas alternativas ao caminho proposto

1. Operação naturalmente idempotente

Quando faz sentido. Estado absoluto versionado

Como implementar. Aplicar a mesma representação sem novos side effects

Limites. Não serve para incrementos ou efeitos financeiros arbitrários

2. Unicidade no domínio

Quando faz sentido. Intenção corresponde a entidade única

Como implementar. Constraint na operação e gravação atômica do resultado

Limites. Exige chave de negócio correta e tratamento de revisões

Referências e alcance da evidência

1. AWS Builders Library - Idempotent APIs

Identidades estáveis, intenção, retries e requisições atrasadas.

https://aws.amazon.com/builders-library/making-retries-safe-with-idempotent-APIs/

2. PostgreSQL - INSERT

ON CONFLICT, RETURNING e constraints usadas para arbitrar inserções.

https://www.postgresql.org/docs/current/sql-insert.html

3. PostgreSQL - Transaction isolation

Snapshots, concorrência, isolamento e falhas de serialização.

https://www.postgresql.org/docs/current/transaction-iso.html

Continue em: T044 · T062 · T063 · T064

T062 / G. Correção e resiliência

Transactional Outbox

Transactional Outbox grava mudança de estado e obrigação de publicar na mesma transação local.

Quando usar. Serviço precisa persistir dados e notificar outro sistema sem perder a obrigação entre essas ações.

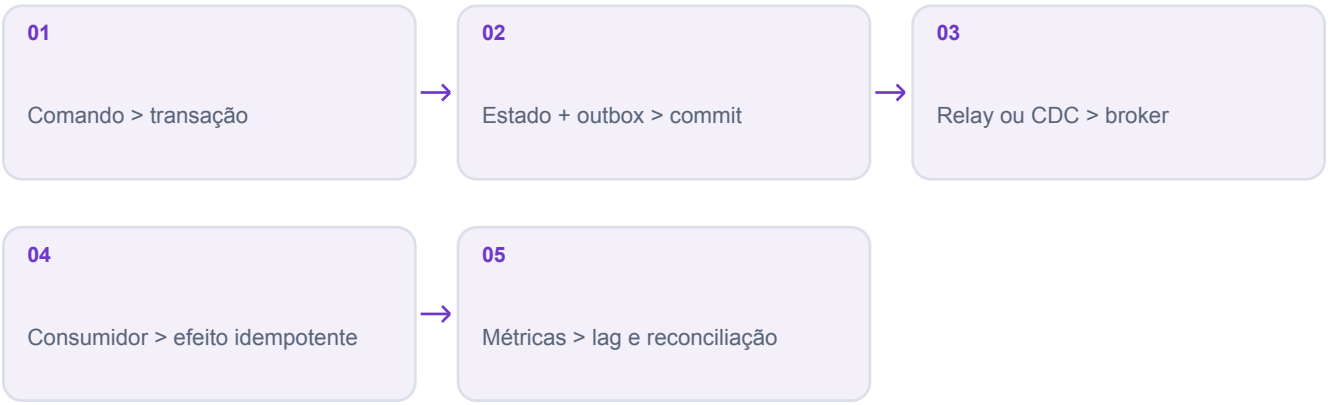
Quando evitar ou limitar. Não afirmar que outbox garante entrega única, ordem global ou efeito único no consumidor.

Uma abertura possível

Vou tornar a publicação recuperável junto do commit de negócio; o relay poderá repetir e o consumidor reconhecerá a identidade.

Peças e tecnologias. PostgreSQL · Debezium · Kafka · SQS · DynamoDB Streams

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Criar event_id estável e versão de agregado

2. Inserir estado e evento na mesma transação

3. Publicar somente registros confirmados

4. Marcar progresso após confirmação sem prometer unicidade

5. Monitorar retenção, lag, replay e consumidores

30 perguntas para sustentar o desenho

01. Qual falha a outbox resolve?

Ela elimina a janela local entre gravar o negócio e esquecer de publicar. Estado e obrigação de saída confirmam juntos; o envio posterior é recuperável, embora possa ocorrer mais de uma vez.

Termo para lembrar: Dual-write problem | Base técnica: referência 1.

02. Por que não publicar antes do commit?

O consumidor poderia agir sobre uma alteração que depois sofre rollback. Publicar depois sem registro durável também falha se o processo cair; a outbox persiste a obrigação dentro da transação autoritativa.

Termo para lembrar: Atomic intent | Base técnica: referência 1.

03. Que campos coloca no evento?

Incluo event_id, agregado, versão, tipo, schema e payload ou referência imutável. Diferencio horário informativo de sequência; os campos precisam permitir deduplicação, interpretação e rastreabilidade sem expor dados desnecessários.

Termo para lembrar: Event envelope | Base técnica: referência 1.

04. Como gera event_id?

Gero uma identidade estável para o evento confirmado. O relay conserva esse ID em cada tentativa; criar um novo por publicação impediria que consumidores reconhecessem a repetição após confirmação ambígua.

Termo para lembrar: Stable event ID | Base técnica: referência 3.

05. O relay pode ler transação incompleta?

Ele deve observar apenas alterações confirmadas. Com leitura transacional apropriada, registros não confirmados não ficam disponíveis; CDC também precisa respeitar os limites de commit previstos pelo conector e origem.

Termo para lembrar: Committed visibility | Base técnica: referência 2.

06. Como dois relays dividem trabalho?

Faço claim em lote limitado com locks curtos ou lease, evitando manter transação aberta durante rede. A proteção admite recuperação e duplicatas; consumidores continuam responsáveis por efeitos idempotentes.

Termo para lembrar: Relay claim | Base técnica: referência 2.

07. SKIP LOCKED garante FIFO?

Não. Ele permite pular linhas ocupadas e melhora concorrência, mas pode inverter ordem de publicação. Se a sequência por agregado importa, controlo claims por agregado ou trato versões e gaps no consumidor.

Termo para lembrar: Ordering boundary | Base técnica: referência 2.

08. O que acontece se publicar e cair antes de marcar?

O evento continua pendente e será publicado novamente. Esse comportamento é esperado; manter event_id e consumidor idempotente evita repetir o efeito, enquanto preservamos recuperação da obrigação.

Termo para lembrar: Publish-ack gap | Base técnica: referência 1.

09. Pode marcar antes de enviar para evitar duplicata?

Não, porque uma queda após marcar e antes do envio perderia a publicação. Prefiro repetição reconhecível a perda silenciosa de trabalho aceito, explicitando essa escolha no contrato do consumidor.

Termo para lembrar: At-least-once relay | Base técnica: referência 1.

10. Publisher confirm significa negócio concluído?

Não. Ele confirma uma etapa de transporte segundo o broker. O consumidor pode ainda estar atrasado ou falhar; monitoro publicação e conclusão de negócio separadamente para não anunciar sucesso cedo.

Termo para lembrar: Transport acknowledgement | Base técnica: referência 1.

11. Como preserva ordem por agregado?

Gravo versão junto do estado e evito que publicadores concorrentes invertam aquela sequência, ou permito ao consumidor adiar gaps. A simples existência da tabela não serializa automaticamente múltiplos relays.

Termo para lembrar: Aggregate sequence | Base técnica: referência 1.

12. Timestamp pode substituir sequência?

Não como regra universal. Relógios, empates e ordem de commit podem divergir. Uso versão gerada na autoridade do agregado quando há dependência, deixando timestamp para análise temporal e auditoria.

Termo para lembrar: Logical version | Base técnica: referência 2.

13. Polling ou CDC?

Polling é simples e explícito, mas adiciona consultas e intervalo. CDC acompanha log transacional e reduz certas leituras, porém exige operação de conectores, offsets e retenção do log de origem.

Termo para lembrar: CDC versus polling | Base técnica: referência 1.

14. CDC dispensa contrato de evento?

Não. Capturar alteração de linha não define automaticamente um evento de domínio estável. Evito acoplar consumidores a detalhes internos de schema, especialmente quando várias tabelas representam uma única intenção.

Termo para lembrar: Domain event | Base técnica: referência 1.

15. Como evita payload mudar antes da publicação?

Persisto o conteúdo relevante ou referência imutável no commit. Se o relay buscar estado atual depois, pode publicar uma versão diferente daquela que gerou a obrigação e perder informação intermediária.

Termo para lembrar: Immutable payload | Base técnica: referência 1.

16. Quando limpar eventos publicados?

Depois de considerar replay, auditoria operacional e confirmação necessária. Limpeza em lotes evita carga abrupta; não apago eventos pendentes nem suponho que todos os consumidores concluíram porque o broker recebeu.

Termo para lembrar: Outbox retention | Base técnica: referência 1.

17. Como dimensiona crescimento da outbox?

Estimo taxa de eventos, bytes e pior duração de indisponibilidade do relay. A tabela pode crescer durante falhas; monitoro espaço, índice, WAL e idade do evento mais antigo para agir antes do limite.

Termo para lembrar: Backlog capacity | Base técnica: referência 2.

18. Se o broker cair, a API pode continuar?

Pode enquanto o banco suporta a obrigação e o contrato admite atraso. Defino limite de backlog e prazo; aceitar indefinidamente sem capacidade futura transforma disponibilidade aparente em descumprimento de negócio.

Termo para lembrar: Admission control | Base técnica: referência 1.

19. Como trata um evento que não publica?

Classifico erro temporário e permanente, preservo identidade e sinalizo pendência. Payload inválido pode exigir correção rastreável; não bloqueio entidades independentes nem removo a obrigação silenciosamente para esvaziar a tabela.

Termo para lembrar: Poison outbox event | Base técnica: referência 1.

20. Outbox garante efeito único no ledger?

Não. Ela garante obrigação local de enviar. O ledger precisa reconhecer operation_id ou oferecer protocolo equivalente; após timeout, a publicação pode repetir enquanto o crédito anterior já aconteceu.

Termo para lembrar: Remote idempotency | Base técnica: referência 3.

21. Como conecta inbox e outbox?

Na transação do consumidor, registro evento recebido, altero estado e crio obrigação de saída. Assim uma reentrega não gera nova transição; a etapa seguinte continua protegida pela própria identidade.

Termo para lembrar: Inbox-outbox chain | Base técnica: referência 1.

22. Pode usar tabela de jobs como outbox?

Pode se ela representa duravelmente a obrigação de saída e participa da mesma transação. O nome da tabela não importa; importam atomicidade, identidade, retomada e semântica de conclusão.

Termo para lembrar: Durable obligation | Base técnica: referência 1.

23. Como publica eventos de várias tabelas?

A aplicação pode formar um evento de domínio explícito na mesma transação. Não deixo o consumidor adivinhar quando um conjunto de mudanças técnicas ficou completo sem um contrato correspondente.

Termo para lembrar: Transaction boundary | Base técnica: referência 2.

24. Como evolui schema do payload?

Versiono contratos e mantenho consumidores compatíveis com eventos antigos ainda pendentes ou reprocessáveis. Um deploy pode encontrar meses de histórico; testar apenas o produtor novo contra consumidor novo é insuficiente.

Termo para lembrar: Schema compatibility | Base técnica: referência 1.

25. Como migra o relay sem perder progresso?

Preservo IDs e estado de publicação, faço corte controlado e aceito duplicatas protegidas durante coexistência. Não uso duas implementações com critérios diferentes de pendência sem reconciliar o conjunto publicado.

Termo para lembrar: Relay migration | Base técnica: referência 3.

26. Como restaura banco depois de desastre?

O backup pode trazer eventos que já foram publicados ou perder os mais recentes conforme RPO. Reconciliamos e mantemos deduplicação no destino; restaurar outbox antiga não significa que consumidores esqueceram seus efeitos.

Termo para lembrar: Restore replay | Base técnica: referência 3.

27. Como testa o padrão?

Interrompo após commit local, após envio e antes de marcar publicação. Verifico eventual saída e ausência de efeito duplicado no consumidor; também testo rollback para comprovar que nada sai de transação abortada.

Termo para lembrar: Fault injection | Base técnica: referência 1.

28. Quais indicadores são prioritários?

Idade do evento pendente mais antigo, taxa de publicação, falhas por causa, espaço ocupado e atraso do consumidor. A API pode continuar rápida enquanto obrigações críticas envelhecem sem chegar ao destino.

Termo para lembrar: Outbox lag | Base técnica: referência 1.

29. Como controla dados sensíveis?

Publico apenas o necessário, com autorização e retenção apropriadas em banco e broker. A outbox cria outra cópia persistente; mascarar a API não protege automaticamente dados já presentes no payload do evento.

Termo para lembrar: Data minimization | Base técnica: referência 1.

30. Como defenderia a solução?

Eu apontaria o commit que torna estado e obrigação indivisíveis, depois a janela em que o relay pode repetir. A defesa inclui consumidor idempotente e métricas de atraso, não uma promessa genérica de exactly-once.

Termo para lembrar: Guarantee boundary | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Polling

Ganho. Implementação explícita

Custo e motivo. Consultas e atraso

Como decidir. Para volume moderado

02. CDC

Ganho. Segue log transacional

Custo e motivo. Conectores e retenção

Como decidir. Quando infraestrutura suporta operação

03. Payload completo

Ganho. Evento reproduzível

Custo e motivo. Mais armazenamento

Como decidir. Para histórico sem consulta posterior

04. Referência imutável

Ganho. Menor evento

Custo e motivo. Dependência de objeto

Como decidir. Para dados grandes

05. Claims paralelos

Ganho. Mais throughput

Custo e motivo. Ordem pode mudar

Como decidir. Para agregados independentes

06. Serialização por agregado

Ganho. Preserva sequência

Custo e motivo. Chave quente

Como decidir. Quando deltas dependem de ordem

07. Retenção longa

Ganho. Replay e diagnóstico

Custo e motivo. Disco e exposição

Como decidir. Conforme janela operacional

08. Lotes grandes

Ganho. Amortiza trabalho

Custo e motivo. Mais memória e atraso

Como decidir. Após medir p99 e limites

09. Aceitar com broker fora

Ganho. Preserva entrada

Custo e motivo. Backlog e prazo

Como decidir. Com capacidade e limite explícitos

10. Mesmo banco

Ganho. Atomicidade local

Custo e motivo. Acoplamento ao domínio

Como decidir. Quando estado e obrigação precisam confirmar juntos

Duas alternativas ao caminho proposto

1. Estado com CDC

Quando faz sentido. Eventos deriváveis de mudança confirmada

Como implementar. Capturar log e construir contrato estável

Limites. Schema interno e recuperação do conector exigem cuidado

2. Publicação direta sem obrigação

Quando faz sentido. Perda é explicitamente tolerada

Como implementar. Enviar telemetria não crítica com métricas de perda

Limites. Inadequado para obrigações financeiras aceitas

Referências e alcance da evidência

1. AWS - Transactional outbox

Gravação atômica de estado e obrigação de publicar; duplicatas ainda possíveis.

https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/transactional-outbox.html

2. PostgreSQL - Transaction isolation

Snapshots, concorrência, isolamento e falhas de serialização.

https://www.postgresql.org/docs/current/transaction-iso.html

3. AWS Builders Library - Idempotent APIs

Identidades estáveis, intenção, retries e requisições atrasadas.

https://aws.amazon.com/builders-library/making-retries-safe-with-idempotent-APIs/

Continue em: T040 · T043 · T044 · T063

T063 / G. Correção e resiliência

Transactional Inbox

Transactional Inbox registra processamento e efeito local numa mesma transação para tolerar reentrega.

Quando usar. Consumidores com incrementos, transições ou criação de obrigações que não podem repetir.

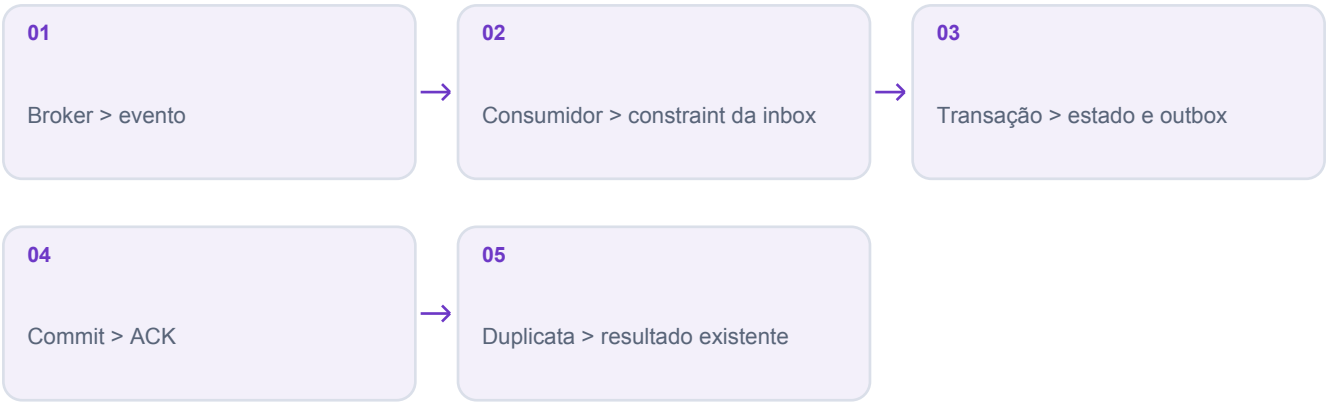
Quando evitar ou limitar. Não usar a marca de recebimento como prova de conclusão nem estender atomicidade local a efeitos remotos.

Uma abertura possível

Vou registrar a identidade recebida junto do efeito e da obrigação de saída; ACK acontece depois dessa durabilidade.

Peças e tecnologias. PostgreSQL · SQS · Kafka · RabbitMQ

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Definir consumidor lógico e event_id

2. Inserir com unicidade e comparar payload

3. Aplicar efeito e obrigação na mesma transação

4. Confirmar somente trabalho durável

5. Testar concorrência, gaps, retenção e restauração

30 perguntas para sustentar o desenho

01. Qual garantia a inbox oferece?

Ela impede reaplicar o mesmo evento ao estado local quando registro e efeito compartilham a transação. O broker pode repetir a entrega; a proteção está na autoridade que confirma a alteração.

Termo para lembrar: Transactional inbox | Base técnica: referência 1.

02. Quando vale adicionar essa tabela?

Quando o efeito não é naturalmente protegido, como incrementar uma projeção ou gerar obrigação financeira. Antes verifico se o próprio registro de negócio já impõe unicidade equivalente, evitando infraestrutura redundante sem benefício.

Termo para lembrar: Idempotent consumer | Base técnica: referência 3.

03. Qual chave escolheria?

Uso consumidor lógico, origem e event_id, com tenant quando necessário. Essa chave permite que consumidores independentes processem o mesmo evento, sem confundir uma nova implantação do serviço com nova responsabilidade de negócio.

Termo para lembrar: Consumer scope | Base técnica: referência 2.

04. Como resolve consumidores concorrentes?

Ambos tentam inserir a mesma chave protegida por constraint. Somente quem insere aplica o efeito; quem encontra conflito verifica hash e estado. Não dependo de uma consulta anterior para excluir concorrência.

Termo para lembrar: ON CONFLICT | Base técnica: referência 2.

05. Por que SELECT seguido de INSERT falha?

Duas transações podem observar ausência e iniciar trabalho simultaneamente. A decisão precisa estar na escrita protegida por unicidade; a consulta posterior serve para recuperar resultado ou classificar conflito, não para arbitrar sozinha.

Termo para lembrar: Check-then-act | Base técnica: referência 2.

06. Como trata ID igual com payload diferente?

Comparo hash de conteúdo normalizado e versão do contrato. Se divergir, registro conflito e não aplico arbitrariamente um dos valores; identidade repetida com intenção diferente é quebra de contrato, não duplicata válida.

Termo para lembrar: Payload conflict | Base técnica: referência 3.

07. O nome do consumer inclui versão do deploy?

Não automaticamente. Ele representa responsabilidade lógica estável. Trocar a versão do deploy na chave faria eventos antigos parecerem novos e poderia repetir efeitos; reconstrução de projeção usa namespace e destino explicitamente separados.

Termo para lembrar: Logical consumer | Base técnica: referência 3.

08. Event_id e operation_id resolvem o mesmo problema?

Não. Event_id identifica ocorrência publicada; operation_id identifica intenção de negócio. Eventos diferentes podem solicitar o mesmo crédito, então o domínio financeiro ainda precisa proteger unicidade da operação autorizada.

Termo para lembrar: Business identity | Base técnica: referência 3.

09. O que ocorre se cair antes do commit?

Registro da inbox, alteração local e obrigação de saída sofrem rollback. Sem confirmação do transporte, outra tentativa pode processar o evento; não deve existir chamada externa irreversível escondida dentro dessa transação local.

Termo para lembrar: Atomic rollback | Base técnica: referência 4.

10. E se cair após commit e antes do ACK?

A mensagem pode voltar, mas a inbox confirma que o efeito local já foi aplicado. A nova tentativa verifica identidade e conteúdo, reconhece a duplicata e confirma sem executar novamente a alteração.

Termo para lembrar: ACK gap | Base técnica: referência 1.

11. Posso confirmar antes de aplicar o efeito?

Somente após persistir uma responsabilidade durável de processamento posterior. Nesse desenho, guardo payload, estado pendente e scheduler recuperável; encontrar RECEIVED não significa que o negócio foi concluído e pode ser abandonado.

Termo para lembrar: Durable intake | Base técnica: referência 1.

12. Visibility timeout substitui inbox?

Não. Ele reduz competição temporária, mas não prova ausência de reentrega. Uma execução lenta pode coexistir com outra tentativa; a constraint e a transação continuam necessárias para proteger o efeito local.

Termo para lembrar: Visibility timeout | Base técnica: referência 5.

13. A inbox evita crédito remoto duplicado?

Não sozinha. Registro a obrigação de crédito junto do estado local e envio operation_id estável ao ledger. O destino precisa reconhecer repetição ou permitir reconciliação de resultado desconhecido após timeout.

Termo para lembrar: Remote side effect | Base técnica: referência 3.

14. Como diferencia recebido e concluído?

Defino estados explícitos: RECEIVED, DEFERRED e COMPLETED quando há ingestão assíncrona. Uma entrada recebida precisa de executor e recuperação; no fluxo direto, a linha confirmada pode representar conclusão local atômica.

Termo para lembrar: Processing state | Base técnica: referência 1.

15. Inbox também resolve ordenação?

Não. E12 pode chegar antes de E11 e não ser duplicata. Se o efeito depende da sequência, verifico a versão esperada do agregado e adio lacunas sem marcar a alteração como concluída.

Termo para lembrar: Sequence gap | Base técnica: referência 4.

16. Como confirma um evento adiado?

Persisto evento e obrigação de retomada antes do ACK. O sistema assume a responsabilidade de buscar a lacuna e executar depois; sem scheduler durável, confirmar um evento adiado poderia perder trabalho silenciosamente.

Termo para lembrar: Deferred event | Base técnica: referência 1.

17. Pode descartar toda versão antiga?

Somente quando a semântica permitir. Um snapshot completo pode substituir outro, mas um delta não aplicado não desaparece por chegar atrasado. Preciso distinguir evento coberto pelo estado e alteração financeira ainda ausente.

Termo para lembrar: Delta versus snapshot | Base técnica: referência 4.

18. O que faz com poison message?

Classifico falha permanente e transitória. Payload inválido exige quarentena com motivo; retry imediato repetiria o mesmo erro. Não registro COMPLETED como se o negócio tivesse ocorrido apenas para esvaziar a fila.

Termo para lembrar: Poison message | Base técnica: referência 5.

19. Como faz redrive com segurança?

Preservo identidade original quando o conteúdo continua igual e controlo vazão. Se corrigir significado, crio evento corretivo rastreável; reutilizar ID com payload diferente quebraria justamente a validação que protege o processamento.

Termo para lembrar: Controlled redrive | Base técnica: referência 3.

20. Qual retenção da inbox?

Considero retenção do broker, DLQ, replay e recuperação. Após remover a marca, a mensagem pode parecer nova; efeitos financeiros exigem proteção duradoura no domínio quando o horizonte de repetição não é curto.

Termo para lembrar: Deduplication retention | Base técnica: referência 3.

21. Como controla tamanho da tabela?

Estimo taxa e retenção, limpo em lotes e monitoro índices e WAL. Particionamento temporal facilita descarte, mas preciso verificar se a estratégia preserva a unicidade desejada entre partições e janelas de replay.

Termo para lembrar: Inbox storage | Base técnica: referência 2.

22. Processaria um lote numa transação?

Pode amortizar commits, mas aumenta locks e alcance do rollback. Uso lotes pequenos e separo mensagens independentes; um evento inválido não deve impedir milhares de outros de progredir sem necessidade de negócio.

Termo para lembrar: Batch transaction | Base técnica: referência 4.

23. Precisa de isolamento Serializable?

Não necessariamente. A constraint resolve duplicata da mesma identidade; invariantes entre eventos diferentes podem exigir locks, CAS ou Serializable. Se usar Serializable, preparo repetição da transação inteira após falha de serialização.

Termo para lembrar: Isolation level | Base técnica: referência 4.

24. Como migra o consumidor?

Preservo identidade lógica e compatibilidade com eventos antigos. Para reconstruir estado, uso destino isolado e controlo side effects; um replay de projeção não pode reenviar créditos ou notificações sem intenção explícita.

Termo para lembrar: Consumer migration | Base técnica: referência 3.

25. O produtor pode trocar event_id no retry?

Não para a mesma publicação lógica. Isso faria a inbox enxergar eventos diferentes; o produtor precisa persistir identidade estável. Correções reais têm nova identidade e relação explícita com o evento anterior.

Termo para lembrar: Producer identity | Base técnica: referência 3.

26. O banco caiu: posso confirmar e processar depois?

Não sem outro armazenamento durável já confirmado. Reduzo consumo e aplico backoff, observando retenção e prazo; confirmar na esperança de recuperar memória depois transforma indisponibilidade temporária em perda de trabalho.

Termo para lembrar: Fail closed | Base técnica: referência 5.

27. Quais métricas monitoraria?

Duplicatas, conflitos de hash, tempo de transação, espera por locks, eventos adiados e idade das obrigações externas. Uma alta de duplicatas pode indicar problema operacional mesmo quando a proteção mantém o estado correto.

Termo para lembrar: Inbox telemetry | Base técnica: referência 1.

28. Como prova a garantia com testes?

Entrego o mesmo evento simultaneamente e interrompo o processo antes e depois do commit. Também testo payload divergente, gap e replay tardio; verifico invariantes no estado autoritativo, não apenas logs de execução.

Termo para lembrar: Fault injection | Base técnica: referência 2.

29. Como backup antigo afeta a proteção?

Pode apagar marcas locais de efeitos já realizados externamente. Reconciliamos antes do replay e preservamos identidade no destino; restaurar inbox, estado e outbox coerentemente é necessário, mas não desfaz o mundo externo.

Termo para lembrar: Restore consistency | Base técnica: referência 3.

30. Como resumiria a escolha?

Eu aceito reentrega e protejo o efeito na transação local. Registro inbox, estado e obrigação de saída juntos; o efeito remoto tem contrato próprio. Assim cada falha possui um caminho explícito de recuperação.

Termo para lembrar: Guarantee boundary | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Inbox explícita

Ganho. Deduplicação genérica

Custo e motivo. Escrita e índice extras

Como decidir. Para efeitos sem proteção própria

02. Unicidade no domínio

Ganho. Menos tabelas

Custo e motivo. Acoplamento à intenção

Como decidir. Quando efeito já é entidade única

03. Retenção longa

Ganho. Protege replay tardio

Custo e motivo. Custo e exposição

Como decidir. Conforme horizonte de repetição

04. Transação por evento

Ganho. Rollback limitado

Custo e motivo. Mais commits

Como decidir. Para tarefas heterogêneas

05. Batch pequeno

Ganho. Amortiza overhead

Custo e motivo. Amplia impacto de falha

Como decidir. Quando eventos são simples

06. Hash de conteúdo

Ganho. Detecta ID reutilizado

Custo e motivo. Normalização versionada

Como decidir. Para contratos críticos

07. Receber e processar depois

Ganho. ACK rápido

Custo e motivo. Scheduler adicional

Como decidir. Para trabalho longo

08. Aplicar antes do ACK

Ganho. Fluxo simples

Custo e motivo. Retém entrega durante processamento

Como decidir. Para efeito local curto

09. Ordem rigorosa

Ganho. Preserva deltas

Custo e motivo. Gap bloqueia agregado

Como decidir. Quando eventos dependem da sequência

10. Mesmo banco do efeito

Ganho. Atomicidade local

Custo e motivo. Acoplamento de storage

Como decidir. Quando garantia é essencial

Duas alternativas ao caminho proposto

1. Registro de negócio único

Quando faz sentido. Cada evento cria um efeito identificável

Como implementar. UNIQUE(operation_id) com efeito e obrigação na mesma transação

Limites. Event_id diferente ainda pode exigir regra de intenção

2. Transição por estado e versão

Quando faz sentido. Sequência de mudanças é inequívoca

Como implementar. UPDATE condicional que registra conclusão e cria outbox atomicamente

Limites. Zero linhas exige distinguir duplicata, conflito e gap; não descartar deltas indiscriminadamente

Referências e alcance da evidência

1. AWS - Transactional outbox

Gravação atômica de estado e obrigação de publicar; duplicatas ainda possíveis.

https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/transactional-outbox.html

2. PostgreSQL - INSERT

ON CONFLICT, RETURNING e constraints usadas para arbitrar inserções.

https://www.postgresql.org/docs/current/sql-insert.html

3. AWS Builders Library - Idempotent APIs

Identidades estáveis, intenção, retries e requisições atrasadas.

https://aws.amazon.com/builders-library/making-retries-safe-with-idempotent-APIs/

4. PostgreSQL - Transaction isolation

Snapshots, concorrência, isolamento e falhas de serialização.

https://www.postgresql.org/docs/current/transaction-iso.html

5. AWS - SQS visibility timeout

Visibilidade temporária, renovação e possibilidade de reentrega.

https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-visibility-timeout.html

Continue em: T043 · T044 · T061 · T062

T064 / G. Correção e resiliência

Timeouts, retries, backoff, jitter e UNKNOWN

Timeout limita espera, retry repete tentativa e UNKNOWN preserva incerteza quando efeito externo pode ter ocorrido.

Quando usar. Chamadas distribuídas, transferência de arquivos e workers sujeitos a falhas transitórias.

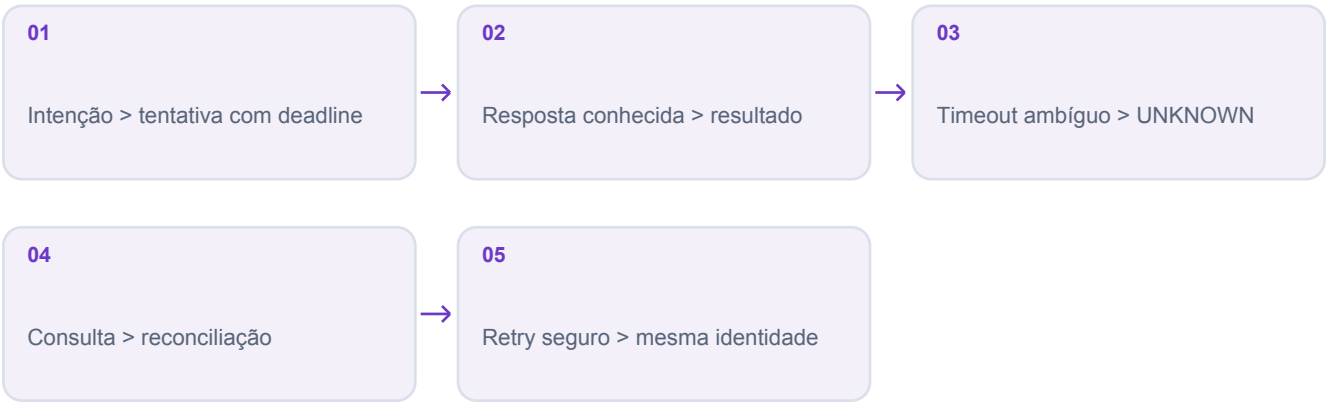
Quando evitar ou limitar. Não repetir comandos sem identidade nem tratar qualquer timeout como rejeição definitiva do destino.

Uma abertura possível

Vou separar falha comprovadamente anterior ao efeito de resposta perdida; só repito quando o contrato torna a tentativa segura.

Peças e tecnologias. Resilience4j · Envoy · SQS · Temporal · HTTP clients

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Definir deadline total e timeouts por estágio

2. Classificar erros e segurança de repetição

3. Persistir identidade e próxima tentativa

4. Aplicar backoff, jitter e orçamento

5. Reconciliar UNKNOWN e testar pontos de queda

30 perguntas para sustentar o desenho

01. Timeout prova que o servidor não executou?

Não. Ele prova que parei de esperar uma resposta dentro do limite. O servidor pode ter confirmado o efeito; por isso separo falha de transporte de rejeição de negócio e preservo UNKNOWN quando necessário.

Termo para lembrar: UNKNOWN | Base técnica: referência 1.

02. Como escolhe timeout de uma chamada?

Uso distribuição de latência, orçamento do usuário e custo de ocupar recursos. Defino margem para conexão e processamento, verificando quais fases o cliente cobre; um número arbitrário pode produzir retries desnecessários.

Termo para lembrar: Timeout budget | Base técnica: referência 1.

03. Connection timeout e read timeout são iguais?

Não. Um limita estabelecer conexão e outro espera de leitura conforme biblioteca. Também considero DNS, TLS, escrita e duração total; configuração parcial pode deixar uma fase fora do limite pretendido.

Termo para lembrar: Timeout phases | Base técnica: referência 1.

04. Qual é o deadline de ponta a ponta?

É o limite total da operação síncrona, propagado entre serviços. Cada etapa consome parte dele; não reinicio um orçamento completo a cada salto, pois isso ampliaria a espera além do compromisso com o cliente.

Termo para lembrar: Deadline propagation | Base técnica: referência 1.

05. Quando é seguro repetir um comando?

Quando há evidência de não execução ou contrato idempotente que reconhece a mesma intenção. Falha após possível envio exige cuidado; uso identidade estável e consulta de status, não nova operação por tentativa.

Termo para lembrar: Retry safety | Base técnica: referência 2.

06. GET pode sempre ser repetido sem custo?

A semântica pode ser idempotente, mas retries ainda consomem capacidade e podem amplificar incidente. Limito tentativas e respeito o deadline; uma leitura cara repetida em massa pode derrubar uma dependência saudável.

Termo para lembrar: Read retry | Base técnica: referência 1.

07. Erro 500 significa rollback?

Não. O servidor pode ter produzido efeito antes de falhar ao montar a resposta. Classifico conforme contrato da API e preservo idempotência; código HTTP sozinho não comprova ausência de consequência financeira.

Termo para lembrar: Ambiguous failure | Base técnica: referência 2.

08. Como trata erro de conexão antes do handshake?

Pode indicar que aquele canal não enviou a requisição, mas verifico o cliente e tentativas internas. A decisão considera evidência real de envio; não generalizo todos os erros de conexão como ausência garantida de efeito.

Termo para lembrar: Failure classification | Base técnica: referência 1.

09. Por que backoff exponencial?

Ele aumenta intervalo entre tentativas para reduzir pressão durante falha. Uso limite máximo e prazo total; esperar cada vez mais não torna uma operação insegura em segura nem resolve erro permanente.

Termo para lembrar: Exponential backoff | Base técnica: referência 1.

10. Para que serve jitter?

Ele espalha tentativas que de outro modo voltariam sincronizadas. Posso sortear atraso dentro de um intervalo limitado; o objetivo é suavizar rajadas, respeitando deadlines e janelas de negócio definidas.

Termo para lembrar: Jitter | Base técnica: referência 1.

11. Qual risco de retries em todas as camadas?

As tentativas se multiplicam. Três tentativas em vários saltos podem gerar muito mais chamadas ao destino; escolho responsabilidade e orçamento global, observando também retries embutidos em SDKs e proxies.

Termo para lembrar: Retry amplification | Base técnica: referência 1.

12. Como decide número máximo de tentativas?

Considero prazo, custo, probabilidade de recuperação e risco de repetição. Limite por contagem não basta: também imponho duração total e orçamento de carga para evitar consumir o serviço durante um incidente.

Termo para lembrar: Retry budget | Base técnica: referência 1.

13. Retry-After deve ser ignorado se o usuário espera?

Não arbitrariamente. Respeito o sinal do serviço e o deadline disponível, apresentando pendência ou indisponibilidade quando necessário. Repetir antes do prazo pode prolongar sobrecarga e prejudicar todos os clientes.

Termo para lembrar: Retry-After | Base técnica: referência 1.

14. Como persiste tentativas de um job?

Registro intenção, attempt_id, next_attempt_at, causa e resultado conhecido. Reiniciar worker não deve apagar backoff ou gerar nova intenção; histórico permite distinguir repetição física de operação financeira única.

Termo para lembrar: Durable retry state | Base técnica: referência 2.

15. O que significa estado UNKNOWN?

Significa que não tenho evidência suficiente para concluir sucesso ou rejeição. O sistema mantém responsabilidade de esclarecer o resultado; não o usa como estado terminal conveniente para abandonar a obrigação.

Termo para lembrar: Unknown outcome | Base técnica: referência 2.

16. Como sai de UNKNOWN?

Consulto o destino por identidade, recibos ou reconciliação prevista no contrato. Só avanço com evidência adequada; se não houver mecanismo suficiente, encaminho intervenção controlada sem fingir que o tempo resolveu a incerteza.

Termo para lembrar: Reconciliation | Base técnica: referência 2.

17. Posso usar outro provedor depois do timeout?

Não automaticamente para o mesmo efeito financeiro. O primeiro pode executar depois; um fallback requer prova de não aceitação, cancelamento confirmado ou coordenação que impeça dois efeitos para a mesma intenção.

Termo para lembrar: Cross-provider retry | Base técnica: referência 2.

18. Posso montar novo lote depois de falha SFTP?

Somente conforme evidência e contrato. Se o arquivo pode ter chegado, preservo batch_id e bytes e reconcilio; um novo lote com os mesmos chargebacks pode escapar da deduplicação do parceiro.

Termo para lembrar: Batch retry | Base técnica: referência 2.

19. Ausência de arquivo remoto resolve UNKNOWN?

Não por si só. O parceiro pode consumir ou mover o arquivo após recebê-lo. Preciso conhecer a semântica do diretório e obter evidência de processamento; ausência não é recibo de rejeição.

Termo para lembrar: SFTP ambiguity | Base técnica: referência 2.

20. Cancelar a conexão cancela o efeito remoto?

Não necessariamente. Cancelamento local libera espera, mas o servidor pode continuar. Uso protocolo explícito de cancelamento quando disponível e acompanho resultado; não anuncio reversão financeira apenas porque fechei o socket.

Termo para lembrar: Cancellation boundary | Base técnica: referência 1.

21. Como protege o usuário contra clique repetido?

O cliente preserva a chave da intenção e mostra estado consultável. Desabilitar botão ajuda experiência, mas não substitui proteção no servidor diante de retransmissão, reconexão ou chamadas concorrentes de outros dispositivos.

Termo para lembrar: Idempotency key | Base técnica: referência 2.

22. Circuit breaker substitui retry?

Não. Retry tenta superar falha transitória; breaker evita insistência numa dependência com sinais persistentes de falha. Coordeno os dois com limites para que a recuperação não libere uma tempestade de chamadas.

Termo para lembrar: Circuit breaker | Base técnica: referência 3.

23. Quando usa hedge request?

Principalmente em leituras seguras com orçamento e cancelamento apropriados. Duplicar comandos financeiros sem garantia forte pode duplicar efeitos; hedging também aumenta carga e precisa de critério baseado em latência de cauda.

Termo para lembrar: Hedging | Base técnica: referência 1.

24. Como evita repetir erro permanente?

Classifico validação, autorização e contrato antes de retentar. Uma credencial inválida ou payload impossível exige correção, não backoff infinito; a política de erro deve ser específica do endpoint e operação.

Termo para lembrar: Permanent error | Base técnica: referência 1.

25. Como lida com timeout de banco no commit?

O resultado pode ser ambíguo se perdi a resposta de confirmação. Consulto pela identidade da operação em conexão recuperada antes de repetir; não presumo rollback apenas porque o cliente recebeu timeout.

Termo para lembrar: Commit uncertainty | Base técnica: referência 2.

26. Como calcula custo de uma política de retry?

Meço chamadas adicionais, tempo ocupado, sucesso recuperado e impacto no destino. Se retries quase nunca recuperam e aumentam p99, reduzo ou removo a política para aquela classe de falha.

Termo para lembrar: Retry effectiveness | Base técnica: referência 1.

27. Como testa uma resposta perdida?

Faço o destino executar e interrompo a resposta antes do cliente recebê-la. Verifico estado UNKNOWN, consulta pela mesma identidade e ausência de novo efeito; simular somente conexão recusada não testa esse risco.

Termo para lembrar: Fault injection | Base técnica: referência 2.

28. Como apresenta UNKNOWN no aplicativo?

Mostro confirmação de recebimento ou verificação conforme o estágio real, com identificador e atualização posterior. Evito recomendar nova tentativa com nova intenção enquanto o resultado anterior pode concluir.

Termo para lembrar: User-visible state | Base técnica: referência 2.

29. Como deadline interage com job durável?

O prazo da conexão pode vencer enquanto a obrigação aceita continua. O cliente acompanha status; se o negócio tem prazo final, o workflow aplica sua política de expiração sem confundir desconexão com cancelamento automático.

Termo para lembrar: Business deadline | Base técnica: referência 1.

30. Como defenderia a política completa?

Eu apresentaria classes de erro, identidade preservada, limites de tempo e carga e caminho de reconciliação. A política busca recuperação sem amplificar falhas ou converter uma resposta perdida em duplicidade financeira.

Termo para lembrar: Resilient retry policy | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Timeout curto

Ganho. Libera recurso cedo

Custo e motivo. Mais falsos timeouts

Como decidir. Quando latência normal é baixa e medida

02. Timeout longo

Ganho. Tolera cauda

Custo e motivo. Ocupa recursos

Como decidir. Quando prazo e capacidade permitem

03. Retry imediato

Ganho. Recupera falha breve

Custo e motivo. Amplifica rajada

Como decidir. Somente com orçamento pequeno

04. Backoff

Ganho. Reduz pressão

Custo e motivo. Aumenta recuperação

Como decidir. Para indisponibilidade transitória

05. Jitter

Ganho. Evita sincronização

Custo e motivo. Tempo menos previsível

Como decidir. Dentro do prazo aceito

06. Retry centralizado

Ganho. Controla amplificação

Custo e motivo. Camada assume política

Como decidir. Para chamadas em cascata

07. Consulta antes de repetir

Ganho. Reduz ambiguidade

Custo e motivo. Mais chamadas e latência

Como decidir. Quando efeito pode ter ocorrido

08. UNKNOWN explícito

Ganho. Preserva verdade operacional

Custo e motivo. Exige reconciliação

Como decidir. Para integração financeira

09. Hedging de leitura

Ganho. Reduz cauda

Custo e motivo. Mais carga

Como decidir. Quando operação é segura e mensurada

10. Persistir next_attempt_at

Ganho. Sobrevive a reinícios

Custo e motivo. Mais estado

Como decidir. Para jobs assíncronos

Duas alternativas ao caminho proposto

1. Falhar sem retry automático

Quando faz sentido. Efeito não pode repetir com segurança

Como implementar. Retornar estado consultável e encaminhar reconciliação

Limites. Recuperação mais lenta e possível intervenção

2. Store-and-forward

Quando faz sentido. Operação pode aguardar

Como implementar. Persistir intenção e drenar com quota e identidade

Limites. Não elimina UNKNOWN no destino externo

Referências e alcance da evidência

1. AWS - Timeouts retries and jitter

Timeouts, backoff, jitter e risco de repetir efeitos após respostas perdidas.

https://aws.amazon.com/builders-library/timeouts-retries-and-backoff-with-jitter/

2. AWS Builders Library - Idempotent APIs

Identidades estáveis, intenção, retries e requisições atrasadas.

https://aws.amazon.com/builders-library/making-retries-safe-with-idempotent-APIs/

3. Microsoft - Circuit breaker

Interrupção temporária de chamadas, recuperação e limites do padrão.

https://learn.microsoft.com/en-us/azure/architecture/patterns/circuit-breaker

Continue em: T045 · T049 · T061 · T065

T065 / G. Correção e resiliência

Leases e fencing tokens

Lease concede posse temporária; fencing impede que um antigo possuidor continue alterando o recurso após perder autoridade.

Quando usar. Workers distribuídos, eleição de líder e retomada de tarefas após falhas.

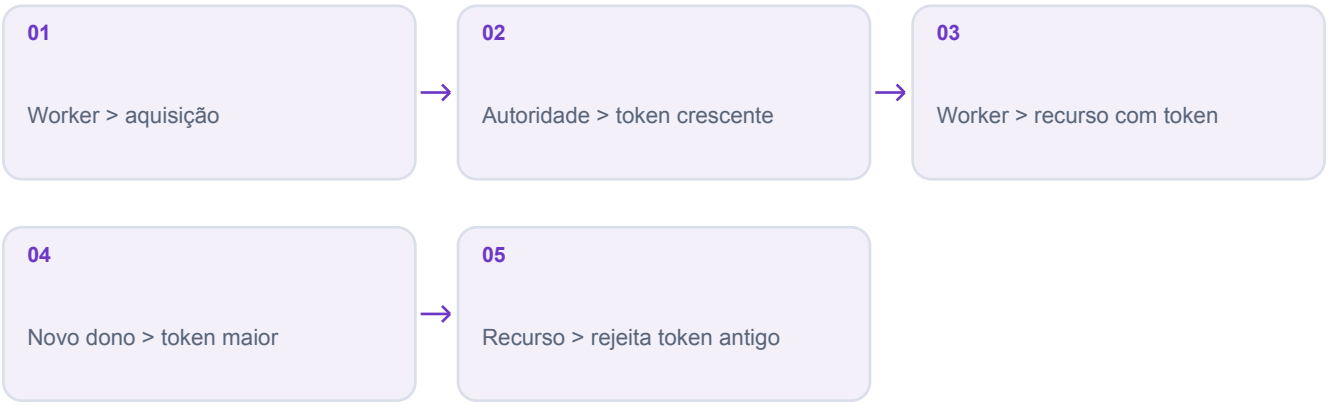
Quando evitar ou limitar. Não assumir que expirar um lock encerra o processo antigo nem que token ignorado pelo destino protege efeitos.

Uma abertura possível

Vou simular um worker pausado voltando depois da lease; a segurança depende de o recurso rejeitar sua autoridade antiga.

Peças e tecnologias. etcd · PostgreSQL CAS · Hazelcast FencedLock · Redis leases

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Definir recurso e autoridade de posse

2. Emitir tokens monotônicos por recurso

3. Validar token atomicamente no destino

4. Renovar lease e recuperar tarefas expiradas

5. Testar pausa longa e retorno de worker antigo

30 perguntas para sustentar o desenho

01. Lease é um lock que mata o processo quando expira?

Não. Ela expira a autorização reconhecida pelo coordenador, mas o processo pode continuar vivo ou voltar de uma pausa. O recurso protegido precisa rejeitar ações obsoletas para impedir concorrência perigosa.

Termo para lembrar: Lease expiration | Base técnica: referência 1.

02. Qual cenário justifica fencing?

Worker A pausa, perde lease e B assume. Quando A retorna, ainda pode tentar escrever. Um fencing token crescente permite ao destino rejeitar a autoridade antiga, evitando que a pausa vire corrupção.

Termo para lembrar: Fencing token | Base técnica: referência 1.

03. Onde o token deve ser validado?

No recurso que produz o efeito, de forma atômica com a operação protegida. Validar apenas no worker deixa uma corrida entre checagem e escrita; um processo atrasado pode agir depois da validação local.

Termo para lembrar: Resource-side fencing | Base técnica: referência 1.

04. Token aleatório serve para fencing?

Ele pode identificar dono, mas não estabelece ordem de autoridades. Para fencing monotônico, preciso distinguir tokens antigos dos novos e preservar essa ordem mesmo durante recuperação; UUID aleatório não oferece essa comparação.

Termo para lembrar: Monotonic epoch | Base técnica: referência 1.

05. Como implementa no PostgreSQL?

Guardo epoch ou versão e faço atualização condicionada à autoridade vigente, junto do efeito local. Se a condição não corresponder, rejeito a escrita; o token precisa fazer parte da transação que altera o estado.

Termo para lembrar: Conditional update | Base técnica: referência 2.

06. Como o destino aprende o novo token?

O protocolo registra ou observa a nova autoridade antes de aceitar operações conflitantes. Apenas gerar token no coordenador não protege automaticamente o recurso; o destino mantém estado suficiente para rejeitar epochs obsoletos.

Termo para lembrar: Epoch enforcement | Base técnica: referência 1.

07. Fencing impede qualquer chamada antiga antes do novo token chegar?

Não necessariamente num protocolo baseado no maior token observado. Defino precisamente quando a nova autoridade se torna efetiva no recurso; para requisitos mais fortes, coordeno validação da posse e atualização autoritativa apropriadas.

Termo para lembrar: Fencing boundary | Base técnica: referência 1.

08. Token também deduplica duas chamadas do mesmo dono?

Não por si só. O mesmo dono pode repetir uma operação com token válido. Uso operation_id para deduplicação e fencing para autoridade; são garantias complementares sobre intenção e posse.

Termo para lembrar: Fencing versus idempotency | Base técnica: referência 3.

09. Redis SET NX com TTL é suficiente?

Pode coordenar trabalho tolerante a duplicatas, mas não prova exclusão de efeitos críticos após expiração. Analiso persistência e failover e mantenho proteção no destino; lock temporário sozinho não garante débito único.

Termo para lembrar: Distributed lease | Base técnica: referência 1.

10. Como escolhe duração da lease?

Equilibro tempo de recuperação e frequência de renovação, considerando pausas e latência. Lease longa atrasa takeover; curta aumenta sobreposição potencial. A correção não deve depender de nunca ocorrer uma pausa maior que o prazo.

Termo para lembrar: Lease duration | Base técnica: referência 1.

11. Heartbeat renova automaticamente em qualquer falha?

Não. Rede ou processo podem impedir renovação, e o worker deve tratar perda de posse. Mesmo interrompendo voluntariamente, mantenho fencing porque pausas e chamadas já em voo podem ultrapassar essa decisão.

Termo para lembrar: Heartbeat failure | Base técnica: referência 1.

12. Como evita liberar lock de outro dono?

A liberação verifica identidade e versão do proprietário de forma atômica. Uma operação de delete cego após expiração pode remover a lease adquirida por outro worker, abrindo concorrência adicional.

Termo para lembrar: Compare-and-delete | Base técnica: referência 2.

13. Como diferencia attempt_id e fencing token?

Attempt_id identifica uma tentativa para auditoria; fencing token ordena autoridades sobre o recurso. Um identificador pode cumprir ambos se o contrato garantir ordem, mas não presumo isso apenas porque ele é único.

Termo para lembrar: Attempt versus epoch | Base técnica: referência 1.

14. O que faz se o coordenador estiver particionado?

Escolho comportamento que preserve a garantia, normalmente recusando nova autoridade sem coordenação suficiente. Disponibilidade durante partição depende do modelo; não permito dois líderes independentes escrevendo num recurso que exige unicidade.

Termo para lembrar: Coordination partition | Base técnica: referência 4.

15. Leitura eventual da lease é segura para decidir escrita?

Não para uma decisão que exige autoridade atual. Uma réplica atrasada pode mostrar dono antigo; uso operação com garantia adequada ou condição no recurso, delimitando consistência do caminho crítico.

Termo para lembrar: Linearizable authority | Base técnica: referência 4.

16. Como protege SFTP com fencing?

Só se o destino ou um intermediário autoritativo validar a autoridade antes do efeito. SFTP comum não entende automaticamente meu token; para arquivos, preciso de identidade, protocolo de aceite e reconciliação além do lock local.

Termo para lembrar: External fencing support | Base técnica: referência 3.

17. Pode colocar o token no nome do arquivo?

Isso ajuda rastreabilidade, mas não obriga o parceiro a rejeitar um arquivo antigo. Sem regra de validação no destino, o token é apenas metadado e pode até criar nomes distintos que escapam da deduplicação.

Termo para lembrar: Metadata versus enforcement | Base técnica: referência 3.

18. Como recupera job cujo worker morreu?

Após expiração apropriada, outro worker assume com nova autoridade e consulta checkpoint. Ele preserva identidade da intenção e verifica efeitos externos; assumir o job não prova que a tentativa anterior não executou nada.

Termo para lembrar: Job takeover | Base técnica: referência 3.

19. Como impede gravação final do worker antigo?

A atualização de conclusão exige attempt_id ou epoch vigente. Se outro worker assumiu, a gravação antiga falha; isso protege estado local, mas não desfaz um efeito remoto que já tenha escapado.

Termo para lembrar: Stale completion | Base técnica: referência 2.

20. Como trata efeito remoto sem fencing nem idempotência?

Reconheço a limitação e evito prometer exclusão completa. Uso serialização controlada, evidências e reconciliação, podendo exigir intervenção; nenhuma combinação de locks locais cria uma garantia ausente no sistema externo.

Termo para lembrar: Uncooperative resource | Base técnica: referência 3.

21. Eleição de líder garante um único executor físico?

Garante autoridade conforme protocolo, mas processos antigos podem continuar rodando. Distingo líder lógico de atividade física e protejo efeitos no recurso; eleição sem fencing pode deixar um antigo líder perigoso.

Termo para lembrar: Leader fencing | Base técnica: referência 1.

22. Como backup afeta contador de fencing?

Restaurar contador antigo pode reutilizar epochs e invalidar a ordem esperada pelo recurso. Planejo recuperação com autoridade monotônica ou nova geração global reconhecida; não reinicio contador arbitrariamente depois de desastre.

Termo para lembrar: Epoch recovery | Base técnica: referência 1.

23. Um contador por recurso é melhor que global?

Pode reduzir contenção e basta quando a autoridade é independente por recurso. Um contador global simplifica certas comparações, mas concentra coordenação; a escolha precisa preservar ordem no escopo protegido.

Termo para lembrar: Token scope | Base técnica: referência 1.

24. Como trata várias operações numa lease?

Cada operação valida autoridade e identidade própria quando necessário. A lease não autoriza um lote ilimitado depois de expirar; dividir trabalho em etapas com checkpoints facilita recuperação e limita consequências de posse perdida.

Termo para lembrar: Lease-scoped work | Base técnica: referência 1.

25. Como cancela trabalho ao perder lease?

Sinalizo cancelamento e paro novas ações, mas não considero isso garantia suficiente. Chamadas já enviadas podem concluir; o destino e a reconciliação tratam essa janela mesmo se a aplicação cooperar corretamente.

Termo para lembrar: Cooperative cancellation | Base técnica: referência 3.

26. Qual custo de fencing?

Exige estado e validação no recurso, além de coordenação para emitir autoridade. Aceito esse custo quando concorrência obsoleta ameaça correção; para trabalho descartável, idempotência simples pode ser suficiente.

Termo para lembrar: Fencing overhead | Base técnica: referência 1.

27. Como testa uma pausa longa?

Suspendo o worker após adquirir lease, deixo outro assumir e retomo o primeiro. Verifico rejeição no recurso e proteção contra efeitos duplicados; apenas matar o processo não testa o retorno tardio mais perigoso.

Termo para lembrar: Pause-and-resume test | Base técnica: referência 1.

28. Quais métricas acompanhar?

Renovações falhas, takeovers, rejeições por token antigo, jobs sem progresso e duração das leases. Rejeição frequente pode indicar timeouts mal dimensionados, pausas de runtime ou problema de rede, mesmo quando a proteção funciona.

Termo para lembrar: Lease telemetry | Base técnica: referência 1.

29. Quando prefere CAS a lock distribuído?

Quando o estado e a invariante cabem no banco autoritativo. Uma atualização condicional pode resolver concorrência com menos componentes; introduzo lock externo somente se houver necessidade concreta além dessa transação.

Termo para lembrar: Compare-and-swap | Base técnica: referência 2.

30. Como defenderia a solução?

Eu mostraria onde a autoridade nasce e onde é rejeitada quando velha. Depois pausaria um worker no desenho; se não consigo explicar sua escrita tardia, o lock ainda não protege a invariante.

Termo para lembrar: Stale-owner proof | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Lease curta

Ganho. Takeover rápido

Custo e motivo. Mais renovação

Como decidir. Com destino protegido

02. Lease longa

Ganho. Menos renovação

Custo e motivo. Recuperação lenta

Como decidir. Para trabalho longo tolerante

03. Fencing no recurso

Ganho. Rejeita dono antigo

Custo e motivo. Estado adicional

Como decidir. Para efeitos críticos

04. Somente lock local

Ganho. Simplicidade

Custo e motivo. Não cobre outros processos

Como decidir. Para recurso exclusivamente local

05. CAS no banco

Ganho. Poucos componentes

Custo e motivo. Contenção por linha

Como decidir. Quando invariante cabe localmente

06. Coordenador externo

Ganho. Autoridade compartilhada

Custo e motivo. Nova dependência

Como decidir. Para múltiplos participantes

07. Token global

Ganho. Ordem uniforme

Custo e motivo. Coordenação central

Como decidir. Quando escopo global importa

08. Token por recurso

Ganho. Menos contenção

Custo e motivo. Gestão de escopos

Como decidir. Para recursos independentes

09. Cancelamento cooperativo

Ganho. Reduz trabalho inútil

Custo e motivo. Não desfaz chamada enviada

Como decidir. Como otimização adicional

10. Recusar sem quorum

Ganho. Preserva autoridade

Custo e motivo. Menor disponibilidade

Como decidir. Quando exclusividade é essencial

Duas alternativas ao caminho proposto

1. Transação com lock local ao banco

Quando faz sentido. Todos os dados cabem no banco

Como implementar. Executar operação curta sob controle transacional

Limites. Não deve manter lock durante rede externa

2. Processamento idempotente concorrente

Quando faz sentido. Trabalho admite repetição segura

Como implementar. Permitir tentativas e deduplicar no efeito

Limites. Não serve para efeitos externos sem proteção

Referências e alcance da evidência

1. Hazelcast - FencedLock

Tokens monotônicos e rejeição de clientes obsoletos pelo recurso protegido.

https://docs.hazelcast.com/hazelcast/latest/data-structures/fencedlock

2. PostgreSQL - Transaction isolation

Snapshots, concorrência, isolamento e falhas de serialização.

https://www.postgresql.org/docs/current/transaction-iso.html

3. AWS Builders Library - Idempotent APIs

Identidades estáveis, intenção, retries e requisições atrasadas.

https://aws.amazon.com/builders-library/making-retries-safe-with-idempotent-APIs/

4. etcd - API guarantees

Linearizabilidade, durabilidade e limites das APIs e watches.

https://etcd.io/docs/v3.5/learning/api_guarantees/

Continue em: T019 · T050 · T061 · T107

T066 / G. Correção e resiliência

Circuit breaker, bulkhead e degradação

Circuit breaker evita insistir em dependência falhando; bulkhead limita propagação; degradação preserva funções semanticamente seguras.

Quando usar. Serviços com múltiplas dependências e risco de falhas em cascata.

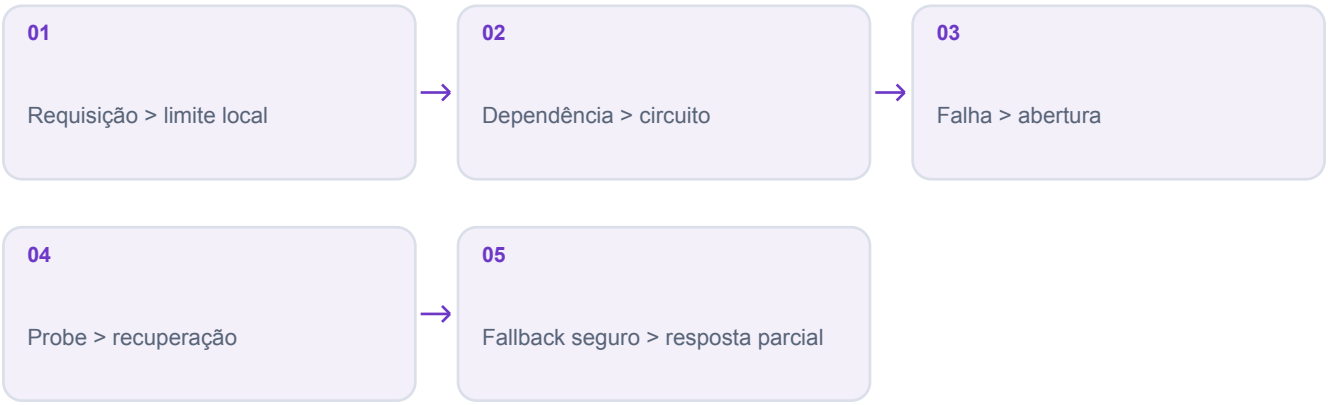
Quando evitar ou limitar. Não usar fallback que inventa sucesso financeiro nem abrir circuito global por erro específico de um cliente.

Uma abertura possível

Vou isolar recursos por dependência e explicar quais funções continuam corretas quando uma parte falha.

Peças e tecnologias. Resilience4j · Envoy · Kong · Semáforos · Pools separados

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Definir falhas que contam para o circuito

2. Separar pools e limites por dependência

3. Escolher fallback semanticamente válido

4. Controlar probes e recuperação gradual

5. Testar cascata, falso positivo e retorno de carga

30 perguntas para sustentar o desenho

01. O que faz um circuit breaker?

Ele interrompe temporariamente chamadas com alta probabilidade de falhar, protegendo recursos e permitindo recuperação. Não corrige a dependência nem resolve operações já desconhecidas; essas continuam exigindo acompanhamento e reconciliação.

Termo para lembrar: Circuit breaker | Base técnica: referência 1.

02. Quais estados você modela?

Normalmente fechado, aberto e semiaberto, com política explícita de transição. Fechado permite chamadas; aberto recusa; semiaberto testa recuperação com poucas tentativas, evitando liberar toda carga na primeira resposta bem-sucedida.

Termo para lembrar: Half-open | Base técnica: referência 1.

03. Erro de validação deve abrir circuito?

Normalmente não, pois indica problema naquela requisição, não indisponibilidade da dependência. Classifico erros relevantes por operação; misturar rejeição de negócio com falha de infraestrutura pode bloquear clientes saudáveis indevidamente.

Termo para lembrar: Error classification | Base técnica: referência 1.

04. Como escolhe threshold?

Uso janela, volume mínimo e taxa de falha ou lentidão conforme comportamento real. Uma única falha em tráfego baixo não deve necessariamente abrir; também não espero média esconder degradação crítica prolongada.

Termo para lembrar: Failure threshold | Base técnica: referência 1.

05. Timeout e breaker são substitutos?

Não. Timeout limita duração de uma chamada; breaker decide se novas chamadas devem começar. Preciso dos dois para impedir que operações em andamento ocupem recursos indefinidamente mesmo antes de o circuito abrir.

Termo para lembrar: Timeout boundary | Base técnica: referência 1.

06. Retry e breaker podem competir?

Sim. Retries excessivos podem alimentar falhas e retardar recuperação. Defino ordem, orçamento e classificação, evitando que cada camada repita uma chamada recusada pelo circuito como se isso fosse uma falha transitória nova.

Termo para lembrar: Retry coordination | Base técnica: referência 3.

07. O que é bulkhead?

É separar capacidade para que uma dependência ou classe não consuma recursos de todas as outras. Posso usar pools, semáforos ou filas distintas, com limites que preservem funções essenciais durante falha parcial.

Termo para lembrar: Bulkhead | Base técnica: referência 2.

08. Pool de threads separado basta?

Ajuda em um recurso, mas posso compartilhar conexões, memória ou banco que ainda propagam a falha. Analiso o caminho completo de recursos; isolamento de threads não é isolamento total do serviço.

Termo para lembrar: Resource isolation | Base técnica: referência 2.

09. Como escolhe tamanho dos pools?

Uso vazão e duração esperadas, com margem e limites do destino. Pools enormes apenas permitem acumular mais chamadas lentas; dimensiono para capacidade sustentável e comportamento degradado, não para esconder fila.

Termo para lembrar: Pool sizing | Base técnica: referência 2.

10. Circuito por serviço ou endpoint?

Depende da correlação de falhas. Um endpoint de relatório lento não deveria bloquear consulta simples saudável; escopo muito fino aumenta estado e ruído. Escolho unidade que represente dependência e risco reais.

Termo para lembrar: Breaker scope | Base técnica: referência 1.

11. Pode abrir circuito por tenant?

Pode quando a falha ou quota é específica daquele tenant. Isso evita penalizar todos, mas exige controle de cardinalidade e política; um identificador arbitrário não deve criar milhões de circuitos em memória.

Termo para lembrar: Tenant isolation | Base técnica: referência 2.

12. Como funciona recuperação semiaberta?

Permito poucas probes representativas e observo resultado antes de ampliar. Uso limites e espalhamento para evitar que todas as instâncias testem simultaneamente, recriando a carga que derrubou a dependência.

Termo para lembrar: Recovery probes | Base técnica: referência 1.

13. Uma resposta boa fecha imediatamente?

Não obrigatoriamente. Defino evidência suficiente e recuperação gradual, porque uma única resposta pode não representar capacidade sob carga. O critério considera taxa, latência e estabilidade dentro da janela escolhida.

Termo para lembrar: Recovery hysteresis | Base técnica: referência 1.

14. Fallback de saldo pode usar zero?

Não. Zero é dado de negócio, não ausência de informação. Mostro indisponibilidade ou último valor identificado quando permitido; comandos que dependem de saldo disponível continuam validando na autoridade financeira.

Termo para lembrar: Semantic fallback | Base técnica: referência 2.

15. Pode aprovar pagamento quando antifraude cai?

Somente se houver política de risco explícita e limites autorizados pelo negócio. Não invento fail-open como requisito técnico de Staff; desenho alternativas e consequências financeiras, incluindo recusa ou análise posterior.

Termo para lembrar: Risk policy | Base técnica: referência 2.

16. Como degrada workspace de atendimento?

Carrego contexto mínimo e isolo widgets opcionais. Se cartões falhar, outros dados continuam; ações dependentes daquele estado ficam indisponíveis com mensagem clara, em vez de toda a tela travar ou fingir sucesso.

Termo para lembrar: Partial functionality | Base técnica: referência 2.

17. Cache antigo é fallback seguro?

Depende do recurso e do uso. Pode servir conteúdo informativo com idade explícita; não deve sustentar autorização revogada ou decisão financeira atual. Defino limite de staleness e comportamento ao excedê-lo.

Termo para lembrar: Stale data policy | Base técnica: referência 2.

18. Quando fail-fast melhora experiência?

Quando esperar não tem probabilidade razoável de sucesso dentro do prazo. Uma resposta rápida e honesta permite recuperação ou alternativa; para comando já aceito, continuo acompanhando a obrigação em vez de simplesmente abandoná-la.

Termo para lembrar: Fail-fast | Base técnica: referência 1.

19. O breaker pode perder trabalho aceito?

Não deve. Se o comando já está durável, a abertura adia execução e preserva estado pendente. A política precisa distinguir recusa antes da aceitação e indisponibilidade de uma etapa posterior do processo.

Termo para lembrar: Accepted obligation | Base técnica: referência 1.

20. Como evitar fallback em cascata perigoso?

Verifico se a alternativa tem capacidade e semântica próprias. Mandar toda carga para outro serviço pode derrubá-lo ou duplicar efeitos; fallback de pagamento exige resolver tentativa anterior UNKNOWN antes de trocar destino.

Termo para lembrar: Fallback capacity | Base técnica: referência 3.

21. Qual problema de circuito global compartilhado?

Ele pode coordenar proteção, mas vira dependência e amplia blast radius de uma decisão incorreta. Circuitos locais reagem rápido, porém podem divergir; escolho conforme necessidade e não uso o estado como verdade financeira.

Termo para lembrar: Distributed breaker | Base técnica: referência 1.

22. Breaker no mesh dispensa código de domínio?

Não. O proxy pode controlar transporte, mas não conhece todas as regras de repetição, compensação e experiência. A aplicação continua responsável por interpretar UNKNOWN, autorização e efeito de negócio.

Termo para lembrar: Infrastructure boundary | Base técnica: referência 1.

23. Como testa isolamento?

Faço uma dependência ficar lenta e observo se funções independentes mantêm latência e capacidade. Verifico conexões, threads e memória compartilhadas; só ver o circuito aberto não comprova que a cascata foi contida.

Termo para lembrar: Bulkhead test | Base técnica: referência 2.

24. Quais métricas acompanho?

Estado e transições do circuito, chamadas recusadas, probes, saturação dos pools e sucesso do fallback. Também meço impacto no usuário; circuito aberto pode ser proteção correta, não um incidente isolado por si só.

Termo para lembrar: Resilience telemetry | Base técnica: referência 1.

25. Como evita alarmes excessivos?

Alerto por impacto e obrigação pendente, usando transições como contexto. Um circuito abrindo brevemente pode ser esperado; páginas repetidas sem ação útil cansam operadores e escondem falhas realmente críticas.

Termo para lembrar: Actionable alerting | Base técnica: referência 1.

26. Qual risco de fallback silencioso?

Pode esconder perda de qualidade e prolongar incidente. Identifico resposta degradada e monitoro sua frequência, sem expor detalhes sensíveis; a operação precisa saber quando está vivendo de alternativas temporárias.

Termo para lembrar: Degradation visibility | Base técnica: referência 2.

27. Como trata clientes que insistem após recusa?

Aplico rate limit e orçamento de retry, com orientação adequada. Fail-fast sem controle pode gerar loop de chamadas muito rápido; proteção do destino precisa acompanhar comportamento dos clientes e proxies.

Termo para lembrar: Retry storm | Base técnica: referência 3.

28. Quando não usar circuit breaker?

Se a camada já controla adequadamente falhas e outro circuito apenas adiciona estados difíceis, posso evitá-lo. A decisão depende de custo de chamadas falhas, recuperação e observabilidade, não de checklist de padrões.

Termo para lembrar: Pattern fit | Base técnica: referência 1.

29. Como volta ao normal sem pico?

Aumento carga gradualmente, limito drenagem do backlog e mantenho jitter. A dependência recém-recuperada pode ter cache frio; liberar tráfego novo e todas as pendências simultaneamente recria a falha.

Termo para lembrar: Recovery ramp | Base técnica: referência 2.

30. Como defenderia o desenho?

Eu escolheria uma dependência lenta e mostraria o limite de recursos, a resposta ao usuário e o caminho de recuperação. A garantia é conter impacto mantendo significado correto, não prometer disponibilidade absoluta.

Termo para lembrar: Failure containment | Base técnica: referência 2.

10 trade-offs: ganho, custo e decisão

01. Threshold sensível

Ganho. Protege cedo

Custo e motivo. Falsos positivos

Como decidir. Para falha cara

02. Threshold tolerante

Ganho. Evita oscilação

Custo e motivo. Mais chamadas ruins

Como decidir. Quando erros breves são comuns

03. Circuito fino

Ganho. Isola operação

Custo e motivo. Mais estado

Como decidir. Para falhas independentes

04. Circuito amplo

Ganho. Simples de operar

Custo e motivo. Bloqueia funções saudáveis

Como decidir. Para dependência realmente comum

05. Pools separados

Ganho. Contém cascata

Custo e motivo. Capacidade pode ficar ociosa

Como decidir. Para dependências críticas

06. Pool compartilhado

Ganho. Melhor utilização

Custo e motivo. Falha se propaga

Como decidir. Para cargas semelhantes e controladas

07. Fallback cacheado

Ganho. Preserva leitura

Custo e motivo. Dado antigo

Como decidir. Com staleness aceitável

08. Fail-closed

Ganho. Preserva invariante

Custo e motivo. Recusa operação

Como decidir. Quando risco é alto

09. Probes poucas

Ganho. Retorno seguro

Custo e motivo. Recuperação lenta

Como decidir. Para destino frágil

10. Recuperação gradual

Ganho. Evita nova queda

Custo e motivo. Drena devagar

Como decidir. Após indisponibilidade longa

Duas alternativas ao caminho proposto

1. Admission control com timeout

Quando faz sentido. Problema principal é saturação previsível

Como implementar. Limitar concorrência e duração sem estados de circuito

Limites. Não evita toda insistência em falha persistente

2. Fila durável por dependência

Quando faz sentido. Trabalho pode aguardar

Como implementar. Aceitar intenção e executar conforme capacidade

Limites. Exige prazo, backlog e idempotência

Referências e alcance da evidência

1. Microsoft - Circuit breaker

Interrupção temporária de chamadas, recuperação e limites do padrão.

https://learn.microsoft.com/en-us/azure/architecture/patterns/circuit-breaker

2. Microsoft - Bulkhead

Separação de pools para reduzir propagação de falhas.

https://learn.microsoft.com/en-us/azure/architecture/patterns/bulkhead

3. AWS - Timeouts retries and jitter

Timeouts, backoff, jitter e risco de repetir efeitos após respostas perdidas.

https://aws.amazon.com/builders-library/timeouts-retries-and-backoff-with-jitter/

Continue em: T045 · T053 · T055 · T064

T067 / G. Correção e resiliência

Backups, PITR, RPO, RTO e disaster recovery

Recuperação exige cópias utilizáveis, objetivos explícitos de perda e tempo, e reconciliação com efeitos externos.

Quando usar. Perda de dados, corrupção, falha regional e exclusão indevida.

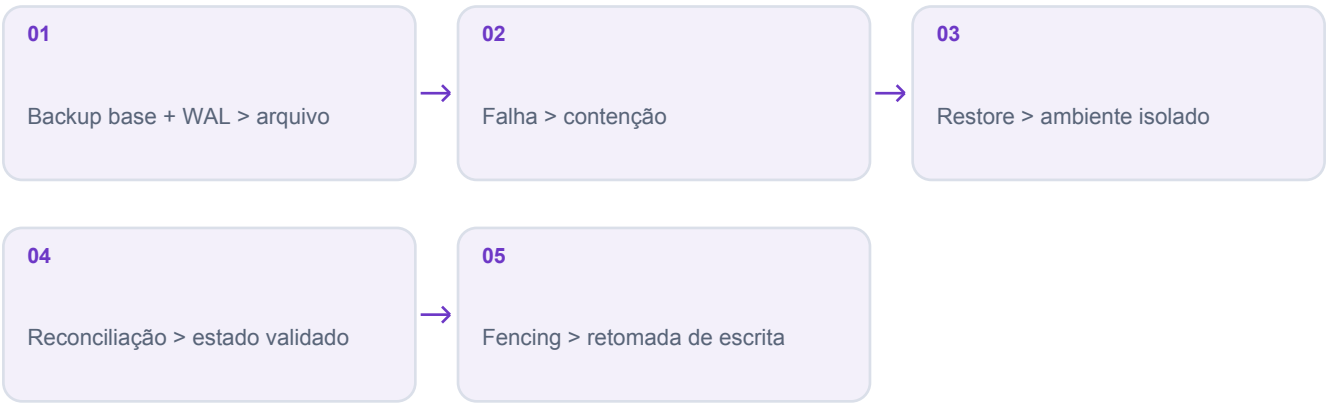
Quando evitar ou limitar. Não tratar réplica como backup nem prometer RPO zero sem durabilidade compatível antes da confirmação.

Uma abertura possível

Vou separar restauração técnica de retomada do negócio e testar quanto tempo leva para recuperar dados corretos e voltar a operar.

Peças e tecnologias. PostgreSQL WAL · PITR · Object storage · Snapshots · Replicação · Runbooks

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Definir RPO e RTO por operação

2. Configurar cópias e retenção independentes

3. Automatizar restore isolado e validação

4. Reconciliar efeitos externos e deduplicação

5. Ensaiar failover e failback com tempos medidos

30 perguntas para sustentar o desenho

01. Qual diferença entre RPO e RTO?

RPO define a perda de dados tolerada em relação ao tempo; RTO define quanto tempo a recuperação pode levar. Ambos são objetivos negociados, não garantias automáticas obtidas apenas por habilitar backup.

Termo para lembrar: RPO RTO | Base técnica: referência 1.

02. RPO zero significa nunca perder nada?

Somente no escopo de falhas e durabilidade projetados. Se confirmo antes de replicar fora da região, posso perder dados numa perda regional; preciso alinhar confirmação e armazenamento ao cenário que prometo suportar.

Termo para lembrar: Durability boundary | Base técnica: referência 1.

03. Réplica é um backup suficiente?

Não necessariamente. Ela pode replicar exclusão ou corrupção lógica rapidamente. Backup com retenção permite voltar a um ponto anterior; uso ambos conforme disponibilidade e recuperação, testando cenários diferentes em vez de tratá-los como equivalentes.

Termo para lembrar: Replication versus backup | Base técnica: referência 1.

04. Como PITR funciona no PostgreSQL?

Combino backup base com sequência necessária de WAL para reconstruir até o ponto escolhido. A recuperação depende de arquivos íntegros e disponíveis; um backup base sem WAL exigido não oferece qualquer ponto arbitrário.

Termo para lembrar: Point-in-time recovery | Base técnica: referência 1.

05. Como detecta falha no arquivamento de WAL?

Monitoro atraso, erros, continuidade e espaço local, não apenas existência do último backup. Se o arquivamento parar, a janela recuperável pode ficar comprometida e o volume local crescer até afetar o banco.

Termo para lembrar: WAL archiving | Base técnica: referência 1.

06. Snapshot de disco sozinho é consistente?

Depende do mecanismo e coordenação com o banco. Não assumo que copiar arquivos em instantes diferentes produz backup utilizável; sigo procedimento suportado e valido restauração, incluindo logs e metadados necessários.

Termo para lembrar: Consistent backup | Base técnica: referência 1.

07. Como escolhe o ponto de restauração?

Identifico o evento de corrupção e suas dependências, escolhendo ponto que preserve dados válidos. Restaurar cedo demais perde trabalho legítimo; tarde demais mantém o defeito. Depois reconcilio efeitos externos que não voltam no tempo.

Termo para lembrar: Recovery target | Base técnica: referência 1.

08. Restaurar banco desfaz pagamento no parceiro?

Não. O parceiro mantém efeitos já executados. Antes de reenviar pendências restauradas, reconcilio operation_ids e resultados externos; um backup antigo pode apagar a memória local de um pagamento real.

Termo para lembrar: External reconciliation | Base técnica: referência 2.

09. Como restore afeta inbox e outbox?

Preciso restaurar estado local coerente, mas eventos antigos podem reaparecer como pendentes e marcas de deduplicação podem sumir. Destinos preservam identidades próprias e a retomada inclui reconciliação antes de repetir efeitos financeiros.

Termo para lembrar: Replay after restore | Base técnica: referência 2.

10. Como impede dois writers após failover?

Faço fencing do writer antigo e controlo promoção e roteamento. Uma região isolada pode continuar ativa; trocar DNS não prova que ela parou de escrever nem resolve split-brain por si só.

Termo para lembrar: Writer fencing | Base técnica: referência 3.

11. Failover automático é sempre melhor?

Não. Reduz tempo em cenários conhecidos, mas pode promover estado inadequado ou ampliar split-brain. Escolho automação com critérios verificáveis e limites; exceções ambíguas exigem procedimento seguro e autoridade clara.

Termo para lembrar: Failover policy | Base técnica: referência 3.

12. Como mede RTO real?

Cronometro detecção, decisão, provisionamento, restore, validação, reconciliação e retomada útil. Medir apenas tempo de subir o banco subestima recuperação; o negócio só voltou quando consegue atender corretamente as funções essenciais.

Termo para lembrar: Recovery timeline | Base técnica: referência 1.

13. RTO de uma hora cabe num SLO de 99,9%?

Pode consumir mais que o orçamento mensal de indisponibilidade de trinta dias, aproximadamente 43,2 minutos. Discuto frequência e definição do SLO; objetivos incompatíveis precisam ser negociados, não escondidos no desenho.

Termo para lembrar: Error budget alignment | Base técnica: referência 1.

14. Como testa backup sem afetar produção?

Restauro em ambiente isolado, valido integridade e executabilidade de consultas essenciais, sem disparar efeitos externos. O teste precisa usar permissões e recursos suficientes para representar recuperação real, protegendo os dados copiados.

Termo para lembrar: Restore drill | Base técnica: referência 1.

15. Backup criptografado resolve tudo?

Não. Preciso recuperar também chaves e permissões, sem permitir acesso indevido. Se perder a chave, o backup pode ficar inutilizável; se ela estiver disponível ao atacante, criptografia não impede destruição autorizada de cópias.

Termo para lembrar: Key recovery | Base técnica: referência 1.

16. Como protege backup de ransomware?

Separo credenciais e domínios de administração, uso retenção protegida quando apropriado e testo restauração. Uma conta comprometida não deveria poder alterar produção e apagar todas as cópias de recuperação no mesmo caminho.

Termo para lembrar: Backup isolation | Base técnica: referência 1.

17. Como define retenção?

Considero tempo para detectar corrupção, obrigações de preservação e necessidade de exclusão, com responsáveis adequados. Retenção maior aumenta opções de recuperação e custo; não adoto duração universal sem contexto do dado e negócio.

Termo para lembrar: Retention policy | Base técnica: referência 1.

18. Qual risco de restaurar aplicação incompatível?

Código novo pode interpretar schema antigo incorretamente, e código antigo pode não compreender dados novos. Mantenho matriz de versões e procedimento de recuperação; restaurar banco sem alinhar aplicação pode criar outra falha.

Termo para lembrar: Version compatibility | Base técnica: referência 1.

19. Como lida com sequências e IDs após restore?

Verifico geração de IDs, epochs e referências externas. Reutilizar identidade antiga pode colidir com operações existentes fora do backup; recuperação precisa preservar unicidade e autoridade, não apenas fazer consultas locais funcionarem.

Termo para lembrar: Identity continuity | Base técnica: referência 2.

20. Como recupera objetos junto do banco?

Reconcilio referências, versões e checksums entre metadados e object storage. Restaurar só um lado pode criar documentos ausentes ou versões erradas; defino política para órfãos e acesso durante a recuperação.

Termo para lembrar: Cross-store recovery | Base técnica: referência 1.

21. Backup multirregional é obrigatório para Staff?

Não. A escolha depende do cenário de risco, RPO, RTO e custo. Explico o que uma região suporta e o que não suporta; nível profissional aparece na decisão fundamentada, não na quantidade de regiões.

Termo para lembrar: Risk-based architecture | Base técnica: referência 1.

22. Como escolhe warm standby ou restore sob demanda?

Warm standby reduz parte do tempo de retomada com custo contínuo maior. Restore sob demanda pode atender RTO mais flexível; comparo tempos medidos, capacidade reservada e dependências externas antes de escolher.

Termo para lembrar: Recovery strategy | Base técnica: referência 1.

23. Active-active elimina recuperação?

Não. Ele pode ampliar disponibilidade para certos fluxos, mas adiciona conflitos e coordenação, e não resolve corrupção lógica propagada. Continuo precisando de backups, fencing ou regras de escrita e reconciliação.

Termo para lembrar: Multi-region boundary | Base técnica: referência 3.

24. Como faz failback?

Trato como migração controlada, verificando autoridade, sincronização e tráfego. Voltar à região original sem reconciliar mudanças pode perder dados ou reabrir dois writers; o runbook inclui critérios e rollback da própria mudança.

Termo para lembrar: Failback | Base técnica: referência 3.

25. Como prioriza funções durante recuperação?

Defino dependências e ordem de retomada, preservando leitura ou comandos essenciais primeiro quando seguro. Não libero todos os consumidores simultaneamente contra banco frio; controle de carga evita transformar recuperação em nova sobrecarga.

Termo para lembrar: Recovery sequencing | Base técnica: referência 1.

26. O que deve constar no runbook?

Critérios de acionamento, autoridade, comandos verificados, validações, comunicação e limites de risco. Incluo evidências de sucesso e pontos de parada; uma lista de comandos sem contexto pode executar a decisão errada rapidamente.

Termo para lembrar: Recovery runbook | Base técnica: referência 1.

27. Como detecta corrupção silenciosa?

Uso verificações e reconciliações apropriadas, amostras de restauração e indicadores de invariantes. Backup bem-sucedido pode copiar dados já incorretos; a estratégia precisa medir qualidade do conteúdo além de disponibilidade dos arquivos.

Termo para lembrar: Data integrity | Base técnica: referência 1.

28. Qual teste financeiro é decisivo?

Restauro estado anterior a um efeito externo conhecido e verifico que a retomada não duplica a operação. Isso exercita a fronteira que testes de banco isolados ignoram e comprova a necessidade de reconciliação.

Termo para lembrar: Financial recovery test | Base técnica: referência 2.

29. Quais métricas acompanho?

Idade do ponto recuperável, sucesso de restores, duração por etapa e pendências pós-recuperação. Um painel de backups verdes não demonstra RTO; mantenho evidência de ensaios e tempo real de validação.

Termo para lembrar: Recovery SLI | Base técnica: referência 1.

30. Como defenderia o plano?

Eu escolheria uma falha concreta e percorreria perda admitida, cópia usada, autoridade de escrita e validação final. A recuperação termina com dados e efeitos coerentes, não somente com infraestrutura novamente acessível.

Termo para lembrar: Recovery proof | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. RPO menor

Ganho. Menos perda

Custo e motivo. Mais replicação e latência

Como decidir. Para dados críticos

02. RTO menor

Ganho. Retomada rápida

Custo e motivo. Capacidade e automação

Como decidir. Para funções essenciais

03. Backup frequente

Ganho. Menor trabalho de replay

Custo e motivo. I/O e armazenamento

Como decidir. Conforme janela recuperável

04. WAL contínuo

Ganho. PITR granular

Custo e motivo. Operação do arquivo

Como decidir. Para recuperação temporal

05. Warm standby

Ganho. Menor provisionamento

Custo e motivo. Custo contínuo

Como decidir. Quando RTO exige

06. Restore sob demanda

Ganho. Menor custo normal

Custo e motivo. Recuperação lenta

Como decidir. Quando prazo permite

07. Failover automático

Ganho. Decisão rápida

Custo e motivo. Risco de promoção errada

Como decidir. Com critérios confiáveis

08. Validação humana

Ganho. Controle em ambiguidade

Custo e motivo. Aumenta RTO

Como decidir. Para exceções de alto risco

09. Cópia isolada

Ganho. Resiste a comprometimento

Custo e motivo. Gestão adicional

Como decidir. Para desastre administrativo

10. Retenção longa

Ganho. Recupera corrupção tardia

Custo e motivo. Custo e exclusão

Como decidir. Conforme risco e política

Duas alternativas ao caminho proposto

1. Alta disponibilidade local

Quando faz sentido. Falha-alvo é nó ou zona

Como implementar. Replicar e promover com fencing

Limites. Não cobre toda perda regional ou corrupção lógica

2. Reconstrução de fonte autoritativa

Quando faz sentido. Dados derivados são recriáveis

Como implementar. Refazer projeção com snapshot e replay

Limites. Fonte e janela de retenção precisam sobreviver

Referências e alcance da evidência

1. PostgreSQL - PITR

Base backup, WAL, recuperação temporal e timelines.

https://www.postgresql.org/docs/current/continuous-archiving.html

2. AWS Builders Library - Idempotent APIs

Identidades estáveis, intenção, retries e requisições atrasadas.

https://aws.amazon.com/builders-library/making-retries-safe-with-idempotent-APIs/

3. Hazelcast - FencedLock

Tokens monotônicos e rejeição de clientes obsoletos pelo recurso protegido.

https://docs.hazelcast.com/hazelcast/latest/data-structures/fencedlock

Continue em: T016 · T020 · T062 · T063

T068 / G. Correção e resiliência

Observabilidade: métricas, logs e traces

Observabilidade conecta sintomas do usuário a comportamento interno por métricas, logs, traces e contexto consistente.

Quando usar. Diagnóstico, SLOs, capacidade, incidentes e validação de mudanças.

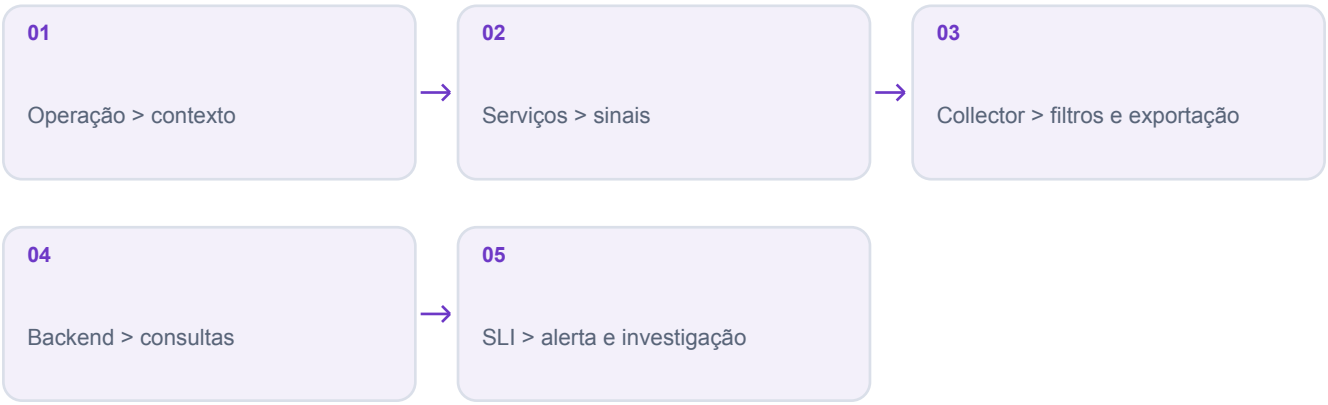
Quando evitar ou limitar. Não coletar tudo indiscriminadamente nem confundir dashboards de infraestrutura com sucesso do negócio.

Uma abertura possível

Vou definir primeiro quais perguntas precisamos responder durante uma falha e instrumentar o caminho completo da operação.

Peças e tecnologias. OpenTelemetry · Prometheus · Grafana · CloudWatch · Tempo · Loki

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Definir SLIs e perguntas operacionais

2. Instrumentar fronteiras e IDs sem segredos

3. Controlar cardinalidade e amostragem

4. Criar alertas com ação e runbook

5. Testar telemetria durante falhas e sobrecarga

30 perguntas para sustentar o desenho

01. Métricas, logs e traces são intercambiáveis?

Não. Métricas resumem comportamento agregado, logs registram eventos e traces conectam etapas de uma operação. Uso sinais complementares e contexto comum, evitando tentar reconstruir tudo apenas por mensagens textuais dispersas.

Termo para lembrar: Observability signals | Base técnica: referência 1.

02. Qual métrica começa pelo usuário?

Escolho resultado útil, como proporção de pagamentos concluídos dentro do prazo. Latência da API de entrada é importante, mas pode estar excelente enquanto operações ficam presas por horas no processamento assíncrono.

Termo para lembrar: Business SLI | Base técnica: referência 2.

03. Quais sinais básicos monitoraria num serviço?

Latência, tráfego, erros e saturação são um ponto inicial útil. Depois adapto ao domínio e às filas; CPU baixa não prova capacidade quando o serviço está esperando rede ou conexões de banco.

Termo para lembrar: Golden signals | Base técnica: referência 2.

04. Por que média de latência é insuficiente?

Ela pode esconder usuários com experiências muito lentas. Observo distribuição e percentis apropriados, além de classes de operação; misturar leitura barata com processamento pesado pode mascarar regressões importantes em um grupo específico.

Termo para lembrar: Tail latency | Base técnica: referência 2.

05. Pode calcular p99 global fazendo média dos p99 locais?

Não. Percentis não se agregam dessa maneira. Uso histogramas ou dados agregáveis adequados e entendo buckets e erro; uma média de percentis pode apresentar uma cauda que nenhum conjunto real possui.

Termo para lembrar: Histogram aggregation | Base técnica: referência 1.

06. O que é cardinalidade de métricas?

É a quantidade de combinações de labels ou dimensões. IDs de usuário e operação podem criar séries em excesso; mantenho dimensões agregáveis e uso logs ou traces para investigação individual autorizada.

Termo para lembrar: Metric cardinality | Base técnica: referência 1.

07. Como correlaciona chamada assíncrona?

Propago contexto apropriado e identidade de operação no evento, respeitando limites e privacidade. A causalidade pode envolver links em vez de uma única árvore contínua; não dependo de manter uma conexão aberta.

Termo para lembrar: Trace context | Base técnica: referência 1.

08. Trace ID substitui operation_id?

Não. Trace ID ajuda diagnóstico de execução; operation_id representa intenção de negócio e pode atravessar retries e vários traces. Preservo ambos quando necessário sem usar telemetria como única autoridade de estado.

Termo para lembrar: Correlation identity | Base técnica: referência 1.

09. Como evita vazar dados em logs?

Defino allowlist de campos, mascaramento e revisão, evitando payloads completos, tokens e segredos. Controle de acesso e retenção também importam; log interno não é automaticamente um lugar seguro para dados bancários.

Termo para lembrar: Data minimization | Base técnica: referência 1.

10. Sampling pode esconder erro raro?

Pode. Defino estratégia de amostragem e retenção conforme pergunta, preservando sinais agregados e casos relevantes. Tail sampling ajuda selecionar traces por resultado, mas adiciona buffers e custo; não garante captura perfeita de tudo.

Termo para lembrar: Trace sampling | Base técnica: referência 1.

11. O collector pode derrubar a aplicação?

Não deveria por desenho. Uso buffers limitados e exportação com política de falha; telemetria não crítica pode degradar antes do serviço. Auditoria obrigatória possui contrato separado e não deve ser descartada silenciosamente.

Termo para lembrar: Telemetry backpressure | Base técnica: referência 1.

12. Logs de auditoria e debug têm a mesma política?

Não. Auditoria registra ações relevantes com integridade e retenção definidas; debug serve diagnóstico e pode ser amostrado ou temporário. Misturar ambos dificulta custo, privacidade e prova do que aconteceu.

Termo para lembrar: Audit versus diagnostics | Base técnica: referência 2.

13. Como monitora fila?

Meço taxa de entrada e conclusão, idade do item mais antigo, backlog por classe e prazo restante. Quantidade isolada engana quando tarefas variam de custo ou uma mensagem crítica fica parada.

Termo para lembrar: Queue age | Base técnica: referência 2.

14. Como monitora idempotência?

Conto duplicatas reconhecidas, conflitos de payload e operações em estado desconhecido. Duplicatas esperadas não significam corrupção, mas aumento repentino pode indicar timeout ruim, relay repetindo ou problema de confirmação do consumidor.

Termo para lembrar: Idempotency telemetry | Base técnica: referência 2.

15. Como mede experiência de uma tela composta?

Registro tempo até contexto útil, falhas por widget e interação no cliente. Um BFF rápido não prova renderização responsiva; correlaciono backend com equipamento e rede representativos sem coletar dados pessoais desnecessários.

Termo para lembrar: Real user monitoring | Base técnica: referência 2.

16. Qual alerta deve acordar alguém?

Um sintoma com impacto relevante e ação possível, associado a responsabilidade e runbook. Evito pager para toda variação interna; alertas sem urgência podem virar ticket ou dashboard para análise posterior.

Termo para lembrar: Actionable alert | Base técnica: referência 2.

17. O que é burn rate?

É a velocidade de consumo do orçamento de erro em relação ao permitido pelo SLO. Ajuda distinguir degradação urgente e lenta; escolho janelas e limiares coerentes com tempo de reação e risco.

Termo para lembrar: Error budget burn | Base técnica: referência 2.

18. Como trata falta de métricas?

Ausência de dados não deve ser interpretada automaticamente como saúde. Monitoro o pipeline e defino comportamento de consultas e alertas quando séries desaparecem, especialmente após deploy, falha do collector ou perda de rede.

Termo para lembrar: Missing data | Base técnica: referência 1.

19. Como evita alertas duplicados num incidente?

Agrupo por sintoma e dependência, preservando contexto para diagnóstico. Se uma falha de banco dispara centenas de páginas, a equipe perde tempo; correlação ajuda sem ocultar impactos diferentes que exigem ações próprias.

Termo para lembrar: Alert grouping | Base técnica: referência 2.

20. Como traces ajudam em fan-out?

Mostram duração e dependências das chamadas, permitindo localizar a cauda e trabalho desnecessário. Ainda comparo volume e amostragem; um trace isolado não prova distribuição global nem explica todos os usuários afetados.

Termo para lembrar: Critical path | Base técnica: referência 1.

21. Como instrumenta retries?

Registro tentativa física, intenção lógica, causa e atraso, sem inflar métricas de sucesso do negócio. Uma operação concluída após cinco tentativas é um resultado, mas cinco consumos de capacidade que precisam ser observados.

Termo para lembrar: Retry observability | Base técnica: referência 1.

22. Como mede custo por operação?

Relaciono consumo de compute, storage, rede e serviços com volume de resultados úteis, reconhecendo custos compartilhados. Isso permite avaliar otimização sem celebrar redução de CPU que aumenta latência ou falhas financeiras.

Termo para lembrar: Unit economics | Base técnica: referência 2.

23. Pode guardar trace completo de cada pagamento para sempre?

Tecnicamente pode ser caro e inadequado à política de dados. Separo trilha financeira autoritativa de telemetria amostrada, com retenção e acesso definidos; diagnóstico não deve virar cópia ilimitada de informação sensível.

Termo para lembrar: Retention design | Base técnica: referência 1.

24. Como testa instrumentação?

Provoco erro conhecido, atraso e mensagem duplicada e verifico se sinais permitem localizar causa e impacto. Uma biblioteca instalada não garante observabilidade; campos, nomes e correlação precisam funcionar em cenários reais.

Termo para lembrar: Observability test | Base técnica: referência 1.

25. Qual risco de dashboard só verde ou vermelho?

Ele pode esconder degradação parcial, tendência e margem de capacidade. Mostro distribuição, segmentação e contexto temporal; um status binário é útil para comunicação, mas insuficiente para diagnóstico e planejamento técnico.

Termo para lembrar: Diagnostic context | Base técnica: referência 2.

26. Como padroniza sinais entre times?

Uso convenções semânticas e atributos de serviço consistentes, com contratos de instrumentação. Padronização facilita consultas, mas permito métricas de domínio; impor somente nomes genéricos pode apagar o que importa ao negócio.

Termo para lembrar: Semantic conventions | Base técnica: referência 1.

27. Como observa um parceiro externo?

Meço latência, erros, quota e resultados conhecidos ou UNKNOWN no adapter, além de prazo de conclusão. Não tenho visibilidade interna do parceiro, então uso evidências contratuais e reconciliação para preencher essa fronteira.

Termo para lembrar: External observability | Base técnica: referência 2.

28. Como protege acesso à telemetria?

Aplico autorização, segregação por ambiente e registro de consultas sensíveis quando necessário. Operadores podem precisar de detalhes, mas acesso ao dashboard não deve conceder automaticamente leitura irrestrita de dados pessoais em logs.

Termo para lembrar: Telemetry authorization | Base técnica: referência 1.

29. Qual risco de alta cardinalidade em baggage?

Propagar dados demais aumenta payload e pode vazar contexto a serviços externos. Uso campos mínimos e limites; baggage não é canal para transportar segredos ou substituir consulta autorizada de dados do usuário.

Termo para lembrar: Baggage hygiene | Base técnica: referência 1.

30. Como defenderia a estratégia?

Eu escolheria um pagamento atrasado e mostraria como encontrar intenção, etapa bloqueada, tentativas e impacto no prazo. A instrumentação é boa quando permite responder essa pergunta com segurança e custo controlado.

Termo para lembrar: Diagnostic workflow | Base técnica: referência 2.

10 trade-offs: ganho, custo e decisão

01. Mais traces

Ganho. Diagnóstico detalhado

Custo e motivo. Custo e exposição

Como decidir. Para caminhos críticos selecionados

02. Sampling

Ganho. Reduz custo

Custo e motivo. Pode perder casos

Como decidir. Com métricas completas apropriadas

03. Labels ricos

Ganho. Segmentação

Custo e motivo. Cardinalidade

Como decidir. Somente para dimensões controladas

04. Logs detalhados

Ganho. Evidência local

Custo e motivo. Volume e privacidade

Como decidir. Com allowlist e retenção

05. Alertar por sintoma

Ganho. Foco no usuário

Custo e motivo. Causa exige investigação

Como decidir. Para pager

06. Alertar por recurso

Ganho. Antecipação

Custo e motivo. Ruído

Como decidir. Quando ação preventiva é clara

07. Collector central

Ganho. Governança

Custo e motivo. Dependência compartilhada

Como decidir. Com buffers e isolamento

08. Agentes locais

Ganho. Isola transporte

Custo e motivo. Mais operação

Como decidir. Para ambientes distribuídos

09. Retenção longa

Ganho. Histórico de investigação

Custo e motivo. Custo e política de dados

Como decidir. Quando uso justifica

10. Convenções comuns

Ganho. Consultas reutilizáveis

Custo e motivo. Governança adicional

Como decidir. Para muitos times

Duas alternativas ao caminho proposto

1. Telemetria nativa da nuvem

Quando faz sentido. Stack concentrada num provedor

Como implementar. Usar métricas e logs gerenciados com correlação

Limites. Portabilidade e custo de consultas precisam avaliação

2. Instrumentação mínima orientada a SLO

Quando faz sentido. Serviço simples e equipe pequena

Como implementar. Medir sucesso, latência e pendências com logs estruturados

Limites. Menos detalhe para investigações complexas

Referências e alcance da evidência

1. OpenTelemetry - Signals

Métricas, logs, traces e sinais de observabilidade.

https://opentelemetry.io/docs/concepts/signals/

2. Google SRE - Monitoring

Sinais operacionais, sintomas e monitoramento orientado ao serviço.

https://sre.google/sre-book/monitoring-distributed-systems/

Continue em: T002 · T064 · T069 · T070

T069 / G. Correção e resiliência

Incidentes, runbooks e post-mortems

Resposta a incidente coordena mitigação, comunicação e recuperação; post-mortem transforma evidência em melhorias verificáveis.

Quando usar. Falhas com impacto em usuários, integridade, segurança ou prazos de negócio.

Quando evitar ou limitar. Não confundir culpabilização com responsabilização nem encerrar incidente apenas porque a infraestrutura voltou.

Uma abertura possível

Vou estabilizar o serviço e preservar evidências, com uma pessoa coordenando decisões e responsáveis claros por execução e comunicação.

Peças e tecnologias. Runbooks · Pager · Dashboards · Incident log · Feature flags · Auditoria

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Definir severidade e papéis

2. Registrar estado e decisões em documento vivo

3. Mitigar com ações limitadas e reversíveis

4. Verificar dados e obrigações pendentes

5. Revisar causas sistêmicas e acompanhar melhorias

30 perguntas para sustentar o desenho

01. Quando declara um incidente?

Quando o impacto ou risco exige coordenação além do atendimento rotineiro. Uso critérios de usuários, dinheiro, segurança e prazo; declarar cedo pode reduzir confusão, sem esperar conhecer a causa raiz completa.

Termo para lembrar: Incident declaration | Base técnica: referência 1.

02. Qual é a primeira prioridade?

Reduzir impacto e preservar segurança e integridade. Investigo em paralelo, mas não espero uma explicação perfeita para aplicar mitigação segura conhecida; também evito mudanças improvisadas que destruam evidência ou ampliem a falha.

Termo para lembrar: Mitigation first | Base técnica: referência 1.

03. O que faz o incident commander?

Mantém visão global, define prioridades e coordena responsáveis. Não precisa executar todos os comandos; separar coordenação de investigação evita que a pessoa mais ocupada tecnicamente também seja gargalo de decisão e comunicação.

Termo para lembrar: Incident command | Base técnica: referência 1.

04. Por que separar comunicação da operação?

Quem executa mitigação precisa foco, enquanto usuários e equipes precisam atualizações claras. Um responsável de comunicação reduz interrupções e mantém informação consistente, sem inventar prazo ou causa ainda não comprovados.

Termo para lembrar: Communication role | Base técnica: referência 1.

05. O que registra no documento vivo?

Impacto, linha do tempo, hipóteses, ações tomadas, responsáveis e próximo checkpoint. Registro fatos e incertezas separadamente; isso permite handoff e evita repetir tentativas perigosas porque ninguém sabe o que já mudou.

Termo para lembrar: Incident state document | Base técnica: referência 1.

06. Como decide severidade?

Considero alcance, duração, integridade financeira e possibilidade de agravamento, não apenas número de alertas. Uma duplicidade de crédito em poucos clientes pode exigir resposta mais rigorosa que indisponibilidade breve de um widget opcional.

Termo para lembrar: Severity model | Base técnica: referência 1.

07. Como usa runbook sem seguir cegamente?

Confirmo se pré-condições e sintomas correspondem ao cenário documentado. O runbook oferece passos e validações, mas não substitui julgamento; uma promoção de banco inadequada pode agravar split-brain apesar de comandos corretos.

Termo para lembrar: Runbook preconditions | Base técnica: referência 1.

08. Rollback é sempre a primeira ação?

Não. Verifico se a mudança causou o problema e se rollback é compatível com schema e efeitos já produzidos. Às vezes desligar uma função ou limitar carga é mais seguro que retornar código antigo.

Termo para lembrar: Rollback assessment | Base técnica: referência 1.

09. Como evita várias pessoas mudando produção ao mesmo tempo?

Centralizo coordenação e registro quem altera o quê, com responsáveis de execução. Investigações podem ocorrer em paralelo, mas mudanças concorrentes sem comunicação tornam causalidade e recuperação difíceis de entender.

Termo para lembrar: Change coordination | Base técnica: referência 1.

10. Como comunica resultado desconhecido financeiro?

Explico o estado conhecido e o processo de verificação, sem anunciar rejeição ou sucesso sem evidência. Internamente acompanho operações UNKNOWN e prioridade; comunicação honesta reduz tentativas duplicadas e expectativas incorretas do cliente.

Termo para lembrar: Financial uncertainty | Base técnica: referência 1.

11. Quando usa kill switch?

Quando ele limita dano de uma função com comportamento conhecido e reversível. Testo e autorizo seu uso previamente; um botão global sem definição pode desligar também recuperação, consulta ou obrigações já aceitas.

Termo para lembrar: Kill switch | Base técnica: referência 1.

12. Como trata backlog após recuperar dependência?

Dreno com limites e prioridades, protegendo tráfego novo e o parceiro recém-recuperado. O incidente não termina quando a conexão volta; obrigações atrasadas e prazos perdidos precisam de acompanhamento e reconciliação.

Termo para lembrar: Backlog recovery | Base técnica: referência 1.

13. Qual risco de reiniciar tudo?

Pode apagar evidência volátil, criar tempestade de reconexão e sobrecarregar dependências. Reinício pode ser mitigação válida, mas aplico de forma controlada e verifico a hipótese em vez de usá-lo automaticamente.

Termo para lembrar: Controlled restart | Base técnica: referência 1.

14. Como faz handoff de incidente?

Transmito impacto, estado atual, alterações, riscos e próximos passos, com confirmação explícita do novo responsável. Trocar turno sem transferência clara pode deixar decisões sem dono e perder continuidade numa recuperação longa.

Termo para lembrar: Incident handoff | Base técnica: referência 1.

15. Quando encerra o incidente?

Quando critérios de recuperação estão atendidos, impacto contido e pendências têm acompanhamento adequado. Infraestrutura verde não basta se clientes continuam com operações presas ou dados inconsistentes; diferencio encerramento operacional de ações posteriores.

Termo para lembrar: Exit criteria | Base técnica: referência 1.

16. O que é post-mortem sem culpabilização?

É analisar como o sistema e o contexto permitiram o incidente, sem reduzir explicação a erro individual. Ainda há responsáveis por corrigir condições e acompanhar ações; ausência de culpa não significa ausência de responsabilidade.

Termo para lembrar: Blameless review | Base técnica: referência 2.

17. Por que causa raiz única pode enganar?

Incidentes costumam envolver múltiplas condições, decisões e falhas de defesa. Procuro fatores contribuintes e barreiras ausentes; dizer apenas erro humano ou falta de teste impede entender por que o sistema aceitou aquela situação.

Termo para lembrar: Contributing factors | Base técnica: referência 2.

18. Como escreve uma boa ação corretiva?

Ela tem problema específico, responsável, prazo e critério verificável de sucesso. Melhorar monitoramento é vago; detectar outbox com mais de dez minutos de atraso e testar o alerta é uma ação avaliável.

Termo para lembrar: Action item | Base técnica: referência 2.

19. Toda ação deve ser automatizar?

Não. Posso simplificar arquitetura, remover fluxo, ajustar contrato ou melhorar documentação. Automação de um processo confuso pode apenas executar o erro mais rápido; escolho a intervenção que reduz risco e carga operacional.

Termo para lembrar: Systemic improvement | Base técnica: referência 2.

20. Como prioriza ações do post-mortem?

Comparo probabilidade, impacto, recorrência e custo de mitigação. Nem toda melhoria precisa bloquear todos os deploys, mas riscos de integridade financeira podem exigir contenção imediata enquanto soluções estruturais são planejadas.

Termo para lembrar: Risk prioritization | Base técnica: referência 2.

21. Como evita post-mortem virar documento esquecido?

Acompanho ações no processo normal de engenharia e verifico sua eficácia em testes ou incidentes posteriores. Publicar a análise sem fechar mudanças não reduz risco; mantenho ownership e revisão de resultados.

Termo para lembrar: Follow-through | Base técnica: referência 2.

22. Quais métricas usa na resposta?

Tempo de detecção, mitigação e recuperação ajudam, mas não bastam. Também observo clientes afetados, obrigações financeiras e recorrência; otimizar apenas um tempo pode incentivar encerrar cedo sem resolver o problema real.

Termo para lembrar: Incident metrics | Base técnica: referência 1.

23. Como prepara a equipe antes da falha?

Faço exercícios com runbooks, acessos e comunicação, incluindo restauração e dependência externa. Um procedimento que ninguém executou pode falhar por permissão ou conhecimento ausente justamente durante a pressão do incidente.

Termo para lembrar: Game day | Base técnica: referência 1.

24. O que muda em incidente de segurança?

Preservo evidências e envolvo responsáveis apropriados, limitando acesso e contenção conforme o caso. Não exponho detalhes sensíveis em canais amplos; recuperação técnica e investigação possuem necessidades que devem ser coordenadas.

Termo para lembrar: Security incident boundary | Base técnica: referência 1.

25. Como lida com acesso emergencial?

Uso mecanismo break-glass com escopo, duração e auditoria definidos. A emergência pode exigir acesso especial, mas não justifica credenciais compartilhadas permanentes ou ações sem rastreabilidade sobre dados bancários.

Termo para lembrar: Break-glass access | Base técnica: referência 1.

26. Pode testar hipótese diretamente em todos os clientes?

Prefiro experimento limitado e reversível, com métrica e condição de parada. Durante incidente, pressão não elimina risco de mudança; uma tentativa ampla sem evidência pode aumentar blast radius e dificultar diagnóstico.

Termo para lembrar: Bounded mitigation | Base técnica: referência 1.

27. Como trata alerta que não tinha ação?

Reviso limiar, roteamento e runbook ou retiro o pager. Alertas úteis exigem decisão possível; acumular notificações sem ação produz fadiga e reduz a chance de responder rápido ao próximo sinal importante.

Termo para lembrar: Alert quality | Base técnica: referência 2.

28. Como envolve outros times sem perder coordenação?

Defino pedidos concretos, responsáveis e canal de atualização, mantendo uma visão compartilhada. Várias equipes podem investigar dependências, mas o comando do incidente precisa entender alterações e impactos cruzados antes de executá-las.

Termo para lembrar: Cross-team coordination | Base técnica: referência 1.

29. Como valida correção definitiva?

Reproduzo a falha ou um contraexemplo equivalente e verifico a nova defesa, incluindo recuperação. Uma correção de código sem teste do cenário e sem observabilidade pode apenas mudar a forma como o problema reaparece.

Termo para lembrar: Regression proof | Base técnica: referência 2.

30. Como mostraria maturidade na entrevista?

Eu explicaria detecção, mitigação, comunicação e reconciliação além do desenho feliz. Demonstraria quem decide, como limitar dano e como transformar o incidente em melhoria verificável, sem prometer que sistemas nunca falharão.

Termo para lembrar: Operational ownership | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Declarar cedo

Ganho. Coordenação rápida

Custo e motivo. Pode mobilizar demais

Como decidir. Quando impacto cresce

02. Investigar em paralelo

Ganho. Mais hipóteses

Custo e motivo. Risco de mudanças conflitantes

Como decidir. Com comando claro

03. Mitigação rápida

Ganho. Reduz dano

Custo e motivo. Pode ser temporária

Como decidir. Quando é segura e reversível

04. Rollback

Ganho. Retorna versão conhecida

Custo e motivo. Pode ser incompatível

Como decidir. Após verificar schema e efeitos

05. Kill switch

Ganho. Contém função

Custo e motivo. Perda de capacidade

Como decidir. Com semântica testada

06. Comunicação frequente

Ganho. Reduz incerteza

Custo e motivo. Consome atenção

Como decidir. Com papel dedicado

07. Runbook detalhado

Ganho. Execução consistente

Custo e motivo. Manutenção contínua

Como decidir. Para cenários recorrentes

08. Automação

Ganho. Reduz toil

Custo e motivo. Amplifica erro de regra

Como decidir. Após validar processo

09. Post-mortem amplo

Ganho. Aprendizado sistêmico

Custo e motivo. Tempo de análise

Como decidir. Para impacto relevante

10. Exercício controlado

Ganho. Prepara equipe

Custo e motivo. Custo e risco de teste

Como decidir. Com escopo e abortagem

Duas alternativas ao caminho proposto

1. Tratamento rotineiro por ticket

Quando faz sentido. Impacto baixo e sem urgência

Como implementar. Responsável corrige com prazo e métricas

Limites. Inadequado para dano crescente ou integridade ameaçada

2. Automação de remediação limitada

Quando faz sentido. Falha conhecida e reversível

Como implementar. Executar ação validada com limite e alerta

Limites. Não substitui coordenação em cenários desconhecidos

Referências e alcance da evidência

1. Google SRE - Managing incidents

Coordenação, papéis e comunicação durante incidentes.

https://sre.google/sre-book/managing-incidents/

2. Google SRE - Postmortem culture

Aprendizado sem culpabilização e ações de melhoria após incidentes.

https://sre.google/sre-book/postmortem-culture/

Continue em: T002 · T067 · T068 · T070

T070 / G. Correção e resiliência

Canary, blue-green e rollback

Deploy seguro limita exposição, mede regressão e preserva compatibilidade; rollback técnico não desfaz efeitos de negócio.

Quando usar. Mudanças de código, configuração, schema e infraestrutura em serviços críticos.

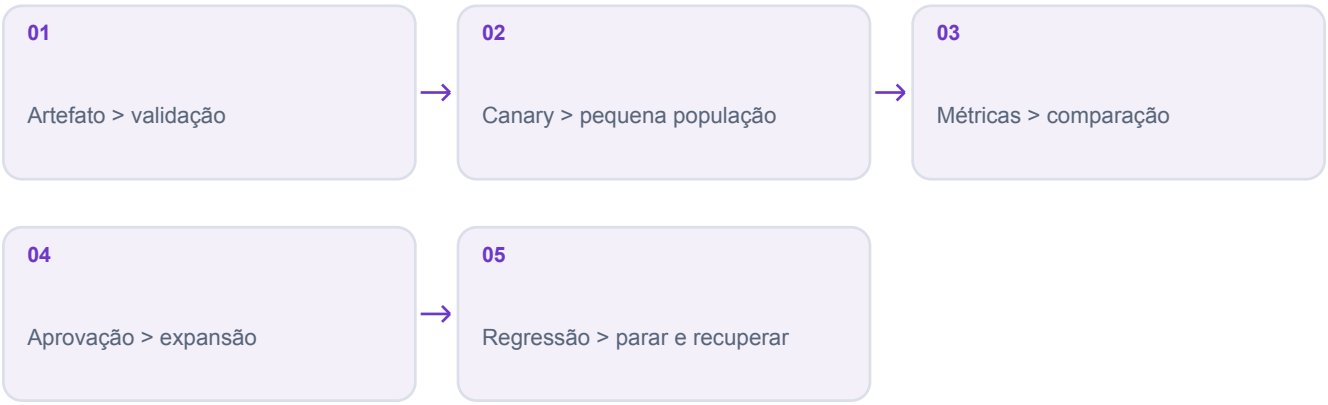
Quando evitar ou limitar. Não prometer segurança apenas por blue-green nem executar shadow traffic com efeitos financeiros reais.

Uma abertura possível

Vou definir população, métricas de comparação e condição de parada antes de liberar a nova versão para mais usuários.

Peças e tecnologias. Canary · Blue-green · Feature flags · Argo Rollouts · Load balancer · CI/CD

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Preparar mudança compatível e reversível

2. Escolher amostra representativa

3. Definir métricas e limites de aborto

4. Expandir gradualmente com capacidade

5. Validar dados e efeitos após rollback

30 perguntas para sustentar o desenho

01. Canary e blue-green são iguais?

Não. Canary expõe parte controlada do tráfego para avaliar a mudança; blue-green mantém ambientes e troca encaminhamento. Posso combinar os dois, mas ambientes duplicados sozinhos não fornecem comparação gradual ou detecção adequada.

Termo para lembrar: Deployment strategy | Base técnica: referência 1.

02. Qual vantagem do canary?

Limita quantidade de usuários expostos enquanto obtenho evidência da versão nova. O benefício depende de amostra e métricas representativas; poucos requests felizes não provam ausência de falha em operações raras ou financeiras.

Termo para lembrar: Canary evaluation | Base técnica: referência 1.

03. Como escolhe a população?

Incluo classes relevantes de clientes, rotas e carga, evitando selecionar apenas casos fáceis. Para operações sensíveis, começo com escopo seguro e expando; a amostra precisa representar riscos sem expor todos imediatamente.

Termo para lembrar: Representative sample | Base técnica: referência 1.

04. Quanto tempo deixa o canary?

Depende de volume e sinais que precisam aparecer, como filas e vazamentos de memória. Uma janela curta pode detectar crash, mas não atraso acumulado; defino duração baseada no comportamento e no orçamento de risco.

Termo para lembrar: Canary duration | Base técnica: referência 1.

05. Que métricas bloqueiam rollout?

Erro, latência, saturação e indicadores de negócio apropriados, comparados com controle. CPU melhor não compensa mais operações incorretas; critérios incluem integridade e conclusão, não apenas disponibilidade da rota inicial.

Termo para lembrar: Release gates | Base técnica: referência 1.

06. Por que comparar com controle simultâneo?

Ajuda separar regressão da versão de mudanças de tráfego ou dependência externa. Comparar manhã com noite pode confundir sazonalidade; ainda verifico interferência entre canary e controle quando compartilham infraestrutura.

Termo para lembrar: Concurrent control | Base técnica: referência 1.

07. Canary pode afetar usuários fora da amostra?

Sim. Ele pode sobrecarregar banco ou alterar dados compartilhados consumidos pela versão antiga. Limito capacidade e avalio isolamento; porcentagem de tráfego não é automaticamente porcentagem máxima de impacto.

Termo para lembrar: Shared dependency blast radius | Base técnica: referência 1.

08. Como faz rollback de schema?

Prefiro expand-contract e mudanças compatíveis para permitir retorno de código. Remover coluna ou mudar significado de dados pode tornar rollback inseguro; planejo migração e reversão antes do deploy, não durante o incidente.

Termo para lembrar: Schema compatibility | Base técnica: referência 2.

09. Rollback desfaz crédito executado?

Não. Ele troca versão ou configuração, mas efeitos financeiros persistem. Reconciliamos e executamos correções de negócio autorizadas quando necessário; não confundimos voltar código com restaurar o mundo anterior ao deploy.

Termo para lembrar: Business rollback boundary | Base técnica: referência 3.

10. Feature flag substitui deploy?

Não. Ela separa ativação de entrega de código e ajuda limitar exposição, mas adiciona configuração e estados de teste. A flag precisa de dono, validade e política; não deve virar dívida permanente invisível.

Termo para lembrar: Feature flag | Base técnica: referência 1.

11. Como escolhe quem recebe uma flag?

Uso atribuição estável e autorizada conforme a experiência desejada, evitando alternância a cada requisição. A flag habilita comportamento, mas não concede permissão financeira nem substitui validação de elegibilidade no domínio.

Termo para lembrar: Stable cohort | Base técnica: referência 1.

12. O que é shadow traffic seguro?

Replico entradas para observar uma versão sem produzir efeitos reais, com dados e acesso controlados. Desabilito ou isolo chamadas financeiras, notificações e gravações externas; sombra que duplica efeitos não é teste seguro.

Termo para lembrar: Shadow traffic | Base técnica: referência 1.

13. Blue-green dobra custo sempre?

Pode exigir capacidade adicional relevante, mas depende de recursos compartilhados e estratégia. Comparo custo com tempo de troca e reversão; compartilhar banco reduz duplicação, porém mantém risco de schema e interferência.

Termo para lembrar: Blue-green capacity | Base técnica: referência 1.

14. Como drena conexões antigas?

Paro novas atribuições e permito conclusão dentro de limite, com reconexão e replay para canais longos. Encerrar instâncias imediatamente pode criar respostas perdidas e retries; comandos continuam protegidos por identidade.

Termo para lembrar: Connection draining | Base técnica: referência 1.

15. Como faz rollout de workers?

Considero coexistência de versões consumindo mensagens antigas e novas, além de leases e checkpoints. Um worker antigo não pode interpretar payload novo incorretamente; contratos e compatibilidade precisam cobrir backlog e redrive.

Termo para lembrar: Consumer rollout | Base técnica: referência 2.

16. Mudar configuração merece canary?

Sim quando pode alterar comportamento ou alcance significativamente. Uma regra de gateway ou quota pode afetar todo tráfego sem mudar código; versiono, valido e libero com observação e reversão apropriadas.

Termo para lembrar: Configuration deployment | Base técnica: referência 1.

17. Como evita duas mudanças confundindo diagnóstico?

Limito sobreposição ou mantenho rastreabilidade e isolamento suficientes. Vários canaries simultâneos no mesmo caminho dificultam atribuição de regressão; velocidade de deploy não deve eliminar capacidade de entender qual mudança causou impacto.

Termo para lembrar: Change attribution | Base técnica: referência 1.

18. Rollback automático é sempre seguro?

Não. Verifico se retornar versão é compatível e se o sinal é confiável. Automatizo ações conhecidas e reversíveis, com limites; algumas mudanças exigem parar expansão e investigar antes de executar uma reversão potencialmente danosa.

Termo para lembrar: Automated rollback | Base técnica: referência 1.

19. Como lida com warm-up?

Aqueço capacidade gradualmente e observo cache, JIT ou pools conforme runtime. Uma réplica fria pode parecer pior temporariamente; ainda não uso warm-up como desculpa para ignorar regressão persistente ou erro funcional.

Termo para lembrar: Warm-up | Base técnica: referência 1.

20. O que faz se o canary tiver pouco tráfego?

Aumento duração, uso tráfego sintético seguro ou expando cuidadosamente uma amostra adequada. Ausência de falhas em poucas requisições oferece evidência fraca; explicito essa incerteza antes de liberar uma mudança crítica.

Termo para lembrar: Statistical power | Base técnica: referência 1.

21. Como mede vazamento de memória no rollout?

Observo crescimento por tempo e volume, incluindo sessões longas e ciclos de carga. Um canary curto pode não revelar vazamento; combino teste de duração e limites operacionais com análise da tendência.

Termo para lembrar: Soak testing | Base técnica: referência 1.

22. Como mantém compatibilidade com app antigo?

Evoluo contrato de forma que versões antigas continuem funcionando dentro da política definida. Usuários não atualizam simultaneamente; rollback do backend e cache de assets precisam considerar essa coexistência prolongada.

Termo para lembrar: Backward compatibility | Base técnica: referência 2.

23. Pode usar migração de dados destrutiva junto do código?

Evito acoplar tudo numa única mudança irreversível. Introduzo novo formato, faço backfill verificável e migro leitura antes de remover antigo; isso reduz risco e permite parar sem perder o caminho anterior.

Termo para lembrar: Expand-contract | Base técnica: referência 2.

24. Como reverte uma feature financeira parcialmente usada?

Desativo novas entradas conforme política e mantenho processamento de intenções já aceitas, com reconciliação. Apagar a rota ou flag não pode abandonar obrigações; recuperação considera estado e contratos produzidos pela versão anterior.

Termo para lembrar: Accepted work continuity | Base técnica: referência 3.

25. Que testes vêm antes do canary?

Testes de contrato, invariantes e cenários de falha relevantes. Canary complementa validação em tráfego real; não é justificativa para enviar código sem testes que detectariam uma duplicidade financeira simples antes da produção.

Termo para lembrar: Preproduction validation | Base técnica: referência 1.

26. Como protege error budget?

Limito exposição e tempo de detecção e reversão, relacionando risco ao SLO. Uma pequena amostra ainda consome orçamento; não trato usuários do canary como impacto gratuito que não conta para disponibilidade.

Termo para lembrar: Release risk budget | Base técnica: referência 1.

27. O que registra para auditoria de deploy?

Artefato, configuração, autorizações, horários, população e decisões de expansão ou parada. Esses dados permitem correlacionar incidente e reconstruir comportamento, sem depender de lembrar qual imagem estava em cada instância.

Termo para lembrar: Release provenance | Base técnica: referência 1.

28. Como testa rollback antes de precisar?

Executo ensaio com dados e contratos representativos, incluindo versões coexistentes e jobs pendentes. Ter imagem antiga armazenada não prova que ela ainda consegue operar após mudanças de schema ou eventos.

Termo para lembrar: Rollback rehearsal | Base técnica: referência 2.

29. Quando blue-green pode ser inadequado?

Quando a duplicação de ambiente é inviável ou a dependência de dados impede troca simples. Posso usar rolling ou canary com compatibilidade; escolho estratégia pelo risco e operação, não pelo nome mais sofisticado.

Termo para lembrar: Deployment fit | Base técnica: referência 1.

30. Como defenderia o plano?

Eu mostraria a primeira população exposta, o sinal que interrompe expansão e o caminho de recuperação de dados e efeitos. O plano é seguro quando uma regressão concreta possui resposta executável e validada.

Termo para lembrar: Release safety proof | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Canary pequeno

Ganho. Menos exposição

Custo e motivo. Pouca evidência

Como decidir. Para primeira etapa

02. Canary maior

Ganho. Sinal representativo

Custo e motivo. Mais usuários afetados

Como decidir. Após validação inicial

03. Janela longa

Ganho. Detecta efeitos lentos

Custo e motivo. Menor velocidade

Como decidir. Para filas e memória

04. Blue-green

Ganho. Troca rápida

Custo e motivo. Capacidade duplicada

Como decidir. Quando ambientes podem coexistir

05. Rolling

Ganho. Menor capacidade extra

Custo e motivo. Coexistência prolongada

Como decidir. Com contratos compatíveis

06. Flag

Ganho. Ativação independente

Custo e motivo. Mais estados e dívida

Como decidir. Com dono e remoção planejada

07. Rollback automático

Ganho. Mitigação rápida

Custo e motivo. Falso sinal pode prejudicar

Como decidir. Para reversão comprovadamente segura

08. Shadow traffic

Ganho. Observa sem responder usuário

Custo e motivo. Risco de side effects

Como decidir. Com isolamento rigoroso

09. Expand-contract

Ganho. Mantém reversibilidade

Custo e motivo. Mais etapas

Como decidir. Para schema e contratos críticos

10. Controle simultâneo

Ganho. Melhor atribuição

Custo e motivo. Infra compartilhada interfere

Como decidir. Com métricas segmentadas

Duas alternativas ao caminho proposto

1. Rolling update conservador

Quando faz sentido. Mudança simples e compatível

Como implementar. Trocar réplicas gradualmente com checks e drenagem

Limites. Sem avaliação de negócio automática por si só

2. Janela de manutenção

Quando faz sentido. Sistema e contrato toleram interrupção

Como implementar. Parar entrada, migrar e validar antes de reabrir

Limites. Custo de disponibilidade e coordenação com usuários

Referências e alcance da evidência

1. Google SRE Workbook - Canarying releases

Avaliação incremental e redução de risco em lançamentos.

https://sre.google/workbook/canarying-releases/

2. PostgreSQL - Transaction isolation

Snapshots, concorrência, isolamento e falhas de serialização.

https://www.postgresql.org/docs/current/transaction-iso.html

3. AWS Builders Library - Idempotent APIs

Identidades estáveis, intenção, retries e requisições atrasadas.

https://aws.amazon.com/builders-library/making-retries-safe-with-idempotent-APIs/

Continue em: T068 · T069 · T076 · T077

T071 / H. Segurança e plataforma

Autenticação: OAuth, OIDC e sessões

Autenticação estabelece identidade; OAuth delega acesso e OIDC adiciona uma camada padronizada de identidade.

Quando usar. Login de clientes e atendentes, SSO e acesso entre aplicações.

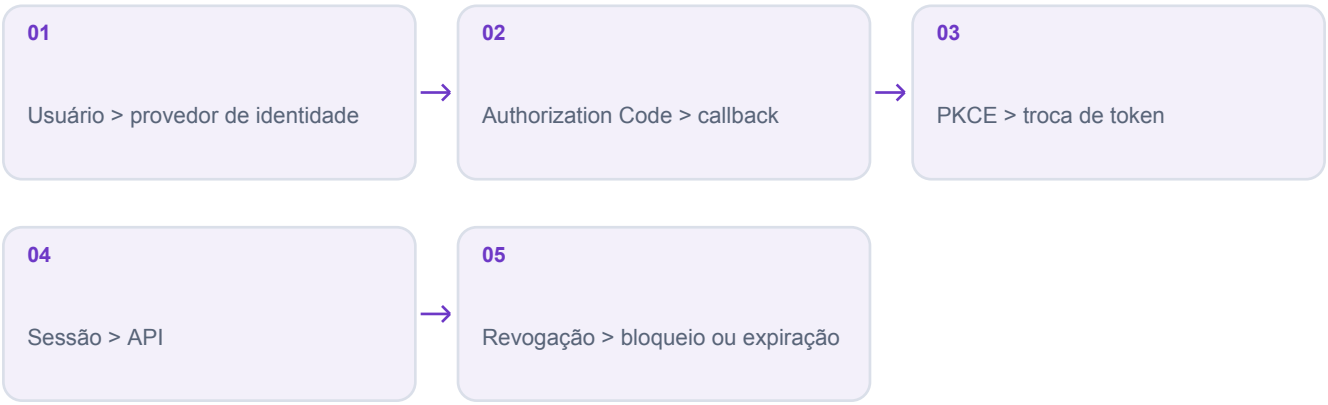
Quando evitar ou limitar. Não usar ID Token como access token indistintamente nem tratar JWT assinado como autorização completa.

Uma abertura possível

Vou definir quem autentica, quem emite tokens, quem valida e como sessões deixam de valer após revogação ou expiração.

Peças e tecnologias. OAuth 2.0 · OIDC · PKCE · JWT · Cookies · Identity Provider

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Escolher fluxo seguro para tipo de cliente

2. Validar issuer, audience, assinatura e contexto

3. Proteger tokens e cookies

4. Definir renovação e revogação

5. Testar replay, logout e troca de chaves

30 perguntas para sustentar o desenho

01. OAuth e OIDC são a mesma coisa?

Não. OAuth trata autorização delegada; OIDC define uma camada de identidade e ID Tokens sobre esse ecossistema. Escolho os artefatos corretos para login e acesso à API, sem inferir identidade por um token qualquer.

Termo para lembrar: OAuth versus OIDC | Base técnica: referência 2.

02. Para que serve PKCE?

Ele vincula a troca do authorization code ao cliente que iniciou o fluxo, reduzindo risco de interceptação. Uso conforme boas práticas atuais e mantenho validações de redirect e contexto; PKCE não substitui todas as proteções.

Termo para lembrar: PKCE | Base técnica: referência 1.

03. ID Token deve ser enviado a qualquer API?

Não. Ele comunica autenticação ao cliente destinatário conforme OIDC. APIs validam o token de acesso apropriado e sua audiência; misturar tipos pode permitir uso fora do contexto para o qual o artefato foi emitido.

Termo para lembrar: Token audience | Base técnica: referência 2.

04. O que valida num JWT?

Assinatura com algoritmo permitido, emissor, audiência, validade e demais condições do contrato. Decodificar o payload não verifica autenticidade; também não aceito algoritmo ou chave escolhidos livremente pelo conteúdo não confiável.

Termo para lembrar: JWT validation | Base técnica: referência 2.

05. JWT é criptografado por definição?

Não. Um token assinado pode ter payload legível por quem o recebe. Evito dados sensíveis desnecessários em claims e protejo transporte e armazenamento; assinatura oferece integridade e autenticidade, não sigilo automático.

Termo para lembrar: Signed versus encrypted | Base técnica: referência 2.

06. Onde guarda token no navegador?

Escolho arquitetura considerando XSS e CSRF, frequentemente usando sessão protegida por cookie no BFF. Não declaro um armazenamento universalmente seguro; reduzo exposição de tokens e aplico controles específicos ao fluxo escolhido.

Termo para lembrar: Token storage | Base técnica: referência 3.

07. O que HttpOnly faz?

Impede acesso ao cookie por APIs comuns de script, reduzindo uma forma de roubo por XSS. Não impede que script malicioso execute ações na sessão; validação, proteção de conteúdo e autorização continuam necessárias.

Termo para lembrar: HttpOnly | Base técnica: referência 3.

08. SameSite resolve todo CSRF?

Não como única medida em qualquer fluxo. Configuro cookies e defesa de requisições conforme contexto, incluindo tokens ou verificações apropriadas; integrações cross-site legítimas podem exigir políticas que precisam ser avaliadas cuidadosamente.

Termo para lembrar: CSRF protection | Base técnica: referência 3.

09. Por que regenerar sessão após login?

Para evitar que um identificador previamente conhecido continue representando a sessão autenticada. Também renovo em mudanças relevantes de privilégio; ciclo de vida da sessão precisa impedir fixation e manter invalidação coerente.

Termo para lembrar: Session fixation | Base técnica: referência 3.

10. Como diferencia idle timeout e duração máxima?

Idle timeout limita inatividade; duração absoluta limita vida mesmo com uso contínuo. Defino ambos pelo risco e experiência, incluindo atendentes e dispositivos; atividade automática não deve perpetuar uma sessão sensível indefinidamente.

Termo para lembrar: Session lifetime | Base técnica: referência 3.

11. Como faz logout efetivo?

Invalido a sessão e credenciais renováveis conforme arquitetura, considerando dispositivos e tokens já emitidos. Um JWT autocontido pode continuar válido até expirar se não houver mecanismo adicional; explicito essa janela de revogação.

Termo para lembrar: Logout semantics | Base técnica: referência 3.

12. Como revoga acesso imediatamente?

Preciso de consulta de sessão, introspecção, lista de revogação ou política equivalente no caminho relevante. Tokens curtos reduzem janela, mas não equivalem a revogação instantânea; avalio custo e disponibilidade dessa verificação.

Termo para lembrar: Revocation | Base técnica: referência 1.

13. Por que access token curto?

Limita exposição após roubo, mas exige renovação e gestão de refresh token. O equilíbrio depende do cliente e risco; reduzir duração sem dimensionar o provedor pode criar falhas e tempestades de renovação.

Termo para lembrar: Token lifetime | Base técnica: referência 1.

14. Como protege refresh token?

Restrinjo armazenamento, escopo e uso, com rotação ou vinculação quando apropriado ao cliente e padrão. Reutilização suspeita exige resposta definida; refresh token duradouro pode valer mais ao atacante que o token de acesso.

Termo para lembrar: Refresh rotation | Base técnica: referência 1.

15. MFA elimina fraude?

Não. Reduz alguns riscos de autenticação, mas não autoriza qualquer transação nem impede sessão roubada ou engenharia social. Operações sensíveis podem exigir confirmação adicional e controles de risco próprios no domínio.

Termo para lembrar: Step-up authentication | Base técnica: referência 3.

16. Como autentica serviço sem usuário?

Uso identidade de workload ou fluxo apropriado para cliente confidencial, com credenciais mínimas e rotação. Não reutilizo token de um funcionário como identidade permanente de uma integração automatizada.

Termo para lembrar: Workload authentication | Base técnica: referência 1.

17. Como lida com provedor de identidade fora?

Sessões já válidas podem continuar conforme política, enquanto novos logins ou renovação ficam afetados. Não ignoro validação para preservar disponibilidade; defino comportamento por risco e preparo redundância e cache seguro de chaves.

Termo para lembrar: Identity dependency | Base técnica: referência 2.

18. Como faz cache de JWKS?

Mantenho política de atualização e validade conforme emissor, tratando rotação e kid desconhecido com limites. Não busco chaves de URLs arbitrárias do token nem deixo uma falha de rede autorizar assinatura não verificada.

Termo para lembrar: JWKS rotation | Base técnica: referência 2.

19. Token de outro ambiente deve funcionar?

Não. Valido emissor, audiência e configuração de ambiente para evitar aceitar credenciais de teste em produção. Separação de chaves e políticas reduz esse risco; nome parecido de serviço não prova contexto correto.

Termo para lembrar: Environment isolation | Base técnica: referência 2.

20. Como valida redirect URI?

Uso correspondência e registro conforme boas práticas do fluxo, evitando redirecionamentos abertos. Um callback permissivo pode entregar códigos a destino controlado por atacante, mesmo quando a autenticação no provedor foi legítima.

Termo para lembrar: Redirect validation | Base técnica: referência 1.

21. Para que servem state e nonce?

Eles vinculam partes do fluxo ao contexto esperado, com funções específicas de proteção e correlação. Uso conforme o protocolo e biblioteca confiável; não substituo esses controles por um único parâmetro genérico sem entender sua semântica.

Termo para lembrar: Authentication context | Base técnica: referência 2.

22. Como trata replay de token roubado?

Reduzo duração e exposição, considero vinculação ao cliente quando apropriada e detecto uso suspeito. Bearer token permite uso por quem o possui; TLS não impede abuso depois que ele foi extraído do endpoint.

Termo para lembrar: Bearer token | Base técnica: referência 1.

23. Autenticação responde se usuário pode ver um caso?

Não. Ela estabelece identidade; autorização verifica ação e recurso. Mesmo após login forte, cada acesso ao caso precisa respeitar tenant, atribuição e política atual, incluindo ações executadas por jobs posteriores.

Termo para lembrar: Authentication boundary | Base técnica: referência 2.

24. Como mantém sessão em múltiplas réplicas?

Uso armazenamento ou token com validação consistente e política de revogação. Afinidade de load balancer não pode ser a única forma de encontrar sessão, pois uma queda deve permitir retomada controlada noutra instância.

Termo para lembrar: Distributed session | Base técnica: referência 3.

25. Pode registrar tokens para debug?

Não registro credenciais completas. Uso identificadores ou hashes apropriados para correlação sem permitir reutilização; logs e traces costumam ter distribuição e retenção amplas, aumentando impacto de um segredo exposto.

Termo para lembrar: Credential hygiene | Base técnica: referência 3.

26. Como protege login contra abuso?

Combino controles de taxa, sinais de risco e proteção contra enumeração, mantendo experiência recuperável. Limite apenas por IP pode penalizar usuários atrás de NAT e não bloquear ataque distribuído; adapto à ameaça observada.

Termo para lembrar: Authentication throttling | Base técnica: referência 3.

27. Como lida com conta bloqueada após token emitido?

Defino se a ação consulta estado atual ou aceita janela limitada do token. Para operações críticas, bloqueio precisa alcançar o caminho de autorização; não presumo que uma claim antiga reflita decisão atual.

Termo para lembrar: Account state validation | Base técnica: referência 1.

28. Como testa troca de chave do emissor?

Emito tokens antes e depois da rotação e verifico validação, cache e retirada das chaves conforme política. Testo também kid desconhecido e falha de atualização, sem permitir fallback para aceitar qualquer assinatura.

Termo para lembrar: Key rotation test | Base técnica: referência 2.

29. Quais eventos audita?

Login, falha relevante, renovação suspeita, logout, alteração de privilégio e uso administrativo, com dados mínimos. Auditoria ajuda investigação, mas não deve conter senha, refresh token ou outros segredos reutilizáveis.

Termo para lembrar: Authentication audit | Base técnica: referência 3.

30. Como defenderia o desenho?

Eu seguiria o token desde emissão até validação, mostrando audiência, armazenamento e fim de vida. Depois revogaria um usuário durante sessão longa para demonstrar que identidade autenticada não vira acesso permanente.

Termo para lembrar: Session security proof | Base técnica: referência 2.

10 trade-offs: ganho, custo e decisão

01. Sessão no servidor

Ganho. Revogação controlada

Custo e motivo. Consulta e armazenamento

Como decidir. Para controle central forte

02. JWT autocontido

Ganho. Validação local

Custo e motivo. Revogação menos imediata

Como decidir. Com janela aceita

03. Token curto

Ganho. Menor exposição

Custo e motivo. Mais renovação

Como decidir. Para clientes com fluxo robusto

04. Refresh duradouro

Ganho. Boa experiência

Custo e motivo. Segredo valioso

Como decidir. Com armazenamento e rotação

05. Cookie HttpOnly

Ganho. Reduz extração por script

Custo e motivo. Exige defesa CSRF

Como decidir. Para aplicações web

06. Step-up

Ganho. Protege ação sensível

Custo e motivo. Fricção

Como decidir. Para risco elevado

07. Cache de chaves

Ganho. Resiliência e latência

Custo e motivo. Rotação precisa de política

Como decidir. Com validação de emissor

08. Identidade de workload

Ganho. Menos segredo estático

Custo e motivo. Integração de plataforma

Como decidir. Para serviço automatizado

09. Revogação online

Ganho. Resposta rápida

Custo e motivo. Dependência adicional

Como decidir. Para ações críticas

10. SSO

Ganho. Experiência e governança

Custo e motivo. Provedor compartilhado

Como decidir. Com disponibilidade e isolamento

Duas alternativas ao caminho proposto

1. Sessão opaca com backend próprio

Quando faz sentido. Aplicação controlada e requisitos simples

Como implementar. Cookie aleatório ligado a sessão autorizada no servidor

Limites. Equipe assume autenticação e segurança operacional

2. Provedor de identidade gerenciado

Quando faz sentido. Deseja reduzir implementação de protocolo

Como implementar. Integrar OIDC com validação e autorização locais

Limites. Configuração incorreta ainda cria vulnerabilidade

Referências e alcance da evidência

1. IETF - OAuth Security BCP

Boas práticas atuais de segurança OAuth, fluxos e proteção de tokens.

https://www.rfc-editor.org/rfc/rfc9700.html

2. OpenID Foundation - OIDC Core

Camada de identidade, claims e validação de ID Tokens.

https://openid.net/specs/openid-connect-core-1_0.html

3. OWASP - Session Management

Cookies, ciclo de vida, expiração e proteção de sessões.

https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html

Continue em: T051 · T072 · T073 · T074

T072 / H. Segurança e plataforma

Autorização, RBAC, ABAC e multi-tenancy

Autorização decide se uma identidade pode executar uma ação sobre um recurso no contexto atual.

Quando usar. APIs bancárias, atendimento, multi-tenancy e ações administrativas.

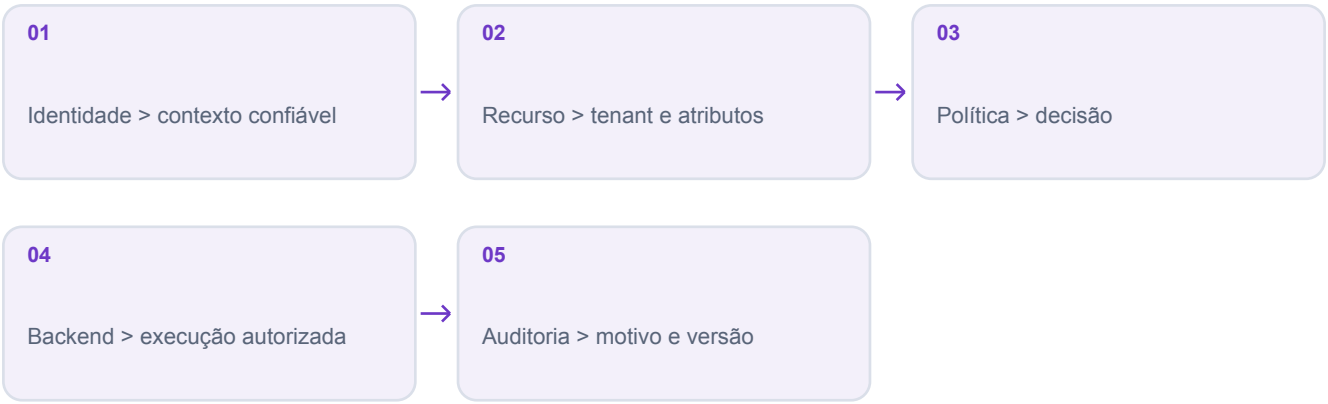
Quando evitar ou limitar. Não confiar em botão oculto, tenant enviado livremente pelo cliente ou papel antigo no JWT como prova suficiente.

Uma abertura possível

Vou definir sujeito, ação, recurso e contexto e aplicar a decisão no backend em cada caminho que produz ou revela dados.

Peças e tecnologias. RBAC · ABAC · ReBAC · OPA · Cedar · PostgreSQL RLS

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Mapear ações e recursos sensíveis

2. Definir política deny by default

3. Aplicar checks no backend e consultas

4. Controlar cache e revogação

5. Testar acesso cruzado e caminhos assíncronos

30 perguntas para sustentar o desenho

01. RBAC é suficiente para um banco?

Pode cobrir papéis gerais, mas regras como atendente atribuído ao caso ou limite por valor exigem contexto adicional. Escolho modelo pelo domínio, evitando criar milhares de papéis para representar cada combinação de recurso.

Termo para lembrar: RBAC | Base técnica: referência 1.

02. Quando usa ABAC?

Quando decisão depende de atributos de sujeito, recurso ou ambiente, como país, valor e classificação. Garanto origem confiável desses atributos; aceitar qualquer atributo declarado pelo cliente transforma política sofisticada em autorização manipulável.

Termo para lembrar: ABAC | Base técnica: referência 1.

03. O que ReBAC acrescenta?

Modela relações, como membro de equipe ou responsável por caso, úteis para permissões derivadas. Ainda preciso definir consistência, revogação e limites de travessia; um grafo não elimina a necessidade de política clara.

Termo para lembrar: Relationship-based access | Base técnica: referência 1.

04. Qual princípio inicial adota?

Deny by default e menor privilégio. Uma ação nova não deve ficar liberada porque ninguém escreveu uma regra de negação; explicito permissões e testo tanto caminhos permitidos quanto proibidos desde o início.

Termo para lembrar: Deny by default | Base técnica: referência 1.

05. Onde aplica autorização?

No backend que lê ou altera o recurso, incluindo jobs e APIs internas. O gateway pode fazer checks gerais, mas não conhece necessariamente estado e ownership atuais; ocultar um botão não protege chamada direta.

Termo para lembrar: Enforcement point | Base técnica: referência 1.

06. Como impede acesso trocando ID na URL?

Carrego ou consulto o recurso dentro do escopo autorizado e verifico a ação solicitada. IDs difíceis de adivinhar ajudam contra enumeração, mas não substituem autorização quando o identificador é conhecido.

Termo para lembrar: BOLA | Base técnica: referência 2.

07. Como protege propriedades de um objeto?

Defino campos que podem ser lidos e alterados por ação e papel. Binding automático do JSON a uma entidade pode permitir mudar status ou owner indevidamente; validação de tipo não resolve autorização por propriedade.

Termo para lembrar: Property authorization | Base técnica: referência 2.

08. Tenant_id do corpo é confiável?

Não sozinho. Derivo tenant do contexto autenticado ou valido explicitamente a relação permitida. O cliente não pode escolher outro tenant apenas preenchendo um campo; essa regra também vale para importações e tarefas em lote.

Termo para lembrar: Tenant context | Base técnica: referência 1.

09. Como impede vazamento em consultas?

Incluo escopo autorizado na consulta ou camada equivalente e testo joins e paginação. Filtrar depois de buscar dados pode vazar contagens ou consumir recursos indevidos; a política precisa cobrir o conjunto retornado.

Termo para lembrar: Query authorization | Base técnica: referência 1.

10. RLS resolve toda autorização?

Pode reforçar isolamento no banco, mas configuração, papéis e caminhos de bypass exigem cuidado. Regras de ação, aprovação e integrações continuam no domínio; uma camada adicional não torna as demais dispensáveis.

Termo para lembrar: Row-level security | Base técnica: referência 1.

11. Como usa cache de permissões?

Defino chave com sujeito, recurso, ação e contexto relevante, além de versão ou TTL compatível com revogação. Cache global por role pode ignorar relações atuais e permitir acesso depois de uma mudança importante.

Termo para lembrar: Authorization cache | Base técnica: referência 1.

12. Revogação imediata combina com JWT longo?

Não automaticamente. Claims antigas podem continuar aceitas; preciso de verificação atual, invalidação ou janela explicitamente tolerada. Para ações financeiras sensíveis, avalio o estado autoritativo no momento da execução, não apenas no login.

Termo para lembrar: Revocation latency | Base técnica: referência 1.

13. Como evita TOCTOU na autorização?

Quando a permissão depende de estado mutável crítico, valido junto da alteração protegida por versão ou transação. Verificar antes e executar muito depois pode usar atribuição ou limite que já deixou de valer.

Termo para lembrar: TOCTOU | Base técnica: referência 1.

14. Como protege maker-checker?

Registro identidade do autor, revisão da decisão e aprovador, impondo separação conforme regra. Aprovação vale para conteúdo específico; alterar valor depois exige nova revisão, não reaproveitar um clique antigo de autorização.

Termo para lembrar: Segregation of duties | Base técnica: referência 1.

15. O job herda permissão do usuário para sempre?

Não necessariamente. Defino se a autorização é capturada como mandato específico ou reavaliada ao executar. Um job durável precisa de contrato para revogação e expiração; não usa automaticamente privilégio administrativo irrestrito.

Termo para lembrar: Delegated authority | Base técnica: referência 1.

16. Como funciona impersonação de suporte?

Exige permissão específica, motivo e auditoria, preservando identidade real do operador e contexto do usuário. Não troco silenciosamente o ator no log; ações sensíveis podem permanecer proibidas mesmo durante assistência autorizada.

Termo para lembrar: Impersonation audit | Base técnica: referência 1.

17. Serviço interno pode confiar em outro serviço?

Confio somente em identidade e contrato verificados, não em estar na rede interna. O chamador pode propagar contexto, mas o destino valida o escopo necessário e evita confused deputy ao agir com privilégios maiores.

Termo para lembrar: Confused deputy | Base técnica: referência 1.

18. Como separa identidade de serviço e usuário?

Registro ambas quando um serviço age em nome de alguém. Isso permite política e auditoria corretas; somente o nome do microsserviço não explica qual cliente autorizou uma transferência ou qual atendente acessou o documento.

Termo para lembrar: Actor chain | Base técnica: referência 1.

19. Como autoriza download com URL assinada?

Verifico acesso antes de emitir e escolho validade compatível com risco. A URL pode continuar utilizável por quem a possui; para revogação forte, considero proxy autorizado ou mecanismos adicionais, não apenas segredo do caminho.

Termo para lembrar: Capability issuance | Base técnica: referência 1.

20. Autorização precisa de serviço central?

Não sempre. Políticas locais reduzem dependência, enquanto decisão central facilita governança e relações globais. Posso distribuir avaliação com dados versionados; escolho consistência, latência e recuperação conforme risco da ação.

Termo para lembrar: Policy architecture | Base técnica: referência 1.

21. O que acontece se o motor de política cair?

Defino comportamento por operação, normalmente recusando ações sensíveis sem decisão válida. Cache de decisão pode ajudar dentro de limites explícitos; não falho aberto indiscriminadamente para manter uma métrica de disponibilidade verde.

Termo para lembrar: Policy failure mode | Base técnica: referência 1.

22. Como versiona políticas?

Trato como código ou configuração governada, com testes, revisão e rollout. Mudança de política pode expor dados sem alterar aplicação; registro versão usada na decisão quando necessário para auditoria e investigação.

Termo para lembrar: Policy versioning | Base técnica: referência 1.

23. Como testa multi-tenancy?

Crio identidades de tenants distintos e tento acessar recursos por IDs, busca, exportação, cache e eventos. O isolamento precisa valer em todos os caminhos, inclusive administração e tarefas assíncronas, não somente numa rota principal.

Termo para lembrar: Cross-tenant testing | Base técnica: referência 2.

24. Como evita role explosion?

Uso papéis para responsabilidades estáveis e atributos ou relações para contexto. Criar role por cliente, país, produto e valor pode ficar inadministrável; modelo permissões com regras compreensíveis e ownership claro.

Termo para lembrar: Role explosion | Base técnica: referência 1.

25. Que dados registra numa decisão?

Ator, ação, recurso, resultado, motivo ou política e contexto mínimo necessário. Não preciso copiar todo conteúdo sensível; auditoria deve explicar acesso e permitir investigação sem criar um segundo repositório desprotegido de dados.

Termo para lembrar: Decision audit | Base técnica: referência 1.

26. Permissão de listar implica ler todos os detalhes?

Não. Defino granularidade de ações e propriedades. Uma busca pode retornar identificação mínima enquanto detalhes exigem acesso adicional; também evito vazar existência ou contagens quando isso viola a política do domínio.

Termo para lembrar: Action granularity | Base técnica: referência 2.

27. Como trata mudança de responsável durante atendimento?

Uso versão da atribuição e revalido em ações relevantes. Uma conexão antiga ou tela aberta não mantém autoridade; mensagens e aprovações do antigo responsável podem ser recusadas após a transferência do caso.

Termo para lembrar: Assignment version | Base técnica: referência 1.

28. Pode usar IA para decidir autorização final?

Não como substituto de política determinística para ações críticas. Um modelo pode auxiliar classificação, mas execução passa por regras verificáveis e contexto confiável; texto gerado não se torna permissão para acessar ou movimentar dinheiro.

Termo para lembrar: Deterministic enforcement | Base técnica: referência 1.

29. Como migra permissões sem liberar demais?

Comparo decisões antigas e novas em modo de observação controlado, reviso divergências e amplio gradualmente. Não uno permissões indiscriminadamente para evitar bloqueio, porque isso pode ampliar acesso de forma invisível.

Termo para lembrar: Policy migration | Base técnica: referência 1.

30. Como defenderia seu modelo?

Eu escolheria uma ação sensível e demonstraria sujeito, recurso, contexto e ponto de decisão. Depois revogaria acesso ou trocaria tenant para mostrar que cache, sessão e processamento assíncrono não contornam a política.

Termo para lembrar: Authorization proof | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. RBAC

Ganho. Fácil compreender

Custo e motivo. Pouco contexto

Como decidir. Para funções estáveis

02. ABAC

Ganho. Expressa contexto

Custo e motivo. Dados e regras complexos

Como decidir. Para decisões condicionais

03. ReBAC

Ganho. Modela relações

Custo e motivo. Consulta e consistência

Como decidir. Para colaboração e hierarquias

04. Decisão central

Ganho. Governança uniforme

Custo e motivo. Dependência e latência

Como decidir. Para política compartilhada

05. Avaliação local

Ganho. Baixa latência

Custo e motivo. Distribuição de política

Como decidir. Com versão e atualização confiáveis

06. Cache de decisão

Ganho. Reduz custo

Custo e motivo. Revogação atrasada

Como decidir. Dentro de janela aceita

07. Check atual

Ganho. Revogação forte

Custo e motivo. Mais consulta

Como decidir. Para ação crítica

08. RLS

Ganho. Defesa no banco

Custo e motivo. Configuração e bypass

Como decidir. Como camada adicional

09. Auditoria detalhada

Ganho. Investigação melhor

Custo e motivo. Retenção e privacidade

Como decidir. Para acesso sensível

10. Fail-closed

Ganho. Preserva proteção

Custo e motivo. Recusa legítimos

Como decidir. Sem decisão confiável

Duas alternativas ao caminho proposto

1. Política simples no domínio

Quando faz sentido. Poucos papéis e relações

Como implementar. Checks explícitos testados em cada comando e consulta

Limites. Pode exigir refatoração ao crescer

2. Motor de políticas dedicado

Quando faz sentido. Muitos serviços e regras compartilhadas

Como implementar. PDP com enforcement local e contexto confiável

Limites. Disponibilidade, versão e cache precisam de desenho

Referências e alcance da evidência

1. OWASP - Authorization

Menor privilégio, deny by default e autorização por requisição.

https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html

2. OWASP - API Security Top 10

Riscos de autorização, autenticação, consumo de recursos e integrações.

https://api-security.owasp.org/editions/2023/en/0x11-t10/

Continue em: T071 · T073 · T074 · T098

T073 / H. Segurança e plataforma

Criptografia, KMS, HSM e segredos

Criptografia protege dados sob chaves; KMS, HSM e gestão de segredos definem custódia, uso, rotação e recuperação dessas chaves.

Quando usar. Dados bancários, credenciais de integração, assinaturas e operações criptográficas sensíveis.

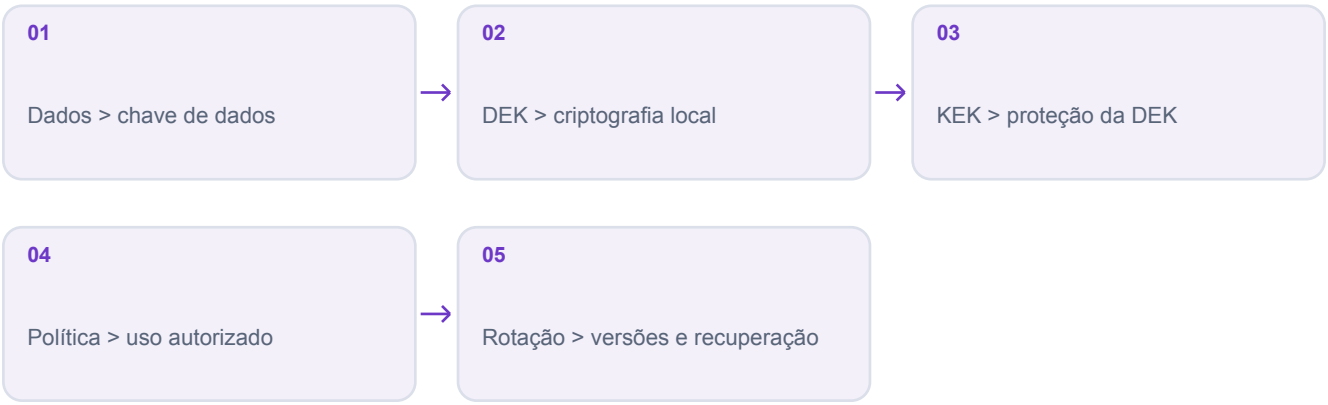
Quando evitar ou limitar. Não criar algoritmo próprio nem confundir envelope encryption de dados com exportação de chaves restritas de PIN.

Uma abertura possível

Vou separar dados, chaves de dados e chaves de proteção, mostrando quem pode usar cada uma e como recuperar ou revogar acesso.

Peças e tecnologias. AWS KMS · Azure Key Vault · HSM · Vault · AEAD · Envelope encryption

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Classificar dados e operações criptográficas

2. Escolher bibliotecas e serviços suportados

3. Definir hierarquia e políticas de acesso

4. Implementar rotação e recarga sem vazamento

5. Testar revogação, indisponibilidade e recuperação de chaves

30 perguntas para sustentar o desenho

01. Criptografia em repouso impede administrador autorizado de ler?

Não necessariamente. Se o administrador ou serviço possui permissão de descriptografar, a criptografia pode ser transparente. Separo proteção de mídia e política de acesso; preciso definir quem pode usar a chave e em qual contexto.

Termo para lembrar: Encryption boundary | Base técnica: referência 1.

02. O que é envelope encryption?

Criptografo dados com uma chave de dados e protejo essa chave com outra autoridade, como KMS. Isso reduz necessidade de enviar grandes payloads ao serviço de chaves, mantendo gestão central da proteção.

Termo para lembrar: Envelope encryption | Base técnica: referência 2.

03. DEK e KEK são iguais?

Não. A DEK protege conteúdo; a KEK protege a DEK conforme a hierarquia. Separar funções facilita rotação e controle, mas exige armazenar metadados suficientes para recuperar a versão correta sem expor material secreto.

Termo para lembrar: Key hierarchy | Base técnica: referência 2.

04. Como guarda a DEK?

Guardo sua forma protegida junto de metadados necessários, não a chave em claro no banco comum. A versão em memória existe apenas conforme necessidade e política, com exposição e ciclo de vida controlados.

Termo para lembrar: Wrapped data key | Base técnica: referência 2.

05. Pode cachear qualquer chave para ganhar performance?

Não. Cache de DEKs pode ser permitido em certos fluxos com limites de tempo e uso, mas chaves restritas de operações como PIN podem exigir permanência no HSM. Avalio contrato e classificação específica.

Termo para lembrar: Key usage boundary | Base técnica: referência 2.

06. KMS e HSM são intercambiáveis?

Não. KMS oferece gestão e APIs de chaves com um modelo de serviço; HSM é uma fronteira criptográfica com capacidades e responsabilidades próprias. Escolho conforme operações, custódia e requisitos, não apenas pelo nome do produto.

Termo para lembrar: KMS versus HSM | Base técnica: referência 1.

07. Como reduz latência sem enfraquecer custódia?

Uso pooling e bibliotecas adequadas, dimensiono quotas e avalio envelope encryption para dados compatíveis. Não exporto chave que deve permanecer no HSM nem substituo operação exigida por uma implementação local improvisada.

Termo para lembrar: Cryptographic throughput | Base técnica: referência 2.

08. Qual papel de AEAD?

Oferece confidencialidade e verificação de integridade autenticada conforme o algoritmo. Uso biblioteca confiável e parâmetros corretos; criptografar sem autenticar pode permitir alterações não detectadas no conteúdo protegido.

Termo para lembrar: Authenticated encryption | Base técnica: referência 2.

09. Nonce pode repetir livremente?

Não. Muitos esquemas exigem propriedades específicas, como unicidade por chave. Sigo a biblioteca e o algoritmo, com geração apropriada; reutilização indevida pode comprometer segurança mesmo quando a chave permanece secreta.

Termo para lembrar: Nonce management | Base técnica: referência 2.

10. AAD é dado secreto?

Não necessariamente. AAD vincula contexto à autenticação do ciphertext, mas não é criptografado por esse mecanismo. Posso ligar tenant ou propósito conforme o contrato, evitando incluir informação sensível achando que ficará oculta.

Termo para lembrar: Additional authenticated data | Base técnica: referência 2.

11. Hash e criptografia resolvem o mesmo problema?

Não. Hash não é uma forma reversível de guardar segredo; criptografia permite recuperar com chave. Para senhas, uso algoritmo de derivação apropriado, não criptografia reversível nem hash rápido genérico escolhido por conveniência.

Termo para lembrar: Cryptographic purpose | Base técnica: referência 3.

12. Como rotaciona uma chave mestra?

Uso mecanismos suportados e metadados de versão, distinguindo novas criptografias de dados antigos. Rotação não significa automaticamente recriptografar todo conteúdo; preciso saber quais versões continuam necessárias para leitura e recuperação.

Termo para lembrar: Key rotation | Base técnica: referência 1.

13. Como rotaciona senha de banco sem outage?

Crio transição com credenciais e pools compatíveis, atualizo consumidores e retiro a antiga após validação. A aplicação precisa recarregar segredo ou reconstruir conexão; trocar apenas no cofre não atualiza processos já em execução.

Termo para lembrar: Secret rotation | Base técnica: referência 3.

14. Variável de ambiente é cofre?

Não. Ela pode expor segredo a dumps, ferramentas ou processos conforme ambiente. Uso entrega controlada e menor privilégio; variável pode ser mecanismo de injeção, mas não resolve custódia, rotação e auditoria sozinha.

Termo para lembrar: Secret delivery | Base técnica: referência 3.

15. Pode colocar segredo na imagem Docker?

Evito. Imagens e camadas se espalham por registries e máquinas, e remover numa camada posterior não garante apagar da anterior. Segredos entram em runtime por mecanismo apropriado, sem ficar no artefato versionado.

Termo para lembrar: Build-time secret leakage | Base técnica: referência 3.

16. Como autentica no cofre sem outro segredo eterno?

Prefiro identidade de workload e credenciais temporárias fornecidas pela plataforma quando disponíveis. Isso reduz o problema de bootstrap, mas permissões e trust policy continuam precisando de escopo e validação adequados.

Termo para lembrar: Workload identity | Base técnica: referência 3.

17. O que acontece se KMS ficar fora?

Operações que precisam da chave podem falhar ou usar cache autorizado dentro de limites. Defino degradação e capacidade; não armazeno chaves em claro indefinidamente apenas para esconder uma dependência de disponibilidade.

Termo para lembrar: KMS availability | Base técnica: referência 1.

18. Como controla quota do HSM?

Meço operações por segundo, latência e filas, com limites de concorrência e pooling. Aumento de workers não cria capacidade criptográfica; protejo o equipamento e priorizo funções críticas sem executar fallback inseguro.

Termo para lembrar: HSM capacity | Base técnica: referência 2.

19. Como protege segredos em logs?

Uso allowlist e redação antes de exportar, evitando tokens, senhas e chaves completas. Também testo erros e dumps; bibliotecas podem incluir parâmetros em exceções, então proteção precisa cobrir caminhos de falha.

Termo para lembrar: Secret redaction | Base técnica: referência 3.

20. Quem pode administrar e usar uma chave?

Separo papéis quando necessário: administrar ciclo de vida não precisa implicar descriptografar dados. Políticas de uso, grants e auditoria devem refletir responsabilidades, evitando um único privilégio amplo para todas as operações.

Termo para lembrar: Separation of duties | Base técnica: referência 1.

21. Como recupera backup criptografado?

Garanto que chaves, versões e permissões necessárias sobrevivam ao cenário de desastre. Um backup íntegro sem chave recuperável é inutilizável; testo restauração completa, incluindo dependências de identidade e serviço criptográfico.

Termo para lembrar: Key recovery | Base técnica: referência 1.

22. Destruir chave apaga todos os dados pessoais?

Não automaticamente. Pode tornar certos ciphertexts inacessíveis, mas existem caches, cópias em claro, índices, backups e outras chaves. Avalio inventário e obrigações aplicáveis; crypto-shredding é mecanismo técnico com condições, não prova universal de exclusão.

Termo para lembrar: Crypto-shredding limits | Base técnica: referência 3.

23. Como responde a chave comprometida?

Contenho uso, identifico escopo, rotaciono material e avalio dados expostos, seguindo procedimento. Rotacionar não torna confidenciais novamente dados que o atacante já leu; investigação e recuperação precisam considerar esse limite.

Termo para lembrar: Key compromise | Base técnica: referência 3.

24. Assinatura digital criptografa o documento?

Não necessariamente. Ela permite verificar autenticidade e integridade conforme chave e algoritmo, mas não esconde conteúdo. Se preciso de sigilo e assinatura, desenho ambos explicitamente com gestão de chaves adequada.

Termo para lembrar: Digital signature | Base técnica: referência 2.

25. Como evita usar chave para propósito errado?

Defino algoritmos, operações permitidas e contexto por chave, além de políticas. Uma chave destinada a assinatura não deve virar credencial genérica; separação reduz impacto e torna auditoria e rotação mais compreensíveis.

Termo para lembrar: Key purpose separation | Base técnica: referência 1.

26. Como audita uso de chaves?

Registro operações, identidade, contexto permitido e resultados sem revelar material secreto. Procuro padrões anômalos e acessos administrativos; auditabilidade depende de retenção e proteção dos próprios registros, não somente da API criptográfica.

Termo para lembrar: Key audit | Base técnica: referência 1.

27. Pode compartilhar segredo entre todos os serviços?

Evito quando aumenta blast radius. Credenciais por função permitem revogar e investigar com precisão; compartilhamento reduz configuração inicial, mas dificulta atribuição e pode dar a um serviço acesso desnecessário a outros domínios.

Termo para lembrar: Credential scope | Base técnica: referência 3.

28. Como testa rotação de verdade?

Rotaciono em ambiente representativo enquanto há tráfego, verifico novas e antigas conexões e simulo falha de recarga. O teste confirma continuidade e revogação; apenas criar uma nova versão no cofre prova pouco.

Termo para lembrar: Rotation drill | Base técnica: referência 3.

29. Qual risco de inventar criptografia própria?

Erros sutis de composição, nonce, validação e geração de chaves podem destruir garantias. Uso padrões e bibliotecas revisadas e foco em integração correta; conhecimento arquitetural não substitui revisão criptográfica especializada.

Termo para lembrar: Vetted cryptography | Base técnica: referência 2.

30. Como defenderia o desenho?

Eu seguiria o dado e sua chave, mostrando custódia, permissões, rotação e recuperação. Depois removeria acesso ou derrubaria o KMS para explicar o comportamento real sem recorrer a chaves expostas ou fallback inseguro.

Termo para lembrar: Key lifecycle proof | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. KMS gerenciado

Ganho. Menos operação de chaves

Custo e motivo. Dependência e quotas

Como decidir. Quando APIs atendem ao requisito

02. HSM dedicado

Ganho. Custódia e operações específicas

Custo e motivo. Custo e capacidade

Como decidir. Para exigências próprias do domínio

03. Envelope encryption

Ganho. Menos payload no KMS

Custo e motivo. Gestão de DEKs

Como decidir. Para dados grandes

04. Cache de DEK permitido

Ganho. Menor latência

Custo e motivo. Janela de exposição

Como decidir. Com limite de tempo e uso

05. Chave por tenant

Ganho. Isolamento

Custo e motivo. Mais objetos e operação

Como decidir. Quando segregação justifica

06. Chave compartilhada

Ganho. Simplicidade

Custo e motivo. Blast radius maior

Como decidir. Para risco compatível

07. Credencial temporária

Ganho. Menor vida de segredo

Custo e motivo. Renovação crítica

Como decidir. Com identidade de workload

08. Rotação frequente

Ganho. Reduz exposição

Custo e motivo. Mais transições

Como decidir. Com recarga testada

09. Papéis separados

Ganho. Menor privilégio

Custo e motivo. Mais coordenação

Como decidir. Para chaves críticas

10. Crypto-shredding

Ganho. Inacessibilidade criptográfica

Custo e motivo. Depende de todas as cópias

Como decidir. Como parte de política validada

Duas alternativas ao caminho proposto

1. Criptografia gerenciada pelo storage

Quando faz sentido. Proteção de mídia é suficiente

Como implementar. Usar serviço com chaves e políticas adequadas

Limites. Aplicação autorizada ainda pode ler dados

2. Criptografia no cliente com custódia própria

Quando faz sentido. Precisa separar provedor e acesso ao conteúdo

Como implementar. Biblioteca confiável e ciclo de vida de chaves explícito

Limites. Recuperação e busca ficam mais complexas

Referências e alcance da evidência

1. AWS - KMS keys

Hierarquia, identificação e controle de chaves KMS.

https://docs.aws.amazon.com/kms/latest/developerguide/concepts.html

2. AWS - KMS cryptography essentials

Operações criptográficas e envelope encryption.

https://docs.aws.amazon.com/kms/latest/developerguide/kms-cryptography.html

3. OWASP - Secrets Management

Ciclo de vida, acesso, rotação e exposição de segredos.

https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html

Continue em: T051 · T067 · T071 · T075

T074 / H. Segurança e plataforma

Threat modeling e OWASP API Security

Threat modeling identifica ativos, fronteiras de confiança, formas de abuso e controles verificáveis no contexto do sistema.

Quando usar. Desenhos com dados sensíveis, integrações, privilégios ou efeitos financeiros.

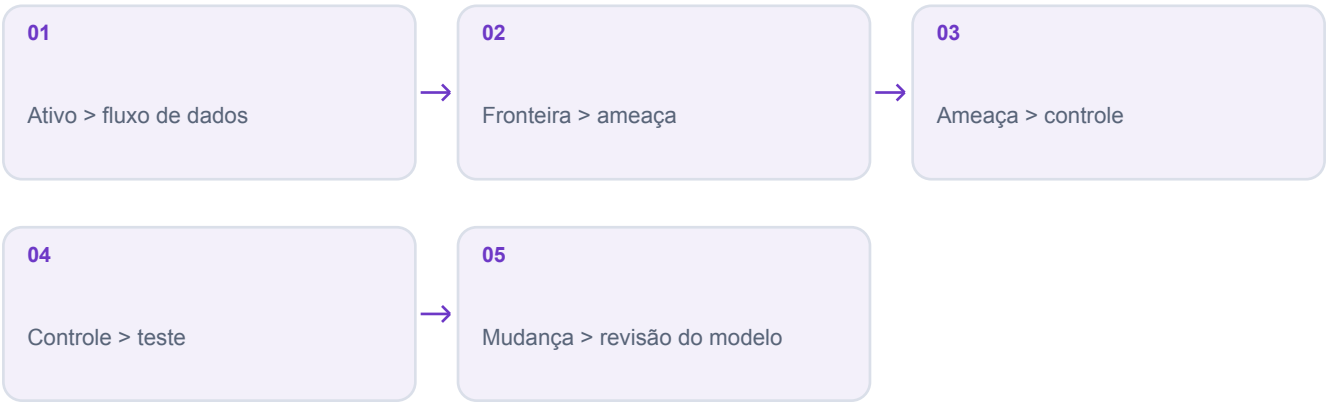
Quando evitar ou limitar. Não usar uma lista de vulnerabilidades como substituto de analisar fluxo real e regras de negócio.

Uma abertura possível

Vou seguir dados e dinheiro por cada fronteira e perguntar como uma identidade ou entrada maliciosa pode violar as invariantes.

Peças e tecnologias. OWASP API Security · STRIDE · Threat model · WAF · SAST · DAST

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Identificar ativos e atores

2. Desenhar fluxos e trust boundaries

3. Priorizar abuso por impacto e plausibilidade

4. Implementar controles no ponto correto

5. Testar e atualizar após mudanças relevantes

30 perguntas para sustentar o desenho

01. Qual é o objetivo do threat model?

Entender o que protegemos, de quem e por quais caminhos o sistema pode ser abusado. Transformo essas hipóteses em controles e testes; o modelo precisa orientar decisões, não ser apenas um desenho arquivado.

Termo para lembrar: Threat modeling | Base técnica: referência 1.

02. Por onde começa numa fintech?

Identifico dinheiro, dados pessoais, credenciais, decisões e continuidade operacional, junto dos atores autorizados. Depois sigo fluxos entre app, serviços e parceiros, procurando onde confiança muda ou um efeito irreversível pode ser solicitado.

Termo para lembrar: Assets and boundaries | Base técnica: referência 1.

03. O que é trust boundary?

É uma fronteira onde mudam identidade, controle ou nível de confiança, como navegador para API ou serviço para parceiro. Valido entradas e autoridade nesse ponto, sem assumir que dados anteriores continuam confiáveis automaticamente.

Termo para lembrar: Trust boundary | Base técnica: referência 1.

04. BOLA significa o quê na prática?

Uma identidade consegue acessar ou alterar objeto de outra pessoa por falha de autorização. Testo trocar IDs em leitura, exportação e comando; identificador aleatório reduz descoberta, mas não corrige a falta de permissão.

Termo para lembrar: Broken object authorization | Base técnica: referência 2.

05. Como mass assignment pode afetar chargeback?

Binding automático pode aceitar campos como approved, owner ou amount autorizado que o cliente não deveria controlar. Defino DTO e campos permitidos por ação, verificando propriedade e estado no domínio antes de persistir.

Termo para lembrar: Property-level authorization | Base técnica: referência 2.

06. WAF elimina injeção SQL?

Não como garantia. Uso consultas parametrizadas e controles adequados no código; WAF pode reduzir certos ataques, mas não entende toda semântica e pode ser contornado ou gerar falsos positivos.

Termo para lembrar: Defense in depth | Base técnica: referência 2.

07. Como identifica SSRF num sistema de anexos?

Uma URL externa pode fazer o backend acessar rede interna ou metadados. Restrinjo destinos e protocolos, valido redirecionamentos e aplico egress; apenas verificar a string inicial pode deixar caminhos de abuso.

Termo para lembrar: SSRF | Base técnica: referência 2.

08. Autenticação forte impede abuso de fluxo?

Não. Um usuário legítimo pode automatizar criação de casos, reservas ou notificações para causar custo ou fraude. Modelo limites e invariantes do fluxo, não somente proteção contra credenciais inválidas.

Termo para lembrar: Business flow abuse | Base técnica: referência 2.

09. Como trata consumo irrestrito de recursos?

Defino limites de payload, paginação, custo de consulta e concorrência por contexto. Uma requisição válida pode ser cara; segurança inclui preservar capacidade e custo, especialmente em APIs que disparam processamento ou chamadas pagas.

Termo para lembrar: Resource consumption | Base técnica: referência 2.

10. Qual risco de API antiga esquecida?

Ela pode manter autenticação ou validação mais fracas e contornar controles novos. Inventario endpoints e versões, observo uso e retiro com política; proteger apenas a rota principal deixa uma superfície paralela aberta.

Termo para lembrar: API inventory | Base técnica: referência 2.

11. Como integra parceiro sem confiar cegamente?

Valido schema, limites, assinatura quando disponível e semântica do resultado. Um parceiro pode falhar ou enviar conteúdo inesperado; dado externo não ganha autoridade irrestrita apenas porque veio de uma integração contratada.

Termo para lembrar: Unsafe API consumption | Base técnica: referência 2.

12. O que é confused deputy?

Um componente com privilégio maior é induzido a agir em nome de quem não possui permissão. Preservo cadeia de identidade e valido recurso e ação, evitando que acesso interno amplio substitua autorização do solicitante.

Termo para lembrar: Confused deputy | Base técnica: referência 1.

13. Como protege webhook contra replay?

Valido assinatura e janela conforme contrato, além de deduplicar identidade do evento no efeito local. Assinatura prova origem e integridade sob condições, mas uma mensagem válida repetida ainda pode duplicar processamento.

Termo para lembrar: Replay protection | Base técnica: referência 2.

14. Como diferencia ameaça e vulnerabilidade?

Ameaça descreve uma forma de dano possível; vulnerabilidade é uma fraqueza explorável no sistema. Relaciono ambas ao ativo e ao controle, evitando discutir riscos abstratos sem caminho concreto de exploração ou mitigação.

Termo para lembrar: Threat versus vulnerability | Base técnica: referência 1.

15. Como prioriza riscos?

Considero impacto, plausibilidade, exposição e controles existentes, com responsáveis do negócio quando necessário. Não conto apenas quantidade de achados; uma falha que duplica dinheiro pode superar dezenas de melhorias cosméticas de segurança.

Termo para lembrar: Risk prioritization | Base técnica: referência 1.

16. STRIDE cobre todo risco financeiro?

Não. Ajuda organizar categorias, mas abuso de lógica e invariantes específicas exigem análise do domínio. Complemento a estrutura com cenários como aprovação indevida, replay de crédito e acesso cruzado entre clientes.

Termo para lembrar: STRIDE limitations | Base técnica: referência 1.

17. Como modela insider risk?

Considero permissões legítimas excessivas, acessos administrativos e capacidade de alterar evidências. Uso menor privilégio, segregação e auditoria, sem presumir má intenção de pessoas específicas; o desenho limita dano por erro ou comprometimento.

Termo para lembrar: Privileged access | Base técnica: referência 1.

18. Como protege logs contra conteúdo malicioso?

Uso estrutura e encoding adequados, limites e tratamento de campos externos. Entradas não confiáveis podem confundir análise ou injetar linhas; logs precisam preservar significado sem executar ou interpretar conteúdo recebido.

Termo para lembrar: Log injection | Base técnica: referência 2.

19. Como evita segredo em resposta de erro?

Retorno mensagens úteis sem stack traces, tokens ou detalhes internos desnecessários. Registro diagnóstico seguro em ambiente autorizado; erro de validação não deve revelar estrutura de banco ou credenciais ao cliente.

Termo para lembrar: Error disclosure | Base técnica: referência 2.

20. Teste de segurança substitui threat model?

Não. Testes procuram falhas em cenários definidos; o modelo ajuda escolher o que testar e projetar. Combino revisão, testes automatizados e exercícios relevantes, reconhecendo que ausência de achado não prova ausência de risco.

Termo para lembrar: Security assurance | Base técnica: referência 1.

21. SAST e DAST respondem a perguntas iguais?

Não. SAST analisa artefatos e código; DAST observa aplicação executando. Ambos têm limites e falsos resultados, especialmente em autorização e lógica; não considero pipeline verde como prova completa de segurança financeira.

Termo para lembrar: Security testing layers | Base técnica: referência 2.

22. Como protege upload de arquivo?

Limito tamanho e tipo real, uso quarentena e processamento isolado e controlo download. Um arquivo pode atacar parser ou consumidor posterior; aceitar Content-Type declarado pelo cliente não valida segurança do conteúdo.

Termo para lembrar: Untrusted file | Base técnica: referência 1.

23. Como trata dependência vulnerável?

Inventario componentes, avalio exposição e aplico correção ou mitigação com teste e rollout. Versão vulnerável não é automaticamente explorável em qualquer contexto, mas ignorar sem análise deixa risco sem responsável e sem prazo.

Termo para lembrar: Dependency risk | Base técnica: referência 1.

24. O que muda com multi-tenancy?

Cada caminho precisa preservar isolamento: consulta, cache, fila, arquivo e observabilidade. Tenant_id não confiável ou chave compartilhada errada pode vazar dados mesmo quando a autenticação funciona e a infraestrutura está criptografada.

Termo para lembrar: Tenant isolation | Base técnica: referência 2.

25. Como modela IA dentro do atendimento?

Trato entradas e saídas como não confiáveis para autorização. Ferramentas têm escopo e validação determinística; prompt injection pode tentar induzir acesso ou ação, então o modelo não decide sozinho permissões financeiras.

Termo para lembrar: Tool authorization | Base técnica: referência 1.

26. Como lida com risco residual?

Documento o que permanece, impacto, dono e condição de revisão. Aceitar risco é decisão informada e autorizada, não esconder limitação; mudanças de volume ou integração podem exigir reavaliar uma aceitação anterior.

Termo para lembrar: Residual risk | Base técnica: referência 1.

27. Quando atualiza o modelo?

Após mudança relevante de fluxo, privilégio, integração, exposição ou incidente. Um modelo antigo pode não representar novas APIs e caminhos; incorporo revisão ao processo de arquitetura sem exigir cerimônia pesada para toda edição trivial.

Termo para lembrar: Threat model lifecycle | Base técnica: referência 1.

28. Como testa uma fronteira de autorização?

Uso identidade válida com recurso proibido, papel reduzido e estado revogado, incluindo chamadas diretas. Esse teste é diferente de enviar token inválido; prova que autenticação não foi confundida com autorização.

Termo para lembrar: Negative authorization test | Base técnica: referência 2.

29. Qual evidência apresenta ao entrevistador?

Um cenário de abuso concreto, o ponto de controle e o teste que comprova a defesa. Por exemplo, evento repetido não cria segundo crédito e operador sem atribuição não aprova caso, mesmo chamando API diretamente.

Termo para lembrar: Control validation | Base técnica: referência 1.

30. Como fecharia a discussão?

Eu relacionaria riscos às invariantes e mostraria controles em camadas, com limites conhecidos. Segurança faz parte do fluxo normal e da recuperação; não aparece apenas como uma caixa WAF adicionada no fim do desenho.

Termo para lembrar: Security by design | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Modelo detalhado

Ganho. Cobertura profunda

Custo e motivo. Tempo de análise

Como decidir. Para fluxo crítico

02. Modelo leve

Ganho. Baixa fricção

Custo e motivo. Pode omitir dependência

Como decidir. Para mudança pequena com revisão

03. Controle no domínio

Ganho. Semântica correta

Custo e motivo. Implementação específica

Como decidir. Para autorização e dinheiro

04. WAF

Ganho. Mitigação na borda

Custo e motivo. Falsos positivos

Como decidir. Como camada adicional

05. Menor privilégio

Ganho. Limita dano

Custo e motivo. Gestão de acesso

Como decidir. Para todos os atores relevantes

06. Isolamento de parser

Ganho. Contém arquivo malicioso

Custo e motivo. Custo operacional

Como decidir. Para conteúdo externo

07. Rate limit

Ganho. Reduz abuso

Custo e motivo. Pode recusar legítimos

Como decidir. Com chaves e quotas justas

08. Auditoria rica

Ganho. Investigação

Custo e motivo. Dados e retenção

Como decidir. Com minimização

09. Automação de scans

Ganho. Escala verificações

Custo e motivo. Falsos negativos

Como decidir. Complementando revisão de domínio

10. Aceitação explícita

Ganho. Decisão transparente

Custo e motivo. Risco permanece

Como decidir. Com dono e prazo de revisão

Duas alternativas ao caminho proposto

1. Checklist de baseline

Quando faz sentido. Sistema simples ou estágio inicial

Como implementar. Aplicar controles conhecidos e testes mínimos

Limites. Não substitui análise de lógica e fronteiras específicas

2. Revisão especializada de segurança

Quando faz sentido. Risco alto ou novidade relevante

Como implementar. Equipe especializada valida ameaças e implementação

Limites. Precisa contexto e ownership contínuo do produto

Referências e alcance da evidência

1. OWASP - Threat Modeling

Modelagem de ativos, ameaças, fronteiras e mitigação.

https://owasp.org/www-community/Threat_Modeling

2. OWASP - API Security Top 10

Riscos de autorização, autenticação, consumo de recursos e integrações.

https://api-security.owasp.org/editions/2023/en/0x11-t10/

Continue em: T053 · T071 · T072 · T073

T075 / H. Segurança e plataforma

Governança, retenção, exclusão e auditoria

Governança define quem pode usar dados, por qual finalidade, por quanto tempo e com qual evidência.

Quando usar. Dados pessoais, auditoria financeira e múltiplas cópias operacionais ou analíticas.

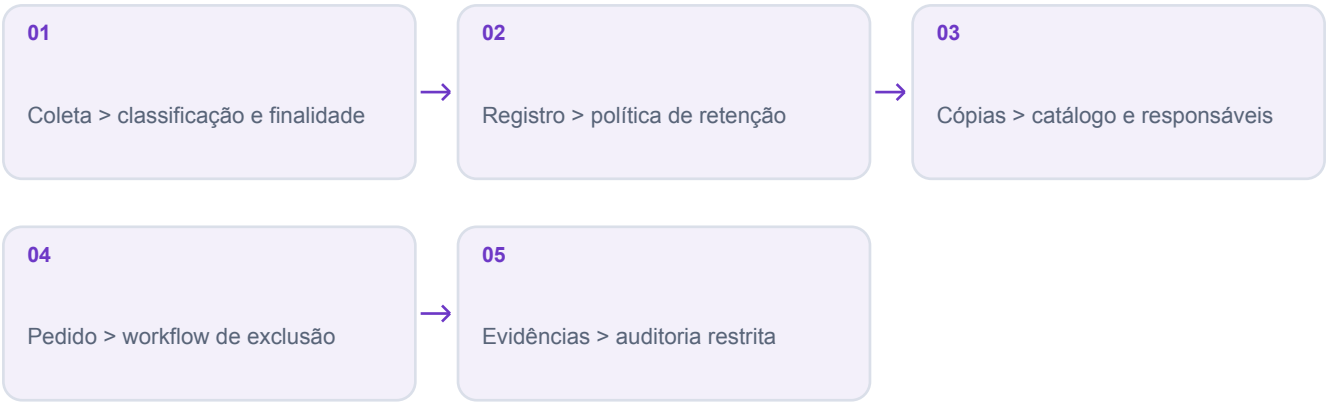
Quando evitar ou limitar. Guardar tudo indefinidamente ou prometer exclusão completa apenas apagando a tabela principal.

Uma abertura possível

Eu desenho o ciclo de vida junto com o fluxo: coleta mínima, acesso, retenção, descarte e prova operacional.

Peças e tecnologias. S3 Object Lock · KMS · Catálogo de dados · PostgreSQL · OpenTelemetry

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Inventariar fontes, derivados, backups e donos

2. Definir política por classe com responsáveis jurídicos e negócio

3. Aplicar acesso mínimo e retenção automatizada

4. Orquestrar exclusão verificável e exceções autorizadas

5. Testar restauração sem ressuscitar exclusões

30 perguntas para sustentar o desenho

01. Como começaria a governança?

Eu inventariaria dados e suas cópias, incluindo logs, índices e exportações. Data lineage permite localizar onde uma informação circula; sem isso, qualquer promessa de exclusão ou retenção fica incompleta.

Termo para lembrar: Data lineage | Base técnica: referência 2.

02. Quem define prazo de retenção?

Eu implemento uma retention policy aprovada pelos responsáveis de negócio, privacidade e jurídico. Não invento um prazo universal: classes diferentes podem ter finalidades, obrigações e exceções distintas, inclusive preservação autorizada.

Termo para lembrar: Retention policy | Base técnica: referência 1.

03. Auditoria exige payload completo?

Não. Eu registro ator, ação, recurso, decisão, horário e correlação, evitando segredos ou conteúdo desnecessário. Data minimization reduz exposição sem perder a capacidade de explicar uma alteração relevante.

Termo para lembrar: Data minimization | Base técnica: referência 2.

04. Como distingue log e auditoria?

Eu separo observabilidade operacional de audit trail, pois finalidade, acesso e retenção diferem. Uma amostra de traces pode desaparecer; a evidência exigida pelo domínio precisa de cobertura e proteção deliberadas.

Termo para lembrar: Audit trail | Base técnica: referência 2.

05. WORM resolve toda governança?

Não. WORM restringe alteração ou remoção conforme configuração, mas não assegura qualidade do dado, autorização correta nem finalidade legítima. Ainda preciso classificar dados e controlar quem grava e quem lê.

Termo para lembrar: WORM | Base técnica: referência 1.

06. Qual cuidado com Object Lock?

Eu verifico retenção por versão, modos de proteção e permissões antes de ativar. Uma retention lock inadequada pode impedir descarte esperado; testes devem usar políticas representativas sem comprometer dados reais.

Termo para lembrar: Retention lock | Base técnica: referência 1.

07. Como protege trilha de auditoria?

Eu separo credenciais de escrita e leitura, restrinjo alterações e monitoro lacunas. Integridade criptográfica pode detectar adulteração, mas não prova que um evento omitido foi registrado corretamente desde a origem.

Termo para lembrar: Tamper evidence | Base técnica: referência 2.

08. Como evita vazamento em logs?

Eu uso allowlist de campos, redaction e testes automáticos com segredos sintéticos. Também restrinjo consultas: mascarar a tela não protege um backend de logs que ainda guarda o valor original.

Termo para lembrar: Redaction | Base técnica: referência 2.

09. Como apaga dado derivado?

Eu propago uma deletion request identificada para projeções, caches e índices, registrando conclusão por destino. A remoção da fonte não garante desaparecimento imediato em todas as cópias materializadas existentes.

Termo para lembrar: Deletion workflow | Base técnica: referência 2.

10. E se uma cópia estiver offline?

Eu mantenho a obrigação de exclusão durável e retento quando o destino voltar. O workflow diferencia pendência de conclusão; não declaro sucesso global enquanto destinos relevantes permanecem sem evidência verificável.

Termo para lembrar: Durable obligation | Base técnica: referência 2.

11. Como impedir ressurreição após restore?

Eu mantenho um deletion ledger protegido, separado do snapshot restaurado, e reaplico exclusões antes de liberar acesso. A restauração precisa validar dados removidos, além de integridade e disponibilidade do banco.

Termo para lembrar: Deletion ledger | Base técnica: referência 2.

12. Pode apagar backup individualmente?

Depende do formato e da política. Eu documento a janela de expiração dos backups e controles de restauração, sem prometer remoção seletiva que o mecanismo de armazenamento não suporta tecnicamente.

Termo para lembrar: Backup lifecycle | Base técnica: referência 1.

13. Crypto-shredding basta?

Eu só considero crypto-shredding efetivo quando todas as cópias relevantes dependem exclusivamente das chaves destruídas. Texto claro em logs, chaves duplicadas ou exportações alternativas invalidam essa premissa; conformidade exige avaliação própria.

Termo para lembrar: Crypto-shredding | Base técnica: referência 1.

14. Como mantém ledger e privacidade?

Eu separo identificadores pessoais de fatos financeiros e minimizo atributos imutáveis. Pseudonymization ajuda, mas uma referência ainda reidentificável não vira automaticamente dado anônimo nem elimina todas as obrigações aplicáveis.

Termo para lembrar: Pseudonymization | Base técnica: referência 2.

15. O que fazer com legal hold?

Eu modelo uma exceção autorizada, com escopo, responsável e revisão, que suspende descarte pertinente. Legal hold não deve virar uma justificativa indefinida para reter indiscriminadamente todos os dados do cliente.

Termo para lembrar: Legal hold | Base técnica: referência 1.

16. Como autoriza exportações?

Eu verifico finalidade, escopo, destinatário e tempo de acesso. Export controls incluem rastreabilidade e expiração; permitir SELECT no sistema não significa permitir baixar irrestritamente toda a base para outra ferramenta.

Termo para lembrar: Export controls | Base técnica: referência 2.

17. Como controla acesso de suporte?

Eu aplico least privilege, justificativa e duração limitada, com auditoria de consultas sensíveis. Dados mascarados por padrão e elevação específica reduzem exposição sem impedir investigação autorizada de um caso real.

Termo para lembrar: Least privilege | Base técnica: referência 2.

18. Como identifica informações sensíveis novas?

Eu combino schema registry, revisão de mudanças e scanning de amostras protegidas. Data classification precisa acompanhar novos campos; confiar apenas no nome da coluna pode deixar conteúdo pessoal em campos livres.

Termo para lembrar: Data classification | Base técnica: referência 2.

19. Como implementa TTL sem perder obrigação?

Eu associo expiração à classe correta e executo remoção observável. TTL operacional não decide sozinho uma obrigação de retenção; exceções, dependências e evidência de descarte precisam integrar o mecanismo de execução.

Termo para lembrar: TTL | Base técnica: referência 1.

20. Qual risco em anonimizar hash simples?

Eu não trataria hash determinístico de identificadores previsíveis como anonimização forte. Dictionary attack pode recuperar valores; avalio finalidade, segredo, ligação entre datasets e possibilidade real de reidentificação no contexto.

Termo para lembrar: Dictionary attack | Base técnica: referência 2.

21. Como audita acesso aos próprios logs?

Eu registro consultas e exportações sensíveis em domínio protegido, com separação de funções. Audit access também é uma atividade relevante; administradores do coletor não devem automaticamente possuir acesso irrestrito ao conteúdo.

Termo para lembrar: Separation of duties | Base técnica: referência 2.

22. Como lida com dados de teste?

Eu prefiro dados sintéticos e subconjuntos minimizados, com políticas equivalentes quando informação real for indispensável. Data masking deve considerar correlação e campos livres, não somente substituir CPF na tabela principal.

Termo para lembrar: Data masking | Base técnica: referência 2.

23. Como valida descarte automático?

Eu executo casos sintéticos rastreáveis e verifico fontes, derivados e backups conforme política. Deletion verification mede cobertura e atrasos; uma resposta HTTP de sucesso não comprova ausência nas demais cópias.

Termo para lembrar: Deletion verification | Base técnica: referência 2.

24. Como escolhe timestamp de auditoria?

Eu armazeno horário do evento e da recepção, identidade e sequência quando disponível. Event time ajuda a análise, mas relógios distribuídos não fornecem ordem causal perfeita entre sistemas independentes.

Termo para lembrar: Event time | Base técnica: referência 2.

25. Como lida com falha do coletor?

Eu defino buffering durável e alarmes conforme criticidade. Para ações que exigem evidência, posso recusar ou registrar uma obrigação local atomicamente; descartar silenciosamente auditoria durante falha enfraquece a garantia declarada.

Termo para lembrar: Audit durability | Base técnica: referência 2.

26. Como evita auditoria duplicada virar fraude aparente?

Eu preservo event_id e operation_id, distinguindo tentativa de efeito concluído. Deduplication na análise mantém tentativas relevantes sem contar duas vezes uma mesma operação financeira repetida por transporte ou recuperação.

Termo para lembrar: Deduplication | Base técnica: referência 2.

27. Governança pode bloquear entrega?

Sim, mas automatizo verificações previsíveis e ofereço caminhos claros de exceção. Policy as code transforma critérios estáveis em feedback rápido, sem substituir julgamento humano quando finalidade ou risco continuam ambíguos.

Termo para lembrar: Policy as code | Base técnica: referência 2.

28. Como mede eficiência da política?

Eu acompanho cobertura do inventário, exclusões vencidas, acessos excepcionais e falhas de retenção. Governance metrics precisam refletir riscos concretos; contar documentos publicados não demonstra controle efetivo sobre cópias e usuários.

Termo para lembrar: Governance metrics | Base técnica: referência 2.

29. Qual erro comum em banco de eventos?

É colocar informação pessoal desnecessária num event log de retenção longa. Eu desenho envelopes mínimos e referências controladas, pois remover conteúdo replicado depois pode ser operacionalmente complexo e semanticamente arriscado.

Termo para lembrar: Event log | Base técnica: referência 1.

30. O que falaria na entrevista?

Eu explicaria quais fatos devem permanecer, quais atributos podem ser descartados e como cada cópia participa. Data lifecycle é uma responsabilidade verificável, com limitações explícitas, e não uma caixa genérica chamada compliance.

Termo para lembrar: Data lifecycle | Base técnica: referência 2.

10 trade-offs: ganho, custo e decisão

01. Retenção longa

Ganho. Histórico e investigação

Custo e motivo. Custo e exposição

Como decidir. Quando finalidade e obrigação justificam

02. Retenção curta

Ganho. Menor exposição

Custo e motivo. Menos histórico

Como decidir. Quando recuperação e finalidade permitem

03. WORM

Ganho. Proteção contra remoção

Custo e motivo. Descarte restrito

Como decidir. Para evidência com política validada

04. Pseudonimização

Ganho. Separa identificadores

Custo e motivo. Ainda pode reidentificar

Como decidir. Para minimizar propagação

05. Auditoria síncrona

Ganho. Evidência antes do sucesso

Custo e motivo. Latência e disponibilidade

Como decidir. Para ações que exigem registro imediato

06. Obrigação assíncrona durável

Ganho. Desacoplamento

Custo e motivo. Atraso na centralização

Como decidir. Com persistência local atômica

07. Chave por sujeito

Ganho. Descarte criptográfico seletivo

Custo e motivo. Gestão de muitas chaves

Como decidir. Quando modelo de cópias sustenta a garantia

08. Inventário central

Ganho. Visão uniforme

Custo e motivo. Dependência organizacional

Como decidir. Com donos de domínio responsáveis

09. Exclusão física imediata

Ganho. Menor permanência

Custo e motivo. Custo em storage e réplicas

Como decidir. Quando suportada e exigida

10. Expiração de backups

Ganho. Operação previsível

Custo e motivo. Janela de permanência

Como decidir. Com política e bloqueio de ressurreição

Duas alternativas ao caminho proposto

1. Governança centralizada

Quando faz sentido. Poucos domínios e necessidade de uniformidade

Como implementar. Equipe central mantém catálogo, políticas e execução

Limites. Pode virar gargalo e desconhecer semântica local

2. Governança federada

Quando faz sentido. Muitos domínios autônomos

Como implementar. Donos locais implementam contratos e evidências sob padrões comuns

Limites. Exige auditoria de aderência e ferramentas compartilhadas

Referências e alcance da evidência

1. AWS - S3 Object Lock

Retenção e proteção WORM de versões de objetos.

https://docs.aws.amazon.com/AmazonS3/latest/userguide/object-lock.html

2. OWASP - Logging

Conteúdo, proteção e riscos de logs de segurança.

https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html

Continue em: T067 · T072 · T073 · T074

T076 / H. Segurança e plataforma

Testes de contrato, propriedades, carga e chaos

Testes verificam contratos, invariantes, capacidade e comportamento sob falha com objetivos diferentes.

Quando usar. Mudanças críticas, protocolos distribuídos e requisitos de desempenho ou recuperação.

Quando evitar ou limitar. Usar somente happy path ou tratar caos aleatório como prova de resiliência.

Uma abertura possível

Eu começo pela invariante e pelo risco: escolho o teste capaz de falsificar a hipótese, depois defino a evidência de sucesso.

Peças e tecnologias. Pact · Hypothesis · k6 · Chaos Mesh · Testcontainers

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Escrever invariantes e critérios de aprovação

2. Verificar contratos entre versões realmente implantadas

3. Gerar sequências e dados de borda

4. Executar carga representativa com limites seguros

5. Injetar falhas com observabilidade e critérios de parada

30 perguntas para sustentar o desenho

01. O que um teste de contrato prova?

Eu verifico que produtor e consumidor compartilham expectativas executáveis para interações específicas. Contract testing reduz incompatibilidades, mas não prova disponibilidade, autorização completa nem correção de todas as combinações de dados do domínio.

Termo para lembrar: Contract testing | Base técnica: referência 1.

02. Como evita contrato tautológico?

Eu parto das necessidades reais do consumidor e verifico o provedor independentemente. Se ambos os testes copiam o mesmo mock inventado, posso validar uma ficção que nunca corresponde ao comportamento implantado.

Termo para lembrar: Consumer-driven contract | Base técnica: referência 1.

03. Quando usaria property-based testing?

Eu o usaria para invariantes como conservação de valor, idempotência e monotonicidade de versão. Property-based testing explora combinações que exemplos manuais esquecem, desde que a propriedade represente uma regra verdadeira.

Termo para lembrar: Property-based testing | Base técnica: referência 2.

04. Como testa idempotência financeira?

Eu gero repetições, concorrência e respostas perdidas para a mesma intenção. A propriedade é um único efeito econômico por identidade, com resposta coerente; contar apenas códigos HTTP de sucesso seria insuficiente.

Termo para lembrar: Idempotency property | Base técnica: referência 2.

05. Como testa estorno?

Eu gero operações e compensações com atrasos e duplicatas. A conservation invariant exige lançamentos equilibrados e vínculos corretos; estorno não apaga histórico nem precisa restaurar cegamente todo estado concorrente anterior.

Termo para lembrar: Conservation invariant | Base técnica: referência 2.

06. O que é shrinking?

Quando um teste gerado falha, shrinking busca um exemplo menor que preserve a falha. Eu salvo esse caso como regressão e investigo a regra violada, em vez de culpar aleatoriedade.

Termo para lembrar: Shrinking | Base técnica: referência 2.

07. Como reproduz teste aleatório?

Eu preservo seed, exemplo reduzido e versões de dependências, além do ambiente relevante. Deterministic reproduction permite investigar o caso; uma falha intermitente sem dados de reprodução perde valor operacional rapidamente.

Termo para lembrar: Deterministic reproduction | Base técnica: referência 2.

08. Carga média é suficiente?

Não. Eu testo distribuição de tamanhos, concentração por chave e picos de chegada. Workload modeling precisa representar o que satura o sistema, incluindo consumidores lentos, clientes impacientes e dependências com caudas longas.

Termo para lembrar: Workload modeling | Base técnica: referência 3.

09. Qual diferença entre load e stress?

Load test verifica uma demanda esperada; stress test ultrapassa limites para descobrir degradação e recuperação. Eu defino objetivos diferentes, pois sobreviver ao pico não prova cumprir latência dentro do regime normal.

Termo para lembrar: Stress testing | Base técnica: referência 3.

10. Para que soak test?

Eu mantenho carga por tempo suficiente para revelar vazamentos, fragmentação, crescimento de filas e rotinas periódicas. Soak testing avalia estabilidade prolongada, que um teste curto de throughput máximo pode não detectar.

Termo para lembrar: Soak testing | Base técnica: referência 3.

11. Como evita coordinated omission?

Eu modelo chegadas independentemente da resposta quando esse é o tráfego real. Closed-loop load pode reduzir requisições justamente quando o serviço fica lento, escondendo latência percebida e capacidade necessária durante sobrecarga.

Termo para lembrar: Coordinated omission | Base técnica: referência 3.

12. Qual métrica decide aprovação?

Eu uso percentis, erros, saturação e invariantes de negócio com thresholds definidos previamente. Throughput isolado engana: responder rapidamente com falhas ou aceitar trabalho que nunca conclui não satisfaz o objetivo.

Termo para lembrar: Thresholds | Base técnica: referência 3.

13. Como testa hot key?

Eu concentro parte significativa da carga numa conta ou tenant e observo locks, partições e fairness. Hot-key testing revela gargalos que uma distribuição uniforme artificial mascara mesmo com volume total semelhante.

Termo para lembrar: Hot-key testing | Base técnica: referência 3.

14. Como testa retry storm?

Eu degrado uma dependência e observo amplificação de tentativas, filas e recuperação. Retry amplification precisa permanecer limitada por orçamento e jitter; aumentar réplicas do cliente pode piorar o sistema dependente.

Termo para lembrar: Retry amplification | Base técnica: referência 4.

15. Caos em produção é obrigatório?

Não. Eu começo por ambientes controlados e só amplio quando observabilidade, isolamento e autorização justificarem. Chaos engineering é experimento orientado por hipótese, não um requisito de senioridade nem destruição aleatória.

Termo para lembrar: Chaos engineering | Base técnica: referência 4.

16. Como define blast radius?

Eu limito tenants, pods, região, duração e volume afetados, com critérios automáticos de parada. Blast radius explícito torna o experimento revisável e reduz o risco de transformar uma hipótese em incidente generalizado.

Termo para lembrar: Blast radius | Base técnica: referência 4.

17. Qual seria uma hipótese de falha?

Eu testaria se perder um worker após commit local mantém um único efeito e permite retomada. Fault injection escolhe um ponto preciso do protocolo, com evidência antes e depois da interrupção.

Termo para lembrar: Fault injection | Base técnica: referência 4.

18. Como testa partição de rede?

Eu isolo caminhos específicos, incluindo falhas assimétricas, e observo timeouts, liderança e escrita obsoleta. Network partition não equivale simplesmente a matar um processo; participantes podem continuar vivos com visões diferentes.

Termo para lembrar: Network partition | Base técnica: referência 4.

19. Como testa lease vencido?

Eu pauso o detentor antigo, concedo nova posse e depois retomo o anterior. O teste verifica rejeição pelo recurso protegido, pois lease expirado não impede fisicamente que o processo antigo continue executando.

Termo para lembrar: Stale owner | Base técnica: referência 4.

20. Como testa restore?

Eu restauro dados num ambiente isolado, valido invariantes e reconcilio obrigações externas antes de liberar tráfego. Recovery testing mede RTO e perda efetiva, não apenas a existência de arquivos de backup.

Termo para lembrar: Recovery testing | Base técnica: referência 4.

21. Shadow traffic pode enviar dinheiro?

Eu desativo side effects e uso destinos isolados no caminho shadow. Shadow testing compara decisões ou respostas sem repetir notificações, pagamentos ou comandos externos; copiar tráfego não autoriza duplicar seus efeitos.

Termo para lembrar: Shadow testing | Base técnica: referência 3.

22. Como testa schema compatível?

Eu verifico consumidores antigos contra payloads novos e produtores antigos contra consumidores novos, conforme janela de coexistência. Backward compatibility envolve significado, valores desconhecidos e campos opcionais, além de desserialização bem-sucedida.

Termo para lembrar: Backward compatibility | Base técnica: referência 1.

23. E se contrato passar e produção falhar?

Eu investigo lacunas de ambiente, dados, autorização e semântica não representada. Contract coverage é parcial por natureza; complemento com integração e telemetria, sem concluir que o teste era inútil ou suficiente.

Termo para lembrar: Contract coverage | Base técnica: referência 1.

24. Como evita teste destrutivo fora do escopo?

Eu uso identidades e recursos dedicados, quotas e guards de ambiente. Safety guardrails devem impedir atingir destinatários reais ou namespaces externos mesmo quando um parâmetro do experimento estiver errado.

Termo para lembrar: Safety guardrails | Base técnica: referência 4.

25. Como testa DLQ e redrive?

Eu introduzo payload inválido, corrijo a causa e reprocesso preservando identidade e ordem necessária. O teste verifica que quarentena não significa conclusão e que redrive não repete um efeito financeiro já aplicado.

Termo para lembrar: Redrive testing | Base técnica: referência 4.

26. Como testa concorrência no banco?

Eu coordeno transações concorrentes com barreiras e verifico a invariante final. Concurrency testing precisa explorar interleavings relevantes, incluindo conflito e retry; executar sequencialmente muitas vezes não reproduz uma corrida real.

Termo para lembrar: Concurrency testing | Base técnica: referência 2.

27. Mocks de cloud bastam?

Não. Eu uso mocks para lógica rápida e testes de integração para garantias específicas do serviço. Emulation pode diferir em consistência, erros e quotas; preciso verificar os comportamentos dos quais minha arquitetura depende.

Termo para lembrar: Emulation | Base técnica: referência 1.

28. Como prioriza testes caros?

Eu relaciono custo ao risco e à frequência de mudança. Risk-based testing concentra experimentos complexos em invariantes críticas, fronteiras externas e caminhos de recuperação, mantendo verificações rápidas no ciclo cotidiano.

Termo para lembrar: Risk-based testing | Base técnica: referência 3.

29. Como trata teste flaky?

Eu investigo nondeterminism, ambiente e sincronização, preservando evidências. Quarentena temporária precisa de dono e prazo; ignorar falhas recorrentes transforma a suíte em ruído e pode esconder uma corrida real.

Termo para lembrar: Flaky test | Base técnica: referência 2.

30. O que apresentaria à banca?

Eu mostraria uma invariante, o ponto de falha escolhido e o resultado observável esperado. Test strategy conecta cada garantia a um experimento concreto, incluindo limites que ainda não foram demonstrados.

Termo para lembrar: Test strategy | Base técnica: referência 4.

10 trade-offs: ganho, custo e decisão

01. Testes unitários

Ganho. Feedback rápido

Custo e motivo. Pouca integração

Como decidir. Para lógica determinística

02. Contratos

Ganho. Compatibilidade explícita

Custo e motivo. Cobertura por exemplos

Como decidir. Para evolução entre equipes

03. Property-based

Ganho. Explora combinações

Custo e motivo. Modelagem difícil

Como decidir. Para invariantes estáveis

04. Integração real

Ganho. Semântica do serviço

Custo e motivo. Tempo e custo

Como decidir. Para garantias específicas

05. Emuladores

Ganho. Execução local barata

Custo e motivo. Diferenças do provedor

Como decidir. Para desenvolvimento sem tratá-los como prova completa

06. Carga aberta

Ganho. Modela chegadas independentes

Custo e motivo. Pode sobrecarregar rápido

Como decidir. Para demandas externas não reguladas pela resposta

07. Carga fechada

Ganho. Modela usuários concorrentes

Custo e motivo. Pode mascarar saturação

Como decidir. Para sessões com espera real

08. Caos em staging

Ganho. Menor impacto

Custo e motivo. Fidelidade limitada

Como decidir. Como primeiro passo

09. Caos em produção

Ganho. Ambiente representativo

Custo e motivo. Risco operacional

Como decidir. Com escopo e parada aprovados

10. Shadow

Ganho. Compara comportamento real

Custo e motivo. Custo e isolamento

Como decidir. Com efeitos externos desabilitados

Duas alternativas ao caminho proposto

1. Verificação baseada em modelos

Quando faz sentido. Protocolos concorrentes pequenos e críticos

Como implementar. Descrever estados e explorar transições possíveis

Limites. Modelo pode divergir da implementação real

2. Rollout progressivo com observabilidade

Quando faz sentido. Mudanças difíceis de reproduzir totalmente

Como implementar. Canary pequeno com métricas e rollback controlado

Limites. Não substitui testes prévios nem garante reversão de dados

Referências e alcance da evidência

1. Pact - Consumer tests

Contratos por exemplos executáveis e verificação de expectativas do consumidor.

https://docs.pact.io/consumer

2. Hypothesis - Documentation

Testes gerados por propriedades e casos extremos.

https://hypothesis.readthedocs.io/en/latest/

3. Grafana - k6 test types

Tipos de teste de carga e seus objetivos.

https://grafana.com/docs/k6/latest/testing-guides/test-types/

4. Chaos Mesh - Overview

Experimentos de falha controlados e mecanismos de simulação.

https://chaos-mesh.org/docs/

Continue em: T061 · T065 · T067 · T070

T077 / H. Segurança e plataforma

Migrações, expand-contract e backfills

Migrações seguras separam mudanças compatíveis, movimentação de dados e retirada de caminhos antigos.

Quando usar. Evolução de schema, troca de storage ou reconstrução de grandes conjuntos sem interrupção prolongada.

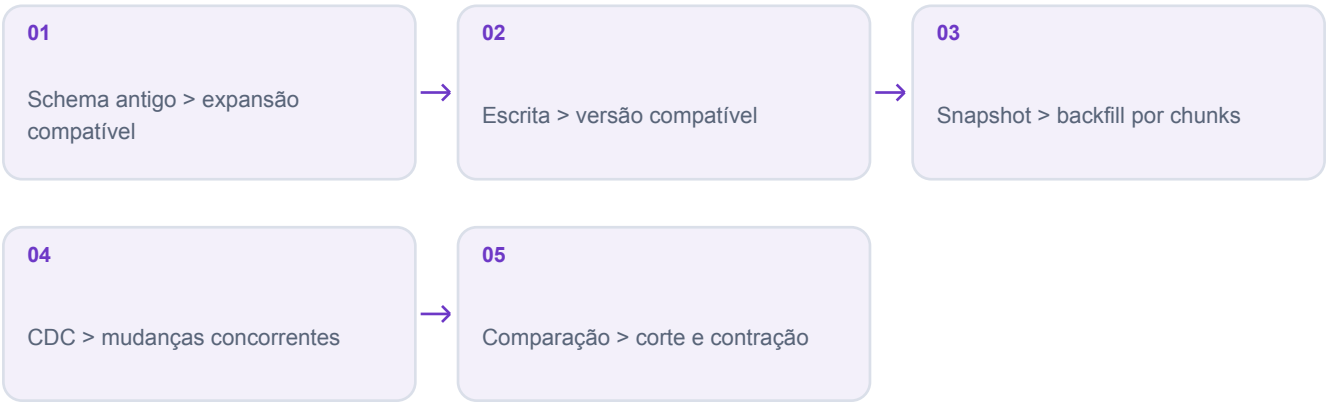
Quando evitar ou limitar. Renomear ou remover campo ainda usado por versões antigas e tratar dual write como atomicidade garantida.

Uma abertura possível

Eu expandiria primeiro, validaria dados e comportamento durante a coexistência e só contrairia quando o caminho antigo não fosse mais necessário.

Peças e tecnologias. PostgreSQL · CDC · Feature flags · Workers de backfill · Índices concorrentes

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Mapear leitores, escritores e versões coexistentes

2. Adicionar estruturas sem remover contratos antigos

3. Executar backfill retomável com limites de carga

4. Comparar resultados e controlar corte de leitura

5. Remover legado após janela de rollback validada

30 perguntas para sustentar o desenho

01. O que é expand-contract?

Eu adiciono uma estrutura compatível, migro escritores e leitores e só depois removo a antiga. Expand-contract permite versões coexistirem; não é seguro contrair enquanto jobs antigos ou clientes ainda dependem do contrato anterior.

Termo para lembrar: Expand-contract | Base técnica: referência 1.

02. Como adiciona coluna obrigatória?

Eu planejo uma etapa compatível para dados existentes e escritores antigos, preencho valores e valido a regra antes de exigir NOT NULL. O custo e os locks dependem da operação e versão concreta.

Termo para lembrar: NOT NULL migration | Base técnica: referência 2.

03. Por que evitar rename direto?

Renomear pode quebrar consultas de versões ainda implantadas. Eu introduzo o novo nome, adapto compatibilidade e migro consumidores gradualmente; schema compatibility precisa cobrir jobs, relatórios e integrações, não somente a API principal.

Termo para lembrar: Schema compatibility | Base técnica: referência 1.

04. Dual write é atômico?

Somente se ambas as alterações participarem da mesma transação suportada. Entre bancos independentes, dual write possui janela de inconsistência; uso uma fonte autoritativa e replicação recuperável, com reconciliação e identidade estável.

Termo para lembrar: Dual write | Base técnica: referência 1.

05. Como combina snapshot e CDC?

Eu estabeleço um ponto de corte consistente e capturo mudanças durante a cópia. Snapshot plus CDC exige tratar sobreposição e ordem, para não perder alterações nem sobrescrever valores novos com dados antigos.

Termo para lembrar: Snapshot plus CDC | Base técnica: referência 1.

06. Como evita backfill sobrescrever atualização recente?

Eu uso versão ou condição que compare a origem do valor antes da escrita. Conditional update impede que um chunk atrasado substitua uma mudança mais nova; relógio de parede sozinho pode ser insuficiente.

Termo para lembrar: Conditional update | Base técnica: referência 1.

07. Como retoma um backfill?

Eu persisto checkpoints por intervalo estável e faço cada chunk idempotente. A retomada aceita repetir trabalho concluído parcialmente, sem depender de uma posição apenas em memória ou de paginação que muda sob inserções.

Termo para lembrar: Checkpoint | Base técnica: referência 1.

08. OFFSET funciona para bilhões de linhas?

Eu prefiro keyset pagination por chave estável, com limites definidos. OFFSET pode exigir percorrer muitas linhas e produzir fronteiras instáveis sob mudanças; particionar intervalos também facilita retomada e paralelismo controlado.

Termo para lembrar: Keyset pagination | Base técnica: referência 1.

09. Como escolhe tamanho de chunk?

Eu começo pequeno e ajusto conforme duração de transação, WAL, lag e impacto no tráfego principal. Chunking limita locks e recuperação; lotes enormes melhoram amortização, mas aumentam risco e tempo de rollback.

Termo para lembrar: Chunking | Base técnica: referência 1.

10. Como controla carga de migração?

Eu uso rate limit e pausa automática quando latência, CPU ou replication lag ultrapassarem limites. Backfill throttling protege o serviço principal; capacidade ociosa momentânea não justifica consumir toda margem de recuperação.

Termo para lembrar: Backfill throttling | Base técnica: referência 1.

11. Como cria índice sem bloquear tanto?

Eu avalio CREATE INDEX CONCURRENTLY e suas restrições, duração e possibilidade de índice inválido após falha. Concurrent indexing reduz bloqueios específicos, mas não elimina consumo de CPU, I/O nem necessidade de monitoramento.

Termo para lembrar: Concurrent indexing | Base técnica: referência 3.

12. Como valida constraint em tabela grande?

Eu separo criação e validação quando o tipo de constraint permite, avaliando locks concretos. Constraint validation precisa demonstrar que dados existentes obedecem à regra sem assumir que adicionar metadata foi suficiente.

Termo para lembrar: Constraint validation | Base técnica: referência 2.

13. Como compara banco novo e antigo?

Eu comparo contagens, checksums por faixas e amostras semânticas, respeitando lag e ponto de corte. Reconciliation deve identificar diferenças relevantes do negócio, pois contagens iguais podem esconder valores ou associações incorretas.

Termo para lembrar: Reconciliation | Base técnica: referência 1.

14. Qual risco de comparar durante escrita?

Diferenças podem refletir momentos distintos, não corrupção. Eu uso watermark ou janela estabilizada para comparação coerente; sem alinhamento temporal, shadow reads podem gerar falsos positivos e ocultar divergências reais na análise.

Termo para lembrar: Watermark | Base técnica: referência 1.

15. Como migra leitura gradualmente?

Eu começo com shadow reads sem efeitos, depois direciono uma fração controlada e monitoro equivalência e latência. Read cutover precisa permitir retorno enquanto a fonte antiga ainda mantém dados suficientemente atualizados.

Termo para lembrar: Read cutover | Base técnica: referência 1.

16. Rollback de código desfaz migração?

Não. Dados transformados ou removidos podem ser irreversíveis para a versão antiga. Eu separo code rollback de data rollback e documento a janela em que ambos os caminhos continuam semanticamente compatíveis.

Termo para lembrar: Data rollback | Base técnica: referência 1.

17. Como remove uma coluna com segurança?

Eu verifico telemetria de acesso, contratos e versões antigas, desativo uso e aguardo a janela necessária. Contract phase só ocorre depois de evidência de abandono, incluindo jobs raros e rotinas de recuperação.

Termo para lembrar: Contract phase | Base técnica: referência 1.

18. Como migra identificador de cliente?

Eu mantenho uma tabela de correspondência com unicidade e proveniência, migrando referências de forma verificável. Identity mapping precisa preservar autorização e idempotência; trocar IDs sem vínculo pode duplicar pessoas ou operações.

Termo para lembrar: Identity mapping | Base técnica: referência 1.

19. Como altera tipo monetário?

Eu defino unidade, escala e regras de arredondamento antes da conversão, comparando invariantes financeiras. Semantic migration não é apenas cast SQL; uma mudança de centavos para decimal pode alterar valores se feita implicitamente.

Termo para lembrar: Semantic migration | Base técnica: referência 1.

20. Como migra enum de estado?

Eu explicito quais versões aceitam novos valores e como leitores antigos reagem. Forward compatibility exige tratar estado desconhecido sem convertê-lo silenciosamente em sucesso, erro definitivo ou autorização indevida no fluxo.

Termo para lembrar: Forward compatibility | Base técnica: referência 1.

21. Como migra chave de partição?

Eu projeto uma fase de roteamento versionado e preservo ordem por agregado durante a transição. Repartitioning pode mover a mesma entidade para outro caminho; deduplicação não corrige, sozinha, inversão de eventos dependentes.

Termo para lembrar: Repartitioning | Base técnica: referência 1.

22. Como trata exclusões no backfill?

Eu propago tombstones ou consulto um registro durável de exclusões, evitando recriar dados apagados durante a cópia. Delete propagation faz parte da migração; copiar somente linhas presentes pode deixar resíduos no destino.

Termo para lembrar: Delete propagation | Base técnica: referência 1.

23. Como evita duas execuções competirem?

Eu uso coordenação por intervalo e writes condicionais, com fencing quando necessário. Worker ownership organiza o trabalho, mas a correção deve sobreviver a retomadas e workers antigos que continuem executando após perder posse.

Termo para lembrar: Worker ownership | Base técnica: referência 1.

24. Como migra outbox e inbox?

Eu preservo identidades, escopo do consumidor e status de conclusão, incluindo obrigações pendentes. Deduplication state não é dado descartável: perdê-lo no corte pode repetir notificações ou efeitos financeiros durante replay e recuperação.

Termo para lembrar: Deduplication state | Base técnica: referência 1.

25. Como planeja pausa e retomada?

Eu torno checkpoints duráveis e limites operacionais configuráveis, sem exigir matar transações longas. Pause-resume permite ceder capacidade durante incidente e continuar depois, mantendo métricas de progresso e estimativa de trabalho restante.

Termo para lembrar: Pause-resume | Base técnica: referência 1.

26. Como calcula tempo de migração?

Eu considero bytes, linhas, custo de índices e vazão permitida após reservar capacidade do serviço. Migration ETA precisa incluir mudanças concorrentes e validação; dividir volume pela capacidade máxima teórica costuma subestimar duração.

Termo para lembrar: Migration ETA | Base técnica: referência 1.

27. Como testa antes de executar?

Eu ensaio com volume e distribuição representativos, incluindo falhas entre checkpoint e commit. Migration rehearsal verifica retomada, compatibilidade e tempo de recuperação, sem usar apenas uma base vazia que mascara locks e custos.

Termo para lembrar: Migration rehearsal | Base técnica: referência 1.

28. Quando aceita manutenção programada?

Eu aceito uma janela quando o custo e risco da migração online superam o impacto acordado. Maintenance window pode simplificar invariantes, desde que comunicação, recuperação e testes sustentem a decisão de indisponibilidade temporária.

Termo para lembrar: Maintenance window | Base técnica: referência 1.

29. Quem decide o corte final?

Eu estabeleço critérios objetivos de completude, divergência, desempenho e prontidão de rollback, com responsáveis definidos. Cutover gate evita que uma data arbitrária substitua evidência de que o novo caminho atende ao contrato.

Termo para lembrar: Cutover gate | Base técnica: referência 1.

30. Como resumiria à banca?

Eu trataria migração como protocolo distribuído com estados, identidade, progresso e recuperação. Zero-downtime migration é uma meta dependente de requisitos e mecanismos, não uma promessa automática obtida apenas com duas versões do schema.

Termo para lembrar: Zero-downtime migration | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Expand-contract

Ganho. Coexistência segura

Custo e motivo. Mais etapas

Como decidir. Para versões implantadas gradualmente

02. Janela de manutenção

Ganho. Menos complexidade

Custo e motivo. Indisponibilidade acordada

Como decidir. Quando impacto é aceitável

03. Dual write local

Ganho. Consistência na mesma transação

Custo e motivo. Acoplamento de schema

Como decidir. Quando ambos destinos são transacionais locais

04. CDC

Ganho. Captura contínua

Custo e motivo. Lag e operação adicional

Como decidir. Para migração entre storages

05. Chunks pequenos

Ganho. Menor impacto

Custo e motivo. Mais overhead

Como decidir. Quando tráfego principal é sensível

06. Chunks grandes

Ganho. Boa amortização

Custo e motivo. Locks e recuperação maiores

Como decidir. Com margem validada

07. Índice concorrente

Ganho. Menos bloqueio de escrita

Custo e motivo. Mais trabalho e restrições

Como decidir. Em tabelas ativas grandes

08. Shadow reads

Ganho. Comparação sem corte

Custo e motivo. Custo extra

Como decidir. Antes de mudar autoridade de leitura

09. Manter legado

Ganho. Rollback viável

Custo e motivo. Custo e complexidade

Como decidir. Durante janela de confiança

10. Contração rápida

Ganho. Menos dívida

Custo e motivo. Perde reversibilidade

Como decidir. Somente após critérios de abandono

Duas alternativas ao caminho proposto

1. Migração offline

Quando faz sentido. Janela de indisponibilidade aceitável

Como implementar. Parar escritores, copiar, validar e reiniciar no destino

Limites. Exige duração e recuperação ensaiadas

2. Strangler por domínio

Quando faz sentido. Sistema divisível em capacidades

Como implementar. Migrar uma capacidade por vez com contrato de roteamento

Limites. Fronteiras e dependências compartilhadas podem dificultar o corte

Referências e alcance da evidência

1. Stripe - Online migrations at scale

Experiência de migração gradual e verificação de dados em produção.

https://stripe.com/blog/online-migrations

2. PostgreSQL - ALTER TABLE

Operações de schema, validação e locks conforme alteração.

https://www.postgresql.org/docs/current/sql-altertable.html

3. PostgreSQL - CREATE INDEX

Construção concorrente de índices e suas restrições.

https://www.postgresql.org/docs/current/sql-createindex.html

Continue em: T043 · T062 · T063 · T070

T078 / H. Segurança e plataforma

Docker, ECS e Kubernetes

Containers empacotam processos; orquestradores gerenciam execução, substituição e escala sem resolver invariantes da aplicação.

Quando usar. Serviços duradouros, workers e ambientes que exigem implantação reproduzível e controle de recursos.

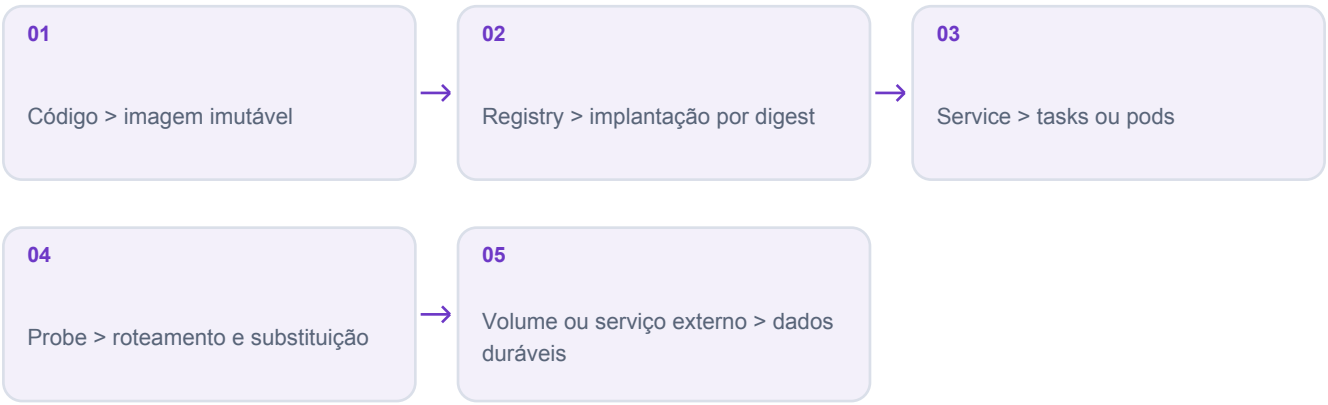
Quando evitar ou limitar. Adotar Kubernetes apenas por prestígio ou confundir container com isolamento absoluto e persistência automática.

Uma abertura possível

Eu escolho a plataforma pelo modelo operacional e pelas necessidades do workload; depois explico como a aplicação sobrevive à substituição de qualquer instância.

Peças e tecnologias. Docker · ECS · Fargate · Kubernetes · EKS · AKS

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Construir imagem mínima com dependências fixadas

2. Executar sem privilégios e com identidade própria

3. Definir recursos, probes e encerramento gracioso

4. Externalizar estado e proteger efeitos repetidos

5. Testar rollout, perda de nó e capacidade de reposição

30 perguntas para sustentar o desenho

01. Container é uma máquina virtual?

Não. Eu trato container como isolamento de processos com recursos do kernel, enquanto uma VM possui outra fronteira de virtualização. Container isolation não elimina a necessidade de patching, restrições de privilégios e proteção do host.

Termo para lembrar: Container isolation | Base técnica: referência 1.

02. O que uma imagem contém?

Ela empacota filesystem e configuração para executar o processo, normalmente em camadas. Immutable image permite reproduzir artefatos, mas dependências externas, segredos e configuração de runtime ainda precisam de gestão própria.

Termo para lembrar: Immutable image | Base técnica: referência 1.

03. Por que fixar digest?

Eu implanto por digest para identificar exatamente o artefato aprovado. Image tag pode ser alterada; o digest melhora rastreabilidade e rollback, embora eu ainda precise verificar origem, vulnerabilidades e compatibilidade do conteúdo.

Termo para lembrar: Image digest | Base técnica: referência 1.

04. Como reduz superfície de ataque?

Eu uso imagem mínima, usuário sem privilégios e filesystem somente leitura quando possível. Least privilege limita consequências de exploração, mas preciso garantir diretórios temporários e permissões suficientes para o comportamento legítimo.

Termo para lembrar: Least privilege | Base técnica: referência 1.

05. Segredo pode entrar na imagem?

Eu não incorporo segredos ao build nem às camadas, pois podem permanecer recuperáveis mesmo após remoção posterior. Runtime secret injection usa identidade e acesso limitado, com rotação e logs que não exponham valores.

Termo para lembrar: Runtime secret injection | Base técnica: referência 1.

06. O que é um Pod?

Eu trato Pod como unidade de agendamento Kubernetes que agrupa containers com contexto compartilhado. Não agrupo serviços independentes apenas por conveniência; o lifecycle conjunto deve corresponder a uma necessidade real de execução.

Termo para lembrar: Pod | Base técnica: referência 2.

07. Quando usa sidecar?

Eu uso sidecar quando um processo auxiliar precisa acompanhar de perto o principal, como proxy ou coleta específica. O acoplamento de recursos e lifecycle tem custo; bibliotecas ou serviços compartilhados podem ser mais simples.

Termo para lembrar: Sidecar | Base técnica: referência 2.

08. ECS Task equivale a serviço?

Task é uma unidade de execução definida por configuração; um service mantém o número desejado e integra implantação e tráfego. Eu diferencio execução pontual de serviço duradouro para escolher recuperação e escalabilidade adequadas.

Termo para lembrar: ECS Task | Base técnica: referência 3.

09. Quando prefere ECS a Kubernetes?

Eu prefiro ECS quando a integração AWS e uma operação mais enxuta atendem ao time. Kubernetes oferece ecossistema e controle adicionais, mas sua flexibilidade traz responsabilidades que precisam justificar o custo operacional.

Termo para lembrar: Managed orchestration | Base técnica: referência 3.

10. Fargate remove operação?

Ele reduz gestão direta dos servidores de execução, mas eu continuo responsável por imagens, limites, observabilidade e semântica da aplicação. Serverless containers não tornam transações idempotentes nem dimensionam automaticamente todas as dependências externas.

Termo para lembrar: Serverless containers | Base técnica: referência 3.

11. Como define requests e limits?

Eu uso medidas de CPU e memória sob carga representativa, mantendo margem e observando throttling ou OOM. Resource sizing precisa considerar picos e concorrência, não somente consumo médio de uma instância ociosa.

Termo para lembrar: Resource sizing | Base técnica: referência 2.

12. O que acontece num OOM?

O processo pode ser encerrado e substituído, perdendo memória local e trabalho não persistido. Eu projeto recuperação por checkpoint e idempotência; restart policy recupera execução, mas não garante conclusão correta do negócio.

Termo para lembrar: OOM recovery | Base técnica: referência 2.

13. Liveness deve consultar banco?

Eu evitaria fazer indisponibilidade transitória do banco provocar restart em massa. Liveness detecta processo irrecuperavelmente travado; readiness controla capacidade de receber trabalho, com cuidado para não retirar toda a frota simultaneamente.

Termo para lembrar: Liveness probe | Base técnica: referência 2.

14. Como encerra um worker?

Eu paro novas admissões, concluo ou abandono trabalho de forma recuperável e respeito o prazo de shutdown. Graceful shutdown reduz duplicatas, mas a correção também precisa sobreviver a uma morte abrupta sem aviso.

Termo para lembrar: Graceful shutdown | Base técnica: referência 3.

15. Como drena conexões HTTP?

Eu retiro a instância do roteamento, aguardo requisições existentes dentro de um limite e encerro o processo. Connection draining precisa alinhar timeouts do balanceador e runtime, inclusive conexões longas e clientes que retentam.

Termo para lembrar: Connection draining | Base técnica: referência 3.

16. Disco do container é durável?

Eu não trato o filesystem gravável da instância como fonte de verdade. Ephemeral storage pode desaparecer na substituição; uso volume apropriado ou serviço externo, com política de backup e recuperação claramente definida.

Termo para lembrar: Ephemeral storage | Base técnica: referência 1.

17. StatefulSet garante banco correto?

Não. Ele ajuda com identidade e associação de storage, mas consenso, replicação e recuperação pertencem ao banco e à operação. Stateful orchestration não substitui uma política testada de durabilidade e failover.

Termo para lembrar: Stateful orchestration | Base técnica: referência 2.

18. Como distribui por zonas?

Eu configuro posicionamento e capacidade para evitar concentração numa única zona, verificando dependências regionais. Failure domain diversity só funciona se dados, rede e capacidade remanescente suportarem a perda considerada no desenho.

Termo para lembrar: Failure domain | Base técnica: referência 3.

19. Autoscaling resolve hot partition?

Não necessariamente. Eu verifico a unidade real de paralelismo e o gargalo antes de escalar réplicas. Horizontal scaling não divide automaticamente uma chave quente, um lock exclusivo ou uma conexão limitada do parceiro.

Termo para lembrar: Horizontal scaling | Base técnica: referência 3.

20. Qual sinal usa para workers?

Eu considero idade da fila, vazão e tempo por tarefa, além de CPU. Queue-based scaling precisa respeitar capacidade do destino e tempo de aquecimento, para não transformar backlog em tempestade de conexões.

Termo para lembrar: Queue-based scaling | Base técnica: referência 3.

21. Como limita acesso cloud por serviço?

Eu uso identidade de workload com permissões específicas, evitando credenciais compartilhadas do nó. Workload identity reduz exposição entre aplicações, mas políticas e confiança precisam impedir que um serviço assuma a identidade de outro.

Termo para lembrar: Workload identity | Base técnica: referência 3.

22. Como atualiza sem perder trabalho?

Eu mantenho compatibilidade entre versões, sobreposição suficiente e drenagem, testando reentrega. Rolling update controla substituição de instâncias; a aplicação ainda deve preservar invariantes quando versões diferentes processam o mesmo domínio.

Termo para lembrar: Rolling update | Base técnica: referência 3.

23. Quantas réplicas são suficientes?

Eu dimensiono para carga, falhas e manutenção simultâneas, considerando capacidade remanescente. Replica count isolado não garante disponibilidade: três pods no mesmo nó ou dependentes de um banco indisponível compartilham o mesmo risco.

Termo para lembrar: Capacity headroom | Base técnica: referência 2.

24. Como coleta logs de containers?

Eu envio logs estruturados para um coletor com buffers e retenção adequados, evitando dependência exclusiva do disco local. Log aggregation preserva investigação após substituição, com redaction e identidade de workload para correlação.

Termo para lembrar: Log aggregation | Base técnica: referência 1.

25. Como impede imagem comprometida?

Eu valido origem, assinatura e políticas de admissão, além de scanning e atualização. Supply-chain controls reduzem risco, mas uma assinatura só demonstra a cadeia declarada; não prova ausência de comportamento malicioso ou vulnerável.

Termo para lembrar: Supply-chain controls | Base técnica: referência 1.

26. Como testa perda de nó?

Eu removo um nó em ambiente controlado e observo reagendamento, capacidade, volumes e efeitos em andamento. Node failure testing verifica recuperação real, pois o scheduler pode não encontrar recursos ou storage compatível imediatamente.

Termo para lembrar: Node failure testing | Base técnica: referência 2.

27. Como evita noisy neighbor?

Eu separo workloads críticos, aplico recursos e limites e observo contenção de CPU, memória e I/O. Resource isolation reduz interferência, mas não transforma hardware compartilhado em capacidade perfeitamente previsível sob qualquer carga.

Termo para lembrar: Resource isolation | Base técnica: referência 2.

28. Kubernetes garante portabilidade total?

Não. Manifestos podem depender de storage, identidade, balanceadores e operadores específicos. Portability precisa ser demonstrada no caminho completo; usar uma API comum de orquestração não elimina dependências particulares do provedor.

Termo para lembrar: Portability | Base técnica: referência 2.

29. Quando evitaria containers?

Eu avaliaria funções para tarefas curtas e eventuais ou serviços gerenciados para capacidades prontas. Operational simplicity pode ser mais valiosa que padronizar tudo em containers quando o workload não exige esse controle.

Termo para lembrar: Operational simplicity | Base técnica: referência 3.

30. O que enfatiza na entrevista?

Eu separo responsabilidades da plataforma e da aplicação, mostrando substituição, recursos e recuperação de efeitos. Orchestration mantém processos disponíveis; consistência financeira, autorização e idempotência continuam sendo decisões explícitas do sistema.

Termo para lembrar: Orchestration | Base técnica: referência 3.

10 trade-offs: ganho, custo e decisão

01. Imagem mínima

Ganho. Menos superfície

Custo e motivo. Debug mais limitado

Como decidir. Para produção com ferramentas externas

02. Imagem completa

Ganho. Diagnóstico fácil

Custo e motivo. Mais dependências

Como decidir. Quando necessidade operacional justifica

03. ECS

Ganho. Integração AWS simples

Custo e motivo. Acoplamento ao provedor

Como decidir. Para equipe focada em AWS

04. Kubernetes

Ganho. Ecossistema e controle

Custo e motivo. Complexidade operacional

Como decidir. Com demandas e capacidade de operação

05. Fargate

Ganho. Menos gestão de hosts

Custo e motivo. Custos e restrições

Como decidir. Para workloads compatíveis

06. Nós próprios

Ganho. Controle e otimização

Custo e motivo. Patching e capacidade

Como decidir. Quando requisitos justificam gestão

07. Sidecar

Ganho. Função próxima ao processo

Custo e motivo. Recursos e lifecycle acoplados

Como decidir. Para necessidade local clara

08. Estado externo

Ganho. Substituição simples

Custo e motivo. Dependência de rede

Como decidir. Para serviços stateless

09. Volumes persistentes

Ganho. Acesso local persistente

Custo e motivo. Agendamento e recuperação complexos

Como decidir. Para workloads stateful justificados

10. Mais réplicas

Ganho. Capacidade e redundância

Custo e motivo. Custo e pressão no destino

Como decidir. Quando gargalo é paralelizável

Duas alternativas ao caminho proposto

1. Plataforma de funções

Quando faz sentido. Execuções curtas e acionadas por eventos

Como implementar. Usar Lambda ou Azure Functions com estado durável externo

Limites. Limites de execução, cold starts e controle do runtime

2. VMs gerenciadas por autoscaling

Quando faz sentido. Legado ou requisitos específicos de host

Como implementar. Empacotar configuração e escalar grupos de instâncias

Limites. Mais responsabilidade por sistema operacional e rollout

Referências e alcance da evidência

1. Docker - Overview

Imagens, containers e componentes do runtime.

https://docs.docker.com/get-started/docker-overview/

2. Kubernetes - Pods

Unidade de execução, ciclo de vida e agrupamento de containers.

https://kubernetes.io/docs/concepts/workloads/pods/

3. AWS - ECS overview

Tasks, services e orquestração de containers.

https://docs.aws.amazon.com/AmazonECS/latest/developerguide/Welcome.html

Continue em: T054 · T065 · T070 · T079

T079 / H. Segurança e plataforma

Serverless: Lambda e Azure Functions

Funções gerenciadas executam código sob demanda com limites de duração, concorrência e ambiente que influenciam o desenho.

Quando usar. Tarefas eventuais, integrações e processamento de eventos com estado externo e duração compatível.

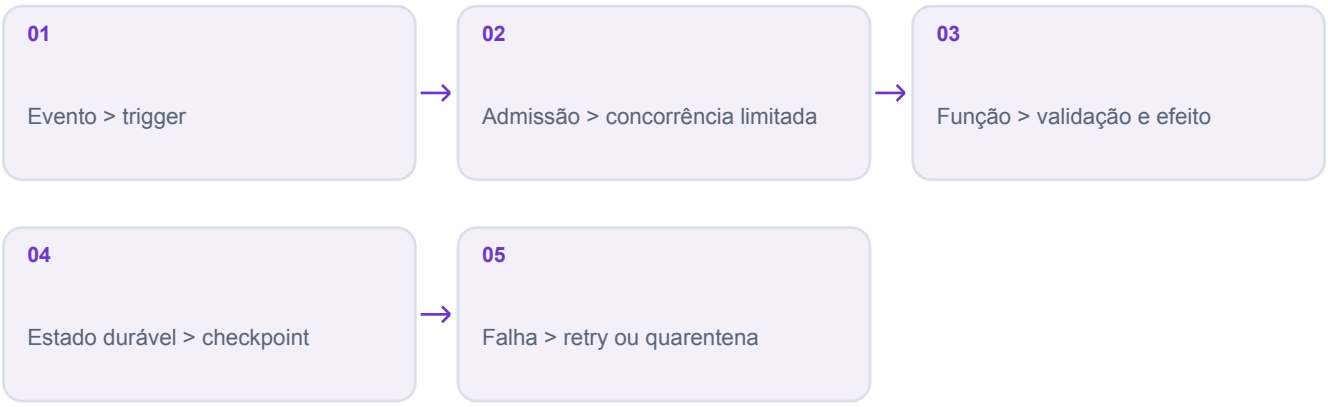
Quando evitar ou limitar. Tratar uma função como processo permanente, guardar progresso apenas em memória ou supor entrega única do trigger.

Uma abertura possível

Eu usaria funções onde o modelo de execução simplifica operação e colocaria estado, idempotência e retomada fora da instância.

Peças e tecnologias. AWS Lambda · Azure Functions · SQS · Step Functions · Durable Functions

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Caracterizar duração, memória, concorrência e dependências

2. Definir identidade estável e persistência do progresso

3. Limitar concorrência conforme capacidade dos destinos

4. Configurar falhas por modalidade de trigger

5. Testar cold start, timeout e repetição após commit

30 perguntas para sustentar o desenho

01. Serverless significa sem servidores?

Não. O provedor administra parte da infraestrutura, enquanto eu projeto código, permissões, capacidade e recuperação. Serverless execution muda a responsabilidade operacional, mas não elimina servidores nem os limites físicos e contratuais do runtime.

Termo para lembrar: Serverless execution | Base técnica: referência 1.

02. Quando escolheria Lambda?

Eu escolheria para tarefas de duração compatível, acionadas por eventos e com demanda variável, após avaliar custo e dependências. Workload fit importa mais que moda: processamento contínuo pesado pode ser melhor em containers.

Termo para lembrar: Workload fit | Base técnica: referência 1.

03. O que é cold start?

É a inicialização necessária quando um ambiente de execução precisa ser preparado. Eu reduzo dependências e trabalho de startup, mas dimensiono o SLO considerando a distribuição observada, não apenas execuções aquecidas.

Termo para lembrar: Cold start | Base técnica: referência 1.

04. Pode reutilizar conexão global?

Eu posso reutilizar clientes e conexões quando o ambiente sobreviver, validando conexões expiradas. Execution environment reuse é uma otimização oportunista; a correção não pode depender de a próxima invocação usar a mesma instância.

Termo para lembrar: Execution environment reuse | Base técnica: referência 1.

05. Pode guardar idempotência em memória?

Não para uma garantia durável. Eu mantenho a identidade e o resultado num storage apropriado, com arbitragem atômica. In-memory deduplication desaparece com substituição e não coordena invocações paralelas em ambientes diferentes.

Termo para lembrar: In-memory deduplication | Base técnica: referência 1.

06. Como controla pressão no banco?

Eu limito concorrência, reutilizo conexões com cuidado e avalio proxy ou pooling compatível. Concurrency limit protege o banco; escalar funções sem esse limite pode esgotar conexões mesmo com CPU disponível na plataforma.

Termo para lembrar: Concurrency limit | Base técnica: referência 1.

07. Timeout significa que nada aconteceu?

Não. O efeito pode ter sido confirmado antes de a invocação terminar. Eu uso operation_id e reconciliação para respostas ambíguas; timeout não autoriza repetir uma operação externa com nova identidade.

Termo para lembrar: Ambiguous outcome | Base técnica: referência 1.

08. Como processa lote parcialmente falho?

Eu registro efeitos individualmente e uso o mecanismo de resposta parcial suportado pelo trigger quando aplicável. Partial batch failure evita repetir trabalho saudável desnecessariamente, mas cada item ainda precisa tolerar reentrega.

Termo para lembrar: Partial batch failure | Base técnica: referência 1.

09. Todos os triggers retentam igual?

Não. Eu verifico a modalidade síncrona, assíncrona ou baseada em polling e suas configurações específicas. Retry semantics pertencem à integração concreta; não generalizo a política de uma fila para uma chamada HTTP direta.

Termo para lembrar: Retry semantics | Base técnica: referência 1.

10. Como usa DLQ corretamente?

Eu configuro destino de falha conforme o trigger e preservo contexto, identidade e motivo. Dead-letter handling exige monitoramento e redrive seguro; enviar para quarentena não equivale a concluir o pedido do cliente.

Termo para lembrar: Dead-letter handling | Base técnica: referência 1.

11. Função é adequada a arquivo gigante?

Depende de duração, memória e streaming. Eu separo manifesto e chunks retomáveis, ou uso container quando o trabalho ultrapassa limites; chunk processing evita carregar o arquivo inteiro e perder todo progresso num timeout.

Termo para lembrar: Chunk processing | Base técnica: referência 1.

12. Como executa tarefa de horas?

Eu uso workflow durável ou worker apropriado, persistindo progresso e dividindo unidades recuperáveis. Durable orchestration coordena a execução longa; não mantenho uma função artificialmente viva nem encadeio retries sem estado explícito.

Termo para lembrar: Durable orchestration | Base técnica: referência 2.

13. Como trata arquivos temporários?

Eu uso armazenamento temporário apenas para dados reconstruíveis e limpo conteúdo sensível. Ephemeral filesystem pode ser reutilizado ou descartado conforme runtime; não é uma fonte de verdade nem isolamento suficiente entre invocações.

Termo para lembrar: Ephemeral filesystem | Base técnica: referência 1.

14. Como dimensiona memória?

Eu faço benchmark representativo e observo duração, CPU associada e custo total, pois recursos podem variar com a configuração. Memory tuning deve medir o workload real, incluindo serialização, bibliotecas e tamanho dos payloads.

Termo para lembrar: Memory tuning | Base técnica: referência 1.

15. Como reduz custo sem piorar SLO?

Eu comparo custo por operação concluída, incluindo retries, cold starts e serviços dependentes. Cost optimization pode favorecer mais recursos por menos tempo; escolher a menor memória ou preço unitário isoladamente costuma enganar.

Termo para lembrar: Cost optimization | Base técnica: referência 1.

16. Como protege segredos?

Eu uso identidade de execução restrita e acesso a um gerenciador de segredos, com cache controlado e rotação. Least privilege inclui quem pode invocar, configurar e publicar a função, além de suas chamadas externas.

Termo para lembrar: Least privilege | Base técnica: referência 1.

17. Como evita duplicata em trigger S3?

Eu uso identidade de objeto e versão adequada, além de registro durável do efeito. Event deduplication considera que eventos podem repetir; nome do arquivo sozinho pode colidir quando um objeto é substituído.

Termo para lembrar: Event deduplication | Base técnica: referência 1.

18. Como conecta função à VPC?

Eu verifico rotas, endpoints, DNS e acesso aos destinos necessários, considerando latência e custos. VPC integration não concede automaticamente acesso à internet nem transforma uma subnet privada em conectividade funcional com qualquer parceiro.

Termo para lembrar: VPC integration | Base técnica: referência 1.

19. Como evita funções conversando em cascata?

Eu agrupo passos simples quando apropriado e uso filas ou orquestração para fronteiras reais. Function chaining excessivo amplia latência, custo e pontos de falha, especialmente quando cada passo faz apenas uma transformação trivial.

Termo para lembrar: Function chaining | Base técnica: referência 2.

20. Como garante ordem por conta?

Eu escolho um mecanismo de particionamento ou sessão compatível e valido sequência no estado do domínio. Ordered processing não surge da função; concorrência e retries podem inverter conclusão mesmo quando a entrega tem alguma ordem.

Termo para lembrar: Ordered processing | Base técnica: referência 1.

21. Como observa uma execução assíncrona?

Eu propago correlation_id, event_id e contexto de trace, medindo idade da mensagem e resultado final. Async observability distingue duração da invocação de tempo total até a conclusão percebida pelo cliente.

Termo para lembrar: Async observability | Base técnica: referência 1.

22. Como faz deploy seguro?

Eu publico versão imutável, direciono tráfego gradualmente quando suportado e acompanho métricas e invariantes. Versioned deployment exige compatibilidade de eventos e estado, porque invocações antigas podem coexistir com a versão nova.

Termo para lembrar: Versioned deployment | Base técnica: referência 1.

23. Como lida com quota atingida?

Eu modelo throttling como sinal de admissão e aplico buffering limitado ou recusa explícita. Service quota precisa entrar no dimensionamento; fila infinita apenas adia a falha e pode ultrapassar prazos do negócio.

Termo para lembrar: Service quota | Base técnica: referência 1.

24. Provisionar capacidade elimina toda latência?

Não. Pode reduzir inicialização de ambientes previstos, mas não elimina rede, dependências lentas ou demanda acima da capacidade preparada. Provisioned capacity deve ser comparada ao SLO e ao custo, com medição de cauda.

Termo para lembrar: Provisioned capacity | Base técnica: referência 1.

25. Como trata recursão acidental?

Eu separo prefixos e eventos de entrada e saída, aplico guards e monitoro invocações anormais. Recursive trigger pode gerar custo e carga rapidamente; a topologia precisa impedir que um resultado volte como nova entrada.

Termo para lembrar: Recursive trigger | Base técnica: referência 1.

26. Como escolhe plano Azure Functions?

Eu comparo limites, escala, rede e previsibilidade do plano com o workload concreto. Hosting plan altera comportamento operacional; não assumo que toda configuração Azure Functions possui os mesmos limites ou características de inicialização.

Termo para lembrar: Hosting plan | Base técnica: referência 2.

27. Como testa localmente?

Eu testo lógica isolada e integração real para comportamento de triggers, permissões e falhas. Local emulation acelera desenvolvimento, mas não comprova quotas, timing de reentrega ou semântica completa do ambiente gerenciado.

Termo para lembrar: Local emulation | Base técnica: referência 2.

28. Quando prefere container?

Eu prefiro container para execução contínua, dependências específicas ou maior controle de recursos e ciclo de vida. Runtime control tem custo operacional, mas pode simplificar workloads que se encaixam mal nos limites das funções.

Termo para lembrar: Runtime control | Base técnica: referência 1.

29. Como calcula concorrência necessária?

Eu estimo chegada por segundo multiplicada pela duração média, depois valido cauda, bursts e limites dos destinos. Little's Law orienta a primeira aproximação, mas demanda não estacionária e retries exigem margem e testes.

Termo para lembrar: Little's Law | Base técnica: referência 1.

30. Qual fala fecha a escolha?

Eu explicaria por que o formato de execução combina com a tarefa e onde ficam as garantias duráveis. Serverless é uma escolha de plataforma; a responsabilidade por efeitos corretos e recuperação continua explícita.

Termo para lembrar: Serverless | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Função sob demanda

Ganho. Escala e pouca gestão

Custo e motivo. Inicialização e limites

Como decidir. Para demanda variável

02. Container contínuo

Ganho. Controle e previsibilidade

Custo e motivo. Capacidade ociosa

Como decidir. Para fluxo sustentado

03. Concorrência alta

Ganho. Drena rápido

Custo e motivo. Pressiona dependências

Como decidir. Quando destinos suportam

04. Concorrência restrita

Ganho. Protege destinos

Custo e motivo. Maior backlog

Como decidir. Quando capacidade externa limita

05. Mais memória

Ganho. Pode reduzir duração

Custo e motivo. Preço por tempo maior

Como decidir. Após benchmark por operação

06. Cache de cliente

Ganho. Menos setup

Custo e motivo. Conexão expirada

Como decidir. Com validação e fallback

07. Lotes grandes

Ganho. Amortiza invocação

Custo e motivo. Mais repetição e latência

Como decidir. Para throughput tolerante

08. Lotes pequenos

Ganho. Isolamento de falha

Custo e motivo. Mais overhead

Como decidir. Para latência ou itens caros

09. Capacidade preparada

Ganho. Menos cold starts

Custo e motivo. Custo reservado

Como decidir. Quando cauda de latência justifica

10. Workflow durável

Ganho. Retomada de fluxo longo

Custo e motivo. Estado e custo adicionais

Como decidir. Para múltiplos passos recuperáveis

Duas alternativas ao caminho proposto

1. Workers em ECS ou Kubernetes

Quando faz sentido. Carga contínua ou duração longa

Como implementar. Consumir fila com checkpoint, limites e shutdown gracioso

Limites. Exige operação da frota e capacidade

2. Serviço gerenciado sem código customizado

Quando faz sentido. Transformação ou integração padronizada

Como implementar. Usar roteamento e integração nativos com contratos claros

Limites. Menos flexibilidade e dependência das capacidades do serviço

Referências e alcance da evidência

1. AWS - Lambda best practices

Execução, reutilização de ambiente, idempotência e escalabilidade.

https://docs.aws.amazon.com/lambda/latest/dg/best-practices.html

2. Microsoft - Azure Functions best practices

Projeto de funções, execução e práticas operacionais.

https://learn.microsoft.com/en-us/azure/azure-functions/functions-best-practices

Continue em: T045 · T048 · T061 · T078

T080 / H. Segurança e plataforma

IaC, Terraform, OpenTofu e GitOps

Infraestrutura declarativa e GitOps tornam mudanças rastreáveis, mas exigem proteção de estado, revisão e controle de reconciliação.

Quando usar. Ambientes repetíveis, múltiplas equipes e necessidade de auditar configuração e mudanças.

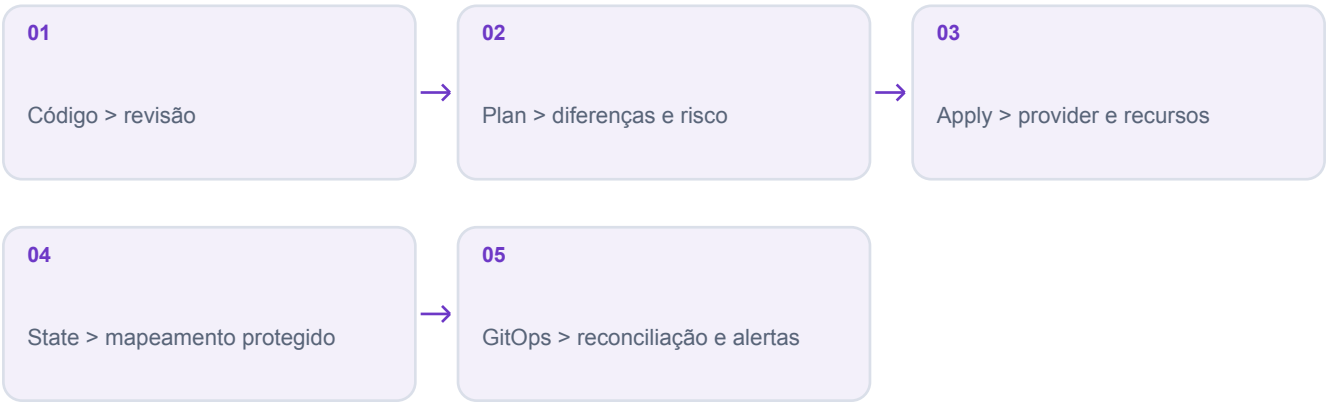
Quando evitar ou limitar. Aplicar planos sem revisar destruição, guardar segredos em repositório ou presumir que Git descreve todos os efeitos reais.

Uma abertura possível

Eu versionaria intenção, revisaria o plano e protegeria o estado; a reconciliação seria uma operação observável com limites e recuperação.

Peças e tecnologias. Terraform · OpenTofu · Argo CD · Flux · Git · Backend remoto de estado

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Separar estados e permissões por fronteira operacional

2. Configurar backend seguro e locking suportado

3. Validar módulos, políticas e plano de mudanças

4. Aplicar com identidade restrita e rastreabilidade

5. Testar drift, importação e recuperação de estado

30 perguntas para sustentar o desenho

01. Para que serve o state?

Eu uso state para mapear recursos declarados aos objetos reais e guardar informações necessárias ao gerenciamento. Terraform state não é apenas cache descartável; perda ou alteração indevida pode comprometer planos e ownership dos recursos.

Termo para lembrar: Terraform state | Base técnica: referência 1.

02. State pode conter segredos?

Sim. Eu trato o arquivo como sensível, restringindo acesso e protegendo armazenamento e backups. Sensitive output pode ocultar exibição, mas não significa que o valor deixou de existir no estado persistido.

Termo para lembrar: Sensitive state | Base técnica: referência 1.

03. Por que backend remoto?

Ele centraliza acesso controlado, persistência e colaboração, conforme capacidades do backend. Remote state não garante automaticamente locking ou criptografia em toda configuração; verifico os mecanismos disponíveis e suas políticas concretas.

Termo para lembrar: Remote state | Base técnica: referência 1.

04. Locking evita toda corrida?

Ele coordena operações sobre o estado quando suportado, mas não impede mudanças manuais ou outros sistemas gerenciando o mesmo recurso. State locking precisa acompanhar ownership claro e processos de alteração fora da ferramenta.

Termo para lembrar: State locking | Base técnica: referência 1.

05. Como divide estados?

Eu separo por fronteira de responsabilidade e ciclo de vida, evitando um estado monolítico que acople tudo. State partitioning reduz blast radius, mas cria dependências e exige contratos explícitos entre stacks.

Termo para lembrar: State partitioning | Base técnica: referência 1.

06. Plan aprovado garante apply idêntico?

Eu verifico o artefato de plano e o contexto de aplicação, mas reconheço mudanças externas e respostas do provider. Execution plan torna a intenção revisável; não é uma garantia absoluta de sucesso ou ausência de drift.

Termo para lembrar: Execution plan | Base técnica: referência 1.

07. Como protege mudança destrutiva?

Eu reviso replacements e deletes, aplico políticas e preparo recuperação antes da execução. Destructive change control depende do recurso: impedir destroy no código não protege contra todas as alterações fora desse caminho.

Termo para lembrar: Destructive change control | Base técnica: referência 1.

08. O que é drift?

É a divergência entre configuração pretendida e infraestrutura observada. Eu detecto e classifico drift antes de reconciliar, porque uma mudança emergencial legítima não deve ser revertida cegamente durante um incidente em andamento.

Termo para lembrar: Drift | Base técnica: referência 1.

09. Como trata hotfix manual?

Eu registro a exceção, preservo evidências e incorporo a mudança ao código ou a reverto de forma deliberada. Drift reconciliation fecha a divergência; deixar o ajuste manual indefinido torna próximos planos imprevisíveis.

Termo para lembrar: Drift reconciliation | Base técnica: referência 1.

10. GitOps é apenas deploy via CI?

Não. GitOps usa estado declarativo versionado e reconciliação contínua por agentes. Uma pipeline que executa comandos uma vez pode ser útil, mas não necessariamente mantém o ciclo de comparação e convergência esperado.

Termo para lembrar: GitOps | Base técnica: referência 3.

11. Qual risco da reconciliação automática?

Ela pode propagar uma configuração ruim ou desfazer intervenção emergencial. Reconciliation loop precisa de políticas, observabilidade e opção de suspensão controlada, com retomada explícita e revisão do estado desejado antes de continuar.

Termo para lembrar: Reconciliation loop | Base técnica: referência 3.

12. Como evita dois controladores competindo?

Eu defino um owner por recurso ou campo conforme o mecanismo utilizado e separo responsabilidades. Controller contention causa alterações repetidas e instabilidade quando Terraform, operador e ação manual tentam impor valores incompatíveis.

Termo para lembrar: Controller contention | Base técnica: referência 3.

13. Como usa módulos sem abstrair demais?

Eu encapsulo padrões estáveis com interfaces pequenas e versões explícitas, mantendo opções relevantes visíveis. Module design deve reduzir repetição sem esconder decisões de rede, segurança e recuperação que a equipe precisa compreender.

Termo para lembrar: Module design | Base técnica: referência 1.

14. Como atualiza provider?

Eu fixo versões, leio mudanças relevantes e executo plano e testes em ambiente representativo antes da promoção. Provider upgrade pode alterar schema ou comportamento; atualizar dependências de infraestrutura também exige compatibilidade e rollback planejado.

Termo para lembrar: Provider upgrade | Base técnica: referência 1.

15. Terraform e OpenTofu são intercambiáveis sempre?

Eu verifico versões, formatos de estado, providers e funcionalidades usadas antes de migrar. Tool compatibility precisa ser demonstrada no projeto concreto; origem comum não garante equivalência permanente entre todas as versões e recursos.

Termo para lembrar: Tool compatibility | Base técnica: referência 2.

16. Como importa recurso existente?

Eu associo o objeto real à configuração e reviso o plano até compreender diferenças. Resource import não escreve automaticamente uma intenção correta para todos os atributos nem resolve conflitos de ownership com ferramentas anteriores.

Termo para lembrar: Resource import | Base técnica: referência 1.

17. Como recupera state perdido?

Eu uso backup protegido, verifico recursos reais e reconstruo mapeamentos cuidadosamente antes de aplicar. State recovery não deve recriar infraestrutura às cegas, pois dados existentes podem ser destruídos ou duplicados por uma interpretação incorreta.

Termo para lembrar: State recovery | Base técnica: referência 1.

18. Como gerencia segredos no GitOps?

Eu versiono referências ou conteúdo cifrado apropriado e entrego segredos por identidade e mecanismo controlado. Secret delivery precisa considerar quem pode descriptografar, histórico do repositório e rotação, sem guardar credenciais em texto claro.

Termo para lembrar: Secret delivery | Base técnica: referência 3.

19. Como aplica menor privilégio na pipeline?

Eu separo identidade de plan e apply quando viável, restringindo escopo e duração das credenciais. Pipeline identity é um alvo sensível; permissão para alterar workflow pode equivaler a permissão para modificar infraestrutura crítica.

Termo para lembrar: Pipeline identity | Base técnica: referência 1.

20. Como revisa uma mudança de rede?

Eu verifico exposição, rotas, dependências e efeito em conexões existentes, além do diff declarativo. Change review precisa traduzir atributos em comportamento, pois uma pequena alteração textual pode abrir acesso amplo ou interromper parceiros.

Termo para lembrar: Change review | Base técnica: referência 1.

21. Como testa IaC?

Eu combino validação estática, políticas, plano e aplicação em ambiente controlado com verificações funcionais. Infrastructure testing precisa confirmar conectividade e permissões reais, porque sintaxe válida não demonstra um serviço utilizável e seguro.

Termo para lembrar: Infrastructure testing | Base técnica: referência 1.

22. Rollback é git revert?

Git revert restaura intenção textual, mas o apply pode não recuperar dados apagados ou recursos substituídos. Infrastructure rollback exige estratégia por recurso e backups; convergir ao código antigo não desfaz automaticamente todos os efeitos.

Termo para lembrar: Infrastructure rollback | Base técnica: referência 3.

23. Como gerencia dependências entre stacks?

Eu exponho outputs mínimos e contratos versionados, evitando ler indiscriminadamente estados completos de outras equipes. Cross-stack dependency deve preservar autonomia e segurança; acesso ao backend de estado pode revelar mais informações que o necessário.

Termo para lembrar: Cross-stack dependency | Base técnica: referência 1.

24. Como controla custo no plano?

Eu relaciono recursos e parâmetros a estimativas e quotas, revisando mudanças de escala ou retenção. Cost guardrail ajuda a detectar regressões, mas tráfego real e uso variável continuam exigindo acompanhamento após a implantação.

Termo para lembrar: Cost guardrail | Base técnica: referência 1.

25. Como evita credencial longa no CI?

Eu prefiro federação de identidade e credenciais temporárias com condições de confiança adequadas. Workload federation reduz segredos persistentes, mas permissões do repositório e contexto autorizado de execução continuam fazendo parte da fronteira.

Termo para lembrar: Workload federation | Base técnica: referência 1.

26. Como promove entre ambientes?

Eu promovo versões de módulos e artefatos com configuração explícita por ambiente, sem copiar estado. Environment promotion precisa respeitar diferenças reais e validar o plano do destino; um sucesso em teste não autoriza qualquer mudança em produção.

Termo para lembrar: Environment promotion | Base técnica: referência 1.

27. Como lida com recurso não declarativo?

Eu isolo a operação imperativa, registro pré-condições e recuperação e evito executá-la implicitamente em qualquer plan. Imperative escape hatch pode ser necessário, mas aumenta risco de repetição e reduz previsibilidade do ciclo declarativo.

Termo para lembrar: Imperative escape hatch | Base técnica: referência 1.

28. Como observa o reconciliador?

Eu monitoro divergência, erros, tempo de convergência e alterações aplicadas, com correlação ao commit. Reconciliation observability precisa mostrar falha persistente, não somente processo vivo; um agente saudável pode estar incapaz de convergir recursos.

Termo para lembrar: Reconciliation observability | Base técnica: referência 3.

29. Quando suspenderia automação?

Eu suspenderia de forma controlada quando ela amplificar um incidente ou quando a intenção no Git estiver incorreta. Automation pause exige responsável, escopo e condição de retomada para não deixar infraestrutura sem governança indefinidamente.

Termo para lembrar: Automation pause | Base técnica: referência 3.

30. Como explicaria o valor para Staff?

Eu conectaria repetibilidade a ownership, revisão e recuperação, mostrando o que acontece quando a ferramenta falha. Declarative infrastructure reduz erros manuais, mas sua segurança depende de processos e garantias concretas sobre estado e mudanças.

Termo para lembrar: Declarative infrastructure | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Estado monolítico

Ganho. Dependências simples

Custo e motivo. Blast radius grande

Como decidir. Para escopo pequeno e coeso

02. Estados separados

Ganho. Autonomia e isolamento

Custo e motivo. Contratos entre stacks

Como decidir. Para fronteiras operacionais distintas

03. Apply automático

Ganho. Feedback rápido

Custo e motivo. Propaga erro rápido

Como decidir. Com políticas e risco limitado

04. Apply revisado

Ganho. Avaliação contextual

Custo e motivo. Maior espera

Como decidir. Para mudanças críticas ou destrutivas

05. Reconciliação contínua

Ganho. Corrige drift

Custo e motivo. Pode desfazer hotfix

Como decidir. Com suspensão e ownership claros

06. Reconciliação sob demanda

Ganho. Controle explícito

Custo e motivo. Drift permanece

Como decidir. Para ambientes com operação específica

07. Módulos genéricos

Ganho. Reuso amplo

Custo e motivo. Interface complexa

Como decidir. Quando variação real justifica

08. Módulos opinativos

Ganho. Padrão simples

Custo e motivo. Menos flexibilidade

Como decidir. Para plataforma com decisões estáveis

09. Backend central

Ganho. Colaboração e backup

Custo e motivo. Concentração de acesso

Como decidir. Com isolamento por política

10. Importar legado

Ganho. Preserva recursos

Custo e motivo. Reconciliação trabalhosa

Como decidir. Quando recriação ameaça dados ou disponibilidade

Duas alternativas ao caminho proposto

1. Infraestrutura gerenciada por plataforma interna

Quando faz sentido. Equipes precisam de interface limitada

Como implementar. Oferecer APIs ou templates aprovados com reconciliação central

Limites. Pode limitar flexibilidade e exige equipe de plataforma

2. Configuração por SDK e pipelines imperativas

Quando faz sentido. Operações não representadas declarativamente

Como implementar. Implementar comandos idempotentes com registro e rollback específico

Limites. Mais lógica própria de estado, drift e recuperação

Referências e alcance da evidência

1. HashiCorp - Terraform State

Mapeamento de recursos, estado e colaboração operacional.

https://developer.hashicorp.com/terraform/language/state

2. OpenTofu - State

Estado, recursos e ciclo de gerenciamento de infraestrutura.

https://opentofu.org/docs/language/state/

3. OpenGitOps - Principles

Estado declarativo versionado e reconciliação contínua.

https://opengitops.dev/

Continue em: T067 · T070 · T073 · T078

T081 / I. Performance e dados

Runtime, threads, event loop e GC

Runtime decide como trabalho concorrente consome CPU, memória e threads. Arquitetura de treino; não descreve implementação interna do Nubank.

Quando usar. Quando latência cresce sem aumento proporcional de tráfego.

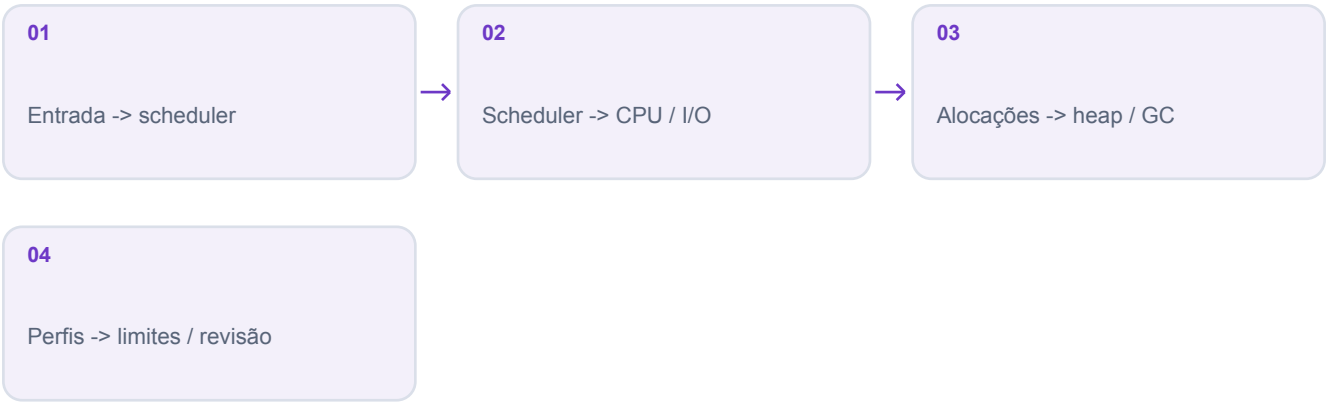
Quando evitar ou limitar. Ajustar GC ou threads sem perfil e limites.

Uma abertura possível

Eu começaria por mover transformação pesada para workers limitados. Depois explicaria dimensionar pool pela espera e capacidade, preservando as restrições do caso.

Peças e tecnologias. Go · JVM · Node.js · pprof · JFR

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Mover transformação pesada para workers limitados; validar atraso do loop durante payloads grandes.

2. Dimensionar pool pela espera e capacidade; validar fila e ocupação do pool.

3. Medir alocação antes de alterar heap; validar pausas e frequência de coleta.

4. Limitar tarefas com semáforo e cancelamento; validar goroutines após interrupção do destino.

5. Usar fila limitada com admissão explícita; validar tempo para saturar o buffer.

30 perguntas para sustentar o desenho

01. Uma transformação JSON trava todos os chats?

Eu retiraria transformação pesada do caminho do event loop, usando workers limitados quando o custo justificar. Mediria tempo bloqueado e custo de transferência, porque mover trabalho também adiciona coordenação e cópias.

Termo para lembrar: Event loop | Base técnica: referência 1.

02. Mais threads resolveriam o extrato lento?

Primeiro distinguiria espera externa de CPU. Aumentaria threads apenas até a capacidade útil das dependências, limitando fila e conexões; mais threads competindo pelo mesmo recurso podem piorar a cauda.

Termo para lembrar: Thread pool | Base técnica: referência 2.

03. Aumentar heap sempre reduz latência?

Compararia alocação, objetos vivos, pausas e CPU de coleta. Heap maior pode reduzir frequência de GC e aumentar memória, mas não resolve retenção indevida; escolheria configuração pelo SLO medido.

Termo para lembrar: GC | Base técnica: referência 1.

04. Posso criar uma goroutine por evento?

Eu limitaria tarefas em voo conforme memória e capacidade do destino. Goroutines são leves, mas não gratuitas; cancelamento e buffers limitados impedem que uma dependência lenta acumule trabalho indefinidamente.

Termo para lembrar: Goroutines | Base técnica: referência 2.

05. Como lidar com produtor mais rápido?

Usaria uma fila com capacidade explícita e política de admissão. Quando cheia, rejeitaria, adiaria ou reduziria trabalho conforme contrato; não permitiria que crescimento de memória fosse a estratégia de absorção.

Termo para lembrar: Backpressure local | Base técnica: referência 1.

06. Como descobrir espera escondida?

Coletaria perfil de espera além de CPU, observando locks, pool e chamadas externas. Uma aplicação pode estar lenta com CPU ociosa porque o recurso limitante é conexão, disco ou serialização de acesso.

Termo para lembrar: Blocking I/O | Base técnica: referência 2.

07. O parser cria objetos demais?

Antes de adicionar pooling, mediria quais alocações dominam custo e quais objetos permanecem vivos. Reutilização exige ownership claro; um pool pode aumentar retenção e complexidade em vez de melhorar a jornada.

Termo para lembrar: Allocations | Base técnica: referência 1.

08. O cliente saiu e o trabalho continua?

Propagaria cancelamento para trabalho ainda reversível e verificaria sua observância. Se um efeito remoto já pode ter ocorrido, manteria acompanhamento e reconciliação; desistência do cliente não constitui rollback financeiro.

Termo para lembrar: Cancellation | Base técnica: referência 2.

09. Um cliente monopoliza o processo?

Separaria orçamento por tenant ou classe de trabalho, evitando que uma tarefa longa bloqueie todas as curtas. Validaria justiça com carga desigual, mantendo limites globais para proteger recursos compartilhados.

Termo para lembrar: Fairness | Base técnica: referência 1.

10. Por que deploy piora p99?

Mediria startup real, JIT quando aplicável e abertura de conexões. Aqueceria recursos seguros antes da prontidão e preservaria margem, sem executar efeitos financeiros artificiais para acelerar o primeiro atendimento.

Termo para lembrar: Warm-up | Base técnica: referência 2.

11. Uma promise transforma cálculo pesado em paralelo?

Não. Promise organiza conclusão assíncrona; o cálculo continua consumindo a thread que o executa. Eu separaria CPU pesada em workers ou processos, com fila limitada e cancelamento definido.

Termo para lembrar: Async versus parallel | Base técnica: referência 1.

12. Hash de senha atrasa leitura de arquivos mesmo com event loop livre?

Eu investigaria contenção no worker pool do Node: operações como pbkdf2 e leitura de arquivos compartilham esses trabalhadores. Mediria espera na fila separadamente do event loop lag, limitaria concorrência de hashes e avaliaria isolamento. Aumentar threads sem medir pode apenas disputar mais CPU.

Termo para lembrar: Worker pool contention | Base técnica: referência 1.

13. Como distinguir vazamento de heap grande legítimo?

Compararia objetos retidos após ciclos equivalentes de trabalho e coleta. Crescimento contínuo de referências vivas sugere retenção indevida; heap alto estabilizado pode representar cache ou conjunto de trabalho necessário.

Termo para lembrar: Retention profiling | Base técnica: referência 1.

14. Thread virtual remove necessidade de pool no banco?

Não. Ela reduz custo de esperar, mas o banco continua tendo conexões e execução finitas. Eu limitaria chamadas concorrentes com semáforo e mediria espera antes de adquirir conexão.

Termo para lembrar: Virtual threads | Base técnica: referência 1.

15. Um collector de baixa pausa é sempre melhor?

Compararia pausas, CPU, memória e throughput com o SLO. O collector pode reduzir interrupções e exigir mais recursos; escolheria pela jornada medida, não somente pelo nome da tecnologia.

Termo para lembrar: GC selection | Base técnica: referência 1.

16. O correlation ID de uma requisição aparece no log de outra: como corrigir?

Eu removeria contexto de requisição de variáveis globais mutáveis. Usaria execution context isolation para propagar identidade e correlation ID por tarefa e callback, com limpeza no encerramento. Testaria requisições intercaladas, exceções e tarefas destacadas; um log atribuído ao cliente errado também compromete investigação e confidencialidade.

Termo para lembrar: Execution context isolation

17. Como evitar deadlock no executor de tarefas?

Não deixaria uma tarefa ocupar worker enquanto espera outra tarefa no mesmo pool saturado. Separaria dependências, usaria composição assíncrona ou reservaria executores apropriados, acompanhando tarefas bloqueadas.

Termo para lembrar: Thread starvation | Base técnica: referência 1.

18. O código usa mutex; como localizar contenção?

Coletaria perfil de bloqueio e duração da seção crítica. Depois reduziria trabalho protegido, retiraria I/O do lock e avaliaria particionamento do estado, mantendo os invariantes necessários.

Termo para lembrar: Lock contention | Base técnica: referência 1.

19. Por que serializar JSON pode causar incidente?

Payload grande exige CPU, aloca memória e pode bloquear execução compartilhada. Eu limitaria tamanho, projetaria campos necessários e consideraria streaming, medindo custo antes de aumentar recursos.

Termo para lembrar: Serialization cost | Base técnica: referência 1.

20. Quando usar processos separados para imagens?

Quando isolamento de memória, bibliotecas nativas ou CPU justificarem. Eu enviaria referência ao objeto, limitaria concorrência e trataria crash por job, evitando transportar arquivos enormes dentro da fila.

Termo para lembrar: Process isolation | Base técnica: referência 1.

21. Context switch aparece como CPU útil?

Consome CPU sem avançar diretamente o trabalho de negócio. Eu investigaria excesso de threads, bloqueios e competição por cores, comparando throughput e latência com diferentes limites de execução.

Termo para lembrar: Context switching | Base técnica: referência 1.

22. Como lidar com pause longa e lease vencida?

O processo pausado pode retomar sem saber que perdeu ownership. Eu validaria epoch no destino antes da escrita e preservaria idempotência, porque ajustar GC não garante exclusividade distribuída.

Termo para lembrar: Pause and fencing | Base técnica: referência 1.

23. Podemos usar objetos mutáveis compartilhados no cache local?

Somente com propriedade e sincronização claras. Eu preferiria valores imutáveis ou cópias controladas, para que uma requisição não altere informação observada por outra sem transição explícita.

Termo para lembrar: Shared mutability | Base técnica: referência 1.

24. Como impedir recursão ilimitada em entrada externa?

Limitaria profundidade e tamanho antes de processar, além de usar algoritmos iterativos quando apropriado. Testaria entradas adversas, porque exaustão de stack pode derrubar execução apesar de CPU disponível.

Termo para lembrar: Stack exhaustion | Base técnica: referência 1.

25. O event loop pode atender milhões de conexões?

Conexão e trabalho são dimensões diferentes. Eu estimaria memória por socket, buffers, descritores e processamento de mensagens; depois limitaria conexões e aplicaria backpressure para consumidores lentos.

Termo para lembrar: Connection budget | Base técnica: referência 1.

26. Por que memória do container excede heap configurado?

Além do heap existem stacks, bibliotecas nativas, buffers e outros consumos cobrados ao container. Eu reservaria margem e observaria RSS, evitando usar todo limite apenas para objetos gerenciados.

Termo para lembrar: Native memory | Base técnica: referência 1.

27. Quando pooling de objetos piora o sistema?

Quando retém grandes objetos, aumenta contenção ou dificulta ownership. Eu compararia alocações evitadas com memória retida e complexidade, preferindo remover o pool quando o runtime já lida bem.

Termo para lembrar: Object pooling | Base técnica: referência 1.

28. Como aquecer sem produzir efeitos reais?

Exercitaria conexões e caminhos seguros com requisições sintéticas controladas. Não enviaria pagamentos falsos a produção; readiness só liberaria tráfego após dependências e recursos essenciais estarem utilizáveis.

Termo para lembrar: Safe warmup | Base técnica: referência 1.

29. CPU alta sempre exige escalar instâncias?

Primeiro identificaria se há trabalho útil, spin, retries ou coleta excessiva. Escalar código ineficiente pode aumentar custo e pressão externa; corrigiria desperdício e mediria a nova curva de capacidade.

Termo para lembrar: Useful throughput | Base técnica: referência 1.

30. Como comparar runtimes de maneira justa?

Usaria o mesmo contrato, carga, limites e critérios de correção. Compararia custo por operação, caudas, memória e comportamento em falhas; um microbenchmark isolado não representa a aplicação bancária.

Termo para lembrar: Representative benchmark | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Event loop versus workers

Ganho. Loop atende I/O com pouca coordenação

Custo e motivo. CPU pesada precisa de isolamento e comunicação adicional

Como decidir. Quando a carga predominante é espera de I/O

02. Threads maiores versus pool limitado

Ganho. Mais tarefas podem progredir durante espera

Custo e motivo. Mais stacks, context switches e pressão no destino

Como decidir. Quando há capacidade externa e espera comprovada

03. Heap maior versus coleta mais frequente

Ganho. Pode reduzir frequência de GC

Custo e motivo. Consome memória e pode alterar pausas

Como decidir. Quando perfil demonstra pressão de coleta

04. Tarefa por pedido versus concorrência limitada

Ganho. Simplifica composição e isolamento lógico

Custo e motivo. Quantidade em voo pode consumir memória substancial

Como decidir. Quando tarefas são limitadas por admissão

05. Fila limitada versus buffer amplo

Ganho. Contém memória e explicita sobrecarga

Custo e motivo. Exige rejeição ou espera mais cedo

Como decidir. Quando capacidade é finita e contrato permite sinalizar pressão

06. I/O assíncrono versus bloqueante

Ganho. Reduz threads dedicadas à espera

Custo e motivo. Fluxo e bibliotecas podem ser mais complexos

Como decidir. Quando muitas chamadas ficam simultaneamente aguardando

07. Pooling versus alocação normal

Ganho. Reduz certas alocações e cópias

Custo e motivo. Retém memória e exige disciplina de ownership

Como decidir. Quando profiling demonstra ganho líquido

08. Cancelamento cooperativo versus conclusão independente

Ganho. Evita trabalho sem consumidor

Custo e motivo. Exige propagação e não revoga efeitos concluídos

Como decidir. Quando trabalho restante é dispensável e reversível

09. Isolamento por tenant versus pool compartilhado

Ganho. Melhora justiça e contenção de abuso

Custo e motivo. Pode deixar capacidade reservada ociosa

Como decidir. Quando clientes têm cargas e criticidades diferentes

10. Warm-up versus startup imediato

Ganho. Reduz latência das primeiras operações

Custo e motivo. Aumenta tempo até prontidão e custo inicial

Como decidir. Quando aquecimento domina a experiência após deploy

Duas alternativas ao caminho proposto

1. Workers de processos

Quando faz sentido. CPU intensa ou isolamento necessário

Como implementar. Separar tarefas pesadas com fila limitada

Limites. IPC e memória aumentam

2. Runtime com threads virtuais

Quando faz sentido. Muitas esperas bloqueantes em plataforma compatível

Como implementar. Usar tarefas leves e limitar dependências

Limites. Não aumenta CPU nem conexões disponíveis

Referências e alcance da evidência

1. Node.js: não bloquear o event loop

Event loop, worker pool e trabalho bloqueante. As respostas e parâmetros de aplicação são propostas autorais.

https://nodejs.org/en/learn/asynchronous-work/dont-block-the-event-loop

2. Go: garbage collector

Heap, GC, limites de memória e custos de execução. As respostas e parâmetros de aplicação são propostas autorais.

https://go.dev/doc/gc-guide

Continue em: T080 · T082

T082 / I. Performance e dados

Hardware, CPU, NUMA, SSD e page cache

Hardware determina limites físicos que abstrações de nuvem não eliminam. Arquitetura de treino; não descreve implementação interna do Nubank.

Quando usar. Ao investigar CPU, memória, disco ou rede saturados.

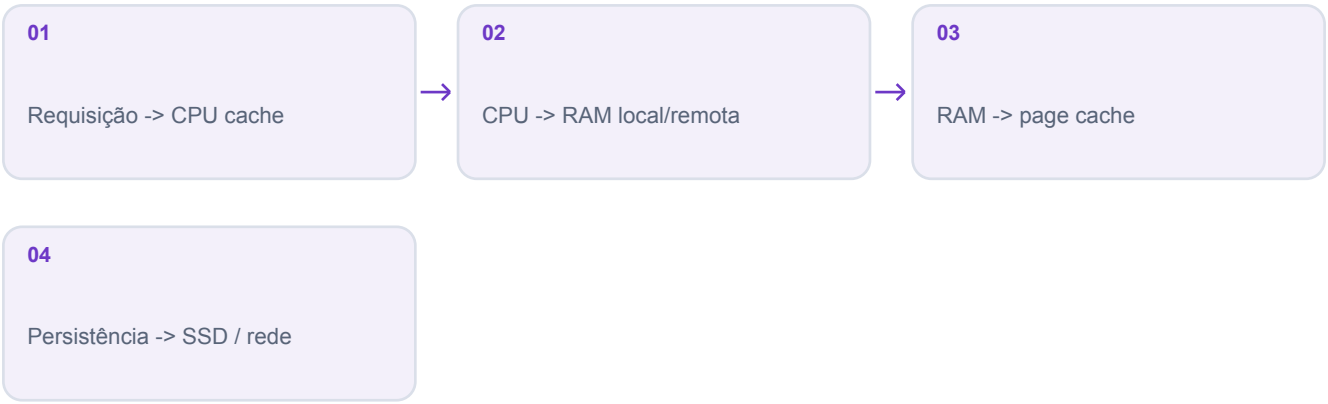
Quando evitar ou limitar. Otimização de baixo nível sem evidência do gargalo.

Uma abertura possível

Eu começaria por favorecer acesso contíguo em trechos críticos. Depois explicaria aproximar threads e memória após profiling, preservando as restrições do caso.

Peças e tecnologias. Linux · perf · numactl · fio · NVMe

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Favorecer acesso contíguo em trechos críticos; validar ciclos e misses por operação.

2. Aproximar threads e memória após profiling; validar comparação local versus memória remota.

3. Separar contadores quentes após medir; validar throughput com layouts diferentes.

4. Medir comportamento quente e frio; validar latência após reinício controlado.

5. Definir confirmação após persistência contratada; validar teste de falha e recuperação.

30 perguntas para sustentar o desenho

01. Por que array supera objetos encadeados?

Eu favoreceria acesso contíguo nos caminhos medidos como críticos, comparando cache misses e throughput. Isso pode exigir reorganizar estruturas; não sacrificaria clareza em trechos que não participam do gargalo.

Termo para lembrar: Cache locality | Base técnica: referência 1.

02. Dois sockets dobram a velocidade?

Verificaria localidade entre threads e memória antes de esperar ganho linear com sockets. Afinidade pode ajudar uma carga e restringir o scheduler; decidiria com medições da topologia utilizada.

Termo para lembrar: NUMA | Base técnica: referência 2.

03. Threads independentes ainda se atrapalham?

Investigaria linhas de cache compartilhadas por contadores atualizados independentemente. Padding ou reorganização pode reduzir invalidação, mas aumenta memória; compararia o layout com a mesma concorrência e volume.

Termo para lembrar: False sharing | Base técnica: referência 1.

04. Por que segunda leitura fica rápida?

Separaria benchmarks com cache quente e frio, observando leituras físicas e major faults. O segundo acesso rápido não demonstra capacidade após restart, migração ou mudança do conjunto de trabalho.

Termo para lembrar: Page cache | Base técnica: referência 2.

05. Gravar no arquivo garante durabilidade?

Definiria em qual confirmação o sistema promete durabilidade e verificaria o caminho completo de persistência. Escrever em buffer não equivale a sobreviver à falha; testes incluem queda e recuperação.

Termo para lembrar: Fsync | Base técnica: referência 1.

06. Como distinguir disco cheio de lento?

Mediria operações por segundo, tamanho, latência e profundidade de fila. Volume livre no disco não indica capacidade de I/O; muitos acessos pequenos podem saturar antes de atingir throughput anunciado.

Termo para lembrar: IOPS | Base técnica: referência 2.

07. CSV grande pede mais IOPS?

Para arquivos grandes, usaria leitura em fluxo e blocos adequados, comparando throughput sustentado e memória. Uma configuração boa para acessos aleatórios pequenos pode desperdiçar capacidade em varredura sequencial.

Termo para lembrar: Throughput de disco | Base técnica: referência 1.

08. Memória usada significa vazamento?

Eu separaria heap, memória nativa, páginas e cache recuperável. A decisão de aumentar RAM depende de pressão e impacto na latência, não apenas da porcentagem de memória que aparece ocupada.

Termo para lembrar: Memory pressure | Base técnica: referência 2.

09. Há CPU livre mas container trava?

Compararia quota efetiva do container e tempo de throttling com demanda por burst. CPU livre no host não significa CPU disponível ao processo; ajustaria limites somente após avaliar isolamento e custo.

Termo para lembrar: CPU throttling | Base técnica: referência 1.

10. Como provar gargalo de banda?

Observaria bytes, pacotes, retransmissões e processamento por conexão. A banda nominal é um limite parcial; payloads e padrões diferentes podem saturar CPU ou taxa de pacotes antes do enlace.

Termo para lembrar: Rede física | Base técnica: referência 2.

11. RAM livre baixa significa que preciso comprar memória?

Não necessariamente: o sistema pode usar memória disponível como cache recuperável. Eu observaria pressão, reclaim, swap e latência, separando uso saudável de cache da competição que prejudica processos.

Termo para lembrar: Memory pressure | Base técnica: referência 1.

12. SSD anuncia alto throughput; por que WAL continua lento?

WAL exige padrões de escrita e confirmação específicos. Eu mediria latência de sync e fila com durabilidade equivalente, pois throughput sequencial de marketing não representa cada commit transacional.

Termo para lembrar: Durable write latency | Base técnica: referência 1.

13. Quando direct I/O ajuda?

Quando a aplicação gerencia seu próprio cache e o padrão justifica evitar duplicação. Eu validaria alinhamento e comportamento de durabilidade, pois bypass da page cache pode piorar cargas comuns.

Termo para lembrar: Direct I/O | Base técnica: referência 1.

14. Que problema surge com swap num serviço de baixa latência?

Páginas necessárias podem exigir acesso lento ao disco. Eu mediria major faults e caudas, ajustaria working set e limites, evitando que o processo dependa de swap para atender normalmente.

Termo para lembrar: Major page faults | Base técnica: referência 1.

15. Por que compressão pode acelerar consulta mesmo usando CPU?

Se I/O ou rede são gargalos, reduzir bytes compensa o trabalho adicional. Eu compararia CPU, tamanho e tempo total com dados reais, incluindo conteúdo pouco compressível e concorrência.

Termo para lembrar: Compression trade-off | Base técnica: referência 1.

16. Como dimensionar armazenamento para muitos arquivos pequenos?

Consideraria operações, metadados e tamanho médio, além dos gigabytes. Agrupar arquivos pode reduzir custo de acesso, mas muda atualização e recuperação; validaria o padrão de consulta e retenção.

Termo para lembrar: Small-file overhead | Base técnica: referência 1.

17. NVMe elimina necessidade de índices?

Não. Ler menos páginas continua economizando CPU e I/O. Eu compararia plano, seletividade e working set; hardware rápido reduz certos custos, mas não corrige consultas com trabalho desnecessário.

Termo para lembrar: Work avoidance | Base técnica: referência 1.

18. Que efeito causa oversubscription de CPU?

Mais tarefas prontas disputam os mesmos cores, aumentando espera e trocas de contexto. Eu limitaria paralelismo útil e mediria fila de execução, distinguindo bloqueio por I/O de trabalho computacional.

Termo para lembrar: Run queue | Base técnica: referência 1.

19. Como saber se limite é bandwidth de memória?

Procuraria baixa eficiência de cache, volume transferido e comportamento ao aumentar threads. Se throughput estagnar sem saturar execução aritmética, avaliaria layout, reutilização e quantidade de dados percorridos.

Termo para lembrar: Memory bandwidth | Base técnica: referência 1.

20. O que significa dirty page para durabilidade?

É uma página modificada ainda pendente de escrita apropriada. Eu distinguiria aceitação pela aplicação, page cache e confirmação durável, alinhando a resposta de sucesso ao contrato do armazenamento.

Termo para lembrar: Dirty pages | Base técnica: referência 1.

21. Por que prefetch pode piorar desempenho?

Pode trazer dados que nunca serão usados e expulsar páginas úteis. Eu verificaria padrão sequencial e taxa de aproveitamento, evitando presumir que leitura antecipada beneficia acessos aleatórios.

Termo para lembrar: Prefetch | Base técnica: referência 1.

22. Como testar disco sem destruir dados de produção?

Usaria ambiente ou volume dedicado com dados descartáveis, configuração equivalente e limites. Confirmaria que o teste não compartilha recursos críticos antes de medir escrita, sync e concorrência.

Termo para lembrar: Safe storage benchmark | Base técnica: referência 1.

23. Quando escolher mais cores em vez de frequência maior?

Depende do paralelismo real. Eu avaliaria fração serial, contenção e particionamento; uma transação sequencial pode beneficiar frequência, enquanto trabalhos independentes aproveitam vários cores com menos coordenação.

Termo para lembrar: Parallel fraction | Base técnica: referência 1.

24. O que é write amplification no SSD?

O dispositivo pode escrever mais dados físicos que a alteração lógica solicitada. Eu consideraria padrão de acesso, manutenção e durabilidade, observando latência sustentada e desgaste, não apenas bursts curtos.

Termo para lembrar: Write amplification | Base técnica: referência 1.

25. Por que o primeiro acesso depois de deploy fica lento?

Caches, páginas e conexões podem estar frios. Eu mediria a trajetória de aquecimento e preservaria margem operacional; copiar resultados de benchmark aquecido esconderia a capacidade durante recuperação.

Termo para lembrar: Cold-start working set | Base técnica: referência 1.

26. Como NUMA afeta containers?

Container não elimina topologia física. Threads e memória podem atravessar nós NUMA; eu mediria localidade e políticas disponíveis antes de fixar afinidade, preservando flexibilidade do scheduler quando suficiente.

Termo para lembrar: NUMA placement | Base técnica: referência 1.

27. O que pode limitar rede além de gigabits por segundo?

Taxa de pacotes, retransmissões, conexões e processamento também limitam vazão. Eu observaria tamanhos e custos por pacote, evitando concluir que banda ociosa significa ausência de gargalo de rede.

Termo para lembrar: Packet rate | Base técnica: referência 1.

28. Como ligar hardware ao SLO?

Traduziria recursos em capacidade medida da jornada: commits por segundo, cauda e recuperação. Manteria folga para falhas previstas, porque especificação de máquina não equivale a garantia de atendimento.

Termo para lembrar: Capacity evidence | Base técnica: referência 1.

29. Um disco replicado substitui backup?

Não. Replicação pode propagar exclusão ou corrupção lógica. Eu manteria recuperação pontual e testes de restauração, separando disponibilidade física da capacidade de voltar a um estado válido.

Termo para lembrar: Replication versus backup | Base técnica: referência 1.

30. Pode migrar para ARM sem testar nada?

Eu verificaria bibliotecas nativas, imagens, precisão e comportamento de desempenho. Depois compararia custo e caudas com a mesma carga, incluindo caminhos raros e ferramentas operacionais usadas pela equipe.

Termo para lembrar: Architecture portability | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Estruturas contíguas versus encadeadas

Ganho. Melhor localidade e menos indireção

Custo e motivo. Inserções e estabilidade de referências podem custar mais

Como decidir. Quando leitura sequencial domina

02. Afinidade NUMA versus scheduler livre

Ganho. Reduz acesso remoto de memória

Custo e motivo. Restringe balanceamento e aumenta configuração

Como decidir. Quando topologia e perfil mostram benefício

03. Padding versus memória compacta

Ganho. Reduz false sharing em escrita concorrente

Custo e motivo. Aumenta footprint e pode piorar cache global

Como decidir. Quando contadores quentes compartilham cache line

04. Page cache versus cache próprio

Ganho. Reaproveita mecanismos do sistema operacional

Custo e motivo. Controle fino e dupla retenção podem ser limitados

Como decidir. Quando acesso comum é bem atendido pelo kernel

05. Sync por operação versus group commit

Ganho. Reduz latência adicional de espera do lote

Custo e motivo. Mais confirmações podem reduzir throughput

Como decidir. Quando prazo individual justifica custo de persistência

06. Disco de mais IOPS versus maior throughput

Ganho. Atende acessos pequenos e concorrentes

Custo e motivo. Preço e configuração podem não beneficiar scans

Como decidir. Quando o workload é limitado por operações

07. Blocos grandes versus pequenos

Ganho. Amortiza overhead em leitura sequencial

Custo e motivo. Aumenta read amplification em consultas pequenas

Como decidir. Quando dados são consumidos em fluxo

08. Mais RAM versus tiering

Ganho. Mantém working set perto da CPU

Custo e motivo. Custo maior e limite físico por nó

Como decidir. Quando acesso frequente justifica memória residente

09. Quota rígida versus burst permitido

Ganho. Melhora isolamento entre workloads

Custo e motivo. Pode elevar cauda durante picos legítimos

Como decidir. Quando previsibilidade de compartilhamento é prioritária

10. Compressão versus bytes brutos

Ganho. Reduz I/O, storage e rede

Custo e motivo. Consome CPU e adiciona processamento

Como decidir. Quando bytes são o gargalo e conteúdo comprime bem

Duas alternativas ao caminho proposto

1. Escala vertical

Quando faz sentido. Conjunto de trabalho cabe num host maior

Como implementar. Aumentar recursos e medir a mesma carga

Limites. Domínio de falha permanece concentrado

2. Particionamento horizontal

Quando faz sentido. Carga excede um host bem utilizado

Como implementar. Dividir dados com ownership explícito

Limites. Coordenação e rebalanceamento custam

Referências e alcance da evidência

1. Linux: conceitos de memória

Memória virtual, páginas e cache. As respostas e parâmetros de aplicação são propostas autorais.

https://docs.kernel.org/admin-guide/mm/concepts.html

2. Linux: NUMA

Localidade entre processadores e memória. As respostas e parâmetros de aplicação são propostas autorais.

https://docs.kernel.org/mm/numa.html

Continue em: T081 · T083

T083 / I. Performance e dados

Algoritmos e estruturas de dados

Estruturas de dados conectam padrões de acesso a custo de CPU, memória e I/O. Arquitetura de treino; não descreve implementação interna do Nubank.

Quando usar. Ao justificar índices, filas, deduplicação e consultas.

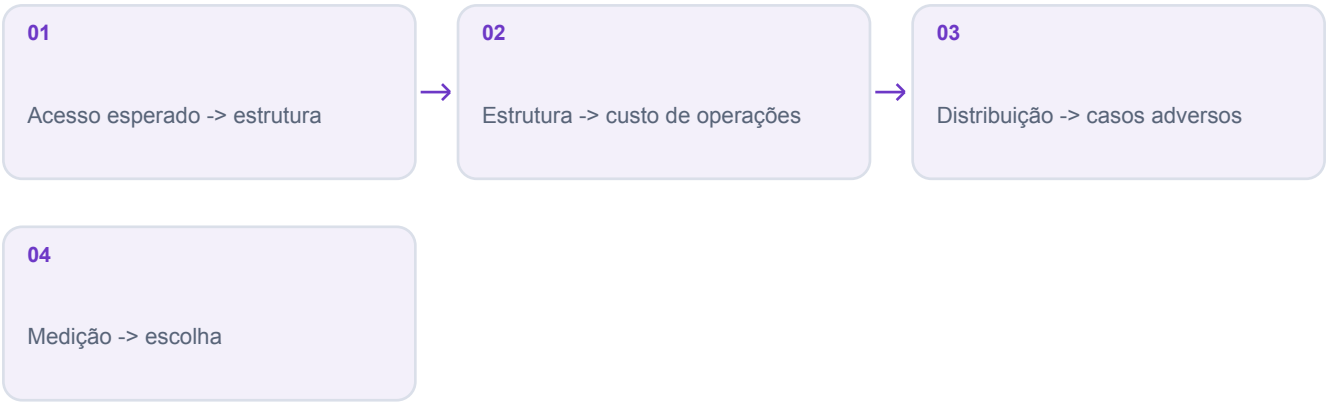
Quando evitar ou limitar. Escolher complexidade assintótica ignorando volume e constantes.

Uma abertura possível

Eu começaria por distinguir custo esperado e pior caso. Depois explicaria manter min-heap com desempate estável, preservando as restrições do caso.

Peças e tecnologias. Hash map · Heap · B-tree · Trie · Bloom filter

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Distinguir custo esperado e pior caso; validar latência com chaves adversas.

2. Manter min-heap com desempate estável; validar ordem após cancelamentos e inserções.

3. Usar ordenação compatível com consulta; validar plano e páginas examinadas.

4. Considerar compaction e amplificação total; validar vazão durante manutenção.

5. Confirmar positivos na fonte autoritativa; validar taxa de falsos positivos.

30 perguntas para sustentar o desenho

01. Busca O(1) é garantida sempre?

Eu explicaria que O(1) costuma ser expectativa, não promessa universal para hash tables. Testaria colisões, crescimento e distribuição, incluindo custos de resize e memória antes de usá-la num caminho sensível.

Termo para lembrar: Hash table | Base técnica: referência 1.

02. Como escolher próximo job por prazo?

Usaria min-heap para retirar o próximo prazo, com desempate estável. Cancelamentos e mudanças de prioridade exigem índice auxiliar ou entradas inválidas controladas; o desenho precisa limitar resíduos acumulados.

Termo para lembrar: Heap | Base técnica: referência 2.

03. Por que índice ajuda intervalos?

Escolheria índice ordenado para igualdade e intervalos compatíveis com o predicado. Validaria o plano e seletividade, lembrando que cada índice também consome armazenamento e manutenção durante as escritas.

Termo para lembrar: B-tree | Base técnica: referência 1.

04. Por que escrita rápida pode piorar leitura?

Avaliaria custo total de escrita, leitura e compaction, não apenas ingestão inicial. LSM pode favorecer determinadas escritas, mas exige espaço e manutenção que afetam a cauda sob carga sustentada.

Termo para lembrar: LSM | Base técnica: referência 2.

05. Posso rejeitar cadastro pelo filtro?

Eu não rejeitaria cliente por um positivo no Bloom filter. Confirmaria na fonte exata; a estrutura reduz consultas em cenários apropriados, mas sua probabilidade de falso positivo precisa ser conhecida.

Termo para lembrar: Bloom filter | Base técnica: referência 1.

06. Autocomplete exige varrer todas palavras?

Para prefixos, avaliaria trie ou índice especializado com normalização consistente. Compararia memória, atualização e idiomas; uma estrutura rápida em um vocabulário pequeno pode ficar cara num corpus extenso.

Termo para lembrar: Trie | Base técnica: referência 2.

07. Como calcular picos numa janela?

Definiria precisão da janela e usaria buckets ou eventos conforme necessidade. Testaria fronteiras temporais, porque aproximações podem ser aceitáveis para métricas e inadequadas para um limite financeiro estrito.

Termo para lembrar: Sliding window | Base técnica: referência 1.

08. Como comparar arquivos maiores que RAM?

Ordenaria arquivos em partes que caibam na memória e faria merge em fluxo. Planejaria disco temporário e retomada; custo de I/O importa tanto quanto a complexidade de comparação nesse volume.

Termo para lembrar: Sort merge | Base técnica: referência 2.

09. Preciso ordenar milhões para mostrar dez?

Manteria heap de tamanho k quando não há índice apropriado entregando a ordem. Isso reduz custo de ordenação, mas ainda exige avaliar candidatos elegíveis e preservar desempate determinístico no resultado.

Termo para lembrar: Top-k | Base técnica: referência 1.

10. Uma operação rápida pode travar ocasionalmente?

Eu distinguiria custo de uma operação e custo amortizado numa sequência. Resize ou manutenção pode produzir picos raros; para SLO de cauda, mediria esses momentos e não apenas a média.

Termo para lembrar: Complexidade amortizada | Base técnica: referência 2.

11. Quando um array supera uma lista encadeada?

Quando acesso contíguo e leitura sequencial dominam. Eu incluiria localidade de cache e alocação na comparação; complexidade assintótica igual não implica o mesmo tempo ou consumo de memória.

Termo para lembrar: Data locality | Base técnica: referência 1.

12. Como remover um job arbitrário do heap?

Manteria um índice auxiliar ou usaria cancelamento lógico com limpeza. A escolha depende da frequência de remoções; verificaria crescimento de entradas inválidas para não criar consumo de memória ilimitado.

Termo para lembrar: Indexed heap | Base técnica: referência 1.

13. Hash map mantém ordem de inserção universalmente?

Isso depende da implementação e contrato. Eu não basearia ordenação financeira em comportamento acidental; manteria sequência explícita ou estrutura ordenada, com desempate estável entre eventos simultâneos.

Termo para lembrar: Ordering contract | Base técnica: referência 1.

14. Como encontrar duplicatas sem guardar tudo em RAM?

Particionaria por hash ou ordenaria externamente, depois compararia identidades na mesma partição. Preservaria confirmação exata quando falsa duplicata puder descartar uma operação financeira real e legítima.

Termo para lembrar: External deduplication | Base técnica: referência 1.

15. Bloom filter aceita remoção simples de bits?

Não no filtro convencional: o bit pode representar outros elementos. Eu reconstruiria o filtro ou avaliaria variante com contadores, considerando memória e segurança da política de remoção.

Termo para lembrar: Counting Bloom filter | Base técnica: referência 1.

16. Como detectar ciclo entre dependências de jobs antes de executá-los?

Eu modelaria jobs como vértices e dependências como arestas. Aplicaria ordenação topológica de Kahn em O(V+E); se não consumir todos os vértices, existe ciclo. Rejeitaria o plano e identificaria as dependências conflitantes, em vez de deixar workers aguardando indefinidamente tarefas que nunca ficam elegíveis.

Termo para lembrar: Topological sort / algoritmo de Kahn

17. Como evitar ataque por colisão de hash?

Usaria implementações resistentes adequadas à linguagem, limites de entrada e observabilidade. Também testaria distribuição adversa, porque a complexidade esperada não deve ser a única defesa contra abuso.

Termo para lembrar: Hash collision defense | Base técnica: referência 1.

18. Qual custo tem ordenar N registros?

Para comparação geral, considero tipicamente N log N, mas também memória e I/O. Se ultrapassar RAM, planejo runs e merge externo, medindo espaço temporário e tempo de recuperação.

Termo para lembrar: External sort | Base técnica: referência 1.

19. Como fazer merge de várias sequências ordenadas?

Usaria um heap com a cabeça de cada sequência, emitindo o menor e avançando sua origem. A memória depende das sequências abertas, e o custo por saída depende dessa quantidade.

Termo para lembrar: K-way merge | Base técnica: referência 1.

20. Um contador aproximado serve para cobrar cliente?

Só se o contrato aceitar explicitamente a aproximação, o que normalmente não assumiria. Para cobrança, manteria eventos verificáveis ou contadores exatos e reconciliação; aproximações servem melhor à observabilidade.

Termo para lembrar: Approximate counting | Base técnica: referência 1.

21. Como manter a mediana exata de valores recebidos sem ordenar todo o histórico?

Eu manteria dois heaps: máximo na metade inferior e mínimo na superior, balanceados após cada inserção. Isso permite inserção O(log n) e consulta O(1), mas armazena O(n) valores do prefixo. Para janela deslizante, precisaria também suportar exclusões; a solução somente append não basta.

Termo para lembrar: Two-heaps running median | Base técnica: referência 1.

22. O que muda numa fila circular limitada?

A capacidade é fixa e exige política quando cheia: bloquear, rejeitar ou descartar conforme semântica. Eu nunca sobrescreveria silenciosamente comandos financeiros ainda não processados para manter throughput aparente.

Termo para lembrar: Ring buffer | Base técnica: referência 1.

23. Trie cresce demais: quais saídas existem?

Avaliaria compressão de prefixos, representação compacta e persistência especializada. Mediria memória por termo e latência de atualização; otimizar apenas a busca pode tornar ingestão e operação muito caras.

Termo para lembrar: Radix tree | Base técnica: referência 1.

24. Como comparar dois datasets sem depender da ordem original?

Normalizaria chaves e compararia conjuntos ou sequências ordenadas. Incluiria multiplicidade, porque transformar tudo em conjunto pode esconder duplicatas que são exatamente o problema de reconciliação procurado.

Termo para lembrar: Multiset comparison | Base técnica: referência 1.

25. O que é algoritmo online nesse contexto?

Processa entradas incrementais sem exigir todo o conjunto disponível. Eu avaliaria memória de estado e exatidão; agregações online podem necessitar revisões quando eventos atrasados mudam resultados previamente publicados.

Termo para lembrar: Online algorithm | Base técnica: referência 1.

26. Uma estrutura lock-free é automaticamente mais rápida?

Não. Contenção, retries de CAS e reclamation de memória podem custar caro. Eu compararia implementação simples protegida por lock com a alternativa usando carga real e testes de correção concorrente.

Termo para lembrar: Lock-free algorithms | Base técnica: referência 1.

27. Como tratar empate de prioridade?

Acrescentaria critério estável, como sequência de aceitação, conforme a política. Isso facilita previsibilidade e testes; também usaria envelhecimento quando prioridades fixas puderem impedir atendimento de tarefas antigas.

Termo para lembrar: Stable priority | Base técnica: referência 1.

28. Como provar que otimização manteve resultado?

Compararia a implementação otimizada com uma referência simples em entradas pequenas geradas, incluindo extremos. Testaria invariantes e determinismo antes de usar benchmarks como evidência de melhoria válida.

Termo para lembrar: Reference implementation | Base técnica: referência 1.

29. Qual problema existe com remoção em array?

Pode exigir deslocamento de elementos e invalidar posições. Eu avaliaria tombstones, troca com último ou estrutura diferente conforme a necessidade de preservar ordem e a frequência real de remoção.

Termo para lembrar: Deletion cost | Base técnica: referência 1.

30. O que você diria sobre O(1) no banco remoto?

Que complexidade local não resume latência de rede, serialização, disco e contenção. Eu explicaria número de acessos e tamanho dos dados, medindo a operação completa sob concorrência representativa.

Termo para lembrar: End-to-end complexity | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Hash versus árvore ordenada

Ganho. Igualdade costuma ter custo esperado menor

Custo e motivo. Não oferece navegação ordenada equivalente

Como decidir. Quando igualdade domina e pior caso é controlado

02. Heap versus lista ordenada

Ganho. Inserir e retirar prioridade têm bom custo

Custo e motivo. Busca arbitrária e atualização precisam de apoio

Como decidir. Quando prioridade é a operação principal

03. Índice extra versus menos índices

Ganho. Acelera padrão específico de leitura

Custo e motivo. Aumenta escrita, espaço e manutenção

Como decidir. Quando economia de leitura supera amplificação

04. LSM versus B-tree

Ganho. Pode amortizar ingestão sequencial de escrita

Custo e motivo. Compaction e amplificação afetam recursos

Como decidir. Quando perfil de escrita justifica manutenção

05. Bloom filter versus conjunto exato

Ganho. Usa menos memória para teste de associação

Custo e motivo. Positivos exigem confirmação exata

Como decidir. Quando evitar consultas negativas gera valor

06. Trie versus hash de termos

Ganho. Permite navegar por prefixo

Custo e motivo. Memória e atualização podem crescer muito

Como decidir. Quando autocomplete é requisito central

07. Buckets versus eventos individuais

Ganho. Reduz estado e custo de agregação

Custo e motivo. Perde precisão nas bordas temporais

Como decidir. Quando erro de aproximação é aceitável

08. Sort-merge versus hash join

Ganho. Permite processar dados maiores que RAM

Custo e motivo. Ordenação externa exige I/O temporário

Como decidir. Quando conjunto não cabe na memória

09. Heap top-k versus ordenação completa

Ganho. Reduz CPU e memória para k pequeno

Custo e motivo. Não entrega ordem completa dos demais itens

Como decidir. Quando apenas melhores candidatos importam

10. Amortização versus custo previsível por operação

Ganho. Melhora custo médio de sequências

Custo e motivo. Operações ocasionais podem ser caras

Como decidir. Quando cauda tolera manutenção ou ela pode ser controlada

Duas alternativas ao caminho proposto

1. Biblioteca padrão

Quando faz sentido. Problema não exige estrutura especializada

Como implementar. Usar implementação testada com benchmarks

Limites. Não cobre toda distribuição de carga

2. Índice no banco

Quando faz sentido. Consulta e consistência pertencem à persistência

Como implementar. Modelar índice pelo predicado real

Limites. Escritas e espaço ficam mais caros

Referências e alcance da evidência

1. Python: heapq

Fila de prioridade e operações sobre heap. As respostas e parâmetros de aplicação são propostas autorais.

https://docs.python.org/3/library/heapq.html

2. PostgreSQL: tipos de índice

Estruturas de índice e tipos de consulta. As respostas e parâmetros de aplicação são propostas autorais.

https://www.postgresql.org/docs/current/indexes-types.html

Continue em: T082 · T084

T084 / I. Performance e dados

Escalabilidade e autoscaling

Escala é aumentar capacidade útil sem violar consistência ou saturar dependências. Arquitetura de treino; não descreve implementação interna do Nubank.

Quando usar. Quando carga cresce, oscila ou possui picos previsíveis.

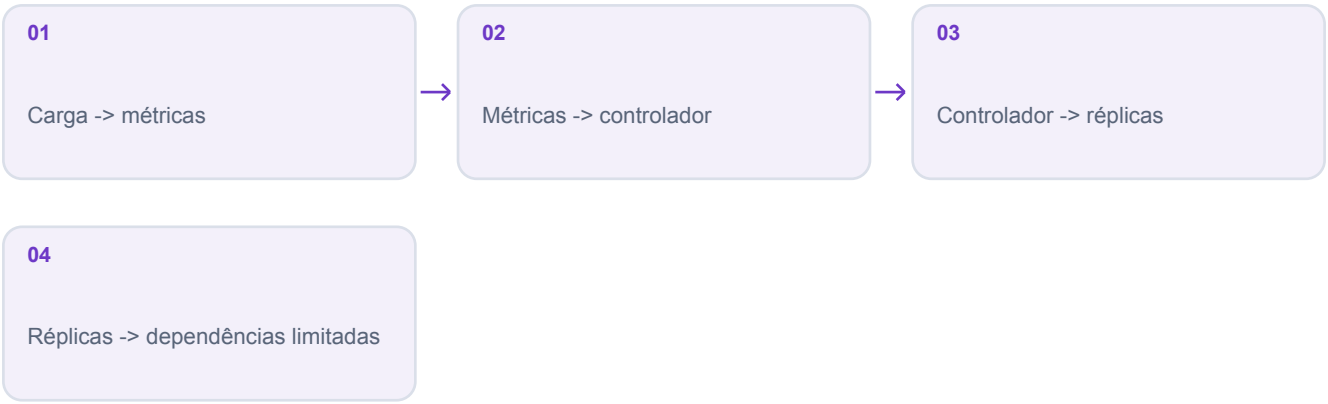
Quando evitar ou limitar. Aumentar réplicas para esconder banco saturado.

Uma abertura possível

Eu começaria por escalar pelo trabalho pendente quando apropriado. Depois explicaria medir prontidão e manter reserva mínima, preservando as restrições do caso.

Peças e tecnologias. Kubernetes HPA · KEDA · EC2 Auto Scaling · ECS

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Escalar pelo trabalho pendente quando apropriado; validar idade e tamanho do backlog.

2. Medir prontidão e manter reserva mínima; validar tempo real até capacidade útil.

3. Drenar tarefas e encerrar conexões controladamente; validar recuperação após encerramento.

4. Aplicar estabilização e passos limitados; validar frequência de mudanças por hora.

5. Impor orçamento global de conexões; validar conexões com máximo de réplicas.

30 perguntas para sustentar o desenho

01. CPU baixa impede autoscaling útil?

Eu escolheria métrica ligada ao trabalho e recurso limitante. Worker bloqueado em I/O pode ter CPU baixa e backlog crescente; observaria idade, vazão e capacidade do destino antes de aumentar réplicas.

Termo para lembrar: Métrica de escala | Base técnica: referência 1.

02. Novas réplicas chegam tarde ao pico?

Mediria tempo até uma réplica estar realmente pronta, incluindo imagem, startup e conexões. Para picos mais rápidos que essa reação, reservaria capacidade ou anteciparia escala, com admissão protegendo o restante.

Termo para lembrar: Startup | Base técnica: referência 2.

03. Como reduzir sem perder mensagens?

Retiraria a instância de novas atribuições e drenaria tarefas dentro do prazo. O que não terminar precisa ser recuperável; identidade do efeito continua protegendo uma execução interrompida em ponto ambíguo.

Termo para lembrar: Downscale | Base técnica: referência 1.

04. Por que réplicas ficam oscilando?

Aplicaria estabilização e limites de variação, verificando se a métrica representa mudança sustentada. Oscilar réplicas desperdiça aquecimento e pode causar rebalanceamentos; compararia tempo de resposta com estabilidade operacional.

Termo para lembrar: Hysteresis | Base técnica: referência 2.

05. Dez réplicas dobram conexões permitidas?

Calcularia conexões no máximo de réplicas, incluindo sobreposição de deploy. Limitaria pools e concorrência global; aumentar processos não cria capacidade no banco e pode apenas multiplicar sua pressão.

Termo para lembrar: Banco compartilhado | Base técnica: referência 1.

06. Mais consumidores sempre aceleram Kafka?

Eu verificaria partições e distribuição das chaves. Consumidores além do paralelismo disponível não aumentam vazão, e uma chave quente pode continuar limitando a jornada mesmo com muitas instâncias ociosas.

Termo para lembrar: Partições | Base técnica: referência 2.

07. Scale out pode falhar na nuvem?

Mapearia quotas, capacidade regional e recursos dependentes antes do pico. Alertaria sobre aproximação dos limites e ensaiaria expansão, porque configuração de autoscaling não garante que a infraestrutura poderá ser criada.

Termo para lembrar: Quotas | Base técnica: referência 1.

08. Como impedir escala ilimitada?

Definiria máximo de escala e comportamento quando ele é atingido, ligando custo a SLO e admissão. Um teto financeiro é uma restrição real; o produto precisa saber qual demanda será priorizada.

Termo para lembrar: Custo | Base técnica: referência 2.

09. Uma partição lenta melhora com réplicas?

Investigaria a semântica da chave quente antes de repartir dados. Salting pode ajudar agregações decomponíveis, mas quebrar ordem ou limite global; a correção determina quais paralelismos são realmente possíveis.

Termo para lembrar: Hot shard | Base técnica: referência 1.

10. Como drenar depois da queda?

Estimaria capacidade líquida acima da chegada normal e o prazo de drenagem. Se processamento apenas iguala entrada, backlog antigo permanece; reservaria recursos sem exceder quota ou saturar dependências na recuperação.

Termo para lembrar: Recovery capacity | Base técnica: referência 2.

11. Como calcular réplicas para absorver demanda?

Mediria throughput sustentável por réplica dentro do SLO, dividiria a carga de pico por essa capacidade e adicionaria margem para falhas. Repetiria o teste com dependências compartilhadas presentes.

Termo para lembrar: Capacity planning | Base técnica: referência 1.

12. Adicionar um sidecar fez o HPA escalar sem aumentar tráfego: por quê?

Eu compararia consumo e requests por container. A métrica de CPU do pod pode incluir o sidecar, e mudar requests altera o denominador da utilização. Avaliaria ContainerResource para medir o container relevante, mantendo capacidade explícita para o auxiliar; crescimento da métrica não prova crescimento de demanda.

Termo para lembrar: HPA ContainerResource / utilization denominator | Base técnica: referência 1.

13. Que métrica usar para worker de documentos?

Combinaria idade do job mais antigo, backlog e taxa de conclusão. Contagem sozinha ignora tamanhos diferentes; estimaria trabalho pendente quando arquivos possuem custos muito desiguais de processamento.

Termo para lembrar: Work-based scaling | Base técnica: referência 1.

14. HPA e VPA podem ajustar CPU e réplicas ao mesmo tempo sem coordenação?

Eu definiria responsabilidade de cada controlador. Se o VPA altera requests e o HPA usa utilização percentual de CPU, muda o denominador que orienta réplicas. Usaria sinais independentes ou uma estratégia coordenada, validando convergência e estabilidade; dois control loops podem disputar o mesmo objetivo.

Termo para lembrar: Control loop interaction

15. O que acontece quando cluster não tem nós livres?

O controlador pode solicitar pods que continuam pendentes. Eu monitoraria capacidade efetivamente pronta e a expansão de nós, mantendo limites e margem compatíveis com o tempo de provisionamento.

Termo para lembrar: Pending capacity | Base técnica: referência 1.

16. Scaling deve contar pods ou operações concluídas?

Pods são um meio. Eu observaria capacidade útil e resultado dentro do prazo; réplicas novas podem estar aquecendo ou bloqueadas no banco, sem aumentar throughput percebido pelos usuários.

Termo para lembrar: Useful capacity | Base técnica: referência 1.

17. CPU pede quatro réplicas e outra métrica pede dez: o que o HPA faz?

Eu compararia recomendações de réplicas, não unidades brutas: o HPA escolhe a maior recomendação, respeitando limites e políticas. Uma métrica indisponível pode impedir redução, enquanto uma válida ainda pode justificar aumento. Exporia o cálculo por métrica para distinguir proteção conservadora de capacidade realmente insuficiente.

Termo para lembrar: Multi-metric HPA recommendation | Base técnica: referência 1.

18. E se métrica de autoscaling desaparecer?

Definiria comportamento conservador e alertaria a perda de observabilidade. Não reduziria agressivamente por falta de dados; manteria limites de segurança enquanto investigo a coleta e a capacidade real.

Termo para lembrar: Metric failure | Base técnica: referência 1.

19. Quando preditivo é melhor que reativo?

Quando existe sazonalidade estável e o recurso demora a iniciar. Eu manteria correção reativa e verificaria erro da previsão, pois campanha inesperada ou incidente pode quebrar padrões históricos.

Termo para lembrar: Predictive scaling | Base técnica: referência 1.

20. Como medir custo de uma réplica adicional?

Compararia aumento de throughput útil com computação, conexões, cache e tráfego gerados. A decisão pode ser otimizar o gargalo compartilhado, porque réplicas adicionais nem sempre reduzem custo por operação.

Termo para lembrar: Marginal capacity cost | Base técnica: referência 1.

21. Posso escalar consumidores durante rebalanceamento contínuo?

Com cuidado: mudanças frequentes podem aumentar pausas e movimentação de trabalho. Eu aplicaria estabilização e observaria lag por partição, evitando reagir a cada oscilação pequena do backlog.

Termo para lembrar: Consumer churn | Base técnica: referência 1.

22. Como lidar com parceiro que aceita cem chamadas por segundo?

Aplicaria limitação global ou particionada coerente com o contrato. Escalar workers não aumenta quota; usaria filas duráveis, prioridades e cálculo do tempo de drenagem para prometer prazos realistas.

Termo para lembrar: Downstream quota | Base técnica: referência 1.

23. Aumentar CPU resolve hot key serializada?

Só até a capacidade daquele caminho e sua contenção. Eu investigaria invariantes por chave e possíveis agregações ou divisão de ownership, evitando particionar uma decisão que precisa permanecer única.

Termo para lembrar: Serial bottleneck | Base técnica: referência 1.

24. Como reservar capacidade para desastre?

Dimensionaria o ambiente sobrevivente para carga essencial e recuperação, testando a perda prevista. Capacidade nominal distribuída não significa que uma zona restante consiga assumir todo tráfego imediatamente.

Termo para lembrar: Failover capacity | Base técnica: referência 1.

25. Spot serve para todos workers?

Usaria onde interrupção e retomada são seguras, com checkpoints e identidade de efeito. Para prazos rígidos, reservaria capacidade estável suficiente; menor preço não compensa perder a janela de negócio.

Termo para lembrar: Interruptible capacity | Base técnica: referência 1.

26. Downscale pode remover cache quente demais?

Pode elevar misses e pressionar a origem. Eu mediria efeitos indiretos, reduziria gradualmente e manteria capacidade mínima quando o aquecimento for caro, especialmente após incidentes ou picos previsíveis.

Termo para lembrar: Cache-aware scaling | Base técnica: referência 1.

27. Como evitar que um tenant cause escala cara para todos?

Aplicaria quotas e isolamento conforme o produto, observando custo e uso por tenant. Uma política global sem atribuição permite que abuso ou carga desbalanceada consuma o orçamento compartilhado.

Termo para lembrar: Tenant fairness | Base técnica: referência 1.

28. CPU target deve ser cem por cento?

Normalmente reservaria margem determinada por testes, porque operar próximo da saturação piora filas e caudas. O valor depende de variabilidade, startup e SLO; não adotaria um percentual universal.

Termo para lembrar: Headroom | Base técnica: referência 1.

29. Qual diferença entre escalabilidade e elasticidade?

Escalabilidade é sustentar mais trabalho com mais recursos; elasticidade ajusta recursos à demanda. Eu testaria ambas: crescer corretamente e reagir no tempo necessário, sem pressupor que um controlador cria escalabilidade.

Termo para lembrar: Elasticity | Base técnica: referência 1.

30. Como validar política antes de produção?

Reproduziria rampas, bursts, falhas e recuperação com limites reais. Registraria decisões do controlador, capacidade pronta e SLO, comparando com configuração fixa para demonstrar benefício e identificar oscilação.

Termo para lembrar: Scaling experiment | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. CPU target versus métrica de trabalho

Ganho. CPU é simples e amplamente disponível

Custo e motivo. Pode ignorar espera e backlog de I/O

Como decidir. Quando CPU acompanha capacidade útil

02. Reserva mínima versus escala a zero

Ganho. Reduz espera de startup

Custo e motivo. Mantém custo ocioso

Como decidir. Quando reação precisa ser mais rápida que provisionamento

03. Drenagem longa versus desligamento rápido

Ganho. Reduz tarefas interrompidas

Custo e motivo. Retarda deploy e redução de custo

Como decidir. Quando tarefas são curtas e vale concluí-las

04. Estabilização versus reação imediata

Ganho. Reduz oscilação e churn

Custo e motivo. Pode responder mais lentamente a mudança real

Como decidir. Quando a métrica é ruidosa

05. Pools menores versus conexões abundantes

Ganho. Protege banco no máximo de réplicas

Custo e motivo. Pode aumentar espera local

Como decidir. Quando dependência compartilhada é o limite

06. Mais partições versus poucas partições

Ganho. Aumenta paralelismo e distribuição

Custo e motivo. Eleva overhead e complexidade de rebalanceamento

Como decidir. Quando consumidores precisam de mais lanes independentes

07. Capacidade pré-reservada versus on-demand

Ganho. Melhora previsibilidade em picos

Custo e motivo. Pode custar mais quando ociosa

Como decidir. Quando disponibilidade de expansão é crítica

08. Teto rígido versus expansão ampla

Ganho. Contém gasto e pressão externa

Custo e motivo. Exige rejeitar ou adiar demanda excedente

Como decidir. Quando orçamento e quota são limites obrigatórios

09. Repartição de chave versus serialização

Ganho. Pode aumentar throughput por entidade agregada

Custo e motivo. Complica ordem e invariantes globais

Como decidir. Quando a semântica permite decomposição

10. Folga de recuperação versus utilização máxima

Ganho. Permite drenar após falha

Custo e motivo. Aumenta capacidade normalmente ociosa

Como decidir. Quando backlog precisa desaparecer dentro de prazo

Duas alternativas ao caminho proposto

1. Capacidade fixa com folga

Quando faz sentido. Picos pequenos e previsíveis

Como implementar. Reservar margem e limitar admissão

Limites. Custo ocioso maior

2. Escala agendada

Quando faz sentido. Horários conhecidos de fechamento

Como implementar. Aquecer recursos antes da janela

Limites. Não responde sozinha a eventos inesperados

Referências e alcance da evidência

1. Kubernetes: HPA

Controle de réplicas a partir de métricas. As respostas e parâmetros de aplicação são propostas autorais.

https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/

2. AWS: dynamic scaling

Políticas de escalabilidade dinâmica. As respostas e parâmetros de aplicação são propostas autorais.

https://docs.aws.amazon.com/autoscaling/ec2/userguide/as-scale-based-on-demand.html

Continue em: T083 · T085

T085 / I. Performance e dados

Little’s Law, tail latency e profiling

Filas e caudas de latência explicam por que médias podem esconder uma experiência ruim. Arquitetura de treino; não descreve implementação interna do Nubank.

Quando usar. Para dimensionar concorrência e localizar o caminho crítico.

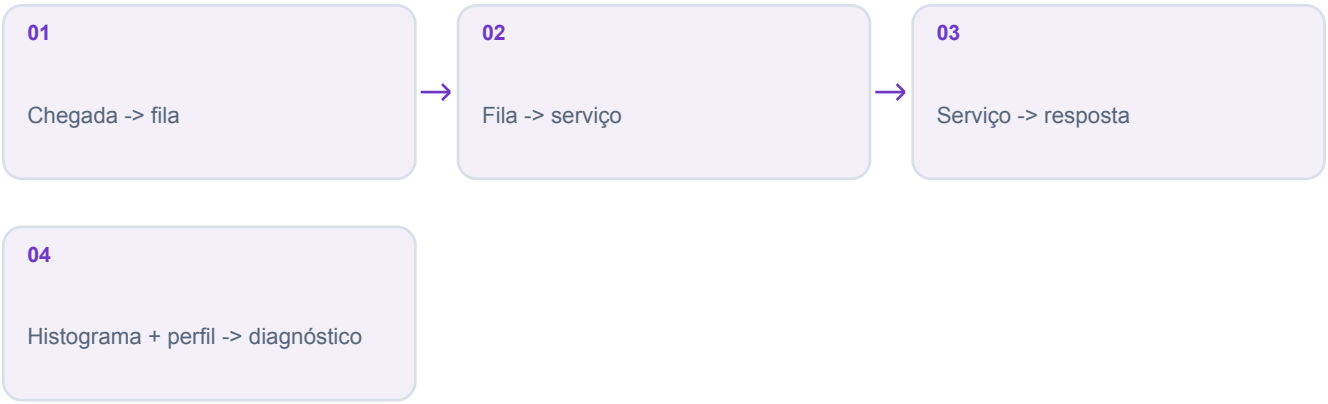
Quando evitar ou limitar. Prometer p99 por cálculo de média ou percentis somados.

Uma abertura possível

Eu começaria por usar L igual lambda vezes W consistentemente. Depois explicaria acompanhar percentis por jornada e classe, preservando as restrições do caso.

Peças e tecnologias. Little’s Law · OpenTelemetry · pprof · Histogramas

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Usar L igual lambda vezes W consistentemente; validar contagem versus duração observada.

2. Acompanhar percentis por jornada e classe; validar p95 e p99 sob pico.

3. Medir o máximo das chamadas paralelas; validar latência conjunta das dependências.

4. Usar traces da jornada completa; validar distribuição ponta a ponta.

5. Testar chegadas independentes da lentidão; validar taxa ofertada versus completada.

30 perguntas para sustentar o desenho

01. Quantas requisições ficam em voo?

Usaria L igual a taxa de chegada multiplicada pelo tempo médio, com unidades e fronteira consistentes. Confirmaria estabilidade antes de interpretar o número, mantendo picos e variabilidade fora dessa estimativa média.

Termo para lembrar: Little’s Law | Base técnica: referência 1.

02. Média de cinquenta milissegundos basta?

Eu mostraria distribuição e percentis por jornada, junto da taxa de sucesso. Média boa pode esconder clientes muito lentos; escolheria o percentil e limite conforme experiência e criticidade do produto.

Termo para lembrar: Tail latency | Base técnica: referência 2.

03. Dez dependências alteram o p99?

No fanout paralelo, a resposta pode depender da chamada mais lenta. Mediria a distribuição conjunta e correlações; aumentar dependências amplia oportunidades de cauda e exige orçamento, isolamento ou resultado parcial.

Termo para lembrar: Fan-out | Base técnica: referência 1.

04. Posso somar p99 dos serviços?

Não somaria p99 dos serviços como se fosse p99 da jornada. Usaria traces e histogramas ponta a ponta, observando sequências e paralelismo para identificar o caminho crítico real.

Termo para lembrar: Percentis | Base técnica: referência 2.

05. Meu gerador espera resposta antes de enviar?

Escolheria geração de carga que não reduza silenciosamente a taxa ofertada quando respostas atrasam. Registraria agendamento e conclusão, pois um teste fechado pode esconder o comportamento durante saturação.

Termo para lembrar: Coordinated omission | Base técnica: referência 1.

06. Onde gasto ciclos no worker?

Coletaria perfis de CPU sob carga representativa, identificando funções dominantes antes de otimizar. Validaria ganho no fluxo completo; acelerar uma função pouco relevante pode não mudar o SLO.

Termo para lembrar: CPU profile | Base técnica: referência 2.

07. Como localizar crescimento de memória?

Compararia alocação com retenção em momentos equivalentes, observando referências que permanecem vivas. Um pico temporário não prova vazamento, e medir somente heap total não identifica qual caminho o sustenta.

Termo para lembrar: Heap profile | Base técnica: referência 1.

08. Por que pequena carga extra explode latência?

Eu mediria espera em fila separada do serviço. Perto da saturação, pequenas variações podem aumentar muito a espera; escolheria margem a partir da curva observada e da capacidade de recuperação.

Termo para lembrar: Queueing | Base técnica: referência 2.

09. Posso publicar benchmark todo aquecido?

Testaria estados frio, quente e de recuperação, com o mesmo padrão de consultas. Um resultado inteiramente aquecido pode ser útil, mas não representa primeiro acesso depois de deploy ou perda de cache.

Termo para lembrar: Warm cache | Base técnica: referência 1.

10. Trace de toda requisição é necessário?

Definiria amostragem com cobertura de erros, caudas e classes relevantes, acompanhando seu viés. Métricas agregadas ajudam detectar problemas não capturados; registrar tudo também possui custo, privacidade e impacto operacional.

Termo para lembrar: Sampling | Base técnica: referência 2.

11. Qual unidade deve entrar em Little’s Law?

Usaria taxa e duração compatíveis na mesma fronteira, como requisições por segundo e segundos dentro do serviço. Misturar latência de rede com concorrência só do banco produz dimensionamento incorreto.

Termo para lembrar: Consistent boundaries | Base técnica: referência 1.

12. Little’s Law vale durante crescimento infinito do backlog?

Não usaria uma leitura ingênua de regime estável nesse cenário. Primeiro mostraria chegada maior que saída e estimaria acúmulo; estabilização exige capacidade adicional ou redução da demanda admitida.

Termo para lembrar: Nonstationary queue | Base técnica: referência 1.

13. Cem chamadas por segundo duram meio segundo: quantas em voo?

A média estimada é cinquenta dentro da fronteira escolhida, assumindo estabilidade. Eu acrescentaria análise de variabilidade e picos antes de definir limites, pois média de concorrência não garante cauda aceitável.

Termo para lembrar: Concurrency estimate | Base técnica: referência 1.

14. O p99 global melhorou quando chegaram muitas consultas baratas: houve otimização?

Eu controlaria o workload mix antes de concluir. Compararia distribuições por operação e população equivalente; mais consultas baratas podem esconder regressão numa jornada crítica. Essa mudança de composição lembra o paradoxo de Simpson. Publicaria indicadores por classe e volume, além do agregado usado no SLO.

Termo para lembrar: Workload mix / paradoxo de Simpson

15. Média de percentis entre servidores é válida?

Não representa necessariamente o percentil da população combinada. Eu agregaria distribuições ou amostras apropriadas, preservando contagens; servidores com volumes diferentes não podem receber peso igual por conveniência.

Termo para lembrar: Histogram aggregation | Base técnica: referência 1.

16. Como escolher buckets do histograma?

Colocaria resolução suficiente perto dos limites do SLO e das latências esperadas. Buckets amplos demais escondem mudanças relevantes; quantidade excessiva aumenta custo de séries e armazenamento operacional.

Termo para lembrar: Histogram resolution | Base técnica: referência 1.

17. Qual diferença entre profiling e tracing?

Profiling explica onde recursos são gastos dentro da execução; tracing conecta etapas de uma jornada. Eu usaria os dois para distinguir CPU local, espera externa e amplificação por chamadas repetidas.

Termo para lembrar: Profile versus trace | Base técnica: referência 1.

18. O que um flame graph largo representa?

Depende do perfil: normalmente maior participação na métrica amostrada, não necessariamente tempo cronológico de uma requisição. Eu verificaria CPU, wall time ou alocações antes de interpretar a imagem.

Termo para lembrar: Profile semantics | Base técnica: referência 1.

19. Como investigar p99 ruim com CPU baixa?

Procuraria espera no pool, locks, disco, rede e limites externos. CPU baixa não exclui saturação de outro recurso; correlacionaria traces lentos com filas e métricas específicas das dependências.

Termo para lembrar: Off-CPU latency | Base técnica: referência 1.

20. Hedging resolve qualquer tail latency?

Ele duplica trabalho e pode piorar sobrecarga. Eu consideraria chamadas seguras, atraso controlado, orçamento e cancelamento, avaliando se as réplicas têm falhas independentes e se o custo compensa.

Termo para lembrar: Hedged requests | Base técnica: referência 1.

21. Timeouts sumiram do histograma e o p99 caiu: como interpretar?

Eu trataria essas observações como censura à direita: sabemos que a conclusão não ocorreu antes do deadline, não sua duração exata. Registraria timeouts percebidos pelo cliente e seu denominador, sem zerá-los nem descartá-los. A melhoria aparente pode resultar de remover justamente as operações mais lentas.

Termo para lembrar: Right censoring / censura à direita

22. Qual carga representa fechamento bancário?

Incluiria distribuição de tamanhos, clientes quentes, cache frio e dependências reais ou simuladas fielmente. A média diária dilui o pico; mediria a janela específica que precisa cumprir o prazo.

Termo para lembrar: Representative workload | Base técnica: referência 1.

23. Por que medir latência de erro separadamente?

Falhas rápidas podem melhorar artificialmente a média quando o serviço deixa de atender. Eu acompanharia latência por resultado e taxa de sucesso, definindo o SLI da experiência válida.

Termo para lembrar: Error latency | Base técnica: referência 1.

24. Como usar amostragem sem perder falhas raras?

Preservaria categorias críticas e consideraria amostragem orientada ao resultado, observando seus limites de memória e privacidade. Também manteria métricas completas para detectar eventos não capturados em traces.

Termo para lembrar: Tail sampling | Base técnica: referência 1.

25. Um benchmark dez vezes mais rápido justifica migração?

Primeiro verificaria equivalência funcional, durabilidade e carga. Depois avaliaria custo operacional e impacto no caminho crítico; acelerar componente irrelevante pode não melhorar a jornada percebida pelo usuário.

Termo para lembrar: Benchmark validity | Base técnica: referência 1.

26. Como definir capacidade máxima sustentável?

Aumentaria carga até violar o SLO ou acumular backlog e escolheria um ponto com margem. Repetiria por duração suficiente para incluir GC, manutenção, aquecimento e limites de dependências.

Termo para lembrar: Sustained throughput | Base técnica: referência 1.

27. Como calcular tempo para drenar backlog?

Dividiria trabalho pendente pela capacidade excedente à chegada, quando essa diferença for positiva e estável. Se chegam cem e saem cem, a fila antiga não diminui apesar de workers ativos.

Termo para lembrar: Drain time | Base técnica: referência 1.

28. Pausas de GC podem aparecer como rede lenta?

Podem atrasar processamento e leitura de respostas, parecendo uma chamada longa. Eu correlacionaria eventos de runtime com traces e relógios, evitando culpar a dependência sem observar o processo local.

Termo para lembrar: Latency attribution | Base técnica: referência 1.

29. Como provar que otimização não só mudou o gargalo?

Mediria toda jornada e recursos antes e depois, incluindo a dependência seguinte. Mais vazão local pode sobrecarregar banco ou parceiro; benefício exige melhora dentro dos limites do sistema completo.

Termo para lembrar: Bottleneck migration | Base técnica: referência 1.

30. Quais números apresentaria ao entrevistador?

Mostraria demanda de pico, tempo de serviço, concorrência estimada, limite observado e margem. Declararia hipóteses e explicaria qual teste as valida, evitando apresentar precisão que os dados não sustentam.

Termo para lembrar: Evidence-based sizing | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Modelo médio versus simulação detalhada

Ganho. Estimativa inicial rápida e explicável

Custo e motivo. Não representa toda variabilidade da fila

Como decidir. Quando há dados limitados e hipóteses declaradas

02. Percentis altos versus média

Ganho. Expõe experiência de cauda

Custo e motivo. Exige volume e estimativa estatística adequada

Como decidir. Quando usuários lentos importam ao SLO

03. Fanout paralelo versus sequência

Ganho. Reduz soma de esperas independentes

Custo e motivo. Aumenta concorrência e exposição a caudas

Como decidir. Quando dependências têm capacidade e podem executar juntas

04. Tracing completo versus amostrado

Ganho. Preserva mais jornadas para diagnóstico

Custo e motivo. Aumenta volume, custo e exposição de dados

Como decidir. Quando risco e orçamento justificam cobertura

05. Carga aberta versus cliente sequencial

Ganho. Revela saturação sob chegada independente

Custo e motivo. Pode exigir gerador e análise mais cuidadosos

Como decidir. Quando produção recebe demanda independente de respostas

06. Profiling em produção versus laboratório

Ganho. Captura comportamento real

Custo e motivo. Exige controle de overhead e acesso

Como decidir. Quando reprodução fora é insuficiente

07. Heap profiling versus só métricas de memória

Ganho. Localiza retenção por origem

Custo e motivo. Coleta e análise têm custo adicional

Como decidir. Quando há suspeita concreta de pressão ou vazamento

08. Headroom versus alta utilização

Ganho. Reduz espera e facilita recuperação

Custo e motivo. Mantém recurso ocioso em parte do tempo

Como decidir. Quando cauda e falhas exigem margem

09. Benchmark frio versus aquecido

Ganho. Representa recuperação e primeiro acesso

Custo e motivo. Pode não refletir operação estável predominante

Como decidir. Quando startup e perda de cache são relevantes

10. Tail sampling versus head sampling

Ganho. Prioriza traces lentos e falhos

Custo e motivo. Precisa aguardar contexto e consumir memória

Como decidir. Quando caudas raras são essenciais ao diagnóstico

Duas alternativas ao caminho proposto

1. Experimento de carga

Quando faz sentido. Distribuição real é desconhecida

Como implementar. Medir curva de saturação progressiva

Limites. Exige ambiente e dados representativos

2. Modelo de filas

Quando faz sentido. Há hipóteses verificáveis de chegada e serviço

Como implementar. Estimar cenários antes do teste

Limites. Modelo inadequado pode enganar

Referências e alcance da evidência

1. Google SRE: monitoramento

Sinais de saúde, distribuições e monitoramento. As respostas e parâmetros de aplicação são propostas autorais.

https://sre.google/sre-book/monitoring-distributed-systems/

2. Go: profiling

Perfis de CPU e memória para investigação. As respostas e parâmetros de aplicação são propostas autorais.

https://go.dev/blog/pprof

Continue em: T084 · T086

T086 / I. Performance e dados

OLTP e OLAP

OLTP e OLAP atendem padrões diferentes de atualização, consulta e consistência. Arquitetura de treino; não descreve implementação interna do Nubank.

Quando usar. Quando relatórios competem com pagamentos ou consultas operacionais.

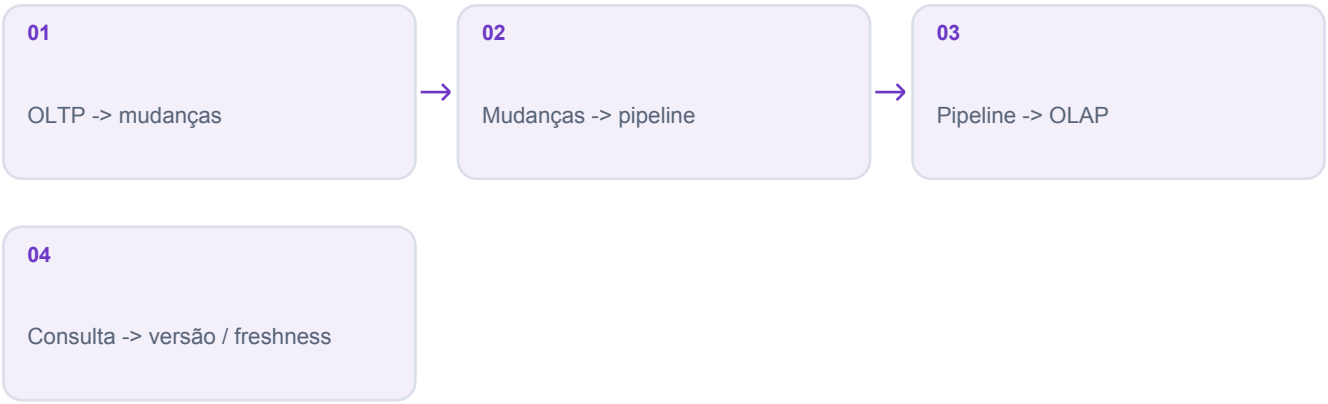
Quando evitar ou limitar. Copiar o banco inteiro sem contrato, proteção e freshness.

Uma abertura possível

Eu começaria por isolar scans analíticos de transações curtas. Depois explicaria usar formato colunar para agregações seletivas, preservando as restrições do caso.

Peças e tecnologias. PostgreSQL · CDC · ClickHouse · BigQuery · Snowflake

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Isolar scans analíticos de transações curtas; validar latência durante fechamento mensal.

2. Usar formato colunar para agregações seletivas; validar bytes lidos por relatório.

3. Negociar atraso aceitável por indicador; validar idade do último dado completo.

4. Capturar mudanças com posição recuperável; validar retomada após interrupção.

5. Estabelecer fronteira consistente de captura; validar contagens antes e após bootstrap.

30 perguntas para sustentar o desenho

01. Por que relatório prejudica o Pix?

Separaria scans e agregações do caminho transacional quando competem por recursos. Mediria impacto de relatórios no SLO de pagamentos; a decisão pode começar por limites ou réplica antes de introduzir outro motor.

Termo para lembrar: Workload | Base técnica: referência 1.

02. Colunas ajudam toda consulta?

Escolheria formato colunar quando consultas leem poucas colunas de muitos registros. Compararia compressão e bytes examinados com custo de atualizações, porque esse layout não é automaticamente melhor para pequenas transações.

Termo para lembrar: Columnar | Base técnica: referência 2.

03. O painel precisa ser em tempo real?

Negociaria freshness pelo uso do indicador. Um painel exploratório pode tolerar atraso que uma decisão operacional não aceita; mostraria instante de referência e completude, evitando apresentar dado parcial como definitivo.

Termo para lembrar: Freshness | Base técnica: referência 1.

04. Como copiar sem dual write frágil?

Usaria captura recuperável com posição e schema controlados, sem dual write ingênuo. Monitoraria lag e retenção do log, incluindo procedimento para reconstrução quando a origem já não conserva a lacuna.

Termo para lembrar: CDC | Base técnica: referência 2.

05. Como iniciar histórico e stream juntos?

Definiria uma fronteira consistente entre snapshot e mudanças incrementais, conforme a ferramenta. Validaria ausência e duplicidade por chave, porque copiar histórico enquanto a origem muda pode deixar uma janela invisível.

Termo para lembrar: Snapshot | Base técnica: referência 1.

06. Mudança de segmento altera histórico?

Escolheria preservar versões de atributos quando perguntas históricas dependem do contexto da época. Sobrescrever pode atender visão atual, mas não deve alterar silenciosamente interpretação de transações já registradas.

Termo para lembrar: Dimensões | Base técnica: referência 2.

07. Como confiar no warehouse?

Confrontaria chaves, totais e períodos entre origem e destino, por moeda quando financeiro. Apenas somas globais podem esconder erros compensados; a validação precisa detectar ausência, duplicação e transformação incorreta.

Termo para lembrar: Reconciliation | Base técnica: referência 1.

08. Analista pode ver todo dado bruto?

Minimizaria dados e aplicaria autorização no acesso analítico, incluindo exportações e derivados. A finalidade de análise não autoriza todas as colunas; o desenho deve permitir rastrear uso e aplicar retenção.

Termo para lembrar: Access control | Base técnica: referência 2.

09. Evento tardio muda relatório fechado?

Modelaria fechamento e revisões explicitamente, preservando versões publicadas. Evento tardio pode corrigir resultado, mas consumidores precisam entender quando isso ocorre e qual evidência sustenta a mudança do indicador.

Termo para lembrar: Corrections | Base técnica: referência 1.

10. Guardo tudo no warehouse quente?

Separaria armazenamento quente e histórico conforme frequência e prazo de acesso. Antes de arquivar, testaria recuperação e custo de consultas, evitando economizar numa camada que torna impossível cumprir a promessa ao usuário.

Termo para lembrar: Retention | Base técnica: referência 2.

11. Uma réplica de leitura é um warehouse?

Não necessariamente. Ela pode isolar parte da leitura, mas mantém modelo transacional e pode atrasar ou sofrer com scans. Eu escolheria conforme consultas, volume e freshness, sem equiparar produtos.

Termo para lembrar: Read replica | Base técnica: referência 1.

12. Posso autorizar crédito com dado do warehouse?

Somente se o contrato de decisão aceitar sua consistência, o que não presumiria. Para limite financeiro, consultaria a autoridade apropriada; análise histórica e autorização operacional possuem exigências distintas.

Termo para lembrar: Authority boundary | Base técnica: referência 1.

13. Como representar correções em dados analíticos?

Manteria identidade, versão e política de atualização dos fatos. Uma correção precisa chegar a agregados e relatórios afetados, com possibilidade de reproduzir o resultado publicado anteriormente e explicar diferenças.

Termo para lembrar: Data correction | Base técnica: referência 1.

14. O que fazer quando CDC fica atrás da retenção do log?

Reconheceria que não há continuidade garantida pelo cursor antigo. Planejaria novo snapshot ou recuperação de fonte preservada, validando lacunas e evitando tratar reinício do consumidor como reparo suficiente.

Termo para lembrar: CDC retention gap | Base técnica: referência 1.

15. Qual risco de consultar réplica após uma escrita?

Ela pode não ter aplicado a mudança ainda. Eu usaria token de versão, leitura no writer ou indicação explícita de atraso, conforme a jornada, preservando a expectativa de quem acabou de escrever.

Termo para lembrar: Read-after-write | Base técnica: referência 1.

16. Por que particionar tabela analítica por data?

Pode reduzir dados examinados em consultas temporais e facilitar retenção. Mas filtros por outras dimensões podem continuar caros; verificaria pruning real e distribuição antes de escolher a granularidade.

Termo para lembrar: Partition pruning | Base técnica: referência 1.

17. Como evitar milhões de arquivos pequenos?

Agruparia ingestão e executaria compactação controlada, preservando publicação consistente. Mediria custo de listar e abrir arquivos, porque armazenamento barato por gigabyte não significa consulta barata por objeto.

Termo para lembrar: Small files | Base técnica: referência 1.

18. Materialized view substitui modelo analítico completo?

Pode resolver agregações específicas com menor complexidade. Eu definiria atualização e tolerância ao atraso; novas perguntas, grandes joins e múltiplas fontes podem justificar uma arquitetura analítica separada.

Termo para lembrar: Materialized view | Base técnica: referência 1.

19. Como dimensionar consultas ad hoc de analistas?

Separaria pools ou quotas, limitaria concorrência e acompanharia bytes processados. Consultas exploratórias variam muito; não permitiria que uma investigação individual consumisse recursos reservados a relatórios com prazo.

Termo para lembrar: Query governance | Base técnica: referência 1.

20. Um join com a dimensão de clientes dobrou a receita: como investigar?

Eu verificaria cardinalidade do join e unicidade da dimensão na chave e período usados. Versões temporais sobrepostas podem multiplicar cada fato. Compararia contagens e somas antes e depois, corrigindo a relação; aplicar DISTINCT indiscriminadamente pode apagar vendas legítimas e esconder a causa.

Termo para lembrar: Join fan-out / cardinalidade

21. Como lidar com dados pessoais em múltiplos datasets?

Mapearia derivados, aplicaria minimização e políticas por finalidade. Exclusão ou restrição precisa alcançar índices e resultados, observando retenções aplicáveis; copiar dados para analytics não elimina responsabilidade de proteção.

Termo para lembrar: Data governance | Base técnica: referência 1.

22. Posso fazer join entre moedas e somar tudo?

Não sem conversão e semântica explícitas. Eu agregaria por moeda ou usaria taxa versionada adequada ao relatório, preservando original e critério para evitar números financeiramente enganosos.

Termo para lembrar: Analytical currency | Base técnica: referência 1.

23. Posso somar clientes distintos por dia para obter clientes únicos do mês?

Eu não somaria essa medida não aditiva: o mesmo cliente pode aparecer em vários dias. Para resultado exato, agregaria a união das identidades no período. Para exploração com erro tolerado, avaliaria sketches combináveis; somar os números diários, mesmo exatos, não calcula a união mensal.

Termo para lembrar: Non-additive measure / distinct aggregation

24. Quando denormalizar ajuda?

Quando reduz joins caros em consultas frequentes e a duplicação pode ser mantida corretamente. Eu incluiria custo de atualização e correções, preferindo derivar cópias de uma fonte identificável e versionada.

Termo para lembrar: Denormalization | Base técnica: referência 1.

25. Como impedir que relatório veja carga pela metade?

Publicaria uma versão ou snapshot completo após validar a carga. Leitores usariam esse marco; objetos presentes isoladamente não comprovariam que todos os dados do período estão disponíveis.

Termo para lembrar: Atomic publication | Base técnica: referência 1.

26. Qual diferença entre latência do pipeline e completude?

Pipeline rápido pode entregar apenas parte das fontes. Eu mediria ambos: idade do último processamento e confirmação de cobertura esperada, evitando exibir um indicador como completo só porque chegou algo recente.

Termo para lembrar: Completeness watermark | Base técnica: referência 1.

27. Como reprocessar sem duplicar métricas?

Usaria identidade dos fatos e uma estratégia explícita de substituição ou merge. Separaria execução do job da versão publicada, testando replay do mesmo período e correções tardias.

Termo para lembrar: Idempotent rebuild | Base técnica: referência 1.

28. Warehouse caro deve ser desligado entre consultas?

Depende do modelo e do tempo de retomada. Eu compararia economia, fila e SLO dos usuários; relatórios críticos podem exigir reserva, enquanto exploração tolera espera maior.

Termo para lembrar: Compute lifecycle | Base técnica: referência 1.

29. Quando separar computação de armazenamento ajuda?

Permite ajustar recursos de consulta sem replicar toda capacidade de disco. Eu avaliaria custo de transferência, caches e isolamento, pois essa separação não elimina limites de leitura ou coordenação.

Termo para lembrar: Storage compute separation | Base técnica: referência 1.

30. Qual desenho inicial apresentaria com volume pequeno?

Começaria com banco operacional protegido e relatórios limitados, talvez réplica ou views. Definiria os sinais para extrair analytics depois, evitando infraestrutura complexa antes de haver necessidade mensurável.

Termo para lembrar: Incremental architecture | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Separação OLTP/OLAP versus banco único

Ganho. Isola carga e otimiza cada workload

Custo e motivo. Adiciona pipeline, cópias e atraso

Como decidir. Quando relatórios ameaçam operações ou exigem outro padrão

02. Colunar versus orientado a linhas

Ganho. Reduz bytes em agregações seletivas

Custo e motivo. Pode encarecer alterações pequenas

Como decidir. Quando scans de poucas colunas predominam

03. Freshness menor versus lotes maiores

Ganho. Dados ficam disponíveis mais cedo

Custo e motivo. Mais processamento contínuo e coordenação

Como decidir. Quando a decisão exige atualização rápida

04. CDC versus extração periódica

Ganho. Captura incrementos com menor atraso

Custo e motivo. Depende de logs, schemas e recuperação de posições

Como decidir. Quando origem suporta captura e atualização frequente importa

05. Snapshot consistente versus cópia livre

Ganho. Define base coerente para reconstrução

Custo e motivo. Pode exigir recursos e coordenação da origem

Como decidir. Quando completude importa durante escrita concorrente

06. Dimensão versionada versus sobrescrita

Ganho. Preserva contexto histórico

Custo e motivo. Mais armazenamento e joins temporais

Como decidir. Quando perguntas dependem da vigência passada

07. Reconciliação por item versus só totais

Ganho. Detecta duplicatas e ausências compensadas

Custo e motivo. Processa mais dados e exige identidade

Como decidir. Quando correção precisa de evidência detalhada

08. Minimização versus cópia bruta ampla

Ganho. Reduz exposição e governança

Custo e motivo. Pode limitar perguntas futuras e exigir nova ingestão

Como decidir. Quando finalidade e sensibilidade restringem retenção

09. Revisão versionada versus atualização silenciosa

Ganho. Permite explicar mudanças publicadas

Custo e motivo. Exige consumidores conscientes de versões

Como decidir. Quando relatórios têm impacto financeiro ou decisório

10. Storage frio versus tudo quente

Ganho. Reduz custo de histórico pouco usado

Custo e motivo. Aumenta latência e custo de recuperação

Como decidir. Quando prazo de acesso permite arquivamento

Duas alternativas ao caminho proposto

1. Réplica relacional

Quando faz sentido. Relatórios moderados usam schema existente

Como implementar. Separar conexão e carga de leitura

Limites. Lag e consultas grandes continuam relevantes

2. Warehouse separado

Quando faz sentido. Agregações extensas e múltiplas fontes

Como implementar. Ingerir dados governados com modelos analíticos

Limites. Custo e consistência eventual adicionais

Referências e alcance da evidência

1. Azure: OLTP

Transações operacionais e modelos relacionais. As respostas e parâmetros de aplicação são propostas autorais.

https://learn.microsoft.com/en-us/azure/architecture/data-guide/relational-data/online-transaction-processing

2. Azure: OLAP

Workloads analíticos e agregações. As respostas e parâmetros de aplicação são propostas autorais.

https://learn.microsoft.com/en-us/azure/architecture/data-guide/relational-data/online-analytical-processing

Continue em: T085 · T087

T087 / I. Performance e dados

Stream processing, Flink e watermarks

Streaming mantém estado sobre eventos contínuos e precisa definir tempo, atraso e recuperação. Arquitetura de treino; não descreve implementação interna do Nubank.

Quando usar. Para agregações incrementais, alertas e reconciliação contínua.

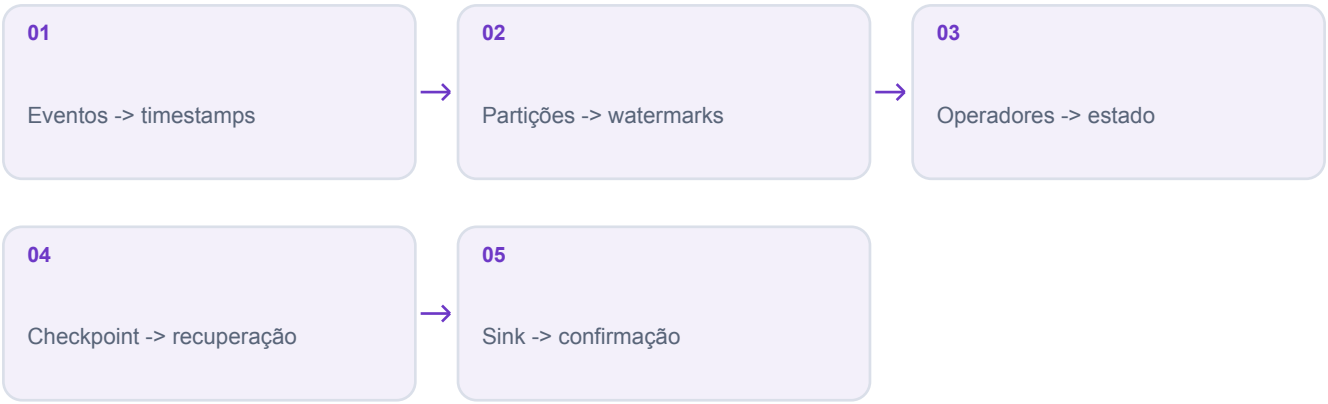
Quando evitar ou limitar. Tratar watermark como prova absoluta de que nada chegará depois.

Uma abertura possível

Eu começaria por usar tempo do evento com contrato explícito. Depois explicaria tratar watermark como progresso estimado, preservando as restrições do caso.

Peças e tecnologias. Flink · Kafka · RocksDB · Object storage

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Usar tempo do evento com contrato explícito; validar eventos com skew e atraso.

2. Tratar watermark como progresso estimado; validar política de late events.

3. Configurar detecção de ociosidade com cautela; validar retomada após período ocioso.

4. Negociar janela de revisão dos resultados; validar distribuição real dos atrasos.

5. Avaliar source estado e sink conjuntamente; validar falha entre gravação e confirmação.

30 perguntas para sustentar o desenho

01. Qual relógio define a janela de fraude?

Eu escolheria event time quando a pergunta depende de quando o fato ocorreu, guardando também chegada. Validaria timestamps e atraso, porque tempo do produtor não é automaticamente confiável nem sincronizado.

Termo para lembrar: Event time | Base técnica: referência 1.

02. Evento após watermark é impossível?

Watermark representa uma política de progresso temporal, não certeza física de ausência de eventos futuros atrasados. Definiria como revisar ou encaminhar late events, com limites de estado e expectativa dos consumidores.

Termo para lembrar: Watermark | Base técnica: referência 2.

03. Uma partição parada trava tudo?

Configuraria tratamento de partições ociosas para não bloquear progresso desnecessariamente, mas testaria a retomada. Uma origem silenciosa pode voltar com eventos antigos; a política precisa preservar o comportamento contratado.

Termo para lembrar: Idle partition | Base técnica: referência 1.

04. Quanto esperar para fechar agregação?

Escolheria tolerância ao atraso a partir da distribuição e custo de revisão. Esperar mais melhora cobertura, mas retém estado e adia resultado; explicaria o que acontece depois do limite.

Termo para lembrar: Allowed lateness | Base técnica: referência 2.

05. Checkpoint entrega exactly-once externo?

Verificaria a composição entre origem, estado e sink antes de prometer exactly-once. Checkpoint recupera o processamento dentro das capacidades suportadas; uma API externa ainda precisa reconhecer identidade ou permitir reconciliação.

Termo para lembrar: Checkpoint | Base técnica: referência 1.

06. Estado grande deve ficar no heap?

Compararia tamanho do estado, latência de acesso, checkpoints e recuperação ao escolher backend. Estado fora do heap pode reduzir pressão de memória, mas adiciona serialização e I/O que precisam de capacidade.

Termo para lembrar: State backend | Base técnica: referência 2.

07. Um cliente concentra todo processamento?

Mediria distribuição por chave e identificaria trabalho decomponível. Não aplicaria salting a um agregado que exige ordem única sem definir recomposição; paralelismo é limitado pela semântica, não apenas pelos workers.

Termo para lembrar: Key skew | Base técnica: referência 1.

08. Como descobrir operador que limita vazão?

Seguiria indicadores do grafo até o operador ou destino limitante. Escalar a fonte enquanto o sink está saturado aumenta backlog; a intervenção deve reduzir trabalho ou aumentar capacidade no ponto correto.

Termo para lembrar: Backpressure | Base técnica: referência 2.

09. Nova versão entende estado antigo?

Versionaria estado e contrato, ensaiando restore antes do deploy. Mudança de código pode tornar snapshot antigo incompatível; manteria caminho de migração ou recuperação que não dependa de improviso durante incidente.

Termo para lembrar: Schema evolution | Base técnica: referência 1.

10. Reexecutar histórico duplica alertas?

Separaria reconstrução de projeções de efeitos externos, preservando identidades. Replay de histórico não autoriza repetir alertas ou pagamentos; o sink precisa distinguir resultado já aplicado de nova intenção legítima.

Termo para lembrar: Reprocessing | Base técnica: referência 2.

11. Processing time e event time podem divergir muito?

Sim, dispositivo offline ou fila congestionada atrasa chegada. Eu escolheria a semântica pela pergunta de negócio e guardaria ambos os tempos, permitindo explicar resultados e investigar atrasos.

Termo para lembrar: Time semantics | Base técnica: referência 1.

12. Uma watermark atrasada significa evento perdido?

Não necessariamente; pode refletir partição lenta ou ociosa. Eu investigaria progresso por origem e política de idle partitions, evitando avançar arbitrariamente e descartar eventos legítimos por conveniência operacional.

Termo para lembrar: Watermark diagnosis | Base técnica: referência 1.

13. O que fazer com evento tarde demais para a janela?

Definiria saída lateral, revisão ou rejeição auditável conforme o produto. A política precisa preservar informação sobre o descarte e permitir reconciliação, sobretudo quando valores financeiros estão envolvidos.

Termo para lembrar: Late-event policy | Base técnica: referência 1.

14. Checkpoint é o mesmo que backup?

Não. Checkpoint apoia recuperação operacional de estado e posição, com retenção e dependências específicas. Eu definiria separadamente recuperação de corrupção, versões incompatíveis e perda do ambiente, testando restaurações completas.

Termo para lembrar: Checkpoint versus backup | Base técnica: referência 1.

15. Como emitir alerta apenas uma vez após restore?

Atribuiria identidade estável ao alerta e faria o destino deduplicar ou confirmar transacionalmente. Recuperar estado do processador não desfaz um e-mail enviado antes da falha de checkpoint.

Termo para lembrar: External effect | Base técnica: referência 1.

16. Um lookup externo de score retorna outro valor após restore: checkpoint resolve?

Eu separaria estado recuperado de dependência externa mutável. Um checkpoint não obriga o provedor a devolver o score anterior. Para replay reproduzível, registraria valor e versão usados ou consultaria uma dimensão temporal, definindo também timeout e ausência. Caso contrário, assumiria explicitamente resultado recalculado.

Termo para lembrar: Deterministic replay / versioned enrichment

17. Como joins de dois streams lidam com atraso diferente?

Manteria estado delimitado por chave e tempo, usando progresso das entradas para expiração. Uma origem lenta aumenta retenção; negociaria janela e tratamento de correspondências que chegam depois.

Termo para lembrar: Stream join | Base técnica: referência 1.

18. Estado por chave pode crescer indefinidamente?

Pode, se não houver expiração ou fechamento definido. Eu projetaria TTL ou compactação conforme semântica e testaria chaves sem novos eventos, evitando descartar estado ainda necessário à correção.

Termo para lembrar: State retention | Base técnica: referência 1.

19. TTL de estado sempre usa event time?

Depende do mecanismo e versão utilizados. Eu verificaria a documentação específica e separaria retenção operacional da semântica do evento; assumir o relógio errado pode remover estado cedo demais.

Termo para lembrar: TTL semantics | Base técnica: referência 1.

20. Qual problema do checkpoint muito frequente?

Ele aumenta trabalho de persistência e coordenação, podendo competir com processamento. Checkpoint raro aumenta trabalho perdido na recuperação; escolheria intervalo pelos custos medidos e objetivos de retomada.

Termo para lembrar: Checkpoint interval | Base técnica: referência 1.

21. Como restaurar com paralelismo diferente?

Usaria mecanismos suportados para redistribuir estado por chave e verificaria compatibilidade. Testaria skew, tempo de restauração e acesso ao storage antes de considerar a mudança segura para a janela operacional.

Termo para lembrar: State rescaling | Base técnica: referência 1.

22. Por que um evento com timestamp futuro é perigoso?

Pode distorcer progresso temporal conforme a estratégia e antecipar fechamento. Eu validaria limites plausíveis da origem, isolaria dados suspeitos e manteria diagnóstico da causa sem corrigir silenciosamente valores financeiros.

Termo para lembrar: Timestamp validation | Base técnica: referência 1.

23. Uma regra transmitida por broadcast muda no meio da janela: qual versão vale?

Eu definiria a fronteira de ativação da regra: por evento ou por janela. Versionaria a política e registraria qual versão produziu cada resultado, preservando o necessário para replay. Misturar partes da agregação sob regras diferentes sem essa semântica gera resultados que ninguém consegue explicar.

Termo para lembrar: Versioned broadcast rules / activation boundary

24. Quando microbatch é uma escolha melhor?

Quando segundos ou minutos de atraso atendem o objetivo e simplificam operação. Eu compararia custo de estado contínuo e recuperação com lotes pequenos, sem tratar menor latência como valor automático.

Termo para lembrar: Microbatch | Base técnica: referência 1.

25. Um operador map lento pode causar fila na origem?

Sim, a pressão se propaga conforme o pipeline e buffers. Eu observaria operadores e sink antes de escalar a fonte, porque aumentar ingestão pode apenas elevar backlog e retenção.

Termo para lembrar: Backpressure propagation | Base técnica: referência 1.

26. Como preservar ordem de eventos por conta?

Escolheria chave consistente e verificaria a ordem efetivamente fornecida pela origem. Se múltiplas fontes concorrem, adicionaria versão de domínio ou regra de resolução, evitando confiar apenas na partição.

Termo para lembrar: Per-key ordering | Base técnica: referência 1.

27. O que salvar para reproduzir uma decisão de janela?

Guardaria versões de regra, entradas relevantes, limites temporais e resultado. Isso permite distinguir mudança de algoritmo de chegada tardia, apoiando investigação e correção controlada de decisões.

Termo para lembrar: Reproducibility | Base técnica: referência 1.

28. Como lidar com alteração de schema durante replay?

Aplicaria evolução compatível ou transformadores versionados, mantendo leitores capazes de entender eventos preservados. Eu testaria amostras de todas as versões e evitaria interpretar silenciosamente campos com significado alterado.

Termo para lembrar: Schema evolution | Base técnica: referência 1.

29. Flink exige Kafka como origem?

Não é uma obrigação conceitual; conectores têm capacidades distintas. Eu escolheria origem pelas garantias de posição, replay e integração, verificando limitações concretas antes de prometer recuperação consistente.

Termo para lembrar: Connector contract | Base técnica: referência 1.

30. Qual indicador de saúde importa além de throughput?

Acompanharia atraso por evento, tamanho do estado, duração e falha de checkpoints, backlog e correção dos resultados. Alta vazão não prova cumprimento do prazo nem processamento completo das origens.

Termo para lembrar: Streaming SLI | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Event time versus processing time

Ganho. Respeita momento do fato de negócio

Custo e motivo. Exige lidar com atraso e clocks da origem

Como decidir. Quando ordem temporal do fato é relevante

02. Watermark conservadora versus agressiva

Ganho. Reduz revisões por eventos tardios

Custo e motivo. Aumenta latência de resultado e estado

Como decidir. Quando completude vale mais que rapidez inicial

03. Detecção de idle versus esperar todas origens

Ganho. Evita bloquear janelas por silêncio

Custo e motivo. Retomadas podem trazer dados fora do progresso atual

Como decidir. Quando fontes realmente ficam ociosas

04. Mais lateness versus fechamento cedo

Ganho. Inclui mais eventos na primeira publicação

Custo e motivo. Retém estado e atrasa definitividade

Como decidir. Quando distribuição de atraso justifica espera

05. Checkpoints frequentes versus espaçados

Ganho. Reduz trabalho a repetir após falha

Custo e motivo. Aumenta I/O e coordenação

Como decidir. Quando recuperação rápida compensa overhead

06. Estado em disco versus heap

Ganho. Suporta maior volume sem heap equivalente

Custo e motivo. Acesso e serialização podem ser mais lentos

Como decidir. Quando volume excede memória eficiente

07. Salting versus chave única

Ganho. Distribui agregações decomponíveis

Custo e motivo. Requer recomposição e pode perder ordem direta

Como decidir. Quando a operação admite divisão matemática

08. Backpressure versus descarte

Ganho. Preserva trabalho ao reduzir velocidade

Custo e motivo. Aumenta atraso e exige armazenamento

Como decidir. Quando perder eventos é inaceitável

09. Migração de estado versus reinício limpo

Ganho. Preserva continuidade de processamento

Custo e motivo. Exige compatibilidade e teste de versões

Como decidir. Quando reconstrução completa é cara ou impossível

10. Replay isolado versus pipeline compartilhado

Ganho. Evita interferir nos efeitos e carga normal

Custo e motivo. Duplica ambiente ou coordenação operacional

Como decidir. Quando recomputação histórica é frequente ou arriscada

Duas alternativas ao caminho proposto

1. Microbatch

Quando faz sentido. Minutos de atraso são aceitáveis

Como implementar. Processar lotes pequenos com checkpoints

Limites. Latência mínima maior

2. Consumidor simples

Quando faz sentido. Transformação sem janelas complexas

Como implementar. Persistir estado e offset de forma coerente

Limites. Coordenação cresce com requisitos

Referências e alcance da evidência

1. Flink: tempo em streaming

Event time, processing time e watermarks. As respostas e parâmetros de aplicação são propostas autorais.

https://nightlies.apache.org/flink/flink-docs-stable/docs/concepts/time/

2. Flink: checkpointing

Checkpoint e recuperação de estado. As respostas e parâmetros de aplicação são propostas autorais.

https://nightlies.apache.org/flink/flink-docs-stable/docs/dev/datastream/fault-tolerance/checkpointing/

Continue em: T086 · T088

T088 / I. Performance e dados

ETL, ELT, data lake e lineage

Pipeline de dados precisa preservar origem, qualidade e possibilidade de reconstrução. Arquitetura de treino; não descreve implementação interna do Nubank.

Quando usar. Ao integrar múltiplas fontes e manter datasets derivados.

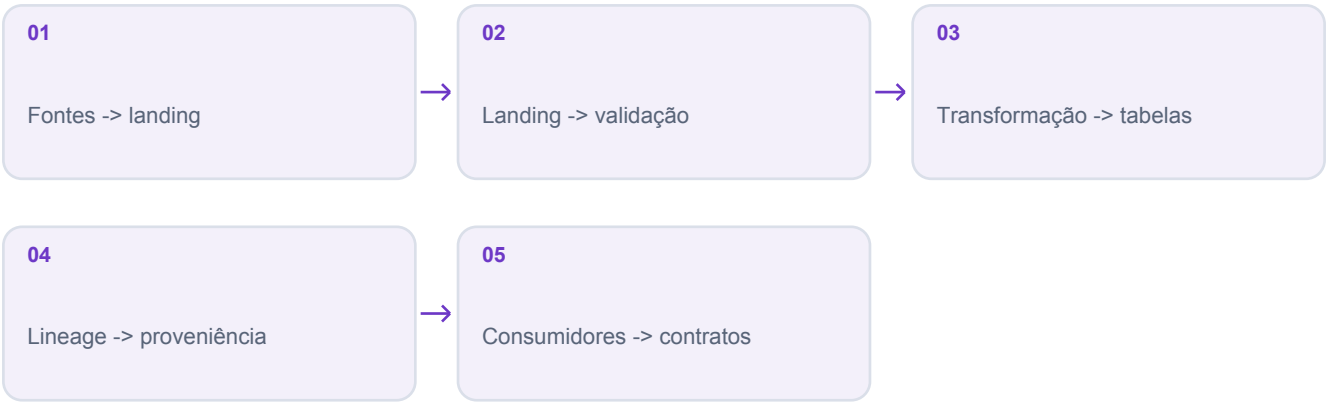
Quando evitar ou limitar. Chamar data lake de governado apenas por armazenar arquivos.

Uma abertura possível

Eu começaria por guardar entrada identificada e checksum. Depois explicaria projetar partições pelo acesso e cardinalidade, preservando as restrições do caso.

Peças e tecnologias. Airflow · Spark · Iceberg · OpenLineage · S3

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Guardar entrada identificada e checksum; validar reprodução do mesmo lote.

2. Projetar partições pelo acesso e cardinalidade; validar tamanho e quantidade de arquivos.

3. Usar evolução suportada com contrato explícito; validar leitura por versões antigas.

4. Registrar datasets jobs e execuções; validar rastreio entre origem e relatório.

5. Validar regras de completude e domínio; validar totais nulos e distribuição.

30 perguntas para sustentar o desenho

01. Posso sobrescrever o arquivo original?

Preservaria identidade, origem e checksum da entrega conforme retenção autorizada. Reprocessamento precisa saber qual entrada foi usada; sobrescrever o arquivo original pode tornar impossível explicar diferenças entre execuções.

Termo para lembrar: Landing | Base técnica: referência 1.

02. Particionar por cliente sempre ajuda?

Escolheria partições pelos filtros e pelo volume, verificando quantidade e tamanho dos arquivos. Cardinalidade excessiva cria overhead; uma chave conveniente para escrita pode prejudicar scans e manutenção posteriores.

Termo para lembrar: Partitioning | Base técnica: referência 2.

03. Renomear coluna quebra histórico?

Usaria mecanismos de evolução com contrato para consumidores, distinguindo nome, tipo e significado. Renomear coluna não deve depender de inferência; testaria leitura histórica e versões suportadas antes da mudança.

Termo para lembrar: Schema evolution | Base técnica: referência 1.

04. Como descobrir quem usou dado incorreto?

Registraria jobs, execuções e datasets de entrada e saída para localizar impacto. O grafo precisa refletir o pipeline real; metadados manuais abandonados não permitem identificar consumidores de um dado incorreto.

Termo para lembrar: Lineage | Base técnica: referência 2.

05. Carga termina mas valores estão errados?

Validaria estrutura, domínio, completude e distribuições relevantes antes da publicação. Job concluído significa execução técnica, não qualidade; uma carga com valores errados pode terminar sem qualquer exceção do processo.

Termo para lembrar: Data quality | Base técnica: referência 1.

06. Como reprocessar cinco anos sem derrubar?

Executaria backfill por partições retomáveis, com capacidade e prioridade separadas. Publicaria versões coerentes e reconciliaria sobreposição com ingestão atual, evitando que reconstrução antiga impeça cumprir freshness do dia.

Termo para lembrar: Backfill | Base técnica: referência 2.

07. Leitores podem ver metade da tabela?

Leitores consumiriam um snapshot ou versão publicada após validação, não uma pasta em preenchimento. A existência de alguns arquivos não comprova completude; a publicação precisa ter um marco identificável.

Termo para lembrar: Atomic publish | Base técnica: referência 1.

08. Apagar origem remove derivados automaticamente?

Mapearia derivados e aplicaria a política de retenção ou exclusão em todas as cópias relevantes. A decisão depende da finalidade e regras aplicáveis; lineage ajuda localizar dados, mas não executa controles sozinho.

Termo para lembrar: Deletion | Base técnica: referência 2.

09. Compactar arquivos muda resultado?

Compactaria preservando conteúdo e mecanismos de publicação do formato. Também reservaria capacidade e retenção de snapshots, porque manutenção disputa recursos e não pode remover arquivos ainda necessários aos leitores.

Termo para lembrar: Compaction | Base técnica: referência 1.

10. Quem corrige uma coluna sem semântica?

Definiria dono, contrato e canal de resolução para cada dataset relevante. Problemas sem responsável persistem entre equipes; mediria tempo de recuperação e qualidade para que ownership seja operação real, não etiqueta.

Termo para lembrar: Ownership | Base técnica: referência 2.

11. Qual diferença prática entre ETL e ELT?

No ETL, transformo antes de carregar o destino; no ELT, carrego e transformo nele. Eu decidiria pela necessidade de minimização, capacidade de reprocessar e governança dos dados brutos.

Termo para lembrar: ETL versus ELT | Base técnica: referência 1.

12. O que torna um arquivo um dataset confiável?

Contrato de schema, origem, período, qualidade e ownership, além do conteúdo. Eu publicaria metadados e critérios de completude, porque encontrar um arquivo no bucket não demonstra que ele está correto.

Termo para lembrar: Dataset contract | Base técnica: referência 1.

13. Como reconhecer reenvio do mesmo arquivo?

Usaria identidade da entrega e checksum conforme contrato, preservando origem e versão. Mesmo conteúdo pode ser legítimo em períodos diferentes; por isso hash sozinho não substitui a intenção da carga.

Termo para lembrar: Ingestion identity | Base técnica: referência 1.

14. Linha inválida deve derrubar o lote inteiro?

Depende do contrato de completude. Eu definiria rejeição total ou quarentena por linha com contagem explícita, sem publicar um resultado parcial como completo quando o consumidor espera totalidade.

Termo para lembrar: Quarantine policy | Base técnica: referência 1.

15. Como escolher partição por dia ou hora?

Compararia volume, filtros e tamanho dos arquivos. Partições muito finas criam overhead; grossas exigem ler dados desnecessários. Validaria consultas dominantes e manutenção antes de fixar a granularidade.

Termo para lembrar: Partition granularity | Base técnica: referência 1.

16. O que lineage oferece durante incidente?

Permite identificar derivados e consumidores potencialmente afetados por uma origem incorreta. Eu usaria essa informação para suspender publicação, reconstruir datasets e comunicar impacto, reconhecendo lacunas na cobertura do grafo.

Termo para lembrar: Impact analysis | Base técnica: referência 1.

17. Schema registry prova qualidade dos valores?

Não. Ele verifica estrutura e compatibilidade conforme regras configuradas. Eu adicionaria validações de domínio, distribuição e relações, porque um valor incorreto pode respeitar perfeitamente o tipo declarado.

Termo para lembrar: Semantic validation | Base técnica: referência 1.

18. A origem muda enquanto o ETL lê: como garantir entradas reproduzíveis?

Eu fixaria snapshots ou versões imutáveis dos objetos num input manifest ligado à execução. Toda leitura seguiria esse conjunto, evitando misturar gerações da origem. Guardaria versões e checksums para replay, com retenção compatível; registrar apenas o caminho atual não permite reconstruir o que foi processado.

Termo para lembrar: Input manifest / snapshot pinning

19. Uma coluna de origem foi corrigida: preciso recalcular todos os derivados?

Eu usaria lineage em nível de coluna e versões das transformações para identificar dependências afetadas. Reprocessaria as saídas atingidas e validaria suas invariantes. Se o grafo estiver incompleto, ampliaria conservadoramente o escopo; schema inalterado não significa ausência de impacto nos valores ou nas decisões.

Termo para lembrar: Column-level lineage / selective recomputation

20. Data mesh significa cada time escolher qualquer formato?

Não precisaria implicar isso. Eu distribuiria responsabilidade com contratos e controles interoperáveis, mantendo padrões mínimos de acesso, qualidade e descoberta para que autonomia não inviabilize consumo entre domínios.

Termo para lembrar: Federated governance | Base técnica: referência 1.

21. Lakehouse elimina necessidade de warehouse?

Não automaticamente. Eu compararia consultas, concorrência, operação e recursos exigidos. Formato de tabela e separação de storage ajudam, mas a adequação depende do motor e dos contratos de uso.

Termo para lembrar: Architecture fit | Base técnica: referência 1.

22. Como versionar lógica de transformação?

Vincularia execução a código, parâmetros e schema, preservando referências das entradas. Assim consigo reconstruir o resultado e distinguir um dado novo de uma mudança de regra no processamento.

Termo para lembrar: Transformation version | Base técnica: referência 1.

23. Qual risco de inferir schema em toda carga?

Amostras e valores diferentes podem alterar tipos de maneira inesperada. Eu usaria contrato explícito para dados relevantes e trataria evolução como mudança versionada, com validação dos consumidores afetados.

Termo para lembrar: Schema inference | Base técnica: referência 1.

24. Posso compactar arquivos enquanto leitores consultam?

Usaria mecanismo de publicação consistente suportado pelo formato, preservando referências necessárias. Não removeria arquivos ainda acessíveis a snapshots válidos; limpeza e retenção precisam respeitar leitores e recuperação.

Termo para lembrar: Snapshot lifecycle | Base técnica: referência 1.

25. Como evitar vazamento em logs do pipeline?

Registraria IDs, contagens e erros sanitizados, sem copiar payloads sensíveis por padrão. Quando amostras forem necessárias, aplicaria acesso, retenção e finalidade explícitos, inclusive nas ferramentas de observabilidade.

Termo para lembrar: Log minimization | Base técnica: referência 1.

26. Qual papel de um catálogo de dados?

Documenta descoberta, ownership, schema e políticas, mas precisa refletir o fluxo real. Eu automatizaria atualização quando possível e verificaria links quebrados, porque documentação abandonada induz decisões erradas.

Termo para lembrar: Data catalog | Base técnica: referência 1.

27. Como medir disponibilidade de um produto de dados?

Definiria consultas ou entregas válidas, prazo de atualização e completude. Um bucket acessível com dados de ontem pode estar indisponível para a decisão que exige fechamento de hoje.

Termo para lembrar: Data product SLO | Base técnica: referência 1.

28. O que fazer quando a origem muda significado sem mudar tipo?

Detectaria por contrato, testes de domínio e alinhamento com o produtor. Compatibilidade sintática não basta; publicaria nova semântica ou versão, evitando reinterpretar histórico silenciosamente para os consumidores.

Termo para lembrar: Semantic drift | Base técnica: referência 1.

29. Como provar que não perdi registros numa migração?

Compararia chaves, contagens, totais e intervalos entre snapshots de referência. Também testaria alterações concorrentes e deletions, pois copiar a quantidade correta não comprova que são os mesmos registros.

Termo para lembrar: Migration reconciliation | Base técnica: referência 1.

30. Quando manter dado bruto é uma escolha ruim?

Quando finalidade, sensibilidade ou custo não justificam retenção. Eu definiria minimização e prazo antes de copiar, preservando somente evidências necessárias para reconstrução e auditoria dentro das regras aplicáveis.

Termo para lembrar: Purpose limitation | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Entrada imutável versus sobrescrita

Ganho. Permite replay e auditoria da entrega

Custo e motivo. Aumenta armazenamento e governança de retenção

Como decidir. Quando reprodução da carga tem valor

02. Partições finas versus grossas

Ganho. Reduz dados lidos por filtros específicos

Custo e motivo. Cria mais arquivos e metadados

Como decidir. Quando volume por partição continua saudável

03. Schema explícito versus inferência

Ganho. Torna contrato previsível

Custo e motivo. Exige gestão de evolução e validação

Como decidir. Quando erro de tipo ou significado tem impacto

04. Lineage automático versus registro manual

Ganho. Melhora cobertura e atualização

Custo e motivo. Exige integração e manutenção dos emissores

Como decidir. Quando muitos pipelines precisam de rastreabilidade

05. Quarentena versus rejeição total

Ganho. Permite tratar erros isoladamente

Custo e motivo. Consumidor precisa entender resultado parcial

Como decidir. Quando contrato permite publicação incompleta identificada

06. Backfill isolado versus compartilhado

Ganho. Protege ingestão e consultas atuais

Custo e motivo. Requer capacidade ou agendas adicionais

Como decidir. Quando histórico grande compete com SLO diário

07. Publicação por snapshot versus arquivo a arquivo

Ganho. Oferece visão coerente aos leitores

Custo e motivo. Adia exposição até completar unidade

Como decidir. Quando completude do dataset é obrigatória

08. Minimização antecipada versus bruto amplo

Ganho. Reduz risco e volume armazenado

Custo e motivo. Diminui flexibilidade de novas análises

Como decidir. Quando finalidade e sensibilidade são restritivas

09. Compactação versus arquivos originais

Ganho. Reduz overhead e melhora scans

Custo e motivo. Consome recursos e requer limpeza coordenada

Como decidir. Quando muitos arquivos pequenos degradam consulta

10. Ownership por domínio versus equipe central

Ganho. Aproxima semântica de quem produz

Custo e motivo. Exige padrões e coordenação entre domínios

Como decidir. Quando escala organizacional justifica distribuição

Duas alternativas ao caminho proposto

1. ETL

Quando faz sentido. Minimização antes de persistir é necessária

Como implementar. Transformar e filtrar antes do destino

Limites. Pode perder flexibilidade de reprocessar

2. ELT

Quando faz sentido. Destino tem capacidade e controles adequados

Como implementar. Carregar dados governados e transformar depois

Limites. Amplia responsabilidade sobre dados brutos

Referências e alcance da evidência

1. OpenLineage: object model

Jobs, runs, datasets e proveniência. As respostas e parâmetros de aplicação são propostas autorais.

https://openlineage.io/docs/spec/object-model/

2. Iceberg: evolução

Evolução de schema e partições. As respostas e parâmetros de aplicação são propostas autorais.

https://iceberg.apache.org/docs/latest/evolution/

Continue em: T087 · T089

T089 / I. Performance e dados

IA, RAG, embeddings e avaliações

RAG recupera evidências para apoiar geração, sem tornar respostas automaticamente verdadeiras. Arquitetura de treino; não descreve implementação interna do Nubank.

Quando usar. Busca assistida, resumo e apoio ao atendimento com fontes autorizadas.

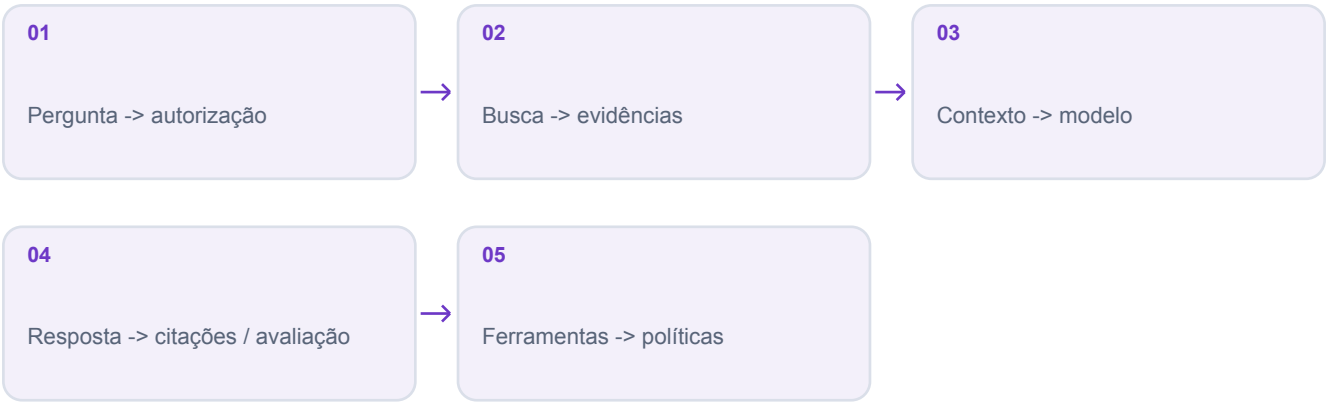
Quando evitar ou limitar. Delegar autorização financeira ao texto de um modelo.

Uma abertura possível

Eu começaria por preservar unidade semântica e referência original. Depois explicaria usar similaridade para recuperar candidatos, preservando as restrições do caso.

Peças e tecnologias. Azure AI Search · Embeddings · Vector DB · Reranker · Eval sets

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Preservar unidade semântica e referência original; validar perguntas que atravessam seções.

2. Usar similaridade para recuperar candidatos; validar relevância avaliada por especialistas.

3. Combinar busca lexical e semântica; validar consultas com siglas e números.

4. Aplicar acesso antes de montar contexto; validar consultas entre perfis distintos.

5. Tratar conteúdo recuperado como dado não confiável; validar testes adversariais no corpus.

30 perguntas para sustentar o desenho

01. Como dividir manuais bancários longos?

Dividiria documentos por unidades semânticas que preservem condições e referências, avaliando perguntas reais. Tamanho de chunk equilibra contexto, relevância e custo; testaria especialmente regras cuja exceção aparece numa seção vizinha.

Termo para lembrar: Chunking | Base técnica: referência 1.

02. Vetores parecidos provam equivalência jurídica?

Usaria embeddings para encontrar candidatos, não para provar equivalência de regras. A avaliação precisa confirmar que evidências corretas entram no contexto, principalmente quando diferenças pequenas alteram uma decisão financeira.

Termo para lembrar: Embeddings

03. Busca vetorial substitui código de produto?

Combinaria busca lexical ou estruturada para números e códigos com recuperação semântica para linguagem aberta. A decisão depende da consulta; similaridade sozinha pode aproximar conceitos e perder identificadores exatos.

Termo para lembrar: Hybrid search | Base técnica: referência 1.

04. Posso filtrar depois da geração?

Aplicaria autorização antes de expor dados ao modelo, incluindo filtros e isolamento do índice. Filtrar somente a resposta é tarde: contexto proibido já entrou numa etapa que não deveria recebê-lo.

Termo para lembrar: Authorization filtering | Base técnica: referência 2.

05. Documento manda ignorar regras: e agora?

Trataria documentos e mensagens como conteúdo não confiável para controle de ferramentas. Instruções recuperadas não podem ampliar permissões; backend valida cada ação independentemente do que o modelo interpretou ou sugeriu.

Termo para lembrar: Prompt injection | Base técnica: referência 1.

06. Citar fonte garante resposta correta?

Verificaria suporte entre afirmações e trechos, não apenas presença de citações. A resposta pode usar uma fonte real incorretamente; avaliações incluem fidelidade, validade temporal e abstenção quando evidência é insuficiente.

Termo para lembrar: Grounding

07. Modelo responde política revogada?

Versionaria corpus, validade e índice, removendo documentos revogados do conjunto elegível. Testaria caches e atrasos de propagação, porque atualizar a fonte não garante que toda consulta deixará imediatamente de recuperar informação antiga.

Termo para lembrar: Freshness | Base técnica: referência 1.

08. Como medir antes de colocar no atendimento?

Montaria avaliações com dúvidas representativas, casos adversos e erros de alta gravidade, revisados por especialistas. Métricas agregadas não bastam; compararia qualidade por categoria e impacto antes de ampliar o uso.

Termo para lembrar: Evaluation

09. IA pode aprovar reembolso sozinha?

Ferramentas financeiras receberiam argumentos validados, identidade e autorização atuais. O modelo pode propor ação dentro de um fluxo, mas o domínio decide elegibilidade e protege efeitos com transações e idempotência.

Termo para lembrar: Tool execution | Base técnica: referência 1.

10. Modelo está lento ou sem evidência?

Quando falta evidência ou o prazo acabou, ofereceria busca ou atendimento humano com contexto autorizado. Eu mediria abstenção e escalonamento, evitando que disponibilidade aparente seja mantida por respostas inventadas.

Termo para lembrar: Fallback

11. RAG atualiza os pesos do modelo automaticamente?

Não. Recupera contexto para a geração, enquanto treinamento é outro processo. Eu versionaria corpus e índices para atualizar conhecimento consultável, avaliando respostas sem presumir que o modelo aprendeu permanentemente.

Termo para lembrar: Retrieval versus training | Base técnica: referência 1.

12. Como ingere PDFs com tabelas e texto em colunas?

Eu preservaria estrutura, cabeçalhos e relação entre células durante extração, validando amostras do documento original. Texto linearizado pode associar uma tarifa à faixa errada. O pipeline registra página e localização para rastrear erros antes de gerar embeddings.

Termo para lembrar: Ingestão e layout

13. Como troca o modelo de embeddings sem interromper a busca?

Criaria um índice novo com dimensão e modelo identificados, reprocessando o corpus e comparando consultas em sombra. Não misturaria vetores incompatíveis. A troca de tráfego exige qualidade medida, cobertura completa e possibilidade de voltar ao índice anterior.

Termo para lembrar: Migração de embeddings

14. Duas perguntas parecidas podem compartilhar uma resposta cacheada?

Somente quando contexto, autorização, validade e condições de resposta também coincidirem. Similaridade textual não prova equivalência entre clientes. Eu incluiria escopo e versões na chave lógica, evitando reutilizar uma orientação personalizada ou desatualizada em outra situação.

Termo para lembrar: Cache semântico

15. Como impedir vazamento entre tenants na busca vetorial?

Aplicaria isolamento e filtros de autorização antes da recuperação ou exposição do contexto, conforme a arquitetura. Testaria consultas cruzadas e falhas do filtro, adotando comportamento seguro diante de dúvida.

Termo para lembrar: Tenant isolation | Base técnica: referência 1.

16. Prompt injection pode vir de PDF interno?

Sim, conteúdo interno também pode conter instruções indevidas ou ser comprometido. Eu trataria documentos como dados, limitaria ferramentas e manteria validações externas ao modelo para ações e acesso.

Termo para lembrar: Untrusted context | Base técnica: referência 1.

17. Você confiaria em outro modelo para dar a nota final?

Usaria avaliação automática como sinal auxiliar calibrado contra revisão humana, incluindo divergências e casos graves. O avaliador pode compartilhar vieses com o gerador. Compararia estabilidade e falsos positivos antes de permitir que a nota determine promoção automática.

Termo para lembrar: Avaliação por modelo

18. Quando a resposta deve ser abstenção?

Quando faltam evidências suficientes, permissões ou confiança avaliada para aquele risco. Eu explicaria a limitação e ofereceria busca ou atendimento humano, evitando transformar uma hipótese plausível em orientação definitiva.

Termo para lembrar: Abstention | Base técnica: referência 1.

19. Modelo maior sempre é melhor?

Não. Compararia qualidade útil, latência, custo e comportamento adversarial no conjunto de avaliação. Um modelo menor com recuperação correta pode atender melhor que outro caro com contexto inadequado.

Termo para lembrar: Model selection | Base técnica: referência 1.

20. Como medir retrieval separado da geração?

Avaliaria se os documentos corretos entram nos candidatos e no contexto final, depois se a resposta usa essas evidências adequadamente. Isso evita tentar corrigir falha de busca apenas com prompt.

Termo para lembrar: Pipeline evaluation | Base técnica: referência 1.

21. Dois documentos autorizados se contradizem; qual resposta gera?

Eu aplicaria precedência de fontes e validade definida pelo domínio, preservando a divergência quando não houver resolução confiável. Não pediria ao modelo que escolha pela redação mais convincente. A resposta indica a incerteza e encaminha a decisão responsável.

Termo para lembrar: Evidências conflitantes

22. Pode executar ferramenta financeira pelo argumento do modelo?

Os argumentos passam por validação, autorização e regras do domínio. Eu preservaria identidade da operação e exigiria confirmação quando apropriado; linguagem natural não amplia a permissão do usuário autenticado.

Termo para lembrar: Tool authorization | Base técnica: referência 1.

23. O que guardar para auditar uma resposta?

Guardaria versões, referências das evidências, política e identificadores necessários, com minimização. Não registraria indiscriminadamente dados pessoais ou prompts completos; auditabilidade precisa coexistir com segurança e retenção adequada.

Termo para lembrar: AI audit trail | Base técnica: referência 1.

24. Como limitar custo de uma conversa longa?

Definiria orçamento de contexto e chamadas, resumindo com cuidado e preservando evidências relevantes. Monitoraria custo por resolução, não só por token, e manteria fallback quando o orçamento acabar.

Termo para lembrar: Inference budget | Base técnica: referência 1.

25. O que muda ao adicionar reranker?

Ele pode melhorar ordenação dos candidatos, mas adiciona latência e custo. Eu avaliaria recall antes dele e qualidade depois, para não esperar que ranking recupere documento que nunca foi buscado.

Termo para lembrar: Reranking | Base técnica: referência 1.

26. Como evitar avaliação contaminada pelo treino?

Manteria conjuntos separados e casos novos representativos, controlando uso das avaliações na otimização. Também observaria desempenho temporal, porque memorizar um benchmark não demonstra generalização para dúvidas futuras.

Termo para lembrar: Evaluation leakage | Base técnica: referência 1.

27. Uma resposta correta pode causar dano mesmo assim?

Pode, se revelada à pessoa errada ou aplicada fora de contexto. Eu avaliaria autorização, validade temporal e consequências da ação, além da correspondência factual entre pergunta e resposta.

Termo para lembrar: Contextual correctness | Base técnica: referência 1.

28. Como fazer rollout de um assistente?

Começaria com escopo restrito e supervisão, medindo erro, escalonamento e tempo de resolução. Ampliaria apenas após evidências, com reversão operacional e caminho humano disponíveis durante falhas.

Termo para lembrar: Controlled rollout | Base técnica: referência 1.

29. Vector database é obrigatório para RAG?

Não. Recuperação pode usar busca lexical, SQL ou combinação, dependendo do corpus. Eu escolheria o mecanismo que encontra evidência adequada com acesso correto e custo aceitável para a jornada.

Termo para lembrar: Retrieval choice | Base técnica: referência 1.

30. Como explicar IA para entrevistador sem prometer perfeição?

Separaria capacidade probabilística das garantias determinísticas do sistema. Apresentaria avaliações, limites, autorização e fallback, mostrando como erros são detectados e contidos antes de atingir ações financeiras relevantes.

Termo para lembrar: Bounded autonomy | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Chunks maiores versus menores

Ganho. Preservam mais contexto próximo

Custo e motivo. Aumentam custo e podem diluir relevância

Como decidir. Quando regras dependem de explicações adjacentes

02. Busca vetorial versus lexical

Ganho. Recupera semelhança sem palavras idênticas

Custo e motivo. Pode perder precisão de códigos e termos exatos

Como decidir. Quando consultas são semânticas e abertas

03. Busca híbrida versus único método

Ganho. Combina recuperação por termo e significado

Custo e motivo. Adiciona ranking e ajuste de pesos

Como decidir. Quando corpus contém texto e identificadores

04. Filtro prévio versus pós-filtragem

Ganho. Evita expor contexto não autorizado

Custo e motivo. Pode reduzir candidatos e exigir integração de ACL

Como decidir. Quando confidencialidade é obrigatória

05. Ferramentas restritas versus autonomia ampla

Ganho. Contém ações e permite validação determinística

Custo e motivo. Limita flexibilidade e exige contratos explícitos

Como decidir. Quando efeitos possuem risco relevante

06. Verificação de evidência versus geração simples

Ganho. Melhora rastreabilidade e detecta afirmações sem suporte

Custo e motivo. Aumenta latência e custo de avaliação

Como decidir. Quando precisão factual importa

07. Reindexação rápida versus em lote

Ganho. Reduz uso de conteúdo revogado

Custo e motivo. Exige mais processamento e coordenação

Como decidir. Quando políticas mudam com urgência

08. Avaliação humana versus só métricas automáticas

Ganho. Captura nuances e gravidade do erro

Custo e motivo. Consome tempo especializado e pode variar

Como decidir. Quando decisões exigem julgamento de domínio

09. Confirmação humana versus execução automática

Ganho. Reduz exposição em ações sensíveis

Custo e motivo. Aumenta fricção e tempo de resolução

Como decidir. Quando risco supera ganho de automação

10. Abstenção versus resposta sempre disponível

Ganho. Evita apresentar hipótese sem evidência

Custo e motivo. Pode aumentar encaminhamento e trabalho humano

Como decidir. Quando confiança demonstrada é insuficiente

Duas alternativas ao caminho proposto

1. Busca tradicional

Quando faz sentido. Resposta pode ser uma lista de documentos

Como implementar. Usar ranking lexical e filtros de acesso

Limites. Usuário interpreta e sintetiza

2. Workflow determinístico

Quando faz sentido. Decisão exige regras verificáveis

Como implementar. Modelar campos regras e aprovações explícitas

Limites. Menor flexibilidade para linguagem aberta

Referências e alcance da evidência

1. Azure AI Search: RAG

Recuperação de contexto para geração. As respostas e parâmetros de aplicação são propostas autorais.

https://learn.microsoft.com/en-us/azure/search/retrieval-augmented-generation-overview

2. OWASP: autorização

Autorização por recurso, deny-by-default e testes. As respostas e parâmetros de aplicação são propostas autorais.

https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html

Continue em: T088 · T090

T090 / I. Performance e dados

UX, leituras desatualizadas e estados pendentes

Interface deve distinguir o que aconteceu do que ainda está sendo confirmado. Arquitetura de treino; não descreve implementação interna do Nubank.

Quando usar. Fluxos assíncronos, projeções atrasadas e conexões instáveis.

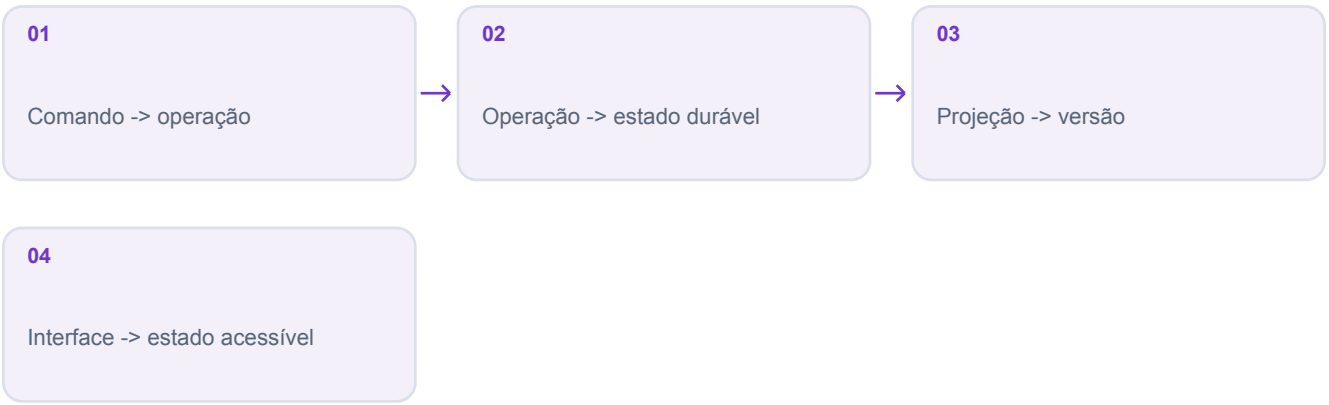
Quando evitar ou limitar. Mostrar falha definitiva após timeout ambíguo ou sucesso financeiro após mera aceitação.

Uma abertura possível

Eu começaria por mostrar operação pendente com versão conhecida. Depois explicaria reservar otimismo para mudanças reversíveis, preservando as restrições do caso.

Peças e tecnologias. React · SSE · WebSocket · ETag · Read-your-writes

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Mostrar operação pendente com versão conhecida; validar jornada antes da projeção atualizar.

2. Reservar otimismo para mudanças reversíveis; validar falha após interação otimista.

3. Propagar versão mínima após comando; validar leitura imediatamente após confirmação.

4. Mostrar idade e política de validade; validar estado com diferentes idades.

5. Retomar por cursor e buscar lacunas; validar desconexão entre duas versões.

30 perguntas para sustentar o desenho

01. O Pix saiu mas saldo ainda não mudou?

Mostraria a intenção recebida e o estado conhecido, com versão ou instante apropriado. A projeção pode atrasar; não declararia falha nem incentivaria novo envio enquanto o resultado financeiro ainda estiver sendo confirmado.

Termo para lembrar: Pending state | Base técnica: referência 1.

02. Posso mostrar reembolso como concluído?

Reservaria atualização otimista a ações com recuperação compreensível e risco aceitável. Para dinheiro, distinguiria visualmente solicitação e confirmação, porque uma animação de sucesso não pode antecipar a garantia do domínio.

Termo para lembrar: Optimistic UI | Base técnica: referência 2.

03. Como evitar que tela volte ao estado antigo?

Após comando, manteria uma versão mínima ou outro mecanismo de continuidade da sessão. Se a réplica não consegue atendê-la, aguardaria dentro do orçamento ou consultaria fonte adequada, sem regredir silenciosamente a tela.

Termo para lembrar: Read your writes | Base técnica: referência 1.

04. Dado antigo é igual a indisponível?

Modelaria dado antigo, ausência de dado e indisponibilidade como estados separados. Exibiria idade quando útil e aplicaria política por finalidade, pois uma consulta informativa pode tolerar atraso que uma decisão financeira não aceita.

Termo para lembrar: Staleness | Base técnica: referência 2.

05. Cliente perdeu eventos enquanto offline?

Retomaria por cursor ou nova consulta de estado, detectando lacunas e deduplicando atualizações. A conexão ao vivo acelera a experiência, mas o aplicativo deve conseguir reconstruí-la depois de ficar offline.

Termo para lembrar: Reconnect | Base técnica: referência 1.

06. Desabilitar botão resolve duplicidade?

Desabilitar botão ajuda experiência, mas a proteção real fica no backend com identidade de intenção. Preservaria a chave entre reenvios e validaria concorrência, incluindo duas abas e respostas perdidas.

Termo para lembrar: Duplicate clicks | Base técnica: referência 2.

07. Como leitor de tela percebe a conclusão?

Anunciaria mudanças relevantes de forma acessível sem depender só de cor ou movimento. Testaria foco, leitura e quantidade de anúncios, para que atualizações frequentes não tornem a interface confusa.

Termo para lembrar: Accessibility | Base técnica: referência 1.

08. O que dizer quando resultado é desconhecido?

Explicaria que o resultado está sendo verificado, oferecendo protocolo e acompanhamento coerentes com o prazo. Evitaria texto que afirme rejeição quando existe possibilidade de execução, especialmente se isso induzir duplicação.

Termo para lembrar: Error wording | Base técnica: referência 2.

09. Um widget caiu: esconder tudo?

Isolaria o widget afetado, preservando contexto válido e indicando indisponibilidade. Valor ausente não vira zero; ações que dependem daquele dado ficam limitadas conforme o contrato, sem bloquear toda tarefa independente.

Termo para lembrar: Partial failure | Base técnica: referência 1.

10. Histórico técnico basta ao atendente?

Traduziria evidências técnicas em marcos de negócio com autoria e referência, mantendo detalhes internos nas ferramentas adequadas. O atendente precisa explicar o estado e agir com segurança, não interpretar uma coleção de stack traces.

Termo para lembrar: Audit timeline | Base técnica: referência 2.

11. O backend respondeu 202: que texto aparece?

Eu mostraria solicitação recebida e um identificador de acompanhamento, sem afirmar execução concluída. A interface acompanharia o estado durável e explicaria eventual atraso segundo o prazo do produto.

Termo para lembrar: Accepted versus completed | Base técnica: referência 1.

12. O saldo antigo deve desaparecer durante atualização?

Depende da decisão que ele suporta. Eu poderia mantê-lo com idade e estado visíveis, mas não permitiria que uma projeção atrasada autorizasse gasto que exige validação atual no domínio.

Termo para lembrar: Stale display | Base técnica: referência 1.

13. O cliente inicia uma transferência offline: posso mostrar que ela foi enviada?

Eu distinguiria intenção armazenada no aparelho de aceitação durável pelo servidor. Mostraria rascunho ou aguardando conexão, com expiração explícita. Na reconexão, revalidaria autorização, saldo e condições antes de executar, preservando a identidade da intenção; ainda não existe transferência pendente no banco só porque o aplicativo a salvou.

Termo para lembrar: Offline command / durable acceptance

14. Quando vale usar atualização otimista?

Em ações reversíveis com recuperação compreensível, como preferências. Para dinheiro, eu diferenciaria intenção de confirmação; qualquer otimismo visual deve ter política explícita para falha e não criar promessa falsa.

Termo para lembrar: Optimistic interaction | Base técnica: referência 1.

15. Um push chega depois de logout e entrada em outra conta: como evitar exposição?

Eu vincularia notificações e resultados ao sujeito autenticado, sem renderizar conteúdo de outra conta na sessão atual. Limparia estado local por identidade e buscaria o resultado autorizado no servidor. O payload do push seria um aviso, não autorização nem fonte confiável para preencher dados financeiros.

Termo para lembrar: Account-scoped state / notification isolation

16. SSE caiu: preciso recarregar a página inteira?

Não necessariamente. Eu retomaria pelo cursor suportado ou consultaria operações ativas para sincronizar. Eventos ao vivo aceleram atualização, mas o estado durável precisa permitir reconstruir a interface.

Termo para lembrar: Resynchronization | Base técnica: referência 1.

17. Quando duas respostas chegam fora de ordem, qual aplicar?

Compararia versão e contexto da consulta. Uma resposta antiga não deve sobrescrever estado mais recente nem dados de outro cliente; cancelamento ajuda no custo, mas validação de identidade protege correção.

Termo para lembrar: Response versioning | Base técnica: referência 1.

18. Como traduzir novos estados de pagamento sem quebrar a lógica da interface?

Eu manteria códigos de estado estáveis no contrato e mensagens traduzidas numa camada separada. Estados desconhecidos teriam apresentação segura, sem inventar sucesso. Versionaria o mapeamento e testaria clientes antigos; regras de negócio nunca dependeriam de comparar texto traduzido ou mensagem interna de erro.

Termo para lembrar: Stable status codes / presentation mapping

19. O atendente precisa ver detalhes de erro de banco?

Mostraria orientação operacional e protocolo, preservando detalhes sensíveis em ferramentas autorizadas. Stack traces não ajudam o cliente e podem expor infraestrutura; o suporte precisa de contexto útil e rastreável.

Termo para lembrar: Error presentation | Base técnica: referência 1.

20. Como lidar com refresh após envio de operação?

A página recuperaria o identificador e consultaria a operação autorizada. Não usaria o carregamento como gatilho para reenviar um comando; leitura de status e execução devem ter responsabilidades distintas.

Termo para lembrar: Refresh safety | Base técnica: referência 1.

21. Leitura em réplica pode regredir o histórico?

Pode se ela estiver atrás da versão já observada. Eu preservaria watermark de sessão ou consultaria fonte capaz de atender a versão mínima, com fallback explícito quando não for possível.

Termo para lembrar: Monotonic reads | Base técnica: referência 1.

22. Qual é a diferença entre vazio, zero e indisponível?

São estados diferentes. Eu modelaria cada um no contrato: nenhuma ocorrência, valor numérico zero ou ausência de resposta confiável. Confundi-los pode levar cliente e atendente a decisões financeiras erradas.

Termo para lembrar: Semantic states | Base técnica: referência 1.

23. Como evitar spinner infinito?

Definiria deadline visual, estado de pendência persistente e ação de acompanhamento. A operação pode continuar no backend quando o contrato permite, mas a interface deve explicar a situação e permanecer navegável.

Termo para lembrar: Finite waiting | Base técnica: referência 1.

24. Como medir UX de um fluxo assíncrono?

Acompanharia tempo até aceitação, tempo até conclusão, abandono, reenvios e contatos ao suporte. Uma API rápida não demonstra jornada boa se o cliente não entende quando sua ação realmente terminou.

Termo para lembrar: Journey metrics | Base técnica: referência 1.

25. Posso estimar posição exata numa fila variável?

Somente com modelo confiável e linguagem adequada à incerteza. Eu evitaria promessa precisa quando prioridades e capacidade mudam, preferindo faixa ou prazo revisável com atualização transparente.

Termo para lembrar: Wait estimation | Base técnica: referência 1.

26. Como tratar mudança de permissão durante a sessão?

Revalidaria ações no backend e atualizaria a interface quando possível. Um botão ainda visível não conserva autorização; a resposta deve orientar o usuário sem revelar dados que ele perdeu acesso.

Termo para lembrar: Permission freshness | Base técnica: referência 1.

27. O que acontece com edição offline conflitante?

Depende do dado. Eu usaria versão e resolução explícita para alterações relevantes, evitando aplicar last-write-wins indiscriminadamente. Operações financeiras offline não podem assumir saldo e autorização atuais por conveniência.

Termo para lembrar: Conflict handling | Base técnica: referência 1.

28. Pode mostrar timestamp do servidor diretamente?

Armazenaria instantes coerentes e apresentaria timezone compreensível, preservando contexto para prazos. Datas locais ambíguas precisam de cuidado; o cliente deve entender qual calendário rege a conclusão prometida.

Termo para lembrar: Temporal UX | Base técnica: referência 1.

29. Como testar estados raros de interface?

Criaria cenários de rede lenta, retorno duplicado, versão antiga, permissão revogada e resultado desconhecido. Verificaria texto, acessibilidade e ações permitidas, não apenas a aparência do caminho feliz.

Termo para lembrar: Failure-state testing | Base técnica: referência 1.

30. Qual fonte autoriza mensagem de operação concluída?

A confirmação definida pelo contrato do domínio, não o término de uma animação ou publicação em fila. Eu mapearia eventos técnicos para estados de negócio e documentaria essa correspondência.

Termo para lembrar: Business confirmation | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Estado pendente versus resposta final imediata

Ganho. Representa honestamente processamento assíncrono

Custo e motivo. Exige acompanhamento e comunicação de prazo

Como decidir. Quando conclusão não cabe na interação inicial

02. UI otimista versus confirmação antes de exibir

Ganho. Melhora sensação de resposta em ações simples

Custo e motivo. Exige desfazer ou explicar falha posterior

Como decidir. Quando ação é reversível e risco é baixo

03. Versão mínima versus leitura de qualquer réplica

Ganho. Evita regressão após escrita observada

Custo e motivo. Pode aumentar espera ou carga no writer

Como decidir. Quando continuidade da sessão é necessária

04. Exibir stale versus esconder dado

Ganho. Mantém contexto durante degradação

Custo e motivo. Usuário precisa entender idade e limites

Como decidir. Quando informação antiga ainda tem utilidade segura

05. Push de estado versus polling

Ganho. Reduz espera e consultas repetidas

Custo e motivo. Mantém conexões e lógica de reconexão

Como decidir. Quando atualizações frequentes justificam transporte ao vivo

06. Persistir intenção no cliente versus nova chave por clique

Ganho. Facilita retomar mesma operação

Custo e motivo. Exige ciclo de vida e armazenamento local cuidadosos

Como decidir. Quando reenvios pertencem à mesma intenção

07. Anúncios acessíveis versus apenas alteração visual

Ganho. Atende usuários de tecnologia assistiva

Custo e motivo. Exige desenho de foco e controle de ruído

Como decidir. Quando a interface muda estado dinamicamente

08. Mensagem detalhada versus genérica

Ganho. Ajuda entender próximo passo

Custo e motivo. Pode expor informação indevida se mal delimitada

Como decidir. Quando contexto pode ser apresentado com segurança

09. Degradação parcial versus bloqueio total

Ganho. Permite tarefas independentes continuarem

Custo e motivo. Exige estados e dependências por funcionalidade

Como decidir. Quando componentes têm falhas isoláveis

10. Timeline de negócio versus logs brutos

Ganho. Reduz esforço de investigação do suporte

Custo e motivo. Exige tradução e manutenção de semântica

Como decidir. Quando operadores precisam explicar a jornada

Duas alternativas ao caminho proposto

1. Leitura autoritativa

Quando faz sentido. Ação exige confirmação imediata do estado

Como implementar. Consultar fonte de verdade com orçamento

Limites. Maior latência e carga

2. Projeção versionada

Quando faz sentido. A jornada tolera pequeno atraso

Como implementar. Aguardar versão mínima ou informar pendência

Limites. Requer versões e tratamento de timeout

Referências e alcance da evidência

1. Azure: CQRS

Separação de escrita/leitura e projeções. As respostas e parâmetros de aplicação são propostas autorais.

https://learn.microsoft.com/en-us/azure/architecture/patterns/cqrs

2. W3C: status messages

Comunicação acessível de mudanças de estado. As respostas e parâmetros de aplicação são propostas autorais.

https://www.w3.org/WAI/WCAG22/Understanding/status-messages.html

Continue em: T089 · T091

T091 / J. Sistemas bancários

Ledger, partidas dobradas e imutabilidade

Ledger preserva lançamentos equilibrados e a história de mudanças financeiras. Arquitetura de treino; não descreve implementação interna do Nubank.

Quando usar. Quando dinheiro exige auditoria, reconciliação e invariantes transacionais.

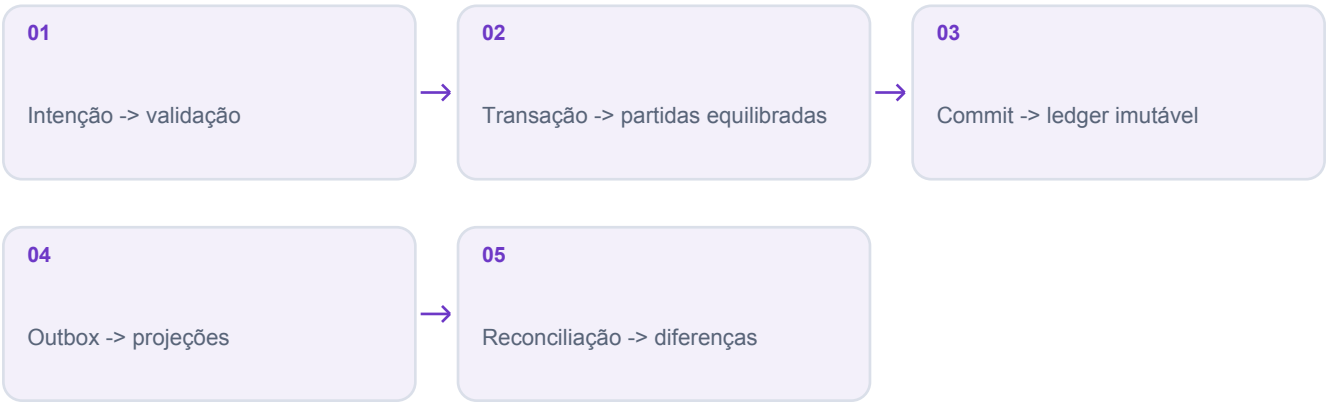
Quando evitar ou limitar. Confundir append-only com ausência de controle de saldo ou com event sourcing obrigatório.

Uma abertura possível

Eu começaria por registrar débito e crédito na mesma moeda. Depois explicaria adicionar reversão vinculada ao lançamento original, preservando as restrições do caso.

Peças e tecnologias. PostgreSQL · Datomic · TigerBeetle · Decimal · Outbox

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Registrar débito e crédito na mesma moeda; validar soma assinada por transação.

2. Adicionar reversão vinculada ao lançamento original; validar reconstrução antes e depois da correção.

3. Confirmar todas as partidas na mesma fronteira; validar interrupção entre preparação e commit.

4. Reutilizar identidade estável reconhecida pelo ledger; validar duas tentativas com mesma intenção.

5. Validar disponibilidade dentro da serialização necessária; validar disputa simultânea pelo último saldo.

30 perguntas para sustentar o desenho

01. Como registrar transferência de cento e cinquenta?

Registraria partidas equilibradas dentro da mesma transação financeira, usando valor e moeda explícitos. A convenção de débito e crédito depende da natureza das contas; o teste verifica conservação sem somar moedas diferentes.

Termo para lembrar: Double entry | Base técnica: referência 1.

02. Como corrigir lançamento errado?

Corrigiria por lançamento de reversão vinculado, preservando a operação original e a justificativa. O novo efeito possui identidade própria e limites, evitando que replay da correção devolva mais que o permitido.

Termo para lembrar: Append only | Base técnica: referência 2.

03. Duas entradas podem confirmar separadamente?

Confirmaria o conjunto completo na fronteira autoritativa do ledger. Não publicaria metade das partidas como sucesso; validaria essa propriedade com falhas entre preparação e commit e com entradas deliberadamente desequilibradas.

Termo para lembrar: Atomicity | Base técnica: referência 1.

04. Resposta perdida permite lançar novamente?

Todas as tentativas da mesma intenção preservam identidade reconhecida pelo ledger. Deduplicação e efeito precisam estar protegidos juntos; criar novo identificador depois de timeout transforma recuperação técnica em outra operação possível.

Termo para lembrar: Idempotency | Base técnica: referência 2.

05. Saldo calculado antes do commit é seguro?

Validaria disponibilidade dentro da serialização necessária, não em consulta anterior sem proteção. Dois débitos concorrentes disputam o mesmo limite; escolheria transação, condição ou engine que preserve esse invariante e mediria contenção.

Termo para lembrar: Concurrency | Base técnica: referência 1.

06. Como saber saldo num momento passado?

Separaria momento de registro e vigência contábil quando o domínio exigir. Uma consulta histórica precisa dizer o que era conhecido e o que era efetivo, principalmente diante de lançamento tardio ou correção.

Termo para lembrar: Time travel | Base técnica: referência 2.

07. Posso somar reais e dólares juntos?

Equilibraria cada moeda separadamente, com contas apropriadas para posição e liquidação cambial. Preservaria vínculo e quote entre as pernas, porque somar números de unidades diferentes não demonstra conservação financeira.

Termo para lembrar: Multi-currency | Base técnica: referência 1.

08. Redis pode autorizar débito pelo saldo?

Redis ou outra projeção pode servir leitura com contrato explícito, mas não concederia autorização financeira baseada em atraso desconhecido. A decisão de débito consulta ou atualiza a autoridade que protege disponibilidade.

Termo para lembrar: Projection | Base técnica: referência 2.

09. Hash de evento prova ausência de fraude?

Combinaria trilha, controle de acesso, validação e reconciliação. Hashes ajudam evidência em certos modelos, mas não provam que a operação era legítima nem impedem uma escrita indevida feita por identidade autorizada.

Termo para lembrar: Audit | Base técnica: referência 1.

10. Conta concentradora vira gargalo: o que fazer?

Mediria a conta concentradora e tentaria reduzir trabalho na seção serializada antes de repartir. Subcontas podem ajudar somente se a consolidação e os limites continuarem corretos; paralelismo não substitui semântica financeira.

Termo para lembrar: Scale | Base técnica: referência 2.

11. Saldo pode ser um campo materializado no ledger?

Pode, desde que seja atualizado atomicamente com as partidas e protegido pelos mesmos invariantes. Eu preservaria histórico e possibilidade de recomputação; imutabilidade dos lançamentos não proíbe projeções eficientes e verificáveis.

Termo para lembrar: Materialized balance | Base técnica: referência 1.

12. Como representar R$ 150 sem float?

Usaria inteiro em centavos ou decimal com escala explícita, sempre associado à moeda. Validaria limites e regras de arredondamento, evitando converter valores monetários por tipos binários durante serialização ou cálculo.

Termo para lembrar: Money representation | Base técnica: referência 1.

13. Qual diferença entre débito e valor negativo?

Débito e crédito são lados contábeis cujo efeito depende da natureza da conta. Eu definiria convenção formal e validaria partidas; apenas armazenar números negativos não estabelece modelo contábil correto.

Termo para lembrar: Debit-credit semantics | Base técnica: referência 1.

14. Uma transferência entre dois shards é uma transação local?

Não automaticamente. Eu precisaria de coordenação, modelo de liquidação ou fronteira alternativa que preserve obrigações. Não prometeria atomicidade entre bancos independentes apenas porque ambos gravam eventos imutáveis.

Termo para lembrar: Cross-shard transfer | Base técnica: referência 1.

15. O que uma conta de compensação resolve?

Representa uma posição intermediária identificável durante trânsito ou liquidação. Eu reconciliaria seus saldos e prazos; a conta ajuda modelagem, mas não faz uma integração externa tornar-se atomicamente confirmada.

Termo para lembrar: Clearing account | Base técnica: referência 1.

16. Posso apagar conta quando cliente encerra relacionamento?

Separaria estado operacional de retenção financeira necessária. A conta pode deixar de aceitar novas operações enquanto referências históricas permanecem protegidas, observando políticas aplicáveis e minimização dos dados pessoais associados.

Termo para lembrar: Account closure | Base técnica: referência 1.

17. Como impedir que aplicação grave só uma partida?

Encapsularia escrita em operação transacional que valida o conjunto e restringiria caminhos de acesso. Testaria interrupções e entradas desequilibradas, porque disciplina de desenvolvedor sem controle efetivo não garante conservação.

Termo para lembrar: Invariant enforcement | Base técnica: referência 1.

18. O transactor elimina qualquer concorrência no sistema?

Ele pode serializar alterações dentro da fronteira da engine, conforme contrato. Serviços externos e múltiplos bancos continuam concorrentes; eu não extrapolaria garantia local para pagamentos, arquivos e projeções distribuídas.

Termo para lembrar: Transaction boundary | Base técnica: referência 1.

19. Como lidar com lançamento retroativo?

Registraria separadamente momento de processamento e data efetiva, com política de fechamento. Consultas históricas precisam declarar qual dimensão usam; uma correção posterior não deve apagar o que era conhecido anteriormente.

Termo para lembrar: Bitemporal accounting | Base técnica: referência 1.

20. Como deduplicar uma reversão?

A reversão recebe identidade própria vinculada à operação original. Eu limitaria valor reversível e consumiria esse limite atomicamente, evitando que duas solicitações distintas devolvam mais do que foi debitado.

Termo para lembrar: Reversal identity | Base técnica: referência 1.

21. O que fazer com lançamento importado sem identificação confiável?

Eu colocaria em quarentena ou processo de matching controlado, preservando origem. Não inferiria unicidade apenas por valor e horário quando operações legítimas podem coincidir nesses campos.

Termo para lembrar: Import integrity | Base técnica: referência 1.

22. Como publicar evento após commit do ledger?

Usaria mecanismo transacional ou log suportado para registrar a obrigação. Consumidores seriam idempotentes; uma publicação repetida não pode reproduzir crédito, e falha de publicação não pode perder a obrigação.

Termo para lembrar: Ledger event delivery | Base técnica: referência 1.

23. Qual teste comprova conservação?

Geraria sequências de transferências, reservas e reversões, verificando equilíbrio por moeda após cada commit. Também testaria falhas e duplicatas, comparando projeções com reconstrução independente do histórico confirmado.

Termo para lembrar: Conservation testing | Base técnica: referência 1.

24. Leitura de saldo pode ser eventual?

Para exibição, talvez, com contrato claro; para autorizar gasto, a decisão precisa da consistência exigida. Eu separaria finalidade da leitura em vez de classificar todo banco como forte ou eventual.

Termo para lembrar: Purpose-specific consistency | Base técnica: referência 1.

25. Como reagir a corrupção lógica descoberta?

Suspenderia o caminho afetado, preservaria evidências e reconciliaria extensão do problema. Correções seriam lançamentos autorizados ou restauração planejada, sem reexecutar efeitos externos automaticamente a partir de estado antigo.

Termo para lembrar: Logical recovery | Base técnica: referência 1.

26. Backup restaurado pode repetir pagamentos já feitos?

Sim, pode perder registros locais de efeitos externos confirmados depois do ponto restaurado. Eu reconciliaria identidades e resultados antes de retomar executores, preservando idempotência no destino.

Termo para lembrar: Restore reconciliation | Base técnica: referência 1.

27. Como calcular throughput máximo por conta?

Mediria contenção e tempo de transação na chave quente. Aumentar consumidores não divide um invariante indivisível; avaliaria agregação ou subcontas somente se o modelo financeiro permitir.

Termo para lembrar: Per-account capacity | Base técnica: referência 1.

28. Todas entradas precisam carregar PII?

Não. Eu manteria referências e dados contábeis necessários, separando informações pessoais quando apropriado. Isso reduz exposição e facilita governança, sem prometer que criptografia sozinha satisfaz toda obrigação de exclusão.

Termo para lembrar: Data minimization | Base técnica: referência 1.

29. É obrigatório usar event sourcing num banco?

Não. Um modelo relacional com partidas imutáveis e estado transacional pode atender. Eu escolheria pela correção, operação e evolução, distinguindo histórico financeiro obrigatório de uma arquitetura completa de eventos.

Termo para lembrar: Architectural choice | Base técnica: referência 1.

30. Como explicaria o ledger numa frase de entrevista?

Eu diria que ele registra efeitos financeiros equilibrados, verificáveis e idempotentes dentro de uma fronteira explícita. Depois mostraria como projeções e integrações preservam essa identidade sem fingir atomicidade global.

Termo para lembrar: Ledger contract | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Partidas dobradas versus registro simples de movimento

Ganho. Facilita conservação e reconciliação contábil

Custo e motivo. Exige plano de contas, contrapartidas e mais registros

Como decidir. Quando dinheiro precisa de trilha e equilíbrio verificáveis

02. Append-only versus edição de lançamentos

Ganho. Preserva história e explicação de correções

Custo e motivo. Aumenta armazenamento e complexidade de consultas históricas

Como decidir. Quando auditoria exige conhecer o estado anterior

03. Transação única versus coordenação entre stores

Ganho. Oferece atomicidade local clara

Custo e motivo. Limita fronteira e pode concentrar contenção

Como decidir. Quando as partidas cabem na mesma autoridade

04. Deduplicação durável versus cache temporário

Ganho. Protege replay após reinício

Custo e motivo. Consome armazenamento e requer política de retenção

Como decidir. Quando repetição de efeito é inaceitável

05. Serialização por conta versus concorrência livre

Ganho. Protege limite compartilhado

Custo e motivo. Reduz paralelismo de uma chave quente

Como decidir. Quando decisões disputam o mesmo saldo

06. Modelo bitemporal versus timestamp único

Ganho. Distingue vigência e conhecimento histórico

Custo e motivo. Amplia schema e complexidade das consultas

Como decidir. Quando registros tardios precisam de explicação

07. Contas por moeda versus saldo convertido único

Ganho. Preserva unidades e posições originais

Custo e motivo. Exige consolidação e regras de câmbio

Como decidir. Quando existem obrigações em moedas diferentes

08. Projeção de leitura versus consulta direta ao ledger

Ganho. Escala consultas específicas e rápidas

Custo e motivo. Introduz atraso e custo de reconstrução

Como decidir. Quando exibição tolera consistência contratada

09. Trilha detalhada versus registro mínimo

Ganho. Melhora investigação e reprodução

Custo e motivo. Aumenta dados, retenção e governança

Como decidir. Quando valor de evidência supera custo e exposição

10. Subcontas versus conta concentradora única

Ganho. Distribui trabalho quando semântica permite

Custo e motivo. Exige consolidação e pode complicar limites globais

Como decidir. Quando contenção medida justifica alterar o modelo

Duas alternativas ao caminho proposto

1. Relacional transacional

Quando faz sentido. Volume e invariantes cabem numa fronteira

Como implementar. Registrar partidas e controles na mesma transação

Limites. Contenção exige medição e particionamento cuidadoso

2. Engine especializada

Quando faz sentido. Carga e domínio justificam investimento

Como implementar. Usar contas transferências e garantias nativas

Limites. Integração e semântica precisam ser aprendidas

Referências e alcance da evidência

1. TigerBeetle: financial accounting

Contas e registro de transferências equilibradas. As respostas e parâmetros de aplicação são propostas autorais.

https://docs.tigerbeetle.com/coding/financial-accounting/

2. TigerBeetle: two-phase transfers

Reservar, confirmar ou cancelar transferências pendentes. As respostas e parâmetros de aplicação são propostas autorais.

https://docs.tigerbeetle.com/coding/two-phase-transfers/

Continue em: T090 · T092

T092 / J. Sistemas bancários

Saldo disponível, reservas e limites

Saldo contábil, saldo disponível e reserva são estados distintos de um compromisso financeiro. Arquitetura de treino; não descreve implementação interna do Nubank.

Quando usar. Autorizações de cartão, Pix, reservas e limites de crédito.

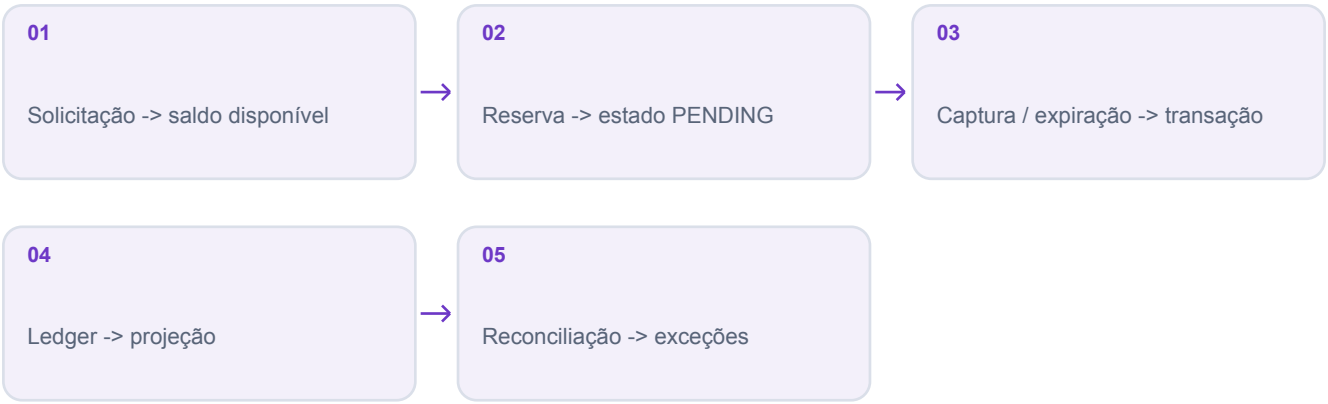
Quando evitar ou limitar. Usar Redis write-behind como única proteção de gasto concorrente.

Uma abertura possível

Eu começaria por validar saldo e reserva atomicamente. Depois explicaria persistir identidade da autorização no destino, preservando as restrições do caso.

Peças e tecnologias. PostgreSQL · TigerBeetle · CAS · Expiry workers

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Validar saldo e reserva atomicamente; validar duas reservas sobre mesmo limite.

2. Persistir identidade da autorização no destino; validar chave repetida com mesmo conteúdo.

3. Consumir reserva e contabilizar no mesmo limite; validar repetição da confirmação externa.

4. Capturar valor permitido e liberar diferença; validar valores menores e múltiplas capturas.

5. Serializar transições pelo estado e versão; validar corrida no instante de expiração.

30 perguntas para sustentar o desenho

01. Saldo de mil permite reservar mil duas vezes?

A reserva consome disponibilidade por uma atualização atômica que verifica o limite. Duas consultas anteriores podem observar o mesmo saldo, então testaria disputa simultânea pela última capacidade antes de considerar o desenho correto.

Termo para lembrar: Available balance | Base técnica: referência 1.

02. Retentativa cria nova reserva?

Preservaria identidade da autorização e seu conteúdo em todas as tentativas. Se a resposta se perder, a consulta ou repetição retorna a mesma reserva; alterar valor exige comando e validação próprios.

Termo para lembrar: Hold identity | Base técnica: referência 2.

03. Como converter reserva em lançamento?

A captura transiciona a reserva e registra o efeito financeiro coerentemente. Eu verificaria remanescente e identidade da captura, impedindo que retornos repetidos consumam ou contabilizem novamente o mesmo compromisso.

Termo para lembrar: Capture | Base técnica: referência 1.

04. Compra final ficou menor que reserva?

Aplicaria a política de captura parcial e liberação do restante, mantendo valores acumulados verificáveis. Se múltiplas capturas forem permitidas, sua soma precisa respeitar o remanescente numa fronteira transacional apropriada.

Termo para lembrar: Partial capture | Base técnica: referência 2.

05. Expiração e captura chegam juntas?

Expiração e captura competem pela mesma transição versionada. Uma delas confirma conforme regra; a outra reavalia estado. O instante do relógio sozinho não protege contra dois workers alterando disponibilidade simultaneamente.

Termo para lembrar: Expiry race | Base técnica: referência 1.

06. Limite caiu com reservas em aberto?

Trataria compromissos já aceitos e novos pedidos separadamente ao reduzir limite. A política define exposição e bloqueios futuros; não apagaria reservas existentes apenas para fazer o novo valor parecer imediatamente disponível.

Termo para lembrar: Limit changes | Base técnica: referência 2.

07. Timeout permite liberar reserva imediatamente?

Timeout remoto pode deixar o compromisso incerto. Eu manteria reserva conforme validade e risco acordados e buscaria confirmação, evitando liberar fundos só porque o processo local deixou de esperar uma resposta.

Termo para lembrar: Pending uncertainty | Base técnica: referência 1.

08. Como escalar carteira de alto movimento?

Começaria medindo contenção, duração de transação e admissão por conta. Uma conta quente pode exigir modelo diferente, mas contadores eventualmente consistentes não preservam automaticamente o limite financeiro global prometido.

Termo para lembrar: Hot account | Base técnica: referência 2.

09. Cliente vê saldo maior que disponível?

Mostraria saldo e reservas com significado claro e versão apropriada, diferenciando contabilizado e disponível. O app pode atrasar, mas a autorização final revalida na fonte responsável para não permitir gasto indevido.

Termo para lembrar: Projection lag | Base técnica: referência 1.

10. Como desfazer captura confirmada?

Reversão é outra intenção, vinculada à captura original e limitada ao valor reversível. Ela mantém histórico e deduplicação próprios, permitindo recuperação sem transformar cada retorno repetido em um crédito adicional.

Termo para lembrar: Reversal | Base técnica: referência 2.

11. Qual fórmula representa saldo disponível?

Depende da convenção, mas eu explicitaria saldo contabilizado, reservas ativas e limites autorizados. Não somaria campos sem definir sinais e natureza das contas; testaria a fórmula em cenários reais.

Termo para lembrar: Available balance formula | Base técnica: referência 1.

12. Uma reserva pendente já aparece no saldo contábil?

Depende do modelo, mas deve ser distinguível de lançamento confirmado. Eu explicaria ambas as visões ao cliente e impediria que a separação permitisse gastar recursos já comprometidos.

Termo para lembrar: Pending versus posted | Base técnica: referência 1.

13. Como reservar valor quando há limite de crédito?

A regra precisa considerar exposição total e limites aplicáveis, não apenas dinheiro depositado. Eu validaria o compromisso na fronteira autoritativa e registraria qual fonte de disponibilidade foi utilizada.

Termo para lembrar: Credit exposure | Base técnica: referência 1.

14. Duas capturas parciais podem ocorrer juntas?

Podem, se o contrato permitir, mas a soma precisa respeitar o remanescente reservado. Eu atualizaria esse limite atomicamente e preservaria identidade de cada captura para tolerar duplicatas.

Termo para lembrar: Partial capture concurrency | Base técnica: referência 1.

15. O que acontece quando captura excede reserva?

Aplicaria a política do produto: rejeitar, exigir autorização adicional ou outro fluxo permitido. Não aceitaria automaticamente valor maior só porque existe uma reserva com o mesmo identificador.

Termo para lembrar: Overcapture policy | Base técnica: referência 1.

16. Como evitar reserva órfã?

Registraria prazo, estado e rotina de revisão, com métricas de idade. A limpeza respeitaria confirmação externa e concorrência; expirar cegamente por relógio pode liberar um compromisso ainda válido.

Termo para lembrar: Orphan reservation | Base técnica: referência 1.

17. Pode usar TTL do Redis para liberar saldo?

Eu não faria dele a única autoridade financeira. A expiração precisa gerar transição durável e verificável na fonte responsável, com proteção contra captura concorrente e reconstrução após falha.

Termo para lembrar: Durable expiry | Base técnica: referência 1.

18. Reserva expirada pode ser reativada?

Somente por nova decisão válida e identidade adequada, após verificar disponibilidade. Reativação silenciosa pode consumir recursos já usados por outra operação; eu preservaria a história e o vínculo com a tentativa.

Termo para lembrar: Reservation renewal | Base técnica: referência 1.

19. Cliente cancela enquanto parceiro confirma: quem vence?

A máquina de estados e o contrato definem precedência. Eu serializaria a decisão local e trataria confirmação tardia por reconciliação ou compensação, evitando presumir que cancelamento de UI revoga efeito remoto.

Termo para lembrar: Cancellation race | Base técnica: referência 1.

20. Como calcular exposição com múltiplas moedas?

Manteria valores e limites por moeda, com conversão versionada quando a política exigir consolidar risco. A taxa usada na autorização precisa ser identificável; somar unidades diferentes não mede exposição.

Termo para lembrar: Multi-currency exposure | Base técnica: referência 1.

21. Conta conjunta exige limite por pessoa ou conta?

Eu esclareceria a regra antes do desenho. Poderiam existir limites agregados e individuais, exigindo validação conjunta ou reserva de capacidade; particionar por usuário sem considerar a conta pode quebrar o teto.

Termo para lembrar: Shared limit | Base técnica: referência 1.

22. Como reduzir contenção sem abandonar correção?

Encurtaria transações, ajustaria índices e limitaria admissão antes de mudar o modelo. Reservas hierárquicas ou particionadas só entram se a política permitir alocar sub-limites sem ultrapassar o total.

Termo para lembrar: Contention reduction | Base técnica: referência 1.

23. Qual evento produz atualização de saldo no app?

Uma transição financeira confirmada ou reserva durável com versão identificável. A projeção aplica eventos idempotentemente e mostra pendências; não depende de interpretar timeout do cliente como alteração de saldo.

Termo para lembrar: Balance projection | Base técnica: referência 1.

24. Quando devolver saldo após autorização recusada?

Após evidência de rejeição conforme contrato e transição válida da reserva. Eu preservaria a referência e impediria que uma confirmação atrasada consumisse novamente recursos já liberados sem tratamento explícito.

Termo para lembrar: Release confirmation | Base técnica: referência 1.

25. Como reconstruir disponíveis após desastre?

Reconciliaria lançamentos, reservas e resultados externos, usando identidades estáveis. Recriar somente saldo contábil deixa compromissos pendentes de fora e pode liberar capacidade financeira que já estava comprometida.

Termo para lembrar: Balance recovery | Base técnica: referência 1.

26. Qual diferença entre limite e reserva?

Limite define capacidade permitida; reserva consome temporariamente parte dessa capacidade para uma intenção. Eu modelaria alterações e expiração separadamente, evitando tratar redução de limite como cancelamento automático de compromissos.

Termo para lembrar: Limit versus hold | Base técnica: referência 1.

27. Como testar uma reserva de valor zero?

Definiria se o domínio permite e qual significado possui. Casos zero, negativos e máximos precisam de validação explícita para evitar atalhos que contornem autorização, contabilização ou regras de disponibilidade.

Termo para lembrar: Boundary values | Base técnica: referência 1.

28. É seguro compensar liberando o valor original inteiro?

Nem sempre: pode existir captura parcial ou liberação anterior. Eu calcularia o remanescente dentro da transação e usaria identidade da compensação, evitando devolver capacidade maior que a ainda comprometida.

Termo para lembrar: Remaining amount | Base técnica: referência 1.

29. Como auditor investiga uma retenção indevida?

Eu apresentaria intenção, valores, estado, prazos e todas as transições autorizadas, incluindo referências externas. A evidência precisa permitir distinguir falta de retorno, falha de projeção e erro efetivo de liberação.

Termo para lembrar: Hold audit | Base técnica: referência 1.

30. O que monitorar além de quantidade de reservas?

Valor exposto, idade, expirações, captura tardia e divergência com parceiros. Uma única reserva grande esquecida pode ser mais importante que milhares pequenas; alertas devem refletir impacto e prazo.

Termo para lembrar: Exposure monitoring | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Reserva prévia versus débito direto

Ganho. Protege capacidade enquanto aguarda confirmação

Custo e motivo. Introduz estados, expiração e saldo temporariamente bloqueado

Como decidir. Quando autorização e execução ocorrem em momentos diferentes

02. ID de intenção persistido versus tentativa independente

Ganho. Evita duplicar compromisso durante retry

Custo e motivo. Requer retenção e comparação de conteúdo

Como decidir. Quando resposta pode se perder após reserva

03. Captura atômica versus etapas locais separadas

Ganho. Mantém compromisso e lançamento coerentes

Custo e motivo. Pode concentrar escrita numa fronteira

Como decidir. Quando os estados precisam mudar juntos

04. Captura parcial versus somente integral

Ganho. Atende valor final variável

Custo e motivo. Exige controle de remanescente e mais transições

Como decidir. Quando o contrato admite execução fracionada

05. Expiração curta versus longa

Ganho. Libera capacidade não utilizada mais cedo

Custo e motivo. Pode aumentar casos de confirmação tardia

Como decidir. Quando prazo e comportamento do parceiro permitem

06. Honrar compromissos versus reduzir exposição imediatamente

Ganho. Preserva obrigações previamente aceitas

Custo e motivo. Mantém risco até resolução das reservas

Como decidir. Quando política não permite revogar compromisso unilateralmente

07. Manter UNKNOWN versus liberar automaticamente

Ganho. Reduz risco de gastar fundos já comprometidos

Custo e motivo. Pode indisponibilizar saldo ao cliente por mais tempo

Como decidir. Quando há possibilidade de execução remota

08. Lock por conta versus CAS com retry

Ganho. Simplifica exclusão numa transação curta

Custo e motivo. Espera de lock limita concorrência quente

Como decidir. Quando conflitos são frequentes e operação é curta

09. Projeção explicativa versus único número de saldo

Ganho. Ajuda cliente entender reservas e pendências

Custo e motivo. Aumenta complexidade de interface e contrato

Como decidir. Quando etapas financeiras possuem estados distintos

10. Reversão explícita versus editar captura

Ganho. Preserva evidência e limita efeito corretivo

Custo e motivo. Adiciona registros e reconciliação

Como decidir. Quando correção financeira precisa ser auditável

Duas alternativas ao caminho proposto

1. Débito direto

Quando faz sentido. Não existe espera ou confirmação posterior

Como implementar. Validar e contabilizar numa transação

Limites. Rejeição posterior exige reversão

2. Reserva em duas fases

Quando faz sentido. Há autorização seguida de confirmação

Como implementar. Reservar e depois capturar ou liberar

Limites. Expiração e captura concorrentes exigem controle

Referências e alcance da evidência

1. TigerBeetle: two-phase transfers

Reservar, confirmar ou cancelar transferências pendentes. As respostas e parâmetros de aplicação são propostas autorais.

https://docs.tigerbeetle.com/coding/two-phase-transfers/

2. PostgreSQL: isolamento

Anomalias de concorrência e níveis de isolamento. As respostas e parâmetros de aplicação são propostas autorais.

https://www.postgresql.org/docs/current/transaction-iso.html

Continue em: T091 · T093

T093 / J. Sistemas bancários

Precisão monetária, arredondamento e câmbio

Dinheiro exige unidade, escala e arredondamento explícitos em cada fronteira. Arquitetura de treino; não descreve implementação interna do Nubank.

Quando usar. Cobrança, impostos, juros, rateios e operações de câmbio.

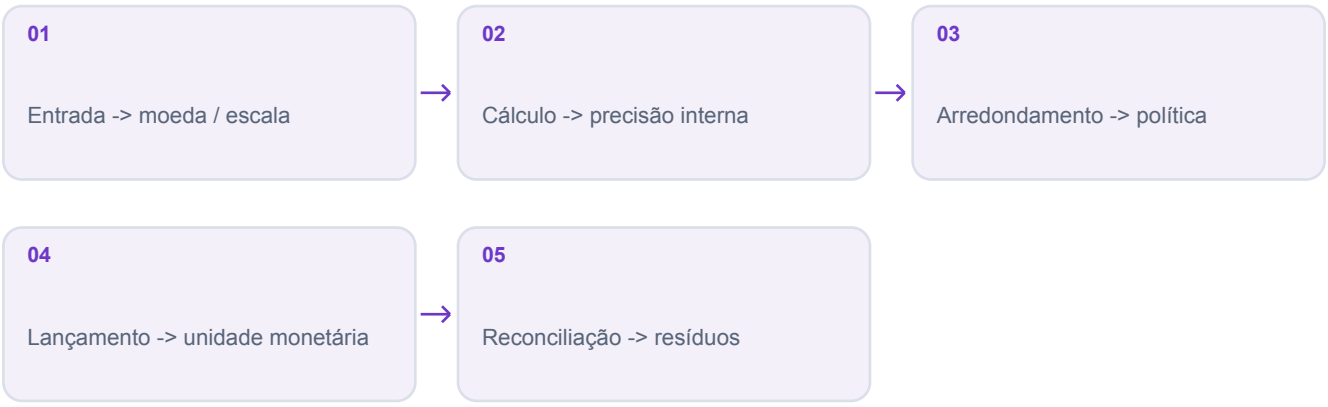
Quando evitar ou limitar. Tratar float binário ou duas casas decimais como representação universal.

Uma abertura possível

Eu começaria por representar valor com decimal ou escala inteira. Depois explicaria modelar escala por moeda e operação, preservando as restrições do caso.

Peças e tecnologias. Decimal · BigDecimal · Inteiros escalados · ISO 4217

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Representar valor com decimal ou escala inteira; validar somas repetidas e comparações exatas.

2. Modelar escala por moeda e operação; validar valores de moedas com escalas distintas.

3. Versionar regra de arredondamento por cálculo; validar casos exatamente no ponto médio.

4. Distribuir resíduo com critério determinístico; validar soma de todas as parcelas.

5. Preservar precisão até fronteiras definidas; validar comparação com cálculo de referência.

30 perguntas para sustentar o desenho

01. Por que não guardar dinheiro em double?

Usaria decimal ou inteiro escalado associado à moeda, com conversões explícitas. Float binário pode introduzir aproximação incompatível com o contrato; testaria soma, serialização e fronteiras de valor entre linguagens.

Termo para lembrar: Binary float | Base técnica: referência 1.

02. Toda moeda tem duas casas?

Modelaria escala por moeda e operação, verificando o contrato do provedor. Duas casas não são regra universal; a API precisa distinguir unidade de armazenamento, precisão de cálculo e apresentação ao cliente.

Termo para lembrar: Minor units | Base técnica: referência 2.

03. Metade arredonda sempre para cima?

Escolheria modo de arredondamento pelo produto e o versionaria com exemplos de referência. Não deixaria o resultado depender do default de uma biblioteca, especialmente quando serviços diferentes calculam a mesma tarifa.

Termo para lembrar: Rounding mode | Base técnica: referência 1.

04. Como repartir cem reais entre três?

Distribuiria o resíduo por critério determinístico acordado, mantendo a soma igual ao original. Reexecutar o rateio deve produzir as mesmas parcelas, e a auditoria precisa explicar quem recebeu o centavo adicional.

Termo para lembrar: Allocation | Base técnica: referência 2.

05. Posso arredondar a cada multiplicação?

Preservaria precisão suficiente durante o cálculo e arredondaria nas fronteiras definidas. Arredondar a cada operação pode acumular diferenças; compararia implementações com casos extremos e referência matemática adequada ao contrato.

Termo para lembrar: Intermediate precision | Base técnica: referência 1.

06. Inteiro elimina todos os riscos?

Inteiros ainda têm faixa finita. Eu validaria valores individuais e agregados com operações verificadas, falhando explicitamente no limite em vez de permitir overflow silencioso alterar sinal ou magnitude de uma cobrança.

Termo para lembrar: Overflow | Base técnica: referência 2.

07. Cotação consultada basta para garantir preço?

Persistiria quote, moedas, taxa, validade e condições de aceitação. Uma cotação informativa não garante execução; confirmaria que a oferta ainda pode ser aceita antes de assumir compromisso de valor com o cliente.

Termo para lembrar: FX quote | Base técnica: referência 1.

08. Tarifa entra antes ou depois do câmbio?

Definiria ordem e moeda de cada tarifa e conversão, preservando parâmetros usados. Aplicar taxa antes ou depois do câmbio pode mudar resultado; exemplos compartilhados com produto evitam interpretações divergentes entre serviços.

Termo para lembrar: Fees | Base técnica: referência 2.

09. JSON number mantém decimal corretamente?

Escolheria representação de intercâmbio que preserve valor e unidade, como string decimal contratada ou inteiro escalado. Testaria roundtrip nos clientes, evitando parsing por locale e conversão inadvertida para precisão insuficiente.

Termo para lembrar: Serialization | Base técnica: referência 1.

10. Onde colocar diferença de arredondamento?

Registraria diferenças autorizadas numa conta ou mecanismo explícito de ajuste, com causa e limites. Resíduo recorrente merece investigação; não usaria tolerância para encobrir erro sistemático no cálculo ou no parceiro.

Termo para lembrar: Residual account | Base técnica: referência 2.

11. Construir Decimal a partir de float é seguro?

Eu evitaria quando a origem precisa ser exata, pois a aproximação binária pode entrar na representação. Preferiria string validada ou inteiro escalado e testaria conversões entre todos os serviços envolvidos.

Termo para lembrar: Decimal construction | Base técnica: referência 1.

12. Onde arredondar uma tarifa percentual?

Na fronteira definida pelo produto, depois de calcular com precisão suficiente. Eu documentaria ordem e modo, preservando intermediários necessários para explicar o resultado e reconciliar com o parceiro.

Termo para lembrar: Rounding boundary | Base técnica: referência 1.

13. O que fazer com divisão que não termina?

Definiria precisão interna e regra de quantização final. Não deixaria o comportamento depender de defaults da linguagem; testaria divisões periódicas e distribuição de resíduos para manter os totais contratados.

Termo para lembrar: Quantization | Base técnica: referência 1.

14. Dois serviços usam modos diferentes de arredondamento: e agora?

Eu centralizaria a especificação e criaria vetores de teste compartilhados entre linguagens. O contrato inclui escala, modo e ordem das operações; biblioteca decimal sozinha não alinha regras comerciais.

Termo para lembrar: Cross-language consistency | Base técnica: referência 1.

15. Como representar cotação e spread?

Guardaria taxa base, spread, moeda de origem/destino, instante e validade, conforme contrato. Isso permite reproduzir a oferta e separar variação cambial de tarifa, sem confundir taxa informativa com preço executável.

Termo para lembrar: Quote components | Base técnica: referência 1.

16. Quote expirou durante a confirmação: posso usar taxa nova?

Somente com política e consentimento adequados. Eu não mudaria silenciosamente o valor prometido; a aceitação precisa validar validade e condições, podendo exigir nova cotação antes de assumir o compromisso.

Termo para lembrar: Quote expiry | Base técnica: referência 1.

17. Decimal aceitou NaN ou Infinity: isso pode entrar numa transação monetária?

Eu exigiria is_finite antes de validar sinal, limites e escala do valor. Decimal representa números não finitos, portanto precisão decimal não garante domínio monetário válido. Rejeitaria esses valores na borda e antes da persistência; sua propagação pode tornar comparações inválidas ou interromper cálculos posteriores.

Termo para lembrar: Decimal.is_finite / numeric domain validation | Base técnica: referência 1.

18. Duas rotinas usam precisões diferentes: como evitar dependência do contexto decimal ambiente?

Eu envolveria o cálculo em localcontext com precisão e arredondamento explícitos, associados à política versionada. Não dependeria do contexto deixado por biblioteca ou execução anterior. Testaria rotinas intercaladas com políticas distintas, assegurando resultado reproduzível sem alterar acidentalmente as configurações de quem chamou.

Termo para lembrar: Decimal localcontext / explicit calculation policy | Base técnica: referência 1.

19. Vírgula decimal deve chegar ao backend financeiro?

A interface pode aceitar locale, mas eu normalizaria por regras explícitas antes da API. Separadores ambíguos precisam ser rejeitados ou esclarecidos, evitando interpretar milhar como fração em transferências.

Termo para lembrar: Locale parsing | Base técnica: referência 1.

20. 1.0 e 1.00 geram hashes diferentes para idempotência: como decidir?

Eu definiria serialização canônica no contrato, incluindo moeda e unidade. Primeiro validaria a escala permitida; depois normalizaria representações economicamente equivalentes conforme a regra acordada. Não arredondaria entrada inválida silenciosamente para fazê-la caber no mesmo hash. Assinatura e deduplicação precisam compartilhar a mesma semântica.

Termo para lembrar: Canonical monetary serialization

21. Como calcular juros por muitos períodos?

Definiria convenção temporal, capitalização, precisão e arredondamento conforme o produto. Eu testaria datas de fronteira e valores de referência; uma fórmula genérica não substitui a semântica contratada.

Termo para lembrar: Interest convention | Base técnica: referência 1.

22. Moeda de apresentação é moeda de liquidação?

Nem sempre. Eu manteria ambas e a conversão aplicada, permitindo reconciliar cada obrigação na sua unidade. A tela não deve apagar a diferença entre estimativa exibida e valor efetivamente liquidado.

Termo para lembrar: Settlement currency | Base técnica: referência 1.

23. O que fazer se provedor usa escala inesperada?

Validaria contrato por operação e moeda, com testes de integração e limites plausíveis. Mudança de escala deve falhar de forma visível, porque aceitar silenciosamente pode multiplicar valores por cem.

Termo para lembrar: Provider unit mismatch | Base técnica: referência 1.

24. Taxa zero e taxa ausente são equivalentes?

Não. Zero pode representar gratuidade; ausência pode indicar erro ou informação indisponível. Eu modelaria estados distintos e impediria cálculo financeiro quando faltam parâmetros obrigatórios, preservando explicação ao usuário.

Termo para lembrar: Missing versus zero | Base técnica: referência 1.

25. Como comparar cotações com moedas invertidas?

Eu normalizaria pares e convenções antes de comparar, verificando precisão e taxas envolvidas. Inverter numericamente uma taxa pode não reproduzir a oferta comercial no sentido contrário, especialmente com spreads.

Termo para lembrar: Currency pair direction | Base técnica: referência 1.

26. Como contabilizar conversão entre moedas?

Usaria partidas equilibradas por moeda e contas de posição ou liquidação apropriadas. Não tentaria equilibrar somando reais com dólares; preservaria vínculo entre as pernas e a cotação acordada.

Termo para lembrar: FX ledger | Base técnica: referência 1.

27. Mudança de regra exige recalcular todo histórico?

Depende da vigência e da política de correção. Eu versionaria a regra e manteria valores confirmados explicáveis, aplicando ajustes explícitos quando necessário em vez de reescrever silenciosamente cobranças antigas.

Termo para lembrar: Rule versioning | Base técnica: referência 1.

28. Por que testar números negativos em operações monetárias?

Porque sinais podem inverter débito, tarifa ou devolução indevidamente. Eu validaria quais operações admitem negativos e usaria comandos de negócio explícitos quando sinal sozinho puder contornar limites e autorização.

Termo para lembrar: Sign validation | Base técnica: referência 1.

29. Como reconciliar resíduo de câmbio?

Compararia valores originais, cotação e arredondamentos por perna, classificando a diferença. Só aplicaria ajuste segundo regra autorizada e identidade estável, sem usar tolerância para esconder erros sistemáticos.

Termo para lembrar: FX reconciliation | Base técnica: referência 1.

30. Qual fala demonstra maturidade nesse tema?

Eu diria que precisão é parte do contrato de negócio: tipo numérico, escala, moeda, ordem e arredondamento precisam ser reproduzíveis. Depois mostraria um rateio e uma conversão como testes concretos.

Termo para lembrar: Monetary contract | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Inteiro escalado versus decimal

Ganho. Armazenamento simples e operações exatas na escala

Custo e motivo. Exige gerir escalas e overflow

Como decidir. Quando unidade persistida é fixa

02. Escala por moeda versus duas casas fixas

Ganho. Representa corretamente diferentes unidades

Custo e motivo. Mais metadados e validação de contratos

Como decidir. Quando sistema atende múltiplas moedas

03. Regra central versus defaults de bibliotecas

Ganho. Mantém cálculo consistente entre serviços

Custo e motivo. Exige governança e vetores de teste comuns

Como decidir. Quando valores são calculados em linguagens distintas

04. Rateio determinístico versus arredondar parcelas isoladas

Ganho. Conserva total e permite reprodução

Custo e motivo. Introduz regra para distribuir resíduos

Como decidir. Quando valor não divide exatamente

05. Precisão interna maior versus quantização precoce

Ganho. Reduz erro acumulado no cálculo

Custo e motivo. Consome mais processamento e exige fronteira clara

Como decidir. Quando operações encadeadas envolvem frações

06. Faixa numérica ampla versus tipos pequenos

Ganho. Acomoda agregados maiores

Custo e motivo. Pode aumentar memória e custo de cálculo

Como decidir. Quando limites previstos excedem tipos menores

07. Quote garantido versus taxa indicativa

Ganho. Oferece previsibilidade ao cliente

Custo e motivo. Cria exposição cambial e gestão de validade

Como decidir. Quando produto promete preço por intervalo

08. Tarifa explícita versus valor líquido único

Ganho. Facilita explicar e reconciliar componentes

Custo e motivo. Amplia contrato e dados persistidos

Como decidir. Quando composição do preço precisa ser auditada

09. String decimal versus JSON number

Ganho. Preserva precisão entre clientes heterogêneos

Custo e motivo. Exige parsing e validação explícitos

Como decidir. Quando tipos numéricos dos clientes têm limites incompatíveis

10. Conta de resíduos versus ajuste oculto

Ganho. Torna diferenças observáveis e conciliáveis

Custo e motivo. Adiciona modelagem e processo contábil

Como decidir. Quando arredondamento legítimo produz diferença

Duas alternativas ao caminho proposto

1. Inteiro escalado

Quando faz sentido. Escala de armazenamento é estável

Como implementar. Guardar quantidade e unidade explicitamente

Limites. Cálculos intermediários exigem escala maior

2. Decimal de precisão controlada

Quando faz sentido. Juros e taxas usam frações extensas

Como implementar. Configurar precisão arredondamento e limites

Limites. Não dispensa política de negócio

Referências e alcance da evidência

1. Python: decimal

Precisão decimal e políticas de arredondamento. As respostas e parâmetros de aplicação são propostas autorais.

https://docs.python.org/3/library/decimal.html

2. Stripe: currencies

Unidades monetárias e particularidades por moeda. As respostas e parâmetros de aplicação são propostas autorais.

https://docs.stripe.com/currencies

Continue em: T092 · T094

T094 / J. Sistemas bancários

Pagamentos, Pix e cartões

Pagamento é uma intenção que atravessa autorização, execução e confirmação com semânticas próprias. Arquitetura de treino; não descreve implementação interna do Nubank.

Quando usar. Desenhos de Pix, cartões e transferências; confirmar sempre o contrato do arranjo.

Quando evitar ou limitar. Assumir que API Pix documenta o core bancário ou que timeout autoriza fallback financeiro.

Uma abertura possível

Eu começaria por separar recepção execução e liquidação. Depois explicaria modelar autorização captura e liquidação separadas, preservando as restrições do caso.

Peças e tecnologias. API Pix · Ledger · PaymentIntent · Outbox · Adapters

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Separar recepção execução e liquidação; validar estados percorridos numa transferência.

2. Modelar autorização captura e liquidação separadas; validar cenários com captura tardia.

3. Respeitar identificadores e escopos do contrato; validar ida e volta da mesma operação.

4. Reconciliar possível execução antes do fallback; validar resposta tardia do primeiro parceiro.

5. Avaliar disponibilidade prazo e elegibilidade juntos; validar resultado por rota e faixa horária.

30 perguntas para sustentar o desenho

01. POST aceito significa dinheiro transferido?

Separaria intenção aceita, execução enviada e resultado confirmado. A resposta da API precisa dizer qual marco garante; só mostraria dinheiro transferido após a evidência exigida pelo contrato financeiro correspondente.

Termo para lembrar: Payment intent | Base técnica: referência 1.

02. Cartão autorizado já liquidou?

Autorização, captura e liquidação representam etapas diferentes. Modelaria prazos e transições de cada uma, permitindo captura tardia ou parcial quando o arranjo suporta, sem tratá-las como um único booleano de sucesso.

Termo para lembrar: Authorization | Base técnica: referência 2.

03. Qual identificador uso na deduplicação?

Usaria identificadores e escopos definidos pelo contrato da integração, preservando também a intenção interna. A correlação precisa atravessar ida, retorno e reconciliação; não inventaria equivalência entre IDs com finalidades distintas.

Termo para lembrar: Pix identifiers | Base técnica: referência 1.

04. Parceiro lento permite trocar de trilho?

Antes de trocar a rota, verificaria se o primeiro destino pode ter executado. UNKNOWN exige consulta ou reconciliação; criar pagamento em outro trilho pode duplicar dinheiro mesmo que aumente sucesso aparente da API.

Termo para lembrar: External timeout | Base técnica: referência 2.

05. Escolho parceiro apenas pelo menor custo?

Escolheria rota por elegibilidade, prazo, capacidade, risco e custo, registrando a decisão. O menor preço não compensa uma rota incapaz de cumprir a promessa; fallback precisa respeitar a execução anterior.

Termo para lembrar: Routing | Base técnica: referência 1.

06. Quando bloquear saldo da transferência?

Reservaria recursos na autoridade apropriada antes do compromisso externo, conforme o fluxo. Validação local sem proteção concorrente pode aprovar vários pagamentos sobre o mesmo saldo, independentemente de haver filas no transporte.

Termo para lembrar: Balance reservation | Base técnica: referência 2.

07. Retorno externo pode ser falso?

Validaria origem, assinatura e contexto do retorno, além da transição de negócio. Evento autenticado ainda pode estar repetido ou atrasado; segurança do canal e idempotência financeira são proteções diferentes.

Termo para lembrar: Webhook | Base técnica: referência 1.

08. Confirmação chega antes de evento intermediário?

Aplicaria máquina de estados e precedência do contrato, preservando evidências. Uma confirmação recebida antes de evento intermediário não deve ser perdida, e mensagem antiga não pode regredir um resultado já estabelecido.

Termo para lembrar: Out of order | Base técnica: referência 2.

09. Devolução reutiliza ID do pagamento?

Criaria intenção de devolução vinculada, com limite reversível e identidade própria. Em devoluções parciais concorrentes, a soma precisa ser controlada atomicamente, evitando que cada pedido isolado pareça válido e exceda o total.

Termo para lembrar: Refund | Base técnica: referência 1.

10. Posso prometer disponibilidade igual para todos trilhos?

Incorporaria calendário, cutoff e condições do arranjo à promessa do produto. Não trataria todos os trilhos como equivalentes; a arquitetura precisa comunicar e operar diferenças de prazo, disponibilidade e confirmação.

Termo para lembrar: Cutoff | Base técnica: referência 2.

11. Como desenhar o caminho mínimo de pagamento?

Eu começaria por intenção durável, validação, execução no arranjo e confirmação correlacionada. Só depois adicionaria filas e projeções, deixando explícitos os estados pendentes e a autoridade de cada decisão.

Termo para lembrar: Payment lifecycle | Base técnica: referência 1.

12. A API Pix é a mesma interface do SPI?

Não assumiria essa equivalência. Eu confirmaria o papel de cada contrato público e integração do participante, evitando usar documentação de uma interface como prova do processamento interno de outra.

Termo para lembrar: Integration scope | Base técnica: referência 1.

13. Autorização do cartão expirada pode ser capturada?

Depende das regras do arranjo e contrato. Eu validaria elegibilidade e trataria exceções explicitamente, sem presumir que possuir um código de autorização permite qualquer captura futura.

Termo para lembrar: Authorization validity | Base técnica: referência 1.

14. Como proteger pagamento contra duplo clique?

Preservaria a intenção no cliente e garantiria idempotência na fronteira responsável. Dois pedidos com chaves distintas ainda exigem regras de negócio quando representam compromisso duplicado; botão desabilitado não é garantia.

Termo para lembrar: Payment deduplication | Base técnica: referência 1.

15. Um pagamento agendado encontra saldo ou permissões diferentes no dia da execução: e agora?

Eu trataria o agendamento como intenção persistida, não autorização irrevogável. Na execução, validaria saldo, autorização e regras vigentes conforme o contrato, preservando a identidade da operação. Se condições essenciais mudarem, registraria impedimento ou pediria confirmação; não alteraria valor ou destinatário silenciosamente para concluir.

Termo para lembrar: Scheduled intent / execution-time validation

16. Erro HTTP 500 permite assumir rejeição?

Não necessariamente. O parceiro pode ter executado e falhado ao responder. Eu consultaria a operação ou aplicaria retry idempotente conforme contrato, mantendo UNKNOWN quando não existe evidência conclusiva.

Termo para lembrar: Ambiguous failure | Base técnica: referência 1.

17. O aplicativo envia nome e conta do favorecido junto ao QR: posso confiar?

Eu trataria os campos enviados pelo cliente como entrada não confiável. Validaria formato e resolveria os dados pelo canal autorizado no contrato, vinculando favorecido e valor confirmados à intenção. Mudança relevante exigiria nova validação; mostrar um nome no frontend não autentica o destino financeiro.

Termo para lembrar: Untrusted payment payload / intent binding

18. Como armazenar credenciais do parceiro?

Usaria identidade de serviço e gerenciador de segredos, com rotação e acesso mínimo. Não colocaria credenciais em eventos ou logs; testaria a troca sem interromper operações pendentes.

Termo para lembrar: Partner credentials | Base técnica: referência 1.

19. Qual relação entre disponibilidade e integridade financeira?

Eu definiria quais ações podem degradar e quais devem aguardar autoridade. Aceitar pagamentos sem preservar limites pode produzir prejuízo; disponibilidade percebida não justifica violar invariantes que o produto considera obrigatórios.

Termo para lembrar: Financial safety | Base técnica: referência 1.

20. Como modelar devolução parcial?

Criaria intenção vinculada ao pagamento e validaria valor remanescente reversível atomicamente. Preservaria cada identidade e resultado externo, para que duplicatas e devoluções concorrentes não ultrapassem o permitido.

Termo para lembrar: Partial refund | Base técnica: referência 1.

21. O que acontece se o cliente fecha o aplicativo?

A operação durável continua conforme contrato, e o cliente consulta depois pelo identificador. Fechar a conexão não representa cancelamento financeiro; esse comando, quando permitido, possui fluxo e resultado próprios.

Termo para lembrar: Detached operation | Base técnica: referência 1.

22. Como lidar com manutenção programada do parceiro?

Incorporaria janela ao roteamento e à admissão, respeitando alternativas seguras e prazo do produto. Operações já incertas continuam em reconciliação; manutenção não autoriza recriar intenções previamente enviadas.

Termo para lembrar: Maintenance handling | Base técnica: referência 1.

23. Duas capturas parciais concorrentes podem ultrapassar a autorização do cartão?

Quando o arranjo permite captura parcial, eu controlaria atomicamente o limite restante na autoridade responsável. Cada captura teria identidade própria e reservaria capacidade incluindo tentativas pendentes de resultado. Um read-then-write concorrente pode exceder a autorização; falhas ambíguas não liberariam essa reserva sem evidência.

Termo para lembrar: Atomic capture reservation / authorized ceiling

24. Como calcular backlog durante indisponibilidade?

Estimaria chegada, capacidade de armazenamento e vazão permitida na recuperação. A fila precisa caber e drenar antes do prazo; adicionar Kafka não altera automaticamente quota ou velocidade do parceiro.

Termo para lembrar: Backlog planning | Base técnica: referência 1.

25. Como operar webhook repetido com outro envelope?

Deduplicaria o evento e aplicaria transições de negócio idempotentes por operação. Identidades de envelope diferentes podem descrever o mesmo fato, então unicidade de transporte não basta para proteger efeito financeiro.

Termo para lembrar: Semantic deduplication | Base técnica: referência 1.

26. Que dados entram na auditoria de pagamento?

Intenção, ator autorizado, valores, moeda, referências, versões de decisão e evidências de resultado. Eu evitaria segredos e dados sensíveis desnecessários, mantendo correlação suficiente para reproduzir e investigar o fluxo.

Termo para lembrar: Payment audit | Base técnica: referência 1.

27. Como limitar fraude sem travar toda jornada?

Aplicaria políticas por risco, com deadline e fallback acordados. A decisão antifraude fica registrada e separada da execução financeira, permitindo revisar resultado sem transformar score em autorização irrestrita.

Termo para lembrar: Risk decision boundary | Base técnica: referência 1.

28. Qual teste de falha você apresentaria primeiro?

Interromperia o worker após possível execução remota e antes do commit local. A solução precisa reconhecer a mesma intenção ou reconciliar, demonstrando que recuperação não depende de acreditar que timeout significa falha.

Termo para lembrar: Critical failure test | Base técnica: referência 1.

29. Como recuperar após perda regional?

Restabeleceria autoridade de escrita com fencing e avaliaria registros perdidos conforme RPO. Antes de retomar efeitos, confrontaria operações externas e identidades preservadas, evitando repetir pagamentos ausentes no backup restaurado.

Termo para lembrar: Regional recovery | Base técnica: referência 1.

30. Quando encerraria a operação como rejeitada?

Quando houver evidência e regra de negócio que sustentem rejeição, não apenas retries esgotados. A exaustão operacional pode exigir intervenção ou estado pendente, mantendo distinta a situação financeira real.

Termo para lembrar: Terminal state | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Aceitação assíncrona versus espera pelo resultado

Ganho. Libera interação e tolera processamento longo

Custo e motivo. Exige estado pendente e acompanhamento

Como decidir. Quando conclusão não cabe no deadline da API

02. Reserva antes de execução versus débito imediato

Ganho. Protege disponibilidade durante confirmação

Custo e motivo. Aumenta estados e pode bloquear saldo temporariamente

Como decidir. Quando há autorização seguida de resultado externo

03. IDs externos preservados versus somente ID local

Ganho. Facilita consulta e reconciliação

Custo e motivo. Acopla parte do modelo ao contrato do parceiro

Como decidir. Quando retorno e suporte dependem dessas referências

04. Rota única versus múltiplas rotas

Ganho. Reduz complexidade e ambiguidade de execução

Custo e motivo. Menos opções diante de indisponibilidade

Como decidir. Quando simplicidade e garantia compensam menor flexibilidade

05. Roteamento por custo versus por prazo

Ganho. Pode reduzir despesa por operação

Custo e motivo. Pode aumentar atraso ou rejeição

Como decidir. Quando rotas elegíveis ainda cumprem o contrato

06. Confirmação forte versus aceitação otimista

Ganho. Preserva certeza do estado mostrado

Custo e motivo. Pode aumentar latência ou pendência

Como decidir. Quando consequência financeira exige evidência

07. Webhook versus polling

Ganho. Reduz consultas e acelera retorno

Custo e motivo. Exige endpoint seguro e tolerância a duplicatas

Como decidir. Quando parceiro oferece eventos confiavelmente correlacionáveis

08. Máquina de estados detalhada versus status simples

Ganho. Representa incerteza e fases distintas

Custo e motivo. Amplia testes e contratos de apresentação

Como decidir. Quando transporte e negócio têm resultados separados

09. Devolução como operação versus edição do pagamento

Ganho. Preserva história e controle de valor

Custo e motivo. Mais entidades e reconciliação

Como decidir. Quando ajustes precisam de evidência própria

10. Fallback financeiro versus aguardar reconciliação

Ganho. Pode recuperar disponibilidade em falha comprovada

Custo e motivo. Exige provar segurança entre rotas

Como decidir. Quando não execução anterior é demonstrável

Duas alternativas ao caminho proposto

1. Fluxo síncrono limitado

Quando faz sentido. Resultado cabe no prazo contratual

Como implementar. Aguardar confirmação com deadline e consulta

Limites. Timeout ainda pode produzir UNKNOWN

2. Fluxo assíncrono explícito

Quando faz sentido. Arranjo e produto permitem conclusão posterior

Como implementar. Aceitar intenção durável e acompanhar estado

Limites. UX e prazos ficam mais complexos

Referências e alcance da evidência

1. Banco Central: API Pix

Contratos públicos da API Pix; não descreve a infraestrutura interna de um banco. As respostas e parâmetros de aplicação são propostas autorais.

https://github.com/bacen/pix-api

2. Stripe: PaymentIntents

Ciclo de vida de uma intenção de pagamento. As respostas e parâmetros de aplicação são propostas autorais.

https://docs.stripe.com/payments/payment-intents

Continue em: T093 · T095

T095 / J. Sistemas bancários

Chargeback: ida, volta e resultados parciais

Chargeback coordena casos e arquivos com retornos potencialmente parciais e efeitos independentes. Arquitetura de treino; não descreve implementação interna do Nubank.

Quando usar. Integração com bandeira, disputa, evidências e crédito provisório.

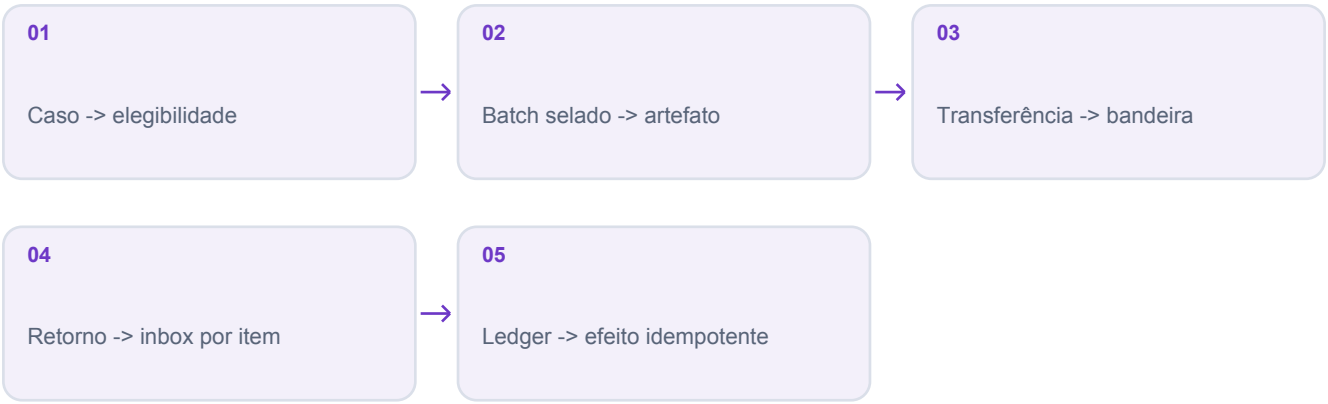
Quando evitar ou limitar. Recolocar itens UNKNOWN em lote novo como se nunca tivessem sido enviados.

Uma abertura possível

Eu começaria por definir identidade pela intenção e escopo permitido. Depois explicaria selar membros bytes e checksum do lote, preservando as restrições do caso.

Peças e tecnologias. PostgreSQL · S3 · SFTP · AWS Transfer Family · Outbox

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Definir identidade pela intenção e escopo permitido; validar casos parciais e motivos diferentes.

2. Selar membros bytes e checksum do lote; validar comparação de artefatos por tentativa.

3. Reservar membros com transação e unicidade; validar dois workers no mesmo cutoff.

4. Separar falha comprovada de resultado desconhecido; validar queda depois do último byte.

5. Selecionar apenas itens comprovadamente elegíveis; validar retorno tardio do lote anterior.

30 perguntas para sustentar o desenho

01. Uma compra pode ter duas contestações?

Definiria identidade da disputa conforme intenção e regras permitidas, separando a chave de retry. Uma compra pode admitir escopos distintos; não usaria purchase_id sozinho sem esclarecer contestações parciais e demais condições.

Termo para lembrar: Business key | Base técnica: referência 1.

02. Posso alterar CSV depois de enviar?

Após selar, preservaria membros, bytes e checksum do lote. Novos casos entram em outro artefato; assim uma tentativa posterior pode ser comparada exatamente com o que o parceiro recebeu anteriormente.

Termo para lembrar: Batch sealing | Base técnica: referência 2.

03. Dois jobs selecionam os mesmos casos?

Associaria os membros ao lote com transação e unicidade apropriadas. Dois workers consultando pending podem selecionar os mesmos registros; a disputa precisa ser resolvida no commit da associação, não por confiança no cron.

Termo para lembrar: Atomic claim | Base técnica: referência 1.

04. Falha de conexão significa lote rejeitado?

Distinguiria falha comprovada antes do envio de resultado incerto após possível transmissão. O estado do transporte não substitui resultado da disputa; manteria evidências e consulta contratual antes de reclassificar itens para novo envio.

Termo para lembrar: Transmission state | Base técnica: referência 2.

05. Posso misturar novos e falhados?

Novo lote só inclui itens elegíveis conforme contrato, preservando identidade da intenção e histórico. UNKNOWN não retorna automaticamente à seleção; mudar envelope ou número de tentativa não elimina possibilidade de processamento anterior.

Termo para lembrar: Retry batch | Base técnica: referência 1.

06. Arquivo voltou com metade dos itens?

Persistiria retorno por item, com correlação e estados permitidos. Ausência de resposta para parte do lote permanece pendente, e duplicatas não repetem efeito; sucesso do arquivo não significa sucesso de todas as disputas.

Termo para lembrar: Partial results | Base técnica: referência 2.

07. Mesmo retorno chegou duas vezes?

Gravaria deduplicação, transição e obrigação de executar efeito na mesma transação local. O ledger recebe uma identidade estável, porque inbox sozinha não impede duplicidade se a chamada externa executar antes de um crash.

Termo para lembrar: Inbox | Base técnica: referência 1.

08. Crédito provisório exige decisão final?

Separaria política de crédito provisório da decisão final da bandeira. A aprovação pode confirmar um efeito já concedido, e a rejeição pode exigir reversão; cada ação possui valor, autorização e identidade próprios.

Termo para lembrar: Provisional credit | Base técnica: referência 2.

09. Quatro jobs por dia bastam?

Calcularia capacidade por janela, quota de transmissões e prazo dos itens, incluindo retries e recuperação. Quatro execuções lógicas não equivalem necessariamente a quatro envios físicos, e backlog pode ameaçar cutoff antes de haver erro técnico.

Termo para lembrar: Deadline | Base técnica: referência 1.

10. Como descobrir solicitação esquecida?

Confrontaria solicitação, manifesto, retorno e ledger por identidade e valor. Uma soma global coincidente pode esconder item ausente e duplicado; casos sem evidência suficiente permanecem investigáveis até resolução autorizada.

Termo para lembrar: Reconciliation | Base técnica: referência 2.

11. Quais entidades separaria no chargeback?

Solicitação, lote, arquivo, tentativa de transferência, retorno por item e efeito financeiro. Cada entidade tem identidade e estado próprios; isso evita que falha de um envio transforme todas as disputas em rejeitadas.

Termo para lembrar: Entity separation | Base técnica: referência 1.

12. Uma disputa parcial coincide com reembolso comercial: como evitar crédito econômico excessivo?

Eu modelaria direitos de reembolso, crédito provisório e liquidação separadamente, conforme as regras do arranjo. Na autoridade financeira, reservaria e reavaliaria atomicamente o valor ainda elegível, considerando operações pendentes. Somar apenas lançamentos concluídos ignora tentativas em andamento e pode autorizar dois créditos para o mesmo direito.

Termo para lembrar: Economic entitlement / concurrent reimbursement

13. Como garantir que membro só pertence ao lote correto?

Usaria constraint ou transição condicional coerente com a política de reenvio. A seleção e associação precisam ser atômicas; duas consultas concorrentes por estado pending não garantem exclusividade.

Termo para lembrar: Batch membership | Base técnica: referência 1.

14. Qual diferença entre batch_id e attempt_id?

Batch identifica o envelope lógico e seu conteúdo selado; attempt identifica uma execução de transporte. Eu preservaria ambos e a identidade dos itens, evitando criar nova intenção financeira a cada retry.

Termo para lembrar: Attempt identity | Base técnica: referência 1.

15. Arquivo ausente no SFTP prova que não chegou?

Não. O parceiro pode mover ou consumir arquivos após recebê-los. Eu procuraria confirmação contratual, consulta ou retorno correlacionado; ausência numa pasta não é evidência universal de não execução.

Termo para lembrar: SFTP uncertainty | Base técnica: referência 1.

16. Pode usar nome temporário e rename?

Se o servidor e contrato suportarem esse protocolo, ajuda evitar consumo de arquivo incompleto. Eu verificaria semântica do rename e confirmação, sem assumir que isso deduplica processamento de negócio automaticamente.

Termo para lembrar: File publication | Base técnica: referência 1.

17. Como processar retorno com item desconhecido?

Preservaria o retorno em quarentena e investigaria correlação, origem e versão. Não criaria automaticamente uma solicitação financeira para encaixá-lo, porque erro de associação pode produzir crédito indevido.

Termo para lembrar: Unmatched return | Base técnica: referência 1.

18. Mesmo item recebeu aprovado e rejeitado: qual vale?

Consultaria regras de versão, etapa e autoridade do arranjo, mantendo evidências. Não aplicaria simplesmente o último recebido; retornos podem representar fases diferentes ou chegar fora de ordem.

Termo para lembrar: Conflicting outcomes | Base técnica: referência 1.

19. O arquivo de retorno usa outro layout, mas o parser não falha: posso processar?

Eu validaria versão negociada do layout e sua associação ao lote, incluindo significado de colunas, unidades e códigos. CSV sintaticamente válido pode interpretar aprovação como outro estado. Colocaria incompatibilidades em quarentena e testaria fixtures do parceiro antes de ativar um parser novo.

Termo para lembrar: Layout version contract / semantic validation

20. Crédito provisório e final podem usar o mesmo comando?

Eu modelaria efeitos conforme política: confirmação pode apenas tornar definitivo um crédito já concedido, ou gerar ajuste específico. Não assumiria que aprovação final sempre exige creditar novamente o valor.

Termo para lembrar: Provisional credit semantics | Base técnica: referência 1.

21. Como representar estorno após derrota na disputa?

Criaria efeito de reversão vinculado ao crédito original, com valor e elegibilidade validados. O histórico permanece; idempotência da reversão evita que retornos repetidos debitem o cliente mais de uma vez.

Termo para lembrar: Dispute reversal | Base técnica: referência 1.

22. Quatro execuções diárias equivalem a quatro tentativas físicas?

Não. Cada job pode provocar retries no SDK, worker ou transferência. Eu registraria consumo conforme a unidade da quota e verificaria timezone e janela, incluindo recuperação e falhas no cálculo.

Termo para lembrar: Physical quota | Base técnica: referência 1.

23. Como reagir se backlog não cabe no próximo lote?

Priorizaria conforme prazo e regras da bandeira, comunicaria risco e controlaria admissão quando necessário. Aumentar tamanho sem verificar limites do arquivo ou processamento apenas desloca a falha.

Termo para lembrar: Cutoff capacity | Base técnica: referência 1.

24. Um item inválido exige reenviar todo arquivo?

Depende do protocolo de rejeição. Eu distinguiria rejeição de envelope e resultado individual, preservando itens já aceitos; reenviar todos sem essa distinção pode duplicar disputas corretamente processadas.

Termo para lembrar: Partial retry | Base técnica: referência 1.

25. Como verificar integridade do arquivo gerado?

Validaria schema, contagem, totais por moeda e checksum contra o manifesto. Também testaria encoding e regras de campo, publicando o artefato apenas após concluir essas verificações.

Termo para lembrar: File validation | Base técnica: referência 1.

26. E se a geração cair depois do upload?

A retomada consultaria estado e artefato identificável, verificando checksum antes de reutilizar ou concluir publicação. Eu não criaria outro lote automaticamente só porque a resposta local foi perdida.

Termo para lembrar: Artifact recovery | Base técnica: referência 1.

27. Quando usar Kafka nesse fluxo?

Quando replay, múltiplos consumidores ou vazão justificarem. Para um processo periódico simples, banco e workers podem bastar; o broker não resolve identidade de lote nem incerteza do parceiro.

Termo para lembrar: Broker justification | Base técnica: referência 1.

28. Como proteger evidências anexadas à contestação?

Armazenaria com autorização por caso, scanner e retenção definida. A exportação ao parceiro seria registrada, evitando URLs amplamente reutilizáveis e conteúdo sensível em logs ou mensagens técnicas.

Termo para lembrar: Evidence security | Base técnica: referência 1.

29. O que suporte deve enxergar durante a indisponibilidade?

Estado da solicitação, último marco confirmado, pendência e prazo aplicável, com protocolo. Não exibiria erro técnico como rejeição da disputa; a operação precisa distinguir atraso de decisão negativa.

Termo para lembrar: Support visibility | Base técnica: referência 1.

30. O cliente retira a contestação depois do envio: basta apagar o caso?

Eu registraria uma solicitação de retirada, preservando evidências e efeitos anteriores. A ação externa dependeria do protocolo e de confirmação da contraparte; localmente, ficaria pendente até obter evidência. Qualquer ajuste do crédito provisório seria explícito. Excluir o caso não desfaz uma disputa já recebida.

Termo para lembrar: Dispute withdrawal / explicit compensating action

10 trade-offs: ganho, custo e decisão

01. Lote periódico versus API por item

Ganho. Amortiza transporte e atende contratos de arquivo

Custo e motivo. Aumenta latência e complexidade de cutoff

Como decidir. Quando parceiro exige ou favorece batch

02. Artefato selado versus arquivo mutável

Ganho. Permite reproduzir e verificar tentativa

Custo e motivo. Novas entradas precisam de outro lote

Como decidir. Quando é necessário provar o conteúdo enviado

03. Claim transacional versus seleção livre

Ganho. Evita associação concorrente indevida

Custo e motivo. Adiciona contenção e gestão de estado

Como decidir. Quando múltiplos workers montam lotes

04. UNKNOWN explícito versus rejeição imediata

Ganho. Preserva incerteza sem induzir reenvio incorreto

Custo e motivo. Exige reconciliação e pendência operacional

Como decidir. Quando parceiro pode ter recebido sem resposta

05. Reenvio do mesmo envelope versus novo envelope

Ganho. Preserva correlação de uma tentativa lógica

Custo e motivo. Depende de suporte e regras do parceiro

Como decidir. Quando contrato reconhece repetição do artefato

06. Resultado por item versus status só do lote

Ganho. Representa aceitação parcial e recuperação precisa

Custo e motivo. Mais registros, estados e consultas

Como decidir. Quando retornos podem ser parciais

07. Outbox e inbox versus chamadas diretas

Ganho. Protegem obrigações locais e repetição

Custo e motivo. Introduzem relay, lag e operação adicional

Como decidir. Quando falhas entre banco e mensageria são relevantes

08. Crédito provisório versus aguardar decisão final

Ganho. Melhora experiência durante disputa

Custo e motivo. Cria exposição e possível reversão posterior

Como decidir. Quando política financeira autoriza esse benefício

09. Reserva de quota versus uso integral imediato

Ganho. Preserva capacidade para recuperação urgente

Custo e motivo. Pode deixar oportunidades normais ociosas

Como decidir. Quando retries e cutoff disputam limite rígido

10. Conciliação detalhada versus somente totais

Ganho. Detecta erros compensados e pendências individuais

Custo e motivo. Consome mais processamento e exige chaves confiáveis

Como decidir. Quando efeito financeiro exige prova por disputa

Duas alternativas ao caminho proposto

1. API por disputa

Quando faz sentido. Parceiro oferece contrato consultável

Como implementar. Usar intenção por item e consulta de resultado

Limites. Quotas e integração continuam relevantes

2. Arquivo em lote

Quando faz sentido. Contrato exige intercâmbio periódico

Como implementar. Selar manifesto e reconciliar cada item

Limites. Cutoffs e falhas parciais ampliam complexidade

Referências e alcance da evidência

1. Stripe: ciclo de disputas

Etapas e resultados de disputas; regras dependem do arranjo. As respostas e parâmetros de aplicação são propostas autorais.

https://docs.stripe.com/disputes/how-disputes-work

2. Stripe: webhooks

Assinatura, duplicatas e entrega de eventos. As respostas e parâmetros de aplicação são propostas autorais.

https://docs.stripe.com/webhooks

Continue em: T094 · T096

T096 / J. Sistemas bancários

Reconciliação operacional e financeira

Reconciliação transforma diferenças entre fontes em evidências e ações controladas. Arquitetura de treino; não descreve implementação interna do Nubank.

Quando usar. Pagamentos, arquivos, ledger, settlement e retomadas após desastre.

Quando evitar ou limitar. Corrigir dinheiro automaticamente só porque uma soma não fechou.

Uma abertura possível

Eu começaria por confrontar intenção execução externa e ledger. Depois explicaria preferir identidade estável e chaves compostas, preservando as restrições do caso.

Peças e tecnologias. PostgreSQL · Object storage · Spark · Flink · Case management

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Confrontar intenção execução externa e ledger; validar matriz de presença por operação.

2. Preferir identidade estável e chaves compostas; validar casos com mesmo valor simultâneo.

3. Validar manifesto e confirmação de completude; validar fonte ausente versus fonte vazia.

4. Separar atraso esperado de divergência vencida; validar distribuição de chegada dos retornos.

5. Comparar itens além dos agregados; validar duplicata e ausência com mesmo valor.

30 perguntas para sustentar o desenho

01. Quais fontes você compararia?

Compararia intenção local, evidência externa e efeito contábil, definindo autoridade por fato. Duas fontes podem concordar por compartilharem o mesmo erro; a conciliação precisa representar obrigações e não apenas copiar um saldo.

Termo para lembrar: Three way match | Base técnica: referência 1.

02. Posso comparar apenas valor e data?

Preferiria identificadores estáveis e cardinalidade contratada, usando critérios auxiliares com cautela. Valor e data iguais podem corresponder a operações legítimas distintas; associações ambíguas ficam em revisão, preservando os candidatos e a evidência.

Termo para lembrar: Matching key | Base técnica: referência 2.

03. Arquivo vazio significa nenhum movimento?

Exigiria manifesto ou outro marco de completude antes de interpretar arquivo vazio. Fonte ausente, carga parcial e período sem movimento são situações diferentes; o relatório deve mostrar qual delas foi comprovada.

Termo para lembrar: Completeness | Base técnica: referência 1.

04. Diferença hoje pode sumir amanhã?

Separaria divergência vencida de atraso ainda esperado, com janela e revisão explícitas. Fechar cedo aumenta falsos incidentes; esperar demais posterga descoberta de perda real, então a escolha depende do prazo e risco.

Termo para lembrar: Temporal windows | Base técnica: referência 2.

05. Total diário igual prova reconciliação?

Compararia itens e multiplicidade além dos totais por moeda e período. Erros de valores iguais podem se compensar numericamente; igualdade agregada é um controle útil, mas não prova correspondência correta entre operações.

Termo para lembrar: Amounts | Base técnica: referência 1.

06. Posso ignorar diferenças pequenas?

Definiria tolerância por causa autorizada, unidade e período, registrando todo desvio. Pequenas diferenças recorrentes podem representar erro sistemático; acompanharia exposição acumulada e não usaria centavos como justificativa automática para ignorar problema.

Termo para lembrar: Tolerance | Base técnica: referência 2.

07. Diferença autoriza lançamento corretivo?

Ajuste financeiro exige regra, evidência, autorização e identidade própria vinculada ao break. Não criaria lançamento apenas para zerar diferença; a correção precisa explicar a causa e permanecer idempotente durante reexecução.

Termo para lembrar: Auto adjustment | Base técnica: referência 1.

08. Como reproduzir um fechamento antigo?

Preservaria referências das fontes, regras e versões usadas no fechamento. Reproduzir resultado antigo exige saber o que estava disponível; consultar tabelas atuais pode mudar a explicação de uma decisão já publicada.

Termo para lembrar: Audit | Base técnica: referência 2.

09. Bilhões de itens cabem numa comparação?

Particionaria por chave apropriada e processaria em fluxo ou sort-merge conforme volume. Estimaria memória, spill e skew, incluindo reinício; reconciliação não deve depender de todo o período caber na RAM.

Termo para lembrar: Scaling join | Base técnica: referência 1.

10. Quando alertar uma pendência?

Priorizaria idade, valor e prazo de cada divergência, com responsável e escalonamento. Contagem baixa não significa baixo risco; um caso antigo e valioso pode merecer mais atenção que muitos pequenos recém-abertos.

Termo para lembrar: Resolution SLA | Base técnica: referência 2.

11. Diferença sempre significa perda financeira?

Não. Pode ser atraso, classificação, taxa ou erro real. Eu classificaria pela evidência e prazo, mantendo valores em aberto até esclarecimento, sem transformar qualquer divergência numa ordem automática de ajuste.

Termo para lembrar: Break classification | Base técnica: referência 1.

12. Como escolher a fonte da verdade?

Definiria autoridade por fato: intenção local, confirmação externa e lançamento contábil podem ter fontes distintas. Reconciliação compara essas obrigações; escolher uma única tabela como verdade universal pode esconder falhas.

Termo para lembrar: Fact authority | Base técnica: referência 1.

13. O que é matching um-para-muitos?

Uma operação pode corresponder a várias liquidações ou parcelas. Eu modelaria essa cardinalidade e conservaria vínculos, evitando exigir igualdade de linhas quando o contrato permite agregação ou desdobramento.

Termo para lembrar: Matching cardinality | Base técnica: referência 1.

14. Como distinguir duplicata de pagamento legítimo igual?

Usaria identificadores e contexto contratual, preservando multiplicidade. Valor e data iguais não provam duplicidade; quando faltarem chaves confiáveis, encaminharia matching ambíguo para decisão controlada em vez de excluir automaticamente.

Termo para lembrar: Ambiguous matching | Base técnica: referência 1.

15. Snapshot de duas fontes no mesmo horário basta?

Não necessariamente, pois cada uma pode fechar ou publicar com atraso distinto. Eu registraria versões e marcos de completude, escolhendo uma janela comparável e revisando retornos que ainda eram esperados.

Termo para lembrar: Comparable snapshots | Base técnica: referência 1.

16. Quando streaming ajuda a conciliar?

Quando reduz tempo até detectar diferenças e as fontes oferecem eventos adequados. Eu ainda manteria fechamento e revisão de atrasados; processamento contínuo não prova completude nem elimina comparação de totais.

Termo para lembrar: Continuous reconciliation | Base técnica: referência 1.

17. O parceiro reutiliza referências externas a cada dia: qual chave de matching usar?

Eu definiria um namespace composto por parceiro, conta, data de negócio e referência, conforme o contrato. Fixaria calendário e fuso dessa data, sem presumir unicidade global do identificador. Colisões ou campos insuficientes iriam para investigação; um match aparentemente exato pode ligar movimentos de dias diferentes.

Termo para lembrar: Composite reconciliation key / namespace

18. Mudamos o algoritmo de matching com divergências em investigação: como migrar?

Eu versionaria o matcher e compararia a nova lógica em shadow, preservando evidências e decisões anteriores. Reassociações ou reaberturas seriam explícitas, com trilha de auditoria. Não deixaria uma mudança de algoritmo substituir silenciosamente correspondências já usadas para ajustes, criando correções financeiras duplicadas.

Termo para lembrar: Versioned matcher / shadow reconciliation

19. Como evitar duas equipes corrigindo o mesmo break?

Usaria caso com ownership e versão, além de identidade única da decisão de ajuste. A execução financeira valida o mesmo efeito, protegendo contra concorrência e repetição além da interface.

Termo para lembrar: Break ownership | Base técnica: referência 1.

20. Qual índice facilita matching de alto volume?

Depende da chave de correspondência; eu indexaria identidade do parceiro, operação e período quando necessário. Validaria seletividade e particionamento, evitando consultas por valor solto que geram muitos candidatos.

Termo para lembrar: Matching index | Base técnica: referência 1.

21. Como lidar com arquivo do parceiro substituído depois?

Preservaria versão, checksum e origem de cada entrega. A nova versão dispara comparação e revisão rastreável; não sobrescreveria evidências usadas no fechamento anterior sem manter vínculo e explicação.

Termo para lembrar: Source revision | Base técnica: referência 1.

22. O painel mostra zero divergências: como saber se a reconciliação realmente executou?

Eu verificaria presença da execução esperada, cobertura dos inputs, watermarks e totais processados antes de interpretar zero breaks. Um monitor independente alertaria sobre execução ausente ou incompleta. Ausência de divergência pode significar ausência de dados; sucesso exige evidência de cobertura, não apenas contador vazio.

Termo para lembrar: Reconciliation coverage / missing-run detection

23. Como testar motor de conciliação?

Geraria fontes com ausência, duplicata, atraso, taxa e moeda divergente, verificando classificação e vínculos. Reexecutaria o mesmo fechamento para garantir que não produz ajustes repetidos nem resultados inconsistentes.

Termo para lembrar: Reconciliation tests | Base técnica: referência 1.

24. Pode reconciliar após restaurar um banco antigo?

É necessário considerar efeitos externos posteriores ao ponto restaurado. Eu recuperaria evidências e identidades antes de liberar executores, distinguindo operação ausente localmente de operação que nunca aconteceu no parceiro.

Termo para lembrar: Post-restore reconciliation | Base técnica: referência 1.

25. Como dimensionar sort-merge de arquivos grandes?

Estimaria volume, memória disponível, runs e espaço temporário, usando streaming. Testaria skew e interrupção, com checkpoints ou reexecução idempotente; carregar tudo em memória torna o fechamento vulnerável ao crescimento.

Termo para lembrar: External merge | Base técnica: referência 1.

26. Quando hash join é uma escolha ruim?

Quando a tabela de build não cabe na memória ou a distribuição provoca pressão excessiva. Eu avaliaria particionamento e spill, comparando custo total com ordenação e merge para aquele volume.

Termo para lembrar: Join strategy | Base técnica: referência 1.

27. O operador deve poder editar valores originais?

Eu preservaria dados de origem e permitiria anotações ou decisões separadas, com autoria. Alterar evidência para fazer matching funcionar destrói auditabilidade e pode esconder a causa real da divergência.

Termo para lembrar: Evidence preservation | Base técnica: referência 1.

28. Como explicar relatório ainda incompleto?

Publicaria estado de cobertura e fontes pendentes, além de totais provisórios claramente identificados. Não exibiria ausência de diferenças como sucesso quando parte dos dados esperados ainda não chegou.

Termo para lembrar: Incomplete reconciliation | Base técnica: referência 1.

29. Que métricas provam melhoria operacional?

Tempo até detectar, tempo até resolver, valor exposto e percentual conciliado com evidência. Reduzir quantidade de alertas só é bom se não esconder diferenças ou aumentar risco financeiro não observado.

Termo para lembrar: Reconciliation SLI | Base técnica: referência 1.

30. Como encerrar um break com segurança?

Exigiria evidência de resolução, decisão autorizada e confirmação do efeito quando houver ajuste. O encerramento registra motivo e referências, permitindo reabrir por nova evidência sem perder a história.

Termo para lembrar: Break closure | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Três fontes versus comparação bilateral

Ganho. Expõe inconsistência entre intenção execução e contabilidade

Custo e motivo. Exige mais contratos e alinhamento temporal

Como decidir. Quando nenhum par sozinho comprova o fluxo

02. Matching exato versus heurístico

Ganho. Reduz associações equivocadas

Custo e motivo. Pode deixar mais casos sem correspondência automática

Como decidir. Quando identificadores confiáveis existem

03. Esperar completude versus publicar provisório

Ganho. Evita conclusões sobre fonte parcial

Custo e motivo. Atrasa disponibilidade do resultado

Como decidir. Quando fechamento exige cobertura total

04. Janela longa versus curta

Ganho. Reduz alertas por atraso normal

Custo e motivo. Posterga descoberta de divergência efetiva

Como decidir. Quando retornos possuem atraso conhecido

05. Comparação por item versus agregados

Ganho. Detecta erros que se compensam nos totais

Custo e motivo. Exige processamento e identidade detalhada

Como decidir. Quando integridade financeira é prioritária

06. Tolerância autorizada versus igualdade estrita

Ganho. Reduz trabalho com resíduos legítimos

Custo e motivo. Pode exigir governança e acompanhamento acumulado

Como decidir. Quando arredondamento documentado explica diferenças

07. Ajuste automático versus revisão humana

Ganho. Reduz tempo de resolução de casos claros

Custo e motivo. Aumenta exigência de regras limites e testes

Como decidir. Quando causa e efeito são inequívocos

08. Snapshots versionados versus consultar dados atuais

Ganho. Permite reproduzir fechamento

Custo e motivo. Aumenta armazenamento e retenção

Como decidir. Quando decisões passadas precisam de explicação

09. Sort-merge versus hash em memória

Ganho. Processa conjuntos maiores com memória limitada

Custo e motivo. Acrescenta ordenação e I/O temporário

Como decidir. Quando dataset excede RAM disponível

10. Prioridade por risco versus FIFO puro

Ganho. Atende maior exposição antes

Custo e motivo. Casos menores exigem aging para não esperar demais

Como decidir. Quando valor e prazo variam significativamente

Duas alternativas ao caminho proposto

1. Batch por fechamento

Quando faz sentido. Fontes fecham em períodos definidos

Como implementar. Comparar snapshots completos por versão

Limites. Descoberta de divergência tem atraso

2. Reconciliação contínua

Quando faz sentido. Eventos chegam com baixa latência

Como implementar. Manter pendências e revisar dados tardios

Limites. Não dispensa fechamento e completude

Referências e alcance da evidência

1. Stripe: reconciliação de saldo

Conferência de saldo e transações por período. As respostas e parâmetros de aplicação são propostas autorais.

https://docs.stripe.com/reports/balance

2. TigerBeetle: financial accounting

Contas e registro de transferências equilibradas. As respostas e parâmetros de aplicação são propostas autorais.

https://docs.tigerbeetle.com/coding/financial-accounting/

Continue em: T095 · T097

T097 / J. Sistemas bancários

Antifraude e revisão humana

Antifraude combina sinais imperfeitos, regras, modelos e decisões humanas auditáveis. Arquitetura de treino; não descreve implementação interna do Nubank.

Quando usar. Pagamentos e acessos com risco variável e resposta limitada por prazo.

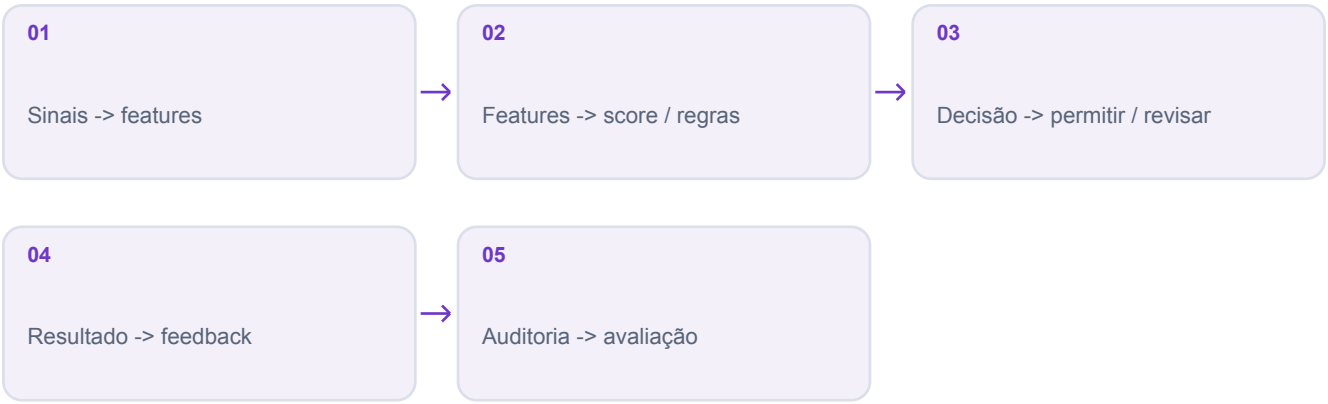
Quando evitar ou limitar. Tratar score probabilístico como verdade ou negar sempre quando modelo cai.

Uma abertura possível

Eu começaria por equilibrar fraude perda legítima e revisão. Depois explicaria registrar origem e idade das features, preservando as restrições do caso.

Peças e tecnologias. Rules engine · Feature store · Model serving · Review queue

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Equilibrar fraude perda legítima e revisão; validar métricas por segmento e severidade.

2. Registrar origem e idade das features; validar qualidade por faixa de idade.

3. Compartilhar definições e testar equivalência temporal; validar comparação offline versus online.

4. Definir fallback por risco e produto; validar simulação de indisponibilidade do modelo.

5. Dimensionar fila humana e prioridade; validar idade e taxa de resolução.

30 perguntas para sustentar o desenho

01. Bloquear mais sempre reduz prejuízo total?

Eu avaliaria perda por fraude junto de bloqueios legítimos, custo de revisão e experiência. Aumentar bloqueio pode reduzir uma métrica e piorar o resultado total; escolheria limiar com evidências e política explícita.

Termo para lembrar: False positive | Base técnica: referência 1.

02. Score pode usar renda desatualizada?

Guardaria origem, instante e validade das features, definindo o que fazer quando estão antigas. Um score rápido com sinal inadequado pode ser pior que uma revisão; freshness faz parte do contrato decisório.

Termo para lembrar: Feature freshness | Base técnica: referência 2.

03. Treino e produção calculam mesma feature?

Manteria definições compatíveis e testes entre cálculo offline e online, respeitando disponibilidade temporal. O mesmo nome não garante a mesma feature; defaults e atrasos podem alterar a decisão produzida em produção.

Termo para lembrar: Serving skew | Base técnica: referência 1.

04. Modelo caiu: aprovar ou bloquear?

Definiria fallback por produto, valor e risco, com limites e monitoramento. Não escolheria fail-open ou fail-closed universalmente; indisponibilidade do modelo não elimina consequências de aprovar fraude ou bloquear clientes legítimos.

Termo para lembrar: Model timeout | Base técnica: referência 2.

05. Quem revisa dez mil casos novos?

Dimensionaria a fila humana pela chegada, duração e especialidade, medindo idade e exposição. Encaminhar mais casos pode melhorar cautela individual e piorar prazo global; o sistema precisa respeitar capacidade de revisão real.

Termo para lembrar: Human review | Base técnica: referência 1.

06. Como explicar uma decisão contestada?

Preservaria versão do modelo, regras e sinais relevantes da decisão, com minimização. Isso permite revisão e explicação posterior; usar dados atuais para reproduzir uma decisão passada pode levar a uma conclusão falsa.

Termo para lembrar: Explanation | Base técnica: referência 2.

07. Resultado futuro pode entrar no treino?

Separaria treino e avaliação pelo tempo e pela disponibilidade real dos sinais. Informação produzida após o resultado não pode prever legitimamente aquela decisão; testaria explicitamente a linha temporal das features.

Termo para lembrar: Feedback leakage | Base técnica: referência 1.

08. Modelo bom piorou após mudança econômica?

Monitoraria distribuição e resultados por coorte, considerando atraso dos rótulos. Drift é um sinal de investigação, não prova isolada de piora; recalibração ou substituição exige evidência e rollout controlado.

Termo para lembrar: Drift | Base técnica: referência 2.

09. Fraudador aprende o limiar: e agora?

Revisaria padrões de abuso e limitaria o que respostas revelam sobre sinais exploráveis. A defesa evolui com adversários, mas mantém explicação e revisão apropriadas para clientes e operadores autorizados.

Termo para lembrar: Adversarial adaptation | Base técnica: referência 1.

10. Motor pode movimentar dinheiro diretamente?

O motor de risco emite decisão dentro de um contrato; execução financeira continua no domínio autorizado. Eu registraria o vínculo e validaria limites, impedindo que um score se transforme em permissão irrestrita.

Termo para lembrar: Action authorization | Base técnica: referência 2.

11. Qual diferença entre fraude e inadimplência?

São problemas diferentes, com sinais e decisões próprios. Eu esclareceria o objetivo antes de escolher modelo ou métrica, evitando usar um score genérico para justificar bloqueios ou concessões sem semântica definida.

Termo para lembrar: Risk problem definition | Base técnica: referência 1.

12. AUC alta basta para aprovar modelo?

Não. Eu avaliaria calibração, desempenho no limiar operacional, custo de erros e segmentos. Uma métrica global pode esconder muitos bloqueios indevidos exatamente na faixa usada para tomar decisões.

Termo para lembrar: Operational threshold | Base técnica: referência 1.

13. Como escolher limiar de revisão humana?

Relacionaria risco esperado, custo de falso positivo e capacidade dos analistas. A fila precisa resolver dentro do prazo; aumentar encaminhamentos sem aumentar capacidade pode apenas atrasar decisões legítimas.

Termo para lembrar: Review threshold | Base técnica: referência 1.

14. Duas compras simultâneas leem o mesmo contador e escapam do limite de velocidade: como resolver?

Se for um limite obrigatório, eu faria check-and-increment ou reserva atômica na autoridade definida, deduplicando por operação. Uma feature aproximada pode alimentar o score quando o risco admite, mas não garante o teto. Separaria sinal antifraude de restrição que precisa resistir à concorrência.

Termo para lembrar: Atomic velocity guard / risk signal

15. Posso reutilizar aprovação antifraude em cache depois de mudar valor ou favorecido?

Eu vincularia a decisão aos inputs canônicos, versões do modelo e da política, além da validade temporal. Alterações críticas exigiriam nova avaliação. Um cache por cliente apenas permite transportar aprovação entre intenções diferentes; só reutilizaria resultado quando a chave representar todas as condições relevantes do contrato.

Termo para lembrar: Decision cache key / intent binding

16. Feedback de chargeback pode demorar meses: o que muda?

Métricas finais chegam atrasadas e podem enviesar avaliação recente. Eu distinguiria sinais intermediários de rótulos consolidados e acompanharia coortes maduras, sem declarar sucesso apenas por ausência inicial de fraude reportada.

Termo para lembrar: Delayed labels | Base técnica: referência 1.

17. Como prevenir vazamento de dados no treino?

Respeitaria cortes temporais e disponibilidade de features, isolando conjuntos de avaliação. Campos produzidos depois da decisão não podem servir como previsão do passado; eu testaria a linha temporal explicitamente.

Termo para lembrar: Temporal leakage | Base técnica: referência 1.

18. O provedor inverteu o score e valores maiores agora significam menor risco: como detectar?

Eu exigiria contrato semântico com direção, faixa, versão e tratamento de ausência. Validaria o adaptador com exemplos rotulados conhecidos antes do rollout, rejeitando versões desconhecidas. Conferir somente que o número está entre zero e um não detecta uma inversão que troca aprovações por bloqueios.

Termo para lembrar: Score semantic contract / directionality

19. Como medir discriminação indevida?

Avaliaria desempenho e impactos por grupos pertinentes, com governança apropriada e dados autorizados. Diferenças exigem investigação de causa e política; uma média boa não resolve riscos de tratamento inadequado.

Termo para lembrar: Fairness evaluation | Base técnica: referência 1.

20. Regra nova deve entrar direto em bloqueio?

Eu poderia começar em observação ou revisão, comparando impacto e falsos positivos antes de ampliar. Para ameaças urgentes, definiria limite e revisão rápida, mantendo responsabilidade explícita pela decisão.

Termo para lembrar: Rule rollout | Base técnica: referência 1.

21. Como lidar com regra e modelo discordando?

Definiria precedência e composição antes da produção, conforme risco e obrigação. Eu registraria quais sinais determinaram a decisão, evitando comportamento acidental dependente da ordem de execução dos componentes.

Termo para lembrar: Decision composition | Base técnica: referência 1.

22. O ataque distribui pequenas transações entre várias contas: como enxergar o padrão?

Eu combinaria sinais por dispositivo, destinatário e relações temporais em janelas delimitadas, respeitando minimização de dados. Um atributo compartilhado não prova fraude; ajudaria a priorizar investigação junto a outros sinais. Limitaria custo e cardinalidade dessas agregações para o próprio detector não virar gargalo.

Termo para lembrar: Cross-entity aggregation / graph signals

23. Como proteger revisão humana de aprovação indevida?

Aplicaria autorização, histórico e segregação quando necessária, vinculando decisão à revisão do caso. Um analista não deve alterar valor depois de aprovação sem novo controle, mesmo sob pressão de fila.

Termo para lembrar: Review authorization | Base técnica: referência 1.

24. Pode compensar falso positivo aprovando pagamento antigo automaticamente?

Não sem verificar validade da intenção e condições atuais. Eu trataria revisão como nova decisão controlada, porque saldo, autorização e consentimento podem ter mudado desde o bloqueio original.

Termo para lembrar: Decision validity | Base técnica: referência 1.

25. Como lidar com feature faltante?

Definiria ausência explicitamente, com política e treinamento compatíveis. Substituir por zero sem significado pode produzir score enganoso; monitoraria taxa de falta e acionaria fallback quando o sinal for essencial.

Termo para lembrar: Missing features | Base técnica: referência 1.

26. Drift de entrada prova que modelo piorou?

Não necessariamente, mas é sinal de investigação. Eu combinaria mudanças de distribuição com calibração, resultados e análise do domínio, evitando trocar modelo automaticamente com base apenas em uma distância estatística.

Termo para lembrar: Drift interpretation | Base técnica: referência 1.

27. Qual arquitetura para latência de poucos milissegundos?

Eu reduziria dependências, pré-computaria sinais apropriados e limitaria inferência pelo orçamento. Confirmaria freshness e fallback; cache de score precisa representar versão e contexto para não reutilizar decisão inválida.

Termo para lembrar: Low-latency scoring | Base técnica: referência 1.

28. Como responder a uma contestação de bloqueio?

O processo deve recuperar evidências e motivo acessível, oferecer revisão autorizada e registrar resultado. Eu separaria explicação ao cliente de detalhes antifraude sensíveis, mantendo rastreabilidade e possibilidade de correção.

Termo para lembrar: Appeal workflow | Base técnica: referência 1.

29. O que torna revisão um gargalo perigoso?

Decisões pendentes acumulam valor, prazo e desgaste do cliente. Eu mediria aging e capacidade por especialidade, ajustando encaminhamento e contingência; adicionar servidores ao modelo não aumenta analistas disponíveis.

Termo para lembrar: Human capacity | Base técnica: referência 1.

30. Como demonstrar nível Staff ao discutir antifraude?

Eu conectaria qualidade do modelo, custo dos erros, autorização, operação humana e efeitos financeiros. O desenho precisa sobreviver à ausência de sinais e permitir contestação, não apenas entregar um score rapidamente.

Termo para lembrar: Socio-technical design | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Limiar mais rigoroso versus permissivo

Ganho. Pode reduzir fraude aprovada

Custo e motivo. Aumenta bloqueios legítimos e revisão

Como decidir. Quando custo e política justificam menor tolerância

02. Features frescas versus pré-computadas

Ganho. Refletem contexto mais recente

Custo e motivo. Aumentam dependências e latência de decisão

Como decidir. Quando mudança rápida altera materialmente o risco

03. Definição compartilhada versus implementações independentes

Ganho. Reduz divergência entre treino e serving

Custo e motivo. Exige coordenação e gestão de versões

Como decidir. Quando features são calculadas em vários ambientes

04. Fallback conservador versus disponibilidade ampla

Ganho. Pode limitar exposição durante falha

Custo e motivo. Pode bloquear operações legítimas

Como decidir. Quando consequência de aprovação indevida é maior

05. Revisão humana versus decisão automática

Ganho. Permite julgamento em casos ambíguos

Custo e motivo. Custa tempo, capacidade e pode variar entre analistas

Como decidir. Quando incerteza e impacto justificam análise

06. Evidência detalhada versus registro mínimo

Ganho. Permite reproduzir e contestar decisão

Custo e motivo. Aumenta dados sensíveis e governança

Como decidir. Quando auditabilidade é necessária

07. Validação temporal versus divisão aleatória

Ganho. Reduz vazamento e aproxima uso real

Custo e motivo. Pode produzir menos dados em certas janelas

Como decidir. Quando contexto muda ao longo do tempo

08. Atualização frequente versus modelo estável

Ganho. Adapta-se mais rápido a novos padrões

Custo e motivo. Exige avaliação rollout e operação mais intensos

Como decidir. Quando drift relevante é confirmado

09. Sinais de defesa privados versus transparência ampla

Ganho. Dificulta adaptação adversarial

Custo e motivo. Limita detalhe de explicação pública

Como decidir. Quando exposição permitiria contornar controles

10. Separar risco e execução versus serviço único

Ganho. Clarifica autoridade e teste de decisões

Custo e motivo. Adiciona contrato e coordenação

Como decidir. Quando modelos evoluem diferente das regras financeiras

Duas alternativas ao caminho proposto

1. Regras determinísticas

Quando faz sentido. Poucos padrões e necessidade de explicação

Como implementar. Versionar regras e acompanhar erros

Limites. Cobertura limitada e manutenção manual

2. Modelo com revisão humana

Quando faz sentido. Há dados e diversidade de padrões

Como implementar. Calibrar score e encaminhar incerteza

Limites. Custo operacional e risco de vieses

Referências e alcance da evidência

1. Stripe: revisão de pagamentos

Revisão humana complementar à automação. As respostas e parâmetros de aplicação são propostas autorais.

https://docs.stripe.com/radar/transaction-reviews

2. OWASP: autorização

Autorização por recurso, deny-by-default e testes. As respostas e parâmetros de aplicação são propostas autorais.

https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html

Continue em: T096 · T098

T098 / J. Sistemas bancários

Gestão de casos e maker-checker

Gestão de casos coordena trabalho humano, decisões versionadas e execução recuperável. Arquitetura de treino; não descreve implementação interna do Nubank.

Quando usar. Disputas, documentos, revisões e decisões com dupla aprovação.

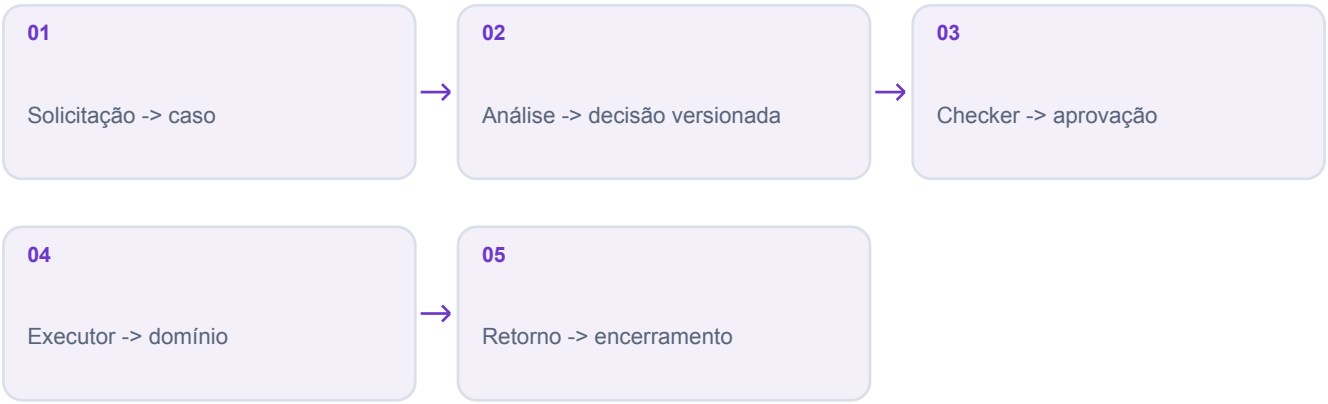
Quando evitar ou limitar. Confundir decisão aprovada com efeito financeiro confirmado.

Uma abertura possível

Eu começaria por atribuir com versão e condição atômica. Depois explicaria validar segregação no backend por identidade, preservando as restrições do caso.

Peças e tecnologias. PostgreSQL · Temporal · Outbox · Object storage · RBAC

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Atribuir com versão e condição atômica; validar cliques concorrentes na mesma fila.

2. Validar segregação no backend por identidade; validar tentativa de autoaprovação.

3. Vincular aprovação à revisão e conteúdo; validar alteração após assinatura do checker.

4. Aguardar confirmação do efeito financeiro; validar timeout após envio ao ledger.

5. Modelar calendário timezone e regra de pausa; validar casos em bordas de calendário.

30 perguntas para sustentar o desenho

01. Dois analistas podem assumir o caso?

Resolveria a disputa no backend por condição e versão da atribuição. A tela pode estar atrasada; assumir caso é uma transição durável, e comandos posteriores revalidam quem tem responsabilidade vigente.

Termo para lembrar: Case ownership | Base técnica: referência 1.

02. Autor da decisão pode aprová-la?

Validaria segregação pela identidade efetiva e regra da decisão, não apenas por rótulo de papel. Quem propôs não aprova quando a política exige independência; a checagem acontece no backend e fica auditável.

Termo para lembrar: Maker checker | Base técnica: referência 2.

03. Aprovação antiga vale após mudar valor?

A aprovação referencia revisão e conteúdo exatos da decisão. Alterar valor, destino ou condição invalida a aprovação anterior conforme política, evitando que um clique genérico permita executar algo diferente do que foi analisado.

Termo para lembrar: Decision revision | Base técnica: referência 1.

04. Quando marcar reembolso como resolvido?

Marcaria execução financeira como confirmada somente após evidência do domínio responsável. Decisão aprovada e efeito pendente são estados distintos; timeout pode manter UNKNOWN sem desfazer a aprovação nem declarar resolução.

Termo para lembrar: Execution state | Base técnica: referência 2.

05. Prazo de três dias é setenta e duas horas?

Usaria calendário, timezone e regra de pausa explícitos, preservando a versão aplicada ao caso. Dias úteis não são um múltiplo fixo de horas; mudanças de política exigem decisão sobre processos em andamento.

Termo para lembrar: Calendar | Base técnica: referência 1.

06. Documento enviado já pode ser aberto?

Anexos entram em fluxo controlado de validação, scanner e autorização antes da liberação. O caso guarda referência e integridade; tamanho ou tipo declarado pelo cliente não bastam para considerar o documento seguro.

Termo para lembrar: Attachment | Base técnica: referência 2.

07. Posso editar motivo de decisão anterior?

Preservaria decisões e correções com autoria e motivo, distinguindo evidência histórica de dados pessoais sujeitos a políticas próprias. Editar o passado sem rastreio dificulta explicar quem autorizou um efeito e por quê.

Termo para lembrar: History | Base técnica: referência 1.

08. Caso encerrado pode voltar à análise?

Reabertura é uma transição autorizada com motivo e versão, não reinício cego de toda execução. Efeitos já confirmados permanecem identificados, para que o processo revisitado não repita crédito ou outra ação anterior.

Termo para lembrar: Reopen | Base técnica: referência 2.

09. Urgentes podem monopolizar analistas?

Combinaria prioridade, prazo e envelhecimento, medindo espera por categoria. Prioridade estrita pode abandonar casos comuns; a política precisa ser explicável e compatível com capacidade e obrigações de cada fila.

Termo para lembrar: Queue fairness | Base técnica: referência 1.

10. Deploy muda casos que duram meses?

Preservaria versão do workflow ou faria migração explícita dos casos, com teste de retomada. Código novo deve entender estados existentes; uma mudança de processo não pode deixar solicitações aceitas impossíveis de concluir.

Termo para lembrar: Workflow version | Base técnica: referência 2.

11. Caso e tarefa devem ser a mesma entidade?

Eu separaria processo de negócio e unidades de trabalho quando possuem ciclos diferentes. Um caso pode reunir tarefas paralelas e decisões; concluir uma tarefa não implica encerrar automaticamente toda a solicitação.

Termo para lembrar: Case versus task | Base técnica: referência 1.

12. Dois casos legítimos tratam a mesma reclamação: como uni-los sem perder obrigações?

Eu escolheria um caso canônico e preservaria referências, evidências e histórico dos dois. Antes de consolidar, conciliaria ações financeiras, aprovações e prazos ainda ativos. A operação seria uma vinculação auditável, não apagar registros; juntar telas não autoriza executar novamente uma obrigação já cumprida.

Termo para lembrar: Case merge / obligation reconciliation

13. Como atribuir caso sem manter transação durante análise?

Atribuição é um registro durável com versão, não um lock de banco aberto por horas. Eu validaria ownership nos comandos e manteria transações curtas para cada transição autorizada.

Termo para lembrar: Human workflow state | Base técnica: referência 1.

14. Supervisor pode assumir caso de agente ausente?

Sim, conforme política e auditoria, alterando ownership e versão. Comandos do responsável anterior precisam falhar ou ser reavaliados, para que a transferência não permita duas decisões conflitantes simultâneas.

Termo para lembrar: Reassignment fencing | Base técnica: referência 1.

15. Acesso ao caso foi revogado depois de gerar uma exportação: como proteger o arquivo?

Eu revalidaria autorização na entrega e no acesso ao artefato, vinculando-o ao escopo correto. Links bearer já emitidos podem continuar válidos; se precisar revogação imediata, usaria um caminho que consulte a autorização atual. Expiração curta reduz exposição, mas não equivale a revogação garantida.

Termo para lembrar: Artifact authorization / revocable delivery

16. Esperar documento pausa SLA sempre?

Só se a regra do processo permitir. Eu registraria intervalo, motivo e condições de pausa, evitando usar estado WAITING para esconder atrasos internos ou alterar métricas sem justificativa auditável.

Termo para lembrar: SLA pause | Base técnica: referência 1.

17. Como anexar documento grande sem passar pela API inteira?

Emitiria autorização curta para upload controlado, validaria o objeto e só o liberaria após verificações. O caso guardaria referência e integridade, mantendo acesso por recurso e scanner fora da transação.

Termo para lembrar: Document workflow | Base técnica: referência 1.

18. O que fazer se integração externa demora dias?

Persistiria estado e prazo, usando job ou workflow durável para consulta e escalonamento. Não manteria thread ou transação bloqueada; a operação precisa sobreviver a reinícios e mudanças de equipe.

Termo para lembrar: Durable waiting | Base técnica: referência 1.

19. Como deduplicar abertura de caso?

Definiria chave da intenção e regras de negócio separadas. O mesmo pedido repetido retorna o caso existente, mas duas disputas parciais legítimas não devem ser proibidas por uma chave ampla demais.

Termo para lembrar: Case identity | Base técnica: referência 1.

20. Pode alterar histórico para corrigir um nome?

Eu distinguiria dado pessoal corrigível de evidência de decisão, observando políticas aplicáveis. Correções relevantes mantêm rastreabilidade; imutabilidade indiscriminada de PII também não deve ser assumida como requisito universal.

Termo para lembrar: History and privacy | Base técnica: referência 1.

21. Quando um workflow engine é excesso?

Quando poucos estados e jobs simples podem ser operados claramente no banco. Eu compararia custo de implementar retomada e timers com o custo de operar a engine, sem adicionar tecnologia apenas por senioridade.

Termo para lembrar: Workflow selection | Base técnica: referência 1.

22. Como evitar que busca desatualizada aprove caso inválido?

A busca apenas localiza candidatos. O comando revalida estado, versão e permissão na fonte autoritativa antes da transição, impedindo que uma projeção antiga conceda autoridade para decidir.

Termo para lembrar: Command revalidation | Base técnica: referência 1.

23. Como versionar formulários e campos obrigatórios?

Vincularia schema à versão do caso ou etapa, com migração explícita. Casos antigos precisam continuar compreensíveis, e novos campos não podem tornar impossível concluir solicitações já aceitas sem política.

Termo para lembrar: Form evolution | Base técnica: referência 1.

24. O que fazer quando analista erra a decisão?

Criaria revisão autorizada e, se necessário, compensação dos efeitos já executados. Não apagaria a decisão anterior; o sistema deve preservar quem decidiu, por quê e como a correção ocorreu.

Termo para lembrar: Decision correction | Base técnica: referência 1.

25. Como tratar ações financeiras manuais de emergência?

Ofereceria caminho controlado com autorização, justificativa, identidade estável e auditoria. Um procedimento emergencial não deve contornar invariantes do ledger, porque falha operacional não torna duplicidade aceitável.

Termo para lembrar: Break-glass operations | Base técnica: referência 1.

26. Pode priorizar somente pelo valor financeiro?

Valor é um critério, mas prazo, urgência e equidade também importam. Eu definiria política transparente e aging para impedir abandono de casos pequenos que ainda precisam de resolução adequada.

Termo para lembrar: Queue policy | Base técnica: referência 1.

27. Como estimar quantidade de analistas?

Usaria chegada por tipo, duração e disponibilidade efetiva, considerando especialidades e variabilidade. A média ajuda dimensionar, mas promessa de percentil de espera exige análise ou simulação adicional da fila.

Termo para lembrar: Workforce sizing | Base técnica: referência 1.

28. Como testar processo de meses em poucos minutos?

Controlaria relógio em testes e simularia sinais, expiração, reabertura e versões. Verificaria invariantes após cada transição, incluindo duplicatas e retorno tardio, sem depender de esperar o tempo real.

Termo para lembrar: Workflow time testing | Base técnica: referência 1.

29. Qual papel da auditoria no caso?

Reconstruir a sequência de decisões e evidências com autoria e contexto. Eu manteria correlação entre tarefas, aprovações e efeitos, evitando que uma coleção de logs técnicos seja a única explicação disponível.

Termo para lembrar: Decision audit | Base técnica: referência 1.

30. O que significa encerrar um caso parcialmente executado?

Precisa de estado e política explícitos. Eu registraria resultados por efeito e pendências remanescentes, sem marcar sucesso global quando parte obrigatória falhou ou permanece UNKNOWN, mesmo que o atendente termine sua tarefa.

Termo para lembrar: Partial completion | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Atribuição exclusiva versus colaboração livre

Ganho. Clarifica responsabilidade e evita decisão concorrente

Custo e motivo. Pode exigir transferência e espera por responsável

Como decidir. Quando uma decisão precisa de dono vigente

02. Maker-checker versus decisão individual

Ganho. Reduz risco de ação unilateral

Custo e motivo. Aumenta tempo e capacidade humana necessária

Como decidir. Quando impacto exige segregação

03. Aprovação por revisão versus aprovação genérica

Ganho. Protege conteúdo efetivamente analisado

Custo e motivo. Mudanças exigem nova aprovação

Como decidir. Quando valor ou destino pode mudar

04. Separar decisão e efeito versus status único

Ganho. Representa pendência e incerteza corretamente

Custo e motivo. Amplia estados e interface operacional

Como decidir. Quando execução depende de sistema externo

05. Calendário versionado versus cálculo simples em horas

Ganho. Preserva prazo conforme regra de negócio

Custo e motivo. Exige manutenção de calendários e versões

Como decidir. Quando dias úteis ou cutoffs importam

06. Quarentena versus liberação imediata de anexo

Ganho. Reduz exposição a arquivos perigosos

Custo e motivo. Adiciona espera e processamento

Como decidir. Quando documentos vêm de fontes não confiáveis

07. Histórico append-only versus edição direta

Ganho. Preserva autoria e sequência de decisões

Custo e motivo. Mais armazenamento e lógica de correção

Como decidir. Quando auditabilidade é necessária

08. Reabertura explícita versus criar caso desconectado

Ganho. Mantém contexto e efeitos anteriores

Custo e motivo. Exige regras para novas revisões

Como decidir. Quando processo legítimo pode ser revisto

09. Aging versus prioridade estrita

Ganho. Evita espera indefinida de classes comuns

Custo e motivo. Pode atrasar alguns casos prioritários

Como decidir. Quando justiça operacional também é requisito

10. Versão fixa versus migrar todos casos

Ganho. Reduz surpresa em processos longos

Custo e motivo. Mantém múltiplas versões operacionais

Como decidir. Quando mudança de regra não deve retroagir

Duas alternativas ao caminho proposto

1. Banco e workers

Quando faz sentido. Poucos fluxos explícitos e equipe pequena

Como implementar. Persistir estados jobs e histórico

Limites. Timers e retomadas exigem implementação cuidadosa

2. Workflow durável

Quando faz sentido. Muitas esperas e integrações prolongadas

Como implementar. Usar motor com versões e activities idempotentes

Limites. Não substitui autorização nem ledger

Referências e alcance da evidência

1. OWASP: autorização

Autorização por recurso, deny-by-default e testes. As respostas e parâmetros de aplicação são propostas autorais.

https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html

2. Azure: saga

Coordenação e compensação de transações locais. As respostas e parâmetros de aplicação são propostas autorais.

https://learn.microsoft.com/en-us/azure/architecture/patterns/saga

Continue em: T097 · T099

T099 / J. Sistemas bancários

Atendimento omnichannel e capacidade

Atendimento omnichannel une conversas duráveis a identidade, roteamento e capacidade humana. Arquitetura de treino; não descreve implementação interna do Nubank.

Quando usar. Chat, e-mail e canais externos com continuidade e histórico.

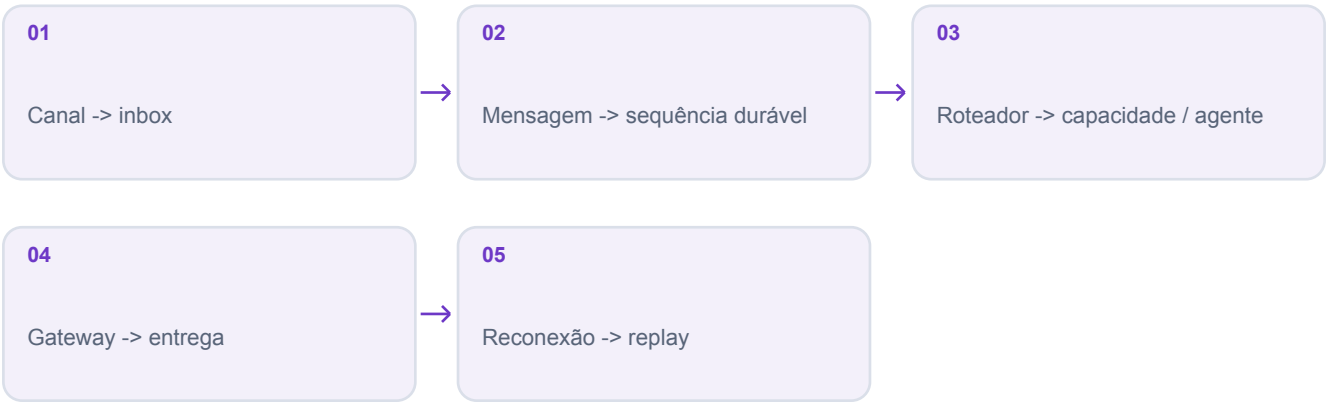
Quando evitar ou limitar. Tratar presença Redis ou confirmação de provedor como prova de leitura pelo cliente.

Uma abertura possível

Eu começaria por confirmar após persistência da mensagem. Depois explicaria atribuir sequência de aceitação por conversa, preservando as restrições do caso.

Peças e tecnologias. WebSocket · PostgreSQL · Redis · Twilio · Amazon Connect

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Confirmar após persistência da mensagem; validar queda imediatamente após confirmação.

2. Atribuir sequência de aceitação por conversa; validar mensagens simultâneas em dispositivos.

3. Retomar cursor e buscar lacunas persistidas; validar reconexão em outro gateway.

4. Dimensionar agentes por chegada e duração; validar ocupação e tempo de primeira resposta.

5. Validar versão vigente da atribuição; validar envio com epoch antigo.

30 perguntas para sustentar o desenho

01. Quando confirmo mensagem ao aplicativo?

Confirmaria recebimento após persistir a mensagem e sua identidade durável. Se o ACK se perder, o cliente repete o mesmo identificador; transporte em tempo real não substitui a garantia de armazenamento.

Termo para lembrar: Durable message

02. Qual ordem vale entre dois dispositivos?

Definiria ordem de aceitação por conversa e preservaria timestamp original como dado adicional. Relógios de dispositivos podem divergir; a sequência usada para replay não deve depender apenas da hora informada pelo cliente.

Termo para lembrar: Sequence | Base técnica: referência 2.

03. WebSocket voltou: como recuperar mensagens?

Ao reconectar, validaria identidade e cursor, buscando histórico e detectando lacunas. Outro gateway deve conseguir recuperar o fluxo; afinidade de sessão pode otimizar, mas não será requisito para a continuidade correta.

Termo para lembrar: Reconnect

04. Mais servidores reduzem espera humana?

Dimensionaria infraestrutura e pessoas separadamente. Mais servidores não aumentam analistas disponíveis; estimaria chegada, duração, simultaneidade e especialidades, depois validaria percentis de espera com variabilidade e capacidade operacional reais.

Termo para lembrar: Agent capacity | Base técnica: referência 2.

05. Agente desconectou e voltou atrasado?

Atribuição tem versão e validade, verificadas nos comandos do agente. Se outro assumir, o antigo não pode continuar agindo com epoch vencido; lease ajuda recuperação, mas não prova que o processo anterior morreu.

Termo para lembrar: Assignment lease

06. WhatsApp prova identidade bancária?

O canal ajuda comunicação, mas não comprova automaticamente identidade bancária. Eu exigiria autenticação apropriada antes de revelar dados ou executar ações, considerando número reciclado, dispositivo compartilhado e troca de titular.

Termo para lembrar: Channel identity | Base técnica: referência 2.

07. Enviado significa lido?

Separaria aceitação interna, aceite pelo provedor, entrega e leitura quando disponíveis. Cada marco possui evidência própria; um e-mail aceito ou push enviado não prova que a pessoa leu a mensagem.

Termo para lembrar: Delivery states | Base técnica: referência 1.

08. Cliente lento pode acumular memória infinita?

Limitaria buffer por conexão e descartaria primeiro sinais efêmeros, como digitação. Mensagens duráveis continuam no histórico; quando necessário, encerraria conexão lenta e exigiria replay em vez de consumir memória ilimitada.

Termo para lembrar: Slow consumer | Base técnica: referência 2.

09. Bot transfere sem perder contexto?

Persistiria contexto autorizado, mensagens e responsabilidade durante transferência ao humano. Resumo ajuda, mas não substitui evidência; respostas atrasadas do bot precisam respeitar a mudança de ownership e não contradizer o atendente.

Termo para lembrar: Human handoff | Base técnica: referência 1.

10. Busca de conversas pode cruzar clientes?

Aplicaria autorização a histórico, busca e anexos, incluindo resultados resumidos. Índices e canais não devem ampliar acesso; testaria perfis e clientes distintos para que conveniência de suporte não produza vazamento.

Termo para lembrar: Privacy | Base técnica: referência 2.

11. Como deduplicar mensagem enviada pelo aplicativo?

O cliente preserva client_message_id antes do primeiro envio, e o servidor impõe unicidade no escopo correto. Resposta perdida devolve a mesma mensagem; hash valida que a chave não mudou de conteúdo.

Termo para lembrar: Message idempotency

12. Como recebe anexos sem expor o atendente a arquivos maliciosos?

Separaria upload em área de quarentena, validação de formato e análise antes de disponibilizar. URLs têm escopo e validade; nomes não viram caminhos confiáveis. A conversa referencia o estado do anexo, permitindo falha de processamento sem perder a mensagem.

Termo para lembrar: Anexos de atendimento

13. Como fazer replay sem lacuna na reconexão?

Coordenaria assinatura e leitura com um limite conhecido, detectando sequências faltantes. O cliente deduplica e busca histórico; conectar no gateway não prova que recebeu todos os eventos anteriores.

Termo para lembrar: Gap-free recovery | Base técnica: referência 1.

14. Redis Pub/Sub basta como histórico de chat?

Não. Eu manteria mensagens duráveis em armazenamento apropriado e usaria distribuição efêmera apenas para atualização. Após perda de conexão, o histórico precisa permitir recuperar conteúdo que não foi recebido ao vivo.

Termo para lembrar: Durable history | Base técnica: referência 3.

15. Como sincroniza mensagens não lidas entre dois aparelhos?

Mantenho por participante um cursor de leitura monotônico sobre a sequência da conversa. Atualizações antigas não reduzem o cursor. A interface pode ter estado local provisório, mas a reconciliação usa o marcador persistido, sem tratar entrega ao aparelho como leitura humana.

Termo para lembrar: Cursor de leitura

16. Sticky session é requisito de correção?

Eu evitaria depender dela para recuperar conversa. Pode otimizar conexão, mas outro gateway deve reconstruir contexto autorizado e cursor; sessão presa a um processo dificulta failover e manutenção.

Termo para lembrar: Gateway failover | Base técnica: referência 1.

17. O que muda para mensagens de e-mail?

Trabalharia com threading, anexos, duplicatas e identidade própria do canal. Não presumiria ordem ou leitura imediata; a integração normaliza mensagens sem apagar as limitações e evidências originais.

Termo para lembrar: Channel semantics | Base técnica: referência 1.

18. Webhook válido autentica o cliente bancário?

Ele autentica a origem da integração conforme contrato, não necessariamente a pessoa. Eu exigiria verificação apropriada antes de revelar dados ou executar ações sensíveis, inclusive quando o canal é conhecido.

Termo para lembrar: Identity boundary | Base técnica: referência 1.

19. Como reservar capacidade de um atendente?

Atualizaria atribuição e contador de slots coerentemente, com condição e versão. A lista de candidatos pode estar atrasada; a confirmação autoritativa precisa impedir atribuições acima da capacidade permitida.

Termo para lembrar: Atomic assignment | Base técnica: referência 1.

20. Atendente pode ter capacidade diferente por canal?

Sim. Voz, chat e e-mail exigem modelos distintos de simultaneidade e habilidade. Eu definiria pesos ou limites por canal, validando na operação para não aplicar uma conta simplista de três atendimentos iguais.

Termo para lembrar: Channel capacity | Base técnica: referência 1.

21. Como transferir conversa sem perder responsabilidade?

Persistiria novo responsável e versão, registrando contexto e pendências. O agente anterior não deve continuar executando comandos sob a atribuição antiga, e o novo precisa confirmar acesso ao histórico autorizado.

Termo para lembrar: Handoff ownership | Base técnica: referência 1.

22. Como impedir starvation de clientes comuns?

Combinaria prioridade com aging e capacidade reservada conforme política. Mediria a espera máxima e percentis por classe, porque uma fila sempre abastecida de urgentes pode impedir atendimento dos demais.

Termo para lembrar: Queue fairness | Base técnica: referência 1.

23. O cliente abriu chat e ligou ao mesmo tempo; cria dois casos?

Eu distinguiria contatos de canal e caso de negócio, vinculando apenas após confirmar identidade e contexto. Dois contatos podem compartilhar investigação sem serem apagados. O roteamento mostra a atividade paralela para evitar trabalho duplicado e respostas contraditórias entre atendentes.

Termo para lembrar: Contatos concorrentes

24. Como unir conversas de dois canais com segurança?

Exigiria vínculo de identidade confiável e autorização, preservando origem. Mesmo telefone ou e-mail aparente pode não bastar; fusão incorreta expõe histórico de uma pessoa a outra.

Termo para lembrar: Conversation linking | Base técnica: referência 1.

25. Pode gravar áudio e transcrever tudo indefinidamente?

Não assumiria essa autorização. Eu definiria finalidade, consentimento quando aplicável, acesso e retenção, incluindo transcrições derivadas. Capacidade técnica de armazenar não determina política legítima de uso.

Termo para lembrar: Conversation retention | Base técnica: referência 1.

26. Qual fallback quando provedor externo cai?

Preservaria mensagens e informaria indisponibilidade conforme contrato, com retry limitado e acompanhamento. Trocar canal exige política e contato autorizado; não enviaria conteúdo sensível por outro meio automaticamente.

Termo para lembrar: Channel failover | Base técnica: referência 1.

27. Como medir qualidade além de tempo médio?

Acompanharia resolução, reabertura, espera por classe, abandono e satisfação, junto da integridade das mensagens. Reduzir duração pode apenas encerrar contatos cedo demais e deslocar trabalho para novos atendimentos.

Termo para lembrar: Support outcomes | Base técnica: referência 1.

28. Como corrige uma mensagem que expôs informação indevida?

Eu registraria a ação de ocultação com motivo e permissão, propagando a mudança para busca e visualizações conforme a política. Evidência restrita pode exigir retenção separada. Não prometo retirar conteúdo já recebido em canais externos ou dispositivos desconectados.

Termo para lembrar: Redação e auditoria

29. Como dimensionar número de conexões?

Estimaria simultaneidade e memória por conexão, buffers, descritores e taxa de mensagens. Pessoas conectadas não equivalem a pessoas ativas; testaria reconexão em massa e distribuição entre gateways.

Termo para lembrar: Connection capacity | Base técnica: referência 1.

30. Qual falha demonstraria no mock?

Derrubaria um gateway após persistir mensagem e antes do ACK. O cliente repete a mesma identidade, reconecta em outro nó e recupera sequência, mostrando durabilidade, deduplicação e continuidade juntas.

Termo para lembrar: Recovery walkthrough | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. ACK após persistência versus antecipado

Ganho. Protege mensagem confirmada contra perda local

Custo e motivo. Aumenta latência de aceitação

Como decidir. Quando histórico durável é requisito

02. Sequência por conversa versus timestamp do cliente

Ganho. Oferece ordem recuperável e desempate

Custo e motivo. Exige coordenação na fronteira da conversa

Como decidir. Quando replay consistente importa

03. Gateway stateless versus sessão presa

Ganho. Facilita failover e balanceamento

Custo e motivo. Exige estado e histórico externos

Como decidir. Quando conexão precisa sobreviver à troca de nó

04. Mais simultaneidade por agente versus atendimento dedicado

Ganho. Aumenta capacidade aparente

Custo e motivo. Pode elevar duração e reduzir qualidade

Como decidir. Quando tarefas permitem alternância sem prejuízo excessivo

05. Lease e versão versus atribuição permanente

Ganho. Recupera agentes indisponíveis

Custo e motivo. Adiciona renovação e validação de epoch

Como decidir. Quando abandono precisa ser detectado automaticamente

06. Autenticação adicional versus confiança no canal

Ganho. Protege ações e dados sensíveis

Custo e motivo. Acrescenta fricção na jornada

Como decidir. Quando identidade do canal é insuficiente

07. Estados detalhados versus indicador único

Ganho. Comunica evidência real de entrega

Custo e motivo. Aumenta complexidade de normalização

Como decidir. Quando canais têm garantias diferentes

08. Buffer limitado versus acumulação livre

Ganho. Contém memória e protege outros clientes

Custo e motivo. Pode exigir reconexão e replay

Como decidir. Quando consumidores lentos são esperados

09. Handoff estruturado versus transferência sem contexto

Ganho. Reduz repetição e melhora continuidade

Custo e motivo. Exige integração e validação do resumo

Como decidir. Quando bot e humano compartilham atendimento

10. Busca central versus acesso por domínio

Ganho. Facilita investigação transversal

Custo e motivo. Amplia superfície de autorização e governança

Como decidir. Quando benefício operacional justifica integração

Duas alternativas ao caminho proposto

1. Serviço de contact center

Quando faz sentido. Capacidades padrão cobrem a operação

Como implementar. Integrar identidade contexto e auditoria

Limites. Custo e limites do fornecedor

2. Plataforma própria

Quando faz sentido. Diferenciação e escala justificam equipe

Como implementar. Construir armazenamento routing e adapters

Limites. Operação contínua e canais são complexos

Referências e alcance da evidência

1. AWS Connect: routing profiles

Filas, capacidade e roteamento de contatos. As respostas e parâmetros de aplicação são propostas autorais.

https://docs.aws.amazon.com/connect/latest/adminguide/concepts-routing.html

2. Twilio: Conversations

Primitivas e integração de conversas. As respostas e parâmetros de aplicação são propostas autorais.

https://www.twilio.com/docs/conversations

3. Redis: Pub/Sub delivery semantics

Entrega at-most-once e perda de mensagens quando assinantes não conseguem recebê-las. O desenho de armazenamento durável e recuperação do chat é uma proposta autoral.

https://redis.io/docs/latest/develop/pubsub/

Continue em: T098 · T100

T100 / J. Sistemas bancários

Shuffle, BFF e composição de interfaces

Bancada inspirada no Shuffle compõe contexto e ações; o case público não revela sua stack interna. Arquitetura de treino; não descreve implementação interna do Nubank.

Quando usar. Plataformas de atendimento com múltiplos produtos e ferramentas.

Quando evitar ou limitar. Colocar regras financeiras no BFF ou apresentar dados de cliente anterior após troca de seleção.

Uma abertura possível

Eu começaria por priorizar contexto mínimo e detalhes sob demanda. Depois explicaria isolar widgets com estados de indisponibilidade, preservando as restrições do caso.

Peças e tecnologias. React · TypeScript · BFF Go · PostgreSQL · SSE · Redis

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Priorizar contexto mínimo e detalhes sob demanda; validar tempo até primeira tarefa útil.

2. Isolar widgets com estados de indisponibilidade; validar queda de uma dependência isolada.

3. Encaminhar regra financeira ao domínio responsável; validar mesma ação por canais diferentes.

4. Limitar concorrência por jornada e destino; validar carga agregada por dependência.

5. Vincular resposta à seleção e cancelar leitura; validar troca rápida entre clientes.

30 perguntas para sustentar o desenho

01. Carregar tudo antes de atender é necessário?

Eu definiria o contexto mínimo necessário para a tarefa e carregaria detalhes por demanda. Isso reduz fanout e exposição; mediria tempo até a primeira ação útil em equipamentos representativos da operação.

Termo para lembrar: Initial context | Base técnica: referência 1.

02. Serviço de cartões caiu: trava a bancada?

Cada widget teria estado próprio e dependências explícitas, permitindo preservar contexto válido quando cartões falham. Ações que exigem esse produto ficam bloqueadas ou pendentes conforme contrato, sem travar tarefas independentes.

Termo para lembrar: Partial response | Base técnica: referência 2.

03. BFF deve calcular saldo disponível?

O BFF compõe e adapta contratos, mas a disponibilidade financeira continua no domínio responsável. Duplicar cálculo de saldo na bancada cria outra regra para manter e pode divergir dos canais do cliente.

Termo para lembrar: BFF boundary | Base técnica: referência 3.

04. Paralelizar vinte chamadas resolve lentidão?

Controlaria concorrência por abertura e por destino, com orçamento total de resposta. Fazer vinte chamadas em paralelo pode reduzir espera individual e saturar serviços; a capacidade agregada determina o limite seguro.

Termo para lembrar: Fanout budget | Base técnica: referência 1.

05. Resposta antiga aparece no novo cliente?

Vincularia toda resposta à seleção e versão atuais do cliente na interface. Cancelamento reduz desperdício, mas a checagem ainda é necessária porque respostas já emitidas podem chegar depois da troca de contexto.

Termo para lembrar: Customer switching | Base técnica: referência 2.

06. Clique em bloquear cartão confirma bloqueio?

Registraria intenção durável e mostraria o marco realmente confirmado. Bloqueio solicitado não é bloqueio concluído; uma operação com resultado desconhecido precisa de acompanhamento, sem induzir o atendente a criar outra ação.

Termo para lembrar: Action intent | Base técnica: referência 3.

07. Esconder botão impede ação indevida?

Autorização é validada no backend para ator, recurso e ação atuais. Esconder controles melhora a interface, mas não impede chamada direta; a auditoria mantém motivo e identidade da operação executada.

Termo para lembrar: Authorization | Base técnica: referência 1.

08. Posso compartilhar resposta entre atendentes?

Cachearia dados com escopo correto, aplicando mascaramento e permissão de forma consistente. Uma resposta montada para supervisor não pode ser reutilizada indiscriminadamente para outro perfil que consulta o mesmo cliente.

Termo para lembrar: Cache scope | Base técnica: referência 2.

09. Máquina do dev representa operação?

Testaria a bancada em hardware e duração de turno representativos, medindo memória, tarefas longas e interação. Experiência boa no computador do desenvolvedor não demonstra produtividade sustentada dos atendentes em operação.

Termo para lembrar: Browser performance | Base técnica: referência 3.

10. Como explicar quem bloqueou o cartão?

Manteria trilha de intenção, ator, motivo, serviço responsável e resultado, ligada a diagnóstico interno autorizado. O suporte precisa explicar o processo sem depender de ler logs brutos nem acessar bancos diretamente.

Termo para lembrar: Audit trail | Base técnica: referência 1.

11. Shuffle público comprova uso de Kafka ou PostgreSQL?

Não. O case descreve contexto e experiência do produto. Eu apresentaria a arquitetura como proposta inspirada nas necessidades observadas, distinguindo evidência pública de escolhas técnicas criadas para o exercício.

Termo para lembrar: Evidence boundary | Base técnica: referência 1.

12. Qual problema o BFF resolve primeiro?

Adapta composição e contrato às tarefas da bancada, reduzindo conhecimento de serviços no navegador. Eu manteria regras financeiras nos domínios, para que a conveniência da interface não crie outra autoridade de negócio.

Termo para lembrar: BFF responsibility | Base técnica: referência 1.

13. Como carregar seis widgets sem esperar o mais lento?

Definiria contexto mínimo e respostas independentes ou progressivas, com deadline por dependência. Um widget atrasado recebe estado explícito, enquanto o atendente continua tarefas que não dependem daquela informação.

Termo para lembrar: Progressive composition | Base técnica: referência 1.

14. BFF pode virar gargalo central?

Sim. Eu limitaria fanout, monitoraria custo por jornada e escalaria processos sem estado durável local. Também separaria caminhos críticos, evitando que uma consulta opcional consuma toda capacidade de composição.

Termo para lembrar: Aggregation bottleneck | Base técnica: referência 1.

15. Como evitar N+1 ao listar cartões e limites?

Usaria APIs em lote ou planejamento de consultas apropriado ao contrato, mantendo autorização por recurso. A solução precisa reduzir chamadas reais, não apenas esconder milhares de requisições atrás de GraphQL.

Termo para lembrar: Batch loading | Base técnica: referência 1.

16. GraphQL elimina necessidade de BFF?

Não automaticamente. Pode expressar consultas, mas composição, custos e políticas continuam existindo. Eu escolheria pela necessidade dos clientes e equipe, limitando profundidade e custo para evitar consultas excessivas.

Termo para lembrar: GraphQL governance | Base técnica: referência 1.

17. Como repartir o tempo disponível entre BFF e dependências?

Eu definiria um deadline total e propagaria o tempo restante, reservando margem para agregação e resposta. Cada integração teria limite próprio; uma chamada iniciada perto do vencimento não deveria receber um timeout completo que ultrapasse o orçamento da jornada.

Termo para lembrar: Deadline propagation

18. Como retornar replay de comando com permissão revogada?

Revalidaria acesso antes de expor operação ou resposta registrada. Idempotência protege efeitos, mas não conserva autorização indefinidamente; a mesma chave não pode virar uma forma de consultar dados após revogação.

Termo para lembrar: Authorized replay | Base técnica: referência 1.

19. Quando usar SSE na bancada?

Quando atualizações principais fluem do servidor e comandos podem seguir por HTTP. Eu manteria recuperação por consulta ou cursor; SSE simplifica transporte, mas não é fonte durável de estados financeiros.

Termo para lembrar: SSE choice | Base técnica: referência 1.

20. Como impedir que retries do BFF multipliquem uma falha?

Eu escolheria uma camada responsável pela repetição e imporia orçamento de tentativas dentro do deadline. Se frontend, BFF e SDK repetem independentemente, a carga cresce multiplicativamente; erros permanentes e comandos sem identidade estável não entram nesse mecanismo.

Termo para lembrar: Retry amplification

21. Como tratar anexos enviados durante o atendimento?

Eu separaria upload e análise do caminho da bancada, limitando tamanho e tipo e mantendo o arquivo em quarentena até validação. O atendente recebe estado explícito; renderizar conteúdo não confiável diretamente na interface amplia a superfície de ataque.

Termo para lembrar: Attachment isolation

22. Como organizar frontend sem microfrontends imediatos?

Começaria com módulos e contratos claros dentro da aplicação. Microfrontends entram quando autonomia de deploy e ownership justificarem custos de integração, performance e consistência visual entre equipes.

Termo para lembrar: Frontend modularity | Base técnica: referência 1.

23. Como manter a bancada utilizável por teclado e leitor de tela?

Eu definiria ordem de foco, nomes acessíveis e anúncio de mudanças relevantes, preservando contexto durante atualizações parciais. Um widget atualizado não deve roubar o foco; validaria as jornadas reais com tecnologias assistivas, além da aparência visual da interface.

Termo para lembrar: Accessible interaction

24. Como executar bloqueio de cartão sem prender a tela?

Persistiria intenção e devolveria operation_id, exibindo estado até confirmação do domínio. A interface continua utilizável; timeout na chamada remota preserva UNKNOWN quando o efeito pode ter ocorrido.

Termo para lembrar: Asynchronous command | Base técnica: referência 1.

25. Qual informação ajuda atendimento a investigar um erro?

Protocolo, marco confirmado, estado atual, dependência afetada e próxima ação permitida. Eu ligaria isso a traces internos autorizados, sem exigir que o atendente interprete stack traces ou consulte bancos manualmente.

Termo para lembrar: Operational context | Base técnica: referência 1.

26. Como tratar dados de produtos com versões diferentes?

Cada widget informa origem e versão adequadas, e a interface não presume snapshot global. Para uma decisão que exige coerência conjunta, consultaria um comando ou serviço capaz de validar as condições necessárias.

Termo para lembrar: Composite consistency | Base técnica: referência 1.

27. Uma busca global pode revelar cliente indevido?

Pode, se autorização for aplicada somente depois de mostrar resultados. Eu filtraria escopo e minimizaria prévias, testando busca, autocomplete e histórico com agentes de países ou equipes diferentes.

Termo para lembrar: Search authorization | Base técnica: referência 1.

28. Como implantar mudança sem quebrar turno aberto?

Manteria contratos compatíveis durante transição e testaria sessões longas. Não presumiria refresh imediato de todos; mudanças de ação sensível precisam continuar validadas no backend para versões antigas.

Termo para lembrar: Long-lived client | Base técnica: referência 1.

29. Como listar muitos resultados de busca sem sobrecarregar a bancada?

Eu retornaria páginas limitadas com cursor opaco e ordenação estável, evitando carregar todos os clientes no navegador. A busca teria tamanho máximo e cancelamento de consultas supersedidas; detalhes sensíveis só seriam carregados após seleção autorizada do resultado.

Termo para lembrar: Bounded search pagination

30. Como fechar esse mock em nível Staff?

Eu conectaria produtividade, autoridade dos domínios, falhas parciais e recuperação de comandos. Explicaria quais decisões começam simples e quais métricas justificam evolução, sem apresentar uma pilha de tecnologias como resultado final.

Termo para lembrar: Architecture synthesis | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Carregamento incremental versus tudo antecipado

Ganho. Reduz tempo inicial e chamadas desnecessárias

Custo e motivo. Detalhes podem exigir espera posterior

Como decidir. Quando tarefas usam apenas parte do contexto

02. Widgets independentes versus resposta indivisível

Ganho. Permite degradação parcial

Custo e motivo. Exige estados e contratos por componente

Como decidir. Quando produtos podem falhar separadamente

03. BFF de composição versus regras duplicadas

Ganho. Mantém autoridade nos domínios

Custo e motivo. Depende de APIs adequadas dos produtos

Como decidir. Quando múltiplos canais compartilham regras financeiras

04. Fanout limitado versus paralelo irrestrito

Ganho. Protege serviços e orçamento de recursos

Custo e motivo. Pode aumentar tempo de widgets não prioritários

Como decidir. Quando dependências têm capacidade finita

05. Versão de seleção versus apenas cancelar requests

Ganho. Impede aplicar resposta de cliente anterior

Custo e motivo. Adiciona controle de contexto na interface

Como decidir. Quando usuário troca rapidamente de atendimento

06. Operação assíncrona versus chamada bloqueante

Ganho. Mantém bancada utilizável durante execução longa

Custo e motivo. Exige acompanhamento e estados intermediários

Como decidir. Quando ação excede prazo interativo

07. Autorização por comando versus somente entrada

Ganho. Respeita recurso e revogação atual

Custo e motivo. Acrescenta avaliação e dependências de política

Como decidir. Quando ações possuem permissões distintas

08. Cache canônico versus resposta pronta por perfil

Ganho. Evita duplicar todas variantes de apresentação

Custo e motivo. Requer autorização e mascaramento após cache

Como decidir. Quando perfis compartilham fatos mas não visibilidade

09. Frontend modular versus microfrontends

Ganho. Reduz coordenação de runtime e bundles duplicados

Custo e motivo. Menos autonomia de deploy por equipe

Como decidir. Quando uma aplicação ainda atende ownership com simplicidade

10. Timeline operacional versus somente logs técnicos

Ganho. Ajuda suporte explicar e resolver casos

Custo e motivo. Exige modelar marcos de negócio

Como decidir. Quando investigação faz parte da rotina do atendimento

Duas alternativas ao caminho proposto

1. Frontend chama APIs independentes

Quando faz sentido. Poucos serviços e contratos estáveis

Como implementar. Aplicar auth e coordenação no cliente

Limites. Mais latência e acoplamento do navegador

2. BFF por jornada

Quando faz sentido. Tela precisa compor contexto específico

Como implementar. Agregar leituras e encaminhar comandos autorizados

Limites. Camada adicional precisa de ownership

Referências e alcance da evidência

1. Azure: gateway aggregation

Agregação, timeouts e falhas parciais. As respostas e parâmetros de aplicação são propostas autorais.

https://learn.microsoft.com/en-us/azure/architecture/patterns/gateway-aggregation

2. Azure: BFF

Backend adaptado às necessidades da interface. As respostas e parâmetros de aplicação são propostas autorais.

https://learn.microsoft.com/en-us/azure/architecture/patterns/backends-for-frontends

3. Case público do Shuffle

Contexto de produto e atendimento; não comprova stack interna. As respostas e parâmetros de aplicação são propostas autorais.

https://www.cristianodalbem.com/shuffle/

Continue em: T099 · T101

T101 / K. Famílias de exercícios

Notificações: push, e-mail e SMS

Notificações entregam informação por canais com garantias e restrições diferentes. Arquitetura de treino; não descreve implementação interna do Nubank.

Quando usar. Alertas financeiros, documentos prontos e comunicação transacional.

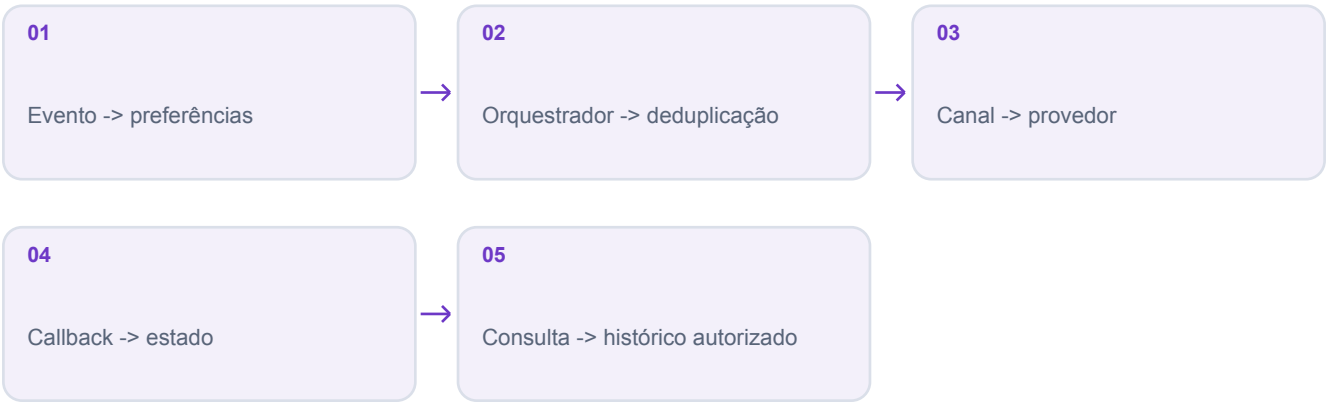
Quando evitar ou limitar. Usar envio de SMS como confirmação financeira ou depender dele para registrar a transação.

Uma abertura possível

Eu começaria por registrar obrigação de notificar junto da transação. Depois explicaria separar aceitação técnica e entrega observada, preservando as restrições do caso.

Peças e tecnologias. FCM · SNS · SES · SMS provider · Outbox

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Registrar obrigação de notificar junto da transação; validar queda entre gravação e publicação.

2. Separar aceitação técnica e entrega observada; validar callbacks reais de cada canal.

3. Invalidar token conforme erro permanente; validar respostas para tokens expirados.

4. Vincular envio à intenção e canal; validar evento repetido antes e após commit.

5. Separar finalidade transacional e marketing; validar matriz de política por mensagem.

30 perguntas para sustentar o desenho

01. Pagamento confirmou mas push sumiu: como evitar?

Registraria a obrigação de avisar junto do evento relevante por mecanismo transacional apropriado. O envio acontece depois, com identidade estável e acompanhamento; uma falha entre banco e provedor não pode apagar silenciosamente essa obrigação.

Termo para lembrar: Outbox notification | Base técnica: referência 1.

02. Provedor aceitou: cliente recebeu?

Aceite do provedor é apenas uma evidência de transporte. Eu manteria estados separados de entrega e leitura quando existirem, sem concluir que o cliente recebeu ou entendeu o aviso apenas pelo retorno da API.

Termo para lembrar: Delivery semantics | Base técnica: referência 2.

03. Push retorna token inválido: retento?

Classificaria o erro conforme contrato do FCM e desativaria tokens permanentemente inválidos quando indicado. Retry indiscriminado consome recursos; atualização de registro e associação ao usuário precisam ser mantidas ao longo do ciclo de vida.

Termo para lembrar: Token lifecycle | Base técnica: referência 1.

04. Mesmo evento dispara dois SMS caros?

Vincularia cada comunicação à intenção, finalidade e canal, com unicidade apropriada. Se o provedor não oferece deduplicação, explicitaria o limite externo e trataria resposta perdida sem prometer entrega única impossível de verificar.

Termo para lembrar: Deduplication | Base técnica: referência 2.

05. Alerta obrigatório segue opt-out comercial?

Separaria marketing e comunicação transacional por finalidade e políticas aplicáveis. Preferências, obrigatoriedade e contingência precisam de regra explícita, evitando que um opt-out amplo silencie alertas que o produto deve fornecer.

Termo para lembrar: Preferences | Base técnica: referência 1.

06. Saldo pode aparecer na tela bloqueada?

Minimizaria conteúdo em tela bloqueada e conduziria o cliente ao aplicativo autenticado para detalhes. Número de telefone ou token pode mudar de titular; o aviso não deve carregar informação financeira desnecessária.

Termo para lembrar: Sensitive content | Base técnica: referência 2.

07. Campanha pode atrasar aviso antifraude?

Reservaria capacidade e prioridade para mensagens críticas, com limites de campanha. Monitoraria prazo por classe; taxa global de envio pode parecer saudável enquanto um alerta importante fica preso atrás de marketing.

Termo para lembrar: Priority | Base técnica: referência 1.

08. Push falhou: envio SMS automaticamente?

Definiria fallback por finalidade, prazo e contato autorizado, considerando entrega tardia do primeiro canal. Trocar automaticamente pode duplicar mensagens e custo; a política precisa distinguir urgência de simples preferência de transporte.

Termo para lembrar: Fallback | Base técnica: referência 2.

09. Como reproduzir texto enviado há meses?

Preservaria template, locale e parâmetros usados, com acesso e retenção adequados. Isso permite explicar o que foi enviado, sem depender da versão atual do texto nem guardar payloads sensíveis indiscriminadamente em logs.

Termo para lembrar: Template version | Base técnica: referência 1.

10. Aviso de código expirado ainda deve sair?

Validaria prazo antes de enviar e descartaria conteúdo vencido com registro apropriado. Um código expirado não deve consumir quota nem confundir o cliente; o backlog precisa considerar utilidade, não apenas ordem de chegada.

Termo para lembrar: TTL | Base técnica: referência 2.

11. Qual entidade representa uma notificação de negócio?

Eu criaria uma intenção com finalidade, destinatário autorizado, conteúdo versionado e prazo. Tentativas por canal são entidades separadas, permitindo recuperar transporte sem transformar cada retry em uma nova comunicação.

Termo para lembrar: Notification intent | Base técnica: referência 1.

12. Notificação deve fazer parte da transação financeira?

A obrigação pode ser registrada atomicamente com o evento financeiro, mas enviar pelo provedor ocorre fora da transação. Eu evitaria manter locks enquanto espero SMS ou e-mail, preservando outbox recuperável.

Termo para lembrar: Transactional obligation | Base técnica: referência 1.

13. Como lidar com push aceito para dispositivo offline?

Respeitaria TTL e semântica do provedor, sem prometer visualização imediata. O aplicativo consulta estado autoritativo depois; uma mensagem de push funciona como aviso, não como fonte única da operação.

Termo para lembrar: Offline delivery | Base técnica: referência 1.

14. Como tratar token que mudou de usuário?

Vincularia registro ao dispositivo e sessão autenticada, atualizando associação e revogação. Também minimizaria conteúdo sensível, porque notificações atrasadas e dispositivos compartilhados tornam identidade de token insuficiente para revelar dados.

Termo para lembrar: Token ownership | Base técnica: referência 1.

15. E-mail devolvido exige retry infinito?

Não. Eu classificaria falha permanente e transitória segundo o provedor, atualizaria qualidade do contato e aplicaria política de fallback. Repetir destino inválido prejudica custo e reputação sem melhorar entrega.

Termo para lembrar: Bounce classification | Base técnica: referência 1.

16. Como evitar avalanche ao recuperar provedor?

Drenaria backlog com rate limit, jitter e prioridades, descartando mensagens vencidas. Capacidade recuperada não autoriza enviar tudo simultaneamente; respeitaria quotas e protegeria avisos novos urgentes.

Termo para lembrar: Recovery throttling | Base técnica: referência 1.

17. Pode consolidar vários alertas financeiros num só?

Somente se a finalidade e o contrato permitirem. Eu preservaria eventos individuais e regras de agrupamento, evitando esconder avisos críticos ou alterar o momento em que o cliente precisa agir.

Termo para lembrar: Notification aggregation | Base técnica: referência 1.

18. Como lidar com callbacks fora de ordem?

Persistiria identidade e estado do provedor, aplicando transições permitidas. Um callback antigo não deve regredir entrega já confirmada; eventos contraditórios ficam investigáveis sem apagar as evidências recebidas.

Termo para lembrar: Delivery state machine | Base técnica: referência 1.

19. Entregue pelo provedor significa exatamente-once ao usuário?

Não. Há limitações de transporte, dispositivo e interface. Eu protegeria a intenção e reduziria duplicatas dentro do contrato, mas não prometeria que uma pessoa verá exatamente uma mensagem em qualquer cenário.

Termo para lembrar: Delivery guarantee | Base técnica: referência 1.

20. Como separar marketing de alertas operacionais?

Usaria finalidade, políticas de preferência, filas e capacidade distintas quando necessário. Uma campanha não deve consumir recursos reservados a avisos críticos; mediria prazo por classe, não apenas taxa global de envio.

Termo para lembrar: Traffic isolation | Base técnica: referência 1.

21. Que conteúdo vai no log de envio?

Identificadores, template, canal, estado e códigos sanitizados. Eu evitaria texto completo e dados pessoais por padrão, mantendo acesso controlado às evidências necessárias para suporte, auditoria e investigação de falhas.

Termo para lembrar: Notification logging | Base técnica: referência 1.

22. Como conciliar horário silencioso com um alerta urgente?

Eu classificaria finalidade e urgência antes do envio e avaliaria preferências aplicáveis ao canal e ao fuso do destinatário. Mensagens adiadas mantêm prazo de validade; uma exceção urgente precisa ser uma política explícita, não uma escolha improvisada pelo worker.

Termo para lembrar: Quiet-hour policy

23. Quando usar fila por canal?

Quando quotas, latência ou falhas precisam de isolamento. Eu manteria prioridade e idempotência por intenção, pois separar filas sem coordenar fallback pode gerar mensagens duplicadas em vários canais.

Termo para lembrar: Channel queues | Base técnica: referência 1.

24. Como impedir SMS para número incorreto?

Manteria verificação e atualização de contato conforme risco, evitando confiar em dado antigo indefinidamente. Conteúdo sensível exige cuidado adicional; entrega bem-sucedida a um número não comprova titularidade atual.

Termo para lembrar: Contact validity | Base técnica: referência 1.

25. Alerta pode conter link de autenticação reutilizável?

Eu usaria tokens de finalidade restrita, validade curta e controles apropriados, evitando credenciais duradouras na mensagem. O desenho considera encaminhamento e vazamento, não apenas o destinatário pretendido.

Termo para lembrar: Link security | Base técnica: referência 1.

26. Qual métrica indica que push está útil?

Mediria entrega disponível, abertura quando apropriada e resultado da jornada, respeitando privacidade. Aceitação pelo provedor sozinha não prova que o aviso ajudou o cliente ou chegou dentro do prazo.

Termo para lembrar: Notification effectiveness | Base técnica: referência 1.

27. Como testar sem enviar mensagens reais a clientes?

Usaria ambientes, destinos e credenciais segregados, simulando callbacks e erros. Para produção, testes sintéticos precisam de contas controladas e limites, evitando disparos acidentais por replay de eventos históricos.

Termo para lembrar: Safe notification testing | Base técnica: referência 1.

28. Reprocessar evento antigo deve reenviar notificação?

Não automaticamente. Eu diferenciaria reconstrução de estado e autorização de envio, consultando identidade e validade da intenção. Replay técnico não renova prazo nem consentimento de uma comunicação já concluída.

Termo para lembrar: Replay policy | Base técnica: referência 1.

29. Como respeitar quotas diferentes por provedor e país?

Eu modelaria orçamento por combinação de canal, provedor e destino, com filas elegíveis e agendamento justo. Um país limitado não deve bloquear todos os demais; ao esgotar quota, considero prazo da mensagem antes de adiar ou selecionar uma rota permitida.

Termo para lembrar: Provider quota scheduling

30. O que mostrar quando todos os canais falham?

O histórico interno mantém falha ou pendência e o produto oferece consulta autenticada. A operação financeira não desaparece porque o aviso falhou; suporte precisa distinguir execução do negócio e comunicação.

Termo para lembrar: Communication failure | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Outbox versus envio inline

Ganho. Preserva obrigação apesar de falha de transporte

Custo e motivo. Adiciona estado, relay e atraso

Como decidir. Quando notificação não pode se perder após commit

02. Estados por canal versus status único

Ganho. Representa garantias reais do provedor

Custo e motivo. Exige normalização e mais testes

Como decidir. Quando canais diferem em recibos disponíveis

03. Limpeza de tokens versus retry persistente

Ganho. Reduz chamadas inúteis e custo

Custo e motivo. Depende de classificação correta e novo registro

Como decidir. Quando erro demonstra invalidade permanente

04. Deduplicação durável versus cache curto

Ganho. Protege replays tardios

Custo e motivo. Consome armazenamento e requer retenção

Como decidir. Quando repetição gera custo ou confusão relevante

05. Política por finalidade versus preferência única

Ganho. Distingue obrigação e marketing

Custo e motivo. Aumenta modelagem e governança

Como decidir. Quando mensagens têm regras diferentes

06. Conteúdo mínimo versus detalhado

Ganho. Reduz exposição em dispositivos compartilhados

Custo e motivo. Exige abrir canal autenticado para detalhes

Como decidir. Quando informação é sensível

07. Capacidade reservada versus fila compartilhada

Ganho. Protege prazo de alertas críticos

Custo e motivo. Pode deixar recursos prioritários ociosos

Como decidir. Quando campanhas disputam quota com avisos urgentes

08. Fallback multicanal versus canal único

Ganho. Aumenta chance de alcançar destinatário

Custo e motivo. Eleva custo e potencial de duplicidade percebida

Como decidir. Quando urgência justifica canal alternativo autorizado

09. Template versionado versus edição no lugar

Ganho. Permite reproduzir comunicação

Custo e motivo. Mantém versões e metadados adicionais

Como decidir. Quando auditoria e suporte precisam do texto exato

10. TTL curto versus longa retenção na fila

Ganho. Evita enviar conteúdo obsoleto

Custo e motivo. Pode reduzir entrega total após falha prolongada

Como decidir. Quando informação perde utilidade rapidamente

Duas alternativas ao caminho proposto

1. Provedor multicanal

Quando faz sentido. Integrações padrão resolvem requisitos

Como implementar. Delegar transporte mantendo intenções e políticas locais

Limites. Dependência e custo do fornecedor

2. Adapters próprios

Quando faz sentido. Há requisitos específicos por país e canal

Como implementar. Isolar SDKs e normalizar estados

Limites. Manutenção de provedores cresce

Referências e alcance da evidência

1. Firebase: erros FCM

Erros de push, tokens e política de repetição. As respostas e parâmetros de aplicação são propostas autorais.

https://firebase.google.com/docs/cloud-messaging/error-codes

2. AWS SNS: delivery status

Observação da entrega aos destinos suportados. As respostas e parâmetros de aplicação são propostas autorais.

https://docs.aws.amazon.com/sns/latest/dg/sns-topic-attributes.html

Continue em: T100 · T102

T102 / K. Famílias de exercícios

Geração e entrega de documentos

Documentos são artefatos derivados que precisam de versão, integridade e acesso controlado. Arquitetura de treino; não descreve implementação interna do Nubank.

Quando usar. Extratos, informes, faturas e evidências geradas assincronamente.

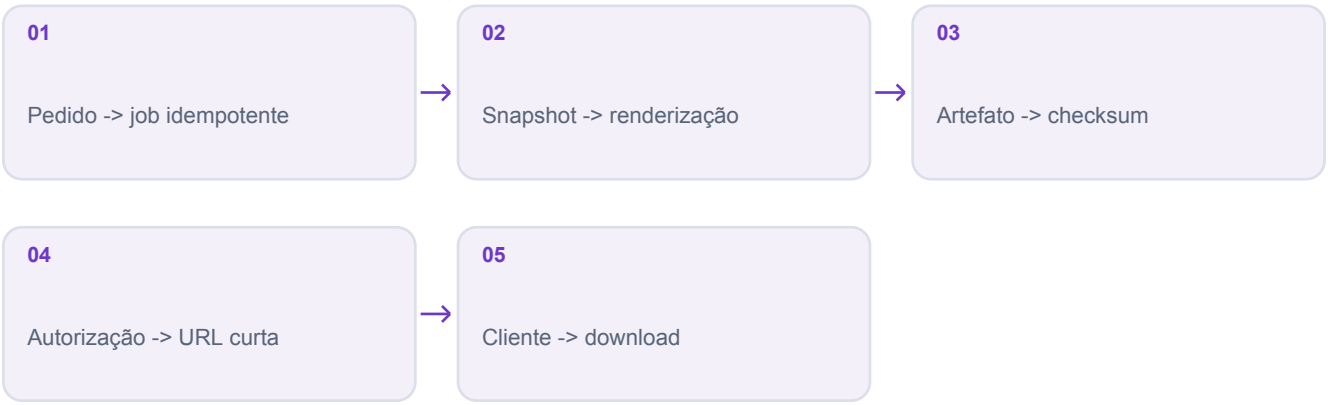
Quando evitar ou limitar. Construir todo PDF em memória ou enviar URL permanente para documento privado.

Uma abertura possível

Eu começaria por fixar versão dos dados e período. Depois explicaria deduplicar por intenção e parâmetros normalizados, preservando as restrições do caso.

Peças e tecnologias. PostgreSQL · S3 · PDF renderer · KMS · Workers

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Fixar versão dos dados e período; validar conferência dos totais por snapshot.

2. Deduplicar por intenção e parâmetros normalizados; validar pedidos equivalentes concorrentes.

3. Usar pipeline limitado ou worker apropriado; validar arquivo máximo com uso de memória.

4. Verificar checksum tamanho e metadados esperados; validar download e validação independente.

5. Tratar URL como credencial temporária delegada; validar acesso após expiração e vazamento.

30 perguntas para sustentar o desenho

01. Extrato pode mudar enquanto PDF é gerado?

Fixaria período e versão dos dados antes de gerar um documento que exige coerência. Sem isso, páginas podem refletir instantes diferentes; validaria totais e referências para demonstrar a origem do artefato final.

Termo para lembrar: Snapshot | Base técnica: referência 1.

02. Cliente pediu cinco vezes: gero cinco arquivos?

Normalizaria parâmetros e associaria o pedido à intenção apropriada, evitando renderizar repetidamente a mesma solicitação por retry. Alterar período ou versão dos dados pode representar documento diferente e precisa de contrato explícito.

Termo para lembrar: Job identity | Base técnica: referência 2.

03. Documento gigante cabe na Lambda?

Estimaria memória, tempo e tamanho máximo, usando streaming ou worker adequado quando necessário. Não presumiria que todo arquivo cabe numa função curta; limites e retomada precisam acompanhar o crescimento do documento.

Termo para lembrar: Streaming | Base técnica: referência 1.

04. Upload terminou: arquivo está correto?

Validaria checksum, tamanho, metadados e conteúdo relevante antes da publicação final. Objeto existente não prova documento correto; a etapa de prontidão deve ser identificável e impedir acesso a artefatos parciais.

Termo para lembrar: Artifact integrity | Base técnica: referência 2.

05. URL assinada autentica quem abriu?

Trataria URL assinada como credencial delegada de duração limitada. Ela não autentica magicamente quem a recebeu depois; definiria validade, escopo e alternativa quando revogação imediata ou acesso mais forte forem necessários.

Termo para lembrar: Signed URL | Base técnica: referência 1.

06. Mesmo sabendo ID posso baixar fatura?

Autorizaria pelo vínculo entre solicitante e documento em cada emissão de acesso. ID opaco reduz enumeração, mas não substitui permissão; testaria consultas cruzadas e sessões cujo acesso foi revogado.

Termo para lembrar: Authorization | Base técnica: referência 2.

07. Worker morreu no multipart: o que fazer?

Persistiria identidade do upload e avaliaria retomar ou abortar partes conforme estado. Limpeza de uploads órfãos entra no ciclo de vida; falha do worker não deve deixar armazenamento temporário crescer indefinidamente.

Termo para lembrar: Partial upload | Base técnica: referência 1.

08. Notificar antes de publicar é seguro?

Publicaria o artefato antes de emitir aviso de prontidão, com obrigação recuperável de comunicação. A sequência precisa tolerar crash, para que o cliente não receba link inexistente nem o documento pronto fique invisível para sempre.

Termo para lembrar: Notification | Base técnica: referência 2.

09. Informe corrigido substitui arquivo silenciosamente?

Criaria revisão identificável, mantendo vínculo e motivo da correção conforme retenção. Substituição silenciosa dificulta explicar o que foi entregue anteriormente; a interface deve indicar qual versão está sendo consultada.

Termo para lembrar: Correction | Base técnica: referência 1.

10. Documento antigo deve ficar sempre quente?

Definiria classes de armazenamento pelo prazo de acesso e frequência, testando recuperação real. Um histórico barato que exige restauração lenta não pode ser oferecido com a mesma promessa de download imediato dos documentos recentes.

Termo para lembrar: Retention | Base técnica: referência 2.

11. Como nomear o objeto sem expor CPF no caminho?

Usaria identificador opaco e metadados autorizados, evitando dados pessoais em chaves, URLs e logs. A autorização continua obrigatória, pois nome difícil de adivinhar não substitui controle de acesso.

Termo para lembrar: Object naming | Base técnica: referência 1.

12. Checksum é igual ao ETag em todos uploads?

Não assumiria isso, especialmente com multipart e configurações específicas. Eu usaria o mecanismo de integridade contratado e armazenaria o valor esperado, verificando documentação antes de tratar ETag como hash universal.

Termo para lembrar: Checksum semantics | Base técnica: referência 1.

13. Como garantir que o PDF corresponde ao período pedido?

Persistiria parâmetros normalizados, versão da fonte e identidade do job, validando metadados e totais antes de publicar. O arquivo precisa estar ligado à solicitação, não apenas ao nome escolhido pelo worker.

Termo para lembrar: Document provenance | Base técnica: referência 1.

14. Como publicar vários arquivos que compõem um único informe?

Eu geraria cada objeto em uma área ainda não divulgada, verificaria integridade e publicaria um manifesto apontando para todas as versões concluídas. O cliente acessa o conjunto pelo manifesto; arquivos parciais não aparecem como um informe completo durante a geração.

Termo para lembrar: Atomic manifest publication

15. Como impedir que renderizador acesse URLs internas maliciosas?

Restringiria rede e fontes permitidas, sanitizaria templates e trataria conteúdo externo como não confiável. Renderização de HTML pode abrir superfície de SSRF; isolamento reduz o impacto de entradas perigosas.

Termo para lembrar: Renderer isolation | Base técnica: referência 1.

16. O que fazer com fontes e assets indisponíveis?

Empacotaria versões necessárias ou definiria fallback validado, evitando dependência imprevisível durante renderização. Testaria caracteres e layout, porque documento tecnicamente criado pode estar ilegível ou com valores cortados.

Termo para lembrar: Deterministic rendering | Base técnica: referência 1.

17. Como cobrar capacidade por documento de tamanho variável?

Estimaria custo por páginas ou volume e usaria classes de execução, com limites. Uma fila só por contagem pode misturar jobs pequenos com enormes e prejudicar prazo dos usuários.

Termo para lembrar: Work estimation | Base técnica: referência 1.

18. Quando retornar arquivo direto na resposta HTTP?

Quando tamanho e duração cabem confortavelmente no contrato. Para geração longa, eu devolveria operação e acompanhamento, evitando timeouts ambíguos e manter conexões presas sem necessidade.

Termo para lembrar: Sync versus async generation | Base técnica: referência 1.

19. URL assinada expirada deve gerar nova cópia?

Não necessariamente. Eu reautorizaria o solicitante e emitiria novo acesso ao mesmo artefato válido. Expiração de credencial de download é diferente de expiração ou invalidação do documento.

Termo para lembrar: Access renewal | Base técnica: referência 1.

20. Como revogar acesso antes do vencimento da URL?

A estratégia depende do mecanismo escolhido; eu consideraria proxy autenticado, política ou revogação suportada. Não prometeria revogação individual imediata quando o contrato da URL assinada não oferece essa propriedade.

Termo para lembrar: Revocation requirement | Base técnica: referência 1.

21. Como comprovar que um PDF baixado é o documento publicado?

Eu manteria um hash da versão final e, quando houver requisito de autenticidade verificável por terceiros, adotaria assinatura digital com gestão de chaves. Hash detecta alteração, mas sozinho não prova quem publicou; identidade, cadeia de confiança e formato precisam ser definidos.

Termo para lembrar: Artifact authenticity

22. Como retomar geração que caiu no meio?

Usaria checkpoints onde fizer sentido ou reexecutaria deterministamente com a mesma identidade. A publicação final é condicional; tentativas concorrentes não podem substituir um artefato válido por um parcial.

Termo para lembrar: Generation recovery | Base técnica: referência 1.

23. O que medir no SLA de documentos?

Tempo até artefato correto disponível, taxa de erro e sucesso de download autorizado. Medir apenas conclusão do worker ignora falhas de publicação, corrupção ou links que o cliente não consegue abrir.

Termo para lembrar: Document SLI | Base técnica: referência 1.

24. Como limitar abuso de pedidos repetidos?

Aplicaria rate limit e deduplicação conforme parâmetros, além de autorização. Também controlaria número de jobs ativos por cliente, evitando que solicitações caras esgotem renderizadores e armazenamento temporário.

Termo para lembrar: Admission control | Base técnica: referência 1.

25. Como informar progresso sem milhares de clientes consultando a cada segundo?

Eu retornaria uma operação consultável com orientação de intervalo e usaria notificações como sinal de mudança quando útil. O cliente aplica backoff e sempre pode recuperar o estado por consulta; perder uma notificação não deve esconder indefinidamente um documento concluído.

Termo para lembrar: Status notification contract

26. Como validar acessibilidade do documento?

Verificaria estrutura, ordem de leitura e representação textual apropriadas, além do visual. Um PDF bonito pode ser inacessível; a aceitação precisa considerar ferramentas assistivas e os usuários que dependem delas.

Termo para lembrar: Document accessibility | Base técnica: referência 1.

27. O que fazer quando um lote gera documentos de clientes trocados?

Interromperia publicação, revogaria acessos possíveis e investigaria extensão pelo lineage. O desenho deve associar fonte, cliente e artefato explicitamente, com testes de isolamento antes da liberação.

Termo para lembrar: Cross-customer isolation | Base técnica: referência 1.

28. Como excluir documentos sem apagar prova necessária?

Aplicaria política por tipo e finalidade, coordenada com retenções aplicáveis. Separaria metadados mínimos de auditoria e conteúdo pessoal quando adequado, sem assumir retenção infinita nem exclusão indiscriminada.

Termo para lembrar: Document lifecycle | Base técnica: referência 1.

29. Pode enviar o PDF inteiro como anexo de e-mail?

Depende de tamanho e confidencialidade. Eu preferiria acesso autenticado ou temporário quando apropriado, considerando encaminhamento e retenção fora do controle do banco; conveniência não elimina exposição.

Termo para lembrar: Delivery channel | Base técnica: referência 1.

30. Qual desenho mínimo você escolheria?

API registra pedido, worker lê snapshot e gera artefato, storage guarda arquivo e API autoriza acesso. Adicionaria componentes por necessidade medida, mantendo idempotência, integridade e observabilidade desde o início.

Termo para lembrar: Minimal document architecture | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Snapshot fixo versus leitura corrente

Ganho. Produz documento internamente coerente

Custo e motivo. Exige retenção ou transação apropriada da fonte

Como decidir. Quando valores não podem mudar entre páginas

02. Deduplicar geração versus renderizar cada pedido

Ganho. Reduz custo de retries e solicitações equivalentes

Custo e motivo. Exige definir identidade e validade do artefato

Como decidir. Quando parâmetros e versão determinam documento reutilizável

03. Streaming versus montar em memória

Ganho. Limita uso de RAM para arquivos grandes

Custo e motivo. Pode complicar layout e etapas de renderização

Como decidir. Quando tamanho excede memória confortável

04. Validação pré-publicação versus disponibilizar cedo

Ganho. Evita entregar artefato incompleto

Custo e motivo. Aumenta tempo até download

Como decidir. Quando integridade documental é essencial

05. URL curta versus proxy autenticado

Ganho. Reduz carga no serviço de aplicação

Custo e motivo. Acesso delegado pode ser compartilhado até expirar

Como decidir. Quando contrato aceita credencial temporária

06. Autorização a cada acesso versus link duradouro

Ganho. Respeita revogação e contexto atual

Custo e motivo. Adiciona dependência e processamento

Como decidir. Quando confidencialidade exige controle contínuo

07. Multipart versus upload único

Ganho. Permite paralelismo e retomada de partes

Custo e motivo. Exige gerenciar partes e limpeza

Como decidir. Quando arquivos grandes justificam overhead

08. Aviso assíncrono versus resposta com arquivo

Ganho. Não prende conexão durante geração

Custo e motivo. Exige acompanhamento e canal de prontidão

Como decidir. Quando geração é longa

09. Revisões preservadas versus sobrescrita

Ganho. Permite explicar correções e entregas anteriores

Custo e motivo. Aumenta armazenamento e gestão de acesso

Como decidir. Quando versão anterior tem valor probatório

10. Arquivo frio versus storage quente

Ganho. Reduz custo de retenção longa

Custo e motivo. Pode aumentar prazo e custo de leitura

Como decidir. Quando frequência baixa e SLA permitem restauração

Duas alternativas ao caminho proposto

1. Geração sob demanda

Quando faz sentido. Poucas consultas e dados recentes

Como implementar. Gerar por pedido com limite e cache versionado

Limites. Primeiro acesso demora

2. Pré-geração em lote

Quando faz sentido. Fechamentos previsíveis e alto acesso

Como implementar. Produzir artefatos antes do pico

Limites. Armazena documentos pouco acessados

Referências e alcance da evidência

1. S3: URLs assinadas

Delegação temporária de acesso a objetos. As respostas e parâmetros de aplicação são propostas autorais.

https://docs.aws.amazon.com/AmazonS3/latest/userguide/using-presigned-url.html

2. S3: multipart upload

Partes, conclusão e recuperação de upload. As respostas e parâmetros de aplicação são propostas autorais.

https://docs.aws.amazon.com/AmazonS3/latest/userguide/mpuoverview.html

Continue em: T101 · T103

T103 / K. Famílias de exercícios

Feeds e timelines

Feeds combinam ingestão de eventos, fanout, ordenação e leitura paginada. Arquitetura de treino; não descreve implementação interna do Nubank.

Quando usar. Timeline de atividade, histórico de atendimento e notificações.

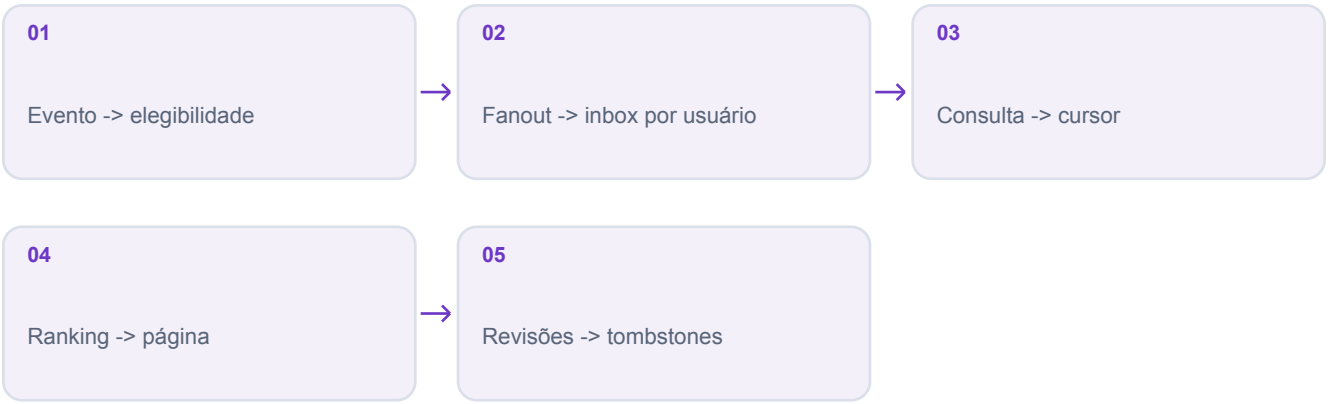
Quando evitar ou limitar. Usar feed eventualmente consistente como autoridade para saldo ou autorização.

Uma abertura possível

Eu começaria por materializar inbox quando leitura justificar custo. Depois explicaria separar autores de alto alcance no modelo, preservando as restrições do caso.

Peças e tecnologias. PostgreSQL · DynamoDB · Kafka · Redis · Cursor pagination

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Materializar inbox quando leitura justificar custo; validar custo por distribuição de seguidores.

2. Separar autores de alto alcance no modelo; validar lag por classe de autor.

3. Usar cursor estável com desempate único; validar novos itens durante paginação.

4. Definir ordenação pelo contrato e desempate; validar eventos simultâneos e atrasados.

5. Revalidar acesso e propagar revogação; validar remoção de acesso após fanout.

30 perguntas para sustentar o desenho

01. Uma publicação precisa copiar para todos?

Escolheria fanout na escrita quando muitas leituras compensam materialização das inboxes. Mediria distribuição de seguidores, porque autores de alto alcance multiplicam escritas mesmo quando poucos destinatários abrem o feed.

Termo para lembrar: Fanout on write | Base técnica: referência 1.

02. Celebridade quebra fila de fanout?

Separaria autores muito populares quando seu fanout domina capacidade, usando composição híbrida se necessário. Isso introduz dois caminhos a ordenar e autorizar; o ganho precisa aparecer em custo e freshness medidos.

Termo para lembrar: Hot publisher | Base técnica: referência 2.

03. Offset permite rolagem sem duplicatas?

Usaria cursor com ordenação e desempate estáveis, evitando deslocamento típico de offset durante inserções. Também definiria comportamento quando ranking muda, pois paginação estável exige contexto adicional em feeds dinâmicos.

Termo para lembrar: Cursor pagination | Base técnica: referência 1.

04. Ordem cronológica significa timestamp do celular?

Definiria o significado da ordem e preservaria sequência ou desempate apropriados. Timestamp do dispositivo não representa causalidade global; eventos de vários produtos podem precisar apenas de uma apresentação consistente e explicável.

Termo para lembrar: Ordering | Base técnica: referência 2.

05. Post privado já copiado perde permissão?

Revogação precisa alcançar leitura autorizada, não apenas futuras publicações. Mesmo com item copiado na inbox, eu revalidaria ou bloquearia exposição conforme o contrato, enquanto a limpeza propagada remove cópias derivadas.

Termo para lembrar: Authorization | Base técnica: referência 1.

06. Excluir publicação apaga todas cópias imediatamente?

Aplicaria tombstone ou mecanismo de invalidação e a política de exclusão aos derivados relevantes. A interface não deve exibir conteúdo revogado enquanto cópias ainda existem; retenção de evidência exige regra separada.

Termo para lembrar: Deletion | Base técnica: referência 2.

07. Replay repete item na timeline?

Usaria identidade estável do evento e inserção idempotente na timeline. Um item pode chegar por replay ou múltiplos caminhos; a deduplicação precisa preservar publicações legítimas distintas, mesmo quando o conteúdo é semelhante.

Termo para lembrar: Deduplication | Base técnica: referência 1.

08. Mudar algoritmo altera páginas já abertas?

Decidiria se a sessão exige snapshot estável ou aceita atualização dinâmica com deduplicação. Mudar ranking entre páginas pode repetir ou omitir itens; o comportamento precisa ser uma escolha de produto, não acidente.

Termo para lembrar: Ranking | Base técnica: referência 2.

09. Cache por usuário pode ficar enorme?

Limitaria janela e conteúdo do cache por usuário, incluindo regras de acesso e invalidação. Personalização multiplica variantes; mediria taxa de acerto e memória para não manter páginas quase nunca reutilizadas.

Termo para lembrar: Cache | Base técnica: referência 1.

10. Novo seguidor recebe todo histórico?

Definiria quanto histórico um novo vínculo deve receber e executaria backfill limitado, separado do tráfego recente. Reprocessamento preserva identidade dos itens e não pode atrasar indefinidamente a atualização dos usuários ativos.

Termo para lembrar: Backfill | Base técnica: referência 2.

11. O feed precisa de ordem global entre todos usuários?

Normalmente não. Eu definiria ordem por timeline ou entidade, evitando coordenação global desnecessária. Se houver requisito específico, explicaria custo e impacto em disponibilidade antes de escolher um sequenciador central.

Termo para lembrar: Ordering scope | Base técnica: referência 1.

12. Como lidar com item editado depois do fanout?

Manteria referência à versão ou estratégia de atualização das cópias. A escolha depende da consistência exigida; revisões devem invalidar conteúdo antigo quando ele não pode mais ser apresentado.

Termo para lembrar: Feed update | Base técnica: referência 1.

13. Qual chave de partição usaria na inbox?

Usuário e janela podem ser candidatos, conforme volume e consultas. Eu testaria usuários muito ativos e limites de partição, evitando uma chave que concentre todo histórico num único ponto quente.

Termo para lembrar: Inbox partitioning | Base técnica: referência 1.

14. Cursor deve conter dados sensíveis?

Eu manteria somente informação necessária, protegendo integridade e autorizando a consulta independentemente do cursor. Cursor alterado não pode permitir acessar timeline de outro usuário ou ignorar filtros de permissão.

Termo para lembrar: Cursor security | Base técnica: referência 1.

15. Como disponibilizar publicações agendadas sem antecipá-las no feed?

Eu separaria criação de conteúdo e elegibilidade para publicação, usando horário do servidor e estado durável. O scheduler pode disparar repetidamente, mas a transição é condicional; o feed só incorpora itens cuja publicação foi confirmada, mesmo se uma tarefa rodar cedo.

Termo para lembrar: Scheduled publication

16. Fanout pode terminar parcialmente sem perder controle?

Sim, com manifesto ou progresso recuperável e inserções idempotentes. Eu registraria o que falta e retomaria, evitando tratar publicação do evento como prova de que todas as inboxes foram atualizadas.

Termo para lembrar: Fanout progress | Base técnica: referência 1.

17. Como excluir dados de usuário em muitas inboxes?

Manteria referências e política de revogação, bloqueando exposição antes de completar limpeza quando necessário. O processo precisa alcançar caches e índices, preservando apenas dados autorizados pela política aplicável.

Termo para lembrar: Feed deletion | Base técnica: referência 1.

18. O que acontece quando consumidor do feed fica dias parado?

Verificaria retenção e capacidade de replay. Se o log não cobre a lacuna, reconstruiria a partir de fonte preservada; iniciar no offset atual sem aviso produz histórico incompleto.

Termo para lembrar: Replay retention | Base técnica: referência 1.

19. Como medir freshness por usuário?

Acompanharia tempo entre evento elegível e disponibilidade na timeline, com percentis e classes de fanout. Lag global pode esconder uma partição quente ou grupo de usuários permanentemente atrasado.

Termo para lembrar: Feed freshness | Base técnica: referência 1.

20. Qual risco de cachear a primeira página por muito tempo?

Ela pode esconder conteúdo novo ou manter conteúdo revogado. Eu definiria TTL e invalidação conforme finalidade, usando versões e proteção de acesso quando o conteúdo tiver sensibilidade relevante.

Termo para lembrar: Page cache | Base técnica: referência 1.

21. Como tratar spam num feed de atividades?

Aplicaria limites e políticas de agrupamento ou supressão, preservando eventos necessários à auditoria. Um resumo visual pode reduzir ruído sem apagar fatos financeiros que continuam consultáveis na fonte apropriada.

Termo para lembrar: Feed noise control | Base técnica: referência 1.

22. Pode reutilizar feed como extrato bancário oficial?

Só se cumprir os requisitos de integridade, completude e semântica do extrato. Uma timeline otimizada para descoberta geralmente aceita atraso e ranking; eu não a promoveria a autoridade contábil por conveniência.

Termo para lembrar: Feed versus statement | Base técnica: referência 1.

23. Como juntar eventos de múltiplos produtos?

Normalizaria envelope e contratos de apresentação, preservando origem e versão. O feed não deve inventar uma ordem causal entre serviços independentes; relações importantes precisam de identificadores explícitos.

Termo para lembrar: Cross-domain timeline | Base técnica: referência 1.

24. Quando uma busca em vez de feed é melhor?

Quando o usuário precisa localizar fatos por critérios, e não acompanhar sequência ou recomendação. Eu separaria índices e SLOs conforme os padrões, sem forçar consultas complexas sobre inbox materializada.

Termo para lembrar: Access pattern fit | Base técnica: referência 1.

25. O que fazer quando usuário segue milhares de fontes?

Mediria custo de composição e escolheria estratégia híbrida. Limites, pré-computação e paginação de candidatos precisam considerar distribuição real, porque média de seguidores esconde usuários extremos.

Termo para lembrar: High-degree users | Base técnica: referência 1.

26. Como testar feed sem verificar só a aparência?

Validaria autorização, duplicatas, ordenação definida, paginação e revogação durante falhas. Também compararia reconstrução com a fonte, garantindo que otimizações de cache e fanout não apaguem itens elegíveis.

Termo para lembrar: Feed invariants | Base técnica: referência 1.

27. Como lidar com notificações e feed do mesmo evento?

Usaria identidades compartilhadas de origem, mas responsabilidades separadas. Publicar na timeline não prova que push foi entregue, e falha de notificação não deve remover o registro consultável pelo usuário.

Termo para lembrar: Channel independence | Base técnica: referência 1.

28. Qual custo principal do fanout on write?

Escritas e armazenamento crescem com destinatários, mesmo que muitos nunca leiam. Eu compararia esse custo com latência e frequência de leitura, considerando limpeza e atualização das cópias materializadas.

Termo para lembrar: Write amplification | Base técnica: referência 1.

29. Como preservar histórico ao mudar schema de item?

Manteria renderização compatível ou migração versionada, evitando que frontend novo deixe eventos antigos ilegíveis. Testaria amostras históricas e defaults, preservando significado de valores e estados apresentados.

Termo para lembrar: Historical compatibility | Base técnica: referência 1.

30. Como fechar o exercício sem superdimensionar?

Apresentaria padrão de acesso, distribuição de audiência e garantia de leitura. Começaria com solução que atende esses dados e indicaria o limite que justificaria fanout híbrido, shards ou ranking mais complexo.

Termo para lembrar: Feed design rationale | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Fanout na escrita versus na leitura

Ganho. Entrega páginas com menor composição

Custo e motivo. Multiplica escrita e armazenamento por destinatário

Como decidir. Quando muitas leituras justificam materialização

02. Modelo híbrido versus único caminho

Ganho. Controla custo de autores muito populares

Custo e motivo. Exige combinar e ordenar origens diferentes

Como decidir. Quando distribuição de audiência é desigual

03. Cursor versus offset

Ganho. Mantém continuação mais estável sob inserções

Custo e motivo. Dificulta salto arbitrário por número de página

Como decidir. Quando navegação é sequencial

04. Ordem cronológica versus ranking

Ganho. Oferece previsibilidade temporal

Custo e motivo. Pode reduzir descoberta de conteúdo relevante

Como decidir. Quando histórico e recência importam mais

05. Revalidar acesso versus confiar na cópia

Ganho. Respeita revogação posterior ao fanout

Custo e motivo. Adiciona custo na leitura

Como decidir. Quando conteúdo possui restrição mutável

06. Tombstones versus remoção imediata de todas cópias

Ganho. Permite bloquear exibição durante propagação

Custo e motivo. Mantém estado e exige limpeza posterior

Como decidir. Quando exclusão distribuída demora

07. Identidade de evento versus dedup por conteúdo

Ganho. Preserva publicações legítimas semelhantes

Custo e motivo. Requer ID estável na origem

Como decidir. Quando replay e múltiplos caminhos são esperados

08. Snapshot de sessão versus ranking vivo

Ganho. Evita repetição e saltos durante paginação

Custo e motivo. Pode mostrar ordem menos atual

Como decidir. Quando estabilidade de navegação importa

09. Cache individual versus composição por consulta

Ganho. Reduz latência de páginas frequentes

Custo e motivo. Consome memória proporcional à personalização

Como decidir. Quando reutilização é suficiente

10. Backfill limitado versus histórico completo

Ganho. Contém custo e tempo de preparação

Custo e motivo. Usuário não recebe toda história automaticamente

Como decidir. Quando produto aceita janela definida

Duas alternativas ao caminho proposto

1. Fanout on read

Quando faz sentido. Poucos autores e muitas publicações

Como implementar. Compor página consultando fontes autorizadas

Limites. Leitura fica mais cara

2. Fanout híbrido

Quando faz sentido. Autores têm distribuições muito diferentes

Como implementar. Pré-computar comuns e buscar populares na leitura

Limites. Dois caminhos precisam de ordenação consistente

Referências e alcance da evidência

1. DynamoDB: índices

Custos e usos de índices secundários. As respostas e parâmetros de aplicação são propostas autorais.

https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/bp-indexes.html

2. DynamoDB: paginação

Continuação de consultas paginadas. As respostas e parâmetros de aplicação são propostas autorais.

https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Query.Pagination.html

Continue em: T102 · T104

T104 / K. Famílias de exercícios

Geolocalização e índices espaciais

Consultas espaciais usam aproximações indexadas e refinamento geométrico com unidades explícitas. Arquitetura de treino; não descreve implementação interna do Nubank.

Quando usar. Busca de agentes, terminais, localidades e análise geográfica autorizada.

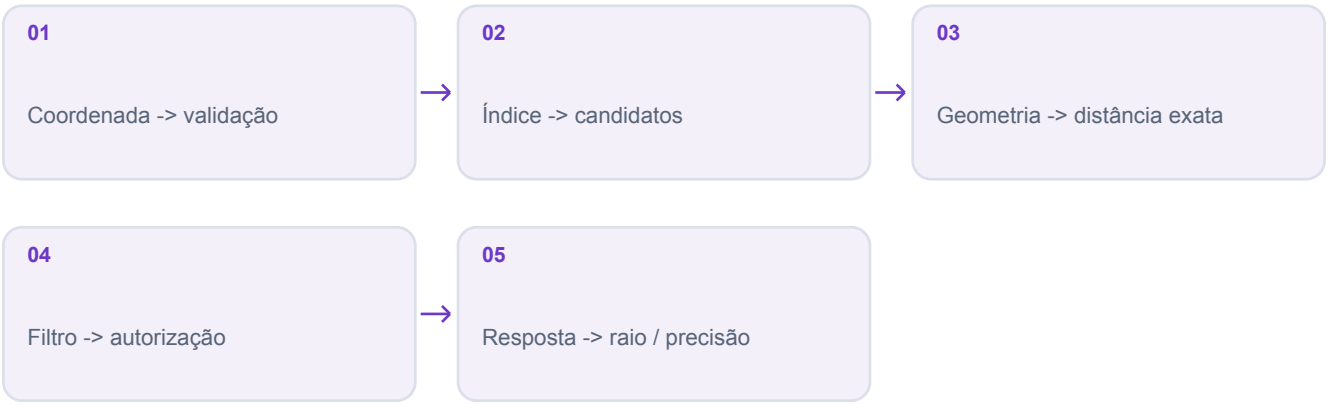
Quando evitar ou limitar. Confundir coordenadas em graus com distância em metros ou tratar GPS como prova absoluta.

Uma abertura possível

Eu começaria por escolher referência e tipo espacial adequados. Depois explicaria filtrar candidatos e aplicar predicado preciso, preservando as restrições do caso.

Peças e tecnologias. PostGIS · GiST · Geography · H3 · Geohash

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Escolher referência e tipo espacial adequados; validar pontos conhecidos em regiões distintas.

2. Filtrar candidatos e aplicar predicado preciso; validar plano e quantidade de refinamentos.

3. Usar distância com unidades contratadas; validar distâncias verificadas em mapas de teste.

4. Confirmar métrica e refinamento necessários; validar pontos perto e longe do equador.

5. Consultar células adjacentes compatíveis com raio; validar pontos em fronteiras de células.

30 perguntas para sustentar o desenho

01. Latitude longitude são metros?

Eu verificaria sistema de referência, tipo e unidades antes de calcular distância. Coordenadas em graus não são metros; escolheria projeção ou função geográfica conforme extensão e precisão esperadas pelo produto.

Termo para lembrar: Coordinate system | Base técnica: referência 1.

02. Índice espacial responde geometria exata?

Usaria índice para reduzir candidatos e predicado adequado para a confirmação geométrica. Bounding boxes podem incluir falsos candidatos; o plano e o refinamento precisam ser observados antes de prometer precisão e desempenho.

Termo para lembrar: Spatial index | Base técnica: referência 2.

03. Como achar terminais a dois quilômetros?

Para um raio em metros, escolheria função e representação que respeitem essa unidade, testando pontos conhecidos. Não converteria latitude por uma constante universal sem avaliar erro e alcance geográfico da aplicação.

Termo para lembrar: Radius query | Base técnica: referência 1.

04. Mais próximo no plano é sempre geográfico?

Definiria a métrica de proximidade e refinaria candidatos quando a aproximação do índice não basta. A ordem percebida depende da distância relevante ao produto, que pode até ser tempo de deslocamento, não linha reta.

Termo para lembrar: Nearest neighbors | Base técnica: referência 2.

05. Usuário perto da borda perde vizinhos?

Consultaria células vizinhas necessárias e aplicaria distância final. Uma partição espacial é uma ferramenta de busca, não uma parede física; testes de borda impedem perder um candidato que está muito próximo.

Termo para lembrar: Cell boundaries | Base técnica: referência 1.

06. Centro urbano concentra consultas: e agora?

Mediria densidade e carga por região, ajustando resolução ou distribuição quando hotspots dominam. Partições uniformes no mapa não têm volume uniforme; a escolha precisa considerar cidades, horários e padrões de consulta.

Termo para lembrar: Dense hotspot | Base técnica: referência 2.

07. Posição de ontem serve para disponibilidade?

Persistiria instante e precisão da localização, aplicando validade por finalidade. Posição antiga pode ser útil como histórico, mas não deve parecer disponibilidade atual de um recurso ou prova presente de localização.

Termo para lembrar: Location freshness | Base técnica: referência 1.

08. Posso guardar trajetória indefinidamente?

Minimizaria precisão, frequência e retenção conforme finalidade, com acesso e auditoria apropriados. Trajetórias revelam comportamento pessoal; índice espacial e analytics precisam das mesmas decisões de governança que a origem.

Termo para lembrar: Privacy | Base técnica: referência 2.

09. GPS prova que cliente está numa agência?

Trataria GPS como sinal sujeito a erro e manipulação, combinado com autenticação e política de risco. Geofencing não substitui identidade financeira; a consequência da decisão determina quanta evidência adicional é necessária.

Termo para lembrar: Geofencing | Base técnica: referência 1.

10. O que muda na linha de data?

Usaria funções e representações adequadas para áreas globais, incluindo testes de antimeridiano e polos. Normalização simples de longitude pode quebrar filtros; os casos extremos precisam ser demonstrados, não assumidos.

Termo para lembrar: Global geometry | Base técnica: referência 2.

11. Geohash de prefixo igual garante proximidade suficiente?

Não. Células aproximam espaço e possuem bordas; eu buscaria vizinhas e refinaria por distância real. O prefixo serve para reduzir candidatos, não para substituir toda validação geométrica.

Termo para lembrar: Spatial approximation | Base técnica: referência 1.

12. Qual diferença entre geometry e geography no PostGIS?

Eu verificaria sistema de referência e unidades das operações. Geography atende cálculos geográficos específicos, enquanto geometry depende da projeção; escolheria pela precisão e extensão espacial do problema, medindo custo.

Termo para lembrar: Spatial type | Base técnica: referência 1.

13. Como consultar dentro de um polígono complexo?

Usaria índice para reduzir candidatos e predicado geométrico apropriado, validando fronteiras e geometria. O contrato precisa definir se pontos na borda contam como dentro, para evitar decisões inconsistentes.

Termo para lembrar: Boundary semantics | Base técnica: referência 1.

14. Posso calcular distância usando diferença simples de latitude?

Isso pode distorcer resultados e ignora geometria terrestre. Eu escolheria projeção ou função adequada, testando diferentes latitudes e extensões; uma aproximação só entra com erro tolerado explicitamente.

Termo para lembrar: Distance model | Base técnica: referência 1.

15. Como associar um ponto a regiões de atendimento sobrepostas?

Eu definiria se a resposta admite múltiplas regiões ou exige precedência, incluindo a regra para pontos na fronteira. O índice encontra candidatos, mas a escolha final segue geometria e política de negócio; proximidade sozinha não resolve a ambiguidade entre áreas elegíveis.

Termo para lembrar: Overlapping spatial regions

16. Como representar precisão ruim do GPS?

Guardaria estimativa de precisão e instante, usando esses dados na decisão. Eu não mostraria uma coordenada como posição exata quando o dispositivo fornece grande incerteza, especialmente em geofencing sensível.

Termo para lembrar: Position uncertainty | Base técnica: referência 1.

17. Como impedir que uma importação use o sistema de coordenadas errado?

Eu exigiria identificação do sistema de referência e validaria faixas e pontos conhecidos antes de transformar coordenadas. Apenas trocar o identificador do sistema não reprojeta os valores; uma carga incorreta deve ficar isolada para não deslocar silenciosamente os resultados da busca.

Termo para lembrar: Coordinate reference validation

18. Como atualizar fronteiras cartográficas sem misturar versões na mesma consulta?

Eu prepararia um conjunto versionado, validaria geometria e índices e fixaria a versão durante cada consulta ou paginação. O corte passa a apontar para o conjunto novo; manter a versão anterior por uma janela permite recuperação e interpretação de resultados históricos.

Termo para lembrar: Versioned geospatial dataset

19. Como atualizar pontos móveis em alta frequência?

Definiria taxa necessária, descartaria atualizações obsoletas por versão e controlaria índice. A frequência máxima do sensor não precisa ser a frequência de persistência; mediria benefício e custo real.

Termo para lembrar: Position ingestion | Base técnica: referência 1.

20. Evento antigo pode sobrescrever posição nova?

Eu impediria com sequência ou versão confiável da origem, considerando troca de dispositivo. Timestamp isolado pode estar errado; a regra de atualização precisa ser explícita para lidar com atraso.

Termo para lembrar: Location ordering | Base técnica: referência 1.

21. Como buscar k vizinhos e filtrar elegibilidade?

Eu recuperaria candidatos suficientes e aplicaria filtros autorizados, repetindo expansão quando necessário. Buscar exatamente k antes de excluir indisponíveis pode retornar menos opções apesar de existirem alternativas próximas.

Termo para lembrar: Filtered nearest neighbors | Base técnica: referência 1.

22. Qual teste cobre antimeridiano e polos?

Usaria conjuntos de referência nessas regiões e compararia distâncias e pertencimento esperados. Casos comuns numa cidade não validam comportamento global; erros de normalização de longitude podem criar resultados absurdos.

Termo para lembrar: Global edge cases | Base técnica: referência 1.

23. Como cachear busca por proximidade?

Agruparia região e parâmetros com cuidado, mantendo filtros de acesso e validade. Precisão do cache determina o erro espacial; posição aproximada não pode servir a uma decisão que exige distância rigorosa.

Termo para lembrar: Spatial cache | Base técnica: referência 1.

24. O que acontece quando um terminal fecha temporariamente?

Disponibilidade precisa ser combinada com localização e freshness. Eu atualizaria o estado operacional e invalidaria projeções conforme prazo, evitando recomendar o ponto mais próximo que já não atende.

Termo para lembrar: Dynamic availability | Base técnica: referência 1.

25. Como paginar resultados espaciais com estabilidade?

Definiria ordenação com distância e desempate, além de contexto da consulta. Mudança de posição ou disponibilidade pode alterar páginas; escolheria snapshot ou deduplicação conforme a experiência contratada.

Termo para lembrar: Spatial pagination | Base técnica: referência 1.

26. Índice espacial garante consulta rápida para qualquer raio?

Não. Um raio enorme pode selecionar quase tudo e reduzir benefício do índice. Eu limitaria consultas caras quando necessário e analisaria plano, candidatos e custo por amplitude espacial.

Termo para lembrar: Selectivity | Base técnica: referência 1.

27. Como importar mapas ou polígonos inválidos?

Validaria formato, sistema de referência e geometria, preservando origem. Corrigir automaticamente pode alterar significado; eu registraria transformações e colocaria casos ambíguos em revisão antes de usar em decisões.

Termo para lembrar: Geometry validation | Base técnica: referência 1.

28. Quando usar H3 em vez de PostGIS?

Avaliaria agregação por células e escala distribuída versus precisão relacional e predicados geométricos. H3 organiza espaço de outra forma, mas ainda exige tratamento de bordas e refinamento para certas consultas.

Termo para lembrar: Spatial alternatives | Base técnica: referência 1.

29. Como limitar consultas exploratórias que rastreiam pessoas?

Aplicaria autorização, minimização, limites e auditoria por finalidade. A API não deve permitir inferir trajetórias privadas por pequenas variações repetidas de consulta, mesmo quando cada resposta parece limitada.

Termo para lembrar: Inference protection | Base técnica: referência 1.

30. Qual métrica prova qualidade da busca geográfica?

Mediria erro espacial, cobertura de candidatos elegíveis, freshness e latência. Retornar rápido o local errado não cumpre o objetivo; validaria com pontos conhecidos e cenários representativos da área atendida.

Termo para lembrar: Spatial quality | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Geography versus geometry projetada

Ganho. Facilita determinadas operações geográficas globais

Custo e motivo. Pode ter custo e recursos diferentes

Como decidir. Quando extensão e unidades justificam o tipo

02. Índice mais refinamento versus scan exato

Ganho. Reduz candidatos examinados

Custo e motivo. Mantém custo de validação final

Como decidir. Quando seletividade espacial é significativa

03. Distância exata versus aproximação

Ganho. Melhora precisão da decisão

Custo e motivo. Consome mais processamento

Como decidir. Quando erro espacial altera resultado relevante

04. Linha reta versus tempo de trajeto

Ganho. Simplifica cálculo e dados necessários

Custo e motivo. Pode representar mal acessibilidade real

Como decidir. Quando proximidade geométrica atende o produto

05. Células vizinhas versus célula única

Ganho. Evita perdas nas bordas

Custo e motivo. Aumenta candidatos e consultas

Como decidir. Quando raio cruza limites da partição

06. Resolução fina versus grossa

Ganho. Reduz candidatos por célula

Custo e motivo. Aumenta metadados e movimentação entre células

Como decidir. Quando densidade justifica granularidade

07. Atualização frequente versus periódica

Ganho. Melhora freshness da posição

Custo e motivo. Aumenta rede, bateria e escrita

Como decidir. Quando movimento rápido importa à decisão

08. Precisão reduzida versus coordenada detalhada

Ganho. Diminui exposição de localização

Custo e motivo. Limita consultas que exigem exatidão

Como decidir. Quando finalidade não precisa de posição precisa

09. GPS combinado versus sinal único

Ganho. Melhora avaliação de confiança

Custo e motivo. Adiciona fricção e integração de sinais

Como decidir. Quando ação possui risco elevado

10. Cobertura global versus modelo local

Ganho. Suporta regiões e bordas complexas

Custo e motivo. Exige mais testes e cuidado geodésico

Como decidir. Quando produto realmente opera globalmente

Duas alternativas ao caminho proposto

1. PostGIS

Quando faz sentido. Consultas relacionais e precisão espacial importam

Como implementar. Usar índices e funções por sistema de referência

Limites. Operação do banco e consultas exigem cuidado

2. Células espaciais

Quando faz sentido. Grande volume e aproximação é aceitável

Como implementar. Buscar células vizinhas e refinar candidatos

Limites. Bordas e densidade precisam de tratamento

Referências e alcance da evidência

1. PostGIS: índices espaciais

Filtragem espacial por índice e refinamento. As respostas e parâmetros de aplicação são propostas autorais.

https://postgis.net/workshops/postgis-intro/indexing.html

2. PostGIS: ST_DWithin

Distância por geometry ou geography e unidades. As respostas e parâmetros de aplicação são propostas autorais.

https://postgis.net/docs/ST_DWithin.html

Continue em: T103 · T105

T105 / K. Famílias de exercícios

UUID, Snowflake e encurtador de URLs

Identificadores precisam de unicidade, distribuição, ordenação e segurança com limites explícitos. Arquitetura de treino; não descreve implementação interna do Nubank.

Quando usar. Chaves de operações, links públicos e geração distribuída.

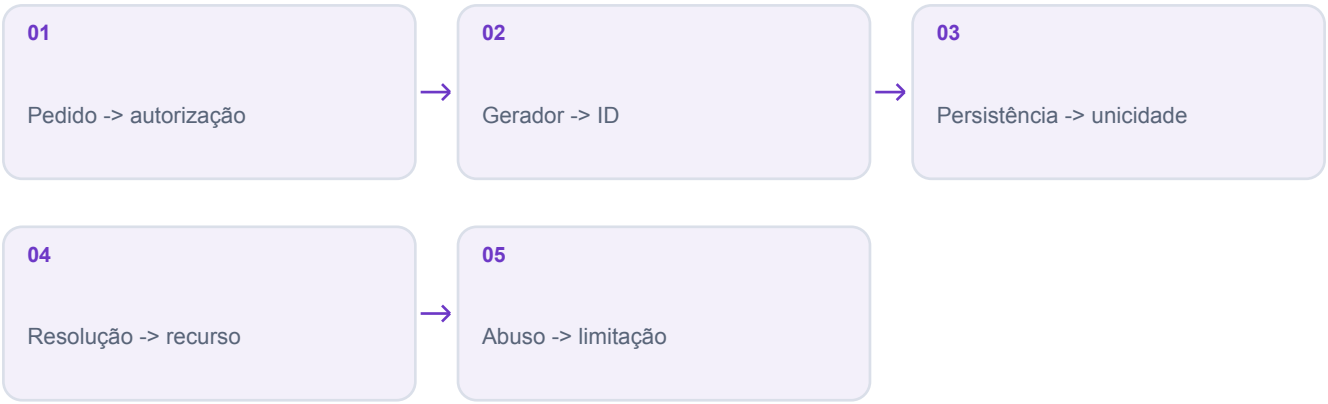
Quando evitar ou limitar. Usar unicidade como autorização ou timestamp como ordem causal global.

Uma abertura possível

Eu começaria por comparar aleatoriedade e localidade temporal. Depois explicaria definir tratamento de regressão e ownership, preservando as restrições do caso.

Peças e tecnologias. UUIDv4 · UUIDv7 · Snowflake · Base62 · Unique index

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Comparar aleatoriedade e localidade temporal; validar inserções e busca sob volume.

2. Definir tratamento de regressão e ownership; validar rollback de relógio no gerador.

3. Atribuir identificadores com exclusividade verificável; validar reinício e atribuição concorrente.

4. Verificar unicidade na persistência autoritativa; validar injeção deliberada de colisão.

5. Separar representação de imprevisibilidade; validar tentativas sequenciais de resolução.

30 perguntas para sustentar o desenho

01. Qual UUID você escolheria para índices?

Compararia propriedades de versões UUID com padrões de acesso e exposição. Localidade temporal pode ajudar índices, mas não define causalidade; escolheria biblioteca adequada e manteria proteção de unicidade na persistência.

Termo para lembrar: UUID version | Base técnica: referência 1.

02. Relógio voltou: posso continuar gerando?

O gerador precisa de política explícita para regressão do relógio, sem reutilizar combinações. Eu testaria saltos e reinícios, avaliando espera ou rejeição controlada conforme a implementação e a disponibilidade exigida.

Termo para lembrar: Snowflake clock | Base técnica: referência 2.

03. Duas instâncias podem compartilhar worker ID?

Atribuiria worker ID com exclusividade verificável durante todo seu uso, incluindo clones e retomadas. Endereço IP ou configuração copiada pode não garantir isso; a operação do gerador faz parte da prova de unicidade.

Termo para lembrar: Worker identity | Base técnica: referência 1.

04. Probabilidade baixa dispensa constraint?

Manteria constraint ou mecanismo autoritativo apropriado, mesmo com probabilidade muito baixa de colisão. Testaria colisões injetadas e reação segura, porque um evento raro não pode associar silenciosamente recursos de clientes distintos.

Termo para lembrar: Collision | Base técnica: referência 2.

05. Codificar ID em Base62 torna secreto?

Base62 é representação compacta, não proteção de acesso. Eu avaliaria enumeração e usaria autorização no destino; IDs difíceis de adivinhar podem reduzir exploração, mas não devem funcionar como permissão implícita.

Termo para lembrar: Base62 | Base técnica: referência 1.

06. Usuário quer o mesmo apelido que outro?

Reservaria alias atomicamente no namespace, validando caracteres e nomes protegidos. Consulta prévia não resolve concorrência; o resultado da gravação decide o vencedor e a API retorna conflito compreensível ao outro solicitante.

Termo para lembrar: Short alias | Base técnica: referência 2.

07. Link mudou e CDN aponta destino velho?

Escolheria cache e status de redirecionamento coerentes com a mutabilidade do destino. Mudança de URL precisa considerar clientes e CDN; uma resposta permanente pode continuar sendo usada depois da alteração no serviço.

Termo para lembrar: Redirect cache | Base técnica: referência 1.

08. Expirar link apaga informação no cache?

Separaria validade do link e retenção do registro, bloqueando resolução conforme o contrato. Cache não pode ampliar uma autorização que precisa expirar; testaria o limite e definiria se revogação imediata é necessária.

Termo para lembrar: Expiry | Base técnica: referência 2.

09. Encurtador vira distribuidor de phishing?

Validaria esquemas e destinos, com processo de abuso e retirada. O serviço precisa reagir a links maliciosos descobertos depois, considerando caches e atualizações, sem virar um mecanismo irrestrito de encaminhamento para ataques.

Termo para lembrar: Malicious destination | Base técnica: referência 1.

10. Retry deve criar outro link para mesma intenção?

Preservaria a intenção de criação quando a mesma solicitação é repetida, retornando resultado existente autorizado. Uma nova URL ou configuração pode representar outra operação; conteúdo e chave precisam ser verificados conjuntamente.

Termo para lembrar: Idempotent create | Base técnica: referência 2.

11. UUIDv7 prova ordem causal entre eventos?

Não. Sua organização temporal ajuda certas propriedades, mas não estabelece causalidade global. Eu manteria versão ou relação explícita quando uma transição depende de outra, especialmente entre relógios e serviços diferentes.

Termo para lembrar: Causal ordering | Base técnica: referência 1.

12. Como lidar com esgotamento de sequência num milissegundo?

O gerador precisa de política definida, como aguardar ou outra estratégia válida, sem reutilizar combinações. Eu mediria limite por nó e trataria regressão de relógio separadamente para preservar unicidade.

Termo para lembrar: Sequence exhaustion | Base técnica: referência 1.

13. Worker ID pode ser simplesmente endereço IP?

Eu verificaria estabilidade e exclusividade; IP pode ser reutilizado ou compartilhar contexto. A geração exige ownership controlado e tratamento de reinícios, não uma suposição sobre topologia de rede.

Termo para lembrar: Generator ownership | Base técnica: referência 1.

14. Por que IDs aleatórios podem afetar B-tree?

Inserções se espalham e podem aumentar trabalho de páginas em certas cargas. Eu compararia versões temporais ou sequências com requisitos de distribuição e exposição, usando benchmark representativo e não regra absoluta.

Termo para lembrar: Index locality | Base técnica: referência 1.

15. Encurtador precisa armazenar URL original normalizada?

Eu preservaria o destino validado e aplicaria normalização cuidadosamente, porque alterações podem mudar significado. Também limitaria tamanho e esquemas permitidos, evitando transformar sanitização em redirecionamento incorreto.

Termo para lembrar: URL validation | Base técnica: referência 1.

16. Qual status HTTP usar para redirect mutável?

Escolheria semântica e cache coerentes com a mutabilidade, verificando comportamento dos clientes. Redirecionamento permanente pode continuar cacheado após alteração; a decisão precisa refletir quanto o destino pode mudar.

Termo para lembrar: Redirect semantics | Base técnica: referência 1.

17. Como criar aliases quando a região responsável pela escrita fica indisponível?

Eu preservaria uma autoridade de unicidade para o namespace e suspenderia novas criações quando não puder garanti-la. Redirecionamentos existentes podem continuar conforme sua política de leitura; aceitar o mesmo alias em regiões isoladas cria um conflito que um ID único interno não resolve.

Termo para lembrar: Identifier authority during partition

18. O link público pode revelar quantidade de clientes?

Sequências previsíveis podem expor padrões e facilitar enumeração. Eu avaliaria IDs opacos para superfície pública, mantendo autorização e rate limit, porque imprevisibilidade reduz exploração mas não concede acesso seguro.

Termo para lembrar: Enumeration resistance | Base técnica: referência 1.

19. Como expirar link sem perder auditoria?

Separaria validade de resolução e retenção dos metadados necessários. O registro pode permanecer para investigação enquanto acesso é recusado; limpeza precisa seguir política e não reativar alias inadvertidamente.

Termo para lembrar: Link lifecycle | Base técnica: referência 1.

20. Alias excluído pode ser reutilizado imediatamente?

Eu avaliaria links antigos, caches e risco de confusão. Reutilização pode direcionar referências existentes para outra pessoa; quarentena ou proibição de reutilização pode ser necessária conforme o produto.

Termo para lembrar: Namespace reuse | Base técnica: referência 1.

21. Como fazer analytics sem atrasar redirecionamento?

Registraria eventos de acesso de forma desacoplada conforme a perda tolerada, com proteção de dados. O caminho de resolução permanece curto, e métricas aproximadas não devem ser confundidas com cobrança exata.

Termo para lembrar: Redirect analytics | Base técnica: referência 1.

22. Como lidar com destino malicioso descoberto depois?

Manteria mecanismo de bloqueio, revisão e invalidação compatível com caches. O sistema precisa retirar resoluções perigosas rapidamente, preservando evidência e evitando que um redirect permanente torne a resposta incontrolável.

Termo para lembrar: Abuse response | Base técnica: referência 1.

23. UUID sozinho serve como segredo de recuperação de senha?

Eu não confundiria identificador com credencial. Tokens de segurança exigem geração, validade, finalidade e uso únicos apropriados, além de proteção no armazenamento e fluxo de revogação definido.

Termo para lembrar: Identifier versus secret | Base técnica: referência 1.

24. Como representar ID de 64 bits no JavaScript?

Usaria formato de intercâmbio que preserve precisão, como string quando necessário. Testaria roundtrip entre serviços e clientes, evitando que conversão numérica altere identidade e associe operações diferentes.

Termo para lembrar: Integer precision | Base técnica: referência 1.

25. Gerar ID antes do commit traz algum problema?

Pode deixar lacunas, o que geralmente é aceitável, mas precisa de contrato. Eu não usaria continuidade da sequência como prova de completude de operações; confirmação e histórico são evidências separadas.

Termo para lembrar: Sequence gaps | Base técnica: referência 1.

26. Como migrar formato de ID sem quebrar clientes?

Aceitaria formatos necessários durante transição, preservando mapeamentos e contratos públicos. Não converteria silenciosamente valores com perda; testaria referências antigas, índices e consumidores antes de descontinuar suporte.

Termo para lembrar: ID migration | Base técnica: referência 1.

27. Como impedir criação massiva de links abusivos?

Aplicaria identidade, quotas e detecção de padrões, com limites por custo. O caminho de criação e resolução possui riscos diferentes; eu protegeria ambos sem exigir armazenamento infinito de solicitações maliciosas.

Termo para lembrar: Abuse admission | Base técnica: referência 1.

28. Quando usar UUIDv4 em vez de ordenação temporal?

Quando geração descentralizada e aleatoriedade forem mais importantes que localidade temporal. Eu avaliaria custo de índice e exposição de tempo, mantendo constraint e biblioteca confiável em qualquer escolha.

Termo para lembrar: UUID selection | Base técnica: referência 1.

29. Como testar gerador distribuído?

Simularia concorrência, reinício, relógio regressivo, esgotamento e duplicação de worker. Verificaria unicidade nas condições suportadas e comportamento explícito fora delas, sem tentar provar confiabilidade apenas gerando uma amostra aleatória.

Termo para lembrar: Generator fault testing | Base técnica: referência 1.

30. Qual é a pergunta principal antes da tecnologia?

Eu perguntaria quais propriedades são necessárias: unicidade em qual escopo, ordenação, previsibilidade, tamanho e disponibilidade. Depois escolheria mecanismo e limites, evitando adotar Snowflake apenas por ser conhecido em entrevistas.

Termo para lembrar: ID requirements | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. UUID temporal versus aleatório

Ganho. Pode melhorar localidade de inserção

Custo e motivo. Expõe componente temporal e não prova causalidade

Como decidir. Quando índice se beneficia da ordenação aproximada

02. Gerador coordenado versus local aleatório

Ganho. Oferece propriedades de sequência controladas

Custo e motivo. Depende de ownership e relógio da infraestrutura

Como decidir. Quando formato e ordenação justificam coordenação

03. Worker ID explícito versus identidade inferida

Ganho. Torna exclusividade administrável

Custo e motivo. Exige alocação e recuperação operacional

Como decidir. Quando esquema usa combinação por nó

04. Constraint final versus confiar só na probabilidade

Ganho. Detecta colisão antes de corromper vínculo

Custo e motivo. Adiciona verificação no caminho de persistência

Como decidir. Quando identidade protege dados relevantes

05. Alias opaco versus sequência pública

Ganho. Dificulta enumeração casual

Custo e motivo. Pode aumentar tamanho e dificultar suporte manual

Como decidir. Quando exposição de sequência é indesejada

06. Alias customizado versus automático

Ganho. Melhora legibilidade e marca

Custo e motivo. Cria disputa por namespace e política de nomes

Como decidir. Quando produto valoriza personalização

07. Redirect permanente versus temporário

Ganho. Pode reduzir consultas por cache

Custo e motivo. Torna mudanças de destino menos controláveis

Como decidir. Quando destino é efetivamente estável

08. Expiração curta versus longa

Ganho. Reduz exposição de links temporários

Custo e motivo. Aumenta necessidade de renovação

Como decidir. Quando acesso delegado possui finalidade limitada

09. Validação de destino versus redirect aberto

Ganho. Reduz abuso e risco ao usuário

Custo e motivo. Acrescenta processamento e operação de revisão

Como decidir. Quando serviço público pode distribuir conteúdo malicioso

10. Criação idempotente versus novo alias por pedido

Ganho. Evita custo e duplicação em retries

Custo e motivo. Exige identidade de intenção e retenção

Como decidir. Quando resposta pode se perder após criação

Duas alternativas ao caminho proposto

1. UUID gerado localmente

Quando faz sentido. Coordenação central é indesejada

Como implementar. Usar biblioteca padrão e constraint final

Limites. Tamanho e propriedades variam por versão

2. Sequência do banco

Quando faz sentido. Um writer atende o volume

Como implementar. Gerar chave sob autoridade transacional

Limites. Expõe ordenação e depende da fronteira

Referências e alcance da evidência

1. RFC 9562: UUID

Formatos e propriedades dos identificadores UUID. As respostas e parâmetros de aplicação são propostas autorais.

https://www.rfc-editor.org/rfc/rfc9562.html

2. Twitter: Snowflake histórico

Referência original de geração distribuída de identificadores; repositório arquivado. As respostas e parâmetros de aplicação são propostas autorais.

https://github.com/twitter-archive/snowflake

Continue em: T104 · T106

T106 / K. Famílias de exercícios

Reservas, estoque e concorrência

Reservas protegem recursos escassos enquanto uma decisão ainda está em andamento. Arquitetura de treino; não descreve implementação interna do Nubank.

Quando usar. Ingressos, estoque, limites e disponibilidade com concorrência.

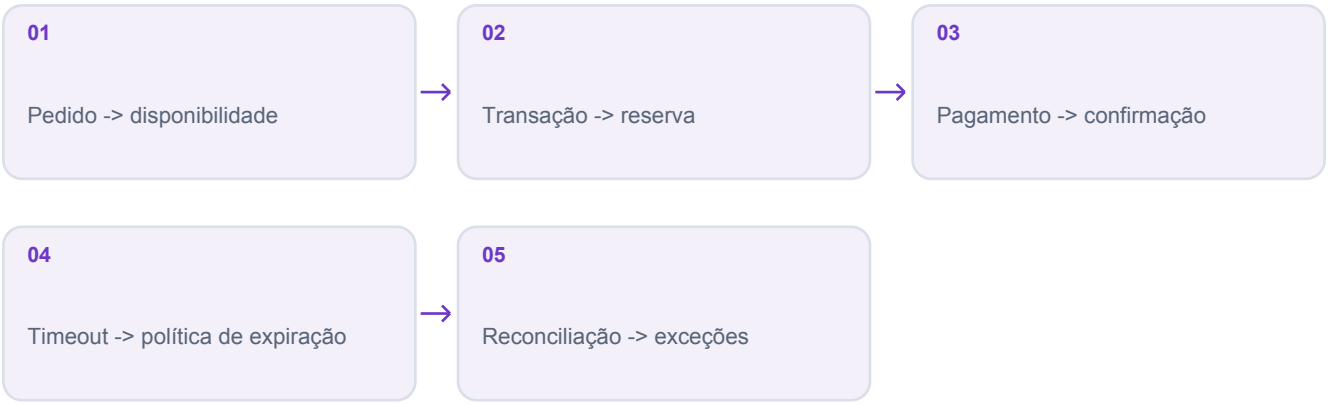
Quando evitar ou limitar. Afirmar que cache eventual evita overselling sem uma fronteira autoritativa.

Uma abertura possível

Eu começaria por atualizar estoque com condição atômica. Depois explicaria serializar expiração e confirmação por versão, preservando as restrições do caso.

Peças e tecnologias. PostgreSQL · CAS · Serializable · Reservation table · Workers

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Atualizar estoque com condição atômica; validar dois clientes disputando a unidade.

2. Serializar expiração e confirmação por versão; validar retorno após limite temporal.

3. Definir atomicidade necessária entre recursos; validar falta de um item no conjunto.

4. Ordenar aquisição e tratar abortos recuperáveis; validar ordens opostas sobre mesmos itens.

5. Limitar admissão antes da fronteira serializada; validar pico sobre uma única unidade.

30 perguntas para sustentar o desenho

01. Última unidade recebe dois pedidos: quem ganha?

A disputa pela última unidade precisa de alteração atômica com condição ou isolamento adequado. Eu testaria dois pedidos simultâneos; verificar estoque antes e atualizar depois, sem proteção, permite que ambos observem a mesma disponibilidade.

Termo para lembrar: Overselling | Base técnica: referência 1.

02. TTL liberou estoque e pagamento chegou depois?

Expiração e confirmação são transições concorrentes sobre a mesma reserva versionada. Definiria precedência e tratamento de retorno tardio, impedindo que dois caminhos liberem e consumam a mesma capacidade independentemente.

Termo para lembrar: Reservation TTL | Base técnica: referência 2.

03. Carrinho exige cinco itens juntos?

Esclareceria se o carrinho precisa de tudo ou admite parcial. Depois escolheria fronteira transacional ou coordenação com reservas e compensações, sem prometer atomicidade global entre estoques independentes apenas por usar uma fila.

Termo para lembrar: Multi item cart | Base técnica: referência 1.

04. Dois carrinhos travam produtos em ordem diferente?

Adquiriria locks numa ordem determinística e manteria a transação curta, com tratamento de aborto recuperável. Isso reduz deadlocks, mas não elimina necessidade de observação e testes com outras operações que compartilham recursos.

Termo para lembrar: Deadlock | Base técnica: referência 2.

05. Produto popular concentra todas escritas?

Limitaria admissão e trabalho concorrente antes da chave quente, medindo a capacidade real da fronteira serializada. Mais retries podem piorar disputa; só repartiria estoque se o modelo preservar conservação e compromissos.

Termo para lembrar: Hot product | Base técnica: referência 1.

06. Cliente atualiza página: reserva novamente?

Associaria reenvios à mesma intenção e verificaria conteúdo da solicitação. Duas abas ou refresh não devem consumir novas reservas; alteração de quantidade exige comando autorizado e validação própria do recurso disponível.

Termo para lembrar: Idempotency | Base técnica: referência 2.

07. Pagamento deu timeout: posso vender a unidade?

Pagamento com resultado incerto não libera automaticamente o compromisso. Aplicaria política de validade e reconciliação, evitando vender a mesma unidade enquanto uma confirmação remota ainda pode demonstrar que o primeiro cliente pagou.

Termo para lembrar: Payment unknown | Base técnica: referência 1.

08. Quantidade física divergiu do sistema?

Registraria ajuste de inventário com origem, motivo e efeito sobre compromissos, sem apagar reservas para fechar número. Divergência física exige decisão operacional e comunicação quando afeta pedidos já aceitos.

Termo para lembrar: Inventory reconciliation | Base técnica: referência 2.

09. Bots podem reservar tudo sem comprar?

Definiria limites por identidade e duração total de reserva, incluindo renovação e sinais de abuso. A política precisa preservar acesso legítimo e justiça; capacidade temporariamente bloqueada também é um recurso escasso.

Termo para lembrar: Fairness | Base técnica: referência 1.

10. Dividir estoque em shards resolve qualquer disputa?

Particionaria por autoridade de recurso quando as operações permitem. Pedidos que atravessam lojas ou shards precisam de semântica explícita de compromisso; aumentar paralelismo não torna essas alterações automaticamente atômicas.

Termo para lembrar: Sharding | Base técnica: referência 2.

11. Uma consulta de estoque seguida de update é suficiente?

Não quando concorrentes podem observar o mesmo valor. Eu faria atualização condicional, lock apropriado ou isolamento que preserve o invariante, testando disputa pela última unidade em vez de confiar no caminho sequencial.

Termo para lembrar: Atomic stock decrement | Base técnica: referência 1.

12. CAS com versão elimina todas as anomalias?

Protege a condição representada naquela atualização, não invariantes arbitrários entre linhas. Eu verificaria o conjunto de recursos envolvidos e escolheria transação ou coordenação adequada quando o limite atravessa várias entidades.

Termo para lembrar: CAS boundary | Base técnica: referência 1.

13. Quando serializable ajuda na reserva?

Quando preciso evitar anomalias não cobertas por atualizações locais simples. Eu trataria abortos de serialização com retry da transação inteira e mediria contenção, sem prometer que isolamento forte elimina custo.

Termo para lembrar: Serializable retry | Base técnica: referência 1.

14. Posso manter lock enquanto usuário paga?

Não manteria uma transação aberta por minutos. Criaria reserva durável com prazo e transições explícitas, liberando locks curtos; pagamento externo e expiração são coordenados por estado, não por conexão presa.

Termo para lembrar: Durable reservation | Base técnica: referência 1.

15. Como oferecer uma lista de espera sem prometer estoque inexistente?

Eu separaria posição de espera de reserva efetiva, definindo prioridade e validade do convite. Quando houver capacidade, uma transição atômica concede a reserva a um elegível; estar na fila não autoriza pagamento nem comprova disponibilidade antes dessa confirmação.

Termo para lembrar: Waitlist admission

16. O cliente pode aumentar quantidade numa reserva existente?

Somente por comando validado contra disponibilidade e versão atuais. Eu registraria alteração e identidade, garantindo que o incremento concorrente não use capacidade já atribuída a outra pessoa.

Termo para lembrar: Reservation amendment | Base técnica: referência 1.

17. Como reservar uma sala por horário, em vez de uma unidade de estoque?

Eu representaria início e fim com convenção explícita de borda e impediria sobreposição por recurso dentro de uma garantia transacional. Somar quantidades não basta nesse domínio; cancelamentos e alterações de período precisam respeitar a mesma regra de exclusão entre intervalos.

Termo para lembrar: Interval exclusion

18. Pagamento falhou definitivamente: como liberar?

Aplicaria transição idempotente da reserva ativa para liberada, atualizando capacidade coerentemente. A evidência de rejeição precisa corresponder à tentativa correta, evitando que resposta antiga desfaça uma confirmação posterior.

Termo para lembrar: Release transition | Base técnica: referência 1.

19. Como impedir que um produto devolvido seja vendido antes da inspeção?

Eu modelaria estoques disponíveis, reservados e em quarentena separadamente, com transições auditáveis. Receber uma devolução aumenta o físico, mas não necessariamente o vendável; somente a inspeção autorizada libera a unidade para que novas reservas possam consumi-la.

Termo para lembrar: Quarantined inventory

20. Como evitar reserva infinita por renovação automática?

Definiria limite de duração e política por identidade, além de sinais de abuso. Renovação compete por capacidade e precisa respeitar usuários aguardando, não apenas prolongar toda sessão indefinidamente.

Termo para lembrar: Reservation fairness | Base técnica: referência 1.

21. Uma fila por produto garante exclusividade?

Ajuda a serializar processamento dentro do ownership correto, mas falhas e consumidores antigos ainda importam. Eu manteria persistência e proteção de efeito, verificando recuperação e transferência de responsabilidade.

Termo para lembrar: Single-writer model | Base técnica: referência 1.

22. Como particionar estoque de várias lojas?

Atribuiria autoridade por loja ou recurso, conforme operações reais. Transferência entre lojas e pedidos multiorigem exigem fluxo explícito; dividir tabelas não torna compromissos distribuídos atomicamente disponíveis.

Termo para lembrar: Inventory ownership | Base técnica: referência 1.

23. Quando escrow de capacidade é útil?

Quando posso distribuir sub-limites sem ultrapassar o total e aceitar recursos temporariamente ociosos em partições. Eu definiria transferência e recuperação desses limites, preservando conservação durante falhas e rebalanceamento.

Termo para lembrar: Escrow allocation | Base técnica: referência 1.

24. Cache pode acelerar consulta de disponibilidade?

Sim, para exibição com freshness definido. A reserva final valida no sistema autoritativo, e a interface explica eventual indisponibilidade; cache de leitura não deve conceder compromisso financeiro ou físico sozinho.

Termo para lembrar: Availability projection | Base técnica: referência 1.

25. Como testar expiração sem esperar minutos?

Usaria relógio controlável e sequências concorrentes de confirmar, cancelar e expirar. Verificaria conservação de capacidade e transições válidas após cada ação, incluindo retorno tardio de pagamento.

Termo para lembrar: Reservation state testing | Base técnica: referência 1.

26. O que significa reserva confirmada em dois sistemas?

Eu definiria qual sistema assume a obrigação e como o outro reconhece. Sem transação distribuída ou protocolo equivalente, há estados intermediários; o desenho precisa recuperar e compensar conforme o contrato.

Termo para lembrar: Distributed commitment | Base técnica: referência 1.

27. Pode resolver conflito com last-write-wins?

Para quantidade escassa, isso pode apagar uma decisão válida e causar overselling. Eu usaria condição ou serialização adequada, deixando last-write-wins para dados cuja semântica realmente aceite substituir informação concorrente.

Termo para lembrar: Conflict semantics | Base técnica: referência 1.

28. Como observar gargalo sem aumentar retries?

Mediria espera de lock, abortos, concorrência por chave e capacidade útil. Limitaria admissão e usaria backoff quando apropriado, evitando que disputa pela mesma unidade gere tempestade de tentativas inúteis.

Termo para lembrar: Contention observability | Base técnica: referência 1.

29. Como recuperar reservas após restart?

Carregaria estados duráveis e retomaria expirações ou reconciliação por identidade, sem recriar compromissos. A ausência de memória local não significa que uma reserva deixou de existir ou que pode ser emitida novamente.

Termo para lembrar: Reservation recovery | Base técnica: referência 1.

30. Como explicar escolha simples em nível Staff?

Eu começaria com transação curta e invariante claro, demonstraria concorrência e falhas, e apresentaria o limite medido para evoluir. Sofisticação só entra quando resolve um problema real sem enfraquecer a garantia.

Termo para lembrar: Simplicity with guarantees | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Update condicional versus leitura seguida de escrita

Ganho. Protege disponibilidade numa operação atômica

Custo e motivo. Pode concentrar contenção na linha

Como decidir. Quando a decisão depende do mesmo contador

02. Reserva com TTL versus bloqueio sem prazo

Ganho. Recupera recursos abandonados

Custo e motivo. Exige resolver corrida com confirmação tardia

Como decidir. Quando usuário pode desistir sem avisar

03. Carrinho atômico versus parcial

Ganho. Oferece compromisso tudo-ou-nada

Custo e motivo. Exige coordenação de múltiplos recursos

Como decidir. Quando produto não aceita entrega incompleta

04. Lock pessimista versus CAS

Ganho. Evita repetição sob certos conflitos

Custo e motivo. Mantém espera e risco de deadlock

Como decidir. Quando transações curtas têm alta contenção

05. Admissão limitada versus concorrência irrestrita

Ganho. Protege recurso quente e reduz tentativas inúteis

Custo e motivo. Pode rejeitar ou enfileirar usuários

Como decidir. Quando capacidade por chave é finita

06. Intenção estável versus reserva nova por tentativa

Ganho. Evita consumo duplicado por retry

Custo e motivo. Exige manter resultado e comparar parâmetros

Como decidir. Quando respostas podem se perder

07. Manter reserva incerta versus liberar cedo

Ganho. Reduz risco de vender recurso comprometido

Custo e motivo. Diminui disponibilidade enquanto reconcilia

Como decidir. Quando pagamento pode ter ocorrido

08. Ajuste auditável versus sobrescrever quantidade

Ganho. Permite explicar divergência física

Custo e motivo. Adiciona histórico e processo operacional

Como decidir. Quando inventário precisa de rastreabilidade

09. Limite por usuário versus alocação aberta

Ganho. Reduz monopólio e abuso de reservas

Custo e motivo. Pode restringir clientes legítimos grandes

Como decidir. Quando escassez exige equidade

10. Sharding por loja versus estoque global único

Ganho. Distribui escrita e ownership

Custo e motivo. Complica carrinhos e transferências entre lojas

Como decidir. Quando recursos têm independência operacional

Duas alternativas ao caminho proposto

1. Lock pessimista

Quando faz sentido. Contenção é tolerável e operação curta

Como implementar. Bloquear recurso durante validação local

Limites. Fila e deadlocks exigem disciplina

2. CAS otimista

Quando faz sentido. Conflitos são relativamente raros

Como implementar. Atualizar quantidade com condição e versão

Limites. Hot keys geram retries e frustração

Referências e alcance da evidência

1. PostgreSQL: isolamento

Anomalias de concorrência e níveis de isolamento. As respostas e parâmetros de aplicação são propostas autorais.

https://www.postgresql.org/docs/current/transaction-iso.html

2. PostgreSQL: locking

Bloqueios, conflito e deadlocks. As respostas e parâmetros de aplicação são propostas autorais.

https://www.postgresql.org/docs/current/explicit-locking.html

Continue em: T105 · T107

T107 / K. Famílias de exercícios

Execução distribuída de jobs

Jobs distribuídos precisam de estado durável, identidade e retomada independente do disparador. Arquitetura de treino; não descreve implementação interna do Nubank.

Quando usar. Lotes, cobranças, documentos e processamento periódico.

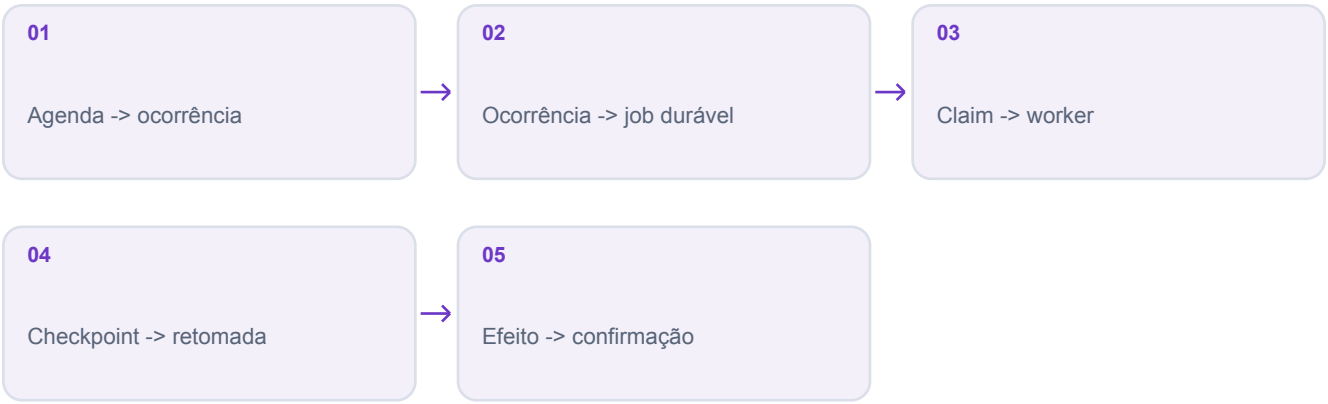
Quando evitar ou limitar. Assumir que líder ou CronJob garante execução única de efeitos externos.

Uma abertura possível

Eu começaria por identificar ocorrência pela agenda e instante lógico. Depois explicaria usar lease com versão de tentativa, preservando as restrições do caso.

Peças e tecnologias. PostgreSQL · Kubernetes CronJob · Temporal · SQS · Fencing

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Identificar ocorrência pela agenda e instante lógico; validar duplicação do evento agendado.

2. Usar lease com versão de tentativa; validar pausa longa seguida de retomada.

3. Vincular checkpoint à confirmação dos itens; validar queda entre processamento e checkpoint.

4. Classificar erro e preservar identidade; validar falhas transitórias e inválidas.

5. Criar partições determinísticas com manifesto; validar união exata das partições.

30 perguntas para sustentar o desenho

01. Scheduler disparou duas vezes: duplico cobrança?

Identificaria cada ocorrência por agenda e instante lógico, com persistência condicional. Disparo repetido retorna ao mesmo trabalho; o scheduler não é a única barreira, pois o efeito externo também precisa de identidade estável.

Termo para lembrar: Occurrence identity | Base técnica: referência 1.

02. Worker travou: como outro assume?

Usaria lease e versão de tentativa para recuperar tarefas abandonadas, validando ownership nas alterações. Um worker pausado pode voltar depois da expiração; proteger o destino exige fencing suportado ou idempotência apropriada.

Termo para lembrar: Lease | Base técnica: referência 2.

03. Posso retomar no último offset salvo?

Alinharia checkpoint à confirmação dos itens conforme a garantia disponível. Avançar posição antes de efeito durável pode perder trabalho; repetir com identidade protegida costuma ser mais seguro do que pular uma incerteza.

Termo para lembrar: Checkpoint | Base técnica: referência 1.

04. Todas falhas merecem nova execução?

Classificaria erros e respeitaria validade, orçamento e contrato do destino. Falha permanente vai para tratamento adequado; timeout após possível efeito pode exigir reconciliação, não simplesmente mais uma execução com novo identificador.

Termo para lembrar: Retries | Base técnica: referência 2.

05. Como dividir um arquivo enorme?

Criaria partições determinísticas com manifesto, cobertura e checkpoints verificáveis. Testaria bordas e sobreposição, garantindo que retomadas preservem identidade dos itens e que um lote maior não dependa de memória proporcional ao total.

Termo para lembrar: Partitioning | Base técnica: referência 1.

06. Job B depende de A: basta horário?

Dependências seriam estado durável, não apenas horários separados no cron. A etapa seguinte verifica conclusão necessária e sua versão, para que atraso ou falha da anterior não produza processamento sobre dados incompletos.

Termo para lembrar: Dependencies | Base técnica: referência 2.

07. Cancelar job desfaz pagamentos já enviados?

Cancelamento interrompe trabalho futuro quando possível e registra o que já foi executado. Efeitos externos confirmados exigem compensação de negócio quando aplicável; mudar um status local não desfaz pagamentos ou notificações.

Termo para lembrar: Cancellation | Base técnica: referência 1.

08. Proibir sobreposição elimina duplicatas?

Política de não sobreposição ajuda operação, mas não substitui idempotência de ocorrência e efeito. Reinício após commit remoto ainda pode repetir trabalho, mesmo quando apenas um job parece ativo no scheduler.

Termo para lembrar: Concurrency policy | Base técnica: referência 2.

09. Um item inválido bloqueia o lote?

Isolaria o item inválido conforme contrato de completude, preservando motivo e possibilidade de revisão. Não descartaria silenciosamente para manter throughput; um lote parcial precisa ser identificado como tal e reconciliável.

Termo para lembrar: Poison job | Base técnica: referência 1.

10. Horário local muda no calendário?

Persistiria timezone e identidade lógica das ocorrências, testando transições do calendário e comportamento de atraso. Intervalo em horas e horário local fixo podem ter semânticas diferentes, então a agenda precisa refletir o negócio.

Termo para lembrar: Scheduling timezone | Base técnica: referência 2.

11. Cron executar no horário prova que trabalho terminou?

Não. Disparo, início, progresso e conclusão são marcos diferentes. Eu persistiria ocorrência e resultado, monitorando prazo do trabalho e capacidade de recuperação, em vez de considerar o log do scheduler como sucesso.

Termo para lembrar: Scheduling versus completion | Base técnica: referência 1.

12. Como deduplicar agenda após eleição de novo líder?

Usaria identidade da ocorrência e inserção condicional na fonte durável. Eleição reduz concorrência do agendador, mas repetição ainda pode ocorrer; workers e efeitos também precisam reconhecer a mesma intenção.

Termo para lembrar: Occurrence deduplication | Base técnica: referência 1.

13. Lease expirada garante que worker antigo parou?

Não. Ele pode estar pausado e retomar. Eu validaria fencing onde suportado e identidade do efeito externo, para que recuperação de ownership não permita duas execuções financeiras independentes.

Termo para lembrar: Stale worker | Base técnica: referência 1.

14. Como escolher duração da lease?

Compararia duração de tarefas, frequência de heartbeat e tempo desejado de recuperação. Lease curta aumenta falsas retomadas; longa atrasa recuperação. Independentemente do valor, correção não depende de ausência de pausas.

Termo para lembrar: Lease tuning | Base técnica: referência 1.

15. Quando heartbeat não deve renovar indefinidamente?

Quando o worker perdeu progresso útil ou ultrapassou validade do job. Eu separaria liveness de progresso, definindo limites e diagnóstico para evitar tarefa presa manter ownership para sempre.

Termo para lembrar: Progress heartbeat | Base técnica: referência 1.

16. Como dividir job sem perder itens na borda?

Usaria intervalos ou partições determinísticas e manifesto verificável, incluindo regras de inclusão. Testaria união e interseção das partes, garantindo cobertura completa e deduplicação de qualquer reexecução.

Termo para lembrar: Partition boundaries | Base técnica: referência 1.

17. Checkpoint antes do efeito é seguro?

Pode perder trabalho se a retomada pular um item ainda não confirmado. Eu alinharia checkpoint e efeitos pela garantia disponível, normalmente aceitando repetição protegida em vez de avançar sem evidência durável.

Termo para lembrar: Checkpoint ordering | Base técnica: referência 1.

18. Como tratar job com efeito remoto UNKNOWN?

Persistiria incerteza e consultaria ou reconciliaria o destino conforme contrato. O scheduler não deve simplesmente reclassificar como pending e criar outra intenção porque a tentativa terminou sem resposta.

Termo para lembrar: Unknown job effect | Base técnica: referência 1.

19. Retry de activity no Temporal dispensa idempotência?

Não. A activity pode ter produzido efeito antes de perder confirmação. Eu preservaria identidade da operação no destino e verificaria política de retry, distinguindo recuperação do workflow de execução única externa.

Termo para lembrar: Activity semantics | Base técnica: referência 1.

20. Como investigar um job cujas etapas acontecem com horas de intervalo?

Eu propagaria identificadores de job, execução e tentativa, relacionando traces de cada etapa sem manter um span aberto por horas. Eventos de estado mostram espera e processamento separadamente; isso permite localizar atrasos e efeitos repetidos sem confundir retry com uma nova intenção.

Termo para lembrar: Asynchronous trace correlation

21. Como impedir job gigante de monopolizar workers?

Dividiria trabalho em unidades retomáveis e aplicaria limites por classe ou tenant. A política de fairness preserva prazos, enquanto checkpoints evitam que preempção exija repetir todo o lote.

Termo para lembrar: Job fairness | Base técnica: referência 1.

22. Fila de jobs precisa guardar payload completo?

Nem sempre; referências a objetos imutáveis podem reduzir tamanho e custo. Eu garantiria autorização, retenção e integridade da referência, para que a tarefa não dependa de um arquivo removido antes da execução.

Termo para lembrar: Payload references | Base técnica: referência 1.

23. Como lidar com mudança de versão do worker?

Manteria compatibilidade de payload e estado ou migração explícita. Jobs antigos precisam de execução suportada; atualizar código não pode transformar silenciosamente significado de parâmetros já aceitos.

Termo para lembrar: Worker compatibility | Base técnica: referência 1.

24. O que fazer com job que sempre mata o processo?

Isolaria tentativa, limitaria repetição e colocaria em quarentena com evidências sanitizadas. O restante da fila continua, e o redrive ocorre após correção, preservando a identidade dos efeitos já possíveis.

Termo para lembrar: Poison job | Base técnica: referência 1.

25. Como controlar quota compartilhada entre workers?

Usaria mecanismo coerente com a unidade do contrato e reserva de capacidade, considerando retries internos. Limite local por processo pode multiplicar consumo quando réplicas aumentam durante um pico.

Termo para lembrar: Global quota | Base técnica: referência 1.

26. Job atrasado deve executar mesmo depois da validade?

Depende do negócio. Eu definiria política de misfire: executar, agrupar, pular com registro ou escalar. O sistema não deve inferir que toda ocorrência atrasada ainda representa uma ação autorizada.

Termo para lembrar: Misfire policy | Base técnica: referência 1.

27. Como observar fila sem apenas contar mensagens?

Mediria idade, prazo restante, custo estimado, tentativas e taxa de conclusão. Mil jobs pequenos e um enorme têm perfis diferentes; a métrica precisa indicar risco de atraso, não só volume.

Termo para lembrar: Queue workload | Base técnica: referência 1.

28. Como reduzir custo com máquinas interrompíveis?

Escolheria tarefas retomáveis com checkpoints e efeitos protegidos, mantendo capacidade estável para prazos críticos. O ganho econômico precisa incluir repetição e tempo de recuperação após interrupções frequentes.

Termo para lembrar: Interruptible workers | Base técnica: referência 1.

29. Qual teste prova recuperação de claim?

Pausaria um worker até perder lease, permitiria outro assumir e depois retomaria o antigo. Verificaria estado, fencing e efeito externo, demonstrando comportamento seguro na sobreposição mais perigosa.

Termo para lembrar: Lease race test | Base técnica: referência 1.

30. Qual solução mínima apresentaria primeiro?

Tabela de jobs duráveis, claim curto, worker limitado, estado de tentativa e recuperação. Explicaria quando filas ou engine de workflow agregam valor, sem delegar invariantes financeiros ao scheduler escolhido.

Termo para lembrar: Job architecture baseline | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Ocorrência durável versus disparo efêmero

Ganho. Permite deduplicar e recuperar agenda

Custo e motivo. Adiciona armazenamento e ciclo de vida

Como decidir. Quando execução não pode desaparecer no scheduler

02. Lease versus ownership permanente

Ganho. Recupera worker indisponível automaticamente

Custo e motivo. Exige renovação e proteção contra dono antigo

Como decidir. Quando processos podem parar sem liberar trabalho

03. Checkpoint frequente versus espaçado

Ganho. Reduz reprocessamento após falha

Custo e motivo. Aumenta persistência e coordenação

Como decidir. Quando tarefas são longas e repetição custa caro

04. Retry automático versus intervenção

Ganho. Reduz trabalho manual em falhas transitórias

Custo e motivo. Consome capacidade e exige classificação segura

Como decidir. Quando a operação é repetível dentro do contrato

05. Partições pequenas versus grandes

Ganho. Melhora balanceamento e retomada

Custo e motivo. Aumenta metadados e overhead por unidade

Como decidir. Quando custos dos itens variam ou falhas são frequentes

06. Dependência explícita versus horários fixos

Ganho. Preserva ordem mesmo com atrasos

Custo e motivo. Exige estado e coordenação de workflow

Como decidir. Quando etapa depende de resultado anterior

07. Cancelamento cooperativo versus conclusão obrigatória

Ganho. Evita trabalho futuro desnecessário

Custo e motivo. Pode deixar resultado parcial que precisa de explicação

Como decidir. Quando tarefas restantes ainda são reversíveis

08. Fila por classe versus compartilhada

Ganho. Isola prazos e tipos de trabalho

Custo e motivo. Pode reduzir utilização agregada

Como decidir. Quando workloads têm criticidades diferentes

09. Quarentena por item versus abortar lote

Ganho. Permite continuar itens válidos

Custo e motivo. Exige contrato de parcialidade e revisão

Como decidir. Quando independência entre itens é legítima

10. Timezone de negócio versus UTC fixo

Ganho. Preserva intenção de horário local

Custo e motivo. Exige tratamento de calendário e transições

Como decidir. Quando prazo é definido pela rotina local

Duas alternativas ao caminho proposto

1. Tabela de jobs

Quando faz sentido. Fluxos simples e volume administrável

Como implementar. Claim com lease e execução idempotente

Limites. Implementar timers e operação manualmente

2. Workflow engine

Quando faz sentido. Esperas e coordenação são complexas

Como implementar. Persistir workflow e activities com retries

Limites. Efeitos externos ainda exigem identidade

Referências e alcance da evidência

1. Kubernetes: CronJob

Agendamento, concorrência e necessidade de jobs idempotentes. As respostas e parâmetros de aplicação são propostas autorais.

https://kubernetes.io/docs/concepts/workloads/controllers/cron-jobs/

2. Temporal: retry policies

Políticas de repetição de execução durável. As respostas e parâmetros de aplicação são propostas autorais.

https://docs.temporal.io/encyclopedia/retry-policies

Continue em: T106 · T108

T108 / K. Famílias de exercícios

Metering, billing e quotas

Metering registra uso; billing interpreta uso por preço; quota controla admissão. Arquitetura de treino; não descreve implementação interna do Nubank.

Quando usar. Cobrança por consumo, limites de API e orçamento de parceiros.

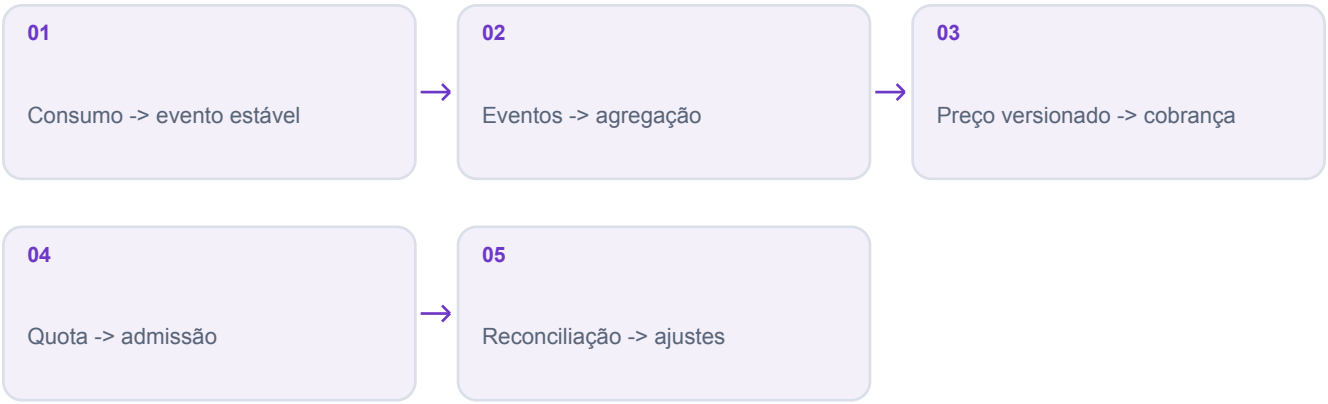
Quando evitar ou limitar. Usar contador aproximado como única autoridade de fatura financeira.

Uma abertura possível

Eu começaria por atribuir identidade estável ao evento de uso. Depois explicaria definir tempo de competência e revisão, preservando as restrições do caso.

Peças e tecnologias. Meter events · PostgreSQL · Kafka · Token bucket · Decimal

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Atribuir identidade estável ao evento de uso; validar mesmo consumo reenviado após falha.

2. Definir tempo de competência e revisão; validar eventos após fechamento.

3. Aplicar tabela versionada por vigência; validar consumo antes e após mudança.

4. Modelar exatamente a unidade contratada; validar falha e retry perto do limite.

5. Usar autoridade consistente quando ultrapassar é proibido; validar concorrência entre instâncias.

30 perguntas para sustentar o desenho

01. Retry do medidor cobra duas vezes?

Identificaria cada consumo legítimo de forma estável e deduplicaria na ingestão apropriada. Reenviar após falha não pode cobrar novamente, mas eventos iguais em valor e horário podem representar usos diferentes.

Termo para lembrar: Usage identity | Base técnica: referência 1.

02. Uso atrasado pertence a qual fatura?

Definiria competência, fechamento e revisão para eventos atrasados, preservando tempo do fato e de chegada. O processamento posterior não deve mudar silenciosamente o período cobrado nem apagar consumo que ainda precisa ser explicado.

Termo para lembrar: Event time | Base técnica: referência 2.

03. Preço mudou no meio do mês: como calcular?

Aplicaria preço pela versão e vigência contratadas, guardando regras e parâmetros necessários à reprodução. Consultar apenas a tabela atual pode alterar faturas passadas; correções geram ajustes explícitos conforme política.

Termo para lembrar: Price version | Base técnica: referência 1.

04. Quatro envios incluem tentativas falhas?

Esclareceria o que a quota conta: solicitação, tentativa física, arquivo ou item aceito. Reservas e consumo duráveis precisam refletir essa unidade, incluindo retries e timezone, para que quatro jobs não excedam quatro envios permitidos.

Termo para lembrar: Quota reservation | Base técnica: referência 2.

05. Posso somar contadores locais para limite rígido?

Para teto estrito, usaria autoridade consistente ou subcotas conservadas conforme o modelo. Somar contadores locais atrasados pode permitir overshoot; aproximação só entra quando o contrato aceita explicitamente esse desvio.

Termo para lembrar: Approximate counters | Base técnica: referência 1.

06. Correção de uso exige apagar evento?

Registraria ajustes vinculados ao uso original, com motivo e autorização. Apagar evento dificulta contestação e replay; uma correção precisa alterar o cálculo de forma reproduzível sem produzir novo desconto a cada execução.

Termo para lembrar: Refund credits | Base técnica: referência 2.

07. Como reduzir bilhões de eventos sem perder prova?

Agregaria por dimensões úteis mantendo referência recuperável à evidência necessária. Isso reduz custo de consulta, mas o nível de resumo precisa permitir explicar fatura e investigar duplicatas, conforme retenção autorizada.

Termo para lembrar: Aggregation | Base técnica: referência 1.

08. Milissegundo e segundo podem se misturar?

Padronizaria unidade no contrato e validaria conversões, escala e limites entre produtores. Misturar segundos e milissegundos pode multiplicar cobrança; testes de roundtrip e valores conhecidos fazem parte da aceitação da integração.

Termo para lembrar: Units | Base técnica: referência 2.

09. Soma dos produtores iguala o faturado?

Confrontaria eventos recebidos, agregados, preços e valores cobrados, com marcos de completude. Uma fatura numericamente plausível pode estar incompleta; reconciliação precisa detectar perda, duplicação e aplicação incorreta de vigência.

Termo para lembrar: Reconciliation | Base técnica: referência 1.

10. Alerta impede gasto excessivo instantaneamente?

Separaria alerta de controle preventivo: notificação chega depois da observação e pode atrasar. Quando gasto não pode ultrapassar o teto, aplicaria admissão ou reserva; alertas servem para acompanhamento e escalonamento.

Termo para lembrar: Budget alerts | Base técnica: referência 2.

11. Metering e billing são a mesma coisa?

Não. Metering registra consumo; billing interpreta esse consumo por preço e contrato. Eu separaria eventos de uso de cálculos monetários para corrigir tarifa ou regra sem perder a evidência original.

Termo para lembrar: Usage versus charge | Base técnica: referência 1.

12. Quota é diferente de rate limit?

Quota limita quantidade em um escopo e período definidos; rate limit controla ritmo de acesso. Podem coexistir, e eu esclareceria unidades e tolerância antes de escolher contador, janela ou reserva.

Termo para lembrar: Quota semantics | Base técnica: referência 1.

13. Como garantir que uma operação cobrável não ocorra sem registrar seu uso?

Eu definiria o ponto que torna o uso faturável e persistiria sua evidência junto do efeito quando houver uma transação comum. Se os sistemas forem separados, usaria entrega durável e reconciliação; confirmar a operação e depois incrementar um contador volátil pode perder cobrança.

Termo para lembrar: Metering commit boundary

14. Quando um contador local é aceitável?

Quando a aproximação ou overshoot calculado atende o contrato, como certas proteções operacionais. Para cobrança ou limite estrito, eu usaria evidência durável e coordenação adequada, explicando a diferença de garantia.

Termo para lembrar: Approximation boundary | Base técnica: referência 1.

15. Como arredondar milhares de valores menores que um centavo?

Eu acumularia com precisão suficiente e aplicaria a regra contratual no nível correto, como linha ou total da fatura. Arredondar cada evento prematuramente pode introduzir viés; manteria rastreáveis valor exato, ajuste e resultado final para explicar diferenças ao cliente.

Termo para lembrar: Monetary rounding policy

16. Como impedir um produtor de atribuir consumo à conta de outro cliente?

Eu derivaria a identidade de cobrança do contexto autenticado e de vínculos autorizados, não de um campo livre no evento. Validaria produtor, recurso e tenant antes da ingestão; um identificador bem formado não prova que o emissor pode cobrar daquela conta.

Termo para lembrar: Trusted billing attribution

17. Preço por faixa e preço por volume são iguais?

Não necessariamente; a aplicação das faixas pode mudar bastante o total. Eu modelaria a regra explicitamente e usaria exemplos de fronteira compartilhados com produto e financeiro antes de implementar.

Termo para lembrar: Tier semantics | Base técnica: referência 1.

18. Como impedir estouro de quota em duas regiões?

Definiria autoridade consistente, subcotas conservadas ou tolerância explícita de overshoot. Contadores eventualmente replicados não garantem teto global estrito; escolheria pela consequência de ultrapassar o limite.

Termo para lembrar: Global quota control | Base técnica: referência 1.

19. Reserva de quota deve ser liberada após timeout?

Somente conforme evidência e unidade do contrato. A tentativa pode ter consumido o recurso externo; eu manteria estado incerto ou reconciliaria, evitando devolver capacidade que já foi utilizada.

Termo para lembrar: Quota uncertainty | Base técnica: referência 1.

20. Como medir custo por cliente sem expor dados?

Usaria identificadores internos e dimensões autorizadas, com agregação e acesso controlado. Atribuição de custo não exige registrar payloads sensíveis; preservaria detalhe suficiente para contestação e planejamento.

Termo para lembrar: Cost attribution | Base técnica: referência 1.

21. Como tratar evento de uso negativo?

Definiria ajuste explícito com referência e autorização, em vez de aceitar qualquer número negativo. Isso evita manipulação do medidor e permite reconciliar correções com o consumo original.

Termo para lembrar: Usage adjustment | Base técnica: referência 1.

22. Reprocessar histórico deve emitir nova cobrança?

Não. O cálculo pode reconstruir resultados, mas publicação e cobrança têm identidades próprias e versões. Eu compararia diferenças antes de gerar ajuste autorizado, impedindo que replay técnico vire débito adicional.

Termo para lembrar: Billing replay | Base técnica: referência 1.

23. Como validar fatura antes de emitir?

Confrontaria consumo, preços, ajustes, impostos aplicáveis e totais, com testes de fronteira. Também verificaria completude das origens, porque ausência de eventos pode parecer uma fatura válida de valor menor.

Termo para lembrar: Invoice validation | Base técnica: referência 1.

24. Qual risco de somar percentis de consumo?

Percentis não são aditivos de forma geral. Para dimensionar e cobrar, eu usaria a agregação definida no contrato e dados apropriados, evitando transformar uma estatística operacional em medida financeira incorreta.

Termo para lembrar: Aggregation semantics | Base técnica: referência 1.

25. Como controlar cardinalidade das dimensões?

Limitando campos e valores permitidos, separando dados de alta cardinalidade quando necessário. Dimensões arbitrárias enviadas pelo cliente podem explodir armazenamento e custo, além de introduzir informação pessoal desnecessária.

Termo para lembrar: Meter cardinality | Base técnica: referência 1.

26. Quando fechar uma janela de quota?

Segundo timezone, calendário e unidade contratados, com transição determinística. Eu testaria eventos próximos da fronteira e relógios divergentes, evitando reset por processo que permita consumo duplicado em reinícios.

Termo para lembrar: Quota window | Base técnica: referência 1.

27. Como lidar com mudança de plano no meio do ciclo?

Aplicaria a regra de vigência e eventual proporcionalidade acordada, preservando versões. A decisão não deve depender da hora em que o worker processou um evento atrasado, e sim do contrato.

Termo para lembrar: Plan transition | Base técnica: referência 1.

28. Como repartir consumo entre centros de custo sem duplicar o faturamento?

Eu manteria uma medida canônica de uso e regras versionadas de alocação, verificando que parcelas somam o total correspondente. Relatórios podem apresentar várias dimensões, mas não devem somá-las como consumos independentes; arredondamentos e valores não atribuídos precisam de tratamento explícito.

Termo para lembrar: Hierarchical cost allocation

29. Qual evidência ajuda contestação de cobrança?

Eventos de uso identificados, unidade, período, preço vigente e ajustes vinculados. Eu forneceria explicação reproduzível sem expor dados de terceiros, permitindo comparar a fatura com a atividade efetiva do cliente.

Termo para lembrar: Billing evidence | Base técnica: referência 1.

30. Como testar pico e recuperação do medidor?

Simularia duplicatas, atraso, queda e replay, verificando que total final corresponde ao uso legítimo. Separaria disponibilidade de ingestão e completude do faturamento, monitorando pendências antes do fechamento.

Termo para lembrar: Meter recovery | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Eventos detalhados versus contador agregado

Ganho. Permitem reprodução e contestação

Custo e motivo. Aumentam volume e retenção

Como decidir. Quando cobrança exige evidência por consumo

02. Fechamento tardio versus correção posterior

Ganho. Inclui mais dados na primeira fatura

Custo e motivo. Atrasa emissão e disponibilidade

Como decidir. Quando atraso das fontes é previsível

03. Preço versionado versus consulta ao preço atual

Ganho. Preserva contrato histórico

Custo e motivo. Exige vigência e armazenamento de regras

Como decidir. Quando tarifas mudam ao longo do tempo

04. Reserva de quota versus consumo após execução

Ganho. Evita iniciar trabalho sem capacidade

Custo e motivo. Pode reter quota durante incerteza

Como decidir. Quando limite externo é rígido

05. Coordenação global versus contadores locais

Ganho. Preserva teto estrito

Custo e motivo. Adiciona latência e dependência de autoridade

Como decidir. Quando overshoot é inaceitável

06. Ajuste explícito versus editar consumo

Ganho. Mantém trilha e replay compreensíveis

Custo e motivo. Adiciona tipos de evento e cálculo

Como decidir. Quando correções precisam de auditoria

07. Pré-agregação versus consulta bruta

Ganho. Reduz custo e tempo de faturamento

Custo e motivo. Complica revisão e dimensões não previstas

Como decidir. Quando padrões de cobrança são estáveis

08. Unidade normalizada versus formatos por produtor

Ganho. Reduz erro de interpretação

Custo e motivo. Exige adapters e validação na entrada

Como decidir. Quando múltiplas equipes emitem consumo

09. Reconciliação completa versus amostragem

Ganho. Detecta divergências em todo universo definido

Custo e motivo. Consome mais processamento e prazo

Como decidir. Quando materialidade exige cobertura total

10. Hard limit versus somente alerta

Ganho. Contém consumo antes do gasto

Custo e motivo. Pode rejeitar demanda legítima no teto

Como decidir. Quando orçamento ou quota são obrigatórios

Duas alternativas ao caminho proposto

1. Contador transacional

Quando faz sentido. Quota estrita cabe numa fronteira

Como implementar. Reservar e consumir capacidade atomicamente

Limites. Contenção limita recursos muito disputados

2. Agregação assíncrona

Quando faz sentido. Cobrança tolera atraso com reconciliação

Como implementar. Persistir eventos e calcular por janela

Limites. Não protege sozinha limite estrito

Referências e alcance da evidência

1. Stripe: registrar utilização

Eventos de consumo e contabilização de uso. As respostas e parâmetros de aplicação são propostas autorais.

https://docs.stripe.com/billing/subscriptions/usage-based/recording-usage-api

2. AWS: Service Quotas

Limites de serviço e sua gestão. As respostas e parâmetros de aplicação são propostas autorais.

https://docs.aws.amazon.com/servicequotas/latest/userguide/intro.html

Continue em: T107 · T109

T109 / K. Famílias de exercícios

A/B tests e feature flags

Flags controlam exposição; experimentos estimam efeitos sob desenho e medição adequados. Arquitetura de treino; não descreve implementação interna do Nubank.

Quando usar. Rollout gradual, desligamento rápido e testes de hipóteses de produto.

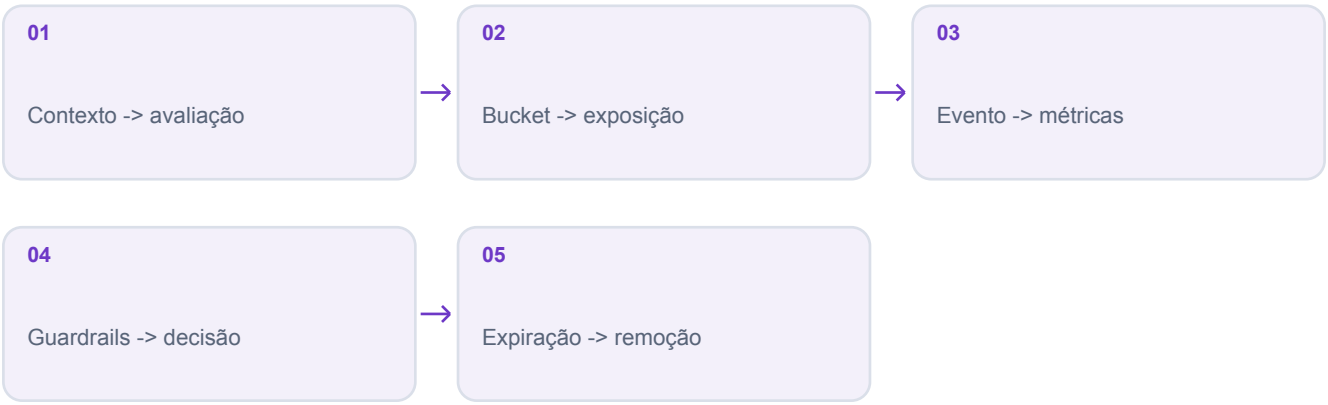
Quando evitar ou limitar. Tratar flag no aplicativo como autorização ou p-valor isolado como decisão suficiente.

Uma abertura possível

Eu começaria por usar bucket determinístico por unidade escolhida. Depois explicaria escolher unidade alinhada à interferência esperada, preservando as restrições do caso.

Peças e tecnologias. OpenFeature · Feature provider · Stable hashing · Experiment metrics

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Usar bucket determinístico por unidade escolhida; validar mesmo usuário em várias sessões.

2. Escolher unidade alinhada à interferência esperada; validar interações entre membros do experimento.

3. Registrar exposição quando funcionalidade participa; validar funil entre avaliação e uso.

4. Priorizar limites de segurança previamente definidos; validar erro latência e reclamação por grupo.

5. Definir validade e default seguro no backend; validar cliente desconectado após desligamento.

30 perguntas para sustentar o desenho

01. Usuário muda de grupo a cada refresh?

Usaria atribuição determinística por unidade apropriada e escopo do experimento, testando estabilidade entre sessões. A estratégia deve preservar grupos conforme o desenho, evitando que cada refresh exponha o cliente a tratamento diferente.

Termo para lembrar: Assignment | Base técnica: referência 1.

02. Randomizo requisição ou cliente?

Escolheria unidade considerando interferência: requisição, pessoa ou empresa produzem interpretações distintas. Se usuários compartilham estado, randomizar individualmente pode contaminar grupos; a análise precisa acompanhar a unidade realmente independente.

Termo para lembrar: Unit of randomization | Base técnica: referência 2.

03. Flag avaliada conta como experiência vista?

Registraria exposição quando o comportamento participa da jornada, com definição clara e versão. Avaliação de flag sem uso não equivale a experiência vista; denominadores inconsistentes podem distorcer o efeito estimado.

Termo para lembrar: Exposure logging | Base técnica: referência 1.

04. Conversão subiu mas erros financeiros também?

Definiria guardrails antes de observar resultados, incluindo falhas, latência e consequências financeiras. Ganho de conversão não autoriza ultrapassar limites essenciais; rollout pode ser interrompido mesmo quando a métrica principal melhora.

Termo para lembrar: Guardrails | Base técnica: referência 2.

05. Flag desligada no servidor muda app offline?

Para recursos sensíveis, backend mantém validação independente e comportamento seguro quando configuração está antiga. Um aplicativo offline pode conservar flag anterior; kill switch precisa de semântica realista sobre onde e quando passa a valer.

Termo para lembrar: Kill switch | Base técnica: referência 1.

06. Habilitar flag autoriza operação restrita?

Flag controla exposição, não concede permissão. Cada ação continua validando identidade, recurso e regra no domínio; alterar configuração local ou chamar endpoint diretamente não deve liberar uma operação restrita.

Termo para lembrar: Authorization | Base técnica: referência 2.

07. Poucos usuários sem diferença provam empate?

Planejaria tamanho amostral e efeito mínimo relevantes à decisão, considerando variabilidade e frequência. Poucos dados sem diferença não demonstram equivalência; apresentaria incerteza e limites antes de declarar um tratamento vencedor.

Termo para lembrar: Statistical power | Base técnica: referência 1.

08. Posso encerrar assim que p ficar baixo?

Escolheria horizonte ou método sequencial apropriado antes da análise. Parar ao primeiro resultado favorável pode aumentar falsos positivos; o procedimento precisa distinguir exploração de uma conclusão que sustenta rollout amplo.

Termo para lembrar: Peeking | Base técnica: referência 2.

09. Centenas de flags antigas são problema?

Cada flag teria dono, finalidade e critério de remoção. Inventariaria combinações e caminhos antigos, porque flags esquecidas aumentam estados possíveis, custo de teste e risco de manter comportamento já não suportado.

Termo para lembrar: Flag debt | Base técnica: referência 1.

10. Serviço de flags caiu: derrubo tudo?

Definiria defaults por semântica e risco, com cache e validade apropriados. Provider indisponível não deve ativar comportamento perigoso nem derrubar toda jornada sem necessidade; testaria falha durante rollout e recuperação.

Termo para lembrar: Provider outage | Base técnica: referência 2.

11. Como distinguir curiosidade inicial de um ganho sustentado no experimento?

Eu definiria horizonte e análise por tempo de exposição antes de interpretar o resultado, acompanhando uso repetido e efeitos tardios. Um pico nos primeiros dias pode refletir novidade; prolongar ou segmentar a observação exige manter critérios estatísticos coerentes com o plano.

Termo para lembrar: Novelty effect

12. Como gerar bucket estável sem expor PII?

Usaria identificador interno apropriado e função determinística com escopo do experimento, evitando dados pessoais crus. A escolha da unidade precisa impedir mudanças inesperadas de grupo entre sessões ou dispositivos.

Termo para lembrar: Stable bucketing | Base técnica: referência 1.

13. Como evitar que um cache compartilhado contamine controle e tratamento?

Eu incluiria na chave as diferenças que alteram a resposta e separaria resultados entre variantes quando necessário. Um usuário do controle não pode receber conteúdo gerado pelo tratamento; também verificaria efeitos compartilhados no backend que podem atravessar a fronteira do experimento.

Termo para lembrar: Cross-cohort cache contamination

14. Mudar porcentagem pode mover pessoas de grupo?

Depende do algoritmo. Eu escolheria estratégia que preserve estabilidade quando necessário e testaria expansão, redução e versões, evitando que usuários alternem de comportamento sem intenção do experimento.

Termo para lembrar: Cohort stability | Base técnica: referência 1.

15. Qual problema de testar muitas métricas até uma melhorar?

Aumenta chance de encontrar efeito aparente por acaso. Eu definiria hipótese, métricas e análise adequadas previamente, distinguindo descoberta exploratória de conclusão confirmatória antes de decidir rollout amplo.

Termo para lembrar: Multiple testing | Base técnica: referência 1.

16. Como reagir a sample ratio mismatch?

Investigaria randomização, exposição e perdas de eventos antes de interpretar resultado. Distribuição inesperada entre grupos pode invalidar comparação; não trataria significância estatística como confiável sem verificar integridade do experimento.

Termo para lembrar: SRM | Base técnica: referência 1.

17. Experimento pode continuar quando error budget estoura?

Eu aplicaria política de segurança e guardrails acordados, podendo suspender exposição. Ganho de conversão não justifica ignorar falhas críticas; a decisão relaciona objetivo de produto e confiabilidade da jornada.

Termo para lembrar: Experiment guardrails | Base técnica: referência 1.

18. Como avaliar resultado em clientes antigos e novos?

Analisaria segmentos relevantes previstos, com cuidado para não inventar histórias após observar dados. Efeitos podem diferir, mas conclusões precisam considerar amostra, incerteza e hipóteses de cada grupo.

Termo para lembrar: Heterogeneous effects | Base técnica: referência 1.

19. Quando randomizar por empresa em vez de usuário?

Quando usuários interagem ou compartilham estado que contamina tratamento. Eu escolheria unidade que reduza interferência, reconhecendo que menos unidades independentes afetam tamanho amostral e precisão da análise.

Termo para lembrar: Cluster randomization | Base técnica: referência 1.

20. Como testar flags combinadas?

Mapearia interações relevantes e restringiria combinações incompatíveis, em vez de assumir independência. Flags financeiras exigem contratos claros; remover flags antigas reduz o espaço de estados e a carga de testes.

Termo para lembrar: Flag interactions | Base técnica: referência 1.

21. Pode confiar em configuração enviada ao mobile offline?

Só dentro de validade e risco definidos. O backend revalida ações sensíveis e versões suportadas; cliente offline não pode manter uma permissão financeira indefinidamente porque recebeu uma flag antes.

Termo para lembrar: Offline flags | Base técnica: referência 1.

22. Quem pode alterar um kill switch?

Papéis autorizados com auditoria e procedimento de operação. Eu evitaria acesso amplo a alterações críticas, mas garantiria capacidade de resposta em incidente, registrando motivo, versão e resultado da intervenção.

Termo para lembrar: Flag governance | Base técnica: referência 1.

23. Como impedir que duas mudanças simultâneas sobrescrevam a configuração de uma flag?

Eu versionaria a configuração e exigiria atualização condicional sobre a revisão conhecida, registrando autor e diff. Em conflito, a segunda mudança é reavaliada; um último escritor indiscriminado pode desfazer silenciosamente um kill switch ou alterar a segmentação aprovada.

Termo para lembrar: Configuration compare-and-swap

24. Qual diferença entre rollout canário e A/B test?

Canário prioriza segurança e comportamento operacional; experimento estima efeito com desenho estatístico. Podem compartilhar mecanismos de exposição, mas objetivos, métricas e critérios de decisão não são equivalentes.

Termo para lembrar: Canary versus experiment | Base técnica: referência 1.

25. Quando manter um grupo de controle depois de ampliar o lançamento?

Eu consideraria um holdout persistente para medir efeitos acumulados que não aparecem num teste curto, com tamanho e duração justificados. Ele também tem custo de oportunidade e restrições éticas; não manteria clientes sem uma melhoria essencial apenas para preservar uma comparação conveniente.

Termo para lembrar: Persistent holdout

26. O que fazer quando métrica principal melhora e retenção cai?

Eu examinaria trade-off e guardrails definidos, com horizonte suficiente. Métricas de curto prazo podem antecipar dano posterior; a decisão precisa considerar valor ao cliente e objetivo real, não apenas cliques.

Termo para lembrar: Metric alignment | Base técnica: referência 1.

27. Como remover uma flag com segurança?

Confirmaria estado final, consumidores e ausência de dependência operacional, depois removeria código e configuração com testes. Flag permanente sem dono vira dívida e pode manter caminhos antigos inseguros ou confusos.

Termo para lembrar: Flag retirement | Base técnica: referência 1.

28. Como versionar regra de segmentação?

Registraria versão e contexto usados na avaliação, permitindo reproduzir exposição. Alteração silenciosa da segmentação no meio do experimento pode mudar população e comprometer comparabilidade dos resultados observados.

Termo para lembrar: Targeting version | Base técnica: referência 1.

29. A/B test pode avaliar prejuízo raro em poucos dias?

Talvez não tenha poder ou maturidade temporal suficiente. Eu consideraria gravidade, frequência e indicadores complementares, sem interpretar ausência de evento raro como prova de segurança do tratamento.

Termo para lembrar: Rare-event evaluation | Base técnica: referência 1.

30. Qual argumento Staff nesse assunto?

Eu conectaria mecanismo de flags a autorização, operação, qualidade experimental e remoção de dívida. A ferramenta facilita exposição, mas uma decisão confiável exige hipótese, evidências e limites de risco explícitos.

Termo para lembrar: Experimentation system | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Bucket estável versus sorteio por requisição

Ganho. Preserva experiência e interpretação por usuário

Custo e motivo. Requer identidade e escopo consistentes

Como decidir. Quando tratamento precisa persistir entre sessões

02. Randomização por cluster versus indivíduo

Ganho. Reduz contaminação entre usuários relacionados

Custo e motivo. Diminui número de unidades independentes

Como decidir. Quando há interação ou estado compartilhado

03. Exposição real versus mera atribuição

Ganho. Melhora denominador da análise

Custo e motivo. Exige instrumentação no ponto de uso

Como decidir. Quando nem todo atribuído encontra a funcionalidade

04. Guardrails rígidos versus otimização da métrica principal

Ganho. Limita dano durante experimento

Custo e motivo. Pode encerrar teste antes de benefício aparecer

Como decidir. Quando segurança e confiabilidade são prioritárias

05. Avaliação backend versus só mobile

Ganho. Mantém controle de ações sensíveis

Custo e motivo. Adiciona dependência de rede para certas decisões

Como decidir. Quando cliente pode estar desatualizado ou manipulado

06. Autorização independente versus embutida em flag

Ganho. Clarifica segurança e revogação

Custo e motivo. Mantém dois mecanismos com responsabilidades distintas

Como decidir. Quando exposição e permissão são conceitos diferentes

07. Amostra maior versus decisão rápida

Ganho. Reduz incerteza para efeito definido

Custo e motivo. Aumenta duração e custo de oportunidade

Como decidir. Quando efeito esperado é pequeno ou variável

08. Análise sequencial versus horizonte fixo

Ganho. Permite decisões antecipadas sob método adequado

Custo e motivo. Exige disciplina estatística específica

Como decidir. Quando acompanhamento contínuo é necessário

09. Remover flags versus preservar flexibilidade

Ganho. Reduz combinações e manutenção

Custo e motivo. Perde capacidade imediata de alternar caminho antigo

Como decidir. Quando rollout terminou e rollback já não depende dela

10. Cache de flags versus consulta central contínua

Ganho. Reduz latência e dependência operacional

Custo e motivo. Atrasa mudanças e revogações

Como decidir. Quando validade contratada permite atraso

Duas alternativas ao caminho proposto

1. Rollout sem experimento

Quando faz sentido. Objetivo é segurança de lançamento

Como implementar. Ampliar cohorts observando guardrails

Limites. Não estima causalidade automaticamente

2. Experimento controlado

Quando faz sentido. Há hipótese e população adequadas

Como implementar. Randomizar unidades e definir métricas antes

Limites. Exige análise e proteção dos participantes

Referências e alcance da evidência

1. OpenFeature: especificação

Avaliação de flags, providers e contexto. As respostas e parâmetros de aplicação são propostas autorais.

https://openfeature.dev/specification/

2. Microsoft Research: experimentação

Pesquisa e práticas de experimentos controlados. As respostas e parâmetros de aplicação são propostas autorais.

https://www.microsoft.com/en-us/research/group/experimentation-platform-exp/

Continue em: T108 · T110

T110 / K. Famílias de exercícios

Recomendação, ranking e personalização

Recomendação recupera candidatos, ordena opções e aplica regras de elegibilidade antes da exposição. Arquitetura de treino; não descreve implementação interna do Nubank.

Quando usar. Personalização de conteúdo e descoberta de produtos permitidos.

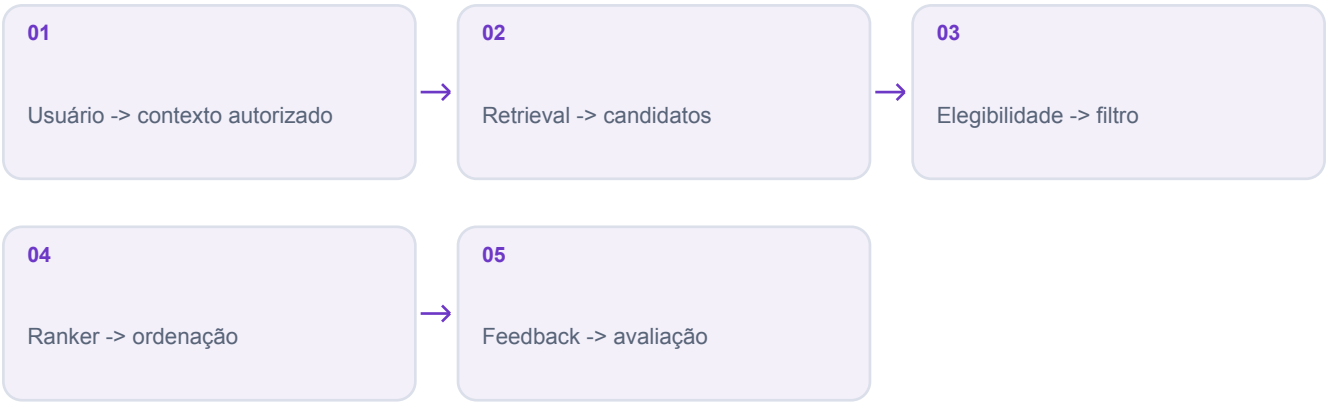
Quando evitar ou limitar. Otimizar cliques ignorando adequação, privacidade ou restrições financeiras.

Uma abertura possível

Eu começaria por reduzir universo antes do ranking caro. Depois explicaria aplicar regras autoritativas de elegibilidade, preservando as restrições do caso.

Peças e tecnologias. Candidate retrieval · Vector search · Ranker · Feature store · Experimentation

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Reduzir universo antes do ranking caro; validar cobertura de candidatos relevantes.

2. Aplicar regras autoritativas de elegibilidade; validar usuários fora dos critérios.

3. Usar contexto permitido e opções gerais; validar qualidade para usuários novos.

4. Considerar seleção e posição no feedback; validar métricas por posição e coorte.

5. Validar qualidade offline e experimento seguro; validar resultados online com guardrails.

30 perguntas para sustentar o desenho

01. Por que não pontuar todo catálogo?

Separaria recuperação de candidatos e ranking para evitar pontuar todo catálogo com modelo caro. Mediria recall da primeira etapa, porque nenhum reranker pode recuperar uma opção relevante que foi descartada antes.

Termo para lembrar: Candidate generation | Base técnica: referência 1.

02. Modelo recomenda crédito não disponível: e agora?

Aplicaria elegibilidade autoritativa independentemente do score, verificando disponibilidade e regras atuais. Uma sugestão bem pontuada não significa aprovação de crédito; personalização deve operar dentro das opções que o cliente pode legitimamente receber.

Termo para lembrar: Eligibility | Base técnica: referência 2.

03. Novo cliente não tem histórico: o que mostrar?

Para usuários novos, usaria contexto permitido e baseline geral, com exploração controlada quando adequada. Não inventaria preferências sem evidência; avaliaria separadamente qualidade e experiência dessa coorte para não esconder cold start na média.

Termo para lembrar: Cold start | Base técnica: referência 1.

04. Cliques refletem preferência ou exposição?

Consideraria que feedback depende do que foi exposto e da posição. Cliques não são preferência pura; a avaliação e a política de exploração precisam evitar reforçar continuamente apenas itens que já receberam destaque.

Termo para lembrar: Feedback bias | Base técnica: referência 2.

05. Recall alto garante melhor experiência?

Compararia métricas offline com resultados online e guardrails, mantendo hipótese clara. Recall alto não garante valor ao cliente; a escolha depende da jornada e pode envolver diversidade, adequação e resultados de longo prazo.

Termo para lembrar: Offline metrics | Base técnica: referência 1.

06. Catálogo mudou e embedding ficou antigo?

Versionaria catálogo, embeddings e índice, revalidando disponibilidade na saída. A atualização assíncrona pode atrasar; item removido ou condição revogada não deve continuar elegível apenas porque seu vetor ainda está presente.

Termo para lembrar: Freshness | Base técnica: referência 2.

07. Ranker excedeu deadline: como responder?

Definiria deadline e fallback elegível que não dependa da mesma falha. Um ranker lento não precisa bloquear toda jornada; a degradação pode reduzir personalização enquanto preserva segurança e utilidade básica.

Termo para lembrar: Latency budget | Base técnica: referência 1.

08. Dez recomendações iguais são boas?

Equilibraria relevância com diversidade e restrições explícitas, medindo resultado por sessão. Dez itens quase iguais podem maximizar um score local e reduzir descoberta; a política precisa refletir o objetivo real do produto.

Termo para lembrar: Diversity | Base técnica: referência 2.

09. Embedding elimina identificação de clientes?

Trataria embeddings e features conforme risco e finalidade dos dados, com acesso e retenção adequados. Transformar informação em vetor não a torna automaticamente anônima nem elimina responsabilidade sobre os derivados.

Termo para lembrar: Privacy | Base técnica: referência 1.

10. Nova versão piora uma minoria: como detectar?

Faria avaliação por segmentos e rollout gradual com versão e reversão operacional. Média melhor pode esconder prejuízo concentrado; guardrails e qualidade por coorte ajudam decidir se expandir, ajustar ou interromper o novo modelo.

Termo para lembrar: Model rollout | Base técnica: referência 2.

11. Retrieval e ranking precisam do mesmo modelo?

Não. Retrieval reduz rapidamente o universo; ranking pode usar sinais mais caros em poucos candidatos. Eu avaliaria o gargalo de qualidade em cada etapa antes de aumentar complexidade indiscriminadamente.

Termo para lembrar: Two-stage recommendation | Base técnica: referência 1.

12. O que é problema de cold start de item?

Um item novo ainda não tem interação suficiente. Eu usaria atributos disponíveis e exploração controlada, mantendo elegibilidade, para não depender exclusivamente de histórico que nunca existirá se ele não for mostrado.

Termo para lembrar: Item cold start | Base técnica: referência 1.

13. Como evitar recomendação de produto já contratado?

Aplicaria filtros de estado e regra do produto na saída, com freshness adequado. Um modelo treinado anteriormente pode continuar pontuando o item, mas não deve superar a elegibilidade atual do cliente.

Termo para lembrar: Eligibility filtering | Base técnica: referência 1.

14. Popularidade é uma baseline aceitável?

Sim, quando comparada com objetivos e restrições. Eu começaria com baseline simples para medir ganho incremental de personalização, evitando operar modelos complexos que não melhoram experiência ou custo de resolução.

Termo para lembrar: Recommendation baseline | Base técnica: referência 1.

15. Cliques são sempre a melhor métrica?

Não. Podem incentivar conteúdo chamativo sem valor ou produto inadequado. Eu alinharia objetivo a resultado útil e guardrails, avaliando efeitos posteriores e segmentos em vez de otimizar apenas interação imediata.

Termo para lembrar: Objective selection | Base técnica: referência 1.

16. Como combater feedback loop de popularidade?

Introduziria diversidade ou exploração controlada e avaliaria exposição desigual. O sistema aprende do que mostra; sem considerar essa seleção, pode reforçar continuamente itens já favorecidos e reduzir descoberta.

Termo para lembrar: Feedback loop | Base técnica: referência 1.

17. Como fazer avaliação temporal offline?

Treinaria com dados anteriores e avaliaria interações posteriores, respeitando disponibilidade dos sinais. Divisão aleatória pode permitir vazamento temporal e produzir qualidade artificial que não aparece em produção.

Termo para lembrar: Temporal evaluation | Base técnica: referência 1.

18. Como evitar que um cliente de grande volume monopolize o serviço de ranking?

Eu aplicaria limites de concorrência e orçamento por tenant, com agendamento justo e isolamento das filas quando necessário. Escalar apenas pelo volume total pode preservar throughput enquanto outros clientes ficam sem resposta; mediria espera e rejeição por classe de serviço.

Termo para lembrar: Inference fairness

19. Quando re-ranking por regras é necessário?

Quando diversidade, elegibilidade ou restrições de negócio precisam ajustar candidatos. Eu tornaria as regras explícitas e auditáveis, observando seu impacto na qualidade para não esconder limitações do modelo.

Termo para lembrar: Post-ranking constraints | Base técnica: referência 1.

20. Como responder quando feature store fica indisponível?

Usaria fallback aprovado com dados suficientes ou reduziria personalização, conforme risco. Não preencheria silenciosamente sinais críticos com valores arbitrários que alterem elegibilidade ou tornem a recomendação enganosa.

Termo para lembrar: Feature fallback | Base técnica: referência 1.

21. Pode cachear lista personalizada por dias?

Depende de validade e mudanças de contexto. Eu incluiria versões relevantes e revalidaria restrições antes da exposição, porque cache prolongado pode recomendar produto indisponível ou conteúdo já inadequado ao cliente.

Termo para lembrar: Recommendation cache | Base técnica: referência 1.

22. Como controlar custo de inferência?

Limitaria candidatos, frequência e recursos, comparando qualidade por custo da jornada. Modelos mais caros precisam justificar ganho; também observaria cauda de latência e capacidade durante picos e recuperação.

Termo para lembrar: Serving economics | Base técnica: referência 1.

23. O que é exploração no sistema?

É reservar oportunidades controladas para aprender sobre opções pouco observadas. Eu definiria limites e elegibilidade, pois experimentar não autoriza expor clientes a produtos inadequados ou ignorar riscos financeiros relevantes.

Termo para lembrar: Controlled exploration | Base técnica: referência 1.

24. Como explicar uma recomendação ao usuário?

Usaria motivos verdadeiros e compreensíveis, derivados do processo permitido, sem inventar causalidade. A explicação não deve revelar dados sensíveis nem prometer aprovação de crédito quando apenas sugere uma opção.

Termo para lembrar: Recommendation explanation | Base técnica: referência 1.

25. Como testar exclusão de dados pessoais?

Verificaria fontes, features, índices e caches afetados, conforme política aplicável. Remover cadastro não garante remoção dos derivados usados em recomendação; a arquitetura precisa conhecer lineage e ciclos de atualização.

Termo para lembrar: Model data governance | Base técnica: referência 1.

26. Modelo novo precisa recalcular todo catálogo?

Depende de compatibilidade dos embeddings e índices. Eu planejaria versões paralelas e troca controlada, testando qualidade e rollback; misturar vetores de espaços incompatíveis pode produzir candidatos sem significado.

Termo para lembrar: Embedding migration | Base técnica: referência 1.

27. Como lidar com item patrocinado?

Identificaria a regra de inserção e transparência exigida, preservando elegibilidade e guardrails. Monetização não deve ser confundida com relevância técnica, e métricas precisam separar efeito comercial de valor ao usuário.

Termo para lembrar: Sponsored ranking | Base técnica: referência 1.

28. Como avaliar recomendações quando conversões são canceladas semanas depois?

Eu separaria sinais provisórios de resultados consolidados e acompanharia coortes com janela suficiente para observar cancelamentos. Treinar apenas com a conversão inicial favorece recomendações que parecem boas cedo; manteria versões dos rótulos e comparações alinhadas ao mesmo grau de maturação.

Termo para lembrar: Delayed outcome labels

29. Como detectar que ranking mudou por erro de dados?

Monitoraria distribuição das features, candidatos e scores junto de resultados. Versionamento e comparação com baseline ajudam localizar a etapa; sem isso, queda de negócio pode ser atribuída incorretamente ao modelo.

Termo para lembrar: Ranking observability | Base técnica: referência 1.

30. Como fechar o design com uma escolha defensável?

Eu apresentaria baseline, restrições de elegibilidade, recuperação de candidatos, ranking e feedback. Explicaria métricas e fallback, deixando claro qual evidência justificaria modelo mais sofisticado e maior custo operacional.

Termo para lembrar: Recommendation rationale | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Retrieval em duas etapas versus pontuar catálogo inteiro

Ganho. Reduz custo e latência do ranking

Custo e motivo. Pode perder candidatos antes da etapa final

Como decidir. Quando catálogo é grande e modelo final é caro

02. Filtro autoritativo versus somente score

Ganho. Preserva regras atuais do produto

Custo e motivo. Adiciona consulta e pode reduzir opções

Como decidir. Quando elegibilidade tem consequência financeira

03. Baseline geral versus personalização imediata

Ganho. Funciona com pouco histórico

Custo e motivo. Oferece menor adaptação individual

Como decidir. Quando cliente ou item é novo

04. Exploração versus aproveitar favoritos conhecidos

Ganho. Produz aprendizado e descoberta

Custo e motivo. Pode reduzir métrica imediata de relevância

Como decidir. Quando há espaço seguro para testar opções

05. Teste online versus avaliação só offline

Ganho. Mede efeito real na jornada

Custo e motivo. Consome tempo e expõe população limitada

Como decidir. Quando ganho precisa ser demonstrado causalmente

06. Reindexação frequente versus em lote

Ganho. Reduz staleness de catálogo

Custo e motivo. Aumenta processamento e operação

Como decidir. Quando disponibilidade muda rapidamente

07. Fallback simples versus depender do ranker

Ganho. Preserva resposta durante falha

Custo e motivo. Reduz qualidade de personalização

Como decidir. Quando disponibilidade da jornada importa

08. Diversidade versus ranking por score único

Ganho. Amplia variedade útil

Custo e motivo. Pode reduzir score imediato médio

Como decidir. Quando descoberta e cobertura são objetivos

09. Minimização de features versus sinal amplo

Ganho. Reduz exposição e governança

Custo e motivo. Pode limitar capacidade preditiva

Como decidir. Quando privacidade e finalidade restringem dados

10. Rollout gradual versus troca total

Ganho. Limita impacto e permite comparação

Custo e motivo. Mantém versões e coordenação adicionais

Como decidir. Quando mudança pode afetar segmentos de forma diferente

Duas alternativas ao caminho proposto

1. Regras e popularidade

Quando faz sentido. Pouco histórico ou catálogo simples

Como implementar. Ordenar opções elegíveis por critérios claros

Limites. Personalização limitada

2. Retrieval e ranking aprendidos

Quando faz sentido. Há dados e ganho demonstrável

Como implementar. Treinar candidatos e ranking com avaliações

Limites. Maior complexidade e vieses

Referências e alcance da evidência

1. Google: recomendações

Problema de recomendação, candidatos e ranking. As respostas e parâmetros de aplicação são propostas autorais.

https://developers.google.com/machine-learning/recommendation/overview

2. TensorFlow: retrieval

Modelo de recuperação de candidatos. As respostas e parâmetros de aplicação são propostas autorais.

https://www.tensorflow.org/recommenders/examples/basic_retrieval

Continue em: T109 · T111

T111 / L. Alcance Staff+/Principal

RFCs e ADRs

RFCs propõem mudanças; ADRs preservam contexto, decisão e consequências para evolução técnica. Arquitetura de treino; não descreve implementação interna do Nubank.

Quando usar. Decisões com impacto duradouro ou coordenação entre equipes.

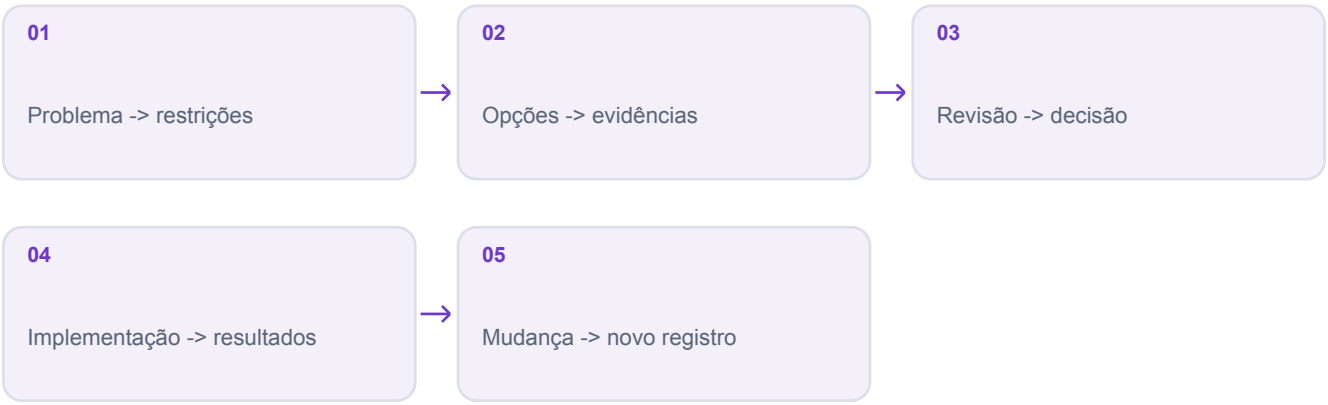
Quando evitar ou limitar. Documentar apenas a tecnologia escolhida ou transformar toda mudança em burocracia.

Uma abertura possível

Eu começaria por registrar problema restrições e critérios de sucesso. Depois explicaria comparar alternativas viáveis pelo mesmo critério, preservando as restrições do caso.

Peças e tecnologias. Markdown · Git · RFC · ADR · Decision log

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Registrar problema restrições e critérios de sucesso; validar ligação entre decisão e objetivo.

2. Comparar alternativas viáveis pelo mesmo critério; validar razões de descarte verificáveis.

3. Explicitar obrigações operacionais e limitações; validar custo e risco previstos.

4. Definir responsável e processo de escalonamento; validar prazo e autoridade da decisão.

5. Testar hipóteses críticas com carga representativa; validar invariantes e SLO medidos.

30 perguntas para sustentar o desenho

01. O que precisa entrar antes da solução?

Começaria pelo problema, restrições e resultado esperado, separando fatos de hipóteses. Sem esse contexto, o documento vira justificativa de preferência; cada escolha precisa poder ser relacionada ao objetivo que motivou a mudança.

Termo para lembrar: Context | Base técnica: referência 1.

02. Preciso registrar opções descartadas?

Compararia alternativas viáveis usando os mesmos critérios e registraria por que foram descartadas agora. Isso evita fingir inevitabilidade e permite revisão quando volume, equipe ou exigência de consistência mudarem depois.

Termo para lembrar: Alternatives | Base técnica: referência 2.

03. Como mostrar custo da decisão?

Descreveria obrigações de operação, manutenção, migração e limitações aceitas, além do benefício. Uma decisão precisa deixar seus custos visíveis, para que quem implementar e operar não descubra a consequência somente após adoção.

Termo para lembrar: Consequences | Base técnica: referência 1.

04. Quem decide quando não há consenso?

Definiria responsável pela decisão e forma de resolver divergências dentro de um prazo. Ouvir afetados é necessário, mas unanimidade não é sempre viável; o processo deve preservar evidência e accountability sem paralisar a entrega.

Termo para lembrar: Ownership | Base técnica: referência 2.

05. Como validar escolha de banco?

Testaria a hipótese que realmente diferencia opções, com carga e garantias equivalentes. Eu documentaria limites do experimento, porque um benchmark artificial ou com durabilidade menor pode sustentar uma escolha inadequada ao requisito.

Termo para lembrar: Evidence | Base técnica: referência 1.

06. Toda decisão merece o mesmo esforço?

Aprofundaria a análise proporcionalmente ao impacto e ao custo de desfazer, incluindo dados e efeitos externos. Nem toda mudança merece RFC extenso, mas uma aparente troca de código pode esconder irreversibilidade importante.

Termo para lembrar: Reversibility | Base técnica: referência 2.

07. Quem precisa revisar o RFC?

Chamaria quem possui contratos, operação e requisitos afetados, usando inventário para localizar consumidores. Revisão não depende apenas de interessados espontâneos; uma equipe ausente pode carregar a maior consequência da escolha.

Termo para lembrar: Review | Base técnica: referência 1.

08. ADR antigo ainda é regra vigente?

Marcaria decisões propostas, aceitas, substituídas e abandonadas de forma visível, ligando registros relacionados. O histórico permanece útil, mas quem chega agora precisa reconhecer qual orientação está vigente e por quê.

Termo para lembrar: Status | Base técnica: referência 2.

09. RFC precisa descrever implantação?

Incluiria etapas de adoção, métricas, compatibilidade e recuperação proporcionais ao risco. Uma boa arquitetura final pode falhar na transição; aprovação do desenho não elimina trabalho de migrar usuários, dados e operação.

Termo para lembrar: Rollout | Base técnica: referência 1.

10. Decisão termina quando documento foi aprovado?

Voltaria às hipóteses após implementação para comparar benefício e custo reais. Quando evidência muda, revisaria a decisão explicitamente; documento aprovado não deve virar argumento para manter uma solução que deixou de atender ao problema.

Termo para lembrar: Follow-up | Base técnica: referência 2.

11. Qual diferença entre RFC e ADR?

Eu usaria RFC para discutir uma proposta e ADR para registrar uma decisão relevante com contexto e consequências. O formato é adaptável; o importante é distinguir discussão aberta de orientação vigente.

Termo para lembrar: Proposal versus decision | Base técnica: referência 1.

12. ADR deve ser um documento enorme?

Não. Eu dimensionaria detalhe pela consequência e reversibilidade. Um registro curto pode bastar para escolha localizada, enquanto mudança entre domínios precisa de evidências, rollout e responsabilidades mais explícitas.

Termo para lembrar: Proportional documentation | Base técnica: referência 1.

13. Como documentar uma hipótese ainda não validada?

Marcaria como hipótese, com teste, responsável e prazo para revisão. Não apresentaria estimativa como fato; a decisão pode ser provisória e depender de um resultado mensurável antes da expansão.

Termo para lembrar: Decision uncertainty | Base técnica: referência 1.

14. Como tratar dois ADRs vigentes que orientam escolhas incompatíveis?

Eu verificaria escopo, premissas e datas antes de presumir conflito. Se ambos governam a mesma decisão, registraria uma resolução explícita com responsável e vínculos de substituição; deixar cada equipe escolher seu documento favorito produz comportamento divergente sem decisão consciente.

Termo para lembrar: Decision consistency

15. Como compartilhar um ADR sem expor informações sensíveis?

Eu separaria o raciocínio necessário para revisão de credenciais, dados de clientes e detalhes exploráveis de segurança. Publicaria uma versão adequada à audiência e manteria anexos restritos quando indispensáveis; transparência da decisão não exige distribuir todos os dados que a embasaram.

Termo para lembrar: Decision-record confidentiality

16. Como documentar a evolução de um contrato público no RFC?

Eu incluiria uma matriz de versões produtoras e consumidoras, exemplos de mudanças compatíveis e critérios para retirar o formato antigo. O revisor precisa enxergar quem continua funcionando em cada etapa, incluindo clientes atrasados e replay, não apenas o schema final desejado.

Termo para lembrar: Contract compatibility matrix

17. Quando atualizar um ADR existente?

Correções pequenas podem ser explicitadas, mas mudança de decisão merece vínculo com novo registro e status de substituição. Eu preservaria o raciocínio histórico para explicar por que a arquitetura evoluiu.

Termo para lembrar: Decision history | Base técnica: referência 1.

18. Como ligar decisão a código e operação?

Referenciaria implementação, contratos, métricas e runbooks relevantes, com ownership. Documento isolado perde valor; quem altera o sistema deve conseguir localizar a decisão e verificar se suas hipóteses continuam válidas.

Termo para lembrar: Traceability | Base técnica: referência 1.

19. Qual risco de documentar depois de tudo pronto?

Pode transformar escolhas contingentes em justificativa retrospectiva sem alternativas reais. Eu registraria decisões importantes durante o processo, preservando incerteza e evidências que existiam no momento, sem exigir burocracia para cada detalhe.

Termo para lembrar: Decision timing | Base técnica: referência 1.

20. Como lidar com decisão emergencial em incidente?

Priorizaria contenção segura e registraria contexto mínimo, responsável e reversibilidade. Depois completaria a análise e revisaria a solução temporária, evitando que urgência vire arquitetura permanente sem avaliação.

Termo para lembrar: Emergency decision | Base técnica: referência 1.

21. RFC deve incluir custo operacional?

Sim, incluindo plantão, migração, manutenção e capacidade da equipe. Uma solução tecnicamente elegante pode ser inadequada se ninguém consegue operá-la; eu compararia custo total com o benefício esperado.

Termo para lembrar: Operational consequences | Base técnica: referência 1.

22. Como evitar RFC virar disputa de preferência?

Estabeleceria critérios e exemplos de carga ou falha antes de comparar ferramentas. Quando houver incerteza, faria experimento limitado, transformando opiniões em hipóteses observáveis sem fingir precisão inexistente.

Termo para lembrar: Criteria-driven review | Base técnica: referência 1.

23. Quem revisa uma mudança de evento público?

Produtor, consumidores relevantes e responsáveis por operação ou governança afetada. Eu usaria inventário e contratos para localizar dependências, evitando confiar apenas em quem participou espontaneamente da discussão.

Termo para lembrar: Stakeholder coverage | Base técnica: referência 1.

24. Uma decisão reversível dispensa análise?

Não dispensa, mas permite análise proporcional e experimento menor. Eu verificaria se é realmente reversível, incluindo dados e efeitos externos; trocar código de volta pode não desfazer compromissos já assumidos.

Termo para lembrar: Reversibility assessment | Base técnica: referência 1.

25. Como expressar gatilho de revisão?

Definiria condição concreta, como atingir limite de capacidade, custo ou requisito novo. Assim a equipe sabe quando reavaliar, em vez de tratar a escolha como eterna ou discutir novamente a cada preferência.

Termo para lembrar: Review trigger | Base técnica: referência 1.

26. ADR precisa ter prazo de validade?

Nem sempre, mas precisa de contexto e status. Eu programaria revisão quando hipóteses são voláteis ou risco é alto, garantindo que mudanças relevantes possam invalidar a decisão de forma visível.

Termo para lembrar: Decision lifecycle | Base técnica: referência 1.

27. Como registrar risco aceito?

Descreveria consequência, mitigação, limite e responsável pela aceitação, conforme alçada. Isso não elimina o risco; permite acompanhar exposição e impede que uma limitação importante fique escondida numa observação vaga.

Termo para lembrar: Risk acceptance | Base técnica: referência 1.

28. Qual sinal mostra documentação excessiva?

Quando custo de produzir e revisar não melhora decisões ou transferência de contexto. Eu reduziria formato e duplicações, medindo utilidade real, sem abandonar registros de escolhas difíceis de desfazer.

Termo para lembrar: Documentation value | Base técnica: referência 1.

29. Como usar ADR numa entrevista?

Eu resumiria contexto, escolha, alternativa e consequência em voz alta. Não preciso escrever um documento completo; a estrutura ajuda demonstrar raciocínio e reconhecer qual mudança de requisito alteraria minha decisão.

Termo para lembrar: Interview decision framing | Base técnica: referência 1.

30. Qual resultado espera de uma boa RFC?

Uma decisão compreensível, implementável e verificável, com responsáveis e caminho de adoção. Aprovação do texto não é o resultado final; o sistema precisa entregar o benefício sem esconder custos assumidos.

Termo para lembrar: Decision outcome | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. ADR curto versus RFC extenso

Ganho. Reduz esforço para decisões localizadas

Custo e motivo. Oferece menos espaço para coordenação complexa

Como decidir. Quando impacto e reversibilidade são limitados

02. Alternativas explícitas versus registrar só vencedor

Ganho. Permite revisão e demonstra julgamento

Custo e motivo. Exige tempo para comparação honesta

Como decidir. Quando decisão tem consequências duradouras

03. Critérios prévios versus escolha por preferência

Ganho. Torna avaliação verificável

Custo e motivo. Pode exigir alinhamento antes do experimento

Como decidir. Quando várias opções são plausíveis

04. Responsável final versus unanimidade

Ganho. Evita bloqueio indefinido

Custo e motivo. Exige legitimidade e registro de divergências

Como decidir. Quando consenso completo não é viável

05. Experimento versus decisão apenas documental

Ganho. Produz evidência sobre hipótese crítica

Custo e motivo. Consome tempo e capacidade de implementação

Como decidir. Quando incerteza muda a escolha

06. Registro imutável de decisão versus edição contínua

Ganho. Preserva contexto histórico

Custo e motivo. Exige navegar versões e substituições

Como decidir. Quando auditoria do raciocínio importa

07. Revisão ampla versus grupo pequeno

Ganho. Revela impactos entre domínios

Custo e motivo. Aumenta coordenação e prazo

Como decidir. Quando mudança afeta vários contratos

08. Status explícito versus documentação sem ciclo

Ganho. Reduz uso de orientação obsoleta

Custo e motivo. Exige manutenção do registro

Como decidir. Quando arquitetura evolui continuamente

09. Plano de adoção versus somente desenho final

Ganho. Torna mudança executável e recuperável

Custo e motivo. Amplia escopo da análise

Como decidir. Quando migração possui risco significativo

10. Revisão por gatilho versus revisão frequente fixa

Ganho. Foca esforço em mudança relevante

Custo e motivo. Pode depender de sinais bem monitorados

Como decidir. Quando hipóteses têm limites observáveis

Duas alternativas ao caminho proposto

1. ADR curto

Quando faz sentido. Decisão localizada e reversível

Como implementar. Registrar contexto escolha e consequências

Limites. Não substitui desenho complexo

2. RFC com experimento

Quando faz sentido. Há incerteza e vários envolvidos

Como implementar. Propor opções e validar hipótese antes de decidir

Limites. Demanda tempo e responsável claro

Referências e alcance da evidência

1. Azure: ADR

Registro de contexto, decisão e consequências. As respostas e parâmetros de aplicação são propostas autorais.

https://learn.microsoft.com/en-us/azure/well-architected/architect-role/architecture-decision-record

2. AWS: excelência operacional

Evolução, operação e aprendizagem. As respostas e parâmetros de aplicação são propostas autorais.

https://docs.aws.amazon.com/wellarchitected/latest/framework/oe-design-principles.html

Continue em: T110 · T112

T112 / L. Alcance Staff+/Principal

Build versus buy

Build versus buy compara capacidade de entregar e operar ao longo do tempo. Arquitetura de treino; não descreve implementação interna do Nubank.

Quando usar. Escolha de plataforma, serviço gerenciado ou implementação própria.

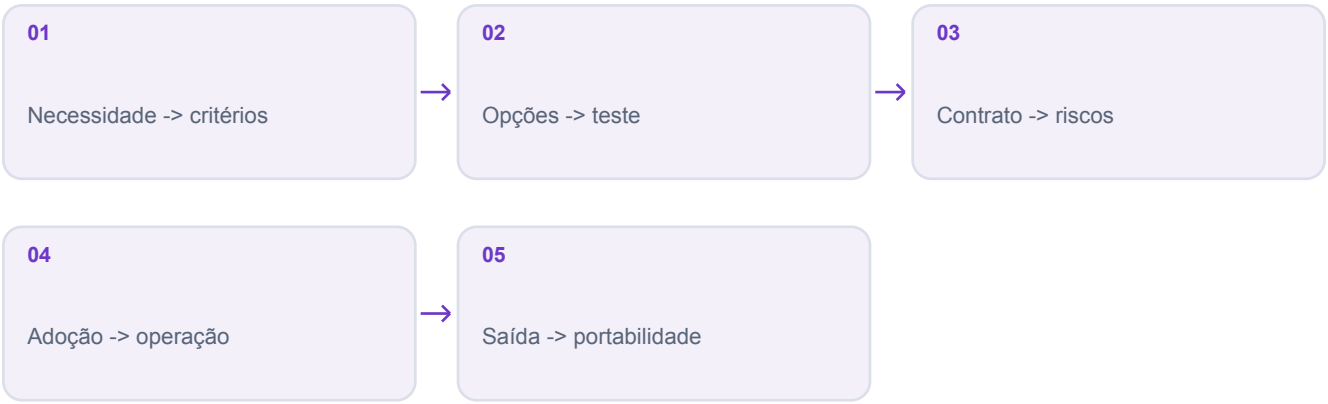
Quando evitar ou limitar. Comparar apenas preço mensal ou assumir que comprar elimina responsabilidade.

Uma abertura possível

Eu começaria por separar diferencial de negócio e infraestrutura comum. Depois explicaria incluir operação integração migração e saída, preservando as restrições do caso.

Peças e tecnologias. Proof of concept · TCO · Vendor assessment · Adapters

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Separar diferencial de negócio e infraestrutura comum; validar benefício específico da customização.

2. Incluir operação integração migração e saída; validar cenários de utilização futura.

3. Mapear dependências e comportamento ponta a ponta; validar falhas e limites do contrato.

4. Isolar contratos onde houver valor concreto; validar teste real de substituição.

5. Validar exportação formato e completude antes; validar ensaio de exportação e restauração.

30 perguntas para sustentar o desenho

01. O que realmente precisa ser próprio?

Separaria capacidades que diferenciam o produto das necessidades comuns já atendidas por soluções maduras. Construir precisa justificar benefício próprio e custo contínuo, incluindo operação; domínio interessante por si só não prova vantagem competitiva.

Termo para lembrar: Differentiation | Base técnica: referência 1.

02. Licença menor significa opção barata?

Compararia custo total no horizonte relevante, incluindo integração, equipe, incidentes, migração e saída. Licença menor pode esconder manutenção maior; projetaria cenários de crescimento e redução de demanda antes de concluir.

Termo para lembrar: TCO | Base técnica: referência 2.

03. SLA do fornecedor garante nosso SLO?

Mapearia a promessa do fornecedor para a jornada real, observando exclusões, quotas e suporte. SLA contratual não garante automaticamente nosso SLO nem compensa todo impacto de negócio; a integração também pode falhar.

Termo para lembrar: Contract | Base técnica: referência 1.

04. Adapter elimina lock-in completamente?

Usaria adapters quando estabilizam contratos relevantes e tornam testes ou troca plausíveis. Não prometeria eliminar lock-in por abstração, porque semântica, dados e operação podem continuar específicos mesmo com uma interface genérica.

Termo para lembrar: Portability | Base técnica: referência 2.

05. Como sair com histórico e evidências?

Testaria exportação e reconstrução de uma amostra fora do produto, preservando identidade e significado. Um botão de exportar não prova portabilidade; histórico e dependências precisam ser utilizáveis para tornar a saída viável.

Termo para lembrar: Data export | Base técnica: referência 1.

06. Gerenciado significa sem plantão?

Definiria quem configura, monitora, responde e escala incidentes entre equipe e fornecedor. Serviço gerenciado reduz certas tarefas, mas a responsabilidade pela jornada permanece; runbooks precisam funcionar com acessos e suporte reais.

Termo para lembrar: Operational burden | Base técnica: referência 2.

07. Depender de feature prometida é seguro?

Basearia compromisso crítico em capacidade disponível ou obrigação verificável, tratando roadmap como incerteza. Se a funcionalidade futura é essencial, manteria alternativa ou condição explícita em vez de assumir entrega prometida informalmente.

Termo para lembrar: Roadmap | Base técnica: referência 1.

08. Comprar produto resolve conformidade sozinho?

Avaliaria o fluxo concreto de dados, identidades e retenção no nosso uso. Certificação e controles do fornecedor são evidências úteis, mas não demonstram que permissões, configuração e integração escolhidas atendem às necessidades específicas.

Termo para lembrar: Security | Base técnica: referência 2.

09. Preço por requisição cresce linearmente para sempre?

Simularia as unidades faturadas com picos, chamadas auxiliares e crescimento, incluindo limites e descontos possíveis. Preço por requisição pode mudar a economia quando retries, fanout ou egress multiplicam o trabalho necessário por operação útil.

Termo para lembrar: Scale economics | Base técnica: referência 1.

10. Plano de saída precisa ser implementado agora?

Provaria os pontos de saída mais difíceis proporcionalmente ao risco, preservando dados e contratos necessários. Não é preciso operar dois fornecedores sempre, mas um plano puramente textual pode falhar quando a migração se torna urgente.

Termo para lembrar: Exit plan | Base técnica: referência 2.

11. Quando construir é uma decisão responsável?

Quando requisitos ou diferenciação justificam custo contínuo e a organização possui capacidade para manter. Eu incluiria operação, segurança e evolução no plano, sem comparar apenas o esforço de uma primeira implementação.

Termo para lembrar: Build rationale | Base técnica: referência 1.

12. Quando comprar é mais adequado?

Quando uma capacidade comum atende requisitos e libera equipe para trabalho de maior valor. Eu validaria contrato, integração e saída, porque serviço gerenciado reduz algumas tarefas, mas não transfere toda responsabilidade.

Termo para lembrar: Buy rationale | Base técnica: referência 1.

13. Como avaliar fornecedor sem acreditar só na demonstração?

Executaria prova com dados e cenários representativos, incluindo falhas, exportação e limites. Também conversaria sobre suporte e operação; uma apresentação feliz não demonstra capacidade de cumprir nossa jornada real.

Termo para lembrar: Vendor validation | Base técnica: referência 1.

14. Lock-in é sempre um erro?

Não. Pode ser uma troca consciente por velocidade ou capacidade. Eu estimaria custo de saída e irreversibilidades, evitando abstrações caras que prometem portabilidade sem serem testadas contra outro fornecedor.

Termo para lembrar: Conscious lock-in | Base técnica: referência 1.

15. Como calcular custo de oportunidade de construir?

Estimaria quais entregas e capacidades serão adiadas pela equipe ocupada na construção e operação. O valor não está só na folha salarial; também inclui tempo até benefício e risco de execução.

Termo para lembrar: Opportunity cost | Base técnica: referência 1.

16. Serviço open source é gratuito operacionalmente?

Não. Licença pode não ter cobrança, mas implantação, atualizações, plantão e especialização custam. Eu compararia essas obrigações com serviço gerenciado e capacidade existente, incluindo o que ocorre durante incidentes.

Termo para lembrar: Open-source TCO | Base técnica: referência 1.

17. Quem deveria controlar as chaves dos dados armazenados no fornecedor?

Eu compararia chaves geridas pelo fornecedor e pelo cliente conforme ameaça, operação e requisitos de acesso. Controle próprio pode facilitar revogação, mas cria risco de indisponibilidade ou perda; validaria backup, rotação e efeitos reais da revogação sobre cópias existentes.

Termo para lembrar: Encryption-key ownership

18. SLA do fornecedor paga o prejuízo do negócio?

Créditos contratuais podem não cobrir impacto real. Eu avaliaria indisponibilidade, suporte e contingência da jornada, distinguindo compensação comercial da capacidade técnica de continuar ou recuperar o serviço.

Termo para lembrar: Contractual SLA | Base técnica: referência 1.

19. Como avaliar limites de capacidade de um fornecedor antes de contratar?

Eu verificaria quotas, burst, concorrência e processo de aumento, testando uma carga representativa dentro das condições autorizadas. Uma demonstração funcional não prova capacidade de pico; distinguiria limite configurável, limite estrutural e compromisso contratual para evitar dependência impossível de escalar.

Termo para lembrar: Vendor capacity validation

20. Quando um adapter vale o custo?

Quando estabiliza contratos relevantes e facilita teste ou troca provável. Eu não esconderia todas as diferenças entre fornecedores, porque uma abstração genérica pode limitar capacidades úteis e criar falsa equivalência.

Termo para lembrar: Abstraction economics | Base técnica: referência 1.

21. Como avaliar risco de descontinuação?

Consideraria exportação, prazo de migração, dependências e alternativas, proporcionalmente à criticidade. Não preciso prever o futuro com certeza; preciso reduzir custo de reação e evitar dependências irreversíveis sem justificativa.

Termo para lembrar: Vendor continuity | Base técnica: referência 1.

22. Comprar solução exige threat modeling?

Sim. A integração cria fluxos de dados, identidades e permissões próprios. Eu avaliaria configuração, retenção e limites de confiança; certificações do fornecedor não provam que nosso uso específico está seguro.

Termo para lembrar: Shared security | Base técnica: referência 1.

23. Como comparar preço por usuário e por operação?

Projetaria cenários reais de crescimento, ociosidade e picos, usando a unidade faturada correta. Também incluiria integrações, suporte e egress, para não escolher pela menor linha de preço inicial.

Termo para lembrar: Pricing model | Base técnica: referência 1.

24. Fornecedor caiu: podemos migrar instantaneamente?

Geralmente não sem preparação e contrato equivalente. Eu definiria degradação e recuperação realistas, testando exportação e dependências; multivendor pode ajudar, mas também adiciona custo e diferenças de comportamento.

Termo para lembrar: Exit readiness | Base técnica: referência 1.

25. Quando multivendor não compensa?

Quando o custo de manter duas integrações corretas supera o risco mitigado ou existe dependência comum. Eu compararia falhas independentes, semântica e volume, evitando duplicar sistemas apenas para parecer resiliente.

Termo para lembrar: Multivendor trade-off | Base técnica: referência 1.

26. Como testar suporte do fornecedor?

Exercitaria caminhos de escalonamento e verificaria tempos, acesso e evidências necessárias. O contrato precisa funcionar durante incidente; descobrir que ninguém tem permissão de abrir chamado crítico é uma falha operacional.

Termo para lembrar: Support readiness | Base técnica: referência 1.

27. Qual papel de compliance na escolha?

Especialistas validam requisitos aplicáveis e evidências, enquanto engenharia demonstra fluxos e controles. Eu não usaria conformidade como selo genérico nem assumiria que um contrato resolve decisões técnicas de acesso e retenção.

Termo para lembrar: Governance collaboration | Base técnica: referência 1.

28. Pode começar comprado e construir depois?

Sim, se houver estratégia de migração e benefício futuro justificável. Eu preservaria contratos e dados necessários, evitando construir abstrações especulativas demais antes de entender o domínio e a demanda real.

Termo para lembrar: Staged sourcing | Base técnica: referência 1.

29. Como rever a decisão após crescimento?

Compararia custo por operação, limitações e esforço de operação com as hipóteses originais. Crescimento pode alterar a economia, mas a migração também custa; decidiria pelo benefício líquido e risco de transição.

Termo para lembrar: Decision reassessment | Base técnica: referência 1.

30. Qual fala final demonstra julgamento?

Eu escolheria a opção que atende requisitos e capacidade organizacional hoje, explicando custos aceitos e gatilhos de revisão. Construir e comprar são estratégias contextuais, não sinais automáticos de senioridade.

Termo para lembrar: Sourcing judgment | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Construir versus comprar

Ganho. Permite adaptação e controle específicos

Custo e motivo. Exige equipe e manutenção contínuas

Como decidir. Quando diferenciação ou requisitos justificam investimento

02. Gerenciado versus self-hosted

Ganho. Reduz certas tarefas operacionais

Custo e motivo. Aumenta dependência e pode elevar preço variável

Como decidir. Quando capacidade operacional é limitada

03. Preço fixo versus por consumo

Ganho. Melhora previsibilidade financeira

Custo e motivo. Pode pagar capacidade não utilizada

Como decidir. Quando demanda é estável e compromisso compensa

04. Adapter específico versus abstração universal

Ganho. Preserva capacidades e semântica úteis

Custo e motivo. Torna troca de fornecedor mais trabalhosa

Como decidir. Quando portabilidade genérica não é requisito real

05. Teste de saída versus confiar no export

Ganho. Demonstra recuperabilidade dos dados

Custo e motivo. Consome tempo antes da adoção

Como decidir. Quando dependência é crítica e difícil de desfazer

06. SLA premium versus suporte básico

Ganho. Pode melhorar resposta a incidentes

Custo e motivo. Aumenta custo contratado

Como decidir. Quando tempo de escalonamento tem impacto relevante

07. Adotar capacidade atual versus esperar roadmap

Ganho. Reduz incerteza de entrega

Custo e motivo. Pode limitar funcionalidade ou atrasar objetivo

Como decidir. Quando promessa futura é essencial mas não verificável

08. Controles próprios versus delegação ampla

Ganho. Permite adequar uso ao risco do produto

Custo e motivo. Adiciona implementação e operação

Como decidir. Quando fornecedor não cobre toda responsabilidade

09. Multivendor versus fornecedor único

Ganho. Pode reduzir dependência específica

Custo e motivo. Duplica integração, testes e diferenças semânticas

Como decidir. Quando independência real justifica o custo

10. Saída preparada versus otimização máxima no fornecedor

Ganho. Reduz custo de reação futura

Custo e motivo. Pode abrir mão de recursos exclusivos úteis

Como decidir. Quando continuidade e portabilidade têm alto valor

Duas alternativas ao caminho proposto

1. Serviço gerenciado

Quando faz sentido. Capacidade comum e equipe limitada

Como implementar. Contratar mantendo limites e estratégia de saída

Limites. Dependência e custo variável

2. Implementação própria

Quando faz sentido. Diferenciação e requisitos justificam

Como implementar. Construir com ownership e orçamento operacional

Limites. Manutenção contínua e oportunidade perdida

Referências e alcance da evidência

1. AWS: cost optimization

Avaliação de custo e eficiência arquitetural. As respostas e parâmetros de aplicação são propostas autorais.

https://docs.aws.amazon.com/wellarchitected/latest/cost-optimization-pillar/welcome.html

2. Azure: ADR

Registro de contexto, decisão e consequências. As respostas e parâmetros de aplicação são propostas autorais.

https://learn.microsoft.com/en-us/azure/well-architected/architect-role/architecture-decision-record

Continue em: T111 · T113

T113 / L. Alcance Staff+/Principal

FinOps, TCO e custo por operação

FinOps conecta custo técnico ao valor e à capacidade entregue com responsabilidade compartilhada. Arquitetura de treino; não descreve implementação interna do Nubank.

Quando usar. Quando escala, desperdício ou escolhas de infraestrutura alteram economia do produto.

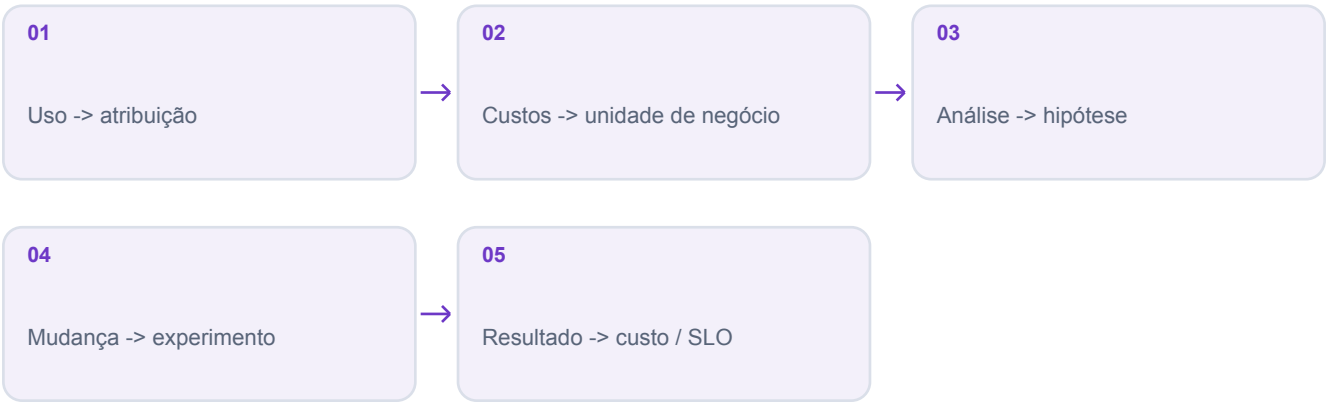
Quando evitar ou limitar. Reduzir custo removendo durabilidade, segurança ou margem sem renegociar requisitos.

Uma abertura possível

Eu começaria por medir custo por operação útil e jornada. Depois explicaria definir critérios de atribuição explicáveis, preservando as restrições do caso.

Peças e tecnologias. FinOps · Cost allocation · Unit economics · Cloud billing

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Medir custo por operação útil e jornada; validar custo com mesmo resultado entregue.

2. Definir critérios de atribuição explicáveis; validar rastreio entre uso e cobrança.

3. Avaliar picos memória e margem de falha; validar SLO durante pico e recuperação.

4. Alinhar classe ao prazo de recuperação; validar consulta após arquivamento.

5. Contabilizar tráfego e operações entre fronteiras; validar bytes úteis e transferências reais.

30 perguntas para sustentar o desenho

01. Qual unidade de custo importa no banco?

Escolheria uma unidade ligada a resultado útil, como pagamento concluído ou caso resolvido, mantendo qualidade comparável. Custo por servidor ajuda diagnóstico, mas pode esconder retries, falhas e trabalho que não entrega valor ao cliente.

Termo para lembrar: Unit economics | Base técnica: referência 1.

02. Quem paga serviço compartilhado?

Definiria atribuição observável e explicável para serviços compartilhados, aceitando limites de precisão. A regra deve apoiar decisões e responsabilidade, não criar incentivos para deslocar custo entre equipes sem reduzir consumo real.

Termo para lembrar: Allocation | Base técnica: referência 2.

03. CPU média baixa prova excesso de recurso?

Analisaria picos, memória, dependências e capacidade após falha, não apenas CPU média. Rightsizing precisa preservar SLO e recuperação; recurso aparentemente ocioso pode ser a margem necessária para operar com segurança.

Termo para lembrar: Rightsizing | Base técnica: referência 1.

04. Histórico frio pode ir para camada barata?

Escolheria classe de armazenamento por acesso e prazo de recuperação, testando consulta após arquivamento. Economia por gigabyte não basta se custo de leitura ou espera torna impossível atender cliente e obrigações de histórico.

Termo para lembrar: Storage tiering | Base técnica: referência 2.

05. Por que integração multicloud custa muito?

Mapearia bytes e chamadas entre fronteiras cobradas, avaliando localidade e cópias desnecessárias. Multicloud e múltiplas regiões podem ter valor, mas a decisão precisa contabilizar tráfego e operação junto do benefício de resiliência.

Termo para lembrar: Egress | Base técnica: referência 1.

06. Reserva de capacidade sempre compensa?

Compararia demanda previsível e flexibilidade antes de assumir compromisso de capacidade. Desconto pode compensar uso estável, mas paga-se o risco de mudança de arquitetura ou queda de demanda durante o período contratado.

Termo para lembrar: Commitments | Base técnica: referência 2.

07. Instância barata serve para pagamento crítico?

Usaria máquinas interrompíveis em trabalho retomável e protegido, mantendo capacidade estável quando o prazo exige. O cálculo inclui recomputação e recuperação; preço baixo da instância não garante custo menor por resultado concluído.

Termo para lembrar: Spot | Base técnica: referência 1.

08. Logs podem dominar a fatura?

Ajustaria volume, cardinalidade e retenção segundo utilidade, preservando diagnóstico e evidências necessárias. Cortar logs indiscriminadamente pode reduzir fatura e aumentar incidentes; observaria custo e valor de cada sinal relevante.

Termo para lembrar: Telemetry cost | Base técnica: referência 2.

09. Como detectar aumento sem esperar mês fechar?

Acompanharia consumo e custo por origem com alertas acionáveis, sem esperar fechamento mensal. A reação depende do que pode ser reduzido com segurança; anomalia financeira precisa de responsável e contexto operacional.

Termo para lembrar: Cost anomaly | Base técnica: referência 1.

10. Como provar economia real?

Compararia custo normalizado antes e depois com SLO e demanda equivalentes, incluindo investimento da mudança. Menos tráfego ou mais falhas também barateiam a conta; economia real precisa preservar o resultado prometido.

Termo para lembrar: Optimization proof | Base técnica: referência 2.

11. Dobrar o tráfego dobra o custo de infraestrutura?

Não necessariamente. Existem capacidade ociosa, faixas de preço e degraus de provisionamento. Eu modelaria custo marginal por intervalo de demanda, incluindo o próximo gargalo. Uma média atual barata pode esconder uma expansão cara ao ultrapassar um limite de capacidade.

Termo para lembrar: Custo marginal

12. Créditos promocionais tornam um fornecedor mais barato no longo prazo?

Eu separaria preço contratado, descontos temporários e custo efetivo normalizado. Projetaria o cenário depois dos créditos e incluiria compromissos assumidos. Uma redução transitória de caixa não demonstra vantagem estrutural quando o produto precisa operar por vários anos.

Termo para lembrar: Custo efetivo

13. Quando reduzir réplica é perigoso?

Quando elimina margem para failover, picos ou manutenção. Eu testaria capacidade sobrevivente e SLO antes da mudança, porque utilização média baixa pode representar redundância necessária, não desperdício removível.

Termo para lembrar: Resilience capacity | Base técnica: referência 1.

14. Qual custo escondido existe em serverless?

Além da execução, considero chamadas, logs, transferência, orquestração e recursos auxiliares. Eu avaliaria o fluxo inteiro e o padrão de carga, evitando concluir que preço por invocação representa o custo total.

Termo para lembrar: Serverless economics | Base técnica: referência 1.

15. Compressão sempre economiza dinheiro?

Depende do equilíbrio entre CPU, storage e rede. Eu mediria custo e latência com dados reais, incluindo conteúdo pouco compressível; economizar bytes pode aumentar processamento e piorar a jornada.

Termo para lembrar: Compression economics | Base técnica: referência 1.

16. Como identificar egress desnecessário?

Mapearia fluxos e bytes entre regiões, zonas ou provedores conforme cobrança aplicável. Reduzir cópias ou aproximar processamento pode ajudar, mas eu preservaria requisitos de disponibilidade e residência dos dados.

Termo para lembrar: Data movement cost | Base técnica: referência 1.

17. Aumentar vCPUs pode custar mais que a própria máquina?

Pode, quando software é licenciado por núcleo ou capacidade provisionada. Eu incluiria licenças e regras contratuais no dimensionamento, avaliando consolidar, reduzir núcleos ou mudar produto. Otimizar preço de computação isoladamente pode ampliar a despesa total do serviço.

Termo para lembrar: Licenciamento

18. Como calcula o custo de sair de um serviço gerenciado?

Estimo exportação, transformação, armazenamento temporário, operação paralela e desenvolvimento de substituição. Também considero prazo e capacidade da equipe. Portabilidade declarada por usar formato aberto não elimina custos de transferência, semântica do produto e dependências operacionais acumuladas.

Termo para lembrar: Portabilidade econômica

19. Como reduzir o custo de consultas cobradas por volume varrido?

Eu verificaria particionamento, projeção de colunas e filtros efetivamente aplicados antes da leitura. Consultas que retornam poucas linhas ainda podem varrer muitos dados. Mediria bytes processados e recorrência, comparando materialização seletiva com seu custo de atualização e armazenamento.

Termo para lembrar: Cobrança por dados lidos

20. Como lidar com índice caro pouco utilizado?

Verificaria frequência e criticidade das consultas antes de remover. Um índice raro pode sustentar fechamento ou incidente; eu testaria planos e impacto nas escritas, mantendo caminho de recuperação da mudança.

Termo para lembrar: Index economics | Base técnica: referência 1.

21. Quando storage frio é inadequado?

Quando o prazo de leitura ou restauração não atende a jornada, mesmo com pouco acesso. Eu calcularia custo total de recuperação e comunicação ao usuário, não apenas preço por gigabyte armazenado.

Termo para lembrar: Archive suitability | Base técnica: referência 1.

22. FinOps deve ficar só no financeiro?

Não. Engenharia explica consumo e alternativas, produto define valor e financeiro apoia orçamento. Eu estabeleceria dados compartilhados para decidir, evitando metas de corte que incentivem violar confiabilidade ou segurança.

Termo para lembrar: Cross-functional FinOps | Base técnica: referência 1.

23. Como detectar desperdício de recursos esquecidos?

Usaria ownership, inventário e sinais de uso, com processo seguro de confirmação e remoção. Recurso sem tráfego aparente pode ser contingência; limpeza automatizada precisa entender exceções e criticidade.

Termo para lembrar: Resource lifecycle | Base técnica: referência 1.

24. Uma arquitetura multirregional tem custo justificável?

Depende de impacto, RTO, RPO e requisitos. Eu compararia estratégias de recuperação e testes, incluindo operação contínua, para demonstrar qual redução de risco é comprada pelo investimento adicional.

Termo para lembrar: Resilience economics | Base técnica: referência 1.

25. Qual métrica evita otimizar componente irrelevante?

Custo por jornada ou resultado de negócio ajuda ligar partes ao total. Eu identificaria maiores contribuições e elasticidade, evitando gastar semanas numa economia minúscula enquanto outro fluxo domina a despesa.

Termo para lembrar: Cost prioritization | Base técnica: referência 1.

26. Como controlar gasto sem bloquear crescimento legítimo?

Definiria orçamentos, alertas e limites por criticidade, distinguindo previsão de restrição obrigatória. Alguns serviços podem escalar com revisão; outros precisam de admissão rígida devido a quota ou risco financeiro.

Termo para lembrar: Budget policy | Base técnica: referência 1.

27. Migração para serviço barato deve incluir qual custo?

Integração, testes, coexistência, transferência de dados, treinamento e recuperação, além do preço final. Eu estimaria prazo de retorno e risco, evitando comparar somente a conta após uma migração idealizada.

Termo para lembrar: Migration TCO | Base técnica: referência 1.

28. Como impedir retry storm de virar problema financeiro?

Aplicaria orçamento de retries, limites e observabilidade de tentativas por intenção. O incidente consome computação e chamadas externas; proteger capacidade também reduz gasto sem precisar esperar a fatura chegar.

Termo para lembrar: Retry amplification cost | Base técnica: referência 1.

29. O que fazer se economia reduz SLO?

Eu trataria como mudança de requisito ou falha do experimento, não sucesso automático. A decisão precisa do responsável adequado e de evidência sobre impacto; custo menor sozinho não autoriza degradar promessa existente.

Termo para lembrar: Cost-quality decision | Base técnica: referência 1.

30. Como apresentar economia à liderança?

Mostraria custo normalizado, qualidade preservada, investimento e risco residual, com período comparável. Também explicaria incertezas e efeito esperado no crescimento, para que o benefício seja verificável e não apenas percentual isolado.

Termo para lembrar: Financial communication | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Custo por jornada versus por recurso

Ganho. Liga investimento ao valor entregue

Custo e motivo. Exige correlação entre vários serviços

Como decidir. Quando decisões precisam refletir resultado de negócio

02. Rateio por uso versus divisão igual

Ganho. Melhora incentivo e atribuição

Custo e motivo. Requer medição e convenções discutíveis

Como decidir. Quando consumo varia bastante entre equipes

03. Rightsizing agressivo versus folga

Ganho. Reduz custo de capacidade ociosa

Custo e motivo. Pode piorar picos e recuperação

Como decidir. Quando testes demonstram margem suficiente

04. Storage frio versus quente

Ganho. Diminui custo de retenção longa

Custo e motivo. Aumenta prazo e custo de recuperação

Como decidir. Quando acesso raro aceita espera

05. Localidade de dados versus distribuição ampla

Ganho. Reduz transferência e latência de certos fluxos

Custo e motivo. Pode limitar independência regional

Como decidir. Quando movimento de dados domina custo

06. Compromisso de uso versus on-demand

Ganho. Reduz preço unitário previsto

Custo e motivo. Cria obrigação financeira com demanda incerta

Como decidir. Quando utilização estável é provável

07. Spot versus capacidade estável

Ganho. Reduz preço de computação elegível

Custo e motivo. Exige retomada e tolerância a interrupção

Como decidir. Quando workload suporta perda de instância

08. Amostragem versus telemetria completa

Ganho. Reduz volume e custo

Custo e motivo. Pode perder detalhes de eventos raros

Como decidir. Quando cobertura restante atende diagnóstico

09. Hard budget versus alertas

Ganho. Contém gasto antes de ultrapassar limite

Custo e motivo. Pode interromper demanda legítima

Como decidir. Quando orçamento é restrição obrigatória

10. Otimização incremental versus migração ampla

Ganho. Reduz risco e investimento inicial

Custo e motivo. Pode ter ganho limitado

Como decidir. Quando desperdícios locais ainda são relevantes

Duas alternativas ao caminho proposto

1. Otimização de utilização

Quando faz sentido. Recursos ociosos dominam custo

Como implementar. Ajustar tamanho agenda e capacidade com métricas

Limites. Pode reduzir margem de recuperação

2. Mudança arquitetural

Quando faz sentido. Padrão de acesso torna recurso inadequado

Como implementar. Alterar armazenamento processamento ou contratos

Limites. Migração tem custo e risco próprios

Referências e alcance da evidência

1. FinOps: framework

Gestão colaborativa de custo e valor. As respostas e parâmetros de aplicação são propostas autorais.

https://www.finops.org/framework/

2. AWS: cost optimization

Avaliação de custo e eficiência arquitetural. As respostas e parâmetros de aplicação são propostas autorais.

https://docs.aws.amazon.com/wellarchitected/latest/cost-optimization-pillar/welcome.html

Continue em: T112 · T114

T114 / L. Alcance Staff+/Principal

Ownership, times e carga cognitiva

Ownership e organização influenciam contratos, operação e velocidade de mudança. Arquitetura de treino; não descreve implementação interna do Nubank.

Quando usar. Sistemas que atravessam equipes, países ou plataformas compartilhadas.

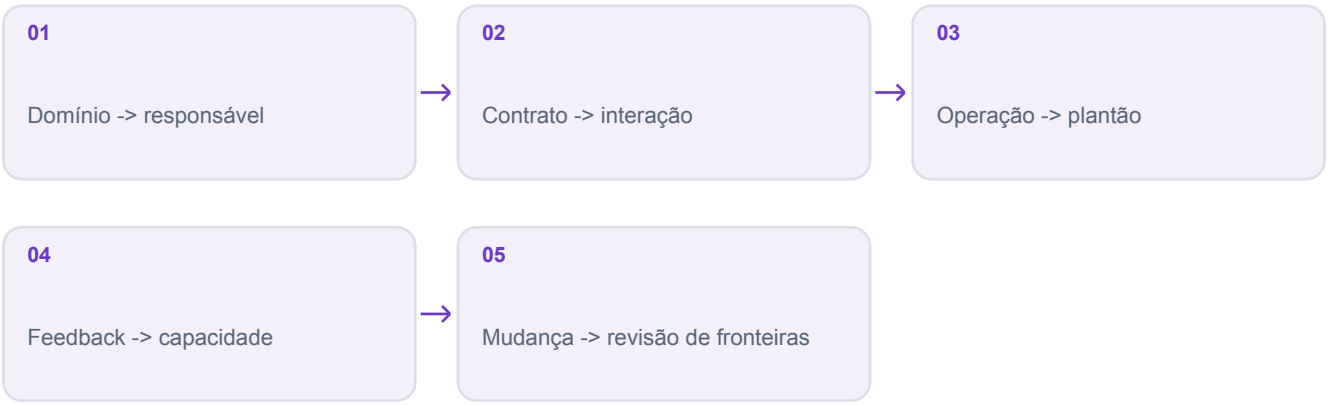
Quando evitar ou limitar. Desenhar organograma como verdade universal ou responsabilizar pessoas sem dar meios.

Uma abertura possível

Eu começaria por definir responsáveis por contrato e jornada. Depois explicaria observar dependências interrupções e tempo de mudança, preservando as restrições do caso.

Peças e tecnologias. Service catalog · On-call · RACI adaptado · Team Topologies

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Definir responsáveis por contrato e jornada; validar simulação de escalonamento entre equipes.

2. Observar dependências interrupções e tempo de mudança; validar métricas e relatos de operação.

3. Alinhar fronteiras quando benefício for real; validar mudanças organizacionais previstas.

4. Oferecer autosserviço com controles verificáveis; validar tempo de espera por dependência.

5. Definir responsabilidade com treinamento e suporte; validar exercícios e qualidade dos runbooks.

30 perguntas para sustentar o desenho

01. Quem responde por falha entre dois serviços?

Definiria responsáveis por contrato e resultado da jornada, com escalonamento entre equipes. Incidente na fronteira não pode ficar sem dono; autoridade e acesso devem acompanhar a responsabilidade de investigar e recuperar.

Termo para lembrar: Ownership | Base técnica: referência 1.

02. Como saber que time cuida de coisas demais?

Observaria interrupções, dependências, onboarding e dificuldade de mudança, combinando métricas e relatos. Carga cognitiva não é um número universal; a análise ajuda decidir se fronteiras, plataforma ou suporte precisam mudar.

Termo para lembrar: Cognitive load | Base técnica: referência 2.

03. Microsserviço precisa corresponder a uma squad?

Alinharia equipe e domínio quando isso reduz coordenação e preserva invariantes, sem tornar o organograma atual uma lei técnica. Times mudam; a arquitetura precisa de contratos que suportem evolução organizacional sem fragmentação arbitrária.

Termo para lembrar: Conway | Base técnica: referência 1.

04. Time de plataforma deve aprovar todo deploy?

Preferiria autosserviço com controles verificáveis para caminhos comuns, deixando revisão humana aos casos relevantes. Exigir aprovação central para tudo pode criar gargalo; autonomia precisa vir com defaults, observabilidade e responsabilidades claras.

Termo para lembrar: Platform interaction | Base técnica: referência 2.

05. Equipe que escreve precisa operar sempre sozinha?

Planejaria operação com treinamento, suporte, runbooks e capacidade sustentável. Quem desenvolve precisa compreender consequências, mas não deve ficar sozinho sem meios; confiabilidade inclui condições reais de manter e recuperar o serviço.

Termo para lembrar: On-call | Base técnica: referência 1.

06. Só uma pessoa conhece o ledger: e agora?

Distribuiria conhecimento por revisão, documentação e exercícios conduzidos por outras pessoas. A prova é alguém conseguir operar e recuperar sem o especialista, não apenas ter recebido uma apresentação ou acesso ao repositório.

Termo para lembrar: Knowledge concentration | Base técnica: referência 2.

07. O que automatizar primeiro?

Priorizaria automação de trabalho repetitivo, previsível e crescente, avaliando retorno e processo subjacente. Automatizar uma rotina errada pode ampliar o dano; primeiro esclareceria regra, exceções e responsabilidade pela manutenção.

Termo para lembrar: Toil | Base técnica: referência 1.

08. Times independentes podem mudar API livremente?

Estabeleceria compatibilidade, testes e depreciação para contratos entre equipes. Autonomia permite mudar internamente sem quebrar consumidores, e não ignorar dependências; decisões que alteram semântica precisam de coordenação proporcional ao impacto.

Termo para lembrar: Contracts | Base técnica: referência 2.

09. Transferência de serviço termina no repositório?

Transferência inclui plantão, permissões, decisões, métricas e incidentes conhecidos, com validação pelo novo responsável. Mudar nome do dono no catálogo não demonstra que a equipe consegue sustentar a operação recebida.

Termo para lembrar: Handoff | Base técnica: referência 1.

10. Como revisar fronteira que não funciona?

Revisaria fronteiras usando evidências de coordenação, tempo de entrega e incidentes, preservando continuidade. Reorganização pode ajudar, mas também perde contexto; eu exigiria benefício claro antes de mover responsabilidades repetidamente.

Termo para lembrar: Feedback | Base técnica: referência 2.

11. Ownership significa uma pessoa única responsável por tudo?

Não. Eu definiria time e responsabilidades claras, com substituição e escalonamento. Dependência de uma pessoa cria risco; autoridade precisa vir acompanhada de conhecimento, acesso e capacidade para operar.

Termo para lembrar: Team ownership | Base técnica: referência 1.

12. Como diferenciar toil de um trabalho técnico difícil, mas valioso?

Eu avaliaria repetição, possibilidade de automação, ausência de melhoria duradoura e crescimento proporcional ao serviço. Uma investigação difícil pode produzir conhecimento estrutural e não ser toil; classificar pelo incômodo ou pela senioridade de quem executa distorce a prioridade de engenharia.

Termo para lembrar: Toil classification | Base técnica: referência 2.

13. Quando juntar dois serviços no mesmo time?

Quando mudanças e operação são fortemente acopladas e a fronteira atual gera coordenação excessiva. Eu avaliaria domínio e capacidade, evitando reorganizar apenas porque duas tecnologias são parecidas.

Termo para lembrar: Team boundary | Base técnica: referência 1.

14. Time de plataforma deve construir tudo internamente?

Não. Seu papel é oferecer capacidades úteis e sustentáveis; pode integrar serviços gerenciados. Eu mediria redução de esforço dos consumidores, não quantidade de código ou ferramentas próprias produzidas.

Termo para lembrar: Platform sourcing | Base técnica: referência 1.

15. O que diferencia suporte de plataforma e gargalo central?

Autosserviço, contratos claros e responsabilidade pelo produto reduzem dependência de tickets. Eu investigaria solicitações repetidas para melhorar caminhos comuns, sem transferir todo trabalho operacional dos consumidores para uma equipe central.

Termo para lembrar: Service interaction | Base técnica: referência 1.

16. Como organizar plantão sem queimar a equipe?

Dimensionaria carga, treinamento, runbooks e escalonamento, atacando incidentes recorrentes e toil. Ownership não justifica disponibilidade humana ilimitada; confiabilidade exige capacidade sustentável e priorização de melhorias.

Termo para lembrar: Sustainable on-call | Base técnica: referência 1.

17. Quem mantém uma biblioteca usada por dezenas de times?

Eu definiria um mantenedor explícito, política de compatibilidade e canal para contribuições, com recursos para atualizações e incidentes. Biblioteca compartilhada não pode depender apenas de voluntariado; os consumidores continuam responsáveis por adotar versões e validar comportamento no contexto de seus serviços.

Termo para lembrar: Shared-library ownership

18. Como resolver disputa entre uma melhoria de plataforma e uma entrega de produto?

Eu tornaria explícitos impacto, urgência, risco e custo de adiamento de cada opção, envolvendo os responsáveis pelos resultados. Quando a plataforma beneficia vários times, o financiamento não pode recair implicitamente sobre um único consumidor; precisamos de prioridade compartilhada e capacidade reservada.

Termo para lembrar: Shared prioritization

19. Como evitar que poucos revisores experientes bloqueiem todas as mudanças?

Eu mediria fila e tempo de revisão, distribuindo conhecimento por domínio e definindo critérios de risco para exigir especialistas. Revisões simples podem seguir um caminho delegado; pareamento e rotação aumentam capacidade sem transformar aprovação em uma assinatura automática sem responsabilidade técnica.

Termo para lembrar: Review queue design

20. Como lidar com responsabilidade sem acesso necessário?

Eu alinharia autoridade e permissões com função, mantendo menor privilégio e caminho de emergência. Não cobraria recuperação de um time que não consegue observar ou agir sobre a dependência crítica.

Termo para lembrar: Authority alignment | Base técnica: referência 1.

21. Uma equipe deve conhecer toda stack da empresa?

Não em profundidade uniforme. Eu definiria contratos e capacidades de plataforma que reduzam detalhes necessários, preservando compreensão suficiente para operar seu domínio e reconhecer quando precisa de apoio especializado.

Termo para lembrar: Bounded expertise | Base técnica: referência 1.

22. Como dividir responsabilidade pelo custo entre produto e infraestrutura?

Eu atribuiria consumo e escolhas de capacidade aos responsáveis que podem agir, distinguindo custos compartilhados dos específicos. A equipe de infraestrutura oferece transparência e opções; o produto decide prioridades com seus SLOs, evitando repassar toda economia como obrigação unilateral de outro time.

Termo para lembrar: Cost ownership

23. Quando um time enabling ajuda?

Quando existe lacuna temporária de capacidade ou adoção que pode ser transferida. Eu definiria objetivo e saída, evitando criar dependência permanente de consultoria para toda decisão cotidiana da equipe.

Termo para lembrar: Enabling interaction | Base técnica: referência 1.

24. Como tratar incidentes entre times sem blame?

Focaria sequência, condições e controles ausentes, atribuindo ações verificáveis com responsáveis. Isso não elimina accountability; evita confundir falha sistêmica com julgamento pessoal que reduz aprendizado e transparência.

Termo para lembrar: Blameless accountability | Base técnica: referência 1.

25. Qual risco de reorganizar frequentemente?

Perde-se contexto, ownership e continuidade de operação. Eu exigiria problema e benefício claros, planejando transferência de conhecimento e contratos, em vez de esperar que novo organograma resolva automaticamente dívida técnica.

Termo para lembrar: Organizational churn | Base técnica: referência 1.

26. Como dividir domínio que cresceu demais?

Mapearia capacidades, invariantes e padrões de mudança, procurando fronteiras com autonomia real. A divisão inclui dados e operação; separar pessoas sem contratos ou autoridade pode aumentar dependências em vez de reduzi-las.

Termo para lembrar: Domain decomposition | Base técnica: referência 1.

27. Pode ter dono técnico e dono de produto diferentes?

Sim, desde que decisões, objetivos e escalonamento estejam claros. Eu conectaria valor de negócio a restrições técnicas e risco, evitando prioridades contraditórias sem uma forma explícita de resolução.

Termo para lembrar: Decision roles | Base técnica: referência 1.

28. Como detectar plataforma que aumenta carga cognitiva?

Observaria etapas manuais, conceitos extras, tickets e tempo até uma entrega útil. Uma plataforma pode esconder detalhes ou acrescentar outra camada difícil; feedback dos consumidores precisa orientar sua evolução.

Termo para lembrar: Platform usability | Base técnica: referência 1.

29. Qual evidência de liderança técnica Staff?

Melhorar decisões e resultados além do código individual, com alinhamento e capacidade durável da equipe. Eu apresentaria impacto, trade-offs e aprendizagem, sem usar tamanho de reunião ou autoridade formal como medida.

Termo para lembrar: Technical influence | Base técnica: referência 1.

30. Como fechar discussão organizacional na entrevista?

Ligaria fronteiras técnicas a ownership, operação e evolução, explicitando quem muda e recupera cada parte. Não desenharia um organograma definitivo sem contexto; mostraria hipóteses e riscos de coordenação.

Termo para lembrar: Socio-technical boundary | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Ownership por domínio versus por tecnologia

Ganho. Aproxima decisão e resultado de negócio

Custo e motivo. Exige competências mais amplas no time

Como decidir. Quando fluxo pode ser atendido de ponta a ponta

02. Equipe pequena especializada versus ampla

Ganho. Aprofunda conhecimento específico

Custo e motivo. Aumenta dependências para entregar jornada completa

Como decidir. Quando especialização é rara e necessária

03. Alinhar serviços a times versus estrutura independente

Ganho. Pode reduzir coordenação cotidiana

Custo e motivo. Mudança organizacional pode pressionar arquitetura

Como decidir. Quando fronteiras de domínio são estáveis

04. Autosserviço versus aprovação central

Ganho. Reduz espera e trabalho repetitivo

Custo e motivo. Exige investimento em controles e caminhos suportados

Como decidir. Quando ações comuns têm regras automatizáveis

05. Plantão compartilhado versus dedicado

Ganho. Distribui carga humana

Custo e motivo. Pode reduzir contexto específico durante incidente

Como decidir. Quando suporte e documentação mantêm recuperação segura

06. Conhecimento distribuído versus especialista único

Ganho. Melhora continuidade e revisão

Custo e motivo. Consome tempo de formação e colaboração

Como decidir. Quando serviço é crítico ou duradouro

07. Automação de toil versus execução manual

Ganho. Reduz esforço recorrente e variabilidade

Custo e motivo. Exige desenvolvimento e manutenção

Como decidir. Quando processo estável se repete com frequência

08. Contrato versionado versus coordenação informal

Ganho. Facilita autonomia de mudança

Custo e motivo. Exige testes e gestão de depreciação

Como decidir. Quando consumidores são independentes

09. Handoff gradual versus transferência imediata

Ganho. Permite aprendizagem e validação

Custo e motivo. Mantém sobreposição temporária de equipes

Como decidir. Quando operação possui conhecimento tácito relevante

10. Reorganizar fronteiras versus melhorar interação

Ganho. Pode remover dependência estrutural

Custo e motivo. Custa contexto e transição

Como decidir. Quando evidência mostra problema persistente de ownership

Duas alternativas ao caminho proposto

1. Time alinhado ao domínio

Quando faz sentido. Há fluxo e fronteira claros

Como implementar. Dar responsabilidade por entrega e operação

Limites. Exige competências suficientes

2. Capacidade de plataforma

Quando faz sentido. Necessidade comum repete entre times

Como implementar. Oferecer serviço e suporte como produto

Limites. Pode virar fila central se mal desenhado

Referências e alcance da evidência

1. Team Topologies: conceitos

Responsabilidade, interação e carga cognitiva. As respostas e parâmetros de aplicação são propostas autorais.

https://teamtopologies.com/key-concepts

2. Google SRE: toil

Trabalho operacional repetitivo e automação. As respostas e parâmetros de aplicação são propostas autorais.

https://sre.google/sre-book/eliminating-toil/

Continue em: T113 · T115

T115 / L. Alcance Staff+/Principal

Plataformas e golden paths

Plataforma interna reduz trabalho repetido com caminhos suportados e possibilidade de exceção governada. Arquitetura de treino; não descreve implementação interna do Nubank.

Quando usar. Necessidades comuns de deploy, identidade, observabilidade e provisionamento.

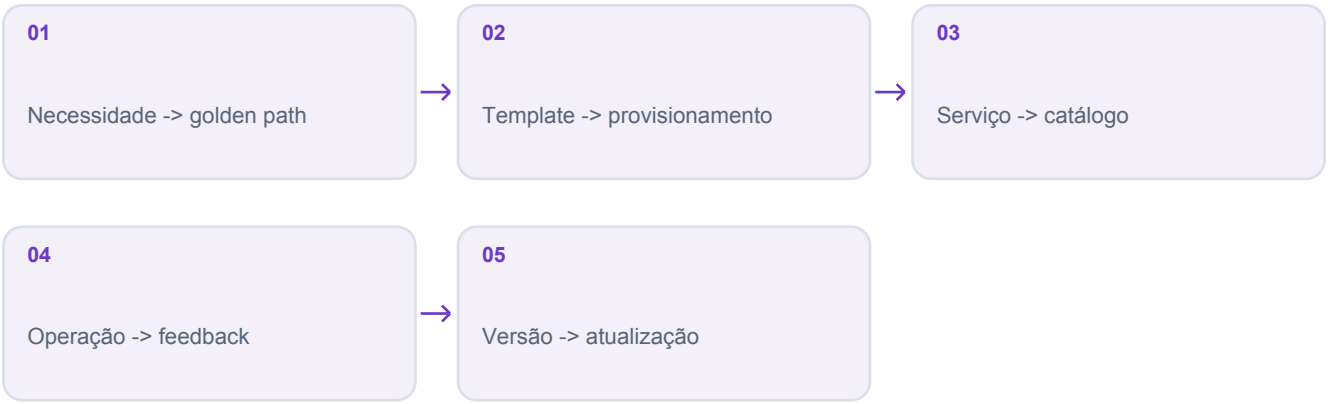
Quando evitar ou limitar. Confundir portal bonito com plataforma útil ou impor solução sem medir adoção.

Uma abertura possível

Eu começaria por oferecer padrão atraente com exceção governada. Depois explicaria ligar ownership documentação e ações úteis, preservando as restrições do caso.

Peças e tecnologias. Backstage · Terraform · GitOps · OpenTelemetry · Policy as code

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Oferecer padrão atraente com exceção governada; validar adoção e razões de abandono.

2. Ligar ownership documentação e ações úteis; validar metadados comparados ao ambiente real.

3. Automatizar políticas e provisionamento auditável; validar criação e limpeza por usuários reais.

4. Planejar atualização e inventário das versões; validar cobertura da mudança publicada.

5. Registrar motivo prazo e responsável; validar inventário de exceções vencidas.

30 perguntas para sustentar o desenho

01. Caminho padrão precisa ser obrigatório?

Ofereceria um caminho padrão útil com controles e suporte, permitindo exceções governadas para necessidades legítimas. Adoção mede valor; obrigatoriedade sem adequação pode criar contornos e aumentar a carga que a plataforma deveria reduzir.

Termo para lembrar: Golden path | Base técnica: referência 1.

02. Catálogo sozinho reduz carga operacional?

Ligaria catálogo a ownership, documentação e ações relevantes, mantendo sincronização com o ambiente. Inventário sozinho não opera serviços; metadados desatualizados podem ser piores que ausência de catálogo ao orientar uma resposta a incidente.

Termo para lembrar: Catalog | Base técnica: referência 2.

03. Como liberar infraestrutura sem ticket?

Automatizaria ações comuns com autorização, políticas e estado durável, preservando auditoria. Autosserviço precisa reconhecer falhas parciais e recursos criados, para que reenvio não multiplique infraestrutura nem deixe custos sem responsável.

Termo para lembrar: Self service | Base técnica: referência 1.

04. Template copiado recebe correção crítica?

Planejaria inventário e atualização de versões desde o primeiro template, incluindo migração de consumidores. Cópias independentes divergem; publicar correção no repositório base não garante que serviços existentes ficarão atualizados.

Termo para lembrar: Versioning | Base técnica: referência 2.

05. Exceção pode virar regra silenciosa?

Exceções teriam motivo, responsável e revisão proporcional ao risco. Elas podem revelar lacuna legítima do produto, mas não devem se tornar divergência permanente sem suporte nem custos explicitamente assumidos pela organização.

Termo para lembrar: Escape hatch | Base técnica: referência 1.

06. Como medir se plataforma ajuda?

Mediria tempo até uma entrega útil, intervenções manuais e fricção dos consumidores, combinando dados e feedback. Número de logins ou serviços cadastrados não demonstra que o desenvolvedor ficou mais autônomo.

Termo para lembrar: Developer experience | Base técnica: referência 2.

07. Plataforma fora derruba serviços de produção?

Separaria control plane e execução dos serviços sempre que fizer sentido, definindo o que para durante indisponibilidade da plataforma. Provisionamento pode ficar suspenso sem derrubar aplicações já em operação, reduzindo blast radius.

Termo para lembrar: Reliability | Base técnica: referência 1.

08. Padrão seguro elimina revisão de domínio?

Automatizaria controles comuns e explicaria seu escopo, mantendo revisão de riscos específicos do domínio. Template seguro não aprova qualquer regra financeira; o consumidor continua responsável pelos comportamentos que a plataforma não conhece.

Termo para lembrar: Security defaults | Base técnica: referência 2.

09. Como evitar autosserviço caro sem percepção?

Apresentaria custo previsto e consumo atribuído junto de limites relevantes. Provisionar facilmente deve vir com visibilidade, sem transformar a plataforma numa barreira arbitrária; a política precisa refletir orçamento e finalidade.

Termo para lembrar: Cost transparency | Base técnica: referência 1.

10. Quem atende dúvida de consumidor?

Definiria suporte, documentação e canais de feedback sustentáveis, usando demandas repetidas para melhorar o produto. A equipe central não deve virar execução manual permanente de toda tarefa que o autosserviço prometeu resolver.

Termo para lembrar: Support model | Base técnica: referência 2.

11. Um portal com catálogo já é uma IDP completa?

Não. Ele pode ser interface útil, mas o valor depende das capacidades reais de provisionar, operar e evoluir serviços. Eu mediria jornadas concluídas e trabalho removido, não apenas páginas disponíveis.

Termo para lembrar: Platform capability | Base técnica: referência 1.

12. Como escolher primeiro golden path?

Selecionaria uma jornada comum e dolorosa, com usuários representativos e resultado mensurável. Um caminho estreito bem mantido pode gerar mais valor que uma plataforma genérica tentando cobrir todos os casos de início.

Termo para lembrar: Platform discovery | Base técnica: referência 1.

13. Padrão deve impedir toda exceção?

Eu permitiria exceções governadas quando necessidades legítimas não cabem, registrando motivo e responsabilidade. Proibir tudo incentiva contornos; liberar tudo elimina suporte. O equilíbrio depende do risco e custo de manutenção.

Termo para lembrar: Governed exceptions | Base técnica: referência 1.

14. Como avaliar um plugin de terceiros antes de instalá-lo na plataforma?

Eu analisaria permissões, dados acessados, código executado e manutenção, isolando credenciais e ações conforme o risco. Um plugin de portal pode alcançar sistemas privilegiados; sua conveniência não justifica herdar acesso administrativo amplo nem executar entradas do usuário sem validação.

Termo para lembrar: Plugin trust boundary

15. Como proteger credenciais no autosserviço?

Usaria identidades e escopos mínimos, separando usuário solicitante de executor controlado. A plataforma registra intenção e autorização, evitando distribuir credenciais amplas apenas para tornar provisionamento mais conveniente.

Termo para lembrar: Provisioning security | Base técnica: referência 1.

16. Pode usar GitOps para qualquer mudança?

Eu avaliaria latência, tipo de recurso e necessidade de aprovação. GitOps oferece trilha e reconciliação declarativa, mas não substitui semântica de operações financeiras nem resolve automaticamente segredos ou migrações de dados.

Termo para lembrar: GitOps scope | Base técnica: referência 1.

17. Como lidar com criação parcial de infraestrutura?

Persistiria operação e recursos associados, com retomada ou limpeza segura. O usuário vê estado real; retry não deve criar duplicatas nem apagar recursos previamente válidos ao tentar compensar uma etapa falha.

Termo para lembrar: Provisioning recovery | Base técnica: referência 1.

18. Como oferecer um golden path em duas nuvens sem esconder diferenças importantes?

Eu definiria capacidades comuns e extensões explícitas para garantias que variam, validando recuperação e operação em cada implementação. Uma interface uniforme não torna bancos ou redes equivalentes; documentaria diferenças que afetam decisões do produto em vez de prometer portabilidade automática.

Termo para lembrar: Platform portability

19. Como medir adoção voluntária?

Observaria uso repetido, tempo poupado, abandono e razões de exceção, junto de entrevistas com equipes. Contagem de serviços criados não prova satisfação quando a ferramenta é obrigatória ou exige suporte constante.

Termo para lembrar: Adoption evidence | Base técnica: referência 1.

20. Plataforma pode oferecer vários bancos?

Pode, se houver necessidade e capacidade de suporte claras. Eu começaria com caminhos limitados e documentados, evitando catálogo amplo de opções sem operação, atualização e critérios de escolha adequados.

Termo para lembrar: Supported choices | Base técnica: referência 1.

21. Quem opera serviço gerado pelo template?

O contrato deve definir responsabilidade do consumidor e da plataforma, incluindo incidentes e atualizações. Provisionar não transfere automaticamente ownership; acesso e treinamento precisam acompanhar a obrigação de manter o serviço.

Termo para lembrar: Operational contract | Base técnica: referência 1.

22. Como impedir que uma ação de autosserviço apague um recurso produtivo por engano?

Eu distinguiria mudanças reversíveis de destrutivas, exibindo o impacto concreto e exigindo autorização proporcional antes da execução. Recursos críticos teriam proteção de exclusão e recuperação testada; um botão conveniente não deve contornar ownership, retenção ou dependências já existentes no ambiente.

Termo para lembrar: Destructive-action guardrails

23. Como incorporar observabilidade por padrão?

Provisionaria sinais básicos, identificação de serviço e exemplos de SLI, permitindo adaptação ao domínio. Dashboard genérico é ponto de partida; sucesso financeiro e prazo de caso exigem instrumentação específica.

Termo para lembrar: Observability defaults | Base técnica: referência 1.

24. Uma plataforma deve prometer zero configuração?

Eu evitaria a promessa absoluta. Bons defaults reduzem trabalho, mas requisitos especiais precisam aparecer; esconder decisões importantes pode produzir serviço inadequado. A experiência deve tornar escolhas necessárias compreensíveis e seguras.

Termo para lembrar: Explicit defaults | Base técnica: referência 1.

25. Como descontinuar versão de um golden path?

Inventariaria consumidores, ofereceria migração e prazo proporcional ao risco, acompanhando bloqueios. Desligar o caminho antigo sem suporte apenas transfere dívida e pode interromper equipes que dependem dele.

Termo para lembrar: Platform deprecation | Base técnica: referência 1.

26. Como escolher entre portal, CLI e API?

Pela jornada e usuários, podendo compartilhar capacidades por múltiplas interfaces. Eu evitaria implementar regras divergentes em cada entrada; autorização, estado e auditoria pertencem ao backend da plataforma.

Termo para lembrar: Interface consistency | Base técnica: referência 1.

27. Qual risco de abstração excessiva?

Ela pode esconder limites e dificultar diagnóstico quando algo sai do padrão. Eu manteria escape e visibilidade proporcionais, documentando garantias reais em vez de fingir que todos os provedores são equivalentes.

Termo para lembrar: Leaky abstraction | Base técnica: referência 1.

28. Como priorizar pedidos concorrentes de times?

Usaria impacto, frequência, risco e custo de suporte, com critérios transparentes. A plataforma é produto compartilhado; atender apenas o solicitante mais influente pode aumentar complexidade e prejudicar caminhos comuns.

Termo para lembrar: Platform prioritization | Base técnica: referência 1.

29. Como evitar que plataforma vire projeto abandonado?

Definiria equipe, orçamento e métricas de ciclo de vida, incluindo suporte e atualizações. Entrega inicial não encerra a obrigação; capacidades compartilhadas precisam evoluir com runtimes, segurança e necessidades dos consumidores.

Termo para lembrar: Platform sustainability | Base técnica: referência 1.

30. Qual desenho Staff apresentaria para IDP?

Mostraria jornada do desenvolvedor, API de operações duráveis, políticas, execução e catálogo, com ownership. Depois discutiria falhas, upgrades e métricas, conectando componentes à redução real de esforço das equipes.

Termo para lembrar: Platform architecture | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Golden path opinativo versus catálogo amplo

Ganho. Reduz decisões e facilita suporte

Custo e motivo. Pode não atender necessidades especiais

Como decidir. Quando jornadas comuns concentram demanda

02. Catálogo integrado versus planilha manual

Ganho. Melhora descoberta e vínculo com ações

Custo e motivo. Exige fontes e manutenção de metadados

Como decidir. Quando inventário muda frequentemente

03. Autosserviço com políticas versus tickets

Ganho. Reduz espera e toil

Custo e motivo. Demanda automação e testes de recuperação

Como decidir. Quando regras comuns são claras

04. Templates copiados versus componentes atualizáveis

Ganho. Dá independência ao consumidor

Custo e motivo. Correções precisam alcançar cópias divergentes

Como decidir. Quando autonomia compensa custo de atualização

05. Escape hatch versus padronização absoluta

Ganho. Atende necessidades legítimas fora do padrão

Custo e motivo. Aumenta diversidade e suporte

Como decidir. Quando exceções possuem justificativa concreta

06. Produto interno versus projeto temporário

Ganho. Mantém evolução orientada ao usuário

Custo e motivo. Exige equipe e orçamento contínuos

Como decidir. Quando capacidades compartilhadas são duradouras

07. Runtime independente versus dependência central

Ganho. Reduz impacto de falha da plataforma

Custo e motivo. Pode limitar controle imediato de algumas ações

Como decidir. Quando serviços devem continuar durante manutenção

08. Defaults seguros versus configuração totalmente livre

Ganho. Reduz erros comuns e onboarding

Custo e motivo. Pode restringir escolhas avançadas

Como decidir. Quando a maioria atende ao mesmo conjunto de requisitos

09. Transparência de custo versus ocultar detalhes

Ganho. Melhora decisão de consumo

Custo e motivo. Exige medição e explicação compreensíveis

Como decidir. Quando equipes controlam recursos faturados

10. Suporte escalável versus atendimento artesanal

Ganho. Reduz dependência da equipe central

Custo e motivo. Requer documentação e melhoria contínua

Como decidir. Quando adoção cresce além de poucos consumidores

Duas alternativas ao caminho proposto

1. Templates leves

Quando faz sentido. Poucos times e padrões simples

Como implementar. Oferecer exemplos mantidos com suporte claro

Limites. Cópias divergem sem atualização

2. Plataforma como produto

Quando faz sentido. Muitas equipes compartilham necessidades

Como implementar. Fornecer APIs autosserviço e ciclo de vida

Limites. Exige equipe e orçamento contínuos

Referências e alcance da evidência

1. CNCF: plataformas

Plataforma como produto e capacidades reutilizáveis. As respostas e parâmetros de aplicação são propostas autorais.

https://tag-app-delivery.cncf.io/whitepapers/platforms/

2. Backstage: software catalog

Catálogo, metadados e ownership de componentes. As respostas e parâmetros de aplicação são propostas autorais.

https://backstage.io/docs/features/software-catalog/

Continue em: T114 · T116

T116 / L. Alcance Staff+/Principal

Strangler, dívida técnica e evolução

Strangler substitui capacidades gradualmente preservando contratos e um caminho de recuperação. Arquitetura de treino; não descreve implementação interna do Nubank.

Quando usar. Modernização de legado com necessidade de continuidade.

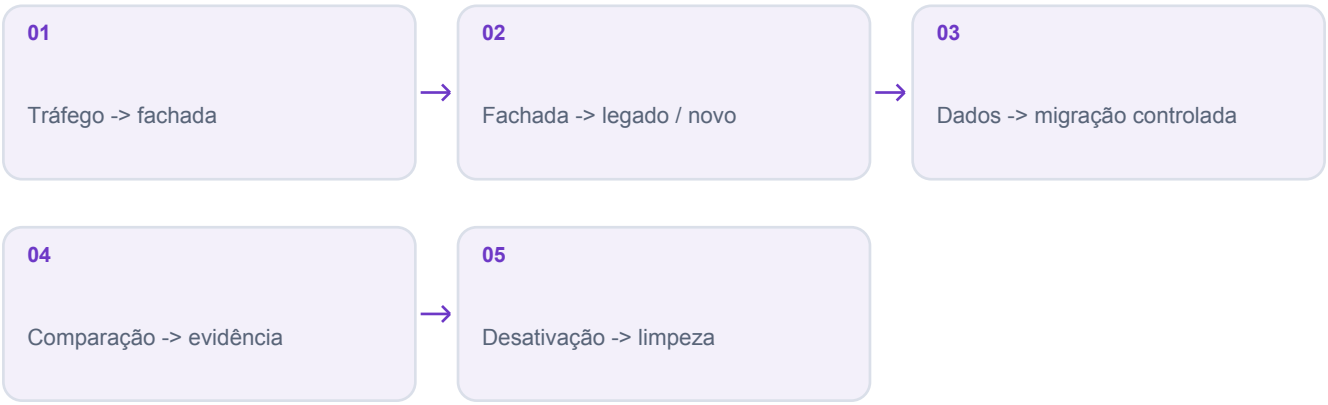
Quando evitar ou limitar. Prometer rollback de código que desfaça efeitos financeiros ou dual write sem coordenação.

Uma abertura possível

Eu começaria por escolher fronteira útil com risco controlável. Depois explicaria rotear por regra determinística e versionada, preservando as restrições do caso.

Peças e tecnologias. Strangler facade · CDC · Contract tests · Feature flags

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Escolher fronteira útil com risco controlável; validar fluxo completo antes e depois.

2. Rotear por regra determinística e versionada; validar mesmo fluxo durante mudanças.

3. Definir autoridade por etapa da migração; validar escritas concorrentes em transição.

4. Espelhar apenas caminhos sem efeito autorizado; validar isolamento de efeitos externos.

5. Coordenar snapshot e captura incremental; validar reconciliação por chave e versão.

30 perguntas para sustentar o desenho

01. Qual pedaço migrar primeiro?

Escolheria uma capacidade com valor observável e fronteira controlável, não apenas uma camada técnica fácil de extrair. O primeiro recorte deve ensinar sobre migração e operação sem assumir exposição maior do que conseguimos recuperar.

Termo para lembrar: Migration slice | Base técnica: referência 1.

02. Como direcionar cliente para novo serviço?

Usaria regra determinística e versionada para direcionar a fatia migrada, testando continuidade da jornada. Alternar destino sem considerar estado pode dividir uma mesma operação entre autoridades incompatíveis durante a transição.

Termo para lembrar: Routing | Base técnica: referência 2.

03. Legado e novo podem escrever mesma conta?

Definiria quem pode escrever cada dado em cada fase, preservando invariantes. Dois writers independentes sobre a mesma conta podem gerar conflito financeiro; coexistência exige protocolo explícito, não apenas comparação posterior.

Termo para lembrar: Data ownership | Base técnica: referência 1.

04. Posso duplicar pagamentos para comparar?

Espelharia somente caminhos cujo efeito esteja isolado ou seja seguro, com credenciais e destinos controlados. Shadow traffic não autoriza repetir pagamento real; comparação de leitura e simulação são diferentes de execução financeira.

Termo para lembrar: Shadow traffic | Base técnica: referência 2.

05. Como mover histórico sem perder mudanças recentes?

Coordenaria snapshot e mudanças incrementais com fronteira recuperável, validando chaves e versões. O processo precisa cobrir alterações e exclusões durante a cópia, sem confiar apenas em contagem final aparentemente próxima.

Termo para lembrar: Backfill | Base técnica: referência 1.

06. Trocar rota de volta desfaz tudo?

Separaria rollback de código, dados e efeitos externos, verificando compatibilidade depois do cutover. Se o legado não entende estados novos, voltar a rota pode ser inseguro; talvez seja necessário roll-forward ou reconciliação.

Termo para lembrar: Rollback | Base técnica: referência 2.

07. Toda dívida merece reescrita?

Priorizaria dívida pelo risco, custo de mudança e valor futuro, comparando alternativas menores. Reescrever não é automaticamente melhorar; o investimento precisa superar custo de transição e preservar obrigações existentes durante o percurso.

Termo para lembrar: Debt | Base técnica: referência 1.

08. API nova pode remover campo antigo?

Manteria compatibilidade ou migraria consumidores com política explícita, incluindo clientes antigos e integrações raras. Uma nova implementação não concede liberdade para remover semântica pública ainda utilizada por quem não atualiza simultaneamente.

Termo para lembrar: Compatibility | Base técnica: referência 2.

09. Quando desligar o legado?

Definiria critérios de retirada: ausência de dependências necessárias, dados preservados e recuperação adequada. Sem dono e prazo de desligamento, a coexistência temporária pode virar custo permanente sem realizar o benefício da migração.

Termo para lembrar: Decommission | Base técnica: referência 1.

10. Como provar que migração valeu?

Compararia resultado com a motivação inicial, incluindo SLO, custo, operação e velocidade de mudança. Concluir tarefas de migração não prova valor; o sistema novo precisa demonstrar melhora sem reduzir garantias silenciosamente.

Termo para lembrar: Success metrics | Base técnica: referência 2.

11. Strangler exige migrar para microsserviços?

Não necessariamente. O padrão substitui capacidades gradualmente; o destino pode ser outro monólito modular ou serviço. Eu escolheria fronteiras pelo problema, evitando distribuir apenas porque estamos modernizando um legado.

Termo para lembrar: Migration target | Base técnica: referência 1.

12. Como escolher fachada de roteamento?

Ela precisa reconhecer a fatia migrada e preservar contratos, sem concentrar regras indevidas. Eu testaria comportamento durante troca de rota e definiria observabilidade, porque a fachada também vira dependência operacional.

Termo para lembrar: Migration facade | Base técnica: referência 1.

13. Como migrar um módulo que mantém dependências cíclicas com o legado?

Eu mapearia chamadas e dados compartilhados para escolher uma fronteira que quebre o ciclo, usando interfaces explícitas e uma transição por etapas. Extrair apenas o código pode preservar o acoplamento pela rede; primeiro preciso reduzir a dependência que impede autonomia operacional.

Termo para lembrar: Dependency-cycle extraction

14. Como migrar entidades quando o sistema novo usa identificadores diferentes?

Eu manteria um mapeamento durável e único entre identidades, preservando referências externas e a identidade das operações em andamento. Converter IDs apenas na aplicação pode deixar eventos antigos incompreensíveis; a tradução precisa acompanhar histórico, replay e vínculos com sistemas que ainda usam o legado.

Termo para lembrar: Identity translation

15. Como trocar um scheduler legado sem perder nem duplicar execuções agendadas?

Eu manteria identidades estáveis de ocorrência e transferiria a autoridade de disparo com um marco verificável. O novo scheduler recupera ocorrências elegíveis sem presumir que o antigo parou; o consumidor continua deduplicando, porque sobreposição e tentativas atrasadas podem ocorrer durante a transição.

Termo para lembrar: Scheduler authority cutover

16. Como comparar saída do legado e novo?

Normalizaria diferenças esperadas e verificaria propriedades relevantes, incluindo valores e autorização. Igualdade textual pode ser inadequada, mas divergências precisam de explicação; eu não descartaria erro como ruído sem investigação.

Termo para lembrar: Differential testing | Base técnica: referência 1.

17. Como impedir que indicadores históricos mudem de significado durante a migração?

Eu versionaria definições e explicitaria diferenças de cálculo, comparando períodos equivalentes antes de substituir relatórios. Se uma regra mudou, não juntaria séries como se fossem idênticas; manteria linhagem suficiente para explicar se a variação veio do negócio ou da transformação técnica.

Termo para lembrar: Historical metric comparability

18. Como evitar duplicar permanentemente custos?

Definiria marcos de saída, consumidores restantes e dono do desligamento. Convivência temporária tem prazo e critério; sem isso, o projeto pode manter legado e novo indefinidamente sem realizar o benefício previsto.

Termo para lembrar: Decommission plan | Base técnica: referência 1.

19. Como migrar cliente mobile que não atualiza?

Manteria contrato compatível ou adapter durante a janela suportada, observando tráfego real. Não presumiria atualização imediata; remoção de campo ou endpoint exige política e evidência de impacto sobre versões antigas.

Termo para lembrar: Long-lived clients | Base técnica: referência 1.

20. Qual dívida técnica merece prioridade?

A que produz risco, atraso ou custo relevante em relação ao esforço de resolver. Eu usaria evidências e dependências, evitando reescrever componentes apenas por preferência de linguagem ou estética arquitetural.

Termo para lembrar: Debt economics | Base técnica: referência 1.

21. Como migrar por tenant sem quebrar operações conjuntas?

Verificaria se a fatia possui independência real. Transações entre tenants ou recursos compartilhados podem atravessar a fronteira; eu definiria roteamento e coordenação antes de usar tenant como unidade de migração.

Termo para lembrar: Migration partition | Base técnica: referência 1.

22. É seguro testar só no ambiente novo vazio?

Não. Histórico, cardinalidade e estados antigos podem revelar incompatibilidades. Eu usaria dados representativos autorizados e ensaios de recuperação, incluindo casos raros que o sistema legado acumulou ao longo do tempo.

Termo para lembrar: Legacy state coverage | Base técnica: referência 1.

23. Como medir benefício de uma refatoração interna?

Compararia tempo de mudança, falhas, testes ou custo operacional ligados ao objetivo. Refatoração não precisa aumentar throughput para valer, mas seu benefício deve ser observável e proporcional ao investimento.

Termo para lembrar: Refactoring outcome | Base técnica: referência 1.

24. O que fazer se a nova solução é mais rápida mas menos correta?

Eu não a consideraria equivalente. Validaria invariantes e garantias antes do desempenho; reduzir durabilidade ou autorização pode melhorar benchmark e tornar a solução inadequada ao requisito original.

Termo para lembrar: Semantic equivalence | Base técnica: referência 1.

25. Como evoluir schema durante coexistência?

Usaria alterações compatíveis, backfill e retirada posterior conforme expand-contract, com validação. Os dois caminhos precisam compreender o estado durante a transição; não dependeria de deploy simultâneo perfeito.

Termo para lembrar: Schema coexistence | Base técnica: referência 1.

26. Quando usar anti-corruption layer?

Quando o modelo legado não deve contaminar o novo domínio ou vice-versa. Eu traduziria contratos e erros explicitamente, preservando semântica, sem esconder diferenças financeiras importantes atrás de uma conversão superficial.

Termo para lembrar: Model isolation | Base técnica: referência 1.

27. Quem decide aceitar divergência temporária?

O responsável pelo requisito, com evidência de impacto e limite claro. Engenharia explica mecanismo e recuperação; não assumiria autorização para perder precisão financeira apenas para facilitar a migração.

Termo para lembrar: Risk ownership | Base técnica: referência 1.

28. Como testar failback depois de cutover?

Executaria uma mudança representativa no novo e verificaria se o antigo consegue operar corretamente depois. Se não houver compatibilidade, definiria roll-forward ou migração reversa explícita, evitando um botão de rollback ilusório.

Termo para lembrar: Failback testing | Base técnica: referência 1.

29. Qual risco de migrar tecnologia e regra de negócio juntas?

Dificulta atribuir diferenças e aumenta espaço de falha. Eu separaria quando possível, ou criaria controles fortes de comparação, deixando claro quais resultados devem permanecer iguais e quais mudanças são intencionais.

Termo para lembrar: Change isolation | Base técnica: referência 1.

30. Como encerrar um plano de evolução na entrevista?

Mostraria estado inicial, fatias, autoridade de dados, validação, contingência e desligamento. A narrativa explica como chegar ao desenho sem interromper obrigações existentes, em vez de apresentar apenas a arquitetura final ideal.

Termo para lembrar: Migration narrative | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Migração incremental versus big bang

Ganho. Limita exposição e produz aprendizado gradual

Custo e motivo. Mantém coexistência e roteamento temporários

Como decidir. Quando continuidade e risco exigem transição controlada

02. Fatia por domínio versus por camada técnica

Ganho. Entrega fluxo utilizável com ownership claro

Custo e motivo. Pode exigir integração mais cuidadosa inicialmente

Como decidir. Quando benefício precisa ser observado ponta a ponta

03. Writer único versus dual write

Ganho. Simplifica autoridade e invariantes

Custo e motivo. Restringe flexibilidade de transição

Como decidir. Quando conflitos financeiros são inaceitáveis

04. Shadow sem efeitos versus teste integral ao vivo

Ganho. Permite comparar sem duplicar compromissos

Custo e motivo. Não exercita toda semântica externa real

Como decidir. Quando segurança impede repetição de efeitos

05. Snapshot mais CDC versus parada para copiar

Ganho. Reduz janela de interrupção

Custo e motivo. Adiciona coordenação e validação incremental

Como decidir. Quando indisponibilidade longa não é aceitável

06. Rollback compatível versus roll-forward

Ganho. Permite retornar a caminho conhecido

Custo e motivo. Exige manter compatibilidade de estados e contratos

Como decidir. Quando mudanças podem ser revertidas com segurança

07. Refatoração local versus reescrita

Ganho. Reduz investimento e risco de transição

Custo e motivo. Pode preservar limitações estruturais

Como decidir. Quando problemas principais cabem na arquitetura atual

08. Adapters de compatibilidade versus migração simultânea

Ganho. Permite consumidores evoluírem em ritmos diferentes

Custo e motivo. Aumenta código temporário e suporte

Como decidir. Quando clientes não atualizam juntos

09. Desligamento rápido versus convivência prolongada

Ganho. Realiza economia e reduz superfícies duplicadas

Custo e motivo. Diminui tempo de fallback

Como decidir. Quando evidências de migração são suficientes

10. Tecnologia isolada versus mudar regras junto

Ganho. Facilita comparação de comportamento

Custo e motivo. Pode adiar benefício de negócio combinado

Como decidir. Quando equivalência é essencial para validar a migração

Duas alternativas ao caminho proposto

1. Refatoração interna

Quando faz sentido. Fronteira atual ainda atende o negócio

Como implementar. Melhorar módulos e testes sem distribuir

Limites. Mantém limitações do deploy conjunto

2. Substituição gradual

Quando faz sentido. Risco permite recortar capacidades

Como implementar. Migrar fatias com contratos e critérios de saída

Limites. Convivência temporária aumenta complexidade

Referências e alcance da evidência

1. Azure: strangler fig

Substituição incremental de funcionalidades. As respostas e parâmetros de aplicação são propostas autorais.

https://learn.microsoft.com/en-us/azure/architecture/patterns/strangler-fig

2. AWS: strangler fig

Roteamento e migração progressiva. As respostas e parâmetros de aplicação são propostas autorais.

https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/strangler-fig.html

Continue em: T115 · T117

T117 / L. Alcance Staff+/Principal

Gestão de riscos e dependências

Risco arquitetural precisa de consequência, probabilidade, responsável e ação verificável. Arquitetura de treino; não descreve implementação interna do Nubank.

Quando usar. Escolhas com dependências externas, dinheiro, dados ou disponibilidade.

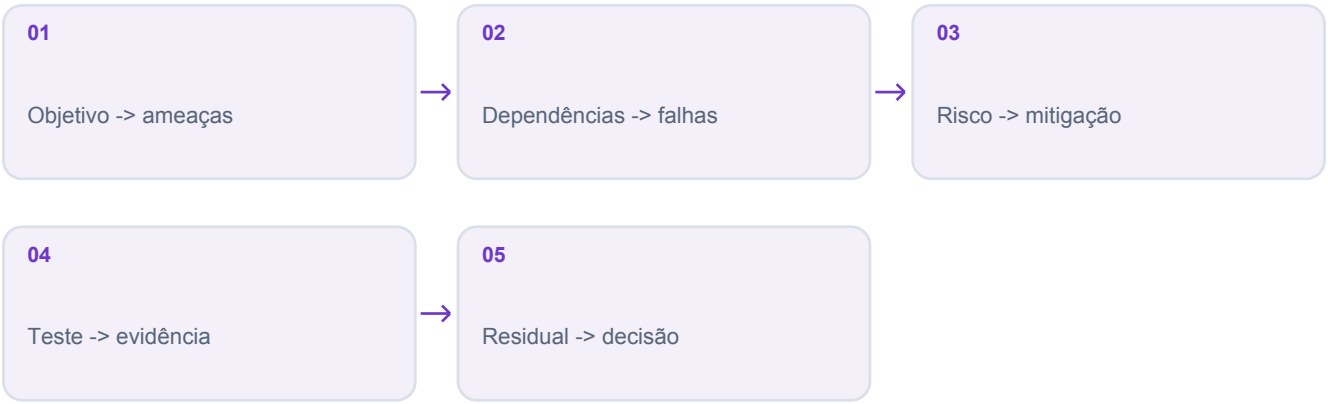
Quando evitar ou limitar. Usar matriz colorida como substituto de teste ou declarar risco zero.

Uma abertura possível

Eu começaria por ligar causa consequência e jornada afetada. Depois explicaria mapear recursos compartilhados e falhas correlacionadas, preservando as restrições do caso.

Peças e tecnologias. Risk register · SLO · Fault tree · Runbooks · Game days

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Ligar causa consequência e jornada afetada; validar cenário de indisponibilidade executável.

2. Mapear recursos compartilhados e falhas correlacionadas; validar perda de região identidade ou DNS.

3. Registrar risco remanescente e aceite responsável; validar limites demonstrados nos testes.

4. Combinar severidade plausibilidade e exposição; validar cenário extremo fundamentado.

5. Ensaiar recuperação com dependências reais; validar tempo e sucesso do exercício.

30 perguntas para sustentar o desenho

01. Como escrever risco de banco indisponível?

Descreveria causa plausível, jornada afetada e consequência mensurável, distinguindo hipótese de problema presente. Risco de banco cair é amplo demais; preciso saber quais operações param, quais ficam incertas e como isso atinge clientes.

Termo para lembrar: Risk statement | Base técnica: referência 1.

02. Conhecer só dependências diretas basta?

Mapearia dependências indiretas e recursos compartilhados, como identidade, DNS e região. Serviços diferentes podem falhar juntos; a avaliação deve revelar correlação antes de contabilizar redundância como se fosse independência garantida.

Termo para lembrar: Dependencies | Base técnica: referência 2.

03. Mitigado significa eliminado?

Registraria o que o controle reduz e o que permanece possível, com responsável pela aceitação. Mitigação não elimina todo risco; testes e limites conhecidos ajudam decidir se a exposição residual é compatível com o negócio.

Termo para lembrar: Residual risk | Base técnica: referência 1.

04. Risco raro nunca merece investimento?

Combinaria consequência, plausibilidade e exposição, explicitando incerteza. Risco raro pode exigir atenção quando o impacto é extremo; não decidiria apenas por média simplificada nem inventaria probabilidades precisas sem base.

Termo para lembrar: Prioritization | Base técnica: referência 2.

05. Runbook sem exercício é suficiente?

Exercitaria runbook com acessos, ferramentas e dependências reais ou representativas, medindo recuperação. Documento não garante execução sob pressão; a prova precisa mostrar que a equipe consegue cumprir o objetivo nas condições previstas.

Termo para lembrar: Contingency | Base técnica: referência 1.

06. Fornecedor garante uptime: acabou risco?

Avaliaria contrato, quotas, integração e contingência do fornecedor dentro da jornada. Disponibilidade da API não significa sucesso do processo; também mediria resultados desconhecidos e tempo de recuperação quando o parceiro falha.

Termo para lembrar: Third party | Base técnica: referência 2.

07. Controle novo pode piorar disponibilidade?

Analisaria custo e falha do próprio controle, definindo comportamento por operação. Segurança e disponibilidade precisam de decisão explícita; um serviço de política indisponível pode bloquear tudo ou permitir risco, conforme configuração.

Termo para lembrar: Security | Base técnica: referência 1.

08. Prazo curto justifica ignorar risco?

Diante de prazo curto, proporia escopo menor ou exposição limitada preservando invariantes essenciais. Risco residual relevante vai ao responsável adequado com evidências; pressão de entrega não é autorização automática para aceitar consequência financeira.

Termo para lembrar: Deadline | Base técnica: referência 2.

09. Registro precisa mudar após incidente?

Atualizaria hipóteses e controles após incidente, ligando ações à causa demonstrada. Post-mortem sem efeito na arquitetura ou operação não reduz risco; acompanharia implementação e evidência de que o mecanismo mudou.

Termo para lembrar: Risk review | Base técnica: referência 1.

10. Quem aceita risco financeiro relevante?

Encaminharia aceitação ao papel com alçada correspondente, explicando impacto, alternativas e limites. Engenharia oferece análise e proposta, mas não deve assumir sozinha decisões de risco de negócio que ultrapassam sua autoridade.

Termo para lembrar: Decision owner | Base técnica: referência 2.

11. Como diferenciar risco de problema já ocorrido?

Risco é possibilidade futura com consequência; problema é condição presente que exige tratamento. Eu registraria ambos sem misturar probabilidades e fatos, usando incidentes como evidência para atualizar a avaliação.

Termo para lembrar: Risk versus issue | Base técnica: referência 1.

12. Como trata uma dependência crítica sem manutenção?

Eu avaliaria exposição, capacidade de corrigir vulnerabilidades e opções de substituição, com dono e horizonte de decisão. Fixar a versão reduz surpresa de atualização, mas não resolve ausência de suporte. O plano inclui testes de migração antes de uma emergência.

Termo para lembrar: Risco de cadeia de software

13. Qual diferença entre mitigação e contingência?

Mitigação reduz chance ou impacto antes da ocorrência; contingência define resposta quando acontece. Eu desenharia ambas para dependências críticas, com responsáveis e testes que mostrem limites reais de recuperação.

Termo para lembrar: Prevention versus response | Base técnica: referência 1.

14. Como identificar falha comum entre provedores distintos?

Mapearia rede, DNS, identidade, região e recursos compartilhados. Dois contratos comerciais não garantem independência técnica; o desenho de redundância precisa evitar depender da mesma causa de falha escondida.

Termo para lembrar: Common-mode failure | Base técnica: referência 1.

15. Quando aceitar risco é uma decisão válida?

Quando impacto e limites são compreendidos, há responsável adequado e a aceitação é compatível com obrigações. Eu registraria gatilho de revisão, evitando confundir falta de tempo com consentimento informado.

Termo para lembrar: Explicit acceptance | Base técnica: referência 1.

16. Como usar error budget na decisão de mudança?

Ligaria a política ao SLO e ao risco de novas alterações, sem tratá-lo como licença para quebrar qualquer invariante. Integridade financeira e segurança podem ter restrições diferentes da disponibilidade medida.

Termo para lembrar: Error budget scope | Base técnica: referência 1.

17. Qual risco de controle com fail-closed?

Pode bloquear tráfego legítimo quando o controle falha. Eu avaliaria consequência e alternativas por operação, preservando regras essenciais; escolher fail-open indiscriminadamente também pode criar exposição inaceitável.

Termo para lembrar: Control availability | Base técnica: referência 1.

18. Como avaliar dependência sem histórico confiável?

Usaria testes, limites contratuais e hipóteses conservadoras explicitadas, com monitoramento após adoção. Não inventaria probabilidade precisa; a incerteza pode justificar rollout menor e um caminho de contingência.

Termo para lembrar: Uncertainty management | Base técnica: referência 1.

19. RTO e RPO precisam de teste conjunto?

Sim. Restaurar rápido um estado muito antigo pode não atender o negócio, e preservar dados sem voltar a operar também falha. Eu ensaiaria recuperação e reconciliação para verificar ambas as metas.

Termo para lembrar: Recovery objectives | Base técnica: referência 1.

20. Uma aceitação de risco continua válida depois de multiplicar a escala?

Não automaticamente. Eu associaria a decisão a limites de volume, exposição e prazo, com gatilhos de revisão. Crescimento pode ampliar consequência ou invalidar a mitigação. O registro precisa explicar em quais condições o responsável aceitou aquele risco residual.

Termo para lembrar: Validade da aceitação

21. Qual benefício de blast radius limitado?

Restringe a população afetada por uma falha ou mudança. O custo é mais isolamento e operação; eu testaria se células realmente não compartilham dependência crítica que invalida a separação pretendida.

Termo para lembrar: Failure containment | Base técnica: referência 1.

22. Durante um incidente, congelaria qualquer mudança?

Eu distinguiria mudanças que ampliam incerteza das necessárias para conter o incidente. Uma correção emergencial passa por revisão proporcional, hipótese e critério de reversão. Congelamento absoluto pode prolongar o dano; pressa também não justifica múltiplas alterações sem rastreabilidade.

Termo para lembrar: Mudança emergencial

23. Como lidar com risco de pessoa-chave?

Distribuiria conhecimento e acessos necessários, com exercícios conduzidos por substitutos. Documentação ajuda, mas a capacidade de executar recuperação e decisões precisa existir além de uma única pessoa.

Termo para lembrar: Key-person risk | Base técnica: referência 1.

24. O que monitorar para revisar risco de parceiro?

Erro, latência, quota, mudanças de contrato e qualidade de suporte, ligados à jornada. Métricas técnicas precisam revelar exposição, como operações UNKNOWN e prazo ameaçado, não apenas disponibilidade da URL.

Termo para lembrar: Dependency monitoring | Base técnica: referência 1.

25. Como testar risco sem causar o incidente real?

Começaria com modelos, ambiente controlado e falhas limitadas, ampliando somente com autorização e proteção adequadas. Experimento deve ter hipótese, abort criteria e observação, evitando transformar teste em exposição desnecessária.

Termo para lembrar: Controlled validation | Base técnica: referência 1.

26. Qual diferença entre redundância e independência?

Redundância duplica capacidade; independência reduz causas comuns. Eu verificaria regiões, energia, identidade e operação, porque duas instâncias podem cair juntas e oferecer menos proteção do que o desenho sugere.

Termo para lembrar: Independent failure domains | Base técnica: referência 1.

27. Como documentar risco residual após automação?

Explicaria falhas ainda possíveis, limites do detector e ação humana necessária. Automatizar um controle muda o risco, mas também cria dependência de código e configuração que precisam de supervisão.

Termo para lembrar: Residual control risk | Base técnica: referência 1.

28. Quando um alerta é uma mitigação fraca?

Quando chega tarde, não tem responsável ou não permite ação antes do impacto. Eu avaliaria tempo de detecção e resposta, preferindo prevenção quando o dano não pode ser revertido após o aviso.

Termo para lembrar: Actionable detection | Base técnica: referência 1.

29. Como decide quando probabilidades de falha são pouco confiáveis?

Eu compararia cenários e variaria hipóteses críticas para observar quando a decisão muda. Se pequenas variações alteram a escolha, buscaria mais evidência ou reduziria exposição. Um número preciso sem fundamento esconde incerteza; intervalos e limites tornam a decisão discutível.

Termo para lembrar: Análise de sensibilidade

30. Como falar de resiliência sem prometer perfeição?

Eu declararia falhas cobertas, hipóteses e comportamento residual, demonstrando uma recuperação concreta. A qualidade está em conhecer limites e preservar propriedades essenciais, não em afirmar que o sistema nunca ficará indisponível.

Termo para lembrar: Bounded resilience | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Prevenção versus recuperação

Ganho. Pode evitar impacto antes da falha

Custo e motivo. Pode custar mais e ainda deixar risco residual

Como decidir. Quando dano é difícil de reverter

02. Redundância independente versus simples duplicação

Ganho. Reduz falhas de causa comum

Custo e motivo. Exige mais domínios e complexidade operacional

Como decidir. Quando continuidade justifica o investimento

03. Mitigar versus aceitar explicitamente

Ganho. Reduz exposição relevante

Custo e motivo. Consome recursos e pode adicionar dependências

Como decidir. Quando benefício supera custo e obrigação exige controle

04. Prioridade por severidade versus frequência

Ganho. Dá atenção a consequências extremas

Custo e motivo. Pode deslocar esforço de falhas mais comuns

Como decidir. Quando impacto raro é material

05. Exercício prático versus revisão documental

Ganho. Revela lacunas de acesso e execução

Custo e motivo. Exige tempo e ambiente controlado

Como decidir. Quando RTO depende de ação operacional

06. Fornecedor alternativo versus contingência local

Ganho. Pode ampliar opções durante falha

Custo e motivo. Adiciona integração e diferenças semânticas

Como decidir. Quando independência e contrato permitem troca segura

07. Fail-closed versus fail-open

Ganho. Preserva determinadas restrições de segurança

Custo e motivo. Pode negar operações legítimas durante falha

Como decidir. Quando permitir sem validação é mais perigoso

08. Escopo reduzido versus prazo adiado

Ganho. Preserva entrega de capacidade essencial

Custo e motivo. Posterga funcionalidades desejadas

Como decidir. Quando invariantes podem ser mantidos num MVP menor

09. Monitoramento contínuo versus revisão periódica

Ganho. Detecta mudança de exposição mais cedo

Custo e motivo. Aumenta custo e ruído operacional

Como decidir. Quando risco muda rapidamente

10. Aceite formal versus decisão implícita

Ganho. Torna responsabilidade e limites visíveis

Custo e motivo. Exige coordenação proporcional ao impacto

Como decidir. Quando consequência ultrapassa escolha técnica local

Duas alternativas ao caminho proposto

1. Mitigação preventiva

Quando faz sentido. Impacto exige reduzir chance de falha

Como implementar. Remover causa ou limitar exposição

Limites. Pode aumentar custo e complexidade

2. Contingência recuperável

Quando faz sentido. Falha não pode ser eliminada

Como implementar. Definir degradação recuperação e responsáveis

Limites. Aceita impacto residual explícito

Referências e alcance da evidência

1. Google SRE: risco

Objetivos de confiabilidade e aceitação de risco. As respostas e parâmetros de aplicação são propostas autorais.

https://sre.google/sre-book/embracing-risk/

2. AWS: reliability

Resiliência e recuperação. As respostas e parâmetros de aplicação são propostas autorais.

https://docs.aws.amazon.com/wellarchitected/latest/reliability-pillar/welcome.html

Continue em: T116 · T118

T118 / L. Alcance Staff+/Principal

Migrações organizacionais e adoção

Adoção é migração de comportamento, responsabilidade e sistemas, não apenas instalação de ferramenta. Arquitetura de treino; não descreve implementação interna do Nubank.

Quando usar. Mudanças entre equipes, contratos e modelos operacionais.

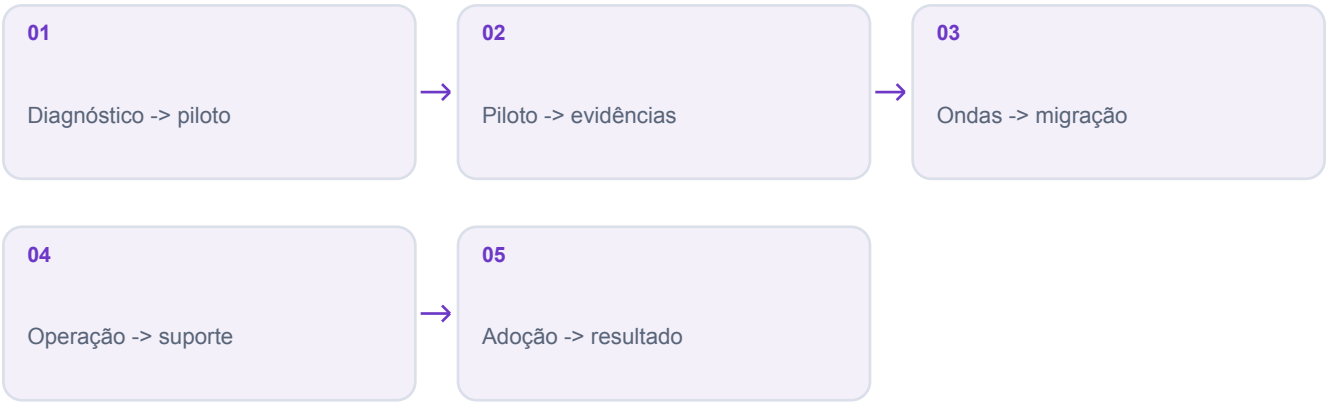
Quando evitar ou limitar. Impor big bang ou medir sucesso só pela quantidade de serviços migrados.

Uma abertura possível

Eu começaria por identificar afetados decisores e operadores. Depois explicaria escolher piloto seguro e representativo, preservando as restrições do caso.

Peças e tecnologias. Pilot · Migration waves · Training · Telemetry · Ownership map

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Identificar afetados decisores e operadores; validar jornada com equipes envolvidas.

2. Escolher piloto seguro e representativo; validar aprendizado transferível do piloto.

3. Agrupar por dependência risco e prontidão; validar capacidade por janela de migração.

4. Combinar exemplos prática e suporte contextual; validar tarefa real por equipe nova.

5. Transferir conhecimento plantão e autoridade juntos; validar exercício de operação independente.

30 perguntas para sustentar o desenho

01. Quem precisa participar da mudança?

Mapearia usuários, operadores, decisores e dependências, entendendo o que muda na rotina de cada grupo. Adoção técnica pode falhar se processo e incentivos continuam incompatíveis; começaria pela jornada real, não pelo anúncio da ferramenta.

Termo para lembrar: Stakeholders | Base técnica: referência 1.

02. Primeira equipe deve ser a mais fácil?

Escolheria piloto representativo com risco controlado, evitando tanto um caso impossível quanto uma equipe excepcional que mascara dificuldades. O objetivo é aprender o que precisa mudar antes de ampliar, com critérios claros de sucesso.

Termo para lembrar: Pilot | Base técnica: referência 2.

03. Como ordenar dezenas de serviços?

Organizaria ondas por dependência, risco e capacidade de suporte, com gates observáveis. Migrar todos simultaneamente pode saturar pessoas e conhecimento, mesmo quando infraestrutura comporta a carga; o cronograma precisa refletir essa restrição.

Termo para lembrar: Migration waves | Base técnica: referência 1.

04. Documentação longa basta para adoção?

Combinaria documentação, exemplos e prática de tarefas reais, incluindo falhas. Treinamento não deve apenas mostrar botões; a equipe precisa conseguir operar com autonomia e saber quando escalar situações fora do caminho padrão.

Termo para lembrar: Training | Base técnica: referência 2.

05. Novo time aceitou repositório: terminou?

Transferiria conhecimento, acessos, plantão e autoridade, validando o receptor em exercício. Mudar ownership no catálogo não garante prontidão; a organização precisa preservar continuidade enquanto o novo time aprende e assume decisões.

Termo para lembrar: Ownership transfer | Base técnica: referência 1.

06. Por que equipes evitam padrão melhor?

Investigaria custos e prioridades locais antes de atribuir resistência à equipe. Uma mudança pode beneficiar a empresa e impor trabalho não financiado ao consumidor; alinharia capacidade e incentivos para tornar adoção viável.

Termo para lembrar: Incentives | Base técnica: referência 2.

07. Como desligar contrato antigo?

Definiria inventário, prazo, comunicação e suporte para depreciação, observando tráfego real. Consumidor desconhecido ou pouco frequente ainda pode depender do contrato; o desligamento exige evidência, não apenas ausência numa lista manual.

Termo para lembrar: Deprecation | Base técnica: referência 1.

08. Mais logins provam valor da plataforma?

Mediria tempo até resultado útil, intervenções, qualidade e abandono do processo antigo. Logins e contagem de migrações podem crescer por obrigação, sem demonstrar que a mudança reduziu esforço ou melhorou segurança.

Termo para lembrar: Adoption metrics | Base técnica: referência 2.

09. Como voltar sem culpar a equipe piloto?

Prepararia contingência e critérios de retorno sem personalizar culpa no piloto. Reversão preserva serviço e aprendizado; pessoas precisam conseguir relatar problemas cedo, em vez de esconder falhas para proteger a reputação da iniciativa.

Termo para lembrar: Rollback organization | Base técnica: referência 1.

10. Projeto acabou: quem mantém a mudança?

Atribuiria orçamento, manutenção e suporte depois da implantação inicial. Uma equipe temporária pode construir o começo, mas capacidades adotadas por muitos consumidores precisam de ownership durável para não degradar após o projeto.

Termo para lembrar: Sustainability | Base técnica: referência 2.

11. Como tratar uma migração interna que conflita com compromissos feitos a clientes?

Eu identificaria interfaces, prazos e capacidades prometidos antes de impor o novo processo, envolvendo os donos desses compromissos. A equipe não pode descumprir um contrato externo só porque o padrão interno mudou; precisamos adaptar a transição, negociar ou manter compatibilidade temporária explícita.

Termo para lembrar: External commitment compatibility

12. Como coordenar adoção entre equipes com fusos e calendários diferentes?

Eu dividiria ondas por disponibilidade real de suporte e responsáveis, registrando janelas locais e critérios de escalonamento. Uma data global aparentemente simples pode cair fora do expediente de parte da operação; a transição precisa preservar capacidade de resposta quando houver falhas.

Termo para lembrar: Time-zone-aware rollout

13. Qual critério libera a próxima onda?

Resultados de funcionamento, operação, suporte e valor previamente definidos. Eu não avançaria apenas porque o cronograma chegou; métricas e bloqueios do piloto precisam informar capacidade de absorver novos consumidores.

Termo para lembrar: Wave readiness | Base técnica: referência 1.

14. Como evitar sobrecarregar equipe de suporte na migração?

Limitaria ondas pela capacidade real, produziria exemplos e trataria dúvidas recorrentes na ferramenta. Adoção simultânea pode criar fila humana, mesmo quando a infraestrutura suporta perfeitamente os novos serviços.

Termo para lembrar: Support capacity | Base técnica: referência 1.

15. Como evitar permissões órfãs depois de uma mudança de equipe?

Eu inventariaria grupos, identidades de serviço e acessos privilegiados ligados à responsabilidade transferida, validando concessões antes de revogar o antigo. A revisão precisa acompanhar recursos reais, não apenas o organograma; mover pessoas no diretório não corrige automaticamente todos os sistemas integrados.

Termo para lembrar: Access lifecycle during reorganization

16. Pode impor prazo único para todos consumidores?

Só com avaliação de dependências e risco. Eu poderia agrupar por complexidade e criticidade, mantendo exceções justificadas; uma data simples administrativamente pode produzir interrupções ou contornos inseguros na prática.

Termo para lembrar: Migration segmentation | Base técnica: referência 1.

17. Como lidar com consumidor desconhecido de API antiga?

Usaria telemetria, inventário e comunicação para identificar uso antes da retirada. Ausência numa lista manual não prova ausência real; o plano precisa considerar versões antigas e integrações pouco frequentes.

Termo para lembrar: Consumer discovery | Base técnica: referência 1.

18. Treinamento deve ensinar botões ou princípios?

Ambos conforme necessidade, mas eu priorizaria tarefas reais e entendimento de limites. Saber navegar não basta para operar falhas; exemplos de recuperação ajudam formar autonomia sem depender sempre do time central.

Termo para lembrar: Operational learning | Base técnica: referência 1.

19. Como preservar a entrega do produto durante uma migração longa?

Eu reservaria capacidade explícita para migração, operação e evolução, limitando trabalho simultâneo e acompanhando filas. Prometer a mesma velocidade sem retirar nenhuma demanda apenas esconde sobrecarga; ajustaria escopo ou prazo conforme a capacidade observada e os riscos das jornadas em produção.

Termo para lembrar: Migration capacity budget

20. Qual risco de manter processo antigo indefinidamente?

Duplica manutenção e permite divergência de controles ou conhecimento. Eu definiria critérios de retirada e migração final, preservando contingência apenas enquanto ela tem propósito operacional claro e custo aceito.

Termo para lembrar: Legacy process retirement | Base técnica: referência 1.

21. Como comunicar mudança de responsabilidade?

Explicitaria decisões, interfaces, plantão e escalonamento, com confirmação dos envolvidos. Comunicação genérica não resolve ambiguidades; durante incidente, cada parte precisa saber quem pode agir e quem responde pela jornada.

Termo para lembrar: Responsibility communication | Base técnica: referência 1.

22. Como evitar que piloto esconda trabalho manual da plataforma?

Mediria intervenções e passos fora do caminho padrão, registrando esforço do fornecedor interno. Um serviço criado rapidamente com ajuda intensiva não prova que o autosserviço funcionará em escala.

Termo para lembrar: Hidden support effort | Base técnica: referência 1.

23. Quando mudar processo antes da ferramenta?

Quando a dor principal vem de regra, aprovação ou ownership mal definido. Automatizar um fluxo inadequado pode acelerar desperdício; eu validaria o processo desejado antes de construir toda a plataforma.

Termo para lembrar: Process-first change | Base técnica: referência 1.

24. Como tratar discordância legítima de uma equipe?

Entenderia requisito e custo específico, comparando com o padrão. Exceção pode ser correta; eu registraria critérios e responsabilidade, evitando transformar uniformidade em objetivo superior ao resultado e à segurança.

Termo para lembrar: Legitimate exception | Base técnica: referência 1.

25. Quem financia esforço de migração?

A decisão precisa reconhecer benefício compartilhado e custo nos consumidores. Eu alinharia capacidade e prioridades com liderança, evitando exigir adoção sem reservar tempo para testes, adaptação e operação segura.

Termo para lembrar: Migration investment | Base técnica: referência 1.

26. Como preparar rollback de uma mudança organizacional?

Preservaria continuidade de operação, acessos e responsáveis durante a transição. Não se trata apenas de mover nomes de volta; conhecimento e trabalho em andamento precisam continuar atendidos sem perda de accountability.

Termo para lembrar: Organizational continuity | Base técnica: referência 1.

27. Qual papel de champions internos?

Podem ajudar descoberta, feedback e disseminação, mas não substituem suporte sustentável. Eu evitaria depender de voluntariado permanente para uma capacidade crítica, tornando aprendizado e documentação acessíveis a todos os consumidores.

Termo para lembrar: Change champions | Base técnica: referência 1.

28. Como distinguir adoção superficial de mudança efetiva?

Observaria se o fluxo real usa novos controles e se práticas antigas foram retiradas. Preencher cadastro ou fazer login não demonstra mudança de comportamento nem redução do risco que motivou o projeto.

Termo para lembrar: Behavioral adoption | Base técnica: referência 1.

29. O que fazer quando benefício previsto não aparece?

Revisaria hipóteses com dados e usuários, podendo reduzir escopo, ajustar ou interromper. Continuar expandindo para justificar investimento passado aumenta custo; a decisão precisa considerar valor futuro e alternativas.

Termo para lembrar: Adoption reassessment | Base técnica: referência 1.

30. Como mostrar alcance Principal nesse tema?

Eu conectaria estratégia, arquitetura, capacidade das equipes e medição de resultado, planejando transição sustentável. Isso é amplitude de responsabilidade e influência, não quantidade de ferramentas implantadas ou autoridade formal declarada.

Termo para lembrar: Organizational technical leadership | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Descoberta com usuários versus desenho central isolado

Ganho. Revela fricções e necessidades reais

Custo e motivo. Consome tempo antes da implementação

Como decidir. Quando mudança afeta rotinas de várias equipes

02. Piloto representativo versus caso mais fácil

Ganho. Produz aprendizado transferível

Custo e motivo. Pode demorar mais para mostrar sucesso inicial

Como decidir. Quando diversidade dos consumidores é relevante

03. Ondas limitadas versus migração simultânea

Ganho. Controla suporte e exposição

Custo e motivo. Prolonga coexistência temporária

Como decidir. Quando capacidade humana é restrita

04. Treino prático versus documentação apenas

Ganho. Demonstra autonomia em tarefas reais

Custo e motivo. Exige tempo de especialistas e participantes

Como decidir. Quando operação possui conhecimento tácito

05. Handoff gradual versus mudança imediata de dono

Ganho. Preserva continuidade e aprendizagem

Custo e motivo. Mantém responsabilidade compartilhada por período

Como decidir. Quando serviço é crítico ou complexo

06. Incentivo alinhado versus mandato puro

Ganho. Aumenta adesão sustentável

Custo e motivo. Pode exigir investimento nos consumidores

Como decidir. Quando custo local bloqueia benefício coletivo

07. Depreciação longa versus curta

Ganho. Dá tempo a clientes e equipes

Custo e motivo. Mantém contratos antigos e custo de suporte

Como decidir. Quando consumidores não evoluem simultaneamente

08. Métricas de resultado versus volume de adoção

Ganho. Demonstra valor efetivo

Custo e motivo. É mais difícil atribuir causalidade

Como decidir. Quando sucesso precisa ir além de uso obrigatório

09. Contingência explícita versus expansão irreversível

Ganho. Permite conter falhas e aprender

Custo e motivo. Exige preservar capacidade e processo anterior temporariamente

Como decidir. Quando mudança ainda possui incerteza relevante

10. Equipe permanente versus projeto temporário

Ganho. Mantém evolução e suporte após lançamento

Custo e motivo. Cria custo recorrente

Como decidir. Quando a capacidade se torna dependência organizacional

Duas alternativas ao caminho proposto

1. Piloto voluntário

Quando faz sentido. Há incerteza sobre utilidade

Como implementar. Testar com equipe representativa e feedback

Limites. Entusiastas podem não representar todos

2. Ondas mandatórias com suporte

Quando faz sentido. Há prazo e necessidade organizacional clara

Como implementar. Migrar por risco com capacidade de atendimento

Limites. Exige coordenação e critérios de exceção

Referências e alcance da evidência

1. Google SRE: lançamentos

Prontidão e coordenação de lançamentos. As respostas e parâmetros de aplicação são propostas autorais.

https://sre.google/sre-book/reliable-product-launches/

2. Team Topologies: conceitos

Responsabilidade, interação e carga cognitiva. As respostas e parâmetros de aplicação são propostas autorais.

https://teamtopologies.com/key-concepts

Continue em: T117 · T119

T119 / L. Alcance Staff+/Principal

Condução da entrevista de 60 minutos

Roteiro de sessenta minutos é uma estratégia flexível de comunicação, não uma rubrica oficial. Arquitetura de treino; não descreve implementação interna do Nubank.

Quando usar. Treino de entrevistas de arquitetura com discussão e mudança de requisitos.

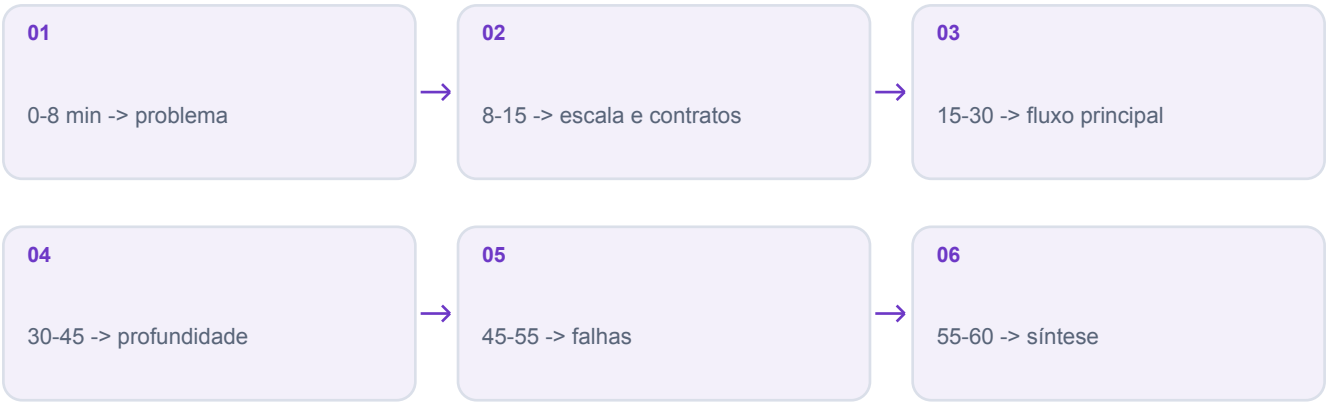
Quando evitar ou limitar. Recitar tecnologias, tentar cobrir enciclopédia ou prometer aprovação garantida.

Uma abertura possível

Eu começaria por confirmar usuário resultado e restrições centrais. Depois explicaria declarar propriedades essenciais antes das peças, preservando as restrições do caso.

Peças e tecnologias. Quadro · Estimativas · Invariantes · Diagramas · Trade-offs

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Confirmar usuário resultado e restrições centrais; validar enunciado reformulado em voz alta.

2. Declarar propriedades essenciais antes das peças; validar contraexemplo simples por propriedade.

3. Estimar números que mudam decisões arquiteturais; validar ordem de grandeza e sensibilidade.

4. Mostrar fluxo mínimo completo primeiro; validar uma solicitação atravessando o desenho.

5. Priorizar risco mais relevante do caso; validar falha ou limite concreto.

30 perguntas para sustentar o desenho

01. Qual primeira pergunta faria ao entrevistador?

Eu confirmaria usuário, resultado e restrições que mudam a arquitetura, reformulando o problema em voz alta. Perguntas iniciais precisam produzir escopo suficiente para começar, sem consumir a entrevista tentando eliminar toda incerteza.

Termo para lembrar: Scope | Base técnica: referência 1.

02. Como escolher o que não pode falhar?

Declararia poucas propriedades essenciais, como não duplicar efeito ou não expor outro cliente. Depois ligaria cada uma a uma fronteira e mecanismo, para que as peças do desenho tenham razão verificável de existir.

Termo para lembrar: Invariants | Base técnica: referência 2.

03. Quantas contas preciso fazer no começo?

Faria contas de ordem de grandeza que influenciam capacidade, armazenamento e prazo, explicitando hipóteses. Não gastaria tempo em precisão artificial; mostraria qual mudança de volume faria escolher outra solução e qual teste validaria.

Termo para lembrar: Sizing | Base técnica: referência 1.

04. Quando desenhar banco e fila?

Desenharia uma solicitação completa, incluindo resultado e consulta de estado, antes de expandir. Banco e fila entram para resolver responsabilidades concretas; um conjunto de caixas sem caminho não demonstra que a jornada funciona.

Termo para lembrar: Happy path | Base técnica: referência 2.

05. Como escolher aprofundamento de Staff?

Escolheria o risco central do caso para aprofundar implementação, concorrência e recuperação. Em chargeback, por exemplo, resultado incerto e identidade importam mais que explicar todo catálogo de tecnologias disponíveis na nuvem.

Termo para lembrar: Deep dive | Base técnica: referência 1.

06. Entrevistador muda requisito: o que dizer?

Ao receber requisito novo, explicaria qual hipótese deixou de valer e ajustaria a decisão. Preservaria invariantes relevantes e discutiria custo adicional, tratando a entrevista como colaboração técnica em vez de defesa rígida do desenho inicial.

Termo para lembrar: Interruption | Base técnica: referência 2.

07. Qual falha testar antes do encerramento?

Percorreria a falha entre commit, efeito externo e resposta, mostrando estados e retomada. Essa sequência demonstra se a solução perde trabalho, duplica ou reconhece incerteza; citar resiliência sem esse mecanismo não basta.

Termo para lembrar: Failure | Base técnica: referência 1.

08. Como evitar resposta depende sem conclusão?

Eu diria de quais condições a escolha depende, assumiria as do exercício e decidiria. Depois apresentaria alternativa e gatilho de revisão, evitando responder depende sem oferecer uma proposta utilizável ao entrevistador.

Termo para lembrar: Tradeoff | Base técnica: referência 2.

09. Ainda faltam dez minutos e desenho está enorme?

Com tempo curto, fecharia fluxo, garantia crítica, falha principal e riscos restantes, em vez de acrescentar componentes periféricos. A síntese precisa continuar coerente e mostrar quais pontos foram conscientemente deixados para aprofundamento posterior.

Termo para lembrar: Pacing | Base técnica: referência 1.

10. O que dizer nos minutos finais?

Resumiria como a solução atende requisitos, quais custos aceita e como evolui com evidência. Não prometeria aprovação nem perfeição; uma boa conclusão deixa claras as fronteiras de garantia e os próximos testes relevantes.

Termo para lembrar: Close | Base técnica: referência 2.

11. Como abrir sem gastar dez minutos perguntando tudo?

Eu resumiria o objetivo e faria poucas perguntas que mudam arquitetura: usuário, operação principal, escala e garantia crítica. Declararia hipóteses restantes e confirmaria antes de aprofundar o desenho.

Termo para lembrar: Focused discovery | Base técnica: referência 1.

12. Se o entrevistador não fornece volume, o que fazer?

Proporia números plausíveis como hipóteses e mostraria sensibilidade. O valor da conta é orientar decisão; eu não fingiria conhecer escala interna nem paralisaria esperando precisão que o exercício não oferece.

Termo para lembrar: Sizing assumptions | Base técnica: referência 1.

13. Quando desenhar primeiro componente?

Depois de alinhar objetivo e invariantes mínimos, eu percorreria uma solicitação completa. Começar cedo ajuda conversa, mas cada caixa precisa ter responsabilidade e fluxo, em vez de uma lista de tecnologias conhecidas.

Termo para lembrar: Incremental drawing | Base técnica: referência 1.

14. Como escolher banco sem parecer dogmático?

Eu ligaria acesso, transação e escala à escolha, apresentando uma alternativa viável e o custo aceito. Se surgir requisito novo, revisaria a decisão; preferência por marca não substitui justificativa.

Termo para lembrar: Database rationale | Base técnica: referência 1.

15. O entrevistador pede aprofundamento que você desconhece: o que dizer?

Eu distinguiria o que sei da hipótese e raciocinaria pelos requisitos, sem inventar garantia. Explicaria o que verificaria na documentação e manteria uma solução cujo limite consigo defender.

Termo para lembrar: Honest uncertainty | Base técnica: referência 1.

16. Como demonstrar idempotência além de citar a palavra?

Eu nomearia a intenção, o escopo da chave, onde a unicidade é imposta e a fronteira atômica do efeito. Depois mostraria o crash após commit e antes da resposta, explicando a retomada.

Termo para lembrar: Idempotency walkthrough | Base técnica: referência 1.

17. Como responder por que não usar Kafka?

Eu verificaria buffering, replay, múltiplos consumidores e operação necessária. Se banco e workers atendem, começaria simples; se o log agrega valor, explicaria esse benefício e os custos sem desqualificar alternativas.

Termo para lembrar: Technology restraint | Base técnica: referência 1.

18. Como mostrar nível Staff sem desenhar quarenta caixas?

Eu aprofundaria uma decisão central, conectando negócio, implementação, falhas e operação. Uma solução pequena com garantias explicadas demonstra mais julgamento que muitas peças sem contrato ou mecanismo verificável.

Termo para lembrar: Depth over inventory | Base técnica: referência 1.

19. Como agir ao perceber que uma estimativa de capacidade feita por você está errada?

Eu corrigiria a conta em voz alta, mostrando qual premissa ou unidade falhou e quais decisões dependem dela. Não reconstruiria tudo automaticamente: verificaria se a ordem de grandeza muda a escolha e atualizaria apenas as consequências relevantes para o desenho apresentado.

Termo para lembrar: Reasoning correction

20. Como conduzir a conversa quando há dois entrevistadores fazendo perguntas diferentes?

Eu reconheceria as duas questões e proporia uma ordem, esclarecendo se uma altera premissas da outra. Ao concluir a primeira, retomaria explicitamente a pendência; isso preserva colaboração e evita abandonar uma pergunta enquanto tento responder tudo simultaneamente sem profundidade suficiente.

Termo para lembrar: Interview collaboration

21. Qual conta simples demonstra entendimento de backlog?

Eu compararia chegada e processamento, estimando crescimento ou drenagem. Se chegam cem e saem cento e vinte por segundo, apenas vinte drenam o acumulado; isso conecta capacidade ao prazo real de recuperação.

Termo para lembrar: Backlog arithmetic | Base técnica: referência 1.

22. Como introduzir SLO sem decorar percentuais?

Eu definiria jornada, resultado válido, limite e janela, perguntando a criticidade. Disponibilidade, latência e integridade são dimensões diferentes; não escolheria números altos apenas para parecer ambicioso sem discutir custo.

Termo para lembrar: SLO framing | Base técnica: referência 1.

23. Quando discutir segurança no mock?

Desde identidade e autorização do fluxo principal, depois aprofundando riscos relevantes. Eu evitaria um checklist final desconectado; dados sensíveis, ações financeiras e integrações precisam de controles nas fronteiras onde acontecem.

Termo para lembrar: Integrated security | Base técnica: referência 1.

24. Qual desenho de falha costuma ser esclarecedor?

Eu marcaria o commit, o efeito externo e a resposta, interrompendo entre eles. Isso revela duplicidade, perda e incerteza, permitindo explicar outbox, idempotência ou reconciliação com propósito concreto.

Termo para lembrar: Failure boundary | Base técnica: referência 1.

25. Como diferenciar requisito de preferência do entrevistador?

Eu confirmaria a consequência esperada: prazo, custo, stack obrigatória ou apenas hipótese. Respeitaria restrições dadas e explicaria trade-offs, sem assumir que uma tecnologia mencionada é necessariamente uma exigência absoluta.

Termo para lembrar: Constraint clarification | Base técnica: referência 1.

26. Pode contestar uma sugestão da banca?

Sim, com respeito e mecanismo concreto. Eu mostraria o risco, proporia alternativa e perguntaria pela hipótese que falta, mantendo colaboração; discordar tecnicamente não exige transformar a conversa numa disputa pessoal.

Termo para lembrar: Constructive disagreement | Base técnica: referência 1.

27. Como pedir alguns segundos para organizar um aprofundamento sem perder a conversa?

Eu diria qual questão estou organizando, por exemplo a fronteira de commit e os efeitos de um timeout, e faria uma pausa curta. Depois apresentaria uma sequência objetiva; preencher silêncio com tecnologias desconexas dificulta acompanhar o raciocínio e avaliar a decisão.

Termo para lembrar: Deliberate communication pause

28. O que preparar antes de quinta-feira?

Eu treinaria casos novos cronometrados, explicando invariantes e falhas sem consulta, e revisaria lacunas observadas. Mais material só ajuda quando transforma execução; não existe lista que garanta aprovação absoluta.

Termo para lembrar: Practice strategy | Base técnica: referência 1.

29. Como avaliar um mock depois?

Compararia escopo, coerência, profundidade, adaptação e tempo, usando exemplos específicos. Identificaria decisões sem mecanismo ou garantias exageradas e repetiria esses trechos, sem reduzir avaliação a quantidade de tecnologias citadas.

Termo para lembrar: Mock review | Base técnica: referência 1.

30. Como responder quando a banca pede pseudocódigo durante o desenho arquitetural?

Eu escolheria o trecho que materializa a garantia discutida, mostrando entradas, validação, transação e resultado de conflito. Declararia o que é atômico e o que permanece remoto; poucas linhas bem escolhidas demonstram implementação sem transformar o restante da entrevista numa codificação completa.

Termo para lembrar: Executable design boundary

10 trade-offs: ganho, custo e decisão

01. Perguntas focadas versus levantamento exaustivo

Ganho. Libera tempo para desenho e aprofundamento

Custo e motivo. Mantém hipóteses que precisam ser declaradas

Como decidir. Quando entrevista possui limite rígido

02. Invariantes primeiro versus tecnologias primeiro

Ganho. Liga mecanismos à correção necessária

Custo e motivo. Exige esclarecer semântica antes das caixas

Como decidir. Quando domínio possui efeitos importantes

03. Estimativa aproximada versus cálculo detalhado

Ganho. Orienta decisão rapidamente

Custo e motivo. Não oferece dimensionamento final de produção

Como decidir. Quando dados do exercício são incompletos

04. Fluxo mínimo versus arquitetura extensa inicial

Ganho. Facilita entendimento e revisão

Custo e motivo. Adia detalhes de escala e operação

Como decidir. Quando é preciso alinhar raciocínio com a banca

05. Deep dive central versus cobertura superficial ampla

Ganho. Demonstra mecanismo e julgamento

Custo e motivo. Deixa temas secundários menos explorados

Como decidir. Quando tempo não permite toda a enciclopédia

06. Adaptar hipótese versus defender solução inicial

Ganho. Mostra colaboração e coerência com requisito

Custo e motivo. Pode exigir redesenhar parte da proposta

Como decidir. Quando nova informação altera a decisão

07. Contraexemplo concreto versus lista de falhas

Ganho. Torna garantia verificável

Custo e motivo. Cobre menos cenários explicitamente

Como decidir. Quando uma falha revela o risco principal

08. Escolha condicionada versus receita universal

Ganho. Explica contexto e alternativa

Custo e motivo. Exige assumir parâmetros com transparência

Como decidir. Quando não existe opção melhor em todos os cenários

09. Síntese antecipada versus adicionar peças até o fim

Ganho. Preserva fechamento compreensível

Custo e motivo. Alguns detalhes ficam como próximos passos

Como decidir. Quando restam poucos minutos

10. Mock sem consulta versus estudo guiado

Ganho. Mede execução e lacunas reais

Custo e motivo. Pode frustrar e exige feedback específico

Como decidir. Quando fundamentos já foram estudados

Duas alternativas ao caminho proposto

1. Mock guiado

Quando faz sentido. Conceitos ainda estão sendo aprendidos

Como implementar. Pausar explicar e repetir decisões

Limites. Não mede desempenho independente

2. Mock cronometrado

Quando faz sentido. É preciso avaliar execução real

Como implementar. Resolver caso novo com interrupções e revisão

Limites. Requer feedback técnico específico

Referências e alcance da evidência

1. Nubank: contratação de engenharia

Orientação pública geral; não é rubrica individual de Staff nem garantia de aprovação. As respostas e parâmetros de aplicação são propostas autorais.

https://building.nu.com/pt-br/como-e-o-processo-de-contratacao-dos-engenheiros-de-software-no-nubank/

2. Azure: princípios de design

Orientação geral de desenho de aplicações. As respostas e parâmetros de aplicação são propostas autorais.

https://learn.microsoft.com/en-us/azure/architecture/guide/design-principles/

Continue em: T118 · T120

T120 / L. Alcance Staff+/Principal

Validação por invariantes e contraexemplos

Invariantes tornam afirmações arquiteturais testáveis; contraexemplos revelam garantias ausentes. Arquitetura de treino; não descreve implementação interna do Nubank.

Quando usar. Revisão de concorrência, dinheiro, migrações e recuperação.

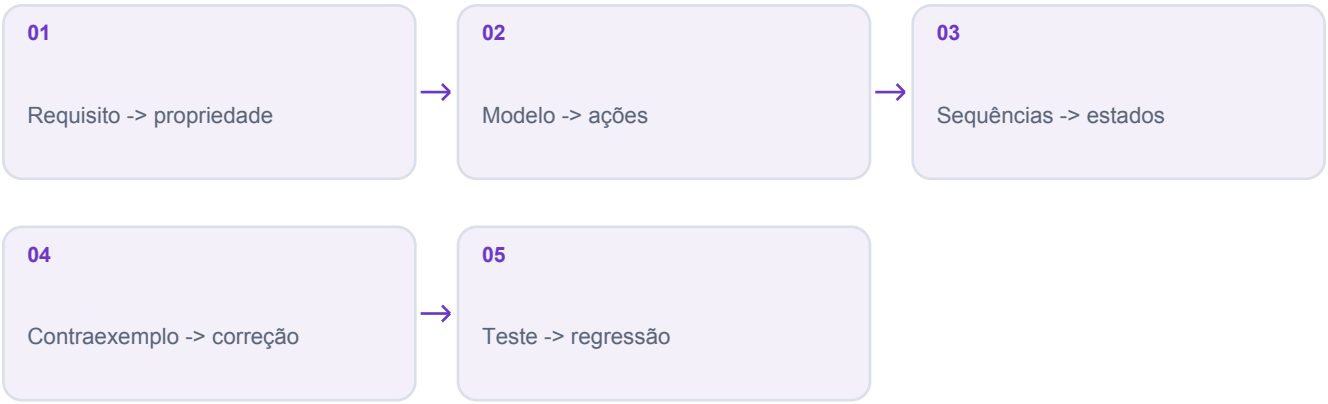
Quando evitar ou limitar. Confundir exemplos felizes ou testes passando com prova universal de correção.

Uma abertura possível

Eu começaria por definir identidade e efeito máximo permitido. Depois explicaria definir progresso sob hipóteses explícitas, preservando as restrições do caso.

Peças e tecnologias. Hypothesis · TLA+ · State machine · Model checking · Fault injection

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Definir identidade e efeito máximo permitido; validar mesma intenção executada duas vezes.

2. Definir progresso sob hipóteses explícitas; validar recuperação após falha transitória.

3. Enumerar transições e pré-condições válidas; validar sequências de comandos concorrentes.

4. Pausar dono até expirar e retomá-lo depois; validar sobreposição controlada de execução.

5. Reduzir sequência preservando violação observada; validar reprodução com poucos passos.

30 perguntas para sustentar o desenho

01. Como escrever dinheiro não duplica de forma testável?

Definiria identidade, efeito e fronteira da propriedade, como no máximo um crédito por intenção válida. Uma frase vaga sobre não duplicar não basta; o teste precisa observar exatamente o resultado que seria incorreto.

Termo para lembrar: Safety | Base técnica: referência 1.

02. Sistema nunca erra mas nunca termina: passou?

Também verificaria progresso sob hipóteses explícitas de recuperação e disponibilidade. Um sistema que nunca executa pode preservar segurança trivialmente e falhar no objetivo; liveness mostra quando uma operação válida deve conseguir avançar.

Termo para lembrar: Liveness | Base técnica: referência 2.

03. Por que modelar estados antes de testes?

Modelaria estados, ações e pré-condições do domínio, incluindo pendência e incerteza. Isso permite gerar sequências além do caminho feliz e detectar transições ilegais que exemplos isolados dificilmente exercitam.

Termo para lembrar: State machine | Base técnica: referência 1.

04. Como desafiar um lock com TTL?

Pausaria o dono do lock até expirar, deixaria outro assumir e depois retomaria o primeiro. A proteção precisa existir no destino ou na identidade do efeito; observar TTL local não prova exclusividade da execução.

Termo para lembrar: Counterexample | Base técnica: referência 2.

05. Falha com mil eventos é impossível investigar?

Reduziria a sequência mantendo a violação, buscando um contraexemplo mínimo reproduzível. Isso facilita entender o mecanismo e criar regressão, em vez de manter um teste enorme que falha sem explicação clara.

Termo para lembrar: Shrinking | Base técnica: referência 1.

06. Modelo simples pode ser mais lento?

Usaria referência simples e independente para comparar a implementação otimizada em entradas pequenas. Ela pode ser lenta, porque o objetivo é clareza de resultado; compartilhar a mesma lógica defeituosa reduziria o valor da comparação.

Termo para lembrar: Reference model | Base técnica: referência 2.

07. Onde interromper pagamento no teste?

Interromperia após possível efeito remoto e antes de confirmação local, depois retomaria a intenção. O teste demonstra se idempotência ou reconciliação realmente protege a fronteira mais perigosa, sem assumir que timeout significa rejeição.

Termo para lembrar: Fault injection | Base técnica: referência 1.

08. Modelo formal prova sistema inteiro?

Declararia hipóteses e partes abstraídas pelo modelo, relacionando-as à implementação. Model checking pode revelar falhas de protocolo, mas não prova automaticamente configuração, código, hardware ou comportamentos que ficaram fora da especificação.

Termo para lembrar: Abstraction | Base técnica: referência 2.

09. Contraexemplo corrigido deve ser descartado?

Preservaria o contraexemplo corrigido como regressão e ligaria à propriedade protegida. Assim uma refatoração futura encontra evidência concreta do risco, sem depender apenas da memória de quem investigou o incidente original.

Termo para lembrar: Regression | Base técnica: referência 1.

10. O que apresentar à banca além do desenho?

Mostraria uma execução que desafia a proposta, acompanhando estados antes, durante e depois. A banca consegue avaliar o mecanismo e seus limites; afirmações de resiliência ficam mais fortes quando existe propriedade verificável associada.

Termo para lembrar: Evidence | Base técnica: referência 2.

11. Qual diferença entre teste de exemplo e propriedade?

Exemplo verifica um caso escolhido; propriedade descreve algo que deve valer para uma classe de execuções. Eu usaria ambos, definindo invariantes claros e gerando sequências que procuram violá-los.

Termo para lembrar: Property-based testing | Base técnica: referência 1.

12. Como escrever conservação de dinheiro?

Definiria soma das partidas por transação e moeda, além de limites de disponibilidade quando exigidos. A propriedade precisa respeitar contas e sinais; somar unidades diferentes produziria um teste aparentemente matemático e incorreto.

Termo para lembrar: Conservation property | Base técnica: referência 1.

13. Exactly-once é uma propriedade suficientemente precisa?

Sozinha, não. Eu perguntaria efeito, identidade, escopo e falhas cobertas. O teste deve dizer, por exemplo, que uma intenção confirmada gera no máximo um efeito financeiro no destino definido.

Termo para lembrar: Guarantee scope | Base técnica: referência 1.

14. Como testar retry depois de commit?

Injetaria falha após gravar o efeito e antes de responder, depois repetiria a mesma intenção. Verificaria resultado existente e ausência de novo efeito, incluindo conteúdo diferente com chave repetida.

Termo para lembrar: Post-commit failure | Base técnica: referência 1.

15. Como testar a recuperação quando o disco enche no meio de uma gravação?

Eu injetaria falha nas etapas de persistência e verificaria quais operações foram reconhecidas como duráveis antes de reiniciar. O estado recuperado precisa respeitar a atomicidade prometida, sem aceitar registros parciais como válidos; também testaria a resposta quando nem o log de erro cabe.

Termo para lembrar: Storage-failure injection

16. Qual contraexemplo derruba dedup só em memória?

Reiniciar o consumidor após efeito e antes de confirmação, depois reenviar. A memória perdeu o registro; eu exigiria proteção durável na fronteira adequada ou idempotência reconhecida pelo destino.

Termo para lembrar: Restart deduplication | Base técnica: referência 1.

17. Como testar saldo com duas leituras concorrentes?

Faria ambos observarem disponibilidade antes dos débitos e controlaria a ordem das escritas. Se cada um decide isoladamente, o total pode exceder saldo; a solução precisa validar dentro da serialização necessária.

Termo para lembrar: Write-skew scenario | Base técnica: referência 1.

18. Safety e liveness podem entrar em tensão?

Sim. Parar tudo pode preservar certas propriedades de segurança e falhar em progresso. Eu declararia hipóteses de disponibilidade e recuperação, verificando que o sistema volta a avançar quando as condições necessárias retornam.

Termo para lembrar: Safety and progress | Base técnica: referência 1.

19. Como descobrir se o gerador de testes nunca alcança os estados perigosos?

Eu mediria cobertura de transições e condições relevantes, construindo exemplos dirigidos para estados raros e comparando com a geração aleatória. Muitas execuções verdes não ajudam se nunca há concorrência ou timeout após commit; o gerador também precisa ser desafiado e validado.

Termo para lembrar: Generator coverage

20. Como escolher estados para uma máquina de testes?

Usaria estados de negócio e evidência, incluindo pendência e UNKNOWN, com ações e pré-condições. Estados somente de interface podem esconder transições financeiras ou recuperação que precisam ser exercitadas.

Termo para lembrar: State model | Base técnica: referência 1.

21. Como validar uma política de retry sem depender de esperas reais e sorte?

Eu usaria relógio controlável e uma sequência programada de respostas, verificando orçamento de tentativas, deadlines e critérios de parada. Testaria jitter por propriedades de intervalo ou distribuição, sem exigir um valor aleatório específico; assim o teste é reproduzível e continua relevante.

Termo para lembrar: Deterministic retry testing

22. Modelo de referência pode compartilhar código com produção?

Com cuidado, pois pode repetir o mesmo defeito. Eu preferiria uma implementação simples e independente da otimização, comparando resultados em entradas pequenas antes de confiar em desempenho da versão complexa.

Termo para lembrar: Independent oracle | Base técnica: referência 1.

23. Como testar ordenação sem exigir ordem global desnecessária?

Definiria propriedade por agregado, versão ou relação causal relevante. Eventos independentes podem chegar em ordens diferentes legitimamente; o teste não deve rejeitar execuções corretas por uma restrição que o produto não exige.

Termo para lembrar: Partial order | Base técnica: referência 1.

24. Como testar recuperação de backup com parceiro externo?

Restauraria estado anterior a alguns efeitos e reintroduziria evidências externas, verificando reconciliação antes de execução. O contraexemplo mostra que restaurar dados locais não desfaz pagamentos ocorridos depois daquele ponto.

Termo para lembrar: Disaster recovery invariant | Base técnica: referência 1.

25. Uma bateria passando prova ausência de bugs?

Não. Ela fornece evidência nas condições exploradas. Eu explicitariaria cobertura, hipóteses e lacunas, combinando testes, revisão e observabilidade, sem apresentar amostragem finita como prova universal de implementação.

Termo para lembrar: Evidence limits | Base técnica: referência 1.

26. Como testar autorização em mudanças concorrentes?

Revogaria permissão durante sessão e tentaria comando ou replay, verificando avaliação no backend. A propriedade precisa impedir acesso indevido mesmo quando interface e tokens locais ainda mostram estado anterior.

Termo para lembrar: Authorization race | Base técnica: referência 1.

27. Qual propriedade aplicar a migração de dados?

Conservação de registros e relações necessárias, com transformação semântica definida. Eu compararia identidades, valores e alterações concorrentes, porque contagem igual pode esconder substituição ou perda compensada por duplicatas.

Termo para lembrar: Migration invariant | Base técnica: referência 1.

28. Como testar parser de arquivo bancário?

Geraria entradas válidas e malformadas, extremos, encoding e truncamentos, verificando rejeição controlada e ausência de efeitos parciais indevidos. O resultado deve preservar manifesto e rastreabilidade, sem confiar apenas em exemplos do parceiro.

Termo para lembrar: Parser properties | Base técnica: referência 1.

29. Quando a propriedade está errada?

Quando não representa o requisito ou proíbe comportamento legítimo. Eu revisaria com domínio e exemplos contrários, porque um teste pode passar perfeitamente e proteger a regra errada com grande rigor técnico.

Termo para lembrar: Specification validation | Base técnica: referência 1.

30. Como usar contraexemplo durante entrevista sem travar conversa?

Eu escolheria uma falha curta e relevante, mostraria a violação e ajustaria o mecanismo. Isso demonstra raciocínio verificável; não preciso enumerar todos os estados para explicar por que a garantia existe.

Termo para lembrar: Counterexample communication | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Propriedades versus exemplos isolados

Ganho. Exploram classes amplas de comportamento

Custo e motivo. Exigem especificar corretamente o que deve valer

Como decidir. Quando entradas e sequências são numerosas

02. Safety e liveness versus só ausência de erro

Ganho. Incluem correção e progresso

Custo e motivo. Precisam de hipóteses sobre ambiente e falhas

Como decidir. Quando parar indefinidamente não atende ao produto

03. Modelo explícito versus testes ad hoc

Ganho. Torna transições e invariantes visíveis

Custo e motivo. Adiciona trabalho de manter modelo e código alinhados

Como decidir. Quando concorrência e estados são complexos

04. Falha controlada versus apenas teste feliz

Ganho. Exercita pontos de recuperação

Custo e motivo. Exige instrumentação e isolamento seguros

Como decidir. Quando crash pode causar perda ou duplicidade

05. Shrinking versus guardar sequência inteira

Ganho. Facilita diagnóstico e regressão

Custo e motivo. Pode custar processamento de redução

Como decidir. Quando gerador encontra falha em cenário grande

06. Referência simples versus só versão otimizada

Ganho. Oferece comparação independente

Custo e motivo. Duplica parte da implementação para testes

Como decidir. Quando otimização dificulta raciocínio direto

07. Teste de fronteira externa versus somente banco local

Ganho. Revela incerteza após efeito remoto

Custo e motivo. Precisa simular ou controlar integração

Como decidir. Quando garantia atravessa processos

08. Model checking versus testes de execução

Ganho. Explora sistematicamente um modelo definido

Custo e motivo. Depende de abstração e limites de estado

Como decidir. Quando protocolo precisa de análise de concorrência

09. Regressão específica versus suíte genérica

Ganho. Preserva evidência de falha conhecida

Custo e motivo. Aumenta manutenção de cenários

Como decidir. Quando bug corrigido teve impacto relevante

10. Contraexemplo curto versus lista extensa de riscos

Ganho. Explica por que um mecanismo é necessário

Custo e motivo. Não cobre sozinho todo espaço de falhas

Como decidir. Quando comunicação precisa ser clara e verificável

Duas alternativas ao caminho proposto

1. Property-based testing

Quando faz sentido. Há implementação e modelo executável

Como implementar. Gerar entradas e sequências com shrinking

Limites. Explora amostras e depende da propriedade

2. Model checking

Quando faz sentido. Estados e concorrência precisam de análise

Como implementar. Explorar modelo abstrato e invariantes

Limites. Limites do modelo não provam todo código

Referências e alcance da evidência

1. Hypothesis: stateful tests

Sequências de ações, estados e invariantes. As respostas e parâmetros de aplicação são propostas autorais.

https://hypothesis.readthedocs.io/en/latest/stateful.html

2. AWS: uso de métodos formais

Modelagem e model checking em sistemas distribuídos. As respostas e parâmetros de aplicação são propostas autorais.

https://lamport.azurewebsites.net/tla/formal-methods-amazon.pdf

Continue em: T119 · T120

C001 / M. Os 50 cenários enviados

Desenhe o sistema de envio e recebimento de Pix

Projetar envio e recebimento de pagamentos instantâneos preservando identidade, saldo e evidências de liquidação. O fluxo abaixo é uma proposta didática, não a arquitetura interna do Nubank.

Quando usar. Quando a jornada envolve autorização, reserva local, comunicação com o arranjo e confirmação posterior.

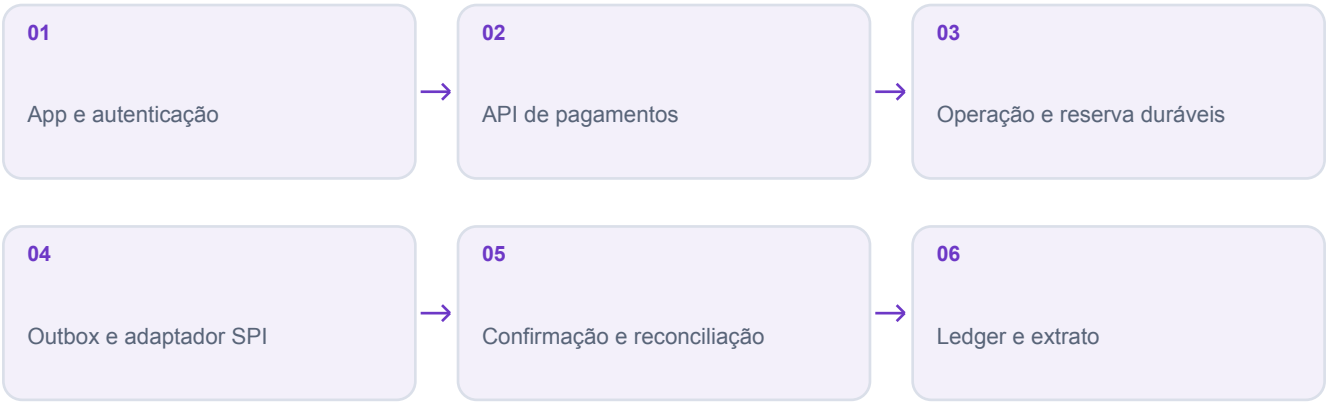
Quando evitar ou limitar. Evitar assumir que timeout é rejeição, que DICT movimenta dinheiro ou que uma compensação local desfaz liquidação externa.

Uma abertura possível

Eu começaria distinguindo intenção, autorização e liquidação. Meu desenho precisa continuar correto quando uma resposta se perde: o cliente verá pendência honesta enquanto buscamos a evidência final.

Peças e tecnologias. Go/Clojure · PostgreSQL ou Datomic · Outbox · Kafka/SQS · Adaptador SPI · Redis somente auxiliar

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. 0-8 min: esclarecer transferência interna versus interbancária, limites, participação no arranjo, jornada de confirmação e fontes normativas aplicáveis.

2. 8-18 min: desenhar operação, identidade externa, reserva, lançamentos e máquina de estados; separar DICT de SPI.

3. 18-32 min: persistir intenção e obrigação de envio atomicamente; autorizar usando a autoridade de saldo e contrato de risco.

4. 32-48 min: tratar resposta perdida, duplicatas, callbacks atrasados, cancelamento e reconciliação; desenhar os caminhos UNKNOWN.

5. 48-60 min: calcular capacidade e backlog, explicar degradação da UX, recuperação regional, métricas e limites restantes.

Cuidado técnico. Atraso entre estados internos e externos é incerteza ou defasagem; não é automaticamente split-brain.

Cuidado técnico. SLA normativo, comportamento de cache e compensação precisam ser verificados por fluxo; não existe um timeout único para todo Pix.

30 perguntas para sustentar o desenho

01. Você desenha Pix interno e externo do mesmo jeito?

Eu separo os caminhos. Uma transferência entre contas sob a mesma autoridade pode confirmar numa transação local; interbancária inclui liquidação externa. Reutilizo identidade e auditoria, mas não invento atomicidade compartilhada com outro participante.

Termo para lembrar: Fronteira de domínio

02. O DICT é o serviço que transfere o dinheiro?

Não. Uso o diretório para endereçamento conforme o contrato; a liquidação pertence ao fluxo apropriado do arranjo. Essa separação permite distinguir erro de consulta de destinatário de incerteza sobre um pagamento já enviado.

Termo para lembrar: DICT

03. Qual regra você escreve antes do diagrama?

Escrevo que uma intenção não pode consumir o saldo duas vezes, que reservas não excedem a política de disponibilidade e que cada liquidação confirmada tem representação contábil reconciliável. Essas invariantes guiam transações e recuperação.

Termo para lembrar: Invariante financeira

04. O usuário toca duas vezes em confirmar. O que acontece?

As duas chamadas carregam a mesma identidade da intenção. A autoridade registra essa identidade com unicidade junto da operação; conteúdo diferente gera conflito. Retorno a operação existente em vez de criar duas ordens financeiras.

Termo para lembrar: Idempotency key

05. Quando você reduz o saldo disponível?

Ao autorizar a intenção, registro uma reserva de forma atômica com a operação, segundo as regras de produto. Liquidação converte a reserva em lançamento definitivo; rejeição confirmada permite liberá-la sem inventar um crédito adicional.

Termo para lembrar: Reserva

06. Duas transferências diferentes tentam gastar o mesmo saldo?

Idempotência não resolve intenções distintas. Serializo a decisão por conta ou uso atualização condicional e isolamento apropriado, validando o saldo disponível no commit. Testo a corrida com as duas transferências concorrentes, não apenas com duplicatas.

Termo para lembrar: Isolamento

07. Como impede operação confirmada localmente sem envio?

A mesma transação que aceita a operação registra sua obrigação de comunicação. Um executor recuperável publica depois. Se o executor cair, a obrigação continua pendente; a repetição preserva a identidade reconhecida no protocolo externo.

Termo para lembrar: Outbox

08. Quais estados você distinguiria?

Proponho criada, autorizada, envio pendente, resultado desconhecido, liquidada e rejeitada, refinados pelo contrato real. Estados representam evidência e responsabilidades. Não exponho um estado genérico failed para juntar rejeição financeira e problema de transporte.

Termo para lembrar: Máquina de estados

09. Qual timeout você usa para o SPI?

Derivo dos tempos normativos aplicáveis à mensagem e do orçamento interno da jornada, verificando a versão vigente. Não confundo prazo do cliente, timeout de socket e prazo de liquidação; cada limite tem ação de recuperação própria.

Termo para lembrar: Deadline

10. O envio terminou em timeout. Devolve o saldo imediatamente?

Não apenas por isso. A liquidação pode ter ocorrido. Mantenho uma situação rastreável e consulto ou reconcilio pelos mecanismos permitidos. Liberar uma reserva sem evidência pode permitir gasto adicional de dinheiro já transferido.

Termo para lembrar: UNKNOWN

11. Recebeu duas confirmações da mesma operação. Credita duas vezes?

O receptor reconhece a identidade da liquidação e aplica a transição e o lançamento uma vez dentro de sua autoridade. Inbox ajuda a deduplicar a mensagem; a identidade financeira também protege contra eventos distintos sobre o mesmo efeito.

Termo para lembrar: Retorno duplicado

12. Uma confirmação chega depois de uma rotina marcar timeout. Quem vence?

Timeout técnico não cria uma decisão financeira final. Valido a evidência externa e uma transição permitida pelo protocolo. Preservo o histórico e o conflito; não deixo a ordem de chegada de workers decidir o destino do dinheiro.

Termo para lembrar: Evento atrasado

13. Usaria Redis para guardar chaves Pix?

Somente dentro das regras e dos controles de segurança do diretório. Defino validade, origem, atualização e tratamento de divergência. Um cache rápido não autoriza endereçar pagamento com dados cuja atualidade não posso justificar.

Termo para lembrar: Cache de endereçamento

14. O DICT ficou lento. O que você degrada?

Reduzo chamadas repetidas, aplico orçamento e informo indisponibilidade da consulta quando necessário. Não invento destinatário ou reciclo informação sensível sem validade. Outros fluxos independentes podem continuar; degradação é por capacidade e por operação.

Termo para lembrar: Graceful degradation

15. O extrato não atualizou, mas o pagamento foi confirmado. Como mostra ao cliente?

A resposta inclui a operação e seu status autoritativo. A tela combina a operação recente com a projeção identificada por versão, sem somar duas vezes. Se o extrato atrasar, mostro atualização pendente em vez de contradizer a confirmação.

Termo para lembrar: Leitura após escrita

16. Qual chave escolhe no Kafka?

Escolho a chave conforme a ordem necessária, por exemplo a conta ou operação. Não assumo ordem entre duas contas apenas por usar Kafka. A autoridade financeira resolve invariantes cruzadas; partições organizam distribuição e consumo.

Termo para lembrar: Particionamento

17. Uma transferência atravessa dois shards do ledger. Como garante equilíbrio?

Escolho explicitamente uma fronteira transacional capaz de registrar a operação ou um protocolo com contas de trânsito e estados recuperáveis. Não faço dois updates independentes e chamo de atômico. O custo da escolha entra no dimensionamento.

Termo para lembrar: Cross-shard

18. Como o caminho de entrada difere do envio?

No recebimento, autentico o canal e identifico a mensagem externa, persisto a evidência e aplico o efeito permitido. Não dependo da disponibilidade do aplicativo. Confirmações e duplicatas seguem o contrato, com reconciliação independente da interface.

Termo para lembrar: Recebimento

19. Devolver um Pix é apagar o lançamento original?

Não. Modelo a devolução como nova operação vinculada à original, com regras, identidade e lançamentos próprios. Preservo valores e evidências anteriores para auditoria e reconciliação; uma correção não deve reescrever o que aconteceu.

Termo para lembrar: Devolução

20. Como faria a conta inicial de infraestrutura?

Estimo pico de ordens, consultas, mensagens por operação e custo das transações. Se cada pagamento gera várias escritas e eventos, dimensiono essa amplificação. Uso carga sintética com contas quentes e latência externa, não só RPS uniforme.

Termo para lembrar: Capacidade

21. O parceiro está lento e a fila cresce. Aceita tudo?

Somente enquanto posso cumprir os compromissos aceitos. Limito backlog por idade e capacidade, separo operações por prazo e reduzo admissão quando necessário. Uma fila durável evita perda, mas não cria capacidade nem estende o prazo financeiro.

Termo para lembrar: Backpressure

22. Abrir o breaker resolve ordens já enviadas?

Não. Ele evita novas chamadas problemáticas, mas ordens ambíguas continuam exigindo resolução. Separo admissão de pagamentos novos das consultas e reconciliações necessárias para concluir os anteriores; abrir o circuito não significa esquecer trabalho existente.

Termo para lembrar: Circuit breaker

23. O que protegeria além de TLS?

Autorização por conta e operação, credenciais do canal, segregação de funções, integridade das mensagens e trilha de acesso. Minimizaria dados em logs e aplicaria controles contra abuso da consulta de destinatários; criptografia não substitui autorização.

Termo para lembrar: Segurança

24. Uma região caiu depois da liquidação externa. O que pode dar errado?

A nova região pode não conhecer uma operação já liquidada se o estado foi replicado com atraso. Antes de retomar, aplico fencing do escritor e reconcilio posições e efeitos. RPO não é apenas uma configuração de armazenamento.

Termo para lembrar: RPO

25. Por que não promover outra região automaticamente em qualquer timeout?

Porque uma partição pode deixar a região antiga ativa. Preciso de autoridade de promoção e proteção contra dois escritores, além de estado recuperável. Automatizo decisões cujas pré-condições consigo verificar e escalo as situações ambíguas.

Termo para lembrar: Failover

26. Como encontra pagamentos que ficaram presos?

Comparo identidades, estados e valores entre operações internas, evidências do arranjo e ledger. Classifico divergência por tipo e idade. Não gero ajuste financeiro automático apenas porque uma mensagem faltou; primeiro determino qual evidência é autoritativa.

Termo para lembrar: Reconciliação

27. Qual disponibilidade você mede?

Separo aceitação válida, confirmação dentro do prazo e disponibilidade de consulta. A home responder não prova que pagamentos estão funcionando. Meço também UNKNOWN envelhecido, reconciliação pendente e taxa de operações incorretas, com denominadores explícitos.

Termo para lembrar: SLO

28. Qual teste derruba uma solução aparentemente boa?

Eu liquidaria no simulador externo e derrubaria a conexão antes da confirmação local. Depois reiniciaria workers e repetiria callbacks. O teste passa se a operação converge com uma identidade financeira e sem liberar dinheiro indevidamente.

Termo para lembrar: Testes

29. Preciso de Datomic para chegar a esse desenho?

Não. PostgreSQL, Datomic ou um ledger especializado podem implementar partes das garantias, com modelos e operação diferentes. Escolho por invariantes, equipe e carga medida; uma tecnologia imutável não implementa sozinha o protocolo externo.

Termo para lembrar: Banco alternativo

30. Como conclui o mock em sessenta minutos?

Recapitulo autoridade de saldo, identidade, estados desconhecidos, caminho de confirmação e recuperação. Explico o gargalo esperado e como validaria. Deixo claro quais números são hipóteses e quais dependem do contrato vigente do arranjo.

Termo para lembrar: Fechamento

10 trade-offs: ganho, custo e decisão

01. Reserva síncrona

Ganho. Evita prometer saldo já comprometido

Custo e motivo. Acrescenta latência e contenção por conta

Como decidir. Quando a autorização depende de disponibilidade financeira atual

02. Consulta autoritativa versus cache

Ganho. Melhora certeza do dado usado

Custo e motivo. Aumenta dependência e custo de rede

Como decidir. Quando endereçamento ou saldo não podem usar dado vencido

03. Outbox versus chamada direta

Ganho. Preserva obrigação após commit

Custo e motivo. Introduz estágio e possível atraso

Como decidir. Quando aceitar a operação implica comunicação externa obrigatória

04. Writer por conta versus escrita multirregional

Ganho. Simplifica decisão concorrente

Custo e motivo. Pode aumentar latência remota

Como decidir. Quando a invariante financeira exige uma ordem clara

05. Consistência da leitura versus projeção

Ganho. Mostra resultado recente com certeza

Custo e motivo. Consulta autoritativa custa mais

Como decidir. Na confirmação de pagamento; extrato histórico pode ser projetado

06. Fila limitada versus admissão ampla

Ganho. Protege prazo e memória

Custo e motivo. Recusa ou adia novas intenções

Como decidir. Quando capacidade de saída e cutoff limitam o trabalho

07. Consulta de status versus reenvio

Ganho. Reduz duplicidade após ambiguidade

Custo e motivo. Depende de evidência disponível no parceiro

Como decidir. Quando o envio pode ter sido aplicado

08. Serialização versus paralelismo

Ganho. Preserva limites por conta

Custo e motivo. Conta quente limita throughput

Como decidir. Quando transferências disputam o mesmo recurso

09. Failover automático versus controlado

Ganho. Reduz tempo de indisponibilidade

Custo e motivo. Exige fencing e estado suficientemente atualizado

Como decidir. Quando pré-condições da promoção são demonstráveis

10. Reconciliação contínua versus batch

Ganho. Detecta divergência cedo

Custo e motivo. Custa processamento e tratamento de eventos atrasados

Como decidir. Quando prazo de resolução exige rapidez sem eliminar conferência de fechamento

Duas alternativas ao caminho proposto

1. PostgreSQL com ledger transacional

Quando faz sentido. Carga e fronteiras cabem numa autoridade relacional

Como implementar. Reservas, operações, lançamentos e outbox com constraints e isolamento

Limites. Shards e multirregião exigem protocolo adicional; medir contenção

2. Ledger especializado com adaptador de pagamentos

Quando faz sentido. Volume e invariantes justificam especialização

Como implementar. Separar autoridade contábil do workflow e preservar operation IDs nos dois

Limites. Novo componente não conhece o resultado do arranjo; exige reconciliação e operação madura

Referências e alcance da evidência

1. BCB: normas e manuais do Pix

Fonte normativa para verificar fluxos, tempos e segurança; este capítulo não fixa números regulatórios.

https://www.bcb.gov.br/estabilidadefinanceira/pix-normas

2. BCB: API DICT

Contrato do diretório e suas operações; não confundir endereçamento com liquidação.

https://bcb.gov.br/content/estabilidadefinanceira/pix/API-DICT.html

3. PostgreSQL: isolamento transacional

Isolamento e necessidade de repetir transações abortadas; não determina a política financeira proposta.

https://www.postgresql.org/docs/current/transaction-iso.html

4. AWS: Transactional Outbox

Atomicidade local entre mudança de estado e obrigação de publicação.

https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/transactional-outbox.html

Continue em: T061 · T062 · T063 · T064 · T091 · T092 · T094 · T096

C002 / M. Os 50 cenários enviados

Como você arquitetaria o autorizador de cartão de crédito?

Autorizador decide rapidamente sobre uma solicitação de cartão, preservando identidade, limites, reservas e evidência. Autorização e liquidação são etapas diferentes.

Quando usar. Quando o emissor precisa responder a solicitações de compra sob prazo de rede e política de risco.

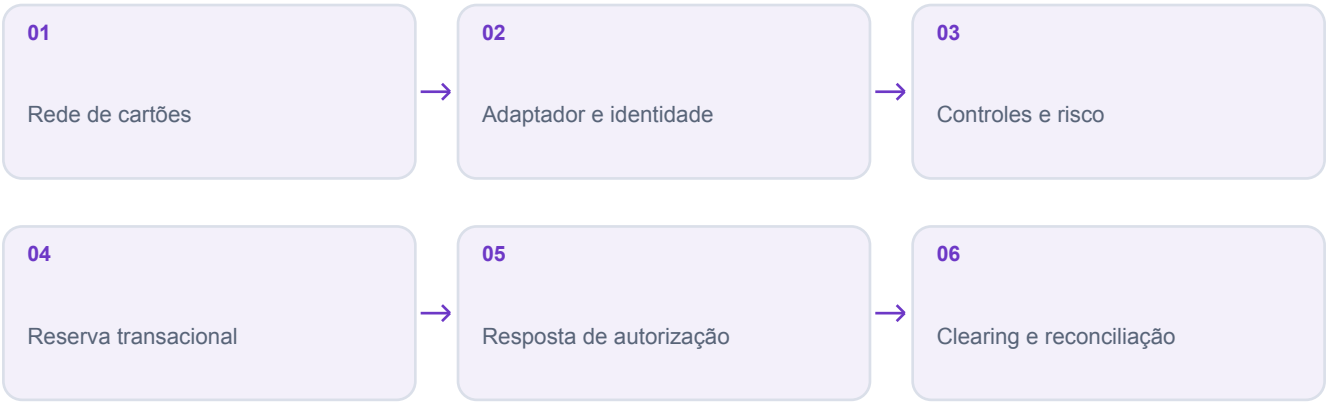
Quando evitar ou limitar. Evitar declarar todo autorizador AP, aprovar por saldo em cache sem limites ou transformar indisponibilidade em aprovação irrestrita.

Uma abertura possível

Vou separar disponibilidade técnica de exposição financeira. A política de contingência será um contrato de risco limitado, enquanto a decisão normal usa uma autoridade consistente de limites e reservas.

Peças e tecnologias. Go/Java/Clojure · PostgreSQL ou ledger especializado · Redis auxiliar · HSM de pagamentos · Kafka · Modelo de risco

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. 0-8 min: esclarecer cartões, tipos de autorização, prazo contratual, limites e contingência permitida.

2. 8-18 min: modelar autorização, reserva, decisão, expiração, captura e reversão com identificadores do protocolo.

3. 18-32 min: desenhar validação, criptografia, risco e atualização atômica dos limites; minimizar dependências síncronas.

4. 32-48 min: resolver duplicatas, respostas perdidas, stand-in, clearing tardio e falhas de HSM ou core.

5. 48-60 min: avaliar p99, contas quentes, capacidade de recuperação, métricas de recusa e rollout de regras.

Cuidado técnico. AP não é requisito universal de um autorizador. Stand-in é contingência sujeita a contrato e risco.

Cuidado técnico. Redis com write-behind não é substituto automático para a autoridade financeira durável.

30 perguntas para sustentar o desenho

01. O autorizador movimenta definitivamente o dinheiro?

Eu diferencio reserva de liquidação. A autorização verifica controles e compromete disponibilidade conforme o produto; captura e clearing confirmam etapas posteriores. Não trato uma resposta aprovada como prova de liquidação final.

Termo para lembrar: Escopo

02. Como define o orçamento de latência?

Começo pelo contrato da rede e divido o tempo entre adaptação, controles, risco, persistência e resposta. Reservo margem de rede e observo percentis. Um número publicado por outro provedor não vira automaticamente o SLA deste emissor.

Termo para lembrar: Deadline

03. Como reconhece retransmissão de uma compra?

Defino a identidade com campos e regras do protocolo, preservando correlação entre autorização e mensagens seguintes. Uma nova conexão não cria nova compra. Também distingo retransmissão de reapresentação legítima, evitando deduplicar intenções diferentes.

Termo para lembrar: Identidade

04. Duas compras simultâneas disputam o limite. O que faz?

Verifico e reservo na mesma autoridade transacional, com condição sobre o disponível ou serialização por conta. Se apenas leio um cache antes de gravar, ambas podem observar o mesmo limite e ultrapassá-lo.

Termo para lembrar: Reserva atômica

05. Onde o Redis ajuda sem virar risco oculto?

Uso para dados auxiliares com validade explícita, como regras distribuídas e metadados não autoritativos. Se armazenar disponibilidade para contingência, imponho exposição limitada e contrato de recuperação. Cache não substitui implicitamente a reserva durável.

Termo para lembrar: Cache

06. Você escolheria AP porque o cartão precisa funcionar?

Não classificaria a jornada inteira assim. Durante partição, preservar certos limites pode exigir recusar ou restringir operações. Uma contingência autorizada aceita risco financeiro delimitado; isso é uma política explícita, não uma aplicação automática do CAP.

Termo para lembrar: CAP

07. Quando aceitaria autorização de contingência?

Somente com regras aprovadas: elegibilidade, valor, frequência, exposição acumulada e tratamento posterior. Registro que a decisão ocorreu em contingência. Não uso o termo como autorização para aprovar qualquer compra quando o core estiver indisponível.

Termo para lembrar: Stand-in

08. Modelo demorou. Aprova baixo valor automaticamente?

Não invento esse limite. Produto e risco definem fallback por segmento, sinal e exposição. Tecnicamente aplico deadline e uma política versionada, registrando a ausência do score. Uma exceção de timeout não deve escolher a política financeira sozinha.

Termo para lembrar: Antifraude

09. Qual dependência removeria do caminho crítico?

Tiraria geração de notificações, analytics e tarefas de enriquecimento dispensáveis à decisão. Persistiria obrigações assíncronas após a decisão. Controles essenciais de segurança e limites permanecem protegidos, mesmo que isso exija uma recusa honesta.

Termo para lembrar: Disponibilidade

10. O HSM ficou lento. O que faz?

Limito concorrência, reaproveito conexões conforme a biblioteca e monitoro filas e latência. Não transfiro PIN ou operação criptográfica sensível para código comum como fallback. A indisponibilidade do controle exige política compatível com o protocolo e segurança.

Termo para lembrar: HSM

11. A reserva confirmou, mas a resposta não chegou à rede?

Uma retransmissão recupera a decisão pela identidade original. Não cria outra reserva. Preservo a decisão e a evidência para reconciliação; também sigo os fluxos de reversão e expiração definidos no contrato.

Termo para lembrar: Resposta perdida

12. Como libera uma autorização cancelada?

Aplico uma transição condicionada e um efeito identificável sobre a reserva. A repetição da reversão não libera duas vezes. Se já houve captura parcial, calculo o restante elegível, em vez de simplesmente devolver o valor original inteiro.

Termo para lembrar: Reversão

13. Uma compra captura menos que o autorizado. Como modela?

Registro valores autorizados, capturados e ainda reservados com relações explícitas. Cada captura tem identidade, e a soma respeita as regras do arranjo. Liberação do excedente é uma transição própria, auditável e repetível com segurança.

Termo para lembrar: Captura parcial

14. E uma autorização que aumenta ao longo da compra?

Trato os incrementos conforme a semântica do protocolo, distinguindo valor acumulado de delta. Uma retransmissão não pode aplicar o aumento novamente. O ledger de reservas mantém correlação e limites acumulados verificáveis.

Termo para lembrar: Incremental authorization

15. A autorização expirou e chegou clearing. Descarta?

Não. Clearing é uma evidência diferente e pode exigir tratamento específico do arranjo. Registro a divergência e encaminho a contabilização ou investigação correta. Um timer local não torna uma mensagem financeira posterior inexistente.

Termo para lembrar: Clearing tardio

16. Como explica depois por que recusou uma compra?

Guardo decisão, códigos de motivo, versão das regras e do modelo, além dos fatos necessários permitidos. Não tento reproduzir a decisão consultando somente regras atuais, porque elas podem ter mudado desde a compra.

Termo para lembrar: Versão de regra

17. Uma conta corporativa recebe milhares de compras. Como escala?

Meço a contenção da autoridade daquela conta. Posso dividir limites previamente alocados por cartão ou dimensão quando o domínio permitir. Não espalho o mesmo limite livremente entre shards e espero que a soma se corrija depois.

Termo para lembrar: Hot account

18. Pode autorizar o mesmo cartão em duas regiões independentes?

Só com um modelo que preserve a exposição acordada: autoridade única, coordenação forte ou alocações limitadas entre regiões. Replicação eventual do saldo com aprovação irrestrita permite decisões incompatíveis durante uma partição.

Termo para lembrar: Multirregião

19. O broker garante que o crédito acontece uma vez?

Não transfiro essa responsabilidade ao broker. Eventos podem repetir e o ledger identifica o efeito financeiro. A reserva, captura e reversão têm identidades distintas; cada uma precisa de proteção adequada no destino.

Termo para lembrar: Idempotência externa

20. O tráfego duplicou. Aumenta o pool do banco?

Primeiro vejo capacidade real e filas. Aumentar conexões pode piorar a saturação. Limito concorrência, protejo o caminho de autorização e removo trabalho secundário; preparo capacidade antes de picos conhecidos e meço o ganho.

Termo para lembrar: Backpressure

21. A transação de reserva deu timeout. Retorna aprovado?

Não invento sucesso nem uma segunda reserva. Recupero a decisão pela identidade dentro do orçamento possível e sigo o protocolo de resultado incerto. O comportamento externo precisa considerar reversão e recuperação sem dupla exposição.

Termo para lembrar: Timeout de banco

22. Quais métricas além de p99 você acompanha?

Taxas de aprovação por motivo, falsas recusas observáveis, contingência, reservas presas, divergência com clearing e disponibilidade das dependências. Separo erro técnico de decisão de risco; aumentar aprovação pode esconder aumento inaceitável de perda.

Termo para lembrar: Observabilidade

23. O que nunca colocaria no log de troubleshooting?

Não registraria PIN, blocos sensíveis ou credenciais. Uso identificadores tokenizados e acesso restrito. A investigação precisa de correlação e motivos, não de copiar todos os dados do protocolo para ferramentas gerais de observabilidade.

Termo para lembrar: Dados sensíveis

24. Como testa uma regra nova?

Executo shadow sem produzir decisão externa, comparo resultados por segmento e depois faço canary limitado. Defino critérios de rollback de risco e de negócio. Reproduzir a requisição em sombra não pode reservar limite outra vez.

Termo para lembrar: Release

25. Pode usar float para limite e reserva?

Uso representação monetária exata com moeda e escala definidas. Valores de rede e liquidação podem ter moedas diferentes; preservo a taxa e o instante aplicáveis. Arredondamento não fica distribuído de forma implícita entre serviços.

Termo para lembrar: Precisão monetária

26. O que precisa sobreviver à perda de uma zona?

Decisões confirmadas, reservas, identidades e obrigações de saída precisam respeitar a durabilidade contratada. Também testo conectividade com HSM e rede externa na zona restante; redundância de aplicações sem essas dependências não basta.

Termo para lembrar: Falha de zona

27. Qual teste revela gasto acima do limite?

Disparo autorizações distintas simultâneas contra o mesmo disponível e introduzo atrasos entre leitura e commit. Depois retransmito algumas e verifico o total reservado. O resultado deve respeitar a política em todos os interleavings testados.

Termo para lembrar: Teste de corrida

28. Você guarda a decisão inteira para sempre?

Defino retenção e minimização por classe de dado e obrigação. Preservo o necessário para explicar efeitos e investigar disputas, sem transformar logging amplo em arquivo permanente de dados sensíveis. Auditoria e telemetria têm finalidades diferentes.

Termo para lembrar: Auditoria

29. PostgreSQL com locks é pouco sofisticado para Staff?

Não. Se atende invariantes, latência e capacidade, é uma escolha defensável. Eu mostraria tempo de lock, contenção e limites de crescimento. Complexidade adicional precisa resolver um gargalo ou requisito demonstrado, não decorar o desenho.

Termo para lembrar: Alternativa simples

30. Qual mensagem final você deixaria à banca?

A decisão é rápida porque reduzimos dependências, e segura porque preservamos identidade e autoridade de limites. A contingência tem exposição limitada e reconciliação. Eu explicaria o que fica indisponível antes de prometer aprovação sem controle.

Termo para lembrar: Conclusão

10 trade-offs: ganho, custo e decisão

01. Reserva consistente versus leitura eventual

Ganho. Preserva limite atual

Custo e motivo. Pode recusar durante falha da autoridade

Como decidir. No caminho normal de autorização

02. Stand-in limitado versus recusa

Ganho. Mantém parte da experiência

Custo e motivo. Assume exposição financeira mensurável

Como decidir. Somente com política de risco e contrato explícitos

03. Score síncrono versus regras locais

Ganho. Usa sinais mais ricos

Custo e motivo. Acrescenta latência e dependência

Como decidir. Quando ganho de risco compensa orçamento

04. Writer por conta versus limite distribuído

Ganho. Facilita concorrência correta

Custo e motivo. Conta quente limita paralelismo

Como decidir. Quando não há alocação independente segura

05. Cache de regras versus consulta central

Ganho. Reduz dependência de rede

Custo e motivo. Exige versão, atualização e fallback

Como decidir. Para regras cuja distribuição atrasada tem política definida

06. Persistência síncrona versus resposta antecipada

Ganho. Decisão recuperável após falha

Custo e motivo. Adiciona tempo de commit

Como decidir. Para não aprovar efeito que pode desaparecer

07. Pooling de HSM versus conexão por operação

Ganho. Reduz setup e jitter

Custo e motivo. Exige limites, saúde e rotação

Como decidir. Quando contrato do HSM suporta reaproveitamento

08. Shadow versus canary

Ganho. Compara sem mudar decisões reais

Custo e motivo. Não mede todos os efeitos de produção

Como decidir. Antes de expor clientes ao novo autorizador

09. Limite por cartão versus compartilhado

Ganho. Distribui contenção e exposição

Custo e motivo. Pode deixar capacidade ociosa fragmentada

Como decidir. Para contas com muitos cartões e orçamento alocável

10. Retenção rica versus minimização

Ganho. Facilita explicar decisões

Custo e motivo. Aumenta exposição e custo

Como decidir. Guardar fatos necessários com acesso e prazo definidos

Duas alternativas ao caminho proposto

1. Autorizador relacional por domínio de conta

Quando faz sentido. Escala medida cabe em partições independentes

Como implementar. Transação curta para decisão, reserva e outbox; leitura auxiliar cacheada

Limites. Requer planejamento para contas quentes e recuperação regional

2. Processamento serializado por conta com log durável

Quando faz sentido. Contenção e previsibilidade justificam ator ou sequenciador

Como implementar. Roteia comandos ao proprietário, persiste antes de confirmar e usa epoch no failover

Limites. Ator em memória não é durabilidade; mudança de proprietário exige coordenação

Referências e alcance da evidência

1. Stripe: real-time authorizations

Exemplo público de autorização síncrona e configuração de timeout; seus prazos não são universais.

https://docs.stripe.com/issuing/controls/real-time-authorizations

2. PostgreSQL: isolamento transacional

Isolamento e necessidade de repetir transações abortadas; não determina a política financeira proposta.

https://www.postgresql.org/docs/current/transaction-iso.html

3. AWS: APIs idempotentes

Identidade estável, repetição e limites do contrato de idempotência.

https://aws.amazon.com/builders-library/making-retries-safe-with-idempotent-APIs/

Continue em: T011 · T019 · T061 · T064 · T092 · T094 · T097

C003 / M. Os 50 cenários enviados

Desenhe o Ledger (Livro-Razão) financeiro do banco

Ledger é a autoridade dos lançamentos financeiros. Um desenho consistente registra partidas equilibradas, identidade da operação, moeda, temporalidade e correções auditáveis.

Quando usar. Quando há saldos, obrigações, liquidação, estornos ou necessidade de explicar a origem de valores.

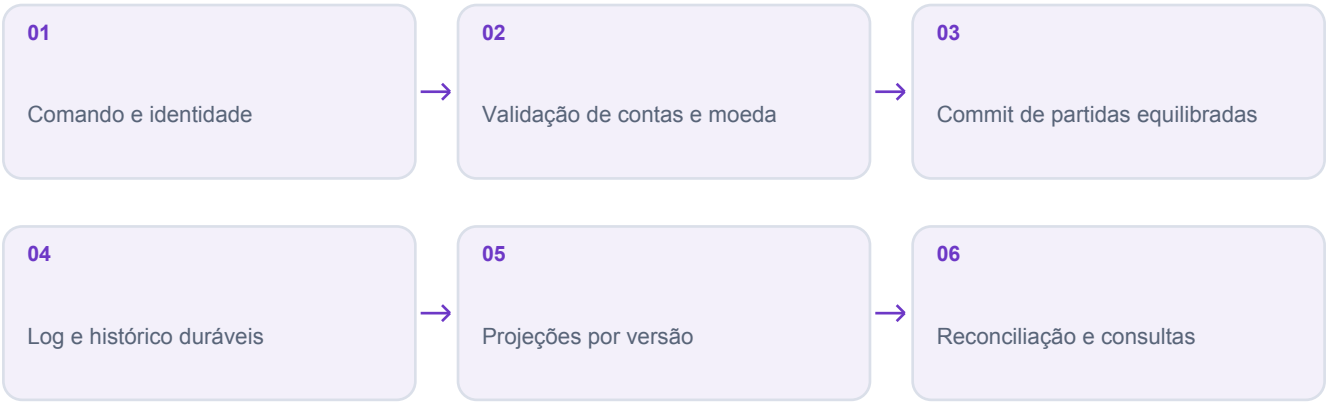
Quando evitar ou limitar. Evitar somar moedas diferentes como se fossem uma só, usar float, reescrever lançamentos confirmados ou exigir event sourcing sem necessidade.

Uma abertura possível

Eu começo pela invariante: cada transação deve estar equilibrada dentro do modelo contábil e da moeda. Depois escolho como garantir atomicidade, identidade e leitura, incluindo o que acontece na recuperação.

Peças e tecnologias. PostgreSQL · Datomic · TigerBeetle como alternativa · Clojure/Go · Outbox/CDC · Projeções de leitura

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. 0-8 min: definir plano de contas, moedas, regras de saldo, histórico e volume; separar contábil de disponível.

2. 8-18 min: modelar transação, entradas, contas, identidade e vínculo de correção; escrever invariantes matemáticas.

3. 18-32 min: demonstrar validação e commit atômico, controle de concorrência e publicação durável.

4. 32-48 min: tratar timeout, estorno, projeção atrasada, partições, replay e restauração de backup.

5. 48-60 min: estimar contenção e I/O, justificar tecnologia, demonstrar testes de propriedades e operação.

Cuidado técnico. Event sourcing não é requisito universal de um ledger correto.

Cuidado técnico. Single-writer ou imutabilidade não dispensam controle de saldo, identidade e protocolo de recuperação.

30 perguntas para sustentar o desenho

01. Qual invariante você garante no commit?

Para cada operação e moeda, verifico que débitos e créditos equilibram conforme o plano contábil. Também valido existência, estado e compatibilidade das contas. Não confundo equilíbrio aritmético com autorização para movimentar qualquer conta.

Termo para lembrar: Partidas dobradas

02. Débito significa sempre diminuir saldo?

Não. O efeito depende da natureza contábil da conta e da perspectiva. Modelo explicitamente ativo, passivo e demais classes necessárias. Para explicar ao cliente, projeto uma visão de saldo, sem apagar a semântica do ledger.

Termo para lembrar: Plano de contas

03. Como representa R$150?

Uso unidade monetária exata e moeda explícita, por exemplo centavos para esse produto. Para instrumentos com outra escala, defino precisão e arredondamento. Não uso ponto flutuante binário como representação do valor financeiro autoritativo.

Termo para lembrar: Representação monetária

04. Como impede gravar só uma das duas partidas?

As entradas da operação e sua identidade confirmam na mesma fronteira transacional. Se uma validação falhar, nada confirma. Se a arquitetura não suporta essa fronteira entre shards, preciso mudar o modelo ou adotar protocolo explícito.

Termo para lembrar: Atomicidade

05. Imutabilidade proíbe qualquer UPDATE no banco?

Não necessariamente. Os lançamentos confirmados são preservados; índices, projeções e metadados operacionais podem mudar sob regras claras. Em PostgreSQL posso proteger o histórico por privilégios e interfaces; o produto não precisa ser um banco imutável para isso.

Termo para lembrar: Append-only

06. O que é um datom?

É um fato com entidade, atributo, valor, transação e indicador de adição ou retração. Uso o modelo para consultar estados históricos. A imutabilidade dos fatos não elimina a necessidade de validar saldos e identidades no momento da transação.

Termo para lembrar: Datomic

07. Serialização do Datomic elimina toda coordenação distribuída?

Não. A ordem transacional vale dentro da base e de suas garantias. Comunicação com broker, parceiro e outra base continua tendo fronteiras próprias. Não estendo a propriedade do transactor a um pagamento externo inteiro.

Termo para lembrar: Transactor

08. Como protege uma transferência repetida?

Associo uma identidade estável à operação e comparo seu conteúdo. A criação dessa identidade e das partidas é atômica. Uma nova tentativa recupera a operação; mesma chave com outro valor ou contas é conflito.

Termo para lembrar: Idempotência

09. Pode calcular disponível apenas somando lançamentos?

Depende do modelo de reservas e limites. Contábil e disponível podem diferir por holds, bloqueios e crédito concedido. Defino uma fórmula verificável e atualizo seus componentes na autoridade correta; uma projeção atrasada não autoriza gasto.

Termo para lembrar: Saldo disponível

10. Duas transferências diferentes consomem a mesma conta. Como evita negativo?

Valido o limite no commit, usando serialização, lock ou operação condicional apropriada. Equilíbrio de partidas não impede gasto acima do permitido. Preciso tanto da invariante contábil quanto da restrição de disponibilidade.

Termo para lembrar: Concorrência

11. Como consulta saldo numa data anterior?

Consulto lançamentos até a fronteira temporal definida ou uso snapshot mais eventos posteriores. Distingo tempo de registro e data efetiva. A pergunta precisa dizer se quer o que sabíamos naquela época ou o saldo corrigido daquele dia.

Termo para lembrar: Time travel

12. Por que guardar effective_at e recorded_at?

Uma correção pode chegar hoje com efeito econômico anterior. Separar os tempos permite reproduzir o conhecimento histórico e recalcular relatórios com ajustes. Sem isso, uma consulta temporal pode mudar de significado sem ninguém perceber.

Termo para lembrar: Bitemporalidade

13. Snapshot substitui o histórico?

Não. É uma aceleração vinculada a uma posição ou versão verificável. Somo apenas eventos posteriores e valido a consistência. Se o snapshot estiver corrompido, reconstruo a projeção a partir da autoridade retida.

Termo para lembrar: Snapshot

14. Pode servir saldo do Redis?

Posso servir uma visão com contrato explícito de frescor. Para autorizar uma operação, uso a autoridade de disponibilidade ou uma estratégia equivalente comprovada. Depois de uma escrita, carrego o resultado recente ou espero uma versão mínima na projeção.

Termo para lembrar: Projeção

15. Como corrige uma transferência de valor errado?

Crio uma operação corretiva vinculada à original, com partidas próprias e autorização adequada. Não apago a evidência anterior. A política define se há reversão completa e novo lançamento ou ajuste incremental, sempre com identidade própria.

Termo para lembrar: Estorno

16. Uma transferência BRL para USD pode ter soma global zero?

Não somo unidades diferentes. Modelo pernas e contas de câmbio para equilibrar cada moeda, guardando cotação, validade e arredondamento. Resultado econômico e exposição cambial exigem representação explícita, não conversão improvisada na leitura.

Termo para lembrar: Moedas

17. Como representa uma tarifa na mesma operação?

Adiciono as partidas para cliente e contas de receita ou obrigação aplicáveis, mantendo equilíbrio por moeda. Não escondo a tarifa diminuindo um saldo fora do ledger. Isso permite conferir o total e explicar a composição.

Termo para lembrar: Taxas

18. Como escala transferências entre shards?

Primeiro tento manter a fronteira necessária sob uma autoridade. Se precisar distribuir, avalio transação distribuída ou contas de trânsito e workflow recuperável. Documentarei estados intermediários; duas escritas independentes não viram uma transferência atômica.

Termo para lembrar: Cross-shard

19. Uma conta de liquidação concentra todas as operações. E agora?

Meço contenção e avalio subdividir contas técnicas por dimensão conciliável, sem mudar o significado contábil. Agregação posterior pode reduzir disputa. Não distribuo livremente uma conta de cliente que exige limite global imediato.

Termo para lembrar: Hot account

20. Quando responde sucesso?

Depois da confirmação compatível com a durabilidade prometida pelo armazenamento. Aceitar no buffer de memória não basta. Distingo confirmação de commit da atualização das projeções e da comunicação externa.

Termo para lembrar: Log durável

21. O commit respondeu timeout. Repete com outra chave?

Não. Preservo a identidade e descubro o resultado na autoridade. Uma segunda chave pode produzir outra transferência válida. Testo esse cenário com perda da resposta após o commit, porque é diferente de rollback comprovado.

Termo para lembrar: Timeout

22. Como informa outros serviços sobre novos lançamentos?

Registro a obrigação de evento com a operação ou uso captura do log com semântica definida. Consumidores lidam com repetição e versão. A atualização do extrato não pode depender de uma chamada volátil após o commit.

Termo para lembrar: Outbox

23. Como impede um administrador de adulterar o histórico?

Uso controles de acesso, separação de funções, trilha independente e verificação de integridade conforme o risco. Append-only na aplicação não bloqueia sozinho um operador privilegiado. Backups e evidências também precisam de proteção e revisão.

Termo para lembrar: Auditoria

24. Imutabilidade significa guardar dados pessoais em todos os eventos?

Não. Minimizaria identificadores sensíveis no ledger e manteria associações sob controles próprios. Exclusão e retenção exigem política por dado e obrigação. Criptografia ajuda, mas não torna qualquer retenção automaticamente adequada.

Termo para lembrar: Privacidade

25. Como restaura sem repetir efeitos externos?

Recupero uma posição consistente e reconcilio com operações que podem ter sobrevivido fora do backup. Não disparo replay com efeitos ativos indiscriminadamente. Identidades financeiras estáveis ajudam a reconhecer o que já foi executado.

Termo para lembrar: Restore

26. Equilíbrio interno prova que o banco está certo?

Não. Um lançamento duplicado pode continuar equilibrado. Comparo ledger com evidências de pagamentos, reservas e parceiros, por identidade, moeda e valor. Equilíbrio é necessário, mas não suficiente para correção de negócio.

Termo para lembrar: Reconciliação

27. Recalcular saldo pelo histórico em toda compra é bom?

É simples conceitualmente, mas custa proporcionalmente ao histórico consultado. Posso manter acumuladores transacionais ou snapshots verificáveis para reduzir leitura. A otimização precisa preservar atomicidade e permitir auditoria contra as entradas de origem.

Termo para lembrar: Complexidade

28. Que propriedades automatizaria?

Equilíbrio por moeda, identidade única, nenhuma perda após falhas admitidas, reservas dentro da política e correções rastreáveis. Gero sequências concorrentes de transferências, falhas e reversões; verifico propriedades finais e intermediárias relevantes.

Termo para lembrar: Teste de propriedades

29. Event sourcing é obrigatório para um ledger Staff?

Não. Um modelo relacional append-only com transações e boas constraints pode atender. Datomic e bancos especializados oferecem outras propriedades e custos. Defendo a escolha por invariantes, operação e carga, não pelo rótulo de senioridade.

Termo para lembrar: Escolha de banco

30. Como encerra a apresentação?

Mostro a operação equilibrada, sua identidade e a fronteira do commit; depois explico leituras, correções e recuperação. O desenho está completo quando consigo narrar tanto o sucesso quanto uma falha após aplicar o efeito sem perder a rastreabilidade.

Termo para lembrar: Conclusão

10 trade-offs: ganho, custo e decisão

01. Histórico append-only versus sobrescrita

Ganho. Preserva fatos e correções

Custo e motivo. Aumenta armazenamento e complexidade de consulta

Como decidir. Para explicar operações financeiras e sua evolução

02. Saldo derivado versus acumulador

Ganho. Derivação facilita conferência

Custo e motivo. Pode custar leitura proporcional ao histórico

Como decidir. Usar acumulador transacional com reconciliação quando latência exigir

03. Writer serializado versus concorrência transacional

Ganho. Simplifica ordem e invariantes

Custo e motivo. Pode criar limite por partição

Como decidir. Quando previsibilidade compensa paralelismo perdido

04. Snapshot frequente versus raro

Ganho. Acelera consultas e recuperação

Custo e motivo. Gera I/O e manutenção adicional

Como decidir. Conforme tempo de replay e volume de alterações

05. Banco relacional versus especializado

Ganho. Relacional favorece equipe e ecossistema

Custo e motivo. Requer implementar semântica financeira cuidadosamente

Como decidir. Especializar quando garantias e carga justificam novo componente

06. Autoridade única versus shards

Ganho. Simplifica atomicidade

Custo e motivo. Limita crescimento ou alcance regional

Como decidir. Distribuir após medir e definir transações entre partições

07. Leitura forte versus projeção

Ganho. Evita dúvida na decisão financeira

Custo e motivo. Custa latência e recursos da autoridade

Como decidir. Projeções para consulta; autoridade para invariantes sensíveis

08. Precisão ampla versus unidades inteiras

Ganho. Suporta instrumentos com escalas diversas

Custo e motivo. Aumenta regras de arredondamento e representação

Como decidir. Escolher por produto, sem usar float para dinheiro autoritativo

09. Correção por novos lançamentos versus edição

Ganho. Mantém rastreabilidade

Custo e motivo. Exige interpretar histórico e vínculos

Como decidir. Quando lançamento já confirmado tem significado auditável

10. Replicação síncrona versus assíncrona

Ganho. Reduz perda admitida após falha

Custo e motivo. Aumenta latência e dependência entre nós

Como decidir. Conforme RPO contratado e distância das réplicas

Duas alternativas ao caminho proposto

1. PostgreSQL com transações append-only

Quando faz sentido. Invariantes e capacidade cabem em autoridade relacional

Como implementar. Operações únicas, partidas numa transação, acumuladores protegidos e outbox

Limites. Constraints entre linhas exigem desenho cuidadoso; sharding não é gratuito

2. Datomic ou ledger especializado

Quando faz sentido. Modelo temporal ou carga financeira justificam ferramenta dedicada

Como implementar. Expressar invariantes na transação e manter identidade até os consumidores

Limites. Operação, capacidade e integração externas continuam sob responsabilidade do desenho

Referências e alcance da evidência

1. TigerBeetle: Financial Accounting

Natureza das contas e interpretação de débitos e créditos.

https://docs.tigerbeetle.com/coding/financial-accounting/

2. Datomic: Transaction Model

Modelo de fatos e transações; não prova arquitetura interna de nenhum banco.

https://docs.datomic.com/transactions/model.html

3. PostgreSQL: isolamento transacional

Isolamento e necessidade de repetir transações abortadas; não determina a política financeira proposta.

https://www.postgresql.org/docs/current/transaction-iso.html

Continue em: T013 · T019 · T028 · T061 · T092 · T093 · T096

C004 / M. Os 50 cenários enviados

Arquitetura para cálculo de rendimento diário de 90 milhões de contas

Calcular rendimento de uma população hipotética de 90 milhões de contas exige definir base temporal, elegibilidade, precisão, contabilização e recuperação. A escala não determina sozinha batch ou streaming.

Quando usar. Quando há cálculo periódico por contrato e necessidade de reproduzir valores e ajustes.

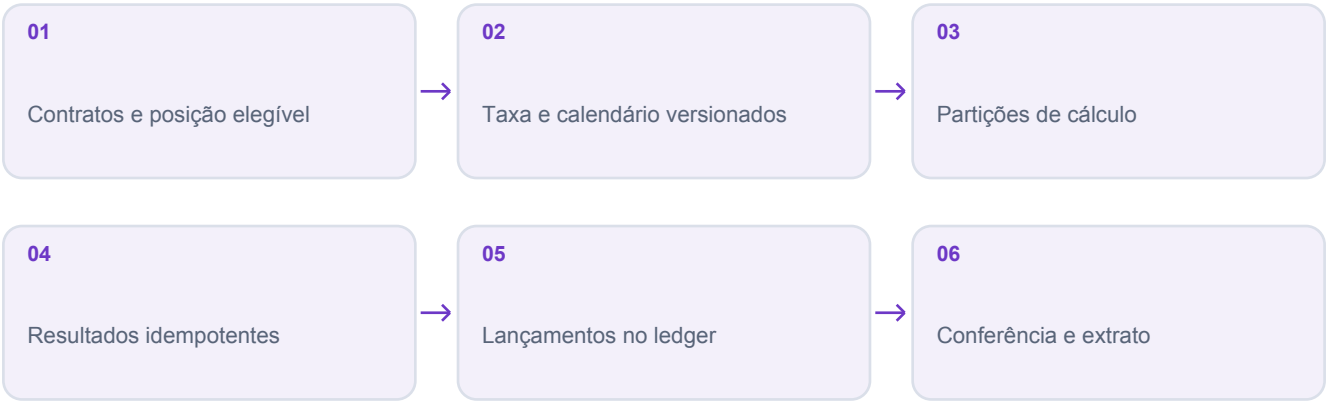
Quando evitar ou limitar. Evitar tratar 90 milhões como número atual do Nubank, somar rendimento visual ao saldo gastável sem liquidação ou descartar batch sem dimensionamento.

Uma abertura possível

Eu fixaria primeiro a regra de rendimento e a data efetiva. Depois mediria quanto trabalho precisa terminar em qual janela; um batch particionado pode ser suficiente e mais simples que manter estado de streaming contínuo.

Peças e tecnologias. PostgreSQL/ledger · S3/Blob · Spark ou workers · Kafka/Flink opcionais · Scheduler durável

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. 0-8 min: definir população hipotética, produtos, calendário, base de cálculo, prazo e diferença entre previsão e crédito.

2. 8-18 min: modelar posição por data, versão de taxa, regra de arredondamento e identidade de cada apuração.

3. 18-32 min: particionar contas, processar snapshots consistentes e registrar resultados e obrigações recuperáveis.

4. 32-48 min: tratar jobs duplicados, taxa corrigida, eventos tardios, falha parcial e backfill sem crédito duplicado.

5. 48-60 min: comparar batch e streaming por custo, prazo e complexidade; conferir totais e demonstrar métricas.

Cuidado técnico. 90 milhões é a população hipotética do enunciado, não uma afirmação sobre o Nubank atual.

Cuidado técnico. Batch pode escalar por particionamento; streaming não é marcador de senioridade.

30 perguntas para sustentar o desenho

01. Qual pergunta vem antes de escolher Spark?

Pergunto qual saldo é elegível, qual período rende, quando credita e quais regras de calendário e arredondamento valem. Sem isso, posso escalar um cálculo incorreto. A arquitetura precisa reproduzir o contrato versionado.

Termo para lembrar: Contrato

02. Noventa milhões de contas obrigam streaming?

Não. Em seis horas, a média é aproximadamente 4.167 contas por segundo, antes de overhead e margem. Meço custo por conta e I/O. Batch particionado pode atender; streaming é uma decisão de frescor e operação.

Termo para lembrar: Janela de processamento

03. Como evita calcular usando saldos de instantes diferentes?

Defino uma posição de corte reproduzível e a origem dos eventos incluídos. Cada resultado referencia essa versão. Não varro uma tabela mutável durante horas e considero todas as leituras como se fossem simultâneas.

Termo para lembrar: Snapshot consistente

04. A taxa foi publicada ou corrigida depois. Como trata?

Persisto a versão e a origem utilizadas. Uma correção produz nova apuração ou ajuste identificado, com comparação contra o já contabilizado. Não recalculo silenciosamente o passado usando a taxa atual.

Termo para lembrar: Taxa versionada

05. Como evita diferença de centavos entre workers?

Uso cálculo decimal ou inteiro com escala e arredondamento especificados. A regra indica em que etapa arredondar. Todos os workers usam a mesma versão, e testes cobrem valores pequenos, limites e acumulação ao longo do tempo.

Termo para lembrar: Precisão

06. Todas as contas recebem crédito todo dia?

Não assumo. Produto, saldo elegível, carência, calendário e outras regras entram no contrato. Materializo os motivos de elegibilidade ou exclusão necessários à explicação; um worker não interpreta regras diferentes por ter recebido dados incompletos.

Termo para lembrar: Elegibilidade

07. Como divide o trabalho?

Distribuo por identificador estável ou ranges de uma fotografia da população, com unidades recuperáveis e tamanho controlado. Mantenho manifestos dos intervalos. Não uso offset crescente sobre uma tabela que está mudando durante a execução.

Termo para lembrar: Particionamento

08. Qual é a chave de uma apuração?

Uso conta, produto, período e versão da apuração, com regra explícita para correções. Reexecutar uma tarefa recupera o mesmo resultado. A identidade do lançamento financeiro também precisa impedir crédito duplicado.

Termo para lembrar: Idempotência de cálculo

09. Checkpoint do Spark garante um único crédito?

Não por si só. O destino financeiro precisa reconhecer a operação. Separaria resultado calculado de obrigação de crédito e usaria identidade estável no ledger. Recuperar processamento não pode repetir um efeito externo como novo.

Termo para lembrar: Exactly-once financeiro

10. Por que separar rendimento exibido e creditado?

Porque estimativa ou provisionamento pode anteceder a contabilização disponível ao cliente. Rotulo a visão e defino quando muda de natureza. Não apresento previsão como saldo gastável sem a regra financeira correspondente.

Termo para lembrar: Provisionado versus liquidado

11. Uma movimentação do dia anterior chega depois do fechamento?

Defino política de fechamento e ajuste. O evento mantém data efetiva e de registro, e o recálculo produz diferença identificada. Não abro o período indefinidamente nem ignoro a movimentação apenas por ter chegado atrasada.

Termo para lembrar: Evento tardio

12. Um worker processou metade do range e caiu. Recomeça tudo?

Posso repetir o range se resultados e créditos forem idempotentes. Checkpoints reduzem trabalho, mas não são a única proteção. Registro progresso em fronteiras consistentes e verifico completude contra o manifesto da população.

Termo para lembrar: Falha parcial

13. Dois workers assumiram o mesmo range. Há duplicidade?

O claim reduz concorrência, mas a correção depende da identidade da apuração e do crédito. Um worker antigo pode voltar após a lease expirar. Condiciono atualizações de progresso à tentativa vigente.

Termo para lembrar: Lease

14. Quando você escolheria streaming?

Quando frescor do provisionamento ou custo de recalcular posições justificar estado contínuo. Ainda mantenho fechamento e conferência periódicos. Streaming exige eventos completos, ordenação temporal e recuperação; não elimina a necessidade de um corte contábil.

Termo para lembrar: Batch versus stream

15. Como reduz trabalho sem perder histórico?

Atualizo posições acumuladas a partir de movimentos identificados, guardando versão e checkpoints. No fechamento, calculo a partir da posição reconciliada. Comparo amostras ou partições completas com uma reconstrução independente para detectar deriva.

Termo para lembrar: Estado incremental

16. Algumas contas têm muito mais movimentos. Como distribui?

Dimensiono por custo estimado, não apenas quantidade de contas. Posso dividir preparação de movimentos em subtarefas e preservar uma conclusão por conta. Uma partição gigante não deve determinar sozinha o prazo do lote inteiro.

Termo para lembrar: Hot partition

17. Qual gargalo pode aparecer antes da CPU?

Ler histórico amplo e escrever milhões de resultados pode saturar banco, rede ou armazenamento. Prefiro dados de entrada compactos e varredura sequencial quando possível. Meço bytes e amplificação, não somente tempo da fórmula de juros.

Termo para lembrar: I/O

18. O ledger aceita menos créditos que os workers calculam. E agora?

Limito a taxa de aplicação e acumulo obrigações duráveis com prazo monitorado. Não aumento workers de cálculo como resposta automática. Separo calcular de contabilizar para enxergar capacidade e atraso de cada estágio.

Termo para lembrar: Backpressure

19. Como faz backfill de um mês?

Crio uma execução com escopo e versão explícitos, congelo entradas e comparo com o histórico. Aplico apenas diferenças autorizadas por identidades próprias. Um backfill não pode reapresentar todos os créditos originais como inéditos.

Termo para lembrar: Reprocessamento

20. Como responde por que o cliente recebeu aquele valor?

Recupero posição elegível, período, taxa, versão da fórmula, arredondamento e referência ao lançamento. Não dependo de reexecutar código atual com dados atuais. A explicação precisa reproduzir o cálculo que fundamentou o crédito.

Termo para lembrar: Auditoria

21. Como detecta contas esquecidas?

Confronto quantidade e identidade das contas esperadas com resultados, excluídas justificadas e pendências. Confiro totais por produto e moeda. Uma soma agregada correta pode esconder uma conta omitida e outra duplicada.

Termo para lembrar: Verificação de totais

22. Uma fórmula errada creditou demais. Apaga os registros?

Preservo resultados e evidências, interrompo novas aplicações e calculo exposição. A correção financeira segue autorização e regras do produto, com novos lançamentos rastreáveis. Rollback de código não desfaz créditos já executados.

Termo para lembrar: Correção

23. A UI precisa cair enquanto fecha o dia?

Não se consultas e fechamento tiverem recursos isolados e versões claras. A interface pode mostrar a última posição válida com status de atualização. Não publico um total parcial como fechamento concluído.

Termo para lembrar: Disponibilidade

24. Onde usaria instâncias interrompíveis?

Em tarefas de cálculo reiniciáveis, com entrada durável, checkpoints e idempotência. Reservo capacidade suficiente para cumprir o prazo. Economizar computação perde sentido se interrupções tornarem o fechamento imprevisível.

Termo para lembrar: Custo

25. O que torna timezone um problema financeiro?

O período contratual pode usar uma zona e calendário específicos. Armazeno instantes e datas de negócio sem confundi-los. Mudanças de horário ou calendário não devem duplicar ou omitir um período por acidente do cron.

Termo para lembrar: Calendário

26. O fechamento caiu durante perda regional. Como retoma?

Recupero manifesto, posição de entrada, progresso e resultados confirmados. Antes de aplicar novamente, verifico créditos que podem ter sobrevivido no ledger. A retomada precisa preservar as mesmas identidades e a fronteira temporal.

Termo para lembrar: DR

27. Qual SLO definiria para esse sistema?

Prazo de conclusão por período, completude da população, correção dos valores e disponibilidade da consulta. Monitoro também idade das pendências e capacidade restante até o cutoff. Média de duração dos workers não mede fechamento completo.

Termo para lembrar: SLO

28. Qual teste valida correção além de performance?

Comparo uma implementação de referência pequena com o pipeline para taxas, arredondamentos, eventos tardios e reexecução. Depois injeto falhas entre cálculo e crédito. O total e a identidade dos efeitos devem permanecer consistentes.

Termo para lembrar: Teste

29. Calcular rendimento na hora da consulta resolve tudo?

Pode reduzir pré-cálculo visual, mas não elimina a obrigação de contabilizar e conferir. Também transfere custo para o pico de consultas. Eu separaria uma estimativa sob demanda do processo financeiro autoritativo.

Termo para lembrar: Alternativa de leitura

30. Como defende sua escolha sem parecer preso a batch?

Apresento números e prazo, mostro recuperação por partição e comparo custo total com streaming. Escolho a solução mais simples que cumpre o contrato e deixo critérios mensuráveis para evoluir, como frescor ou volume de reprocessamento.

Termo para lembrar: Fechamento do mock

10 trade-offs: ganho, custo e decisão

01. Batch versus streaming

Ganho. Batch simplifica corte e operação

Custo e motivo. Pode atrasar visão; streaming exige estado contínuo

Como decidir. Escolher pelo prazo e frescor, não por rótulo

02. Snapshot completo versus incremental

Ganho. Snapshot facilita reprodução

Custo e motivo. Lê mais dados; incremental exige verificar eventos

Como decidir. Conforme volume de mudanças e custo de reconstrução

03. Partição grande versus pequena

Ganho. Amortiza overhead

Custo e motivo. Piora stragglers e recuperação

Como decidir. Ajustar pelo custo por conta e janela

04. Calcular e creditar juntos versus separados

Ganho. Menos estados intermediários

Custo e motivo. Acopla throughput ao ledger

Como decidir. Separar quando há taxas e falhas diferentes

05. Spot versus capacidade reservada

Ganho. Reduz custo de cálculo reiniciável

Custo e motivo. Interrupções ameaçam prazo

Como decidir. Manter margem e fallback para cutoff

06. Arredondar diariamente versus acumular precisão

Ganho. Regra diária simplifica explicação

Custo e motivo. Pode alterar resultado acumulado

Como decidir. Seguir contrato, nunca escolher só por performance

07. Correção incremental versus recálculo integral

Ganho. Processa menos dados

Custo e motivo. Exige base anterior confiável

Como decidir. Para ajustes delimitados e auditáveis

08. Provisionamento rápido versus valor final

Ganho. Melhora experiência de consulta

Custo e motivo. Exige rotulagem e limites claros

Como decidir. Quando previsão não equivale a saldo disponível

09. Retenção de insumos versus apenas resultado

Ganho. Permite reprodução detalhada

Custo e motivo. Custa armazenamento e proteção

Como decidir. Guardar fatos mínimos necessários por política

10. Verificação integral versus amostral

Ganho. Cobertura completa de população

Custo e motivo. Consome mais tempo e I/O

Como decidir. Conferir identidades/totais integralmente e usar amostras para análises adicionais

Duas alternativas ao caminho proposto

1. Batch particionado com manifesto

Quando faz sentido. Janela de fechamento permite processamento periódico

Como implementar. Snapshot versionado, workers idempotentes, resultados e crédito separados

Limites. Precisa dimensionar pico, stragglers e recuperação antes do prazo

2. Posição incremental com fechamento periódico

Quando faz sentido. Movimentos contínuos e frescor justificam streaming

Como implementar. Atualizar posição por evento e fechar período sobre versão reconciliada

Limites. Estado contínuo, eventos tardios e reconstrução tornam operação mais complexa

Referências e alcance da evidência

1. PostgreSQL: transações e isolamento

Garantias locais para atualização concorrente; o workflow proposto é autoral.

https://www.postgresql.org/docs/current/transaction-iso.html

2. Flink: timely stream processing

Event time, processamento temporal e watermarks; não estabelece política financeira.

https://nightlies.apache.org/flink/flink-docs-stable/docs/concepts/time/

3. AWS: design de workloads batch

Serviço de execução batch; escolha e dimensionamento são propostas do exercício.

https://docs.aws.amazon.com/batch/latest/userguide/what-is-batch.html

Continue em: T049 · T050 · T061 · T087 · T091 · T093

C005 / M. Os 50 cenários enviados

Sistema de detecção de fraude transacional em tempo real

Antifraude em tempo real combina dados, regras e modelos sob orçamento de latência e risco. A decisão, suas versões e o fallback precisam ser rastreáveis.

Quando usar. Quando decisões de transação exigem sinais recentes e equilíbrio entre perda, fricção e disponibilidade.

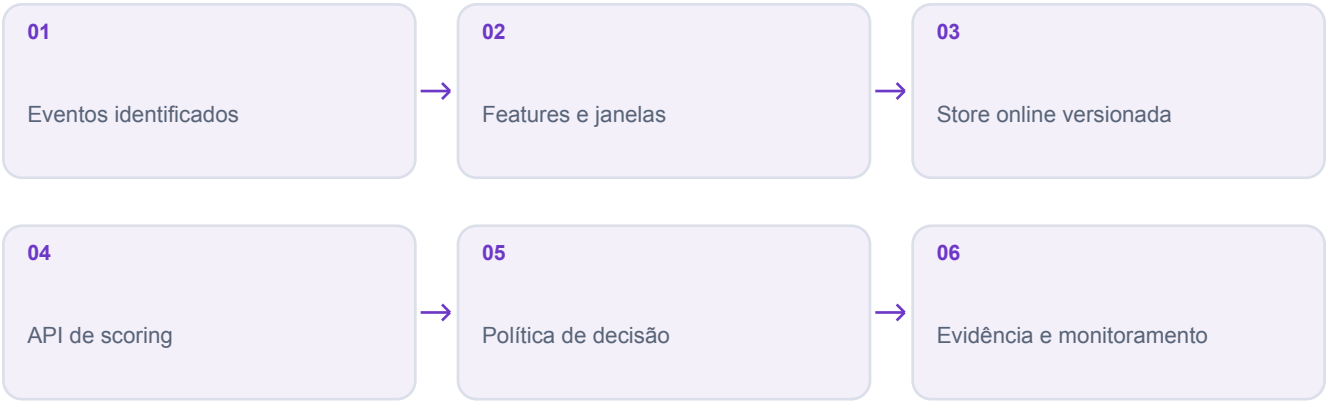
Quando evitar ou limitar. Evitar aprovar por timeout com um valor arbitrário, confundir checkpoint com decisão única ou presumir que feature store elimina diferenças entre treino e produção.

Uma abertura possível

Eu desenharia dois caminhos: atualização de sinais e decisão da transação. O segundo tem deadline e política de risco explícita; o primeiro controla qualidade, frescor e reprodução temporal dos dados.

Peças e tecnologias. Kafka · Flink · Feature Store/Feast · Redis ou DynamoDB para features · Serving de modelo · Regras versionadas

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. 0-8 min: definir ação de fraude, prazo, perdas toleradas, revisão humana e disponibilidade de labels.

2. 8-18 min: separar dados históricos, features online, modelo e decisão; definir identidade e versões.

3. 18-32 min: desenhar pipeline de sinais e serving com limites de latência, segurança e fallback aprovado.

4. 32-48 min: tratar atraso, skew, modelo indisponível, replay, shadow e vieses de observação.

5. 48-60 min: avaliar falsos positivos, perda, drift, capacidade, custo e rollback.

Cuidado técnico. Não há threshold universal de 100 ms ou baixo valor que autorize aprovação por timeout.

Cuidado técnico. Feature store reduz divergência de definições, mas não garante por si só equivalência entre treino e produção.

30 perguntas para sustentar o desenho

01. Qual métrica de negócio orienta o modelo?

Defino perda financeira, falsos positivos, fricção e tempo de decisão por segmento. Acurácia isolada engana em classes desbalanceadas. A arquitetura preserva dados para avaliar essas métricas depois que os resultados reais ficarem disponíveis.

Termo para lembrar: Objetivo

02. O stream processor decide cada compra diretamente?

Pode atualizar sinais continuamente, enquanto uma API de decisão consulta features e aplica política sob deadline. Se a decisão estiver no stream, preciso demonstrar latência e correlação da resposta. Não misturo os dois caminhos sem contrato.

Termo para lembrar: Caminho crítico

03. Quais sinais você calcularia?

Exemplos incluem velocidade de transações, mudanças de dispositivo e padrões de destinatário, sujeitos a finalidade e qualidade. Registro timestamp e versão. Não trato ausência de um sinal como valor zero sem definir seu significado no modelo.

Termo para lembrar: Features

04. Como evita usar o futuro no treinamento?

Construo datasets com os valores que estariam disponíveis no instante da decisão e verifico tempo de evento e de disponibilidade. Correções tardias podem vazar informação. Testo a consulta temporal e não confio apenas no nome da feature store.

Termo para lembrar: Point-in-time

05. Feature store garante igualdade entre treino e produção?

Ajuda a compartilhar definições e recuperação, mas não elimina diferenças de frescor, código, fontes e versões. Comparo distribuições e amostras reproduzíveis. O modelo deve saber lidar com features ausentes ou atrasadas conforme contrato.

Termo para lembrar: Training-serving skew

06. Por que usar tempo do evento?

Uma transação atrasada pertence ao instante de ocorrência, não necessariamente ao momento de consumo. Defino janelas e política de atraso para sinais de velocidade. Essa política afeta o score e precisa ser testada com reordenação.

Termo para lembrar: Event time

07. Watermark prova que nenhum evento antigo chegará?

Não. Representa uma política de progresso temporal. Eventos mais atrasados ainda podem aparecer e exigir tratamento. Decido entre corrigir sinais, emitir ajuste ou registrar atraso, sem fingir completude absoluta do mundo externo.

Termo para lembrar: Watermark

08. Uma conta comercial concentra milhões de eventos. Como escala?

Avalio agregações parciais quando a função permite combinar resultados e mantenho limites para estado e tráfego. Nem toda feature é decomponível. Monitoro skew por chave para não esconder um gargalo atrás da média do cluster.

Termo para lembrar: Hot key

09. Evento repetido pode aumentar o score indevidamente?

Sim, se incrementar contadores novamente. Preservo identidade e deduplico na fronteira adequada ou uso operações que reconheçam a sequência. A janela de retenção precisa cobrir replay; o checkpoint não corrige identidade ruim na origem.

Termo para lembrar: Deduplicação

10. Como distribui o tempo da decisão?

Separo consulta de features, inferência e política final, reservando margem para a resposta. Defino timeout por dependência e um limite total. Uma chamada que já perdeu o prazo não deve continuar consumindo recursos sem utilidade.

Termo para lembrar: Deadline

11. O modelo não respondeu. Qual decisão toma?

Uma política aprovada por risco define recusa, desafio adicional, revisão ou aceitação limitada conforme contexto. Registro que o modelo não participou. Não codifico aprovação automática de baixo valor sem discutir exposição acumulada e abuso.

Termo para lembrar: Fallback

12. O que salva junto da decisão?

Identidade da transação, modelo, regras, versões e frescor das features essenciais, resultado e motivo. Minimizando dados, mantenho evidência suficiente para auditoria e replay explicativo. Reexecutar o modelo atual não reproduz uma decisão antiga.

Termo para lembrar: Versões

13. Quanto tempo uma feature pode ficar velha?

Cada feature tem requisito próprio: endereço histórico e velocidade de compras não envelhecem igual. O serving informa idade e ausência. A política decide quais dados vencidos ainda são aceitáveis; TTL de cache não é decisão de risco universal.

Termo para lembrar: Feature TTL

14. Redis ou DynamoDB para features?

Comparo latência, acesso por chave, persistência, custo e operação. O store serve sinais, não a autoridade do dinheiro. Cache perdido pode mudar a confiança do score; o fallback deve reconhecer essa condição explicitamente.

Termo para lembrar: Online store

15. Usaria GPU para toda inferência?

Só após medir custo, throughput e latência do modelo. Batching pode melhorar eficiência e piorar cauda. Modelos menores ou pré-computação podem atender melhor o caminho crítico; não escolho hardware pelo prestígio técnico.

Termo para lembrar: Modelo pesado

16. Qual problema de agrupar requisições de inferência?

Amortiza execução, mas introduz espera e acopla respostas. Limito tamanho e tempo do lote pelo deadline. Também separo prioridades para não deixar uma jornada urgente atrás de processamento analítico tolerante a atraso.

Termo para lembrar: Microbatch

17. Como testa modelo novo sem afetar clientes?

Copio insumos para inferência em sombra, sem produzir bloqueios ou efeitos financeiros. Comparo decisões e custo por segmento. Shadow mede comportamento observado, mas não revela completamente consequências de decisões que nunca foram aplicadas.

Termo para lembrar: Shadow

18. Como faz rollout de um modelo?

Escolho coortes estáveis, limites de exposição e métricas com atraso conhecido. Mantenho versão anterior e critérios de interrupção. Um aumento de aprovação imediato não comprova melhora antes de observar fraude e contestação posteriores.

Termo para lembrar: Canary

19. Como avalia fraude quando o resultado leva semanas?

Separo métricas rápidas de saúde e proxies das métricas finais de perda. Versiono datasets e datas de maturação. Não uso ausência temporária de contestação como prova definitiva de transação legítima.

Termo para lembrar: Labels atrasados

20. Drift significa necessariamente modelo pior?

Não. Mudança de distribuição é sinal para investigar; efeito depende do modelo e da relação com o alvo. Monitoro qualidade por segmento e dados faltantes. Retreinar automaticamente sem validação pode amplificar um problema de dados.

Termo para lembrar: Drift

21. Bloquear transações muda os dados de treino?

Sim. Decisões afetam o que observamos e quais labels aparecem. Registro política e seleção, e avalio viés com métodos apropriados. Não interpreto o conjunto aprovado como amostra neutra de todas as tentativas.

Termo para lembrar: Feedback loop

22. Como responde a uma contestação sobre o bloqueio?

Recupero motivos e evidências permitidas da decisão, com revisão humana quando aplicável. Diferencio explicação operacional de divulgação de regras exploráveis. O sistema precisa preservar contexto sem expor dados de outros clientes.

Termo para lembrar: Explainability

23. O atacante pode aprender seus limites?

Pode explorar feedback e repetir padrões. Limito exposição de detalhes sensíveis, monitoro abuso e aplico controles independentes. Não revelo thresholds internos na resposta pública; ainda forneço uma experiência clara e caminhos legítimos de resolução.

Termo para lembrar: Abuso

24. O pipeline de features está atrasado. Continua decidindo?

O serving sinaliza frescor e a política considera degradação. Protejo o caminho de decisão de filas ilimitadas e priorizo sinais críticos. Não escondo atraso retornando um score aparentemente normal baseado em estado antigo.

Termo para lembrar: Backpressure

25. Como recupera o stream processor?

Restaura estado e posição compatíveis, respeitando garantias da fonte e do sink. Depois verifica frescor e reconstrução. Durante recuperação, o caminho de decisão usa política explícita; reiniciar o job não significa features imediatamente corretas.

Termo para lembrar: Resiliência

26. Pode reprocessar eventos antigos contra o endpoint de decisão?

Para análise, uso um ambiente sem efeitos e dados temporais reproduzíveis. Replay não deve bloquear contas ou lançar novas ações reais. Se quiser corrigir estado operacional, crio uma campanha controlada com identidades e autorização próprias.

Termo para lembrar: Replay

27. Guardaria todas as features para sempre?

Não. Defino finalidade, retenção e acesso por categoria. Posso guardar evidência mínima e versões para reprodução controlada. Um feature store central não autoriza acesso indiscriminado nem reutilização de dados em qualquer modelo.

Termo para lembrar: Privacidade

28. Qual SLO além da latência?

Disponibilidade de decisão, frescor de features, ausência de decisões duplicadas e prazo de recuperação. Métricas de risco complementam o SLO técnico. Uma API rápida que silenciosamente decide com dados vencidos pode estar funcionalmente degradada.

Termo para lembrar: SLO

29. Como testa uma falha perigosa?

Atraso uma feature crítica sem derrubar o endpoint, mudo schema e duplico eventos. Verifico se o sistema detecta degradação e aplica a política correta. Testar só indisponibilidade total não cobre falhas silenciosas de qualidade.

Termo para lembrar: Teste

30. Como defende a arquitetura final?

Mostro separação entre sinais, inferência e política, com identidade e versões. Explico como decidimos com dados ausentes, como medimos perda e como revertemos mudanças. A sofisticação está na decisão controlada, não apenas em adicionar ML.

Termo para lembrar: Conclusão

10 trade-offs: ganho, custo e decisão

01. Modelo complexo versus regras simples

Ganho. Pode capturar padrões mais ricos

Custo e motivo. Custa latência, explicação e operação

Como decidir. Quando ganho medido compensa complexidade

02. Features frescas versus disponibilidade

Ganho. Melhora informação da decisão

Custo e motivo. Aumenta dependência do pipeline

Como decidir. Definir política por feature e risco

03. Inferência síncrona versus pré-cálculo

Ganho. Usa contexto imediato

Custo e motivo. Consome deadline

Como decidir. Pré-calcular sinais estáveis e manter decisão crítica controlada

04. Batching versus cauda

Ganho. Melhora eficiência de hardware

Custo e motivo. Adiciona espera

Como decidir. Limitar por orçamento e prioridade

05. Shadow versus decisão real

Ganho. Reduz risco inicial de rollout

Custo e motivo. Não revela todos efeitos contrafactuais

Como decidir. Usar antes de canary com exposição limitada

06. Fallback permissivo versus restritivo

Ganho. Reduz recusa durante falha

Custo e motivo. Aumenta exposição a fraude

Como decidir. Somente com política de risco explícita

07. Retenção de evidência versus minimização

Ganho. Facilita auditoria e reprodução

Custo e motivo. Aumenta dados sensíveis retidos

Como decidir. Guardar fatos necessários com prazo e acesso

08. Atualização contínua versus versão congelada

Ganho. Adapta sinais rapidamente

Custo e motivo. Dificulta reprodução se versões não forem preservadas

Como decidir. Versionar modelos, regras e dados de referência

09. Agregação por chave versus subagregação

Ganho. Simplifica semântica temporal

Custo e motivo. Hot keys limitam throughput

Como decidir. Subdividir somente funções combináveis

10. Automação de retreino versus aprovação

Ganho. Reduz tempo de adaptação

Custo e motivo. Pode promover modelo contaminado

Como decidir. Exigir validação, métricas e capacidade de rollback

Duas alternativas ao caminho proposto

1. Motor de regras versionado

Quando faz sentido. Sinais e política são suficientes para a fase inicial

Como implementar. Regras auditáveis, features limitadas e revisão humana

Limites. Menor capacidade de capturar padrões complexos; medir perda e manutenção

2. Streaming de features com serving separado

Quando faz sentido. Volume e frescor justificam pipeline contínuo

Como implementar. Flink/Kafka alimentam store online; API aplica modelo e política sob deadline

Limites. Mais estado e operação; feature store não elimina skew automaticamente

Referências e alcance da evidência

1. Feast: point-in-time joins

Recuperação histórica temporal e cuidado com disponibilidade tardia dos dados.

https://docs.feast.dev/getting-started/concepts/point-in-time-joins

2. Feast: feature retrieval

Separação de recuperação online, offline e batch.

https://docs.feast.dev/getting-started/concepts/feature-retrieval

3. Flink: timely stream processing

Event time, processamento temporal e watermarks; não estabelece política financeira.

https://nightlies.apache.org/flink/flink-docs-stable/docs/concepts/time/

Continue em: T064 · T087 · T089 · T097 · T110

C006 / M. Os 50 cenários enviados

Como lidar com picos de tráfego extremos (ex: quinto dia útil ou Black Friday)?

Absorver pico exige proteger capacidade útil e compromissos do produto, combinando previsão, admissão, filas limitadas, isolamento e recuperação gradual.

Quando usar. Quando há campanhas, datas previsíveis, rajadas ou falhas que redirecionam tráfego.

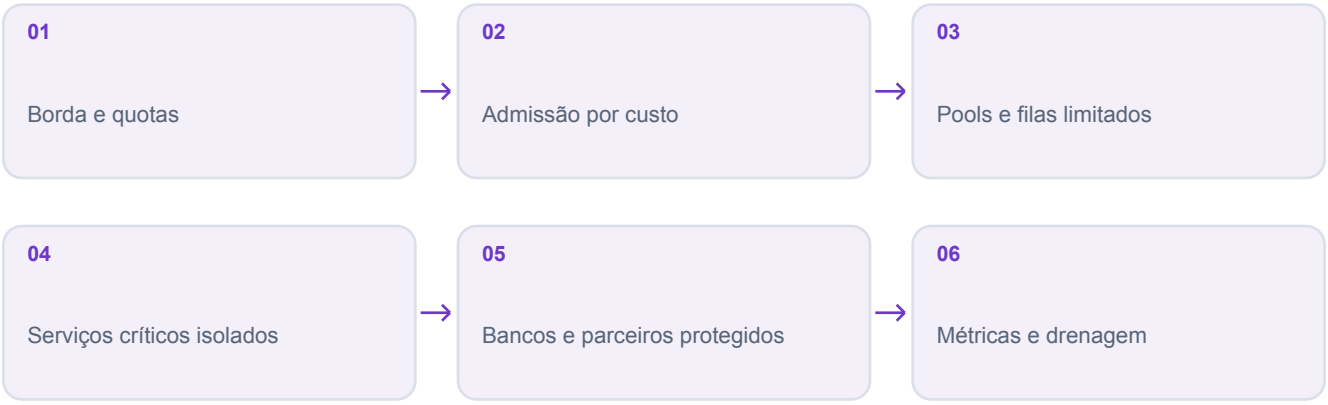
Quando evitar ou limitar. Evitar tratar autoscaling de CPU e aumento do pool como resposta universal, ou assumir que fila infinita torna qualquer carga aceitável.

Uma abertura possível

Eu começaria pelo recurso que satura e pelo trabalho que precisamos preservar. Quero manter o caminho financeiro correto e previsível, degradando leituras e tarefas adiáveis antes de transformar o pico numa falha em cascata.

Peças e tecnologias. Envoy/Kong · Redis rate limiter · Kubernetes/ECS · SQS/Kafka · PostgreSQL pool · OpenTelemetry

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. 0-8 min: caracterizar pico, duração, mix de operações, prioridades e compromissos aceitáveis.

2. 8-18 min: estimar capacidade por recurso e desenhar filas, pools, dependências e amplificação de chamadas.

3. 18-32 min: definir admissão, quotas, isolamento, cache e capacidade pré-aquecida.

4. 32-48 min: simular dependência lenta, tempestade de retry, cache frio, perda de zona e recuperação.

5. 48-60 min: demonstrar testes de carga, SLOs por jornada, critérios de rejeição e plano de expansão.

Cuidado técnico. Aumentar pool pode piorar a saturação.

Cuidado técnico. Operações financeiras aceitas não podem ser descartadas como simples load shedding.

30 perguntas para sustentar o desenho

01. RPS sozinho descreve o pico?

Não. Uma consulta simples e uma exportação grande custam diferente. Estimo mix, bytes, CPU, I/O, concorrência e fan-out. O limite de admissão deve refletir custo útil, não apenas quantidade de requisições.

Termo para lembrar: Perfil de carga

02. Por que latência crescente aumenta pressão?

Com taxa de chegada estável, mais tempo em serviço aumenta trabalho em andamento. Isso ocupa conexões e memória. Uso essa relação para dimensionar concorrência e identificar quando esperar mais está alimentando a saturação.

Termo para lembrar: Little's Law

03. Onde você rejeita tráfego excedente?

O mais cedo possível com informação suficiente, preservando capacidade para trabalho prioritário. Rejeitar depois de autenticar, consultar banco e executar fan-out já consumiu recursos caros. A resposta deve orientar o cliente sem gerar retries sincronizados.

Termo para lembrar: Admissão

04. Como decide o que degradar primeiro?

Mapeio impacto e prazo: uma autorização em andamento difere de exportar cinco anos de histórico. Defino classes e capacidade reservada. Prioridade não significa deixar tarefas de baixa classe sem progresso para sempre.

Termo para lembrar: Prioridades

05. CPU alta é sempre o melhor sinal?

Não. Uma aplicação bloqueada em banco pode ter CPU baixa e filas enormes. Uso métricas de saturação e capacidade do estágio limitante. Escalar aplicações sem capacidade downstream pode aumentar o número de concorrentes e piorar a falha.

Termo para lembrar: Autoscaling

06. Por que não aumentar o pool do banco?

Porque o banco tem capacidade finita de executar e coordenar trabalho. Mais sessões podem aumentar memória, context switches e contenção. Limito concorrência e meço throughput e p99; o pool também funciona como proteção.

Termo para lembrar: Pool de conexões

07. Por que uma fila durável precisa de limite?

Ela preserva dados, mas pendências ainda têm prazo e custo. Defino capacidade por idade, quantidade e bytes. Se não conseguirei concluir no prazo, preciso reduzir admissão ou negociar uma experiência assíncrona explícita.

Termo para lembrar: Fila limitada

08. Como comunica lentidão ao produtor?

Uso limites de consumo, erros ou sinais apropriados ao protocolo e orçamento de retries. O produtor reduz envio em vez de criar trabalho ilimitado. Contratos assíncronos incluem capacidade e atraso, não apenas endereço do tópico.

Termo para lembrar: Backpressure

09. Como impede que falha triplique o tráfego?

Escolho uma camada de retry, aplico backoff com jitter e orçamento global de tentativas adicionais. Inspeciono SDKs e proxies. Durante sobrecarga, uma repetição sem chance de sucesso disputa capacidade com trabalho útil.

Termo para lembrar: Retry storm

10. Abrir breaker resolve saturação local?

Pode evitar chamadas a uma dependência falhando, mas não substitui limite de recursos locais. Também preciso controlar filas e concorrência. Abertura e recuperação devem ser observáveis; sondagens em massa podem recriar o pico.

Termo para lembrar: Circuit breaker

11. O que isolaria no desenho?

Pools, filas e limites de clientes ou jornadas com perfis diferentes. Um parceiro lento não deve ocupar todos os workers de pagamento. Isolamento consome capacidade reservada, então escolho fronteiras pelo impacto potencial.

Termo para lembrar: Bulkhead

12. O deploy limpou cache durante o pico. E agora?

Evito invalidação global desnecessária, preaqueço dados previsíveis e limito recomputação concorrente. Uso single-flight e validade escalonada. O banco precisa suportar uma política de miss controlada, não todos os clientes ao mesmo tempo.

Termo para lembrar: Cache frio

13. TTL com jitter resolve sozinho?

Ajuda a dispersar expirações, mas uma chave popular ainda pode causar várias recomputações. Combino coalescência, refresh controlado e limites. Se servir dado antigo, explicito onde isso é aceitável e por quanto tempo.

Termo para lembrar: Stampede

14. Um cliente domina a carga. Como protege os demais?

Aplico quotas por identidade relevante e monitoro custo, evitando depender só do IP. Posso isolar clientes muito grandes. A chave de distribuição não deve concentrar todo o tráfego de uma campanha num único shard.

Termo para lembrar: Hot key

15. Que resposta parcial é aceitável numa home bancária?

Exibo os componentes disponíveis e marco dados indisponíveis ou antigos. Não substituo saldo desconhecido por zero. O BFF aplica deadlines independentes e não deixa um domínio opcional derrubar toda a composição.

Termo para lembrar: Degradação

16. Como o BFF amplifica o pico?

Cada acesso pode gerar várias chamadas e retries. Calculo essa multiplicação e uso batching ou pré-composição quando apropriado. Limito paralelismo e cancelo trabalho dispensável; uma interface rápida pode esconder carga enorme nos serviços internos.

Termo para lembrar: Fan-out

17. Quando reserva capacidade antes do pico?

Quando a data é previsível e o tempo de inicialização ou a quota limita reação. Preparo aplicações, caches e dependências. Autoscaling continua útil, mas não substitui planejamento para uma rajada que chega antes dos novos nós.

Termo para lembrar: Prewarming

18. Autoscaling pediu instâncias e não conseguiu. Como evita surpresa?

Verifico quotas, capacidade regional e dependências antes do evento. Planejo margem e alternativas operacionais. Um manifesto declarando dez réplicas não garante que a infraestrutura conseguirá entregá-las no momento necessário.

Termo para lembrar: Quota cloud

19. O sistema suporta pico e perda de zona juntos?

Essa combinação precisa entrar no requisito e no teste de capacidade. Distribuir instâncias entre zonas não garante capacidade remanescente suficiente. Defino o nível de serviço reduzido e a admissão durante esse cenário.

Termo para lembrar: Perda de zona

20. Descartar requisição é sempre falha de arquitetura?

Não quando a alternativa é colapso que prejudica todos. Rejeito de forma controlada antes de assumir obrigação, preservando operações já aceitas. Não descarto silenciosamente uma tarefa financeira durável só para reduzir o gráfico de backlog.

Termo para lembrar: Load shedding

21. Como evita starvation de tarefas menos prioritárias?

Reservo uma parcela de capacidade ou uso escalonamento ponderado e envelhecimento de prioridade. Limites por classe impedem monopolização. A política deve respeitar prazos reais, não apenas uma fila fixa de alta e baixa prioridade.

Termo para lembrar: Fairness

22. O aplicativo também participa do controle?

Sim. Evito polling excessivo, aplico backoff, reutilizo resultados e uso feedback claro de pendência. O cliente não deve repetir uma criação com identidade nova a cada erro. A experiência influencia diretamente a carga de recuperação.

Termo para lembrar: Cliente

23. Qual teste faria antes da Black Friday?

Reproduzo mix e distribuição de chaves, duração, rajadas, cache frio e dependência lenta. Procuro o ponto em que p99 e erros degradam, não só o maior RPS momentâneo. Testo também recuperação após reduzir a carga.

Termo para lembrar: Teste de carga

24. Por que testar durante horas?

Vazamentos, crescimento de filas, fragmentação e tarefas de manutenção aparecem com tempo. Um benchmark de minutos pode esconder saturação progressiva. Meço estabilidade de memória, conexões, backlog e latência ao longo de ciclos reais.

Termo para lembrar: Soak test

25. Qual painel guia a resposta ao pico?

Mostro demanda, trabalho admitido, rejeitado e concluído por jornada, mais saturação e idade das filas. Separo erro técnico de recusa deliberada. CPU média isolada não mostra uma partição quente ou um parceiro bloqueando tudo.

Termo para lembrar: Observabilidade

26. Depois que o pico acabou, libera o backlog inteiro?

Não. Dreno com limites para não provocar uma segunda sobrecarga. Balanceio pendências antigas e tráfego novo, removendo trabalho expirado conforme a regra. A recuperação precisa de capacidade excedente e acompanhamento até convergir.

Termo para lembrar: Recuperação

27. Superdimensionar tudo é a solução mais segura?

Pode desperdiçar recursos e ainda não resolver limites de parceiros ou contenção. Avalio margem por risco, capacidade elástica e isolamento. O custo relevante inclui indisponibilidade e operação, mas não autoriza infraestrutura ilimitada sem medição.

Termo para lembrar: Custo

28. Como mede a degradação sem maquiar disponibilidade?

Defino antecipadamente o que é sucesso por jornada e quais respostas parciais contam. Uma página renderizada sem informação essencial não vira sucesso por conveniência. Também acompanho obrigações aceitas que perderam prazo.

Termo para lembrar: SLO

29. Quando precisa de sharding em vez de mais réplicas?

Quando o limite está na autoridade de escrita ou conjunto de dados que não escala pelas opções simples. Réplicas de leitura não aumentam automaticamente capacidade de commit. Antes de shardear, meço consultas, índices, contenção e carga evitável.

Termo para lembrar: Mudança arquitetural

30. Como resumiria sua proposta à banca?

Eu preservo trabalho crítico com admissão, isolamento e capacidade medida. Cache e autoscaling ajudam, mas cada dependência tem limites. Explico como o usuário percebe a degradação e como drenamos pendências sem perder correção financeira.

Termo para lembrar: Conclusão

10 trade-offs: ganho, custo e decisão

01. Capacidade reservada versus elasticidade

Ganho. Reduz surpresa na rajada

Custo e motivo. Aumenta custo ocioso

Como decidir. Para picos previsíveis ou componentes lentos de iniciar

02. Rejeição precoce versus fila

Ganho. Protege recursos e prazo

Custo e motivo. Pode exigir nova tentativa do cliente

Como decidir. Antes de aceitar trabalho que não pode cumprir

03. Prioridade estrita versus fairness

Ganho. Protege jornada crítica

Custo e motivo. Pode causar starvation

Como decidir. Reservar progresso mínimo e usar prazo por classe

04. Pool pequeno versus grande

Ganho. Limita pressão downstream

Custo e motivo. Pode restringir throughput se subdimensionado

Como decidir. Escolher por medição, não por máximo de conexões

05. Cache velho versus erro explícito

Ganho. Mantém leitura útil

Custo e motivo. Pode induzir decisão com dado vencido

Como decidir. Somente em leituras cujo contrato permite

06. Isolamento dedicado versus compartilhamento

Ganho. Reduz blast radius

Custo e motivo. Fragmenta capacidade e aumenta custo

Como decidir. Para clientes e dependências com risco desigual

07. Retry agressivo versus orçamento

Ganho. Recupera falhas muito curtas

Custo e motivo. Amplifica carga durante incidente

Como decidir. Usar tentativas limitadas com evidência de benefício

08. Autoscaling reativo versus previsão

Ganho. Adapta custo à demanda

Custo e motivo. Reage depois do início da saturação

Como decidir. Combinar com prewarming para eventos conhecidos

09. Degradação parcial versus resposta completa

Ganho. Preserva parte da jornada

Custo e motivo. Exige contratos e UX mais complexos

Como decidir. Quando componentes opcionais podem faltar honestamente

10. Drenagem rápida versus controlada

Ganho. Reduz idade do backlog

Custo e motivo. Pode recriar saturação

Como decidir. Limitar taxa e manter capacidade para tráfego atual

Duas alternativas ao caminho proposto

1. Capacidade pré-planejada e limites simples

Quando faz sentido. Pico previsível e domínio relativamente estável

Como implementar. Reservar margem, limitar concorrência e priorizar jornadas com poucos mecanismos

Limites. Custo ocioso e menor adaptação a mudanças inesperadas

2. Admissão adaptativa e isolamento por células

Quando faz sentido. Escala e diversidade justificam controle mais fino

Como implementar. Medir saturação por célula, limitar trabalho e redistribuir somente dentro de capacidade segura

Limites. Controlador também precisa de estabilidade, observabilidade e testes

Referências e alcance da evidência

1. Google SRE: Handling Overload

Controle de sobrecarga, capacidade e degradação; os limites do exercício são hipóteses.

https://sre.google/sre-book/handling-overload/

2. Kubernetes: Horizontal Pod Autoscaling

Escalonamento por métricas; não garante capacidade das dependências.

https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/

3. AWS: retries e jitter

Limites e riscos das retentativas em sistemas distribuídos.

https://aws.amazon.com/builders-library/timeouts-retries-and-backoff-with-jitter/

Continue em: T003 · T025 · T034 · T045 · T055 · T064 · T084 · T085

C007 / M. Os 50 cenários enviados

Desenhe a infraestrutura de Open Finance (iniciação de pagamentos)

Iniciação de pagamentos em Open Finance combina participantes autenticados, consentimento válido, autorização do cliente e execução rastreável. A versão e o perfil de segurança aplicáveis devem ser verificados no contrato oficial.

Quando usar. Quando uma instituição iniciadora solicita pagamento em uma detentora por uma jornada autorizada.

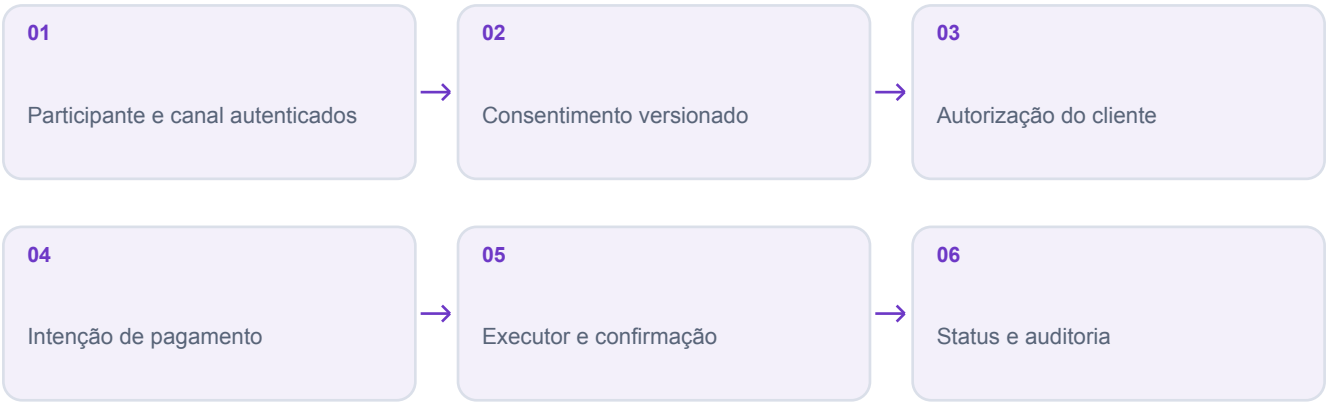
Quando evitar ou limitar. Evitar equiparar JWT válido a consentimento vigente, confundir autenticação do participante com autorização da conta ou apresentar assinatura como garantia jurídica isolada.

Uma abertura possível

Eu separo identidade do participante, consentimento e operação financeira. Cada fronteira precisa demonstrar quem pode fazer o quê, por quanto tempo e como uma revogação ou resposta perdida altera o fluxo.

Peças e tecnologias. OAuth/OIDC conforme perfil · mTLS ou sender-constrained tokens aplicáveis · Gateway · PostgreSQL · Outbox · KMS/HSM

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. 0-8 min: confirmar modalidade, versão de API, participantes, jornada e requisitos normativos de segurança.

2. 8-18 min: modelar consentimento, escopo, validade, autorização e identidade do pagamento.

3. 18-32 min: desenhar validações na borda e no domínio, uso atômico do consentimento conforme contrato e execução financeira.

4. 32-48 min: tratar revogação concorrente, repetição, certificado expirado, callback perdido e parceiro indisponível.

5. 48-60 min: discutir quotas por participante, auditoria, testes de conformidade, métricas e evolução de versão.

Cuidado técnico. Autenticação, consentimento e autorização financeira são fronteiras distintas.

Cuidado técnico. Assinatura não garante não repúdio jurídico isoladamente; verifique o perfil e a evidência exigidos.

30 perguntas para sustentar o desenho

01. Quem são os atores desse sistema?

Eu separo cliente, instituição iniciadora, detentora da conta e serviços de confiança exigidos pelo ecossistema. Identifico quem autentica cada ator e quem autoriza o pagamento. Essa distinção evita confiar no parceiro como se fosse o titular.

Termo para lembrar: Participantes

02. Token válido basta para pagar?

Não. Valido identidade, escopo, vínculo ao recurso e estado do consentimento aplicável. A assinatura do token não prova que o cliente ainda permite aquela operação. A decisão de negócio precisa consultar ou validar a autoridade correta.

Termo para lembrar: Consentimento

03. Qual versão da API você desenha?

Declaro a versão e modalidade escolhidas como premissa e consulto os documentos vigentes. Evito misturar requisitos de jornadas diferentes. No mock, explico os mecanismos sem inventar campos obrigatórios ou prazos regulatórios.

Termo para lembrar: Versionamento normativo

04. O que persiste no consentimento?

Identidade, participante, titular ou contexto autorizado, escopo, validade, estado e versão, além da evidência necessária. Dados concretos seguem a API aplicável. Não uso um booleano autorizado sem histórico de como e para quê foi concedido.

Termo para lembrar: Modelo de dados

05. Como liga a aprovação do cliente ao pagamento correto?

Vinculo a evidência de autorização ao conteúdo e escopo consentidos. Se valor ou destinatário mudar, verifico se o contrato permite ou exige nova autorização. Não aprovo um objeto e executo outro apenas porque compartilham um token.

Termo para lembrar: Autorização

06. Revogou enquanto o pagamento era iniciado. Quem vence?

Defino o ponto de aceitação e a ordem das transições na autoridade local, conforme as regras da jornada. Revogação não desfaz automaticamente um efeito já liquidado. Registro versões e evidências para explicar a decisão concorrente.

Termo para lembrar: Revogação

07. Pode cachear consentimento?

Somente com uma garantia de frescor compatível com revogação e risco. Para decisões sensíveis, uso a autoridade ou um mecanismo de invalidação e versão adequado. TTL arbitrário não equivale a revogação imediata.

Termo para lembrar: Cache

08. O que mTLS acrescenta?

Pode autenticar o cliente no canal e vincular o uso de credenciais conforme o perfil adotado. Ainda preciso de autorização por operação e validação da aplicação. Canal autenticado não concede acesso irrestrito a todas as contas.

Termo para lembrar: mTLS

09. Por que restringir token ao remetente?

Reduz a utilidade de um token roubado quando o uso exige prova associada à chave do cliente. Implemento o mecanismo previsto no perfil. Isso não protege sozinho uma aplicação autorizada comprometida ou uma decisão de negócio incorreta.

Termo para lembrar: Sender-constrained token

10. JWS garante não repúdio em qualquer situação?

Não isoladamente. A assinatura demonstra propriedades criptográficas sob hipóteses de chaves e verificação. Evidência, custódia, contexto, retenção e regras aplicáveis também importam. Eu não transformo uma biblioteca de assinatura numa conclusão jurídica universal.

Termo para lembrar: Assinatura

11. Uma requisição assinada pode ser repetida?

Pode, se o protocolo não limitar reutilização e a operação não reconhecer identidade. Verifico os mecanismos de frescor e replay exigidos, além da idempotência financeira. Assinatura válida não significa requisição inédita.

Termo para lembrar: Replay

12. Como trata duas iniciações da mesma intenção?

Preservo a identidade definida pelo contrato e comparo o conteúdo. A autoridade registra operação e obrigações de forma atômica. A repetição retorna estado conhecido; não cria outro pagamento porque o parceiro perdeu a resposta.

Termo para lembrar: Idempotência

13. O parceiro não sabe se o pagamento foi aceito. Como responde?

Forneço consulta e correlação estáveis, distinguindo recebido, em processamento e resultado final conforme o contrato. Não transformo timeout em rejeição definitiva. A recuperação utiliza evidências do fluxo financeiro e reconciliação.

Termo para lembrar: UNKNOWN

14. Por que quota por participante e não só por IP?

Vários participantes podem compartilhar infraestrutura e um participante pode usar vários IPs. A identidade autenticada permite limites mais coerentes. Também protejo recursos por operação e custo para impedir consultas caras de consumir toda a capacidade.

Termo para lembrar: Rate limit

15. Um participante válido começa a enviar carga destrutiva. E agora?

Aplico limites, isolamento e monitoramento por identidade, com respostas e procedimentos compatíveis com o contrato. Autenticação não elimina abuso. Preservo capacidade para resolver pagamentos já aceitos e para participantes não afetados.

Termo para lembrar: Abuso

16. Como rotaciona sem derrubar tudo?

Planejo sobreposição e atualização de confiança conforme o perfil, valido expiração e testo o caminho de troca. Monitoro falhas por certificado e participante. Não dependo de reiniciar simultaneamente todas as aplicações na hora da validade terminar.

Termo para lembrar: Certificados

17. Onde guarda chaves privadas?

Em mecanismos de custódia adequados ao risco e ao perfil, com acesso mínimo, rotação e auditoria. Não distribuo chaves em imagens ou logs. A arquitetura especifica quem pode assinar e como uma chave comprometida é retirada.

Termo para lembrar: Segredos

18. Toda validação fica no gateway?

Coloco controles comuns de canal e tráfego na borda, mas a autorização da conta e da transição continua no domínio. O serviço não confia cegamente em um header interno que poderia ser forjado por outro componente.

Termo para lembrar: Gateway

19. Por que usar outbox na iniciação?

Depois de aceitar uma operação, a obrigação de executá-la precisa sobreviver à queda da API. Registro estado e obrigação juntos. O executor preserva identidade e acompanha resultado; a fila não substitui a autorização inicial.

Termo para lembrar: Outbox

20. O retorno do pagamento chegou duas vezes?

Autentico a origem, valido o contrato e aplico a transição idempotente. Um evento duplicado não altera novamente o ledger. Também trato eventos fora de ordem, sem deixar a última chegada sobrescrever uma evidência mais forte.

Termo para lembrar: Callback

21. Posso servir status de uma réplica atrasada?

Depende da promessa de frescor. Depois de aceitar a iniciação, devolvo a referência autoritativa e considero leitura por versão mínima. Não uso ausência numa projeção atrasada como prova de que o pagamento nunca existiu.

Termo para lembrar: Consulta de status

22. Guarda toda a jornada para sempre?

Defino dados necessários, finalidade, obrigações de retenção e acesso. Auditoria precisa de identidade e decisões, mas isso não autoriza replicar dados pessoais em todos os logs. Separo telemetria operacional de evidência controlada.

Termo para lembrar: Retenção

23. Como testa interoperabilidade?

Uso testes de contrato, segurança e conformidade correspondentes à versão, além de simuladores de estados e falhas. O caminho feliz com um parceiro não prova aderência. Exercito revogação, repetição, credencial inválida e resposta perdida.

Termo para lembrar: Conformidade

24. Como mantém revogação e pagamento coerentes entre regiões?

Defino a autoridade da decisão e o mecanismo de roteamento ou coordenação. Réplicas eventuais independentes podem discordar sobre consentimento. O failover precisa preservar estados e impedir dois escritores incompatíveis.

Termo para lembrar: Multirregião

25. O parceiro pode consultar qualquer recurso do cliente autenticado?

Não. O acesso é limitado ao escopo e ao recurso autorizados. Valido a relação na operação concreta, não apenas no login. Também evito respostas de erro que revelem dados de contas fora do consentimento.

Termo para lembrar: Privacidade

26. Como muda a versão sem quebrar participantes?

Planejo coexistência, testes e migração pelos prazos e políticas aplicáveis. Monitoro uso e erros por versão. Não retiro a antiga porque meu código novo já está pronto; a integração envolve participantes externos.

Termo para lembrar: Depreciação

27. O que mediria na jornada?

Disponibilidade e latência por etapa, conversão de autorização, erros técnicos, rejeições legítimas e tempo até resultado financeiro. Denominadores precisam ser claros. Uma API de consentimento saudável não prova que a execução do pagamento funciona.

Termo para lembrar: SLO

28. Servidor de autorização fora: aceita pagamentos antigos?

Avalio validade do token, consentimento e requisitos da operação, sem inventar um bypass universal. Algumas verificações locais podem continuar; outras dependem de estado atual. A política de degradação precisa respeitar a garantia exigida.

Termo para lembrar: Falha de autorização

29. Precisa construir um servidor OAuth do zero?

Não. Prefiro implementação madura e compatível, verificando suporte ao perfil e operação. Personalização de domínio fica separada. Usar um produto conhecido não dispensa configuração correta, testes de segurança e gestão de chaves.

Termo para lembrar: Tecnologia

30. Qual é a defesa central da solução?

Identidade, consentimento e efeito financeiro são verificações distintas. Mostro onde cada uma é autoritativa e como revogação, repetição e incerteza são tratadas. A segurança deve permanecer explicável durante falhas, não só no fluxo de autorização bem-sucedido.

Termo para lembrar: Conclusão

10 trade-offs: ganho, custo e decisão

01. Validação central versus distribuída

Ganho. Central facilita estado atual

Custo e motivo. Aumenta dependência e latência

Como decidir. Distribuir apenas garantias compatíveis com frescor exigido

02. Cache de consentimento versus leitura autoritativa

Ganho. Reduz chamadas

Custo e motivo. Pode atrasar revogação

Como decidir. Usar somente com contrato claro de validade e invalidação

03. Token curto versus longo

Ganho. Limita janela de exposição

Custo e motivo. Aumenta renovação e dependência

Como decidir. Dimensionar pelo perfil e experiência sem inventar prazos

04. Gateway forte versus domínio autônomo

Ganho. Reutiliza controles comuns

Custo e motivo. Pode concentrar falhas e lógica indevida

Como decidir. Manter autorização de recurso no serviço responsável

05. Quota por participante versus global

Ganho. Protege fairness

Custo e motivo. Exige identidade e contadores distribuídos

Como decidir. Combinar limite local, por participante e de capacidade total

06. Assinatura de payload versus só transporte

Ganho. Preserva integridade além do salto TLS

Custo e motivo. Exige chaves, canonicalização e verificação

Como decidir. Seguir contrato, sem prometer não repúdio isolado

07. Retorno push versus consulta

Ganho. Reduz polling e latência

Custo e motivo. Callbacks podem repetir ou se perder

Como decidir. Manter consulta/reconciliação como mecanismo complementar

08. Construir versus adotar servidor de autorização

Ganho. Produto maduro reduz superfície própria

Custo e motivo. Exige avaliação de compatibilidade e dependência

Como decidir. Preferir aderência demonstrada ao perfil aplicável

09. Multirregião versus autoridade simples

Ganho. Pode melhorar continuidade regional

Custo e motivo. Coordenação e revogação ficam mais difíceis

Como decidir. Justificar por RTO/RPO e invariantes

10. Auditoria detalhada versus minimização

Ganho. Ajuda explicar consentimento e execução

Custo e motivo. Amplia dados sensíveis retidos

Como decidir. Guardar evidência necessária com acesso e política definidos

Duas alternativas ao caminho proposto

1. Plataforma de autorização compatível com domínio próprio

Quando faz sentido. Equipe quer reduzir implementação de protocolos

Como implementar. Adotar servidor avaliado e manter consentimento/pagamento sob serviços de domínio

Limites. Fornecedor não decide regras financeiras nem garante configuração correta

2. Integração por plataforma especializada de Open Finance

Quando faz sentido. Cobertura e certificação justificam fornecedor

Como implementar. Delegar parte do protocolo, preservar identidades e evidências locais

Limites. Avaliar dependência, saída, custos, responsabilidades e recuperação de estados

Referências e alcance da evidência

1. Open Finance Brasil: pagamentos v5

Visão da API de consentimento e iniciação; verificar modalidade e versão aplicáveis.

https://openfinancebrasil.atlassian.net/wiki/spaces/OF/pages/1767146135

2. OpenID Foundation: FAPI 2.0 Security Profile

Perfil de segurança e tokens vinculados ao remetente; não substitui o perfil local aplicável.

https://openid.net/specs/fapi-security-profile-2_0-final.html

3. IETF: OAuth Security BCP

Práticas de segurança de OAuth e modelos de ameaça.

https://www.rfc-editor.org/rfc/rfc9700.html

Continue em: T060 · T061 · T071 · T072 · T073 · T094

C008 / M. Os 50 cenários enviados

Migração de um banco de dados relacional grande para particionamento (Sharding)

Migrar um banco grande para shards é uma mudança de autoridade e de roteamento, não apenas uma cópia. O plano precisa preservar dados, transações, identidades e um caminho de reversão.

Quando usar. Quando um limite medido de capacidade ou isolamento justifica distribuir a autoridade de dados.

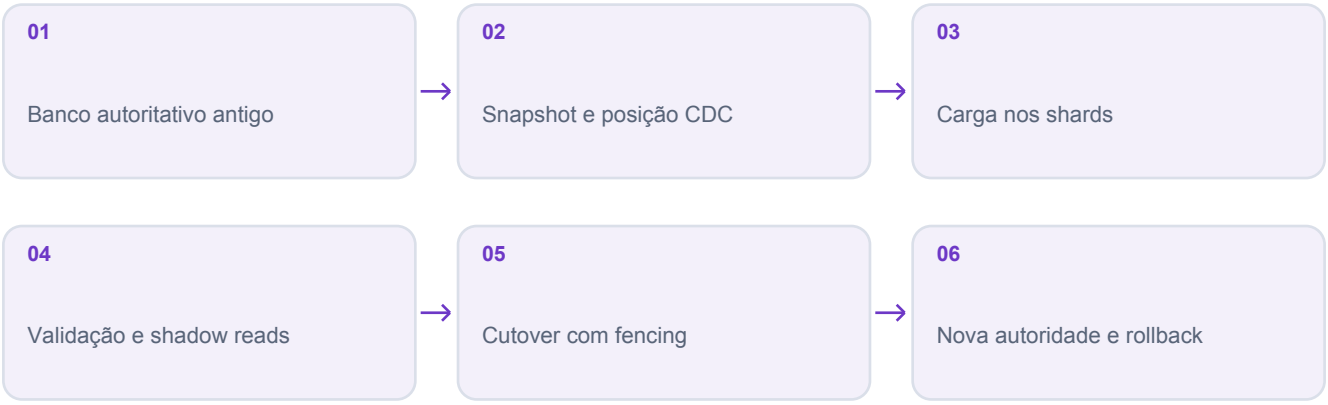
Quando evitar ou limitar. Evitar dual write independente sem reparação, prometer zero downtime sem definir a jornada ou trocar leitura antes de verificar lag e equivalência.

Uma abertura possível

Eu dividiria a migração em preparar, copiar, acompanhar mudanças, comparar e transferir autoridade por unidade controlada. O ponto mais importante é impedir dois escritores e saber quando voltar ainda é seguro.

Peças e tecnologias. PostgreSQL · CDC/Debezium ou AWS DMS · Roteador de shards · S3 para checkpoints · Shadow reads · Feature flags

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. 0-8 min: comprovar motivo da migração, limites, jornadas e tolerância a pausa; mapear transações e consumidores.

2. 8-18 min: escolher chave, diretório de roteamento, unidade de migração e identidade global.

3. 18-32 min: preparar schema, capturar mudanças, copiar snapshot e validar destino sem gerar efeitos reais.

4. 32-48 min: demonstrar cutover por shard/tenant, fencing, requisições em voo, reversão e perda de CDC.

5. 48-60 min: avaliar capacidade de origem e destino, rebalancing, custo, ownership e critérios de conclusão.

Cuidado técnico. Dual write independente cria uma nova janela de falha; não é receita universal de migração sem interrupção.

Cuidado técnico. Zero downtime precisa ser definido pela experiência e demonstrado, não apenas anunciado.

30 perguntas para sustentar o desenho

01. Como prova que precisa de sharding?

Mostro o limite observado: escrita, volume, isolamento ou manutenção. Antes comparo índices, consultas, partições locais e capacidade vertical. Sharding acrescenta roteamento, transações distribuídas e operação; não é resposta automática para banco grande.

Termo para lembrar: Motivação

02. Como escolhe a chave?

Analiso padrões de acesso, distribuição e fronteiras transacionais. Customer ID pode funcionar, mas clientes grandes e operações entre clientes precisam de tratamento. A chave deve estar disponível no caminho de roteamento sem consulta central cara por requisição.

Termo para lembrar: Shard key

03. Hash fixo ou tabela de roteamento?

Hash distribui com simplicidade; diretório permite movimentar clientes e isolar grandes contas. Defino versão e cache do mapeamento. Qualquer estratégia precisa evitar que clientes antigos continuem escrevendo no shard anterior após a mudança.

Termo para lembrar: Diretório

04. Como preserva IDs únicos entre shards?

Escolho identificadores e regras de unicidade que funcionem no novo escopo. Uma constraint local não prova unicidade global. Também mantenho identidades de operações já existentes para que retries durante a migração não criem duplicatas.

Termo para lembrar: Unicidade

05. Como copia enquanto produção continua escrevendo?

Uso uma fotografia consistente associada a uma posição de mudanças recuperável. Aplico a cópia e depois as alterações correspondentes sem lacunas. A ferramenta precisa suportar esse protocolo para o banco e o schema utilizados.

Termo para lembrar: Snapshot

06. Capturar WAL significa atraso zero?

Não. A captura e aplicação têm capacidade e lag variáveis. Monitoro posição, idade e bytes retidos, além da taxa do destino. Cutover exige um critério verificável de sincronização, não apenas o processo estar rodando.

Termo para lembrar: CDC

07. Por que não escrever nos dois bancos diretamente?

Uma escrita pode confirmar e a outra falhar. Sem obrigação durável, reconciliação e regra de autoridade, crio divergência. Prefiro manter um escritor autoritativo e propagar mudanças, ou explicitar todo o protocolo que torna dual write recuperável.

Termo para lembrar: Dual write

08. Como evita derrubar o banco durante a cópia?

Leio em ranges ou unidades limitadas, controlo throughput e monitoro impacto no primário e réplicas. Pauso quando o orçamento operacional é ameaçado. Uma migração não recebe capacidade ilimitada só por ser temporária.

Termo para lembrar: Backfill

09. Uma alteração recente chega antes do snapshot da mesma linha?

O protocolo de carga e CDC precisa impedir que o snapshot antigo sobrescreva a mudança nova. Uso fronteiras de posição ou versões compatíveis com a ferramenta. Não aplico arquivos e eventos em ordem arbitrária.

Termo para lembrar: Ordem

10. Como migra registros apagados durante o processo?

Capturo exclusões e sua identidade, com tratamento de tombstones ou operações equivalentes. Verifico que a cópia não ressuscita uma linha removida. Comparar apenas registros presentes no destino deixa esse tipo de divergência escondido.

Termo para lembrar: Deletes

11. Pode alterar schema no meio da migração?

Somente com política de compatibilidade e suporte do pipeline. Coordeno DDL, versão de consumidores e transformação. Uma alteração aparentemente pequena pode quebrar captura ou validação; congelo mudanças incompatíveis durante etapas críticas.

Termo para lembrar: Schema

12. O que compara nas leituras em sombra?

Comparo respostas semanticamente equivalentes sob posições temporais conhecidas, normalizando diferenças irrelevantes. Uma divergência causada por lag não é igual a corrupção. Registro causas e não envio efeitos externos a partir do caminho de sombra.

Termo para lembrar: Shadow reads

13. Contar linhas basta?

Não. Confiro chaves, conteúdo, exclusões, relações e invariantes de negócio, com checksums e comparações apropriadas. A soma de linhas pode coincidir enquanto valores estão errados. Também verifico pendências e limitações da ferramenta de validação.

Termo para lembrar: Validação

14. Cria todos os índices antes da carga?

Avalio custo de ingestão versus necessidade de consulta e validação. Alguns índices podem ser criados em etapa controlada, mas os de integridade exigem cuidado especial. O destino precisa estar pronto antes de receber tráfego real.

Termo para lembrar: Índices

15. Qual é o instante de troca da autoridade?

Defino uma barreira: interromper ou drenar escritas da unidade, alcançar a posição final, validar e mudar roteamento com fencing. Requisições em voo precisam de tratamento. Trocar DNS não prova que todos os escritores antigos pararam.

Termo para lembrar: Cutover

16. Como impede um processo antigo de continuar escrevendo?

Uso uma versão de propriedade ou controle de acesso validado pela autoridade de escrita. O processo precisa ser rejeitado quando estiver desatualizado. Apenas atualizar uma configuração central não interrompe uma instância pausada que depois retorna.

Termo para lembrar: Fencing

17. Posso voltar para o antigo a qualquer momento?

Não se o novo já recebeu escritas que o antigo desconhece. Defino até quando rollback é simples e quando exige sincronização reversa ou recuperação diferente. Reverter aplicação sem recuperar dados pode perder operações confirmadas.

Termo para lembrar: Rollback

18. Migra um cliente primeiro ou uma tabela inteira?

Escolho unidade que preserve as fronteiras de negócio, como tenant ou conjunto coeso. Um canary representativo revela carga e relações reais. Não separo tabelas de uma transação crítica entre autoridades sem protocolo adicional.

Termo para lembrar: Canary

19. O que acontece com um JOIN entre clientes?

Pode exigir fan-out, agregação ou nova projeção. Identifico consultas afetadas antes da migração e meço custo. Sharding não é transparente para todo acesso; endpoints e relatórios podem precisar de outro desenho.

Termo para lembrar: Cross-shard

20. Uma transferência atravessa dois shards. Como procede?

Escolho coordenação transacional suportada ou workflow financeiro com estados e contas de trânsito explícitos. Não substituo uma transação antiga por duas gravações independentes sem discutir a mudança de garantia e sua recuperação.

Termo para lembrar: Transações

21. O maior cliente não cabe no shard. Qual opção?

Posso dedicar capacidade ou subdividir o domínio quando invariantes e acesso permitirem. Diretório facilita isolar o cliente. Medir distribuição por bytes e trabalho evita balancear somente número de tenants e manter o hotspot.

Termo para lembrar: Hot shard

22. Como moverá dados novamente daqui a um ano?

Reutilizo o protocolo de cópia, CDC, validação e transferência de propriedade. Automação exige observabilidade e limites de impacto. Uma arquitetura sharded precisa de manutenção recorrente, não apenas do projeto inicial de migração.

Termo para lembrar: Rebalancing

23. O que faz com autoincrementos antigos?

Preservo IDs existentes e defino geração futura sem colisão entre autoridades. Verifico também referências externas e APIs. Não renumero dados por conveniência de armazenamento se isso quebra identidade ou deduplicação.

Termo para lembrar: Sequências

24. O log necessário foi removido. Como retoma?

Detecto a perda da posição recuperável e interrompo a promessa de sincronização. Refaço a carga ou uso outro mecanismo validado de reconciliação. Reiniciar o conector na posição atual pode pular alterações silenciosamente.

Termo para lembrar: Falha de CDC

25. Qual backup restaura durante a migração?

Mantenho documentação da autoridade por unidade e posições consistentes de dados e captura. Um restore deve recuperar também roteamento e identidades. Misturar snapshots de fases diferentes pode produzir escrita duplicada ou referências quebradas.

Termo para lembrar: DR

26. Quais sinais autorizam avançar de fase?

Lag dentro do critério, validações concluídas, carga suportável, erros conhecidos resolvidos e rollback ensaiado. Defino gates por unidade. Um dashboard verde de CPU não comprova equivalência dos dados nem segurança do cutover.

Termo para lembrar: Observabilidade

27. Pode prometer indisponibilidade zero?

Defino o que o usuário observará e os limites do protocolo. Uma breve drenagem pode ser absorvida por retries seguros, mas isso precisa de teste. Não uso o rótulo para esconder pausas, erros ou perda de garantias.

Termo para lembrar: Zero downtime

28. Quem assume os shards depois da migração?

Defino donos, runbooks, capacidade, backup, rebalanceamento e escalonamento de incidentes antes da troca. A equipe que copia dados não pode desaparecer deixando o novo sistema sem operação. O custo permanente entra na decisão.

Termo para lembrar: Ownership

29. Quando a migração está terminada?

Quando autoridade, consumidores, backups e operação estão no novo modelo e o antigo pode ser retirado com evidência. Também removo caminhos temporários e permissões. Manter dual write para sempre pode perpetuar complexidade e riscos.

Termo para lembrar: Conclusão

30. Que ensaio faria antes do cutover real?

Simulo queda após a última escrita na origem, atraso de CDC e um escritor antigo retornando. Depois tento rollback após escrita no destino. O ensaio deve provar ausência de perda e explicitar quando a reversão exige outro procedimento.

Termo para lembrar: Teste decisivo

10 trade-offs: ganho, custo e decisão

01. Hash versus diretório

Ganho. Hash simplifica roteamento

Custo e motivo. Dificulta movimentação seletiva

Como decidir. Diretório quando isolamento e rebalanceamento exigem flexibilidade

02. CDC versus dual write de aplicação

Ganho. Mantém uma autoridade inicial clara

Custo e motivo. Exige log, offsets e recuperação

Como decidir. Usar dual write somente com protocolo de reparação explícito

03. Cópia rápida versus impacto na produção

Ganho. Encurta janela do projeto

Custo e motivo. Pode saturar origem e rede

Como decidir. Limitar por orçamento de recursos e SLO

04. Índices antes versus depois

Ganho. Índices prontos simplificam validação

Custo e motivo. Custam ingestão e espaço

Como decidir. Separar integridade obrigatória de otimização adiável

05. Cutover global versus por unidade

Ganho. Global reduz coexistência

Custo e motivo. Aumenta blast radius

Como decidir. Preferir unidade coesa quando roteamento permite

06. Rollback simples versus operação contínua

Ganho. Voltar antes de novas escritas é fácil

Custo e motivo. Após cutover exige sincronização reversa

Como decidir. Definir ponto de não retorno e alternativa de recuperação

07. Validação integral versus amostral

Ganho. Aumenta confiança de equivalência

Custo e motivo. Custa I/O e tempo

Como decidir. Usar invariantes e comparações proporcionais ao risco

08. Pausa curta versus coordenação contínua

Ganho. Simplifica transferência de autoridade

Custo e motivo. Pode afetar a jornada

Como decidir. Comparar custo real com exigência de disponibilidade

09. Shard grande versus pequeno

Ganho. Menos componentes para operar

Custo e motivo. Hotspots e manutenção maiores

Como decidir. Definir tamanho máximo por testes e recuperação

10. Compatibilidade longa versus retirada rápida

Ganho. Permite migração gradual de consumidores

Custo e motivo. Mantém caminhos e custos duplicados

Como decidir. Retirar após evidência, com prazo e ownership

Duas alternativas ao caminho proposto

1. Otimizar e particionar dentro do banco atual

Quando faz sentido. Gargalo pode ser resolvido sem múltiplas autoridades

Como implementar. Revisar índices, consultas, arquivos históricos e particionamento local

Limites. Não elimina limite de escrita ou falha compartilhada do primário

2. Banco distribuído com transações suportadas

Quando faz sentido. Requisitos justificam coordenação embutida

Como implementar. Migrar mantendo contratos e avaliando latência entre nós e regiões

Limites. Ferramenta não elimina custo de coordenação, operação e migração de dados

Referências e alcance da evidência

1. AWS DMS: CDC

Carga e captura contínua, posições e limites de replicação.

https://docs.aws.amazon.com/dms/latest/userguide/CHAP_Task.CDC.html

2. AWS DMS: validação

Comparações, divergências e consumo adicional de recursos.

https://docs.aws.amazon.com/dms/latest/userguide/CHAP_Validating.html

3. PostgreSQL: logical replication

Snapshot inicial, aplicação de mudanças e conflitos possíveis.

https://www.postgresql.org/docs/current/logical-replication.html

Continue em: T017 · T040 · T065 · T067 · T077 · T118

C009 / M. Os 50 cenários enviados

Sistema centralizado de Notificações (Push, Email, SMS)

Uma plataforma de notificações transforma intenções de comunicação em entregas rastreáveis por canal, respeitando preferência, prioridade, validade e limites dos provedores.

Quando usar. Quando vários produtos precisam comunicar eventos sem implementar separadamente roteamento, retries e observabilidade.

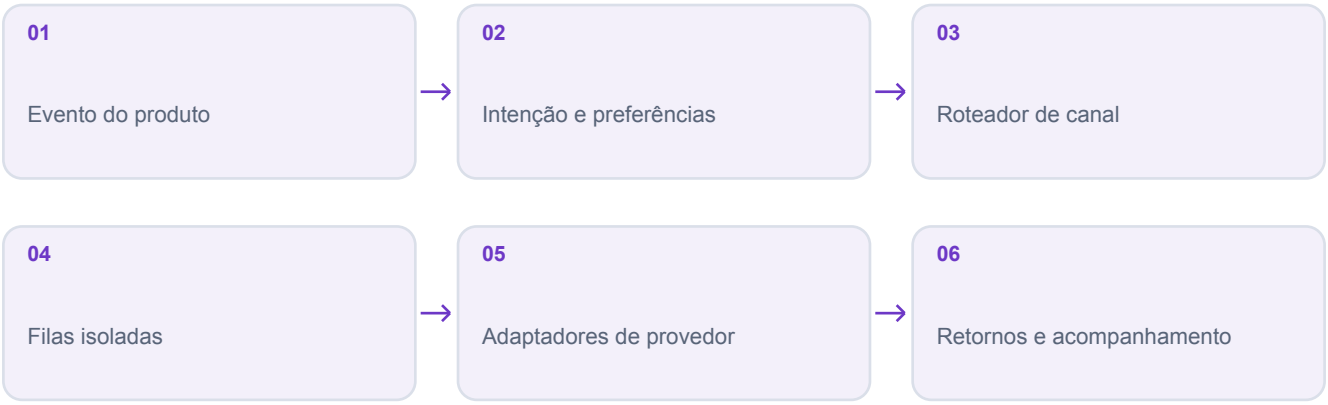
Quando evitar ou limitar. Evitar deduplicar apenas por texto, considerar aceitação do provedor como leitura do cliente ou tentar todos os canais em paralelo sem uma política.

Uma abertura possível

Vou distinguir a intenção de notificar das tentativas de entrega. Uma mensagem crítica e uma campanha têm prazos e permissões diferentes; o desenho deve preservar isso quando um provedor cair ou responder tarde.

Peças e tecnologias. Outbox · SQS/Kafka · PostgreSQL · FCM/APNs · SES/SMS providers · Redis para limites auxiliares

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. 0-8 min: classificar comunicação, prioridade, prazo, preferências e significado de entrega.

2. 8-18 min: modelar intenção, destinatário, canal, template e tentativa; definir identidade e estados.

3. 18-32 min: desenhar filas, limites e adaptadores, preservando resultado por provedor e canal.

4. 32-48 min: resolver duplicata, timeout, fallback tardio, token inválido e campanhas massivas.

5. 48-60 min: justificar centralização, medir latência e custo, proteger dados e explicar recuperação.

Cuidado técnico. Aceitação pelo provedor não comprova leitura humana.

Cuidado técnico. Hash do conteúdo não distingue sozinho repetição de intenção legítima.

30 perguntas para sustentar o desenho

01. O que significa notificação entregue?

Defino etapas: intenção aceita, provedor aceitou, canal reportou entrega e usuário leu quando houver evidência. Esses estados não são equivalentes. A plataforma não promete leitura de SMS ou push apenas porque uma API respondeu sucesso.

Termo para lembrar: Semântica

02. Como reconhece a mesma notificação?

Uso evento de negócio, destinatário e finalidade, com regras para campanhas e lembretes. Conteúdo igual pode ser legítimo em momentos diferentes. Um hash do texto sozinho pode suprimir comunicações válidas e não expressa a intenção.

Termo para lembrar: Identidade

03. O pagamento confirmou, mas a notificação não foi criada. Como evita?

O produto registra a obrigação de comunicação junto da mudança que a exige, ou usa captura durável equivalente. A plataforma recebe depois com deduplicação. Não dependo de uma chamada HTTP volátil após o commit financeiro.

Termo para lembrar: Outbox

04. Marketing pode atrasar alerta de segurança?

Não deveria compartilhar capacidade sem proteção. Separo classes, filas e quotas com reserva para mensagens críticas. Também defino limites para evitar que prioridade alta seja usada por todo produtor e perca significado.

Termo para lembrar: Prioridade

05. Quando consulta preferência do usuário?

Defino no contrato se vale no momento da intenção ou do envio, considerando o tipo de comunicação. Para campanhas, normalmente preciso respeitar mudanças recentes. Mensagens obrigatórias seguem política específica, não um bypass genérico de preferências.

Termo para lembrar: Preferências

06. Como impede que uma atualização mude mensagens já aceitas?

Vinculo a intenção a versão de template e dados necessários, ou explicito a política de renderização posterior. Uma alteração de marketing não deve reescrever silenciosamente uma comunicação transacional cujo conteúdo precisa ser explicável.

Termo para lembrar: Template

07. Como lida com idioma e moeda?

Renderizo por locale definido e formato valores a partir de dados tipados. Não monto texto por concatenação improvisada. Testo comprimentos, pluralização e canais limitados; uma tradução truncada pode esconder informação essencial.

Termo para lembrar: Localização

08. FCM informa token não registrado. Retenta?

Classifico como erro que exige atualizar ou remover aquele destino conforme o contrato do provedor. Repetir indefinidamente consome recursos sem entregar. Preservo a intenção e avalio outro canal somente se a política permitir.

Termo para lembrar: Token inválido

09. Quando retenta um envio?

Quando a classe de erro e o contrato permitem, com backoff, jitter, prazo e orçamento. A mesma intenção mantém identidade. Um erro permanente de endereço ou conteúdo não melhora apenas esperando alguns segundos.

Termo para lembrar: Erro transitório

10. O provedor pode ter enviado apesar do timeout?

Sim. Registro a tentativa como ambígua e uso identidade ou consulta quando disponíveis. Se o provedor não deduplica, existe risco de mensagem repetida. Defino o tratamento por criticidade, sem afirmar exactly-once fim a fim.

Termo para lembrar: Timeout

11. Push sem confirmação significa tentar SMS imediatamente?

Não necessariamente. Ausência de confirmação pode ser atraso, e o push ainda chegar depois. Defino prazo, prioridade e política de canais. Para alertas críticos, duplicidade pode ser aceitável; para campanhas, pode virar spam e custo.

Termo para lembrar: Fallback

12. SMS chegou depois de o usuário resolver o problema. Como trata?

Cada intenção tem validade e semântica de atualização. Não consigo recolher toda mensagem enviada, então evito texto que se torne perigoso e direciono a uma consulta atual autorizada. Cancelo tentativas ainda não iniciadas quando aplicável.

Termo para lembrar: Entrega tardia

13. Como evita avisar estorno antes da compra?

Se a ordem for necessária, mantenho sequência por operação ou destinatário e política para lacunas. Canais externos podem reordenar entrega; o conteúdo deve incluir contexto. Não prometo ordem global que o último estágio não suporta.

Termo para lembrar: Ordenação

14. Como limita spam sem perder alertas críticos?

Uso quotas por finalidade e janelas, com exceções explícitas para classes justificadas. Posso agrupar mensagens informativas, mas preservo eventos que exigem ação individual. Um limite único para todos os canais e produtos é insuficiente.

Termo para lembrar: Rate limit por usuário

15. Milhões de mensagens excedem a quota. O que faz?

Controlo taxa antes do adaptador e acompanho prazo das intenções. Distribuo capacidade por prioridade e provedor. Adicionar workers sem quota disponível apenas aumenta erros e retries; capacidade de saída é parte do planejamento.

Termo para lembrar: Quota do provedor

16. Um fornecedor ficou lento. Como isola?

Separo concorrência e filas para que chamadas presas não ocupem todos os workers. Aplico deadline e circuito conforme o padrão de falha. Preservo operações pendentes e evito abrir uma tempestade ao retomar.

Termo para lembrar: Bulkhead

17. Quando agrupa mensagens?

Para amortizar chamadas ou criar resumos quando a semântica permitir. Limito espera e tamanho. Agrupar alertas transacionais independentes pode mudar a informação recebida; eficiência não pode alterar silenciosamente o significado da comunicação.

Termo para lembrar: Batching

18. O que faz com mensagens na DLQ?

Registro motivo e dono, corrijo a causa e reenvio apenas intenções ainda válidas, preservando identidade. Uma campanha expirada não deve sair semanas depois. DLQ precisa de política de descarte ou resolução auditável.

Termo para lembrar: DLQ

19. Como processa retorno de entrega duplicado?

Valido a origem e correlaciono com tentativa e intenção. A transição é idempotente e considera ordem de evidência. Um callback antigo não deve regredir um estado mais forte nem contabilizar custo duas vezes.

Termo para lembrar: Callback

20. Pode colocar saldo e CPF no push?

Minimizo conteúdo exposto em tela bloqueada e canais externos. Uso referências para abrir uma área autenticada quando apropriado. Dados sensíveis não entram em logs ou templates sem necessidade e política de proteção.

Termo para lembrar: Segurança

21. Como protege links de ação?

Uso tokens restritos, validade e autorização no destino. Clicar no link não substitui automaticamente autenticação para uma operação sensível. Também evito URLs com dados pessoais e registro apenas correlação segura.

Termo para lembrar: Links

22. Uma plataforma única vira gargalo organizacional?

Pode virar. Defino contratos estáveis, self-service de templates e quotas, além de ownership de cada mensagem pelo produto. Centralizo mecanismos repetidos sem exigir que um time aprove manualmente toda comunicação de todas as squads.

Termo para lembrar: Centralização

23. Como envia uma campanha a milhões de usuários?

Materializo ou percorro uma audiência versionada em lotes recuperáveis, com identidade por destinatário. Evito uma transação gigante e protejo o banco de preferências. O progresso deve permitir retomar sem duplicar toda a campanha.

Termo para lembrar: Escala de fan-out

24. Pode cancelar uma campanha já iniciada?

Interrompo novas tentativas e marco intenções ainda não enviadas conforme política. Mensagens aceitas pelo provedor podem continuar chegando. Comunico esse limite; cancelar o job local não desfaz efeitos externos já iniciados.

Termo para lembrar: Cancelamento

25. Quais métricas por canal?

Atraso até tentativa, aceitação, entrega observada, falhas permanentes, ambiguidades, custo e idade de backlog. Separo por prioridade e provedor. Um agregado alto de sucesso pode esconder que alertas críticos estão perdendo prazo.

Termo para lembrar: Observabilidade

26. Como testa sem enviar mensagens reais?

Uso adaptadores de teste e ambientes com destinatários controlados, mantendo contrato e estados. Simulo timeout após aceite, callback duplicado e token inválido. Shadow de tráfego não pode disparar uma segunda comunicação aos clientes.

Termo para lembrar: Teste

27. Replay de eventos antigos deve notificar novamente?

Não por padrão. Reconstrução de estado e emissão de comunicação são modos diferentes. A identidade e a validade impedem reenvio acidental. Uma campanha de reaviso é nova intenção deliberada, com escopo e autorização próprios.

Termo para lembrar: Reprocessamento

28. Como escolhe entre push, e-mail e SMS?

Comparo urgência, alcance, preferência, evidência de entrega e custo por tentativa. O canal barato pode ser inadequado para a jornada. A política é configurável e versionada, com limites para fallback que multiplica despesas.

Termo para lembrar: Custo

29. Preciso de Kafka para começar?

Não. Uma fila gerenciada e armazenamento de intenções podem atender bem. Escolho Kafka quando replay, múltiplos consumidores e operação justificarem. A garantia importante é preservar a obrigação e tratar tentativas, não o nome do broker.

Termo para lembrar: Alternativa simples

30. Como fecha a defesa?

Mostro a intenção única, tentativas separadas, prioridade e validade. Explico que entrega externa tem limites e que fallback pode duplicar contato. A plataforma protege experiência, capacidade e rastreabilidade sem prometer que toda pessoa lerá a mensagem.

Termo para lembrar: Conclusão

10 trade-offs: ganho, custo e decisão

01. Plataforma central versus envio por produto

Ganho. Reutiliza controles e provedores

Custo e motivo. Pode virar dependência e gargalo organizacional

Como decidir. Centralizar mecanismos e manter ownership do conteúdo

02. Push versus SMS

Ganho. Push reduz custo e pode ser rico

Custo e motivo. Depende de dispositivo e permissões

Como decidir. SMS para casos em que alcance e política justificam custo

03. Fallback rápido versus espera

Ganho. Aumenta chance de contato urgente

Custo e motivo. Pode duplicar mensagem e custo

Como decidir. Definir por criticidade e evidência disponível

04. Batching versus envio imediato

Ganho. Amortiza chamadas e reduz ruído

Custo e motivo. Adiciona atraso e pode alterar semântica

Como decidir. Para resumos e campanhas, não toda comunicação

05. Snapshot de template versus versão atual

Ganho. Preserva conteúdo aceito

Custo e motivo. Retém mais metadados e versões

Como decidir. Para mensagens que precisam ser reproduzidas

06. Fila por provedor versus compartilhada

Ganho. Isola lentidão e quotas

Custo e motivo. Aumenta infraestrutura e roteamento

Como decidir. Quando falhas e limites diferem significativamente

07. Retenção de entrega versus minimização

Ganho. Facilita investigação e custo

Custo e motivo. Acumula dados de comunicação

Como decidir. Guardar metadados necessários com prazo definido

08. Deduplicação forte versus risco de repetição

Ganho. Reduz spam e cobrança duplicada

Custo e motivo. Pode exigir estado e contrato do provedor

Como decidir. Preservar identidade e explicitar limite externo

09. Campanha versionada versus audiência dinâmica

Ganho. Permite reprodução e retomada

Custo e motivo. Pode usar preferência desatualizada

Como decidir. Revalidar condições necessárias no envio

10. Prioridade versus fairness

Ganho. Protege alertas urgentes

Custo e motivo. Pode atrasar comunicação comum indefinidamente

Como decidir. Reservar progresso e prazo por classe

Duas alternativas ao caminho proposto

1. Fila gerenciada com adaptadores simples

Quando faz sentido. Poucos canais e volume moderado

Como implementar. Persistir intenção, enfileirar, limitar tentativas e acompanhar callbacks

Limites. Exige implementar preferências, validade e idempotência com cuidado

2. Plataforma de comunicação gerenciada

Quando faz sentido. Entrega e canais justificam fornecedor

Como implementar. Integrar por contrato, manter identidade e estado local de negócio

Limites. Fornecedor não substitui autorização, política de contato nem estratégia de saída

Referências e alcance da evidência

1. Firebase: códigos de erro do FCM

Classificação de erros de entrega e tratamento por tipo.

https://firebase.google.com/docs/cloud-messaging/error-codes

2. AWS SES: notificações de eventos

Retornos de entrega, bounce e complaint; duplicatas podem ocorrer.

https://docs.aws.amazon.com/ses/latest/dg/monitor-sending-activity-using-notifications.html

3. Stripe: webhooks

Exemplo público de assinatura, duplicatas e entrega de eventos.

https://docs.stripe.com/webhooks

Continue em: T042 · T046 · T060 · T061 · T064 · T101

C010 / M. Os 50 cenários enviados

Onboarding de usuários e KYC (Know Your Customer)

Onboarding é um processo durável de coleta, verificação e decisão, frequentemente com fornecedores e revisão humana. Verificar documento não equivale automaticamente a autorizar todos os produtos financeiros.

Quando usar. Quando etapas demoram, exigem evidências e precisam continuar após fechamento do aplicativo ou falhas externas.

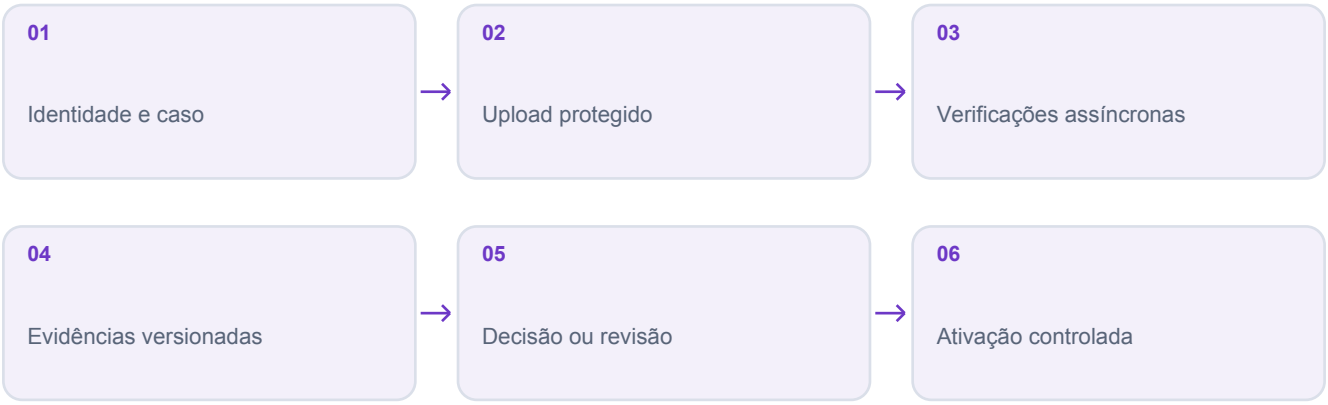
Quando evitar ou limitar. Evitar manter uma requisição aberta por horas, copiar documentos para logs, liberar acesso por timeout ou tratar uma compensação como apagamento mágico de dados enviados.

Uma abertura possível

Eu modelaria o onboarding como um caso versionado. O cliente pode sair e voltar, fornecedores podem responder fora de ordem e uma pessoa pode revisar; a decisão final precisa usar evidências válidas e a versão correta do caso.

Peças e tecnologias. Workflow durável/Temporal · PostgreSQL · Object storage privado · Outbox e inbox · Adaptadores KYC · Console de revisão

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. 0-8 min: definir produtos, verificações necessárias, tempo de análise, privacidade e revisão humana.

2. 8-18 min: modelar caso, evidências, tentativas de verificação, versões e permissões provisórias.

3. 18-32 min: desenhar upload, workflow, fornecedores e retorno persistido; liberar apenas capacidades autorizadas.

4. 32-48 min: tratar abandono, reenvio de documento, callback atrasado, indisponibilidade e decisão concorrente.

5. 48-60 min: discutir retenção, auditoria, throughput humano, métricas de conversão e rollout de regras.

Cuidado técnico. Verificação de documento não equivale automaticamente à aprovação de todos os produtos.

Cuidado técnico. Compensação não apaga o passado nem prova exclusão de dados já enviados a terceiros.

30 perguntas para sustentar o desenho

01. O que significa onboarding concluído?

Defino quais verificações e aprovações habilitam quais capacidades do produto. Uma identidade verificada pode ser apenas uma etapa. Não uso um único booleano para representar documento válido, risco aprovado e conta pronta para movimentação.

Termo para lembrar: Domínio

02. Por que não manter a chamada HTTP aberta?

Porque verificações e revisão podem durar muito além da interação. Persisto o caso e devolvo um identificador. A aplicação retoma consultando estado, enquanto um executor durável coordena tarefas, prazos e resultados independentemente da sessão.

Termo para lembrar: Workflow

03. Quais estados precisa distinguir?

Rascunho, aguardando evidência, em verificação, requer correção, revisão humana, aprovado, rejeitado e expirado, conforme o produto. Cada transição tem pré-condições e motivo. Erro técnico de fornecedor não deve virar rejeição automática do cliente.

Termo para lembrar: Máquina de estados

04. O usuário reinstalou o app. Cria outro onboarding?

Recupero o caso permitido pela identidade e pelo contrato, sem depender do dispositivo original. Distingo retomada de nova solicitação legítima. A criação tem chave estável e proteção concorrente para evitar casos acidentais duplicados.

Termo para lembrar: Identidade

05. Como recebe documentos grandes com segurança?

Uso upload restrito para armazenamento privado, limites de tamanho e tipo, validação e varredura apropriadas. O objeto só entra como evidência após conclusão e associação autorizada ao caso. URL assinada não autoriza qualquer arquivo ou usuário.

Termo para lembrar: Upload

06. Onde guarda imagens e resultados?

Separo objetos sensíveis de metadados operacionais, com criptografia, acesso mínimo e retenção definida. Logs recebem correlação e códigos seguros. Um desenvolvedor investigando timeout não precisa ter acesso irrestrito à foto do documento.

Termo para lembrar: Dados sensíveis

07. O cliente enviou outro documento antes do retorno. Qual vale?

Cada submissão tem versão e identidade. O callback referencia a verificação e a evidência correspondentes. Um resultado antigo não aprova automaticamente o documento novo nem sobrescreve a análise corrente.

Termo para lembrar: Versão da evidência

08. Como evita criar duas verificações cobradas?

Preservo identidade da etapa e uso idempotência ou consulta oferecida pelo provedor. Registro a obrigação antes da chamada e a tentativa separadamente. Se o resultado for ambíguo, recupero evidência antes de iniciar outra verificação.

Termo para lembrar: Fornecedor

09. Como valida o retorno externo?

Autentico ou verifico assinatura conforme contrato, confiro identidade, versão e associação ao caso. Persisto o recebimento e a transição de forma segura. Callback válido no transporte ainda pode ser duplicado, antigo ou irrelevante para a etapa atual.

Termo para lembrar: Callback

10. Se há webhook, por que manter consulta?

Porque callbacks podem falhar ou atrasar. Consulta ou reconciliação recupera resultados pendentes com limite de taxa e prazo. Não uso polling agressivo para todos os casos, mas um mecanismo controlado de fechamento de lacunas.

Termo para lembrar: Polling

11. O fornecedor recebeu o documento, mas a resposta sumiu?

Registro a tentativa ambígua e busco status pela referência estável. Não concluo que o documento foi rejeitado. Também não prometo que cancelar localmente removeu os dados já transmitidos ao fornecedor.

Termo para lembrar: UNKNOWN

12. Qual compensação faz sentido nesse processo?

Compensações são ações de negócio, como cancelar uma etapa ainda pendente ou revogar acesso provisório. Não desfazem o passado nem toda divulgação externa. Defino o que é reversível e o que exige retenção, correção ou tratamento próprio.

Termo para lembrar: Saga

13. Como inclui analistas sem travar o workflow?

Crio tarefa durável com evidências e prazo, e recebo decisão como comando autenticado e versionado. O workflow aguarda um evento, não uma thread ocupada. A fila humana tem capacidade, prioridade e métricas próprias.

Termo para lembrar: Revisão humana

14. Quando exige duas pessoas?

Quando o risco e a política pedirem segregação de funções. A segunda aprovação se vincula à versão e aos fatos revisados. Se o caso mudar, a aprovação anterior pode perder validade; não guardo apenas nomes numa tabela.

Termo para lembrar: Maker-checker

15. Dois analistas decidem ao mesmo tempo?

Uso versão esperada e transição condicional. Uma decisão vence conforme a regra; a outra recebe conflito e precisa revisar o estado atual. Não deixo last-write-wins escolher silenciosamente entre aprovar e rejeitar uma pessoa.

Termo para lembrar: Concorrência

16. Pode abrir a conta antes de todas as verificações?

Somente com uma matriz explícita de capacidades permitidas e restrições aprovadas. Tecnicamente separo criação de identidade, conta e habilitação de operações. Não trato workflow ainda pendente como autorização genérica para movimentar dinheiro.

Termo para lembrar: Permissões provisórias

17. O usuário não termina. Como limpa?

Defino expiração por etapa e política de retenção por dado. Cancelo trabalho ainda não necessário e preservo evidências obrigatórias de forma controlada. Apagar o caso local não garante exclusão de todas as cópias derivadas ou externas.

Termo para lembrar: Abandono

18. Quais falhas recebem nova tentativa?

Erros transitórios de infraestrutura, dentro de prazo e contrato. Documento ilegível exige ação do usuário; rejeição de regra exige outra decisão. Repetir tudo indiscriminadamente aumenta custo e pode irritar o cliente sem melhorar o resultado.

Termo para lembrar: Retry

19. Como evita que um fornecedor lento paralise outros casos?

Isolo concorrência e filas por etapa ou destino, com backpressure. Casos independentes avançam onde possível. Mantenho visibilidade de dependências pendentes; paralelizar não permite aprovar antes de uma verificação obrigatória.

Termo para lembrar: Fila

20. Qual gargalo pode ser mais importante que CPU?

A capacidade de revisão manual. Estimo chegada, tempo por caso e distribuição de complexidade, com prioridade por prazo. Escalar workers não reduz uma fila que depende de analistas e de evidências faltantes.

Termo para lembrar: SLA humano

21. Como informa rejeição sem expor regras exploráveis?

Apresento motivo e ação permitidos pelo produto e pelas obrigações aplicáveis, preservando detalhes sensíveis. A equipe autorizada acessa a evidência completa necessária. Transparência da experiência não significa publicar toda regra antifraude.

Termo para lembrar: Explicação

22. Como prova por que uma conta foi ativada?

Vinculo decisão, versão de política, evidências e responsáveis ao estado ativado. Não dependo de relatórios atuais do fornecedor para explicar o passado. O histórico registra também exceções e alterações posteriores.

Termo para lembrar: Auditoria

23. Mudou o código enquanto há casos antigos. O que acontece?

Mantenho compatibilidade com execuções em andamento ou faço migração explícita. No motor escolhido, respeito regras de replay e versionamento. Não suponho que deploy de código novo reinterpretará com segurança todo histórico antigo.

Termo para lembrar: Versionamento do workflow

24. Criptografar documentos elimina o problema de retenção?

Não. A retenção deve ter finalidade, prazo e controles de acesso e exclusão. Chaves também precisam de gestão. Criptografia reduz exposição, mas não autoriza manter todos os dados em qualquer ambiente indefinidamente.

Termo para lembrar: Privacidade

25. Como testa onboarding sem copiar clientes reais?

Uso dados sintéticos e ambientes controlados dos fornecedores, simulando respostas e documentos de teste apropriados. Valido contratos e falhas. Se houver dados reais em algum processo autorizado, minimização e controle precisam ser definidos, não presumidos.

Termo para lembrar: Sandbox

26. Pode reexecutar todas as verificações após um bug?

Defino escopo, versão e efeitos pretendidos, separando análise de nova chamada cobrada ou mudança de status. Reprocessamento pode exigir nova autorização ou revisão. Não reaplico automaticamente decisões históricas com regras atuais.

Termo para lembrar: Reprocessamento

27. Quais métricas mediria?

Conversão por etapa, tempo até decisão, pendências por motivo, custo por verificação, duplicatas e fila humana. Separo falha técnica de rejeição legítima. Uma taxa alta de conclusão não prova qualidade da decisão nem experiência adequada.

Termo para lembrar: Observabilidade

28. O que precisa sobreviver a uma falha regional?

Casos, versões, referências de evidências, resultados e obrigações pendentes, além do controle de acesso. Verifico também disponibilidade dos objetos e chaves. Recuperar só o banco de estados não restaura um processo utilizável.

Termo para lembrar: DR

29. Temporal é obrigatório para esse desenho?

Não. Um modelo relacional com jobs duráveis pode atender uma máquina de estados simples. Temporal ajuda em coordenação longa e recuperação, mas exige aprendizado e operação. Escolho pelo número de etapas, evolução e custo de manter a lógica própria.

Termo para lembrar: Ferramenta

30. Como fecha a arquitetura?

Mostro um caso que sobrevive à saída do cliente, uma evidência versionada e uma decisão autorizada. Explico retorno atrasado, revisão humana e ativação limitada. O workflow organiza o processo; a correção depende das transições e políticas que definimos.

Termo para lembrar: Conclusão

10 trade-offs: ganho, custo e decisão

01. Workflow engine versus jobs próprios

Ganho. Motor reduz infraestrutura de coordenação própria

Custo e motivo. Adiciona plataforma e regras de execução

Como decidir. Quando duração e evolução justificam complexidade

02. Verificações paralelas versus sequenciais

Ganho. Reduz tempo total

Custo e motivo. Pode aumentar custo em casos que seriam rejeitados cedo

Como decidir. Ordenar etapas baratas ou obrigatórias conforme política

03. Webhook versus polling

Ganho. Reduz consultas e latência

Custo e motivo. Retorno pode se perder ou repetir

Como decidir. Combinar callback com recuperação controlada

04. Evidência versionada versus substituição

Ganho. Preserva contexto de decisões

Custo e motivo. Aumenta armazenamento e gestão

Como decidir. Quando respostas chegam fora de ordem ou há revisão

05. Ativação gradual versus tudo ao final

Ganho. Pode reduzir fricção

Custo e motivo. Exige matriz de permissões e risco

Como decidir. Somente para capacidades explicitamente permitidas

06. Automação versus revisão humana

Ganho. Aumenta throughput

Custo e motivo. Erros podem escalar rapidamente

Como decidir. Definir exceções, qualidade e capacidade de contestação

07. Um fornecedor versus vários

Ganho. Simplifica integração e consistência

Custo e motivo. Concentra indisponibilidade e dependência

Como decidir. Diversificar quando benefício supera custo de comparar contratos

08. Retenção rica versus minimização

Ganho. Facilita auditoria

Custo e motivo. Amplia exposição de documentos

Como decidir. Guardar o necessário por finalidade e prazo

09. Expiração curta versus longa

Ganho. Reduz casos e dados abandonados

Custo e motivo. Pode frustrar retomada legítima

Como decidir. Escolher por etapa, risco e experiência

10. Reprocessamento automático versus campanha controlada

Ganho. Corrige rapidamente grande volume

Custo e motivo. Pode repetir custos e alterar decisões indevidamente

Como decidir. Controlar escopo, identidade e autorização dos efeitos

Duas alternativas ao caminho proposto

1. Máquina de estados relacional e workers

Quando faz sentido. Poucas etapas e equipe com domínio do banco

Como implementar. Transições condicionais, outbox, jobs com lease e reconciliação

Limites. Evolução, timers e histórico de coordenação tornam-se responsabilidade própria

2. Workflow durável com adaptadores de domínio

Quando faz sentido. Processo longo, muitas etapas e revisão humana

Como implementar. Motor coordena timers e retries; atividades usam identidades e evidências versionadas

Limites. Motor não valida KYC sozinho nem garante unicidade de efeitos externos

Referências e alcance da evidência

1. Stripe Identity: Verification Sessions

Exemplo público de sessão, retomada e resultados; não define toda obrigação de KYC bancário.

https://docs.stripe.com/identity/verification-sessions

2. Temporal: Workflows

Coordenação durável, histórico e replay; atividades externas exigem contratos próprios.

https://docs.temporal.io/workflows

3. Stripe: Webhooks

Validação e tratamento de eventos externos em um contrato público.

https://docs.stripe.com/webhooks

Continue em: T047 · T048 · T058 · T061 · T063 · T072 · T075 · T098

C011 / M. Os 50 cenários enviados

Arquitetura Multi-Região / Active-Active

Uma arquitetura multi-região distribui execução e dados entre domínios de falha. Active-active descreve tráfego ativo em mais de um lugar, mas não exige permitir escrita concorrente sobre o mesmo saldo.

Quando usar. Quando continuidade, latência geográfica ou isolamento justificam custo operacional e um contrato explícito de recuperação.

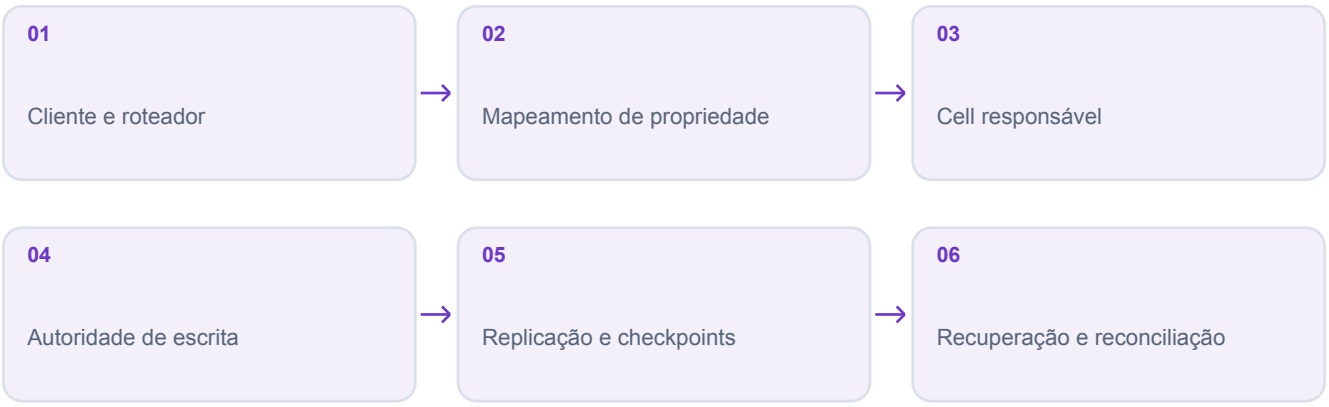
Quando evitar ou limitar. Evitar replicar escritores financeiros sem resolver autoridade, conflitos, fencing e recuperação. Uma segunda região sem testes não é uma estratégia de continuidade.

Uma abertura possível

Eu começaria separando disponibilidade da aplicação de autoridade sobre os dados. Posso servir clientes em duas regiões e ainda manter um único escritor por conta. A pergunta decisiva é quem pode aceitar uma operação quando a comunicação entre regiões cai.

Peças e tecnologias. Cells · Roteamento por conta · PostgreSQL/consenso distribuído · Replicação · Fencing token · Runbooks de recuperação

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. 0-8 min: definir jornada crítica, perda admissível, tempo de recuperação e residência dos dados.

2. 8-18 min: separar unidade de roteamento, autoridade de escrita e interações entre contas.

3. 18-32 min: desenhar cells, replicação, diretório e condições para assumir propriedade.

4. 32-48 min: simular partição, failover sem confirmação, escritor antigo e retorno de região.

5. 48-60 min: comparar custo de reserva, testar capacidade de recuperação e explicar risco residual.

Cuidado técnico. Active-active de tráfego não implica escrita concorrente na mesma conta.

Cuidado técnico. Cells não precisam coincidir com regiões.

30 perguntas para sustentar o desenho

01. O que significa active-active aqui?

Especifico se as duas regiões recebem tráfego, processam clientes diferentes ou escrevem a mesma entidade. São problemas distintos. Para saldo, começo com autoridade única por conta e explico como a propriedade muda com segurança.

Termo para lembrar: Escopo

02. Uma cell é uma região inteira?

Não necessariamente. É uma unidade de isolamento com capacidade e dependências limitadas. Uma região pode hospedar várias cells. Se todas dependem do mesmo banco global ou deploy simultâneo, parte do isolamento anunciado desaparece.

Termo para lembrar: Cells

03. Como encontra a cell de uma conta?

Uso mapeamento estável e versionado, consultado por um roteador simples. Posso manter cache desse diretório, mas o destino valida propriedade. Um roteamento antigo não deve autorizar escrita num proprietário que já foi substituído.

Termo para lembrar: Roteamento

04. Qual regra vem antes da disponibilidade?

Para uma operação que não pode gastar duas vezes o mesmo recurso, preciso impedir autoridades concorrentes. Durante uma partição, posso suspender aquela escrita. Outras jornadas, como consulta histórica claramente identificada, podem continuar.

Termo para lembrar: Invariante

05. Assíncrona permite recuperação sem perda?

Não garante. Um commit reconhecido pode ainda não existir na região de recuperação. Meço atraso e defino política para operações nesse intervalo. Se perda zero for obrigatória, preciso de confirmação durável no domínio exigido ou outro mecanismo equivalente.

Termo para lembrar: Replicação

06. Qual custo da replicação síncrona distante?

O caminho de commit depende da comunicação entre participantes e de sua disponibilidade. Aumenta latência e sensibilidade à rede. Comparo esse custo com o requisito real de durabilidade, em vez de vender distância geográfica como benefício gratuito.

Termo para lembrar: Latência

07. Qual é o primeiro passo ao promover outro escritor?

Estabelecer que a nova autoridade pode escrever e que a antiga não pode produzir efeitos válidos. DNS e balanceador não bastam. Verifico fencing, estado recuperado, dependências e operações ambíguas antes de reabrir a jornada.

Termo para lembrar: Failover

08. Como funciona a proteção contra escritor antigo?

Cada mudança de propriedade recebe uma época crescente, validada no recurso que aceita a escrita. Uma operação de época vencida é rejeitada. O token precisa ser imposto no destino; carregar um número no log não protege nada.

Termo para lembrar: Fencing

09. Heartbeat sumiu. A região morreu?

Não sei. Pode existir apenas uma partição entre observadores. Por isso não promovo autoridade com base somente na ausência de heartbeat. O protocolo de eleição ou a intervenção precisa preservar exclusão e reconhecer as limitações do ambiente.

Termo para lembrar: Split-brain

10. Um banco distribuído resolve toda a aplicação?

Ele pode oferecer garantias para suas transações e configuração. Não torna atômicos os efeitos em bandeiras, e-mails ou sistemas legados. Ainda preciso de identidade, publicação confiável e recuperação nas fronteiras externas.

Termo para lembrar: Consenso

11. Como transfere dinheiro entre duas cells?

Defino onde mora a obrigação e quais transições podem ocorrer em cada autoridade. Reservas, contas de trânsito e coordenação durável podem ser necessárias. Não descrevo dois commits locais como se fossem uma única transação global.

Termo para lembrar: Cross-cell

12. A região sobrevivente precisa ter quanto espaço?

Depende da carga a absorver e do tempo para expandir. Dimensiono reserva, quotas, bancos e conexões, incluindo retries e recuperação. Duplicar apenas instâncias da aplicação não cria capacidade de armazenamento ou do fornecedor externo.

Termo para lembrar: Capacidade

13. Como evita um bug derrubar todas as cells?

Faço rollout progressivo com observação e possibilidade de interromper, respeitando compatibilidade de dados. O isolamento físico não ajuda se a mesma configuração inválida chega simultaneamente a todas as unidades.

Termo para lembrar: Deploy

14. E se o serviço de mapeamento cair?

Mantenho decisões conhecidas utilizáveis conforme a política, limitando mudanças de propriedade durante a indisponibilidade. O destino continua validando a época. Preciso de um caminho de recuperação que não dependa circularmente do próprio diretório indisponível.

Termo para lembrar: Diretório

15. Como muda uma conta de cell?

Copio estado e acompanho alterações, estabeleço um ponto de corte e transfiro propriedade com fencing. Revalido consultas e efeitos pendentes. Manter dois escritores livres durante a cópia simplifica o desenho, mas pode quebrar a conta.

Termo para lembrar: Migração

16. Posso consultar qualquer réplica?

Somente conforme a consistência exigida. Uma leitura informativa pode aceitar atraso visível; uma decisão de autorização pode exigir a autoridade atual. Registro versão ou posição observada para oferecer leitura após escrita quando necessário.

Termo para lembrar: Leitura

17. O cache global acompanha o failover?

Não automaticamente. Incluo época ou versão e invalidade de entradas antigas na estratégia. Uma réplica de cache atualizada não prova que o banco promovido tem todos os commits que originaram aqueles valores.

Termo para lembrar: Cache

18. Onde fica a deduplicação numa troca regional?

Junto do estado que define o efeito, recuperável no mesmo contrato de durabilidade. Se o efeito sobrevive e a identidade se perde, uma tentativa repetida pode duplicar. Recuperar as duas coisas precisa fazer parte do plano.

Termo para lembrar: Idempotência

19. Como evita republicar eventos de forma perigosa?

Preservo identidade estável e consumidores idempotentes. Após recuperação, posso reenviar obrigações ainda não confirmadas. Não assumo que perder a confirmação de publicação significa que o evento nunca chegou ao broker ou aos consumidores.

Termo para lembrar: Outbox

20. Quando devolve tráfego para a região antiga?

Depois de restabelecer dados, capacidade e autoridade com um plano controlado. O retorno não deve ser automático por simples melhora de health check. A região antiga pode conter operações divergentes que exigem investigação.

Termo para lembrar: Failback

21. Como testa desastre sem causar outro?

Começo com escopo limitado e condições de interrupção, usando evidência de backups, fencing e capacidade. Exercito perda de comunicação, não só desligamento limpo. Verifico resultados do negócio, não apenas se a página voltou.

Termo para lembrar: Teste

22. Como mede perda potencial?

Comparo o ponto durável recuperável com operações reconhecidas, usando checkpoints e evidência do sistema. Atraso médio de replicação não é garantia de perda máxima. Também avalio mensagens, objetos e chaves necessários para reconstruir o serviço.

Termo para lembrar: RPO

23. O tempo termina quando a API sobe?

Não. Termina no critério de recuperação combinado: jornada útil, dados consistentes e capacidade suficiente. Uma API respondendo erro ou devolvendo saldos incompletos não cumpre automaticamente o objetivo de continuidade financeira.

Termo para lembrar: RTO

24. Dados podem atravessar qualquer região?

Depende das restrições do produto, contratos e obrigações aplicáveis. Trato residência e acesso como requisitos a confirmar com responsáveis. Não escolho região só por latência nem faço uma afirmação legal universal no desenho.

Termo para lembrar: Jurisdicação

25. Onde está o custo escondido?

Em capacidade ociosa, transferência, operação duplicada, testes e maior complexidade de mudança. Comparo isso ao impacto de indisponibilidade. Às vezes redundância entre zonas e recuperação regional ensaiada atende melhor ao estágio do negócio.

Termo para lembrar: Custo

26. Que visão precisa existir durante um desastre?

Propriedade atual, época, posição replicada, operações ambíguas, backlog e sucesso da jornada por cell. A telemetria também precisa sobreviver ao incidente. Um painel central indisponível não deve impedir decisões básicas de recuperação.

Termo para lembrar: Observabilidade

27. Quem decide uma promoção manual?

Um procedimento com critérios, responsáveis e registro, incluindo verificação de autoridade e risco de dados. Automatizar pode reduzir tempo, mas uma automação sem condições de segurança apenas produz erros mais rápido.

Termo para lembrar: Controle

28. Réplicas substituem backups?

Não. Corrupção lógica ou exclusão pode se propagar. Mantenho recuperação para pontos anteriores e testo restauração. Uma cópia disponível do mesmo erro não oferece a capacidade de voltar a um estado correto.

Termo para lembrar: Backup

29. Quando ficaria em uma região?

Quando múltiplas zonas, backups e recuperação regional atendem os objetivos com custo menor. Deixo uma evolução possível. Não uso multi-região como prova de senioridade quando o requisito não paga sua complexidade.

Termo para lembrar: Alternativa

30. Como fecha a defesa?

Mostro quem escreve cada conta, como a propriedade muda e quais operações param numa partição. Explico perda tolerada, tempo de retomada e teste. A arquitetura vale pelo comportamento durante a falha, não pela quantidade de regiões.

Termo para lembrar: Conclusão

10 trade-offs: ganho, custo e decisão

01. Um escritor por chave versus multiwriter

Ganho. Simplifica preservação de invariantes

Custo e motivo. Pode limitar disponibilidade daquela chave

Como decidir. Preferir para recursos que não toleram conflito arbitrário

02. Replicação síncrona versus assíncrona

Ganho. Reduz janela de perda reconhecida

Custo e motivo. Acrescenta latência e dependência de rede

Como decidir. Escolher pela durabilidade exigida e distância

03. Cells pequenas versus grandes

Ganho. Limita impacto de falhas

Custo e motivo. Aumenta unidades e custo operacional

Como decidir. Dimensionar pelo blast radius e eficiência

04. Roteamento estável versus livre distribuição

Ganho. Mantém afinidade e autoridade claras

Custo e motivo. Exige migração para rebalancear

Como decidir. Quando o estado segue uma unidade de negócio

05. Failover automático versus manual

Ganho. Reduz tempo de reação

Custo e motivo. Pode promover em condição mal compreendida

Como decidir. Automatizar critérios testáveis e manter controle de exceção

06. Capacidade reservada versus expansão posterior

Ganho. Melhora retomada imediata

Custo e motivo. Custa mesmo sem desastre

Como decidir. Quando a demanda não pode esperar provisionamento

07. Leitura local versus autoritativa

Ganho. Reduz latência e dependência

Custo e motivo. Pode mostrar estado atrasado

Como decidir. Separar informação de decisão financeira

08. Diretório central versus hash puro

Ganho. Permite migrações explícitas

Custo e motivo. Cria dependência de controle

Como decidir. Manter roteador simples e estado versionado

09. Failback rápido versus planejado

Ganho. Restaura distribuição original

Custo e motivo. Pode reintroduzir divergência

Como decidir. Retornar só depois de validar dados e autoridade

10. Multi-região versus recuperação ensaiada

Ganho. Oferece mais opções de continuidade

Custo e motivo. Multiplica complexidade e custo

Como decidir. Comparar objetivos mensuráveis de recuperação

Duas alternativas ao caminho proposto

1. Uma região com múltiplas zonas e recuperação regional

Quando faz sentido. Objetivos permitem tempo de restauração e eventual perda definida

Como implementar. Replicar entre zonas, manter backups fora do domínio e testar promoção regional

Limites. Não oferece escrita regional imediata durante todo desastre

2. Banco distribuído com consenso e propriedade lógica

Quando faz sentido. Transações e latência justificam participantes distribuídos

Como implementar. Usar topologia de quórum, colocação e contratos de consistência explícitos

Limites. Não elimina latência física nem atomicidade de efeitos externos

Referências e alcance da evidência

1. AWS: bulkheads e cells

Isolamento por cells, roteamento e limitação de dependências compartilhadas.

https://docs.aws.amazon.com/wellarchitected/latest/reliability-pillar/rel_fault_isolation_use_bulkhead.html

2. PostgreSQL: replicação e alta disponibilidade

Opções de replicação e limites de disponibilidade e consistência.

https://www.postgresql.org/docs/current/high-availability.html

Continue em: T020 · T061 · T062 · T064

C012 / M. Os 50 cenários enviados

Gerenciamento de chaves e segurança PCI-DSS para PIN do cartão

Criptografia de pagamentos protege operações como verificação e tradução de PIN por protocolos e chaves de uso restrito. Não é simplesmente armazenar um PIN cifrado e depois decifrá-lo na aplicação.

Quando usar. Quando o sistema participa de processamento de cartão que exige operações criptográficas específicas e segregação de responsabilidades.

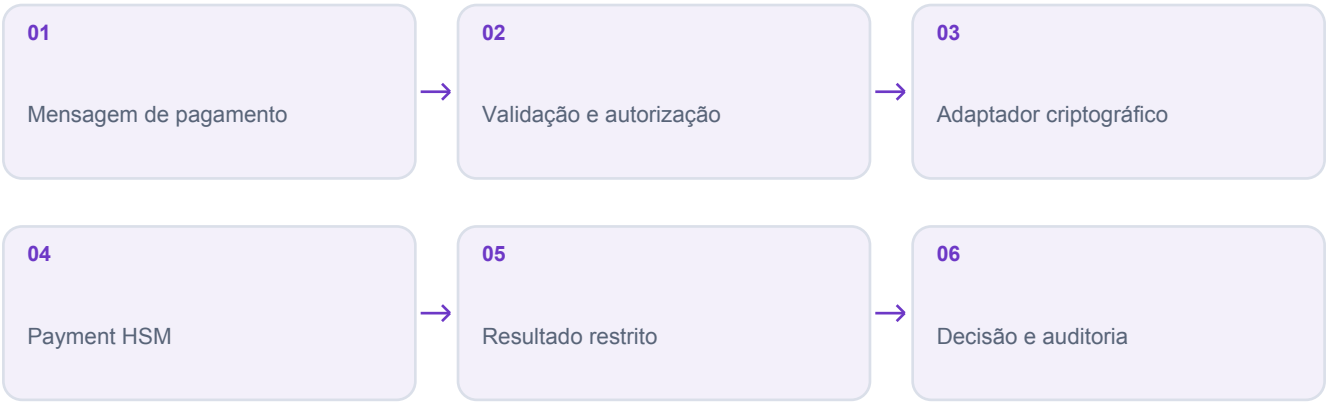
Quando evitar ou limitar. Evitar PIN em claro, logs de material sensível, chaves genéricas para todos os usos e promessas de conformidade baseadas apenas na contratação de um HSM.

Uma abertura possível

Eu isolaria as operações criptográficas de pagamento do código de produto. Primeiro confirmaria o papel da instituição e os protocolos com o processador; depois desenharia uma API mínima que usa a chave para uma finalidade autorizada sem entregar o segredo à aplicação.

Peças e tecnologias. Payment HSM · AWS Payment Cryptography · Azure Payment HSM · Controle de acesso · Key blocks · Auditoria segregada

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. 0-8 min: confirmar papel, operação necessária, protocolos, dados sensíveis e responsáveis por conformidade.

2. 8-18 min: desenhar fronteira criptográfica, identidades e operações permitidas, sem PIN em claro na aplicação.

3. 18-32 min: modelar ciclo de vida, finalidade e versões de chaves; integrar ao autorizador.

4. 32-48 min: simular timeout, rotação, perda de zona, credencial comprometida e esgotamento de quota.

5. 48-60 min: discutir custo, segregação, auditoria e testes sem segredos reais.

Cuidado técnico. Este cenário descreve engenharia e limites; não é uma declaração de conformidade PCI.

Cuidado técnico. Não armazenar PIN em claro nem expor chaves protegidas para otimizar latência.

30 perguntas para sustentar o desenho

01. KMS e payment HSM são a mesma coisa?

Não. Um serviço geral de chaves não implementa automaticamente as operações e protocolos de pagamentos exigidos. Escolho pela função concreta, como verificação ou tradução de PIN, pelos contratos e pelos controles aplicáveis ao papel da instituição.

Termo para lembrar: Escopo

02. Guardaria o PIN para comparar depois?

Eu não proporia um campo de PIN recuperável pela aplicação. O desenho usa mecanismos de verificação definidos pelo protocolo e executados na fronteira criptográfica apropriada. Os detalhes dependem da rede, do emissor e do processador.

Termo para lembrar: PIN

03. O que o serviço criptográfico expõe?

Operações específicas autorizadas, com referências de chave e parâmetros validados, retornando o mínimo necessário. Não ofereço uma API genérica que permita a qualquer consumidor decifrar qualquer dado ou exportar material sensível.

Termo para lembrar: API mínima

04. Por que distinguir finalidade da chave?

Porque uma chave autorizada para uma operação não deve poder ser reutilizada arbitrariamente em outra. Modelo finalidade, algoritmo e permissões como atributos controlados. Isso reduz abuso por erro de configuração ou comprometimento de um consumidor.

Termo para lembrar: Separação de uso

05. Para que servem blocos de chave protegidos?

Permitem transportar material de chave com proteção e atributos ligados ao seu uso no protocolo suportado. A integração precisa validar compatibilidade e confiança entre participantes. Não trato uma string cifrada qualquer como equivalente a um key block.

Termo para lembrar: Key blocks

06. Como importa uma chave de parceiro?

Uso o procedimento e os formatos suportados pelas duas pontas, com autorização e evidência de origem. Evito componentes secretos em tickets ou scripts comuns. A cerimônia e os controles precisam ser definidos por especialistas responsáveis.

Termo para lembrar: Importação

07. Permissão de usar a chave inclui administrá-la?

Não deveria ser uma concessão automática. Separo uso operacional, gestão, exportação quando permitida e auditoria. Um serviço que valida pagamentos não precisa poder desativar todas as chaves ou alterar seus atributos.

Termo para lembrar: IAM

08. Como troca a chave sem interromper tudo?

Versiono referências e coordeno compatibilidade com participantes, mantendo a janela necessária para mensagens em trânsito. Verifico quais operações usam a versão antiga. Trocar um alias local não atualiza magicamente o parceiro externo.

Termo para lembrar: Rotação

09. Uma chave comprometida deve seguir a rotação normal?

Não necessariamente. Comprometimento pode exigir interrupção ou substituição urgente, com tratamento de operações pendentes. Diferencio manutenção programada de resposta a incidente; disponibilidade não justifica continuar usando uma chave considerada insegura.

Termo para lembrar: Revogação

10. Como evita um HSM único parar o autorizador?

Uso redundância suportada e capacidade por domínio de falha, com chaves e permissões disponíveis conforme o modelo do produto. Testo falha real. Ter duas caixas não basta se só uma possui o material e configuração necessários.

Termo para lembrar: Disponibilidade

11. Pode cachear resultado de validação de PIN?

Não generalizo isso. Uma validação está ligada à mensagem, contexto e controles de tentativa. Reusar resultado sem contrato pode aprovar uma operação diferente. Otimizo conexões e capacidade antes de inventar cache de decisão sensível.

Termo para lembrar: Latência

12. Posso copiar a chave para Redis para reduzir custo?

Não como atalho para chaves que devem permanecer protegidas pela fronteira criptográfica. Isso altera radicalmente a ameaça e os controles. Avalio capacidade, quotas e operações suportadas, mantendo a finalidade de proteção do HSM.

Termo para lembrar: Chave em cache

13. Uma operação criptográfica expirou. Repito?

Classifico a operação e seus efeitos, incluindo contadores, estado e cobrança, conforme o contrato. Propago um deadline útil e evito retries cegos. Mesmo quando o cálculo é repetível, o processo de autorização ao redor pode não ser.

Termo para lembrar: Timeout

14. Por que limitar conexões ao HSM?

Para proteger sessões, throughput e latência de cauda. Mantenho fila limitada e observabilidade da espera. Abrir uma conexão por requisição ou aumentar concorrência indefinidamente pode piorar a disponibilidade em vez de elevar capacidade.

Termo para lembrar: Pooling

15. O que faz quando toda capacidade está ocupada?

Aplico admissão e prioridade definidas, preservando a jornada crítica e retornando falha controlada quando necessário. Não transformo indisponibilidade criptográfica em aprovação automática. Uma política excepcional exige autorização específica de risco.

Termo para lembrar: Sobrecarga

16. O que precisa registrar?

Identidade do chamador, operação, referência e versão da chave, resultado seguro e correlação, sem segredos. Os registros devem permitir investigação e controle de acesso. Logar o payload completo por conveniência pode destruir a proteção do desenho.

Termo para lembrar: Auditoria

17. Rede privada basta para confiar no chamador?

Não. Exijo identidade, autorização e validação da operação, além de controles de rede. Um workload comprometido dentro da rede continua sendo uma ameaça. A política deve restringir o que ele consegue pedir ao serviço.

Termo para lembrar: Rede

18. Qual o valor da separação de funções?

Reduz a possibilidade de uma única identidade criar, alterar e usar chaves sem controle. Defino responsabilidades e procedimentos proporcionais ao risco. Também preciso de recuperação de acesso, para a segregação não bloquear resposta legítima a incidentes.

Termo para lembrar: Segregação

19. Usar serviço certificado torna tudo conforme?

Não. A certificação e o escopo do serviço são apenas parte da avaliação. Aplicação, processos, acesso, integração e dados continuam sob responsabilidade definida. Eu confirmaria requisitos e evidências com a área especializada, sem prometer conformidade por diagrama.

Termo para lembrar: Conformidade

20. Como valida sem usar PIN real de clientes?

Uso vetores e materiais de teste apropriados, separados de produção, e contratos de integração controlados. Também testo negação de acesso e falha de dependência. Dados sensíveis reais não são necessários para provar o comportamento operacional básico.

Termo para lembrar: Teste

21. Como recupera uma chave perdida?

Depende do modelo suportado, procedimentos seguros e acordos com parceiros. Planejo recuperação antes da ativação e testo sem expor material. Não suponho que um snapshot da aplicação contenha ou deva conter as chaves necessárias.

Termo para lembrar: Backup

22. Pode levar chaves para qualquer região?

Somente conforme suporte do serviço, arquitetura e restrições aplicáveis. Disponibilidade regional exige planejamento de material e confiança, não apenas deploy do código. A decisão deve incluir participantes externos e responsáveis pelos controles.

Termo para lembrar: Região

23. Como diferencia erro de PIN de falha técnica?

Uso resultados tipados e seguros do contrato. Uma falha de infraestrutura não deve incrementar indevidamente uma classificação de credencial inválida nem ser comunicada como erro do cliente. Também evito revelar detalhes que facilitem exploração.

Termo para lembrar: Erro

24. Quem controla tentativas inválidas?

O domínio responsável mantém controles duráveis e coerentes com o protocolo. Instâncias independentes não podem cada uma oferecer uma nova quota completa. Cache auxiliar não deve apagar o histórico relevante após reinicialização.

Termo para lembrar: Tentativas

25. Como compara serviço gerenciado e HSM próprio?

Incluo throughput, operações suportadas, redundância, pessoal, certificações, cerimônias, integração e recuperação. O preço por chamada não captura todo custo. A opção própria exige capacidades operacionais que precisam existir de fato.

Termo para lembrar: Custo

26. Escolheria algoritmo só por ser mais novo?

Não. Preciso de compatibilidade, uso permitido e requisitos atuais do ecossistema de pagamentos. Uma migração envolve mensagens e parceiros. A entrevista deve mostrar critérios e dependências, não inventar um protocolo criptográfico novo.

Termo para lembrar: Algoritmo

27. O HSM decide se a compra tem saldo?

Não. Ele fornece operações criptográficas delimitadas. O autorizador ainda aplica saldo, limite e regras do produto. Misturar essas responsabilidades dificulta segurança e evolução; validação criptográfica não equivale a aprovação financeira.

Termo para lembrar: Autorização

28. Quais métricas ajudam sem expor segredo?

Latência por operação, espera por sessão, quota, erros técnicos e negações, usando dimensões controladas. Não incluo PIN, chave ou dados de cartão em labels. Investigação usa correlação segura e acesso restrito.

Termo para lembrar: Observabilidade

29. Como implanta uma nova versão do adaptador?

Valido compatibilidade com vetores e contratos, faço rollout restrito e monitoro resultados por versão. O rollback precisa respeitar chaves e formatos ativos. Reverter binário não desfaz uma mudança de protocolo já adotada pelo parceiro.

Termo para lembrar: Mudança

30. Como encerra a proposta?

Mostro a fronteira onde segredos permanecem, quem pode executar cada operação e como o serviço falha com segurança. Explico rotação e recuperação coordenadas. O mecanismo protege o pagamento sem transformar a aplicação em depositária de chaves.

Termo para lembrar: Conclusão

10 trade-offs: ganho, custo e decisão

01. Gerenciado versus equipamento próprio

Ganho. Reduz operação física e parte da gestão

Custo e motivo. Limita opções ao contrato e oferta do fornecedor

Como decidir. Comparar operações, controle e capacidade da equipe

02. API específica versus genérica

Ganho. Reduz superfície de abuso

Custo e motivo. Exige modelar novos casos de uso

Como decidir. Preferir operações autorizadas por finalidade

03. Rotação gradual versus troca imediata

Ganho. Preserva mensagens em trânsito

Custo e motivo. Mantém versões simultâneas por uma janela

Como decidir. Usar gradual em manutenção, incidente pode exigir corte

04. Redundância versus custo mínimo

Ganho. Suporta falha de componente

Custo e motivo. Aumenta custo e gestão de material

Como decidir. Dimensionar para jornada e recuperação exigidas

05. Pooling versus conexão por chamada

Ganho. Amortiza criação e protege capacidade

Custo e motivo. Requer limite, timeout e descarte de sessão

Como decidir. Quando o protocolo e cliente suportarem

06. Auditoria detalhada versus minimização

Ganho. Melhora investigação

Custo e motivo. Pode aumentar exposição e custo

Como decidir. Registrar metadados úteis sem segredos

07. Chave por finalidade versus reutilização

Ganho. Limita impacto e uso indevido

Custo e motivo. Amplia inventário e rotação

Como decidir. Quando protocolos e controles exigem separação

08. Falha fechada versus política excepcional

Ganho. Preserva controles obrigatórios

Custo e motivo. Pode recusar transações legítimas na falha

Como decidir. Qualquer exceção deve ser formal e delimitada

09. Rede isolada versus identidade por operação

Ganho. Reduz acesso de superfície

Custo e motivo. Rede sozinha não impede abuso interno

Como decidir. Combinar controles sem substituir autorização

10. Migração de protocolo versus compatibilidade

Ganho. Permite evolução de proteção

Custo e motivo. Depende de parceiros e transição

Como decidir. Planejar versões e testes antes de ativar

Duas alternativas ao caminho proposto

1. Serviço especializado gerenciado

Quando faz sentido. Operações e requisitos cabem no produto disponível

Como implementar. Integrar AWS Payment Cryptography ou oferta especializada equivalente com permissões restritas

Limites. Verificar regiões, limites e responsabilidade compartilhada

2. Payment HSM dedicado

Quando faz sentido. Protocolos, controle ou integração exigem equipamento dedicado

Como implementar. Operar cluster, material, cerimônias, capacidade e recuperação com equipe especializada

Limites. Custo e exigência operacional maiores; general purpose HSM não é substituto automático

Referências e alcance da evidência

1. AWS: Payment Cryptography

Escopo de funções criptográficas de pagamentos e distinção de serviços gerais.

https://docs.aws.amazon.com/payment-cryptography/latest/userguide/what-is.html

2. Azure: Payment HSM

Oferta de HSM voltada a pagamentos; capacidades e responsabilidades devem ser confirmadas.

https://learn.microsoft.com/en-us/azure/payment-hsm/overview

Continue em: T071 · T072 · T077 · T091

C013 / M. Os 50 cenários enviados

Agregação de saldo (Conta, Cartão, Cripto, Investimentos) na tela inicial

A home agrega visões de produtos com autoridades e ritmos de atualização distintos. O BFF compõe uma experiência útil sem transformar indisponibilidade parcial em um saldo enganoso.

Quando usar. Quando o aplicativo precisa de uma resposta adequada ao dispositivo a partir de múltiplos domínios independentes.

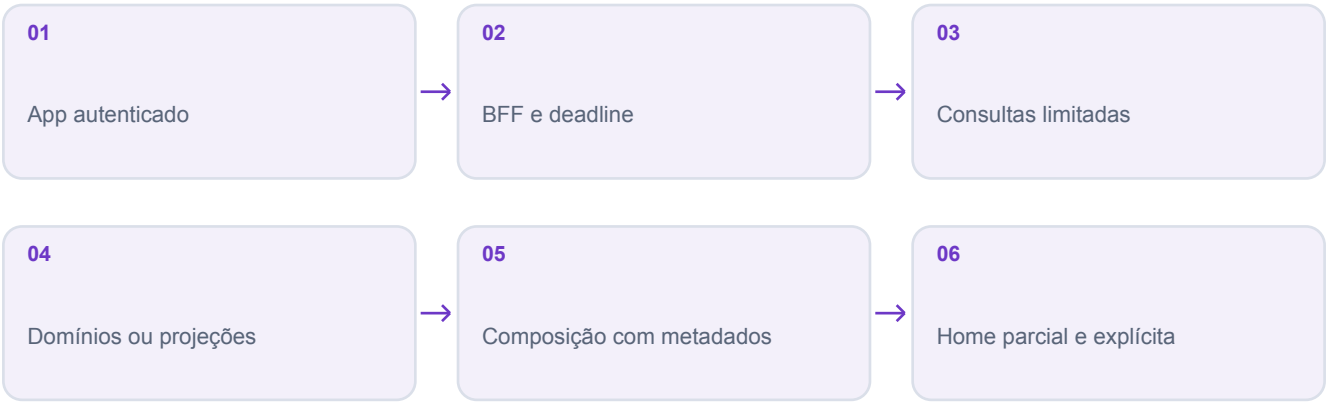
Quando evitar ou limitar. Evitar somar valores de moedas e significados diferentes, cache público de informações privadas e fan-out sem limite ou prazo.

Uma abertura possível

Eu definiria o que cada valor significa antes de agregá-lo. Saldo disponível, investimento e limite do cartão não são a mesma coisa. A home pode degradar por seção, mas precisa mostrar o que está atualizado e o que está indisponível.

Peças e tecnologias. BFF REST/GraphQL · HTTP/gRPC interno · Deadlines · Bulkheads · Projeções · Cache privado versionado

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. 0-8 min: definir seções, valores, frescor, acesso e tolerância a falhas por produto.

2. 8-18 min: desenhar contrato com estados por seção, moeda, data e versão.

3. 18-32 min: construir fan-out limitado, deadlines, autorização e política de cache.

4. 32-48 min: simular produto lento, leitura após pagamento, cache errado e pico de login.

5. 48-60 min: comparar projeção e consulta síncrona, medir cauda e testar experiência degradada.

Cuidado técnico. Cache de borda para dados privados exige configuração específica; não é equivalente a cache de assets.

Cuidado técnico. Saldo desconhecido não deve ser representado como zero.

30 perguntas para sustentar o desenho

01. Pode somar todos os saldos numa variável total?

Só se o produto definir o significado e a conversão. Limite disponível, patrimônio e dinheiro movimentável não são intercambiáveis. Prefiro valores tipados por moeda e finalidade, com data e fonte, para evitar uma soma financeiramente enganosa.

Termo para lembrar: Semântica

02. Qual responsabilidade fica no agregador?

Compor dados e experiência para aquele cliente, aplicando autorização e políticas de falha. A regra financeira continua no domínio responsável. Se o BFF recalcular limite ou aprovar pagamento, começa a criar uma segunda autoridade difícil de manter.

Termo para lembrar: BFF

03. Paralelizar quatro chamadas reduz sempre a latência?

Pode reduzir o tempo comparado à execução sequencial, mas a resposta completa depende da mais lenta. O custo total de chamadas permanece e a cauda pode piorar. Defino prazos e quais seções podem faltar.

Termo para lembrar: Fan-out

04. Como distribui o orçamento de tempo?

Parto do prazo útil da interação e reservo tempo para autenticação, composição e rede. Cada dependência recebe um limite coerente. Não dou a cada chamada o prazo total inteiro e depois somo esperas e retries.

Termo para lembrar: Deadline

05. Cancelar a chamada desfaz trabalho no serviço?

Não necessariamente. Propago cancelamento para liberar recursos quando suportado, mas não suponho rollback de efeitos já iniciados. Consultas da home devem evitar efeitos de negócio e limitar trabalho mesmo quando o cliente abandona a tela.

Termo para lembrar: Cancelamento

06. Cripto caiu. A home inteira falha?

Se o contrato permitir, retorno as outras seções e um estado explícito para cripto. Não mostro zero como se fosse saldo real. O aplicativo diferencia indisponível, carregando, desatualizado e valor confirmado.

Termo para lembrar: Falha parcial

07. O que a chave de cache precisa incluir?

A identidade e o escopo de autorização, além de versão, produto e parâmetros relevantes. Nunca uso uma chave genérica de home para dados privados. Mudança de permissão também precisa invalidar ou limitar reutilização da resposta.

Termo para lembrar: Cache

08. Colocaria essa resposta em cache público?

Não por padrão. Conteúdo financeiro personalizado exige política estrita de privacidade e separação. CDN pode servir assets; cache de API privada só com contrato explícito e configuração verificada, sem misturar respostas de usuários.

Termo para lembrar: CDN

09. Como informa que um saldo pode estar atrasado?

Retorno metadados de atualização ou versão e uma política de uso pela interface. A tolerância depende da seção. Uma projeção antiga não pode ser apresentada como confirmação de que um pagamento recente ainda não ocorreu.

Termo para lembrar: Frescor

10. O usuário acabou de pagar. Como evita susto?

Levo uma referência de versão ou operação para a consulta, aguardo a projeção dentro de prazo ou consulto a autoridade quando viável. Também posso mostrar a operação pendente explicitamente. Não invento um saldo definitivo no cliente.

Termo para lembrar: Read-your-writes

11. Quando criaria uma visão materializada da home?

Quando consultas repetidas e fan-out custam demais e o produto aceita o modelo de frescor. Consumo eventos com identidade e versão. A visão acelera leitura, mas introduz atraso, reprocessamento e responsabilidade por convergência.

Termo para lembrar: Projeção

12. GraphQL resolve automaticamente o fan-out?

Não. Ele organiza contrato e seleção de campos, mas resolvers podem gerar muitas chamadas e consultas caras. Preciso de batching, limites de custo, autorização e observabilidade. A escolha do protocolo não substitui desenho de dependências.

Termo para lembrar: GraphQL

13. Como evita consultar um serviço por item?

Agrupo chaves e uso APIs de lote ou carregamento por requisição quando apropriado. Defino limites para o lote. Um cache global improvisado pode reduzir chamadas e ao mesmo tempo vazar dados ou servir resultados incorretos.

Termo para lembrar: N+1

14. Uma API REST de home é inferior?

Não. Para uma tela estável, um contrato específico pode ser simples e eficiente. GraphQL ajuda quando flexibilidade e diversidade de clientes justificam sua governança. Comparo necessidades reais, não trato protocolo como nível de senioridade.

Termo para lembrar: REST

15. Por que usar gRPC internamente?

Pode oferecer contratos tipados, geração de clientes e recursos de RPC úteis. Ainda preciso controlar prazo e compatibilidade. Não assumo que o app móvel ou cada dependência se beneficia igualmente; HTTP com JSON pode ser suficiente.

Termo para lembrar: gRPC

16. Como protege o BFF de um produto travado?

Limito concorrência e recursos por dependência, evitando que uma fila esgote todas as threads ou conexões. Resposta parcial precisa de orçamento próprio. Sem isolamento, a tolerância declarada desaparece quando o processo inteiro fica saturado.

Termo para lembrar: Bulkhead

17. O BFF deve retentar toda consulta falha?

Somente quando houver tempo útil, erro transitório e orçamento. Repetições em várias camadas multiplicam carga. Para uma seção opcional, devolver indisponibilidade pode ser melhor do que gastar todo o prazo e prejudicar a home inteira.

Termo para lembrar: Retry

18. Circuito aberto significa que o dado é zero?

Não. É um estado operacional da dependência. Retorno indisponibilidade ou valor anterior identificado conforme a política. O cliente não deve interpretar a decisão do circuito como um resultado financeiro legítimo.

Termo para lembrar: Circuit breaker

19. Basta autenticar no gateway?

Não. Confiro que o usuário pode acessar cada conta e produto solicitado, com escopos apropriados também nas chamadas internas. Um identificador recebido do cliente não pode substituir verificação de ownership no domínio.

Termo para lembrar: Autorização

20. Todos os valores representam o mesmo instante?

Não automaticamente. Domínios independentes respondem em momentos e versões diferentes. Se uma visão global consistente for necessária, ela exige um mecanismo e custo próprios. Para a home, posso explicitar datas e evitar conclusões que exigem simultaneidade.

Termo para lembrar: Consistência global

21. Milhões abrem o app após uma campanha. Como protege?

Planejo cache seguro, limites e capacidade das dependências, com degradação por seção. Meço fan-out por login e evita expiração sincronizada. Escalar o BFF sozinho pode apenas transferir o pico para bancos menores.

Termo para lembrar: Pico

22. Como reduz custo na rede móvel?

Retorno dados necessários para a tela, comprimo quando apropriado e evito chamadas redundantes. Também considero cache de assets e atualização incremental. Reduzir bytes não compensa uma dependência que demora segundos sem deadline.

Termo para lembrar: Payload

23. Atualiza saldo a cada segundo?

Não como padrão. Defino eventos de atualização, intervalos proporcionais e pausas quando a tela está inativa. Push pode sinalizar mudança, mas a consulta autorizada recupera o estado. Polling agressivo multiplica custo sem garantir frescor absoluto.

Termo para lembrar: Polling

24. O que mostra sem conexão?

Uma visão anterior claramente identificada e ações limitadas, conforme a política. Não apresento informação local como autorização atual de gasto. O usuário precisa distinguir ausência de conexão de saldo realmente indisponível.

Termo para lembrar: Offline

25. Qual métrica mede sucesso da home?

Tempo até conteúdo útil, completude por seção, frescor, erros e latência de cauda. Não basta medir HTTP duzentos. Uma resposta rápida com todas as seções indisponíveis não representa a mesma experiência que uma home completa.

Termo para lembrar: Observabilidade

26. Como testa degradação da interface?

Simulo uma dependência lenta, outra com erro e uma projeção atrasada, verificando contrato e apresentação. Também testo dados de usuários distintos no cache. O teste precisa impedir zero falso e exposição cruzada, não apenas validar JSON.

Termo para lembrar: Teste

27. Como evolui a home sem quebrar apps antigos?

Uso evolução compatível, campos opcionais bem definidos e contratos testados. A interface antiga precisa entender estados suportados. Não removo uma semântica só porque a maioria dos clientes já atualizou.

Termo para lembrar: Versões

28. Quem responde por uma seção errada?

O domínio mantém a autoridade e contrato; o BFF responde pela composição e apresentação técnica. Defino donos e indicadores por fronteira. Uma plataforma central sem ownership de produtos pode esconder o local da falha.

Termo para lembrar: Ownership

29. Quando deixaria o app consultar diretamente os serviços?

Quando poucas chamadas e contratos seguros atendem a experiência sem expor acoplamento excessivo. Avalio rede, versionamento e autenticação. O BFF é uma ferramenta de composição, não uma camada obrigatória em todo sistema.

Termo para lembrar: Alternativa

30. Como fecha essa proposta?

Mostro uma resposta com valores, versões e estados por seção, e percorro a falha de um produto. Explico que a home permanece útil sem inventar informação. A autoridade financeira continua onde suas invariantes são garantidas.

Termo para lembrar: Conclusão

10 trade-offs: ganho, custo e decisão

01. Fan-out síncrono versus projeção

Ganho. Consulta fontes atuais diretamente

Custo e motivo. Depende de várias caudas e disponibilidades

Como decidir. Projeção quando frescor tolerado e custo de leitura justificam

02. Resposta completa versus parcial

Ganho. Completa simplifica interpretação

Custo e motivo. Uma dependência pode bloquear tudo

Como decidir. Parcial com estados claros quando produto permite

03. GraphQL versus REST específico

Ganho. Flexibilidade de seleção

Custo e motivo. Exige limites de custo e governança de resolvers

Como decidir. Escolher pela diversidade real de clientes

04. Cache versus consulta direta

Ganho. Reduz carga e latência

Custo e motivo. Introduz atraso e risco de chave incorreta

Como decidir. Cache privado com escopo e frescor explícitos

05. Leitura autoritativa versus réplica

Ganho. Melhora certeza para decisões

Custo e motivo. Aumenta dependência do escritor

Como decidir. Separar consulta informativa de confirmação crítica

06. Deadline curto versus completude

Ganho. Protege experiência e recursos

Custo e motivo. Pode omitir dados recuperáveis com mais tempo

Como decidir. Distribuir orçamento por importância da seção

07. Retry versus degradação

Ganho. Pode recuperar falha breve

Custo e motivo. Aumenta carga e latência

Como decidir. Retentar só com orçamento e benefício esperado

08. Agregador central versus BFF por cliente

Ganho. Reutiliza lógica de composição

Custo e motivo. Pode acumular requisitos incompatíveis

Como decidir. Separar quando experiências e ciclos diferirem

09. Atualização frequente versus custo

Ganho. Melhora sensação de frescor

Custo e motivo. Consome rede, bateria e capacidade

Como decidir. Atualizar por necessidade e eventos relevantes

10. Estado local offline versus vazio

Ganho. Mantém contexto ao usuário

Custo e motivo. Pode ser interpretado como informação atual

Como decidir. Identificar data e limitar ações financeiras

Duas alternativas ao caminho proposto

1. Endpoint REST de composição enxuto

Quando faz sentido. Tela com contrato estável e poucas fontes

Como implementar. Fan-out limitado, metadados por seção e cache privado quando permitido

Limites. Continua dependente de disponibilidade das fontes

2. Projeção de leitura por usuário

Quando faz sentido. Grande volume de leituras e frescor tolerado

Como implementar. Consumir eventos versionados e materializar seções com reconciliação

Limites. Exige tratar atraso, replay e leitura após escrita

Referências e alcance da evidência

1. Microsoft: Gateway Aggregation

Composição, falhas parciais, gargalos e limites do agregador.

https://learn.microsoft.com/en-us/azure/architecture/patterns/gateway-aggregation

2. gRPC: deadlines

Prazos e propagação no ciclo de chamadas RPC.

https://grpc.io/docs/guides/deadlines/

3. GraphQL: especificação

Modelo de execução, tipos e tratamento de resultados.

https://spec.graphql.org/October2021/

Continue em: T008 · T034 · T064 · T100

C014 / M. Os 50 cenários enviados

Prevenção de duplicidade em compras com cartão físico intermitente

Prevenir duplicidade em pagamentos exige reconhecer a intenção financeira através de mensagens, tentativas e etapas distintas. Uma repetição de transporte não é necessariamente uma nova compra, e uma apresentação posterior pode ser legítima.

Quando usar. Quando redes e terminais repetem mensagens, respostas se perdem e o autorizador precisa preservar reservas e lançamentos corretos.

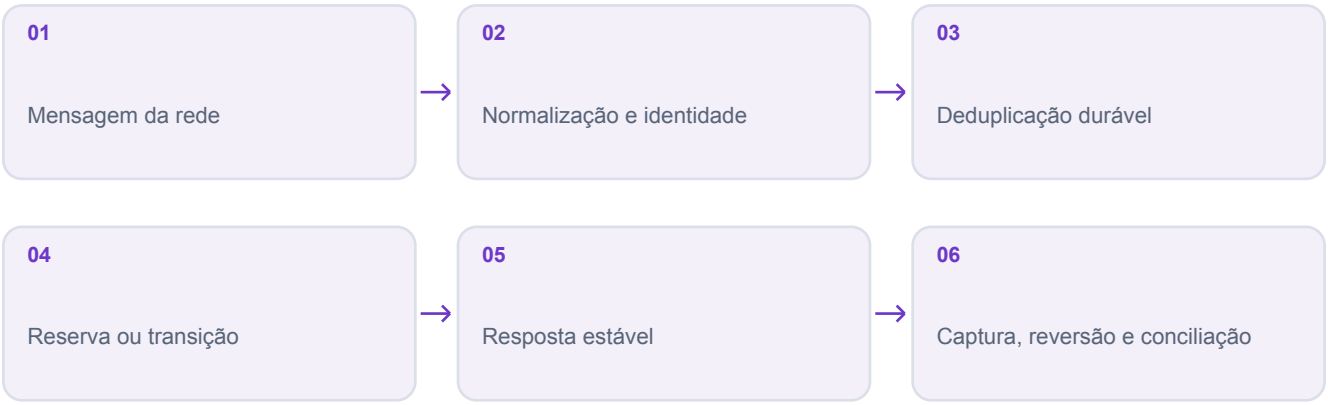
Quando evitar ou limitar. Evitar identificar compra apenas por valor e horário, usar Redis como única memória ou confundir autorização, captura e reversão.

Uma abertura possível

Eu separaria identidade da operação, identidade da mensagem e tentativa de entrega. A deduplicação deve estar ligada ao efeito durável. Uma constraint no banco é uma ferramenta forte; o trabalho é escolher a chave e a fronteira certas.

Peças e tecnologias. Identidade de rede · PostgreSQL unique constraints · Reserva de limite · Outbox/inbox · Estado UNKNOWN · Reconciliação

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. 0-8 min: confirmar mensagens de rede, etapas, prazos e o que caracteriza uma mesma operação.

2. 8-18 min: modelar compra, mensagem, tentativa, reserva e resultado durável.

3. 18-32 min: desenhar transação local com identidade e efeito, resposta e eventos.

4. 32-48 min: simular resposta perdida, concorrência, reversão antecipada e apresentação tardia.

5. 48-60 min: discutir retenção, reconciliação, limites de Redis e evidências de correção.

Cuidado técnico. Redis não substitui a proteção durável ligada ao efeito.

Cuidado técnico. Referências e janelas exatas dependem do protocolo da rede.

30 perguntas para sustentar o desenho

01. Qual chave usaria para deduplicar?

A chave vem da semântica e do contrato da rede, combinando referências estáveis quando necessário. Não invento um identificador só no gateway se a repetição externa não o preserva. Documento colisões, escopo e retenção.

Termo para lembrar: Identidade

02. Mensagem e compra têm o mesmo identificador?

Não necessariamente. Uma compra pode gerar várias mensagens, autorizações, capturas e reversões. Modelo essas entidades separadamente e mantenho referências entre elas. Isso permite repetir transporte sem apagar uma etapa financeira legítima.

Termo para lembrar: Tentativa

03. Uma unique constraint é uma solução junior?

Não. É um mecanismo robusto de exclusão concorrente para a chave definida. O nível da solução depende de integrar a restrição ao efeito, à resposta e à recuperação, e de compreender o que a chave representa.

Termo para lembrar: Constraint

04. Onde liga a chave ao débito ou reserva?

Na mesma transação ou autoridade que aceita o efeito. Se registro a chave e depois falho antes da reserva, posso perder a operação; se faço o inverso, posso duplicá-la. O estado precisa representar também execução pendente.

Termo para lembrar: Atomicidade

05. Duas instâncias recebem a mesma mensagem juntas?

A autoridade durável arbitra a identidade. Uma vence a criação ou transição; a outra observa o resultado ou o estado em andamento. Um teste prévio seguido de insert sem proteção deixa uma janela de corrida.

Termo para lembrar: Concorrência

06. A mesma chave chegou com valor diferente. E agora?

Comparo os campos relevantes ou um hash canônico do contrato. Trato como conflito e investigo, em vez de reutilizar silenciosamente a resposta. Canonicalização precisa preservar moeda, valor e identidade, sem depender da ordem do JSON.

Termo para lembrar: Payload

07. Onde Redis ajuda?

Pode acelerar consulta de resultados e reduzir carga, respeitando versão e escopo. A proteção financeira continua na autoridade durável. Eviction, reinício ou expiração do cache não devem tornar uma compra antiga novamente executável.

Termo para lembrar: Cache

08. Quanto tempo guarda a deduplicação?

Pelo ciclo de vida e pelas janelas de repetição e apresentação do contrato, com política de retenção apropriada. Um TTL curto escolhido apenas por memória pode liberar duplicação tardia. Diferencio dados operacionais e evidência histórica.

Termo para lembrar: TTL

09. Reservou saldo, mas a resposta sumiu. O que retorna no retry?

Recupero o resultado associado à mesma intenção, sem reservar novamente. Se ainda não houver resultado conclusivo, exponho estado apropriado ao protocolo e continuo recuperação. Não interpreto ausência de resposta como ausência de efeito.

Termo para lembrar: Resposta perdida

10. O que significa resultado desconhecido?

Que a evidência disponível não permite afirmar sucesso ou falha. Preservo identidade e recursos conforme a política, consulto ou reconcilio e evito nova intenção financeira. UNKNOWN é um estado operacional útil, não uma desculpa para esquecer a operação.

Termo para lembrar: UNKNOWN

11. Autorização já é a liquidação da compra?

Não. Ela pode reservar capacidade, enquanto captura e liquidação seguem etapas próprias. A modelagem precisa refletir o contrato. Deduplicar todas essas mensagens pela mesma chave e descartá-las indiscriminadamente pode perder efeitos legítimos.

Termo para lembrar: Autorização

12. Como lida com reversão antes da autorização conhecida?

Correlaciono e registro a evidência para resolução conforme o protocolo. Não crio um crédito solto sem relação. Uma máquina de estados pode aguardar dependência ou aplicar regra definida para mensagens fora de ordem.

Termo para lembrar: Reversão

13. Duas capturas sempre são duplicata?

Não. O contrato pode permitir capturas parciais ou múltiplas apresentações. Modelo identidade e limites de cada etapa, controlando total acumulado. A regra de negócio decide a validade; igualdade de referência principal não basta.

Termo para lembrar: Captura parcial

14. Duas compras iguais no mesmo minuto são a mesma?

Não necessariamente. Essa heurística pode detectar suspeita, mas não substitui identidade contratual. Bloquear automaticamente pode negar compras legítimas. Se a rede não oferece referência suficiente, explicito a ambiguidade e a política de risco.

Termo para lembrar: Valor igual

15. Como evita duas compras diferentes gastarem o mesmo limite?

Além de deduplicação, uso controle concorrente sobre o recurso compartilhado, como atualização condicional ou transação com isolamento adequado. Identidades diferentes não se excluem automaticamente. O saldo disponível precisa considerar reservas ativas.

Termo para lembrar: Saldo

16. Preciso de lock distribuído antes do banco?

Não por padrão. Se o banco já arbitra estado e efeito atomicamente, um lock externo pode acrescentar falhas sem melhorar a garantia. Avalio contenção e transações antes de introduzir outra autoridade de exclusão.

Termo para lembrar: Lock

17. A chave pode cair em dois shards?

A regra de roteamento deve ser estável e acompanhar a autoridade do recurso. Durante migração, fencing e versão impedem escritores concorrentes. Uma unique constraint local não garante unicidade global se a mesma intenção puder aparecer em dois lugares.

Termo para lembrar: Shard

18. Como publica a autorização sem perder evento?

Registro a obrigação de publicação com a mudança local e envio depois, preservando event_id. O consumidor deduplica. Isso fecha a lacuna entre commit e broker, mas não promete uma única entrega física.

Termo para lembrar: Outbox

19. Como o ledger recebe repetição do evento?

Registra recebimento e aplica o efeito na mesma fronteira transacional quando possível. Um identificador de negócio estável protege contra reenvios com envelope diferente. A confirmação ao broker ocorre depois da persistência necessária.

Termo para lembrar: Inbox

20. Chegou aprovação depois de timeout. Aceita?

Depende do estado atual e do contrato, incluindo eventual reversão ou expiração. Uma transição condicional evita ressuscitar uma operação encerrada indevidamente. O evento tardio vira evidência para reconciliação, não um update cego.

Termo para lembrar: Retorno tardio

21. Reserva expirada pode ser capturada depois?

A regra precisa ser definida com o protocolo e o produto. Trato apresentação tardia como evento próprio, com contabilização e exceções necessárias. Não suponho que expirar uma reserva local apague obrigações externas.

Termo para lembrar: Expiração

22. Por que reconciliar se há idempotência?

Porque identidade local não prova completude nem correspondência com a rede. Comparo autorizações, apresentações, reversões e valores para encontrar faltas e divergências. Idempotência reduz duplicação; reconciliação detecta lacunas e estados incompatíveis.

Termo para lembrar: Reconciliação

23. Quem deve repetir a operação?

Defino a responsabilidade por camada e limito tentativas com prazo. Se terminal, gateway e autorizador repetem livremente, a carga explode. Todas as tentativas legítimas mantêm a identidade necessária para recuperar o resultado.

Termo para lembrar: Retry budget

24. Como prova que não debitou duas vezes?

Relaciono mensagens e tentativas a uma operação e aos lançamentos duráveis, com restrições verificáveis. Logs ajudam investigação, mas não são a autoridade. Consultas de consistência e reconciliação verificam o resultado financeiro.

Termo para lembrar: Auditoria

25. Qual teste é indispensável?

Envio a mesma intenção simultaneamente e interrompo processos antes e depois do commit. Verifico um único efeito e resposta recuperável. Também testo uma compra legítima parecida, para garantir que a deduplicação não bloqueia tudo.

Termo para lembrar: Teste concorrente

26. Como simula a falha mais perigosa?

Faço o destino confirmar o efeito e descarto a resposta. O cliente repete com a mesma identidade. Esse teste verifica a janela ambígua que um simples erro antes da conexão não exercita.

Termo para lembrar: Teste de rede

27. Pode usar dados de cartão como chave em logs?

Evito expor dados sensíveis e uso referências seguras ou tokenizadas adequadas ao domínio. Uma chave de idempotência não precisa carregar o segredo. Correlacionar operação deve ser possível sem divulgar informações de pagamento.

Termo para lembrar: Privacidade

28. Qual indicador mostra falha de identidade?

Conflitos de payload, repetição por operação, efeitos financeiros divergentes e casos UNKNOWN envelhecidos. Também observo falsos bloqueios. Uma taxa alta de cache hit não demonstra que a deduplicação está correta.

Termo para lembrar: Métrica

29. Consigo garantir exatamente uma compra em toda rede?

Só posso afirmar garantias dentro de contratos e fronteiras controladas. Fora delas, preciso de referências, protocolos e reconciliação. Explico o risco residual quando o participante não oferece deduplicação ou consulta suficiente.

Termo para lembrar: Limite

30. Como resumiria a solução?

Cada intenção mantém identidade, cada tentativa deixa evidência e cada efeito é protegido na autoridade durável. Redis acelera, mas não decide a correção. Captura, reversão e reconciliação completam a jornada depois da autorização.

Termo para lembrar: Conclusão

10 trade-offs: ganho, custo e decisão

01. Unicidade durável versus cache exclusivo

Ganho. Preserva proteção após falhas

Custo e motivo. Custa armazenamento e coordenação

Como decidir. Obrigatória onde duplicação altera recurso financeiro

02. Chave contratual versus heurística

Ganho. Distingue repetição com precisão

Custo e motivo. Depende das referências externas

Como decidir. Heurística apenas como sinal quando identidade é insuficiente

03. Resultado persistido versus recomputado

Ganho. Permite resposta estável

Custo e motivo. Exige retenção e proteção de dados

Como decidir. Para operações cujo efeito não pode se repetir

04. Retenção longa versus TTL curto

Ganho. Cobre reapresentação tardia

Custo e motivo. Aumenta volume e governança

Como decidir. Dimensionar pelo ciclo real da operação

05. Lock local no banco versus distribuído

Ganho. Mantém arbitragem perto do efeito

Custo e motivo. Pode gerar contenção em recurso quente

Como decidir. Começar pela autoridade transacional existente

06. Reserva versus débito definitivo

Ganho. Representa autorização antes de captura

Custo e motivo. Exige expiração e transições adicionais

Como decidir. Quando o protocolo separa autorização e liquidação

07. Particionamento por conta versus por mensagem

Ganho. Agrupa concorrência do saldo

Custo e motivo. Pode concentrar contas muito ativas

Como decidir. Escolher pela invariante compartilhada

08. Resposta pendente versus falha definitiva

Ganho. Preserva honestidade diante da dúvida

Custo e motivo. Exige jornada de acompanhamento

Como decidir. Quando houve possível efeito sem confirmação

09. Deduplicação estrita versus falsos positivos

Ganho. Reduz efeitos repetidos

Custo e motivo. Chave errada pode bloquear compra legítima

Como decidir. Validar semântica e colisões do identificador

10. Reconciliação contínua versus periódica

Ganho. Reduz tempo de divergência

Custo e motivo. Custa processamento e integração

Como decidir. Escolher pela criticidade e disponibilidade de evidência

Duas alternativas ao caminho proposto

1. Banco relacional como árbitro

Quando faz sentido. Volume e fronteira transacional cabem numa autoridade

Como implementar. Constraint, transação de reserva e outbox com resultado persistido

Limites. Sharding exige preservar autoridade e unicidade relevante

2. Motor de ledger com identidade de transferências

Quando faz sentido. Produto se beneficia de primitivas financeiras dedicadas

Como implementar. Mapear mensagens a transferências e estados com invariantes explícitas

Limites. Ainda exige integração correta com rede, captura e reconciliação

Referências e alcance da evidência

1. PostgreSQL: constraints

Garantias de unicidade e restrições no banco.

https://www.postgresql.org/docs/current/ddl-constraints.html

2. AWS: retries idempotentes

Identidade do chamador e semântica de novas tentativas.

https://aws.amazon.com/builders-library/making-retries-safe-with-idempotent-APIs/

3. Stripe: autorizações em tempo real

Exemplo específico de fluxo de autorização; prazos do fornecedor não são universais.

https://docs.stripe.com/issuing/controls/real-time-authorizations

Continue em: T061 · T062 · T063 · T064 · T091 · T092

C015 / M. Os 50 cenários enviados

Distribuição e consumo massivo de webhooks para parceiros B2B

Uma plataforma de webhooks entrega eventos a destinos externos com isolamento, autenticação e rastreabilidade de tentativas. O emissor controla a obrigação de entregar, mas não controla toda a disponibilidade do parceiro.

Quando usar. Quando parceiros precisam reagir a eventos sem consultar continuamente o sistema e aceitam um contrato assíncrono explícito.

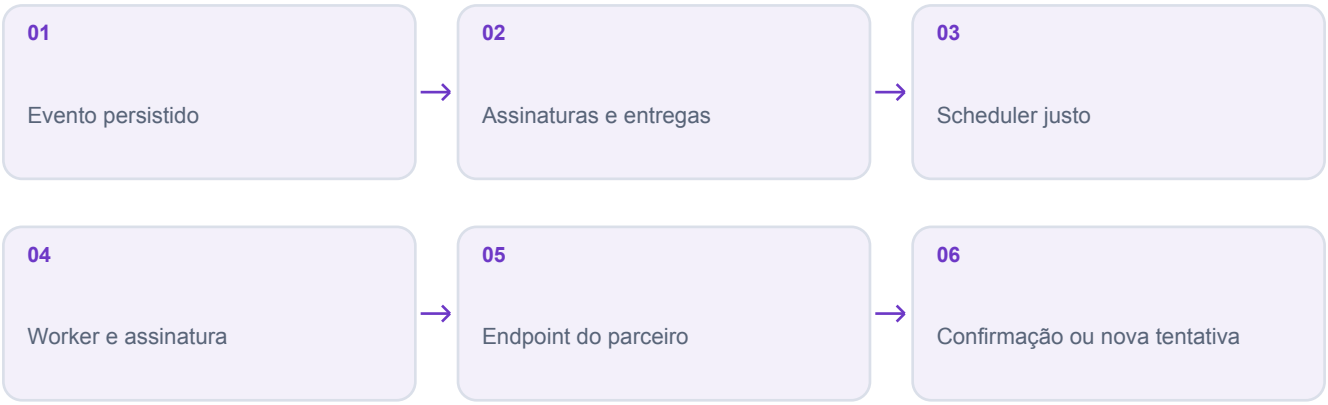
Quando evitar ou limitar. Evitar chamada externa dentro da transação principal, confiança em URL arbitrária e promessa de ordem ou entrega única sem suporte de ponta a ponta.

Uma abertura possível

Eu separaria evento de negócio, entrega para uma assinatura e tentativa HTTP. O parceiro pode cair por horas; isso não deve travar o produto nem outros parceiros. A recuperação precisa preservar identidade, prazo e uma forma de buscar eventos perdidos.

Peças e tecnologias. Outbox · PostgreSQL · SQS/Kafka · Workers por limites · HMAC e rotação · API de replay

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. 0-8 min: definir eventos, prazo, ordenação, retenção, escala e responsabilidade de cada parceiro.

2. 8-18 min: modelar evento, assinatura, entrega, tentativa e identidade estável.

3. 18-32 min: desenhar filas, scheduler, assinatura, limitação e resposta de recebimento.

4. 32-48 min: simular parceiro lento, duplicata, resposta perdida, SSRF e recuperação massiva.

5. 48-60 min: comparar filas dedicadas e compartilhadas, custo, replay e observabilidade.

Cuidado técnico. Um webhook com HTTP de sucesso não prova leitura humana nem conclusão de todo processamento remoto.

30 perguntas para sustentar o desenho

01. Por que separar evento e tentativa?

O evento representa um fato, a entrega representa sua obrigação para um destino e a tentativa é uma chamada concreta. Essa separação permite retry, múltiplos assinantes e métricas sem recriar o fato a cada falha.

Termo para lembrar: Modelo

02. Como garante que o evento será enviado após commit?

Uso outbox ou captura durável equivalente ligada à mudança de negócio. A plataforma materializa entregas de forma idempotente. Chamar o parceiro depois do commit sem registrar obrigação deixa uma janela de perda.

Termo para lembrar: Persistência

03. Como envia um evento para milhares de assinaturas?

Crio entregas em lotes recuperáveis, com chave por evento e assinatura. O progresso é durável e não depende de uma transação enorme. Quotas e particionamento limitam impacto de um produtor muito popular.

Termo para lembrar: Fan-out

04. Precisa de uma fila física por parceiro?

Não sempre. Posso usar filas compartilhadas com limites e agendamento por destino, reservando filas dedicadas para casos relevantes. Milhões de filas podem complicar operação; o requisito é isolamento efetivo de recursos e progresso.

Termo para lembrar: Isolamento

05. Como evita um parceiro consumir todos os workers?

Mantenho concorrência e taxa por destino ou classe, com scheduler que dá progresso aos demais. Um backlog grande não recebe capacidade ilimitada. Também separo tentativas novas de recuperação para evitar starvation.

Termo para lembrar: Fairness

06. Quanto tempo espera a resposta HTTP?

Defino um prazo curto o suficiente para proteger capacidade e compatível com o contrato de recebimento. O parceiro deve persistir e confirmar, processando depois quando necessário. Não espero toda a regra de negócio remota terminar.

Termo para lembrar: Timeout

07. HTTP duzentos prova que o parceiro executou o efeito?

Não. Prova somente o significado acordado para aquela resposta, normalmente recebimento aceito. O consumidor precisa persistir antes de confirmar. Se exige confirmação de processamento, modelo outro estado ou evento explícito.

Termo para lembrar: ACK

08. O parceiro processou, mas o ACK sumiu. E agora?

A entrega será tentada novamente com a mesma identidade. O consumidor deve deduplicar o efeito. Não existe como inferir sucesso apenas do timeout; a garantia depende também do contrato e da implementação do destinatário.

Termo para lembrar: Resposta perdida

09. Como protege autenticidade do payload?

Assino o conteúdo e metadados relevantes com o mecanismo definido, como HMAC, e documento verificação sobre bytes corretos. Uso transporte seguro e segredo por escopo. Assinatura não substitui autorização do evento nem criptografia em trânsito.

Termo para lembrar: Assinatura

10. Uma assinatura válida basta para aceitar sempre?

Não. O consumidor valida janela temporal quando prevista e identidade do evento, além da assinatura. Uma mensagem antiga autêntica ainda pode ser repetida maliciosamente. A janela e o replay legítimo precisam coexistir por contrato.

Termo para lembrar: Replay attack

11. Como troca segredo sem interromper entregas?

Versiono identificadores e mantenho uma janela controlada de verificação de chaves, conforme o contrato. Assino novas tentativas com material autorizado. O consumidor precisa saber quais versões aceita e quando deixa de aceitá-las.

Termo para lembrar: Rotação

12. Por que cadastro de URL é uma superfície de ataque?

Porque o servidor fará conexões em nome do usuário. Valido destinos e protocolos, restrinjo redes internas e redirects, e considero resolução DNS no momento da conexão. Validar apenas a string inicial deixa caminhos de abuso.

Termo para lembrar: SSRF

13. Retenta qualquer resposta diferente de sucesso?

Não. Classifico erros conforme contrato, distinguindo limitação, indisponibilidade e destino inválido. Defino prazo e escalada. Repetir para sempre uma configuração incorreta gasta recursos e oculta a necessidade de ação do parceiro.

Termo para lembrar: Erro permanente

14. Como escolhe intervalo de retry?

Uso atraso crescente com jitter e respeito sinais de limitação quando aplicável. O orçamento considera prazo e número de destinos. Após uma queda geral, evito que todos voltem a chamar no mesmo segundo.

Termo para lembrar: Backoff

15. O circuito pode descartar o evento?

Não deveria confundir pausa operacional com cumprimento da obrigação. Mantenho entrega pendente até expiração ou resolução contratual. Circuito limita tentativas durante falha; retenção e decisão de encerramento são políticas separadas.

Termo para lembrar: Circuit breaker

16. Como garante que atualizado não chega antes de criado?

Se necessário, serializo por entidade ou forneço sequência e recuperação de lacunas. Ordenação global custa caro e pode bloquear todos por um evento ruim. O consumidor deve conhecer as garantias reais, incluindo retries.

Termo para lembrar: Ordenação

17. Um evento ruim bloqueia todos os próximos?

Depende da unidade de ordem. Posso isolar por entidade e encaminhar exceção, mas pular evento muda a garantia. Defino como sinalizar lacuna e permitir recuperação; não prometo ordem estrita enquanto descarto silenciosamente falhas.

Termo para lembrar: Head-of-line

18. Envia snapshot completo ou só identificador?

Snapshot facilita processamento sem consulta, mas aumenta exposição e versionamento. Referência reduz payload, porém depende da API e pode refletir estado posterior. Escolho conforme semântica do evento e necessidades de reconstituição.

Termo para lembrar: Payload

19. Como evolui contrato sem quebrar parceiro?

Versiono schema e documente mudanças compatíveis, com período de migração. Testo payloads reais de contrato e preservo histórico necessário. Alterar o significado de um campo existente é mais perigoso que apenas adicionar um opcional.

Termo para lembrar: Versão

20. Como parceiro recupera eventos que perdeu?

Ofereço consulta ou replay por cursor e janela, com autorização e identidade original. Replay não cria um fato novo. Se a retenção expirou, explicito limite e caminho de reconciliação por estado de negócio.

Termo para lembrar: Replay

21. A DLQ encerra a responsabilidade?

Não. Ela organiza exceções com motivo, dono e ação. Um processo decide corrigir, reenviar, expirar ou contatar o responsável. Guardar indefinidamente sem observabilidade não torna o sistema resiliente.

Termo para lembrar: DLQ

22. Todo assinante recebe todos os campos?

Não. Aplico escopo e minimização por assinatura e finalidade. Verifico autorização também no replay. Um webhook é uma saída de dados para terceiro e precisa manter o mesmo cuidado de uma API consultada diretamente.

Termo para lembrar: Privacidade

23. Parceiro cancelou a assinatura. O que faz com pendentes?

Defino política de corte e retenção, suspendendo novas entregas conforme contrato. Registro a decisão para explicar o que não foi enviado. Não continuo divulgando dados porque um worker guardou uma cópia antiga da configuração.

Termo para lembrar: Desativação

24. O broker acumulou dias de eventos. Adiciona workers?

Só se houver capacidade e quota nos destinos. Caso contrário, mais workers aumentam erros. Priorizo entregas válidas, limito replay e acompanho idade do backlog para decidir expansão, renegociação ou expiração.

Termo para lembrar: Backpressure

25. Qual é o SLO mais útil?

Tempo entre obrigação criada e recebimento aceito, por classe e destino, separando indisponibilidade externa e falha interna. Também acompanho idade, tentativas e eventos expirados. Uma média global esconde parceiros críticos sem entrega.

Termo para lembrar: Métricas

26. Como testa o consumidor de exemplo?

Simulo duplicata, ordem invertida, assinatura inválida, timeout após persistência e payload futuro. Verifico um único efeito de negócio e confirmação durável. Testar só a chamada feliz não valida o contrato operacional.

Termo para lembrar: Teste

27. Que índices precisa no scheduler?

Acesso por estado e próxima tentativa, com particionamento ou índices compatíveis ao volume e à seletividade. Evito varrer todas as entregas a cada segundo. Claims concorrentes precisam impedir trabalho simultâneo ou tolerá-lo com identidade.

Termo para lembrar: Banco

28. Qual custo cresce mais rapidamente?

Número de entregas por evento e tentativas, não apenas eventos originais. Incluo egress, armazenamento e conectividade. Contratos de retenção e quotas impedem que um parceiro indisponível transforme um evento barato em custo ilimitado.

Termo para lembrar: Custo

29. Quando usaria pull em vez de webhook?

Quando o consumidor prefere controlar ritmo, conectividade de entrada é difícil ou replay é central. Uma API por cursor ou stream autorizado pode ser melhor. O custo muda para consultas e retenção, sem eliminar identidade e autorização.

Termo para lembrar: Alternativa

30. Como resume a garantia?

Preservo a obrigação, tento com limites e identidade estável e dou recuperação por replay. O parceiro confirma de forma durável e deduplica. Falhas ficam visíveis sem paralisar o produto ou outros destinatários.

Termo para lembrar: Conclusão

10 trade-offs: ganho, custo e decisão

01. Push versus pull

Ganho. Reduz latência e polling do parceiro

Custo e motivo. Emissor gerencia falhas de destino

Como decidir. Pull quando controle de ritmo e conectividade pesam mais

02. Fila dedicada versus compartilhada

Ganho. Isola recursos com clareza

Custo e motivo. Aumenta infraestrutura por destino

Como decidir. Combinar limites lógicos e dedicação seletiva

03. Ordem por entidade versus global

Ganho. Preserva sequência relevante com paralelismo

Custo e motivo. Exige chave e tratamento de lacunas

Como decidir. Evitar ordem global sem requisito concreto

04. Snapshot versus referência

Ganho. Snapshot reproduz o fato recebido

Custo e motivo. Aumenta dados e evolução de schema

Como decidir. Referência quando consulta atual é semântica suficiente

05. Retry longo versus prazo finito

Ganho. Recupera quedas prolongadas

Custo e motivo. Retém estado e pode entregar informação vencida

Como decidir. Contrato define validade e recuperação alternativa

06. HMAC versus assinatura assimétrica

Ganho. HMAC é simples e eficiente

Custo e motivo. Compartilha segredo entre participantes

Como decidir. Assimétrica quando modelo de confiança justifica gestão extra

07. ACK rápido versus processamento síncrono

Ganho. Libera conexões e reduz timeout

Custo e motivo. Exige persistência e execução posterior no parceiro

Como decidir. Confirmar depois de receber duravelmente

08. Replay amplo versus retenção curta

Ganho. Facilita recuperação histórica

Custo e motivo. Aumenta custo e exposição de dados

Como decidir. Definir janela e reconciliação além dela

09. Isolamento estrito versus utilização máxima

Ganho. Protege outros parceiros

Custo e motivo. Pode deixar capacidade ociosa

Como decidir. Reservar mínimos e compartilhar excedente controlado

10. Payload rico versus minimização

Ganho. Reduz consultas adicionais

Custo e motivo. Amplia divulgação e compatibilidade

Como decidir. Enviar apenas dados necessários ao contrato

Duas alternativas ao caminho proposto

1. API de eventos por cursor

Quando faz sentido. Consumidor quer controlar taxa e recuperar histórico

Como implementar. Paginar log autorizado com cursor estável e retenção definida

Limites. Mais polling e necessidade de acompanhar lag

2. Broker ou serviço de eventos gerenciado para parceiros

Quando faz sentido. Participantes suportam integração e identidade da plataforma

Como implementar. Publicar em destinos autorizados com quotas e contratos de consumo

Limites. Acoplamento ao fornecedor e semântica de entrega ainda exigem cuidado

Referências e alcance da evidência

1. Stripe: webhooks

Exemplo de assinaturas, duplicatas, retries e comportamento de entrega.

https://docs.stripe.com/webhooks

2. OWASP: prevenção de SSRF

Controles para conexões do servidor a destinos fornecidos externamente.

https://cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevention_Cheat_Sheet.html

3. AWS: backoff e jitter

Prazos e limites de retries para evitar amplificação de carga.

https://aws.amazon.com/builders-library/timeouts-retries-and-backoff-with-jitter/

Continue em: T042 · T046 · T061 · T062 · T063 · T064

C016 / M. Os 50 cenários enviados

Tolerância a falhas na emissão de cartões físicos

Emissão física é um processo durável que liga pedido, produção, expedição e entrega. Arquivos e APIs transportam instruções, mas confirmação de transporte não comprova produção do cartão.

Quando usar. Quando um fornecedor produz ou entrega um objeto físico e o processo precisa sobreviver a falhas, lotes e retornos tardios.

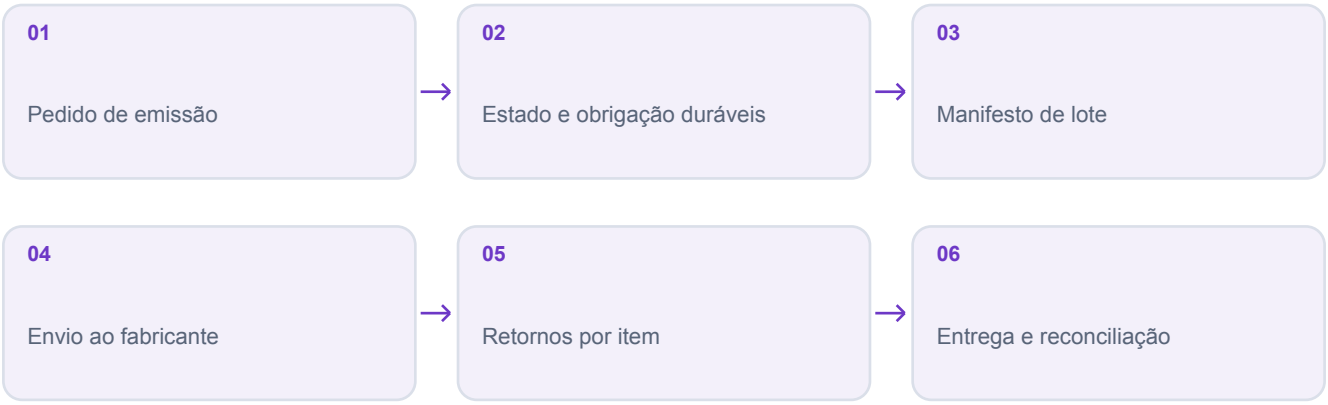
Quando evitar ou limitar. Evitar recriar pedido a cada timeout, reenviar ambíguos com nova identidade e tratar lote enviado como todos os itens produzidos.

Uma abertura possível

Eu manteria uma identidade de emissão por intenção do cliente, separada do lote e da tentativa de envio. Um novo lote pode agrupar trabalho novo e retries seguros, mas não pode transformar uma dúvida de recebimento em uma segunda produção física.

Peças e tecnologias. PostgreSQL · Outbox/inbox · S3 ou Blob Storage · SFTP/API de parceiro · Scheduler durável · Reconciliação

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. 0-8 min: definir intenção de emissão, etapas externas, cancelamento, prazos e identidade do parceiro.

2. 8-18 min: modelar pedido, lote, item, tentativa e retorno, com estado ambíguo separado.

3. 18-32 min: desenhar arquivo imutável, manifesto, envio e ingestão idempotente de respostas.

4. 32-48 min: simular queda durante upload, aceite sem resposta, lote parcial e endereço alterado.

5. 48-60 min: discutir automação de retries, limites de cancelamento, rastreabilidade e recuperação.

Cuidado técnico. Novo lote não é nova intenção de produção.

Cuidado técnico. Erro comprovado antes do envio e timeout após possível recebimento são situações diferentes.

30 perguntas para sustentar o desenho

01. Qual entidade não pode mudar no retry?

A intenção de emissão e sua referência estável para o parceiro. Lote e tentativa podem mudar sem criar nova intenção. Se o fornecedor usa outra chave para deduplicar, o contrato precisa preservar essa correspondência.

Termo para lembrar: Identidade

02. Quais estados diferencia?

Solicitado, validado, pronto para envio, envio em andamento, resultado desconhecido, recebido, em produção, expedido, entregue e exceções aplicáveis. Distingo falha técnica de rejeição de negócio. Um único campo failed perde informação importante.

Termo para lembrar: Estados

03. Quando responde ao aplicativo?

Depois de persistir a solicitação e a obrigação de avançar, devolvendo identificação e estado. Não espero fabricação. O texto para o cliente distingue pedido recebido de cartão produzido, e permite acompanhamento durável.

Termo para lembrar: Aceite

04. Como não perde o pedido após gravar no banco?

Registro a obrigação na mesma transação do pedido ou uso mecanismo equivalente. Um executor recuperável encontra trabalho pendente. Disparar um evento em memória depois do commit não garante continuidade após queda do processo.

Termo para lembrar: Outbox

05. O lote é a identidade da emissão?

Não. É um agrupamento de transporte. Cada item tem identidade própria, estado e vínculo com tentativas. Um lote pode ser parcialmente aceito, e a troca do arquivo não deve autorizar produção duplicada do mesmo item.

Termo para lembrar: Lote

06. O que guarda junto do arquivo?

Identidade do lote, versão do layout, itens, contagens, checksum, estado e referências do objeto. O manifesto permite verificar conteúdo e reconciliar retornos. Não dependo apenas de um nome de arquivo e da data da pasta.

Termo para lembrar: Manifesto

07. Pode sobrescrever um arquivo já enviado?

Evito. Um artefato publicado deve continuar identificável por versão ou conteúdo. Correções geram nova versão ou lote com relação explícita. Sobrescrever impede provar o que o fornecedor recebeu numa tentativa anterior.

Termo para lembrar: Imutabilidade

08. Como evita o parceiro ler arquivo incompleto?

Uso o protocolo acordado, como nome temporário seguido de publicação ou manifesto de conclusão. Verifico as garantias concretas do servidor. A existência de alguns bytes no destino não significa que o lote esteja pronto para consumo.

Termo para lembrar: Publicação

09. Uma notificação de objeto chega uma única vez?

Não devo assumir isso. Processamento deduplica por identidade e versão do objeto e registra estado durável. Também mantenho varredura ou reconciliação para obrigações não concluídas, em vez de depender de uma única notificação.

Termo para lembrar: S3 event

10. Upload concluído significa que o cartão foi produzido?

Não. Significa uma etapa de transporte conforme a evidência disponível. Preciso de confirmação de recepção, validação e produção, que podem chegar depois e por item. Misturar essas etapas cria uma experiência enganosa.

Termo para lembrar: SFTP

11. Se falhou antes de conectar, pode tentar de novo?

Se há evidência confiável de que nenhum envio foi iniciado, a tentativa é segura sob essa condição. Ainda preservo identidade e limites. Diferencio essa falha de timeout após escrita, quando o destino pode ter recebido conteúdo.

Termo para lembrar: Erro de conexão

12. O upload terminou, mas a confirmação sumiu. Reenvia em novo lote?

Não automaticamente com nova identidade de negócio. Consulto estado, verifico evidência ou uso deduplicação contratual. Marco UNKNOWN até resolver. Um novo arquivo pode ser interpretado como novo pedido pelo parceiro se o contrato não protege itens.

Termo para lembrar: Ambiguidade

13. Pode juntar pedidos novos e retries?

Sim, quando os itens de retry são elegíveis e mantêm identidade estável. Excluo ou trato separadamente os ambíguos conforme contrato. O lote anterior permanece auditável; o novo lote não apaga o histórico de tentativas.

Termo para lembrar: Novo lote

14. Rodar a cada quatro horas é suficiente?

Depende do prazo, cutoff e capacidade do parceiro. Pode ser adequado para lote, mas não garante recuperação por si. A seleção precisa considerar próximo horário, validade, estado e limites, sem reabrir itens já confirmados.

Termo para lembrar: Scheduler

15. Dois jobs selecionam o mesmo pedido. Como evita?

Uso claim transacional ou lease com controle de versão, além da identidade no efeito externo. O claim reduz trabalho duplicado; não substitui proteção contra uma tentativa antiga que continua viva após perder a posse.

Termo para lembrar: Concorrência

16. O fornecedor aceitou metade do lote. O que faz?

Atualizo por item e mantenho agregação do lote como derivada. Reenvio apenas itens elegíveis, preservando referências. Marcar o lote inteiro como failed e repetir tudo pode produzir cartões que já estavam em fabricação.

Termo para lembrar: Retorno parcial

17. Arquivo rejeitado deve entrar em retry automático?

Somente depois de corrigir a causa e gerar artefato válido com rastreabilidade. Repetir o mesmo layout inválido não ajuda. Diferencio erro permanente de dados, incompatibilidade de versão e falha transitória de infraestrutura.

Termo para lembrar: Layout inválido

18. Como processa o mesmo retorno duas vezes?

Valido origem, registro identidade do retorno e aplico transição condicional com o efeito local. O segundo recebimento recupera resultado sem repetir notificações ou mudanças financeiras. Inbox e outbox podem compartilhar a transação apropriada.

Termo para lembrar: Callback duplicado

19. Entregue chegou antes de expedido. Regride depois?

Não por ordem de chegada. Uso sequência ou precedência e evidência do contrato, preservando histórico. Um retorno antigo não deve voltar o pedido para uma etapa anterior. Inconsistências exigem investigação quando não houver regra segura.

Termo para lembrar: Ordem

20. O cliente pode cancelar a qualquer momento?

Depende da etapa. Antes de produção pode ser reversível; depois pode exigir bloqueio, descarte ou nova logística. Cancelamento local não recolhe um cartão já fabricado. A interface precisa comunicar o limite real.

Termo para lembrar: Cancelamento

21. O usuário mudou endereço durante produção. Qual vale?

Versiono os dados confirmados para aquela emissão e defino o ponto de corte. Mudança de cadastro não altera silenciosamente uma ordem já enviada. Se houver redirecionamento, trato como nova ação autorizada e rastreável.

Termo para lembrar: Endereço

22. Um cartão extraviado pode reutilizar a mesma intenção?

Normalmente é uma nova ação de negócio ligada à anterior, com regras de bloqueio e produção próprias. Não a confundo com retry de transporte. A identidade deve refletir que há um novo objeto autorizado a existir.

Termo para lembrar: Reposição

23. Como protege arquivos e logs?

Uso armazenamento privado, criptografia, acesso mínimo, retenção definida e campos necessários ao fornecedor. Logs guardam referências e resultados seguros. Arquivo temporário e diretório de transferência também entram no modelo de proteção.

Termo para lembrar: Dados sensíveis

24. O que compara periodicamente?

Pedidos locais, manifestos, recebimentos, produção e tracking do fornecedor. Procuro itens sem evolução, retornos órfãos e contagens divergentes. O objetivo é fechar obrigações, não apenas verificar que algum arquivo existe no S3.

Termo para lembrar: Conciliação

25. Quando manda para intervenção humana?

Quando a causa exige correção, evidência externa ou decisão de negócio que o algoritmo não possui. Registro contexto seguro e ação esperada. Automação reduz trabalho manual, mas não deve inventar sucesso ou duplicar produção para evitá-lo.

Termo para lembrar: DLQ

26. Como absorve uma campanha de emissão?

Persisto pedidos, controlo taxa por fornecedor e acompanho idade por prazo. Escalo preparação de arquivos sem exceder produção e logística. O gargalo real pode ser capacidade física, que mais pods não aumentam.

Termo para lembrar: Pico

27. Qual indicador representa o cliente?

Tempo até cada etapa prometida e pedidos sem avanço, segmentados por fornecedor e motivo. Distingo recebimento do pedido de entrega física. Um job bem-sucedido não prova que o cliente recebeu o cartão.

Termo para lembrar: Métrica

28. Qual falha precisa ser simulada?

Aceite do lote sem resposta, retorno parcial e callback duplicado após reinício. Verifico que cada emissão autorizada produz no máximo o efeito permitido e que pendências continuam recuperáveis. Teste só de upload não cobre o risco principal.

Termo para lembrar: Teste

29. API elimina a necessidade de lote e reconciliação?

Pode reduzir espera e simplificar transporte, mas ainda há timeout e processamento físico assíncrono. Identidade, estados e recuperação continuam necessários. Escolho API ou arquivo conforme o contrato, não como indicador automático de modernidade.

Termo para lembrar: Alternativa

30. Como fecha o desenho de ida e volta?

Mostro pedido durável, artefato identificável, tentativa rastreada e retorno por item. Só retento o que é seguro, reconcilio dúvidas e separo produção de entrega. Assim o sistema avança automaticamente sem transformar falha de rede em duplicação física.

Termo para lembrar: Conclusão

10 trade-offs: ganho, custo e decisão

01. Lote versus API

Ganho. Lote amortiza transporte e atende legado

Custo e motivo. Adiciona espera e retorno agregado

Como decidir. Escolher pelo contrato e prazo de negócio

02. Identidade por item versus só por arquivo

Ganho. Protege reenvio parcial e rastreabilidade

Custo e motivo. Exige modelo mais detalhado

Como decidir. Necessária quando um lote pode ter resultados mistos

03. Arquivo imutável versus sobrescrita

Ganho. Preserva evidência do enviado

Custo e motivo. Aumenta artefatos e retenção

Como decidir. Versionar correções e relacionar tentativas

04. Retry automático versus reconciliação

Ganho. Automação recupera falhas conhecidas

Custo e motivo. Na ambiguidade pode duplicar efeito externo

Como decidir. Retentar apenas com identidade ou evidência suficiente

05. Agendamento fixo versus próximo retry por item

Ganho. Fixo simplifica operação

Custo e motivo. Pode atrasar urgentes ou repetir simultaneamente

Como decidir. Combinar janela de lote e elegibilidade individual

06. Claim transacional versus processamento livre

Ganho. Reduz concorrência duplicada

Custo e motivo. Exige controle de posse e recuperação

Como decidir. Ainda proteger efeitos com identidade estável

07. Cancelamento cedo versus flexibilidade tardia

Ganho. Cedo é mais simples e previsível

Custo e motivo. Limita mudanças após corte

Como decidir. Explicar limites físicos e contratos de fornecedor

08. Dados congelados versus cadastro atual

Ganho. Reproduz ordem autorizada

Custo e motivo. Pode usar endereço que depois mudou

Como decidir. Alterações precisam de processo explícito

09. Retenção longa versus minimização

Ganho. Facilita investigação e prova

Custo e motivo. Aumenta exposição e custo

Como decidir. Separar metadados necessários de dados sensíveis

10. Um fornecedor versus múltiplos

Ganho. Simplifica integração e operação

Custo e motivo. Concentra risco e capacidade

Como decidir. Diversificar quando volume e continuidade justificam

Duas alternativas ao caminho proposto

1. Workflow durável com API do fabricante

Quando faz sentido. Parceiro oferece API e consulta por referência

Como implementar. Coordenar etapas, callbacks e retries com identidade estável

Limites. Processamento físico continua assíncrono e pode ficar ambíguo

2. Pipeline de arquivos com manifesto e scheduler

Quando faz sentido. Contrato exige lotes por SFTP

Como implementar. Gerar objetos imutáveis, publicar, receber retornos por item e reconciliar

Limites. Cutoffs, layouts e evidência de recepção exigem governança

Referências e alcance da evidência

1. Amazon S3: notificações de eventos

Entrega pelo menos uma vez e mecanismos de notificação de objetos.

https://docs.aws.amazon.com/AmazonS3/latest/userguide/EventNotifications.html

2. AWS: APIs idempotentes

Identidade e tratamento de novas tentativas sem recriar intenção.

https://aws.amazon.com/builders-library/making-retries-safe-with-idempotent-APIs/

3. Temporal: workflows

Modelo de execução durável; não torna efeitos externos automaticamente únicos.

https://docs.temporal.io/workflows

Continue em: T048 · T061 · T062 · T063 · T064 · T096

C017 / M. Os 50 cenários enviados

Serviço de Chat de Atendimento ao Cliente

Chat de atendimento combina comunicação em tempo real, histórico durável e distribuição de trabalho humano. A conexão é temporária; a conversa e suas obrigações precisam sobreviver a reconexões e trocas de atendente.

Quando usar. Quando cliente, bot e equipe humana precisam compartilhar uma conversa com contexto, prioridade e acompanhamento.

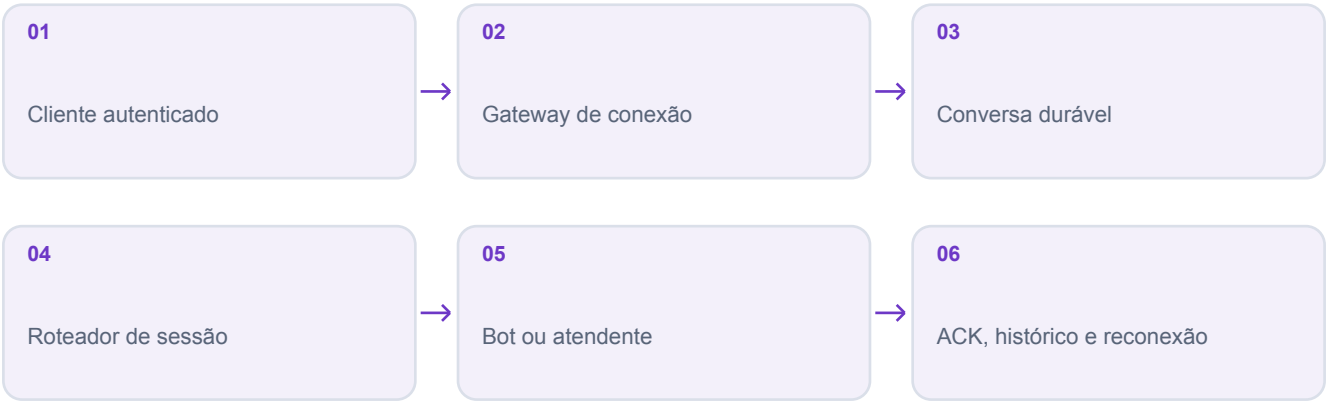
Quando evitar ou limitar. Evitar usar conexão como identidade, Redis Pub/Sub como histórico e presença como prova de entrega ou leitura.

Uma abertura possível

Eu separaria mensagem persistida, mensagem entregue ao dispositivo e mensagem lida. WebSocket melhora interação, mas não é o banco da conversa. Depois desenharia como o cliente recupera mensagens e como um atendente assume o caso sem perder contexto.

Peças e tecnologias. WebSocket/SSE · Banco de mensagens · Sequência por conversa · Outbox · Redis para presença · Roteamento e fila humana

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. 0-8 min: definir participantes, anexos, promessa de entrega, tempo de resposta e retenção.

2. 8-18 min: modelar conversa, mensagem, sequência, recibo e atribuição de atendimento.

3. 18-32 min: desenhar persistência, conexão, roteamento e retomada por cursor.

4. 32-48 min: simular rede móvel, nó perdido, handoff concorrente e fila humana saturada.

5. 48-60 min: comparar protocolos, custo de conexões, segurança e métricas da experiência.

Cuidado técnico. Redis Pub/Sub não é um log durável de chat.

Cuidado técnico. Presença, entrega e leitura têm significados diferentes.

30 perguntas para sustentar o desenho

01. A conversa pertence à conexão WebSocket?

Não. A conversa é uma entidade durável autorizada, enquanto a conexão é uma sessão substituível. O cliente pode trocar de rede ou dispositivo e continuar o caso. Identificadores de socket não devem definir o histórico.

Termo para lembrar: Identidade

02. Quando confirma que recebeu a mensagem?

Depois da persistência exigida pelo contrato. O recibo distingue aceite pelo servidor de entrega ou leitura pelo destinatário. Responder antes de guardar pode fazer a interface mostrar uma mensagem que desaparece após falha.

Termo para lembrar: Persistência

03. O app reenviou após reconectar. Como evita duas mensagens?

O cliente mantém um identificador estável da mensagem naquela conversa. A escrita durável protege unicidade e devolve o mesmo resultado. Uma nova conexão não gera uma nova intenção só porque o ACK anterior se perdeu.

Termo para lembrar: Deduplicação

04. Como ordena mensagens de vários dispositivos?

Defino uma sequência atribuída pela autoridade da conversa, junto de identidades e tempos auxiliares. Relógio do aparelho não é ordem confiável. Se houver múltiplos escritores, explico o mecanismo e as garantias de ordenação disponíveis.

Termo para lembrar: Ordem

05. Como recupera o que perdeu offline?

O cliente guarda a última sequência confirmada e consulta as posteriores, com paginação e autorização. A entrega ao vivo e o replay podem se sobrepor; deduplicação na interface evita repetição. Detecto lacunas em vez de assumir continuidade.

Termo para lembrar: Cursor

06. Redis Pub/Sub garante mensagens para quem estava desconectado?

Não. Uso esse mecanismo para sinalização efêmera quando apropriado, mantendo histórico em armazenamento durável. Após reconexão, consulto por cursor. Perder uma notificação não deve apagar a mensagem nem impedir a recuperação.

Termo para lembrar: Pub/Sub

07. Online significa que a pessoa leu?

Não. Presença é uma estimativa baseada em sessões e heartbeat. Leitura exige um recibo com significado definido, que ainda representa comportamento do cliente. Não uso presença como confirmação de que uma comunicação importante foi compreendida.

Termo para lembrar: Presença

08. O usuário reconectou em outro gateway. Como envia?

Atualizo o diretório efêmero de sessões e sinalizo o nó atual. Mensagens permanecem recuperáveis pelo histórico caso o mapeamento esteja atrasado. A correção não pode depender de um único registro de presença perfeito.

Termo para lembrar: Roteamento

09. Preciso fixar sempre o usuário no mesmo servidor?

Não para preservar a conversa. Afinidade pode reduzir movimentação de sessão, mas o estado durável e a retomada devem permitir outro nó. Caso contrário, uma falha transforma sticky session em perda de atendimento.

Termo para lembrar: Sticky session

10. Um cliente lento acumula mensagens em memória?

Limito buffers e envio, interrompendo ou reduzindo a sessão quando necessário. O cliente recupera histórico depois. Uma conexão lenta não pode consumir memória ilimitada nem impedir entrega aos demais usuários do gateway.

Termo para lembrar: Backpressure

11. Como envia arquivos sem bloquear o chat?

Uso armazenamento privado com upload controlado, validação e referência na mensagem. A disponibilidade do anexo tem estado próprio. Não coloco arquivos grandes no mesmo caminho de mensagens pequenas sem limites e controle de segurança.

Termo para lembrar: Anexos

12. Conhecer o ID da conversa dá acesso?

Não. Cada operação verifica vínculo e permissões atuais. O mesmo vale para anexos, histórico e recibos. Um link compartilhado ou identificador previsível não deve permitir ler atendimento financeiro de outra pessoa.

Termo para lembrar: Autorização

13. Como passa do bot para uma pessoa?

Crio uma transição durável com motivo, resumo verificável e histórico acessível. A atribuição define quem responde e quais ações o bot ainda pode executar. Não dependo apenas de uma mensagem informal dizendo que transferiu.

Termo para lembrar: Handoff

14. Dois atendentes assumiram o caso. Quem responde?

Uso atribuição versionada e regras explícitas de colaboração. Se só um pode atuar, uma transição condicional arbitra a posse. Atualizações antigas não podem substituir o responsável atual silenciosamente.

Termo para lembrar: Concorrência humana

15. Qual pode ser o maior gargalo?

Capacidade de atendimento e complexidade dos casos. Estimo chegada, duração e ocupação, considerando variação e prioridades. Escalar gateways não resolve espera por analistas; a arquitetura precisa tornar essa fila visível e gerenciável.

Termo para lembrar: Fila humana

16. Todo cliente urgente pode furar a fila?

A política precisa de critérios e proteção contra abuso, com progresso para classes comuns. Posso considerar risco e prazo, mas não invento prioridade apenas por ordem técnica. Métricas devem revelar starvation.

Termo para lembrar: Prioridade

17. Mede tempo de resposta do socket ou do atendente?

Ambos são diferentes. Defino entrega técnica, primeira resposta humana, resolução e abandono. Um chat com baixa latência de rede pode oferecer atendimento ruim se o cliente espera horas por alguém disponível.

Termo para lembrar: SLA

18. O bot pode movimentar dinheiro sozinho?

Somente por fluxos autorizados e controles do produto, com confirmação e identidade de operação quando exigidos. Resposta textual não é autorização financeira. Ações sensíveis usam APIs com as mesmas invariantes e auditoria de outros canais.

Termo para lembrar: Bot

19. Um resumo gerado substitui o histórico?

Não. Ajuda o atendente, mas pode omitir ou interpretar fatos. Mantenho acesso à evidência original e separo resumo de registro. Decisões sensíveis não devem depender exclusivamente de uma síntese sem possibilidade de conferência.

Termo para lembrar: Contexto

20. Pode editar ou apagar mensagem já enviada?

Depende da política. Modelo versões ou tombstones e preservo o que for necessário para auditoria e retenção. Não confundo alteração na interface com eliminação de todas as cópias e registros derivados.

Termo para lembrar: Edição

21. Guarda o chat para sempre?

Não por padrão. Defino finalidade, acesso, prazos e tratamento de anexos e backups. Arquivamento frio reduz custo, mas não remove obrigação de proteger dados. A recuperação precisa atender usos legítimos previstos.

Termo para lembrar: Retenção

22. Como permite que atendente encontre conversas?

Uso índices autorizados por escopo, evitando que pesquisa global exponha clientes indevidos. O índice é derivado e tem política de atualização e exclusão. Resultado de busca não substitui a verificação de acesso ao conteúdo.

Termo para lembrar: Busca

23. Manda push para cada mensagem?

Depende de presença, preferências e prioridade, com limites para evitar ruído. O push sinaliza nova atividade e não substitui histórico durável. Conteúdo sensível pode ficar fora da tela bloqueada e ser acessado autenticado.

Termo para lembrar: Notificação

24. O gateway morreu. Perde conversa?

Sessões caem e clientes reconectam com backoff, mas mensagens confirmadas permanecem no armazenamento. Refaço roteamento e replay por cursor. Preciso planejar tempestade de reconexão, não apenas capacidade estável de sockets.

Termo para lembrar: Queda de nó

25. Como atualiza gateways com conexões longas?

Dreno conexões progressivamente, sinalizo reconexão e mantenho compatibilidade de protocolo. Novas sessões vão para instâncias saudáveis. Encerrar todas simultaneamente pode gerar pico de autenticação, histórico e reconexões.

Termo para lembrar: Deploy

26. Como estima memória do gateway?

Considero conexões, buffers, estado de sessão, TLS e trabalho por mensagem. Número de sockets não equivale a throughput. Testo consumidores lentos e picos de fan-out para medir limites reais.

Termo para lembrar: Escala

27. WebSocket é sempre melhor que SSE?

Não. WebSocket permite comunicação bidirecional contínua; SSE com envio por HTTP pode atender alguns fluxos com simplicidade. Comparo suporte, proxies, autenticação, reconexão e custo operacional, preservando o mesmo modelo durável.

Termo para lembrar: Protocolo

28. Qual cenário comprova recuperação?

Persisto mensagem, perco o ACK, reconecto em outro nó e reenvio. Verifico uma única mensagem com sequência recuperável. Depois simulo handoff simultâneo e consumidor lento para testar responsabilidades e limites.

Termo para lembrar: Teste

29. Atendente pode ver todos os dados da conta?

Apenas o necessário para sua função e caso, com ações auditadas. Dados sensíveis podem exigir elevação ou mascaramento. Um console de atendimento não deve virar acesso administrativo irrestrito por conveniência operacional.

Termo para lembrar: Privacidade

30. Como resume o desenho?

A conexão entrega rápido, o histórico garante continuidade e a fila organiza trabalho humano. Mensagens têm identidade, sequência e recibos claros. Assim a conversa sobrevive à rede móvel sem confundir socket aberto com atendimento resolvido.

Termo para lembrar: Conclusão

10 trade-offs: ganho, custo e decisão

01. WebSocket versus SSE e HTTP

Ganho. Bidirecionalidade reduz atrito de interação

Custo e motivo. Exige gestão de conexão e infraestrutura compatível

Como decidir. SSE pode bastar quando fluxo de saída domina

02. Pub/Sub efêmero versus log durável

Ganho. Sinalização tem baixa latência e simplicidade

Custo e motivo. Perde mensagens para desconectados

Como decidir. Usar histórico durável para recuperação

03. Ordem por conversa versus global

Ganho. Preserva contexto com paralelismo

Custo e motivo. Exige autoridade da sequência local

Como decidir. Evitar coordenação global sem necessidade

04. Sticky session versus roteamento flexível

Ganho. Reduz mudança de nó

Custo e motivo. Pode concentrar carga e criar dependência

Como decidir. Retomada deve funcionar em qualquer nó autorizado

05. Recibo persistido versus imediato

Ganho. Melhora confiança na entrega aceita

Custo e motivo. Adiciona latência de armazenamento

Como decidir. Confirmar conforme a promessa apresentada ao usuário

06. Resumo do bot versus leitura completa

Ganho. Acelera handoff

Custo e motivo. Pode omitir contexto relevante

Como decidir. Manter evidência e capacidade de conferir

07. Prioridade versus ordem de chegada

Ganho. Protege casos críticos

Custo e motivo. Pode atrasar casos comuns

Como decidir. Definir fairness e métricas por classe

08. Histórico quente versus arquivo frio

Ganho. Melhora acesso recente

Custo e motivo. Custa mais armazenamento e índice

Como decidir. Mover por uso e retenção, preservando recuperação

09. Buffer amplo versus desconexão controlada

Ganho. Tolera cliente lento por mais tempo

Custo e motivo. Aumenta risco de esgotar memória

Como decidir. Limitar e recuperar por cursor

10. Automação versus atendimento humano

Ganho. Aumenta capacidade em casos simples

Custo e motivo. Pode errar contexto ou frustrar usuário

Como decidir. Transferência clara com limites de ação

Duas alternativas ao caminho proposto

1. SSE para receber e HTTP para enviar

Quando faz sentido. Interação não exige canal bidirecional completo

Como implementar. Persistir mensagens, emitir eventos e retomar por cursor

Limites. Ainda precisa de autorização, limites e tratamento de desconexão

2. Plataforma de atendimento gerenciada

Quando faz sentido. Operação humana e integrações justificam fornecedor

Como implementar. Integrar identidade, eventos e ações sensíveis por contratos restritos

Limites. Retenção, exportação, custo e dependência do fornecedor precisam ser avaliados

Referências e alcance da evidência

1. IETF: WebSocket, RFC 6455

Protocolo de comunicação WebSocket; persistência de mensagens é responsabilidade da aplicação.

https://www.rfc-editor.org/rfc/rfc6455

2. Redis: Pub/Sub

Semântica efêmera de entrega; não oferece histórico para clientes desconectados.

https://redis.io/docs/latest/develop/pubsub/

Continue em: T057 · T060 · T061 · T099

C018 / M. Os 50 cenários enviados

Processamento de fechamento da fatura do cartão

Fechamento de fatura consolida obrigações de um ciclo segundo regras versionadas de datas, lançamentos e ajustes. Fechar, gerar documento e vencer são momentos distintos.

Quando usar. Quando compras, pagamentos, taxas e correções precisam formar uma fatura reproduzível e comunicada dentro do prazo.

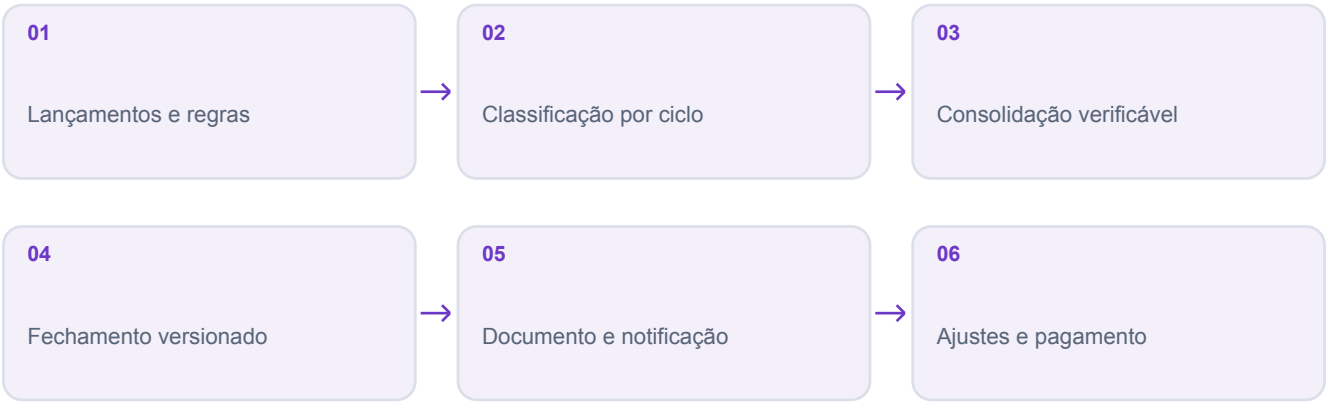
Quando evitar ou limitar. Evitar somar uma projeção atrasada e chamá-la de definitiva, fechar por relógios locais inconsistentes ou tratar geração do PDF como autoridade financeira.

Uma abertura possível

Eu separaria o ciclo financeiro do documento. Primeiro definiria quais lançamentos pertencem ao período e como lidar com atrasados; depois criaria um fechamento idempotente e verificável, deixando PDF e comunicação fora da transação crítica.

Peças e tecnologias. Ledger · Projeção por ciclo · PostgreSQL · Scheduler durável · Object storage · Outbox e reconciliação

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. 0-8 min: distinguir fechamento e vencimento, regras de corte, moedas e lançamentos elegíveis.

2. 8-18 min: modelar ciclo, lançamentos, versão de cálculo e identidade do fechamento.

3. 18-32 min: desenhar acumulação ou batch, barreira de completude e transação de fechamento.

4. 32-48 min: simular evento atrasado, job duplicado, correção e falha na geração do PDF.

5. 48-60 min: comparar custo, reconciliação, rollout de regras e experiência do cliente.

Cuidado técnico. Fechamento e vencimento são momentos distintos.

Cuidado técnico. Streaming não torna um sistema automaticamente mais sênior que batch.

30 perguntas para sustentar o desenho

01. Fechamento ocorre no dia do vencimento?

Não necessariamente. São eventos diferentes, com intervalo e regras do produto. Confirmo calendário, fuso e prazo de comunicação. Confundir os dois muda quais compras entram no ciclo e quando o cliente precisa pagar.

Termo para lembrar: Domínio

02. O PDF é a fatura financeira?

É uma representação de uma versão do estado financeiro, não a autoridade dos lançamentos. Registro o fechamento e seus dados verificáveis antes de gerar o documento. Um problema de renderização não deve recalcular ou duplicar a obrigação.

Termo para lembrar: Autoridade

03. Como decide a qual ciclo pertence uma compra?

Uso regras explícitas sobre datas e etapas relevantes, como processamento ou apresentação, conforme contrato. Armazeno os tempos necessários e a versão da regra. Não dependo somente do horário de chegada ao consumidor.

Termo para lembrar: Ciclo

04. Data da compra e data de processamento são iguais?

Não. Mensagens podem chegar tarde e operações podem ser apresentadas depois. Distingo tempo efetivo e tempo em que o sistema soube do fato. Essa diferença é central para explicar ajustes e reconstruir o que foi informado.

Termo para lembrar: Event time

05. Por que acumular durante o mês?

Reduz trabalho no momento do fechamento e permite acompanhar valores. Porém a projeção precisa ser idempotente, completa e reconciliável. Pré-calcular não elimina a validação necessária para declarar uma versão final.

Termo para lembrar: Pré-cálculo

06. Um batch no fechamento seria inadequado?

Não por definição. Pode ser simples e suficiente com particionamento, capacidade e tempo de execução medidos. Comparo volume, janela e custo. Streaming adiciona atualização contínua, mas também estado e tratamento de atraso.

Termo para lembrar: Batch

07. Como sabe que a projeção recebeu tudo?

Uso posições ou checkpoints das fontes e uma política de corte, além de reconciliação. Um consumidor sem lag aparente não prova ausência de dados ainda não produzidos. O contrato define o que significa completar o ciclo.

Termo para lembrar: Completude

08. O scheduler disparou duas vezes. Fecha duas faturas?

A identidade inclui conta, ciclo e versão ou operação de fechamento. Uma transação condicional cria um resultado único e recuperável. O scheduler pode repetir; a autoridade financeira protege a transição e seus efeitos.

Termo para lembrar: Idempotência

09. Uma compra chega durante o fechamento. O que acontece?

A regra e a fronteira transacional determinam o ciclo elegível. Uso versão, estado e corte explícitos para evitar inclusão parcial. Não deixo a ordem acidental entre duas threads escolher a obrigação do cliente.

Termo para lembrar: Concorrência

10. Precisa bloquear a conta inteira por minutos?

Não como primeira opção. Preparo cálculos fora de uma transação longa e faço uma transição curta validando versão e precondições. O mecanismo exato depende do banco; o objetivo é evitar lock prolongado sobre operações não relacionadas.

Termo para lembrar: Fechamento

11. Uma apresentação legítima chegou após fechar?

Aplico a política de ajuste ou ciclo seguinte, preservando vínculo e datas. Não altero silenciosamente uma fatura já comunicada. Se uma versão corrigida for necessária, ela deve ser identificável e explicável ao cliente.

Termo para lembrar: Atrasado

12. Como corrige valor cobrado errado?

Registro ajuste ou reversão autorizada e recalculo o que a política permitir, mantendo histórico das versões. Não apago o lançamento original para fazer a soma parecer correta. O documento atualizado precisa corresponder ao estado registrado.

Termo para lembrar: Correção

13. Como modela uma compra parcelada?

Distingo a compra original, o plano e cada obrigação de parcela com identidade e regras de calendário. Evito derivar tudo de texto descritivo. Alterações ou cancelamentos precisam preservar a relação entre parcelas e ajustes.

Termo para lembrar: Parcelamento

14. Usa float para somar a fatura?

Não para valores monetários que exigem precisão decimal definida. Uso unidades inteiras ou decimal com escala e regras de arredondamento explícitas. O ponto onde arredondo pode mudar o total e precisa ser reproduzível.

Termo para lembrar: Precisão

15. Pode somar lançamentos em moedas diferentes?

Somente depois de aplicar a política de conversão, com taxa, momento e versão registrados. Preservo valores originais e resultados. Uma consulta à cotação atual não deve reescrever o valor de uma fatura histórica.

Termo para lembrar: Moedas

16. Pagamento durante fechamento altera o ciclo?

Depende da alocação e regras do produto. Modelo pagamento como fato próprio e sua aplicação a obrigações. Não misturo redução de saldo devedor com remoção de compras, o que destruiria a explicação do extrato.

Termo para lembrar: Pagamento

17. Onde coloca cálculo de encargos?

Num componente ou módulo com regras versionadas, entradas identificáveis e precisão definida. A arquitetura deve permitir auditar e testar cálculo. Fórmulas e obrigações concretas precisam ser validadas pelos responsáveis do produto, não inventadas na entrevista.

Termo para lembrar: Juros

18. Muitas contas fecham no mesmo instante. Como distribui?

Particiono trabalho e espalho execução dentro da janela permitida, mantendo regras de corte iguais. Dimensiono bancos, geração de documentos e notificações separadamente. Adicionar todos os jobs simultaneamente pode produzir uma tempestade desnecessária.

Termo para lembrar: Escala

19. Uma conta tem milhões de lançamentos. E agora?

Evito carregar tudo em memória e uso agregações verificáveis, paginação ou processamento incremental. A unidade de fechamento continua tendo identidade. Dimensiono exceções grandes sem presumir que todas as contas têm tamanho médio.

Termo para lembrar: Hotspot

20. O gerador caiu depois do fechamento. Precisa fechar novamente?

Não. A obrigação de gerar documento fica durável e usa a versão já consolidada. Retento a renderização sem refazer efeitos financeiros. O artefato recebe referência e checksum para relacioná-lo à versão correta.

Termo para lembrar: PDF

21. E-mail enviado significa que a fatura foi entregue?

É uma evidência de uma etapa do canal, não necessariamente leitura humana. A fatura deve estar disponível no canal autenticado conforme o produto. Comunicação tem seus próprios estados e recuperação, ligados ao fechamento.

Termo para lembrar: Notificação

22. O que compara antes e depois de fechar?

Lançamentos elegíveis, agregações, ajustes e total consolidado, com contagens e somas por moeda e categoria. Verifico também obrigações sem documento. Um checksum de arquivo não substitui a comparação financeira.

Termo para lembrar: Reconciliação

23. Reprocessar eventos pode fechar faturas antigas de novo?

Não deve produzir efeitos externos por padrão. Separo reconstrução de projeção e emissão de novas obrigações ou comunicações. Identidades e modos de execução evitam que um backfill envie documentos ou cobre clientes novamente.

Termo para lembrar: Replay

24. Mudou a regra no meio do mês. Qual vale?

A política define vigência e escopo; registro a versão aplicada. Execuções em andamento precisam de compatibilidade ou migração explícita. Reprocessar com código atual sem preservar regra histórica pode produzir resultado diferente sem explicação.

Termo para lembrar: Versionamento

25. Quais métricas importam?

Percentual de ciclos concluídos no prazo, divergências, idade de pendências, falha de documento e latência por etapa. Separo fechamento financeiro de publicação. Contar jobs executados não mede quantas faturas corretas ficaram disponíveis.

Termo para lembrar: Observabilidade

26. Como recupera após restaurar um backup antigo?

Reconstituo estado com evidência durável e reconcilio efeitos externos já ocorridos. Não reenvio automaticamente todos os documentos ou obrigações como se fossem novos. A restauração precisa preservar identidade e reconhecer a lacuna temporal.

Termo para lembrar: DR

27. Que casos de fronteira testaria?

Corte de ciclo, mudança de fuso, evento atrasado, arredondamento, cancelamento, pagamento simultâneo e job repetido. Comparo resultado com invariantes e exemplos aprovados. O teste deve verificar total e explicação, não apenas um PDF gerado.

Termo para lembrar: Teste

28. Como mostra uma fatura em processamento?

Distingo valor parcial, fechamento em andamento e versão final, com expectativas claras. Não apresento uma projeção incompleta como cobrança definitiva. Se houver atraso, mantenho o cliente informado conforme as regras da jornada.

Termo para lembrar: Experiência

29. Preciso de event sourcing completo?

Não. Um modelo relacional com lançamentos duráveis, versões e jobs idempotentes pode atender. Event sourcing pode ajudar reconstrução, mas exige governança e compatibilidade. O requisito é explicabilidade e correção, não um estilo obrigatório.

Termo para lembrar: Alternativa

30. Como fecha a proposta?

Mostro o corte, a fonte dos lançamentos e uma versão financeira reproduzível. PDF e comunicação são derivados recuperáveis. A solução suporta repetição e atraso sem alterar silenciosamente o que o cliente deve ou já recebeu.

Termo para lembrar: Conclusão

10 trade-offs: ganho, custo e decisão

01. Batch versus pré-cálculo

Ganho. Batch reduz estado contínuo e operação

Custo e motivo. Concentra trabalho na janela de fechamento

Como decidir. Pré-cálculo quando volume e experiência justificam

02. Fechamento curto versus cálculo transacional longo

Ganho. Reduz contenção

Custo e motivo. Exige validar versão e completude do cálculo preparado

Como decidir. Preferir transição curta com precondições verificáveis

03. Evento atual versus data efetiva

Ganho. Chegada é simples de processar

Custo e motivo. Pode classificar incorretamente fatos tardios

Como decidir. Preservar dimensões temporais exigidas pelo produto

04. Fatura imutável versus edição direta

Ganho. Mantém explicação histórica

Custo e motivo. Correções precisam de versões e ajustes

Como decidir. Quando informação já foi comunicada

05. Armazenar agregados versus calcular toda leitura

Ganho. Reduz custo recorrente

Custo e motivo. Exige reconciliação e atualização correta

Como decidir. Para consultas frequentes e volumes altos

06. Distribuir jobs versus disparo simultâneo

Ganho. Suaviza carga

Custo e motivo. Exige janela e agendamento controlado

Como decidir. Manter o mesmo corte lógico independentemente da hora do job

07. Documento síncrono versus assíncrono

Ganho. Síncrono entrega artefato imediatamente

Custo e motivo. Acopla fechamento a renderização e storage

Como decidir. Assíncrono com obrigação durável quando prazo permite

08. Retenção de versões versus economia de espaço

Ganho. Permite explicar mudanças

Custo e motivo. Aumenta armazenamento e governança

Como decidir. Guardar evidência necessária com política definida

09. Regra configurável versus código fixo

Ganho. Facilita vigência e evolução

Custo e motivo. Exige validação e controle de mudanças

Como decidir. Versionar decisões relevantes em ambos os casos

10. Conciliação contínua versus no corte

Ganho. Detecta divergência mais cedo

Custo e motivo. Consome capacidade durante o ciclo

Como decidir. Combinar verificações proporcionais ao risco

Duas alternativas ao caminho proposto

1. Batch particionado com ledger relacional

Quando faz sentido. Janela permite consolidar volume com simplicidade

Como implementar. Selecionar lançamentos por corte, calcular de forma determinística e fechar idempotentemente

Limites. Precisa medir capacidade e tratar alterações concorrentes

2. Projeção incremental com barreira de fechamento

Quando faz sentido. Volume e consultas durante o mês justificam estado derivado

Como implementar. Aplicar eventos idempotentes, verificar checkpoints e materializar versão final

Limites. Atraso, replay e completude exigem mecanismos adicionais

Referências e alcance da evidência

1. PostgreSQL: tipos numéricos

Precisão de tipos numéricos e diferenças de representação.

https://www.postgresql.org/docs/current/datatype-numeric.html

2. Apache Flink: tempo e processamento

Distinção entre tempo de evento e processamento; regras financeiras são específicas do domínio.

https://nightlies.apache.org/flink/flink-docs-stable/docs/concepts/time/

3. AWS: transactional outbox

Publicação durável de obrigações após mudanças locais.

https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/transactional-outbox.html

Continue em: T042 · T061 · T062 · T091 · T093

C019 / M. Os 50 cenários enviados

Rastreabilidade distribuída em centenas de microsserviços

Tracing distribuído reconstrói relações causais e tempos de uma execução através de serviços. É uma ferramenta diagnóstica com amostragem e perdas possíveis, não o registro autoritativo da operação financeira.

Quando usar. Quando latência e falhas atravessam várias dependências e a equipe precisa localizar onde o tempo e o erro se originam.

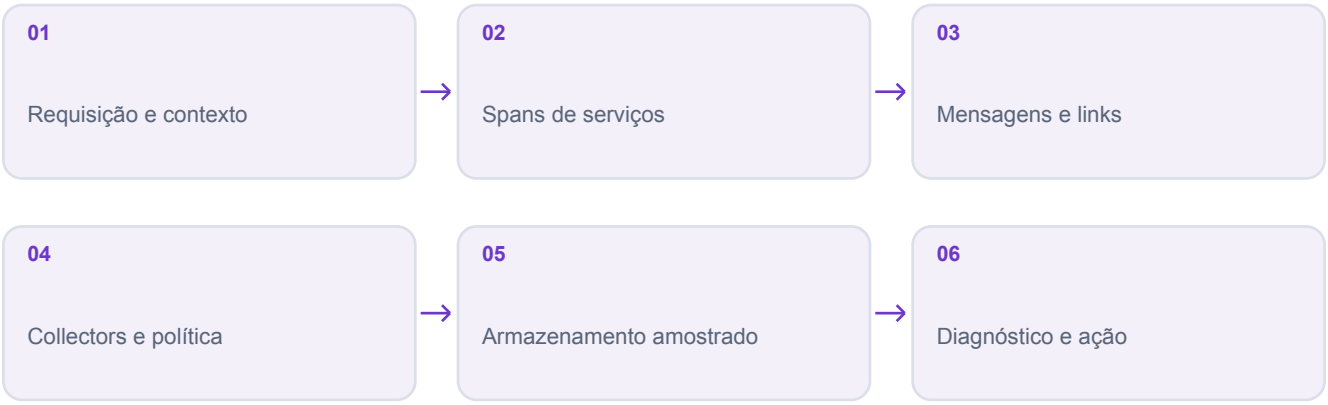
Quando evitar ou limitar. Evitar coletar payloads sensíveis, confundir trace_id com identidade de negócio ou prometer todos os erros quando parte dos traces já foi descartada na origem.

Uma abertura possível

Eu começaria por quais perguntas o time precisa responder num incidente. Depois padronizaria propagação e nomes, protegeria dados e definiria orçamento. Guardar tudo não é a única forma de observar bem, e descartar cedo demais limita o que consigo selecionar depois.

Peças e tecnologias. OpenTelemetry SDK · W3C Trace Context · Collectors · Head/tail sampling · Backend de traces · Logs e métricas correlacionados

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. 0-8 min: definir perguntas de diagnóstico, jornadas críticas, privacidade e orçamento.

2. 8-18 min: desenhar contexto, convenções, identidade de negócio separada e fronteiras de confiança.

3. 18-32 min: instrumentar fluxo síncrono e assíncrono, collectors e política de amostragem.

4. 32-48 min: simular collector indisponível, trace incompleto, backlog e pico de cardinalidade.

5. 48-60 min: definir SLO da telemetria, rollout, ownership e critérios de custo-benefício.

Cuidado técnico. Reter todos os erros não é garantido se houve descarte anterior ou perda de coleta.

Cuidado técnico. Trace não substitui registro financeiro durável.

30 perguntas para sustentar o desenho

01. Que problema o trace resolve melhor que um log isolado?

Mostra relações e duração entre operações de uma execução, ajudando a localizar dependências lentas e falhas propagadas. Logs continuam úteis para eventos específicos. O valor está na correlação e na pergunta diagnóstica, não em substituir todos os sinais.

Termo para lembrar: Finalidade

02. Trace_id pode ser a chave de idempotência?

Não como regra. Uma intenção pode atravessar várias tentativas e traces, e um trace pode conter várias operações. Mantenho identidade de negócio separada e correlacionada. Amostragem ou mudança de contexto não deve alterar proteção financeira.

Termo para lembrar: Identidade

03. Como o contexto passa entre serviços?

Uso formato padronizado, como Trace Context, e instrumentação compatível nos clientes e servidores. Valido entradas e limites. Injetar headers manualmente em alguns pontos não garante continuidade em bibliotecas, filas e chamadas internas.

Termo para lembrar: Propagação

04. O que merece virar um span?

Uma operação útil para entender latência e causalidade, como chamada externa ou etapa relevante. Evito criar spans para cada instrução. Granularidade excessiva aumenta custo e ruído sem melhorar a decisão durante um incidente.

Termo para lembrar: Span

05. Uma mensagem deve manter o mesmo contexto?

Preservo relação causal conforme a semântica de instrumentação, usando contexto e links quando apropriado. Processamento atrasado ou em lote pode não caber numa árvore simples. Não invento uma única duração HTTP para trabalho que acontece horas depois.

Termo para lembrar: Assíncrono

06. Um consumidor processa cem mensagens. Quem é o pai?

Pode haver múltiplas causas; links representam essa relação sem escolher arbitrariamente uma mensagem como origem de todas. Sigo convenções do ecossistema e registro dados úteis. A modelagem deve refletir o trabalho real.

Termo para lembrar: Batch

07. Qual é a limitação de decidir amostra no início?

A decisão pode ser barata e consistente, mas ainda não conhece o resultado final. Um erro posterior pode ocorrer num trace descartado. Não prometo reter todos os erros se a informação foi eliminada antes de chegar ao coletor.

Termo para lembrar: Head sampling

08. Como escolhe traces depois de ver o resultado?

Retenho dados por uma janela e aplico políticas sobre o trace observado, como erro ou duração. Isso consome memória e exige roteamento coerente. Traces tardios ou incompletos limitam a decisão e precisam ser considerados.

Termo para lembrar: Tail sampling

09. Posso garantir cem por cento dos traces com erro?

Só dentro de condições muito explícitas de coleta, capacidade e definição de erro; perdas ainda podem ocorrer. Uma política de seleção não é uma garantia absoluta. Preservo métricas e registros obrigatórios por caminhos apropriados.

Termo para lembrar: Todos os erros

10. O collector deve ficar no caminho crítico da compra?

A exportação diagnóstica normalmente deve ter buffers e limites que evitem bloquear a jornada. Se o coletor cair, aceito perda controlada conforme política. Auditoria obrigatória é um requisito separado, com durabilidade própria.

Termo para lembrar: Collector

11. Por que não guardar telemetria ilimitadamente em memória?

Porque uma falha do destino pode consumir recursos do serviço observado e causar outro incidente. Defino limites, descarte e métricas da perda. O sistema de observabilidade não deve derrubar o produto para preservar todos os spans.

Termo para lembrar: Buffer

12. Por que tail sampling precisa reunir spans relacionados?

A decisão depende de uma visão suficiente do trace. Se fragmentos chegam a coletores independentes sem coordenação, cada um pode decidir com informação incompleta. Planejo distribuição por trace e comportamento durante mudanças de topologia.

Termo para lembrar: Roteamento

13. Qual o problema de colocar user_id em toda métrica?

Cria quantidade potencialmente enorme de séries, elevando custo e instabilidade. Em traces, atributos também têm custo e risco de dados. Escolho dimensões de agregação controladas e correlação específica quando autorizada.

Termo para lembrar: Cardinalidade

14. Pode registrar corpo de requisição para facilitar debug?

Não por padrão. Payloads podem conter segredos e dados financeiros. Prefiro atributos permitidos e sanitizados, com acesso e retenção definidos. Redação precisa acontecer antes de exposição, não apenas num dashboard posterior.

Termo para lembrar: Privacidade

15. Baggage é um bom lugar para CPF e token?

Não. Ele pode propagar por diversas fronteiras e fornecedores. Uso apenas dados apropriados e mínimos, com limites. Contexto de tracing não deve virar canal de divulgação de informações sensíveis ou credenciais.

Termo para lembrar: Baggage

16. Um cliente pode forçar amostragem cara pelo header?

Não deveria controlar irrestritamente recursos internos. Trato contexto externo como entrada não confiável e aplico política na fronteira. Continuidade diagnóstica não exige aceitar qualquer tamanho, valor ou solicitação de coleta do chamador.

Termo para lembrar: Confiança

17. Pode subtrair timestamps de hosts para medir latência precisa?

Com cuidado: desalinhamento pode distorcer relações e durações aparentes. Uso durações locais e interpreto o trace com limitações de relógio. Não concluo causalidade impossível apenas porque dois timestamps distribuídos parecem invertidos.

Termo para lembrar: Relógios

18. Saldo insuficiente é sempre erro de infraestrutura?

Não. Classifico resultado esperado de negócio separadamente de falha técnica. Caso contrário, alertas e amostragem ficam ruidosos. A taxonomia precisa ajudar a equipe a identificar comportamento anormal, não pintar toda recusa de vermelho.

Termo para lembrar: Erro de negócio

19. Por que ainda precisa de métricas se tem traces?

Métricas agregadas ajudam medir população e objetivos sem depender da amostra diagnóstica. Traces aprofundam casos. Inferir taxa global diretamente de uma amostra enviesada por erro ou latência pode produzir conclusões erradas.

Termo para lembrar: Métricas

20. Como correlaciona logs sem duplicar tudo?

Incluo identificadores de contexto e eventos estruturados úteis, evitando copiar a mesma informação extensa em cada camada. Defino severidade e retenção. Um trace aponta para os detalhes necessários, não precisa carregar todos os logs dentro dele.

Termo para lembrar: Logs

21. Como controla custo sem perder o que importa?

Defino orçamento por serviço ou jornada, limites de atributos e políticas de amostra revisadas com incidentes reais. Meço utilidade e volume. Reduzir tudo uniformemente pode eliminar exatamente o caminho raro mais importante.

Termo para lembrar: Custo

22. O volume de erros explodiu. Tail sampling guarda tudo?

Uma política ingênua pode ultrapassar capacidade justamente no incidente. Defino limites e prioridades, preservando métricas de perda e amostras representativas. Coleta diagnóstica precisa de proteção contra tempestade tanto quanto qualquer outro sistema.

Termo para lembrar: Pico

23. Quanto tempo guarda traces?

Pelo tempo útil de investigação, custo e sensibilidade, com regras por classe quando necessário. Não uso traces como arquivo permanente de transações. Evidência financeira ou regulatória pertence a um modelo de retenção e durabilidade específico.

Termo para lembrar: Retenção

24. Como evita nomes de spans incompatíveis entre squads?

Padronizo convenções mínimas, bibliotecas e testes de instrumentação, com evolução versionada. Evito exigir aprovação central para cada detalhe. A plataforma oferece um caminho simples e mede adoção e qualidade.

Termo para lembrar: Versões

25. Como sabe que o observador está falhando?

Meço exportação, filas, descartes, atraso, fragmentação e cobertura das jornadas, por fontes independentes quando possível. Um painel vazio pode significar ausência de tráfego ou falha de coleta; preciso distinguir as duas situações.

Termo para lembrar: SLO da telemetria

26. Qual teste valida propagação assíncrona?

Executo uma jornada que publica, espera, consome em lote e retenta, conferindo relações e identidades. Também testo headers inválidos e ausência de contexto. Ter um trace bonito no caminho síncrono não prova cobertura de mensageria.

Termo para lembrar: Teste

27. O trace amostrado mostra latência no banco. Qual próximo passo?

Correlaciono espera, consultas, pool, locks e métricas do banco para separar causa de sintoma. Um span longo identifica uma fronteira, mas não explica sozinho por que ela está lenta. Evito escalar recursos sem evidência adicional.

Termo para lembrar: Incidente

28. Por que não usar tracing como trilha de chargeback?

Porque traces podem ser amostrados, descartados e retidos por pouco tempo. O processo precisa de estado e eventos duráveis próprios. Correlaciono os dois para investigação sem confundir diagnóstico com prova completa da operação.

Termo para lembrar: Auditoria

29. Quando começaria com menos tracing?

Em sistema simples, métricas e logs estruturados podem resolver as perguntas principais. Instrumento primeiro jornadas e dependências críticas. Uma plataforma complexa de tail sampling antes da necessidade pode consumir mais esforço do que economiza.

Termo para lembrar: Alternativa

30. Como fecha a proposta?

Mostro uma jornada rastreável, sua relação com filas e uma política de coleta que cabe no orçamento. Explico perdas e privacidade. O objetivo é reduzir tempo de diagnóstico sem comprometer o serviço nem fingir uma auditoria completa.

Termo para lembrar: Conclusão

10 trade-offs: ganho, custo e decisão

01. Head versus tail sampling

Ganho. Head decide cedo com menor custo

Custo e motivo. Não conhece falha ou latência final

Como decidir. Tail quando seleção por resultado justifica buffer e complexidade

02. Amostra ampla versus orçamento restrito

Ganho. Mais contexto para casos raros

Custo e motivo. Mais armazenamento, ingestão e exposição

Como decidir. Priorizar jornadas e medir utilidade

03. Spans finos versus operações maiores

Ganho. Detalham etapas internas

Custo e motivo. Aumentam ruído e custo

Como decidir. Criar unidade que ajude decisão diagnóstica

04. Collector local versus gateway central

Ganho. Local reduz dependência imediata

Custo e motivo. Distribui operação e limites

Como decidir. Combinar conforme tráfego e política de processamento

05. Exportação não bloqueante versus durabilidade

Ganho. Protege jornada em falha da telemetria

Custo e motivo. Pode perder spans

Como decidir. Separar diagnóstico de registros obrigatórios

06. Atributos ricos versus minimização

Ganho. Facilitam investigação

Custo e motivo. Elevam cardinalidade e risco de dados

Como decidir. Lista permitida e acesso proporcionais

07. Retenção longa versus recente

Ganho. Permite investigar incidentes antigos

Custo e motivo. Custa e acumula dados

Como decidir. Guardar conforme utilidade e finalidade

08. Convenções comuns versus autonomia total

Ganho. Melhoram consulta transversal

Custo e motivo. Exigem governança e migração

Como decidir. Padronizar o mínimo que gera valor

09. Trace único longo versus links

Ganho. Árvore única parece simples

Custo e motivo. Pode representar mal batch e causalidade múltipla

Como decidir. Usar links quando o trabalho tiver múltiplas origens

10. Seleção por erro versus amostra representativa

Ganho. Preserva casos problemáticos

Custo e motivo. Introduz viés para inferência estatística

Como decidir. Usar métricas para população e traces para diagnóstico

Duas alternativas ao caminho proposto

1. Head sampling com métricas e logs estruturados

Quando faz sentido. Jornadas e orçamento permitem uma estratégia simples

Como implementar. Propagar contexto, amostrar na entrada e correlacionar sinais

Limites. Pode perder traces de erros que surgem depois da decisão

2. Tail sampling em collectors com capacidade planejada

Quando faz sentido. Diagnóstico depende de resultado e cauda

Como implementar. Agrupar spans por trace, aplicar políticas e limitar buffers

Limites. Mais custo operacional e limitações para traces tardios ou incompletos

Referências e alcance da evidência

1. OpenTelemetry: sampling

Diferenças e limites de head e tail sampling.

https://opentelemetry.io/docs/concepts/sampling/

2. W3C: Trace Context

Formato e propagação de contexto entre componentes.

https://www.w3.org/TR/trace-context/

3. OpenTelemetry: messaging spans

Relações e convenções de instrumentação para mensageria.

https://opentelemetry.io/docs/specs/semconv/messaging/messaging-spans/

Continue em: T078 · T079 · T080

C020 / M. Os 50 cenários enviados

Gerenciamento de campanhas de Cashback/Rewards

Rewards mantém direitos de benefício derivados de regras de campanha e eventos elegíveis. Pontos pendentes, disponíveis, resgatados e expirados têm significados distintos e precisam de rastreabilidade.

Quando usar. Quando cashback ou pontos dependem de compras e podem sofrer cancelamento, estorno, limites e expiração.

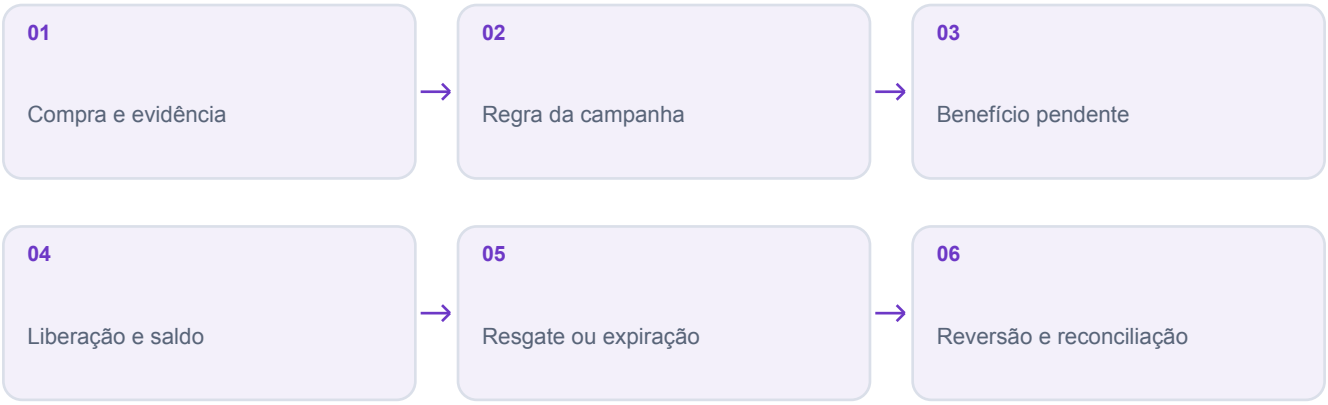
Quando evitar ou limitar. Evitar creditar a cada evento recebido sem identidade, recalcular campanha antiga com regra atual e supor que saldo negativo é sempre permitido.

Uma abertura possível

Eu trataria recompensa como uma obrigação própria, ligada à compra e à versão da campanha. O core publica fatos; rewards decide elegibilidade. Depois eu mostraria o caso difícil: o cliente resgata os pontos e a compra é devolvida. A política desse caso precisa ser explícita.

Peças e tecnologias. Eventos de compra · Outbox/inbox · Motor de regras versionado · Ledger de benefícios · Scheduler de expiração · API de resgate

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. 0-8 min: definir benefício, elegibilidade, liberação, devolução, orçamento e expiração.

2. 8-18 min: modelar campanha versionada, concessão, saldo, resgate e vínculo com compra.

3. 18-32 min: desenhar consumo idempotente, cálculo, liberação e execução de resgate.

4. 32-48 min: simular devolução após uso, concorrência, campanha esgotada e replay.

5. 48-60 min: comparar consistência, custo, antifraude, métricas e reconciliação.

Cuidado técnico. Saldo negativo de pontos é uma política possível, não uma regra universal.

Cuidado técnico. Compra autorizada, capturada e liquidada podem ter elegibilidades diferentes.

30 perguntas para sustentar o desenho

01. Pontos e dinheiro são o mesmo recurso?

Não necessariamente. Defino unidade, valor, regras de uso e obrigações de cada benefício. Cashback em dinheiro pode exigir integração financeira diferente de pontos promocionais. Não misturo os saldos apenas porque ambos são números.

Termo para lembrar: Domínio

02. Qual fato gera a recompensa?

A política pode usar autorização, captura, liquidação ou outro marco. Escolho explicitamente e preservo a referência. Creditar por qualquer mensagem de compra pode antecipar benefício de uma operação que depois não se concretiza.

Termo para lembrar: Evento

03. Por que separar pontos pendentes e disponíveis?

Porque elegibilidade inicial pode depender de confirmação ou janela de devolução. A separação reduz risco de resgatar benefício ainda não consolidado. Também melhora a explicação ao cliente, desde que o prazo de liberação seja claro.

Termo para lembrar: Pendente

04. Como impede crédito duplicado de pontos?

Uso identidade da concessão ligada a compra, campanha e regra aplicável, protegida na transação que registra o benefício. Event_id ajuda, mas envelopes diferentes do mesmo fato não devem criar direitos duplicados.

Termo para lembrar: Identidade

05. Como preserva a regra que o cliente aceitou?

Versiono critérios, vigência e cálculo, vinculando a concessão à versão utilizada. Uma edição posterior não muda silenciosamente o passado. Correções retroativas precisam de escopo e efeitos autorizados, com rastreabilidade.

Termo para lembrar: Campanha

06. Consulta dados atuais do cliente para evento antigo?

Só quando essa for a semântica definida. Caso contrário, preciso dos atributos válidos no momento relevante ou de evidência preservada. Usar categoria atual pode alterar o benefício de uma compra histórica sem justificativa.

Termo para lembrar: Elegibilidade

07. Como evita diferenças de arredondamento?

Defino unidade, escala e momento de arredondar, com exemplos testados. Calcular por item ou pelo total pode gerar resultados diferentes. O motor precisa reproduzir a regra da campanha e explicar o valor concedido.

Termo para lembrar: Cálculo

08. Como impede ultrapassar teto da campanha?

Se o teto é estrito, a concessão precisa reservar ou consumir orçamento com coordenação adequada. Contadores eventualmente consistentes podem excedê-lo. Posso aceitar excesso limitado somente como política explícita, não como surpresa de concorrência.

Termo para lembrar: Orçamento

09. Um contador global vira gargalo. Como resolve?

Avalio particionar orçamento em quotas com reconciliação ou reduzir coordenação onde o produto tolera margem. Isso troca utilização perfeita por paralelismo. Se o teto global deve ser exato, algum ponto de arbitragem continua necessário.

Termo para lembrar: Escala

10. Dois resgates usam os mesmos pontos. O que protege?

Uma transação ou atualização condicional sobre o saldo disponível e a identidade do resgate. Deduplicar apenas o pedido não impede duas intenções distintas concorrentes. A invariante é não consumir mais benefício do que a política permite.

Termo para lembrar: Resgate

11. Precisa reservar pontos antes de chamar parceiro?

Quando a entrega externa é assíncrona, uma reserva evita liberar os mesmos pontos para outro resgate. Defino confirmação, liberação e UNKNOWN. Não devolvo pontos automaticamente se o parceiro pode ter entregue o benefício.

Termo para lembrar: Reserva

12. O parceiro pode ter emitido o voucher apesar do erro?

Sim. Preservo identidade e consulto ou reconcilio antes de criar outro voucher. A política de retry depende do contrato. Timeout não autoriza devolver pontos e ao mesmo tempo deixar o benefício externo válido sem controle.

Termo para lembrar: Timeout

13. Compra cancelou antes de liberar pontos. O que faz?

Cancelo ou reverto a concessão pendente de forma vinculada e idempotente. Não removo a evidência original. Eventos fora de ordem precisam convergir para o resultado correto sem criar disponibilidade temporária indevida.

Termo para lembrar: Devolução

14. Cliente usou pontos e depois devolveu a compra. Saldo fica negativo?

Depende da política aprovada: débito futuro, bloqueio de novos usos, compensação ou outra ação permitida. Modelo a obrigação explicitamente. Saldo negativo não é uma consequência universal nem algo que engenharia decide sozinha.

Termo para lembrar: Após resgate

15. Devolveu parte da compra. Como ajusta?

Aplico a regra proporcional ou específica da campanha, preservando precisão e histórico de ajustes acumulados. O total revertido não pode exceder o benefício correspondente sem uma justificativa de negócio. Identifico cada evento de devolução.

Termo para lembrar: Estorno parcial

16. Cancelamento chegou antes da compra. Descarta?

Não. Guardo evidência correlacionável ou uso uma visão autoritativa para resolver dependência. Descartar por falta de entidade pode deixar pontos indevidos quando a compra chegar. A estratégia precisa lidar com lacunas e retenção.

Termo para lembrar: Ordem

17. Como expira pontos sem disputar com resgate?

Uso lotes ou buckets de validade e uma transição concorrente segura. Expiração e consumo seguem regras de prioridade e tempo definidas. Um job repetido não pode remover novamente pontos já expirados ou consumidos.

Termo para lembrar: Expiração

18. Quais pontos são usados primeiro?

A política pode priorizar os que vencem antes, exigindo controle por concessão ou bucket. Um saldo agregado sozinho pode não explicar validade. Comparo precisão da alocação com custo de armazenamento e atualização.

Termo para lembrar: FIFO

19. Por que não colocar marketing no autorizador de cartão?

Porque campanhas mudam em ritmo diferente e geralmente não precisam bloquear a autorização. Eventos permitem separar falhas e evolução. Se houver benefício em tempo real obrigatório, desenho uma fronteira e orçamento específicos em vez de acoplar tudo.

Termo para lembrar: Separação

20. Pontos podem demorar para aparecer?

Se o produto permitir e informar, uma projeção eventual pode atender. Resgate, porém, precisa verificar a autoridade do benefício disponível. A tela atrasada não deve autorizar gasto nem causar um crédito duplicado para compensar a demora.

Termo para lembrar: Consistência

21. Reprocessar compras cria pontos novos?

Não por padrão. A identidade da concessão e a versão de regra preservam o resultado. Se o objetivo for recalcular uma campanha, trato como operação controlada com comparação e ajustes, sem disparar automaticamente efeitos externos.

Termo para lembrar: Replay

22. Como reduz compra e devolução para extrair benefício?

Alinho liberação, reversão e limites à política de risco, com sinais e revisão proporcionais. Não basta deduplicar mensagens. O abuso pode usar operações formalmente distintas e explorar o momento do resgate.

Termo para lembrar: Fraude

23. Como explica o saldo de pontos ao cliente?

Mostro concessões, liberações, usos, ajustes e expirações vinculados às suas origens. O saldo é consequência desses movimentos. Um número mutável sem histórico dificulta contestação, suporte e verificação de regras.

Termo para lembrar: Auditoria

24. Mudança de pontos precisa avisar sempre?

Depende da relevância e preferência. Gero intenções idempotentes para eventos escolhidos e evito spam em reprocessamento. O conteúdo deve explicar pendência ou reversão sem prometer dinheiro que ainda não está disponível.

Termo para lembrar: Notificação

25. O que reconcilia nesse domínio?

Compras elegíveis, concessões, orçamento, resgates e entrega externa, além de totais por unidade. Procuro benefícios sem origem e obrigações sem conclusão. Um pipeline sem lag pode ainda aplicar uma regra errada perfeitamente rápido.

Termo para lembrar: Reconciliação

26. Quais sequências de teste são essenciais?

Compra duplicada, cancelamento antecipado, devolução parcial, resgate simultâneo e expiração concorrente. Verifico conservação de benefícios e orçamento. Testes de propriedade ajudam explorar combinações que exemplos isolados não cobrem.

Termo para lembrar: Teste

27. Como mede saúde técnica e do negócio?

Atraso para concessão, divergências, resgates ambíguos, custo por campanha e exceções de orçamento. Separo falha técnica de inelegibilidade legítima. Conversão alta pode esconder concessão indevida se correção não for medida.

Termo para lembrar: Métrica

28. Pode trocar o motor de regras em produção?

Sim, com replay controlado ou shadow sem efeitos, comparação de resultados e versões explícitas. Migro campanhas e concessões existentes conforme política. Dois motores em paralelo não devem ambos conceder o mesmo benefício.

Termo para lembrar: Migração

29. Preciso de ledger dedicado para uma campanha pequena?

Não necessariamente. Tabelas relacionais de concessões e movimentos com transações podem atender. Um motor dedicado faz sentido com complexidade e volume. O importante é preservar identidade, saldo e explicação, não adotar um produto por prestígio.

Termo para lembrar: Alternativa

30. Como fecha a arquitetura?

Mostro a compra originando um benefício versionado, seu caminho até disponibilidade e resgate, e a reversão após devolução. As regras decidem quem deve o quê; os mecanismos garantem que retries e concorrência não alterem essa decisão.

Termo para lembrar: Conclusão

10 trade-offs: ganho, custo e decisão

01. Crédito imediato versus pendente

Ganho. Imediato melhora percepção de recompensa

Custo e motivo. Aumenta risco de uso antes de reversão

Como decidir. Escolher pela política de elegibilidade e fraude

02. Teto exato versus excesso limitado

Ganho. Coordenação mantém orçamento estrito

Custo e motivo. Pode reduzir paralelismo e disponibilidade

Como decidir. Excesso só com autorização explícita e mensurável

03. Saldo agregado versus movimentos por concessão

Ganho. Agregado simplifica leitura

Custo e motivo. Perde detalhes de origem e validade se isolado

Como decidir. Manter movimentos quando expiração e contestação exigem

04. Motor central versus regra por produto

Ganho. Centraliza governança e reuso

Custo e motivo. Pode virar gargalo de evolução

Como decidir. Separar contrato comum de regras específicas

05. Consumo síncrono versus eventos

Ganho. Síncrono oferece resposta imediata

Custo e motivo. Acopla disponibilidade do core ao marketing

Como decidir. Eventos quando atraso é aceitável

06. Expiração por item versus bucket

Ganho. Item oferece precisão fina

Custo e motivo. Custa mais estado e operações

Como decidir. Buckets quando semântica e escala permitem

07. Regra histórica versus regra atual

Ganho. Histórica reproduz decisão original

Custo e motivo. Exige versões e dados de contexto

Como decidir. Atual só para recalculação deliberada

08. Saldo negativo versus restrição de uso

Ganho. Negativo representa obrigação remanescente

Custo e motivo. Pode confundir ou conflitar com política

Como decidir. Decisão de produto, risco e regras aplicáveis

09. Fornecedor de rewards versus solução própria

Ganho. Acelera recursos e integração

Custo e motivo. Traz custo, dependência e limites de contrato

Como decidir. Comparar diferenciação e controle necessários

10. Revisão manual versus automação total

Ganho. Resolve exceções sem inventar regra

Custo e motivo. Custa e pode atrasar casos

Como decidir. Automatizar decisões bem definidas, escalar ambiguidades

Duas alternativas ao caminho proposto

1. Modelo relacional de concessões e movimentos

Quando faz sentido. Campanhas e volume cabem numa autoridade simples

Como implementar. Transações para crédito, reserva, resgate e ajustes com regras versionadas

Limites. Precisa evoluir índices, retenção e concorrência conforme uso

2. Serviço especializado de benefícios

Quando faz sentido. Múltiplos programas e parceiros justificam plataforma

Como implementar. Integrar eventos, identidade de cliente e reconciliação por contratos claros

Limites. Não terceiriza responsabilidade por elegibilidade, dados e efeitos financeiros

Referências e alcance da evidência

1. PostgreSQL: isolamento transacional

Mecanismos e limites de isolamento para saldo e orçamento.

https://www.postgresql.org/docs/current/transaction-iso.html

2. AWS: transactional outbox

Conexão durável entre mudanças locais e eventos.

https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/transactional-outbox.html

3. TigerBeetle: contabilidade financeira

Princípios de movimentos e contas; regras específicas de rewards são propostas do exercício.

https://docs.tigerbeetle.com/coding/financial-accounting/

Continue em: T061 · T062 · T063 · T091 · T092 · T093

C021 / M. Os 50 cenários enviados

Criação de um ambiente de homologação fidedigno (Sandbox)

Sandbox é um ambiente isolado que reproduz contratos, estados e falhas relevantes de produção com dados controlados. Fidelidade deve ser definida por risco, não por uma cópia indiscriminada da infraestrutura.

Quando usar. Para validar integrações bancárias, mudanças de contrato, cenários de recusa e recuperação antes de exposição controlada a clientes.

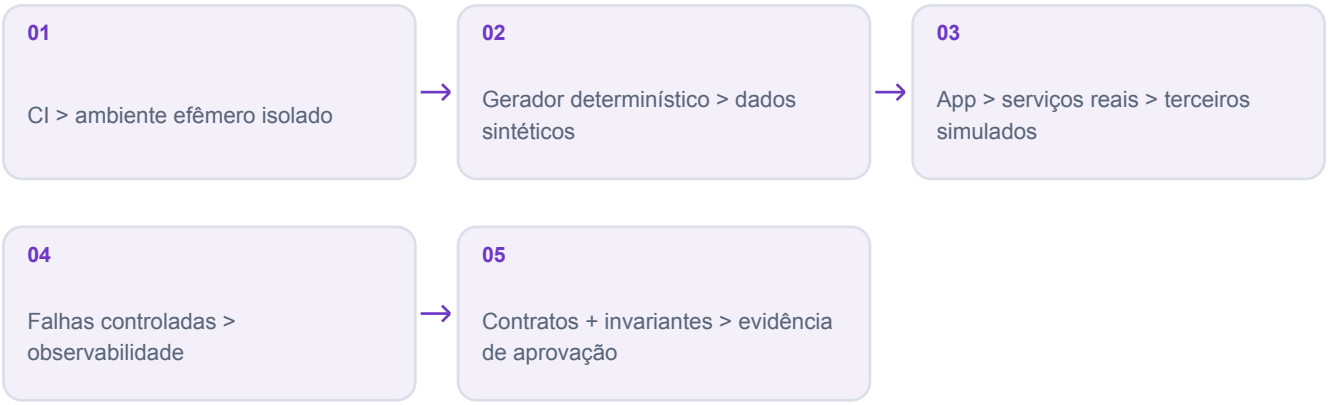
Quando evitar ou limitar. Evitar tratá-lo como prova absoluta de capacidade ou copiar dados reais sem necessidade, autorização e controles. Anonimização não deve ser presumida apenas porque nomes foram removidos.

Uma abertura possível

Eu separaria três fidelidades: contrato, comportamento e capacidade. Criaria dados sintéticos com casos difíceis, terceiros simulados e bloqueio de saída financeira. Depois verificaria onde esse ambiente ainda diverge de produção.

Peças e tecnologias. Terraform · Kubernetes namespaces ou contas AWS isoladas · PostgreSQL · Pact · WireMock · Envoy · Toxiproxy

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Mapear fluxos críticos, invariantes e diferenças permitidas para produção.

2. Provisionar infraestrutura versionada, identidade e saída de rede isoladas.

3. Gerar dados sintéticos com relações, distribuições e casos extremos.

4. Validar contratos e injetar falhas com critérios de interrupção.

5. Registrar resultados, limitações e destruir recursos conforme política.

30 perguntas para sustentar o desenho

01. O que precisa ser igual a produção?

Eu manteria contratos, topologia crítica, configurações de consistência e mecanismos de falha compatíveis. Tamanho pode variar, desde que os testes não extrapolem resultados de capacidade sem justificar distribuição, concorrência e volume equivalentes.

Termo para lembrar: Risk-based fidelity | Base técnica: referência 1.

02. Por que não copiar simplesmente o banco real?

Eu perguntaria qual comportamento exige dados reais. Relações e distribuições geralmente podem ser reproduzidas sinteticamente; copiar produção amplia acesso e exposição. Remover nomes não elimina identificabilidade em históricos financeiros raros ou combinados.

Termo para lembrar: Data minimization | Base técnica: referência 1.

03. Como gerar transações sintéticas úteis?

Eu geraria contas e lançamentos com sementes reproduzíveis, mantendo partidas equilibradas e adicionando casos inválidos deliberados. Incluiria moedas, datas de corte, duplicatas e contas quentes para exercitar invariantes e distribuição.

Termo para lembrar: Property-based generation | Base técnica: referência 1.

04. Como testar integração com uma bandeira?

Eu simularia o protocolo contratado, inclusive confirmação tardia, arquivo parcialmente aceito e conexão interrompida. Compararia periodicamente o simulador com homologação do parceiro, porque um stub criado por nós pode preservar nossas próprias suposições erradas.

Termo para lembrar: Service virtualization | Base técnica: referência 1.

05. Mock de sucesso já basta?

Eu começaria também por recusas válidas, payload malformado, timeout depois do efeito e resposta duplicada. Esses caminhos revelam se o sistema distingue falha definitiva de resultado desconhecido e se consegue reconciliar corretamente.

Termo para lembrar: Negative-path testing | Base técnica: referência 1.

06. Como impedir um Pix real no sandbox?

Eu usaria credenciais exclusivas, destinos permitidos e bloqueio de rede para endpoints produtivos. Um cabeçalho de teste não basta: a infraestrutura e o adaptador devem impedir efeitos externos mesmo diante de configuração incorreta.

Termo para lembrar: Egress isolation | Base técnica: referência 1.

07. Qual papel do Pact?

Eu registraria expectativas observáveis dos consumidores e verificaria o provedor contra elas. Isso detecta incompatibilidade de campos e respostas; não demonstra desempenho, disponibilidade ou correção de toda a jornada financeira distribuída.

Termo para lembrar: Consumer-driven contracts | Base técnica: referência 1.

08. Como tratar serviços ainda indisponíveis?

Eu injetaria portas de integração com implementações reais ou simuladas por ambiente. O domínio permanece igual, e a seleção é explícita. Evitaria condicionais de teste espalhados que alterem regras financeiras apenas na homologação.

Termo para lembrar: Dependency injection | Base técnica: referência 1.

09. Vale espelhar tráfego de produção?

Eu consideraria uma amostra sanitizada quando o benefício superar a exposição. A cópia deve ter efeitos bloqueados e recursos isolados; o resultado do shadow não participa da resposta ao cliente real.

Termo para lembrar: Traffic shadowing | Base técnica: referência 1.

10. O shadow pode duplicar mensagens?

Pode, se apontar para brokers, notificações ou parceiros produtivos. Eu isolaria identidades e destinos de saída, incluindo webhooks, e verificaria essa barreira automaticamente antes de liberar qualquer espelhamento de tráfego real.

Termo para lembrar: Side-effect containment | Base técnica: referência 1.

11. Como comparar duas implementações?

Eu executaria a mesma entrada contra versões com estado inicial equivalente e compararia campos semanticamente relevantes. Normalizaria timestamps e identificadores não determinísticos; diferenças financeiras não seriam escondidas por tolerâncias genéricas de comparação.

Termo para lembrar: Differential testing | Base técnica: referência 1.

12. Como testar timeout após commit?

Eu interromperia a resposta depois da persistência, repetiria a mesma intenção e verificaria um único resultado financeiro. O teste precisa observar estado e efeitos posteriores, não apenas o código HTTP retornado pela aplicação.

Termo para lembrar: Fault injection | Base técnica: referência 1.

13. Como reproduzir problemas de relógio?

Eu injetaria uma abstração de relógio para regras de prazo e testaria viradas de dia, fuso e horário de verão aplicável. Para leases reais, também avaliaria pausas e relógios divergentes no ambiente distribuído.

Termo para lembrar: Controllable clock | Base técnica: referência 1.

14. Uma infraestrutura menor valida escala?

Ela valida alguns mecanismos, mas não prova comportamento de cache, partições quentes ou compaction em escala real. Eu registraria essa limitação e faria ensaios específicos com cardinalidade e saturação representativas antes da mudança crítica.

Termo para lembrar: Scale fidelity | Base técnica: referência 1.

15. Como evitar ambiente compartilhado instável?

Eu criaria ambientes por mudança quando o custo permitir, com dados e namespaces próprios. Dependências caras podem ser compartilhadas por tenants isolados, desde que quotas e reset não prejudiquem testes de outras equipes.

Termo para lembrar: Ephemeral environments | Base técnica: referência 1.

16. Como limpar dados após cada execução?

Eu usaria snapshots sintéticos versionados ou recriação do schema, esperando o término dos consumidores antes da limpeza. Apagar tabelas enquanto mensagens antigas continuam circulando pode ressuscitar estados e produzir testes intermitentes difíceis de interpretar.

Termo para lembrar: Deterministic reset | Base técnica: referência 1.

17. Como testar o broker realisticamente?

Eu manteria versão e configurações relevantes de retenção, partições e entrega. Injetaria redelivery e rebalanceamento; substituir Kafka por uma lista em memória esconderia justamente concorrência, duplicatas e falhas que precisamos estudar.

Termo para lembrar: Broker parity | Base técnica: referência 1.

18. Qual métrica define sucesso do sandbox?

Eu mediria falhas relevantes detectadas antes do lançamento, tempo para diagnóstico e estabilidade dos testes. Uma taxa de testes verdes isolada é fraca se todos exercitam apenas respostas felizes e pouca variabilidade.

Termo para lembrar: Defect escape rate | Base técnica: referência 1.

19. Como lidar com segredos?

Eu provisionaria identidade própria com privilégios mínimos e credenciais curtas quando possível. Segredos produtivos não entram em fixtures, imagens ou logs; o pipeline precisa validar também essa separação, além do funcionamento da aplicação.

Termo para lembrar: Workload identity | Base técnica: referência 1.

20. Como verificar isolamento de tenants?

Eu geraria dois clientes sintéticos e tentaria acessar objetos cruzados com identificadores conhecidos. O resultado esperado é negação consistente, inclusive em busca, cache e arquivos, sem depender de identificadores difíceis de adivinhar.

Termo para lembrar: Authorization testing | Base técnica: referência 1.

21. Como controlar custos dos ambientes?

Eu aplicaria TTL, quotas e identificação do responsável em cada recurso, com destruição automática auditável. Exceções para investigações precisam de prazo; ambientes esquecidos acumulam custo e superfícies de acesso desnecessárias ao projeto.

Termo para lembrar: Resource lifecycle | Base técnica: referência 1.

22. Como evitar divergência de configuração?

Eu derivaria produção e sandbox dos mesmos módulos declarativos, parametrizando diferenças documentadas. Compararia configurações efetivas, porque código de infraestrutura igual não impede alterações manuais ou defaults distintos dos serviços gerenciados utilizados.

Termo para lembrar: Configuration drift | Base técnica: referência 1.

23. O que fazer com dados difíceis de sintetizar?

Eu identificaria a propriedade necessária, como correlação entre renda e atraso, e criaria fixtures que a preservem sem reproduzir pessoas. Se dados reais forem indispensáveis, envolveria governança e controles específicos antes da utilização.

Termo para lembrar: Representative fixtures | Base técnica: referência 1.

24. Como testar indisponibilidade regional?

Eu bloquearia dependências de uma região dentro do experimento e verificaria recuperação com objetivos definidos. Desligar um pod não representa perda regional se banco, DNS e credenciais continuam acessíveis no mesmo domínio de falha.

Termo para lembrar: Failure-domain testing | Base técnica: referência 1.

25. Como impedir testes instáveis?

Eu substituiria esperas fixas por condições observáveis com timeout, controlaria sementes e isolaria recursos concorrentes. Quando houver intermitência, guardaria traces e eventos para descobrir a causa em vez de repetir até passar.

Termo para lembrar: Deterministic orchestration | Base técnica: referência 1.

26. Qual a diferença entre homologação e canário?

Homologação oferece controle sobre entradas e falhas, enquanto o canário mede comportamento real com exposição limitada. Eu combinaria os dois, com critérios de rollback; nenhum elimina sozinho o risco das interações desconhecidas de produção.

Termo para lembrar: Progressive delivery | Base técnica: referência 1.

27. Como lidar com contratos externos mudando?

Eu versionaria exemplos oficiais e executaria verificações periódicas no ambiente homologado do parceiro. Divergências geram revisão do simulador e do adaptador, sem assumir que documentação ou mocks permanecem corretos indefinidamente após sua criação.

Termo para lembrar: Contract drift | Base técnica: referência 1.

28. Como testar reconciliação?

Eu introduziria propositalmente eventos faltantes, duplicados e confirmações tardias, depois verificaria detecção e reparação rastreável. O resultado precisa preservar os lançamentos originais e produzir ajustes identificáveis, sem simplesmente sobrescrever divergências silenciosamente.

Termo para lembrar: Reconciliation testing | Base técnica: referência 1.

29. Quem aprova experimentos de caos?

Eu definiria dono, hipótese, escopo, métricas de parada e recuperação antes da execução. Em ambiente compartilhado, coordenaria a janela; automação não significa liberdade para afetar outras equipes sem delimitar o impacto previamente.

Termo para lembrar: Experiment guardrails | Base técnica: referência 1.

30. O que demonstra maturidade nesta proposta?

Eu explicaria quais riscos o sandbox cobre e quais permanecem abertos, com evidências reproduzíveis. A decisão madura não é ter mais ferramentas, mas ligar cada teste a uma hipótese relevante e um limite conhecido.

Termo para lembrar: Evidence-based validation | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Dados sintéticos

Ganho. Reduzem exposição e facilitam repetição

Custo e motivo. Podem perder correlações reais

Como decidir. Quando propriedades relevantes podem ser modeladas e verificadas

02. Ambientes efêmeros

Ganho. Isolam mudanças e permitem reset

Custo e motivo. Aumentam provisionamento e custo

Como decidir. Quando interferência entre equipes prejudica diagnóstico

03. Stub estrito

Ganho. Falhas reproduzíveis e execução rápida

Custo e motivo. Pode divergir do parceiro

Como decidir. Quando combinado com verificação periódica de contrato

04. Terceiro real em homologação

Ganho. Valida protocolo efetivamente oferecido

Custo e motivo. Quotas e disponibilidade externas

Como decidir. Para testes selecionados de compatibilidade

05. Shadow traffic

Ganho. Reproduz distribuição observada

Custo e motivo. Exige sanitização e contenção de efeitos

Como decidir. Quando há lacunas relevantes nos testes sintéticos

06. Infraestrutura equivalente

Ganho. Revela mecanismos semelhantes

Custo e motivo. Custa mais que uma simulação local

Como decidir. Para falhas dependentes de topologia e configuração

07. Escala reduzida

Ganho. Acelera feedback e reduz custo

Custo e motivo. Não demonstra capacidade produtiva

Como decidir. Para validação funcional com limites documentados

08. Injeção de falhas

Ganho. Exercita recuperação deliberadamente

Custo e motivo. Pode desestabilizar ambiente compartilhado

Como decidir. Com hipótese e critérios de parada

09. Contratos automatizados

Ganho. Detectam incompatibilidade cedo

Custo e motivo. Não validam toda semântica do domínio

Como decidir. Em integrações com consumidores independentes

10. TTL de ambientes

Ganho. Evita recursos esquecidos

Custo e motivo. Pode interromper investigações longas

Como decidir. Com extensão explícita e responsável definido

Duas alternativas ao caminho proposto

1. Testes de componente com containers

Quando faz sentido. Para mudanças locais de feedback rápido

Como implementar. Executar dependências reais mínimas e contratos em CI

Limites. Não reproduz a jornada distribuída ou capacidade completa

2. Canário controlado em produção

Quando faz sentido. Para validar efeitos impossíveis de reproduzir integralmente

Como implementar. Expor coorte pequena com métricas, limites e rollback

Limites. Exige barreiras operacionais e não autoriza experiências financeiras irreversíveis

Referências e alcance da evidência

1. AWS: Test resiliency using chaos engineering

Experimentos controlados e hipóteses de resiliência; a arquitetura de sandbox apresentada é uma proposta autoral.

https://docs.aws.amazon.com/wellarchitected/latest/reliability-pillar/rel_testing_resiliency_failure_injection_resiliency.html

2. Pact: Introduction

Testes de contrato entre consumidores e provedores; não substituem carga, segurança ou comportamento completo de produção.

https://docs.pact.io/

3. Envoy: Route mirroring policies

Mecanismo de espelhamento de requisições; sanitização e bloqueio de efeitos externos são responsabilidades adicionais do desenho.

https://www.envoyproxy.io/docs/envoy/latest/start/sandboxes/route-mirror

Continue em: T008 · T010 · T013

C022 / M. Os 50 cenários enviados

Mudança de esquema de banco de dados sem indisponibilidade

Migração de schema sem interrupção planejada combina compatibilidade entre versões, operações de banco avaliadas por lock e custo, backfill controlado e retirada gradual do formato antigo.

Quando usar. Quando aplicações antigas e novas precisam coexistir e a base não pode parar durante alterações estruturais relevantes.

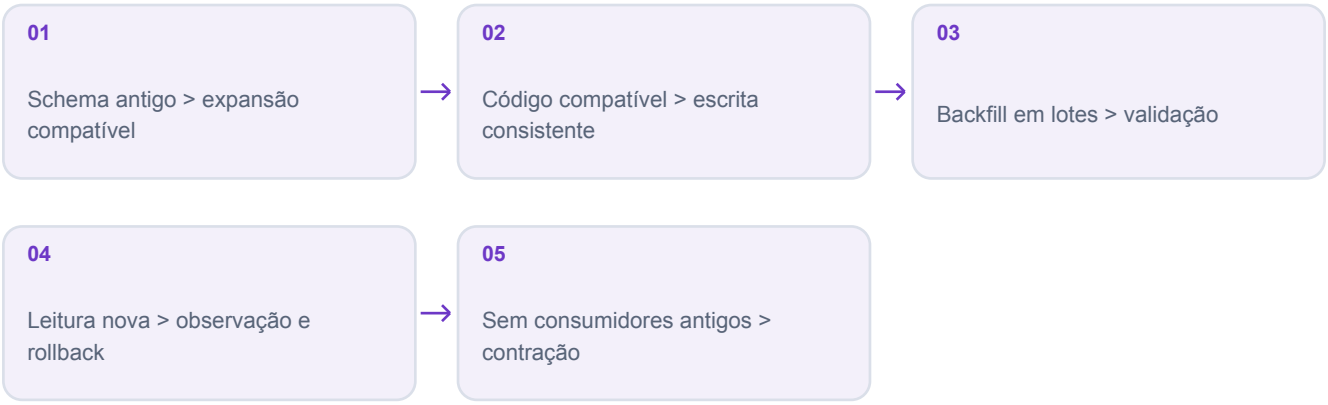
Quando evitar ou limitar. Evitar afirmar que todo ALTER TABLE reescreve a tabela ou que expand-contract garante zero impacto. Operação, versão, transações existentes e volume determinam os riscos.

Uma abertura possível

Eu começaria pelo contrato entre versões e pelo comportamento exato do DDL. Expandiria de forma compatível, migraria os dados em lotes medidos, trocaria leitores e só removeria o legado depois de comprovar que ninguém depende dele.

Peças e tecnologias. PostgreSQL · Flyway ou Liquibase · pg_stat_activity · CREATE INDEX CONCURRENTLY · Feature flags · CDC quando necessário

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Inventariar leitores, escritores, versões e comportamento de locks do DDL.

2. Adicionar estrutura compatível e publicar código que mantenha a regra de consistência.

3. Executar backfill retomável com limites de IO e comparação dos resultados.

4. Migrar leituras gradualmente observando métricas e divergências.

5. Remover formato antigo após janela de compatibilidade e verificação de dependências.

30 perguntas para sustentar o desenho

01. Todo ALTER TABLE causa downtime?

Não. Eu avaliaria a operação e a versão do banco: algumas alterações são rápidas em catálogo, outras varrem ou reescrevem dados. Mesmo um lock breve pode esperar atrás de transações longas e bloquear trabalho posterior.

Termo para lembrar: DDL lock behavior | Base técnica: referência 1.

02. Qual primeira pergunta sobre a mudança?

Eu perguntaria quais versões de aplicação coexistirão e quem escreve diretamente na tabela. Definiria uma matriz de leitura e escrita antes do SQL, porque um DDL válido ainda pode quebrar clientes antigos imediatamente.

Termo para lembrar: Compatibility matrix | Base técnica: referência 1.

03. Como renomear uma coluna usada pelo app?

Eu adicionaria a representação nova e manteria compatibilidade durante a transição. Migraria escritores, dados e leitores em etapas observáveis; só excluiria a coluna antiga quando versões e integrações dependentes realmente deixassem de utilizá-la.

Termo para lembrar: Expand and contract | Base técnica: referência 1.

04. Escrever em duas colunas é dual-write inseguro?

Se ambas pertencem à mesma transação do banco, posso atualizá-las atomicamente. O problema reaparece com bancos diferentes ou escritores esquecidos; eu definiria autoridade, transformação e verificação para evitar formatos divergentes durante a coexistência.

Termo para lembrar: Local atomicity | Base técnica: referência 1.

05. Como lidar com escritores antigos?

Eu os inventariaria antes do backfill. Enquanto existirem, uma coluna nova pode voltar a divergir; usaria compatibilidade na aplicação, trigger cuidadosamente limitado ou captura de mudanças, conforme o controle que temos sobre esses consumidores.

Termo para lembrar: Write compatibility | Base técnica: referência 1.

06. Como migrar bilhões de linhas?

Eu processaria faixas pequenas por chave estável, com commits curtos, checkpoint e limite adaptativo de carga. Evitaria OFFSET crescente e uma única transação enorme, que ampliam trabalho, retenção de versões e custo de recuperação.

Termo para lembrar: Chunked backfill | Base técnica: referência 1.

07. Como evitar sobrescrever uma alteração recente?

Eu atualizaria apenas linhas ainda não migradas ou cuja versão corresponde à lida. Se o usuário mudou o registro, o lote reavalia a transformação; o backfill nunca deve restaurar silenciosamente um valor antigo.

Termo para lembrar: Conditional backfill | Base técnica: referência 1.

08. Como retomar após uma falha?

Eu persistiria o progresso somente depois do commit correspondente e tornaria cada lote repetível. O checkpoint reduz trabalho, mas a condição na atualização garante correção se a resposta do commit for perdida.

Termo para lembrar: Resumable migration | Base técnica: referência 1.

09. Qual limite impõe ao backfill?

Eu observaria latência de produção, lag de réplica, WAL e pressão de IO, reduzindo ou pausando o trabalho ao ultrapassar limites. Quantidade fixa de linhas por segundo pode ser inadequada quando o tráfego varia.

Termo para lembrar: Adaptive throttling | Base técnica: referência 1.

10. Como adicionar um índice sem bloquear escrita continuamente?

No PostgreSQL, eu avaliaria CREATE INDEX CONCURRENTLY, que permite escrita durante boa parte do processo. Ainda há esperas, carga e restrições; acompanharia validade final e trataria índices inválidos deixados por falhas.

Termo para lembrar: Concurrent index build | Base técnica: referência 1.

11. Como adicionar uma foreign key numa tabela grande?

Eu avaliaria criar a constraint NOT VALID e depois validá-la separadamente, conforme suporte da operação. Isso distingue proteção de novas escritas da verificação histórica, mas não elimina todos os locks ou custos da validação.

Termo para lembrar: Deferred constraint validation | Base técnica: referência 1.

12. Como tornar coluna NOT NULL?

Eu primeiro impediria novos nulos, corrigiria os históricos e verificaria uma constraint compatível com a versão utilizada. Só então aplicaria a restrição final, avaliando se o banco pode evitar uma varredura adicional desnecessária.

Termo para lembrar: Constraint rollout | Base técnica: referência 1.

13. E um valor default na coluna nova?

Eu verificaria a versão e a expressão do default. PostgreSQL pode evitar reescrita em casos suportados, mas expressões voláteis e outras mudanças têm comportamento diferente; não generalizaria uma otimização para qualquer alteração de schema.

Termo para lembrar: Fast default | Base técnica: referência 1.

14. Como trocar tipo de dinheiro?

Eu definiria escala, arredondamento e intervalo aceitos antes da migração, usando representação decimal ou unidades inteiras apropriadas. Valores incompatíveis iriam para análise explícita; truncar silenciosamente centavos seria uma falha financeira, não apenas técnica.

Termo para lembrar: Lossless conversion | Base técnica: referência 1.

15. Como validar dados migrados?

Eu compararia contagens, nulos, agregados e amostras determinísticas por faixa, além da equivalência semântica dos campos transformados. Um checksum só ajuda quando a normalização é definida e divergências podem ser localizadas para correção.

Termo para lembrar: Semantic reconciliation | Base técnica: referência 1.

16. Como trocar a leitura com segurança?

Eu faria leitura comparativa sem efeitos por uma coorte e mediria divergência antes de servir o novo resultado. A flag deve permitir recuo rápido enquanto os dois formatos permanecem completos e compatíveis.

Termo para lembrar: Read cutover | Base técnica: referência 1.

17. Rollback significa executar DROP da coluna nova?

Não necessariamente. Eu reverteria primeiro o comportamento da aplicação, preservando dados úteis para diagnóstico. Se a transformação perdeu informação ou novas escritas já dependem do formato novo, o caminho seguro pode ser corrigir para frente.

Termo para lembrar: Rollback versus roll-forward | Base técnica: referência 1.

18. Quando remover a coluna antiga?

Eu esperaria o fim da janela de compatibilidade, confirmaria ausência de consumidores e manteria evidência de recuperação. Tráfego zero em uma hora não prova desuso de jobs mensais, relatórios ou versões móveis antigas.

Termo para lembrar: Contract phase | Base técnica: referência 1.

19. Como evitar esperar um lock indefinidamente?

Eu configuraria um limite curto e uma política de repetição fora do pico, inspecionando transações bloqueadoras. Cancelar e tentar depois é preferível a manter um DDL aguardando enquanto novas operações se acumulam atrás dele.

Termo para lembrar: Lock timeout | Base técnica: referência 1.

20. Como migrar tabelas particionadas?

Eu avaliaria DDL e índices por partição, compatibilidade do catálogo pai e volume de cada faixa. Aplicar a mesma rotina indiscriminadamente pode prolongar locks ou deixar partições futuras sem a estrutura necessária para novas escritas.

Termo para lembrar: Partition-aware migration | Base técnica: referência 1.

21. CDC resolve toda migração entre bancos?

Ele transporta alterações, mas ainda preciso de snapshot consistente, posição de continuidade e transformação correta. O corte de autoridade exige controlar escritores e atraso; replicação funcionando não torna dois bancos automaticamente equivalentes ou atomicamente atualizados.

Termo para lembrar: Change data capture | Base técnica: referência 1.

22. Como lidar com mudança de chave primária?

Eu manteria um mapeamento estável entre identidades antigas e novas, atualizando referências e eventos de forma compatível. Reutilizar identificadores ou mudar sua semântica durante o processo pode quebrar deduplicação, links e auditoria histórica.

Termo para lembrar: Identity migration | Base técnica: referência 1.

23. Qual efeito do backfill nas réplicas?

Ele gera WAL e consumo de IO que podem atrasar réplicas e aumentar dados retidos. Eu acompanharia aplicação do log e capacidade de disco, suspendendo a migração antes de comprometer leituras ou recuperação.

Termo para lembrar: Replication lag | Base técnica: referência 1.

24. Por que transações curtas importam?

Transações longas mantêm snapshots e podem atrasar limpeza de versões antigas, além de prolongar conflitos e rollback. Eu limitaria cada lote para reduzir retenção, tempo de bloqueio e impacto quando um processamento falhar.

Termo para lembrar: MVCC retention | Base técnica: referência 1.

25. Como evitar duas migrações concorrentes conflitantes?

Eu serializaria mudanças incompatíveis no pipeline e registraria versão e estado da execução. Um lock de coordenação não substitui análise: duas alterações independentes no papel ainda podem disputar IO e locks na mesma tabela.

Termo para lembrar: Migration coordination | Base técnica: referência 1.

26. Como versionar eventos durante a mudança?

Eu preservaria campos consumidos e adicionaria novos de forma compatível, com transformação documentada. Alterar o banco não autoriza mudar o significado de um evento publicado; consumidores atrasados e replays também precisam continuar interpretando corretamente.

Termo para lembrar: Schema evolution | Base técnica: referência 1.

27. Como testar antes da produção?

Eu ensaiaria com volume e distribuição representativos, incluindo escritor antigo concorrente, cancelamento e retomada. Mediria locks e WAL, não apenas duração total, e documentaria diferenças de hardware que limitam a extrapolação do resultado.

Termo para lembrar: Migration rehearsal | Base técnica: referência 1.

28. Qual SLO observar durante o rollout?

Eu observaria latência e erro das operações afetadas, tempo de espera por lock e disponibilidade dos leitores. Um backfill rápido que consome o orçamento de erro não é uma migração bem-sucedida para o usuário.

Termo para lembrar: Migration guardrails | Base técnica: referência 1.

29. Como tratar transformação não reversível?

Eu preservaria a origem ou uma representação suficiente para auditoria e reconstrução durante a janela necessária. Antes de apagar informação, validaria impacto funcional e retenção; rollback de código não recupera dados descartados pela transformação.

Termo para lembrar: Information preservation | Base técnica: referência 1.

30. Qual discurso final de maturidade?

Eu apresentaria etapas, critérios de avanço e condições de parada, deixando claro que indisponibilidade zero é um objetivo operacional. A garantia depende dos mecanismos do banco, da compatibilidade dos clientes e das evidências medidas durante a execução.

Termo para lembrar: Safe schema evolution | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Expand-contract

Ganho. Permite coexistência entre versões

Custo e motivo. Mantém complexidade temporária

Como decidir. Quando rollout não pode ser atômico entre todos os clientes

02. Escrita dupla local

Ganho. Atualiza representações na mesma transação

Custo e motivo. Aumenta custo e lógica de escrita

Como decidir. Quando ambas ficam no mesmo banco e possuem transformação definida

03. Trigger temporário

Ganho. Cobre escritores legados

Custo e motivo. Oculta lógica e exige operação cuidadosa

Como decidir. Quando não é possível atualizar todos os escritores imediatamente

04. Backfill pequeno

Ganho. Reduz bloqueio e recuperação

Custo e motivo. Aumenta duração total

Como decidir. Quando produção precisa preservar latência

05. Backfill agressivo

Ganho. Conclui mais rapidamente

Custo e motivo. Amplifica WAL, IO e lag

Como decidir. Quando há margem medida e janela operacional

06. Índice concorrente

Ganho. Reduz bloqueio contínuo de escrita

Custo e motivo. Mais trabalho, esperas e recuperação de falha

Como decidir. Para tabelas ativas onde a operação é suportada

07. Validação separada

Ganho. Divide alteração estrutural e verificação histórica

Custo e motivo. Exige acompanhamento de estado intermediário

Como decidir. Para constraints e versões que suportam essa estratégia

08. Leitura comparativa

Ganho. Detecta divergências antes do corte

Custo e motivo. Duplica parte do trabalho de leitura

Como decidir. Em coortes limitadas com orçamento de capacidade

09. Flag de leitura

Ganho. Permite reversão rápida do comportamento

Custo e motivo. Não desfaz perda de informação

Como decidir. Enquanto os dois formatos continuam consistentes

10. Retenção do legado

Ganho. Preserva recuperação e compatibilidade

Custo e motivo. Consome espaço e manutenção

Como decidir. Até comprovar ausência de dependências e validar a nova representação

Duas alternativas ao caminho proposto

1. Janela de manutenção explícita

Quando faz sentido. Quando o custo de coexistência supera a indisponibilidade aceita

Como implementar. Parar escritores, migrar, validar e reabrir com plano de recuperação

Limites. Precisa de acordo operacional e pode ser inviável para fluxos críticos

2. Banco paralelo com replicação e corte

Quando faz sentido. Quando a mudança exige novo motor ou estrutura radical

Como implementar. Sincronizar snapshot e mudanças, validar e transferir autoridade controladamente

Limites. Aumenta custo e exige resolver continuidade, lag e rollback de escritas

Referências e alcance da evidência

1. PostgreSQL: ALTER TABLE

Níveis de lock, validação de constraints e operações que podem reescrever tabelas; o roteiro de migração é síntese autoral.

https://www.postgresql.org/docs/current/sql-altertable.html

2. PostgreSQL: CREATE INDEX

Criação concorrente de índices e seus limites operacionais; confirmar versão e tipo de índice utilizados.

https://www.postgresql.org/docs/current/sql-createindex.html

Continue em: T013 · T022 · T040

C023 / M. Os 50 cenários enviados

Prevenção contra ataques de Credential Stuffing/Brute Force no Login

Proteção de login combina autenticação forte, limitação de abuso e avaliação contextual para reduzir credenciais reutilizadas e tentativas automatizadas sem transformar a defesa em indisponibilidade para vítimas.

Quando usar. Em login, recuperação de conta e emissão de sessões expostos à internet, especialmente quando bots distribuem tentativas entre milhares de endereços.

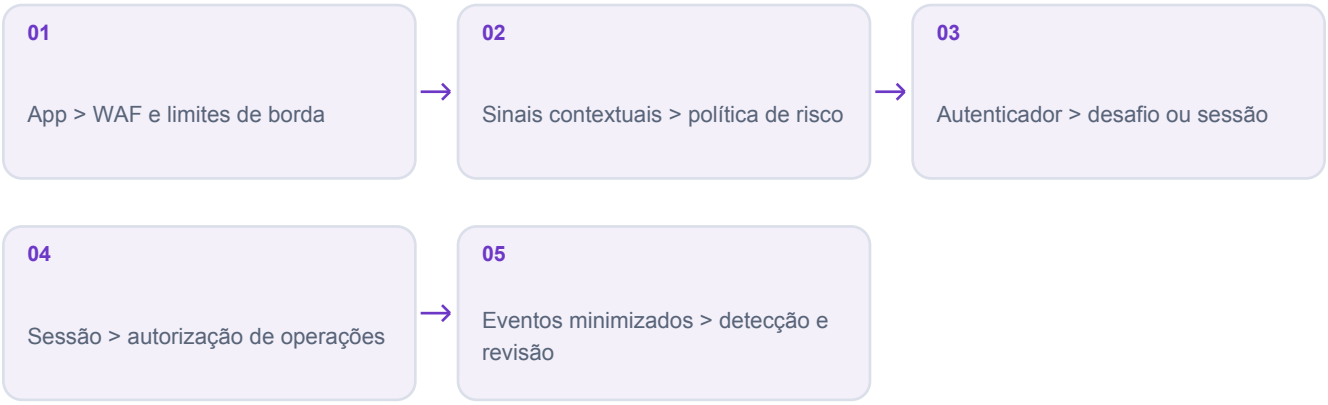
Quando evitar ou limitar. Evitar depender apenas de IP, fingerprint ou bloqueio permanente da conta. Nenhum sinal identifica perfeitamente uma pessoa, e desafios excessivos também excluem usuários legítimos.

Uma abertura possível

Eu trataria login como uma superfície de abuso com dois objetivos: impedir tomada de conta e preservar acesso legítimo. Usaria sinais combinados, autenticação resistente a phishing e decisões graduais, medindo também falsos positivos.

Peças e tecnologias. WAF · Envoy ou API Gateway · Redis para limites rápidos · WebAuthn/passkeys · OIDC · Kafka ou SQS para sinais assíncronos · SIEM

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Definir ameaça, métricas de fraude e limites aceitáveis de atrito.

2. Aplicar controles por origem, conta, dispositivo e campanha com expiração.

3. Implementar autenticação forte e recuperação proporcionalmente protegida.

4. Separar emissão de sessão de autorização para operações sensíveis.

5. Monitorar ataques e falsos positivos, ajustando regras com rollout controlado.

30 perguntas para sustentar o desenho

01. Credential stuffing difere de brute force?

Sim. No stuffing, o atacante testa pares obtidos de outros vazamentos; no brute force, tenta descobrir o segredo. Eu priorizaria autenticação forte e detecção de campanhas, porque uma senha correta não comprova legitimidade do acesso.

Termo para lembrar: Threat classification | Base técnica: referência 1.

02. Por que limitar apenas IP falha?

Uma botnet distribui tentativas entre origens, enquanto usuários legítimos compartilham NAT. Eu combinaria limites por conta, origem e contexto, evitando tanto liberar uma campanha distribuída quanto bloquear uma rede inteira por poucos agentes maliciosos.

Termo para lembrar: Distributed abuse | Base técnica: referência 1.

03. Quais dimensões entrariam no rate limit?

Eu usaria conta normalizada, origem, dispositivo reconhecido e volume global, com janelas e custos distintos. O objetivo é limitar campanhas sem depender de uma única chave facilmente rotacionável ou controlada pelo próprio atacante.

Termo para lembrar: Composite rate limiting | Base técnica: referência 1.

04. Bloquear a conta após cinco erros é seguro?

Isso pode permitir negação de serviço contra uma vítima. Eu aplicaria atraso progressivo e desafios proporcionais, preservando caminhos seguros de recuperação; a decisão de bloquear totalmente exigiria evidência adicional e política explícita de risco.

Termo para lembrar: Account lockout abuse | Base técnica: referência 1.

05. Como funciona o limite distribuído?

Eu faria consumo atômico de tokens por chave em um armazenamento rápido, com TTL e limites de cardinalidade. Para regiões independentes, definiria quanto desvio agregado aceitamos; somar contadores locais não cria um teto global estrito.

Termo para lembrar: Token bucket | Base técnica: referência 1.

06. E se o Redis de limites cair?

Eu adotaria limites locais conservadores e priorizaria dispositivos previamente confiáveis conforme política. Não liberaria tráfego irrestrito silenciosamente; também evitaria bloquear todo acesso sem avaliar recuperação, capacidade e risco durante a degradação temporária.

Termo para lembrar: Degraded authentication | Base técnica: referência 1.

07. Fingerprint prova identidade?

Não. Ele pode mudar legitimamente e ser imitado. Eu o usaria como sinal complementar, com minimização e retenção definida, nunca como substituto de credencial forte ou autorização para uma operação financeira sensível.

Termo para lembrar: Probabilistic signal | Base técnica: referência 1.

08. Qual benefício de passkeys?

Eu avaliaria passkeys para reduzir dependência de senhas reutilizadas e vincular autenticação ao serviço esperado. Ainda preciso proteger registro, recuperação e sessões; melhorar o autenticador não elimina abuso posterior de uma sessão já comprometida.

Termo para lembrar: Phishing-resistant authentication | Base técnica: referência 1.

09. MFA por SMS resolve sozinho?

Não. Eu avaliaria ameaças de troca de SIM, phishing e recuperação, além de acessibilidade. O fator escolhido deve ser proporcional ao risco, e operações sensíveis podem exigir confirmação contextual além do login inicial.

Termo para lembrar: Factor assurance | Base técnica: referência 1.

10. Quando usar CAPTCHA?

Eu o reservaria para sinais de automação ou situações específicas, medindo abandono e acessibilidade. CAPTCHA não é prova forte de identidade, pode ser terceirizado pelo atacante e não substitui proteção da conta ou do autenticador.

Termo para lembrar: Adaptive challenge | Base técnica: referência 1.

11. Como evitar revelar contas existentes?

Eu manteria respostas externas consistentes para credenciais inválidas e recuperação, sem diferenças triviais de mensagem. Também observaria timing e códigos, porque ocultar apenas o texto não impede inferência por outras características do protocolo.

Termo para lembrar: Enumeration resistance | Base técnica: referência 1.

12. Como armazenar senhas?

Eu usaria algoritmo apropriado de derivação de senha, salt único e parâmetros calibrados, com migração progressiva. Hash rápido genérico não é adequado; o custo de verificação também precisa ser protegido contra exaustão por tráfego malicioso.

Termo para lembrar: Password hashing | Base técnica: referência 1.

13. Por que o login pode derrubar CPU?

Verificar senhas corretamente consome recursos deliberadamente. Eu filtraria abuso antes dessa etapa e limitaria concorrência, preservando capacidade para usuários legítimos; apenas aumentar o custo do hash pode ampliar a vulnerabilidade operacional a sobrecarga.

Termo para lembrar: Expensive-operation admission | Base técnica: referência 1.

14. Como detectar senha vazada sem expô-la?

Eu avaliaria mecanismos que evitem transmitir a senha em texto e respeitem o contrato do provedor, como consulta por prefixos de hash quando apropriada. O resultado informa orientação e política, sem registrar o segredo nos logs.

Termo para lembrar: Credential screening | Base técnica: referência 1.

15. Como proteger recuperação de conta?

Eu trataria recuperação como uma forma alternativa de autenticação, com controles equivalentes ao risco. Caso contrário, um login forte perde valor quando o atacante consegue trocar fatores por um fluxo de suporte fraco ou automatizado.

Termo para lembrar: Recovery assurance | Base técnica: referência 1.

16. Sessão deve valer indefinidamente?

Eu definiria expiração, rotação e revogação conforme sensibilidade e contexto. Mudança de fator ou suspeita relevante pode exigir reautenticação; tokens assinados longos sem mecanismo de revogação deixam exposição mesmo após detectar comprometimento da conta.

Termo para lembrar: Session lifecycle | Base técnica: referência 1.

17. O que acontece após senha correta com risco alto?

Eu evitaria emitir imediatamente uma sessão com todos os privilégios. Solicitaria um fator adicional ou restringiria operações sensíveis, registrando o motivo; senha correta em dispositivo desconhecido pode ser justamente o resultado de credential stuffing.

Termo para lembrar: Step-up authentication | Base técnica: referência 1.

18. Como evitar ataques de baixo volume?

Eu agregaria padrões entre contas, redes e intervalos maiores, além dos limites instantâneos. Ataques lentos podem escapar de cada contador isolado; a detecção precisa combinar sinais sem tratar qualquer semelhança como prova definitiva de fraude.

Termo para lembrar: Campaign correlation | Base técnica: referência 1.

19. O que registrar para investigação?

Eu registraria decisão, motivo, identificador pseudonimizado e correlação da tentativa, evitando senhas, tokens e dados excessivos. Acesso e retenção seriam controlados, porque logs de segurança também podem virar uma fonte importante de exposição.

Termo para lembrar: Security telemetry | Base técnica: referência 1.

20. Como medir falsos positivos?

Eu compararia desafios e bloqueios com recuperação, abandono e confirmações de usuários legítimos por segmento. Taxa de ataques bloqueados sozinha pode parecer ótima enquanto a defesa impede clientes reais de acessar o serviço diariamente.

Termo para lembrar: Detection precision | Base técnica: referência 1.

21. Como notificar o usuário sem gerar spam?

Eu agregaria eventos semelhantes e notificaria alterações relevantes de segurança, com limites por conta. Um atacante não deve conseguir disparar milhares de mensagens; o aviso precisa oferecer ação segura sem incentivar cliques em links suspeitos.

Termo para lembrar: Notification deduplication | Base técnica: referência 1.

22. Como lidar com VPN e viagem?

Eu não trataria mudança geográfica ou uso de VPN como fraude automática. Combinaria histórico, dispositivo e força do autenticador, permitindo verificação adicional; sinais contextuais reduzem incerteza, mas precisam admitir mudanças legítimas de comportamento.

Termo para lembrar: Contextual risk | Base técnica: referência 1.

23. Como proteger limites contra cardinalidade maliciosa?

Eu limitaria criação de chaves, normalizaria entradas e expiraria estado rapidamente, mantendo controles agregados na borda. Se cada username aleatório cria memória ilimitada, o mecanismo de defesa se torna um caminho para derrubar o sistema.

Termo para lembrar: State exhaustion | Base técnica: referência 1.

24. Qual parte fica assíncrona?

Eu enviaria correlação histórica, alertas e análise de campanha para processamento assíncrono. A decisão mínima de autenticação permanece no caminho crítico; um consumidor atrasado não pode ser a única barreira contra acesso claramente abusivo.

Termo para lembrar: Detection pipeline | Base técnica: referência 1.

25. Como liberar uma regra nova?

Eu executaria a regra inicialmente sem bloquear, comparando decisões e impacto por segmento. Depois avançaria em coortes com rollback rápido, porque uma regra aparentemente precisa em dados históricos pode excluir usuários legítimos em produção.

Termo para lembrar: Shadow policy | Base técnica: referência 1.

26. Como enfrentar retries dos clientes?

Eu retornaria orientação de espera e faria o cliente aplicar backoff, sem repetir automaticamente uma senha incorreta. O backend mantém seus próprios limites, pois um agente malicioso não respeitará a política implementada no aplicativo.

Termo para lembrar: Retry discipline | Base técnica: referência 1.

27. Qual relação entre login e autorização financeira?

Entrar na conta não autoriza qualquer transferência. Eu revalidaria permissões, limites e contexto da operação no serviço responsável; uma sessão válida não deve contornar controles financeiros nem ampliar acesso a objetos de outro cliente.

Termo para lembrar: Continuous authorization | Base técnica: referência 1.

28. O que fazer após detectar comprometimento?

Eu revogaria sessões afetadas, protegeria fatores e preservaria evidências, oferecendo recuperação verificada. A resposta deve distinguir suspeita de confirmação e coordenar transações em andamento; apenas trocar a senha pode deixar sessões existentes ainda utilizáveis.

Termo para lembrar: Incident containment | Base técnica: referência 1.

29. Como testar a defesa sem atacar clientes?

Eu usaria contas sintéticas e tráfego controlado para testar distribuição de IPs, enumeração, exaustão e recuperação. Avaliaria também usuários compartilhando NAT, para verificar se a proteção funciona sem transformar padrões normais em bloqueios indevidos.

Termo para lembrar: Abuse simulation | Base técnica: referência 1.

30. O que explica como Staff nesta solução?

Eu mostraria mecanismos, limites e métricas de fraude e acesso legítimo, com política de degradação definida. Não venderia fingerprint ou CAPTCHA como solução mágica; explicaria como cada controle reduz um risco específico e onde ainda falha.

Termo para lembrar: Risk-balanced design | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Limite por IP

Ganho. Filtra abuso simples cedo

Custo e motivo. Afeta NAT e perde botnets

Como decidir. Como camada inicial combinada com outros sinais

02. Limite por conta

Ganho. Detecta tentativas distribuídas contra vítima

Custo e motivo. Pode ser explorado para bloquear acesso

Como decidir. Com desafios graduais e recuperação segura

03. Passkeys

Ganho. Reduzem risco de senha reutilizada e phishing

Custo e motivo. Exigem adoção e recuperação bem desenhada

Como decidir. Quando dispositivos e experiência permitem autenticação apropriada

04. MFA contextual

Ganho. Aumenta proteção em eventos arriscados

Custo e motivo. Adiciona atrito e suporte

Como decidir. Quando risco exige verificação além da credencial inicial

05. CAPTCHA adaptativo

Ganho. Eleva custo de algumas automações

Custo e motivo. Acessibilidade e eficácia limitadas

Como decidir. Como sinal complementar em tráfego suspeito

06. Fingerprint

Ganho. Enriquece contexto

Custo e motivo. É mutável, imitável e envolve privacidade

Como decidir. Com minimização e sem tratá-lo como identidade

07. Contador global

Ganho. Coordena um limite agregado

Custo e motivo. Acrescenta latência e dependência

Como decidir. Quando desvio entre regiões é inaceitável

08. Limites locais

Ganho. Mantêm resposta em falhas centrais

Custo e motivo. Permitem desvio agregado controlado

Como decidir. Como degradação com orçamento explícito

09. Logs detalhados

Ganho. Ajudam investigação e ajuste

Custo e motivo. Aumentam exposição e custo

Como decidir. Com minimização, acesso restrito e retenção definida

10. Bloqueio agressivo

Ganho. Reduz janela de abuso

Custo e motivo. Pode impedir acesso legítimo

Como decidir. Para evidência forte e caminhos seguros de recuperação

Duas alternativas ao caminho proposto

1. Provedor de identidade gerenciado

Quando faz sentido. Quando a equipe prefere delegar protocolos e autenticadores

Como implementar. Integrar OIDC e políticas de risco mantendo autorização no domínio

Limites. Não elimina responsabilidade sobre recuperação, configuração e efeitos financeiros

2. Autenticação prioritariamente sem senha

Quando faz sentido. Quando há estratégia de adoção e suporte a dispositivos

Como implementar. Usar passkeys e fluxos de recuperação protegidos

Limites. Sessões, registro de fatores e usuários sem dispositivos compatíveis continuam exigindo projeto

Referências e alcance da evidência

1. OWASP: Credential Stuffing Prevention

Defesas complementares contra automação e abuso de credenciais; thresholds e topologia propostos são exemplos autorais.

https://cheatsheetseries.owasp.org/cheatsheets/Credential_Stuffing_Prevention_Cheat_Sheet.html

2. OWASP: Authentication

Princípios de autenticação, respostas de erro e proteção de fluxos de acesso; não oferece garantia contra todo ataque.

https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

Continue em: T008 · T019 · C021

C024 / M. Os 50 cenários enviados

Sistema de simulação de empréstimo (Loan Engine)

Um loan engine separa cálculo de simulação, elegibilidade, oferta versionada, contratação e desembolso. Uma resposta rápida de simulação não equivale a aprovação nem a reserva financeira definitiva.

Quando usar. Para comparar prazos e parcelas, apresentar ofertas explicáveis e converter uma escolha em contratação com condições e limites verificáveis.

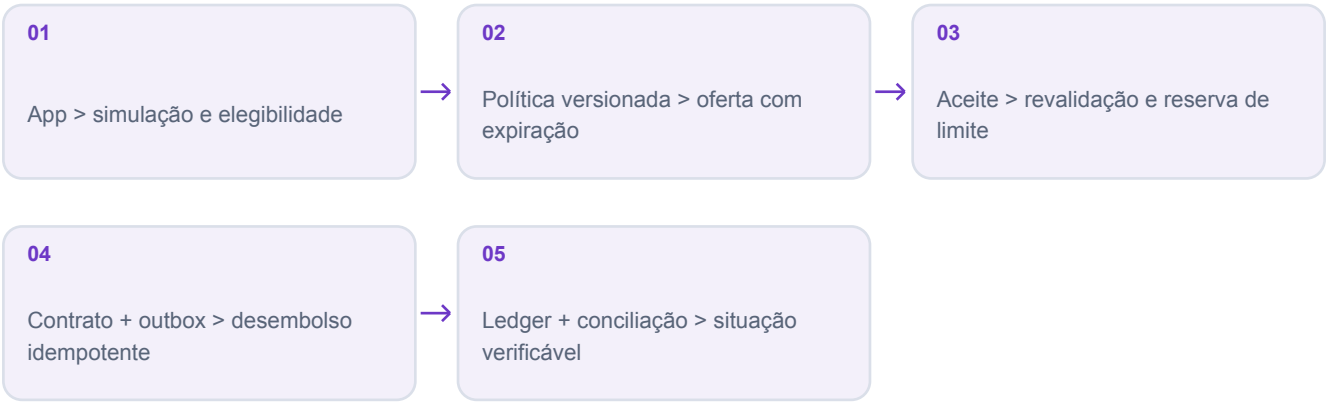
Quando evitar ou limitar. Evitar pré-aprovação sensível em cache público de borda ou assumir que um circuit breaker genérico limita crédito concedido. Limites financeiros exigem contabilização e coordenação próprias.

Uma abertura possível

Eu separaria simular de contratar. O cálculo usa política versionada e valores precisos; a oferta tem validade e condições. Na contratação, revalido elegibilidade e reservo limites atomicamente antes de iniciar o desembolso rastreável.

Peças e tecnologias. PostgreSQL · Decimal ou BigDecimal · DMN ou regras versionadas em código · Redis privado para dados derivados · Outbox · Kafka ou SQS · Feature flags

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Definir produto, fórmula, arredondamento e diferença entre estimativa e oferta.

2. Versionar políticas e calcular simulações sem criar obrigações financeiras.

3. Emitir oferta identificada com termos, validade e dados de decisão.

4. Contratar com idempotência, controle de concorrência e reserva de exposição.

5. Desembolsar e reconciliar por estados duráveis, mantendo rastreabilidade.

30 perguntas para sustentar o desenho

01. Simulação já representa aprovação?

Eu separaria estimativa de parcelas, elegibilidade e oferta vinculada a condições. O app deve mostrar qual resposta recebeu; calcular uma parcela rapidamente não significa que o cliente possui crédito disponível ou que o banco assumiu obrigação.

Termo para lembrar: Decision boundaries | Base técnica: referência 1.

02. Como calcular parcelas sem erros de centavos?

Eu usaria representação decimal com precisão intermediária suficiente e regra explícita de arredondamento. Validaria soma, saldo residual e última parcela; usar ponto flutuante binário sem tratamento pode acumular diferenças difíceis de explicar ao cliente.

Termo para lembrar: Decimal arithmetic | Base técnica: referência 2.

03. Qual fórmula escolher?

Eu confirmaria o sistema de amortização, calendário e encargos do produto antes de implementar. Fórmulas de parcelas constantes e amortização constante produzem cronogramas diferentes; não escolheria uma apenas por ser um exemplo comum de entrevista.

Termo para lembrar: Product-specific amortization | Base técnica: referência 1.

04. O que armazenar numa oferta?

Eu persistiria identificador, principal, prazo, condições, versão da política, validade e referência dos dados usados. Isso permite reproduzir a decisão e verificar o aceite, evitando que uma mudança posterior altere silenciosamente o que foi apresentado.

Termo para lembrar: Versioned offer | Base técnica: referência 1.

05. Como lidar com oferta expirada?

Eu recusaria a contratação daquela oferta ou emitiria uma nova com consentimento explícito, conforme o contrato do produto. Não alteraria taxa ou parcela silenciosamente durante o aceite; o cliente precisa compreender as condições efetivamente assumidas.

Termo para lembrar: Offer validity | Base técnica: referência 1.

06. Vale usar rule engine?

Eu usaria quando regras são numerosas, versionadas e revisadas por diferentes responsáveis. Para políticas simples, código testável pode ser suficiente; uma engine adiciona linguagem, governança e operação, sem tornar automaticamente a decisão correta ou explicável.

Termo para lembrar: Decision modeling | Base técnica: referência 1.

07. Como garantir reprodução de uma decisão antiga?

Eu guardaria versão imutável das regras, entradas relevantes e referência do modelo utilizado, com controles de acesso. Reexecutar a política atual sobre dados atuais não reconstrói uma decisão histórica tomada em outro contexto financeiro.

Termo para lembrar: Decision provenance | Base técnica: referência 1.

08. O risco pode ser calculado assincronamente?

Sim, para pré-avaliações com validade e atraso aceitáveis. Eu carregaria versão e frescor do resultado, revalidando condições críticas na contratação; um score calculado ontem não comprova capacidade atual de assumir uma nova obrigação.

Termo para lembrar: Precomputed eligibility | Base técnica: referência 1.

09. Onde cachear pré-aprovações?

Eu preferiria cache privado com chave por cliente, versão e expiração, protegido por autorização. Uma CDN pública é inadequada para dados sensíveis; a leitura do cache não substitui validação e limites no momento da contratação.

Termo para lembrar: Private derived cache | Base técnica: referência 1.

10. Como impedir contratação duplicada?

Eu associaria uma chave estável à intenção de aceite e imporia unicidade no contrato criado. Repetições retornam o mesmo resultado; payload diferente com a mesma chave é conflito, não uma autorização para criar outro empréstimo.

Termo para lembrar: Idempotent acceptance | Base técnica: referência 4.

11. Duas ofertas podem ser aceitas simultaneamente?

Podem chegar juntas, então eu reservaria a exposição em uma transação ou atualização condicional no serviço responsável pelo limite. Verificar saldo de limite e depois gravar separadamente permite que ambas aprovações consumam a mesma capacidade.

Termo para lembrar: Exposure concurrency | Base técnica: referência 1.

12. Como limitar concessão total por hora?

Eu manteria um orçamento de exposição e reservas com autoridade definida, ou distribuiria cotas conservadoras entre células. Um circuit breaker baseado em erros não soma valores concedidos; o limite financeiro precisa controlar concorrência e liberações.

Termo para lembrar: Financial admission control | Base técnica: referência 1.

13. O limite global vira gargalo?

Pode. Eu avaliaria cotas por segmento ou célula, cuja soma respeite o teto global, com redistribuição controlada. Isso reduz coordenação por contrato, mas pode recusar concessões localmente mesmo quando outra célula ainda possui capacidade.

Termo para lembrar: Escrow allocation | Base técnica: referência 1.

14. E se a taxa publicada estiver errada?

Eu bloquearia novas contratações da versão afetada e identificaria ofertas e contratos expostos para tratamento responsável. Um kill switch limita novas perdas, mas não desfaz obrigações existentes; a resposta exige análise do produto e rastreabilidade.

Termo para lembrar: Policy kill switch | Base técnica: referência 1.

15. Como lançar uma nova política?

Eu compararia resultados da política candidata com a vigente sem conceder crédito adicional, avaliando divergências e segmentos. Depois avançaria por coortes com limites de exposição; apenas medir latência não valida qualidade ou impacto financeiro da decisão.

Termo para lembrar: Shadow decision | Base técnica: referência 1.

16. Como tratar serviço de risco fora do ar?

Eu definiria quais ofertas previamente válidas podem prosseguir e quais ficam pendentes, conforme política aprovada. Não inventaria uma autorização permissiva por disponibilidade; desconhecer risco é diferente de comprovar elegibilidade para assumir obrigação financeira.

Termo para lembrar: Risk-aware degradation | Base técnica: referência 1.

17. Como desenhar a API de simulação?

Eu receberia valor, prazo e contexto autorizado, devolvendo cronograma, premissas e identificador quando houver oferta persistida. Separaria endpoints de simular e contratar, para que retries de uma consulta não criem efeitos financeiros inadvertidamente.

Termo para lembrar: Resource-oriented API | Base técnica: referência 1.

18. Qual estado usar após aceite?

Eu usaria estados explícitos como ACCEPTED, FUNDING_PENDING, FUNDED e FAILED_FINAL, com transições permitidas. O aceite e o dinheiro disponível são momentos diferentes; a interface deve refletir isso sem prometer conclusão antes da confirmação necessária.

Termo para lembrar: Contract state machine | Base técnica: referência 1.

19. Como integrar o desembolso?

Eu gravaria contrato e intenção de desembolso na mesma transação local, publicando depois. O consumidor usa identidade estável e reconciliação; outbox evita perder a intenção, mas não torna o ledger e o contrato uma transação distribuída única.

Termo para lembrar: Transactional outbox | Base técnica: referência 3.

20. Timeout no desembolso significa falha?

Não. Eu consultaria o resultado pela identidade da operação antes de tentar uma nova intenção financeira. O estado permanece pendente enquanto há incerteza; repetir com novo identificador pode desembolsar duas vezes para o mesmo contrato.

Termo para lembrar: Unknown outcome | Base técnica: referência 4.

21. Quando liberar uma reserva de limite?

Eu liberaria após cancelamento ou falha definitiva comprovada, conforme a regra de negócio. Um timeout não basta, porque o desembolso pode ter ocorrido; a reserva precisa de reconciliação antes de devolver capacidade para outras concessões.

Termo para lembrar: Reservation lifecycle | Base técnica: referência 4.

22. Como impedir alteração do valor no cliente?

Eu reconstituiria condições a partir da oferta persistida e validaria autorização e integridade no servidor. Campos enviados pelo app são intenção, não autoridade; assinar uma oferta pode ajudar integridade, mas não substitui expiração e revalidação.

Termo para lembrar: Server-side authority | Base técnica: referência 1.

23. Como dimensionar muitas simulações?

Eu manteria o cálculo puro e escalável horizontalmente, usando políticas versionadas em memória quando apropriado. Escritas e consultas caras de risco ficam controladas; mover toda simulação para o banco desperdiça capacidade transacional sem necessidade.

Termo para lembrar: Compute versus state separation | Base técnica: referência 1.

24. Como evitar abuso de simulações?

Eu limitaria concorrência e taxa por cliente e origem, validando parâmetros antes do cálculo. Prazo ou precisão absurdos podem criar trabalho desnecessário; limites devem preservar exploração legítima de opções sem permitir saturação do serviço.

Termo para lembrar: Workload admission | Base técnica: referência 1.

25. Quais testes matemáticos aplicaria?

Eu verificaria que amortizações somam o principal, saldos seguem o cronograma e arredondamentos obedecem à regra definida. Incluiria juros zero, prazo mínimo, valores extremos e calendário irregular, comparando exemplos aprovados pelo responsável do produto.

Termo para lembrar: Financial invariants | Base técnica: referência 2.

26. Como explicar uma recusa ao cliente?

Eu retornaria motivos adequados ao produto e à política aplicável, preservando controles antifraude e privacidade. Internamente, manteria a trilha das regras acionadas; um resultado opaco dificulta suporte, contestação e investigação de erros sistemáticos.

Termo para lembrar: Decision explainability | Base técnica: referência 1.

27. Como proteger dados usados pelo modelo?

Eu definiria origem, frescor, acesso e retenção de cada atributo, evitando reutilizar dados sem finalidade apropriada. Também monitoraria ausência e distribuição; um campo que vira nulo pode alterar decisões em massa sem derrubar a API.

Termo para lembrar: Feature governance | Base técnica: referência 1.

28. Qual observabilidade importa além de latência?

Eu acompanharia divergência de parcelas, taxa de aprovação por versão, exposição reservada e desembolsos pendentes. Esses sinais podem revelar erro financeiro antes de erros HTTP; a API responder rápido não demonstra que as decisões estão corretas.

Termo para lembrar: Business guardrails | Base técnica: referência 1.

29. Como reconciliar contrato e ledger?

Eu compararia contratos que deveriam ter desembolso com operações efetivamente lançadas, usando identificadores estáveis. Divergências geram investigação e reparação rastreável; não marcaria um contrato como financiado apenas porque uma mensagem foi publicada no broker.

Termo para lembrar: Financial reconciliation | Base técnica: referência 1.

30. O que diferencia esta proposta de complexidade gratuita?

Eu justificaria coordenação apenas onde há obrigação financeira e manteria simulação simples e escalável. A maturidade está em separar estimativa, promessa e efeito, com limites claros de risco, e não em adicionar uma engine para impressionar.

Termo para lembrar: Invariant-driven design | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Cálculo puro

Ganho. Escala sem contenção transacional

Custo e motivo. Depende de entradas e política corretas

Como decidir. Para simulações sem efeito financeiro

02. Oferta persistida

Ganho. Permite reprodução e aceite verificável

Custo e motivo. Adiciona armazenamento e expiração

Como decidir. Quando condições precisam ser identificadas e auditadas

03. Regras em código

Ganho. Facilitam testes e ferramentas existentes

Custo e motivo. Mudanças exigem ciclo de engenharia

Como decidir. Para políticas simples e equipe técnica responsável

04. Rule engine

Ganho. Explicita tabelas e versões de decisão

Custo e motivo. Cria linguagem e governança adicionais

Como decidir. Quando complexidade e colaboração justificam

05. Pré-cálculo de risco

Ganho. Reduz latência percebida

Custo e motivo. Pode ficar desatualizado

Como decidir. Com validade explícita e revalidação crítica

06. Reavaliação síncrona

Ganho. Usa informação recente

Custo e motivo. Acopla contratação à disponibilidade do risco

Como decidir. Para condições que não aceitam atraso

07. Reserva central

Ganho. Impõe teto financeiro compartilhado

Custo e motivo. Pode gerar contenção e latência

Como decidir. Quando um limite global estrito é necessário

08. Cotas por célula

Ganho. Reduzem coordenação entre regiões

Custo e motivo. Podem deixar capacidade ociosa e recusar localmente

Como decidir. Quando cotas somadas respeitam o limite e há redistribuição controlada

09. Desembolso assíncrono

Ganho. Isola falhas e permite recuperação

Custo e motivo. Cliente vê estado pendente

Como decidir. Quando o produto admite confirmação posterior

10. Política em shadow

Ganho. Detecta diferenças sem exposição financeira

Custo e motivo. Não mede integralmente comportamento de clientes

Como decidir. Antes de liberar regras com limite progressivo de concessão

Duas alternativas ao caminho proposto

1. Motor simples de cálculo e políticas em código

Quando faz sentido. Para produto inicial com poucas regras

Como implementar. Implementar funções determinísticas versionadas e contratação transacional

Limites. Pode exigir reorganização quando regras e responsáveis se multiplicarem

2. Motor de decisão especializado com orquestrador de contratação

Quando faz sentido. Para portfólio complexo e governança formal

Como implementar. Separar avaliação versionada, oferta e workflow de efeitos

Limites. Integração, dependência operacional e explicabilidade continuam sob responsabilidade da solução

Referências e alcance da evidência

1. OMG: Decision Model and Notation 1.5

Padrão para modelagem de decisões; políticas de crédito e arquitetura de concessão abaixo são hipóteses didáticas.

https://www.omg.org/spec/DMN/1.5/About-DMN

2. Python: decimal

Representação decimal, contexto de precisão e arredondamento; fórmulas contratuais dependem do produto financeiro definido.

https://docs.python.org/3/library/decimal.html

3. AWS: Transactional outbox

Persistência de intenção de publicação na transação local e tratamento de duplicatas; não há atomicidade automática entre todos os serviços.

https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/transactional-outbox.html

4. Amazon Builders Library: Making retries safe with idempotent APIs

Identidade estável da intenção, resultado incerto após timeout e repetição sem efeitos adicionais; reservas e desembolso deste cenário são desenho autoral.

https://aws.amazon.com/builders-library/making-retries-safe-with-idempotent-APIs/

Continue em: T004 · T013 · T037

C025 / M. Os 50 cenários enviados

Lidando com versionamento de APIs REST consumidas pelo App Mobile

Evoluir uma API móvel exige compatibilidade com clientes já instalados, contratos semânticos estáveis, introdução gradual de capacidades e política explícita de retirada das versões antigas.

Quando usar. Quando o backend muda mais rapidamente que a distribuição e adoção de versões dos aplicativos, ou quando integrações externas têm ciclos independentes.

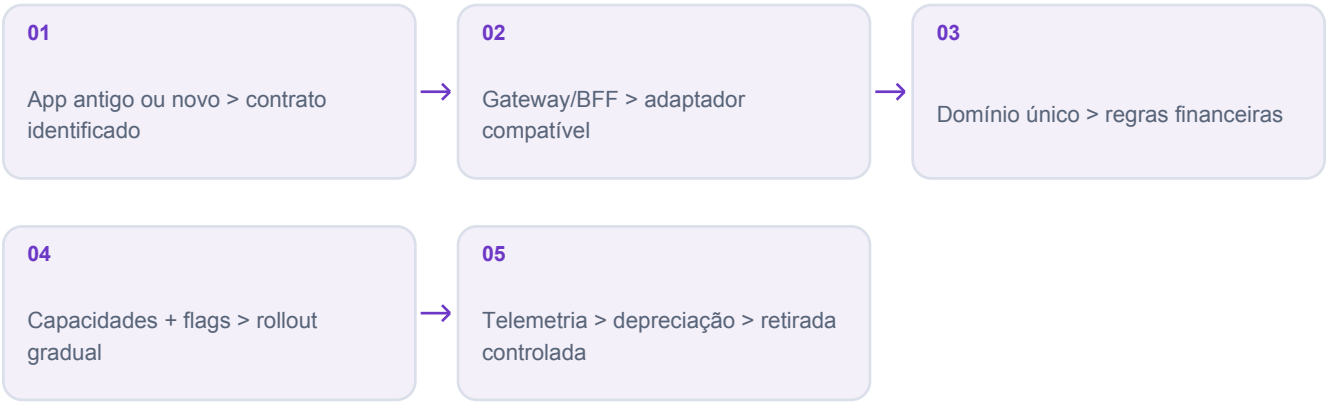
Quando evitar ou limitar. Evitar presumir que GraphQL elimina incompatibilidade, que um header de versão é confiável para segurança ou que todos os clientes podem atualizar imediatamente.

Uma abertura possível

Eu partiria da realidade de clientes antigos ainda ativos. Evoluiria contratos de forma aditiva quando possível, isolaria mudanças incompatíveis e mediria adoção por capacidade antes de desligar qualquer caminho usado pelos clientes.

Peças e tecnologias. OpenAPI · Pact · API Gateway · Feature flags · REST ou GraphQL conforme contrato · Telemetry por versão · BFF quando necessário

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Inventariar versões ativas e definir contratos de request, response e erro.

2. Preferir extensões compatíveis e criar nova versão para semântica incompatível.

3. Testar contratos com clientes suportados e manter autoridade no backend.

4. Liberar capacidades em coortes e medir erro, uso e adoção.

5. Comunicar depreciação e retirar apenas com política e alternativas definidas.

30 perguntas para sustentar o desenho

01. Colocar v1 e v2 na URL resolve?

Ajuda a identificar contratos incompatíveis, mas não define como eles coexistem ou evoluem. Eu ainda preciso mapear consumidores, preservar semântica, testar versões e planejar retirada; apenas duplicar endpoints pode criar duas implementações divergentes de negócio.

Termo para lembrar: Contract versioning | Base técnica: referência 1.

02. O que é mudança realmente compatível?

Eu avaliaria consumidores reais, não apenas JSON válido. Adicionar campo opcional costuma ajudar, mas um cliente com desserializador estrito pode falhar; mudar significado, unidade ou ordenação pode quebrar comportamento sem alterar o schema.

Termo para lembrar: Backward compatibility | Base técnica: referência 1.

03. Adicionar um valor de enum é seguro?

Depende do contrato e do cliente. Eu testaria tratamento de valores desconhecidos e definiria fallback explícito; um switch exaustivo pode falhar ao receber um novo estado mesmo quando nenhum campo foi removido da resposta.

Termo para lembrar: Open versus closed enums | Base técnica: referência 1.

04. Como mudar reais para centavos?

Eu criaria um campo ou contrato novo com unidade explícita, preservando o antigo durante a transição. Reutilizar o mesmo nome com outra escala gera erros financeiros silenciosos, mesmo que tipo e validação sintática continuem corretos.

Termo para lembrar: Semantic compatibility | Base técnica: referência 1.

05. Como evitar manter duas regras financeiras?

Eu colocaria a tradução de contratos na borda e manteria o domínio central compartilhado. O adaptador converte formatos e capacidades, mas não deveria criar uma política financeira paralela que evolui independentemente da regra principal.

Termo para lembrar: Compatibility adapter | Base técnica: referência 1.

06. Qual função do BFF?

Eu usaria um BFF quando a composição e o formato para mobile justificarem uma camada própria. Ele pode reduzir chamadas e adaptar versões, mas adiciona operação e não deve duplicar autoridade sobre saldos ou permissões.

Termo para lembrar: Backend for frontend | Base técnica: referência 1.

07. Feature flag substitui versionamento?

Não. A flag controla exposição de comportamento já suportado por um contrato. Eu combinaria capacidade declarada e validação no servidor, mantendo defaults seguros; ativar uma flag não ensina um aplicativo antigo a interpretar uma estrutura desconhecida.

Termo para lembrar: Capability rollout | Base técnica: referência 1.

08. Como identificar capacidades do cliente?

Eu manteria um conjunto explícito de recursos suportados ou uma matriz de versões testada, com comportamento conservador quando ausente. Essa informação orienta experiência, mas não é prova de identidade nem substitui autorização no backend.

Termo para lembrar: Capability negotiation | Base técnica: referência 1.

09. Versão enviada pelo app é confiável?

Não para segurança. Um cliente modificado pode declarar qualquer versão, então eu verificaria permissões e invariantes no servidor. A versão serve para compatibilidade e observabilidade, enquanto controles críticos independem da honestidade desse campo informado.

Termo para lembrar: Untrusted client metadata | Base técnica: referência 1.

10. GraphQL elimina APIs antigas?

Não. Campos, argumentos e significado continuam sendo contratos consumidos por clientes instalados. Eu usaria depreciação e observação de operações, mantendo compatibilidade; remover um campo necessário ainda quebra o aplicativo mesmo sem uma URL versionada.

Termo para lembrar: Schema evolution | Base técnica: referência 1.

11. Como tratar um campo obrigatório novo?

Eu introduziria um caminho compatível ou uma nova operação, definindo o comportamento dos clientes que não enviam o campo. Um default só é aceitável quando representa uma decisão válida, não quando inventa consentimento ou informação financeira.

Termo para lembrar: Request evolution | Base técnica: referência 1.

12. Como evoluir respostas de erro?

Eu manteria códigos de negócio legíveis por máquina e mensagens separadas para apresentação. Clientes devem reagir a categorias estáveis; mudar texto traduzido não pode alterar lógica de retry, autenticação ou confirmação de uma operação financeira.

Termo para lembrar: Stable error contract | Base técnica: referência 1.

13. Como preservar idempotência entre versões?

Eu definiria escopo e semântica da intenção independentemente do formato do endpoint, evitando que migrar de v1 para v2 crie um novo efeito. Se o payload muda semanticamente, o servidor precisa detectar conflito, não deduplicar incorretamente.

Termo para lembrar: Stable operation identity | Base técnica: referência 1.

14. O que medir para retirar v1?

Eu mediria usuários ativos, operações críticas e versões realmente chamadas ao longo de ciclos relevantes. Volume agregado baixo pode esconder um fluxo essencial ou clientes que acessam mensalmente; decisão de retirada precisa considerar impacto, não só porcentagem.

Termo para lembrar: Consumer telemetry | Base técnica: referência 1.

15. Deprecation significa desligamento?

Não. Eu usaria depreciação para comunicar que o contrato deve ser substituído e uma data de retirada quando definida. Headers e documentação ajudam, mas aplicativos antigos podem ignorá-los; a comunicação de produto também precisa existir.

Termo para lembrar: Deprecation versus sunset | Base técnica: referência 2.

16. Quando obrigar atualização?

Eu reservaria atualização obrigatória para risco ou incompatibilidade justificados, com comunicação e caminho de suporte. A medida pode excluir dispositivos e clientes sem acesso imediato à loja; deve ser uma decisão consciente, não um atalho operacional.

Termo para lembrar: Minimum supported version | Base técnica: referência 1.

17. Como testar clientes muito antigos?

Eu manteria fixtures e contratos de versões suportadas, complementados por builds representativos em testes de integração. Não é necessário testar todas as combinações imagináveis, mas o conjunto escolhido deve cobrir mudanças reais de serialização e comportamento.

Termo para lembrar: Consumer contract matrix | Base técnica: referência 4.

18. Como evitar explosão de versões?

Eu definiria uma política de suporte e favoreceria mudanças aditivas com adaptadores limitados. Cada versão tem custo de segurança, testes e operação; manter tudo para sempre não é sustentabilidade, mas retirar sem evidência também é arriscado.

Termo para lembrar: Compatibility budget | Base técnica: referência 1.

19. Como evoluir paginação?

Eu preservaria a semântica do cursor e informaria expiração ou incompatibilidade explicitamente. Trocar ordenação sem invalidar cursores pode repetir ou pular itens; uma nova versão do token deve carregar informações suficientes para interpretar o contexto.

Termo para lembrar: Pagination contract | Base técnica: referência 1.

20. Como cachear respostas versionadas?

Eu incluiria contrato, parâmetros e contexto autorizado necessários na chave, seguindo a política de privacidade. Se duas versões recebem formatos diferentes, compartilhar o mesmo objeto cacheado pode causar erro de parsing ou exposição entre clientes indevidamente.

Termo para lembrar: Cache key correctness | Base técnica: referência 1.

21. Como tratar mudança de autenticação?

Eu manteria um período de coexistência controlada e avaliaria risco do protocolo antigo, com revogação e recuperação. A migração deve abranger renovação de sessão e dispositivos inativos, não apenas o login de quem instala a versão nova.

Termo para lembrar: Protocol migration | Base técnica: referência 1.

22. Como fazer rollback do backend?

Eu verificaria se a versão anterior ainda entende dados e requests já produzidos pela nova. Voltar o deploy não é seguro automaticamente; mudanças de schema e efeitos externos podem exigir manter adaptadores ou corrigir para frente.

Termo para lembrar: Bidirectional compatibility | Base técnica: referência 1.

23. E usuários que ficam meses offline?

Eu definiria validade de credenciais, cursores e dados locais, oferecendo ressincronização ou atualização quando necessário. O retorno de um cliente antigo deve produzir uma resposta compreensível, não uma sequência de falhas indefinidas sem recuperação possível.

Termo para lembrar: Dormant client handling | Base técnica: referência 1.

24. Como migrar de endpoint síncrono para assíncrono?

Eu não mudaria silenciosamente uma confirmação final para mera aceitação. Criaria contrato que explicite status pendente e consulta posterior, porque clientes antigos podem interpretar HTTP de sucesso como conclusão financeira e apresentar informação incorreta ao usuário.

Termo para lembrar: Behavioral versioning | Base técnica: referência 1.

25. Como desativar uma função vulnerável?

Eu bloquearia o comportamento no servidor e ofereceria uma resposta compatível com os clientes suportados. Esconder o botão por flag não impede chamadas diretas; a autoridade para permitir a operação permanece no backend em qualquer versão.

Termo para lembrar: Server-side kill switch | Base técnica: referência 1.

26. O que fazer com links antigos?

Eu preservaria identificadores e rotas quando possível, ou ofereceria redirecionamento compatível e documentado. Alterar URL de recurso não deve criar uma entidade nova; histórico, notificações e suporte podem continuar apontando para referências antigas existentes.

Termo para lembrar: Stable resource identity | Base técnica: referência 1.

27. Como medir impacto do rollout?

Eu segmentaria erros, latência e conclusão de jornadas por versão e capacidade, sem cardinalidade descontrolada. Uma média saudável pode esconder falha completa numa versão antiga usada por uma parcela pequena, porém relevante, dos clientes.

Termo para lembrar: Version-segmented SLO | Base técnica: referência 1.

28. Como lidar com evolução de consentimento?

Eu criaria fluxo e contrato que representem a nova decisão do usuário, sem inferir aceite de campos ausentes. Compatibilidade técnica não autoriza transformar uma opção antiga em permissão diferente ou mais ampla por conveniência de implementação.

Termo para lembrar: Explicit semantic change | Base técnica: referência 1.

29. Quem é responsável pelo contrato?

Eu definiria dono, política de suporte e processo de revisão com consumidores, incluindo mobile e atendimento. Sem ownership, endpoints legados ficam sem manutenção e cada equipe presume que outra conhece o impacto de uma mudança incompatível.

Termo para lembrar: API ownership | Base técnica: referência 1.

30. Qual defesa principal numa entrevista?

Eu mostraria como clientes antigos continuam seguros enquanto o produto evolui, com critérios de retirada e custo conhecido. A qualidade está em preservar significado e experiência, não em escolher REST ou GraphQL como solução universal.

Termo para lembrar: Evolution without surprise | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Mudança aditiva

Ganho. Reduz necessidade de versões paralelas

Custo e motivo. Depende de tolerância dos clientes

Como decidir. Quando não altera significado ou requisitos existentes

02. Nova versão explícita

Ganho. Isola contrato incompatível

Custo e motivo. Aumenta manutenção e testes

Como decidir. Para mudanças semânticas que não admitem adaptação segura

03. Adaptador na borda

Ganho. Centraliza compatibilidade

Custo e motivo. Adiciona tradução e possível latência

Como decidir. Quando domínio pode permanecer único

04. BFF móvel

Ganho. Otimiza composição para o aplicativo

Custo e motivo. Cria mais um serviço para operar

Como decidir. Quando necessidades de interface diferem materialmente

05. Flags de capacidade

Ganho. Permitem exposição gradual

Custo e motivo. Criam combinações e dívida temporária

Como decidir. Com dono, validade e defaults seguros

06. Suporte prolongado

Ganho. Preserva usuários antigos

Custo e motivo. Eleva custo de segurança e operação

Como decidir. Quando adoção lenta e risco permitem

07. Atualização obrigatória

Ganho. Remove contratos inseguros ou inviáveis

Custo e motivo. Pode excluir dispositivos e usuários

Como decidir. Com justificativa, comunicação e suporte

08. GraphQL evolutivo

Ganho. Permite seleção de campos e depreciação

Custo e motivo. Não remove contratos nem custo de queries

Como decidir. Quando flexibilidade de leitura compensa governança adicional

09. Telemetria por versão

Ganho. Localiza impacto em clientes específicos

Custo e motivo. Pode aumentar cardinalidade

Como decidir. Com dimensões limitadas e jornadas críticas identificadas

10. Janela de coexistência

Ganho. Permite rollback e adoção gradual

Custo e motivo. Prolonga duplicidade de formatos

Como decidir. Até critérios mensuráveis de migração e segurança serem cumpridos

Duas alternativas ao caminho proposto

1. Contrato estável com extensões de capacidade

Quando faz sentido. Quando evolução é majoritariamente aditiva

Como implementar. Manter uma API com campos opcionais e negociação explícita

Limites. Não acomoda toda mudança de significado e exige clientes tolerantes

2. Versões maiores com camada de tradução

Quando faz sentido. Quando há rupturas inevitáveis

Como implementar. Publicar contratos separados sobre domínio comum com prazo de suporte

Limites. Adaptadores custam manutenção e não devem mascarar diferenças financeiras incompatíveis

Referências e alcance da evidência

1. Microsoft: Web API design

Princípios de contratos HTTP e evolução de APIs; a política de compatibilidade móvel abaixo é proposta autoral.

https://learn.microsoft.com/en-us/azure/architecture/best-practices/api-design

2. IETF: RFC 9745 Deprecation

Sinalização de depreciação por header e link, não atualização automática de aplicativos.

https://www.rfc-editor.org/rfc/rfc9745.html

3. IETF: RFC 8594 Sunset

Comunicação de retirada futura de um recurso HTTP, distinta da mera depreciação.

https://www.rfc-editor.org/rfc/rfc8594.html

4. Pact: Introduction

Testes de contrato entre consumidores e provedores; não substituem carga, segurança ou comportamento completo de produção.

https://docs.pact.io/

Continue em: T008 · T010 · C022

C026 / M. Os 50 cenários enviados

Ingestão massiva de logs de aplicativos móveis

Ingestão de logs móveis é um pipeline de telemetria com buffers limitados, envio oportunista, admissão no servidor e armazenamento por utilidade. A observabilidade não deve comprometer bateria, privacidade ou operações bancárias.

Quando usar. Para diagnosticar crashes, degradação por versão e falhas de jornada em milhões de dispositivos com conectividade e capacidade variáveis.

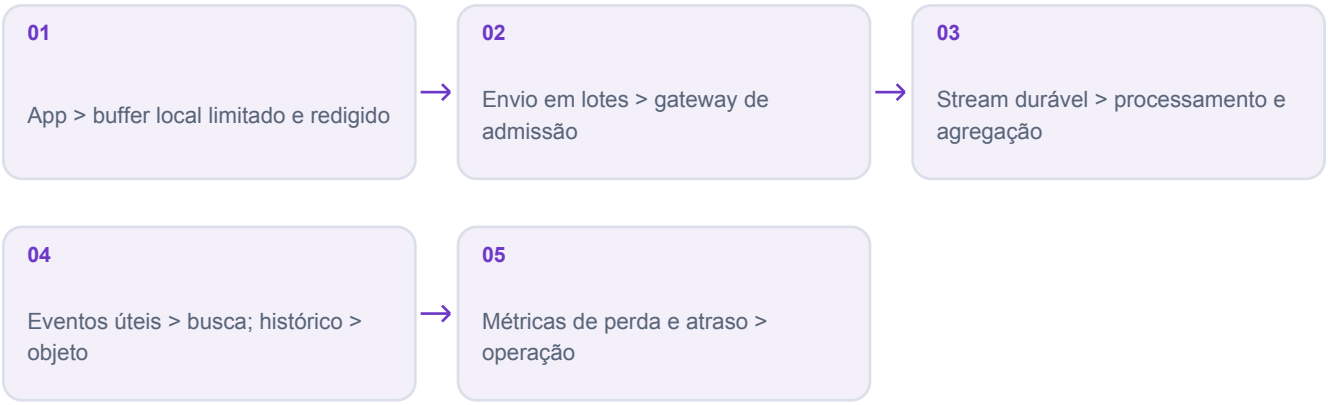
Quando evitar ou limitar. Evitar prometer entrega perfeita após crash ou reinstalação, usar logs como ledger ou transmitir payloads financeiros e segredos indiscriminadamente.

Uma abertura possível

Eu desenharia o pipeline para falhar sem derrubar o app. O dispositivo acumula um volume limitado e envia lotes conforme condições; a borda protege o backend, e o pipeline prioriza eventos úteis com perdas explícitas quando necessário.

Peças e tecnologias. SQLite ou buffer local limitado · WorkManager ou mecanismo equivalente da plataforma · HTTPS · Kinesis ou Kafka · OpenTelemetry Collector · S3 · OpenSearch

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Definir eventos permitidos, prioridade, orçamento de bytes e retenção.

2. Implementar buffer local e envio com batch, compressão e retry limitado.

3. Autenticar e limitar ingestão antes de aceitar dados no stream.

4. Processar com schema versionado, deduplicação quando necessária e destinos por utilidade.

5. Monitorar perda, atraso, custo e impacto no app, testando surtos e falhas.

30 perguntas para sustentar o desenho

01. Por que não enviar cada log imediatamente?

Cada envio tem custo de rede, rádio e protocolo. Eu agruparia eventos até um limite de tempo ou tamanho, preservando urgência de erros relevantes; isso reduz overhead, mas aumenta atraso e exige buffer local controlado.

Termo para lembrar: Batching | Base técnica: referência 1.

02. Como evitar consumir toda memória do celular?

Eu imporia limites de bytes e quantidade, descartando primeiro eventos menos úteis conforme política. Um logger não pode causar o crash que tenta diagnosticar; a ocupação e as perdas precisam ser observáveis sem gerar mais logs infinitamente.

Termo para lembrar: Bounded buffer | Base técnica: referência 1.

03. Buffer em memória ou disco?

Eu usaria memória para sinais descartáveis e armazenamento local limitado para eventos importantes. Disco sobrevive a parte das falhas, mas custa IO e exige proteção; nem ele garante entrega após desinstalação, corrupção ou perda do dispositivo.

Termo para lembrar: Durability budget | Base técnica: referência 1.

04. O que acontece durante um crash?

Eu evitaria operações complexas e alocações excessivas no momento da falha, usando suporte apropriado da plataforma. Persistiria contexto mínimo quando possível e enviaria na próxima execução; não prometeria uma chamada de rede bem-sucedida durante o crash.

Termo para lembrar: Crash-safe capture | Base técnica: referência 1.

05. Como respeitar bateria e rede limitada?

Eu agendaria envios considerando conectividade, carga e prioridade, com limites de bytes por período. Eventos urgentes podem ter política diferente, mas o pipeline deve respeitar restrições do sistema operacional e não manter o rádio ativo continuamente.

Termo para lembrar: Opportunistic scheduling | Base técnica: referência 1.

06. Como lidar com app offline por dias?

Eu manteria retenção local limitada e descartaria dados antigos conforme utilidade, preservando contadores de perda. Ao reconectar, o cliente não envia tudo de uma vez: usa orçamento e jitter para não provocar um pico coordenado.

Termo para lembrar: Retention window | Base técnica: referência 1.

07. Qual identidade cada evento recebe?

Eu atribuiria um identificador ao criar o evento e o preservaria em retries, junto da versão do schema. Isso permite reconhecer duplicatas quando necessário; gerar outro identificador a cada tentativa transforma redelivery em novos eventos aparentemente distintos.

Termo para lembrar: Stable event identifier | Base técnica: referência 1.

08. Precisa deduplicar todos os logs?

Depende do uso. Para diagnóstico exploratório, duplicatas limitadas podem ser toleráveis; para métricas que dirigem decisões, eu deduplicaria ou estimaria o viés. Não pagaria custo global de unicidade sem relacioná-lo ao requisito de análise.

Termo para lembrar: Approximate analytics | Base técnica: referência 1.

09. Quando confirmar o batch ao cliente?

Eu confirmaria depois de aceitar o lote na fronteira de durabilidade prometida, como o stream configurado. Receber bytes em memória não é o mesmo que persistir; o contrato de resposta precisa deixar claro qual garantia oferece.

Termo para lembrar: Durable acceptance | Base técnica: referência 1.

10. Como tratar falha parcial no lote?

Eu identificaria os itens aceitos e rejeitados para repetir apenas o necessário, ou definiria atomicidade do envelope explicitamente. Repetir tudo sem identidade estável pode inflar contagens e aumentar tráfego justamente durante uma falha de ingestão.

Termo para lembrar: Per-item acknowledgement | Base técnica: referência 1.

11. Como proteger contra um app com bug gerando milhões de logs?

Eu aplicaria limites por instalação, versão e conta, além de orçamento global e prioridade. O cliente também limita produção; confiar apenas no backend desperdiça bateria e rede antes que o descarte aconteça na infraestrutura.

Termo para lembrar: Admission control | Base técnica: referência 1.

12. Qual papel de Kafka ou Kinesis?

Eu usaria o stream para absorver variação e desacoplar ingestão de consumidores, com retenção e capacidade definidas. Ele não cria capacidade infinita: atraso crescente precisa gerar ação antes que o consumidor perca dados por expiração.

Termo para lembrar: Durable buffering | Base técnica: referência 1.

13. Como escolher a chave de partição?

Eu distribuiria por identificador estável com boa cardinalidade quando a ordem por dispositivo importar. Usar apenas versão do app concentra um lançamento inteiro; o desenho deve equilibrar contexto de sessão e distribuição da carga recebida.

Termo para lembrar: Partition distribution | Base técnica: referência 1.

14. Como dimensionar o pipeline?

Eu estimaria dispositivos ativos, eventos por dispositivo, tamanho médio e pico por versão, distinguindo bytes comprimidos e expandidos. Depois dimensionaria ingestão, retenção e consumo; contar apenas requests esconde o custo real de lotes grandes.

Termo para lembrar: Byte-rate capacity | Base técnica: referência 1.

15. O que priorizar numa sobrecarga?

Eu preservaria amostras de crashes e falhas críticas, descartando debug repetitivo e reduzindo frequência de sucesso. A política precisa evitar que uma única assinatura monopolize espaço e deve publicar métricas explícitas do que foi perdido.

Termo para lembrar: Priority shedding | Base técnica: referência 1.

16. Como evitar uma tempestade de retries?

Eu usaria espera exponencial limitada e aleatoriedade, respeitando sinalização de sobrecarga do servidor. O cliente mantém um orçamento de tentativas; retries sem limite transformam uma indisponibilidade curta em pressão prolongada sobre rede e backend.

Termo para lembrar: Backoff with jitter | Base técnica: referência 1.

17. Como tratar logs com dados pessoais?

Eu definiria uma lista permitida de campos e redigiria dados antes de sair do dispositivo, com validação adicional no servidor. Remover depois do armazenamento é tarde para prevenir exposição em trânsito, filas e acessos intermediários.

Termo para lembrar: Telemetry minimization | Base técnica: referência 1.

18. É seguro registrar request e response completos?

Em geral eu evitaria, especialmente em autenticação e operações financeiras. Preferiria códigos, tamanhos, duração e identificadores de correlação controlados; payload completo pode incluir tokens, documentos e detalhes que não são necessários para diagnosticar o problema.

Termo para lembrar: Sensitive payload avoidance | Base técnica: referência 1.

19. Como correlacionar app e backend?

Eu propagaria identificadores de trace ou requisição sem transformá-los em autorização, registrando apenas contexto necessário. O servidor valida formato e tamanho; valores enviados pelo cliente não devem criar cardinalidade ilimitada ou permitir injeção nos logs.

Termo para lembrar: Trace correlation | Base técnica: referência 1.

20. Qual relógio usar para ordenar?

Eu guardaria horário informado pelo dispositivo e horário de ingestão, além de sequência local quando útil. Relógios móveis podem estar errados; análises não devem presumir ordem global confiável apenas porque eventos carregam timestamps semelhantes.

Termo para lembrar: Event time versus ingest time | Base técnica: referência 1.

21. Como evoluir o schema dos eventos?

Eu versionaria envelopes e manteria parsers tolerantes a campos adicionais, validando limites e tipos essenciais. Clientes antigos continuam enviando formatos anteriores; um deploy do consumidor não deve descartar silenciosamente toda telemetria dessas versões ainda suportadas.

Termo para lembrar: Telemetry schema evolution | Base técnica: referência 1.

22. Qual destino de armazenamento escolher?

Eu colocaria dados recentes e úteis para busca em um índice dimensionado, e histórico selecionado em armazenamento de objetos. Indexar tudo indefinidamente eleva custo; consultas raras podem aceitar processamento mais lento sobre dados comprimidos.

Termo para lembrar: Hot and cold telemetry | Base técnica: referência 1.

23. Como lidar com cardinalidade alta?

Eu limitaria dimensões de métricas e agregaria assinaturas normalizadas de erro, separando detalhes de diagnóstico. Identificador de usuário como label pode explodir séries; manter o contexto em logs controlados costuma ser mais apropriado que indexar cada valor.

Termo para lembrar: Cardinality control | Base técnica: referência 1.

24. Como detectar perda do próprio pipeline?

Eu compararia eventos produzidos, aceitos e processados por janelas, acompanhando rejeições, lag e ocupação de buffers. Ausência de logs pode significar saúde ou pipeline quebrado; sinais independentes são necessários para distinguir essas situações operacionalmente.

Termo para lembrar: Telemetry pipeline SLI | Base técnica: referência 1.

25. Como impedir que a API de logs vire vetor de ataque?

Eu limitaria tamanho descomprimido, profundidade de objetos, formatos e taxa antes do processamento caro. Telemetria vem de dispositivos não confiáveis; autenticação não elimina payload malicioso nem autoriza consumir recursos ilimitados do backend de observabilidade.

Termo para lembrar: Untrusted ingestion | Base técnica: referência 1.

26. Como tratar compressão maliciosa?

Eu imporia limite ao tamanho expandido e à razão de compressão, com timeout e rejeição antecipada. Um corpo pequeno pode expandir enormemente; aceitar apenas pelo Content-Length comprimido não protege memória e CPU do serviço.

Termo para lembrar: Decompression limits | Base técnica: referência 1.

27. O que fazer com evento inválido?

Eu rejeitaria ou isolaria o item com motivo rastreável, sem bloquear indefinidamente a partição inteira. A quarentena também possui limites e retenção; guardar cada payload inválido sem controle pode criar outra fonte de custo e exposição.

Termo para lembrar: Poison-event handling | Base técnica: referência 1.

28. Como testar um lançamento problemático?

Eu simularia uma versão que aumenta crashes e repete a mesma assinatura, medindo buffers, atraso e descarte. O teste deve confirmar que o app continua responsivo e que o pipeline preserva evidências suficientes para diagnosticar o incidente.

Termo para lembrar: Burst simulation | Base técnica: referência 1.

29. Logs podem confirmar um pagamento?

Não. Eu usaria registros transacionais e reconciliação do sistema financeiro como autoridade. Logs podem faltar, duplicar ou ser amostrados; são evidência operacional auxiliar e não devem determinar sozinhos se uma obrigação monetária foi executada.

Termo para lembrar: Authority separation | Base técnica: referência 1.

30. Qual resumo defenderia na entrevista?

Eu mostraria um caminho de coleta limitado, privado e recuperável, com perdas deliberadas onde aceitáveis. A meta não é guardar cada byte a qualquer custo, mas obter diagnóstico útil sem prejudicar o cliente ou o sistema principal.

Termo para lembrar: Observability without harm | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Batch maior

Ganho. Reduz overhead de rede

Custo e motivo. Aumenta atraso e tamanho de retry

Como decidir. Quando urgência permite acumular eventos

02. Persistência local

Ganho. Sobrevive a parte das falhas

Custo e motivo. Consome disco e exige proteção

Como decidir. Para eventos de maior utilidade diagnóstica

03. Envio oportunista

Ganho. Poupa bateria e dados móveis

Custo e motivo. Atrasa visibilidade do incidente

Como decidir. Para telemetria que não exige tempo real

04. Compressão

Ganho. Reduz bytes transferidos

Custo e motivo. Consome CPU e exige limites de expansão

Como decidir. Quando ganho medido supera custo no dispositivo

05. Stream durável

Ganho. Desacopla ingestão e processamento

Custo e motivo. Adiciona custo, lag e operação

Como decidir. Quando há picos e múltiplos consumidores

06. Deduplicação

Ganho. Melhora precisão de contagens

Custo e motivo. Consome estado e processamento

Como decidir. Para análises sensíveis a retries

07. Amostragem

Ganho. Reduz volume e custo

Custo e motivo. Pode esconder eventos raros

Como decidir. Com política por prioridade e avaliação de viés

08. Busca quente

Ganho. Acelera investigação recente

Custo e motivo. Custa indexação e armazenamento

Como decidir. Para janela operacional de alta consulta

09. Arquivo em objetos

Ganho. Barateia retenção selecionada

Custo e motivo. Aumenta latência de consulta

Como decidir. Para histórico pouco acessado com necessidade definida

10. Load shedding

Ganho. Preserva infraestrutura e app

Custo e motivo. Perde sinais deliberadamente

Como decidir. Com prioridades, limites e métricas de descarte explícitos

Duas alternativas ao caminho proposto

1. Plataforma gerenciada de crash reporting

Quando faz sentido. Quando a equipe precisa de diagnóstico móvel rapidamente

Como implementar. Integrar SDK com filtros, políticas de coleta e limites apropriados

Limites. Dependência externa, residência de dados e controle de custo precisam ser avaliados

2. Uploads periódicos para armazenamento de objetos

Quando faz sentido. Para diagnóstico em lotes sem busca imediata

Como implementar. Enviar arquivos pequenos autorizados e processar assincronamente

Limites. Maior atraso e necessidade de controlar acesso, tamanho e ciclo de vida dos objetos

Referências e alcance da evidência

1. Android: Build an offline-first app

Armazenamento local e sincronização considerando rede e bateria; políticas financeiras e protocolos específicos são autorais.

https://developer.android.com/topic/architecture/data-layer/offline-first

2. OpenTelemetry: Scaling the Collector

Escala e organização de pipelines de telemetria; garantias de entrega dependem da configuração e dos componentes escolhidos.

https://opentelemetry.io/docs/collector/scaling/

3. AWS: Kinesis Data Streams concepts

Streams, shards, produtores e consumidores; dimensionamento e política de descarte do caso são hipóteses de projeto.

https://docs.aws.amazon.com/streams/latest/dev/key-concepts.html

Continue em: T003 · T038 · C025

C027 / M. Os 50 cenários enviados

Sincronização offline-first do aplicativo para consultas sem internet

Offline-first para consultas bancárias mantém uma cópia local autorizada e identificada por frescor, sincronizando mudanças de forma retomável. Consultar dados antigos não autoriza executar transações financeiras offline.

Quando usar. Para extratos recentes, comprovantes já baixados e informações que continuam úteis sem conexão, com data da última atualização visível.

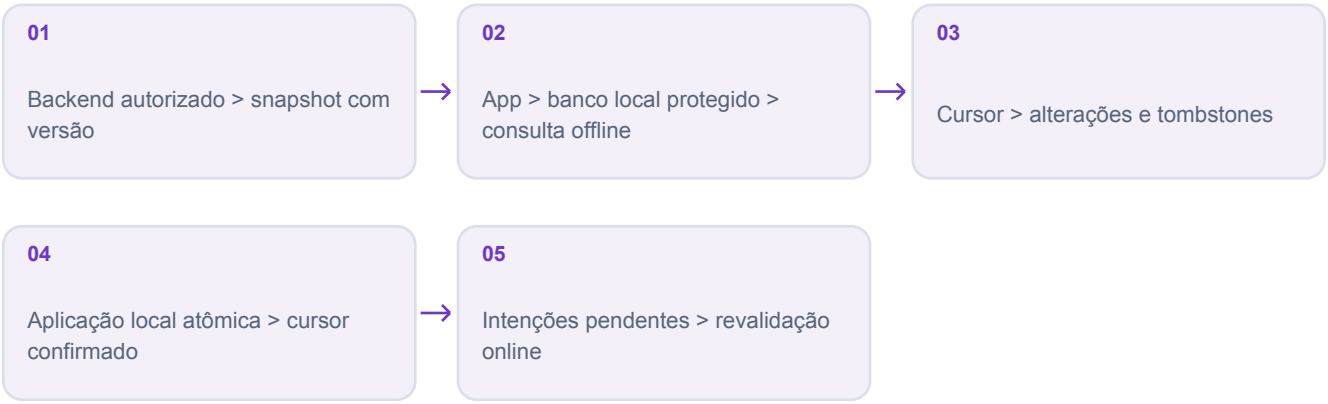
Quando evitar ou limitar. Evitar prometer saldo atual offline, tratar ETag como protocolo completo de delta ou usar CRDT para resolver automaticamente invariantes de dinheiro e limites.

Uma abertura possível

Eu definiria primeiro o que pode funcionar offline. O app mostra a última visão conhecida com seu horário; o backend continua sendo autoridade financeira. Sincronização incremental cuida de alterações e exclusões, enquanto intenções de escrita permanecem explicitamente pendentes.

Peças e tecnologias. SQLite ou Room · Keychain ou Android Keystore para proteção de chaves · API de sincronização com cursor · ETag/If-None-Match · WorkManager ou equivalente · PostgreSQL ou projeção de leitura

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Classificar dados e ações que podem ser usados sem rede.

2. Persistir snapshot local com autorização, versão e horário de atualização.

3. Implementar cursor incremental com exclusões, paginação e retenção definida.

4. Aplicar cada lote e checkpoint atomicamente, com ressincronização quando necessário.

5. Distinguir dados conhecidos de efeitos confirmados e testar reconexão, logout e troca de conta.

30 perguntas para sustentar o desenho

01. O que o usuário consegue fazer sem internet?

Eu permitiria consultar dados previamente sincronizados e criar rascunhos quando apropriado. Operações financeiras dependentes de saldo atual ficam indisponíveis ou pendentes; a interface não pode apresentar uma intenção local como transferência concluída pelo banco.

Termo para lembrar: Offline capability boundary | Base técnica: referência 1.

02. Como mostrar saldo offline?

Eu exibiria o último saldo conhecido junto do horário de atualização e indicação clara de desconexão. Ele serve como referência histórica, não como autorização para gastar; o valor disponível pode ter mudado em outro dispositivo ou canal.

Termo para lembrar: Freshness disclosure | Base técnica: referência 1.

03. Banco local é fonte de verdade?

Pode ser a fonte imediata da interface, oferecendo leitura estável, mas não a autoridade financeira global. Eu separaria esses significados: o app renderiza do banco local enquanto o servidor valida obrigações e produz estados confirmados.

Termo para lembrar: Presentation source of truth | Base técnica: referência 1.

04. ETag já implementa delta sync?

Não. ETag permite verificar se uma representação mudou e evitar transferi-la quando permanece igual. Para receber apenas alterações de vários registros, eu preciso de um protocolo adicional com cursor, versões, exclusões e comportamento de expiração.

Termo para lembrar: Conditional representation | Base técnica: referência 1.

05. Como seria uma API incremental?

Eu retornaria alterações desde um cursor opaco, junto do próximo cursor e indicador de continuidade. O cursor precisa representar uma posição consistente no feed autorizado; usar apenas horário do celular deixa lacunas e problemas de ordenação.

Termo para lembrar: Change cursor | Base técnica: referência 1.

06. Como evitar perder alterações durante o snapshot inicial?

Eu associaria o snapshot a uma posição de mudanças e continuaria a partir dela, evitando intervalo entre cópia e assinatura. Sem essa coordenação, uma atualização ocorrida durante o download pode nunca aparecer no dispositivo depois.

Termo para lembrar: Snapshot-to-log handoff | Base técnica: referência 1.

07. Como aplicar uma página de mudanças?

Eu gravaria alterações e novo cursor na mesma transação local. Se o app cair antes do commit, repete a página; se confirmar depois, avança com segurança. Atualizar o cursor antes dos dados pode perder mudanças definitivamente.

Termo para lembrar: Atomic local checkpoint | Base técnica: referência 1.

08. Como lidar com páginas repetidas?

Eu usaria identificador e versão do registro para aplicar somente mudanças novas ou equivalentes. Repetir uma página não pode duplicar lançamentos no extrato; a lógica de merge precisa suportar redelivery normal após interrupções de rede.

Termo para lembrar: Idempotent merge | Base técnica: referência 1.

09. Como sincronizar exclusões?

Eu enviaria marcadores de exclusão versionados pelo período de retenção do feed. Sem eles, um registro removido no servidor permanece no aparelho; dispositivos muito antigos precisam refazer o snapshot quando já não há histórico suficiente.

Termo para lembrar: Tombstones | Base técnica: referência 1.

10. O que acontece quando o cursor expira?

Eu retornaria uma condição explícita que exige novo snapshot, preservando rascunhos locais separadamente. Não tentaria continuar com um cursor inválido silenciosamente, pois isso pode deixar exclusões e alterações antigas ausentes da cópia local do cliente.

Termo para lembrar: Resynchronization protocol | Base técnica: referência 1.

11. Como paginar sem pular mudanças concorrentes?

Eu fixaria um limite superior de posição para a sessão de sincronização e paginaria dentro dele. Novas mudanças entram na próxima rodada; paginação por offset sobre um conjunto mutável pode duplicar ou omitir registros durante o processo.

Termo para lembrar: Stable synchronization window | Base técnica: referência 1.

12. Precisa de vector clocks?

Não para uma cópia de leitura cuja autoridade é o servidor. Versões monotônicas por entidade ou cursor podem bastar; vector clocks fazem sentido quando precisamos detectar causalidade entre escritores independentes e aceitamos o custo desse modelo.

Termo para lembrar: Conflict model selection | Base técnica: referência 1.

13. Quando CRDT seria adequado?

Eu consideraria para preferências ou coleções colaborativas com semântica de merge compatível. Não usaria convergência como prova de saldo correto: limites e transferências exigem invariantes que não desaparecem só porque réplicas eventualmente concordam no mesmo estado.

Termo para lembrar: Convergent data types | Base técnica: referência 1.

14. Como editar um apelido de conta offline?

Eu guardaria a intenção com a versão base e enviaria depois, usando comparação de versão no servidor. Em conflito, aplicaria a política definida ou pediria escolha; sobrescrever cegamente pode apagar uma mudança feita em outro dispositivo.

Termo para lembrar: Optimistic concurrency | Base técnica: referência 1.

15. Last-write-wins é suficiente?

Pode ser aceitável para dados de baixo risco com perda tolerável, mas eu definiria qual ordem vale e explicaria a perda possível. Relógio do cliente não é autoridade confiável, especialmente com dispositivos desajustados ou intencionalmente manipulados.

Termo para lembrar: Conflict resolution policy | Base técnica: referência 1.

16. Como tratar um Pix iniciado offline?

Eu mostraria um rascunho ou intenção ainda não enviada, exigindo validação online antes do efeito. Na reconexão, saldo, destinatário, limites e consentimento podem precisar de revisão; não prometeria execução automática em condições financeiras diferentes.

Termo para lembrar: Pending intent | Base técnica: referência 1.

17. Como repetir uma escrita após reconexão?

Eu criaria a identidade da intenção antes do primeiro envio e a manteria no armazenamento local. Se a resposta se perder, consulto ou repito a mesma operação; gerar outra chave pode duplicar o efeito financeiro no servidor.

Termo para lembrar: Persistent operation key | Base técnica: referência 1.

18. Como tratar logout?

Eu encerraria sessões e aplicaria a política de remoção ou inacessibilidade dos dados locais, incluindo caches e arquivos. Trocar a conta ativa sem isolar o armazenamento pode mostrar informações do usuário anterior no mesmo aparelho compartilhado.

Termo para lembrar: Local data lifecycle | Base técnica: referência 1.

19. Como proteger a base no dispositivo?

Eu utilizaria proteção de armazenamento e chaves oferecida pela plataforma, minimizando dados guardados. Criptografia reduz exposição em alguns cenários, mas não protege tudo num dispositivo comprometido; controle de sessão e limites de retenção continuam necessários.

Termo para lembrar: At-rest protection | Base técnica: referência 1.

20. O que sincronizar primeiro?

Eu priorizaria estado essencial e dados recentes, deixando histórico volumoso sob demanda. Um download completo antes de mostrar qualquer informação prejudica a experiência; a interface pode indicar progresso enquanto preserva uma visão local consistente e utilizável.

Termo para lembrar: Priority synchronization | Base técnica: referência 1.

21. Como economizar bateria na sincronização?

Eu combinaria atualização sob demanda, notificações como sinal e trabalho em segundo plano permitido pela plataforma. Polling frequente custa bateria; o agendador precisa respeitar conectividade e restrições do sistema, sem prometer execução exata em qualquer horário.

Termo para lembrar: Adaptive sync scheduling | Base técnica: referência 1.

22. Push notification garante que o app está atualizado?

Não. Eu trataria push como sugestão para sincronizar, porque entrega pode atrasar ou falhar. O app verifica atualização ao abrir ou recuperar rede; a correção não deve depender exclusivamente de receber todas as notificações enviadas.

Termo para lembrar: Invalidation hint | Base técnica: referência 1.

23. Como lidar com evento fora de ordem?

Eu compararia a versão recebida com a armazenada e impediria regressão do registro. Quando uma mudança depende de outra entidade, o protocolo precisa fornecer ordem ou recuperação explícita, em vez de assumir que a rede preserva dependências automaticamente.

Termo para lembrar: Version-aware application | Base técnica: referência 1.

24. Como impedir vazamento entre contas no cache?

Eu incluiria identidade autorizada no particionamento local e nas chaves do servidor, limpando contexto na troca de usuário. O endpoint também filtra por autorização; um cursor válido de outra conta não pode permitir acesso cruzado aos dados.

Termo para lembrar: Tenant-scoped storage | Base técnica: referência 1.

25. Como evoluir o schema local?

Eu versionaria migrações do banco móvel e testaria atualização a partir das versões suportadas. Se houver reconstrução, preservaria intenções pendentes necessárias; apagar toda a base indiscriminadamente pode perder trabalho do usuário ainda não sincronizado com o servidor.

Termo para lembrar: Local database migration | Base técnica: referência 1.

26. Qual limite de armazenamento impor?

Eu definiria janela de histórico e tamanho máximo, removendo dados recuperáveis menos recentes primeiro. Rascunhos e operações pendentes têm política separada; uma limpeza por idade não pode apagar silenciosamente intenções que o usuário acredita ainda aguardarem envio.

Termo para lembrar: Local retention | Base técnica: referência 1.

27. Como tratar exclusão de acesso enquanto offline?

Eu reconheceria que um dispositivo desconectado não recebe revogação imediatamente. Por isso minimizo conteúdo, uso expiração local quando apropriada e proteção do aparelho; prometer apagar remotamente uma cópia sem conexão seria uma garantia impossível.

Termo para lembrar: Revocation limitation | Base técnica: referência 1.

28. Como testar sincronização robusta?

Eu interromperia o processo antes e depois de aplicar dados e cursor, repetiria páginas e expiraria o histórico. Também testaria troca de conta e relógio incorreto; a validação deve mostrar convergência sem duplicar ou perder registros autorizados.

Termo para lembrar: Fault-sequence testing | Base técnica: referência 1.

29. Qual métrica acompanha a experiência?

Eu mediria idade da visão local após reconexão, duração da sincronização e falhas de ressincronização por versão. Disponibilidade da API sozinha não revela se o cliente continua vendo um extrato antigo apesar de já ter internet.

Termo para lembrar: Data freshness SLI | Base técnica: referência 1.

30. Qual defesa final do desenho?

Eu ofereceria utilidade sem mascarar incerteza: leitura conhecida continua disponível, escrita tem estado explícito e a autoridade financeira permanece online. A sofisticação está no protocolo e na experiência coerente, não em exigir CRDT para qualquer cache móvel.

Termo para lembrar: Honest offline semantics | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Leitura local imediata

Ganho. Mantém interface útil sem rede

Custo e motivo. Pode mostrar dados antigos

Como decidir. Com frescor visível e autoridade financeira separada

02. Snapshot completo

Ganho. Simplifica recuperação inicial

Custo e motivo. Consome rede e tempo

Como decidir. Para bases pequenas ou cursores expirados

03. Feed incremental

Ganho. Reduz bytes transferidos

Custo e motivo. Exige retenção, tombstones e continuidade

Como decidir. Quando volume e frequência justificam protocolo próprio

04. ETag

Ganho. Evita baixar representação inalterada

Custo e motivo. Não entrega mudanças de coleção sozinho

Como decidir. Para recursos cuja representação pode ser revalidada

05. Retenção longa de tombstones

Ganho. Suporta dispositivos ausentes por mais tempo

Custo e motivo. Consome armazenamento e manutenção

Como decidir. Quando retorno tardio precisa evitar snapshot frequente

06. Escrita offline

Ganho. Melhora produtividade sem rede

Custo e motivo. Cria conflitos e intenções pendentes

Como decidir. Para ações cujo risco e consentimento permitem execução posterior

07. Servidor como autoridade

Ganho. Simplifica invariantes financeiras

Custo e motivo. Exige conexão para confirmar efeitos

Como decidir. Para saldo, limites e transferências

08. CRDT

Ganho. Permite merge automático em tipos adequados

Custo e motivo. Impõe semântica e metadados específicos

Como decidir. Para dados colaborativos com invariantes compatíveis

09. Histórico local amplo

Ganho. Facilita consulta antiga offline

Custo e motivo. Aumenta exposição e uso de disco

Como decidir. Com necessidade explícita e controles proporcionais

10. Push como sinal

Ganho. Reduz polling desnecessário

Custo e motivo. Pode atrasar ou não chegar

Como decidir. Com sincronização de recuperação independente

Duas alternativas ao caminho proposto

1. Cache local somente de leitura

Quando faz sentido. Quando a necessidade é consultar histórico sem internet

Como implementar. Baixar snapshots versionados e revalidar ao reconectar

Limites. Não permite mutações offline nem garante frescor durante desconexão

2. Sincronização gerenciada de dados colaborativos

Quando faz sentido. Quando o produto exige múltiplos escritores offline

Como implementar. Adotar plataforma com merge e políticas por tipo de dado

Limites. Garantias, autorização e conflitos financeiros precisam de análise própria; não terceirizam invariantes

Referências e alcance da evidência

1. Android: Build an offline-first app

Armazenamento local e sincronização considerando rede e bateria; políticas financeiras e protocolos específicos são autorais.

https://developer.android.com/topic/architecture/data-layer/offline-first

2. IETF: HTTP Semantics RFC 9110

ETag, condicionais HTTP e semântica de métodos; um protocolo de delta precisa de desenho adicional.

https://www.rfc-editor.org/rfc/rfc9110

Continue em: T012 · T018 · T032

C028 / M. Os 50 cenários enviados

Desenho de um sistema de "Distributed Lock" (Bloqueio Distribuído)

Um lock distribuído coordena participantes sobre um recurso, mas leases expiráveis não interrompem processos antigos. Correção exige proteção no local do efeito, como fencing, CAS, transação ou identidade de operação.

Quando usar. Para coordenar trabalho exclusivo entre processos quando a semântica do recurso e o modelo de falhas estão definidos e mecanismos locais não bastam.

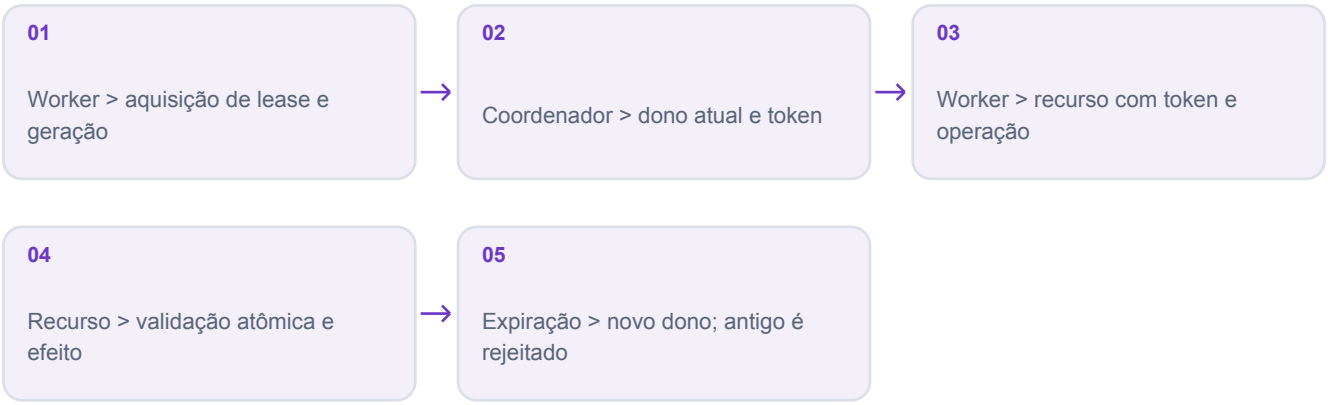
Quando evitar ou limitar. Evitar usar SETNX isolado como garantia de exclusividade financeira, presumir que TTL mata o dono anterior ou confundir token aleatório de propriedade com fencing monotônico.

Uma abertura possível

Eu perguntaria qual efeito precisa ser protegido. Se o banco pode validar a condição na mesma transação, prefiro isso. Quando um coordenador é necessário, uso lease com token de geração e faço o recurso rejeitar donos obsoletos.

Peças e tecnologias. PostgreSQL row locks e CAS · etcd leases e transactions · Redis para coordenação de risco limitado · Fencing tokens · Fila particionada por entidade

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Definir invariante, recurso protegido e consequência de execução concorrente.

2. Preferir constraint, CAS ou transação local quando suficientes.

3. Escolher coordenador e hipóteses de lease para o domínio de falha.

4. Validar fencing ou condição no recurso junto do efeito.

5. Testar pausas, partições, expiração, retomada e resultados desconhecidos.

30 perguntas para sustentar o desenho

01. Qual problema um lock distribuído resolve?

Ele coordena participantes para reduzir ou impedir trabalho concorrente sobre um recurso, conforme suas garantias. Eu começaria pelo efeito protegido, porque exclusividade no serviço de lock não implica automaticamente exclusividade num banco ou parceiro externo.

Termo para lembrar: Mutual exclusion | Base técnica: referência 1.

02. Quando dispensaria o lock distribuído?

Se a condição e o efeito cabem no mesmo banco, eu preferiria constraint, atualização condicional ou transação adequada. Isso reduz componentes e coloca a validação junto da escrita, evitando depender de um lease separado para preservar a regra.

Termo para lembrar: Invariant localization | Base técnica: referência 1.

03. SETNX sozinho é suficiente?

Não para uma garantia geral. Eu precisaria de expiração e identificação do dono, além de analisar persistência e failover. Mesmo com isso, um processo pausado pode voltar depois da expiração e continuar produzindo efeitos no recurso protegido.

Termo para lembrar: Lease acquisition | Base técnica: referência 1.

04. Por que TTL não resolve processo travado?

O TTL remove a autorização no coordenador, mas não encerra a execução do processo. Se ele retoma após outro worker adquirir o recurso, ambos podem agir; o destino precisa reconhecer e rejeitar a geração antiga.

Termo para lembrar: Stale owner | Base técnica: referência 1.

05. Como funciona fencing?

Cada nova posse recebe uma geração monotônica, e o recurso rejeita operações de gerações obsoletas de forma atômica com o efeito. O token só ajuda quando o destino realmente o valida; incluí-lo no log não protege nada.

Termo para lembrar: Fencing token | Base técnica: referência 1.

06. Token aleatório é fencing?

Não. Um valor aleatório identifica quem pode liberar aquele lock, mas não informa qual posse é mais recente. Para fencing, preciso de uma ordem confiável de gerações e validação persistente no recurso que executa o efeito.

Termo para lembrar: Ownership versus ordering | Base técnica: referência 1.

07. Onde armazenar a última geração?

Eu a manteria no próprio domínio transacional do recurso, atualizando condição e efeito juntos. Verificar o token num serviço e depois escrever sem condição reabre uma janela em que o dono antigo pode interferir novamente.

Termo para lembrar: Resource-side validation | Base técnica: referência 1.

08. Fencing prova que o lease ainda está válido?

Não necessariamente. Um recurso que só compara a maior geração já vista rejeita antigos após conhecer uma geração superior. Se a regra exige validade atual do lease em cada ação, preciso de protocolo adicional ou autoridade transacional integrada.

Termo para lembrar: Fencing scope | Base técnica: referência 1.

09. Como liberar um lock Redis corretamente?

Eu removeria a chave apenas se o valor ainda corresponder ao meu identificador, usando operação atômica. Um DELETE incondicional após expiração pode apagar o lock que outro worker adquiriu e permitir concorrência indevida entre participantes.

Termo para lembrar: Compare-and-delete | Base técnica: referência 1.

10. Como renovar o lease?

Eu renovaria somente comprovando propriedade e dentro de uma política de prazo, interrompendo novas ações quando houver incerteza. A renovação não desfaz efeitos em andamento; a proteção no recurso continua necessária para processos atrasados ou particionados.

Termo para lembrar: Lease renewal | Base técnica: referência 1.

11. Qual TTL escolher?

Eu equilibraria tempo de recuperação e duração esperada do trabalho, medindo pausas e latência, mas não trataria o TTL como prova de segurança. Trabalhos longos precisam de renovação ou divisão em etapas, além de validação no destino.

Termo para lembrar: Lease duration | Base técnica: referência 1.

12. E se o coordenador ficar indisponível?

Eu impediria novas posses quando não puder provar exclusividade, se a invariante exigir isso. Trabalho já autorizado segue apenas dentro do protocolo definido; liberar todos os workers por disponibilidade pode violar exatamente a regra que motivou o lock.

Termo para lembrar: Safety versus liveness | Base técnica: referência 1.

13. O que acontece numa partição de rede?

Um lado pode perder contato com o coordenador enquanto ainda alcança o recurso. Eu testaria esse caso explicitamente: o novo dono pode surgir do outro lado, então o destino precisa impedir que a geração antiga continue alterando estado.

Termo para lembrar: Partition behavior | Base técnica: referência 1.

14. Como um serviço com consenso ajuda?

Ele oferece uma ordem consistente para aquisição e estado do coordenador dentro de suas garantias, sacrificando progresso sem quorum. Isso melhora a coordenação, mas não transforma automaticamente chamadas externas em transações atômicas nem interrompe clientes pausados.

Termo para lembrar: Linearizable coordination | Base técnica: referência 1.

15. Redlock resolve todos esses problemas?

Eu analisaria suas hipóteses de tempo, falhas e uso, sem chamá-lo de garantia universal. Para efeito financeiro crítico, exigiria uma prova operacional compatível com o recurso e proteção final, em vez de confiar apenas no nome do algoritmo.

Termo para lembrar: Assumption-sensitive algorithm | Base técnica: referência 1.

16. Como implementar CAS no banco?

Eu atualizaria a linha com WHERE version = esperada e incrementaria a versão na mesma escrita. Se nenhuma linha mudar, releio e reavalio a decisão; repetir cegamente valores calculados sobre estado antigo pode violar a regra de negócio.

Termo para lembrar: Compare-and-swap | Base técnica: referência 1.

17. CAS resolve qualquer invariante entre várias linhas?

Não. Se a regra depende de múltiplas linhas ou da ausência de registros, um CAS isolado pode ser insuficiente. Eu escolheria constraint, bloqueios apropriados ou isolamento serializável com retry, conforme o conjunto que precisa permanecer consistente.

Termo para lembrar: Invariant scope | Base técnica: referência 1.

18. Como impedir deadlocks?

Eu adquiriria recursos em ordem determinística e manteria a seção crítica curta, evitando chamadas remotas enquanto seguro locks do banco. Ainda trataria abortos e retries, pois outras rotas e operações administrativas podem introduzir ciclos de espera inesperados.

Termo para lembrar: Lock ordering | Base técnica: referência 1.

19. Advisory lock é igual a row lock?

Não. Advisory locks coordenam participantes que obedecem ao protocolo, mas não impõem automaticamente a regra sobre linhas. Eu verificaria escopo de sessão ou transação e comportamento do pool, para não liberar ou manter locks de maneira acidental.

Termo para lembrar: Advisory coordination | Base técnica: referência 1.

20. Como lidar com pool de conexões?

Locks vinculados à sessão exigem cuidado: devolver a conexão ao pool pode manter estado ou perder associação lógica. Eu preferiria escopo transacional quando adequado e testaria o modo do pool antes de depender de afinidade entre chamadas.

Termo para lembrar: Session affinity | Base técnica: referência 1.

21. Qual perigo de fazer HTTP segurando lock?

A rede pode atrasar indefinidamente e prolongar contenção, bloqueando outros clientes enquanto o banco mantém recursos. Eu separaria estado durável e efeito assíncrono quando possível, preservando idempotência e reconciliação em vez de manter uma transação aberta.

Termo para lembrar: Long critical section | Base técnica: referência 1.

22. Lock substitui idempotência?

Não. Ele pode serializar duas tentativas, mas ambas ainda podem executar uma após a outra. Eu manteria identidade estável e registro do resultado, porque exclusividade simultânea não impede duplicação sequencial causada por retries ou redelivery.

Termo para lembrar: Concurrency versus duplication | Base técnica: referência 1.

23. Timeout ao adquirir significa que não adquiri?

Não necessariamente: a resposta pode ter se perdido após a aquisição. Eu consultaria propriedade pelo identificador quando o protocolo permitir e evitaria agir sem confirmação; tentativas ambíguas precisam de recuperação definida, não de suposição conveniente.

Termo para lembrar: Unknown acquisition outcome | Base técnica: referência 1.

24. Como proteger chamada a parceiro sem fencing?

Se o parceiro não aceita geração, eu buscaria idempotência por operação e consulta de resultado, reduzindo concorrência pelo workflow. Não consigo impor fencing externo só no meu banco; a garantia final depende também do contrato oferecido pelo parceiro.

Termo para lembrar: External-effect limitation | Base técnica: referência 1.

25. Fila por entidade elimina locks?

Ela reduz concorrência dentro da partição, mas rebalances, retries e múltiplos produtores ainda exigem cuidado. Eu manteria versão e idempotência no armazenamento; um consumidor único momentâneo não prova que nenhum worker antigo continuará tentando produzir efeitos.

Termo para lembrar: Serialized processing | Base técnica: referência 1.

26. Como evitar starvation?

Eu observaria tempo de espera e usaria política de fila ou backoff com jitter conforme o coordenador. Locks rápidos sem justiça podem favorecer alguns clientes; a solução precisa definir se ordem de chegada é requisito ou apenas desejável.

Termo para lembrar: Fairness and contention | Base técnica: referência 1.

27. Como monitorar o serviço de lock?

Eu mediria espera, duração de posse, renovações falhas, rejeições por geração e contenção por recurso. Taxa de aquisição isolada é insuficiente: muitas rejeições de fencing podem revelar pausas, rede instável ou bugs de ciclo de vida.

Termo para lembrar: Coordination SLI | Base técnica: referência 1.

28. Qual teste de falha é indispensável?

Eu pausaria o worker após adquirir, deixaria o lease expirar, executaria outro dono e retomaria o primeiro. O teste passa quando o efeito obsoleto é rejeitado no recurso, não apenas quando o coordenador mostra um novo proprietário.

Termo para lembrar: Pause-and-resume test | Base técnica: referência 1.

29. Como evitar um lock global gargalo?

Eu escolheria a menor unidade que preserva a invariante, como conta ou agregado, medindo hotspots. Dividir demais pode quebrar regras globais; manter tudo sob um lock único serializa trabalho independente e limita capacidade desnecessariamente.

Termo para lembrar: Lock granularity | Base técnica: referência 1.

30. Qual argumento principal para a banca?

Eu separaria coordenação, exclusividade do efeito e deduplicação, mostrando onde cada garantia vive. A maturidade não é escolher Redis ou etcd por preferência, mas demonstrar o que acontece com um dono antigo que volta exatamente no pior momento.

Termo para lembrar: End-to-end correctness | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Lock no banco

Ganho. Protege efeito no mesmo domínio transacional

Custo e motivo. Pode causar contenção e deadlocks

Como decidir. Quando a invariante cabe no banco

02. CAS

Ganho. Evita posse prolongada

Custo e motivo. Conflitos exigem reavaliação e retry

Como decidir. Quando versões e condição de atualização representam a regra

03. Lease curto

Ganho. Recupera dono morto rapidamente

Custo e motivo. Aumenta expirações e renovações

Como decidir. Quando pausas e custo de coordenação são considerados

04. Lease longo

Ganho. Tolera pausas maiores

Custo e motivo. Atrasa recuperação de falhas

Como decidir. Para trabalhos longos com proteção final no recurso

05. Fencing

Ganho. Rejeita gerações obsoletas

Custo e motivo. Exige suporte atômico do destino

Como decidir. Quando um dono antigo pode continuar executando

06. Coordenador com consenso

Ganho. Ordena posses de forma forte

Custo e motivo. Perde progresso sem quorum e adiciona latência

Como decidir. Para coordenação que exige essas garantias

07. Redis de coordenação

Ganho. Oferece baixa latência e simplicidade

Custo e motivo. Garantia depende de falhas e configuração

Como decidir. Para risco tolerável ou quando efeito já é protegido independentemente

08. Lock por agregado

Ganho. Permite paralelismo entre entidades

Custo e motivo. Não cobre regras entre agregados sozinho

Como decidir. Quando a fronteira de consistência está bem definida

09. Fila serial por chave

Ganho. Simplifica processamento ordenado

Custo e motivo. Hot keys limitam capacidade

Como decidir. Para fluxos cuja ordem por entidade importa

10. Retry com jitter

Ganho. Reduz disputa sincronizada

Custo e motivo. Aumenta tempo até progresso

Como decidir. Com limites e reavaliação da condição de negócio

Duas alternativas ao caminho proposto

1. Transação com constraint ou CAS

Quando faz sentido. Quando estado e efeito estão no mesmo armazenamento

Como implementar. Validar condição e aplicar mudança atomicamente no banco

Limites. Regras entre sistemas externos continuam exigindo protocolo próprio

2. Workflow serial por agregado com idempotência

Quando faz sentido. Quando operações podem ser enfileiradas

Como implementar. Particionar por entidade, registrar estado durável e rejeitar versões antigas

Limites. Consumidores antigos e efeitos externos ainda precisam de proteção; fila não é prova universal de exclusividade

Referências e alcance da evidência

1. etcd: Concurrency API

Locks e eleições vinculados a leases no etcd; proteção dos efeitos em recursos externos exige mecanismos adicionais.

https://etcd.io/docs/v3.6/dev-guide/api_concurrency_reference_v3/

2. Redis: Distributed locks

Aquisição, identificação de dono e hipóteses de locks temporários; não equivale a garantia financeira fim a fim.

https://redis.io/docs/latest/develop/clients/patterns/distributed-locks/

3. PostgreSQL: Explicit locking

Locks de linha, tabela e advisory locks, incluindo concorrência e deadlocks no banco.

https://www.postgresql.org/docs/current/explicit-locking.html

Continue em: T014 · T019 · T020

C029 / M. Os 50 cenários enviados

Otimização de custos de infraestrutura no pico da escalabilidade

Otimização de custo em escala reduz recursos por resultado útil sem violar requisitos de confiabilidade, segurança e recuperação. O foco inclui computação, dados, rede, operação e custo de falhas, não apenas preço de instância.

Quando usar. Quando crescimento aumenta custo unitário, há capacidade ociosa relevante ou diferentes classes de trabalho permitem opções de execução e armazenamento mais econômicas.

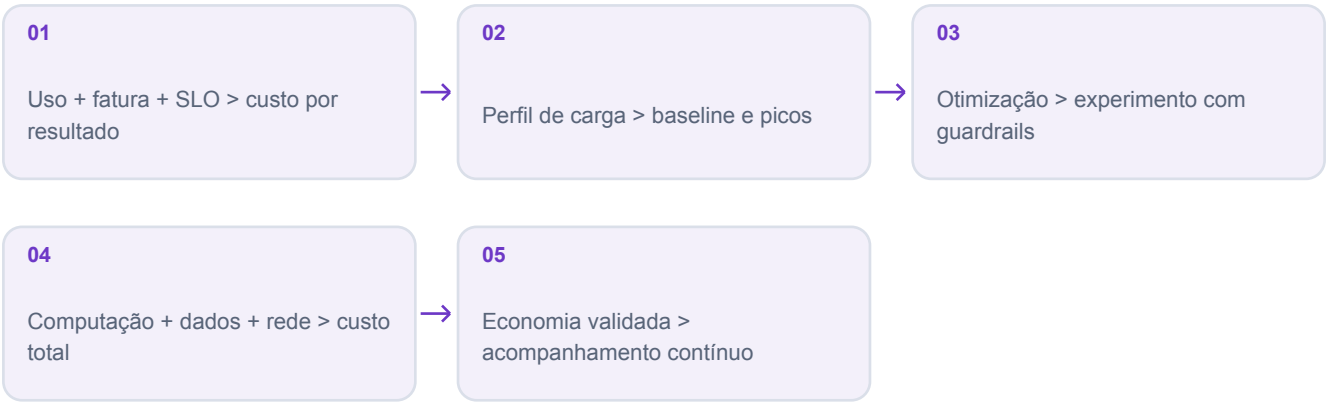
Quando evitar ou limitar. Evitar remover redundância crítica, depender de Spot como capacidade garantida ou arquivar dados antigos sem verificar acesso, retenção, recuperação e custos de leitura.

Uma abertura possível

Eu começaria medindo custo por operação útil e por jornada, separando baseline de pico. Depois atacaria desperdício e distribuição de demanda, preservando margem de recuperação. Cada economia seria validada junto de latência, risco e esforço operacional.

Peças e tecnologias. AWS Cost Explorer ou equivalente · Tags e alocação de custos · Autoscaling · EC2 Spot para tarefas tolerantes · S3 Lifecycle · Parquet · OpenTelemetry · Terraform

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Medir custos completos e atribuir responsáveis e unidades de negócio.

2. Identificar recursos ociosos, hot paths e picos previsíveis.

3. Classificar cargas e dados por tolerância a interrupção, atraso e recuperação.

4. Aplicar mudanças graduais com orçamento de risco e reversão.

5. Verificar economia líquida, SLO e efeitos sobre operação e crescimento.

30 perguntas para sustentar o desenho

01. Qual primeira métrica de custo escolher?

Eu relacionaria custo a um resultado útil, como transação concluída ou cliente ativo, separando etapas relevantes. Fatura total cresce com o negócio; custo unitário e margem de capacidade ajudam a distinguir crescimento saudável de desperdício arquitetural.

Termo para lembrar: Unit economics | Base técnica: referência 1.

02. Por que CPU média baixa não prova desperdício?

A média esconde picos, latência de cauda e capacidade necessária para perder uma zona. Eu analisaria distribuição, memória, IO e cenários de falha antes de reduzir instâncias; margem ociosa pode ser parte deliberada da confiabilidade.

Termo para lembrar: Tail-capacity planning | Base técnica: referência 1.

03. Como separar baseline de pico?

Eu mediria demanda mínima estável, picos recorrentes e eventos excepcionais, incluindo tempo para escalar. Capacidade previsível pode usar compromissos apropriados; picos precisam de elasticidade ou admissão, sem presumir que novos recursos aparecem instantaneamente quando necessários.

Termo para lembrar: Capacity decomposition | Base técnica: referência 1.

04. Quando usar Spot?

Eu usaria para trabalho tolerante a interrupção, com estado durável e retomada, como processamento de lotes. Não colocaria toda capacidade indispensável ali; preço menor não garante disponibilidade nem tempo suficiente para finalizar uma tarefa específica.

Termo para lembrar: Interruptible compute | Base técnica: referência 1.

05. Como tornar um worker apto a interrupção?

Eu dividiria trabalho em unidades pequenas, registrando progresso após efeitos confirmados e usando identidade estável. Uma interrupção repete somente o necessário; checkpoint sem idempotência pode duplicar efeitos se o resultado anterior ocorreu antes da queda.

Termo para lembrar: Checkpointed processing | Base técnica: referência 1.

06. Pode depender do aviso de interrupção?

Eu o usaria para acelerar drenagem quando disponível, mas desenharia recuperação para perda abrupta também. Processos podem falhar por outras causas; a correção não deve depender de sempre receber e processar um aviso antecipado do provedor.

Termo para lembrar: Interruption assumption | Base técnica: referência 1.

07. Basta trocar Spot por on-demand quando faltar?

Não há garantia de reposição imediata, especialmente na mesma família e zona sob pressão. Eu manteria capacidade indispensável e flexibilidade planejadas, com backlog e prazos explícitos, evitando vender fallback como disponibilidade infinita automática do provedor.

Termo para lembrar: Capacity uncertainty | Base técnica: referência 1.

08. Como economizar em jobs noturnos?

Eu executaria trabalho dentro da janela disponível, aproveitando capacidade flexível e reduzindo simultaneidade quando não há urgência. O objetivo é cumprir o prazo com custo menor, observando dependências; atrasar um lote pode bloquear relatórios e conciliação posterior.

Termo para lembrar: Deadline-aware scheduling | Base técnica: referência 1.

09. Quando autoscaling piora o custo?

Se o gargalo está no banco, adicionar aplicações aumenta conexões e retries sem elevar throughput útil. Eu identificaria saturação real e usaria métricas de fila e latência, não apenas CPU de um componente isolado.

Termo para lembrar: Scaling bottleneck | Base técnica: referência 1.

10. Qual risco de scale-to-zero?

Ele reduz custo ocioso, mas cria latência de partida e dependências de provisionamento. Eu reservaria para cargas que toleram essa espera; uma jornada crítica pode precisar de instâncias aquecidas mesmo quando o tráfego fica baixo.

Termo para lembrar: Cold-start tradeoff | Base técnica: referência 1.

11. Como otimizar o banco antes de reduzi-lo?

Eu procuraria scans desnecessários, consultas repetidas, índices inadequados e transações longas, validando planos com carga real. Melhorar o acesso pode economizar mais que diminuir a máquina e evita transformar uma economia aparente em latência elevada para todos.

Termo para lembrar: Query efficiency | Base técnica: referência 1.

12. Cache sempre economiza dinheiro?

Não. Eu compararia taxa de acerto, custo de memória, invalidação e carga no banco evitada. Um cache grande para dados pouco reutilizados pode custar mais que as leituras originais, além de adicionar operação e modos de falha.

Termo para lembrar: Cache economics | Base técnica: referência 1.

13. Como identificar custo de rede escondido?

Eu mapearia tráfego entre zonas, regiões, serviços e internet, incluindo replicação e observabilidade. Reduzir CPU pode ser irrelevante se o maior custo vem de transferir os mesmos dados repetidamente entre fronteiras tarifadas e arquiteturalmente desnecessárias.

Termo para lembrar: Data-transfer accounting | Base técnica: referência 1.

14. Vale concentrar tudo numa zona para economizar?

Eu não removeria separação de falhas sem revisar requisitos de disponibilidade e recuperação. Economia de tráfego precisa ser comparada ao impacto de perder a zona; o desenho deve otimizar dentro do risco aceito, não ignorá-lo silenciosamente.

Termo para lembrar: Failure-domain cost | Base técnica: referência 1.

15. Como escolher dados para armazenamento frio?

Eu avaliaria frequência de acesso, prazo de recuperação, tamanho e necessidade de retenção, não apenas idade. Um documento antigo ainda pode ser consultado frequentemente; classes mais baratas podem cobrar recuperação e impor latências incompatíveis com a experiência esperada.

Termo para lembrar: Access-based tiering | Base técnica: referência 1.

16. Arquivar significa apagar do banco imediatamente?

Eu primeiro copiaria, verificaria integridade e testaria recuperação, mantendo índices e referências necessários. Só removeria a origem após cumprir critérios de negócio e retenção; uma cópia existente não prova que o sistema consegue localizá-la e usá-la corretamente.

Termo para lembrar: Verified archival | Base técnica: referência 1.

17. Como servir uma fatura arquivada?

Eu manteria metadados pesquisáveis e iniciaria recuperação quando a classe exigir, mostrando estado e prazo coerentes. Se o produto exige acesso imediato a qualquer fatura, escolheria armazenamento compatível; não imporia espera inesperada apenas para reduzir a fatura técnica.

Termo para lembrar: Asynchronous retrieval | Base técnica: referência 1.

18. Parquet ajuda em quais casos?

Eu o usaria para análises que leem subconjuntos de colunas e grandes volumes, com partições e tamanhos de arquivo adequados. Ele não substitui automaticamente um banco transacional para atualizações pontuais, constraints ou consultas de baixa latência por entidade.

Termo para lembrar: Columnar analytics | Base técnica: referência 1.

19. Como evitar milhões de arquivos minúsculos?

Eu agruparia eventos em arquivos de tamanho apropriado ao padrão de consulta, com compactação controlada. Arquivos pequenos elevam operações, metadados e planejamento; arquivos enormes também podem limitar paralelismo, então a escolha precisa ser medida na carga real.

Termo para lembrar: Small-file compaction | Base técnica: referência 1.

20. Qual custo da observabilidade?

Eu alocaria orçamento por sinal e utilidade, limitando cardinalidade, retenção e logs repetitivos. Cortar tudo prejudica diagnóstico e recuperação; a economia deve preservar evidências essenciais e medir o custo operacional de incidentes mais difíceis de investigar.

Termo para lembrar: Telemetry budget | Base técnica: referência 1.

21. Como avaliar um compromisso de capacidade?

Eu o aplicaria à demanda estável e previsível, considerando horizonte, crescimento e mudanças tecnológicas. Desconto nominal não significa economia se a arquitetura muda e a capacidade contratada fica ociosa; compararia utilização esperada e flexibilidade oferecida.

Termo para lembrar: Commitment risk | Base técnica: referência 1.

22. Serverless é sempre mais barato?

Não. Eu compararia duração, memória, frequência, picos e custos adjacentes de rede e serviços. Serverless pode reduzir operação e ociosidade, enquanto carga constante intensa pode favorecer outra opção; a decisão depende do custo total e dos requisitos.

Termo para lembrar: Workload-dependent pricing | Base técnica: referência 1.

23. Como medir economia de uma refatoração?

Eu compararia custo por unidade útil em janelas e cargas equivalentes, incluindo efeitos sobre falhas e operação. Uma queda na fatura durante menor tráfego não prova melhoria; também preciso considerar esforço de implementação e manutenção futura.

Termo para lembrar: Controlled cost experiment | Base técnica: referência 1.

24. Como evitar otimização local prejudicial?

Eu somaria computação, armazenamento, rede, licenças e trabalho da equipe, incluindo recuperação. Migrar para uma tecnologia aparentemente barata pode exigir especialistas e plantões; a decisão precisa considerar o sistema completo e o horizonte de uso esperado.

Termo para lembrar: Total cost of ownership | Base técnica: referência 1.

25. Qual papel de tags e ownership?

Eu identificaria serviço, ambiente e responsável para atribuir consumo e discutir prioridades. Tags não reduzem gasto sozinhas, mas tornam recursos órfãos e anomalias acionáveis; sem dono, a recomendação de economia costuma ficar sem execução ou avaliação.

Termo para lembrar: Cost allocation | Base técnica: referência 1.

26. Como controlar ambientes de desenvolvimento?

Eu aplicaria quotas, desligamento quando ocioso e TTL em ambientes efêmeros, preservando exceções justificadas. Essa economia costuma ter risco menor que reduzir redundância produtiva, mas precisa evitar apagar dados de investigação ou interromper trabalho sem política clara.

Termo para lembrar: Resource lifecycle | Base técnica: referência 1.

27. Como incluir recuperação no orçamento?

Eu dimensionaria capacidade e armazenamento para cenários de recuperação escolhidos, testando restauração e retomada de backlog. Operar permanentemente no limite pode ser barato no dia normal e incapaz de absorver qualquer falha ou reconstrução necessária.

Termo para lembrar: Resilience reserve | Base técnica: referência 1.

28. O que fazer quando o custo dispara?

Eu correlacionaria a mudança com tráfego, deploys, cardinalidade, retries e abuso, aplicando limites onde seguro. Não desligaria automaticamente componentes críticos só por orçamento; uma resposta financeira precisa preservar obrigações e distinguir incidente de crescimento legítimo do produto.

Termo para lembrar: Cost anomaly response | Base técnica: referência 1.

29. Como apresentar uma recomendação à liderança?

Eu apresentaria economia estimada, investimento, prazo de retorno e riscos mensuráveis, com alternativas. Isso permite escolher conscientemente; uma lista de instâncias menores sem impacto sobre usuários e recuperação não sustenta uma decisão de arquitetura responsável.

Termo para lembrar: Decision economics | Base técnica: referência 1.

30. Qual conclusão defenderia na entrevista?

Eu buscaria menor custo por resultado dentro dos requisitos, começando por desperdício comprovado e preservando margem de falha. Staff não significa sempre usar Spot ou cold storage; significa demonstrar quando essas escolhas realmente compensam e quais compromissos introduzem.

Termo para lembrar: Cost-aware reliability | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Rightsizing

Ganho. Reduz capacidade sem uso útil

Custo e motivo. Pode diminuir margem para picos e falhas

Como decidir. Após análise de cauda, IO, memória e recuperação

02. Spot

Ganho. Reduz custo de trabalho flexível

Custo e motivo. Pode interromper e não estar disponível

Como decidir. Para tarefas retomáveis com prazo tolerante

03. Capacidade comprometida

Ganho. Pode reduzir custo do baseline

Custo e motivo. Cria obrigação e risco de ociosidade

Como decidir. Quando demanda estável e horizonte justificam

04. Autoscaling

Ganho. Acompanha variação de demanda

Custo e motivo. Tem atraso e pode amplificar gargalos

Como decidir. Com métrica ligada à capacidade útil e limites corretos

05. Scale-to-zero

Ganho. Evita custo em períodos ociosos

Custo e motivo. Adiciona cold start

Como decidir. Para jornadas que toleram inicialização

06. Cache

Ganho. Evita leituras repetidas caras

Custo e motivo. Consome memória e operação

Como decidir. Quando reutilização medida supera custo de manter o cache

07. Tiering de dados

Ganho. Reduz custo de retenção

Custo e motivo. Pode cobrar recuperação e aumentar espera

Como decidir. Conforme frequência de acesso e compromisso do produto

08. Compactação de arquivos

Ganho. Reduz operações e melhora análise

Custo e motivo. Consome processamento e atrasa disponibilização

Como decidir. Quando tamanho e padrão de consulta justificam

09. Retenção menor de logs

Ganho. Reduz armazenamento e indexação

Custo e motivo. Pode limitar investigação histórica

Como decidir. Com política baseada em utilidade e obrigações reais

10. Serviço gerenciado

Ganho. Reduz esforço operacional

Custo e motivo. Pode aumentar preço direto e dependência

Como decidir. Quando custo total e competências favorecem delegar operação

Duas alternativas ao caminho proposto

1. Otimização na arquitetura atual

Quando faz sentido. Quando há desperdício mensurável antes de uma migração

Como implementar. Corrigir queries, limites, retenção e recursos órfãos com experimentos

Limites. Economias têm teto e não resolvem um modelo estruturalmente inadequado

2. Redesenho por classes de serviço

Quando faz sentido. Quando cargas têm requisitos muito diferentes

Como implementar. Separar caminhos críticos, batch e histórico com SLO e custo próprios

Limites. Aumenta complexidade e exige medir benefício líquido e operação adicional

Referências e alcance da evidência

1. AWS: Cost Optimization Pillar

Otimização vinculada a requisitos e resultados; estimativas e decisões específicas são análises autorais, não preços ou promessas de economia.

https://docs.aws.amazon.com/wellarchitected/latest/cost-optimization-pillar/welcome.html

2. AWS: EC2 Spot best practices

Interrupções, flexibilidade e ausência de garantia de capacidade Spot; não assegura capacidade substituta imediata.

https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/spot-best-practices.html

3. AWS: S3 object lifecycle

Transições e expiração de objetos com custos e condições; retenção de dados do negócio precisa ser definida separadamente.

https://docs.aws.amazon.com/AmazonS3/latest/userguide/object-lifecycle-mgmt.html

Continue em: T002 · T003 · T024

C030 / M. Os 50 cenários enviados

Criação de um Internal Developer Platform (IDP)

Uma Internal Developer Platform oferece capacidades reutilizáveis de entrega e operação como produto interno, com autosserviço, contratos e ownership. Portal e templates são interfaces possíveis, não a plataforma inteira.

Quando usar. Quando equipes repetem trabalho operacional, dependem de tickets para tarefas comuns e precisam de caminhos seguros e sustentáveis para entregar serviços.

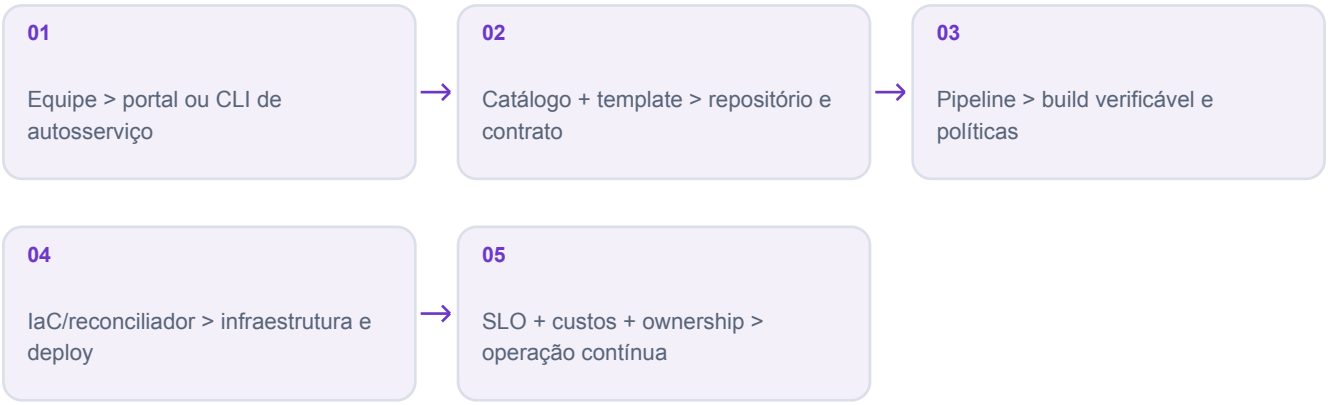
Quando evitar ou limitar. Evitar construir uma plataforma universal antes de entender usuários, centralizar toda autonomia ou afirmar que SAST, DAST e infraestrutura como código garantem conformidade por si sós.

Uma abertura possível

Eu começaria por uma jornada concreta, como publicar um serviço com identidade, observabilidade e rollback. Entregaria um caminho recomendado mensurável, com escape controlado, e evoluiria a plataforma pelo uso real das equipes, não pelo número de ferramentas instaladas.

Peças e tecnologias. Backstage · Terraform · GitHub Actions ou equivalente · Argo CD quando aplicável · OpenTelemetry · OIDC workload identity · OPA/Gatekeeper · SBOM e provenance

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Pesquisar jornadas e medir gargalos recorrentes das equipes consumidoras.

2. Criar um caminho inicial com contrato, dono e limites explícitos.

3. Automatizar provisionamento, build e deploy com identidade e evidência.

4. Operar a própria plataforma com SLO, suporte e recuperação.

5. Medir adoção, tempo para valor e risco, evoluindo padrões e exceções.

30 perguntas para sustentar o desenho

01. IDP é instalar Backstage?

Não. Backstage pode organizar catálogo e experiência, mas as capacidades reais incluem provisionamento, entrega, identidade e operação. Eu começaria pelas necessidades das equipes; um portal bonito sobre processos manuais lentos não resolve o gargalo principal de desenvolvimento.

Termo para lembrar: Platform product | Base técnica: referência 1.

02. Qual seria o primeiro caso de uso?

Eu escolheria uma jornada frequente e dolorosa, como criar e publicar um serviço com padrões mínimos. Entregaria o fluxo completo para poucas equipes, medindo tempo e falhas antes de ampliar escopo para diferentes workloads e ambientes.

Termo para lembrar: Thin-slice platform | Base técnica: referência 1.

03. Como descobrir o que automatizar?

Eu observaria trabalho real, tickets e tempo de espera, conversando com consumidores de diferentes equipes. Pedidos isolados podem representar exceções; priorizaria capacidades repetidas cujo custo e risco justificam uma solução compartilhada com suporte contínuo.

Termo para lembrar: Developer research | Base técnica: referência 1.

04. O que significa paved road?

É um caminho recomendado e suportado que reduz decisões repetitivas, não uma proibição de alternativas. Eu definiria quais garantias oferece, quais workloads atende e como uma equipe solicita exceção quando seus requisitos não cabem no padrão disponível.

Termo para lembrar: Golden path | Base técnica: referência 1.

05. Template copiado é suficiente?

Ele acelera o início, mas pode envelhecer em centenas de repositórios independentes. Eu combinaria bibliotecas ou módulos versionados, automação de upgrades e ownership, para que correções de segurança e operação não dependam de copiar novamente um projeto inteiro.

Termo para lembrar: Lifecycle ownership | Base técnica: referência 1.

06. Como evitar uma plataforma burocrática?

Eu automatizaria tarefas comuns dentro de limites claros e reservaria revisão humana para riscos relevantes. Se cada criação exige vários tickets, apenas deslocamos o gargalo; o usuário precisa saber o que pode fazer autonomamente e receber feedback imediato.

Termo para lembrar: Self-service boundaries | Base técnica: referência 1.

07. Qual informação entra no catálogo?

Eu registraria responsável, domínio, criticidade, dependências e links operacionais, mantendo atualização próxima do código quando possível. Catálogo sem dados confiáveis vira diretório decorativo; a informação deve ajudar incidentes, mudanças e descoberta de capacidades realmente existentes.

Termo para lembrar: Service ownership | Base técnica: referência 1.

08. Como manter metadados atualizados?

Eu coletaria informações de fontes autorizadas e verificaria inconsistências, com responsável por correção. Campos declarados manualmente podem divergir do runtime; a plataforma precisa distinguir intenção e realidade em vez de apresentar tudo como igualmente confiável aos usuários.

Termo para lembrar: Catalog reconciliation | Base técnica: referência 1.

09. Como desenhar a API de autosserviço?

Eu permitiria declarar intenção, como serviço e classe de armazenamento, devolvendo estado e identificador da operação. Provisionamento pode demorar ou falhar parcialmente; a API precisa expor progresso, repetir com segurança e permitir reconciliação de recursos já criados.

Termo para lembrar: Declarative platform API | Base técnica: referência 1.

10. Como evitar recursos duplicados num retry?

Eu atribuiria identidade estável à solicitação e reconciliaria recursos pelo estado desejado, verificando o que já existe. Reexecutar um workflow após timeout não deve criar outro banco ou outra conta sem reconhecer a execução anterior parcialmente concluída.

Termo para lembrar: Idempotent provisioning | Base técnica: referência 1.

11. O que fazer após falha parcial?

Eu registraria etapas e recursos criados, retomando ou compensando conforme a segurança da ação. Apagar tudo automaticamente pode destruir dados; a plataforma deve diferenciar recurso vazio descartável de um ambiente que já recebeu uso legítimo da equipe.

Termo para lembrar: Reconciliation loop | Base técnica: referência 1.

12. Terraform resolve sozinho o ciclo de vida?

Ele ajuda a declarar e aplicar infraestrutura, mas ainda preciso de estado protegido, revisão, identidade, limites e recuperação. Um plan correto não prova que a mudança atende requisitos de produto ou que todas as dependências operacionais foram consideradas.

Termo para lembrar: Infrastructure workflow | Base técnica: referência 1.

13. Como proteger credenciais do pipeline?

Eu preferiria identidades temporárias vinculadas ao contexto de execução, com privilégios mínimos e separação por ambiente. Segredos administrativos permanentes no CI ampliam o impacto de comprometimento; a plataforma deve impedir que qualquer repositório obtenha acesso produtivo indiscriminadamente.

Termo para lembrar: Workload identity | Base técnica: referência 1.

14. Como tratar segredos em templates?

Eu geraria referências e políticas de acesso, não valores secretos embutidos no código produzido. Rotação, revogação e auditoria fazem parte do serviço; um template seguro no primeiro dia ainda pode se tornar inseguro sem manutenção posterior.

Termo para lembrar: Secret lifecycle | Base técnica: referência 1.

15. O que SAST e DAST garantem?

Eles detectam classes de problemas conforme ferramentas e configuração, mas não provam ausência de vulnerabilidade. Eu combinaria análise, revisão proporcional ao risco e testes relevantes, mantendo evidência de cobertura e exceções em vez de prometer conformidade automática.

Termo para lembrar: Security coverage | Base técnica: referência 1.

16. Qual papel de policy as code?

Eu codificaria restrições verificáveis, como exposição de rede e imagens permitidas, com mensagens acionáveis. Políticas precisam de testes e rollout; uma regra errada distribuída globalmente pode bloquear entregas ou recuperação durante incidentes importantes para a organização.

Termo para lembrar: Automated guardrails | Base técnica: referência 1.

17. Como introduzir uma política nova?

Eu começaria em modo de observação, mediria violações e ajudaria equipes a corrigir antes de bloquear. Exceções têm dono e prazo; ativar rejeição imediata sem avaliar impacto pode interromper serviços ou impedir correções urgentes em produção.

Termo para lembrar: Progressive enforcement | Base técnica: referência 1.

18. Como tratar exceções ao caminho padrão?

Eu permitiria justificativa, responsável, controles compensatórios e revisão com prazo, preservando visibilidade. Proibir toda exceção incentiva soluções escondidas; aceitar qualquer uma para sempre dissolve os benefícios da plataforma e torna suporte impossível de sustentar operacionalmente.

Termo para lembrar: Managed exceptions | Base técnica: referência 1.

19. Como verificar origem do artefato?

Eu ligaria artefato ao código, processo e identidade de build com evidência verificável e política de consumo. Provenance ajuda a rastrear origem, mas não demonstra correção funcional nem ausência de vulnerabilidades no conteúdo que foi compilado.

Termo para lembrar: Build provenance | Base técnica: referência 1.

20. Por que gerar SBOM?

Eu usaria inventário de componentes para localizar dependências afetadas e acelerar resposta a vulnerabilidades. A lista precisa acompanhar o artefato efetivo; gerar um arquivo sem integração com investigação e atualização não reduz risco automaticamente para os serviços publicados.

Termo para lembrar: Dependency inventory | Base técnica: referência 1.

21. Como a plataforma ajuda observabilidade?

Eu entregaria instrumentação, dashboards e alertas iniciais com guias de adaptação ao domínio. Defaults reduzem trabalho, mas a equipe dona define sinais de negócio; um painel genérico de CPU não identifica todas as falhas relevantes para clientes.

Termo para lembrar: Operational defaults | Base técnica: referência 1.

22. Quem atende um incidente do serviço criado?

Eu definiria responsabilidades entre equipe do produto e plataforma, incluindo escalonamento e dependências. A plataforma responde pelas capacidades que oferece, enquanto regras e operação do domínio continuam com seu dono; indefinição nesse ponto prolonga incidentes reais.

Termo para lembrar: Shared responsibility | Base técnica: referência 1.

23. Qual SLO a plataforma precisa?

Eu definiria objetivos separados para provisionamento, entrega e capacidades de runtime, porque seus impactos diferem. Uma indisponibilidade do portal não deveria derrubar serviços já executando; o desenho precisa limitar essa dependência e documentar alternativas operacionais seguras.

Termo para lembrar: Platform reliability | Base técnica: referência 1.

24. Como evitar que o portal seja ponto único de falha?

Eu manteria serviços em execução independentes da interface de autosserviço e disponibilizaria caminhos operacionais autorizados para recuperação. A queda do controle pode impedir novas mudanças, mas não deveria interromper automaticamente todas as transações dos produtos que dependem da plataforma.

Termo para lembrar: Control-plane separation | Base técnica: referência 1.

25. Como medir sucesso sem contar templates?

Eu mediria tempo até primeira entrega, espera por infraestrutura, falhas de mudança e satisfação dos usuários, junto de riscos operacionais. Quantidade de componentes criados pode crescer enquanto a experiência piora; o resultado precisa representar trabalho efetivamente facilitado.

Termo para lembrar: Developer outcomes | Base técnica: referência 1.

26. Como lidar com diferentes linguagens e bancos?

Eu começaria com combinações prioritárias e níveis explícitos de suporte, evoluindo conforme demanda. Suportar tudo desde o início dilui qualidade; o catálogo deve mostrar o que é plenamente operado, experimental ou responsabilidade adicional da equipe consumidora.

Termo para lembrar: Supported capability tiers | Base técnica: referência 1.

27. Como distribuir custos entre equipes?

Eu associaria recursos a responsáveis e unidades de consumo, oferecendo estimativas antes do provisionamento e acompanhamento depois. Transparência ajuda decisões; cobrança interna mal desenhada pode incentivar economias locais que prejudicam confiabilidade ou duplicam infraestrutura compartilhada necessária.

Termo para lembrar: Platform cost transparency | Base técnica: referência 1.

28. Como atualizar componentes de centenas de serviços?

Eu manteria versões e inventário, automatizando propostas de atualização com testes e rollout progressivo. Mudanças universais precisam de compatibilidade e rollback; centralizar um módulo reduz trabalho, mas também aumenta o alcance de um erro distribuído para toda a frota.

Termo para lembrar: Fleet lifecycle management | Base técnica: referência 1.

29. Quando comprar em vez de construir?

Eu compararia diferenciação necessária, integrações, custo total e capacidade de operação da equipe. Compraria componentes comuns quando fizer sentido, mantendo contratos e saída viável; construir tudo internamente pode consumir o tempo que deveria melhorar a experiência dos desenvolvedores.

Termo para lembrar: Build versus buy | Base técnica: referência 1.

30. Qual mensagem final demonstra maturidade?

Eu defenderia uma plataforma adotada porque resolve problemas reais, com limites, suporte e métricas claros. Padronização é um meio para reduzir trabalho e risco; não substitui entendimento do domínio nem justifica impor a mesma arquitetura a todos os produtos.

Termo para lembrar: Platform as an internal product | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Golden path

Ganho. Reduz decisões repetitivas e erros comuns

Custo e motivo. Não atende todos os workloads

Como decidir. Para jornadas frequentes com requisitos compartilhados

02. Templates

Ganho. Aceleram criação inicial

Custo e motivo. Podem divergir e envelhecer

Como decidir. Com estratégia de atualização e módulos reutilizáveis

03. Módulos centrais

Ganho. Propagam melhorias com menos duplicação

Custo e motivo. Ampliam impacto de regressões

Como decidir. Com versões, compatibilidade e rollout gradual

04. Autosserviço

Ganho. Reduz espera por tickets

Custo e motivo. Exige limites e feedback confiável

Como decidir. Para operações recorrentes com risco controlável

05. Políticas bloqueantes

Ganho. Impedem configurações proibidas

Custo e motivo. Podem travar entregas e recuperação

Como decidir. Após validação, adoção assistida e exceções governadas

06. Modo de auditoria

Ganho. Revela impacto antes de impor

Custo e motivo. Não impede violações imediatamente

Como decidir. Durante implantação de regras e levantamento de legado

07. Portal único

Ganho. Facilita descoberta e experiência

Custo e motivo. Pode concentrar dependências de controle

Como decidir. Com APIs e separação do runtime

08. Catálogo declarativo

Ganho. Aproxima ownership do código

Custo e motivo. Pode divergir do ambiente real

Como decidir. Com coleta e reconciliação contínuas

09. Amplo suporte tecnológico

Ganho. Atende equipes diversas

Custo e motivo. Eleva custo e reduz profundidade

Como decidir. Quando demanda e equipe sustentam níveis explícitos de suporte

10. Compra de componentes

Ganho. Acelera capacidades comuns

Custo e motivo. Introduz dependência e limita customização

Como decidir. Quando custo total e contratos favorecem adoção externa

Duas alternativas ao caminho proposto

1. Enablement enxuto com módulos e documentação

Quando faz sentido. Quando a organização ainda tem poucas equipes ou demandas incertas

Como implementar. Oferecer bibliotecas, IaC e apoio direto antes de construir um portal

Limites. Autosserviço e visibilidade podem se tornar insuficientes com crescimento

2. Plataforma gerenciada integrada

Quando faz sentido. Quando capacidades comuns atendem a maior parte das necessidades

Como implementar. Configurar serviços externos com identidade, políticas e integração ao fluxo existente

Limites. Custo, portabilidade e limites de customização precisam ser avaliados; ownership permanece interno

Referências e alcance da evidência

1. Backstage: Software Catalog

Catálogo de componentes, metadados e ownership; a plataforma completa envolve outros serviços e processos.

https://backstage.io/docs/features/software-catalog/

2. Backstage: Software Templates

Criação de componentes a partir de modelos e ações; não garante operação segura ou conformidade automaticamente.

https://backstage.io/docs/features/software-templates/

3. HashiCorp: Terraform core workflow

Fluxo de escrita, planejamento e aplicação de infraestrutura declarativa; governança e reconciliação são decisões adicionais.

https://developer.hashicorp.com/terraform/intro/core-workflow

4. SLSA: Provenance 1.2

Metadados verificáveis sobre origem e processo de build; não comprovam ausência de vulnerabilidades ou correção funcional.

https://slsa.dev/spec/v1.2/provenance

5. Gatekeeper: Introduction

Aplicação de políticas em recursos Kubernetes; limites de cobertura e exceções dependem da configuração e governança.

https://open-policy-agent.github.io/gatekeeper/website/docs/

Continue em: T005 · T006 · T010

C031 / M. Os 50 cenários enviados

Arquitetura Analítica: Data Mesh vs Data Lakehouse

Projetar uma plataforma analítica exige separar organização dos domínios, formato de tabelas e motores de processamento. Data Mesh e Lakehouse podem coexistir.

Quando usar. Análises de transações, risco e produto com múltiplos produtores, volumes grandes e requisitos de qualidade.

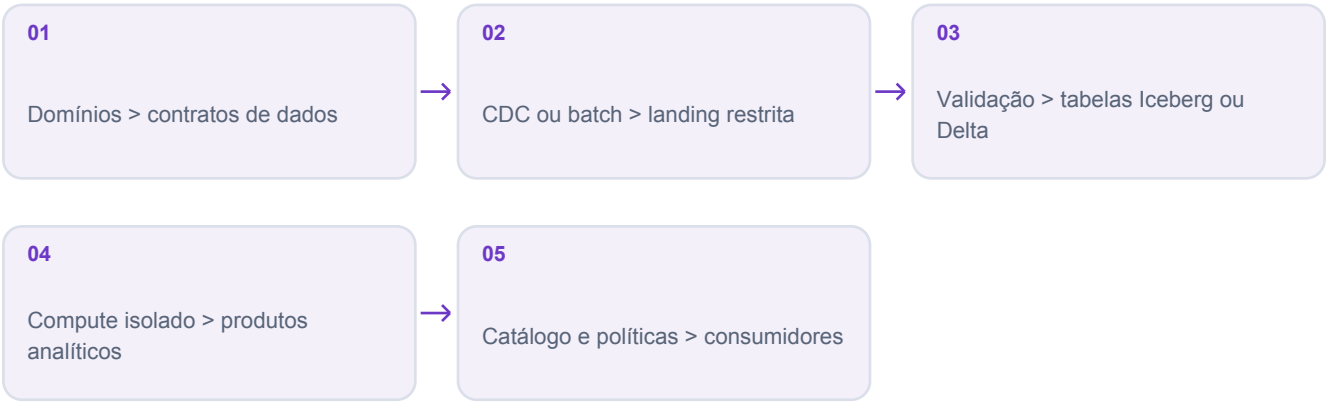
Quando evitar ou limitar. Escolher Mesh ou Lakehouse como selo de senioridade ou consultar o ledger operacional com scans analíticos pesados.

Uma abertura possível

Eu perguntaria quem consome os dados, qual frescor precisa e quem responde pela qualidade. Depois separaria ownership, armazenamento e execução, sem tratar Mesh e Lakehouse como rivais.

Peças e tecnologias. Apache Iceberg · Delta Lake · Spark · Trino · S3 · ADLS · Catálogo de dados

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Definir consumidores, frescor, qualidade e responsáveis

2. Escolher ingestão por fonte sem sobrecarregar OLTP

3. Publicar tabelas versionadas com validação e lineage

4. Isolar compute e aplicar políticas de acesso

5. Testar replay, correção, exclusão e recuperação do catálogo

30 perguntas para sustentar o desenho

01. Mesh substitui Lakehouse?

Eu separaria as dimensões: Data Mesh organiza ownership e produtos de dados, enquanto Lakehouse descreve capacidades de armazenamento e análise. Posso usar tabelas Iceberg numa plataforma compartilhada com responsabilidades federadas entre domínios.

Termo para lembrar: Data Mesh | Base técnica: referência 1.

02. Qual primeira pergunta ao negócio?

Eu perguntaria se o caso exige decisão em segundos, relatório diário ou exploração histórica. Freshness SLO orienta ingestão e custo; não construiria streaming contínuo para uma necessidade que tolera atualização noturna.

Termo para lembrar: Freshness SLO | Base técnica: referência 2.

03. Como evita afetar o ledger?

Eu extraio mudanças por um mecanismo suportado ou exportações controladas, isolando compute analítico. Workload isolation impede que scans extensos disputem recursos críticos com autorização de pagamentos e gravações transacionais do sistema operacional.

Termo para lembrar: Workload isolation | Base técnica: referência 2.

04. Quem é dono do dado de Pix?

Eu atribuiria ao domínio responsável pela semântica do evento, com contrato e suporte definidos. Data product ownership inclui qualidade, documentação e evolução; publicar arquivos num bucket não transforma automaticamente dados em produto confiável.

Termo para lembrar: Data product | Base técnica: referência 1.

05. O que contém o contrato?

Eu definiria schema, significado dos campos, unidade monetária, identidade, frescor e política de correção. Data contract deve explicar como interpretar cancelamentos e ajustes, evitando que cada consumidor invente sua própria definição de receita ou saldo.

Termo para lembrar: Data contract | Base técnica: referência 1.

06. Por que formato de tabela aberto?

Eu avaliaria Iceberg ou Delta para snapshots, evolução e integração com motores compatíveis. Table format oferece metadata e protocolo além dos arquivos, mas preciso verificar funcionalidades concretas entre versões e engines escolhidas.

Termo para lembrar: Table format | Base técnica: referência 3.

07. Parquet sozinho oferece transação de tabela?

Não. Arquivos colunares ajudam leitura e compressão, mas uma tabela precisa coordenar quais arquivos formam um snapshot válido. Atomic table commit depende do catálogo e protocolo, não apenas da extensão dos objetos armazenados.

Termo para lembrar: Atomic table commit | Base técnica: referência 3.

08. Como lida com escritores concorrentes?

Eu uso o mecanismo de commit e validação do formato, tratando conflitos com retry adequado. Optimistic concurrency evita publicar metadata inconsistente, mas conflitos semânticos exigem verificar a operação e não apenas repetir indefinidamente.

Termo para lembrar: Optimistic concurrency | Base técnica: referência 3.

09. Pode assumir transação entre todas as tabelas?

Não sem suporte explícito do engine e catálogo utilizados. Eu documento a fronteira de atomicidade e uso publication watermark ou versões de produto para consumidores que precisam combinar datasets publicados de forma coerente.

Termo para lembrar: Publication watermark | Base técnica: referência 3.

10. Como escolhe partição analítica?

Eu considero filtros frequentes, distribuição e tamanho dos arquivos, como data de evento com ajustes apropriados. Partition pruning reduz leitura, mas particionar por cliente indiscriminadamente pode gerar milhões de arquivos e metadata excessiva.

Termo para lembrar: Partition pruning | Base técnica: referência 2.

11. Como resolve small files?

Eu monitoro tamanho e quantidade de arquivos e executo compaction controlada, preservando snapshots necessários. Small files aumentam overhead de planejamento e acesso; ingestão muito granular pode comprometer custo mesmo quando o volume total parece modesto.

Termo para lembrar: Small files | Base técnica: referência 2.

12. Como distingue bronze, silver e gold?

Eu usaria camadas apenas se expressarem responsabilidades claras de captura, validação e consumo. Medallion architecture é uma organização possível; nomes bonitos não substituem contratos, qualidade nem ownership de cada transformação publicada.

Termo para lembrar: Medallion architecture | Base técnica: referência 2.

13. Como trata evento atrasado?

Eu registro event time e ingestion time e defino política de reabertura ou correção das agregações. Late-arriving data exige comunicar quando um relatório pode mudar, especialmente em fechamentos e análises financeiras.

Termo para lembrar: Late-arriving data | Base técnica: referência 2.

14. Como evita duplicar transação na análise?

Eu mantenho identidade de negócio e versão, aplicando deduplicação conforme a semântica da fonte. Analytical deduplication diferencia reentrega de uma nova movimentação; igualdade de valor e horário não é uma chave segura.

Termo para lembrar: Analytical deduplication | Base técnica: referência 3.

15. Como corrige dado historicamente errado?

Eu publico uma correção versionada e recomputo derivados afetados com lineage conhecido. Data restatement precisa preservar explicação da mudança, para que consumidores entendam por que um indicador histórico mudou após uma reconciliação.

Termo para lembrar: Data restatement | Base técnica: referência 1.

16. Como controla acesso a CPF?

Eu aplico políticas no catálogo e motores, com mascaramento ou projeções mínimas e controle dos arquivos subjacentes. Column-level security perde efeito se o consumidor puder contornar o engine e ler diretamente o storage.

Termo para lembrar: Column-level security | Base técnica: referência 2.

17. Como evita vazamento por joins?

Eu avalio combinações de datasets e finalidade, não somente permissões isoladas de colunas. Reidentification risk pode surgir quando dados aparentemente limitados são unidos; catálogo e revisão de acesso precisam considerar esse contexto.

Termo para lembrar: Reidentification risk | Base técnica: referência 1.

18. Como mede qualidade de um produto?

Eu defino regras de completude, unicidade, validade e reconciliação com a origem, com responsável e alerta. Data quality SLO deve detectar impacto no consumidor, e não apenas confirmar que o job terminou sem exceção.

Termo para lembrar: Data quality SLO | Base técnica: referência 1.

19. Como distingue zero de dado ausente?

Eu publico status de completude e watermark junto ao dataset. Missing data não pode virar valor zero silenciosamente, pois isso pode alterar indicadores de fraude, receita ou inadimplência sem que o consumidor perceba.

Termo para lembrar: Completeness watermark | Base técnica: referência 1.

20. Como recupera pipeline parado?

Eu retomo de checkpoint e reprocesso intervalos com identidade preservada, verificando retenção da origem. Replay window precisa cobrir a falha; se dados expiraram, uso novo snapshot e reconcilio o ponto de corte.

Termo para lembrar: Replay window | Base técnica: referência 3.

21. Como controla custo de consultas?

Eu separo pools, imponho quotas e acompanho bytes lidos e tempo de execução por domínio. Compute governance permite exploração sem deixar uma consulta exploratória consumir toda a capacidade destinada a relatórios críticos.

Termo para lembrar: Compute governance | Base técnica: referência 2.

22. Lakehouse elimina warehouse?

Não necessariamente. Eu comparo concorrência, latência, ferramentas e maturidade operacional. Um warehouse pode continuar sendo a melhor camada de serving; arquitetura analítica deve atender consumidores sem transformar uma preferência tecnológica em obrigação.

Termo para lembrar: Analytical serving | Base técnica: referência 2.

23. Como escolhe batch ou streaming?

Eu comparo frescor necessário com custo de operação e correção. Batch processing simplifica muitos relatórios, enquanto streaming atende decisões rápidas; posso combinar ambos e reconciliar resultados sem declarar que um é sempre superior.

Termo para lembrar: Batch processing | Base técnica: referência 2.

24. Como versiona mudanças incompatíveis?

Eu publico nova versão do produto e ofereço janela de migração com validação dos consumidores. Schema evolution técnica não resolve mudança de significado; renomear ou reinterpretar um campo exige comunicação e contrato explícito.

Termo para lembrar: Schema evolution | Base técnica: referência 1.

25. Como apaga informação pessoal histórica?

Eu identifico snapshots, arquivos, derivados e caches afetados e sigo a política aplicável. Data lifecycle precisa considerar que excluir uma linha da versão atual não remove automaticamente todas as versões históricas e cópias.

Termo para lembrar: Data lifecycle | Base técnica: referência 3.

26. Quem mantém plataforma self-service?

Eu definiria uma equipe de plataforma que oferece catálogo, templates e controles, enquanto domínios respondem pela semântica. Self-service platform reduz duplicação, mas precisa ser um produto utilizável, com suporte e limites claros.

Termo para lembrar: Self-service platform | Base técnica: referência 1.

27. Como evita cada domínio criar seu padrão?

Eu estabeleço padrões mínimos de identidade, contratos, segurança e publicação com governança federada. Federated governance mantém interoperabilidade sem centralizar toda decisão de transformação numa equipe que desconhece os detalhes de cada domínio.

Termo para lembrar: Federated governance | Base técnica: referência 1.

28. Como testa recuperação do catálogo?

Eu ensaio restaurar metadata e referências a objetos, verificando snapshots e permissões antes de liberar consultas. Catalog recovery é crítico: arquivos intactos não bastam se a plataforma perdeu o mapa consistente das tabelas.

Termo para lembrar: Catalog recovery | Base técnica: referência 3.

29. Qual seria a evolução inicial?

Eu começaria por poucos produtos prioritários, contratos claros e infraestrutura proporcional, medindo uso e qualidade. Incremental adoption evita impor uma reorganização completa antes de comprovar que autonomia federada resolve um problema real.

Termo para lembrar: Incremental adoption | Base técnica: referência 1.

30. O que diferencia a resposta madura?

Eu justificaria frescor, ownership, garantias e custo, incluindo quando um warehouse central simples basta. Architecture fit é mais importante que mencionar Mesh, Spark ou Iceberg sem explicar o problema e a operação resultante.

Termo para lembrar: Architecture fit | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Mesh federado

Ganho. Ownership próximo ao domínio

Custo e motivo. Coordenação e maturidade

Como decidir. Quando centralização virou gargalo real

02. Equipe analítica central

Ganho. Uniformidade e simplicidade

Custo e motivo. Fila de demandas

Como decidir. Para poucos domínios ou fase inicial

03. Lakehouse

Ganho. Storage flexível e vários engines

Custo e motivo. Catálogo e manutenção

Como decidir. Para workloads variados e escala analítica

04. Warehouse

Ganho. Serving e operação integrados

Custo e motivo. Custo e dependência

Como decidir. Para BI com concorrência previsível

05. Streaming

Ganho. Frescor baixo

Custo e motivo. Operação e correções contínuas

Como decidir. Quando decisão exige segundos

06. Batch

Ganho. Simplicidade e custo

Custo e motivo. Maior atraso

Como decidir. Quando relatório tolera janela

07. Partição granular

Ganho. Pruning específico

Custo e motivo. Small files e metadata

Como decidir. Quando filtros e volume justificam

08. Compaction frequente

Ganho. Leitura eficiente

Custo e motivo. Compute adicional

Como decidir. Quando arquivos pequenos afetam consultas

09. Snapshots longos

Ganho. Reprodutibilidade

Custo e motivo. Storage e retenção sensível

Como decidir. Com política e utilidade demonstradas

10. Compute isolado

Ganho. Proteção entre consumidores

Custo e motivo. Capacidade reservada

Como decidir. Para workloads críticos concorrentes

Duas alternativas ao caminho proposto

1. Warehouse central gerenciado

Quando faz sentido. BI predominante e equipe menor

Como implementar. Ingerir fontes controladas e publicar modelos governados

Limites. Menos autonomia e possível gargalo organizacional

2. Produtos analíticos por domínio sem Lakehouse

Quando faz sentido. Necessidades heterogêneas

Como implementar. Cada domínio publica APIs ou datasets com contratos comuns

Limites. Interoperabilidade e governança exigem disciplina adicional

Referências e alcance da evidência

1. Zhamak Dehghani - Data Mesh Principles

Princípios de ownership por domínio, dados como produto, plataforma e governança federada.

https://martinfowler.com/articles/data-mesh-principles.html

2. Databricks - Data lakehouse

Arquitetura analítica, processamento separado do armazenamento e camadas de dados.

https://docs.databricks.com/aws/en/lakehouse/

3. Apache Iceberg - Reliability

Snapshots, commits atômicos de tabelas e conflitos entre escritores.

https://iceberg.apache.org/docs/latest/reliability/

Continue em: T075 · T077 · C034

C032 / M. Os 50 cenários enviados

Direito ao Esquecimento (LGPD/GDPR) em sistemas imutáveis

Atender solicitações de eliminação em sistemas imutáveis exige políticas por classe, inventário de cópias e mecanismos verificáveis. Crypto-shredding é uma opção condicionada, não uma solução jurídica universal.

Quando usar. Eventos históricos, dados pessoais replicados e necessidade de preservar fatos financeiros legitimamente retidos.

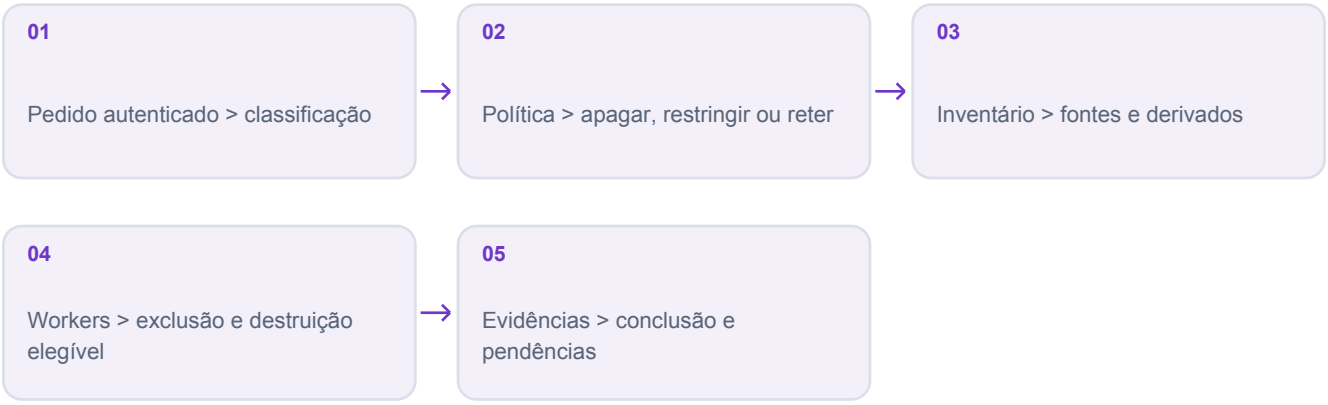
Quando evitar ou limitar. Prometer apagar todo histórico financeiro ou declarar conformidade apenas destruindo uma chave sem verificar cópias e exceções.

Uma abertura possível

Eu separaria fatos financeiros de atributos pessoais e desenharia a eliminação como workflow durável. As regras aplicáveis seriam validadas pelos responsáveis, e cada cópia teria um caminho verificável de tratamento.

Peças e tecnologias. KMS · Envelope encryption · S3 Object Lock · Catálogo de dados · Workflow durável

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Validar identidade, escopo e política aplicável

2. Mapear dados pessoais, chaves e cópias derivadas

3. Executar ações idempotentes por destino autorizado

4. Registrar exceções e verificar descarte efetivo

5. Testar restauração e impedir ressurreição de dados eliminados

30 perguntas para sustentar o desenho

01. Você apagaria tudo ao receber o pedido?

Eu primeiro classificaria o pedido e as bases de conservação aplicáveis com os responsáveis. Retention policy diferencia dados elimináveis de registros que precisam de tratamento distinto; não assumo que o direito de eliminação seja absoluto.

Termo para lembrar: Retention policy | Base técnica: referência 1.

02. LGPD e GDPR são a mesma regra?

Não. Eu trataria jurisdição e contexto como entradas da política, sem transpor automaticamente condições de uma norma para outra. Policy mapping é validado por especialistas; a arquitetura implementa decisões aprovadas e registra evidências verificáveis.

Termo para lembrar: Policy mapping | Base técnica: referência 2.

03. Como autentica quem pede exclusão?

Eu verifico identidade e poderes de representação proporcionalmente ao risco, sem coletar informação excessiva. Request authentication impede que um atacante apague dados de terceiros e mantém o pedido vinculado ao sujeito correto.

Termo para lembrar: Request authentication | Base técnica: referência 1.

04. Como separa evento financeiro de PII?

Eu colocaria no evento somente referências e fatos mínimos, mantendo atributos pessoais em um domínio controlado. Data minimization reduz propagação, mas a referência ainda pode ser dado pessoal quando permite associação ou reidentificação.

Termo para lembrar: Data minimization | Base técnica: referência 1.

05. Imutável significa impossível de apagar?

Imutabilidade é uma propriedade do modelo ou da política de armazenamento, não uma licença para ignorar requisitos de ciclo de vida. Storage immutability precisa ser desenhada junto com retenção, exceções e possibilidades técnicas de descarte.

Termo para lembrar: Storage immutability | Base técnica: referência 4.

06. Como funciona crypto-shredding?

Eu cifro dados com chaves cuja destruição torne as cópias relevantes indecifráveis, após validar elegibilidade. Crypto-shredding só sustenta essa garantia técnica se não existirem outras chaves, plaintext ou caminhos alternativos de recuperação.

Termo para lembrar: Crypto-shredding | Base técnica: referência 3.

07. Uma chave por cliente é sempre melhor?

Ela permite granularidade de descarte, mas aumenta gestão e exige mapear dados compartilhados. Key granularity deve corresponder ao escopo de eliminação; uma chave compartilhada entre clientes impede apagar um sem afetar os demais.

Termo para lembrar: Key granularity | Base técnica: referência 3.

08. Envelope encryption muda essa análise?

Sim. Eu rastreio DEKs, versões e suas cópias cifradas, além das chaves que podem desembrulhá-las. Envelope encryption cria uma hierarquia; destruir uma referência sem eliminar outros caminhos de unwrap pode não impedir recuperação.

Termo para lembrar: Envelope encryption | Base técnica: referência 3.

09. Rotacionar chave equivale a apagar?

Não. Rotação costuma preservar capacidade de ler dados antigos, conforme configuração. Key rotation e destruição têm objetivos diferentes; eu verifico versões e material ainda acessível antes de afirmar que um dado tornou-se irrecuperável.

Termo para lembrar: Key rotation | Base técnica: referência 3.

10. Desabilitar chave é destruição definitiva?

Não necessariamente. Desabilitação pode ser reversível e serviços podem ter janelas ou processos próprios de eliminação. Key lifecycle precisa ser observado no provedor escolhido, sem confundir bloqueio temporário de acesso com destruição concluída.

Termo para lembrar: Key lifecycle | Base técnica: referência 3.

11. Como trata cache com plaintext?

Eu invalido ou expiro caches relevantes e verifico logs e dumps que possam conter o conteúdo. Plaintext copies continuam legíveis independentemente da chave original, portanto precisam de mecanismos próprios de descarte e controle de acesso.

Termo para lembrar: Plaintext copies | Base técnica: referência 3.

12. Como trata índice de busca?

Eu envio uma obrigação de remoção por identidade e verifico resultado, incluindo índices antigos e reconstruções. Search deletion não pode depender somente de excluir a base principal, pois a projeção possui retenção e lifecycle independentes.

Termo para lembrar: Search deletion | Base técnica: referência 1.

13. Como trata data lake histórico?

Eu localizo snapshots, arquivos e produtos derivados afetados, aplicando a política por cópia. Data lineage permite planejar reescrita, expiração ou outras medidas; excluir apenas a tabela atual pode deixar versões antigas acessíveis.

Termo para lembrar: Data lineage | Base técnica: referência 1.

14. E se o backup não permite remoção seletiva?

Eu documento a política de expiração e controlo qualquer restauração, conforme decisão aprovada. Backup retention não deve ser ocultada do desenho; o sistema precisa impedir uso indevido e ressurreição de dados já tratados.

Termo para lembrar: Backup retention | Base técnica: referência 1.

15. Como impede ressurreição no restore?

Eu reaplico um registro protegido de exclusões e restrições antes de reabrir acesso à base restaurada. Restore gate verifica a política vigente, porque o snapshot representa um passado em que o dado ainda podia existir.

Termo para lembrar: Restore gate | Base técnica: referência 1.

16. Como mantém prova sem manter PII desnecessária?

Eu registro identidade de solicitação, escopo técnico, decisão e evidências mínimas de execução. Deletion audit precisa comprovar o processo sem recriar, no próprio log, todo o conteúdo pessoal que o workflow deveria eliminar.

Termo para lembrar: Deletion audit | Base técnica: referência 1.

17. Um hash do CPF resolve anonimização?

Eu não assumiria isso: identificadores previsíveis podem ser recuperados por tentativa ou correlacionados entre bases. Pseudonymization reduz exposição, mas anonimização depende do risco de reidentificação no contexto, não apenas de aplicar uma função hash.

Termo para lembrar: Pseudonymization | Base técnica: referência 1.

18. Como trata dados compartilhados por duas pessoas?

Eu modelo ownership e finalidade por atributo, preservando direitos e obrigações de outros envolvidos. Shared data exige decisão específica; destruir uma chave comum ou remover o registro inteiro pode prejudicar informações que precisam permanecer.

Termo para lembrar: Shared data | Base técnica: referência 1.

19. Como funciona legal hold no fluxo?

Eu registro exceção autorizada com escopo, fundamento informado pelos responsáveis e revisão. Legal hold bloqueia a ação pertinente, mas não deve suspender indiscriminadamente o tratamento de dados sem relação com a necessidade de conservação.

Termo para lembrar: Legal hold | Base técnica: referência 4.

20. Como lida com destino indisponível?

Eu mantenho a tarefa pendente de forma durável, com retries limitados e escalonamento por prazo. Deletion obligation não é concluída por envio da mensagem; preciso de confirmação verificável do tratamento naquele destino.

Termo para lembrar: Deletion obligation | Base técnica: referência 1.

21. Como torna exclusão idempotente?

Eu uso request_id e sujeito estável, registrando conclusão por destino e versão da política. Idempotent deletion tolera repetição sem apagar dados de uma nova identidade que reutilizou um identificador técnico antigo.

Termo para lembrar: Idempotent deletion | Base técnica: referência 1.

22. Como trata recriação legítima de conta?

Eu separo identidade histórica e nova relação, com regras aprovadas para os dados novos. Identity lifecycle impede que uma tarefa atrasada de exclusão alcance um cadastro posterior apenas porque email ou telefone foram reutilizados.

Termo para lembrar: Identity lifecycle | Base técnica: referência 1.

23. Como evita corrida com uma exportação?

Eu verifico restrições vigentes no início e antes de disponibilizar o artefato, revogando acessos quando necessário. Authorization freshness precisa acompanhar o workflow; uma exportação longa não pode ignorar uma restrição aprovada durante sua geração.

Termo para lembrar: Authorization freshness | Base técnica: referência 1.

24. Pode confiar no parceiro que recebeu dados?

Eu registro destinatários e obrigações contratuais, propagando solicitações pelo canal definido e acompanhando evidências. Downstream propagation torna a dependência visível; não prometo controle físico direto sobre sistemas de terceiros que não opero.

Termo para lembrar: Downstream propagation | Base técnica: referência 1.

25. Como decide manter estatística agregada?

Eu avalio se a agregação ainda permite identificar pessoas e qual finalidade sustenta seu uso. Aggregation não garante anonimização automaticamente, especialmente em grupos pequenos ou combináveis com outros dados disponíveis ao consumidor.

Termo para lembrar: Aggregation | Base técnica: referência 1.

26. Como testa o mecanismo?

Eu crio sujeitos sintéticos rastreáveis, distribuo dados pelos destinos e executo o fluxo completo, incluindo restore. Deletion verification precisa buscar resíduos e caminhos alternativos, não apenas conferir que cada worker retornou sucesso.

Termo para lembrar: Deletion verification | Base técnica: referência 1.

27. Qual SLO operacional escolheria?

Eu alinharia prazos aplicáveis com responsáveis e definiria metas internas menores, acompanhando idade das pendências. Deletion SLO mede tempo até tratamento verificável por destino, com exceções explícitas, sem inventar um prazo legal universal.

Termo para lembrar: Deletion SLO | Base técnica: referência 1.

28. Como protege contra destruição maliciosa de chaves?

Eu aplico separação de funções, revisão e controles de acesso proporcionais ao impacto. Key destruction é uma operação potencialmente irreversível; a mesma credencial de leitura cotidiana não deveria possuir poder irrestrito de eliminar chaves.

Termo para lembrar: Key destruction | Base técnica: referência 3.

29. E quando o sistema legado não suporta?

Eu explicito a limitação e desenho controles compensatórios e plano de migração com responsáveis. Technical limitation precisa aparecer na análise e no acompanhamento; não renomeio bloqueio de acesso como exclusão física concluída.

Termo para lembrar: Technical limitation | Base técnica: referência 1.

30. O que corrigiria na proposta simplificada?

Eu diria que crypto-shredding é um mecanismo possível dentro de um processo maior. End-to-end deletion exige política, inventário, cópias, chaves, backups e evidência; nenhuma técnica isolada permite declarar automaticamente conformidade completa.

Termo para lembrar: End-to-end deletion | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Referência pessoal externa

Ganho. Menos PII no log

Custo e motivo. Dependência para enriquecimento

Como decidir. Quando eventos não precisam do atributo

02. PII cifrada no evento

Ganho. Payload autocontido protegido

Custo e motivo. Gestão de chaves e cópias

Como decidir. Quando necessidade está demonstrada

03. Chave por sujeito

Ganho. Descarte granular

Custo e motivo. Muitas chaves e referências

Como decidir. Para escopo individual consistente

04. Chave compartilhada

Ganho. Operação simples

Custo e motivo. Descarte seletivo difícil

Como decidir. Para dados com lifecycle comum

05. Exclusão física

Ganho. Remove cópia acessível

Custo e motivo. Custo e limites do storage

Como decidir. Quando suportada e apropriada

06. Crypto-shredding

Ganho. Inacessibilidade por chave

Custo e motivo. Premissas fortes sobre cópias

Como decidir. Quando todas as dependências são verificadas

07. WORM

Ganho. Protege evidência

Custo e motivo. Impede remoção antecipada

Como decidir. Com retenção aprovada antes da gravação

08. Registro de exclusões

Ganho. Evita ressurreição

Custo e motivo. Retenção de metadata sensível

Como decidir. Com acesso mínimo e escopo definido

09. Workflow assíncrono

Ganho. Coordena muitos destinos

Custo e motivo. Conclusão não imediata

Como decidir. Com estado durável e prazo monitorado

10. Auditoria detalhada

Ganho. Comprova execução

Custo e motivo. Pode reter dados demais

Como decidir. Com minimização deliberada

Duas alternativas ao caminho proposto

1. Separação de PII em cofre mutável

Quando faz sentido. Eventos precisam só de referência

Como implementar. Guardar atributos em serviço controlado e remover vínculos elegíveis

Limites. Referências e derivados ainda exigem avaliação

2. Reescrita e expiração controlada de arquivos

Quando faz sentido. Storage permite substituir segmentos

Como implementar. Reconstruir dados sem atributos elegíveis e expirar versões conforme política

Limites. Custo alto e necessidade de cobrir snapshots e backups

Referências e alcance da evidência

1. ANPD - Direitos dos titulares

Direitos de titulares e referências a hipóteses de conservação previstas na LGPD.

https://www.gov.br/anpd/pt-br/assuntos/titular-de-dados-1/direito-dos-titulares

2. EDPB - Data protection FAQ

Direitos de titulares e caráter não absoluto do direito de apagamento.

https://www.edpb.europa.eu/sme/find-practical-info/faq_en

3. AWS - KMS keys

Hierarquia, identificação e controle de chaves KMS.

https://docs.aws.amazon.com/kms/latest/developerguide/concepts.html

4. AWS - S3 Object Lock

Retenção e proteção WORM de versões de objetos.

https://docs.aws.amazon.com/AmazonS3/latest/userguide/object-lock.html

Continue em: T073 · T075 · C031 · C034

C033 / M. Os 50 cenários enviados

Engenharia do Caos e Testes em Produção

Experimentos de falha verificam hipóteses de resiliência com escopo, observabilidade e parada definidos. Produção é uma escolha de risco, não um requisito de senioridade.

Quando usar. Validar recuperação de pagamentos, filas e dependências após testes seguros e preparação operacional.

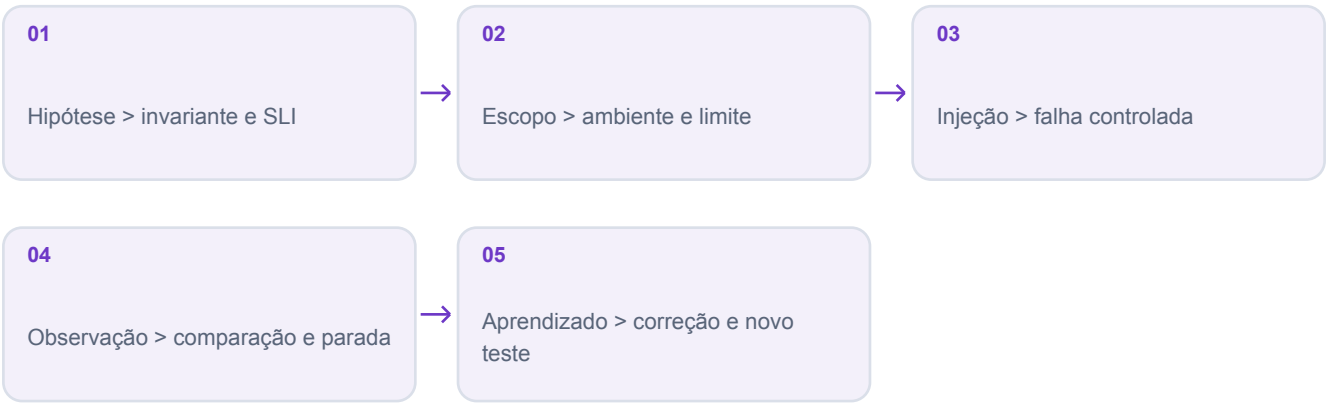
Quando evitar ou limitar. Injetar falhas aleatórias continuamente, espelhar efeitos financeiros ou usar rollback como promessa de desfazer dados.

Uma abertura possível

Eu escolheria uma hipótese concreta, como perder um worker após o commit, e mostraria a evidência que provaria ou refutaria nossa recuperação antes de ampliar o experimento.

Peças e tecnologias. Chaos Mesh · k6 · OpenTelemetry · Canary · Feature flags · Shadow traffic

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Escolher invariante e falha específica

2. Validar observabilidade e mecanismo de interrupção

3. Ensaiar em ambiente controlado com dados sintéticos

4. Executar escopo autorizado com limites automáticos

5. Registrar resultados e transformar lacunas em ações verificáveis

30 perguntas para sustentar o desenho

01. Começaria derrubando produção?

Não. Eu começaria pela hipótese e pelo ambiente menos arriscado que produza evidência útil. Chaos experiment em produção exige maturidade e autorização proporcionais; estágio controlado pode revelar o mesmo defeito com impacto muito menor.

Termo para lembrar: Chaos experiment | Base técnica: referência 1.

02. Qual hipótese usaria em chargeback?

Eu testaria se um worker morto após persistir a obrigação de envio retoma sem criar nova intenção financeira. Recovery invariant relaciona falha, estado durável e efeito esperado, evitando um experimento genérico sem critério de sucesso.

Termo para lembrar: Recovery invariant | Base técnica: referência 1.

03. Como define estado saudável?

Eu escolho SLIs de conclusão, latência e integridade do negócio antes da injeção. Steady state não é apenas CPU baixa: o sistema pode responder normalmente e ainda deixar solicitações financeiras presas sem progresso.

Termo para lembrar: Steady state | Base técnica: referência 2.

04. Como limita o experimento?

Eu restrinjo identidade sintética, namespace, número de instâncias, duração e caminhos de rede. Blast radius é uma configuração verificável, com proteção contra seleção ampla acidental e dependências compartilhadas que ampliem o impacto.

Termo para lembrar: Blast radius | Base técnica: referência 1.

05. Qual condição dispara parada?

Eu defino limites de erro, latência, backlog e qualquer violação de invariante, com abort automático e operador responsável. Stop condition precisa funcionar mesmo se o mecanismo de injeção perder comunicação com seu controlador.

Termo para lembrar: Stop condition | Base técnica: referência 1.

06. Como testa queda do banco?

Eu simulo indisponibilidade limitada e observo recusa, buffering e recuperação, sem alterar dados reais indiscriminadamente. Dependency failure deve verificar pressão de retries e comportamento do cliente, além de a aplicação voltar a conectar.

Termo para lembrar: Dependency failure | Base técnica: referência 1.

07. Como testa perda de resposta do parceiro?

Eu permito que o efeito remoto ocorra e removo a resposta no ambiente controlado. Ambiguous outcome verifica se o sistema consulta estado ou reconcilia pela mesma identidade, em vez de reenviar como uma operação nova.

Termo para lembrar: Ambiguous outcome | Base técnica: referência 1.

08. Como testa Kafka duplicado?

Eu reentrego eventos conhecidos e reinicio consumidores em pontos de commit distintos. Duplicate delivery deve manter um único efeito local e obrigações corretas; métricas de consumo sozinhas não demonstram idempotência de negócio.

Termo para lembrar: Duplicate delivery | Base técnica: referência 1.

09. Como testa eventos fora de ordem?

Eu atraso uma versão e entrego a seguinte, observando deferimento, recuperação de lacunas e alarmes. Sequence gap não deve ser tratado como duplicata concluída se o evento ainda é necessário para o estado correto.

Termo para lembrar: Sequence gap | Base técnica: referência 1.

10. Como testa fencing?

Eu suspendo o worker antigo até expirar sua posse, inicio outro e retomo o primeiro. Fencing validation precisa ocorrer no recurso protegido; observar apenas que o lock mudou de dono não comprova rejeição da escrita obsoleta.

Termo para lembrar: Fencing validation | Base técnica: referência 1.

11. Como testa saturação sem derrubar tudo?

Eu aumento carga gradualmente num escopo isolado, mantendo margem e limites de parada. Load shedding deve recusar trabalho de forma controlada antes de esgotar recursos, preservando fluxos prioritários e capacidade de recuperação.

Termo para lembrar: Load shedding | Base técnica: referência 2.

12. Qual diferença de stress e caos?

Stress testing explora capacidade e comportamento sob carga; chaos engineering injeta falhas para testar hipóteses de resiliência. Posso combiná-los, mas separo variáveis inicialmente para entender qual condição realmente causou a degradação observada.

Termo para lembrar: Stress testing | Base técnica: referência 1.

13. Como usa shadow traffic?

Eu espelho entradas para uma versão isolada e comparo decisões ou respostas, bloqueando efeitos externos. Shadow traffic não pode enviar pagamentos, emails ou arquivos reais novamente, mesmo que a resposta sombra nunca chegue ao cliente.

Termo para lembrar: Shadow traffic | Base técnica: referência 3.

14. Dados reais podem ir ao shadow?

Eu aplico minimização, autorização e isolamento compatíveis com a finalidade do teste, preferindo dados sintéticos quando possível. Shadow data handling deve considerar retenção e acesso; copiar tráfego também copia potencialmente informações pessoais e segredos.

Termo para lembrar: Shadow data handling | Base técnica: referência 3.

15. Como compara respostas não determinísticas?

Eu normalizo campos irrelevantes e comparo propriedades semânticas, como valores e permissões, dentro de tolerâncias explícitas. Differential testing precisa distinguir diferença esperada de regressão, sem ignorar campos financeiros apenas porque variam frequentemente.

Termo para lembrar: Differential testing | Base técnica: referência 3.

16. Canary e shadow são equivalentes?

Não. Canary atende uma fração real do tráfego, enquanto shadow executa um caminho paralelo sem controlar a resposta principal. Canary analysis mede impacto real, mas expõe usuários do escopo escolhido ao comportamento da versão nova.

Termo para lembrar: Canary analysis | Base técnica: referência 3.

17. Como usa error budget?

Eu relaciono a tolerância de falhas do serviço ao risco de mudanças e experimentos, com política acordada. Error budget não é licença para causar dano financeiro; invariantes de integridade podem exigir limites diferentes de disponibilidade.

Termo para lembrar: Error budget | Base técnica: referência 2.

18. Rollback automático é sempre seguro?

Não. Eu verifico compatibilidade de schema, eventos e efeitos já produzidos antes de automatizar. Rollback safety depende do tipo de mudança; reverter binário não desfaz um débito externo nem recupera dados removidos.

Termo para lembrar: Rollback safety | Base técnica: referência 3.

19. Como testa o próprio abort?

Eu executo um ensaio curto e verifico remoção da falha, liberação de recursos e retorno ao baseline. Kill switch não deve existir apenas no documento; precisa ser testado e acessível aos operadores responsáveis.

Termo para lembrar: Kill switch | Base técnica: referência 1.

20. Como protege dependências compartilhadas?

Eu identifico bancos, NAT, filas e quotas compartilhadas antes de escolher o escopo. Dependency mapping evita que um experimento aparentemente local consuma toda a capacidade de um componente usado por vários serviços críticos.

Termo para lembrar: Dependency mapping | Base técnica: referência 2.

21. Como evita confundir incidente real com experimento?

Eu registro janela, identificador e mudanças aplicadas, mantendo monitoramento normal e comunicação operacional. Experiment tagging facilita correlação, mas não deve silenciar alertas de impacto inesperado nem presumir que toda falha durante a janela foi intencional.

Termo para lembrar: Experiment tagging | Base técnica: referência 2.

22. Como mede recuperação?

Eu marco início da falha, detecção, mitigação e retomada do resultado de negócio. Recovery time inclui drenagem do backlog e reconciliação, não apenas o momento em que os pods voltaram a responder health checks.

Termo para lembrar: Recovery time | Base técnica: referência 2.

23. Como testa falta de capacidade após failover?

Eu removo uma zona simulada e mantenho carga representativa no restante, com limites seguros. Capacity failover verifica headroom e quotas; redundância nominal não garante que as instâncias sobreviventes suportem o volume exigido.

Termo para lembrar: Capacity failover | Base técnica: referência 1.

24. Como testa corrupção de mensagem?

Eu introduzo payload inválido controlado e verifico quarentena, isolamento por agregado e recuperação após correção. Poison-message testing demonstra se um único registro pode bloquear toda a partição ou provocar repetição infinita sem progresso.

Termo para lembrar: Poison-message testing | Base técnica: referência 1.

25. Como seleciona experimentos prioritários?

Eu uso histórico de incidentes, risco financeiro, mudanças recentes e dependências frágeis. Risk-based experimentation concentra esforço onde uma falha plausível ameaça invariantes importantes, em vez de maximizar a quantidade de falhas diferentes injetadas.

Termo para lembrar: Risk-based experimentation | Base técnica: referência 4.

26. O que faz se o teste revelar defeito?

Eu interrompo conforme critério, preservo evidências e atribuo ação com dono e validação posterior. Learning loop só fecha quando a correção é demonstrada; uma apresentação sobre o experimento não elimina a vulnerabilidade encontrada.

Termo para lembrar: Learning loop | Base técnica: referência 4.

27. Como evita culpabilizar equipe?

Eu descrevo condições, decisões e mecanismos que permitiram a falha, preservando responsabilidade por correções. Blameless analysis favorece informação útil sem transformar o exercício em julgamento individual ou desculpa para não executar ações concretas.

Termo para lembrar: Blameless analysis | Base técnica: referência 4.

28. Qual frequência de testes escolheria?

Eu a ajustaria ao risco, mudanças e capacidade operacional, automatizando casos maduros. Experiment cadence não precisa ser contínua em produção; frequência alta sem análise e correção pode apenas repetir riscos conhecidos sem aprendizado.

Termo para lembrar: Experiment cadence | Base técnica: referência 1.

29. Como apresenta resultado negativo?

Eu diria que a hipótese foi refutada, mostraria o limite observado e proporia a correção e novo ensaio. Falsifiable hypothesis torna um teste que encontra falha valioso, desde que o escopo tenha protegido o negócio.

Termo para lembrar: Falsifiable hypothesis | Base técnica: referência 1.

30. O que demonstra nível Staff aqui?

Eu conectaria hipótese, segurança, evidência e melhoria operacional, escolhendo ferramentas proporcionalmente. Resilience validation é julgamento de engenharia; usar Chaos Mesh em produção não é automaticamente superior a um ensaio bem desenhado em ambiente controlado.

Termo para lembrar: Resilience validation | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Staging

Ganho. Menor impacto

Custo e motivo. Fidelidade parcial

Como decidir. Para primeira validação e riscos altos

02. Produção limitada

Ganho. Comportamento real

Custo e motivo. Exposição de usuários

Como decidir. Com maturidade e autorização

03. Falha única

Ganho. Causa mais clara

Custo e motivo. Menos combinações

Como decidir. Para entender mecanismo inicial

04. Falhas combinadas

Ganho. Explora cascatas

Custo e motivo. Diagnóstico e risco maiores

Como decidir. Após validar falhas isoladas

05. Shadow

Ganho. Comparação sem resposta ao cliente

Custo e motivo. Custo e isolamento

Como decidir. Com efeitos externos bloqueados

06. Canary

Ganho. Evidência de uso real

Custo e motivo. Impacta pequena população

Como decidir. Com métricas e recuperação definidas

07. Abort automático

Ganho. Resposta rápida

Custo e motivo. Pode parar por ruído

Como decidir. Com sinais confiáveis

08. Operador dedicado

Ganho. Julgamento contextual

Custo e motivo. Custo humano e tempo

Como decidir. Para experimentos novos ou críticos

09. Teste frequente

Ganho. Detecta regressões

Custo e motivo. Carga operacional

Como decidir. Quando mecanismo está maduro

10. Rollback automático

Ganho. Mitigação rápida

Custo e motivo. Pode incompatibilizar estado

Como decidir. Somente para mudanças reversíveis validadas

Duas alternativas ao caminho proposto

1. Simulação determinística de falhas

Quando faz sentido. Protocolos críticos e ambiente sensível

Como implementar. Controlar tempo, rede e interleavings num modelo ou harness

Limites. Pode omitir detalhes reais da infraestrutura

2. Game day operacional

Quando faz sentido. Validar pessoas e runbooks

Como implementar. Ensaiar cenários e decisões com falhas limitadas ou simuladas

Limites. Menos cobertura automática e dependência da participação

Referências e alcance da evidência

1. Chaos Mesh - Overview

Experimentos de falha controlados e mecanismos de simulação.

https://chaos-mesh.org/docs/

2. Google SRE - Monitoring

Sinais operacionais, sintomas e monitoramento orientado ao serviço.

https://sre.google/sre-book/monitoring-distributed-systems/

3. Google SRE Workbook - Canarying releases

Avaliação incremental e redução de risco em lançamentos.

https://sre.google/workbook/canarying-releases/

4. Google SRE - Postmortem culture

Aprendizado sem culpabilização e ações de melhoria após incidentes.

https://sre.google/sre-book/postmortem-culture/

Continue em: T064 · T065 · T069 · T076

C034 / M. Os 50 cenários enviados

Motor de busca para Histórico de Transações de milhões de clientes

Busca de transações é uma projeção autorizada e reconstruível, com atraso explícito em relação à fonte financeira.

Quando usar. Pesquisar descrições, estabelecimentos e filtros combinados em históricos extensos de clientes.

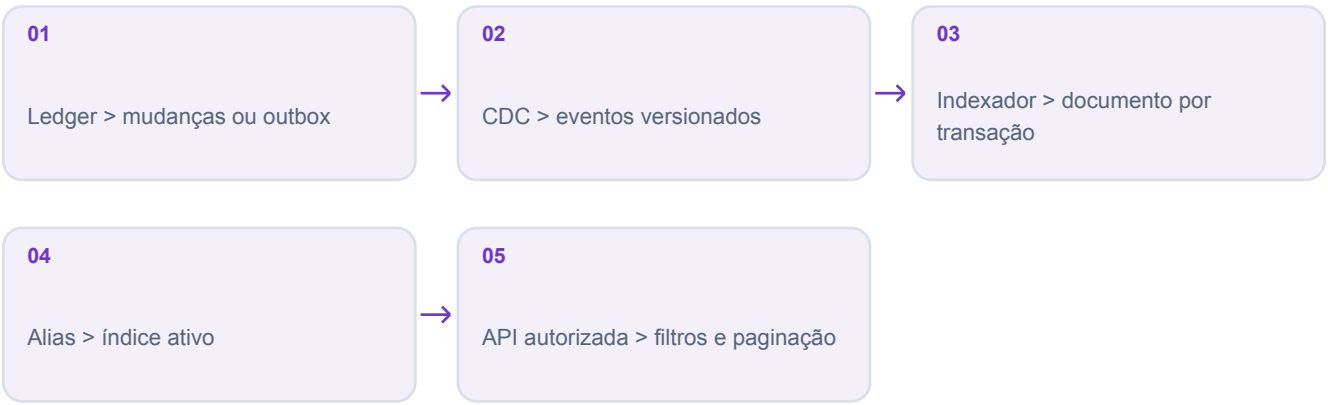
Quando evitar ou limitar. Usar resultado de busca como saldo autoritativo ou expor documentos sem autorização por conta e tenant.

Uma abertura possível

Eu começaria pelos filtros e pelo frescor esperado. Manteria o ledger como autoridade e construiria uma projeção de busca com identidade, versão, acesso por cliente e reconstrução verificável.

Peças e tecnologias. PostgreSQL · Debezium · Kafka · Elasticsearch · OpenSearch · S3

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Definir filtros, volume, frescor e contrato de autorização

2. Capturar mudanças e manter posição recuperável

3. Indexar por identidade estável e versão do domínio

4. Servir consultas limitadas por cliente e cursor

5. Reconstruir índice paralelo e trocar alias após validação

30 perguntas para sustentar o desenho

01. Precisa de Elasticsearch desde o início?

Eu verificaria se índices relacionais e busca textual existente atendem ao volume e aos filtros. Search engine especializado agrega relevância e flexibilidade, mas introduz projeção, atraso e operação que precisam justificar o custo.

Termo para lembrar: Search engine | Base técnica: referência 3.

02. Qual é a fonte de verdade?

Eu manteria o sistema transacional como autoridade financeira e trataria busca como read model. Uma transação ausente no índice pode indicar lag; não concluiria que ela não existe nem recalcularia saldo autoritativo pela busca.

Termo para lembrar: Read model | Base técnica: referência 1.

03. Qual documento indexaria?

Eu usaria um documento por transação ou visão claramente definida, com transaction_id, account_id, versão, datas e campos necessários. Document identity estável permite reentrega e atualização sem criar múltiplos registros para o mesmo fato.

Termo para lembrar: Document identity | Base técnica: referência 3.

04. Como captura mudanças sem dual write?

Eu usaria CDC ou outbox conforme o contrato da fonte, acompanhando posição e recuperação. Change data capture reduz a janela de escrever banco e índice separadamente, mas o consumidor ainda precisa tolerar repetição e atrasos.

Termo para lembrar: Change data capture | Base técnica: referência 4.

05. CDC expõe semântica de negócio automaticamente?

Não. Ele observa mudanças de armazenamento, que podem não representar eventos estáveis de domínio. Semantic projection exige tradução e contrato; renomear uma tabela não deveria quebrar silenciosamente todos os consumidores de histórico financeiro.

Termo para lembrar: Semantic projection | Base técnica: referência 4.

06. Como faz carga inicial?

Eu estabeleço snapshot e ponto de corte compatível com mudanças concorrentes, depois alcanço o fluxo incremental. Bootstrap consistency evita perder transações ocorridas durante a cópia ou substituir documentos novos com versões antigas do snapshot.

Termo para lembrar: Bootstrap consistency | Base técnica: referência 4.

07. Como trata eventos duplicados?

Eu faço upsert por identidade estável com controle de versão ou semântica equivalente. Idempotent indexing evita duplicatas físicas, mas também precisa impedir que uma versão atrasada sobrescreva uma correção já aplicada ao documento.

Termo para lembrar: Idempotent indexing | Base técnica: referência 4.

08. Como trata exclusões?

Eu propago tombstones ou ações explícitas e preservo informação suficiente para evitar ressurreição durante replay. Delete propagation inclui índices antigos e reconstruções; apagar só o índice ativo deixa cópias e caminhos de retorno.

Termo para lembrar: Delete propagation | Base técnica: referência 4.

09. Refresh é confirmação financeira?

Não. Refresh torna mudanças disponíveis à busca conforme configuração; não é a confirmação do ledger. Search visibility e commit financeiro são marcos diferentes, e o cliente precisa receber status coerente quando a projeção ainda está atrasada.

Termo para lembrar: Search visibility | Base técnica: referência 1.

10. Como mostra transação recém-criada?

Eu posso combinar leitura autoritativa do detalhe ou overlay temporário identificado com a busca, mostrando o estado real. Read-your-writes precisa evitar duplicar o item quando a projeção alcançar a mesma versão posteriormente.

Termo para lembrar: Read-your-writes | Base técnica: referência 1.

11. Forçaria refresh a cada escrita?

Somente se a necessidade justificar o custo medido. Refresh policy muito agressiva aumenta trabalho e pode reduzir throughput; para histórico, atraso pequeno explícito pode ser preferível a pagar esse custo em cada documento.

Termo para lembrar: Refresh policy | Base técnica: referência 1.

12. Como escolhe text e keyword?

Eu uso text para análise linguística quando necessária e keyword para identificadores, filtros exatos e agregações apropriadas. Mapping design define comportamento de consulta; analisar um identificador financeiro como texto pode produzir resultados incorretos.

Termo para lembrar: Mapping design | Base técnica: referência 3.

13. Como evita mapping explosion?

Eu defino campos esperados e limito metadata dinâmica, usando representação apropriada para atributos variáveis. Mapping explosion pode consumir recursos e dificultar operação quando cada chave arbitrária de payload vira um campo novo.

Termo para lembrar: Mapping explosion | Base técnica: referência 3.

14. Como guarda valores monetários?

Eu defino unidade e representação exata, como inteiro em unidade mínima quando adequado, e não uso relevância textual para valores. Numeric mapping precisa preservar filtros e ordenação sem introduzir arredondamento incompatível com o domínio.

Termo para lembrar: Numeric mapping | Base técnica: referência 3.

15. Como impede busca de outra conta?

Eu obtenho o escopo autorizado no servidor e o aplico a toda consulta, incluindo agregações e exportações. Tenant isolation não pode depender de um account_id enviado livremente pelo cliente ou de ocultar campos na interface.

Termo para lembrar: Tenant isolation | Base técnica: referência 5.

16. Alias filtrado é toda autorização?

Eu não o trataria como única barreira sem validar cada API e caminho de acesso. Authorization enforcement precisa existir no serviço e nas permissões do cluster, impedindo consultas diretas que contornem o escopo pretendido.

Termo para lembrar: Authorization enforcement | Base técnica: referência 5.

17. Como pagina milhões de resultados?

Eu uso cursor com ordenação estável e mecanismo de snapshot de busca quando necessário, evitando offset profundo. Stable pagination precisa de desempate por identidade, para não repetir ou omitir transações quando há datas iguais.

Termo para lembrar: Stable pagination | Base técnica: referência 1.

18. Como lida com transações mudando durante paginação?

Eu defino se a consulta representa snapshot ou visão viva e comunico a semântica. Search snapshot melhora consistência da navegação, mas retém recursos e precisa de expiração e limites por cliente.

Termo para lembrar: Search snapshot | Base técnica: referência 1.

19. Como escolhe shards?

Eu estimo volume, tamanho de índice, throughput e custo de fan-out, validando por benchmark. Shard sizing evita tanto unidades enormes difíceis de mover quanto milhares de shards pequenos que consomem metadata e memória.

Termo para lembrar: Shard sizing | Base técnica: referência 3.

20. Índice por cliente é uma boa ideia?

Eu só consideraria para casos muito específicos, pois milhões de índices amplificam overhead operacional. Multi-tenant indexing costuma compartilhar índices com filtros e routing apropriados, preservando autorização e isolamento de recursos por cliente.

Termo para lembrar: Multi-tenant indexing | Base técnica: referência 3.

21. Routing por conta elimina hot tenants?

Não. Ele reduz fan-out de algumas consultas, mas uma conta muito ativa pode concentrar carga. Routing strategy precisa considerar skew, fallback e requisitos de distribuição, sem confundir localização eficiente com balanceamento perfeito.

Termo para lembrar: Routing strategy | Base técnica: referência 3.

22. Como reindexa sem interromper consulta?

Eu construo um índice novo, reproduzo mudanças até um watermark validado e troco o alias atomicamente. Alias swap resolve o apontamento, mas não garante sozinho que o novo índice contém todos os dados corretos.

Termo para lembrar: Alias swap | Base técnica: referência 2.

23. Como reverte índice novo ruim?

Eu mantenho o anterior durante uma janela, garantindo atualização suficiente para retorno ou replay controlado. Search rollback precisa considerar mudanças ocorridas após o corte; apontar para uma cópia congelada pode devolver histórico incompleto.

Termo para lembrar: Search rollback | Base técnica: referência 2.

24. Quando usa hot, warm e cold?

Eu uso tiers quando padrões de acesso e custo justificarem diferenças de hardware e latência. Data tiering não é obrigatório: histórico antigo ainda pode ser crítico para auditoria ou consultas que exigem resposta previsível.

Termo para lembrar: Data tiering | Base técnica: referência 3.

25. Como observa o pipeline?

Eu meço atraso desde o commit da origem, erros de indexação e idade da mensagem mais antiga, por partição. Projection lag é mais útil ao usuário que apenas CPU do cluster ou quantidade total de documentos.

Termo para lembrar: Projection lag | Base técnica: referência 4.

26. Como lida com documento inválido?

Eu isolo o registro com identidade, versão e motivo e preservo progresso seguro dos demais conforme dependências. Quarantine evita retry infinito, mas exige recuperação e não permite declarar completude enquanto documentos relevantes continuam ausentes.

Termo para lembrar: Quarantine | Base técnica: referência 4.

27. E se o índice inteiro cair?

Eu degrado busca avançada e preservo caminhos críticos apoiados na fonte ou em projeções simples, conforme capacidade. Graceful degradation deve limitar fallback para não transferir toda a carga de busca ao banco financeiro.

Termo para lembrar: Graceful degradation | Base técnica: referência 1.

28. Como evita consulta muito cara?

Eu limito período, paginação, operadores e custo estimado, com timeout e quotas por cliente. Query admission protege o cluster; permitir regex arbitrária ou agregações enormes pode tornar uma única requisição mais cara que milhares normais.

Termo para lembrar: Query admission | Base técnica: referência 3.

29. Como verifica completude da reconstrução?

Eu comparo faixas, versões e amostras semânticas contra um ponto de corte da origem, incluindo exclusões. Reindex reconciliation não se resume a contar documentos, pois contagens iguais podem esconder transações faltantes e duplicadas.

Termo para lembrar: Reindex reconciliation | Base técnica: referência 4.

30. O que demonstraria à banca?

Eu mostraria busca útil sem comprometer autoridade financeira, privacidade e recuperação. Projection architecture é uma troca consciente entre flexibilidade de consulta e consistência imediata, com mecanismos concretos para atrasos, reconstrução e falhas.

Termo para lembrar: Projection architecture | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Busca no SQL

Ganho. Menos componentes

Custo e motivo. Limites de relevância e carga

Como decidir. Quando filtros e escala cabem no banco

02. Engine dedicado

Ganho. Consulta flexível

Custo e motivo. Projeção e operação extra

Como decidir. Quando requisitos justificam

03. Refresh frequente

Ganho. Maior frescor

Custo e motivo. Custo de indexação

Como decidir. Para visibilidade quase imediata necessária

04. Refresh relaxado

Ganho. Mais throughput

Custo e motivo. Atraso de busca

Como decidir. Quando UX tolera lag explícito

05. Mapping explícito

Ganho. Previsibilidade

Custo e motivo. Evolução planejada

Como decidir. Para documentos financeiros conhecidos

06. Mapping dinâmico

Ganho. Flexibilidade inicial

Custo e motivo. Explosão de campos

Como decidir. Para atributos limitados e controlados

07. Routing por conta

Ganho. Menor fan-out

Custo e motivo. Skew por cliente

Como decidir. Quando distribuição é favorável

08. Índice por período

Ganho. Lifecycle simples

Custo e motivo. Consultas atravessam índices

Como decidir. Quando filtros temporais predominam

09. Manter índice antigo

Ganho. Rollback

Custo e motivo. Storage e atualização extra

Como decidir. Durante janela de validação

10. Fallback ao SQL

Ganho. Disponibilidade parcial

Custo e motivo. Risco de sobrecarga

Como decidir. Com consultas limitadas e capacidade reservada

Duas alternativas ao caminho proposto

1. PostgreSQL com índices e busca textual

Quando faz sentido. Escopo moderado e poucos filtros

Como implementar. Projetar índices compostos, GIN quando apropriado e paginação por chave

Limites. Pode exigir separação de leitura conforme volume

2. Projeção dedicada em banco de leitura

Quando faz sentido. Consultas previsíveis sem relevância textual complexa

Como implementar. Materializar por conta e tempo em storage adequado

Limites. Menos flexibilidade para pesquisa ad hoc e linguagem natural

Referências e alcance da evidência

1. Elastic - Near real-time search

Refresh e visibilidade de documentos em consultas de busca.

https://www.elastic.co/docs/manage-data/data-store/near-real-time-search

2. Elastic - Aliases

Indireção de índices e troca de aliases por operações atômicas.

https://www.elastic.co/docs/manage-data/data-store/aliases

3. Elastic - Mapping

Tipos, campos e escolhas entre mapeamento explícito e dinâmico.

https://www.elastic.co/docs/manage-data/data-store/mapping

4. Debezium - PostgreSQL connector

Snapshots, captura de mudanças, offsets e operação do conector PostgreSQL.

https://debezium.io/documentation/reference/stable/connectors/postgresql.html

5. OWASP - Authorization

Menor privilégio, deny by default e autorização por requisição.

https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html

Continue em: T043 · T063 · T072 · T077

C035 / M. Os 50 cenários enviados

Service Mesh e Comunicação Inter-serviços (L7)

Service mesh oferece mecanismos compartilhados de identidade e tráfego entre serviços, mas acrescenta uma camada operacional e não substitui autorização de negócio.

Quando usar. Muitos serviços com necessidades repetidas de mTLS, observabilidade e políticas de tráfego administradas por plataforma.

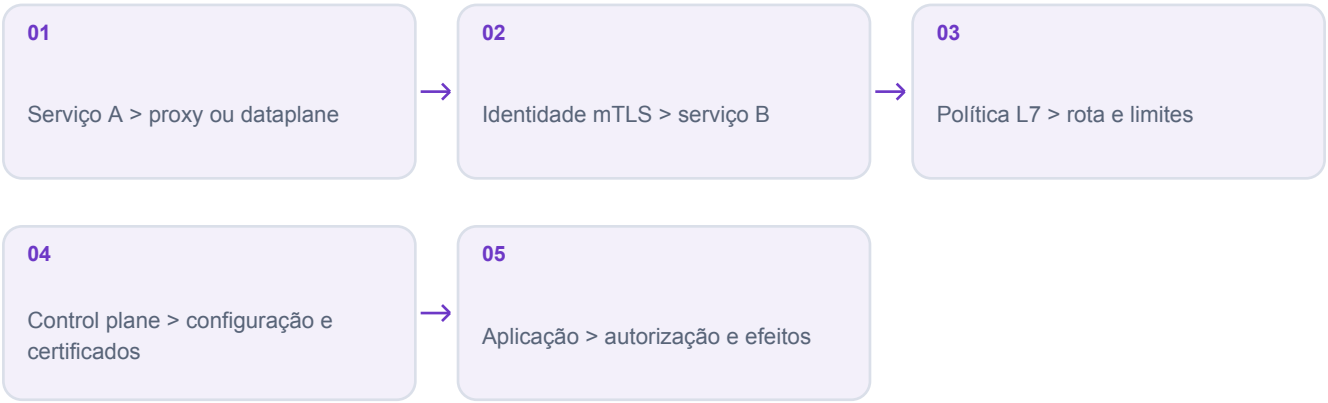
Quando evitar ou limitar. Adotar mesh por senioridade ou mover retries financeiros para proxies sem contrato de idempotência e deadline.

Uma abertura possível

Eu perguntaria qual problema transversal queremos padronizar. Um mesh pode ajudar, mas manteria decisões de negócio na aplicação e mediria custo, falhas do plano de controle e impacto na latência.

Peças e tecnologias. Istio · Envoy · Kubernetes · SPIFFE · OpenTelemetry

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Inventariar fluxos e objetivos de segurança e tráfego

2. Escolher modo de dataplane compatível e medir overhead

3. Configurar identidades e políticas mínimas por serviço

4. Alinhar timeouts e retries com semântica da aplicação

5. Ensaiar rotação, falha de controle e rollout do mesh

30 perguntas para sustentar o desenho

01. Mesh é obrigatório para microserviços?

Não. Eu o adotaria quando benefícios transversais superarem custo e complexidade. Service mesh pode ser excessivo para poucos serviços; bibliotecas consistentes e infraestrutura gerenciada podem atender aos requisitos com menos componentes operacionais.

Termo para lembrar: Service mesh | Base técnica: referência 1.

02. Gateway e mesh resolvem o mesmo problema?

Eles se sobrepõem em algumas capacidades, mas o gateway costuma controlar uma fronteira de entrada e o mesh comunica workloads internos. East-west traffic tem topologia e identidade próprias, sem exigir passar tudo por um gateway central.

Termo para lembrar: East-west traffic | Base técnica: referência 2.

03. mTLS significa autorização completa?

Não. mTLS autentica pares e protege o canal, mas não decide se um usuário pode estornar uma transação específica. Domain authorization continua no serviço com contexto do recurso, mesmo quando a identidade do workload é confiável.

Termo para lembrar: Domain authorization | Base técnica: referência 1.

04. Como identifica um workload?

Eu uso identidade emitida e validada pela infraestrutura, vinculada ao contexto autorizado de execução. Workload identity precisa de cadeia de confiança e políticas; nome DNS ou IP de origem sozinho não estabelece autorização suficiente.

Termo para lembrar: Workload identity | Base técnica: referência 1.

05. O que fica no control plane?

Ele distribui configuração e material necessário ao dataplane conforme a implementação. Control plane separation exige entender quais operações continuam com configuração existente e quais dependem de novas atualizações ou emissão de certificados durante uma falha.

Termo para lembrar: Control plane separation | Base técnica: referência 1.

06. Queda do control plane derruba tudo?

Não necessariamente, mas eu não prometo continuidade indefinida. Data plane autonomy depende da configuração já recebida, expiração de credenciais e eventos de escala; testo serviços existentes, novos pods e rotação durante a indisponibilidade.

Termo para lembrar: Data plane autonomy | Base técnica: referência 1.

07. Sidecar é a única implantação possível?

Não. Eu comparo modos suportados, incluindo abordagens sem sidecar por pod quando disponíveis, e suas diferenças de recursos e políticas. Data plane topology altera isolamento, custo e superfície operacional, exigindo avaliação concreta do ambiente.

Termo para lembrar: Data plane topology | Base técnica: referência 1.

08. Qual custo do proxy por pod?

Eu meço CPU, memória, latência e conexões adicionais, especialmente em chamadas pequenas e frequentes. Proxy overhead pode ser relevante em fan-out; o benefício de padronização precisa compensar esse custo no workload real.

Termo para lembrar: Proxy overhead | Base técnica: referência 2.

09. Moveria todos os retries ao mesh?

Não. Eu só habilitaria retries compatíveis com a semântica e o orçamento da operação. Retry ownership precisa ser claro para evitar multiplicação entre cliente, proxy e aplicação, especialmente quando uma chamada pode produzir efeito financeiro.

Termo para lembrar: Retry ownership | Base técnica: referência 2.

10. Proxy sabe se pagamento foi aplicado?

Normalmente não possui essa semântica. Eu trataria timeout após envio como resultado potencialmente ambíguo e usaria identidade e consulta de estado no domínio. Transport retry não deve inventar uma nova intenção de pagamento.

Termo para lembrar: Transport retry | Base técnica: referência 2.

11. Como alinha deadlines?

Eu propago um orçamento total e distribuo limites por etapa, considerando retries e tempo de fila. Deadline propagation evita que proxies continuem consumindo recursos depois de o chamador já ter desistido da operação.

Termo para lembrar: Deadline propagation | Base técnica: referência 2.

12. Circuit breaker do proxy basta?

Ele pode limitar conexões ou remover endpoints problemáticos, mas decisões de degradação de negócio continuam específicas. Circuit breaking precisa distinguir falha técnica de rejeição legítima, para não expulsar um serviço saudável que responde erros de domínio.

Termo para lembrar: Circuit breaking | Base técnica: referência 2.

13. Como faz canary no mesh?

Eu direciono uma fração controlada para a versão nova e observo métricas por versão e invariantes. Traffic splitting não garante compatibilidade de dados nem rollback seguro; essas condições precisam ser estabelecidas antes do experimento.

Termo para lembrar: Traffic splitting | Base técnica: referência 2.

14. Sticky session ajuda no canary?

Pode manter uma sessão na mesma versão quando necessário, mas cria dependência de afinidade e distribuição. Session affinity não deve ser usada para esconder estado local que impede recuperação após perda da instância.

Termo para lembrar: Session affinity | Base técnica: referência 2.

15. Como evita bypass do mTLS?

Eu verifico portas, caminhos diretos, serviços fora do mesh e políticas de rede, além do modo de autenticação. Policy enforcement precisa cobrir rotas alternativas; configurar criptografia numa rota não protege automaticamente todas as conexões possíveis.

Termo para lembrar: Policy enforcement | Base técnica: referência 1.

16. Como migra de modo permissivo para estrito?

Eu inventario clientes, observo tráfego não autenticado e migro por escopo, corrigindo incompatibilidades antes de exigir a política. Strict mTLS rollout reduz exposição sem produzir uma interrupção ampla por dependências esquecidas.

Termo para lembrar: Strict mTLS rollout | Base técnica: referência 1.

17. Como roda rotação de certificados?

Eu testo emissão, distribuição e renovação antes da expiração, com alerta de validade restante e falhas de atualização. Certificate rotation precisa cobrir conexões novas e existentes conforme implementação, sem depender de reinícios manuais coordenados.

Termo para lembrar: Certificate rotation | Base técnica: referência 1.

18. Mesh implementa zero trust sozinho?

Não. Zero trust envolve identidade, autorização, contexto e minimização de confiança em várias camadas. O mesh pode fornecer mecanismos úteis, mas credenciais amplas, endpoints vulneráveis ou autorização incorreta continuam permitindo abuso.

Termo para lembrar: Zero trust | Base técnica: referência 1.

19. Como protege saída para parceiro?

Eu defino destinos permitidos, identidade e observabilidade, avaliando egress gateway quando houver necessidade de controle central. Egress policy deve considerar DNS, TLS e caminhos alternativos; centralizar saída também cria capacidade e disponibilidade a gerenciar.

Termo para lembrar: Egress policy | Base técnica: referência 1.

20. Como evita configuração ruim global?

Eu valido políticas, aplico por escopo e uso rollout progressivo com capacidade de reversão. Configuration blast radius pode ser maior que um deploy de aplicação, porque uma regra compartilhada altera vários fluxos simultaneamente.

Termo para lembrar: Configuration blast radius | Base técnica: referência 2.

21. Como observa falha entre proxies?

Eu correlaciono spans, métricas de conexão e resposta, distinguindo erro da aplicação de rejeição ou timeout no dataplane. Proxy telemetry facilita diagnóstico, mas cardinalidade e dados sensíveis precisam ser controlados no armazenamento.

Termo para lembrar: Proxy telemetry | Base técnica: referência 2.

22. O mesh elimina tracing na aplicação?

Não. Ele observa transporte, mas não conhece etapas internas como validação de saldo ou commit. Application tracing complementa o caminho de rede e ajuda a explicar onde o tempo e a falha de negócio ocorreram.

Termo para lembrar: Application tracing | Base técnica: referência 2.

23. Como lida com protocolos não HTTP?

Eu verifico suporte e nível de inspeção disponível, evitando atribuir políticas L7 a tráfego opaco. Protocol support varia; encaminhar TCP não significa compreender mensagens financeiras, retries seguros ou códigos semânticos desse protocolo.

Termo para lembrar: Protocol support | Base técnica: referência 2.

24. Como evita retry storm na recuperação?

Eu limito tentativas e concorrência, uso jitter quando aplicável e recupero tráfego gradualmente. Recovery ramp-up evita que endpoints recém-recuperados recebam simultaneamente backlog, conexões novas e retries de toda a malha.

Termo para lembrar: Recovery ramp-up | Base técnica: referência 2.

25. Como atualiza o mesh?

Eu testo compatibilidade entre versões do control plane, dataplane e aplicações, usando escopo progressivo. Mesh upgrade tem blast radius próprio; trato certificados, políticas e encerramento de conexões como partes da validação operacional.

Termo para lembrar: Mesh upgrade | Base técnica: referência 1.

26. Como lida com latência de fan-out?

Eu reduzo dependências e paralelismo desnecessário, medindo cauda de cada salto, incluindo proxies. Fan-out amplification pode dominar o SLO; adicionar mesh não resolve uma topologia que exige muitas chamadas para uma resposta simples.

Termo para lembrar: Fan-out amplification | Base técnica: referência 2.

27. Como testa isolamento entre tenants?

Eu verifico autorização no domínio e propagação de contexto autenticado, além das políticas entre workloads. Tenant isolation não pode depender apenas de um certificado compartilhado pelo serviço que atende todos os clientes.

Termo para lembrar: Tenant isolation | Base técnica: referência 1.

28. Quem deve operar a malha?

Eu definiria ownership de plataforma e responsabilidades das equipes de aplicação, com contratos de configuração e suporte. Platform ownership precisa evitar que políticas sejam modificadas por ninguém ou por todos sem revisão e rastreabilidade.

Termo para lembrar: Platform ownership | Base técnica: referência 1.

29. Quando removeria o mesh?

Eu reconsideraria se overhead, incidentes e complexidade superarem benefícios medidos, mantendo os requisitos por outros mecanismos. Architecture simplification é válida quando preserva segurança e operação; uma ferramenta adotada não precisa virar compromisso permanente.

Termo para lembrar: Architecture simplification | Base técnica: referência 1.

30. Como fecharia a resposta?

Eu mostraria quais garantias ficam no transporte e quais continuam no domínio, com custo e recuperação. Responsibility boundary é o ponto central: mesh padroniza mecanismos, mas não converte automaticamente comunicação em transação correta.

Termo para lembrar: Responsibility boundary | Base técnica: referência 2.

10 trade-offs: ganho, custo e decisão

01. Mesh

Ganho. Políticas transversais

Custo e motivo. Camada operacional extra

Como decidir. Para muitos serviços com necessidades repetidas

02. Bibliotecas

Ganho. Menos infraestrutura

Custo e motivo. Atualização por aplicação

Como decidir. Para stack controlada e escopo menor

03. Sidecar

Ganho. Isolamento próximo do workload

Custo e motivo. Recursos por pod

Como decidir. Quando esse modo atende políticas e operação

04. Dataplane compartilhado

Ganho. Menos componentes por pod

Custo e motivo. Fronteira operacional diferente

Como decidir. Após validar recursos e isolamento

05. mTLS estrito

Ganho. Identidade de canal exigida

Custo e motivo. Migração e compatibilidade

Como decidir. Quando todos os clientes estão preparados

06. Retry no proxy

Ganho. Padronização de transporte

Custo e motivo. Semântica limitada

Como decidir. Para operações comprovadamente seguras

07. Retry na aplicação

Ganho. Conhecimento do domínio

Custo e motivo. Código e manutenção

Como decidir. Para efeitos e resultados ambíguos

08. Egress central

Ganho. Controle e auditoria

Custo e motivo. Gargalo e dependência

Como decidir. Quando governança de saída justifica

09. Configuração global

Ganho. Uniformidade

Custo e motivo. Blast radius amplo

Como decidir. Com validação e rollout gradual

10. Tracing de rede

Ganho. Visibilidade uniforme

Custo e motivo. Não explica lógica interna

Como decidir. Como complemento à instrumentação da aplicação

Duas alternativas ao caminho proposto

1. SDKs e bibliotecas padronizadas

Quando faz sentido. Poucas linguagens e equipes alinhadas

Como implementar. Implementar identidade, limites e telemetria com versões gerenciadas

Limites. Atualizações exigem rollout das aplicações

2. Serviços gerenciados e gateways seletivos

Quando faz sentido. Necessidades concentradas em fronteiras

Como implementar. Usar load balancing, identidade e políticas nas rotas relevantes

Limites. Menor uniformidade de tráfego interno e dependência do provedor

Referências e alcance da evidência

1. Istio - Security

Identidade de workloads, autenticação mútua e políticas de autorização.

https://istio.io/latest/docs/concepts/security/

2. Istio - Traffic management

Roteamento, timeouts, retries e políticas de tráfego no mesh.

https://istio.io/latest/docs/concepts/traffic-management/

Continue em: T051 · T053 · T064 · T072

C036 / M. Os 50 cenários enviados

Integração com APIs legadas externas (ex: SPB, Câmaras de Compensação)

Integração financeira legada exige tradução de domínio, envio durável, controle de capacidade e reconciliação de resultados ambíguos.

Quando usar. Parceiros com HTTP, arquivos ou SFTP, quotas limitadas, janelas operacionais e confirmações assíncronas.

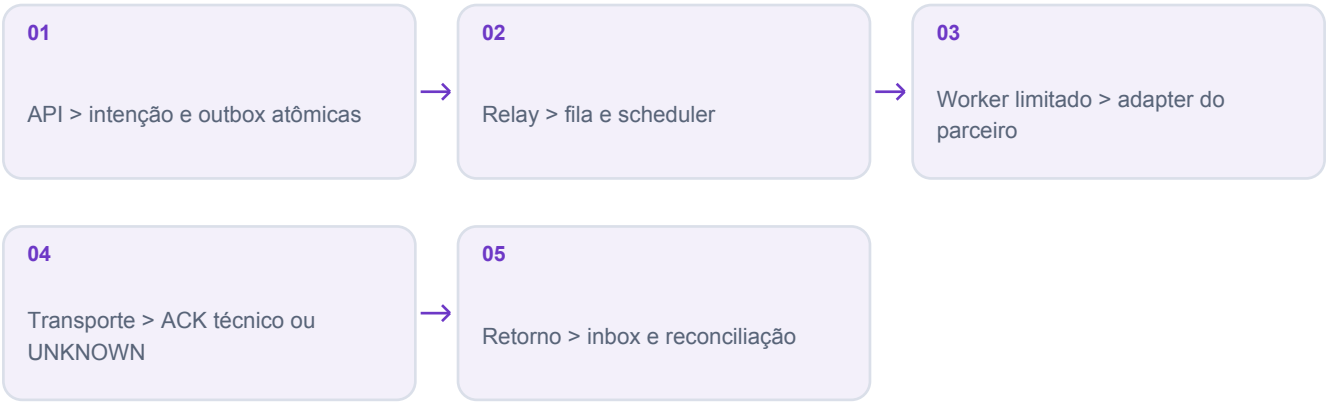
Quando evitar ou limitar. Retentar cegamente após timeout, criar nova identidade para uma intenção incerta ou assumir que transferência de arquivo significa aceite financeiro.

Uma abertura possível

Eu separaria aceitação interna, transporte e confirmação de negócio. Cada intenção teria identidade estável e estado durável; falhas certas permitem retry, enquanto resultado desconhecido exige consulta ou reconciliação.

Peças e tecnologias. PostgreSQL · Outbox · SQS ou Kafka · ECS · S3 · AWS Transfer Family · SFTP · Adapter ACL

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Mapear contrato, códigos, quotas e identidades do parceiro

2. Persistir intenção e obrigação de envio atomicamente

3. Enviar com limites, deadlines e bytes de lote imutáveis

4. Separar ACK técnico, aceite e resultado desconhecido

5. Processar retorno idempotente e reconciliar pendências por prazo

30 perguntas para sustentar o desenho

01. Por que uma anti-corruption layer?

Eu isolaria códigos, formatos e estados legados do modelo interno, traduzindo semântica explicitamente. Anti-corruption layer evita espalhar detalhes do parceiro por todos os serviços, mas não deve esconder incerteza nem converter qualquer erro em rejeição definitiva.

Termo para lembrar: Anti-corruption layer | Base técnica: referência 1.

02. Qual primeira confirmação ao cliente?

Eu confirmaria apenas a aceitação durável da solicitação, com identificador e estado pendente quando o processamento for assíncrono. Acceptance boundary deixa claro que registrar internamente não significa que a câmara ou bandeira aprovou a operação.

Termo para lembrar: Acceptance boundary | Base técnica: referência 4.

03. Como evita perder envio após gravar pedido?

Eu gravo pedido e outbox na mesma transação local, depois publico por relay recuperável. Transactional outbox remove a janela de obrigação perdida, embora a publicação possa repetir e o consumidor precise tolerar essa repetição.

Termo para lembrar: Transactional outbox | Base técnica: referência 4.

04. Kafka é obrigatório para retry?

Não. Eu posso manter trabalho pendente numa fila ou tabela durável com next_attempt_at, conforme escala e operação. Durable retry importa mais que a tecnologia: identidade, estado e critérios de reenvio precisam permanecer explícitos.

Termo para lembrar: Durable retry | Base técnica: referência 2.

05. Como respeita quota do parceiro?

Eu modelo a regra real: taxa, burst, janela, concorrência ou limites por credencial, coordenando instâncias. Rate limiting precisa corresponder ao contrato; token bucket não representa automaticamente qualquer quota rígida por janela do fornecedor.

Termo para lembrar: Rate limiting | Base técnica: referência 2.

06. Como evita dois workers enviarem o mesmo lote?

Eu arbitro posse e transição de estado atomicamente, preservando identidade e mecanismo de proteção no destino quando disponível. Worker lease reduz concorrência, mas não interrompe um processo antigo nem garante sozinho ausência de envio duplicado.

Termo para lembrar: Worker lease | Base técnica: referência 5.

07. Como identifica lote e tentativa?

Eu separo batch_id da tentativa física e mantenho operation_id por item, com manifesto e checksum. Attempt identity permite repetir transporte sem inventar uma nova intenção financeira ou perder o vínculo com retornos atrasados.

Termo para lembrar: Attempt identity | Base técnica: referência 5.

08. Pode misturar novos e antigos num novo lote?

Somente para itens comprovadamente não processados e se o contrato aceitar esse reagrupamento. Rebatching de itens UNKNOWN pode duplicar efeitos; preservo referências e verifico a semântica de deduplicação do parceiro antes de decidir.

Termo para lembrar: Rebatching | Base técnica: referência 5.

09. Erro de conexão sempre permite retry?

Não basta o rótulo genérico. Eu verifico em qual fase falhou e quais tentativas anteriores existiram; após envio parcial ou resposta perdida, o resultado pode ser UNKNOWN mesmo que o cliente reporte erro de conexão.

Termo para lembrar: UNKNOWN | Base técnica: referência 2.

10. Quando classifica como falha certa?

Eu classifico quando há evidência suficiente de não aplicação, como rejeição definitiva correlacionada ou falha antes de qualquer envio naquela tentativa sem incerteza anterior. Definitive failure depende do contrato e da história completa da intenção.

Termo para lembrar: Definitive failure | Base técnica: referência 2.

11. O que faz após timeout de SFTP?

Eu preservo arquivo, identidade e estado incerto e consulto evidências de transporte e retorno de negócio. SFTP timeout não prova ausência no destino; reenviar como novo lote pode duplicar chargebacks já recebidos pelo parceiro.

Termo para lembrar: SFTP timeout | Base técnica: referência 3.

12. Arquivo ausente no diretório prova que não foi processado?

Não. O parceiro pode mover ou remover arquivos após ingestão. Negative evidence precisa de contrato confiável, como consulta de status ou confirmação explícita; ausência numa listagem não autoriza concluir que nenhum efeito ocorreu.

Termo para lembrar: Negative evidence | Base técnica: referência 3.

13. ACK do transfer job é sucesso financeiro?

Não. Ele indica um marco técnico do transporte conforme o serviço. Business acknowledgment é outra etapa, associada a lote ou item; eu manteria estados distintos para enviado, recebido, aceito, rejeitado e concluído conforme contrato.

Termo para lembrar: Business acknowledgment | Base técnica: referência 3.

14. Como usa arquivo temporário e rename?

Eu avaliaria upload com nome temporário e publicação por rename se o parceiro garantir essa convenção. Atomic handoff reduz leitura de arquivo incompleto, mas não oferece deduplicação financeira nem transação distribuída com meu banco.

Termo para lembrar: Atomic handoff | Base técnica: referência 3.

15. Como trata retorno parcial de lote?

Eu reconcilio por item e identidade, preservando totais e estado agregado derivado. Partial acceptance impede marcar o lote inteiro como falho e reenviar itens já aceitos; pendências individuais continuam com causa e próxima ação.

Termo para lembrar: Partial acceptance | Base técnica: referência 5.

16. Como valida arquivo de retorno?

Eu verifico origem, integridade, formato, referência e valores esperados antes de aplicar mudanças. Return validation inclui limites de tamanho e parser seguro; um arquivo tecnicamente legível não é automaticamente um resultado autorizado do parceiro.

Termo para lembrar: Return validation | Base técnica: referência 3.

17. Como aplica retorno duplicado?

Eu registro inbox e transição de domínio na mesma transação local, com chave do evento ou referência de resultado apropriada. Idempotent return processing tolera reentrega sem creditar, notificar ou encerrar duas vezes a mesma operação.

Termo para lembrar: Idempotent return processing | Base técnica: referência 5.

18. Como trata retorno atrasado após cancelamento?

Eu verifico máquina de estados e evidência financeira, sem simplesmente ignorar o evento. Late acknowledgment pode exigir ajuste ou compensação; a decisão deve preservar histórico e evitar que cancelamento local apague um efeito remoto já realizado.

Termo para lembrar: Late acknowledgment | Base técnica: referência 5.

19. Circuit breaker resolve indisponibilidade prolongada?

Ele reduz chamadas inúteis, mas eu também preciso de storage, prazos e capacidade de recuperação. Outage buffering deve ser dimensionado e monitorado; aceitar indefinidamente novas solicitações pode ultrapassar cutoff e compromissos com clientes.

Termo para lembrar: Outage buffering | Base técnica: referência 2.

20. Por que retry a cada quatro horas pode ser ruim?

Eu comparo a janela com prazo, recuperação do parceiro e volume acumulado. Retry schedule pode usar backoff e jitter dentro do contrato; uma periodicidade fixa não garante tempo adequado para todos os cenários de chargeback.

Termo para lembrar: Retry schedule | Base técnica: referência 2.

21. Como evita tempestade quando parceiro volta?

Eu reabro gradualmente, respeitando quota e priorizando prazos, enquanto separo consultas de reconciliação de novos envios. Recovery throttling impede que backlog e tráfego novo consumam simultaneamente toda a capacidade disponível do parceiro.

Termo para lembrar: Recovery throttling | Base técnica: referência 2.

22. O que vai para DLQ?

Eu isolo falhas não recuperáveis automaticamente ou que exigem análise, com identidade, histórico e motivo. Quarantine não equivale a falha financeira final; casos incertos podem precisar reconciliação antes de qualquer compensação ou redrive.

Termo para lembrar: Quarantine | Base técnica: referência 2.

23. Pode automatizar tudo sem intervenção humana?

Eu automatizo caminhos conhecidos, mas mantenho escalonamento para evidência contraditória, prazos críticos e contratos sem consulta confiável. Exception handling é parte da resiliência; prometer ausência absoluta de intervenção esconderia limites reais da integração.

Termo para lembrar: Exception handling | Base técnica: referência 1.

24. Como troca parceiro sem duplicar pagamento?

Eu preservo a intenção global e só faço failover quando sei que o anterior não aplicou ou existe deduplicação compartilhada confiável. Provider failover após UNKNOWN pode produzir dois efeitos em destinos que não compartilham identidade.

Termo para lembrar: Provider failover | Base técnica: referência 5.

25. Como lida com mudança de layout?

Eu versiono parser e serializador, valido fixtures oficiais e mantenho compatibilidade com retornos de lotes antigos. Protocol versioning precisa acompanhar a identidade do lote, pois um retorno atrasado pode usar o layout anterior.

Termo para lembrar: Protocol versioning | Base técnica: referência 1.

26. Como mede SLA dessa integração?

Eu separo aceitação, tempo de envio, idade de UNKNOWN e conclusão de negócio, com prazos por classe. End-to-end SLA não é a disponibilidade da minha API; inclui dependências e estados que o usuário ainda percebe como pendentes.

Termo para lembrar: End-to-end SLA | Base técnica: referência 2.

27. Como recupera após restaurar banco antigo?

Eu congelo envios relevantes e reconcilio com registros externos e identidades preservadas antes de retomar. Disaster reconciliation evita que uma restauração esqueça efeitos já realizados pelo parceiro e os execute novamente como se fossem novos.

Termo para lembrar: Disaster reconciliation | Base técnica: referência 5.

28. Como implementa compensação?

Eu crio uma operação de negócio vinculada à original, com identidade própria e sem apagar histórico. Compensation depende de confirmar o efeito que será revertido; não estorno automaticamente só porque a resposta do transporte foi perdida.

Termo para lembrar: Compensation | Base técnica: referência 5.

29. Como testa o adapter?

Eu simulo rejeição, timeout após aplicação, arquivo parcial, duplicata e retorno fora de ordem, verificando estados e invariantes. Contract fault testing precisa demonstrar que o sistema distingue não aplicado, aplicado e desconhecido.

Termo para lembrar: Contract fault testing | Base técnica: referência 1.

30. Qual fala final usaria?

Eu explicaria que resiliência combina automação com honestidade sobre o estado remoto. Store-and-forward garante obrigação local recuperável; efeito único depende de identidade, contrato do parceiro e reconciliação das janelas em que não conhecemos o resultado.

Termo para lembrar: Store-and-forward | Base técnica: referência 4.

10 trade-offs: ganho, custo e decisão

01. Chamada síncrona

Ganho. Resposta imediata

Custo e motivo. Acoplamento à disponibilidade

Como decidir. Para contrato rápido e deadline compatível

02. Store-and-forward

Ganho. Aceitação durável

Custo e motivo. Estado pendente e atraso

Como decidir. Para parceiro instável ou batch

03. Fila gerenciada

Ganho. Operação simplificada

Custo e motivo. Semântica e quotas próprias

Como decidir. Para tarefas recuperáveis

04. Tabela de trabalho

Ganho. Estado e consulta juntos

Custo e motivo. Polling e contenção

Como decidir. Para volume e operação compatíveis

05. Lote grande

Ganho. Amortiza transporte

Custo e motivo. Falha e retorno mais complexos

Como decidir. Quando contrato e prazo permitem

06. Lote pequeno

Ganho. Isola itens e reduz espera

Custo e motivo. Mais chamadas e overhead

Como decidir. Quando quota e custo suportam

07. Retry automático

Ganho. Recupera falhas transitórias

Custo e motivo. Risco de duplicata

Como decidir. Com identidade e evidência adequadas

08. Reconciliação antes do retry

Ganho. Evita efeito duplicado

Custo e motivo. Maior latência

Como decidir. Para resultado UNKNOWN

09. Failover de parceiro

Ganho. Continuidade potencial

Custo e motivo. Risco entre destinos

Como decidir. Só com não aplicação conhecida ou dedupe compartilhada

10. Exceção humana

Ganho. Resolve ambiguidade real

Custo e motivo. Custo e tempo

Como decidir. Para casos sem resolução automática confiável

Duas alternativas ao caminho proposto

1. Integração direta síncrona protegida

Quando faz sentido. Volume baixo e parceiro com contrato idempotente

Como implementar. Chamar com deadline, mesma identidade e consulta de status

Limites. Disponibilidade percebida depende do parceiro

2. Orquestrador durável

Quando faz sentido. Fluxos longos com vários retornos e timers

Como implementar. Modelar envio, espera, reconciliação e compensação como estados duráveis

Limites. Não elimina semântica externa nem necessidade de adapter

Referências e alcance da evidência

1. Microsoft - Anti-corruption layer

Tradução entre subsistemas com modelos e semânticas diferentes.

https://learn.microsoft.com/en-us/azure/architecture/patterns/anti-corruption-layer

2. AWS - Timeouts retries and jitter

Timeouts, backoff, jitter e risco de repetir efeitos após respostas perdidas.

https://aws.amazon.com/builders-library/timeouts-retries-and-backoff-with-jitter/

3. AWS - Monitoring SFTP connectors

Status de transferência, eventos e logs; processamento de negócio exige confirmação própria.

https://docs.aws.amazon.com/transfer/latest/userguide/track-connector-progress.html

4. AWS - Transactional outbox

Gravação atômica de estado e obrigação de publicar; duplicatas ainda possíveis.

https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/transactional-outbox.html

5. AWS Builders Library - Idempotent APIs

Identidades estáveis, intenção, retries e requisições atrasadas.

https://aws.amazon.com/builders-library/making-retries-safe-with-idempotent-APIs/

Continue em: T049 · T050 · T060 · T064

C037 / M. Os 50 cenários enviados

GraphQL Federation para dezenas de Frontends/Mobile Apps

GraphQL Federation compõe contratos de domínios em um grafo comum, exigindo governança, limites de consulta e autorização em cada fronteira relevante.

Quando usar. Múltiplas equipes e interfaces precisam combinar dados com autonomia de schema e composição controlada.

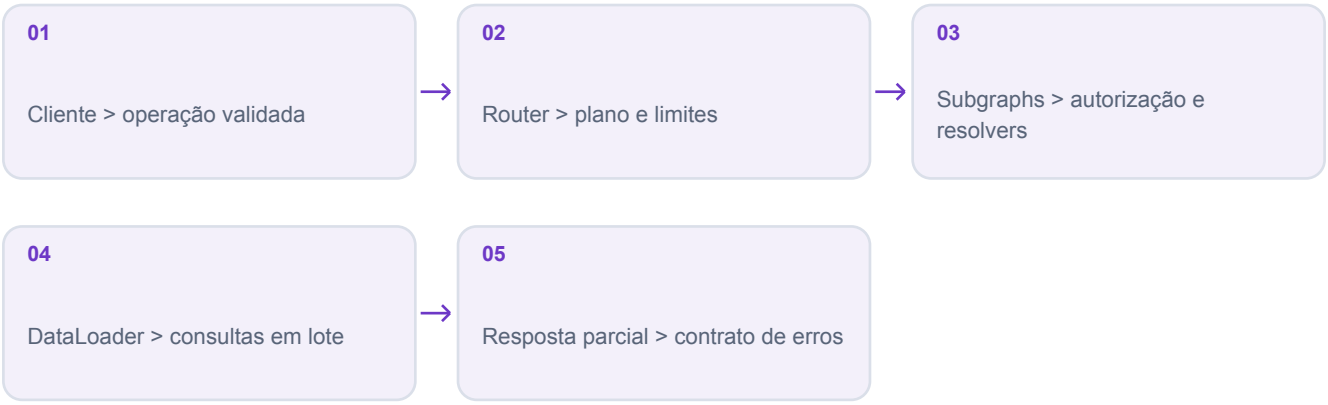
Quando evitar ou limitar. Federar por prestígio, consultar bancos de todos os domínios no gateway ou supor que uma única query produz snapshot transacional global.

Uma abertura possível

Eu separaria experiência de consulta e ownership dos dados. O router comporia subgraphs com contratos claros, enquanto cada domínio continuaria responsável por autorização, eficiência e efeitos de suas operações.

Peças e tecnologias. Apollo Federation · Apollo Router · GraphQL · DataLoader · OpenTelemetry · Schema registry

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Mapear jornadas e limites de domínio antes do schema

2. Definir entidades, chaves e ownership por subgraph

3. Validar composição e compatibilidade de operações existentes

4. Limitar custo, profundidade e concorrência com autorização

5. Testar N+1, falhas parciais e rollout independente

30 perguntas para sustentar o desenho

01. Por que federar em vez de um servidor único?

Eu federaria quando autonomia de equipes e domínios justificar composição distribuída. Federation não é requisito de senioridade; um servidor modular pode ser mais simples quando ownership, volume e mudanças ainda cabem numa equipe.

Termo para lembrar: Federation | Base técnica: referência 1.

02. Como escolhe fronteiras de subgraphs?

Eu parto de capacidades e responsabilidade pelos dados, não de tabelas ou telas individuais. Subgraph ownership precisa permitir evolução coerente; dividir cada campo em um serviço cria dependências e fan-out sem autonomia útil.

Termo para lembrar: Subgraph ownership | Base técnica: referência 1.

03. O router deve acessar todos os bancos?

Eu evitaria isso, mantendo acesso e regras nos domínios responsáveis. Domain encapsulation impede que o gateway se torne um monólito acoplado aos schemas físicos e concentre permissões amplas sobre dados financeiros de toda a organização.

Termo para lembrar: Domain encapsulation | Base técnica: referência 1.

04. Como define uma entidade federada?

Eu escolho identidade estável e contrato de resolução, com ownership explícito dos campos. Entity key deve ser suficiente para localizar o recurso sem expor informação sensível desnecessária ou permitir atravessar tenants por colisão de identificador.

Termo para lembrar: Entity key | Base técnica: referência 1.

05. Como resolve N+1?

Eu meço o plano e os resolvers, usando batching e cache por requisição quando apropriado. N+1 pode existir dentro do subgraph ou entre dependências; o router sozinho não corrige consultas SQL individuais disparadas por cada item.

Termo para lembrar: N+1 | Base técnica: referência 2.

06. DataLoader global é seguro?

Eu normalmente limitaria cache ao contexto da requisição ou incluiria escopo de autorização rigoroso. Request-scoped cache evita compartilhar dados entre usuários; um cache global por ID pode devolver um objeto que o chamador não deveria acessar.

Termo para lembrar: Request-scoped cache | Base técnica: referência 2.

07. Batching preserva ordem automaticamente?

Eu implemento correspondência explícita entre chaves e resultados, tratando ausências e erros por item. Batch resolver precisa devolver valores coerentes com o contrato esperado, mesmo quando o banco retorna linhas em ordem diferente da solicitação.

Termo para lembrar: Batch resolver | Base técnica: referência 2.

08. Como impede query muito profunda?

Eu imponho limites de profundidade e custo, considerando cardinalidade de listas e campos caros. Query complexity vai além de contar níveis: uma consulta rasa pode produzir enorme fan-out ou agregações custosas em vários subgraphs.

Termo para lembrar: Query complexity | Base técnica: referência 3.

09. Como estima custo antes de executar?

Eu uso pesos e limites de cardinalidade conhecidos, rejeitando ou restringindo operações acima do orçamento. Static cost analysis é aproximação; comparo estimativa com execução real para ajustar regras e detectar subestimação sistemática.

Termo para lembrar: Static cost analysis | Base técnica: referência 3.

10. Query persistida resolve segurança?

Ela restringe operações conhecidas quando usada como allowlist, mas não substitui autorização nem validação de variáveis. Persisted query pode continuar muito cara com parâmetros amplos ou acessar recursos de outro cliente se o domínio não verificar escopo.

Termo para lembrar: Persisted query | Base técnica: referência 3.

11. Como autoriza campos sensíveis?

Eu verifico identidade, ação e recurso no domínio, inclusive campos aninhados e resolução de entidades. Field authorization não deve depender apenas de ocultar o campo no cliente ou validar JWT uma vez na entrada.

Termo para lembrar: Field authorization | Base técnica: referência 4.

12. O subgraph pode confiar no router?

Eu autentico o canal e valido o contexto encaminhado, mantendo controles necessários no serviço. Trust boundary precisa considerar acesso direto e clientes internos; um header livre não pode representar identidade de usuário sem validação.

Termo para lembrar: Trust boundary | Base técnica: referência 4.

13. Como limita custo por tenant?

Eu combino orçamento de query com quotas e concorrência por escopo autenticado, observando consumo real. Tenant fairness impede que um cliente com operações válidas monopolize resolvers caros e degrade a experiência dos demais usuários.

Termo para lembrar: Tenant fairness | Base técnica: referência 3.

14. Uma query tem snapshot consistente entre subgraphs?

Não automaticamente. Cada domínio pode ler um momento diferente. Cross-service consistency exige contrato específico, versões ou uma projeção conjunta quando necessário; GraphQL compõe respostas, mas não fornece transação global só por usar uma operação.

Termo para lembrar: Cross-service consistency | Base técnica: referência 1.

15. Como mostra falha parcial?

Eu defino nullability e erros de domínio para que a interface diferencie informação indisponível de valor ausente. Partial response pode preservar parte da experiência, mas não deve representar saldo desconhecido como zero ou sucesso financeiro.

Termo para lembrar: Partial response | Base técnica: referência 1.

16. Como escolhe timeout?

Eu distribuo o deadline da jornada entre o router e dependências, incluindo fila e fan-out. Query deadline deve limitar trabalho abandonado e permitir degradação controlada; timeouts independentes longos podem exceder o orçamento total.

Termo para lembrar: Query deadline | Base técnica: referência 3.

17. Pode retentar mutation automaticamente?

Somente com contrato seguro e identidade estável para o efeito. Mutation retry após timeout pode repetir uma operação já aplicada; o domínio precisa oferecer idempotência ou consulta de estado, independentemente de a chamada usar GraphQL.

Termo para lembrar: Mutation retry | Base técnica: referência 4.

18. Mutation em vários subgraphs é transação?

Não. Eu modelaria workflow ou saga com estados e compensações quando houver efeitos distribuídos. Distributed mutation não ganha atomicidade apenas porque o cliente a enviou numa requisição; limites transacionais precisam permanecer explícitos.

Termo para lembrar: Distributed mutation | Base técnica: referência 1.

19. Como evita breaking changes?

Eu verifico composição e operações usadas por clientes, oferecendo depreciação e janela de migração. Schema compatibility inclui significado e nullability; remover campo pouco usado pode quebrar versões mobile ainda ativas por meses.

Termo para lembrar: Schema compatibility | Base técnica: referência 1.

20. Como trata app mobile antigo?

Eu acompanho uso por versão e mantenho contratos necessários enquanto houver suporte acordado, evitando exigir atualização instantânea. Client compatibility orienta depreciação; a equipe de backend não controla quando todos os aparelhos instalarão a versão nova.

Termo para lembrar: Client compatibility | Base técnica: referência 1.

21. Como migra campo entre subgraphs?

Eu planejo ownership e compatibilidade de resolução durante a transição, validando o plano composto antes do corte. Field migration precisa evitar ambiguidades e mudanças inesperadas de fan-out, mesmo quando o schema público permanece igual.

Termo para lembrar: Field migration | Base técnica: referência 1.

22. Como observa gargalo do router?

Eu meço tempo de planejamento, chamadas por operação, concorrência e custo dos subgraphs, com assinatura normalizada da query. Query-plan observability distingue overhead de composição de lentidão real no domínio que fornece os dados.

Termo para lembrar: Query-plan observability | Base técnica: referência 1.

23. Como evita alta cardinalidade nas métricas?

Eu uso nome ou assinatura normalizada de operação, sem registrar valores pessoais das variáveis como labels. Telemetry cardinality precisa ser limitada para que observabilidade não vire vazamento de dados ou custo descontrolado de armazenamento.

Termo para lembrar: Telemetry cardinality | Base técnica: referência 3.

24. Como faz cache de resposta?

Eu considero identidade, autorização, variáveis e frescor por campo, evitando compartilhar respostas privadas indevidamente. Response cache pode ser difícil em consultas compostas, pois o menor requisito de segurança e atualidade influencia a resposta inteira.

Termo para lembrar: Response cache | Base técnica: referência 4.

25. Como trata revogação de acesso?

Eu avalio autorização atual nos caminhos necessários e invalido caches conforme o contrato de revogação. Authorization freshness não pode depender exclusivamente de roles antigas no token ou de uma resposta cacheada antes da mudança.

Termo para lembrar: Authorization freshness | Base técnica: referência 4.

26. Como evita que um subgraph derrube todo o grafo?

Eu uso budgets, isolamento de concorrência e contratos de resposta parcial quando apropriados. Subgraph isolation precisa respeitar campos obrigatórios; algumas jornadas não podem continuar corretamente sem o domínio crítico, mesmo com outros serviços saudáveis.

Termo para lembrar: Subgraph isolation | Base técnica: referência 3.

27. Como testa composição antes do deploy?

Eu valido schema proposto contra o supergraph e operações relevantes, além de comportamento dos resolvers em integração. Composition check encontra incompatibilidades estruturais, mas não prova latência, autorização ou significado correto de cada campo.

Termo para lembrar: Composition check | Base técnica: referência 1.

28. Como define governança do grafo?

Eu estabeleço donos, convenções, revisão de mudanças compartilhadas e critérios de depreciação, com autonomia dentro de limites. Schema governance evita que a federação substitua um gargalo central por conflitos distribuídos de identidade e semântica.

Termo para lembrar: Schema governance | Base técnica: referência 1.

29. Quando escolheria BFF REST?

Eu escolheria BFF quando jornadas forem previsíveis e composição específica por interface simplificar segurança e desempenho. API fit depende dos consumidores; GraphQL não é automaticamente melhor quando flexibilidade de consulta traz mais custo que benefício.

Termo para lembrar: API fit | Base técnica: referência 1.

30. O que enfatizaria no encerramento?

Eu mostraria autonomia com contratos, custo controlado e autorização correta, incluindo falhas parciais e compatibilidade mobile. Federated architecture vale pelo problema organizacional e técnico resolvido, não pelo número de subgraphs ou pela adoção de uma ferramenta.

Termo para lembrar: Federated architecture | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Federação

Ganho. Autonomia por domínio

Custo e motivo. Composição e governança

Como decidir. Quando várias equipes evoluem contratos

02. Servidor modular

Ganho. Simplicidade operacional

Custo e motivo. Ownership central

Como decidir. Para escopo menor e equipe coesa

03. Query flexível

Ganho. Cliente escolhe dados

Custo e motivo. Custo difícil de prever

Como decidir. Com análise e limites

04. Operação persistida

Ganho. Controle e cache

Custo e motivo. Processo de publicação

Como decidir. Para clientes conhecidos e risco elevado

05. Batching

Ganho. Menos chamadas

Custo e motivo. Espera e tratamento por item

Como decidir. Quando várias chaves chegam juntas

06. Cache por request

Ganho. Evita repetição segura

Custo e motivo. Vida curta

Como decidir. Para resolvers com autorização contextual

07. Cache compartilhado

Ganho. Maior reutilização

Custo e motivo. Risco de vazamento e stale

Como decidir. Com chave e política rigorosas

08. Resposta parcial

Ganho. Preserva experiência

Custo e motivo. Complexidade no cliente

Como decidir. Quando dados opcionais falham

09. Campos não nulos

Ganho. Contrato forte

Custo e motivo. Propagação de falha

Como decidir. Quando ausência torna resposta inválida

10. Subgraphs pequenos

Ganho. Ownership específico

Custo e motivo. Fan-out e dependências

Como decidir. Somente com fronteiras de domínio reais

Duas alternativas ao caminho proposto

1. BFF REST por experiência

Quando faz sentido. Jornadas estáveis e interfaces distintas

Como implementar. Compor endpoints de domínio em respostas específicas

Limites. Pode duplicar composição e limitar exploração flexível

2. GraphQL modular único

Quando faz sentido. Equipe menor ou domínio coeso

Como implementar. Separar módulos e resolvers mantendo um runtime e governança simples

Limites. Escala organizacional e implantação permanecem compartilhadas

Referências e alcance da evidência

1. Apollo - Federation

Composição de subgraphs e organização de um supergraph.

https://www.apollographql.com/docs/graphos/schema-design/federated-schemas/federation

2. Apollo - Handling N+1

Redução de chamadas repetidas e batching em consultas federadas.

https://www.apollographql.com/docs/graphos/schema-design/guides/handling-n-plus-one

3. Apollo - Demand control

Análise de custo e limites de operações GraphQL.

https://www.apollographql.com/docs/graphos/routing/security/demand-control

4. OWASP - Authorization

Menor privilégio, deny by default e autorização por requisição.

https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html

Continue em: T053 · T064 · T072 · C035

C038 / M. Os 50 cenários enviados

Controle de Acesso Baseado em Regras e Relacionamentos (Global IAM/RBAC)

Autorização global combina identidade, políticas e relações, com requisitos explícitos de revogação, isolamento e consistência entre decisão e uso do recurso.

Quando usar. Contas compartilhadas, empresas, procurações, suporte e recursos com permissões finas distribuídas.

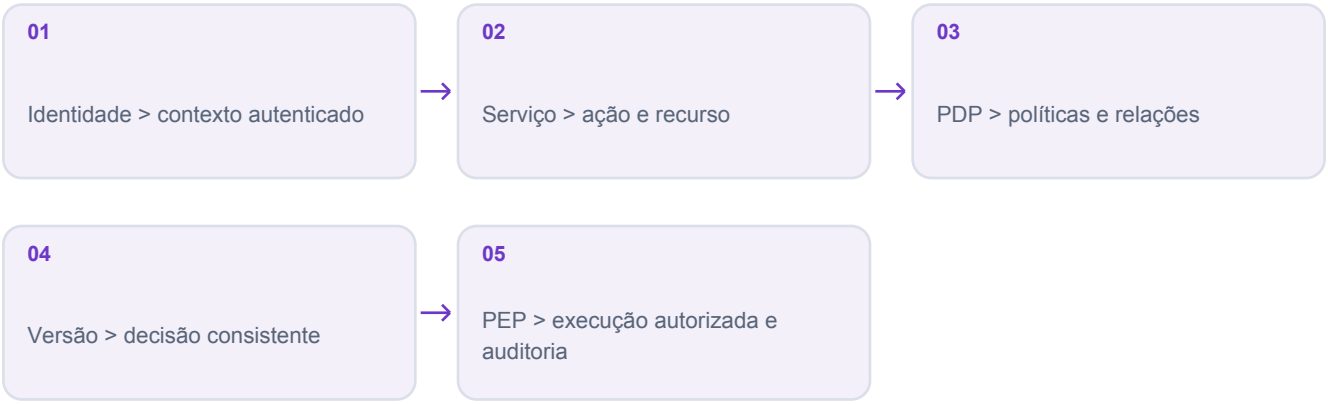
Quando evitar ou limitar. Tratar JWT assinado como permissão eterna, adotar Zanzibar sem necessidade ou confiar apenas em checks no gateway.

Uma abertura possível

Eu começaria pelas perguntas quem pode fazer o quê, sobre qual recurso e até quando. Escolheria RBAC, ABAC ou relações conforme o domínio e desenharia revogação e falhas antes de otimizar cache.

Peças e tecnologias. OIDC · RBAC · ABAC · ReBAC · OPA · OpenFGA · SpiceDB · PostgreSQL

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Modelar ações, recursos, tenants e relações permitidas

2. Separar autenticação de decisão e enforcement

3. Definir revogação, cache e consistência exigida por ação

4. Aplicar checks em todos os caminhos de acesso

5. Testar bypass, mudança concorrente e indisponibilidade do autorizador

30 perguntas para sustentar o desenho

01. RBAC basta para qualquer cenário?

Não. Eu usaria roles para responsabilidades estáveis, mas relações e atributos podem ser necessários para contas compartilhadas ou limites contextuais. Authorization model deve refletir regras reais sem criar milhares de papéis para cada combinação de recurso.

Termo para lembrar: Authorization model | Base técnica: referência 1.

02. Quando escolhe ABAC?

Eu escolho ABAC quando a decisão depende de atributos como tenant, classificação, horário ou limite, com fontes confiáveis. Attribute provenance importa: um atributo enviado livremente pelo cliente não pode conceder privilégio sem validação.

Termo para lembrar: ABAC | Base técnica: referência 1.

03. Quando escolhe ReBAC?

Eu escolho relações quando acesso deriva de vínculos, como membro de organização ou procurador de conta. ReBAC expressa esses caminhos, mas exige limites de profundidade, consistência e governança das alterações no grafo de permissões.

Termo para lembrar: ReBAC | Base técnica: referência 2.

04. Zanzibar é obrigatório para Staff?

Não. Eu usaria suas ideias quando escala e relações justificarem complexidade semelhante, sem copiar a arquitetura inteira. Architecture fit pode favorecer uma tabela relacional e checks transacionais para um domínio menor com requisitos claros.

Termo para lembrar: Architecture fit | Base técnica: referência 2.

05. JWT válido significa acesso permitido?

Não. A assinatura valida determinadas propriedades do token, mas a permissão pode ter sido revogada ou não abranger aquele recurso. Authorization freshness exige política atual ou uma janela de validade explicitamente aceita pelo negócio.

Termo para lembrar: Authorization freshness | Base técnica: referência 1.

06. Como separa autenticação e autorização?

Eu uso autenticação para estabelecer identidade e contexto validado; depois avalio ação, recurso e política. Authentication boundary não substitui o check de acesso, pois a mesma pessoa autenticada pode consultar uma conta e não outra.

Termo para lembrar: Authentication boundary | Base técnica: referência 3.

07. O que são PDP e PEP?

Eu separo o ponto que decide a política do ponto que a aplica, garantindo que toda rota relevante passe pelo enforcement. Policy enforcement point deve falhar de forma definida e não aceitar decisões fornecidas pelo próprio cliente.

Termo para lembrar: Policy enforcement point | Base técnica: referência 1.

08. Onde aplica o check?

Eu aplico no serviço responsável pelo recurso, com contexto autenticado, além de controles úteis na borda. Resource authorization protege caminhos internos, jobs e chamadas diretas que poderiam contornar uma validação feita somente no gateway.

Termo para lembrar: Resource authorization | Base técnica: referência 1.

09. Como modela tenant?

Eu incluo tenant na identidade lógica dos recursos e nas relações, validando coerência com o contexto. Tenant boundary precisa impedir colisões e travessias entre organizações, inclusive em caches, listagens, exports e resolução de IDs globais.

Termo para lembrar: Tenant boundary | Base técnica: referência 1.

10. Como resolve revogação rápida?

Eu defino o prazo máximo tolerado e uso consulta atual, invalidação ou versões conforme risco. Revocation latency é um requisito mensurável; cache com TTL longo não pode ser descrito como revogação instantânea sem mecanismo adicional.

Termo para lembrar: Revocation latency | Base técnica: referência 2.

11. Cache de deny é perigoso?

Ele pode atrasar uma concessão legítima, enquanto cache de allow pode prolongar acesso revogado. Authorization caching exige TTL, versão e escopo adequados; escolho comportamento segundo risco e expectativa de cada tipo de mudança.

Termo para lembrar: Authorization caching | Base técnica: referência 1.

12. Como evita chave de cache incompleta?

Eu incluo sujeito, ação, recurso, tenant e contexto relevante, além da versão de política quando necessário. Cache key completeness impede reutilizar uma decisão válida para outro usuário ou condição que alteraria a autorização.

Termo para lembrar: Cache key completeness | Base técnica: referência 1.

13. Como trata mudança de permissão durante operação longa?

Eu defino se autorização vale na aceitação ou precisa ser reavaliada antes do efeito e da entrega. Authorization checkpoint depende do risco; uma exportação longa pode exigir novo check antes de disponibilizar dados sensíveis.

Termo para lembrar: Authorization checkpoint | Base técnica: referência 1.

14. Qual é o problema TOCTOU?

Uma decisão pode ficar obsoleta entre check e uso se recurso ou permissão mudar. TOCTOU mitigation exige transação, versão ou contrato de validade; fazer duas chamadas independentes não oferece atomicidade por si só.

Termo para lembrar: TOCTOU | Base técnica: referência 1.

15. Como garante ordem entre conteúdo e permissão?

Eu avaliaria versões ou tokens de consistência que preservem a relação causal necessária entre alterações e checks. Causal authorization evita decisões sobre uma visão antiga de permissões quando o conteúdo já reflete uma mudança relevante.

Termo para lembrar: Causal authorization | Base técnica: referência 2.

16. Toda leitura precisa da consistência mais forte?

Não. Eu classifico ações pelo risco e pela tolerância a mudanças recentes, documentando a janela aceita. Consistency budget permite otimizar consultas de baixo impacto sem aplicar a mesma tolerância a transferências ou exportações sensíveis.

Termo para lembrar: Consistency budget | Base técnica: referência 2.

17. O que faz se o autorizador cair?

Eu defino comportamento por ação, normalmente negando operações sensíveis sem decisão confiável. Fail-closed reduz risco de acesso indevido, mas precisa de alta disponibilidade e comunicação adequada para que indisponibilidade não seja confundida com permissão removida.

Termo para lembrar: Fail-closed | Base técnica: referência 1.

18. Usaria fail-open em pagamentos?

Eu não o adotaria como fallback genérico de autorização. Risk-based fallback só existe com regra de negócio explícita e controles próprios; disponibilidade não justifica permitir qualquer ação quando o serviço de políticas estiver inacessível.

Termo para lembrar: Risk-based fallback | Base técnica: referência 1.

19. Como modela acesso emergencial de suporte?

Eu uso privilégio temporário, justificativa, aprovação proporcional e auditoria, restringindo recursos e ações. Break-glass access precisa expirar e ser revisado; uma role permanente de superadmin compartilhada enfraquece rastreabilidade e isolamento.

Termo para lembrar: Break-glass access | Base técnica: referência 1.

20. Como autoriza listagens eficientes?

Eu aplico filtros autorizados ou mecanismos de listagem de recursos permitidos, evitando buscar tudo e filtrar tardiamente. Authorization-aware query deve impedir vazamento em contagens, paginação e agregações, além dos documentos retornados.

Termo para lembrar: Authorization-aware query | Base técnica: referência 1.

21. Como evita explosão de travessias no grafo?

Eu limito modelos, profundidade e custo, usando cache e índices conforme semântica de consistência. Relationship traversal precisa de orçamento para ciclos e caminhos amplos; expressividade sem limites pode transformar um check simples em consulta imprevisível.

Termo para lembrar: Relationship traversal | Base técnica: referência 2.

22. Como trata grupos aninhados?

Eu defino regras de herança, ciclos e precedência, validando mudanças com testes. Permission inheritance deve ser compreensível para administradores; uma relação aparentemente local pode conceder acesso a muitos recursos por caminhos indiretos.

Termo para lembrar: Permission inheritance | Base técnica: referência 2.

23. Como versiona políticas?

Eu publico versões testadas e observo decisões em modo sombra antes de ativar mudanças relevantes. Policy rollout precisa de rollback e auditoria, pois um pequeno ajuste pode negar acesso legítimo ou ampliar permissões em toda a organização.

Termo para lembrar: Policy rollout | Base técnica: referência 1.

24. Pode confiar em roles recebidas do frontend?

Não. Eu obtenho atributos autorizativos de fontes confiáveis e valido tokens segundo emissor e audiência esperados. Trusted attributes não são simplesmente campos presentes numa requisição; o cliente não pode escolher a própria autoridade.

Termo para lembrar: Trusted attributes | Base técnica: referência 3.

25. Como explica uma negação?

Eu retorno mensagem segura ao cliente e mantenho evidência interna da política e contexto usados, sem expor relações de terceiros. Decision audit ajuda suporte e investigação, mas também contém informações sensíveis que precisam de acesso controlado.

Termo para lembrar: Decision audit | Base técnica: referência 1.

26. Como testa autorização?

Eu construo matriz de sujeitos, ações e recursos, incluindo acessos cruzados, revogações e caminhos alternativos. Negative authorization testing tenta provar que o usuário indevido não acessa dados nem provoca efeitos, mesmo chamando APIs diretamente.

Termo para lembrar: Negative authorization testing | Base técnica: referência 1.

27. Como migra do JWT com roles para política central?

Eu introduzo checks compatíveis, comparo decisões e migro ações por risco, mantendo controle sobre versões antigas. Authorization migration precisa evitar que um fallback legado aceite uma operação negada pelo modelo novo durante a transição.

Termo para lembrar: Authorization migration | Base técnica: referência 1.

28. Como protege escrita de relações?

Eu autorizo quem pode conceder ou revogar cada vínculo e registro mudanças com identidade e versão. Permission administration é um domínio crítico: proteger somente a API de consulta deixa o atacante alterar o próprio grafo autorizativo.

Termo para lembrar: Permission administration | Base técnica: referência 2.

29. Como dimensiona latência do check?

Eu meço cauda, cache e custo de travessia dentro do orçamento da jornada, evitando chamadas redundantes sem semântica. Authorization latency precisa ser previsível, mas reduzir milissegundos não justifica aceitar permissões obsoletas além do contrato.

Termo para lembrar: Authorization latency | Base técnica: referência 2.

30. Qual seria sua conclusão?

Eu mostraria um modelo de regras simples o suficiente para explicar e forte o suficiente para os riscos identificados. Least privilege, revogação e enforcement verificável importam mais que escolher um produto sofisticado ou repetir o nome Zanzibar.

Termo para lembrar: Least privilege | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. RBAC

Ganho. Simplicidade de papéis

Custo e motivo. Pouca granularidade contextual

Como decidir. Para funções estáveis

02. ABAC

Ganho. Regras contextuais

Custo e motivo. Atributos e políticas complexos

Como decidir. Quando contexto muda a decisão

03. ReBAC

Ganho. Expressa compartilhamento

Custo e motivo. Travessias e consistência

Como decidir. Para relações entre sujeitos e recursos

04. PDP central

Ganho. Política uniforme

Custo e motivo. Dependência de rede

Como decidir. Com disponibilidade e latência adequadas

05. Decisão local

Ganho. Baixa latência

Custo e motivo. Distribuição de políticas

Como decidir. Com versões e atualização controladas

06. Cache de allow

Ganho. Menos chamadas

Custo e motivo. Revogação atrasada

Como decidir. Somente dentro da janela aceita

07. Cache de deny

Ganho. Protege backend

Custo e motivo. Concessão atrasada

Como decidir. Quando atraso de acesso é tolerável

08. Check forte

Ganho. Atualidade maior

Custo e motivo. Latência e disponibilidade

Como decidir. Para ações sensíveis

09. Token de consistência

Ganho. Preserva relação causal

Custo e motivo. Integração mais complexa

Como decidir. Quando conteúdo e permissão mudam juntos

10. Break-glass

Ganho. Recuperação emergencial

Custo e motivo. Privilégio excepcional

Como decidir. Com escopo, expiração e auditoria

Duas alternativas ao caminho proposto

1. Autorização relacional no próprio domínio

Quando faz sentido. Regras locais e escala moderada

Como implementar. Tabelas de vínculo e checks transacionais junto ao recurso

Limites. Difícil uniformizar muitos domínios independentes

2. Engine de políticas ABAC

Quando faz sentido. Decisões dominadas por atributos

Como implementar. Distribuir políticas versionadas e avaliar contexto confiável

Limites. Não resolve sozinho armazenamento e consistência de relações complexas

Referências e alcance da evidência

1. OWASP - Authorization

Menor privilégio, deny by default e autorização por requisição.

https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html

2. Google Research - Zanzibar

Modelo de relações e consistência causal de autorização no sistema descrito pelo artigo.

https://research.google/pubs/zanzibar-googles-consistent-global-authorization-system/

3. OpenID Foundation - OIDC Core

Camada de identidade, claims e validação de ID Tokens.

https://openid.net/specs/openid-connect-core-1_0.html

Continue em: T071 · T072 · T074 · C037

C039 / M. Os 50 cenários enviados

Gerenciamento e Rotação de Segredos

Segredos precisam de emissão, entrega, uso, rotação e revogação verificáveis. Atualizar um valor no cofre não atualiza automaticamente conexões e processos que o utilizam.

Quando usar. Credenciais de bancos, parceiros e serviços que precisam limitar exposição e manter continuidade durante mudanças.

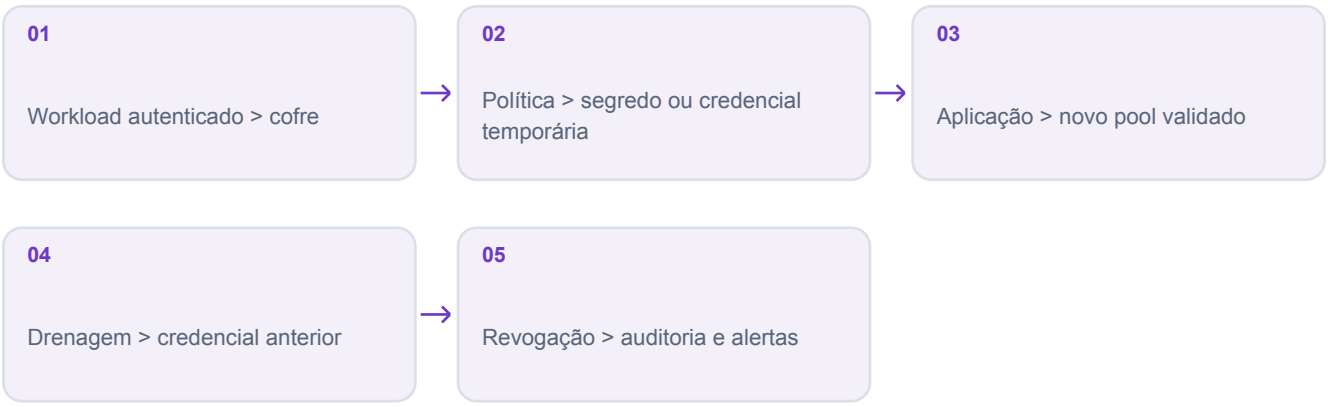
Quando evitar ou limitar. Guardar segredos no código, supor que toda credencial é renovável ou confundir identidade humana com credencial dinâmica de workload.

Uma abertura possível

Eu desenharia o ciclo completo do segredo, incluindo como cada aplicação recebe a nova versão e abandona a antiga. A rotação só termina quando o destino e os consumidores demonstram uso correto.

Peças e tecnologias. Vault · AWS Secrets Manager · Azure Key Vault · Workload identity · Connection pools · KMS

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Inventariar consumidores, destinos e tipos de segredo

2. Preferir identidade temporária quando suportada

3. Definir rotação com validação e sobreposição compatível

4. Atualizar clientes e drenar conexões antigas

5. Testar falhas do cofre, expiração e revogação emergencial

30 perguntas para sustentar o desenho

01. Por onde começa a gestão de segredos?

Eu inventario segredos, consumidores, privilégios e formas de revogação, definindo um dono por integração. Secret inventory evita rotação incompleta: uma credencial aparentemente usada por um serviço pode estar em jobs, scripts e ferramentas esquecidas.

Termo para lembrar: Secret inventory | Base técnica: referência 1.

02. Credencial dinâmica é sempre possível?

Não. Eu verifico suporte do destino e do mecanismo de emissão. Dynamic credentials funcionam quando posso criar e revogar identidades apropriadas; muitos parceiros oferecem apenas chaves estáticas, exigindo outro protocolo de rotação.

Termo para lembrar: Dynamic credentials | Base técnica: referência 2.

03. Vault troca qualquer senha automaticamente?

Ele depende de engines, plugins, permissões e configuração compatíveis com o destino. Secret engine não elimina integração: preciso testar criação, renovação e revogação na versão concreta do banco ou serviço utilizado.

Termo para lembrar: Secret engine | Base técnica: referência 2.

04. IAM Identity Center substitui esse fluxo?

Eu distinguiria identidade humana de identidade de workload e verificaria a capacidade específica necessária. Workload authentication deve usar o mecanismo adequado ao serviço; uma solução de acesso de pessoas não gera automaticamente senhas efêmeras para qualquer banco.

Termo para lembrar: Workload authentication | Base técnica: referência 1.

05. Por que evitar segredo na imagem?

Porque camadas, caches de build e registries podem preservar cópias mesmo após remoção posterior. Secret injection deve acontecer por canal controlado no runtime, com identidade restrita e sem expor valor em argumentos, logs ou artefatos.

Termo para lembrar: Secret injection | Base técnica: referência 1.

06. Variável de ambiente é sempre proibida?

Eu avalio riscos do runtime, dumps e ferramentas, mas o principal é entrega, acesso e lifecycle. Environment secret pode ser aceitável em contexto controlado; não resolve rotação automática nem impede exposição por processos privilegiados.

Termo para lembrar: Environment secret | Base técnica: referência 1.

07. Como a aplicação obtém o primeiro acesso?

Eu uso identidade de workload ou bootstrap controlado com privilégio mínimo, evitando uma senha mestre permanente distribuída manualmente. Secret zero não desaparece magicamente; a confiança inicial precisa de mecanismo verificável e revogável.

Termo para lembrar: Secret zero | Base técnica: referência 1.

08. Como define TTL?

Eu equilibro exposição, capacidade de renovação, duração das operações e tolerância à indisponibilidade do emissor. Credential TTL curto reduz janela de abuso, mas pode aumentar falhas e pressão no cofre se o sistema não renovar corretamente.

Termo para lembrar: Credential TTL | Base técnica: referência 2.

09. Toda lease pode ser renovada?

Não. Eu verifico se é renovável, prazo máximo e comportamento do engine, planejando obtenção de nova credencial quando necessário. Lease renewal não deve ser um loop infinito que presume validade prolongável sem limite.

Termo para lembrar: Lease renewal | Base técnica: referência 2.

10. Atualizar cofre atualiza o pool?

Não automaticamente. Eu implemento detecção de versão ou renovação, crio conexões com a credencial nova e dreno as antigas. Connection pool rotation precisa ser parte da aplicação ou plataforma, com erro e progresso observáveis.

Termo para lembrar: Connection pool rotation | Base técnica: referência 1.

11. Como evita downtime na troca?

Quando o destino permite, eu uso sobreposição controlada, valido a credencial nova e migro consumidores antes de revogar a antiga. Rotation overlap reduz interrupção, mas prolonga temporariamente a exposição e precisa de prazo e monitoramento.

Termo para lembrar: Rotation overlap | Base técnica: referência 1.

12. E se o destino só aceita uma senha?

Eu planejo uma troca coordenada e tolerância limitada a falhas de autenticação, ou uma janela quando necessário. Single-credential rotation não oferece sobreposição por desejo arquitetural; preciso trabalhar com as capacidades reais do destino.

Termo para lembrar: Single-credential rotation | Base técnica: referência 1.

13. Revogar senha encerra sessões existentes?

Depende do banco e do mecanismo. Eu testo autenticação nova e sessões já abertas, definindo se devo encerrá-las explicitamente. Session revocation não pode ser inferida apenas porque o cofre marcou a credencial como expirada.

Termo para lembrar: Session revocation | Base técnica: referência 2.

14. Como lida com transação em andamento?

Eu dreno o pool respeitando limites e sem interromper indiscriminadamente commits críticos, salvo necessidade de segurança. Graceful credential rollover distingue conexões novas de trabalho em andamento e registra resultado ambíguo se a conexão cair durante commit.

Termo para lembrar: Graceful credential rollover | Base técnica: referência 1.

15. O que faz se a nova credencial falhar?

Eu paro a promoção, preservo a versão anterior quando ainda segura e investigo permissões ou sincronização. Rotation validation precisa testar o acesso necessário, não apenas confirmar que uma nova string foi gravada no gerenciador.

Termo para lembrar: Rotation validation | Base técnica: referência 1.

16. Como evita todas as instâncias renovarem juntas?

Eu aplico jitter e antecedência de renovação, com limites de concorrência e cache local controlado. Renewal stampede pode derrubar o emissor justamente perto da expiração, então distribuo a carga sem ultrapassar a validade da credencial.

Termo para lembrar: Renewal stampede | Base técnica: referência 2.

17. Como opera quando o cofre cai?

Eu uso credenciais ainda válidas conforme política e sinalizo tempo restante, evitando ampliar validade sem autorização. Vault outage planning inclui disponibilidade, capacidade e recuperação; uma aplicação reiniciada pode não ter o mesmo cache de outra já ativa.

Termo para lembrar: Vault outage planning | Base técnica: referência 2.

18. Cache de segredo é perigoso?

Ele reduz latência e dependência do cofre, mas prolonga presença do valor e pode atrasar rotação. Secret caching precisa de expiração, proteção de memória e comportamento definido para credencial revogada ou versão mais nova.

Termo para lembrar: Secret caching | Base técnica: referência 1.

19. Como revoga após vazamento?

Eu identifico escopo, revogo ou substituo credenciais afetadas, investigo uso e verifico consumidores. Emergency rotation prioriza contenção conforme risco; não basta apagar o segredo do Git se ele já foi copiado ou utilizado.

Termo para lembrar: Emergency rotation | Base técnica: referência 1.

20. Como detecta segredo no Git?

Eu uso scanning preventivo e histórico, com processo de revogação ao encontrar exposição. Secret scanning reduz risco, mas não comprova ausência de vazamento em logs, tickets ou máquinas; credencial detectada deve ser tratada conforme seu potencial comprometimento.

Termo para lembrar: Secret scanning | Base técnica: referência 1.

21. Como audita sem registrar o valor?

Eu registro identidade, operação, recurso, versão e resultado, evitando conteúdo do segredo. Secret audit permite rastrear leitura e mudança com minimização, pois o sistema de auditoria não deve virar uma segunda base de credenciais.

Termo para lembrar: Secret audit | Base técnica: referência 1.

22. Quem pode ler e quem pode rotacionar?

Eu separo permissões conforme função, restringindo administração e acesso ao conteúdo. Separation of duties reduz o alcance de uma credencial comprometida; o consumidor cotidiano não precisa necessariamente criar políticas ou destruir segredos de outros serviços.

Termo para lembrar: Separation of duties | Base técnica: referência 1.

23. Como usa credencial por instância?

Eu avalio identidades específicas para limitar revogação e melhorar atribuição, considerando custo de emissão e quotas. Per-instance credential facilita investigação, mas não é sempre necessária; por serviço pode ser suficiente conforme risco e escala.

Termo para lembrar: Per-instance credential | Base técnica: referência 2.

24. Como protege segredo de parceiro em suporte?

Eu ofereço ferramentas que executam ações autorizadas sem revelar a credencial, com escopo e auditoria. Brokered access reduz compartilhamento humano e mantém o valor dentro de uma fronteira controlada, sem impedir operações legítimas de diagnóstico.

Termo para lembrar: Brokered access | Base técnica: referência 1.

25. Como testa permissões após rotação?

Eu executo operações mínimas representativas, incluindo leitura ou escrita autorizada e negação de ações proibidas. Privilege validation evita promover uma credencial funcional porém ampla demais, ou uma que passa health check mas falha no fluxo real.

Termo para lembrar: Privilege validation | Base técnica: referência 1.

26. Como recupera backup do cofre?

Eu ensaio restauração com proteção das chaves e reconcilio leases e credenciais com os destinos. Secret recovery precisa considerar que a realidade externa continuou mudando; restaurar metadata antiga pode reintroduzir referências inválidas ou desconhecer revogações.

Termo para lembrar: Secret recovery | Base técnica: referência 2.

27. Como gerencia segredo compartilhado por serviços?

Eu priorizo separar identidades quando possível; enquanto houver compartilhamento, coordeno consumidores e acompanho adoção da nova versão. Shared credential aumenta blast radius e dificulta atribuição, então sua rotação exige inventário especialmente confiável.

Termo para lembrar: Shared credential | Base técnica: referência 1.

28. Como trata certificado e chave privada?

Eu gerencio emissão, cadeia de confiança, distribuição e expiração, protegendo a chave privada e testando renovação. Certificate lifecycle inclui conexões e peers que validam a cadeia; trocar apenas um arquivo pode não atualizar o processo em execução.

Termo para lembrar: Certificate lifecycle | Base técnica: referência 1.

29. Qual métrica prova rotação concluída?

Eu acompanho versão ativa por consumidor, falhas de autenticação, conexões antigas e revogação efetiva no destino. Rotation completion é um estado observado do sistema inteiro, não apenas o sucesso de uma função que atualizou o cofre.

Termo para lembrar: Rotation completion | Base técnica: referência 1.

30. O que destacaria como Staff?

Eu mostraria o protocolo completo e seus limites, incluindo falha do emissor e destino sem sobreposição. Secret lifecycle management combina segurança e continuidade; escolher Vault ou um serviço cloud não substitui integração correta com os consumidores.

Termo para lembrar: Secret lifecycle management | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Credencial dinâmica

Ganho. Exposição limitada

Custo e motivo. Dependência de emissão

Como decidir. Quando destino suporta criação e revogação

02. Credencial estática rotacionada

Ganho. Compatibilidade ampla

Custo e motivo. Janela maior de exposição

Como decidir. Para parceiros sem emissão dinâmica

03. TTL curto

Ganho. Revogação natural rápida

Custo e motivo. Pressão de renovação

Como decidir. Com disponibilidade e integração adequadas

04. TTL longo

Ganho. Menos dependência imediata

Custo e motivo. Maior janela de abuso

Como decidir. Quando risco e operação justificam

05. Cache local

Ganho. Menos chamadas ao cofre

Custo e motivo. Segredo permanece em memória

Como decidir. Com validade e proteção claras

06. Busca por operação

Ganho. Atualidade maior

Custo e motivo. Latência e disponibilidade

Como decidir. Quando taxa e risco permitem

07. Identidade por instância

Ganho. Revogação granular

Custo e motivo. Mais identidades

Como decidir. Para alta necessidade de atribuição

08. Identidade por serviço

Ganho. Gestão simples

Custo e motivo. Blast radius do serviço

Como decidir. Quando escopo é aceitável

09. Sobreposição de versões

Ganho. Troca gradual

Custo e motivo. Duas credenciais válidas

Como decidir. Quando destino permite e prazo é limitado

10. Troca coordenada

Ganho. Compatível com segredo único

Custo e motivo. Risco de interrupção

Como decidir. Quando não há mecanismo de sobreposição

Duas alternativas ao caminho proposto

1. Identidade federada sem segredo persistente

Quando faz sentido. Destino aceita tokens temporários do workload

Como implementar. Trocar identidade de execução por credenciais de curta duração

Limites. Depende de suporte e configuração de confiança

2. Proxy de acesso a dados

Quando faz sentido. Aplicações podem delegar autenticação

Como implementar. Proxy mantém credenciais e gerencia conexões ao destino

Limites. Cria dependência central e exige isolamento e capacidade

Referências e alcance da evidência

1. OWASP - Secrets Management

Ciclo de vida, acesso, rotação e exposição de segredos.

https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html

2. HashiCorp - Database secrets engine

Credenciais dinâmicas, papéis estáticos, leases e suporte conforme plugin.

https://developer.hashicorp.com/vault/docs/secrets/databases

Continue em: T051 · T071 · T073 · T080

C040 / M. Os 50 cenários enviados

Consistência e Idempotência em Sagas Distribuídas

Sagas coordenam efeitos locais duráveis e compensações de negócio. Idempotência, ordenação e fencing resolvem problemas distintos e precisam ser aplicados nas fronteiras corretas.

Quando usar. Pagamentos, reservas e chargebacks que atravessam serviços ou parceiros sem transação ACID única.

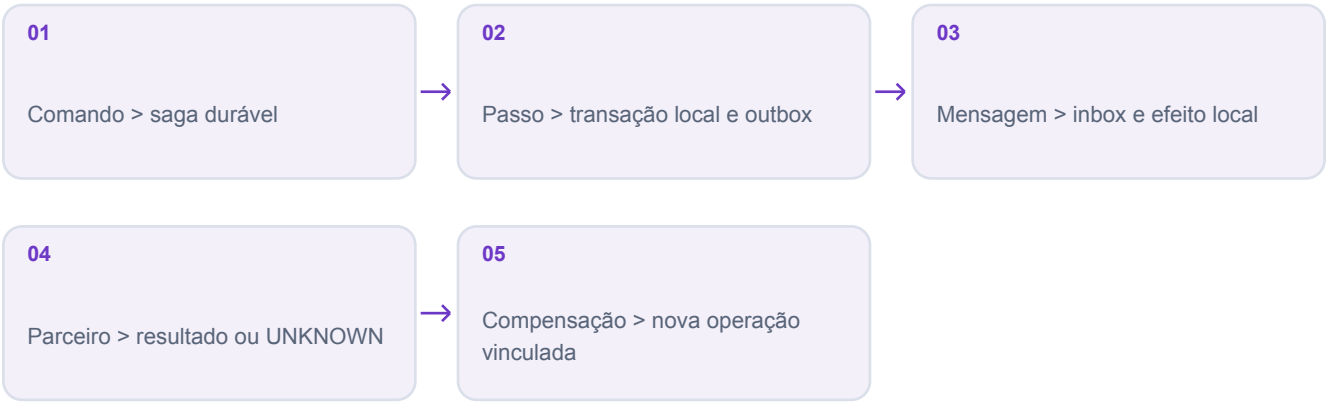
Quando evitar ou limitar. Confundir event_id com garantia de efeito único, tratar compensação como rollback técnico ou usar fencing como substituto universal de deduplicação.

Uma abertura possível

Eu identificaria cada efeito irreversível e daria uma identidade estável a cada passo. Depois mostraria commits locais, mensagens, estados desconhecidos e compensações, explicando exatamente o que acontece se cair entre duas etapas.

Peças e tecnologias. PostgreSQL · Transactional Outbox · Transactional Inbox · Kafka ou SQS · Temporal · Step Functions

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. Definir invariantes, passos e pontos de não retorno

2. Criar identidades estáveis de saga, passo e compensação

3. Persistir efeitos locais e obrigações atomicamente

4. Tratar duplicatas, lacunas e resultados externos desconhecidos

5. Testar replay, concorrência, compensação e recuperação após restore

30 perguntas para sustentar o desenho

01. Por que uma saga aqui?

Eu usaria saga quando o fluxo atravessa fronteiras que não compartilham uma transação local adequada. Saga coordination torna progresso e compensação explícitos, mas não oferece isolamento global nem elimina estados intermediários visíveis aos demais participantes.

Termo para lembrar: Saga coordination | Base técnica: referência 1.

02. Qual invariante começaria definindo?

Eu definiria que cada intenção produz no máximo o efeito econômico autorizado e que todo valor movimentado permanece contabilmente explicável. Business invariant orienta identidades, transições e compensações antes de escolher broker, orquestrador ou banco.

Termo para lembrar: Business invariant | Base técnica: referência 1.

03. Um transaction_id no evento basta?

Não. Eu preciso arbitrar atomicamente a aplicação daquele efeito no receptor e preservar sua identidade entre retries. Idempotency key é um dado do protocolo; sem armazenamento e regra de conflito, duas execuções ainda podem produzir dois efeitos.

Termo para lembrar: Idempotency key | Base técnica: referência 4.

04. Como separa IDs?

Eu uso saga_id para o fluxo, step_id para uma obrigação lógica e event_id para uma publicação, além de attempt_id para execução física. Identity hierarchy evita confundir nova tentativa com novo efeito ou deduplicar passos distintos da mesma saga.

Termo para lembrar: Identity hierarchy | Base técnica: referência 4.

05. Como protege um efeito local?

Eu gravo deduplicação, alteração de domínio e obrigações de saída na mesma transação quando compartilham o banco. Local atomicity impede marcar concluído sem efeito ou aplicar efeito sem registrar que já ocorreu naquela fronteira.

Termo para lembrar: Local atomicity | Base técnica: referência 2.

06. Como garante publicação após commit?

Eu uso transactional outbox e um relay recuperável, aceitando publicação repetida. Outbox relay não precisa de uma transação mágica com o broker, mas exige identidade e consumo idempotente para tolerar a janela entre publicação e marcação local.

Termo para lembrar: Transactional outbox | Base técnica: referência 2.

07. Inbox protege chamada HTTP externa?

Não por si só. Eu persisto uma obrigação de chamada e uso contrato idempotente ou reconciliação com o destino. External side effect fica fora da transação local; marcar inbox antes ou depois não elimina todas as janelas de falha.

Termo para lembrar: External side effect | Base técnica: referência 4.

08. Quando confirma a mensagem?

Eu confirmo após tornar o efeito local ou a obrigação durável conforme o protocolo. ACK after commit permite reentrega após queda sem perder trabalho; confirmar antes pode fazer o broker esquecer um passo que nunca foi aplicado.

Termo para lembrar: ACK after commit | Base técnica: referência 2.

09. E se cair depois do commit antes do ACK?

A mensagem pode voltar, e o receptor identifica a mesma obrigação já concluída sem repetir o efeito. Duplicate delivery é esperada nesse desenho; o teste precisa comprovar que a resposta e as obrigações persistidas permanecem coerentes.

Termo para lembrar: Duplicate delivery | Base técnica: referência 4.

10. Exactly-once é impossível em qualquer contexto?

Não. Existem garantias delimitadas, como transações em um sistema específico. Exactly-once scope precisa ser declarado; não amplio uma garantia do Kafka para um banco externo ou parceiro que não participa daquele protocolo transacional.

Termo para lembrar: Exactly-once scope | Base técnica: referência 5.

11. Produtor Kafka idempotente resolve a saga?

Não. Ele protege um escopo de produção, enquanto a saga envolve efeitos e estados em outros participantes. End-to-end idempotency depende de cada fronteira relevante, incluindo republicação, replay, APIs externas e compensações de negócio.

Termo para lembrar: End-to-end idempotency | Base técnica: referência 5.

12. Fencing e deduplicação são iguais?

Não. Fencing rejeita um detentor obsoleto por token validado no recurso, enquanto deduplicação reconhece repetição da mesma intenção. Posso precisar dos dois: um worker atual ainda pode receber duas vezes o mesmo comando.

Termo para lembrar: Fencing | Base técnica: referência 3.

13. Onde valida o fencing token?

No recurso que executa o efeito, atomicamente com a operação protegida, usando a regra de token definida. Resource-side fencing não funciona apenas porque o coordenador emitiu um número; o destino precisa efetivamente rejeitar a escrita obsoleta.

Termo para lembrar: Resource-side fencing | Base técnica: referência 3.

14. Lease expirado mata o worker?

Não. Ele pode continuar depois de pausa ou partição, por isso a posse temporária não basta para proteger efeitos. Stale owner precisa ser rejeitado pelo destino ou contido por outro protocolo que preserve a invariante.

Termo para lembrar: Stale owner | Base técnica: referência 3.

15. Como trata eventos fora de ordem?

Eu uso versão ou sequência por agregado e guardo mensagens que dependem de uma lacuna, com recuperação e alerta. Deferred processing não deve marcar como concluído um evento necessário que apenas chegou antes de sua precondição.

Termo para lembrar: Deferred processing | Base técnica: referência 4.

16. Pode ignorar qualquer versão antiga?

Somente se a semântica permitir e houver evidência de que o efeito correspondente já está incorporado. Version guard pode funcionar para estado absoluto, mas descartar deltas ou passos independentes por comparação ingênua pode perder operações válidas.

Termo para lembrar: Version guard | Base técnica: referência 4.

17. Como compensa uma reserva?

Eu crio uma operação de liberação vinculada à reserva original, com identidade e transição condicionada ao estado. Compensation deve considerar concorrência e uso parcial; não restauro cegamente um snapshot que apague mudanças legítimas posteriores.

Termo para lembrar: Compensation | Base técnica: referência 1.

18. Estorno deve reutilizar a chave do débito?

Eu uso identidade própria para a compensação, vinculada ao efeito original, e a preservo entre retries. Compensation identity distingue desfazer um efeito de repetir sua execução; reutilizar a mesma chave pode causar conflito semântico no receptor.

Termo para lembrar: Compensation identity | Base técnica: referência 4.

19. Compensação pode falhar?

Sim. Eu a trato como obrigação durável com estado, retry seguro, observabilidade e escalonamento. Compensating transaction não é um bloco finally infalível; a saga pode permanecer em recuperação até obter evidência suficiente de conclusão.

Termo para lembrar: Compensating transaction | Base técnica: referência 1.

20. Como lida com efeito externo UNKNOWN?

Eu mantenho estado explícito e consulto ou reconcilio pela identidade original antes de decidir repetir ou compensar. Unknown outcome impede assumir falha: o parceiro pode ter aplicado o efeito mesmo quando a resposta nunca chegou.

Termo para lembrar: Unknown outcome | Base técnica: referência 4.

21. Timeout deve disparar estorno imediatamente?

Não genericamente. Eu verifico se houve efeito e qual ação o contrato permite, evitando compensação que concorra com uma confirmação atrasada. Timeout handling precisa preservar a relação entre intenção, resultado remoto e operação de correção.

Termo para lembrar: Timeout handling | Base técnica: referência 4.

22. Como evita corrida entre cancelar e confirmar?

Eu uso máquina de estados e transições condicionais, tratando eventos atrasados como evidência a reconciliar. State-machine concurrency define precedência e ações compensatórias; timestamps de máquinas diferentes não resolvem sozinhos qual decisão financeira prevalece.

Termo para lembrar: State-machine concurrency | Base técnica: referência 1.

23. Orquestração é melhor que coreografia?

Depende da visibilidade e complexidade do fluxo. Orchestration facilita timers e acompanhamento central, enquanto coreografia reduz coordenação explícita, mas pode ocultar dependências; eu escolho conforme ownership e necessidade de explicar recuperação ponta a ponta.

Termo para lembrar: Orchestration | Base técnica: referência 1.

24. Como evita acoplamento excessivo no orquestrador?

Eu mantenho contratos de comandos e resultados, deixando invariantes locais nos domínios. Workflow ownership coordena sequência e compensação sem transformar o orquestrador num serviço que acessa diretamente todos os bancos e regras internas.

Termo para lembrar: Workflow ownership | Base técnica: referência 1.

25. Como apresenta estado ao usuário?

Eu exponho estados compreensíveis como recebido, em processamento e concluído, diferenciando pendência de rejeição definitiva. User-visible state deve refletir evidência durável e permitir consulta pelo mesmo identificador, sem prometer sucesso enquanto existe incerteza relevante.

Termo para lembrar: User-visible state | Base técnica: referência 1.

26. Como decide retenção da deduplicação?

Eu considero retries tardios, replay, restauração e prazo de reapresentação do negócio, preservando identidade enquanto efeitos puderem voltar. Deduplication retention curta pode permitir duplicata antiga; TTL não deve ser escolhido apenas para economizar storage.

Termo para lembrar: Deduplication retention | Base técnica: referência 4.

27. Como evolui schema durante saga longa?

Eu versiono comandos e estado, mantendo interpretação dos passos já iniciados e testando coexistência. Workflow versioning precisa preservar decisões anteriores; um deploy não pode reinterpretar silenciosamente uma obrigação financeira antiga com significado novo.

Termo para lembrar: Workflow versioning | Base técnica: referência 1.

28. Como recupera depois de restore?

Eu reconcilio progresso restaurado com efeitos externos e históricos independentes antes de retomar envios. Restore reconciliation evita repetir passos que o backup esqueceu, mas que outros participantes já concluíram e continuam considerando válidos.

Termo para lembrar: Restore reconciliation | Base técnica: referência 4.

29. Como testa a saga de verdade?

Eu injeto queda antes e depois de cada commit e chamada externa, além de duplicatas e ordem invertida. Failure matrix verifica invariantes e retomada por fronteira; happy path não demonstra correção de um protocolo distribuído.

Termo para lembrar: Failure matrix | Base técnica: referência 1.

30. Como concluiria a defesa da arquitetura?

Eu diria exatamente onde há atomicidade, onde há repetição e como resolvemos incerteza. Explicit guarantees tornam a solução defensável: idempotência, fencing, ordenação e compensação colaboram, mas nenhum desses mecanismos substitui automaticamente os demais.

Termo para lembrar: Explicit guarantees | Base técnica: referência 1.

10 trade-offs: ganho, custo e decisão

01. Saga

Ganho. Coordena fronteiras independentes

Custo e motivo. Estados intermediários

Como decidir. Quando transação única não atende

02. Transação local maior

Ganho. Atomicidade simples

Custo e motivo. Acoplamento e contenção

Como decidir. Quando domínio e storage permitem

03. Orquestração

Ganho. Fluxo e timers visíveis

Custo e motivo. Coordenador adicional

Como decidir. Para processos longos e auditáveis

04. Coreografia

Ganho. Autonomia de eventos

Custo e motivo. Fluxo implícito complexo

Como decidir. Para interações simples com ownership claro

05. Outbox

Ganho. Obrigação atômica

Custo e motivo. Relay e duplicatas

Como decidir. Para estado local mais publicação

06. Inbox

Ganho. Deduplicação local durável

Custo e motivo. Storage e retenção

Como decidir. Quando receptor aplica efeitos locais

07. Fencing

Ganho. Rejeita dono obsoleto

Custo e motivo. Suporte no recurso

Como decidir. Para posse exclusiva com execução atrasada

08. Compensação imediata

Ganho. Recuperação rápida

Custo e motivo. Pode conflitar com UNKNOWN

Como decidir. Somente com evidência e regra adequadas

09. Reconciliação

Ganho. Resolve incerteza externa

Custo e motivo. Latência e operação

Como decidir. Quando resposta não prova resultado

10. Retenção longa de IDs

Ganho. Protege replay tardio

Custo e motivo. Custo de storage

Como decidir. Quando obrigações podem reaparecer por longo período

Duas alternativas ao caminho proposto

1. Uma transação ACID local

Quando faz sentido. Capacidades podem compartilhar domínio e banco

Como implementar. Executar invariantes e mudanças atomicamente sem saga distribuída

Limites. Pode limitar autonomia e aumentar contenção

2. Workflow durável com atividades idempotentes

Quando faz sentido. Fluxo longo e necessidade de timers e histórico

Como implementar. Usar Temporal ou Step Functions com identidades e contratos por atividade

Limites. Orquestrador não torna efeitos externos atomicamente únicos

Referências e alcance da evidência

1. Microsoft - Compensating transaction

Compensação específica do negócio e consideração de mudanças concorrentes.

https://learn.microsoft.com/en-us/azure/architecture/patterns/compensating-transaction

2. AWS - Transactional outbox

Gravação atômica de estado e obrigação de publicar; duplicatas ainda possíveis.

https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/transactional-outbox.html

3. Hazelcast - FencedLock

Tokens monotônicos e rejeição de clientes obsoletos pelo recurso protegido.

https://docs.hazelcast.com/hazelcast/latest/data-structures/fencedlock

4. AWS Builders Library - Idempotent APIs

Identidades estáveis, intenção, retries e requisições atrasadas.

https://aws.amazon.com/builders-library/making-retries-safe-with-idempotent-APIs/

5. Apache Kafka - Design

Log particionado, offsets, replicação e fronteiras de garantias transacionais.

https://kafka.apache.org/41/design/design/

Continue em: T047 · T061 · T062 · T063 · T065

C041 / M. Os 50 cenários enviados

MLOps e Model Serving em Alta Concorrência (Scoring de Crédito)

Model serving transforma um artefato de modelo e features versionadas em uma previsão sob orçamento de latência e recursos. A decisão de crédito envolve ainda política, autorização, evidência e acompanhamento de resultado.

Quando usar. Quando previsões precisam entrar em jornadas com volume, controle de versões e recuperação, sem confundir treinamento com atendimento de requisições.

Quando evitar ou limitar. Evitar pickle não confiável, feature store como garantia automática de paridade e fallback que concede crédito só porque o modelo ficou indisponível.

Uma abertura possível

Eu separaria previsão de decisão. O modelo produz um score a partir de uma versão conhecida de features; uma política decide como usá-lo. Antes de exigir inferência em tempo real, eu confirmaria quanto frescor a jornada realmente precisa.

Peças e tecnologias. Feature pipelines · Feast ou equivalente · Model registry · Serving CPU/GPU · Canary e shadow · Monitoramento de dados e resultados

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. 0-8 min: definir previsão, decisão, frescor, latência, explicação e consequências de erro.

2. 8-18 min: modelar versões de features, modelo, política e dados disponíveis no instante da decisão.

3. 18-32 min: desenhar treinamento separado, publicação, serving e registro de evidência.

4. 32-48 min: simular feature ausente, modelo lento, drift, rollout e atraso de rótulos.

5. 48-60 min: comparar batch e online, custo, capacidade, governança e recuperação.

Cuidado técnico. Feature store não garante sozinha paridade entre treinamento e serving.

Cuidado técnico. Score e decisão de crédito são responsabilidades diferentes.

30 perguntas para sustentar o desenho

01. O modelo aprova crédito sozinho?

Não necessariamente. Ele produz uma previsão usada por uma política com outras regras e controles. Separo score, decisão e execução contratual, registrando versões. Isso permite explicar quando uma mudança veio do modelo ou da política.

Termo para lembrar: Objetivo

02. Toda previsão precisa ser calculada na hora?

Não. Se as variáveis mudam lentamente e a jornada aceita validade definida, batch pode reduzir custo e dependências. Inferência online entra quando frescor altera a decisão de forma relevante e justifica latência e operação adicionais.

Termo para lembrar: Online

03. Ela garante que treino e produção usam dados idênticos?

Não por si. Ajuda a compartilhar definições e recuperar dados, mas pipelines, versões, disponibilidade temporal e transformações ainda podem divergir. Testo paridade e registro linhagem. Instalar uma ferramenta não elimina training-serving skew.

Termo para lembrar: Feature store

04. Como evita usar informação do futuro no treino?

Faço joins com tempo de evento e, quando necessário, tempo de disponibilidade. Uma informação sobre o passado pode só ter chegado depois da decisão. Reproduzir o que era conhecido exige mais cuidado que filtrar apenas a data do fato.

Termo para lembrar: Tempo

05. Por que resultado de crédito demora a avaliar?

Porque inadimplência e outros desfechos aparecem depois da concessão e podem ter janelas diferentes. Métricas imediatas de API não medem qualidade preditiva. Planejo monitoramento inicial e avaliação posterior com coortes comparáveis.

Termo para lembrar: Rótulo

06. O que registra em cada decisão?

Referências de modelo, features e política, tempos relevantes, resultado e evidência necessária com minimização. Isso permite reconstruir a decisão dentro da retenção definida. Não dependo apenas do modelo atual no servidor.

Termo para lembrar: Versão

07. É seguro carregar qualquer arquivo pickle recebido?

Não. Formatos que executam código ao desserializar exigem origem e controle rigorosos. Uso artefatos aprovados, verificação de integridade e ambiente restrito. Conveniência de publicação não pode abrir execução arbitrária no serving.

Termo para lembrar: Artefato

08. Para que serve um registro de modelos?

Controla versões, metadados e promoção de artefatos, com relação a testes e aprovação. Não substitui avaliação nem processo decisório. A versão em produção precisa estar identificável e corresponder ao artefato realmente carregado.

Termo para lembrar: Registry

09. Preenche qualquer feature faltante com zero?

Não sem semântica treinada e validada. Zero pode significar um valor real e enviesar a previsão. Distingo ausente, atrasada e inválida, aplicando estratégia definida ou interrompendo a decisão quando a feature é necessária.

Termo para lembrar: Feature ausente

10. Como detecta feature velha?

Carrego timestamp e validade por feature ou conjunto, comparando com o requisito da decisão. Um cache hit não prova frescor. Também observo atraso do pipeline para evitar que dados antigos pareçam saudáveis por continuarem disponíveis.

Termo para lembrar: Frescor

11. Pode cachear o score do cliente?

Quando a decisão admite validade e a chave inclui versões e contexto relevantes. Mudança de dados ou política pode invalidar o resultado. Não reutilizo score antigo para uma operação diferente sem uma regra explícita.

Termo para lembrar: Cache

12. Onde costuma estar o gargalo?

Pode estar na recuperação de features, serialização, filas ou carregamento, não só no cálculo do modelo. Meço cada etapa e a cauda. Escolher GPU antes de identificar o custo dominante pode aumentar despesas sem melhorar a jornada.

Termo para lembrar: Latência

13. Quando GPU compensa?

Quando o modelo e o volume aproveitam paralelismo suficiente para compensar transferência, batching e custo. Modelos menores podem atender melhor em CPU. Comparo custo por inferência e latência sob carga real, incluindo baixa utilização.

Termo para lembrar: GPU

14. Agrupar inferências aumenta sempre desempenho útil?

Pode melhorar throughput, mas adiciona espera e pode prejudicar prazo individual. Limito tamanho e tempo de formação do lote. Uma boa taxa de inferências por segundo não basta se clientes passam do deadline.

Termo para lembrar: Batching

15. Como evita esgotar memória do serving?

Limito requisições em voo, tamanho de entrada e filas, dimensionando cópias de modelo e buffers. Backpressure deve ocorrer antes do colapso. Escalar réplicas sem considerar memória e tempo de carregamento pode chegar tarde.

Termo para lembrar: Concorrência

16. Uma réplica pronta pode atender imediatamente?

Preciso distinguir processo vivo de modelo carregado e aquecido conforme o runtime. Testo readiness e capacidade inicial. Direcionar pico para réplicas frias pode piorar latência mesmo quando o autoscaler criou instâncias suficientes.

Termo para lembrar: Warmup

17. Se o modelo cair, aprova por padrão?

Não. A política de risco define rejeitar, adiar, revisar ou usar alternativa validada. A escolha precisa ser conhecida antes do incidente. Uma falha técnica não autoriza engenharia a conceder crédito sem os controles exigidos.

Termo para lembrar: Fallback

18. Como testa modelo novo sem afetar cliente?

Executo a previsão em paralelo com isolamento de recursos e sem efeitos de decisão, comparando resultados. Ainda cuido de dados e custo. Shadow não demonstra automaticamente qualidade final quando os rótulos reais só chegam depois.

Termo para lembrar: Shadow

19. Como libera uma versão gradualmente?

Escolho coortes e limites, monitoro erros técnicos e indicadores adequados, preservando possibilidade de interromper. Comparações precisam considerar diferenças de população. Canary de modelo exige mais que observar HTTP duzentos.

Termo para lembrar: Canary

20. Reverter modelo desfaz decisões já tomadas?

Não. Afeta próximas inferências; decisões anteriores exigem tratamento de negócio se houver problema. Preservo versões e evidência para identificar impacto. Um botão de rollback não reverte contratos ou dinheiro automaticamente.

Termo para lembrar: Rollback

21. Mudança de distribuição prova que o modelo piorou?

Não necessariamente. É um sinal que exige investigação junto de qualidade de dados, população e resultados. Posso ter drift sem degradação relevante ou degradação sem alerta evidente. Defino resposta proporcional e revisão.

Termo para lembrar: Drift

22. Detectou drift, treina e promove automaticamente?

Não como regra universal. Treinamento pode ser automatizado, mas promoção precisa de critérios de qualidade, risco e compatibilidade. Dados recentes também podem estar corrompidos ou incompletos. Automatizar sem controles pode amplificar o problema.

Termo para lembrar: Retraining

23. Como fornece uma decisão compreensível?

Registro fatores e regras permitidos pelo modelo e pelo processo, com explicação adequada ao usuário e aos responsáveis. Não invento uma justificativa textual depois. A exigência concreta deve ser definida com produto e especialistas.

Termo para lembrar: Explicação

24. Quem pode consultar features sensíveis?

Somente consumidores autorizados por finalidade e escopo, com auditoria e minimização. Feature store não deve virar um banco universal aberto. Separar treino e serving também ajuda a controlar dados e permissões distintas.

Termo para lembrar: Acesso

25. Como testa paridade de features?

Comparo transformações e recuperação online com exemplos históricos controlados, incluindo ausência, atraso e atualização. Verifico tipos e versões. O teste precisa reproduzir disponibilidade temporal, não apenas igualdade de nomes das colunas.

Termo para lembrar: Teste

26. Repetir uma consulta de score é sempre inofensivo?

O cálculo pode ser sem efeito, mas registro de decisão, contratação ou consulta cobrada podem ter consequências. Separo previsão de comando de negócio e uso identidade onde há efeito. Uma segunda previsão pode usar dados diferentes.

Termo para lembrar: Idempotência

27. Como estima capacidade e orçamento?

Meço custo por inferência, armazenamento e atualização de features, treinamento e observabilidade. Considero pico, reserva e tempo de carregamento. Um modelo mais preciso pode não compensar custo ou latência na jornada específica.

Termo para lembrar: Custo

28. Quais sinais acompanham o serviço?

Latência, erros, saturação, idade e ausência de features, versões ativas e resultados por coorte. Qualidade técnica e preditiva são dimensões diferentes. Um endpoint disponível pode servir previsões incorretas de forma consistente.

Termo para lembrar: Monitoramento

29. Quando escolheria regras determinísticas?

Quando o problema é simples, exige comportamento explícito ou não há dados suficientes para validar modelo. Regras também precisam de versões e testes. Machine learning é uma opção para o objetivo, não uma marca de maturidade obrigatória.

Termo para lembrar: Alternativa

30. Como encerra a proposta?

Mostro dados temporalmente corretos, artefato aprovado e decisão versionada, com limites de falha. Explico quando batch basta e como detectar degradação. A arquitetura conecta qualidade de dados, custo e consequência da decisão.

Termo para lembrar: Conclusão

10 trade-offs: ganho, custo e decisão

01. Batch versus online

Ganho. Batch reduz custo e dependências na jornada

Custo e motivo. Pode usar informação menos fresca

Como decidir. Online quando frescor muda a decisão de modo relevante

02. CPU versus GPU

Ganho. CPU pode simplificar modelos pequenos

Custo e motivo. Menor paralelismo para certas cargas

Como decidir. GPU quando utilização e modelo justificam custo

03. Batching versus inferência imediata

Ganho. Melhora throughput por recurso

Custo e motivo. Adiciona espera para formar lote

Como decidir. Limitar pela latência permitida

04. Feature store versus pipelines específicos

Ganho. Compartilha definições e acesso

Custo e motivo. Adiciona plataforma e governança

Como decidir. Quando reuso e número de modelos pagam a complexidade

05. Score em cache versus recálculo

Ganho. Reduz latência e custo

Custo e motivo. Pode ficar inválido com dados ou versão nova

Como decidir. Usar validade e chave coerentes com contexto

06. Shadow versus canary

Ganho. Shadow não muda decisões

Custo e motivo. Duplica custo e não mede todos os efeitos reais

Como decidir. Canary após validação e com limites de risco

07. Modelo complexo versus simples

Ganho. Pode capturar relações mais ricas

Custo e motivo. Aumenta custo, explicação e manutenção

Como decidir. Escolher por ganho comprovado na jornada

08. Fallback conservador versus continuidade

Ganho. Protege controles quando falta evidência

Custo e motivo. Pode interromper decisões legítimas

Como decidir. Política explícita conforme risco e obrigação

09. Retenção de evidência versus minimização

Ganho. Permite reconstruir decisões

Custo e motivo. Acumula dados sensíveis

Como decidir. Guardar o necessário com acesso e prazo definidos

10. Retraining frequente versus estabilidade

Ganho. Adapta a mudanças mais rápido

Custo e motivo. Pode introduzir variação e dados ainda imaturos

Como decidir. Promover com critérios e resultados verificáveis

Duas alternativas ao caminho proposto

1. Scoring batch com validade explícita

Quando faz sentido. Features e jornada toleram atualização periódica

Como implementar. Calcular scores versionados e revalidar condições críticas na contratação

Limites. Não serve quando informação fresca muda imediatamente o risco

2. Serviço online enxuto com modelo simples

Quando faz sentido. Baixa latência e poucas features bastam

Como implementar. Carregar artefato aprovado, buscar dados tipados e aplicar política separada

Limites. Exige evolução quando volume, modelos ou features aumentarem

Referências e alcance da evidência

1. Feast: point-in-time joins

Recuperação histórica temporal; disponibilidade real e regras de dados precisam ser consideradas.

https://docs.feast.dev/getting-started/concepts/point-in-time-joins

2. Feast: feature retrieval

Padrões de recuperação online e offline de features.

https://docs.feast.dev/getting-started/concepts/feature-retrieval

3. Python: pickle

Riscos de desserialização de dados não confiáveis.

https://docs.python.org/3/library/pickle.html

Continue em: T087 · T088 · T089

C042 / M. Os 50 cenários enviados

Agendamento Distribuído de Jobs (Distributed Cron)

Agendamento distribuído cria execuções lógicas a partir de calendários e políticas. O disparo pode repetir ou atrasar; a correção dos efeitos depende de identidade, estado durável e controle de concorrência.

Quando usar. Quando rotinas precisam cumprir prazos, recuperar execuções perdidas e distribuir trabalho por várias instâncias.

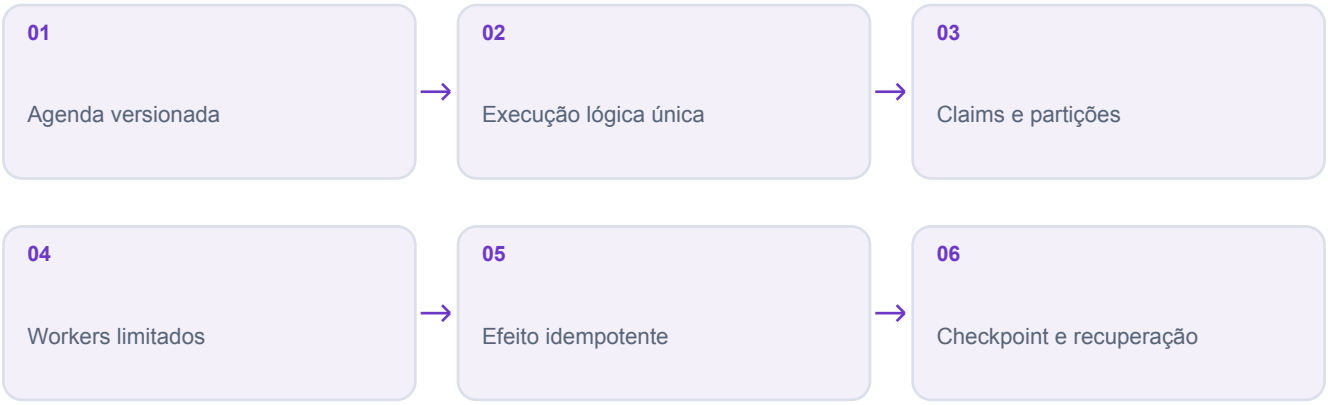
Quando evitar ou limitar. Evitar declarar exactly-once porque há eleição de líder e tratar CronJob como inadequado sem avaliar a tarefa.

Uma abertura possível

Eu separaria agenda, execução lógica e tentativa. Um scheduler pode repetir; a execução financeira não pode repetir seu efeito. Por isso a identidade inclui a janela ou unidade de trabalho, e a recuperação consulta estado durável antes de continuar.

Peças e tecnologias. Kubernetes CronJob · Temporal Schedules · PostgreSQL · Leases e fencing · Filas particionadas · Checkpoints

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. 0-8 min: definir calendário, fuso, prazo, política de atraso e sobreposição.

2. 8-18 min: modelar agenda, janela lógica, execução, tentativa e checkpoint.

3. 18-32 min: desenhar disparo redundante, claims, partições e efeito protegido.

4. 32-48 min: simular líder isolado, execução longa, relógio, duplicata e retomada.

5. 48-60 min: comparar ferramenta pronta e scheduler próprio, medir atraso e capacidade.

Cuidado técnico. Eleição de líder não garante exactly-once para efeitos externos.

Cuidado técnico. CronJob não é uma escolha inferior por definição; depende da tarefa e das garantias adicionais.

30 perguntas para sustentar o desenho

01. Qual diferença entre agenda e execução?

Agenda define quando e com quais regras iniciar trabalho. Execução identifica uma ocorrência lógica, enquanto tentativa representa uma execução concreta por um worker. Essa separação permite retentar sem criar um novo efeito de negócio.

Termo para lembrar: Modelo

02. Como identifica o job diário de rendimento?

Uso a unidade de negócio e a janela aplicável, incluindo versão quando isso fizer parte da semântica. Não uso só UUID de tentativa. A mesma obrigação diária precisa continuar reconhecível após reinício ou mudança de scheduler.

Termo para lembrar: Identidade

03. Kubernetes CronJob pode atender um sistema financeiro?

Pode, se a tarefa for idempotente e as políticas de horário, atraso e concorrência atenderem o requisito. O scheduler não precisa prometer perfeição para ser útil. A aplicação protege o efeito e acompanha execuções esperadas.

Termo para lembrar: CronJob

04. Eleger um líder garante efeito único?

Não. Um líder antigo pode continuar executando ou uma tentativa pode ter produzido efeito antes de cair. Eleição coordena agendamento; identidade durável e, quando necessário, fencing protegem efeitos e posse no destino.

Termo para lembrar: Leader election

05. Onde valida o token de posse?

No recurso que aceita a operação protegida, com uma regra capaz de rejeitar proprietários obsoletos. Só carregar token na mensagem não basta. Para efeito externo sem suporte, preciso de identidade contratual e recuperação adicional.

Termo para lembrar: Fencing

06. O lease expirou. Posso assumir que o worker parou?

Não. Ele pode estar pausado, isolado ou executando uma chamada externa. Outro worker pode assumir trabalho, mas o sistema deve impedir ou tolerar o efeito do antigo. Expiração é evidência de posse perdida, não de morte.

Termo para lembrar: Lease

07. O job anterior não terminou. Roda o próximo?

A política pode permitir, enfileirar, substituir ou ignorar, conforme a tarefa. Explicito consequências para janelas de negócio. Um fechamento financeiro e uma coleta de métricas não precisam da mesma regra de sobreposição.

Termo para lembrar: Sobreposição

08. O scheduler ficou fora por seis horas. Executa tudo atrasado?

Depende da política de catch-up, prazo e utilidade. Posso recuperar cada janela, consolidar trabalho ou marcar perda com tratamento explícito. Executar tudo de uma vez pode sobrecarregar dependências e produzir resultados sem sentido.

Termo para lembrar: Atraso

09. Por que não usar o relógio local de cada pod?

Porque calendário e fuso precisam ser consistentes e explícitos. Mudanças sazonais podem repetir ou eliminar horários em algumas regiões. Armazeno instantes e regras necessárias, e testo transições relevantes para o produto.

Termo para lembrar: Fuso

10. NTP resolve toda disputa de agendamento?

Ajuda sincronização, mas não substitui identidade e coordenação. Relógios podem saltar ou divergir temporariamente. A elegibilidade da execução precisa tolerar essas condições sem criar efeitos duplicados ou perder janelas silenciosamente.

Termo para lembrar: Relógio

11. Como divide milhões de contas entre workers?

Crio unidades estáveis de trabalho por faixa ou chave, com progresso e limites. O scheduler cria a execução; workers processam partições recuperáveis. Evito uma mensagem gigante com todos os clientes e uma transação única.

Termo para lembrar: Partições

12. Quando grava o progresso?

Depois de confirmar a unidade de efeito, ou junto dela quando possível. Se o checkpoint avança antes, pode pular trabalho; se vem depois, pode repetir. Idempotência da unidade fecha a janela de retomada.

Termo para lembrar: Checkpoint

13. Como dois workers não pegam a mesma partição?

Uso claim condicional ou transacional com validade e versão, adequado ao armazenamento. Mesmo assim, considero perda de posse e execução antiga. O claim reduz concorrência indesejada; o efeito precisa continuar protegido.

Termo para lembrar: Claim

14. Toda falha merece repetir a partição inteira?

Não. Distingo unidade concluída, erro transitório e dado inválido. Retomo de checkpoints e preservo identidade. Repetir uma partição inteira pode ser aceitável se cada item for seguro, mas o custo precisa ser medido.

Termo para lembrar: Retry

15. Um registro inválido impede o lote inteiro?

Defino se ele deve bloquear a consistência do conjunto ou virar exceção isolada. Não pulo silenciosamente uma obrigação financeira. Registro motivo e dono, permitindo que itens independentes avancem quando a regra permitir.

Termo para lembrar: Poison item

16. O worker caiu após chamar um parceiro. Como retoma?

Consulta a referência estável ou reconcilia o resultado antes de recriar a intenção. O estado local pode ser ambíguo. Marcar a tentativa como failed não prova que o parceiro deixou de executar o efeito.

Termo para lembrar: UNKNOWN

17. Cancelar a execução para tudo imediatamente?

Não necessariamente. Sinalizo cancelamento e impeço novas unidades, mas chamadas já iniciadas podem concluir. Defino compensação e estado final conforme a tarefa. O painel precisa distinguir cancelamento solicitado de trabalho efetivamente encerrado.

Termo para lembrar: Cancelamento

18. O prazo do job é igual ao prazo de cada chamada?

Não. Existe orçamento total, por unidade e por dependência. Reservo tempo para registrar progresso e recuperar. Uma chamada que consome toda a janela pode impedir encerramento seguro e atrasar outras unidades.

Termo para lembrar: Timeout

19. Como evita pico na virada do dia?

Espalho início dentro da janela permitida, limito concorrência e priorizo prazos. O corte lógico pode permanecer igual mesmo com processamento distribuído no tempo. Jitter operacional não deve alterar a data financeira aplicada.

Termo para lembrar: Capacidade

20. Um job enorme pode impedir rotinas pequenas?

Se todos compartilham fila sem política, sim. Defino classes, limites e recursos reservados quando necessário. O agendador precisa considerar prazo e custo, evitando que volume de uma tarefa elimine progresso das demais.

Termo para lembrar: Fairness

21. Mudou o código enquanto o job rodava. O que faz?

Vinculo execução à versão relevante ou garanto compatibilidade de retomada. Dados e checkpoints precisam continuar interpretáveis. Um deploy não deve fazer metade das contas usar uma regra e a outra metade outra sem intenção.

Termo para lembrar: Versionamento

22. Como permite rerun sem duplicar?

A interface diferencia retomar a mesma execução de criar uma nova operação de negócio. Preserva identidade, escopo e autorização. O botão de rerun não deve simplesmente gerar novos UUIDs para tudo que já foi aplicado.

Termo para lembrar: Reexecução manual

23. Quais métricas mostram cumprimento da agenda?

Execuções esperadas versus criadas, atraso de início, conclusão no prazo, progresso, exceções e idade de UNKNOWN. Contar pods bem-sucedidos não prova que todas as unidades financeiras foram processadas corretamente.

Termo para lembrar: Observabilidade

24. Como explica quem disparou uma rotina extraordinária?

Registro origem, motivo, parâmetros, versão e escopo autorizados, ligados à execução. Uma rotina manual pode ter o mesmo mecanismo seguro da automática. Evito scripts ad hoc sem rastreabilidade sobre dados de produção.

Termo para lembrar: Auditoria

25. Backup restaurado pode repetir jobs antigos?

Sim, se o estado voltar atrás enquanto efeitos externos permanecerem. Recuperação precisa reconciliar janelas e identidades antes de retomar. O scheduler não pode assumir que ausência no banco restaurado significa que o efeito nunca ocorreu.

Termo para lembrar: DR

26. Como testa calendário e falhas sem esperar dias?

Uso relógio controlável e disparos de teste com janelas explícitas. Simulo duplicata, atraso, sobreposição, queda após efeito e retomada. Verifico unidades concluídas e invariantes, não apenas horário de criação do pod.

Termo para lembrar: Teste

27. Quando Temporal ajuda mais que cron?

Quando há coordenação longa, etapas, espera por eventos e recuperação complexa. Cron pode iniciar uma tarefa simples com estado próprio. Temporal reduz infraestrutura de workflow, mas efeitos externos continuam exigindo idempotência.

Termo para lembrar: Ferramenta

28. Vale construir um scheduler próprio?

Somente se requisitos e escala justificarem manter calendário, recuperação, observabilidade e operação. Ferramentas existentes podem atender. O custo principal não é desenhar uma tabela de horários, mas lidar corretamente com anos de falhas e mudanças.

Termo para lembrar: Custo

29. Qual risco de um job privilegiado?

Ele pode afetar muitos registros rapidamente. Restringe-se escopo, credenciais e parâmetros, com dry-run útil e controle de acesso. Uma agenda interna não dispensa proteção contra configuração errada ou execução não autorizada.

Termo para lembrar: Segurança

30. Como fecha a garantia?

O calendário cria obrigações identificáveis; workers podem repetir tentativas, mas a autoridade protege cada efeito. Checkpoints e reconciliação garantem progresso recuperável. Não preciso prometer um disparo único para entregar processamento correto.

Termo para lembrar: Conclusão

10 trade-offs: ganho, custo e decisão

01. Cron simples versus workflow durável

Ganho. Cron tem operação e conceito simples

Custo e motivo. Coordenação longa precisa de estado adicional

Como decidir. Workflow quando etapas e recuperação justificam

02. Líder único versus disparos concorrentes idempotentes

Ganho. Líder reduz duplicação de agendamento

Custo e motivo. Exige eleição e ainda não garante efeito único

Como decidir. Escolher pela escala e custo de coordenação

03. Catch-up completo versus consolidação

Ganho. Completo preserva cada janela

Custo e motivo. Pode gerar tempestade após indisponibilidade

Como decidir. Consolidar somente se a semântica permitir

04. Partições pequenas versus grandes

Ganho. Melhoram retomada e distribuição

Custo e motivo. Aumentam metadados e overhead

Como decidir. Ajustar por custo de item e prazo

05. Checkpoint frequente versus espaçado

Ganho. Reduz trabalho repetido

Custo e motivo. Aumenta escrita e coordenação

Como decidir. Balancear custo de progresso e reexecução

06. Sobreposição versus serialização

Ganho. Aumenta throughput de janelas independentes

Custo e motivo. Pode disputar recursos e invariantes

Como decidir. Serializar quando a mesma autoridade exige ordem

07. Agenda exata versus janela flexível

Ganho. Exata atende compromissos específicos

Custo e motivo. Concentra picos e exige capacidade reservada

Como decidir. Espalhar quando horário de processamento não muda negócio

08. Cancelamento cooperativo versus interrupção forçada

Ganho. Permite terminar unidade com segurança

Custo e motivo. Pode levar mais tempo

Como decidir. Interrupção exige recuperação e efeitos ambíguos tratados

09. Scheduler próprio versus gerenciado

Ganho. Próprio oferece controle específico

Custo e motivo. Exige manter muitos modos de falha

Como decidir. Usar pronto quando contratos atendem

10. Retry automático versus exceção revisada

Ganho. Recupera falhas transitórias sem intervenção

Custo e motivo. Pode repetir erro permanente ou ambíguo

Como decidir. Classificar causas e preservar identidade

Duas alternativas ao caminho proposto

1. Kubernetes CronJob com execução idempotente em banco

Quando faz sentido. Tarefa periódica simples e ambiente já usa Kubernetes

Como implementar. Criar ocorrência por janela, processar unidades com checkpoints e monitorar atrasos

Limites. Cron não substitui deduplicação nem recuperação de efeitos externos

2. Temporal Schedules e workflows

Quando faz sentido. Coordenação duradoura, políticas de sobreposição e etapas complexas

Como implementar. Agendar workflows versionados e usar activities com identidade e limites

Limites. Adiciona plataforma e exige entender replay, retries e efeitos externos

Referências e alcance da evidência

1. Kubernetes: CronJob

Semântica de agendamento, concorrência e limitações que exigem jobs idempotentes.

https://kubernetes.io/docs/concepts/workloads/controllers/cron-jobs/

2. Temporal: Schedule

Agendamento, políticas e execução de workflows.

https://docs.temporal.io/schedule

3. AWS: retries idempotentes

Identidade de intenção e segurança de novas tentativas.

https://aws.amazon.com/builders-library/making-retries-safe-with-idempotent-APIs/

Continue em: T048 · T061 · T064 · T065

C043 / M. Os 50 cenários enviados

Defesa L7 na Borda (Edge Computing e WAF Avançado)

Defesa de borda combina filtragem, limitação e validação inicial para reduzir abuso antes de consumir a origem. Autorização e invariantes do negócio continuam sendo verificadas no domínio responsável.

Quando usar. Quando APIs expostas recebem tráfego abusivo, picos e entradas inválidas que podem esgotar recursos ou comprometer jornadas.

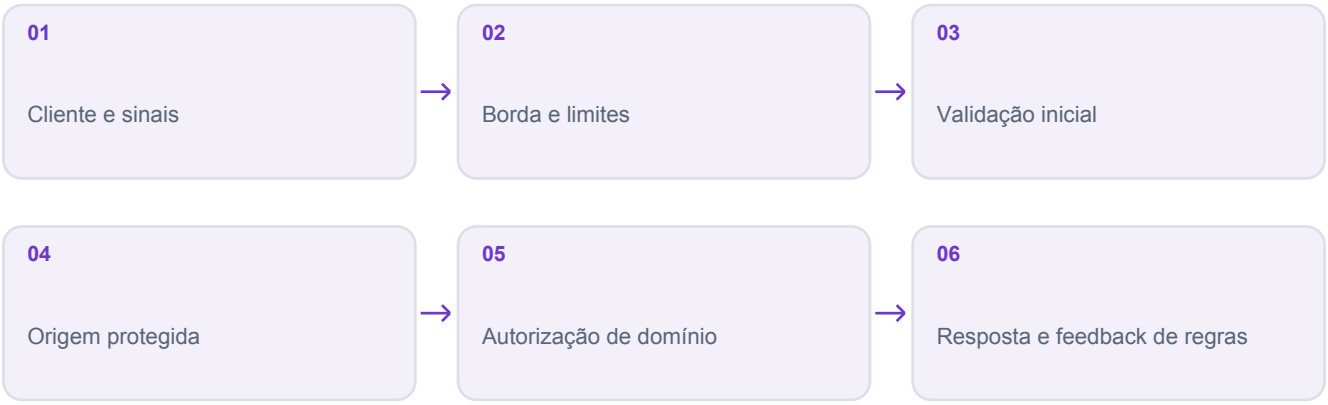
Quando evitar ou limitar. Evitar tratar WAF como autorização, confiar apenas em endereço IP e distribuir lógica financeira complexa por funções de borda.

Uma abertura possível

Eu protegeria capacidade e identidade em camadas. A borda rejeita tráfego claramente inválido e aplica limites; a aplicação decide o que aquela pessoa pode fazer. A defesa também precisa medir quantos clientes legítimos ela impede de acessar a própria conta.

Peças e tecnologias. CDN e WAF · Rate limiting · Validação JWT · Bot signals · Origin authentication · Observabilidade por jornada

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. 0-8 min: definir endpoints críticos, abuso observado, capacidade e impacto de falsos positivos.

2. 8-18 min: separar validação na borda, autenticação e autorização de negócio.

3. 18-32 min: desenhar regras, limites por identidade, proteção da origem e rollout.

4. 32-48 min: simular pico, rotação de chave, origem exposta e regra que bloqueia clientes reais.

5. 48-60 min: comparar custo, dependência de fornecedor, fallback e métricas de eficácia.

Cuidado técnico. WAF e validação de token não substituem autorização de domínio.

Cuidado técnico. Aproximações de rate limit não devem arbitrar saldo financeiro.

30 perguntas para sustentar o desenho

01. O que o WAF consegue decidir?

Ele aplica regras sobre tráfego e sinais disponíveis, bloqueando padrões e entradas conforme configuração. Não conhece automaticamente ownership de contas ou limite financeiro. Autorização e regras do domínio precisam continuar na aplicação.

Termo para lembrar: Escopo

02. Se existe WAF, posso deixar a origem pública sem proteção?

Não como desenho pretendido. Um atacante pode tentar contornar a borda. Restrinjo acesso à origem por mecanismos apropriados, autentico o caminho e verifico configurações. A defesa só funciona se o tráfego relevante passar pelos controles.

Termo para lembrar: Origem

03. Validar JWT na borda substitui validação na API?

Pode reduzir trabalho, mas preciso de um modelo de confiança explícito entre borda e origem. Verifico emissor, público, algoritmo e validade conforme contrato. A aplicação ainda autoriza o recurso e não confia em headers forjáveis.

Termo para lembrar: JWT

04. Como lida com rotação da chave de assinatura?

Mantenho cache e atualização controlados do conjunto autorizado, com tratamento de chave desconhecida e indisponibilidade. Não busco uma URL arbitrária indicada pelo token. Política de fallback precisa preservar autenticação, sem aceitar assinatura não verificada.

Termo para lembrar: Chaves

05. Um JWT válido garante acesso até expirar?

Depende da política de sessão e revogação. Tokens autossuficientes criam uma janela entre mudança de estado e expiração se não houver consulta adicional. Escolho validade e controles proporcionais, sem prometer revogação instantânea gratuita.

Termo para lembrar: Revogação

06. Limitar por IP resolve abuso de login?

Só parcialmente. IP pode ser compartilhado por clientes legítimos ou distribuído por atacantes. Combino dimensões apropriadas, como conta e sinais de dispositivo, com cuidado de privacidade e falsos positivos. Nenhuma dimensão isolada resolve todo abuso.

Termo para lembrar: Rate limit

07. Uma quota global pode ser estrita em toda borda?

Exige coordenação ou desenho específico; contadores locais podem permitir excesso agregado. Para proteção de capacidade, aproximação pode bastar. Para uma invariante financeira exata, não uso o contador da borda como autoridade.

Termo para lembrar: Quota

08. Token bucket ou janela fixa?

Token bucket permite controlar taxa sustentada e rajadas; janela fixa é simples, mas pode concentrar tráfego nas fronteiras. Escolho pelo comportamento desejado e custo. A política precisa dizer se uma rajada legítima é aceitável.

Termo para lembrar: Algoritmo

09. Todo tráfego automatizado é malicioso?

Não. Integrações e recursos legítimos podem ser automatizados. Uso sinais e políticas por endpoint e finalidade, com meios seguros de acesso para parceiros. Bloquear por um rótulo genérico pode interromper fluxos válidos.

Termo para lembrar: Bot

10. CAPTCHA em todo pedido bancário é uma boa defesa?

Pode criar atrito e excluir usuários sem resolver abuso sofisticado. Aplico desafios quando adequados ao canal e ao risco, com acessibilidade e alternativas. Não transformo toda transação legítima em uma barreira indiscriminada.

Termo para lembrar: Challenge

11. Bloquear por país garante que usuário está lá?

Não. Localização de rede é um sinal imperfeito e pode ser alterada. Também há clientes viajando. Uso a regra somente com uma política clara e meios de recuperação, sem tratá-la como prova de identidade ou residência.

Termo para lembrar: Geo-block

12. Como limita requisições caras?

Defino tamanho, estrutura e custos permitidos por endpoint, antes de alocar trabalho excessivo. O limite de bytes não captura toda complexidade, como consultas profundas. A aplicação precisa validar custo semântico quando a borda não consegue.

Termo para lembrar: Payload

13. WAF torna qualquer arquivo seguro?

Não. Upload exige validação, armazenamento isolado, limites e análise apropriada ao tipo. A borda pode barrar algumas entradas, mas conteúdo e processamento posterior têm riscos próprios. Não executo arquivos só porque passaram por um filtro.

Termo para lembrar: Upload

14. Pode cachear resposta de saldo na borda?

Somente sob contrato rigoroso de privacidade, chave e frescor, se realmente fizer sentido. Por padrão separo assets públicos de dados personalizados. Um erro de cache pode expor contas, não apenas mostrar conteúdo antigo.

Termo para lembrar: Cache privado

15. Como publica uma regra nova sem derrubar clientes?

Uso observação ou modo de contagem quando disponível, escopo limitado e rollout progressivo. Monitoro impacto na jornada e tenho rollback. Uma regra aparentemente boa num teste pequeno pode bloquear um padrão legítimo de aplicativo antigo.

Termo para lembrar: Regras

16. Como percebe que a defesa está prejudicando usuários?

Correlaciono bloqueios com sucesso de login, conversão, suporte e versões de cliente, com dados minimizados. Taxa de bloqueio alta não é sucesso por si. Preciso distinguir abuso rejeitado de cliente legítimo impedido.

Termo para lembrar: Falso positivo

17. Um parceiro pede bypass total. Aceita?

Prefiro identidade e política específica com limites, prazo e justificativa. Um bypass amplo pode remover a proteção da origem e ser explorado após comprometimento. Exceções devem ser rastreáveis e tão restritas quanto necessário.

Termo para lembrar: Exceção

18. A borda caiu. Abre acesso direto à origem?

Não automaticamente. Isso pode remover controles essenciais justamente durante um incidente. Planejo continuidade com segurança equivalente ou aceito degradação definida. O caminho alternativo precisa ser testado e autorizado antes da emergência.

Termo para lembrar: Indisponibilidade

19. Toda validação pode falhar aberta para manter disponibilidade?

Não. Distingo controles auxiliares de controles obrigatórios. Falha de autenticação não vira autorização. Qualquer modo degradado precisa limitar capacidades e ser uma política explícita, não uma decisão improvisada do código.

Termo para lembrar: Fail-open

20. Proteção de aplicação resolve ataque volumétrico de rede?

Não sozinha. Camadas diferentes têm recursos e mecanismos distintos. Preciso de capacidade e proteção de rede apropriadas além de regras HTTP. O desenho deve mostrar onde cada classe de exaustão é mitigada.

Termo para lembrar: L7 e L4

21. A borda pode salvar uma origem saturada?

Ajuda com admissão, limites e respostas controladas, mas precisa refletir capacidade e evitar retries sincronizados. Se toda requisição passa antes de descobrir saturação, o custo já chegou. A origem também mantém limites locais.

Termo para lembrar: Backpressure

22. Posso confiar em X-Forwarded-For enviado pelo cliente?

Não sem uma cadeia de proxies confiável e normalização. Removo ou substituo headers relevantes na fronteira e defino quem pode acrescentá-los. Caso contrário, limites e auditoria baseados em endereço podem ser enganados.

Termo para lembrar: Headers

23. Devo armazenar todo corpo bloqueado para investigar?

Não por padrão. Pode conter senhas, tokens e dados pessoais. Uso metadados e amostras minimizadas com controles. A defesa não deve criar um repositório de segredos para provar que está funcionando.

Termo para lembrar: Logs

24. Edge functions são sempre mais baratas?

Depende de invocações, tempo, transferência e limites do runtime. Processamento distribuído também complica depuração e versões. Mantenho na borda o trabalho que se beneficia dela, evitando mover todo domínio apenas por proximidade.

Termo para lembrar: Custo

25. Uma mudança de regra chega instantaneamente a todos os pontos?

Não assumo isso. Considero propagação e versões da configuração. Durante a transição, a aplicação mantém invariantes essenciais. Regras de segurança críticas precisam de um plano que reconheça a janela de distribuição.

Termo para lembrar: Consistência

26. Como testa a defesa sem atacar produção indiscriminadamente?

Uso cenários autorizados e ambiente controlado, medindo limites e tráfego legítimo representativo. Em produção, escopo e interrupção são definidos. O teste deve validar recuperação e falso positivo, não apenas capacidade de bloquear.

Termo para lembrar: Teste

27. Usar dois provedores de borda elimina dependência?

Pode reduzir certos riscos, mas aumenta diferenças de regras, certificados, roteamento e operação. A configuração precisa continuar coerente. Multi-provider sem teste pode criar rotas desprotegidas ou comportamento divergente para o mesmo usuário.

Termo para lembrar: Fornecedor

28. Quem mantém as regras e responde a incidentes?

Defino colaboração entre segurança, plataforma e donos das jornadas, com critérios de mudança e acesso. Centralizar tudo num time sem contexto de produto pode gerar bloqueios ruins. Autonomia sem governança também cria lacunas.

Termo para lembrar: Ownership

29. Quando uma solução mais simples atende?

Quando APIs e volume são limitados, um gateway com regras gerenciadas, limites e origem protegida pode bastar. Evoluo com evidência de abuso e capacidade. Não preciso começar com lógica dinâmica própria em dezenas de locais.

Termo para lembrar: Alternativa

30. Como resume a arquitetura?

A borda reduz tráfego prejudicial e protege capacidade, enquanto a aplicação mantém identidade e autorização. Regras evoluem com medição e rollback. A defesa é boa quando bloqueia abuso preservando a jornada legítima e a recuperação.

Termo para lembrar: Conclusão

10 trade-offs: ganho, custo e decisão

01. Validação na borda versus só na origem

Ganho. Rejeita cedo e reduz carga

Custo e motivo. Distribui configuração e diagnóstico

Como decidir. Manter autorização final no domínio responsável

02. Limite por IP versus múltiplos sinais

Ganho. IP é barato e simples

Custo e motivo. Pode punir redes compartilhadas e falhar contra distribuição

Como decidir. Combinar dimensões conforme endpoint e privacidade

03. Quota aproximada versus global exata

Ganho. Aproximada reduz coordenação e latência

Custo e motivo. Pode permitir excesso agregado

Como decidir. Usar para proteção, não para invariante financeira estrita

04. Regras agressivas versus menor falso positivo

Ganho. Bloqueiam mais padrões suspeitos

Custo e motivo. Podem impedir acesso legítimo

Como decidir. Medir impacto na jornada e aplicar progressivamente

05. Challenge versus bloqueio direto

Ganho. Permite recuperação de alguns usuários legítimos

Custo e motivo. Adiciona atrito e acessibilidade a resolver

Como decidir. Escolher pelo canal e risco

06. Cache de chaves versus consulta constante

Ganho. Reduz latência e dependência

Custo e motivo. Exige lidar com rotação e validade

Como decidir. Atualização controlada com modelo de confiança explícito

07. Edge function versus serviço central

Ganho. Aproxima trabalho simples do usuário

Custo e motivo. Limita runtime e complica versões distribuídas

Como decidir. Mover apenas funções que ganham com proximidade

08. Um provedor versus múltiplos

Ganho. Um simplifica operação e políticas

Custo e motivo. Concentra dependência

Como decidir. Diversificar só com necessidade e testes de equivalência

09. Log detalhado versus minimização

Ganho. Ajuda investigação

Custo e motivo. Acumula dados sensíveis e custo

Como decidir. Metadados úteis e acesso restrito

10. Falha fechada versus modo degradado

Ganho. Preserva controles essenciais

Custo e motivo. Pode reduzir disponibilidade

Como decidir. Definir por controle e capacidade, nunca como regra universal

Duas alternativas ao caminho proposto

1. Gateway e WAF gerenciados com regras enxutas

Quando faz sentido. Necessidade inicial é proteção básica e previsível

Como implementar. Aplicar limites, autenticação inicial e acesso restrito à origem

Limites. Exige ajuste por jornada e não resolve autorização de negócio

2. Borda programável com política controlada

Quando faz sentido. Escala e casos específicos justificam processamento distribuído

Como implementar. Versionar funções e regras, testar propagação e manter rollback

Limites. Mais complexidade operacional, observabilidade e dependência de runtime

Referências e alcance da evidência

1. Cloudflare: WAF

Recursos e organização de regras de proteção na borda.

https://developers.cloudflare.com/waf/

2. IETF: JWT Best Current Practices

Validação segura de tokens e cuidados de algoritmos e contexto.

https://www.rfc-editor.org/rfc/rfc8725

3. Google SRE: handling overload

Proteção de capacidade e comportamento sob sobrecarga.

https://sre.google/sre-book/handling-overload/

Continue em: T051 · T052 · T071 · T072

C044 / M. Os 50 cenários enviados

Roteamento Inteligente de Pagamentos P2P e B2B

Roteamento de pagamentos escolhe um caminho elegível para uma intenção conforme capacidade, custo, prazo e contrato. Trocar de caminho após resultado ambíguo pode produzir dois pagamentos e exige recuperação antes de nova execução.

Quando usar. Quando há vários parceiros ou meios compatíveis com a intenção e o negócio precisa decidir entre eles com controle e evidência.

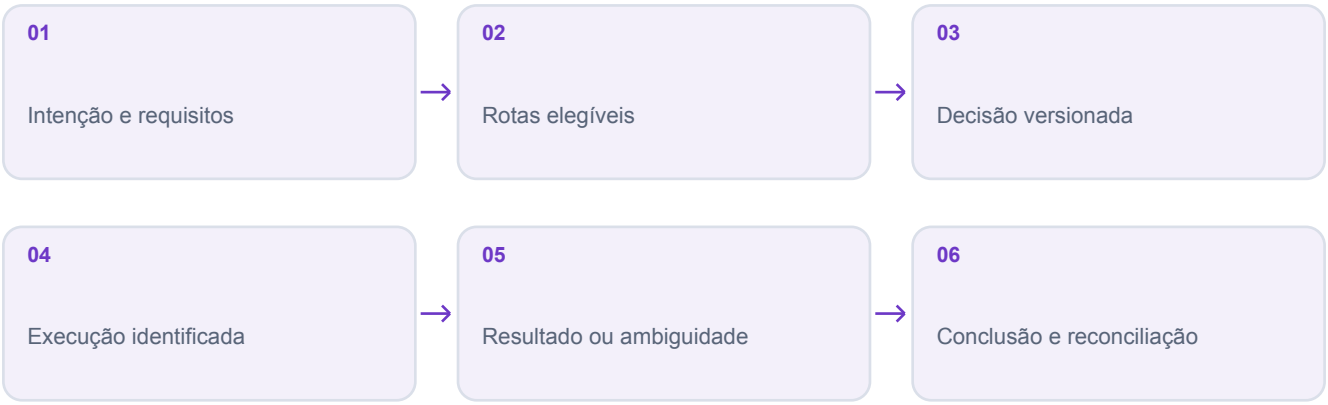
Quando evitar ou limitar. Evitar fallback Pix para outro meio após simples timeout, escolher só por menor preço e tratar mudança de configuração como autorização ilimitada.

Uma abertura possível

Eu começaria preservando uma intenção única, independentemente do provedor. O roteador escolhe entre caminhos permitidos; o executor acompanha cada tentativa. Se um caminho fica ambíguo, eu não pago por outro até ter uma condição segura para isso.

Peças e tecnologias. Payment intent · Motor de regras versionado · Adaptadores de provedor · Idempotência · Reservas e UNKNOWN · Reconciliação

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. 0-8 min: definir intenção, meios permitidos, prazo, custo, consentimento e critérios de conclusão.

2. 8-18 min: modelar intenção, decisão de rota, tentativa, reserva e resultado.

3. 18-32 min: desenhar elegibilidade, ranking, adaptadores e persistência antes de efeitos.

4. 32-48 min: simular timeout após aceite, provedor degradado, fallback e retorno tardio.

5. 48-60 min: comparar regras e otimização, métricas, rollout e reconciliação.

Cuidado técnico. Timeout não é evidência suficiente para trocar Pix por outro meio.

Cuidado técnico. Fallback entre provedores não cria atomicidade global.

30 perguntas para sustentar o desenho

01. O pagamento é a chamada ao provedor?

Não. A intenção do cliente existe independentemente de tentativas e rotas. Registro beneficiário, valor, moeda e condições autorizadas. Isso permite recuperar falhas sem confundir uma nova conexão com uma nova obrigação financeira.

Termo para lembrar: Intenção

02. Pode escolher qualquer meio disponível?

Não. A rota precisa atender identidade do destinatário, moeda, prazo, limites e condições autorizadas. Meios diferentes podem ter contratos distintos. O roteador não transforma consentimento para uma operação em autorização genérica para outra.

Termo para lembrar: Elegibilidade

03. Escolhe sempre o parceiro mais barato?

Não. Primeiro filtro elegibilidade e correção, depois avalio custo, latência, capacidade e confiabilidade dentro do prazo. Um caminho barato que aumenta falhas ou atraso pode custar mais no total e piorar a jornada.

Termo para lembrar: Ranking

04. Quais estados ajudam a evitar duplicação?

Intenção aceita, rota escolhida, tentativa iniciada, aceita externamente, concluída, rejeitada e resultado desconhecido, conforme contrato. Separar tentativa de resultado evita que erro de transporte seja interpretado como rejeição financeira definitiva.

Termo para lembrar: Estado

05. Pix expirou na chamada. Pode tentar TED?

Não automaticamente. O Pix pode ter ocorrido. Primeiro resolvo a ambiguidade por consulta ou reconciliação, ou uso garantia contratual suficiente. Trocar de meio sem isso pode pagar duas vezes a mesma intenção.

Termo para lembrar: Timeout

06. Quando uma nova rota é segura?

Quando há evidência de que a tentativa anterior não pode produzir o efeito, ou um protocolo que coordene exclusão e identidade. Ainda verifico prazo e autorização. Um texto genérico de erro nem sempre oferece essa prova.

Termo para lembrar: Erro comprovado

07. Uma chave global resolve duplicação em todos os provedores?

Só se os participantes respeitarem uma semântica compartilhada, o que não posso presumir. Mantenho identidade da intenção e referências por provedor. A chave local não impede dois fornecedores independentes de executar a mesma ordem.

Termo para lembrar: Idempotência

08. O que grava antes da chamada externa?

A intenção, a decisão e a tentativa com referência estável e estado necessário para recuperação. Depois registro evidência do resultado. A fronteira externa não é atômica com o banco; o desenho precisa representar essa lacuna.

Termo para lembrar: Persistência

09. Como protege saldo enquanto tenta pagar?

Reservo ou contabilizo conforme o fluxo, vinculando o recurso à intenção. Não libero automaticamente por timeout se o pagamento pode ter sido aceito. A política de resolução e expiração precisa evitar gastar novamente o mesmo dinheiro.

Termo para lembrar: Reserva

10. A rota antiga confirmou depois de escolher outra. E agora?

Esse é precisamente o risco que o controle de ambiguidade deve evitar. Registro e reconcilio todas as evidências, interrompendo ações adicionais e tratando impacto. Não ignoro sucesso antigo só porque o estado local mudou.

Termo para lembrar: Retorno tardio

11. Circuito aberto pode mudar rota imediatamente?

Pode impedir uma nova tentativa ainda não iniciada, usando outra rota elegível. Não resolve o destino de uma tentativa já enviada. Estado de saúde do provedor e resultado de um pagamento específico são informações diferentes.

Termo para lembrar: Circuit breaker

12. Como mede disponibilidade do parceiro?

Uso resultado e latência por operação, região e período, distinguindo rejeição legítima de falha técnica. Um health check simples pode estar verde enquanto pagamentos falham. Evito reagir a ruído com mudanças constantes de rota.

Termo para lembrar: Saúde

13. Como evita alternar provedores a cada minuto?

Uso janelas, limiares e histerese apropriados, com limites de mudança. A configuração deve ser observável e reversível. Alternância excessiva pode atrapalhar capacidade, investigação e previsibilidade sem melhorar sucesso real.

Termo para lembrar: Oscilação

14. Rota saudável, mas sem capacidade. O que faz?

Considero quota e concorrência na elegibilidade operacional, reservando capacidade quando necessário. Não envio tudo ao melhor provedor até saturá-lo. O roteamento precisa distribuir carga e respeitar compromissos de cada integração.

Termo para lembrar: Quota

15. Regras sem deploy são sempre mais seguras?

Não. Mudanças de configuração podem ter impacto igual ou maior que código. Versiono, valido, simulo e libero progressivamente, com responsável e rollback. Um editor visual não elimina a necessidade de governança.

Termo para lembrar: Configuração

16. Como responde por que escolheu uma rota?

Registro versão da política, candidatos elegíveis e critérios relevantes com dados minimizados. Isso permite investigar custo, falha e comportamento. Não preciso armazenar todo sinal sensível para preservar uma justificativa útil.

Termo para lembrar: Explicação

17. Como compara custo total de duas rotas?

Incluo tarifa, falhas, retries, reconciliação, prazo e operação, além de condições contratuais. O menor preço por chamada pode gerar mais custo por pagamento concluído. A função de decisão deve refletir o objetivo real.

Termo para lembrar: Preço

18. Como respeita o prazo do cliente?

Propago um orçamento entre seleção, tentativa, consulta e conclusão, reservando tempo para recuperação. Uma rota só é elegível se puder atender o contrato. Não gasto o prazo inteiro num provedor e depois prometo fallback instantâneo.

Termo para lembrar: Tempo

19. Cancelar a intenção impede pagamento externo?

Só dentro das condições do protocolo e da etapa. Depois de enviado, cancelamento pode ser uma nova operação ou não estar disponível. Distingo pedido de cancelamento de confirmação, preservando reservas e evidências até resolver.

Termo para lembrar: Cancelamento

20. Um segundo pagamento pode ser corrigido com estorno automático?

Nem sempre o meio permite reversão imediata ou garantida. Por isso prevenção é melhor que supor compensação perfeita. Qualquer devolução é outra operação com identidade, autorização e risco próprios, vinculada ao erro original.

Termo para lembrar: Reversão

21. O status pode ser eventual no aplicativo?

Pode haver atualização assíncrona, desde que a interface distinga processamento, dúvida e conclusão. Não mostro failed quando só perdi resposta. A experiência deve orientar o cliente a acompanhar a mesma intenção, evitando novas ordens acidentais.

Termo para lembrar: Consistência

22. Por que não espalhar if/else por todos os produtos?

Um contrato de execução comum concentra diferenças de protocolos e resultados, facilitando testes e evolução. Ainda preservo semânticas que não são equivalentes. Abstração não deve esconder que dois meios têm garantias diferentes.

Termo para lembrar: Adaptadores

23. O que fecha uma tentativa UNKNOWN?

Evidência do provedor ou processo de conciliação compatível com o contrato, correlacionada por referência estável. Defino prazo e escalada. Um cron que muda tudo para failed após algumas horas não resolve a incerteza externa.

Termo para lembrar: Reconciliação

24. Qual teste deve existir antes de habilitar fallback?

O provedor executa e a resposta se perde. Verifico que o sistema não inicia outro pagamento sem condição segura. Também testo rejeição comprovada para confirmar que fallback permitido continua funcionando e não fica bloqueado indevidamente.

Termo para lembrar: Teste

25. Pode reprocessar histórico para testar novas regras?

Sim em modo sem efeitos, comparando escolhas e restrições. Não chamo provedores reais nem reaplico pagamentos. Também reconheço que dados históricos podem não conter todos os sinais disponíveis no momento da decisão original.

Termo para lembrar: Replay

26. Pode fazer A/B com qualquer rota?

Somente dentro de elegibilidade, riscos e contratos autorizados. Defino limites e critérios de parada, preservando correção financeira. Experimentar não justifica expor clientes a caminhos que não cumprem requisitos essenciais.

Termo para lembrar: Experimentação

27. O que mede além de taxa de sucesso?

Tempo até conclusão, ambiguidade, custo por intenção concluída, retries, divergências e experiência por rota. Rejeições legítimas não devem ser mascaradas como falha técnica. Uma taxa agregada pode esconder uma classe de clientes prejudicada.

Termo para lembrar: Métrica

28. Duas regiões podem executar a mesma intenção?

O desenho precisa de autoridade e deduplicação coerentes para impedir isso. Replicação eventual de estado local não basta para exclusão. Failover preserva identidade e trata tentativas em andamento antes de abrir outra execução.

Termo para lembrar: Multi-região

29. Quando um roteador estático é melhor?

Quando há poucos caminhos e regras estáveis, uma seleção determinística versionada é simples e auditável. Otimização dinâmica entra com dados e benefício medido. Complexidade sem sinais confiáveis pode piorar decisões.

Termo para lembrar: Alternativa

30. Como fecha a solução?

Mostro uma intenção, várias tentativas possíveis e uma regra que impede efeitos concorrentes indevidos. Seleção otimiza apenas rotas válidas; recuperação resolve dúvidas. Disponibilidade maior não pode ser comprada com risco escondido de pagar duas vezes.

Termo para lembrar: Conclusão

10 trade-offs: ganho, custo e decisão

01. Rota estática versus dinâmica

Ganho. Estática é previsível e fácil de auditar

Custo e motivo. Adapta menos a custo e saúde momentâneos

Como decidir. Dinâmica quando sinais confiáveis e benefício justificam

02. Menor tarifa versus melhor conclusão

Ganho. Tarifa reduz custo imediato

Custo e motivo. Pode elevar falhas, atraso e suporte

Como decidir. Comparar custo por intenção concluída

03. Fallback rápido versus resolver ambiguidade

Ganho. Rápido ajuda em falha comprovada

Custo e motivo. Na dúvida pode duplicar pagamento

Como decidir. Só mudar rota com condição segura

04. Política central versus regras por produto

Ganho. Central melhora governança e reuso

Custo e motivo. Pode esconder necessidades diferentes

Como decidir. Contrato comum com restrições explícitas por jornada

05. Reserva prolongada versus liberação cedo

Ganho. Preserva recurso durante UNKNOWN

Custo e motivo. Pode reduzir saldo disponível do cliente

Como decidir. Resolver dúvida com prazo e comunicação claros

06. Health check versus resultado real

Ganho. Check é simples e barato

Custo e motivo. Pode não representar pagamentos

Como decidir. Combinar sinais por operação e janela

07. Configuração dinâmica versus deploy

Ganho. Permite ajuste rápido

Custo e motivo. Muda comportamento crítico fora do binário

Como decidir. Versionar, revisar e testar ambos

08. Abstração uniforme versus semântica específica

Ganho. Uniformidade simplifica consumidores

Custo e motivo. Pode esconder diferenças de meios

Como decidir. Expor estados e limites que afetam correção

09. Consulta ativa versus espera por callback

Ganho. Consulta fecha lacunas mais cedo

Custo e motivo. Consome quota e pode ampliar carga

Como decidir. Usar polling limitado e reconciliação

10. Otimização agressiva versus estabilidade

Ganho. Pode capturar oportunidades de custo

Custo e motivo. Oscilações complicam operação e capacidade

Como decidir. Histerese, limites e evidência de benefício

Duas alternativas ao caminho proposto

1. Roteamento determinístico com fallback restrito

Quando faz sentido. Poucos provedores e regras estáveis

Como implementar. Selecionar rota por contrato, persistir tentativa e recuperar ambiguidades

Limites. Menor adaptação automática a custo e saúde

2. Motor de políticas com execução durável

Quando faz sentido. Múltiplas rotas e objetivos justificam evolução

Como implementar. Filtrar elegibilidade, pontuar opções e registrar decisão versionada separada do executor

Limites. Exige qualidade de sinais, governança e proteção contra oscilação

Referências e alcance da evidência

1. AWS: APIs idempotentes

Semântica de intenção e novas tentativas.

https://aws.amazon.com/builders-library/making-retries-safe-with-idempotent-APIs/

2. AWS: timeouts, retries e jitter

Falhas ambíguas, prazos e controle de repetição.

https://aws.amazon.com/builders-library/timeouts-retries-and-backoff-with-jitter/

3. Banco Central: normas do Pix

Referência para consultar contratos e regras atuais do arranjo; o roteador apresentado é uma proposta.

https://www.bcb.gov.br/estabilidadefinanceira/pix-normas

Continue em: T061 · T064 · T091 · T092 · C001

C045 / M. Os 50 cenários enviados

Cálculo de Saldo em Tempo Real com Extrema Concorrência

Saldo sob concorrência exige uma autoridade que preserve a relação entre lançamentos, reservas e limite disponível. O mecanismo pode usar transações, atualização condicional ou processamento serializado, mas não pode perder efeitos reconhecidos.

Quando usar. Quando várias operações disputam os mesmos recursos financeiros e o sistema precisa de alta vazão com resultados verificáveis.

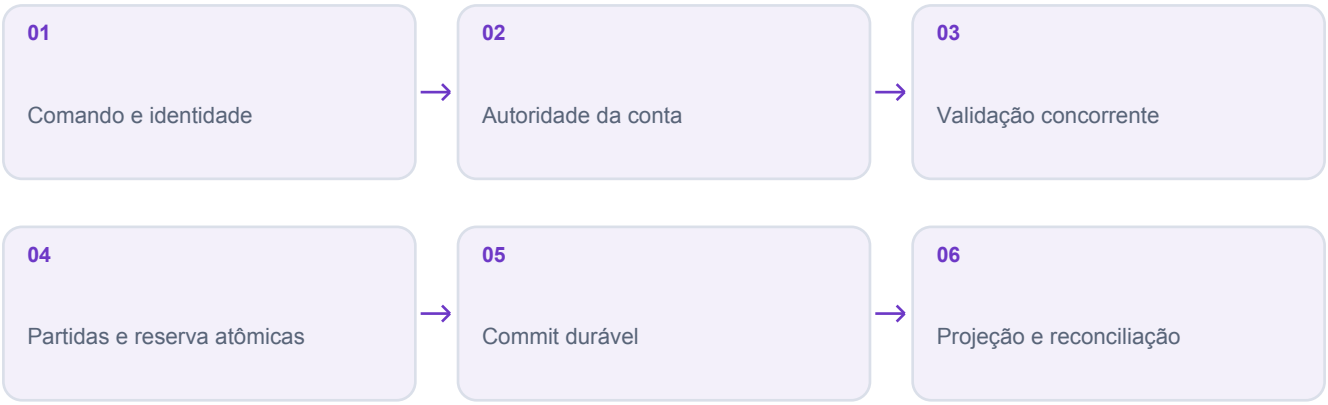
Quando evitar ou limitar. Evitar declarar locks sempre ruins, usar Redis write-behind como autoridade sem contrato de durabilidade e prometer paralelizar livremente a mesma invariante.

Uma abertura possível

Eu começaria com uma transação curta que protege o saldo e registra as partidas. Mediria contenção antes de trocar a ferramenta. Se uma conta fica muito quente, posso mudar a forma de serializar, mas ainda preciso de uma ordem válida e de persistência antes de confirmar dinheiro.

Peças e tecnologias. PostgreSQL · Atualização condicional · Ledger append-only · Reservas · Actor com persistência · Particionamento por autoridade

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. 0-8 min: definir saldo contábil, disponível, reservas, limite e falhas que não podem ocorrer.

2. 8-18 min: modelar lançamentos, identidade, concorrência e unidade de autoridade.

3. 18-32 min: implementar fluxo com transação curta ou comando serializado durável.

4. 32-48 min: simular conta quente, corrida, timeout após commit, perda de líder e cache atrasado.

5. 48-60 min: comparar mecanismos, medir capacidade, planejar sharding e recuperação.

Cuidado técnico. Locks não são inerentemente inadequados para alta performance.

Cuidado técnico. Actor, Disruptor e Redis não substituem por si só identidade, durabilidade e invariantes financeiras.

30 perguntas para sustentar o desenho

01. Qual regra precisa ser preservada?

Defino saldo e reservas conforme o produto, impedindo consumo além do permitido e mantendo lançamentos equilibrados. O valor exato da regra depende de limites e crédito. Uma única variável chamada saldo pode esconder recursos diferentes.

Termo para lembrar: Invariante

02. Esse lock é necessariamente ruim?

Não. Pode ser uma escolha simples e correta para uma transação curta sobre a conta. Meço espera e contenção antes de substituir. O problema é segurar locks por muito tempo ou em ordem inconsistente, não a existência de exclusão.

Termo para lembrar: SELECT FOR UPDATE

03. Como debita sem um SELECT separado?

Posso usar atualização com condição sobre disponibilidade e verificar linhas afetadas, dentro da transação que registra efeitos necessários. Isso reduz uma janela de corrida. A expressão precisa representar corretamente reservas e limites do produto.

Termo para lembrar: Update condicional

04. Read committed sempre basta?

Depende das operações e invariantes. Uma atualização condicional sobre a mesma linha pode arbitrar aquele recurso, mas regras entre múltiplas linhas podem exigir locks ou isolamento mais forte. Escolho pelo conflito real e testo anomalias.

Termo para lembrar: Isolamento

05. Qual falha uma leitura consistente pode não impedir?

Duas transações podem ler condições compatíveis e alterar linhas distintas, quebrando uma regra conjunta. Snapshot isolado não elimina todo write skew. Identifico o recurso compartilhado e uso um mecanismo que arbitre a invariante completa.

Termo para lembrar: Write skew

06. Atualizar saldo e gravar histórico em chamadas separadas é seguro?

Não sem um protocolo que preserve atomicidade ou recuperação equivalente. Se uma parte falha, saldo e evidência divergem. Prefiro registrar partidas e estado materializado na mesma autoridade transacional quando esse é o limite do sistema.

Termo para lembrar: Partidas

07. Onde protege a operação repetida?

Na mesma fronteira durável do efeito, com identidade e payload compatíveis. Se uma resposta se perde após commit, recupero o resultado. Um cache de chaves separado pode desaparecer e permitir repetir o débito.

Termo para lembrar: Idempotência

08. Um contador numérico qualquer serve para dinheiro?

Não. Uso representação e escala monetária definidas, com limites e arredondamento explícitos. Inteiros também podem estourar e moedas podem ter regras diferentes. O tipo e a validação fazem parte da correção.

Termo para lembrar: Float

09. Por que saldo contábil não basta para autorizar?

Porque parte do recurso pode estar comprometida por operações ainda não liquidadas. O disponível considera reservas e regras do produto. Ignorar compromissos permite autorizar gastos que parecem caber no saldo, mas não no recurso livre.

Termo para lembrar: Reserva

10. Pode liberar reserva só porque passou o timeout da API?

Não se o efeito externo ainda for possível. Expiração de transporte e resolução financeira são coisas diferentes. Uso estado ambíguo e reconciliação, com política de prazo que não permita gastar novamente dinheiro já comprometido.

Termo para lembrar: Expiração

11. Um actor por conta elimina toda concorrência?

Serializa comandos dentro do modelo de execução enquanto a posse é válida. Ainda preciso de persistência, exclusão de proprietários antigos e recuperação. Duas instâncias pensando ser o mesmo actor podem quebrar a garantia se o destino não protege autoridade.

Termo para lembrar: Actor

12. Posso confirmar depois de atualizar só a memória do actor?

Somente se a promessa de durabilidade permitir perder esse efeito, o que geralmente não atende dinheiro reconhecido. Persisto conforme o contrato antes de confirmar. Alta vazão em benchmark de memória não equivale a ledger durável.

Termo para lembrar: Memória

13. LMAX Disruptor resolve durabilidade e failover?

Não por si. É uma estrutura e modelo de comunicação eficiente entre etapas em memória. O sistema ainda precisa de journal, replicação, recuperação e semântica de confirmação. Não confundo desempenho de uma fila com garantia financeira completa.

Termo para lembrar: Disruptor

14. INCR atômico garante saldo bancário correto?

Garante a operação dentro do mecanismo e configuração considerados, não todas as invariantes e a durabilidade da jornada. Preciso de identidade, múltiplos efeitos, failover e persistência. Um incremento isolado não modela transferência equilibrada.

Termo para lembrar: Redis

15. Por que escrever no banco depois pode ser perigoso?

Se confirmo antes de tornar o efeito durável no contrato exigido, uma falha pode perder dinheiro reconhecido. Também posso reordenar ou repetir gravações. Cache assíncrono é útil para projeções, mas não vira autoridade financeira sem desenho específico.

Termo para lembrar: Write-behind

16. Como escala uma conta com muitas operações?

Primeiro reduzo trabalho dentro da seção crítica e agrupo o que for semanticamente seguro. Posso serializar comandos eficientemente, mas a invariante da mesma conta ainda impõe coordenação. Particionar contas diferentes não remove esse hotspot individual.

Termo para lembrar: Hot account

17. Pode dividir o saldo em quotas independentes?

Em alguns modelos, aloco direitos limitados a partições para permitir decisões locais sem exceder o total. Isso exige conservar e transferir quotas corretamente. Troco utilização imediata perfeita por paralelismo; não é um contador eventual irrestrito.

Termo para lembrar: Escrow

18. Qual chave escolheria?

Uma chave que mantenha a maior parte das invariantes dentro de uma autoridade, frequentemente conta ou agrupamento definido. Considero transferências e contas quentes. Distribuir por transaction_id pode espalhar justamente os dados que precisam ser coordenados.

Termo para lembrar: Sharding

19. Transferir entre shards é só debitar e creditar?

Não. Dois commits independentes podem deixar uma metade aplicada. Uso coordenação adequada, reservas, contas de trânsito ou transação distribuída suportada, conforme requisitos. Explicito estados e recuperação em vez de chamar a soma de atômica.

Termo para lembrar: Cross-shard

20. Como reduz deadlocks entre duas contas?

Acesso recursos em ordem determinística quando aplicável, mantenho transações curtas e trato abortos com retry seguro. Deadlock detectado não é corrupção automática; o banco pode abortar uma transação para preservar progresso.

Termo para lembrar: Deadlock

21. Uma falha de serialização pode ser repetida?

Em geral posso repetir a transação conforme o contrato do banco, preservando intenção e sem efeitos externos não protegidos dentro dela. Limito tentativas e observo contenção. Retentar infinitamente pode agravar saturação.

Termo para lembrar: Retry

22. O cliente viu erro, mas o banco confirmou. O que faz?

Consulta ou repete pela mesma identidade para recuperar o resultado durável. Não cria outra transferência. A API deve representar processamento ou dúvida quando necessário, mantendo uma forma de acompanhar a intenção original.

Termo para lembrar: Timeout após commit

23. Como oferece saldo rápido sem usar projeção para autorizar?

Sirvo consultas informativas por projeção com frescor definido e uso a autoridade para decisões que exigem estado atual. Posso fornecer versão ou leitura após escrita. Não confundo otimização de leitura com permissão de gasto.

Termo para lembrar: Leitura

24. Precisa somar todos os lançamentos em cada pagamento?

Não. Posso manter saldo materializado atomicamente e verificar por recomputação periódica ou sob demanda. O histórico preserva explicação e recuperação. Ler todo o passado a cada comando pode ser caro sem acrescentar correção.

Termo para lembrar: Histórico

25. O que verifica entre histórico e saldo?

Somas por conta e moeda, reservas, totais equilibrados e vínculo de identidades, respeitando estados de trânsito. Uma divergência exige investigação e correção controlada. Não atualizo o saldo silenciosamente apenas para fazer um relatório fechar.

Termo para lembrar: Reconciliação

26. Qual momento define sucesso para o cliente?

O ponto em que o contrato de commit e durabilidade foi cumprido, incluindo replicação necessária. Explico o que pode ser perdido em falha e qual é a autoridade recuperável. Resposta rápida sem essa definição esconde risco.

Termo para lembrar: Durabilidade

27. Como compara banco, actor e Redis de modo justo?

Uso a mesma garantia de durabilidade, concorrência e correção, além de carga representativa. Meço latência de cauda e falhas, não só operações por segundo. Comparar memória volátil com commit durável produz uma conclusão enganosa.

Termo para lembrar: Benchmark

28. Failover pode ressuscitar saldo antigo?

Sim se dados reconhecidos não estiverem na autoridade promovida. O plano precisa considerar perda possível, identidade e escritor antigo. Uma réplica de cache atualizada não prova que a nova autoridade financeira está completa.

Termo para lembrar: DR

29. Que propriedades verificaria sob falhas?

Conservação por moeda, ausência de efeito duplicado, limites de disponível e transições de reserva. Gero concorrência e quedas em pontos de commit, incluindo resposta perdida. Testes de carga só são úteis se também validam o resultado.

Termo para lembrar: Teste

30. Como fecha a escolha técnica?

Começo pela opção simples que preserva a invariante e mede contenção. Evoluo serialização, particionamento ou quotas com evidência. O mecanismo pode mudar; identidade, durabilidade e uma autoridade clara sobre o recurso continuam obrigatórias.

Termo para lembrar: Conclusão

10 trade-offs: ganho, custo e decisão

01. Lock pessimista versus controle otimista

Ganho. Lock evita trabalho concorrente conflitante

Custo e motivo. Pode aumentar espera e deadlock

Como decidir. Otimista quando conflitos são raros e retries baratos

02. Update condicional versus leitura e lock

Ganho. Reduz etapas para uma condição simples

Custo e motivo. Pode não expressar invariantes entre várias linhas

Como decidir. Usar conforme a regra realmente protegida

03. Banco transacional versus actor durável

Ganho. Banco reúne restrições e persistência maduras

Custo e motivo. Pode ter custo de coordenação por transação

Como decidir. Actor quando serialização e operação justificam nova infraestrutura

04. Histórico puro versus saldo materializado

Ganho. Histórico facilita reconstrução e explicação

Custo e motivo. Consulta completa pode ser cara

Como decidir. Materializar atomicamente e reconciliar

05. Um escritor por conta versus multiwriter

Ganho. Simplifica ordem e invariantes

Custo e motivo. Limita paralelismo da mesma chave

Como decidir. Multiwriter só com mecanismo que preserve conflitos relevantes

06. Commit síncrono versus escrita posterior

Ganho. Síncrono torna confirmação mais forte

Custo e motivo. Adiciona latência de persistência e replicação

Como decidir. Não confirmar dinheiro com perda incompatível ao contrato

07. Batching de comandos versus imediato

Ganho. Amortiza custo de persistência

Custo e motivo. Adiciona espera e complexidade de resultados

Como decidir. Quando latência e semântica permitem

08. Quota particionada versus saldo global

Ganho. Permite decisões locais limitadas

Custo e motivo. Pode deixar recurso ocioso numa partição

Como decidir. Quando direitos podem ser conservados e redistribuídos

09. Shard por conta versus por evento

Ganho. Conta aproxima dados concorrentes

Custo e motivo. Pode concentrar contas quentes

Como decidir. Escolher pela invariante e operações entre contas

10. Leitura de réplica versus autoridade

Ganho. Reduz carga e latência local

Custo e motivo. Pode mostrar informação atrasada

Como decidir. Separar consulta informativa de autorização

Duas alternativas ao caminho proposto

1. Transação relacional curta com saldo e partidas

Quando faz sentido. Volume e contenção cabem numa autoridade conhecida

Como implementar. Aplicar condição ou lock, registrar identidade e partidas, confirmar duravelmente

Limites. Escala por chave quente e transferências entre shards exigem evolução

2. Processador serializado com journal durável

Quando faz sentido. Perfil de carga justifica controle explícito de execução

Como implementar. Ordenar comandos por autoridade, persistir antes de confirmar e recuperar com fencing

Limites. Operação, replicação e recuperação precisam ser implementadas ou fornecidas pela plataforma

Referências e alcance da evidência

1. PostgreSQL: explicit locking

Locks, modos e conflitos no banco.

https://www.postgresql.org/docs/current/explicit-locking.html

2. PostgreSQL: transaction isolation

Anomalias, isolamento e tratamento de conflitos.

https://www.postgresql.org/docs/current/transaction-iso.html

3. LMAX Disruptor

Biblioteca de comunicação em memória; durabilidade e semântica financeira exigem desenho adicional.

https://lmax-exchange.github.io/disruptor/

4. Redis: persistence

Opções e compromissos de persistência; não constitui garantia automática de ledger.

https://redis.io/docs/latest/operate/oss_and_stack/management/persistence/

Continue em: T022 · T023 · T061 · T091 · T092 · T106

C046 / M. Os 50 cenários enviados

Motor de Reconciliação Contábil Massiva (Banco x BACEN)

Reconciliação compara evidências independentes de operações e posições, preservando diferenças temporais, registros ausentes e correções auditáveis. Este projeto de treino não descreve um protocolo real do Banco Central nem presume que streaming substitua fechamento financeiro.

Quando usar. Quando o ledger, os retornos operacionais e as posições externas precisam concordar dentro de critérios de completude e prazo explícitos.

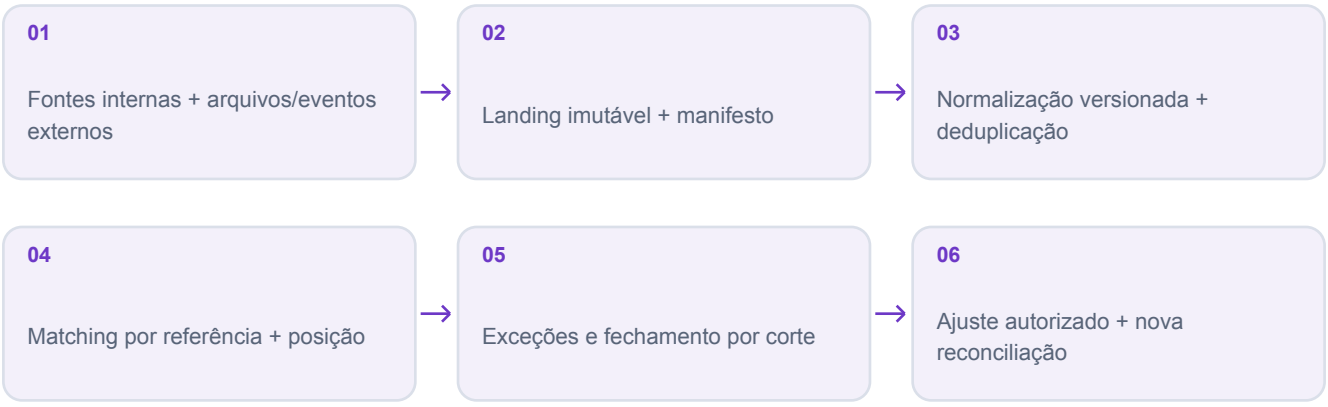
Quando evitar ou limitar. Não ajustar saldo automaticamente apenas porque um total divergiu. Watermark, entrega de arquivo e igualdade de totais não provam correção por operação.

Uma abertura possível

Eu separaria três perguntas: recebemos tudo que era esperado, cada operação corresponde à evidência externa e os saldos fecham? Desenho essas verificações antes de escolher streaming. Diferença vira um caso explicável; qualquer lançamento corretivo exige causa, autorização e identidade estável.

Peças e tecnologias. PostgreSQL e índices compostos · S3 ou armazenamento imutável equivalente · Kafka e Flink quando baixa latência justificar · Manifestos e checksums · Outbox e inbox · Ledger e workflow de exceções

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. 0-8 min: combinar fontes, calendário, corte, latência tolerada e significado de conciliado.

2. 8-18 min: modelar item original, versão normalizada, vínculo, diferença e execução.

3. 18-32 min: desenhar ingestão imutável, prova de completude e matching determinístico.

4. 32-48 min: simular duplicata, atraso, arquivo corrigido, retorno parcial e ajuste repetido.

5. 48-60 min: definir prazo por exceção, controles de fechamento, capacidade de replay e responsabilidade.

Cuidado técnico. O título representa um cenário hipotético; a interface externa depende do contrato aplicável.

Cuidado técnico. Streaming melhora tempo de detecção quando necessário, mas não substitui completude e fechamento.

30 perguntas para sustentar o desenho

01. O que exatamente você está conciliando?

Primeiro identifico a entidade externa, o tipo de operação e o período. Separaria confirmação operacional, liquidação e posição contábil. Um pagamento aceito pelo parceiro pode ainda não estar liquidado; comparar esses estados diretamente produziria uma divergência artificial.

Termo para lembrar: Escopo

02. Como sabe que recebeu todos os arquivos?

Exigiria um contrato de completude: sequência, manifesto, contagem esperada ou confirmação de fechamento. Registro arquivos ausentes e duplicados. Sem essa evidência, apresento o período como incompleto, mesmo quando todos os registros recebidos encontram correspondência.

Termo para lembrar: Completude

03. Por que preservar o arquivo original?

Guardo bytes originais, origem, horário de recebimento e checksum antes de normalizar. Assim posso reproduzir uma análise com outra versão do parser e provar quais dados sustentaram a decisão, sem depender de campos transformados que perderam contexto.

Termo para lembrar: Landing imutável

04. Qual chave você usaria para encontrar correspondências?

Prefiro referência externa estável combinada com participante e tipo de operação. Valor e data ajudam na validação, mas não identificam sozinhos um pagamento. Duas transações legítimas podem ter o mesmo valor e acontecer no mesmo segundo.

Termo para lembrar: Identidade

05. E quando uma operação interna corresponde a várias linhas externas?

Modelo grupos de correspondência com relação um para muitos, componentes de tarifa e liquidação parcial. Valido conservação por moeda e natureza. Não forço uma junção individual que esconderia componentes legítimos ou classificaria uma liquidação parcelada como duplicata.

Termo para lembrar: Matching

06. Qual data entra na comparação?

Persisto instante do evento, data contábil, data de liquidação e recebimento separadamente. A regra de reconciliação escolhe o corte adequado e seu fuso. Uso calendários versionados porque feriados e horários de fechamento afetam o conjunto esperado.

Termo para lembrar: Corte temporal

07. Um watermark permite fechar o dia?

Não sozinho. Ele orienta o tratamento temporal do stream segundo uma política de atraso. Fechar contabilmente exige evidência adicional das fontes esperadas. Um arquivo que nunca chegou não se torna correto porque o relógio do processamento avançou.

Termo para lembrar: Watermark | Base técnica: referência 1.

08. O que faz com um retorno recebido depois do fechamento?

Preservo o fechamento anterior e abro uma revisão vinculada ao período afetado. Recalculo apenas a partição necessária, mantendo causa e versão. Se houver impacto contábil, aplico o procedimento de correção aprovado, sem reescrever silenciosamente os números publicados.

Termo para lembrar: Atraso

09. O mesmo arquivo foi entregue três vezes; o que acontece?

Identifico o arquivo por origem e referência, comparo checksum e registro as entregas. Os itens também têm identidade própria. Duplicata idêntica não reaplica efeito; mesma referência com conteúdo diferente vai para investigação de revisão ou corrupção.

Termo para lembrar: Deduplicação

10. Como tratar um arquivo substituto com dados corrigidos?

Não sobrescrevo a evidência anterior. Registro relação de substituição, versão e motivo informado. A reconciliação calcula o efeito da revisão sobre vínculos existentes e exige autorização para qualquer correção financeira decorrente dessa nova evidência.

Termo para lembrar: Correção externa

11. Se a soma dos dois lados bate, está conciliado?

Ainda não. Um débito omitido pode compensar um crédito incorreto no total. Verifico quantidade, valores por moeda, referências, estados e correspondências individuais ou agrupadas. Totais são controles úteis, mas não substituem a identificação das diferenças.

Termo para lembrar: Totais de controle

12. Quais estados colocaria no motor?

Usaria recebido, inválido, aguardando contraparte, conciliado, divergente e em revisão, com razão explícita. Fechamento pertence ao período e tem critérios próprios. Evito um booleano conciliado que mistura ausência temporária, incompatibilidade e decisão humana.

Termo para lembrar: Estados

13. Dois workers podem conciliar o mesmo item?

Uso unicidade para vínculos que devem ser exclusivos e transação para validar a versão antes de persistir a correspondência. Um conflito obriga nova leitura. Particionar o trabalho reduz disputa, mas não substitui as restrições que protegem o resultado.

Termo para lembrar: Concorrência | Base técnica: referência 3.

14. Como distribuir bilhões de registros sem perder pares?

Particiono pela referência normalizada ou por um agrupamento que mantenha possíveis pares juntos. Datas servem para limitar busca, com sobreposição para atrasos. Identifico chaves muito frequentes e grupos grandes para evitar que uma partição concentre o trabalho.

Termo para lembrar: Particionamento

15. Qual o custo do matching em lote?

Com identificadores normalizados, posso fazer hash join em tempo esperado linear, condicionado à memória, ou sort merge com custo de ordenação. Regras aproximadas custam mais; aplico primeiro filtros baratos para reduzir candidatos e conservar explicabilidade.

Termo para lembrar: Complexidade

16. Como executar novamente sem criar ajustes duplicados?

Versiono a execução e separo cálculo de propostas de aplicação financeira. A proposta tem identidade de negócio independente do job. O ledger aceita cada correção autorizada uma vez, mesmo quando o mesmo conjunto é recalculado por outro worker.

Termo para lembrar: Reprocessamento

17. Como compara fontes que continuam recebendo operações?

Defino snapshots ou posições de leitura com cortes registrados. Caso contrário, uma consulta longa compara momentos diferentes e inventa diferenças. O relatório informa até qual posição de cada fonte foi coberto e quais entradas continuam pendentes.

Termo para lembrar: Snapshot | Base técnica: referência 3.

18. O que faz com uma diferença de poucos centavos?

Investigo regra de arredondamento, tarifa, moeda e versão do cálculo. Uma tolerância pode classificar prioridade, mas não autoriza apagar a diferença. Se houver ajuste legítimo, ele precisa de conta de destino, razão e aprovação proporcionais ao risco.

Termo para lembrar: Exceções

19. Pode gerar lançamentos automáticos para zerar a divergência?

Somente para classes previamente definidas, com causa verificável, política aprovada e limites explícitos. Divergência desconhecida vira investigação. Zerar um número sem explicar a origem transforma reconciliação em um mecanismo que pode ocultar perdas ou corrupção.

Termo para lembrar: Ajustes | Base técnica: referência 4.

20. Como impede que um operador invente uma correção?

Exijo permissões separadas para propor e aprovar classes sensíveis, vinculando evidência, valores e hash da proposta. Qualquer alteração invalida a aprovação anterior. O executor valida a decisão registrada e aplica a mesma identidade de correção no ledger.

Termo para lembrar: Maker-checker

21. Qual métrica realmente importa para a operação?

Mediria exposição financeira não conciliada por idade e motivo, além da cobertura das fontes e tempo de resolução. Contar apenas itens processados mascara um único pagamento relevante parado. O prazo de exceção acompanha criticidade e calendário operacional.

Termo para lembrar: SLA operacional

22. O que acontece se a fonte externa parar?

Continuo recebendo dados internos e marco a cobertura externa como incompleta. Evito anunciar fechamento. Alertas consideram prazo e exposição, enquanto o adaptador retenta com orçamento. A recuperação processa o atraso sem bloquear os períodos já comprovadamente completos.

Termo para lembrar: Indisponibilidade

23. Como evita que o fechamento derrube o ledger?

Leio por CDC, réplica adequada ou exportação consistente, limitando concorrência e consultas amplas. Mantenho orçamento de recursos para o fechamento e prioridade para transações online. A necessidade de investigar uma diferença não justifica saturar a fonte operacional.

Termo para lembrar: Backpressure

24. Como um auditor reproduziria sua conclusão?

Entregaria referências dos dados originais, versão do parser, regra aplicada, cortes, correspondências e decisões humanas. A execução deve ser reproduzível sobre dados preservados. Logs técnicos sem esses vínculos não explicam por que o resultado financeiro foi considerado correto.

Termo para lembrar: Evidência

25. O analista precisa acessar todos os dados pessoais?

Não. A interface mostra referências e atributos necessários ao diagnóstico, com mascaramento e controle por função. Acesso excepcional fica auditado. A capacidade de reconciliar valores não requer divulgar livremente documentos, dados de cartão ou detalhes pessoais completos.

Termo para lembrar: Dados sensíveis

26. Streaming é obrigatório para demonstrar nível Staff?

Não. Escolho pela janela de decisão e pela forma como a contraparte publica evidência. Se a confirmação oficial chega diariamente, um lote bem controlado pode ser suficiente. Streaming é útil para antecipar diferenças, não para fabricar confirmação externa.

Termo para lembrar: Batch ou stream

27. Como dimensiona estado de operações sem contraparte?

Estimo taxa de itens multiplicada pelo tempo máximo de espera e tamanho do estado, incluindo índices e réplicas. Diferencio atraso normal de retenção para auditoria. O crescimento de pendências precisa de limites operacionais e alertas antes de esgotar armazenamento.

Termo para lembrar: Estimativa

28. Qual teste revelaria um erro financeiro escondido?

Criaria duas operações de mesmo valor, removendo uma e duplicando a outra. O total permanece igual, mas a correspondência por identidade deve falhar. Também testaria arquivos fora de ordem, correções tardias e replay após aplicar um ajuste.

Termo para lembrar: Testes

29. Como recupera o motor depois de perder seu estado?

Restauro checkpoints ou recalculo a partir da landing preservada, mantendo versões de regras e decisões aplicadas. Antes de liberar novos ajustes, verifico identidades já registradas no ledger. Recuperar cálculo não autoriza repetir efeitos financeiros anteriores.

Termo para lembrar: Recuperação

30. Como encerraria a apresentação dessa arquitetura?

Eu mostraria quais fontes estão completas, qual exposição continua aberta e quem decide cada exceção. A solução entrega evidência rastreável, não apenas um número igual. Encerraria demonstrando uma duplicata, um atraso e uma correção autorizada ponta a ponta.

Termo para lembrar: Fechamento

10 trade-offs: ganho, custo e decisão

01. Batch versus streaming

Ganho. Lote reduz estado contínuo e combina com arquivos externos

Custo e motivo. A detecção espera a janela de execução

Como decidir. Preferir batch quando a decisão e a evidência têm periodicidade definida

02. Matching exato versus aproximado

Ganho. Identificadores exatos geram explicações determinísticas

Custo e motivo. Não resolvem referências ausentes ou corrompidas

Como decidir. Usar matching exato primeiro e revisar candidatos aproximados

03. Dados originais preservados versus somente normalizados

Ganho. Permite replay e investigação independente do parser

Custo e motivo. Aumenta armazenamento e governança de acesso

Como decidir. Preservar evidência quando decisões financeiras dependem dela

04. Fechamento estrito versus provisório

Ganho. Estrito reduz risco de declarar cobertura inexistente

Custo e motivo. Pode atrasar relatórios por uma fonte pequena

Como decidir. Escolher conforme obrigação de fechamento e explicitar pendências

05. Estado em memória versus armazenamento externo

Ganho. Memória reduz latência do matching

Custo e motivo. Capacidade e recuperação exigem cuidado adicional

Como decidir. Usar memória para janelas limitadas com persistência apropriada

06. Partição por referência versus data

Ganho. Referência reúne pares e distribui matching

Custo e motivo. Consultas por período precisam de índices ou organização adicional

Como decidir. Priorizar referência no processamento e data na retenção

07. Ajuste automatizado versus revisão humana

Ganho. Automação reduz tempo para classes conhecidas

Custo e motivo. Exige regras estreitas, evidência e manutenção da política

Como decidir. Automatizar apenas causas determinísticas autorizadas

08. Recalcular tudo versus reprocessar partições

Ganho. Replay completo simplifica raciocínio sobre regras novas

Custo e motivo. Consome mais CPU e aumenta tempo de recuperação

Como decidir. Usar completo em validação; incremental com linhagem confiável

09. Snapshot consistente versus consulta corrente

Ganho. Snapshot evita diferenças criadas pela movimentação da fonte

Custo e motivo. Mantém versões ou demanda exportação e coordenação

Como decidir. Necessário quando o corte financeiro deve ser reproduzível

10. Tolerância zero versus classificação por materialidade

Ganho. Tolerância zero evidencia qualquer diferença

Custo e motivo. Aumenta filas de investigação por variações conhecidas

Como decidir. Classificar prioridade por materialidade sem apagar diferenças

Duas alternativas ao caminho proposto

1. Reconciliação batch determinística

Quando faz sentido. A evidência externa e o prazo de decisão são diários ou por janela.

Como implementar. Ingerir arquivos imutáveis, validar manifestos e executar matching sobre cortes consistentes, com exceções e replay.

Limites. Não antecipa problemas anteriores à disponibilidade externa; exige calendário e prova de completude.

2. Reconciliação híbrida

Quando faz sentido. Há necessidade de detecção rápida e fechamento posterior com evidência consolidada.

Como implementar. Manter correspondências preliminares por eventos e confirmar no fechamento usando posições e arquivos oficiais.

Limites. Estados preliminares não equivalem a fechamento; versões e diferenças entre fontes continuam explícitas.

Referências e alcance da evidência

1. Flink: tempo de evento e watermarks

Watermarks e processamento temporal; não fornecem prova contábil de completude.

https://nightlies.apache.org/flink/flink-docs-stable/docs/concepts/time/

2. Stripe: relatório de saldo

Exemplo público de evidência para reconciliação; não representa o protocolo do Banco Central.

https://docs.stripe.com/reports/balance

3. PostgreSQL: isolamento de transações

Concorrência na aplicação de vínculos e decisões.

https://www.postgresql.org/docs/current/transaction-iso.html

4. TigerBeetle: contabilidade financeira

Modelagem de contas, partidas e preservação de evidência financeira.

https://docs.tigerbeetle.com/coding/financial-accounting/

Continue em: T087 · T091 · T096 · C044

C047 / M. Os 50 cenários enviados

Integração e Pooling de Hardware Security Modules (HSMs)

Integração com HSM exige delimitar operações criptográficas, política de chaves e capacidade antes de otimizar sessões. O projeto distingue HSM de propósito geral, payment HSM e criptografia de envelope; nenhum deles é substituto automático dos demais.

Quando usar. Quando operações de pagamento ou proteção de chaves exigem execução em módulos criptográficos e a aplicação precisa controlar latência, concorrência e recuperação.

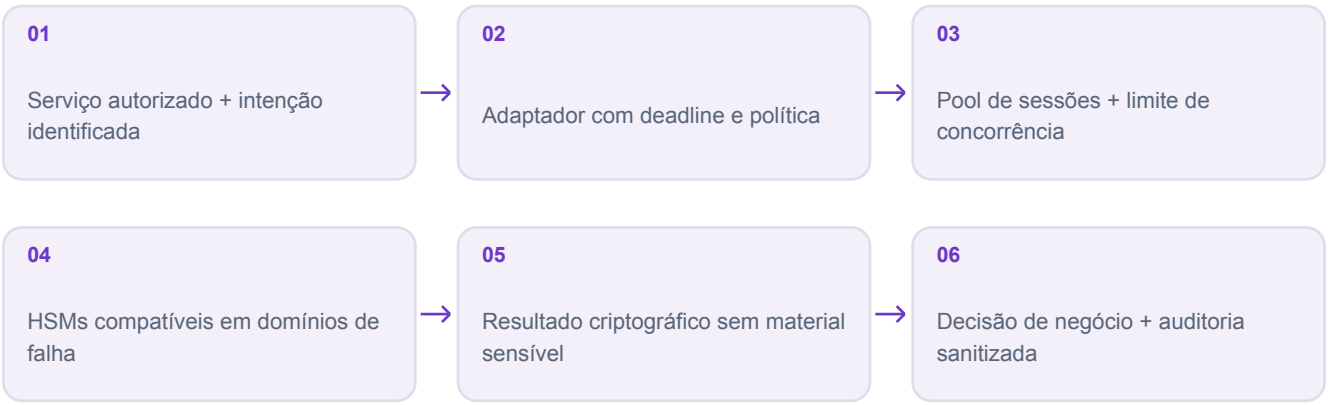
Quando evitar ou limitar. Não extrair ou cachear chaves de PIN em memória para acelerar verificações. Não prometer failover entre aparelhos sem confirmar chaves, políticas, compatibilidade e contrato operacional.

Uma abertura possível

Eu começaria pelas operações autorizadas e pela fronteira de segurança. Depois mediria o custo de cada comando e dimensionaria sessões e concorrência. O pool melhora transporte e reutilização; ele não permite mover operações protegidas para fora do HSM nem transforma uma falha ambígua em rejeição financeira.

Peças e tecnologias. Payment HSM ou AWS Payment Cryptography conforme requisitos · AWS CloudHSM para operações compatíveis de propósito geral · SDK do fabricante e PKCS#11 quando aplicável · Pooling limitado e admission control · mTLS ou transporte exigido pelo produto · Auditoria com metadados e rotação de chaves

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. 0-8 min: definir PIN, assinatura, MAC ou envelope e requisitos de proteção aplicáveis.

2. 8-18 min: mapear chaves, permissões, dependências e limites por operação.

3. 18-32 min: desenhar adaptador, sessões, autenticação e proteção contra saturação.

4. 32-48 min: simular timeout, rotação, falha de nó, credencial expirada e retomada.

5. 48-60 min: validar capacidade com perda de nó, recuperação de chaves e observabilidade sem segredos.

Cuidado técnico. Pooling depende do protocolo e SDK; conexões e sessões não são sinônimos universais.

Cuidado técnico. Envelope encryption para dados gerais não autoriza exportar chaves protegidas de PIN.

30 perguntas para sustentar o desenho

01. Que perguntas faria antes de desenhar o pool?

Perguntaria quais comandos serão usados, tipo de chave, volume, latência e requisitos de proteção. Verificar PIN, assinar documentos e criptografar arquivos têm contratos diferentes. Sem essa distinção, posso otimizar uma integração que nem suporta a operação necessária.

Termo para lembrar: Escopo criptográfico

02. CloudHSM e payment HSM são intercambiáveis?

Não assumiria isso. Verificaria comandos, certificações, formatos de chave e integrações exigidas pelo arranjo. Um HSM de propósito geral pode atender assinatura e criptografia sem oferecer o conjunto de operações e controles esperado para processamento de pagamentos.

Termo para lembrar: Tipo de HSM | Base técnica: referência 3.

03. Por que não guardar a chave de PIN no Redis?

Isso moveria material protegido para uma fronteira diferente e ampliaria radicalmente a exposição. Para operações de PIN, mantenho as chaves conforme a política do payment HSM e envio comandos autorizados. Cache não substitui a proteção exigida para esse material.

Termo para lembrar: Fronteira de segurança | Base técnica: referência 2.

04. Em qual situação criptografia de envelope ajuda?

Para dados gerais, posso cifrar conteúdo com uma chave de dados e proteger essa chave com outra gerenciada pelo serviço apropriado. Isso reduz chamadas por volume de bytes. A política de exposição da chave de dados continua explícita e restrita.

Termo para lembrar: Envelope encryption

05. Um pool TCP resolve toda a latência?

Não. Separo transporte, autenticação, sessão criptográfica e execução. Alguns SDKs já gerenciam parte disso. Meço cada fase antes de criar um proxy; um pool redundante pode adicionar fila e complexidade sem reduzir o tempo dominante.

Termo para lembrar: Conexão e sessão | Base técnica: referência 1.

06. Como escolheria o tamanho inicial do pool?

Usaria taxa alvo, tempo de serviço por comando e limites do fabricante para estimar concorrência, depois validaria sob carga. O objetivo é manter utilização segura e fila limitada, incluindo capacidade para continuar operando quando um nó ficar indisponível.

Termo para lembrar: Dimensionamento

07. Posso compartilhar uma sessão entre várias threads?

Somente se o SDK e a operação permitirem. Eu verificaria estado por sessão, autenticação e buffers. Onde houver restrição, alocaria sessões por worker ou empréstimo exclusivo do pool, sem criar uma serialização global desnecessária na aplicação.

Termo para lembrar: Thread safety | Base técnica: referência 1.

08. O pool está ocupado; você abre mais conexões?

Não ilimitadamente. Defino concorrência máxima, fila curta e prazo para adquirir sessão. Se não houver capacidade, rejeito ou adio conforme o contrato da operação. Abrir conexões sem controle pode derrubar exatamente o recurso que já está saturado.

Termo para lembrar: Backpressure

09. Como impedir que relatórios prejudiquem autorizações?

Reservaria capacidade ou filas separadas por classe de comando e criticidade. Trabalho em lote pode esperar ou ser limitado. A prioridade precisa de limites para evitar starvation, e o monitoramento distingue tempo de espera de execução criptográfica.

Termo para lembrar: Prioridade

10. O timeout inclui esperar no pool?

Sim, parto de um prazo absoluto da operação e desconto espera, conexão, autenticação e execução. Se o orçamento acabar antes do envio, não inicio trabalho inútil. Também propago cancelamento quando suportado, sem assumir que ele desfaz comandos enviados.

Termo para lembrar: Deadline | Base técnica: referência 4.

11. Um timeout significa que o comando não executou?

Não. A resposta pode ter se perdido depois da execução. Classifico a operação antes de repetir: cálculo sem efeito persistente difere de criação, importação ou rotação de chave. Para efeitos duráveis, recupero status ou identidade segundo o contrato.

Termo para lembrar: Timeout | Base técnica: referência 4.

12. Uma assinatura criptográfica repetida duplica o pagamento?

A assinatura e o pagamento são camadas diferentes. Mesmo que repetir o cálculo seja permitido, o efeito financeiro precisa de identidade e deduplicação próprias. Não uso igualdade do resultado criptográfico como substituto da idempotência da intenção de negócio.

Termo para lembrar: Idempotência

13. Como evita uma tempestade de login após deploy?

Faço rollout gradual, aqueço capacidade conforme a documentação do SDK e limito inicializações concorrentes. Credenciais têm ciclo de vida definido. Milhares de pods autenticando juntos podem saturar o HSM antes de qualquer carga útil de clientes chegar.

Termo para lembrar: Autenticação | Base técnica: referência 1.

14. Pode distribuir comandos por round-robin simples?

Só entre destinos capazes de executar a operação com a chave e política corretas. Considero saúde, capacidade e restrições de sessão. Um nó respondendo TCP não prova que tenha a chave necessária nem que consiga processar o comando solicitado.

Termo para lembrar: Balanceamento

15. O que precisa existir antes de perder um HSM?

Chaves e políticas precisam estar disponíveis no destino autorizado, com procedimento validado de recuperação ou replicação. Também preciso de capacidade remanescente e descoberta adequada. Um endereço secundário configurado não basta para garantir continuidade criptográfica.

Termo para lembrar: Failover

16. Ativar outra região resolve um desastre automaticamente?

Não. Confirmo disponibilidade das chaves, formatos, permissões, dependências e regras de residência no destino. Testo o caminho completo. Replicação de dados da aplicação não implica replicação de chaves, e a transição pode exigir um procedimento operacional específico.

Termo para lembrar: Multi-região

17. Como gira chaves sem quebrar mensagens antigas?

Associo cada operação à versão da chave e mantenho as versões necessárias durante a janela permitida de validação. Novas gravações usam a versão ativada. A aposentadoria considera dados e parceiros ainda dependentes, com evidência antes da destruição definitiva.

Termo para lembrar: Rotação

18. É seguro resolver um alias de chave a cada retry?

Se o alias puder mudar, a tentativa pode executar com outra versão e alterar a semântica. Para operações que exigem estabilidade, fixo a versão aprovada na intenção. A rotação governa novas intenções e define explicitamente a transição das existentes.

Termo para lembrar: Aliases

19. Todos os serviços podem chamar qualquer comando?

Não. Separo uso de chaves, administração e exportação autorizada, restringindo comandos por identidade e propósito. Um serviço de verificação não precisa criar ou remover chaves. Essa separação limita o impacto de credenciais comprometidas e de erros de aplicação.

Termo para lembrar: Permissões

20. O que você colocaria nos logs?

Registro identidade do chamador, operação, referência não sensível da chave, duração, resultado e correlação. Não registro PIN, material de chave ou payloads sensíveis indiscriminadamente. A auditoria deve permitir investigar uso sem se tornar outra base de segredos.

Termo para lembrar: Auditoria

21. Como verifica saúde sem sobrecarregar o equipamento?

Combino sinais do SDK, erros reais e sondas leves autorizadas com baixa frequência. Diferencio falha de rede, autenticação e indisponibilidade de comando. Sondas pesadas ou sincronizadas consomem capacidade e podem agravar uma degradação existente.

Termo para lembrar: Health checks

22. Abriria um circuito para qualquer erro criptográfico?

Não. PIN incorreto ou assinatura inválida é resultado de domínio, não falha de infraestrutura. O circuito considera erros transitórios e saturação relevantes. Misturar rejeições legítimas com indisponibilidade permitiria que tráfego inválido bloqueasse operações válidas.

Termo para lembrar: Circuit breaker

23. Dá para cachear verificações e evitar o HSM?

Eu avaliaria cada operação e sua política antes de propor cache. Identidades, validade e contexto podem mudar, e resultados sensíveis podem vazar informação. Não reutilizaria respostas de PIN como atalho genérico para reduzir carga criptográfica.

Termo para lembrar: Cache de resultados

24. Qual teste demonstra que seu HA funciona?

Retiraria um nó durante carga representativa por comando e mediria erros, fila e latência de cauda. A topologia precisa suportar a perda prevista sem ultrapassar limites. Ter dois aparelhos não prova capacidade suficiente com apenas um disponível.

Termo para lembrar: Capacidade degradada

25. O CPU do serviço está baixo; por que a latência subiu?

Investigaria espera por sessão, fila interna, autenticação, rede e mistura de comandos. CPU local não mede saturação remota. Separar essas durações ajuda a decidir entre aumentar capacidade criptográfica, reduzir concorrência ou corrigir um vazamento de sessões.

Termo para lembrar: P99

26. Como evita exaurir sessões depois de erros?

Uso aquisição e liberação estruturadas, descarto sessões inválidas e acompanho empréstimos em aberto. Erros de transporte não devem devolver objetos corrompidos ao pool. Um teste com falhas repetidas verifica se a capacidade retorna ao patamar inicial.

Termo para lembrar: Vazamento de sessão

27. Por que criar um serviço intermediário de criptografia?

Ele pode centralizar políticas, auditoria e limites quando muitos clientes precisam da mesma integração. Em troca, acrescenta latência e domínio de falha. Eu justificaria esse custo pela necessidade de governança, evitando criar um gargalo compartilhado sem isolamento.

Termo para lembrar: Proxy

28. Um mock do HSM é suficiente para validar produção?

Serve para contratos e lógica de erro, mas não demonstra latência, limites de sessão, compatibilidade ou comportamento durante rotação. Eu complementaria com ambiente apropriado do fornecedor e testes controlados de recuperação, sem usar material sensível de produção.

Termo para lembrar: Teste local

29. Se o HSM cair, você aprova transações sem verificar?

Não assumo essa permissão. O sistema aplica a política previamente aprovada para a operação, que pode ser recusar ou manter pendente. Disponibilidade não autoriza inventar uma decisão criptográfica nem ignorar controles necessários à segurança do pagamento.

Termo para lembrar: Falha segura

30. Qual é a principal defesa dessa arquitetura?

Eu preservo a fronteira das chaves e controlo a capacidade escassa com filas e concorrência limitadas. Depois demonstro recuperação por tipo de operação. O ganho de desempenho vem da integração medida, não de remover a proteção que motivou o HSM.

Termo para lembrar: Encerramento

10 trade-offs: ganho, custo e decisão

01. SDK direto versus proxy central

Ganho. SDK direto remove um salto e reduz componentes

Custo e motivo. Cada aplicação precisa integrar políticas e operação corretamente

Como decidir. Preferir direto com poucos clientes e biblioteca governada

02. Sessões reutilizadas versus criação por chamada

Ganho. Reutilização reduz inicialização e autenticação repetidas

Custo e motivo. Exige ciclo de vida, higiene e respeito à thread safety

Como decidir. Usar conforme suporte do SDK e perfil de tráfego

03. Pool pequeno versus amplo

Ganho. Pool pequeno limita pressão no recurso remoto

Custo e motivo. Pode aumentar espera mesmo havendo capacidade livre

Como decidir. Dimensionar com latência por comando e testes de saturação

04. Fila curta versus fila extensa

Ganho. Fila curta limita cauda e trabalho vencido

Custo e motivo. Rejeita mais cedo durante picos

Como decidir. Preferir quando existe prazo interativo rígido

05. Capacidade reservada versus compartilhada

Ganho. Reserva protege operações críticas contra lotes

Custo e motivo. Mantém parte da capacidade ociosa em alguns períodos

Como decidir. Aplicar quando classes têm prioridades e SLOs diferentes

06. HSM dedicado versus serviço gerenciado

Ganho. Dedicado oferece controle operacional e integrações específicas

Custo e motivo. Exige capacidade, manutenção e recuperação próprias

Como decidir. Escolher conforme comandos, política e TCO verificados

07. Região única versus recuperação regional

Ganho. Uma região simplifica gestão de chaves e roteamento

Custo e motivo. Concentra o risco de indisponibilidade regional

Como decidir. Expandir quando RTO e política justificarem a complexidade

08. Versão de chave fixada versus alias dinâmico

Ganho. Versão fixada torna tentativas reproduzíveis

Custo e motivo. Precisa coordenar aposentadoria e novas intenções

Como decidir. Usar quando a operação depende de uma versão específica

09. Isolamento por cliente versus pool comum

Ganho. Isolamento reduz noisy neighbor e blast radius

Custo e motivo. Aumenta fragmentação e capacidade reservada

Como decidir. Usar para cargas com prioridades ou riscos distintos

10. Aquecimento gradual versus inicialização sob demanda

Ganho. Aquecimento evita picos de autenticação na primeira carga

Custo e motivo. Consome sessões antes de haver tráfego

Como decidir. Aplicar em deploys e retomadas com volume previsível

Duas alternativas ao caminho proposto

1. Serviço gerenciado de criptografia de pagamentos

Quando faz sentido. Os comandos, contratos, regiões e requisitos são atendidos pelo produto gerenciado.

Como implementar. Integrar API autenticada, controlar quotas e deadlines, preservar identidades e auditar operações permitidas.

Limites. Não elimina política de chaves, integração financeira nem dependência do provedor; não é substituto universal de qualquer HSM.

2. SDK direto com biblioteca compartilhada

Quando faz sentido. Poucas aplicações precisam de operações compatíveis e um proxy não agrega governança suficiente.

Como implementar. Padronizar autenticação, sessões, métricas e tratamento de erros em biblioteca mantida, usando capacidades nativas do SDK.

Limites. Clientes continuam responsáveis por rollout e configuração; a biblioteca não cria compatibilidade entre produtos criptográficos diferentes.

Referências e alcance da evidência

1. AWS CloudHSM: integração de aplicações

Uso do SDK, autenticação, concorrência, throttling e recuperação específicos de CloudHSM.

https://docs.aws.amazon.com/cloudhsm/latest/userguide/bp-application-integration.html

2. AWS Payment Cryptography: proteção dos dados

Fronteira criptográfica e proteção do material de chaves de pagamento.

https://docs.aws.amazon.com/payment-cryptography/latest/userguide/data-protection.html

3. AWS Payment Cryptography: detalhes criptográficos

Distinção entre serviço criptográfico de pagamentos e integração de aplicação.

https://docs.aws.amazon.com/payment-cryptography/latest/userguide/cryptographic-details.html

4. AWS Builders Library: timeouts e retries

Orçamento de tentativas, jitter e resultados ambíguos; desenho aplicado é autoral.

https://aws.amazon.com/builders-library/timeouts-retries-and-backoff-with-jitter/

Continue em: T064 · T081 · T085 · T094

C048 / M. Os 50 cenários enviados

Controle de Feature Toggles em Dispositivos Móveis

Feature flags móveis controlam apresentação e rollout com configuração versionada, comportamento offline e decisões estáveis. Permissões financeiras continuam no backend; um aplicativo desconectado não recebe uma desativação instantânea.

Quando usar. Quando a equipe precisa liberar capacidades progressivamente, testar variantes ou reduzir exposição sem depender de nova publicação nas lojas.

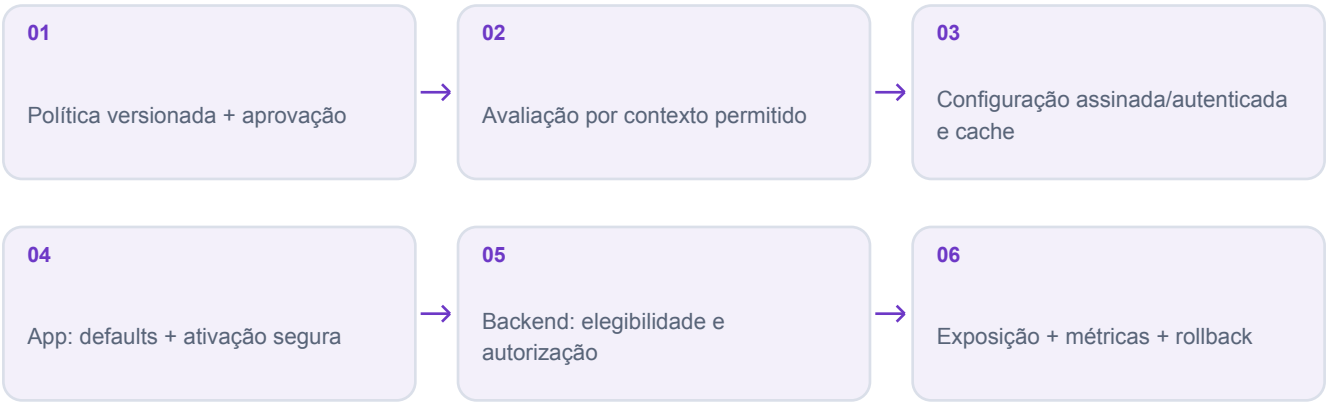
Quando evitar ou limitar. Não usar flags como autorização, prometer kill switch instantâneo offline ou adotar conexão permanente e consistent hashing sem necessidade demonstrada.

Uma abertura possível

Eu separaria disponibilidade visual, elegibilidade de negócio e autorização. O app recebe uma configuração pequena e versionada, com defaults seguros. O backend continua validando operações críticas. Para rollout, mantenho uma atribuição estável e observo exposição real, erros e efeito no cliente antes de ampliar.

Peças e tecnologias. Firebase Remote Config ou serviço equivalente · OpenFeature para interface de avaliação quando adequado · HTTP ETag e cache local · Hash determinístico com salt por experimento · BFF e autorização no backend · Eventos de exposição e guardrails

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. 0-8 min: separar release flag, experimento, permissão e operação de emergência.

2. 8-18 min: definir contexto, identidade de atribuição, defaults, TTL e versões suportadas.

3. 18-32 min: desenhar fetch, cache, ativação, validação do backend e métricas.

4. 32-48 min: simular app offline, flag antiga, rollout parcial e erro de configuração.

5. 48-60 min: justificar atualização periódica ou em tempo real, remoção das flags e critérios de ampliação.

Cuidado técnico. Rollout determinístico por hash não exige consistent hashing de um anel de servidores.

Cuidado técnico. Uma flag não altera o binário de versões antigas que desconhecem a funcionalidade.

30 perguntas para sustentar o desenho

01. O que você controla com uma feature flag?

Definiria se a flag controla apresentação, liberação operacional ou experimento. Cada propósito tem prazo e risco diferentes. Uma opção escondida no app não é uma barreira de segurança; o backend precisa decidir se a operação solicitada é permitida.

Termo para lembrar: Escopo | Base técnica: referência 3.

02. O que acontece quando o cliente abre sem internet?

O app usa configuração local ainda válida ou defaults embarcados definidos por função. Exibo indisponibilidade onde a operação exige confirmação atual do servidor. Não bloqueio toda a navegação apenas porque não foi possível atualizar uma decisão de apresentação.

Termo para lembrar: Offline

03. Todas as flags devem ter default desligado?

Não necessariamente. Uma flag que habilita o fluxo antigo pode precisar ficar ligada para preservar continuidade. Defino fallback explícito por jornada e risco, testado sem rede. O nome da flag não deve esconder qual comportamento é seguro na ausência de configuração.

Termo para lembrar: Defaults

04. Que campos você persistiria em uma definição?

Guardaria chave, tipo, valor padrão, regras, versão, responsável, finalidade e data de expiração. Experimentos também precisam de unidade de atribuição e critérios de parada. Esse contrato permite revisar mudanças e remover decisões temporárias antes que virem dependências permanentes.

Termo para lembrar: Configuração

05. Como escolher o tempo de validade do cache?

Eu compararia tolerância à desatualização, custo de rede e risco da função. Configuração visual pode durar mais; uma decisão crítica é revalidada no backend. TTL define comportamento do cliente, mas não garante revogação imediata em um aparelho desconectado.

Termo para lembrar: TTL

06. Você consegue desligar uma função instantaneamente em todos os celulares?

Não em aparelhos offline ou suspensos. Para efeitos financeiros, desativo a autorização no backend e adapto a resposta ao app. A atualização visual pode chegar depois. Essa separação evita depender da entrega de uma configuração para impedir uma operação perigosa.

Termo para lembrar: Kill switch | Base técnica: referência 3.

07. Quando usaria atualizações em tempo real?

Quando reduzir a latência de propagação durante sessões ativas tem valor mensurável. Mesmo assim mantenho fetch e cache como recuperação. A conexão depende do ciclo de vida do aplicativo; não trato sua presença como garantia de recebimento contínuo.

Termo para lembrar: Tempo real | Base técnica: referência 1.

08. Polling é uma solução inferior?

Não. Fetch ao iniciar, voltar ao foreground e em intervalos moderados pode atender o SLO com menor complexidade. Uso ETag e jitter para reduzir tráfego e sincronização. Escolho atualização contínua apenas quando a necessidade de propagação justificar o custo.

Termo para lembrar: Polling

09. Como manter o mesmo usuário na mesma variante?

Calculo um bucket com hash estável da identidade escolhida e do identificador do experimento, usando salt controlado. Comparo com faixas de alocação. Não preciso de um anel de consistent hashing; preciso de uma regra reproduzível entre avaliações.

Termo para lembrar: Atribuição

10. Aumentar de 5% para 10% troca os primeiros usuários?

Se mantenho hash, salt e faixas cumulativas, o grupo inicial continua incluído. Alterar a função ou a identidade pode redistribuir pessoas. Versiono essas decisões e verifico estabilidade antes de ampliar, especialmente quando a experiência mantém estado entre sessões.

Termo para lembrar: Rollout

11. Você sorteia por aparelho, conta ou sessão?

Depende da hipótese e do risco de contaminação. Para uma jornada financeira persistente, conta pode ser mais adequada que sessão. Para comportamento anônimo, dispositivo pode servir. Documento a unidade porque ela determina estabilidade, análise e experiência em múltiplos aparelhos.

Termo para lembrar: Identidade

12. Como trata um usuário que começa anônimo e entra na conta?

Defino uma transição explícita entre identidade anônima e autenticada. Posso manter a variante durante a jornada atual e aplicar a atribuição da conta depois. Trocar interface no meio de uma operação pode quebrar entendimento e contaminar a medição.

Termo para lembrar: Login

13. Mandaria todo o perfil do cliente para o serviço de flags?

Não. Enviaria apenas atributos necessários e permitidos, preferindo avaliação no backend para regras sensíveis. Reduzo dados pessoais no dispositivo e nos eventos analíticos. A plataforma de configuração não precisa se tornar uma cópia indiscriminada do cadastro financeiro.

Termo para lembrar: Contexto

14. Um usuário pode ativar uma função editando o armazenamento local?

Ele pode manipular sua interface, por isso o servidor valida permissão, estado e limites em cada comando. A flag ajuda na experiência, mas não concede direitos. Testaria chamadas diretas à API sem depender do comportamento do aplicativo oficial.

Termo para lembrar: Autorização | Base técnica: referência 3.

15. Como libera uma flag para versões antigas do app?

Segmento por capacidades ou versão suportada e mantenho fallback compatível. O servidor não manda componentes que o binário desconhece. Uma flag ativa código já presente; ela não instala automaticamente uma implementação nova em aplicativos antigos.

Termo para lembrar: Compatibilidade

16. Aplicaria uma configuração nova no meio de um pagamento?

Eu fixaria a configuração relevante durante a jornada ou a intenção, exceto controles de segurança revalidados pelo servidor. Atualizações visuais entram em pontos seguros. Uma mudança remota não deve trocar regras de preenchimento ou sumir com uma confirmação em andamento.

Termo para lembrar: Ativação

17. App e backend discordam sobre uma flag; quem vence?

O backend governa aceitação e regras de negócio. O app trata uma recusa como estado previsto e atualiza a experiência com mensagem clara. Registro versões para diagnóstico; não tento criar consistência global instantânea entre todos os dispositivos.

Termo para lembrar: Consistência

18. Como evitar baixar a configuração inteira a cada abertura?

Uso validação condicional por versão ou ETag e mantenho payload compacto. O cache precisa respeitar segmentação e identidade; uma resposta personalizada não pode ser reutilizada para outra conta por uma chave incorreta de CDN ou armazenamento local.

Termo para lembrar: Cache HTTP

19. O sistema de flags caiu; o app inteiro para?

Não para decisões que tenham fallback definido. O app usa configuração conhecida dentro da política e o backend preserva suas verificações. Flags críticas precisam de comportamento explícito na falha, evitando que uma dependência auxiliar vire indisponibilidade total.

Termo para lembrar: Falha do serviço

20. Você mede quem foi sorteado ou quem viu a variante?

Registro ambos quando necessário, mas exposição representa a experiência efetivamente apresentada. Ser elegível não significa usar a função. Incluo experimento, variante e versão para analisar resultados, deduplicando eventos sem confundir ausência de telemetria com ausência de uso.

Termo para lembrar: Exposição

21. Qual métrica faria você interromper um experimento?

Definiria antes limites para erro, abandono, latência e efeitos financeiros relevantes, além da métrica de resultado. Uma melhoria de conversão não compensa violar invariantes. O mecanismo de parada e o responsável precisam existir antes de expor clientes.

Termo para lembrar: Guardrails

22. Duas flags podem tornar o teste inválido?

Sim. Experimentos simultâneos podem interagir e alterar a interpretação do resultado. Eu documentaria dependências, usaria grupos mutuamente exclusivos quando necessário e testaria combinações críticas. O sistema não deve assumir independência só porque cada flag tem uma chave diferente.

Termo para lembrar: Interferência

23. Uma variante ganhou ontem; você libera para todos?

Não concluiria por uma oscilação isolada. Avaliaria desenho do experimento, duração, tamanho da amostra, segmentação e guardrails com a equipe responsável. A decisão precisa distinguir significância prática, risco operacional e efeitos que aparecem apenas com maior exposição.

Termo para lembrar: Resultados | Base técnica: referência 4.

24. Quem pode mudar uma flag financeira em produção?

Defino permissões e aprovação proporcionais ao impacto, com histórico imutável de alterações e rollback. Mudanças de alto risco não dependem apenas de acesso à interface administrativa. A configuração é parte do comportamento de produção e merece controle correspondente.

Termo para lembrar: Governança

25. Como desfaz uma configuração errada?

Reativo uma versão conhecida e verifico propagação e métricas, mantendo autorização central para efeitos críticos. O rollback de interface não desfaz operações já concluídas. Se a configuração causou efeitos financeiros, trato esses casos por recuperação de negócio específica.

Termo para lembrar: Rollback

26. Como evita centenas de condicionais esquecidos?

Cada flag temporária recebe dono, objetivo e prazo de revisão. Após estabilização, removo variantes antigas, configuração e testes obsoletos de forma coordenada. Um catálogo de flags sem processo de remoção apenas organiza a dívida, não a reduz.

Termo para lembrar: Dívida de flags

27. Você testa todas as combinações de cem flags?

Isso seria impraticável. Limito dependências, testo configurações suportadas e adiciono cobertura dirigida a pares ou grupos de risco. Também valido defaults e configurações inválidas. A arquitetura deve reduzir combinações permitidas, não depender de explorar todo o espaço exponencial.

Termo para lembrar: Teste combinatório

28. Como diagnostica um bug visto só por alguns clientes?

Registro versão do app, versão da configuração e variantes relevantes, sem dados pessoais desnecessários. Consigo reproduzir um contexto sanitizado e comparar grupos. Logs que dizem apenas flag ligada não explicam qual regra ou versão produziu a decisão.

Termo para lembrar: Observabilidade

29. Como evita picos quando publica uma nova configuração?

Uso distribuição cacheável quando apropriada, invalidação leve e fetch com jitter e limites. A publicação não deve forçar milhões de clientes a baixar simultaneamente um documento grande. Dimensiono o pico de atualização, não apenas a média diária.

Termo para lembrar: Custo

30. Como resumiria a qualidade desse desenho?

Eu demonstraria uma liberação gradual, um aparelho offline e uma desativação crítica no backend. A solução mantém experiência estável e decisão segura sem prometer atualização instantânea universal. Também mostro como medir efeito real e retirar a flag depois.

Termo para lembrar: Encerramento

10 trade-offs: ganho, custo e decisão

01. Fetch periódico versus conexão contínua

Ganho. Fetch reduz conexões e complexidade operacional

Custo e motivo. A propagação espera a próxima atualização

Como decidir. Preferir quando o SLO tolera atraso e cache atende a jornada

02. Avaliação no servidor versus no cliente

Ganho. Servidor preserva regras e atributos sensíveis

Custo e motivo. Acrescenta dependência de rede para decisões atuais

Como decidir. Usar para elegibilidade e regras financeiras

03. Cache duradouro versus TTL curto

Ganho. Cache duradouro melhora experiência offline e reduz tráfego

Custo e motivo. Mantém decisões visuais antigas por mais tempo

Como decidir. Adequado a funções com fallback seguro e pouca urgência

04. Bucket por conta versus dispositivo

Ganho. Conta mantém experiência entre aparelhos

Custo e motivo. Exige identidade autenticada e política para anonimato

Como decidir. Usar quando a jornada e a análise pertencem à conta

05. Ativação imediata versus por fronteira de jornada

Ganho. Ativação na fronteira evita mudar interface durante operação

Custo e motivo. Atrasa aplicação visual da configuração

Como decidir. Preferir para formulários e fluxos com estado

06. Rollout cumulativo versus reatribuição

Ganho. Cumulativo preserva a experiência de grupos anteriores

Custo e motivo. Restringe mudanças na função de atribuição

Como decidir. Usar para ampliação progressiva da mesma versão

07. Configuração central versus por domínio

Ganho. Central facilita catálogo e auditoria

Custo e motivo. Pode concentrar dependência e governança

Como decidir. Centralizar fundamentos e delegar ownership por risco

08. Muitas flags pequenas versus poucas configurações compostas

Ganho. Flags pequenas permitem controle granular

Custo e motivo. Multiplicam combinações e testes

Como decidir. Usar granularidade com dependências e expiração explícitas

09. Telemetria detalhada versus minimizada

Ganho. Detalhe melhora diagnóstico e análise de exposição

Custo e motivo. Aumenta custo e exigências de privacidade

Como decidir. Coletar atributos necessários à hipótese e ao suporte

10. Produto gerenciado versus plataforma própria

Ganho. Gerenciado acelera operação e entrega de SDKs

Custo e motivo. Impõe custos, limites e integração do fornecedor

Como decidir. Escolher quando atende offline, governança e escala requeridos

Duas alternativas ao caminho proposto

1. Configuração por fetch com cache e ETag

Quando faz sentido. Propagação em minutos ou na abertura é suficiente.

Como implementar. Buscar ao iniciar e voltar ao foreground, validar versão, ativar em ponto seguro e manter defaults locais.

Limites. Não entrega desativação instantânea offline; decisões críticas continuam no backend.

2. Avaliação central no BFF com capacidades do app

Quando faz sentido. A interface é composta pelo servidor e regras sensíveis não devem ser expostas no cliente.

Como implementar. O BFF avalia contexto permitido e devolve capacidades compatíveis, mantendo atribuição e registro de exposição.

Limites. Aumenta dependência do backend e precisa de fallback; composição visual continua separada de autorização.

Referências e alcance da evidência

1. Firebase: atualização de Remote Config em tempo real

Sinalização de atualização com app em foreground; não implica entrega offline.

https://firebase.google.com/docs/remote-config/ios/real-time

2. OpenFeature: especificação

Interfaces de avaliação e contexto; a política de rollout é definida pelo produto.

https://openfeature.dev/specification/

3. OWASP: autorização

Autorização precisa ser verificada no lado confiável, independentemente da interface.

https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html

4. Microsoft: plataforma de experimentação

Referência primária de experimentação e avaliação; critérios concretos são proposta do capítulo.

https://www.microsoft.com/en-us/research/group/experimentation-platform-exp/

Continue em: T090 · T100 · T109 · T118

C049 / M. Os 50 cenários enviados

Evolução de Contratos de Eventos e Schema Registry

Contratos de eventos precisam de compatibilidade estrutural, significado estável e estratégia de implantação e replay. Schema Registry ajuda a validar formatos, mas não conhece sozinho invariantes financeiros, unidades ou efeitos dos consumidores.

Quando usar. Quando produtores e consumidores evoluem de forma independente e dados antigos continuam disponíveis para replay ou auditoria.

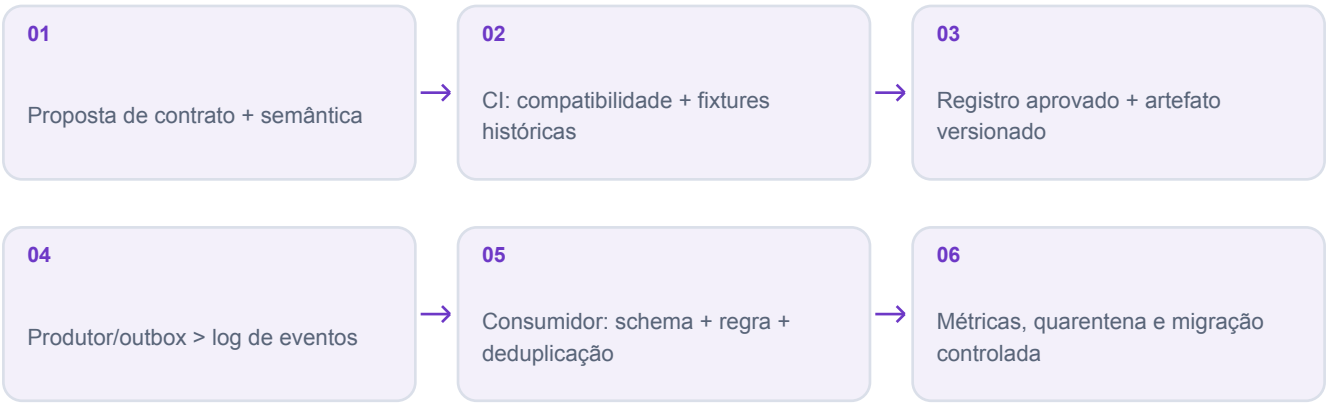
Quando evitar ou limitar. Não tratar JSON como ausência inevitável de contrato nem Protobuf como garantia automática de evolução. Não reutilizar tags, mudar unidades silenciosamente ou publicar dois efeitos financeiros durante migração.

Uma abertura possível

Eu separaria o contrato em envelope, payload e significado. Definiria quais versões precisam conversar e quais eventos históricos devem continuar legíveis. A validação de schema entra no CI, mas completo com testes de consumidor, exemplos históricos e um rollout que preserve a identidade do evento e do efeito.

Peças e tecnologias. Kafka e consumidores idempotentes · Schema Registry compatível com o formato escolhido · Avro ou Protobuf ou JSON Schema · CI de compatibilidade e testes de contrato · Outbox e registro de schema por artefato · DLQ com replay controlado

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. 0-8 min: identificar consumidores, retenção, formatos e garantias de negócio.

2. 8-18 min: definir envelope, identidade, unidades e matriz de versões suportadas.

3. 18-32 min: desenhar CI, registro, publicação, cache de schema e evolução aditiva.

4. 32-48 min: simular consumidor antigo, registry indisponível, replay e mudança semântica.

5. 48-60 min: apresentar rollout, rollback compatível, remoção de versões e ownership.

Cuidado técnico. Compatibilidade estrutural não prova equivalência financeira ou semântica.

Cuidado técnico. Defaults de Avro pertencem à resolução de leitura; não dispensam escrever campos conforme o schema do produtor.

30 perguntas para sustentar o desenho

01. O que existe além da definição dos campos?

Documento significado, unidade, cardinalidade, identidade, momento do evento e invariantes. Um campo decimal pode continuar desserializando depois de mudar de reais para centavos, mas os consumidores calcularão valores errados. Compatibilidade precisa incluir esse contrato semântico.

Termo para lembrar: Contrato

02. Quais campos colocaria no envelope do evento?

Usaria identidade do evento, tipo, versão de contrato, origem, correlação e identidade do agregado quando existir. Sequência só aparece se houver garantia real. Separaria instante de ocorrência e publicação para não confundir atraso de transporte com ordem de negócio.

Termo para lembrar: Envelope

03. O que significa compatibilidade backward?

Significa que o leitor novo consegue interpretar dados escritos conforme o contrato antigo, nas regras do formato escolhido. Isso ajuda atualização de consumidores e replay. Não demonstra automaticamente que um consumidor antigo entenda eventos produzidos pela versão nova.

Termo para lembrar: Backward | Base técnica: referência 3.

04. E compatibilidade forward, resolve qual situação?

Ela protege a leitura de dados novos por leitores antigos, conforme o formato e as mudanças permitidas. É útil durante coexistência de versões. Ainda preciso confirmar o comportamento de negócio dos leitores antigos diante de informações que eles ignoram.

Termo para lembrar: Forward | Base técnica: referência 3.

05. Por que validar só contra a versão anterior pode falhar?

Porque consumidores e arquivos históricos podem estar várias versões atrás. Uma sequência de mudanças aceitas par a par não garante compatibilidade com toda a história necessária. Defino a janela suportada e testo as versões relevantes, usando política transitiva quando apropriado.

Termo para lembrar: Transitividade | Base técnica: referência 3.

06. Adicionar um campo com default significa que ele não será escrito?

Não. Em Avro, o default ajuda o leitor quando o campo não existe no schema de escrita correspondente. O produtor continua seguindo seu próprio schema. Eu verificaria fixtures de escrita antiga e leitura nova para demonstrar o comportamento esperado.

Termo para lembrar: Avro defaults | Base técnica: referência 1.

07. Por que você nunca reutilizaria uma tag removida?

A tag identifica o campo no formato binário. Reutilizá-la pode fazer um leitor interpretar dados antigos como outro conceito. Reservo os números removidos e considero também nomes quando existem representações JSON ou textuais em uso.

Termo para lembrar: Protobuf tags | Base técnica: referência 2.

08. Zero e campo ausente são equivalentes?

Só se o contrato disser isso. Para valores financeiros, zero pode ser informação válida e ausência pode significar desconhecido. Escolho representação com presença explícita quando necessário e testo como cada linguagem e formato preserva essa distinção.

Termo para lembrar: Presença

09. Como um consumidor antigo reage a um novo estado?

Defino comportamento para valor desconhecido e verifico o runtime utilizado. O consumidor pode preservar, ignorar ou colocar em quarentena conforme a operação. Não converto um estado desconhecido em sucesso apenas porque o código antigo não possui aquele ramo.

Termo para lembrar: Enums

10. Posso manter o mesmo campo e mudar sua moeda?

Não silenciosamente. A moeda deve ser explícita e o contrato precisa preservar a interpretação histórica. Se a semântica mudar, introduzo novo campo ou evento com migração definida. O schema não detecta sozinho que um número passou a representar outra obrigação.

Termo para lembrar: Unidades

11. Por que escolher Avro em vez de JSON Schema?

Eu compararia ecossistema, tamanho, resolução de schemas e ferramentas operacionais. Avro pode combinar bem com pipelines de eventos; JSON facilita inspeção humana. Ambos exigem contrato e validação. Não escolheria formato como um marcador de senioridade.

Termo para lembrar: Formato

12. Quando Protobuf seria uma escolha adequada?

Quando geração de código e integração entre linguagens já fazem parte do ecossistema e o formato atende às consultas e ferramentas. Eu manteria disciplina de tags e presença, além de testes. O formato compacto não elimina coordenação de mudanças semânticas.

Termo para lembrar: Protobuf

13. O registry valida cada mensagem automaticamente?

Depende da integração. Registrar um schema não impede qualquer produtor de enviar bytes inválidos por outro caminho. Eu controlaria publicação, serializadores e permissões, e validaria no consumo conforme o risco. A plataforma precisa fazer cumprir a política declarada.

Termo para lembrar: Registry

14. Se o registry cair, todo o Kafka para?

Não necessariamente. Clientes podem continuar com schemas conhecidos em cache, conforme sua implementação. Novos schemas e inicializações sem cache podem falhar. Eu testo esses caminhos e evito introduzir versões novas durante uma indisponibilidade do controle de contratos.

Termo para lembrar: Indisponibilidade

15. É seguro deixar qualquer schema no cache para sempre?

Schemas imutáveis identificados corretamente podem ser reutilizados, mas o cache precisa de limites e origem confiável. Política de autorização e retenção são assuntos separados. Também verifico se o cliente consegue resolver todas as dependências necessárias ao decodificar um evento antigo.

Termo para lembrar: Cache

16. Quais verificações colocaria antes do merge?

Executaria compatibilidade de schema, fixtures históricas, testes dos consumidores críticos e validação de unidades e invariantes. A mudança também informa dono e rollout. Aprovação estrutural é necessária para certos caminhos, mas insuficiente para declarar segurança de negócio.

Termo para lembrar: CI

17. Quem atualiza primeiro: produtor ou consumidor?

Depende da direção de compatibilidade e do histórico disponível. Em uma mudança apenas backward, atualizo consumidores antes de emitir o novo formato. Para qualquer estratégia, valido a matriz real de versões e a possibilidade de rollback, não apenas o deploy ideal.

Termo para lembrar: Deploy | Base técnica: referência 3.

18. Posso voltar o consumidor depois de publicar eventos novos?

Só se a versão antiga puder ler e tratar corretamente esses eventos. Rollback de binário não apaga o log. Antes do lançamento, testo retorno de versão com dados novos já persistidos ou preparo um caminho de recuperação que não dependa desse rollback.

Termo para lembrar: Rollback

19. Como migraria uma mudança realmente incompatível?

Criaria um contrato novo com período de coexistência e adaptação explícita, medindo consumidores migrados. Se publicar representações paralelas, preservo a identidade lógica para evitar efeitos duplicados. Retiro o contrato antigo apenas após verificar dependências e necessidade de replay.

Termo para lembrar: Breaking change

20. Publicar em dois tópicos pode cobrar o cliente duas vezes?

Pode, se ambos alimentarem executores do mesmo efeito sem coordenação. A migração precisa de identidade de negócio estável e um único responsável pela execução. Consumidores de sombra comparam resultados sem aplicar efeitos financeiros enquanto validamos o novo caminho.

Termo para lembrar: Dual publishing | Base técnica: referência 4.

21. Como lê cinco anos de eventos com o código atual?

Preservo schemas de escrita e versões necessárias à interpretação, usando adaptação explícita quando fizer sentido. Testo amostras históricas e casos extremos. A estratégia precisa distinguir reconstrução de estado de repetir comandos externos que já produziram efeitos.

Termo para lembrar: Eventos históricos

22. Um upcaster pode inventar dados ausentes no evento antigo?

Não deve fabricar certeza. Ele pode transformar representação e preencher defaults semanticamente válidos, mantendo indicação de ausência quando necessário. Se uma informação não existia, consulto evidência autorizada ou preservo desconhecido; adivinhação prejudica auditoria e decisões financeiras.

Termo para lembrar: Upcaster

23. Schema Registry garante que abertura de conta venha antes do crédito?

Não. Ordenação depende do log, chave de partição, produção e consumo. O contrato pode expressar versão do agregado, mas o consumidor precisa tratar lacunas e pré-condições. Validação de formato não resolve causalidade entre eventos ou serviços.

Termo para lembrar: Ordenação

24. Qual identificador usaria para deduplicar no consumidor?

Separaria identidade do evento de identidade do efeito de negócio. Redelivery repete o evento; duas representações podem descrever a mesma intenção. Persisto a decisão e o efeito de forma atômica quando possível, evitando que a migração altere a chave financeira.

Termo para lembrar: Idempotência | Base técnica: referência 4.

25. Um evento ilegível deve bloquear a partição para sempre?

Defino política por criticidade e ordenação. Posso registrar a falha e colocar em quarentena, mas isso não autoriza processar eventos dependentes como se nada faltasse. A operação precisa ver a lacuna, sua idade e o procedimento seguro de replay.

Termo para lembrar: Poison message

26. É uma boa ideia colocar o cadastro completo em todo evento?

Não por padrão. Publico o necessário para o contrato e considero retenção, acesso e exposição em consumidores. Duplicar dados pessoais em logs duradouros dificulta governança. Identificadores e projeções controladas podem atender sem espalhar informações indiscriminadamente.

Termo para lembrar: Dados pessoais

27. Quem decide se um contrato pode mudar?

O domínio produtor é responsável pelo significado, com consumidores relevantes envolvidos em mudanças de impacto. A plataforma fornece ferramentas e políticas comuns. Um comitê central para qualquer campo vira gargalo; autonomia precisa de inventário, compatibilidade e responsabilidade explícitos.

Termo para lembrar: Ownership

28. Como descobre um consumidor antigo que ninguém lembra?

Mantenho catálogo de assinantes, versões observadas e métricas de leitura, complementados por responsáveis declarados. Ausência recente de tráfego não prova abandono, pois há jobs raros e recuperação histórica. A retirada exige janela e comunicação compatíveis com esses usos.

Termo para lembrar: Observabilidade

29. Que exemplo pegaria uma quebra que o schema aceita?

Manteria o tipo inteiro e mudaria o significado de valor bruto para líquido. A desserialização passa, mas tarifas seriam descontadas duas vezes. Fixtures com partidas e reconciliação esperadas revelam a mudança semântica que uma verificação estrutural não consegue conhecer.

Termo para lembrar: Teste financeiro

30. Como demonstraria a solução na entrevista?

Eu evoluiria um evento com campo aditivo, mostraria um consumidor antigo e faria replay de dados históricos. Depois apresentaria uma mudança de unidade rejeitada por contrato. Assim fica claro quais garantias vêm do formato e quais dependem da engenharia do domínio.

Termo para lembrar: Encerramento

10 trade-offs: ganho, custo e decisão

01. Avro versus JSON Schema

Ganho. Avro pode reduzir payload e explicitar resolução de schemas

Custo e motivo. Exige ferramentas para inspeção e schemas de escrita disponíveis

Como decidir. Usar quando o ecossistema de dados já suporta esse fluxo

02. Protobuf versus JSON

Ganho. Protobuf oferece formato binário e geração de código

Custo e motivo. Depuração manual e evolução de tags exigem ferramentas e disciplina

Como decidir. Escolher quando integração entre linguagens e desempenho justificarem

03. Backward versus full compatibility

Ganho. Backward permite mais mudanças e protege leitores novos

Custo e motivo. Não protege automaticamente leitores antigos contra novos eventos

Como decidir. Usar com atualização coordenada dos consumidores

04. Validação transitiva versus apenas última versão

Ganho. Transitiva cobre versões históricas do conjunto verificado

Custo e motivo. Restringe evolução e amplia manutenção de compatibilidade

Como decidir. Usar quando replay e consumidores antigos são requisitos reais

05. Registry central versus registries por domínio

Ganho. Central simplifica catálogo e política comum

Custo e motivo. Concentra dependência e disputa de governança

Como decidir. Centralizar quando disponibilidade e autonomia estiverem resolvidas

06. Schema em cache versus consulta por mensagem

Ganho. Cache reduz latência e dependência de rede

Custo e motivo. Precisa controlar memória e inicialização de schemas desconhecidos

Como decidir. Usar para schemas imutáveis identificados de forma confiável

07. Evento novo versus alteração aditiva

Ganho. Novo evento permite semântica distinta com clareza

Custo e motivo. Exige migração, coexistência e retirada do contrato anterior

Como decidir. Preferir quando a mudança altera significado ou invariantes

08. Upcasting na leitura versus migração histórica

Ganho. Upcasting preserva a evidência original e evita reescrita

Custo e motivo. Acumula adaptadores e custo de interpretação

Como decidir. Usar quando transformações são determinísticas e auditáveis

09. Tópico separado versus contrato compartilhado

Ganho. Separação isola evolução e consumidores novos

Custo e motivo. Aumenta fluxos, operação e risco de duplicar efeitos

Como decidir. Usar para rupturas com plano explícito de coexistência

10. Validação só estrutural versus testes semânticos

Ganho. Testes semânticos capturam unidade, estado e invariantes

Custo e motivo. Exigem fixtures e manutenção por domínio

Como decidir. Necessários quando desserializar corretamente não garante efeito correto

Duas alternativas ao caminho proposto

1. JSON Schema com contratos e CI

Quando faz sentido. Legibilidade, ferramentas HTTP e simplicidade operacional pesam mais que compactação.

Como implementar. Versionar schemas, validar produtores e consumidores, manter fixtures históricas e aplicar política de compatibilidade apropriada.

Limites. JSON não elimina custo de validação; semântica e migração continuam sob responsabilidade dos times.

2. Novo contrato com adaptador de migração

Quando faz sentido. A alteração de significado não pode ser expressa de forma compatível.

Como implementar. Publicar contrato distinto, adaptar leitores gradualmente, comparar em sombra e preservar a identidade dos efeitos.

Limites. Coexistência tem prazo e custo; o adaptador não pode inventar dados nem duplicar execução financeira.

Referências e alcance da evidência

1. Apache Avro: especificação 1.12.0

Resolução entre schema de escrita e leitura, defaults e tipos lógicos.

https://avro.apache.org/docs/1.12.0/specification/

2. Protocol Buffers: guia proto3

Identidade de campos, reserva de tags e evolução do formato binário.

https://protobuf.dev/programming-guides/proto3/

3. Confluent: evolução e compatibilidade de schemas

Backward, forward, full, transitividade e implicações para atualização de clientes.

https://docs.confluent.io/platform/current/schema-registry/fundamentals/schema-evolution.html

4. AWS: APIs idempotentes

Identidade de intenção e deduplicação; aplicação a consumidores e migrações é proposta do capítulo.

https://aws.amazon.com/builders-library/making-retries-safe-with-idempotent-APIs/

Continue em: T025 · T056 · T057 · T061 · T118

C050 / M. Os 50 cenários enviados

Câmbio e Pagamentos Cross-Border (Operações de FX)

Pagamentos internacionais combinam cotação, aceite, reservas, obrigações por moeda, execução externa e liquidação. A arquitetura de treino preserva identidade e resultados desconhecidos; equilíbrio contábil é verificado em cada moeda, sem somar quantidades monetárias incompatíveis.

Quando usar. Quando o produto converte moedas e entrega valor por provedores ou contas de liquidez com prazos, tarifas e estados independentes.

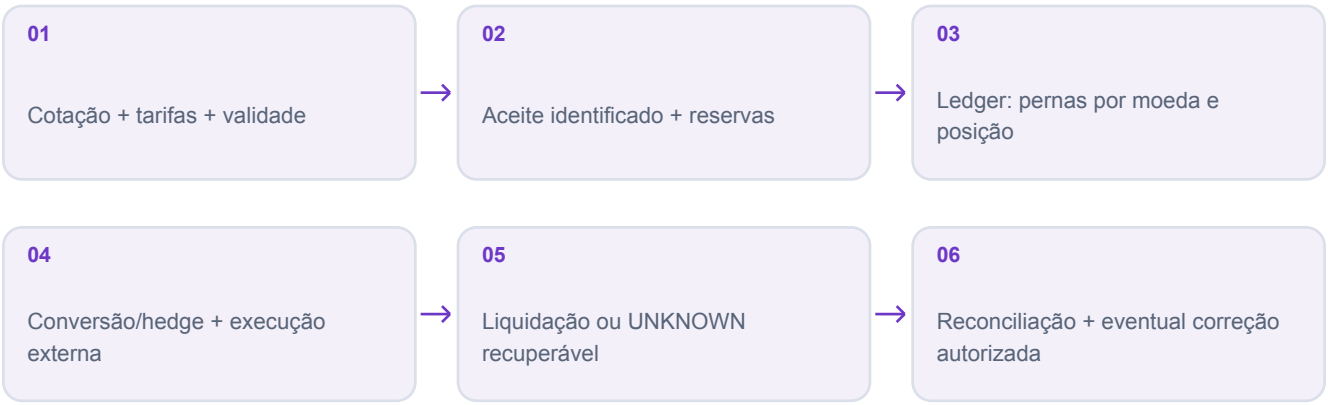
Quando evitar ou limitar. Não usar taxa indicativa como promessa executável, recalcular um aceite silenciosamente, repetir pagamento por outro provedor após timeout ou supor que compensação desfaz uma conversão ao mesmo preço.

Uma abertura possível

Eu modelaria uma cotação com validade e condições explícitas, depois uma intenção de pagamento aceita pelo cliente. Reservo os recursos e registro obrigações em cada moeda. A partir daí separo conversão, execução e liquidação, mantendo qualquer resultado ambíguo em recuperação até obter evidência suficiente.

Peças e tecnologias. Quote service e motor de preços versionado · Decimal ou inteiros com escala explícita · Ledger multi-moeda e contas de posição · Workflow durável e outbox · Adaptadores de provedores com idempotência · Reconciliação por moeda e por referência

Mapa do raciocínio



Leia os blocos na ordem numérica. O mapa resume o fluxo principal; as garantias, retornos e falhas são detalhados a seguir.

Construção passo a passo

1. 0-8 min: definir moedas, valor de origem ou destino, prazo, taxas e promessa ao cliente.

2. 8-18 min: modelar quote, aceite, intenção, reservas, pernas e identidades externas.

3. 18-32 min: desenhar registro atômico local, workflow e contas por moeda.

4. 32-48 min: simular expiração, falta de liquidez, timeout, rejeição e devolução parcial.

5. 48-60 min: explicar risco cambial, reconciliação, métricas e recuperação sem pagamento duplicado.

Cuidado técnico. A arquitetura não declara o funcionamento interno do Nubank nem recomenda uma estratégia financeira de hedge.

Cuidado técnico. Prazos, tarifas, exigências e garantias de liquidação dependem do contrato e do arranjo aplicável.

30 perguntas para sustentar o desenho

01. Qual é sua primeira pergunta sobre o produto?

Eu perguntaria se o cliente fixa o valor a enviar ou o valor a receber, quais moedas e países participam e qual prazo será prometido. Essas escolhas determinam cálculo de tarifas, necessidade de liquidez e tratamento de variações cambiais.

Termo para lembrar: Escopo

02. Uma consulta à taxa de mercado já é uma quote executável?

Não. Uma taxa indicativa informa preço aproximado; a cotação executável precisa definir condições, validade, valores e quem assume o risco até o aceite. Eu persistiria a oferta apresentada e não transformaria uma consulta informativa em compromisso sem esse contrato.

Termo para lembrar: Cotação | Base técnica: referência 1.

03. Quais dados guardaria na cotação?

Guardaria identificador, moedas, montantes, taxa e sua convenção, tarifas discriminadas, expiração, versão de preços e condições do provedor. Também identificaria o cliente ou contexto permitido. O aceite referencia essa versão para impedir mudança silenciosa da oferta durante o fluxo.

Termo para lembrar: Modelo

04. A quote venceu enquanto o cliente apertava confirmar; o que faz?

O backend verifica validade no ponto de aceite autoritativo, considerando o contrato externo. Se não puder honrar as condições, devolve nova oferta para confirmação. Não altera taxa e valor recebido silenciosamente apenas porque a interface ainda mostrava a cotação anterior.

Termo para lembrar: Expiração

05. Confiaria no relógio do celular para aceitar a oferta?

Não. O servidor ou provedor responsável define a validade e o instante de aceite, com relógios monitorados. O contador do app é informativo. Também deixo margem para transporte, porque uma oferta válida localmente pode chegar expirada ao executor externo.

Termo para lembrar: Tempo

06. Como evita reservar dinheiro sem criar a intenção?

Gravo aceite, identidade da intenção e reserva na mesma fronteira transacional quando compartilham o ledger. Quando não compartilham, uso estados e recuperação explícitos. O próximo passo só acontece após evidência durável, evitando depender de memória do processo.

Termo para lembrar: Atomicidade local

07. O cliente tocou confirmar três vezes; surgem três câmbios?

Não se a intenção tiver identidade estável e restrição de unicidade. Repetições com os mesmos parâmetros retornam o resultado conhecido; parâmetros diferentes com a mesma chave geram conflito. Uma nova conexão ou tentativa não representa uma nova autorização de conversão.

Termo para lembrar: Idempotência

08. Por que não usar float para calcular o valor recebido?

Uso decimal ou inteiros com escala explícita porque aproximações binárias podem alterar centavos e reconciliação. Defino precisão intermediária e arredondamento na fronteira contratual. A regra é versionada e testada com valores pequenos, grandes e taxas fracionárias.

Termo para lembrar: Precisão | Base técnica: referência 3.

09. O que significa uma taxa BRL/USD de 0,20?

Eu escreveria a unidade explicitamente: quantos USD são obtidos por um BRL, ou o inverso, conforme o contrato. A sigla sozinha pode ser interpretada de maneiras diferentes. Testes dimensionais impedem multiplicar quando deveria dividir ou inverter spread.

Termo para lembrar: Convenção da taxa

10. Onde registra a diferença de arredondamento?

Defino a regra por moeda e produto, preservando valor bruto, tarifa e valor final. Diferenças legítimas vão para contas apropriadas segundo a política contábil, com rastreabilidade. Não espalho arredondamento arbitrário por serviços que depois discordam sobre o mesmo pagamento.

Termo para lembrar: Arredondamento

11. Como prova que uma operação BRL para USD está equilibrada?

Verifico débitos e créditos separadamente em BRL e em USD, usando contas de posição ou contrapartida por moeda. Não somo reais e dólares para obter zero. O vínculo cambial conecta as pernas e registra taxa, mas não mistura unidades contábeis.

Termo para lembrar: Partidas por moeda | Base técnica: referência 2.

12. Como desenharia a troca simplificada de R$ 500 por US$ 100?

Mostraria uma perna de R$ 500 entre a conta do cliente e a posição em BRL, e outra de US$ 100 entre a posição em USD e a obrigação de destino. Cada perna conserva sua moeda; o vínculo registra a conversão.

Termo para lembrar: Exemplo contábil | Base técnica: referência 2.

13. Saldo contábil em USD significa que posso liquidar agora?

Não necessariamente. Preciso saber onde os recursos estão disponíveis, horários, reservas e limitações da conta de liquidação. Uma posição contábil pode representar recebível ainda não disponível. O executor verifica liquidez operacional antes de prometer um prazo de entrega.

Termo para lembrar: Liquidez

14. Reserva em BRL garante a perna de saída em USD?

Ela protege os recursos de origem, mas não cria liquidez em outra moeda. Eu separo reserva do cliente, disponibilidade da conta externa e obrigação cambial. O workflow só avança conforme as condições exigidas para cada perna e a política de risco.

Termo para lembrar: Reserva

15. Você precisa fazer um hedge individual para cada pagamento?

Não é uma obrigação arquitetural universal. A política de risco pode definir execução individual, posições agregadas ou outra estratégia aprovada. Meu sistema registra exposição e vínculos necessários, sem confundir pagamento ao beneficiário com a decisão financeira de cobertura.

Termo para lembrar: Hedge

16. Como coordena conversão e pagamento em instituições diferentes?

Uso workflow durável com passos identificados, estados observáveis e recuperação por operação. Não prometo uma transação ACID entre parceiros. As compensações dependem de quais efeitos realmente ocorreram e podem ter custos ou restrições, mantendo evidência de cada perna.

Termo para lembrar: Orquestração

17. O provedor aceitou o pagamento, mas a resposta se perdeu; e agora?

Marco a tentativa como resultado desconhecido, mantenho a identidade externa e consulto status ou reconciliação conforme o contrato. Não envio automaticamente por outra rota. O timeout informa falta de resposta, não prova que o beneficiário deixou de receber.

Termo para lembrar: UNKNOWN | Base técnica: referência 4.

18. Quando pode trocar de provedor com segurança?

Quando há evidência suficiente de que o caminho anterior não criou uma obrigação incompatível, ou um protocolo explícito de deduplicação entre rotas. Além disso, a nova rota precisa preservar condições autorizadas. Disponibilidade melhor não justifica arriscar dois pagamentos.

Termo para lembrar: Fallback

19. Qual a diferença entre aceito, enviado e liquidado?

São marcos distintos definidos pelo contrato. Aceito pode indicar que o provedor assumiu processamento; enviado pode indicar encaminhamento; liquidado exige a evidência pertinente. A interface mostra progresso sem transformar um ACK técnico em confirmação final de recebimento.

Termo para lembrar: Liquidação

20. Como feriados e horários de corte entram no design?

O cálculo de prazo considera calendário e fuso dos participantes relevantes, versionados como dados operacionais. Reavalio a promessa quando a janela é perdida. Um agendamento em horário local sem essas regras pode atrasar pagamentos mesmo quando todos os serviços funcionam.

Termo para lembrar: Calendário

21. O beneficiário foi rejeitado depois da conversão; como desfaz?

Primeiro identifico quais pernas aconteceram e quais recursos estão recuperáveis. Reverter o pagamento não garante desfazer o câmbio ao preço original. Registro a decisão de devolução, custos e eventual diferença conforme o contrato, criando lançamentos vinculados sem apagar a história.

Termo para lembrar: Rejeição

22. O parceiro devolveu menos que o valor enviado; isso é corrupção?

Pode ser tarifa, retenção contratual ou erro, então não concluo automaticamente. Registro valor e motivo por moeda, comparo com as condições e abro exceção quando necessário. A devolução parcial tem identidade própria e não encerra silenciosamente a obrigação restante.

Termo para lembrar: Devolução parcial

23. Um webhook de sucesso chegou depois de um timeout; o que faz?

Valido autenticidade, deduplico o evento e aplico a transição permitida pela versão atual da operação. A evidência pode resolver UNKNOWN. Se contrariar outra evidência final, abro investigação; ordem de chegada sozinha não determina qual fato financeiro é verdadeiro.

Termo para lembrar: Webhooks

24. Quais fontes compararia no fechamento?

Compararia ledger, execuções dos provedores, movimentos de contas de liquidação e tarifas, com cortes compatíveis. Faço matching por referências e moeda, mantendo itens ausentes e parciais. Um total convertido para moeda base pode ocultar diferenças e não substitui essa verificação.

Termo para lembrar: Reconciliação

25. Como impedir consumo excessivo de liquidez por um parceiro?

Mantenho limites e reservas sobre a disponibilidade autorizada por moeda e conta, com atualização concorrente protegida. O roteamento considera esses limites antes de executar. Métricas atrasadas ajudam a observar, mas não podem ser a única barreira contra exceder a capacidade.

Termo para lembrar: Limites

26. Validar o formato da conta é suficiente para liberar?

Não. A operação precisa cumprir os requisitos aplicáveis ao corredor, participante e produto, usando resultados verificáveis dos controles definidos. Se houver revisão pendente, preservo esse estado. Validação sintática não prova elegibilidade nem garante que o destino aceitará o pagamento.

Termo para lembrar: Dados do beneficiário

27. Se a fonte de preços cair, você usa a última taxa?

Só se a política autorizar uma oferta com validade e risco conhecidos. Caso contrário, suspendo novas cotações executáveis, mantendo consulta e recuperação de operações existentes. Não reutilizo indefinidamente um preço antigo para preservar uma aparência de disponibilidade.

Termo para lembrar: Disponibilidade

28. Qual SLO mostraria além da latência da API?

Mediria tempo para cotação, aceite, resolução de UNKNOWN e conclusão por corredor, além de exposição pendente e falhas por motivo. A API responder rapidamente não significa entregar o dinheiro. A promessa ao cliente precisa refletir as dependências externas.

Termo para lembrar: SLO

29. Qual sequência de falhas você testaria primeiro?

Eu simularia aceite persistido, timeout após execução externa e webhook tardio repetido. A invariância é uma única obrigação de pagamento, com partidas equilibradas por moeda. Depois testaria quote expirada, rotação de rota e devolução parcial para validar a recuperação.

Termo para lembrar: Teste adversarial

30. Como terminaria a defesa dessa solução?

Eu mostraria a jornada normal e uma tentativa ambígua, destacando identidades, reservas e evidências de liquidação. A arquitetura protege a promessa aceita pelo cliente e torna exposição visível. Seu valor está em explicar o dinheiro em trânsito, não apenas encaminhar mensagens.

Termo para lembrar: Encerramento

10 trade-offs: ganho, custo e decisão

01. Cotação fixa versus indicativa

Ganho. Fixa dá previsibilidade ao cliente durante a validade

Custo e motivo. Transfere risco de preço e exige condições de cobertura

Como decidir. Usar quando o produto e o provedor conseguem honrar a oferta

02. Valor de origem versus destino fixado

Ganho. Fixar destino torna o recebimento previsível

Custo e motivo. Origem e tarifas precisam absorver variações conforme contrato

Como decidir. Escolher conforme promessa do produto e consentimento

03. Pré-financiamento versus liquidez sob demanda

Ganho. Pré-financiamento reduz espera na execução

Custo e motivo. Imobiliza capital e exige gestão por conta e moeda

Como decidir. Adequado a corredores com volume e prazo exigente

04. Execução individual versus agregação de posição

Ganho. Individual facilita vínculo entre operação e conversão

Custo e motivo. Pode aumentar tarifas e quantidade de execuções

Como decidir. Escolher segundo política de risco e escala, não por senioridade

05. Workflow orquestrado versus coordenação por eventos

Ganho. Orquestração facilita visualizar pernas e recuperação

Custo e motivo. Concentra lógica e exige operação do coordenador durável

Como decidir. Preferir quando etapas externas e compensações são complexas

06. Reserva antecipada versus no momento da execução

Ganho. Antecipada protege disponibilidade de recursos

Custo e motivo. Reduz saldo disponível enquanto a operação aguarda

Como decidir. Usar quando há compromisso assumido antes da execução

07. Um provedor versus múltiplos

Ganho. Múltiplos ampliam opções de corredor e capacidade

Custo e motivo. Aumentam reconciliação, contratos e risco de fallback incorreto

Como decidir. Expandir quando os ganhos justificarem a complexidade operacional

08. Prazo conservador versus agressivo

Ganho. Conservador reduz promessas descumpridas

Custo e motivo. Pode tornar a oferta menos atraente

Como decidir. Definir com dados por corredor e limites externos

09. Consulta de status versus espera por callback

Ganho. Consulta pode resolver incerteza sem aguardar entrega de evento

Custo e motivo. Consome quotas e exige frequência controlada

Como decidir. Combinar com callbacks quando o contrato permite recuperação por referência

10. Ledger central versus separados por domínio

Ganho. Central facilita atomicidade entre pernas internas

Custo e motivo. Concentra coordenação e escala da escrita

Como decidir. Escolher conforme fronteiras; externos continuam fora da transação local

Duas alternativas ao caminho proposto

1. Provedor integrado de conversão e payout

Quando faz sentido. Um parceiro cobre moedas, corredores e condições necessárias com contrato adequado.

Como implementar. Manter quote, intenção, ledger e reconciliação próprios, delegando execução cambial e pagamento a uma API integrada.

Limites. Reduz integrações, mas não elimina risco do parceiro, UNKNOWN, tarifas nem necessidade de evidência de liquidação.

2. Plataforma com contas pré-financiadas e adaptadores separados

Quando faz sentido. Volume e requisitos justificam controlar liquidez e selecionar parceiros por etapa.

Como implementar. Gerir posições e reservas por moeda, coordenar conversão e payout em workflow durável e reconciliar cada conta e perna.

Limites. Exige capacidade operacional, gestão de liquidez e controles adicionais; não cria atomicidade entre instituições.

Referências e alcance da evidência

1. Wise Platform: quotes

Exemplo público de cotação com moeda, valor, tipo de taxa e expiração; condições variam por contrato.

https://docs.wise.com/api-reference/quote

2. TigerBeetle: currency exchange

Pernas contábeis de câmbio e contas de posição por moeda.

https://docs.tigerbeetle.com/coding/recipes/currency-exchange/

3. Python: decimal

Representação decimal e controle explícito de arredondamento.

https://docs.python.org/3/library/decimal.html

4. AWS Builders Library: timeouts e retries

Ambiguidade de resultado e controle de tentativas; aplicação financeira é desenho autoral.

https://aws.amazon.com/builders-library/timeouts-retries-and-backoff-with-jitter/

Continue em: T091 · T092 · T093 · T094 · T096 · C044